The pdp64 Architecture Manual

A 64-bit extension of the PDP-11

Digital Equity Collective's Pragmatic Digital Processor, 64 bit

David Walther

Copyright © 2026 David Walther. Released under the BSD 2-Clause licence (the file `COPYING` of the repository).
The source of this book is https://github.com/moebiusV/pdp64 (`docs/manual/`, the manual of each member in `docs/manual/pdp64-r20/`, `pdp64-16/`, `pdp64-32/`); the instruction dictionary is generated from the machine-readable specification. Typeset with Typst in EB Garamond and DejaVu Sans Mono.

CONTENTS

About this manual ..... **ii**
Part I — The architecture **1**
Chapter 1 — Introduction 2
Chapter 2 — The bus 7
Chapter 3 — Graphics: the terminal and its devices 23
Chapter 4 — Boot, the BIOS, the disk label and the GPT 57
Chapter 5 — The device driver model 80
Chapter 6 — The kernel: modes, exceptions and memory management 85
Chapter 7 — Multiple processors 107
Chapter 8 — Vector work, AI and array languages: the device route 112
Chapter 9 — The instruction set 116
Chapter 10 — Conclusion: from FPGA to silicon 444
Part II — The instruction reference tables **447**
The instruction reference tables 448
The second page: one-parcel abbreviations 449
Assembler aliases and old spellings 451
The architectural constants 452
The mnemonics, alphabetically 455
The opcodes, in order 461
The bit layouts of every opcode 464
Part III — Appendices **501**
Appendix A — Possible improvements not decided on 502
Appendix B — Decisions pending 505
Appendix C — Glossary 511
Appendix D — The ABI, version 1 515
Appendix E — The assembly language and the object format 516
Appendix F — The evidence behind the manual 518
Appendix G — The evolution of the design 520

About this manual

Each section is a few numbered aphorisms followed by a commentary. The aphorisms say what is true of the machine, in plain short statements, and read on their own as the main text. The commentary under them gives, as they apply: the *Precedent* (what earlier machines did), the *Trade-offs* (what the choice costs), the *Improvements* (how the pdp64 departs from the precedent), the *Benefits*, the *Modern comparables* (what current architectures do), and, for an instruction that is not obvious, its *Use*. Every decision also carries an *Impact* line saying what it means for an efficient **kernel**, for efficient **compilation** and for an efficient **hardware layout**: the three things a processor designer asks of a choice. An instruction as plain as `MOV` or `ADD` needs no statement of use. The reference entries follow the layout of the classic CPU manuals, in which every instruction has its encoding diagram, its operation and its effects.

Each aphorism carries a status. *(D)* means decided. *(P)* means a proposal that is not decided. *(S)* means a stub: the statement names what is left unspecified. An aphorism with no tag describes the architecture as it is built and tested.

Numbers. Where a figure is measured, the text says what was measured; where it is an estimate, it says so. Claims about other machines are checked against the fact-check record listed in appendix F.

The PDF. The same text as a book, `book/pdp64-r20.pdf`, built by buildabook and Typst from the sources of this directory (`tools/mkbook.sh`; see `book/README.md`).

Evidence documents. All of them are listed in appendix F: the full instruction set text (`../pdp64-isa.md`), the record of decisions (`../errata.md`) and the open decisions with five options each (`../decisions-pending.md`).

Part I — The architecture

Chapter 1 — Introduction

The pdp64 is a 64-bit successor to the PDP-11, designed to be as pleasant to program as the PDP-11 and the VAX-11 were, and as small to build as the PDP-11 was. It is a little-endian machine with sixteen 64-bit registers, instructions of two, four or six bytes, and the PDP-11's addressing modes and branches, brought up to date with what fifty years of compilers and operating systems have shown to matter. It is a 64-bit machine that supports several cores from the start (symmetric multiprocessing, chapter 7), and it is designed to be efficient with in-order and out-of-order execution implementations (1.4.4). Sections 1.1 to 1.4 give the reasons for the machine, what made its ancestors good, and the principles the rest of the manual follows; section 1.8 says how the pdp64 and the encodings tested against it relate.

1.1 Why the machine exists

1.1.1 The PDP-11 and the VAX-11 were pleasant, even fun, to program in assembly language.

1.1.2 Afterward computer design went in many directions, and that pleasure was largely lost.

1.1.3 The pdp64 carries it forward, using the lessons of the last fifty years: which instructions are needed to get things done and which are not, what a processor is used for, and which instructions it needs the most.

1.1.4 The PDP-11 is a source, not an obligation. What earned its place is kept, what did not is dropped, and what fifty years showed to be missing is added.

Commentary. **Precedent.** The claim in 1.1.1 is old and widely made of the PDP-11: a small, regular instruction set that a programmer could learn in a week and keep in mind, with a set of addressing modes that fit the C language so closely that the language and the machine seem to have grown together. **Improvements.** Measurements contradict several received ideas. A second unsigned multiply is the same instruction as the first, while the instruction that returns the *high half* of a product is the one multi-precision arithmetic needs. A rotate by a register count has almost no users, because cryptographic code rotates by constants. A divide is rare, because compilers turn division by a constant into a multiply and a shift. Dedicated absolute-value, minimum and maximum instructions are worth under a tenth of a percent of compiled code. And CRC instructions become unnecessary once a carry-less multiply exists. **Benefits.** A machine built on these findings is small enough for a single engineer to hold in mind, and, as later chapters show, small enough to fit in a modest FPGA. **Modern comparables.** The same re-examination produced the RISC designs of the 1980s and, more recently, the compact RISC-V and LoongArch instruction sets; the pdp64 differs by keeping the PDP-11's memory operands, addressing modes and variable-length encoding where those were found to save code and to cost little. **A thread between the two lineages.** The pdp64 joins a DEC machine's instruction set to a Commodore machine's devices, and the two companies are linked by at least one person: Ira Velinsky, the industrial designer credited with modifying the Commodore 64's case, had earlier worked at Digital Equipment Corporation (he is credited there with the Rainbow 100 of 1981), and a Commodore history credits his DEC work with drawing the attention of Commodore's management. The author predicted such a link before finding it. The hypothesis he draws from it is that personnel passed between the two companies and that the similarities between the C64 and the PDP-11 are too many to be chance *(P)*; the source itself claims only the person, and no influence of DEC designs on Commodore's, and the similarities are not yet listed with sources (the record is in `docs/c64-devices-from-chat.md`, section 8).

1.2 What made the PDP-10, the PDP-11 and the VAX-11 good to program

1.2.1 The PDP-10 gave a programmer one word per instruction that said everything: the operation, the register, the index register, the indirect bit and the address.

1.2.2 The PDP-10's registers were also memory: the sixteen accumulators answered to the addresses 0 to 15.

1.2.3 The PDP-10 had half-word and byte operations, stack and skip instructions, and a procedure call that was one instruction.

1.2.4 The PDP-11 made every instruction and every addressing mode available with every register, so that the language had few exceptions to remember.

1.2.5 The PDP-11's program counter and stack pointer were ordinary registers, and its autoincrement and autodecrement modes made `*p++` and `*--p` single instructions.

1.2.6 The PDP-11 had no input or output instructions: a device was a few memory locations, and a driver was an ordinary program.

1.2.7 The VAX-11 widened the PDP-11 to 32 bits and added three-operand arithmetic, a complete virtual-memory system, and instructions for the jobs programs actually do: procedure calls, queues, character strings, array indexing and polynomial evaluation.

Commentary. **Precedent.** These are the qualities the design starts from. The PDP-10's simplicity of format and its habit of making the machine's own state addressable (1.2.1, 1.2.2) are the origin of the idea that the machine should hold together in the head. The PDP-11's orthogonality (1.2.4) is the property that mattered most to assembly programmers and, later, to compiler writers: an instruction that takes a register takes an address, an immediate or a stack slot with the same syntax. Its memory-mapped devices (1.2.6) are the reason the pdp64's devices are still memory locations in the one address space (chapters 2 and 5). **Trade-offs.** Each of these qualities had a price that the later machines paid. The VAX-11's richness (1.2.7) bought programmability and cost an unbounded amount of hidden state in its longest instructions, so that a page fault in the middle of a character move could not be resumed without a special bit in the status word. The Motorola 68000, which took the PDP-11's orthogonality and widened it, could not restart an instruction after a bus error and so could not support demand-paged virtual memory until its successor patched the problem. **Improvements.** The pdp64 keeps what made programming pleasant and removes what made implementation hard: no hidden instruction state (every instruction can be restarted), at most one memory operand, a fixed small maximum instruction length, and no mode bits that change what unrelated instructions mean unless their benefit outweighs the cost. **Benefits.** The measurements in the instruction-set chapter show where this paid off: the processor fits in about 16,000 FPGA lookup tables and the compiler's code is the size of `gcc -Os`. **Modern comparables.** Of today's architectures, x86-64 kept the memory operands and variable-length encoding of this lineage; AArch64 and RISC-V chose fixed-length register-to-register encodings; the pdp64 sits between them, closer to x86 in the form of its instructions and closer to RISC-V in the discipline of its encoding.

1.3 More minimal than its ancestors, where hindsight allows

1.3.1 In several ways the pdp64 is more minimal than the PDP-11 and the VAX-11: it gives up theoretical completeness where fifty years of use showed the completeness was never exercised, and it spends what that saves on power that was.

1.3.2 Not every instruction works with every addressing mode on every operand: at most one operand of an instruction is a memory reference. *(D)*

1.3.3 The deferred addressing modes are gone; an indirection is a `JMPI`, a `CALLI` or an ordinary load. *(D)*

1.3.4 The ALU has one width, there is no arithmetic in the narrow sizes, and there is no conversion opcode group: conversions are moves. *(D)*

1.3.5 There is one rotate direction, and no slot exists only to complete a table.

1.3.6 The VAX's packed decimal, string editing and `CALLS`/`CALLG` procedure-call instructions are absent. *(D)*

1.3.7 There are no transcendental instructions, no separate floating-point register file and no rounding-mode bits. *(D)*

1.3.8 What the saving buys is power that programs use: a bit unit (population count, count of leading and trailing zeros, byte reversal, bitfields, a carry-less multiply), block operations that restart from their registers, a fused multiply-add with the IEEE exception flags, the full set of atomics, UTF-8 instructions, and a user-memory guard. *(D)*

Commentary. **Precedent.** The PDP-11 was orthogonal, any operation with any mode on any register, and the VAX carried that to completeness: every operation in every data type, with up to six operands, each operand of any mode. Both were designed when memory was dear and compilers were weak, and the completeness did two things at once: it made assembly programming regular and pleasant, and it made a compiler's job simple. **Trade-offs.** The completeness was never free. It bought a decoder that had to parse specifier after specifier before the next instruction's address was known, and an instruction that could fault in the middle with unbounded state to save. What hindsight lets the pdp64 do is separate the two benefits: keep the regularity that programmers and compilers use (every register is general, every mode means the same thing everywhere it is allowed) and drop the completeness that nobody exercised (a memory-to-memory arithmetic instruction,

a deferred mode on every operand, a sized version of every operation). The price is a few more instructions in rare cases, such as a load and a store where the VAX had one memory-to-memory move. **Improvements.** Each piece of dropped completeness was weighed against what it freed: the one memory operand lets the fault matrix collapse to three rows with the third empty and pays for sixteen registers and a 12-bit displacement in the same parcel; dropping the sized ALU removes byte-lane multiplexing and partial-register writes altogether; dropping the transcendentals leaves the `FMA` datapath, which is what accurate library maths is built from. **Benefits.** The decoder is small, the exception machinery is small, and the die area that completeness would have used goes to a bit unit and a block-move sequencer that together cost less than 5% of the core. **Modern comparables.** The RISC machines of the 1980s went further and made every instruction register to register except loads and stores; x86 kept one memory operand and survived on it; the pdp64 takes the middle position, which keeps the compact code of a memory operand and the restartability of a load/store design.

1.4 The principles

1.4.1 The design follows these priorities, in this order: **security and safety**, meaning errors that can never arise, then errors that are easy to detect, then errors that are cheap to recover from (restart and recovery go together, and an instruction does what its name says and nothing else); **opcode choice**, so that fewer instructions executed means fewer memory fetches; **opcode encoding**, dense, with rare instructions in longer encodings; **speed and power**, served by a low gate count; a **good compiler target**; and **pleasant in assembler**. *(D)*

1.4.2 All the rules are soft rules: a rule gives way when breaking it is worth more than it costs, with the cost written down. *(D)*

1.4.3 Between two designs that both work, prefer by this order: the one with no strange side effects (an instruction does what its name says and nothing else); the one that does not push an existing instruction into a longer encoding. Beside the ranking, at every level, do not multiply entities without benefit. Mode bits are decided case by case: by default a mode is visible state, and a bit in a register that is saved anyway costs nothing extra.

1.4.4 The architecture is designed to be efficient with in-order and out-of-order implementations. The first implementations are in order; nothing is specified in a way that is safe only because the core does not speculate or reorder, and nothing is specified that an out-of-order core could implement only at a large cost. *(D)*

1.4.5 An instruction is judged by its weight in the loops where it is used, not by how often it appears: a rarely used instruction that sits inside a critical, tight hot loop can earn its place. The measure is profile-weighted (executions multiplied by the instructions and bytes it saves), not a static count. *(D)*

Commentary. **Precedent.** Security and safety rank first; opcode choice and encoding, which cut memory fetches, rank next, then speed and power with the gate count that serves them. The priorities agree on almost everything, because a small, regular machine is also compact in code and cheap in gates; the stated order decides which wins when they conflict. Code size here means the instructions a program executes and the bytes it fetches, not the number of opcodes the machine has. **Trade-offs.** Error prevention, detection and recovery is the principle that most often changes a decision. It means counting the cost of an option not only in gates and bytes but in the code a kernel must write to handle its failures: a feature that adds a new fault, a new error return or a new half-finished state costs recovery code in every system that uses it, and a feature that makes an operation repeatable from the start costs none. **Improvements.** The soft-rules principle covers the two strongest defaults, "no mode bits" and "natural alignment of quad accesses". Breaking either has a large cost, but each is weighed like any other choice. Three mode bits are accepted in the design on those terms: the translation-enable bit that is set once at boot, the user-memory guard that lives in the status word, and the sticky floating-point flags, which never change what an instruction does. **Benefits.** A reader can tell why a choice was made by asking which principle it serves. **Modern comparables.** Most architectures state no priorities; the RISC-V project's published principle of keeping the base small and the extensions optional is the nearest in spirit.

1.5 What has been built and measured

1.5.1 The machine is specified by machine-readable grammars; the assembler, linker, simulator, a compiler back end and the QEMU target are generated from or checked against them, so that the numbers cannot drift apart.

1.5.2 On 827 real source files that every compiler accepts, the first compiler's output, in code and data, is within 0.2% of `gcc -Os` and 6.5% smaller than `clang -Os` on amd64, using 1.19 and 1.10 times as many (shorter) instructions.

1.5.3 516 C test programs pass on the simulator and on QEMU.

1.5.4 The finish line before silicon is an assembler, a linker, an LLVM compiler back end, an emulator, and a port of OpenBSD that can be logged into with `ssh`. *(D)*

Commentary. **Precedent.** The habit of generating every tool from one description, rather than writing each and hoping they agree, is the discipline that other instruction-set projects adopted late; the pdp64 follows it from the start, which is why a change to an opcode number is made once. **Trade-offs.** The measurements are of compiled C at the compiler's size-optimising setting; they are static counts of what is written, not run-time measurements, and the compiler is one of two (the LLVM back end, which OpenBSD needs, has an assembler and object writer that agree with the project's own on every form tested). **Benefits.** The comparison gives the architecture a number against which later changes are measured: the flag rules cost 0.39% in code size, accepted for the benefit they give to faster implementations. **Modern comparables.** `gcc` and `clang` at `-Os` are the standard measuring sticks for code size on x86-64 and AArch64; the pdp64 figure is compared with them on the same source files.

1.6 Does the instruction set matter?

1.6.1 The instruction set is not interchangeable with any other: it fixes what every implementation must preserve, and so what a processor costs to build, to verify, to program and to secure.

1.6.2 The common view, that a good implementation hides the instruction set, is true of one thing: the peak speed of a very large core, where a great deal of hardware can absorb almost any encoding.

1.6.3 It is not true of the cost of a small core, the size of the code, the amount of software a system needs, or the number of ways a system can fail; those follow the instruction set, and they can be measured.

1.6.4 Each of the rows below is a measurement or an estimate from this project, and each is explained in the chapter named beside it.

What differs	The figure	Where
The logic of a complete implementation, same company and era	the PDP-11's T-11, a full integer core, was about 12,000 transistors; the MicroVAX 78032, a subset of the VAX with no floating point, about 125,000	10.1, 10.2
What it takes to find where the next instruction starts	pdp64: two bits of the first parcel; PDP-11: both operand specifiers; x86: prefixes, opcode, ModR/M and SIB in sequence; VAX: every specifier in order	9.4
Whether flags block a faster implementation	68.2% of compiled instructions wrote a flag and 1.5% read one, so the flag rules are set at a measured cost of 0.39% in code size	9.3, 10.3
Size of compiled code	for the same 827 source files and the same compiler family, the pdp64 output is 0.86 times the size of its amd64 output, and 1.002 times `gcc -Os`	1.5
System-call path, instructions in and out (paper estimates)	x86-64 Linux about 45 and 30; AArch64 about 60 and 45; RISC-V about 66 and 45; VAX about 11 and 8; m68k 6 and 8; pdp64 4 to 5 and 4	6.5
Lines of kernel code to start a second core	x86: 1,528 plus 1,258 of real-mode code; arm64: 124 or 127; RISC-V: 108 or 58; pdp64: one register write	4.5, 7.8
Machine-dependent code in an OpenBSD port	riscv64 35,600 lines; arm64 61,613; amd64 79,167	10.4
Errata, workaround and quirk lines per 1,000 code lines in Linux's architecture directories	x86 0.8; arm64 0.7; arm 0.5; powerpc 0.4; mips 0.3; m68k 0.1; riscv 0.1; s390 0.0	pain-points report
Hardware-erratum options in Linux's configuration	arm64 64; arm 32; riscv 14	pain-points report

Commentary. **Precedent.** The view that instruction sets are equivalent has real support. Published comparisons of x86 and ARM implementations (the 2013 study usually cited is "Power Struggles: Revisiting the RISC vs. CISC Debate on Contemporary ARM and x86 Architectures", a citation not verified) found that at the same performance level the instruction set explains little of the difference in power or speed, and the very large cores of both families have converged. The pdp64 does not dispute that. **Trade-offs.** The claim is about steady-state performance of comparable large cores, and the table is about other costs. The table's figures are measurements of real machines and real software, not controlled experiments: some of the differences between architectures in the Linux figures are partly a matter of age and deployment (x86 is the oldest and most widely deployed), and the system-call and start-up figures are estimates and line counts of what is written, not run times. They show that the instruction set is correlated with these costs, and the chapters show the mechanism in each case. **Improvements.** Each row is a place where this design made a different choice from the architecture it is compared with, and the choices were made against these costs, not against peak speed. **Benefits.** A reader who thinks the instruction

set does not matter can check any row against its chapter: the answer is in what the instruction set obliges an implementation to do (precise exceptions, ordering, flag behaviour, instruction length, restart) and in the code a system must then write. **Modern comparables.** RISC-V's own case for a clean base instruction set and x86's survival by microarchitecture are the two sides of the argument.

1.7 How the manual is organised

1.7.1 Chapter 2 is the bus. Chapter 3 is graphics: the terminal, based on the C64 and the Blit and modernised, and how it drives HDMI and the other ports. Chapter 4 is boot. Chapter 5 is the device driver model. Chapter 6 is the kernel: modes, exceptions and memory management. Chapter 7 is multiple processors. Chapter 8 is vector work, AI and array languages, done by devices. Chapter 9 is the instruction set. Chapter 10 is the path to silicon.

1.7.2 Each section states its aphorisms first and its commentary after; the status tag *(D)* or *(P)* marks what is decided and what is proposed.

Commentary. **Precedent.** The form follows the old commentary traditions, in which a short text can be read straight through and the discussion of each line sits beneath it; and the layout of the classic CPU manuals for the reference material, in which every instruction has a fixed entry. **Benefits.** A designer who wants only the architecture reads the aphorisms; one who wants the reasons reads on.

1.8 The pdp64 and the encodings tested against it

1.8.1 The pdp64 is pdp64/32: thirty-two registers and an instruction encoded by a prefix code over the first parcel, with 5-bit register fields in the one-parcel forms. Two earlier encodings, pdp64/r20 (the original: sixteen registers, a first parcel of `len`, opcode, `reg` and `low`) and pdp64/16 (sixteen registers, a prefix code), were built and measured against it; they are tested alternatives, not products, and each keeps its own manual so that the comparison can be read. They share with the pdp64 the roles of the registers, the memory model, the exception mechanism, the page tables, the bus and the devices, the calling convention and the assembly language, and differ in how an instruction is encoded and in the size of the register file. *(D)*

1.8.2 This manual is the manual of pdp64/r20, a tested alternative; the manual of the pdp64 is that of pdp64/32. *(D)*

1.8.3 The three manuals are generated from one text, so that what is common is written once. The assembler selects the encoding (`-m pdp64/r20`, `-m pdp64/16`, `-m pdp64/32`), the object records it, and the linker refuses to link objects of different encodings. *(D)*

1.8.4 This CPU has 106 instruction entries (270 mnemonic words with the aliases); pdp64/16 and pdp64/32 have the same instructions and 70 more words, the fused pairs, the selects, the checked arithmetic, `TRAPCC`, the deferred and user-qualified moves and the AES and SHA words, none of which this CPU accepts. *(D for the fused pairs, the selects, `MOVQL`, the checked arithmetic and `TRAPCC`; P for the rest, tagged in their entries)*

1.8.5 pdp64/32 was chosen for performance, by testing against the other two on the same programs. Over six programs (lz4, zlib, Lua and three BearSSL kernels) compiled for each and run on the simulator, pdp64/32 executes 572,801,956 instructions against 609,755,229 for pdp64/16 and 755,816,022 for pdp64/r20 (the original compiler for the latter); its frame loads and stores fall to 24,276,195 against 65,203,703 and 68,936,548; and on the in-order timing model (an ideal memory, no branch predictor, issue width 1 or 2) its cycle counts are 22 to 28% below pdp64/r20's and, with a fetch window of 4 bytes or more, 7 to 8% below pdp64/16's (the model is a check of the pipeline it states, not hardware). Its code is larger than pdp64/16's: 3.18% in instruction bytes over the 841-file corpus and 2.92% in bytes fetched over the six programs. *(measured; the record is `../encoding-work/integration.md`, section 4)*

The manuals of the tested alternatives: pdp64/16; the manual of the pdp64 is pdp64/32; the landing page is `docs/manual/`.

Commentary. **Precedent.** One architecture with more than one encoding is not new: AArch32 has the ARM and Thumb encodings and RISC-V a compressed extension; the PDP-11 family had many implementations of one instruction set. **Trade-offs.** Three manuals cost the maintainers a generator; one manual with a line of availability on every instruction would have mixed three layouts on every page, and the author decided against it. **Benefits.** A reader of one manual meets only the encoding of that CPU; the bus, the kernel and the devices are the same text in all three, and the reader who wants the evidence for the choice of pdp64/32 has the other two manuals to compare with. **Impact.** *Kernel:* the system architecture is common, so one kernel serves all three. *Compiler:* one back end with three targets (`ccom -m`), a different set of instructions on pdp64/r20. *Hardware:* the CPUs have different decoders and register files and the same programming model above the instruction set.

Chapter 2 — The bus

The processor reaches the world through one bus, Q64, which is PCI Express with convenience and security circuitry added; every standard device is a virtio device (the disk is virtio-blk) or a USB controller, and any other PCI Express device may be attached, all found by PCI Express enumeration; every transfer of data is a descriptor in a queue in memory that the device carries out and marks done. This chapter is written for a reader who knows the PDP-11 and its Unibus, or who is learning computing from the start, and who has not met anything that was invented after the 1970s: it begins with what the Unibus did, explains each modern idea by contrast with it (a PCI Express link, the configuration space, base-address registers, bus-mastering DMA, message-signalled interrupts, virtio, the IOMMU), and only then gives the rules the processor and the bus keep. The rules are the numbered statements; the rest is explanation. Decided statements are tagged *(D)*; proposed ones *(P)*; unspecified ones *(S)*; a fact about PCI Express or virtio that could not be checked on this machine is marked *unverified* and listed in section 2.12. How a driver talks to a device, the virtio transport and the default devices are in chapter 5; interrupts and modes are chapter 6, and the other cores are chapter 7. The full discussion is in `../pdp64-q64-bus.md`; the options still open are in `../decisions-pending.md`.

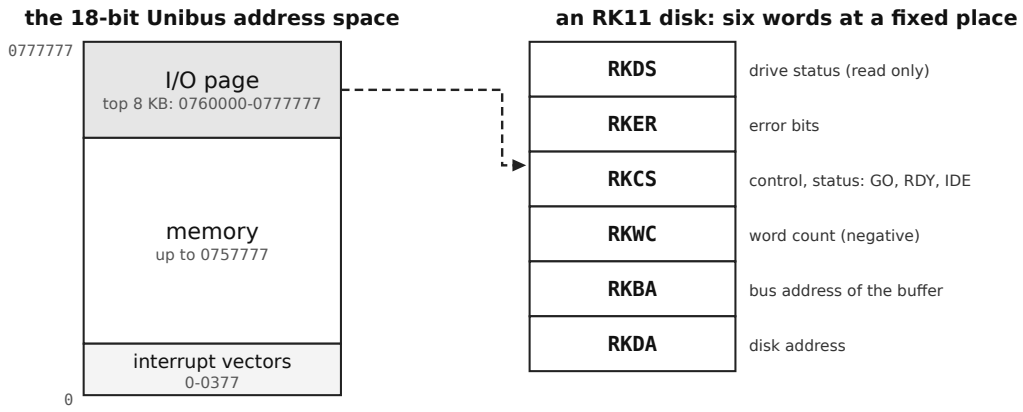
How to read this chapter. A term is defined where it first appears, as the technical word followed by a plain restatement. Boxes-and-arrows pictures are the kind a textbook draws for a linked list: a box is a piece of memory, an arrow is an address stored in it. Code blocks are pseudocode (C-like, with `mmio_read16(address)` and the like standing for an ordinary load or store to a device register, section 2.3); the PDP-11 assembly is the real thing, quoted from a 2.11BSD driver or written in its style.

2.1 What you already know: the Unibus

A PDP-11 has a **bus** (a set of shared wires that every part of the machine hangs on) called the Unibus. Everything the machine knows about the world it knows through that bus, and four ideas of it matter here, because every modern idea in this chapter is one of them, kept or replaced.

Registers at fixed addresses. The Unibus has 18 address lines, so 256 KB of **address space** (the set of numbers that name locations). The top 8 KB of it is the **I/O page** (the region where devices, not memory, answer). A device is a handful of 16-bit **registers** (small storage cells in the device that the processor can read or write) at addresses that were set by switches on the card. Nothing tells the processor what is plugged in: the kernel is built with a table that says "there is an RK11 at 0177400" and the kernel's probe routine tests whether anything answers there. A driver is written against a C structure laid over those addresses; this is the whole of 2.11BSD's declaration for the RK05 disk controller (`rkreg.h`):

```
struct rkdevice {
        short   rkds;   /* drive status         */
        short   rker;   /* error bits           */
        short   rkcs;   /* control and status   */
        short   rkwc;   /* word count           */
        caddr_t rkba;   /* bus address          */
        short   rkda;   /* disk address         */
};
```



a driver holds one pointer, the address of RKDS; each register is a field
(the wiring, the vector and the CSR address, was set by switches on the card)

The CSR convention. Most devices have a **CSR** (control and status register: one word whose bits both command the device and report on it). The RK11's has GO in bit 0 (do it now), READY in bit 7 (the device is idle and may be given work), IDE in bit 6 (interrupt on done enable) and ERROR in bit 15 (`rkreg.h`: `RKCS_GO` 01, `RKCS_RDY` 0200, `RKCS_IDE` 0100, `RKCS_ERR` 0100000). The same shape, with a **DBR** (data buffer register: the word the data passes through) beside it, is repeated on the serial lines and the line printer.

DMA. To move a block, the processor does not copy words: it writes the memory address into a register (`rkba`), a negative count into another (`rkwc`), the disk address into a third and then GO. The controller becomes a **bus master** (a part that is allowed to start transfers itself, not only answer them) and **DMA** (direct memory access: moving data between a device and memory without the processor's help) does the copying, requesting the bus by **NPR** (non-processor request: the Unibus line a device raises to be given the bus for a data transfer at once, at a priority above every interrupt). The 18-bit bus address was too narrow once memory grew, and a **map register** (a table entry that translates a device's address to a memory address) appears in 2.11BSD as the UNIBUS map (`mapalloc`, in the machine-dependent code). Keep this: it is the ancestor of the IOMMU of section 2.11.

Interrupts. A device that wants attention asserts one of four **BR** lines (bus request, levels 4 to 7: the priority at which it wishes to interrupt); when the processor's priority is lower, it grants the bus, and the device places its own **interrupt vector** (the address of a pair of words, new program counter and new status word, that the processor loads) on the data lines. The vector too was set by switches on the card. The interrupt routine returns with `RTI` (return from interrupt). A driver that does not want interrupts **polls** (reads the status register again and again until the bit it wants changes).

Here is the RK11 driver in the two styles, both real or in the style of the real code. The interrupt-driven start, from 2.11BSD `pdpuba/rk.c`:

```
rkaddr->rkda = (dn << 13) | (cn << 4) | sn;     /* disk address     */
rkaddr->rkba = bp->b_un.b_addr;                  /* where in memory  */
rkaddr->rkwc = -(bp->b_bcount >> 1);             /* words, negative  */
com = ((bp->b_xmem & 3) << 4) | RKCS_IDE | RKCS_GO;
com |= (bp->b_flags & B_READ) ? RKCS_RCOM : RKCS_WCOM;
rkaddr->rkcs = com;                              /* go; return; rkintr() runs when done */
```

and the same command polled, in PDP-11 assembly (the register addresses are the usual ones for the first RK11, 0177400 and up, *unverified* in this repository; the structure above fixes their order):

```
        MOV  #<disk address>, @#177412     ; RKDA
        MOV  #<buffer>,       @#177410     ; RKBA
        MOV  #-256.,          @#177406     ; RKWC: 256 words, negative
        MOV  #RCOM+GO,        @#177404     ; RKCS: read, go, no interrupt enable
1$:     TSTB @#177404                      ; bit 7 of the low byte is READY
        BPL  1$                            ; plus means not yet: look again
        TST  @#177404                      ; bit 15 is ERROR
        BMI  error
```

Notice what the processor must do: **it** knows the register layout, **it** loads each register, **it** waits, and it knows the meaning of every bit of that one device. When the next device arrives, with different registers, the driver is written again. The rest of this chapter is the answer the industry reached, and the pdp64 adopts: do not make every device a different set of registers; make every device the same shape.

What does not carry over, and what does. Gone: the fixed I/O page, the switches, the vectors on the data lines, the bus request levels, the single shared set of wires (and with it the locked read-modify-write `DATIP`/`DATO`, section 2.10). Kept: devices are locations in the one address space, reached by ordinary loads and stores, with no I/O instructions (2.3.2); the device moves its own data by DMA; the interrupt says only "look at the device"; polling remains allowed (2.7.4).

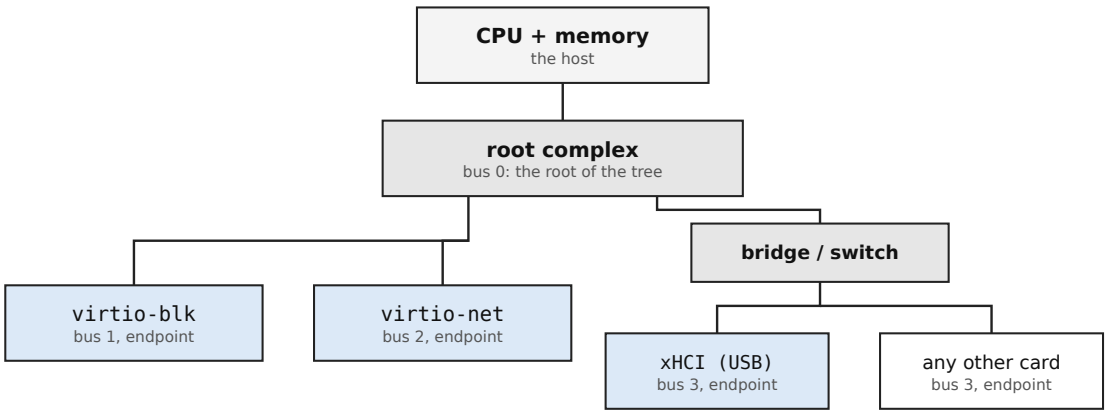
2.2 PCI Express in one section

PCI Express (written PCIe) is the bus of every PC and server made since about 2005. Its ideas, each against the Unibus.

A link, not a shared bus. A Unibus is one set of wires that all devices share, so only one transfer happens at a time and every card loads the wires. PCI Express is a **point-to-point link** (a connection between exactly two ends; nothing else is on it). A **lane** is one such connection's unit of width: a pair of wires carrying data one way and a second pair the other way, so a lane sends and receives at the same time; a link has 1, 2, 4, 8, 16 or more lanes (*unverified* as a list; the link-width field of `pci_regs.h` is six bits wide, so the specification allows a range of widths), and gets faster by having more of them. Each lane runs at a fixed signalling rate, which has doubled with each generation: the speed codes of `pci_regs.h` are 2.5, 5, 8, 16, 32 and 64 **GT/s** (gigatransfers a second: the number of signal changes, not yet bytes). The software does not see any of this: it sees addresses.

A tree. Because links are point-to-point, a machine is a tree. The root is the **root complex** (the part, in the processor chip or beside it, that connects the processor and memory to the tree and is the PCI Express counterpart of the Unibus's processor-side logic); below it are **root ports** (its links to the outside), **switches** and **bridges** (parts with one link up and several down, which pass traffic between them; to software a switch is a set of bridges) and at the leaves **endpoints** (the devices). The QEMU documentation of the tree (`docs/pcie.txt`) uses exactly these words: root complex, root port, downstream port, bridge, integrated endpoint.

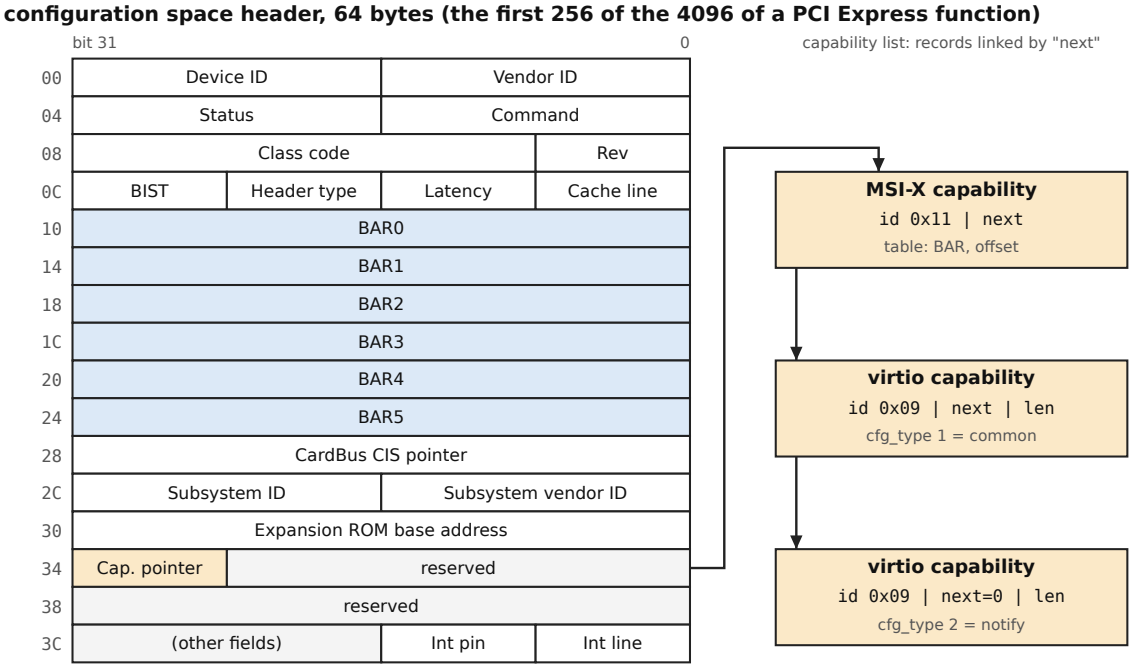
a PCI Express tree: a bus of point-to-point links, not a shared set of wires



each line is a link: one or more lanes, each a pair of one-way wire pairs
Bus numbers and the shape are examples.

Packets. Nothing in PCI Express is a wire that goes high. Everything is a **packet** (a small message with a header that says what it is, where it goes and how much data follows). The packets that carry reads and writes are **TLPs** (transaction layer packets: the unit software cares about, as distinct from the lower layers that frame bytes and retry on error). The kinds a programmer meets (*unverified*, from general knowledge of the specification, which is not on this machine): a memory write (address and data; no answer comes back, which is called **posted**: the sender has handed it over and moves on), a memory read (address and length) which is answered later by a **completion** (a packet carrying the data back), configuration reads and writes, and messages. A read is therefore a **split transaction** (the request and the answer are separate packets, and the link is free in between), where on the Unibus the bus master held the bus, with `MSYN` asserted, until the slave replied with `SSYN`. Two consequences matter to a programmer: a write has already "happened" in the program's mind while it is still on its way, which is why section 2.8 has a `FENCE`; and an interrupt (section 2.7) can be a packet like any other.

Configuration space: the device describes itself. This is the idea the Unibus lacked. Every device, which PCI Express calls a **function** (one logical device; a card may carry several), has a **configuration space** (a block of 4096 bytes of registers, separate from the device's working registers, whose first 64 bytes are the same for every device on earth: who made it, what it is, how much address space it wants). The first 256 bytes are the older PCI part; the rest, 4096 bytes in all, is the PCI Express part (`PCI_CFG_SPACE_SIZE` 256 and `PCI_CFG_SPACE_EXP_SIZE` 4096). A function is named by three numbers, its **bus**, **device** and **function** numbers (the position in the tree), and the processor reaches the configuration space of function (b, d, f) with ordinary loads and stores to one address range, in the usual arrangement: byte `offset` of function (b, d, f) is at `base + (b << 20) + (((d << 3) + f) << 12) + offset` (this is **ECAM**, enhanced configuration access mechanism; the shifts are from QEMU's `pcie_host.h`, `PCIE_MMCFG_BUS_BIT` 20 and `PCIE_MMCFG_DEVFN_BIT` 12). Where `base` is on the pdp64 is not specified (2.4.8).



The 64-byte header of an ordinary function (type 0) holds, as the figure shows and `pci_regs.h` defines: the **vendor ID** and **device ID** (two 16-bit numbers that name the maker and the product; 0xFFFF as a vendor ID means that no function answers), the **command** register (three of its bits matter here: bit 1 lets the function answer memory addresses, bit 2 lets it be a bus master, bit 10 turns off its old-style interrupt pin), the **status** register, the **class code** (what kind of device), the **header type** (0 for an ordinary function, 1 for a bridge, with bit 7 set when the card has several functions), six **base-address registers** (section 2.4) and a pointer to the capability list.

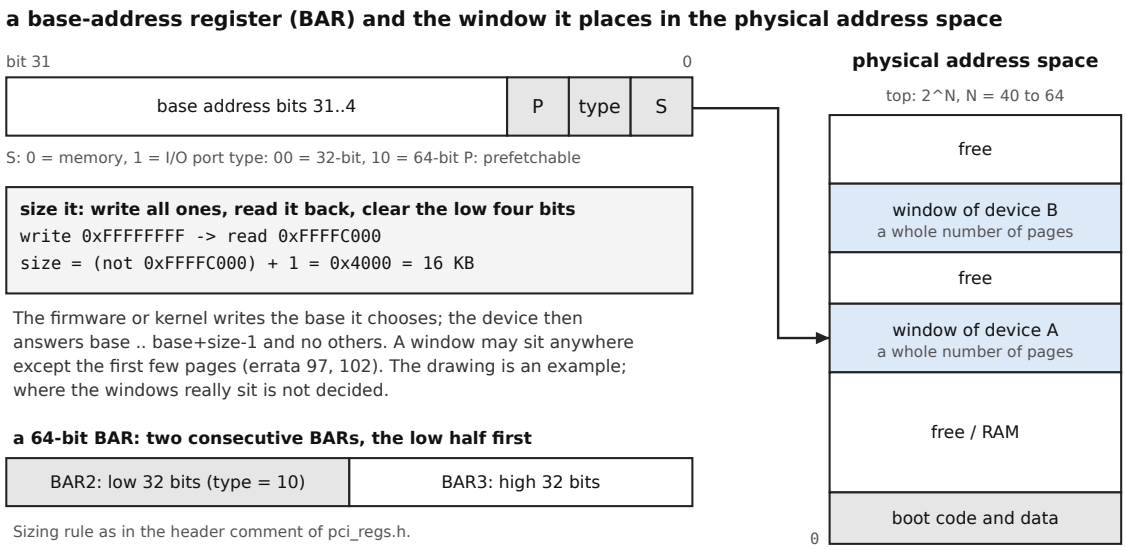
Capabilities. The header is fixed; everything optional hangs off a **capability list**: a linked list, inside the configuration space, of small records, each starting with an identifier byte and a byte holding the offset of the next record (0 ends the list). It is a linked list of exactly the kind drawn in section 2.6. The records a pdp64 driver looks for are the MSI-X record (identifier 0x11, section 2.7), the PCI Express record (0x10) and the vendor-specific record (0x09) which virtio uses to say where its structures are (section 2.6).

Backward compatibility, and what the pdp64 adds. A PCI Express endpoint is written to look to software like a **PCI** device (PCI is the parallel bus of the 1990s from which PCIe took its configuration space, its addressing and its software model), which is why one driver model has served thirty years of hardware. The author's decision (errata 80) is that Q64 is PCI Express and so an ordinary card works on it unchanged, with circuitry added on the processor side for convenience and security (2.3.5).

2.3 Q64 and the address space

Memory-mapped registers. On the PDP-11 the registers of a device are **memory-mapped** (they have addresses in the same address space as memory, so the instruction that reads memory reads a register). Q64 keeps this and widens it: a

function's registers are in **windows** (ranges of the physical address space that the function answers), and a driver reaches them with ordinary loads and stores, with no input or output instructions. What changed is who chooses the addresses. On the Unibus the switches on the card did. In PCI Express the function does not know where it will live; it says how much room it needs, the system assigns an address, and the function answers there (section 2.4). A window in the address map looks like this, and the **base-address register** (BAR: a register in the configuration-space header that holds where a window starts and, by a trick of section 2.4, tells how big it is) is what places it:



Two rules of the old design are gone and a third is new. The PDP-11's fixed I/O page at the top of memory is gone (the author's decision, errata 97, 102: no address bit says "device"); the address is wider (40 to 64 bits, against the Unibus's 18); and a window may be placed anywhere except the first few pages, where the boot code and boot data are loaded.

2.3.1 Q64 is the system bus: PCI Express, so that an ordinary PCI Express device works on it unchanged, with the programming model of the PDP-11 Q-bus (devices as memory locations in the one address space) widened to 64-bit data and a physical address that is not limited to 40 bits (the simulator and the emulator implement 40), and with convenience and security circuitry added. *(D)*

2.3.2 Devices live in the ordinary address space; the processor has no input or output instructions. *(D)*

2.3.3 The windows of a device (the regions its base-address registers describe) start on 4 KB page boundaries and are a whole number of pages, so that one device can be mapped to one protection domain. *(P)*

2.3.4 Every page has a memory type: normal cached, normal uncached, device strongly ordered, or device weakly ordered. *(D)*

2.3.5 The transactions of Q64 are those of PCI Express; the circuitry the pdp64 adds is for convenience (overlays, helper registers) and security (the IOMMU, error and reset rules), and is measured against the list of pain points of `kernel-pain-points.md`. An extension is announced to a card, through a PCI Express extended capability, only if the card must take part in it; a mitigation that lives on the processor side is invisible to cards. *(D)* The detail of the extensions is not specified. *(S)*

2.3.6 No bit of the physical address selects device space: a device window is an ordinary range of physical addresses, found by enumeration (section 2.4), and the physical address is at least 40 bits and at most 64, an implementation choosing its width in that range, so that mappings of petabytes are possible. *(D)* A window can be mapped anywhere in the physical address space except the first few pages, where the boot code and boot data are loaded. *(D)* How many pages are "the first few" is not decided, *(P)* and nor is how software discovers the implemented width. *(S)* The simulator and the emulator place their devices in the upper half of a 40-bit space, from `0x8000000000`, and check the memory type of a page against that bit; those two arrangements belong to the present implementation, not to the architecture.

Commentary on 2.3.6. **Precedent.** The VAX-11/780 divided its physical address space in two: memory in the low half and the I/O space above `0x20000000` (the 4.3BSD kernel's `NEX780`), which holds one 8 KB window (`NEXSIZE`) per adaptor slot; the 4.3BSD `nexus.h` also records the 8600's, 8200's and MicroVAX's different bases, so the VAX itself kept changing the map

from model to model. The PDP-11 put its devices in a fixed page at the top of the address space (a V7 driver's pointer to its registers is a constant in that page). PCI Express places each function's windows where enumeration assigns them. **Trade-offs.** A decoding bit makes the first routing decision in the bus fabric trivial and makes a mapping of the wrong memory type detectable, but it costs half of the physical address space; the author dropped it because mapping petabytes needs those bits. Without it, a device's DMA into another device's windows has to be refused by the IOMMU (errata 85) and not by the address. **Modern comparables.** Most systems split the map by address ranges chosen by the platform; x86 has a separate port space. **Impact.** *Kernel:* devices are mapped where enumeration says, and a device mapping is recognised by its memory type and its window, not by an address bit. *Compiler:* none. *Hardware:* the root complex decodes the windows it has assigned.

2.4 Enumeration and hot-plug

Finding the devices. The Unibus kernel is told what is there; the Q64 kernel asks. **Enumeration** (walking the whole tree and reading each function's configuration space to find out what is there) goes: for every bus, for every device number 0 to 31 and function number 0 to 7, read the vendor ID; if a function answers, read its header type; if it is a bridge, give the bus behind it a number and walk that bus too; if it is an ordinary function, size and place its windows and record it. The pseudocode (the bus numbers follow the `PCI_PRIMARY_BUS`, `PCI_SECONDARY_BUS` and `PCI_SUBORDINATE_BUS` bytes at offsets 0x18 to 0x1A of a bridge's header):

```
next_bus = 1                                # bus 0 is the root complex's own

scan(bus):
    for dev in 0 .. 31:
        for fn in 0 .. 7:
            vendor = cfg_read16(bus, dev, fn, 0x00)
            if vendor == 0xFFFF:            # nothing here
                if fn == 0: break           # no function 0, no card: skip its other functions
                continue
            htype = cfg_read8(bus, dev, fn, 0x0E)
            if (htype & 0x7F) == 1:         # a bridge: a bus lies behind it
                secondary = next_bus++
                cfg_write8(bus, dev, fn, 0x18, bus)          # primary bus
                cfg_write8(bus, dev, fn, 0x19, secondary)    # secondary bus
                cfg_write8(bus, dev, fn, 0x1A, 255)          # subordinate: provisional
                scan(secondary)                              # the recursion
                cfg_write8(bus, dev, fn, 0x1A, next_bus - 1)  # now the real highest number
            else:
                size_and_place_bars(bus, dev, fn)
                add_to_device_list(bus, dev, fn, vendor, cfg_read16(bus, dev, fn, 0x02))
            if fn == 0 and not (htype & 0x80): break         # single-function card
```

(`cfg_read16` is a load from the ECAM address of section 2.2. The test for an empty position reads all ones in standard PCI Express and in QEMU; the pdp64's root complex answers all ones for a position that never held a device (2.4.9). The windows of a bridge must also be programmed so that the traffic for the functions behind it passes; that step is omitted here.)

Sizing a window. The register must say how big its window is, and it is the same register that holds the address. The trick is that the bits below the size are not storage: they always read as zero. So the system writes all ones, reads the register back, and the bits that stuck show the size (the header comment of `pci_regs.h` says the same: "writing a value of 0xffffffff to the register, and reading it back. Only 1 bits are decoded"; QEMU builds the answer as `~(size - 1)`). The low four bits of a memory BAR are not part of the address: bit 0 is 0 for memory and 1 for an I/O port, bits 2:1 give the width (00 is a 32-bit address, 10 a 64-bit one that uses the **next** BAR as its high half), and bit 3 says the memory is **prefetchable** (reading it has no side effects, so it can be read ahead).

```
size_bar(bus, dev, fn, index):                      # returns (size, is_64bit)
    reg = 0x10 + 4 * index
    cmd = cfg_read16(bus, dev, fn, 0x04)
    cfg_write16(bus, dev, fn, 0x04, cmd & ~0x2)     # stop the function answering memory while we play
    old = cfg_read32(bus, dev, fn, reg)
    cfg_write32(bus, dev, fn, reg, 0xFFFFFFFF)
    got = cfg_read32(bus, dev, fn, reg)
```

```
    cfg_write32(bus, dev, fn, reg, old)              # put it back
    is64 = ((got >> 1) & 3) == 2
    if is64:                                         # the same dance on the high half
        oldh = cfg_read32(..., reg + 4)
        cfg_write32(..., reg + 4, 0xFFFFFFFF)
        goth = cfg_read32(..., reg + 4)
        cfg_write32(..., reg + 4, oldh)
        mask = (goth << 32) | (got & 0xFFFFFFF0)
        size = (~mask + 1) & 0xFFFFFFFFFFFFFFFF
    else:
        mask = got & 0xFFFFFFF0
        size = (~mask + 1) & 0xFFFFFFFF
    cfg_write16(bus, dev, fn, 0x04, cmd)             # restore the command register
    return (size, is64)

size_and_place_bars(bus, dev, fn):
    index = 0
    while index < 6:
        (size, is64) = size_bar(bus, dev, fn, index)
        if size != 0:
            base = allocate(size, align = size)      # a window starts on a multiple of its own size
            cfg_write32(bus, dev, fn, 0x10 + 4*index, base & 0xFFFFFFFF)
            if is64: cfg_write32(bus, dev, fn, 0x14 + 4*index, base >> 32)
        index += 2 if is64 else 1
    cmd = cfg_read16(bus, dev, fn, 0x04)
    cfg_write16(bus, dev, fn, 0x04, cmd | 0x2 | 0x4) # answer memory (bit 1); may be a bus master (bit 2)
```

The worked example of the figure: reading back 0xFFFFC000 means the low 14 bits are not storage, so the size is 2^{14} = 16 KB. On the pdp64 a window is at least a page, 4 KB, and a whole number of pages (2.3.3), so that one device can be given to one protection domain. Note the last line: until bit 2 of the command register is set the function may not start a transfer of its own, which is the step that makes it a bus master (section 2.5).

2.4.1 The processor finds the devices by PCI Express enumeration: it reads the configuration space of each function through the root complex, sizes the base-address registers of the function and assigns its windows in device space. *(D)*

2.4.2 A function is a virtio device when its configuration space says so (the virtio vendor identifier and a device identifier that carries the virtio device type); the capabilities in its configuration space locate the virtio structures in its windows (chapter 5). One transport driver serves every virtio device. *(D)*

2.4.3 A function may itself be a bus: a PCI Express bridge presents the functions behind it, and a USB host controller (itself a PCI Express function) enumerates its own tree of hubs and devices, so any bus can be attached to Q64 and its devices enumerated, to any depth. The kernel's enumeration is one recursive walk. *(D)*

2.4.4 The bus alerts the processor when a device enters or leaves, by an interrupt; the kernel then rescans the bus, which is also what it does at boot. *(P)*

2.4.5 When a device leaves, an access to its windows is at once a precise bus fault, not a read of all ones: a read of the window faults, a posted write is dropped and counted, and the device's in-flight bus transactions are aborted. *(P)*

2.4.6 The queued work of a departed device stays in memory untouched, so the driver can read, save, resend or fail it. *(P)*

2.4.7 A bus reset produces no removal events, so a reset is not mistaken for the removal of every device, and the identity of every function is defined after the bus has lost power. *(D)*

2.4.8 Where the configuration space and the windows of the functions sit in the physical address space (fixed bases and sizes) is unspecified, as is how a hot-plug event behind a bridge is reported. *(S)*

2.4.9 A read of a configuration position that never held a function completes normally and returns all ones in every byte, so a vendor ID of `0xFFFF` means that nothing is there; it is not a bus fault (the read of other non-existent addresses stays a precise `BUSFAULT`, 6.4.3). *(D)*

2.4.10 The root complex records such a miss in a sticky bit of its own status register, cleared by writing a one to it, so that a dead link can be told from an empty slot; the register and its place are not specified. *(D)*

Commentary. **Precedent.** The VAX scanned its interconnect: `probenexi()` in 4.3BSD reads each nexus slot's status register, whose type field says what is plugged in; the Unibus had no enumeration at all, with devices at switch-set addresses and `badaddr()` probes, and nothing told the kernel when a device left. PCI sizes each device's windows by writing all ones to

its base registers and assigns addresses; Amiga AutoConfig lets each board announce itself at a fixed address and be told where to live; NuBus gave each slot a fixed share of the address space. **Trade-offs.** Assignment on demand supports any layout at the cost of a sizing step at boot; a fixed window per slot needs no assignment and limits the slot count. Standard enumeration costs the processor nothing it does not already do for a PCI Express device, and means an ordinary card works unchanged. **Improvements.** On a surprise removal PCI returns all ones, so every driver must treat all ones as a possible error; the pdp64 uses the precise bus fault the instruction set already defines, so the driver has a single recovery path. Rescan on an event makes hot-plug and boot the same code. **Benefits.** The kernel needs one address, the root complex's, to find every device. **Modern comparables.** PCI Express enumerates through configuration space with hot-plug signalled by a slot-status interrupt; USB hubs report port changes through a status-change endpoint; NVMe and `virtio` keep their queues in host memory and re-queue them after a reset. **Impact.** *Kernel:* Enumeration is one recursive loop over the functions of each bus; hot-plug is an interrupt and a rescan. *Compiler:* None. *Hardware:* The root complex is a standard one; the pdp64 adds the precise fault for an absent device.

Commentary on 2.4.3. **Precedent.** The VAX did this: the Unibus hung off the VAX-11/780's interconnect through a Unibus adapter, a device on the nexus whose own registers include a table of map registers for DMA and whose Unibus memory (the devices' registers) appears in a window of its own (`UMEM780(i)` in `vaxuba/ubareg.h`, 4.3BSD); `unifind()` in `vax/autoconf.c` probes the devices behind each adapter. PCI-to-PCI bridges nest buses, a USB host controller enumerates a tree of hubs and devices, and Thunderbolt tunnels PCI Express over a cable. **Trade-offs.** Each level adds a mapping of windows and a hop of latency, and a departure deep in the tree must be reported to a driver that does not know the path; the cure for the first is that a bridge's windows are ordinary device space (ordinary device windows, section 2.3.6). **Improvements.** A new bus costs one bridge and no change to the processor, the kernel's walk or the driver model. **Benefits.** Any bus a board offers is connected by a bridge. **Modern comparables.** PCI Express switches and bridges, USB hubs, Thunderbolt, CXL, and the device-tree and ACPI descriptions of nested buses. **Impact.** *Kernel:* one recursive enumeration routine and one hot-plug path for every bus. *Compiler:* none. *Hardware:* a bridge is a standard PCI Express bridge.

2.5 DMA, coherence and the hand-off

Bus mastering. On the Unibus any device could be a master after winning an NPR grant. In PCI Express a function is a **bus master** (it may originate memory reads and writes of its own, as packets, instead of only answering) when the processor sets bit 2 of its command register; until then it only answers. A DMA transfer is therefore a function sending memory-write packets (to deliver data into memory) or memory-read packets (to fetch data from it) whose addresses are in the memory the processor gave it. There is no request line, no grant chain and no priority between devices: the tree's routers carry the packets. The danger is the one the Unibus map registers hinted at: a device that can write anywhere can write into the kernel. That is the subject of section 2.11.

The hand-off. Moving data is the easy half; telling each side that the data is ready is the hard half, and it is what a queue in memory is for (section 2.6). Statement 2.5.3 below is the five steps in order.

2.5.1 Between processors the data caches are coherent without any software action, by snooping with three states (modified, shared, invalid), for two to four cores. *(D)*

2.5.2 Between a processor and a DMA device they are not: buffers are mapped uncached and the hand-off is explicit. *(D)*

2.5.3 The hand-off is five steps: the processor builds the descriptors and data in uncached memory, executes `FENCE`, writes the device's notification address; the device walks the descriptors, moves the data by DMA, writes the result and its used-ring entry, and raises an interrupt; the processor reads the result. *(D)*

2.5.4 The write that raises a device's interrupt (a message-signalled write) cannot overtake the data writes that precede it on the same path. *(D)*

Commentary. **Precedent.** The 11/70's cache watched Unibus writes so that DMA appeared coherent; the VAX's Unibus adapter had buffered data paths that a driver had to purge. **Trade-offs.** Snooping the DMA path is expensive and DMA transfers are large, infrequent and have an explicit hand-off, so two mechanisms are used, each where it is cheap; the cost is that the processor reads received data from uncached memory, which is slow, and a cache-maintenance instruction would remove it for a non-coherent device that may come later. **Benefits.** No driver can forget a cache clean or invalidate, because there is none to forget. **Modern comparables.** x86 DMA is coherent; ARM and MIPS systems with non-coherent DMA need a DMA-mapping layer in the kernel to place cache maintenance correctly. **Impact.** *Kernel:* Uncached DMA buffers

mean no driver cache code. *Compiler:* None. *Hardware:* A snoop port on each data cache for core coherence only; no snoop port for DMA.

2.6 virtio: one shape for every device

The Unibus gave every device its own registers, and every driver learned them. **virtio** (a specification for devices that are meant to be imitated in software: the name is from "virtual I/O") takes the other road: every device, whatever it does, is the same few structures in memory, and only the meaning of the requests differs. A **driver** (the program that runs a device) and a **device** (the hardware, or the emulator that pretends to be it) talk by **shared memory** (both sides read and write the same region of memory, according to a fixed protocol, instead of passing values through registers). The processor decided on it for every standard device (errata 95, 79, 101): the network card, the graphics device, the audio device and the disk are virtio-net, virtio-gpu, virtio-snd and virtio-blk, so the one transport driver of section 2.6.2 and a thin driver for each kind of device serve the machine, and the same devices run in QEMU, the simulator and the FPGA. Chapter 5 states the rules; this section teaches the shape.

2.6.1 The five ideas, against the RK11

the RK11	virtio
the kernel is told where the device is	the device is found by enumeration, by vendor and device ID (2.6.2)
six registers, each with bits of its own	a standard set of registers of **feature bits** and a **status** byte, the same for every device (2.6.3)
one command at a time: load the registers, GO	a **virtqueue**, a queue of commands in memory, so many are outstanding at once (2.6.4)
the command is in the registers	the command is a **descriptor chain**, boxes in memory linked like a list (2.6.5)
READY bit, or an interrupt, says it is done	the device writes the finished chain into the **used ring** in memory, and may interrupt (2.6.6, 2.7)

2.6.2 Finding a virtio device

A virtio function is recognised by its identifiers: the vendor ID is 0x1AF4 (the vendor ID of Red Hat, which made the specification; QEMU's `pci.h` has `PCI_VENDOR_ID_REDHAT_QUMRANET`), and its device ID is 0x1040 plus the **device type** (QEMU's `PCI_DEVICE_ID_VIRTIO_10_BASE`). The types are in the Linux header `virtio_ids.h`: network 1, block 2, console 3, GPU 16, sound 25, so virtio-blk is device ID 0x1042, virtio-net 0x1041, virtio-gpu 0x1050 and virtio-snd 0x1059. (The "modern" devices; an older numbering at 0x1000 and up exists and is not used, section 2.6.3.)

The registers are not at fixed places in the window. The function's **vendor-specific capabilities** (capability identifier 0x09 in the list of section 2.2) say where each structure is. Each record is the 16 bytes of `struct virtio_pci_cap`: the identifier, the next-record offset, the record length, a `cfg_type` byte that says which structure it describes, a `bar` byte (which of the six windows), an `id` byte, two bytes of padding, a 32-bit `offset` into that window and a 32-bit `length`. The `cfg_type` values are 1 common configuration, 2 notifications, 3 interrupt status, 4 device-specific configuration, 5 a PCI configuration access window, 8 shared memory, 9 vendor data (`virtio_pci.h`); a driver for the pdp64 needs 1 to 4. The notification record is two words longer: it ends with a 32-bit `notify_off_multiplier`.

```
find_virtio_structures(fn):
    cap = cfg_read8(fn, 0x34)                       # head of the capability list
    while cap != 0:
        if cfg_read8(fn, cap + 0) == 0x09:          # vendor-specific: could be virtio
            type   = cfg_read8 (fn, cap + 3)
            bar    = cfg_read8 (fn, cap + 4)
            offset = cfg_read32(fn, cap + 8)
            length = cfg_read32(fn, cap + 12)
            where[type] = bar_base(fn, bar) + offset         # the physical address of the structure
            if type == 2: notify_multiplier = cfg_read32(fn, cap + 16)
        cap = cfg_read8(fn, cap + 1)                # follow "next"
```

2.6.3 The handshake: status and features

The **common configuration** structure is a block of registers, the same in every device; chapter 5 (5.2.2) lists them with their offsets, from `device_feature_select` at 0x00 to `queue_device` at 0x30. Three are the handshake.

`device_status` is one byte whose bits the driver sets in order, each meaning "I have got this far": ACKNOWLEDGE (1, "I have seen you"), DRIVER (2, "I know how to drive you"), FEATURES_OK (8, "I have chosen my features"), DRIVER_OK

(4, "I am running"); the device may set NEEDS_RESET (0x40) and the driver FAILED (0x80) (`virtio_config.h`). Writing 0 resets the device. This is the READY and GO of the CSR, spread over a conversation.

Feature bits are how two sides that were not written together agree. The device offers a 64-bit set of **features** (optional abilities, each one a bit; for example "I can chain descriptors indirectly", bit 28), reading them through a window of 32 bits at a time selected by `device_feature_select`; the driver writes back the subset it will use. Nothing outside the agreed subset is used. A new device can add a feature without breaking an old driver, which simply never asks for it: this is how the same driver keeps working as devices gain abilities. One feature is required of every device the pdp64 drives, `VIRTIO_F_VERSION_1` (bit 32: "modern layout"; errata 95 and 5.2.5). Two more matter to the processor and are open (2.13): `VIRTIO_F_ACCESS_PLATFORM` (bit 33: device addresses go through the platform's translation, section 2.11) and `VIRTIO_F_ORDER_PLATFORM` (bit 36: the device respects the platform's ordering rules).

```
init_device(fn):
    status = 0;               write_status(0)                  # reset
    write_status(ACKNOWLEDGE)                                  # 1
    write_status(status |= DRIVER)                             # 2
    wanted = (1 << 32) | class_driver_features                 # VERSION_1 and what this driver uses
    offered = read_features_64()                               # 2 reads through device_feature_select
    if (offered & (1 << 32)) == 0: fail()
    write_features_64(wanted & offered)                        # driver_feature_select, driver_feature
    write_status(status |= FEATURES_OK)                        # 8
    if (read_status() & FEATURES_OK) == 0: fail()              # the device may refuse the subset
    for q in 0 .. num_queues - 1: setup_queue(q)               # section 2.6.4
    write_status(status |= DRIVER_OK)                          # 4: go
```

2.6.4 A virtqueue: three areas of memory

A **virtqueue** (a queue of requests that lives in ordinary memory, shared by the driver and the device) is three arrays, each in memory the driver allocates: the **descriptor table** (the requests), the **available ring** (the driver's list of which requests are new) and the **used ring** (the device's list of which are finished). The size, a power of two, is a number of entries chosen by the driver up to the device's maximum. Sizes: a descriptor is 16 bytes aligned to 16, the available ring is aligned to 2 and the used ring to 4 (the `VRING_*_ALIGN_SIZE` of `virtio_ring.h`); all fields are little-endian (5.1.3). The driver tells the device where the three are by writing their physical addresses into `queue_desc`, `queue_driver` and `queue_device` (offsets 0x20, 0x28 and 0x30) after selecting the queue with `queue_select`, and then sets `queue_enable`.

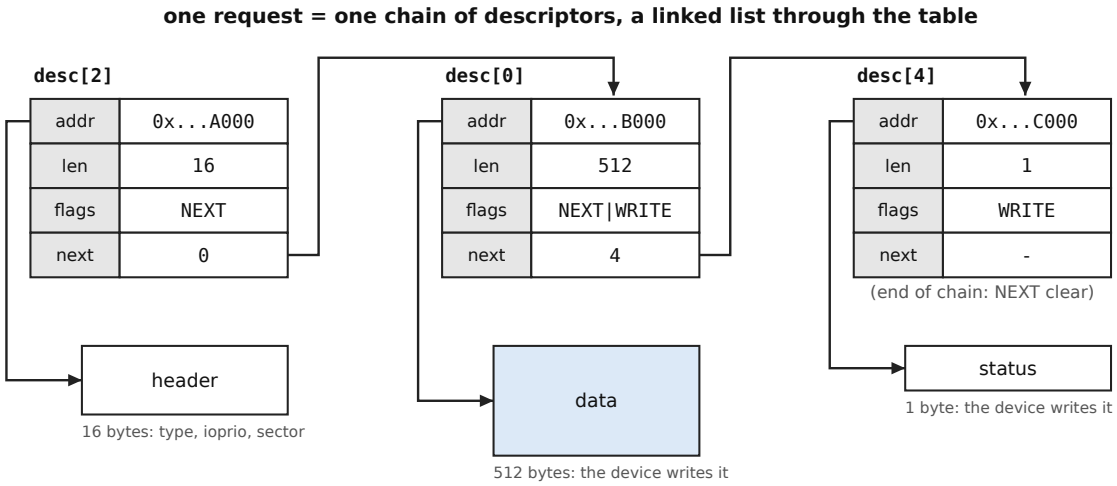
```
setup_queue(q):
    write16(queue_select, q)
    n = min(read16(queue_size), 256)               # the device proposes a size; the driver may lower it
    write16(queue_size, n)
    desc  = alloc_uncached(16 * n, align = 16)     # the descriptor table
    avail = alloc_uncached(6 + 2 * n, align = 2)   # flags, idx, ring[n], one optional word
    used  = alloc_uncached(6 + 8 * n, align = 4)   # flags, idx, ring[n] of 8-byte elements
    zero(desc, avail, used)
    write64(queue_desc, desc);  write64(queue_driver, avail);  write64(queue_device, used)
    write16(queue_msix_vector, vector_for(q))      # section 2.7
    write16(queue_enable, 1)
    notify_addr[q] = where[2] + read16(queue_notify_off) * notify_multiplier    # the doorbell of this queue
    free_head = 0; link all n descriptors into a free list through their `next` fields
    avail_idx = 0; last_used = 0                   # the driver's private counters
```

(The queue memory is in uncached memory on the pdp64, 2.5.2, so no cache instructions appear anywhere. The sizes 6 + 2n and 6 + 8n follow from the structures `struct vring_avail` and `struct vring_used` of `virtio_ring.h`, each two 16-bit fields followed by the ring, and a final optional 16-bit field, section 2.6.9.)

2.6.5 The descriptor, and the chain

A **descriptor** is 16 bytes, four fields (`struct vring_desc`): `addr`, a 64-bit physical address of a buffer; `len`, a 32-bit length in bytes; `flags`, 16 bits; and `next`, a 16-bit index of another descriptor. The flags are `NEXT` (1: the request continues in the descriptor `next` names), `WRITE` (2: the device will write into this buffer; absent, the device only reads it) and `INDIRECT` (4: the buffer is itself a table of descriptors). One request is a **chain** (a sequence of descriptors, each pointing to the next by index, ending at one

without NEXT): in the language of a textbook, a linked list whose nodes are the 16-byte descriptors, whose pointers are table indices and whose payloads are the buffers.

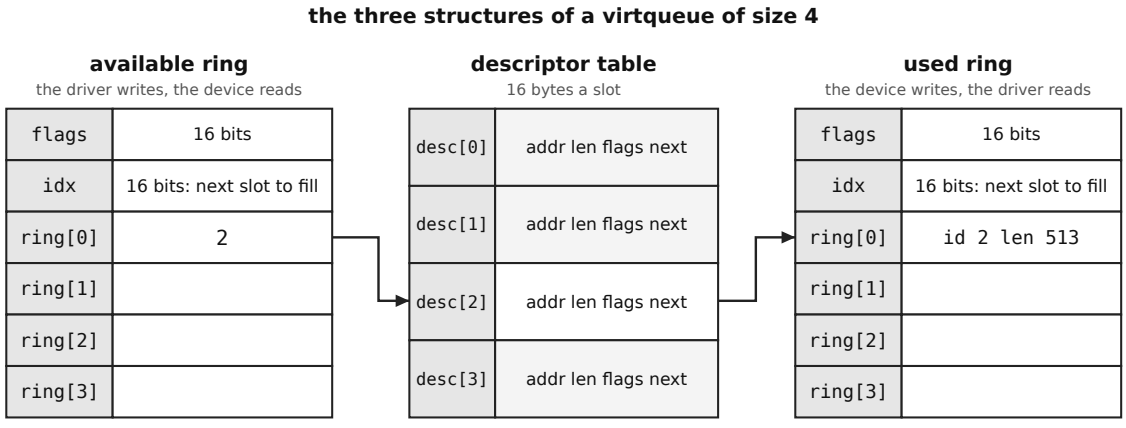


The cards are not in the table in order: the chain 2, 0, 4 is the order of the links, as in any linked list. Indices are examples.

The figure is a virtio-blk read of one sector. The first box is the header the driver filled in (`struct virtio_blk_outhdr`: a 32-bit request type, a 32-bit priority, a 64-bit sector number counted in 512-byte units; type 0 is a read, 1 a write, 4 a flush: `virtio_blk.h`); it has no `WRITE` flag because the device only reads it. The second is the 512-byte buffer for the data, with `WRITE`, because for a read the device fills it. The third is one byte the device writes with the result (0 is OK, 1 an I/O error, 2 unsupported). A chain may use any free slots of the table in any order; the free ones are kept on a list of the driver's own. A buffer that straddles a page is simply two descriptors (5.4.3). Compare the RK11: the whole request was in registers, and could not be more than one; here it is memory the device reads, and the table holds many.

2.6.6 The available ring and the used ring

The two rings are arrays of 16-bit and 32-bit numbers with an index each.



The driver offers a chain by writing its head into ring[idx mod size] and then adding 1 to idx.
The device reports it done by writing (head, bytes written) into the used ring and then adding 1 to its idx.
Each side keeps its own private count of how far it has read the other side's ring. Indices are examples.

The **available ring** (`struct vring_avail`) is a 16-bit `flags`, a 16-bit `idx` and `ring[]`, 16-bit entries. The driver puts the index of the **head** (first descriptor) of a chain into `ring[idx mod size]` and then adds 1 to `idx`; `idx` only ever increases and wraps at 65536, so that "how many have been offered in all" is the difference of two readings and no extra state is kept. The **used ring** (`struct`

`vring_used`) is the mirror: `flags`, `idx` and `ring[]` of **used elements** (`struct vring_used_elem`), each a 32-bit `id` (the head of the finished chain) and a 32-bit `len` (the number of bytes the device wrote into the chain's buffers). The device writes the element and then adds 1 to its own `idx`. Each side keeps a private count of how far it has read the other's ring (`last_used` for the driver); the rings are single-writer, so neither side ever waits for the other to release anything: the one place a lock would be needed on the Unibus (two parties writing one location) does not occur. This is the **producer-consumer ring** (a fixed circular array with one writer and one reader, each owning one index) that the 4.3BSD DEUNA's transmit and receive rings already were, except that the "own" bit of each DEUNA entry is replaced by the two indices.

2.6.7 Posting a buffer and reaping the completions

```
post(q, head):                                      # head: first descriptor of a built chain
    avail.ring[avail_idx mod n] = head              # 1. name the chain
    FENCE                                           # 2. the descriptors and the ring entry are visible...
    avail_idx = avail_idx + 1 (mod 65536)
    avail.idx = avail_idx                           # 3. ...before the device is told there is one more
    FENCE
    if (used.flags & NO_NOTIFY) == 0:               # the device may ask not to be rung (section 2.6.9)
        mmio_write16(notify_addr[q], q)             # 4. the doorbell

reap(q):                                            # called from the interrupt routine, or when polling
    while last_used != used.idx:                    # the device has added finished chains
        FENCE                                       # read the index first, then the element and the data
(5.4.4)
        e = used.ring[last_used mod n]              # e.id = head of chain, e.len = bytes written
        complete(e.id, e.len)                       # the class driver reads status, frees the chain
        free_chain(e.id)
        last_used = last_used + 1 (mod 65536)
```

The doorbell is an ordinary 16-bit write to a notification address: the address is the notification area's base plus `queue_notify_off` times the `notify_off_multiplier` (QEMU decodes the queue from the address, `addr / multiplier`; what value the specification asks the driver to write is *unverified*: the code above writes the queue number, which QEMU ignores). The `FENCE` between step 1 and step 3 is the pdp64's statement 2.8.2; the processor owes nothing else (2.8.1).

A complete read of one sector, from the driver's side, putting the pieces of this section together:

```
blk_read(sector, buf):                              # buf: a physical address of 512 bytes
    h = alloc_hdr();  h.type = 0; h.ioprio = 0; h.sector = sector      # 16 bytes
    s = alloc_status_byte()
    d0 = get_free();  d1 = get_free();  d2 = get_free()
    desc[d0] = { addr: h,   len: 16,  flags: NEXT,         next: d1 }
    desc[d1] = { addr: buf, len: 512, flags: NEXT | WRITE, next: d2 }
    desc[d2] = { addr: s,   len: 1,   flags: WRITE,        next: 0  }
    post(q, d0)
    wait_for_completion(d0)                         # polled or interrupt-driven
    return status_byte(s) == 0
```

2.6.8 The polled driver, two ways

The two loops side by side, the RK11 of section 2.1 and the same job as virtio-blk:

```
RK11 (the PDP-11 way)                       virtio-blk (the virtio way)
------------------------------------------  ------------------------------------------
write RKDA, RKBA, RKWC                      build 3 descriptors in memory (blk_read above)
write RKCS = READ | GO                      post(q, head)           # one doorbell write
while (RKCS & READY) == 0 : spin            while last_used == used.idx : spin   # reads memory
if RKCS & ERROR : read RKER, reset          status = the byte the device wrote in d2
one command in flight                       up to `size` commands in flight
the registers are this device's             the structures are every device's
```

The only difference in the shape of the polling loop is that the thing being read is a word of memory, the used index, and not a register. A memory read is an ordinary load, cheap and cacheable-or-not as the page says, not a transaction on the device's link; this is also why a driver can poll the used ring while the device is on the other side of a bridge.

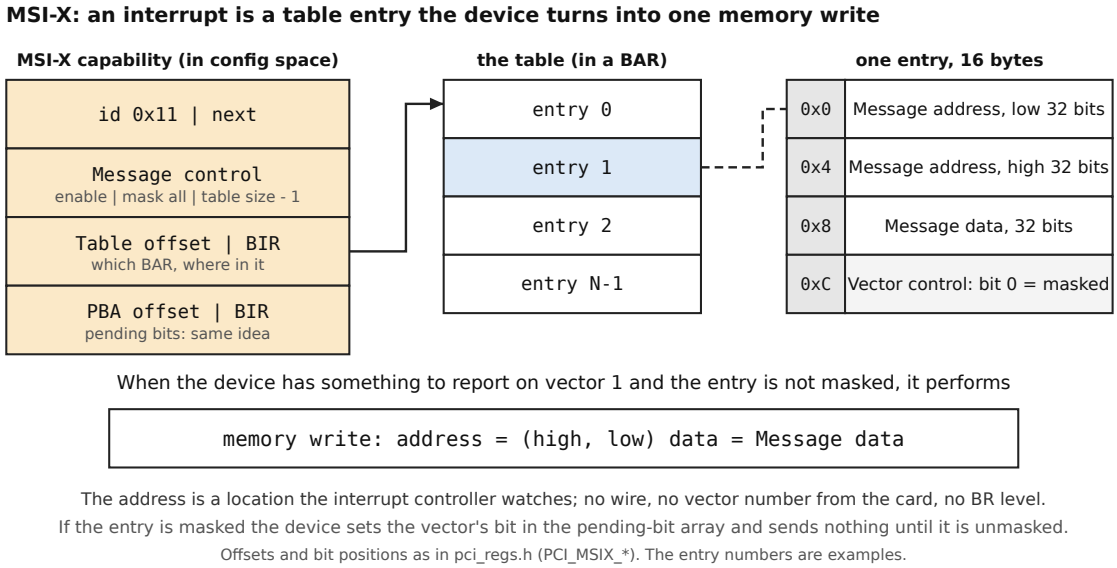
2.6.9 Suppressing notifications and interrupts

Doorbells and interrupts cost time on both sides, so each side can ask the other to stay quiet. Both flags are in the rings' `flags` fields and both are **hints** (a request the other side may, for a short time, not honour; the driver must work either way). The device sets `VRING_USED_F_NO_NOTIFY` (bit 0 of `used.flags`) to say "do not ring the doorbell; I am looking at the available ring anyway", which is what a device that is busy with a stream does. The driver sets `VRING_AVAIL_F_NO_INTERRUPT` (bit 0 of `avail.flags`) to say "do not interrupt; I am polling", which is how a driver turns interrupts off for a burst of completions. A finer scheme exists when feature bit 29 (`VIRTIO_RING_F_EVENT_IDX`) is agreed: each side writes into an extra 16-bit word at the end of its ring the **index** at which it wants to be woken, and the other side then rings or interrupts only when it crosses that number (`virtio_ring.h`: the driver "publishes the used index for which it expects an interrupt at the end of the avail ring"; the host "publishes the avail index for which it expects a kick at the end of the used ring"). Whether the pdp64 uses it is not decided (2.13). A driver may mask a device's interrupts and poll the used ring (2.7.4).

2.7 Interrupts that are writes: MSI and MSI-X

On the Unibus an interrupt was a signal: a BR line went up, the processor granted the bus, and the device put its vector on the data lines. A PCI Express link has no such lines to raise. The first answer, kept for old devices, was four shared **interrupt pins** per slot (`INTA` to `INTD`; the header's **interrupt line** and **interrupt pin** bytes at 0x3C and 0x3D describe them, and bit 10 of the command register turns the function's pin off), which several devices share, so that the handler must ask each in turn whether it was the one. PCI Express emulates the pins with messages, for the sake of old software, and the pdp64 does not use them for its standard devices.

The answer for new devices is that **an interrupt is a memory write**. A function that wants attention does not raise a signal; it performs a write, of a value the system chose, to an address the system chose, and the part of the machine that owns that address (the **interrupt controller**: the unit that decides which core runs which handler) takes the arrival of the write as the interrupt. **MSI** (message-signalled interrupts, capability identifier 0x05) gives a function one address and one data value, with a handful of low bits of the data varied to give several interrupts. **MSI-X** (capability 0x11, the "extended" form, and the one the pdp64's virtio devices use, errata 95 and 5.7.1) gives a function a **table** (an array in one of its windows) of entries, up to 2048 of them (the table-size field is 11 bits, `PCI_MSIX_FLAGS_QSIZE` 0x07FF), each with a separate address and data, so that every queue of every device can have an interrupt of its own, aimed at a core of its own.



The capability record (`pci_regs.h`, offsets from the record's start) is: the identifier and next byte; a 16-bit **message control** at 2, whose bit 15 enables MSI-X, bit 14 masks every vector of the function and bits 10:0 hold the table size minus one; a 32-bit word at 4 whose low three bits say which BAR holds the table (the **BIR**, BAR indicator register) and the rest an offset in it; and a 32-bit word at 8 of the same form for the **PBA**, the **pending-bit array** (one bit per vector, set by the function when it wanted to interrupt and could not because the entry was masked). An entry is 16 bytes: the message address, low

half at 0 and high half at 4; the message data at 8; and at 12 the **vector control**, whose bit 0 is the **mask** (while it is set the function sends nothing for that vector and records the wish in the PBA).

```
enable_msix(fn, nvec):
    cap  = find_capability(fn, 0x11)
    ctrl = cfg_read16(fn, cap + 2)
    table_bar    = cfg_read32(fn, cap + 4) & 7
    table_offset = cfg_read32(fn, cap + 4) & ~7
    table = bar_base(fn, table_bar) + table_offset
    cfg_write16(fn, cap + 2, ctrl | 0x8000 | 0x4000)     # enable, but mask every vector for now
    for v in 0 .. nvec - 1:
        entry = table + 16 * v
        (addr, data) = interrupt_controller_allocate(core_for(v), handler_for(v))
        mmio_write32(entry + 0,  addr & 0xFFFFFFFF)
        mmio_write32(entry + 4,  addr >> 32)
        mmio_write32(entry + 8,  data)
        mmio_write32(entry + 12, 0)                      # unmask this vector
    cfg_write16(fn, cap + 2, (ctrl | 0x8000) & ~0x4000)  # lift the function-wide mask

# in virtio, each queue is told its vector, and the device-wide changes have one too:
write16(config_msix_vector, 0);  write16(queue_msix_vector, 1)    # 0xFFFF means no vector
```

When the device raises vector *v*, it sends a write of `data[v]` to `address[v]`, if the entry is not masked; a handler then learns what happened by reading the used ring (2.6.7) and, for the device as a whole, the interrupt-status structure of the virtio capabilities (type 3, section 2.6.2). Compare the Unibus: there were four levels, so a handler had to find out which of the devices at its level had spoken, and the vector was a switch on a card; here there is a vector for each queue, assigned by software, and no wire.

2.7.1 A device interrupts by a message-signalled write (MSI-X): the configuration change and each queue have a vector that the driver sets in the common configuration, and the interrupt status structure says why. *(P)* (This is 5.7.1.)

2.7.2 The write that raises an interrupt cannot overtake the data writes that precede it on the same path, so a handler that runs because of the interrupt finds the data in memory (2.5.4). *(D)*

2.7.3 Which unit of the processor receives the writes, how a core is chosen, how a vector number reaches a handler and how the interrupt controller is programmed (the address and data that `interrupt_controller_allocate` returns above) are not specified. *(S)* This is the interrupt-controller decision R1 of appendix B; the pseudocode shows the shape that any choice has.

2.7.4 A driver may mask a device's interrupts and poll the used ring instead (2.6.9, 5.7.2). *(D)*

Commentary. **Precedent.** The PDP-11's four BR levels and switch-set vectors are a shared, scarce resource; the VAX's Unibus adapters mapped each level's vectors in a table. **Trade-offs.** A message carries its own vector, so no switch and no assignment table are needed, and there can be thousands; the cost is that the interrupt controller must be an address that many devices write, and that a write to the wrong address by a confused device is an interrupt (the IOMMU of section 2.11 is what stops that: it can refuse a write that is not to an allowed address). **Modern comparables.** MSI-X is how PCI Express devices interrupt; the x86 local APIC and the ARM GIC's interrupt translation service both receive writes. **Impact.** *Kernel:* an interrupt routine reads the used ring and the status structure; the table is programmed once at attach. *Compiler:* none. *Hardware:* an MSI-X target in the interrupt controller.

2.8 What the processor owes the bus

2.8.1 The processor owes the bus exactly three things: a memory type per page, a `FENCE` instruction whose meaning against Q64 is defined, and atomics that can reach device memory. *(D)*

2.8.2 A driver can be sure a descriptor is visible before it notifies the device: it builds the descriptor and publishes it in the queue, executes `FENCE`, and writes the queue's notification address (the doorbell). *(D)*

Commentary. **Precedent.** The PDP-11 needed none of this because there was one processor and no cache. **Trade-offs.** Three obligations is the whole contract, which keeps the device side free to change without respecifying the processor; a virtio device is written against the virtio specification and these three promises alone. **Benefits.** A device and a driver can be written against these three promises without knowing the processor's pipeline. **Modern comparables.** ARM's device-memory types and `DMB`/`DSB`, and RISC-V's `FENCE` with I/O bits, play the same roles. **Impact.** *Kernel:* One `FENCE` before the

doorbell is the whole ordering rule. *Compiler:* `FENCE` is an intrinsic. *Hardware:* The memory type is two bits per page-table entry and the bus honours it.

2.9 When an access fails

2.9.1 A read or instruction fetch of non-existent memory is a precise abort with cause `BUSFAULT`, at the faulting instruction. *(D)*

2.9.2 A posted write that times out, and an uncorrectable memory error, are machine checks: imprecise and not restartable. A `FENCE` before a probe makes even a posted write precise at a known point. *(D)*

2.9.3 `PROBER` and `PROBEW` test whether an address can be read or written without faulting. *(D)*

2.9.4 Resetting the bus is a write to a register of the root complex, not an instruction. *(D)*

Commentary. **Precedent.** A PDP-11 read timeout was an unrestartable bus error, fatal to the process; the `nofault` idiom of V6 and V7 existed to probe for devices without dying. **Improvements.** The read timeout is a precise, restartable abort, and `PROBE` does the work of the idiom. **Benefits.** A driver that probes for a device handles a missing one in a few lines of ordinary fault-handling code, which is the recoverability principle at work. **Modern comparables.** x86 machine checks and ARM's `SError` correspond to the imprecise class; PCI returns all ones for a read of a removed device, which every driver must then learn to distrust, and the pdp64 avoids that convention. **Impact.** *Kernel:* A driver probes a missing device in a few lines of ordinary fault handling. *Compiler:* None. *Hardware:* A timeout counter in the bridge and a way to make a read abort precise.

2.10 Atomics on the bus

2.10.1 The atomics are defined on a monitor: a load-linked takes a reservation, a write by any initiator clears it, and a store-conditional succeeds only if it is intact; the Unibus's `DATIP`/`DATO` locked pair is not an architectural primitive. *(D)*

2.10.2 A device that writes memory shared with a core takes part by clearing reservations. *(D)*

Commentary. **Precedent.** The PDP-11's locked read-modify-write and the VAX's interlocked instructions are bus locks. **Trade-offs.** A lock is simple and a bottleneck as cores multiply; a monitor maps onto AXI's exclusive access and the memory systems of FPGAs and modern chips, at the cost of forward-progress rules. **Modern comparables.** RISC-V `LR`/`SC`, AArch64's exclusive loads and stores, and the large-system atomics that perform the operation at the memory. **Impact.** *Kernel:* Lock words and lock-free structures use the atomics of chapter 7. *Compiler:* C11 atomics lower to them. *Hardware:* A reservation monitor and an exclusive access, not a bus lock.

2.11 The IOMMU, honestly

A device that is a bus master (section 2.5) can write anywhere its packets say, and a descriptor is only an address that the driver gave it. If a driver is wrong, or the device is, or the device is a stick of hardware someone has modified, the address can be the kernel's. The Unibus had a gentle form of the cure, the map registers of section 2.1, which exist so that an 18-bit address could reach more memory, and which also, as a side effect, limited what a device could reach. The modern cure is the **IOMMU** (input-output memory management unit: a translator between the device's addresses and memory's addresses, with a page table of its own for each device, exactly as the processor's MMU does for a program). A device is given **device addresses** (also called I/O virtual addresses) in its descriptors; the IOMMU walks that device's table to turn each into a physical address, and refuses the access if the table has no entry for it, or the entry forbids that kind of access. Because the table is per device (the unit is called a **domain**: a set of mappings one or more devices share), a network card can reach its buffers and nothing else.

What is decided, and what is not. The author decided that an IOMMU is part of the design (errata 75). The five recommendations of the synthesis in `decisions-pending.md` (F25) are accepted (errata 85): translation is on by default; each device is its own domain; descriptors contain no kernel pointers (a device never sees an address that is useful to an attacker as an address of the kernel's data); the mappings of the rings, which stay put for the life of the queue, are permanent; and a small buffer is **bounced** (copied into a buffer that is mapped for the device, which can then be the only memory it can reach) instead of mapping the page that holds it with other data. The IOMMU's leaf entry shares bits 0 to 39 with the processor's page-table entry, and bits 40 to 63 of it are free (errata 85). That is all that is decided. The format of the device table, where the unit sits in the machine (in the root complex, as on most systems), how a fault is reported, and the questions of section 2.13 on virtualization are open. Until it exists, a device addresses all of memory and a user-space driver can direct its device to write anywhere (5.4.2, 5.8.2).

Two other facts to know. First, with no address bit that says "device" (errata 97), the address of another device's window is an ordinary address, so the IOMMU is also what refuses a device's DMA into another device's window (2.3.6, commentary). Second, in virtio the feature bit `VIRTIO_F_ACCESS_PLATFORM` (33) is the device's statement that the addresses in its descriptors are subject to the platform's translation, which is the bit that a driver on a machine with an IOMMU must have agreed (section 2.6.3, 2.13).

2.12 What was checked, and what was not

The rule of the project is: check, do not guess. For the facts of this chapter about PCI Express and virtio the specifications themselves are not on this machine. What was available, and what each fact rests on:

- **Checked against the Linux headers on this machine** (`/usr/include/linux/pci_regs.h`, `virtio_pci.h`, `virtio_config.h`, `virtio_ring.h`, `virtio_blk.h`, `virtio_ids.h`): the configuration-space header offsets and the command, status and header-type bits; the base-address register fields and the all-ones sizing rule; the capability-list fields and the MSI-X record, table entry, mask and size fields; the virtio vendor-specific capability layout and its `cfg_type` values; the common configuration structure; the status bits; feature bits 28, 29, 32, 33 and 36; the descriptor, the available ring, the used ring and their flags and alignments; the virtio-blk header, request types and status values; the device types.
- **Checked against the QEMU sources** (`/home/david/claude/qemu-src`): the vendor ID 0x1AF4 and the device-ID base 0x1040; the ECAM address arithmetic; the doorbell address decoding; the MSI-X entry mask handling and the behaviour of a masked vector; the BAR sizing mask; the PCI Express tree vocabulary in `docs/pcie.txt`.
- **Checked against 2.11BSD** (`/home/david/claude/facetlang/unixtree/2.11BSD/sys`): the RK11 register structure, its bit values and the code of `rkstart` and `rkintr`; the Unibus map (`mapalloc`); the console register and the status word as fixed addresses (0177560 and 0177776).
- **Unverified** (general knowledge, the specifications not being on this machine): the 18-bit width and the 8 KB I/O page (the 18 bits are in the repository's text, the I/O page's extent is not) and the place of the vectors at the bottom of memory; the Unibus's BR levels 4 to 7, NPR, `MSYN`/`SSYN` and the grant daisy chain; the RK11's register addresses 0177400 and up; the kinds of TLP, posted writes and completions; that lane counts are powers of two; that the width and speed of a link are negotiated; the value the specification asks a driver to write to a doorbell; that the specification asks for a memory barrier between publishing the available index and reading the used flags; that an empty configuration position reads all ones on real hardware; the size of the PCI Express maximum payload and the packet header formats (not used here).
- **Decided by the author and recorded in `errata.md`**: everything stated as a decision with a tag *(D)*; nothing else in this chapter is a decision.

2.13 Still open

The bus chapter's open items, each with options in the decisions document: the interrupt controller and routing, and so how the address and data of an MSI-X entry are made (R1, 2.7.3); starting the other cores and inter-processor interrupts (R2, B2); coherence beyond four cores (R3); the detail of the IOMMU (its five recommendations are accepted, errata 85; the format of its table, where it sits and how it reports a fault are open); errors raised by devices (R7); virtualization (R8); where the configuration space and the windows sit in the physical address space and how wide that address is (how software finds the width: no register is defined); how many pages "the first few" are; the exact feature subset of each virtio device, and whether the pdp64 uses the `EVENT_IDX` feature (2.6.9) and declares `ACCESS_PLATFORM` and `ORDER_PLATFORM` (2.6.3); and where the root complex's status register that records such a miss sits (2.4.10).

Chapter 3 — Graphics: the terminal and its devices

> ***Iteration two.*** *The devices of this chapter (`VIC-64x` with its 48-bit RGBA pixel, `SID-64x` with 48-bit samples, `BLT64`, and the eight-register face) are planned for the future, together with the author's 12-register network, sound and graphics cards (errata 75, 2026-11-03). The first iteration uses the most standard, secure and easy-to-support network, graphics and audio interfaces over PCI Express, NVMe and USB (since errata 79 and 95: virtio-gpu and virtio-snd); this chapter describes what follows them. Statements keep their tags; the decisions of errata 73 and 74 stand. Errata 105 records how this chapter was rewritten and what is still open in it.*

The pdp64's display is meant to bring back what the Blit terminal and the Commodore 64 had, text and graphics on the screen at once, set by simple stores to memory, at a modern colour depth and screen size; its output is HDMI or another standard graphics output; its form is not yet fixed, and the standard device of the first iteration is virtio-gpu. This chapter is written for a reader who knows the PDP-11 and its teletype, or who is learning computing from the start, and who has not met anything that was invented after the 1970s. It tells the story of how a computer's display grew, in the order it happened (the teletype, the screen with escape codes, the bitmap, the Commodore 64, the PC's graphics adapters, the graphics processor), because each stage is the answer to a complaint about the one before and the pdp64's terminal is a choice among those answers. Then it says what the author has asked for, what the repository specifies, what is decided and what is open, and ends with a section of sketches for a design that is still being worked out. Tags: *(D)* decided; *(P)* proposed; *(S)* stub (not specified). A fact about a historical machine or about a graphics processor that could not be checked on this machine is marked *unverified* and listed in section 3.22.

How to read this chapter. A term is defined where it first appears, as the technical word followed by a plain restatement. Boxes-and-arrows pictures show memory as a textbook shows a linked list: a box is a piece of memory, an arrow is an address or an index stored in it. Code blocks are pseudocode (C-like; `poke(address, value)` and `peek(address)` stand for an ordinary store and load, as in chapter 2) unless they are PDP-11 assembly, which is marked. The numbered statements are the rules; the history and the explanation are prose without numbers, as in chapter 2.

3.1 The teletype: one line at a time

A PDP-11 in 1975 talked to a person through a **teletype** (a keyboard and a printer on one machine, joined to the computer by a serial line: characters go one after another along a single wire pair, each as a short burst of bits). The machine knew nothing about the page. It could send a character, and the printer put it at the right of the last one; a carriage-return character moved the print head back to the left margin, a line-feed character moved the paper up one line, and that was the whole of its picture of the world. Nothing could be erased, because ink was on paper. This is **line-at-a-time output**: the program's only notion of position is "the end of what I have printed".

At the hardware level it was the same shape as every Unibus device of chapter 2: a few registers. The console of 2.11BSD is a KL11/DL11, declared in `pdp/cons.h` as four words, receive status, receive buffer, transmit status and transmit buffer, at address 0177560 (`cnaddr` in `cons.c`), with the transmitter-ready bit in the transmit status word being `DLXCSR_TRDY`, 0200 (checked in the 2.11BSD tree on this machine). Printing a line was a loop that waits for ready and stores a byte:

```
print_line(string):
    for each byte c in string:
        while (peek(XCSR) & TRDY) == 0:     # the printer is slow: ten characters a second or so
(unverified)
            wait
        poke(XBUF, c)
    poke(XBUF, CR); poke(XBUF, LF)          # carriage return, line feed
```

and in PDP-11 assembly, with the register addresses of the console (the fourth word is the transmit buffer; the loop is in the style of the `rk` driver of chapter 2, not copied from a driver):

```
        MOV  #msg, R0
1$:     MOVB (R0)+, R1        ; next character; R0 now points at the one after
        BEQ  3$               ; a zero byte ends the string
2$:     TSTB @#177564         ; XCSR: bit 7 is READY
        BPL  2$               ; plus: not ready, look again
        MOVB R1, @#177566     ; XBUF: this sends it
        BR   1$
3$:
```

Everything a program can say to such a device is a stream of characters. That is why the Unix idea of a **tty** (the name is short for teletype: the device file that a program reads and writes characters from and to) is a stream, and why programs written for it, the shell, the editor `ed`, the compiler's messages, think in lines. The cost shows as soon as the printer is replaced by a screen: the stream model has no way to say "put this here".

3.2 The glass teletype and its escape codes

A **video terminal** is a keyboard and a screen on the end of the same serial line. At first it simply imitated the teletype (the "glass teletype": characters appear at the bottom and the page scrolls up) and then it began to do what paper could not: move the cursor to any place and change what was there. The program told it how with **escape sequences**: characters the terminal does not display but obeys, introduced by the escape character (code 27, written ESC), so that the stream carries both text and commands.

Two generations of them matter here. The DEC VT52 was one of the early ones, and the repository's own records preserve one of its codes, cursor-to-position, because it is the textbook example of the problem (c64stuff.txt, the author's chat of 2026-06-18): ESC Y, then a row character and a column character, each with 32 added so that the numbers arrive as printable characters. In pseudocode:

```
goto(row, col):
    send(ESC); send('Y'); send(row + 32); send(col + 32)   # the VT52 form; details unverified
send_string("Hello")                                       # the text appears at the cursor
```

A later and enormously influential generation, the DEC VT100 and the ANSI standard that followed it, uses ESC followed by an opening square bracket, then numbers and a letter: ESC, bracket, 12, a semicolon, 40, `H` moves the cursor to row 12, column 40, and ESC, bracket, 31, `m` makes what follows red (unverified in detail; the repository's records name "VT100/ANSI" and the "xterm-256color" subset as what ordinary programs speak, c64stuff.txt line 218 and 4888). The author's message names a "vt55" between the teletype and the Blit. **The repository's texts name the VT52 and the VT100 and nowhere a VT55, so what "VT55" means is the author's naming** (it may be the VT52 or a model of its family; the DEC model of that number was a graphics terminal, which would fit the author's order, but that is general knowledge and *unverified*); this chapter describes the VT52 and the ANSI codes by what the records say and marks the rest unverified, and it says what a terminal of that kind could not do without relying on the VT55.

What the escape-code terminal gave, and what it did not. It gave a **character-cell display** (a screen that is a grid, 80 columns by 24 rows is the famous size, of cells that each hold one character) that a program could address by position. It did not give a program any control over the shape of a character, a drawing, a mix of fonts or a picture, and it put a slow serial line, with a language of escape codes whose details differed from one make to the next, between the program and its screen. Unix answered the variety with a database of terminal descriptions (**termcap**, then **terminfo**: a file that says, for each make of terminal, which bytes mean "move the cursor") and a library (**curses**: a program that works out the fewest bytes to send to turn the screen as it was into the screen as wanted). The authors of *The UNIX-HATERS Handbook* complain about exactly that ("Original Sin", read for the repository in `docs/kernel-pain-points.md`, section 14I: "Unfortunately for us, Unix was designed in the days of teletypes"). Each character cell needed several bytes of command to change and the bandwidth was low, which is the complaint the next stage answers.

3.3 The Blit: a bitmap, and the blit

In the early 1980s the Bell Labs terminal called the **Blit** (Rob Pike, Locanthi and Reiser; the repository records their papers as *Blit 1983* and *Tradeoffs 1985* in `docs/kernel-pain-points.md`, section 14I) took the opposite course. Instead of a terminal that

obeys commands about a grid of characters, it had a **bitmap display** (a screen in which every dot, every pixel, is one or more bits in memory, so a program draws anything by changing bits) and a Motorola 68000 processor, 8 MHz, to do the drawing. Its screen memory was in the processor's address space like any other memory. A program that ran in the terminal changed the picture by storing into that memory.

The memory a bitmap needs is a picture stored row after row (the figure shows it in 3.5, where the PC meets it again), and the operation that makes a bitmap usable is the **bit-block-transfer**, written **bitblt** and said "bit blit": copy a rectangle of bits from one place to another, combining each source bit with the destination bit by a chosen rule, the **raster operation** (RasterOp: the rule that says, bit by bit, what the result is, from "replace" to "OR" to "exclusive-OR"; a rule of two inputs has sixteen possible forms, and the Blit authors found that four sufficed: store, OR, clear, XOR, with XOR the most used because drawing a shape twice with it erases it again, which is how a cursor moves without a saved copy of what was beneath it). With bitblt a program scrolls a window (copy the rectangle up by one line of text), moves a window (copy it), draws a letter (copy the letter's rectangle from a stored font into the screen) and fills an area. In pseudocode of the Blit's own library style:

```
bitblt(source_bitmap, source_rect, destination_bitmap, destination_point, operation)
bitblt(font, rect_of('A'), screen, (x, y), OR)       # draws an A with its top left at (x, y)
bitblt(screen, window_rect, screen, window_rect + (0, -line_height), STORE)   # scroll one line
```

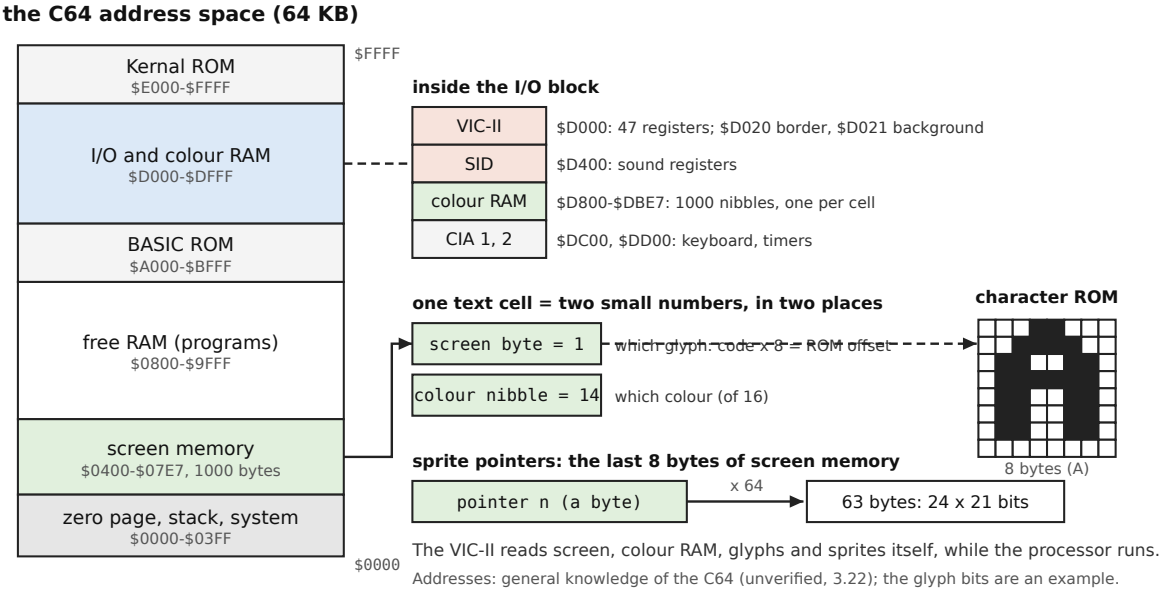
The Blit's central point was that the terminal was a small computer, not a protocol. The Blit authors recorded the reason in their own words: "To draw graphics in the windows, sending escape sequences is traditional but makes poor use of the processing power of the terminal"; a predecessor that interpreted escape sequences had failed because "too much state information was maintained by the terminal and guessed at by the Unix program". The Blit received downloaded programs over its serial line (RS-232: a cheap choice, and the authors say that "the greatest problem, though, is certainly the low bandwidth"). The repository's chapters give both halves of that lesson, which the pdp64 keeps: the framebuffer is in the address space, and the blit is an assist that can be bypassed (3.10.6).

The Blit was the inspiration of the window systems that followed it: on the author's account, of rio (Plan 9) and of NeWS (Sun's), and the sources confirm the line from the Blit's `mux` through 8½ to rio; what they were and what the pdp64 could carry over from their simplicity is in 3.19.

One more Blit idea belongs here because the pdp64 inherits the question it raises: the Blit drew its cursor itself, in software, in XOR, at each clock interrupt; other machines had **sprites** (small pictures that the display hardware overlays on the screen, at a position the program gives, without touching the screen memory beneath). The next machine had eight of them.

3.4 The Commodore 64: text, graphics, sprites and sound together

The Commodore 64 (1982) was a home computer built around a 6510 processor (a 6502 with a few input and output lines added; unverified detail) and two custom chips: the **VIC-II** (the video interface chip, the video chip) and the **SID** (sound interface device). It was built to play games, and a game needs a lot at once: a background that scrolls, characters that move over it, bright colours, and music and noise. The design answer was to make the chips read their data by themselves from memory and to give the programmer every control as a memory location to **peek** (read a byte) or **poke** (write a byte): the names are the C64 BASIC commands for a load and a store, which is where the author's phrase "peek poke style" comes from.



Text mode. The standard screen is 40 columns by 25 rows, 1000 cells. The picture is *two* arrays in memory: screen memory, 1000 bytes, normally at $0400 (the number of the glyph for each cell), and **colour RAM**, 1000 four-bit entries at $D800 (the colour of each cell, one of 16). The shape of each glyph is not in these arrays; it is in the **character ROM** (a read-only memory of 8 bytes a glyph, one byte a row of eight dots, 256 glyphs: 2 KB for one set), which the video chip reads, and which a program can also replace by pointing the chip at glyphs it has made in RAM. To put an A at the top left, in red, the BASIC programmer writes two pokes, and the video chip, which reads screen memory sixty times a second by itself, shows it (the screen codes, the colour numbers and the addresses are general knowledge of the C64, *unverified* here):

```
poke(1024, 1)          # $0400: screen code 1 is the letter A
poke(55296, 2)         # $D800: colour 2 is red
poke(53280, 0)         # $D020: the border colour, black
poke(53281, 6)         # $D021: the background colour, blue
```

This is exactly the "peek poke" model: the screen is memory, a cell is an address, and the chip does the display. No escape codes, no stream: the program that pokes position 1024 + 40 * row + column has put a character there, at any time, in any order.

Bitmap mode. The same chip could instead show a bitmap, 320 by 200 dots, in memory, with colour attributes per 8 by 8 block (unverified detail), and **multicolour** modes that trade width for colour. The point the author makes about the C64 is that these were *modes of one chip*, and that the chip could mix them: text in one band of the screen and graphics in another by changing mode at a chosen line.

Sprites. The VIC-II had eight **sprites**: movable objects of 24 by 21 dots, each described by 63 bytes of data in memory, a pointer byte (the last eight bytes of screen memory, $07F8 to $07FF, say which 64-byte block holds each sprite's shape) and a few registers (x position, y position, colour, enable, size, priority; collision registers that say which sprites touched). The chip draws the sprites over the background as the picture is read out; to move a sprite, the program changes two registers, and the picture underneath is never disturbed. Sprites were why games were easy: a ship, a bullet, an enemy are each a sprite and a game's movement is a handful of pokes per frame.

The raster and the raster interrupt. The VIC-II draws the screen by reading it in order, row by row, while an electron beam sweeps the tube (the **raster**; the whole sweep is a **frame**, 50 or 60 times a second depending on the television standard). A register holds the number of the line being drawn, and another can be set to a line number so that the chip interrupts the processor when the beam reaches it: the **raster interrupt**. Programs used it to change a register mid-screen, for example the background colour or a sprite position, and so to get more than the chip nominally offered, and to make changes while the beam was not looking (the subject of 3.8).

Sound. The SID is a three-voice synthesiser with filters, again with registers that the programmer pokes: a voice has a frequency, a waveform (triangle, sawtooth, pulse, noise), and an attack, decay, sustain and release envelope. A note is a handful of pokes (unverified in detail). A game's sound was a part of the same few chips, controlled in the same way, and that is the C64's real lesson: **one machine, one address space, a few chips whose registers are memory, and a program that sets them directly**.

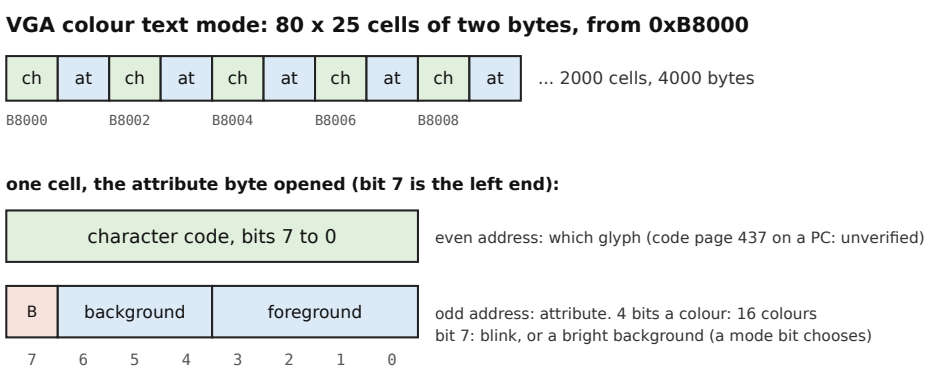
The C64's limits are the limits of 1982: 16 colours, 40 columns, 8 by 8 cells, 8 sprites, three voices. The author's request is for the same ease with none of those limits (3.10).

3.5 The PC years: VGA, SVGA, VESA

The IBM PC took a different road. Its first video adapters were separate cards, with their own memory and their own registers, and the software model was a mixture of memory and input/output ports (the I/O instructions that the PDP-11 and the C64 did not have). The **VGA** (Video Graphics Array, IBM, 1987) became the baseline that every later adapter imitated.

VGA text mode. The text mode is the C64's idea, with the attribute folded into the cell: the screen is 80 columns by 25 rows, 2000 cells, in memory at 0xB8000, two bytes a cell: the character code and an **attribute byte** (colours: four bits of foreground, three of background, one bit for blink, or for a bright background; the exact layout is *unverified* general knowledge). The shapes come from a font in the adapter's own memory, which a program can replace. To put a red A at the top left:

```
poke(0xB8000, 'A')          # the character
poke(0xB8001, 0x04)         # attribute: black background, red foreground
```

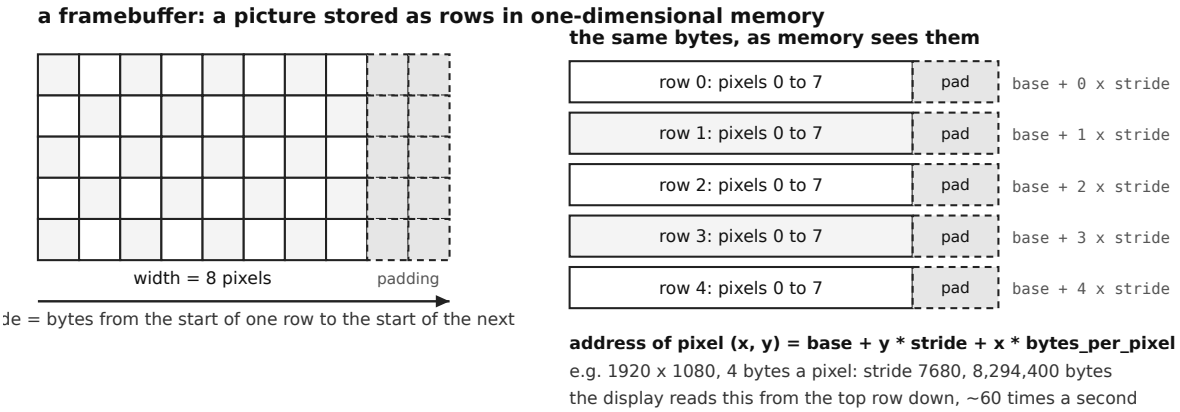


The glyph shapes are in the display adapter, not in this memory; the cell says which, and in what colours.
Layout as in general knowledge of the IBM PC adapters (unverified here, 3.22).

VGA graphics. The same card had graphics modes, and here the trouble began. Mode 13h (320 by 200, 256 colours, one byte a pixel, at 0xA0000) was the easy one: the framebuffer is a single array and to plot a pixel you write one byte at y * 320 + x. A byte names one of 256 palette entries, and the palette (a table of 256 colours, which the program can load through I/O ports) is in the card (unverified detail). Modes with 16 colours and higher resolutions were **planar**: the four bits of a pixel's colour were stored in four separate **planes** (four arrays, the same pixel position in each), so that to set one pixel's colour the program had to select planes through registers, and the plane that a write reached depended on register settings. It was fast for some operations and a puzzle for the rest. The adapter's window onto its memory was 64 KB at 0xA0000, so any picture larger than 64 KB needed **bank switching** (the card has more memory than the window shows; a register chooses which 64 KB slice appears in the window, and the program must change the register whenever its drawing crosses a slice boundary).

SVGA and VESA. Every manufacturer's Super VGA added modes (800 by 600, 1024 by 768, more colours) and, since there was no standard, each used its own registers to switch to them and to select a bank. **VESA** (a standards body of display makers) wrote the **VBE** (VESA BIOS Extensions: calls into the adapter's ROM that list the modes, set one, and switch banks), and a later version let the card present its whole memory as one **linear framebuffer** at a high address, ending the banks (unverified in detail). The picture is now what the chapter has been approaching: a block of memory, a **width** and **height** in pixels, a number of bits per pixel, and a **stride** (also called pitch: the number of bytes from the start of one row to

the start of the next, which can be more than the width times the bytes per pixel, because rows are padded to a convenient boundary).



Padding makes every row start on an aligned address; software must use the stride, not width x bytes.

What the programmer had to know. The price of all this was paid by the programmer, and it was large. A graphics programmer of the late 1980s and the 1990s carried in the head: the VGA's registers, set through ports, that choose which planes a write reaches, how the bits of a write combine with what is already there, and how the card's address counters step; the planar memory layout, in which one pixel's colour is spread over four arrays; **Mode X** (a name from the PC games community for an unofficial VGA mode, 320 by 240 with square pixels, that used the planar memory to give more than one screen of pixels and faster fills; unverified detail); the bank register of each Super VGA chip, which differed from chip to chip, and the per-chip quirks that the VESA calls papered over only in part; the palette, whose entries could be rewritten to make colours cycle or fade without touching the picture; the **retrace**, since a change made at the wrong moment tore, and the program timed its drawing against the beam; and the **inner loops**, written by hand in assembly, for the fill, the copy and the line, because the processor was only just fast enough and a few cycles a pixel decided whether a game ran. The author's referent is the books of Michael Abrash on PC graphics programming, which he recalls as "800 page michael abrash books" (the author's words and figure; the titles, the dates and the page counts are *unverified*: no copy is on this machine and none of the repository's records names a title). The point is not that these programmers did it wrongly: they did it with great skill, and the depth of the books is the measure of what the hardware pushed onto them. The pdp64's aim (3.9) is the opposite: that the program needs none of it.

The uncomfortable split. By then the PC had two worlds. In **text mode** the hardware drew the characters, so text was fast, small (4000 bytes) and easy, and a program could not have anything else on the screen. In **graphics mode** a program could have anything, but it drew every dot: even a letter, then, was a program's job (copy its rectangle from a font, which is the Blit's bitblt, now in software), and the text a user had was suddenly slow and unlovely. To change between the two, a program switched modes, the screen blanked, and the other world had none of the conveniences of the first. The C64 had had a mixture (text and bitmap and sprites in one frame); the PC made the programmer choose. Windowing systems, which want text and graphics together, chose graphics and drew everything.

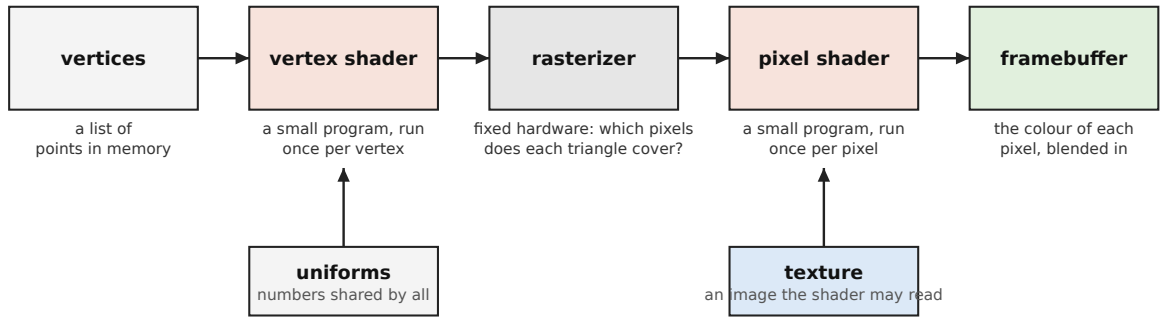
Why sprites vanished. The PC games of the VGA era had no sprites (the VGA had none) and drew every moving object into the framebuffer in software, often with the processor doing the work while a second copy of the screen was being prepared (3.8). Three things made this acceptable: the processor became fast enough, by the early 1990s, to move the dots of a screenful in a frame time; the framebuffer was ordinary memory, so any program could write it; and windowing systems composed overlapping windows in software, which made a hardware plane or sprite a special case that the window system would not use. The loss was not free: moving a mouse cursor, a game's ship or a window now cost writing and restoring pixels in the framebuffer, and (3.8) doing it while the display was reading the same pixels.

3.6 The GPU: pipelines, vertices, shaders

In the 1990s a new kind of chip took the drawing from the processor: the **GPU** (graphics processing unit), first for three-dimensional games, now for every screen. To see why it looks as it does, start from what a three-dimensional scene is: a list of

vertices (points in space with a position, and perhaps a colour, a direction the surface faces and a place in a picture), joined into **triangles** (three vertices: the one shape that is always flat, so any surface can be built from them), seen by a camera. To draw it, the machine must (a) work out where on the screen each vertex lands, (b) find which pixels each triangle covers (**rasterization**: turning a shape into the set of pixels it covers), and (c) decide each pixel's colour, which is usually where a **texture** (a picture stored in memory, which is looked up at a position given by the vertex, to paint a surface with, say, brick) comes in. The three steps are the **graphics pipeline**.

the graphics pipeline: data goes in at the left, pixels come out at the right



Every vertex goes through the same program at the same time on different lanes; so does every pixel.
The programmer writes the two shaders (pink); the rasterizer between them is fixed.
This is the model of general knowledge of current GPUs (unverified here, 3.22); stages such as geometry and tessellation are left out.

The early GPUs did each step in fixed hardware. The change that made the modern GPU is that two of the steps became **programmable**: the programmer supplies a **shader** (a small program, written in a special language, that the GPU runs; the name is historical, because shaders began as programs that computed the shading of a surface). A **vertex shader** runs once for every vertex and says where it goes; a **pixel shader** (also called a fragment shader) runs once for every pixel a triangle covers and says what colour it is. The rasterizer between them stays fixed, because it is the same arithmetic for every program.

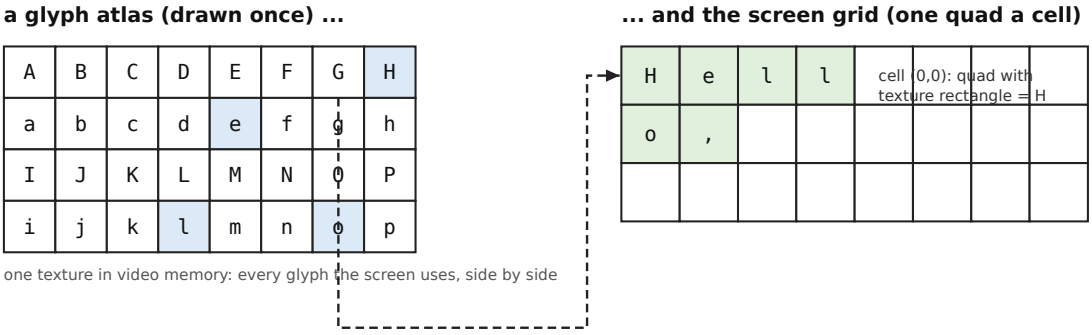
Why a GPU is fast. A processor core is built to run one thread of arbitrary code as quickly as possible: it has a deep pipeline, large caches and a lot of logic to guess and reorder. A GPU is built for the case where *the same small program is run on a very large number of independent items*: a million pixels each need the same few dozen instructions. So it has thousands of simple **lanes** (units that each do arithmetic on one item), grouped so that a group of 32 or 64 lanes (the group size varies by maker; unverified) executes one instruction at the same time on different data (**SIMD**: single instruction, multiple data), and it has memory with an enormous **bandwidth** (bytes per second) to feed them. When one group waits for memory the hardware switches to another, so the lanes are always busy. Three more things make it fast in practice. The CPU does not touch pixels: it describes the work (a **command buffer**: a list of commands in memory, such as "bind this texture, use these shaders, draw these 3,000 vertices as triangles", which the GPU reads and carries out by itself, in the manner of the descriptor rings of chapter 2) and goes on. Work is **batched**: one command can draw thousands of things, so the cost of the command is paid once. And the GPU's memory is organised for it: textures are stored in a layout that keeps neighbouring pixels near each other, and reading a texture is hardware that does the filtering.

The cost is that the GPU is poor at anything with decisions, or with items that depend on each other, and that the programmer must learn a different way of working: describe the whole job as a pipeline, write the shaders, and feed it in large batches. Sections 3.8 and 3.9 show how much of the boilerplate of graphics programming comes from that model.

3.7 Why a terminal emulator uses the GPU, and why a font needs three dimensions

A **terminal emulator** (a program that shows a grid of text in a window of a graphical desktop, and acts like the VT-series terminal of 3.2) has a simple job: the screen is a grid of cells, each a glyph with a foreground and a background colour. Several well-known ones draw their grid on the GPU (Alacritty, kitty and WezTerm are the standard examples, and the repository's chat names kitty; the claim that they use it is general knowledge, *unverified* here). Why would a program that shows only text want the machinery built for three-dimensional games?

The way a GPU draws text. The text is the same glyphs, over and over. So the program draws each glyph once, with a font rasteriser, into a **glyph atlas** (one texture that holds every glyph the screen uses, side by side), then describes the screen as a list with one entry per cell: where it is, which part of the atlas to show, the two colours. The GPU draws each cell as a **quad** (a rectangle of two triangles) with the glyph's part of the atlas as its texture, and the pixel shader mixes foreground and background by the glyph's value at each pixel. The whole screen, however many cells, is **one draw call** (one command that draws a whole list).



To draw the screen: send the GPU a list with one entry per cell, and run one draw call.

entry = (cell column, cell row, atlas rectangle of the glyph, foreground, background)
vertex shader: turn the cell position into screen corners; pixel shader: look the glyph up, mix fg and bg
a change of one cell, or a scroll by a row, is a change to the list or to a number, not to 2 million pixels

The atlas is made by a font rasteriser (CPU) and uploaded once; new glyphs are added as they first appear.

What this buys. The processor never touches a pixel: it changes a cell in the list when the text changes, and scrolling by a line is a change of one number (where the list starts) or a shift of the list, not a copy of two million pixels. Colour and attribute changes cost nothing more. The glyph is stored as coverage values, so the shader can place it at a fraction of a pixel and blend it, which is **antialiasing** (smoothing an edge by giving the pixels it crosses an in-between colour) and **subpixel positioning** (placing a glyph at a position finer than the pixel grid). And the GPU does all of this with the bandwidth of its own memory, leaving the processor free to run the program whose output is being shown.

Why a font is a three-dimensional problem. The question deserves an honest answer in two parts. First, a glyph is not a bitmap: in a TrueType font it is an **outline** (curves, in a coordinate system independent of size), and to show it the outline must be turned into pixels, which is **rasterization** again: the same job the GPU's rasterizer does for triangles. A font rasteriser on the processor produces the atlas once. There are also ways of using the GPU for the curves themselves: tessellating the outline into triangles, or evaluating in the pixel shader whether each pixel is inside the curve, or storing a **signed distance field** (a texture in which each pixel holds its distance to the nearest edge of the glyph, positive outside and negative inside, so that a pixel shader can draw the glyph sharply at any size from a small texture; this technique is credited to Valve's 2007 paper, *unverified*). Each of these is the 3D pipeline used on a flat shape.

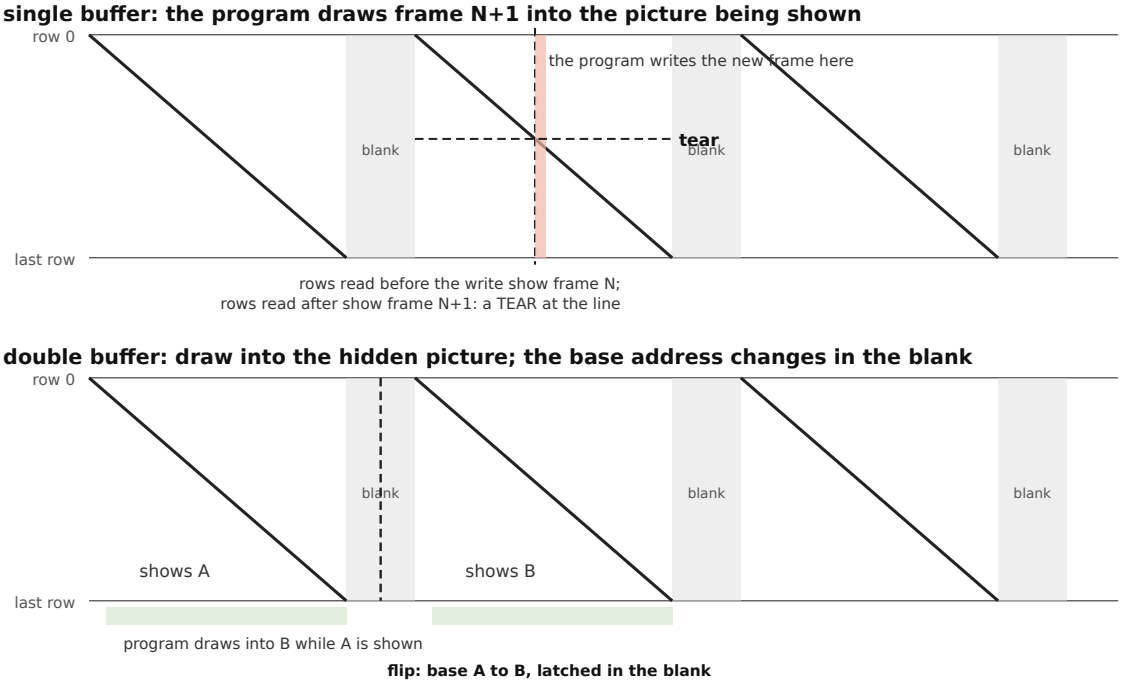
Second, the pipeline is three-dimensional only by convention. A modern GPU has **no 2D mode**: the 2D accelerator of the 1990s, with its blits, line draws and rectangle fills, has been dropped from the hardware, and the drivers implement a rectangle as two triangles with a shader. Drawing a flat picture is the special case of the 3D machinery in which every z value is the same and the camera looks straight on. This is a choice of the industry and not a necessity: a flat-rectangle engine would be smaller and would do the terminal's job; the reason it is not built is that the GPU has to be built for the harder case anyway, and the 3D case pays for the chip. That is why the *terminal* ends up on the *3D* chip, and it is the reason the pdp64, which will not necessarily have a 3D chip, must decide what to put in its place (3.21).

3.8 Scanout, tearing and double buffering

Scanout. A display does not hold a picture; it is **refreshed**. The adapter reads the framebuffer from the first row to the last, in order, at a fixed rate, and sends each pixel to the monitor as it is read: this is **scanout** (reading the picture out of memory to the screen). The rate is the **refresh rate**, 60 times a second on most monitors (an example; the C64's was 50 or 60 by television standard). Between one frame and the next, and between rows, there are short **blanking intervals** (times

when the monitor is not being sent pixels: the beam on a tube had to fly back, and the name **vertical blank** stuck to the gap between the last row of a frame and the first of the next; VGA called the event the vertical **retrace**). The C64's raster register and raster interrupt of 3.4 are the program's view of the same thing: the number of the row being read now, and a way to be told when it reaches a chosen row.

Tearing. Suppose the program changes the picture while the scanout is reading it. The part of the screen above the row being read shows the old picture and the part below shows the new one, and the seam is a **tear**: a horizontal line where an object that moved appears cut in two. A related fault is **flicker**: a program that clears the screen and then draws the new picture lets the scanout catch the screen half-drawn, or empty. The figure shows time to the right and the row being read, rising from the first row to the last once a frame.



Time runs to the right; a frame is about 16.7 ms at 60 Hz (an example rate). The slanted line is the display reading one row after another.

Double buffering. The standard fix is two pictures. One, the **front buffer**, is being shown; the program draws the next frame in the other, the **back buffer**, and when it is complete the two exchange roles at the moment when the scanout is in the vertical blank, when no row is being read. This is **double buffering**, and the exchange is a **swap** or a **page flip**. The swap can be done by *changing the base address* the scanout reads from (a register write; nothing is copied) or by *copying* the back buffer into the front buffer (a **blit**, cheap on a machine with a blitter and slow on one without, and the copy itself can tear if it is not finished within the blank). The two ways in pseudocode, as a program would write them, in peek/poke style, with the single-buffer way first:

```
# 1. single buffer: tears and flickers
loop:
    clear(FRAME)                        # the scanout can see an empty screen here
    draw_scene(FRAME)                   # and half a scene here
```

```
# 2. double buffer, swap by changing the base address (a page flip), done by polling
front = BUF_A; back = BUF_B
poke(SCANOUT_BASE, front)
loop:
    draw_scene(back)                    # nobody is reading `back`
    while peek(RASTER_LINE) != 0:       # wait for the start of the blank: busy-waiting
        wait                            # (or wait for a vertical-blank interrupt)
    poke(SCANOUT_BASE, back)            # the flip: takes effect now
    swap(front, back)                   # draw next into the picture just retired
```

The cost of double buffering is real. It **needs twice the memory** for the picture: at 1920 by 1080 and 4 bytes a pixel that is 8.3 MB twice (3.15.2). It **needs a deadline**: if drawing the next frame takes longer than a frame time the flip misses the blank, and the program either waits for the next one (the frame rate halves from 60 to 30 and then to 20: the "half frame rate" cost recorded in `kernel-pain-points.md`, 0.6 #3) or flips at once and tears. **Triple buffering** (three pictures, so that the program can start the next frame while the finished one waits for the blank) removes the wait and adds a frame of memory and of **latency** (the delay between the program's input and what the screen shows). Polling the blank occupies the processor; the interrupt version needs a driver. And, where the swap is a base-address register, the register must change *in the blank*, which a program that is itself being interrupted cannot always manage: the Linux display driver for one common graphics chip disables interrupts and busy-waits for the row to pass before it updates registers (the "vblank evasion" of `kernel-pain-points.md` 0.6 #13, from the driver source `intel_pipe_update_start`).

Is this still a problem? The author asked (2026-11-10): "with modern hdmi is tearing even a concern?" The honest answer is that tearing is not a property of HDMI at all. HDMI (like DVI and DisplayPort, 3.14) carries a stream of pixels at a steady rate, one row after another, and has no idea whether the picture it is carrying was complete when it was read. A tear happens one step earlier, in the display controller that reads the framebuffer out while a program is changing it; the cable only repeats what it was given. On a phone or a desktop it is largely hidden, because between the program and the controller there is a **compositor** (a program of the system that collects the pictures of all the windows and builds the one that is shown) and a driver, and together they draw the next frame in a spare buffer and flip it at the vertical blank: tear-free by construction. That machinery *is* the boilerplate of 3.9, which the author wants the architecture to sugar away. A program on the pdp64 that writes straight into the framebuffer, with no operating system in between (as a boot program, a game written to the metal or a C64-style program would), can still tear, so a tear-free flip has to come from the architecture or the device, not from the cable. Variable refresh (HDMI VRR, adaptive sync) turns a late frame into a short delay in the display instead of a tear or a missed blank, which reduces the need for a blank-time flip but does not remove it: the frame still must not change *while* it is read. Other displays add their own behaviour: a television may hold a whole frame before showing it (added **latency**), and a laptop panel may refresh itself from its own memory when nothing changes (panel self-refresh). Of this, what is *checked* is only what the repository and the headers say: that HDMI is a pixel-stream transmitter (3.14.1) and that the DRM interface on this machine has a page-flip call with an event on completion and an asynchronous flag that allows a flip with tearing (`DRM_MODE_PAGE_FLIP_EVENT` and `DRM_MODE_PAGE_FLIP_ASYNC` in `drm_mode.h`). What is *general knowledge, unverified*: the HDMI VRR and adaptive-sync standards and what a sink does with a late frame, the latency of a television's processing, panel self-refresh, and what phone and desktop compositors do. Whether a tear-free flip is worth hardware if the display side is variable-refresh is a question for the author (3.23).

What the repository already decides. The video device of 3.10 (`VIC-64X`) already has the first piece of sugar: every register that takes effect at a frame boundary is shadowed, one write to its `ARM` register latches them all together at the start of the next vertical blank, a status bit says whether the arm was taken in this frame or missed it, a sticky underrun bit reports a scan-out that ran dry, and the vertical-blank interrupt occurs at a fixed line (statement 3.10.9, errata 71, from the pain-point list of `kernel-pain-points.md`). A base address in that register set therefore flips at the blank without any program arranging it. The chat of the author's device design had said the same thing as a rule in 2026-08 ("multi-register updates are atomic at a named raster line or vblank", `docs/c64-devices-from-chat.md` section 2) and, for the eight-way machine, "flip on vsync unconditionally with an overrun interrupt instead of a barrier". What no text yet decides is anything beyond one shadowed register set: independent layers, damage, variable rate, fences, and a default that a program does not have to set up.

3.8.1 The architecture is to offer conveniences that avoid tearing, flicker and the other faults that forced programmers to double-buffer, and to do it so that the common program needs no more than a store; the form of those conveniences is open. *(P)* (The author, 2026-11-10: "we want to put in nice architectural sugar to avoid 'tearing' and all those other issues graphics programmers had to use double buffering to fix", errata 105.)

3.8.2 The mechanism that is decided is the shadowed registers and the one `ARM` write of 3.10.9; everything else in this section is a candidate. *(D)*

The candidates, each with what it costs, and none decided:

- **A scanout base register that latches at the vertical blank** (a page flip: the register holds the address of the picture to show and takes the new value only in the blank), with a completion **interrupt** or a **fence** (a counter that the device advances when the flip has happened, which the program can read or wait on, so it knows that the old picture is free). *Gives:* no tearing, no polling of the raster; the program writes one register and goes on. *Costs:* two pictures of memory

for the program that uses it, and a rule for what the register shows if it is written twice in one frame (the first is dropped, or the second waits). *Already in 3.10.9, except the fence.*

- **Per-layer latching**, in a design with layers (3.21, sketches 1 and 2): each layer has its own base register and flips independently, so a cursor or a sprite layer can move every frame while the text layer flips only when its text changes. *Gives:* the cheapest case for text, a moving cursor with no redraw. *Costs:* one set of registers per layer.
- **An atomic commit of several registers**: several registers are staged and applied together at the blank (this is what `ARM` of 3.10.9 is for the single set; Linux calls the same idea "atomic modesetting" and offers `DRM_MODE_ATOMIC_TEST_ONLY` to ask "would this work?" and `DRM_MODE_ATOMIC_NONBLOCK` to ask for it without waiting, in `drm.h` and `drm_mode.h` on this machine). *Gives:* a change of mode, picture and cursor that cannot be seen half-done. *Costs:* the shadow copies of the registers, which a small device pays in flip-flops; an optional test-before-commit is more.
- **A raster-position register and a raster interrupt** (the C64 way): the program can know the row being read and be interrupted at a chosen row. *Gives:* mid-frame changes (the C64's trick of a different colour or sprite position per band) and a way to time drawing behind the beam. *Costs:* one register and an interrupt source; encourages programs that are tied to the display's timing.
- **Variable refresh** (the monitor waits for the frame instead of reading at a fixed rate: so a frame that is late is not a missed blank but a short wait; the standards are called adaptive sync, FreeSync, G-Sync and, for HDMI, VRR, variable refresh rate; unverified general knowledge). *Gives:* a late frame costs only its own delay, not a halving of the rate. *Costs:* the output must support it (the HDMI transmitter chip of 3.14.1 and the monitor), and the device's mode registers must say so.
- **Damage rectangles** (the program says which part of the picture changed and only that part is sent on): this is what virtio-gpu's `TRANSFER_TO_HOST_2D` and `RESOURCE_FLUSH` take as a `virtio_gpu_rect` (3.16). *Gives:* less memory traffic when little changes (a terminal). *Costs:* the program must track what it changed.
- **A hardware cursor and sprites**: objects that the scanout overlays at a position without touching the picture under them, so moving one never needs a redraw and never tears. *Gives:* the cursor, and in a game its moving objects, for the price of two registers each. *Costs:* gates in the scanout path and a limit on how many. (virtio-gpu has a cursor in its protocol: a separate cursor queue, `UPDATE_CURSOR` and `MOVE_CURSOR`, `virtio_gpu.h`.)
- **Versioned or copy-on-write buffers**: the device keeps the picture the scanout is reading and the program writes to a fresh version that replaces it atomically when the program says it is done, so the program can never see or show a half-drawn frame and never has to name two buffers. *Gives:* the sugar nearest to "nothing to do": a program draws and says `commit`. *Costs:* the memory and copying are the device's; a page-granular copy-on-write needs the memory-management machinery of chapter 6 or a device-side equivalent; a whole-frame version is twice the memory with a rule of who allocates it.
- **A fence or sequence counter**: a register that the device increments when each commit has been applied, which the program can read (poll) or wait for on an interrupt, to know when a buffer is free. *Gives:* synchronisation with no guess. *Costs:* a counter and a rule for wraparound.

And the same loop, in the style the pdp64 could offer, written with three of the candidates (a latching base register, a fence, and the interrupt) in the form that the program meets:

```
# 3. the flip at the blank, with the sugar: no polling of the raster, no waiting for the flip to be safe
poke(SCANOUT_BASE, front)
loop:
    draw_scene(back)
    poke(SCANOUT_BASE, back); poke(ARM, 1)    # staged now, latched by the device at the next blank (3.10.9)
    wait_until(peek(FLIP_COUNT) >= expected)  # a fence: the old picture is free (a candidate, not defined)
    swap(front, back)
```

Which sketch gives which sugar cheaply (3.21): the layer sketches (1 and 2) give per-layer latching and a hardware cursor and sprites almost for free, because the compositing is in the scanout and the shadowing is in its registers; the command-queue sketch (3) gives the fence for free (a command-queue device already completes commands in order and signals them) and the atomic commit (a queue entry is applied whole); the virtio-gpu route (4) gives damage rectangles and fences (both are in its protocol) and nothing about the raster. The names `FLIP_COUNT` and `ARM` above are placeholders; the repository defines an `ARM` register only as a name in 3.10.9.

3.9 What made graphics programming annoying

The author, 2026-11-10: "basically, anything that made graphics programming annoying or cumbersome or boiler platey, we want to sugar away to almost nothing." That is a goal and not a design: it says what the architecture should aim at, and the means, device by device, are open. **Boilerplate** (code that has to be written the same way in every program before the program can do anything of its own, which carries no idea of the program's) is what the goal is about, so the way to take it seriously is to list the boilerplate that exists, show it, and for each piece ask what the architecture could do so that the program does not have to.

3.9.1 The aim is that what made graphics programming annoying, cumbersome or boilerplate is reduced by the architecture to almost nothing; the means are open and each is a candidate (section 3.9 and its table). *(P)* (The author, 2026-11-10, errata 105.)

3.9.2 The standard first-iteration device, virtio-gpu, keeps its own protocol (3.16); where this section says that a sugar "would" appear, it appears in a device of the second iteration (3.10) or as a library over virtio-gpu, and no sugar is specified. *(S)*

3.9.3 The measure that every design in this chapter is held to is the author's: "graphics programming should be as simple and pleasurable as the c64 was." *(P)* (2026-11-10, errata 105; a goal, not a design.)

Before the pairs, the measure of the burden. The books of Michael Abrash on VGA and PC graphics programming (the author's "800 page michael abrash books"; titles and page counts unverified, see 3.5) exist because a programmer could not get a fast picture without knowing the VGA's registers, its planar memory, its retrace, the banking of each chip and how to hand-tune the inner loops; a book of that size is what it took to teach it. Each row of the table below is a piece of that knowledge, or its modern successor (the long lists of the Linux display interface and of Vulkan), that the author would like the architecture to make unnecessary. It is said with respect for the craft: the knowledge was real, and it was needed.

Three worked pairs. Each shows the boilerplate as the real systems have it (the names are those of the real interfaces, checked in the headers on this machine where marked, otherwise general knowledge, *unverified*), and then what the pdp64 would offer. The right-hand versions are sketches and use placeholder names; they are not registers.

Pair 1: getting a picture on the screen at all. On a VGA/VESA PC the program asked the BIOS for the list of modes, picked one, set it, and found the framebuffer:

```
# VESA (VBE), 1990s: about 15 lines in a real program (illustrative, not measured)
info = vbe_get_controller_info()
for m in info.mode_list:
    mi = vbe_get_mode_info(m)
    if mi.width == 1024 and mi.height == 768 and mi.bits_per_pixel == 32 and mi.has_linear_framebuffer:
        mode = m
vbe_set_mode(mode | LINEAR_FRAMEBUFFER_BIT)
fb = map_physical(mi.physical_base, mi.pitch * mi.height)
```

On Linux today (DRM/KMS: the kernel's display interface) the same job is longer. The program opens the device, lists **connectors** (the outputs), reads the monitor's list of modes (**EDID**: the data block that the monitor supplies describing the modes it supports, a general-knowledge name), chooses one (`struct drm_mode_modeinfo`: pixel clock, horizontal and vertical total and sync positions, refresh rate, in `drm_mode.h`), allocates a **dumb buffer** (a plain pixel buffer), registers it as a framebuffer (`DRM_IOCTL_MODE_ADDFB2`, with a format code and a pitch), maps it, and sets the mode on a **CRTC** (the unit that does scanout; the name is from cathode-ray tube controller):

```
# Linux DRM/KMS, one output, no atomic: about 40 lines in a real program (illustrative, not measured)
fd = open("/dev/dri/card0")
res = drmModeGetResources(fd)
conn = first connected connector in res
mode = conn.modes[0]                          # the monitor's preferred mode, by convention first
crtc = a crtc the encoder of conn can use
buf = create_dumb(fd, mode.hdisplay, mode.vdisplay, 32)    # returns handle, pitch, size
fb_id = drmModeAddFB2(fd, w, h, XRGB8888, handle, pitch)
ptr = mmap(fd, map_dumb(buf).offset, size)
drmModeSetCrtc(fd, crtc, fb_id, 0, 0, conn, mode)
```

A program on Vulkan or OpenGL does not do that itself but meets another list: create an instance, enumerate physical devices, choose a queue family, create a logical device, create a window surface, create a swap chain and its images,

create image views, a render pass, framebuffers, command pools and buffers and synchronisation objects (the names `vkCreateInstance`, `vkCreateSwapchainKHR` and the like are Vulkan's; the headers are not on this machine, *unverified*; OpenGL's `glGenBuffers`, `glCompileShader`, `glLinkProgram` are in `glext.h` here). A "hello triangle" in Vulkan is commonly several hundred lines (general knowledge, illustrative). In virtio-gpu the equivalent is the six commands of 3.16, about 25 lines of the real driver: create resource, attach backing, set scanout, draw, transfer, flush.

What the pdp64 could offer is **a device that already shows something**:

```
# the pdp64 way, open (a sketch)
# the device powers up with a sane default mode, in the monitor's preferred mode, with a canonical pixel
format:
fb = peek(DISPLAY_BASE)            # where the picture is, always mapped
w  = peek(DISPLAY_WIDTH); h = peek(DISPLAY_HEIGHT)
poke(fb + (y * w + x) * 4, 0xFFFFFFFF)     # a white pixel: done
```

Options for the sugar, with trade-offs. (a) The device **reports its mode and a sane default** (a register holds the width, the height and the refresh rate of the mode chosen at start-up from the monitor's data, as the virtio-gpu `GET_DISPLAY_INFO` already returns the width and height of each output in a `virtio_gpu_resp_display_info`); nothing to set. *Cost:* the device reads the EDID itself (a small I2C engine, which 3.14.1 already puts at the transmitter chip) and picks a policy; a program that wants another mode still has the registers. (b) A **fixed canonical pixel format** with the device converting at scanout (the repository's 48-bit RGBA pixel, errata 73, would be that format on the native device; virtio-gpu offers only 32-bit formats with eight bits per channel: `VIRTIO_GPU_FORMAT_B8G8R8A8_UNORM` and its siblings, in `virtio_gpu.h`, so that the 48-bit pixel does not exist on the standard device, an inconsistency that is listed in 3.23). *Cost:* a conversion stage in the scanout, and memory a program wastes if it wants fewer bits. (c) **Stride equal to width** unless the program asks for padding, and the framebuffer window **always mapped**: the program has no `mmap`, no bank register, no `map_physical`; the window is in its address space from the start, a decision of the page tables and the bus (2.3.3 and 3.15.4). *Cost:* a fixed place in the address space, and a rule for what a program does that wants two framebuffers.

Pair 2: the cache and the flush. On a modern computer the processor has a **cache** (a small fast memory that keeps recently used data close to the core). When the program writes pixels they may be in the cache and not yet in memory, and a device that reads memory by DMA sees the old data. Programs on non-coherent systems therefore have to flush:

```
# ARM or x86 with a non-coherent display: about 3 lines per update, in every program (illustrative)
draw(framebuffer)
cache_clean_range(framebuffer, size)       # push the dirty lines to memory
memory_barrier()                           # and make sure the writes are ordered before the notification
poke(DOORBELL, 1)
```

The pdp64 already has this sugar, and chapter 2 says how: processor caches are coherent with each other, a device and the processor are not, and the answer is not a flush instruction but a memory type, **uncached**: "buffers are mapped uncached and the hand-off is explicit" (2.5.2), and "no driver can forget a cache clean or invalidate, because there is none to forget" (2.5 commentary); the framebuffer is mapped uncached too (3.15.1). The cost, which the commentary of 3.15 states, is that a long run of writes is no slower per byte but a single write is slow; the candidate improvement is a write-combining memory type for the framebuffer (3.15, Modern comparables), which the architecture's four memory types of 2.3.4 do not name. The sugar is therefore decided for correctness (D) and open for speed (P).

Pair 3: drawing text. On a GL or Vulkan system text is the longest boilerplate of all, because the GPU has no text:

```
# a GPU text path (illustrative; about 150 lines in a real small program, not measured)
font = ft_load("DejaVuSansMono.ttf"); ft_set_size(font, 16)
atlas = new_texture(1024, 1024)
for each character c needed:
    bitmap = ft_render_glyph(font, c)          # the font rasteriser (CPU)
    rect = pack_into_atlas(atlas, bitmap)      # find room: a bin-packing problem
    upload(atlas, rect, bitmap)
    remember(c, rect, advance, bearing)        # metrics: how far to move on, where the glyph sits
for each cell on screen:
    append_quad(vertex_list, x, y, rect_of(char), fg, bg)
upload(vertex_list); bind(atlas, shader); draw(vertex_list)
# plus kerning, ligatures, right-to-left runs, combining marks, fallback fonts, scaling for HiDPI
```

The pdp64's terminal would do it as the C64 did, with one store:

```
# the pdp64 way, open (a sketch; names are placeholders)
poke(CELL(row, col).glyph, SLOT_OF('A'))     # or: poke(CELL(row, col).char, 'A')
poke(CELL(row, col).fg, RED)
# a new glyph: one library call fills a slot from the TrueType engine
font_render_into_slot(font, 'λ', slot)       # rasterises the outline and writes the bitmap into the slot
```

Here the author's glyph slots do the work of the atlas, and a library call replaces the first half of the boilerplate (3.11). The candidates are (a) the cell holds the **character code**, and the device or a table maps the code to a slot, so the program never names slots (the author's c64x terminal of 2026-06-18, `c64stuff.txt`, line 5015: "each cell holds its actual data, not a reference to the data"); (b) the cell holds the **slot number** directly, which is the simplest hardware and makes the program responsible for assigning slots. The trade-off, which is open (3.23): (a) is nicer for the program and costs a lookup table in the scanout; (b) is nearly free and puts a small allocator in a library.

The pain points, in a table

Each row names a thing that has annoyed graphics programmers, the typical size of the boilerplate (the line counts are *illustrative*, from the typical shape of such code, **not measured**), the sugar the architecture could offer, and where it appears in the sketches of 3.21 (S1 layers, S2 layers plus blitter, S3 command queue, S4 plain virtio-gpu with a library). "Decided" means that the repository already decides it.

Pain point	Typical boilerplate (illustrative)	The sugar that would remove it	Where (3.21)
Mode setting and discovering what the display supports (EDID, modes)	15 lines (VESA) to 40 (DRM)	the device reads the monitor and starts in its preferred mode; a register reports it	S1 S2 S3 native; S4: `GET_DISPLAY_INFO` gives the size, a library the rest
Pixel formats	one format code to get right, and conversion code when it is not	one canonical format, the device converts at scanout	S1 S2 S3; S4 has 32-bit formats only
Stride and pitch	one multiply in every pixel address	stride equal to width unless asked; the address of pixel (x, y) is a register-reported formula	S1 S2 S3
Endianness and colour order (RGB or BGR, alpha first or last)	a swap or a table, in every program	one defined order, little-endian (decided for the machine, errata 72); the colour field order named once	all
Bank switching and mapping the framebuffer	5 to 20 lines	a window always mapped, no banks	all (3.15.4 is the rule for the type)
Double buffering and tearing	10 to 30 lines, plus a frame loop	a latching base register, a fence, per-layer flip, versioned buffers (3.8)	S1 S2 per layer; S3 queue; S4 flush
Synchronising with the display	polling or a vblank interrupt handler	raster interrupt and a fence (3.8); decided: vblank interrupt at a fixed line (3.10.9)	all
Cache coherence between CPU writes and the device	3 lines per update, forgotten once per project	decided: uncached memory type, no flush exists (2.5.2, 3.15.1); open: write-combining for speed	all
Device initialisation and context creation	50 to 200 lines (Vulkan), 25 (virtio-gpu)	the device works at power-up; a single `enable` bit, a default	S1 S2 S3; S4 via a library
Buffer and memory management (allocate, map, upload, barriers, fences)	20 to 100 lines	all video memory is a plain array in a window (S1 S2); no upload, nothing to map; or the device allocates (S3)	S1 S2 plain; S3 managed
Shader compilation and pipeline state	50 to 300 lines	no shaders at all in S1 S2; a few fixed programs in S3, chosen by a field	S1 S2 none; S3 fixed
Drawing text (glyph atlases, kerning, layout)	100 to 300 lines	a one-store put-character; glyph slots filled by a library call (3.11)	all native; S4 in a library
Clipping and windows	10 to 30 lines per drawing call	a clip rectangle register per layer; a window is a layer	S2 S3
Sprites and cursors	20 lines to save, draw and restore	hardware sprites and a hardware cursor, two registers each	S1 S2 S3; S4 has the cursor queue only

Pain point	Typical boilerplate (illustrative)	The sugar that would remove it	Where (3.21)
Colour spaces and gamma	a table and a conversion, or wrong colours	one defined space (linear or sRGB, to be decided) with the device applying the transfer curve	open: S1 S2 S3
Resolution changes and hot-plug	20 to 60 lines of event handling	the bus reports a plug as an event (decided in chapter 2, 2.4); the device re-reads the monitor and signals "mode changed"	all
Fonts and scaling (HiDPI: screens with several pixels to a logical dot)	a scale factor threaded through every program	cells and glyph sizes defined in physical units by the device; the TrueType engine renders for the real pixel size	S1 S2 S3
Input of palettes	256 writes through ports (VGA)	a palette is an array in memory, loaded by `copy`; or none (true colour)	optional
Sound and video sync	a timestamp and a resampler	one clock for both devices (the audio clock relation of 3.14.4, decided as a duty of the mode setting) and a frame counter that audio can read	all

Each of these has a cost in gates or in flexibility, and the table's sugar column is not a recommendation: it is the list of things to decide. The ones the repository has *already* decided are the coherence rule (D), the vertical-blank latch (D), the one uncached framebuffer (D) and little-endian (D); every other cell in the sugar column is a candidate.

3.10 The terminal: VIC-64X, SID-64X and BLT64

3.10.1 The graphics terminal is based on two machines, not one: the **Commodore 64**, for its video and sound as chips with registers that a program writes, and the **Blit terminal** (Rob Pike's bit-mapped windowing terminal), for its framebuffer in the address space and the bit-block-transfer operation that moves rectangles of pixels. The pdp64 modernises and extends both. *(D)*

3.10.2 The video device `VIC-64X` is the Commodore 64's video chip (the VIC-II) extended: a framebuffer in the address space with 48-bit pixels, scanned out through a line buffer. A pixel is RGBA with 12 bits per channel (errata 73, the author's decision of 2026-10-24). The register formats and the other pixel formats are unspecified (3.13). *(D)*

3.10.3 The sound device `SID-64X` is the C64's sound chip (the SID) extended: voices with filters, and also a general sound path that is not limited to the SID's voices; audio samples are 48 bits (errata 74, the author's decision of 2026-10-24). The register formats and the layout of a 48-bit sample are unspecified (3.13). *(D)*

3.10.4 The blitter `BLT64`, the Blit's bit-block-transfer operation as a device, copies and combines rectangles of pixels with raster operations: registers for the source and destination base addresses, the strides, the width and height, a raster operation (the sixteen binary operations of source against destination and the common ternary ones), a mask or transparent colour, and a control register with direction, go, busy and interrupt on done. The device chat chose a three-operand `draw` instead (`decisions-pending.md` E3). *(P)*

3.10.5 Each of the three is a device in a slot: it is enumerated by the bus, mapped into its own 4 KB page, and found by its identification register like any other (chapters 2 and 5). The device chat put the C64 devices in one 8 KB window per process (`decisions-pending.md` E5). *(P)*

3.10.6 The framebuffer stays plain memory, so that software can draw without `BLT64`; the blitter is an assist that can be bypassed, not the only way to draw (the Blit authors' advice in 1985 was "transparent, so it can be bypassed"). *(P)*

3.10.7 Four raster operations are required of `BLT64`: store, OR, clear and exclusive-OR (these four sufficed on the Blit, and exclusive-OR dominated its use); the other twelve binary operations and the ternary ones are optional. *(P)*

3.10.8 Whether the video device has a hardware cursor or sprite is unspecified (the vertical-blank interrupt is 3.10.9); the Blit read the mouse position at the retrace interrupt and drew its cursor in software, in exclusive-OR. *(S)*

3.10.9 Every register of the video device that takes effect at a frame boundary is shadowed: one write to its `ARM` register latches them all together at the start of the next vertical blank, and a status bit says whether the arm was taken in this frame or missed it. A sticky underrun bit and an underrun counter report a scan-out that ran dry. The vertical-blank interrupt occurs at a

fixed line of the frame, whose number a register holds. The sound path's audio clock relation (3.14.4) is part of the same mode setting. *(D)*

Commentary. **Precedent.** The Commodore 64's VIC-II and SID were chips whose registers a program poked directly, with a screen in memory that the video chip read by direct memory access; **the Blit**, Rob Pike's terminal, had a framebuffer in the address space and a bit-block-transfer operation that the terminal did in software (Pike, Locanthi and Reiser, 1985: the terminal had "no pixel-moving hardware"), where the Amiga and other 1980s machines added hardware blitters. The Blit was a Motorola 68000 machine (8 MHz) that received downloaded code, characters and editing messages from its host over a serial line and drew every pixel itself; its description as a precursor of the X11 protocol and of the NeWS window system is not stated by any source read *(S)*, while the lineage the sources confirm runs from the Blit's `mux` to 8½ and `rio` (`docs/kernel-pain-points.md` section 14I). The display was a small computer driven by messages, where here it is a set of devices on the bus driven by registers and descriptors, so the pdp64 needs no display protocol between host and terminal (the nearest thing is the descriptor ring of chapter 5); the pdp64's terminal takes the framebuffer and the bit-block-transfer operation from the Blit (the pdp64 adds the hardware engine, `BLT64`) and the video and sound chips from the C64, and both are modernised. The Blit authors give the cost of the split (a slow link, and a program in two halves) as the reason their design was what it was; the pdp64 avoids the split by putting the display on the bus. Plan 9 later moved its window system into files (`/dev/bitblt`, then `/dev/draw`) for the sake of size and remote use: a stream of small commands written to a device, the same shape as an entry in a descriptor ring (reading of the commands as a ring is an inference, not a source's statement). **Trade-offs.** The blitter covers scrolling a terminal, moving a window, drawing a glyph, filling a rectangle and exclusive-oring a cursor, which is the working set of a text-and-windows machine; it does not do three-dimensional graphics. `MCPY` is no substitute: it is one-dimensional and occupies the processor, where a blit has a stride and a raster operation and runs concurrently. 48-bit pixels (presumably 16 bits per channel) exceed what any display connector carries today, so the display device converts to the connector's depth on the way out; 48-bit audio samples likewise exceed what a converter takes, so the sound device keeps the precision for mixing and filtering and reduces it at the output. **Improvements.** The C64's video and sound registers were scattered and irregular; the pdp64 versions use the uniform register face, so one driver skeleton serves them. **Benefits.** The blitter is about 1,500 lookup tables and a line buffer, nearly all address arithmetic; the whole terminal is a handful of small devices. **Modern comparables.** Every embedded display controller has a 2-D blitter and a scan-out engine; sound chips are audio codecs behind an I2S interface. **Impact.** *Kernel:* a framebuffer, a sound and a blit driver are each a mapped page and a few register writes; no cache maintenance is needed because the framebuffer is uncached. *Compiler:* none. *Hardware:* a scan-out engine, a line buffer, a small address-arithmetic unit for the blitter and a mixer for the sound device, each independent of the processor and of the others.

3.11 The terminal as the author describes it: peek/poke text and graphics, and glyph slots

The author, 2026-11-10: "our virtio-console device is an attempt to bring the nice properties of the blit terminal and c64 for doing simple and easy peek poke style display of text and also doing graphics, but with modern color depths and screen sizes, and with glyph maps we can even have a ttf engine generate glyphs and they will go into their memory slots for use."

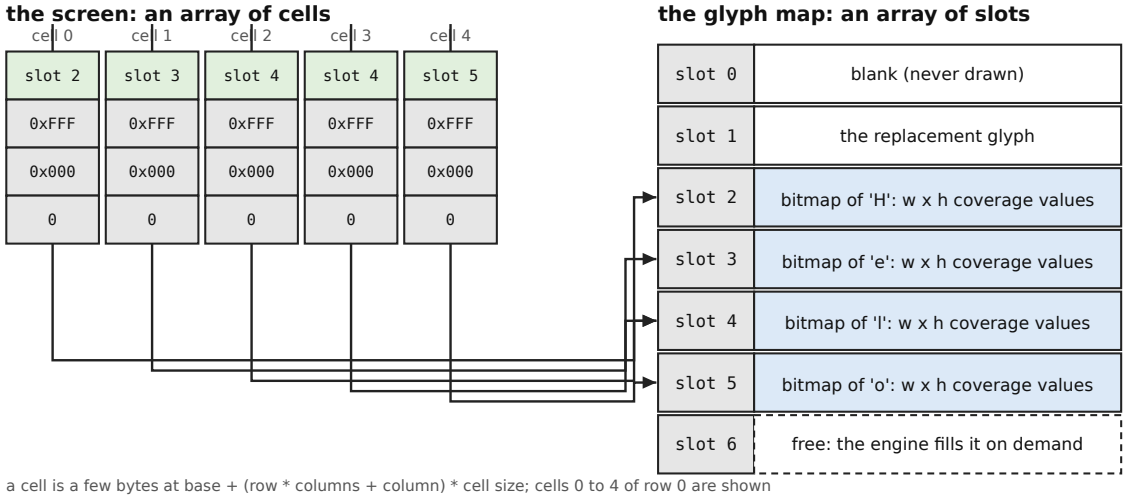
3.11.1 The pdp64's terminal is meant to bring the properties of the Blit terminal and the C64 that the earlier sections describe: a display that a program drives by simple peeks and pokes, for text and for graphics together, at modern colour depths and screen sizes; and a **glyph map** (a table of slots, each a place in memory for the picture of one glyph) that a TrueType engine fills, so that the shapes of the characters are made by software and used by the display through their slots. *(P)*

3.11.2 What the author calls "our virtio-console device" is not specified in this repository: no document defines a device of that name, its identification, its queues or its registers, and the form of the terminal device (a new virtio device, an extension of virtio-gpu, or the native `VIC-64X` family of 3.10) is **open**. *(S)*

The difference, stated exactly, and not resolved. The repository's chapter 3 describes the native devices `VIC-64X`, `SID-64X` and `BLT64`, and these were **deferred** by the author's decision of 2026-11-03 (errata 78) and made obsolete as a model by "forget the native 8 register devices. gone. virtio across the board." (errata 95, 2026-11-09). The standard video and audio devices are **virtio-gpu in 2D mode and virtio-snd** (errata 79). **virtio-console**, in the virtio specification as the Linux header `virtio_console.h` on this machine shows it, is a *serial text device*: device type 3 (`VIRTIO_ID_CONSOLE`), a configuration structure of `cols`, `rows`, `max_nr_ports` and `emerg_wr`, the feature bits `SIZE`, `MULTIPORT` and `EMERG_WRITE`, and control messages for ports (`PORT_ADD`, `PORT_REMOVE`, `RESIZE`, ...); the data are byte streams over queues, one receive and one transmit queue for each port; it carries no

cells, colours, glyphs or pixels. In this repository virtio-console appears only as a candidate for the *console* of the boot ROM and the kernel (statement 5.5.5 and the open question Q6 of the decision records: "virtio-console optional for ordinary use", beside "a minimal fixed UART kept for the boot ROM and panic"), that is as a text stream like the teletype of 3.1, not as a display. So "virtio-console" as the author uses it names a device with properties (cells, colours, glyph slots, graphics) that **neither** the virtio-console of the specification **nor** virtio-gpu 2D has, and that the native devices were to have. Four readings are possible and this chapter chooses none: (1) a new virtio device type of the pdp64's own, with the cell array and glyph slots as the device's memory; (2) virtio-gpu extended by a text layer and glyph slots; (3) virtio-console used as the text *stream* (the path for existing tty programs, 3.2) beside a separate display device; (4) the native device family renamed. The choice is the first question of 3.23.

What the author's earlier terminal chat adds (2026-06-18, recorded in `docs/c64-devices-from-chat.md` section 6; a separate project, not a pdp64 decision): a cell holds a 32-bit foreground colour, a 32-bit background colour, a UTF-8 character and about eight bits of attributes (bold, italic, underline, blink); "the number of cells take up so little memory, the 64 bits devoted to color is trivial"; "each cell holds its actual data, not a reference to the data"; sprites are a separate plane that composites over the cells; existing tty programs must run through a shim and native programs use the terminal fully, "peek and poke is the internal interface". That chat-era design fits the cell array of the figure below; it differs from the author's new words in one place: **the cell there holds the character code itself, and the glyph map of the new words implies a slot** (3.9, pair 3, candidate (a) against (b)).



The same slot 4 serves both letters l: a glyph is drawn once and used by every cell that names it.

A TrueType engine rasterises each glyph into its slot (3.11.3). Field names and sizes are placeholders, not a specification.
Slot 6 shows an empty slot, to be filled when a cell first needs it.

3.11.3 A glyph map is a table of **slots**; a slot holds the picture of one glyph (for example, a rectangle of coverage values, one byte for each pixel, which tells how much of the pixel the glyph covers, so that the display can antialias), and the screen is an array of cells each of which names a glyph (by slot, or by a character code that is looked up to a slot) and gives its colours and attributes. A TrueType engine, which is software, rasterises a glyph from the font's outline at the pixel size of the display and writes the result into a slot; a glyph is made once and used by every cell that names it. The layout of the slot (its width, height, pixel format, and whether a slot may hold several cells' width, for a wide glyph) and the number of slots are unspecified. *(S)*

As an arithmetic example, with the chat-era cell (about 13 bytes: 4 + 4 + 4 for a character + 1 for attributes) and a 1920 by 1080 screen of 8 by 16 pixel cells, there are 240 columns and 67 rows, 16,080 cells and about 209 KB of cells; with slots of 16 by 32 coverage bytes (512 bytes) and 4,096 slots, the glyph map is 2 MB. Both are small against a 12.4 MB frame at 48 bits (3.15.2), which is the author's observation of 2026-06-18: the cells are small, and text is cheap to store. These are examples, not decisions.

3.11.4 Whether the terminal has sprites is not stated in the author's description of 2026-11-10; the C64 had them and the earlier terminal chat gave sprites a separate plane, and statement 3.10.8 leaves the pdp64's hardware cursor and sprites unspecified. *(S)*

3.12 A peek/poke program for the terminal

This is what the author's description would let a program do. **No register of the terminal is specified** (3.13), so the names are placeholders in capitals, to be replaced by the real ones when the device is defined; where a number appears it is an example.

```
# placeholders: SCREEN is the base of the cell array, COLS its width in cells,
#               GLYPHS the base of the glyph map, GLYPH_SIZE the size of one slot

put_char(row, col, ch, fg, bg):                    # one cell: a few stores
    cell = SCREEN + (row * COLS + col) * CELL_SIZE
    poke32(cell + GLYPH, slot_of(ch))              # which glyph (or the character code, 3.11)
    poke32(cell + FG, fg)                          # colours, modern depth: the pixel format of 3.10.2
    poke32(cell + BG, bg)
    poke8 (cell + ATTR, 0)

define_glyph(slot, font, codepoint):               # a glyph the program has just thought of
    bitmap = ttf_rasterise(font, codepoint, CELL_W, CELL_H)   # software: the TrueType engine
    copy(GLYPHS + slot * GLYPH_SIZE, bitmap, GLYPH_SIZE)      # an ordinary memory copy

put_char(0, 0, 'A', RED, BLACK)                    # an A, red on black, top left: four stores
define_glyph(200, my_font, 0x03BB)                 # a lambda goes into slot 200
put_char(0, 1, 200, WHITE, BLACK)                  # and is used by naming slot 200
```

Graphics beside text, in the C64 manner, are another array and one more register set:

```
# the graphics plane: a framebuffer of FB_W by FB_H pixels (3.10.2: 48 bits, or 32 on the standard device)
put_pixel(x, y, colour):
    poke32(FB_BASE + (y * FB_STRIDE + x) * 4, colour)       # stride from the register FB_STRIDE
# sprites, if the design has them (3.11.4: not decided):
poke(SPRITE_N + X, 100); poke(SPRITE_N + Y, 80); poke(SPRITE_N + SHAPE, shape_slot)
```

Three observations. Each line is one store, so the program is as small as the C64's was. Everything is memory, so `memcpy` fills a glyph slot as it fills any buffer. And the *anti-annoyance* of 3.9 is visible in what is missing: no mode setting, no format conversion, no flush (the memory type of 3.15.1 does that), no atlas to pack, no shader, no command buffer. What the terminal is *not* is a three-dimensional machine; for that the standard device, virtio-gpu, remains (3.16, 3.17).

3.13 The stub: the C64 and Blit documentation

3.13.1 The register-level definitions of `VIC-64X`, `SID-64X` and `BLT64` (register names, offsets, bit fields, the 48-bit pixel and sample formats, the sprite, raster-interrupt and voice models inherited from the C64, and the bit-block-transfer operations and windowing model inherited from the Blit) are a stub: only the `BLT64` registers listed in 3.10.4 are specified. The author's earlier device chat decided much of the rest (register conventions, the 8502 island, the display rules); it is extracted in `docs/c64-devices-from-chat.md` and awaits adoption (`decisions-pending.md` E1 to E11). *(S)*

3.13.2 What is fixed: the devices exist, they follow the eight-register convention in the terminal's slots, and the interrupt, `FENCE` and uncached-framebuffer rules of section 3.15 apply. *(S)*

Commentary. This section is the placeholder for the C64- and Blit-derived specification. The Blit side draws on the papers read in section 14I of `docs/kernel-pain-points.md` (the four primitives `bitblt`, `texture`, `segment` and `point`, the `Bitmap`, `Rectangle`, `Point` and `Texture` types, half-open rectangles, and overlapping windows with obscured parts kept off screen); the video chip is built after the platform it runs on. **Impact.** *Kernel:* none until defined. *Compiler:* none. *Hardware:* the block RAM budget (section 3.15) is settled before the video device is built.

3.14 How the terminal drives HDMI and the other ports

3.14.1 The display connector is standard kit: HDMI (and DVI and DisplayPort) is a transmitter chip that takes pixels and sync signals in parallel and does the link protocol, the encoding and the electrical signalling itself; on the first board it is the DE10-Nano's ADV7513, set up once over an I2C-style control bus. *(D)*

3.14.2 The video device's output side is therefore a pixel stream with horizontal and vertical sync, a clock and a data-enable, which is what every such transmitter accepts; the device's registers set the mode (width, height, blanking, pixel clock, the pixel format), the base address of the framebuffer and the display on and off. *(P)*

3.14.3 Other ports are the same shape: a different transmitter (VGA digital-to-analogue converter, DisplayPort, an LVDS panel) takes the same pixel stream, so the pdp64's side does not change, and each port is one more slot or one more output of the video device. *(P)*

3.14.4 The sound device's output is likewise a standard digital audio stream (I2S or the HDMI audio channel) into a converter or the transmitter. The processor never sees the HDMI packets, but whoever sets the mode must also program the audio clock relation: the transmitter needs N (and CTS, or a hardware CTS calculation) for each pixel clock and sample rate, and a driver such as the ADV7511's computes both. *(P)*

3.14.5 The ports that carry data, SATA for storage and USB for peripherals, follow the same pattern: the protocol is in the chip or controller block, and the processor's side is a slot with the common registers and a ring (chapter 5). *(D)*

Commentary. **Precedent.** The MiSTer framework on the DE10-Nano already does exactly this (the retro cores produce a pixel stream and the framework's scaler and the HDMI transmitter do the rest), which is why the plan buys HDMI there. **Trade-offs.** The control bus of the transmitter needs a small amount of one-time setup that a boot routine does by writing a few registers; the pixel clock must be generated to the mode, which on an FPGA is a phase-locked loop and in silicon is the same block. **Improvements.** Because the interface is a stream and a register set, replacing the transmitter or the connector changes nothing above the video device, as a SATA controller behind its ring can be replaced without changing the disk driver. **Benefits.** No part of the operating system carries an HDMI, SATA or USB protocol stack: each is one driver of the common shape. **Modern comparables.** Single-board ARM computers with a display controller and an HDMI transmitter; the video output of every SoC. **Impact.** *Kernel:* a display driver sets a mode and a base address and nothing else; hot-plug of a monitor is a bus event if the transmitter reports one. *Compiler:* none. *Hardware:* a pixel generator, a clock and an off-chip transmitter; no protocol logic in the pdp64's own gates.

3.15 The framebuffer, the memory type and the memory budget

3.15.1 The framebuffer is mapped with the uncached memory type, or the processor's writes sit in the data cache while the video device scans out stale memory; a `FENCE` separates writing a device's registers from setting its go bit. *(D)*

3.15.2 On the first board the block RAM (about 5.5 megabits on the DE10-Nano) cannot hold a modern frame (a 1920 by 1080 frame at 32 bits is 8.3 MB, and at the 48-bit pixel of 3.10.2 it is 12.4 MB), so the framebuffer lives in main memory and the block RAM holds the scan-out line buffer of a few kilobytes. *(D)*

3.15.3 The block RAM budget (caches, translation buffer, the blitter's scratch and the sound voices' state) is settled before the video device is built. *(D)*

3.15.4 The framebuffer is ordinary memory and the devices' registers are device windows (section 2.3.6), so a page-table entry that maps the framebuffer as a device, or a register page as cached memory, is a mistake that the memory type of the page makes visible to the kernel. *(P)*

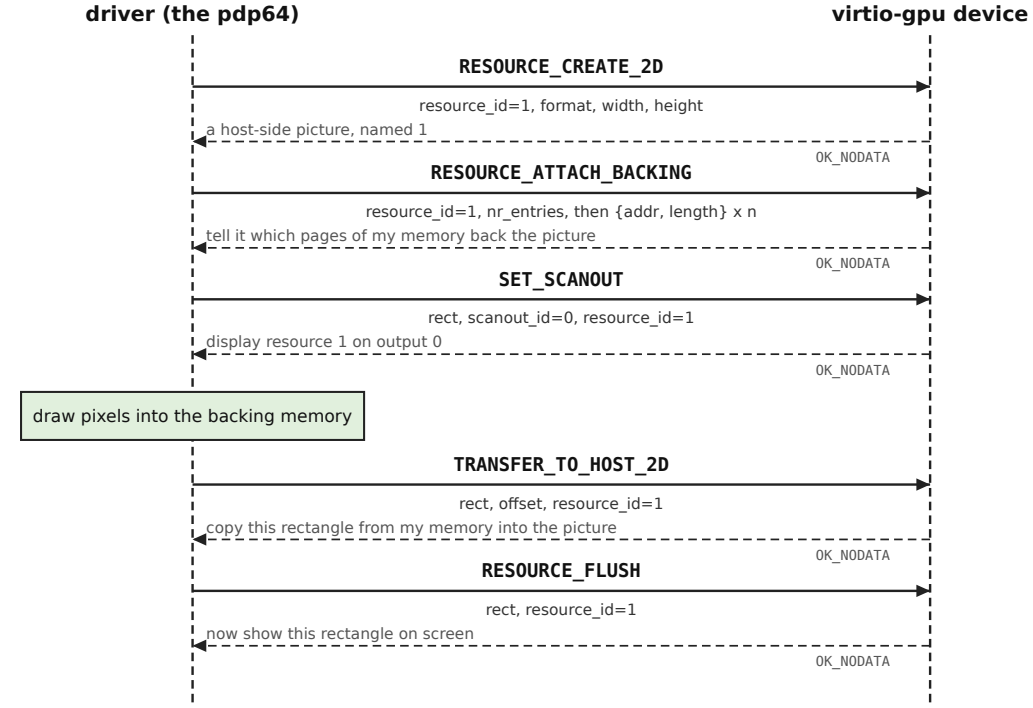
Commentary. **Trade-offs.** An uncached framebuffer makes a single pixel write slow and a long run of writes no slower per byte, which suits the blitter (it writes the bus directly) and punishes pixel-by-pixel drawing from the processor, which is what a blitter is for. **Modern comparables.** Write-combining memory types on x86 and ARM serve the same purpose. **Impact.** *Kernel:* the mapping rule is one memory-type value in the page-table entry. *Compiler:* none. *Hardware:* the scan-out reads main memory by bus mastering, which competes with the processor for bandwidth.

3.16 virtio-gpu: the standard graphics device

The first iteration's display (errata 79: "virtio is fine"; virtio-gpu in its 2D mode) is a virtio device like the disk of chapter 2, so everything of 2.6 applies: it is found by enumeration, its queues are rings of descriptors in memory, and the work is commands in a queue. What the commands are, in the Linux header `virtio_gpu.h` and as the QEMU emulator (`hw/display/virtio-gpu.c`) carries them out, is this.

The vocabulary. A **resource** is a picture that the device owns, named by a number the driver chooses (`resource_id`), with a pixel format, a width and a height. Its **backing** is the driver's own memory that holds the pixels: a list of `{addr, length}` entries (`struct virtio_gpu_mem_entry`) naming the pages. A **scanout** is one output of the device (up to `VIRTIO_GPU_MAX_SCANOUTS`, 16, in the header); `SET_SCANOUT` says which resource, and which rectangle of it, an output shows. Two commands move and show the pixels: `TRANSFER_TO_HOST_2D` copies a rectangle of the backing memory into the device's picture, and `RESOURCE_FLUSH` tells the device that a rectangle changed and should be shown. The pixel formats are 32 bits with 8 bits a channel: `B8G8R8A8_UNORM`, `B8G8R8X8_UNORM`, `A8R8G8B8_UNORM`, `X8R8G8B8_UNORM`, `R8G8B8A8_UNORM`, `X8B8G8R8_UNORM`, `A8B8G8R8_UNORM`, `R8G8B8X8_UNORM` (values 1 to 4, 67, 68, 121 and 134 in the

header; the names say the order of the channels, which is the *colour order* annoyance of 3.9). Every command starts with a `virtio_gpu_ctrl_hdr`: a type, flags, a 64-bit fence id, a context id and a ring index; the flag `VIRTIO_GPU_FLAG_FENCE` asks that the response carry the fence id once the work is done, which is the device's only completion-ordering tool. The rectangle is `struct virtio_gpu_rect`: x, y, width, height, four 32-bit numbers.



Each line is one command in the control queue, answered by a response; the header of every command is a type, flags, a fence id, a context id.

```
# the whole of a virtio-gpu 2D start-up and one update, as pseudocode over the queue of chapter 2.6
send(RESOURCE_CREATE_2D,     resource_id=1, format=X8R8G8B8, width=1024, height=768)
send(RESOURCE_ATTACH_BACKING, resource_id=1, entries=[{addr=PAGES, length=1024*768*4}])
send(SET_SCANOUT,            scanout_id=0, resource_id=1, rect={0,0,1024,768})

draw_into(PAGES)                                  # the driver writes pixels: ordinary stores
send(TRANSFER_TO_HOST_2D,    resource_id=1, rect={0,0,1024,768}, offset=0)
send(RESOURCE_FLUSH,         resource_id=1, rect={0,0,1024,768})
```

What QEMU's code does (read in `virtio-gpu.c` here): `transfer_to_host_2d` rejects a rectangle that is not inside the resource with `VIRTIO_GPU_RESP_ERR_INVALID_PARAMETER`, then copies it **row by row** from the backing into the resource, using the resource's stride, or in one copy if the rectangle spans whole rows; `resource_flush` rejects a rectangle outside the resource in the same way and then asks each scanout showing the resource to update the part of its display that the rectangle covers. So the transfer is the **upload** and the flush is the **present**, and the program must do both after drawing.

What it does not define. Neither the header nor the QEMU function names anything about the vertical blank, a page flip, a refresh rate or tearing: the flush says "show this rectangle now" and the host decides when its own display next reads it. The only timing handle is the fence of the header. The emulator's configuration structure has a `refresh_rate` field for each output that a host can set (`virtio-gpu.h`), and the device can report an EDID with the optional feature `VIRTIO_GPU_F_EDID`; the header does not say that either of them changes how a flush is timed. This chapter does not claim more than that. The standard device therefore gives a program the damage rectangle (the flush rectangle), a cursor (the cursor queue and its `UPDATE_CURSOR` and `MOVE_CURSOR`), a fence and display information (`GET_DISPLAY_INFO`: the size and enabled flag of each output), and none of the anti-tearing sugar of 3.8 beyond that.

3.16.1 The standard graphics device of the first iteration is virtio-gpu in 2D mode over the virtio PCI Express transport (errata 79); its commands are the six of the figure, plus the cursor queue and the display-information query. *(D)*

3.16.2 The feature subset of virtio-gpu that the pdp64 requires (the EDID feature, the fence flag, the cursor queue, the blob and 3D features of the header, which are not used for 2D) is not specified. *(S)*

3.16.3 The framebuffer of virtio-gpu is the backing memory that the driver allocates; on the pdp64 it is ordinary memory mapped uncached (3.15.1), and the transfer command is the hand-off that tells the device to read it (the five steps of 2.5.3). *(P)*

3.16.4 The standard device has no 48-bit pixel and no text layer: the 48-bit pixel of 3.10.2 belongs to the native device that is deferred, and a text terminal over virtio-gpu is a program that draws glyphs into the backing memory with the processor (the way 3.5 ends) or with a library. *(S)*

Commentary. **Precedent.** Every virtualised PC uses virtio-gpu or a card of the same shape: the guest draws into its own memory and tells the host which rectangle to show. **Trade-offs.** The route is the most standard (one driver, tested, the same in QEMU, `sim64` and the FPGA, as errata 79 says) and does everything in software; the processor writes every pixel and the host copies them, which for a terminal is cheap if the damage is small and for a game is the very cost of 3.5. **Improvements.** The sugar of 3.9 can be added as a library over it (a library that sets up the six commands, allocates the picture, draws text from a glyph cache and flushes only the damage), which is sketch 4 of 3.21. **Benefits.** No special hardware; the display of the first iteration works in the emulator now. **Modern comparables.** Linux's `virtio_gpu` driver, which sits on the DRM interface of 3.9. **Impact.** *Kernel:* a driver for the six commands, one queue of chapter 2.6, and a cursor queue. *Compiler:* none. *Hardware:* none beyond the virtio device and the bus.

3.17 A standard graphics card

3.17.1 A PCI Express graphics card is not excluded by the architecture: a bridge could present PCI Express functions as slots on the bus, enumerated like any other device. *(P)*

3.17.2 Against a standard card: PCI Express configuration space, base-address mapping and message-signalled interrupts, a driver stack for a programming model that for some vendors must be reverse-engineered, the card's firmware and the initialisation a PC's firmware performs, and no hard PCI Express block on the first board (the Cyclone V and the Zynq 7020 lack one; a Zynq UltraScale+ has it). *(D)*

3.17.3 On the other side PCI Express is a standard enumerated bus, which suits the bus-as-a-device design (each function a slot, as each USB device is), and OpenBSD carries graphics drivers for several vendors' cards ported from Linux *(not confirmed in the sources read)*. *(P)*

Commentary. **Trade-offs.** The terminal described here works on the first board and costs a few thousand lookup tables; a standard card needs a root complex, which is a large block when it is not already on the chip, and a graphics driver ported to the pdp64, which is a larger task than the architecture. **Modern comparables.** PowerPC and RISC-V workstations plug in PCI Express graphics cards with open drivers. **Impact.** *Kernel:* a PCI Express card would need a large graphics driver ported. *Compiler:* none. *Hardware:* a PCI Express root complex and bridge to the bus.

3.18 A c64 mode: a small computer on the die that acts as the terminal

The author, 2026-11-10: "we will have a c64 mode, where a c128 is built onto the die with sid etc, so a bit of code loaded into its address space by pcie dma will let it act as a terminal the way the blit and vt55 did." This is a **proposed design idea**, not a design: it says what the author would like to have and leaves every question about it open, and this section writes it up for the two readers and states those questions.

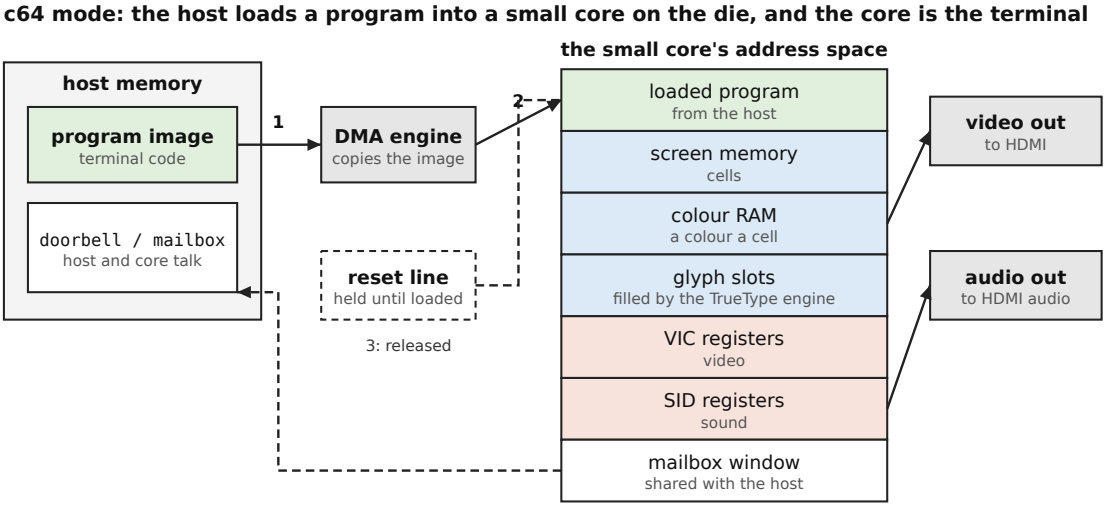
What the Blit did. The Blit (3.3) was a terminal with a computer of its own, a Motorola 68000 (8 MHz, general knowledge of the repository's records), and the host did not send it escape codes to interpret: it sent it **programs**. The records of the repository (`docs/kernel-pain-points.md` section 14I, from the Blit papers) say that the terminal received downloaded code over its serial line, that the terminal software `mpxterm` (1,627 lines of C and 204 of assembler, as recorded) multiplexed several host programs over the one line, and that the authors' reason was that "an intelligent terminal is not an idiot savant, it is one that can be educated". So a **terminal with its own small computer** means this: the display, the keyboard and the mouse are driven by a processor on the terminal's side of the cable, which runs a program that the host loaded; the host talks to that program by messages, and the program does the drawing. How the download worked in detail (the loading protocol, the format of a program) is not in the records and is *unverified* here. The VT52 and VT100 were the opposite, a fixed circuit that obeyed a fixed set of codes (3.2); and "vt55" is, as 3.2 says, the author's naming.

Why a small core with the C64's chips is a terminal on a chip. Take a small computer, give it the C64 family's display and sound circuits, the **VIC** (the video chip: text cells, sprites, colours), the **SID** (the sound chip), **screen memory**, **colour RAM**, **glyph memory** (3.11) and a window towards the host, and let the host load its program: that is a terminal, with

every part of it in one place. The program on the small core is the terminal's firmware: it receives bytes or messages from the host (by the mailbox), puts characters into screen memory, plays a sound, moves a sprite. The host's operating system sees a device that takes characters, as it saw a teletype, and the author's old problem (3.2) is solved the Blit's way: the cleverness is in a program that the host can replace, and the pdp64 core is not burdened with the drawing.

What "loaded by PCI Express DMA into its address space" means, mechanically. The host places the program image (a block of bytes, the terminal's code and its initial data) in its own memory. It tells the device's **DMA engine** (the part that copies memory without the processor's help, chapter 2) the address of the image, its length and the address in the small core's memory where it is to go. The engine reads the host's memory over the PCI Express link (the bus-master read of 2.5) and writes the bytes into the core's memory. The core has been held in **reset** (a line that keeps a processor from running until it is released) since the machine started, so that it executes nothing while its memory is incomplete; when the copy is done the host releases the reset, and the core starts at its fixed start address. In pseudocode, with placeholder names (no register of any such device exists):

```
load_terminal(image, length):                    # on the host, as a driver would
    poke(CORE_CTRL, HOLD_RESET)                  # the core is stopped
    poke(DMA_SRC,  physical_address_of(image))   # where the image is, in host memory
    poke(DMA_DST,  CORE_LOAD_ADDRESS)            # where it must go, in the core's address space
    poke(DMA_LEN,  length)
    poke(DMA_GO,   1)
    wait_until(peek(DMA_DONE) == 1)              # or an interrupt
    poke(CORE_CTRL, RELEASE_RESET)               # the core starts executing the loaded code
    # the host now talks to it through the mailbox:
    mailbox_send(PUT_TEXT, row, col, "hello")    # a message the loaded program understands
```



1 the host places the image in its own memory; the device copies it into the core by DMA (over PCI Express).
2 the program lands at the core's load address; 3 the core is let out of reset and runs it as the terminal.

Which core, what the mailbox is, and how the image is checked are open (3.18); sizes and order are an example.

The picture of the core's address space is the C64's (3.4) with a mailbox added. Everything the loaded program needs is in one flat space, as in the 1982 machine: its own code, the screen and colour memory, glyph slots, the video and sound registers, and a window that the host also sees. Video goes out to the display connector and audio to its audio channel (3.14); the sound path to HDMI audio brings the audio clock relation of 3.14.4 with it, which the small core's program would have to honour or the transmitter's own calculation would have to supply.

3.18.1 A **c64 mode** may be offered: a small core on the die, with the C64 family's circuits (SID and others), into whose address space a program is loaded by DMA over PCI Express so that it acts as a terminal, as the Blit's terminal program did. *(P)* (The author, 2026-11-10, errata 105.)

3.18.2 C64 mode is the enabling or disabling of the on-die c128 as a PCI Express device: when it is enabled it takes over the graphics and audio output, or sends them to a compositor; the form of the compositor path is open. *(P)* (The author,

2026-11-10: "the mode is just enabling/disabling the c128 as a pcie device so it takes over the graphics and audio output... or sends them to a compositor. not sure what that would look like. is there virtio-compositor?")

3.18.3 Everything about this idea other than the two sentences above is open and unspecified: which core; whether "c64 mode" also means running unmodified C64 software; how video memory is shared; the form of the compositor path; how the core and the host talk; how the loaded image is authenticated and contained; and how the idea relates to the native devices and to virtio. *(S)*

The questions the author's words leave open, stated and not answered:

1. **Which core?** He wrote "a c128". The C128 (general knowledge, *unverified* here) combined an 8502 (a faster member of the 6502 family, like the C64's 6510), a Z80 (a second, unrelated processor, used for CP/M), the VIC-IIe (the video chip with extra registers), the 8563 **VDC** (a separate 80-column video chip, with its own memory) and a SID, with a memory-management chip for banking. The repository's records mention the 8502 and not the Z80 or the VDC: the author's earlier device chat (`docs/c64-devices-from-chat.md` section 4, chat-era, nothing decided) put an **"8502 compatibility island"** in the design: a soft 8502 held in reset at power-on, with its registers readable and writable by the host, running at the C64's clock, so that Commodore software could run. The questions: is it (a) a real 6502-family core, as a hard block or as a soft core in the programmable logic; (b) an *emulation* of one, in software or in hardware; or (c) a core of the pdp64's own instruction set given the C64's registers? Does "c128" mean the 8502 only, or the Z80, the VDC and the memory-management chip too?
2. **What does "c64 mode" mean, beyond the author's clarification?** He says the mode is "just" the on-die c128 appearing as a PCI Express device, taking over the graphics and audio output or sending them to a compositor (3.18.2). Still open: does the loaded program run **unmodified C64 software** (the Kernal, BASIC, games, which need the exact chips and timing: the "island" idea of the chat) or only **terminal firmware** written for the small core? And what is the compositor path (below)?
3. **How is video memory shared** with the host and with virtio-gpu (3.16)? Is the core's screen memory its own, behind a window (VRAM in the sense of 3.21), or in host memory that the video chip reads by DMA (3.15.2)? How does the core's picture reach the display when the standard device is virtio-gpu, which has a resource of its own, and who owns the scanout?
4. **How do the core and the host talk?** A mailbox and a doorbell (registers and a write that interrupts), a virtio device with queues (as everything else, 2.6), or a shared window of memory that both can read and write? The figure draws a mailbox only as a placeholder.
5. **How is the host's program image authenticated and contained?** A program loaded from the host runs on the die with access to the core's whole address space and, depending on the design, to the bus. The IOMMU of errata 75 and 85 (the table of which memory a device may touch) is the architecture's answer for devices that master the bus; whether the small core's DMA goes through it, whether the image must be signed, and who holds the signing key, are the author's decision and are listed here only as questions.
6. **How does it relate to the deferred native devices (`VIC-64X`, `SID-64X`, `BLT64`, errata 78) and to virtio across the board (errata 95)?** The tension, stated plainly and not resolved: the author said "forget the native 8 register devices. gone. virtio across the board." (errata 95) and, earlier, that "iommu will allow us to map in c64/c128 and i think we should, but that is extraneous" (errata 75, 2026-11-03). A small core with the C64 family's chips is a native, register-level device by definition, so it is either (i) the surviving form of the deferred native devices, serving as a terminal, with the VIC and SID as its private chips and not as the system's devices; (ii) something that speaks virtio to the host on its outside (a virtio device served by the core's program) and the C64's registers only on its inside, which would honour errata 95 on the bus; or (iii) a thing apart from the standard, an optional extra that errata 95 did not have in view. Which is meant is the author's.
7. **What does it cost** in gates and die area? A 6502-class core is a few thousand gates (general knowledge, *unverified*); the SID and VIC, the memories and the DMA engine are what the FPGA's block RAM (3.15.2) must also hold; **an estimate, mine, unmeasured**: the core itself is small beside the memory of the display, which is why the idea is attractive. A hard block on the die costs area for everyone; a soft core exists only in the programmable logic of the FPGA phase.
8. **How does its sound reach HDMI audio?** Through the same 48-bit-sample sound path (errata 74, 3.10.3) or the SID's own output converted to the audio stream of 3.14.4.

The options, with their trade-offs, none decided. (a) **A hard core on the die** (a real 6502-family core plus the chips): fastest and smallest per core; permanent die area, and it fixes the idea in silicon before the author has answered the questions above. (b) **A soft core in the FPGA phase only**: costs nothing in silicon and lets the idea be tried on the first board (the programmable logic can hold a small core and the chips, as the MiSTer framework does, 3.14); gone from a later chip unless it is made hard. (c) **Software emulation of the C64 machine on a pdp64 core** (the pdp64 runs an emulator of the 6502, VIC and SID as a program): no hardware, easiest to change, uses a core and loses the C64's timing exactness; it is the Blit's idea turned around, the emulator being the "terminal program". (d) **None, and the terminal is a virtio device served by a host process**: the standard route (errata 95), the terminal's firmware is a program on the host (as a terminal emulator is, 3.7); nothing on the die; it loses the property that the terminal is a computer of its own, which the author values in the Blit. Option (c) and option (d) differ in where the C64 machine runs, not in whether it exists.

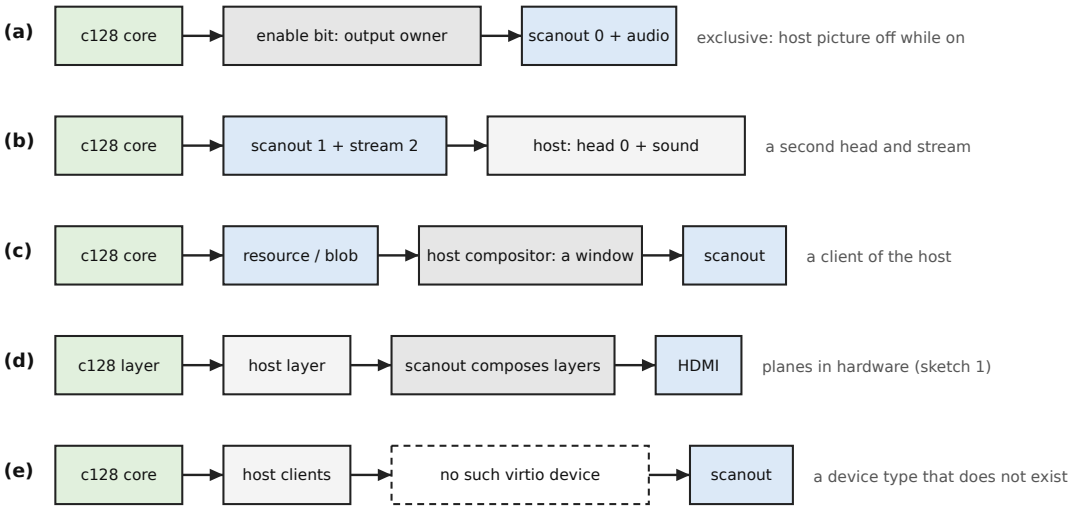
The compositor question

The author, 2026-11-10: "the mode is just enabling/disabling the c128 as a pcie device so it takes over the graphics and audio output... or sends them to a compositor. not sure what that would look like. is there virtio-compositor?"

What a compositor is. A **compositor** is the program (or the hardware) that takes the pictures of several windows or sources and combines them into the one picture that is sent to the display, which is then scanned out row by row as in 3.8, with its own flipping. A desktop's window system is one: each program draws into its own picture (its **surface**) and the compositor places the pictures, one over another, into the frame that the scanout reads. In 3.5 the windowing systems did it in software; in a sketch with layers (3.21, sketches 1 and 2) the scanout hardware does it, as the C64's chip did with sprites.

Why "takes over" is the simple case. If the c128 owns the output outright when it is enabled, there is nothing to compose: the machine has one scanout and one audio path, and the question is only who drives them. The multiplexer is a single enable bit (path (a) below), and switching the bit is the whole of the mode change. The compositor case is harder in exactly one way: the core is no longer the owner of the display but one client among others, so its picture and its sound must be handed over in some form that the host's compositor, or the display controller's layers, can place.

five ways the c64-mode core and the host could share the display and the audio



Green: the on-die core. Blue: output. Grey: the arbiter. Dashed: a thing that would have to be invented.
In (a) the multiplexer is one enable bit; in (b) to (e) the core becomes a client of something else.

The hand-over of any of these should happen in a vertical blank, with the audio muted by a short ramp (3.18).

What exists, checked. There is **no virtio device called a compositor**: the device types listed in `virtio_ids.h` on this machine are network, block, console, random numbers, balloon, SCSI, 9p, GPU (16), clock, input (18), socket, crypto, IOMMU, memory, sound (25), file system, persistent memory, video encoder and decoder, I2C, watchdog, CAN, DMA buffer (37), Bluetooth, GPIO and SPI, with a few others (network wireless, firmware and security modules), and the standard headers on this machine are `virtio_9p`, `balloon`, `blk`, `bt`, `console`, `crypto`, `fs`, `gpio`, `gpu`, `i2c`, `input`, `iommu`, `mem`, `net`, `pmem`, `rng`, `rtc`, `scmi`, `scsi`, `snd`, `spi`

and vsock, among the transport files. What virtio-gpu offers that comes near a compositor, in virtio_gpu.h: up to 16 scanouts (VIRTIO_GPU_MAX_SCANOUTS), so the host can present several outputs; SET_SCANOUT_BLOB, which sets a scanout from a blob resource (a memory object shared with the host without copying); a **cursor queue** separate from the control queue, with UPDATE_CURSOR and MOVE_CURSOR; capability sets (VIRTIO_GPU_CAPSET_VIRGL, VIRGL2 and GFXSTREAM_VULKAN) selected with context_init so that several rendering contexts, each with its own ordering (the "timelines" of the header comment), share the device. QEMU's documentation (docs/system/devices/virtio/virtio-gpu.rst) describes a variant, virtio-gpu-rutabaga, that provides "gfxstream rendering and Wayland display passthrough", with a capset named cross-domain for Linux guests and an experimental x-gfxstream-composer for Android guests; it says no more than that about what the cross-domain path carries. What I take from it, as general knowledge and *unverified* beyond those sentences: the crosvm project's use of the same mechanism so that a guest's windows appear as windows of the host's Wayland compositor, which is the path (c) below; Wayland is the Linux display protocol in which clients hand their surfaces to a compositor. **virtio-snd** and **virtio-input** exist for audio and for the keyboard and mouse. So **path (b) and path (c) are built from parts that exist**, and **path (e) is a new device type**: the process by which a virtio device type gets its number is the specification's own, and is *unverified* here.

The options, none decided.

- **(a) Exclusive takeover.** When the c128 is enabled it owns scanout 0 and the audio path; the host's picture and sound stop. *Gives:* the simplest hardware (an enable bit and a mute); the C64's one-screen model. *Costs:* the host cannot show anything while the core runs; a text terminal that wants to coexist with a window system cannot. Right for a console in the Blit's sense, wrong for a window.
- **(b) The core is one more virtio-gpu scanout and one virtio-snd stream.** It is a second head of the display device (or a window on the host's, if the host maps heads to windows) and one more stream of the sound device. *Gives:* nothing new to invent; the host's existing drivers and compositor handle it; it is the shape of errata 95. *Costs:* the core must speak the virtio-gpu and virtio-snd protocols, so a small core would need those in firmware or hardware, which is the larger part of a driver.
- **(c) A surface for the host's compositor (the author's stated preference, below).** The core renders into a resource (or a blob) that the host's compositor treats as a window's surface, as any client's. *Gives:* the core appears as a window, with the host's window management, overlap and flipping; the path that Wayland passthrough takes (general knowledge, unverified). *Costs:* the compositor must know that the surface is the core's; a copy or a shared blob; the core's frame timing becomes the compositor's.
- **(d) Layers at scanout** (sketch 1 of 3.21). The display controller has a layer for the core and a layer for the host's picture and composes them as it reads them out. *Gives:* the cheapest path and no copy; the core's picture is a plane in video memory; a vertical-blank latch per layer (3.8) switches it without a tear. *Costs:* the display controller must be the layered one of sketch 1, which is not the first-iteration device.
- **(e) A new virtio device type for compositing.** It does not exist. What it would need to be useful: a way for a client to give the device a surface (a resource or blob), to say where it goes and how it overlaps, to swap it atomically at a blank, to ask for the frame's timing and to hand over the matching audio. Most of those are what virtio-gpu's resources, scanouts and fences already say; *a new device type would repeat them*, and it is the author's call whether a compositor is a device or a protocol of existing ones. *Gives:* one model for every client of the display, the core included. *Costs:* a specification to write and have accepted (unverified: the virtio process), and a driver for every host.

3.18.4 Of the five compositor paths the author prefers (c), that the c128 renders into a buffer that the host's compositor treats as a surface: "c128 drawing into a buffer makes most sense" (2026-11-10, errata 106). A stated preference, and not a decision. *(P)*

The author's leaning. He prefers path (c): the c128 draws into a buffer, and the host compositor places it. It is the path that makes the core a client like any other, the one the rio design takes for every window (3.19: a window is a buffer and a rectangle), and the one whose parts exist (a virtio-gpu resource or blob as the surface, 3.16); what remains open under it is how the compositor learns that the buffer is the core's, who owns the copy or the shared memory, and how the core's frame timing meets the compositor's. The audio form of the same choice is in 3.20.

How the switch avoids tearing and pops. Switching the mode (enable or disable) changes who feeds the scanout and the audio path. If it changes in the middle of a frame the picture tears (3.8) and the sound clicks (a sudden change of the waveform's value is heard as a pop). The remedy has the shape of 3.8: make the hand-over at a **vertical blank**, as the latch of 3.10.9 does for registers (the enable bit as one more shadowed register that ARM latches), and **mute with a short ramp**

(a fade of a few milliseconds down before and up after, rather than a cut). Which of these a design uses depends on the path: in (a) the enable bit is the shadowed register and the mute belongs to the audio path; in (b) to (e) the compositor does the hand-over and the core only stops or starts its stream. The audio clock relation of 3.14.4 must be unchanged across the switch, or the transmitter loses its lock and the sound drops.

Commentary. **Precedent.** The Blit loaded its terminal program from the host (above); the C128 put several complete computers in one box (general knowledge, *unverified*); the MiSTer framework runs C64 and C128 cores in programmable logic; a PC's graphics card carries its own processor and firmware (*unverified*). **Trade-offs.** A computer of its own makes the terminal programmable and independent of the host's drawing, and makes it a second machine to load, secure and keep consistent with the first. **Impact.** *Kernel:* a driver for the loader and the mailbox. *Compiler:* none. *Hardware:* a core, memory, DMA, the C64 family's circuits; or nothing, in option (d).

3.19 Windows: from the Blit to rio and NeWS

The author, 2026-11-10: "it should mention blit terminal was inspiration for the rio and NeWS windowing systems, and we'd like to bring over what we can of that simplicity also."

What the sources confirm and what they do not. That the Blit was the inspiration of **rio** is borne out by the papers recorded in `docs/kernel-pain-points.md` section 14I: the Blit ran `mux`, a window system in the terminal; its successor on Plan 9 was **8½**, whose paper says "Primarily for reasons of history and familiarity, the general model and appearance of 8½ are similar to those of mux", and **rio** replaced 8½ ("Note that 8½ has been replaced by rio (see rio(1))"; rio(1): "Rio manages asynchronous layers of text, or windows, on a raster display"; the word *layers* is the Blit's). That the Blit inspired **NeWS** is the author's statement; **no source read for the repository makes that claim**, and the glossary says so, so here it is the author's account, not a checked fact. What the repository does record about NeWS is from *The UNIX-HATERS Handbook* (section 14G of the same file): NeWS was an extensible window server in which "the client can download programs and data structures to draw the whole schematic", and "ultimately, NeWS was not economically or politically viable" (the quoted reasons are about licensing). That NeWS was Sun's, that its programs were written in PostScript (a page-description language that is also a programming language) and that James Gosling designed it are general knowledge, *unverified* here.

What rio is, for the two readers. rio is the window system of Plan 9 (the research operating system that followed Unix at Bell Labs; Rob Pike wrote 8½ and its descendant). Its central idea, as the 8½ paper records it, is that the window system is a **file server**: "The most important contributor to its small size, though, is its overall design as a file server." A program uses a window by opening files (`/dev/cons` for text, `/dev/mouse`, `/dev/bitblt` or later `/dev/draw` for graphics): each window is a private screen that the program draws on, and the server keeps the window's bitmap and carries out the drawing for the program ("no bitmap graphics operations are executed in the client at all"). There is one small server and no **toolkit** (a library of buttons, menus and other widgets that every program of a window system links in; X and its descendants have several, rio has none in the base system). The result, in the paper's words, is a window system of a size that a programmer can read in an afternoon (the paper says "small"; I give no line count).

What NeWS was, for the two readers. In a conventional window system a program on the host sends the display server drawing commands one at a time over a slow link, and each round trip costs time (the "latency" complaint of 3.2). NeWS made the server itself programmable: the host *downloaded a program* into the display server, and the server ran it locally for every mouse movement, so that the interactive loop (dragging a shape across the screen) did not cross the link. This is the Blit's idea one step on: the Blit downloaded a program into a *terminal*; NeWS downloaded programs into a *window server*. The repository's own comparison (`kernel-pain-points.md` row 10 and its note) says the Handbook's argument "is the same shape as the Blit lineage", and notes that it does not claim a historical link beyond what the sources say.

3.19.1 The pdp64 is to bring over what it can of the simplicity of the Blit's descendants rio and NeWS (the author: "we'd like to bring over what we can of that simplicity also"); a goal, and the means are open. *(P)*

3.19.2 Which of the options below the pdp64 takes, if any, is open. *(S)*

What simplicity could be carried over, as options with their trade-offs, none decided.

- **A window is a plain buffer and a rectangle.** The program owns a pixel buffer (or a cell array, 3.11) and the system knows only where to place it. *Gives:* the model of 3.9's goal; drawing is writes to memory with nothing to ask for. *Costs:* a buffer per window (memory), and a rule for who may write whose (the memory protection of chapter 6). *Ties to:* path (c) of the compositor question (3.18), where the c128 draws into a buffer that the compositor places.

- **Drawing is writes to that buffer.** No drawing protocol in the base system: the program uses `memcpy`, a library, or the blitter (3.21 sketch 2). *Gives:* no round trips, no server-side drawing language. *Costs:* every program draws for itself (the opposite of rio's "the server carries out the drawing"); a library must exist for text and shapes.
- **The server composes the buffers.** A small program (or the display hardware, sketch 1) places the buffers into the one picture. *Gives:* one place that handles overlap, flip and tear (3.8). *Costs:* a copy or a layer per window; the server must be trusted with every buffer.
- **Downloadable server-side code.** A program is loaded into the display server, as the Blit's terminal program and NeWS's PostScript were: here, into the c128 core of 3.18, which would be the display server. *Gives:* the display can be given new abilities without changing the host; interaction stays local. *Costs:* a second program to secure (3.18 question 5), and the two-halves problem of the Blit ("writing one program in two pieces is much harder than writing two programs", `kernel-pain-points.md` row 12).
- **No toolkit in the base system.** The system provides windows, text and pixels; widgets, if any, are a library the program chooses. *Gives:* the size rio had; nothing to keep consistent. *Costs:* every program draws its own look, and users see the inconsistency.
- **Windows as files, or as virtio queues.** rio exposes a window as files; the pdp64's devices are virtio queues (chapter 2). A window could be a queue of drawing messages (one more virtio shape), or a file the kernel maps, or both. *Gives:* the uniformity rio and 8½ had ("the same device interface at every level"). *Costs:* a naming and permission model for files (a chapter 6 matter) or a queue per window.

3.20 The audio path: samples, mixers and the pdp64

The author asked, 2026-11-10: "what is the audio equivalent, what do those mixers do to combine multiple programs audio?" The answer is that audio has the same structure as pixels, with a **mixer** where video has a compositor, and this section takes the parts in the same order as 3.8 and 3.18.

What a sound card does. A sound is a pressure wave, stored as a list of **samples** (numbers, each the height of the wave at one instant) taken at a fixed rate: 44,100 or 48,000 a second are the usual rates (44.1 kHz is the compact disc's and 48 kHz the video world's; general knowledge, *unverified*, though both appear in the virtio-snd rate list, below), with each sample a 16-bit integer, or 24 bits, or a floating-point number. A **DAC** (digital-to-analogue converter: the chip that turns a number into a voltage) takes one sample on each tick of a clock, and the speaker moves. The device is fed from a **ring buffer**: a block of memory, treated as a circle, which the program fills and the device empties, the device reading the next sample at each tick and the program writing ahead of it. It is the scanout idea of 3.8 exactly: the display reads the framebuffer at a fixed rate, the DAC reads the ring buffer at a fixed rate; a **buffer underrun** (the device reaches samples the program has not yet written) is **audio's tearing**, heard as a click or a gap, and is as sure to happen when the program is late as a tear is when it draws late.

```
# fill the ring buffer: the program writes ahead of the read pointer, and notices an underrun
fill_ring(ring, next_frame):                # called by an interrupt each period, or in a loop
    while free_space(ring) >= PERIOD:
        samples = generate_or_read(PERIOD)   # this period's samples
        write_to_ring(ring, samples)
    if device_read_pointer_passed_write_pointer(ring):    # underrun
        count_underrun(); write_silence_up_to(...)        # a click is unavoidable; at least not noise
```

The SID's way. The C64 did not store samples at all (3.4): the SID has three **voices**, each an oscillator with a waveform, an envelope and a filter, in hardware, programmed by registers: a program says "this frequency, this waveform, this attack and release" and the chip *synthesises* the sound. The sound is never in memory as numbers; there is nothing to underrun and nothing to mix in software, because the chip mixes its three voices and its filter inside itself. It is exactly the difference between the text mode of 3.4 (the chip draws from a description) and a framebuffer (the program draws every dot).

What a mixer does. When several programs each play sound, the card has one ring buffer and one DAC, so something must combine their sounds into the one stream: the **mixer**, "the audio compositor". Its job, per **period** (a short block of samples, a few milliseconds):

```
mix_period(streams, out):                   # one block of the output; runs once per period
    for each frame i in 0 .. PERIOD-1:
```

```
        sum = 0
        for s in streams:
            x = s.next_sample()             # after converting s to the output's rate and format
            sum += x * s.volume             # per-stream volume
        out[i] = clip(sum)                  # or a limiter: a gentle gain reduction instead of a hard cut
    write_to_ring(out)
```

It must also **convert each stream to a common sample rate and format**, because one program plays 44.1 kHz music, another 48 kHz speech, a third floating-point effects; this is **sample-rate conversion** (resampling: computing the samples of a wave at the new rate), a filtering job, and it is why a mixer is not just an adder. It **applies per-stream volume**, and **clips or limits** the sum, because two loud samples added exceed what the output can hold (**clipping**: a hard cut of the top of the wave, heard as distortion; a **limiter** reduces the gain smoothly to avoid it). There is a **latency against underrun** trade-off: a small buffer means the sound follows the program's action quickly (low latency) and leaves little margin for the program being late; a large one is safe and late. And there is **clock drift**: the program's idea of time and the DAC's clock are different crystals, so over minutes one runs ahead of the other; the remedy is for the mixer to be driven by the *device's* clock (it writes when the ring buffer has room, not when the program's timer says) and to resample slightly (a fraction of a percent) to absorb the difference.

Who the mixer is in real systems (general knowledge, *unverified* except where noted): on Linux a **sound server** (PulseAudio and, more recently, PipeWire) or ALSA's software mixing plug-in (`dmix`); on macOS CoreAudio; on Windows WASAPI's shared mode. On this machine there is a `pulseaudio` program and the ALSA library (`libasound`) installed; PipeWire is not. QEMU's own audio has a mixing engine (`audio/mixeng.c`, with `audio-mixeng-be.c`) and backends for ALSA, PulseAudio, PipeWire, CoreAudio, DirectSound, JACK and SDL (the file names are in `qemu-src/audio`): a virtual machine's sound is mixed by the **host**, as it is for any other program of the host.

What virtio-snd defines (checked in `/usr/include/linux/virtio_snd.h`). Four queues: control, event, transmit (`TX`) and receive (`RX`). Streams are **PCM streams** (pulse-code modulation: plain samples) with a direction (output or input), described by `virtio_snd_pcm_info`: a bitmap of formats (from 4-bit ADPCM and 8-bit mu-law through `S16`, `S24`, `S32`, `FLOAT` and `FLOAT64` to DSD and IEC958), a bitmap of rates (5,512 Hz to 384,000 Hz, including 44,100 and 48,000), and a range of channels. The driver sets a stream's parameters (`PCM_SET_PARAMS`: `buffer_bytes`, `period_bytes`, channels, format, rate), prepares, starts and stops it, then sends audio as buffers on the TX queue, each tagged with a `stream_id` and answered with a `virtio_snd_pcm_status` that carries a `status` and the **current device latency** in bytes. The event queue reports `PCM_PERIOD_ELAPSED` and `PCM_XRUN` (an underrun or overrun: the same word as the ring buffer's), and jack events; the feature bits include shared-memory modes, polling and period and xrun events. Controls (volume and the like) are optional (`VIRTIO_SND_F_CTLS`). What the header says about **who mixes**: nothing; it defines one stream at a time, the way the device presents *one card*, and the mixing of several programs happens on the host side of the device (in QEMU's engine above), or, in a machine with no host, in the pdp64's own kernel or library before the stream is sent. The pdp64 *is* that machine, so who mixes is the pdp64's open question.

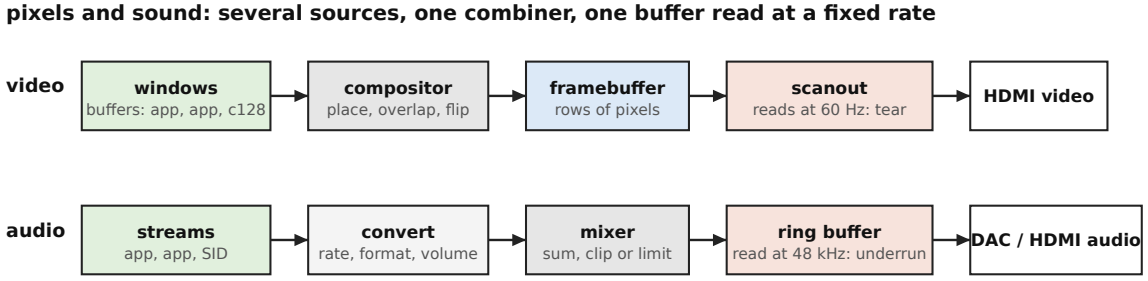
3.20.1 The standard audio device is virtio-snd (errata 79); the device carries one stream at a time as a PCM stream of the formats and rates the device lists. *(D)*

3.20.2 The pdp64's mixer (the program or device that combines the audio of several programs into one stream, converts rates and formats and applies volume), where it lives (the kernel, a library or the device) and who drives it, are unspecified. *(S)*

The audio of the c128 core, the options, parallel to the pixel options of 3.18, none decided. The author's stated preference for the pixel path (the c128 draws into a buffer, 3.18) has an audio form, and it is a natural pairing, but the author has stated no preference for audio.

- **Exclusive takeover of the audio path** (parallel to 3.18 (a)): when the core is enabled it owns the audio output and the host's sound stops. *Gives:* nothing to mix; the SID's output goes to the converter or the HDMI audio channel. *Costs:* no music while the terminal bells.
- **The SID's output as a virtio-snd stream** that the host mixes with the others (parallel to 3.18 (c)): the core renders its voices to samples (or the SID model does), and the samples go into a stream as any program's would. *Gives:* the core is a client of the mixer; the host's volume, rate conversion and routing apply. *Costs:* the SID must produce samples at a rate (a design choice: the 48-bit sample path of errata 74 is exactly that), and its latency is the stream's.

- **A mix inside the device with a fixed gain:** the core's voices and the host's stream are summed by a small mixer in the device, at a fixed or registered gain. *Gives:* nothing for the host to know; cheap (an adder and a clamp). *Costs:* a fixed policy; the device has its own clock and a rate converter, or both streams must already be at one rate.
- **Hand-over with a fade:** whichever is chosen, switching the core on or off ramps the volume over a few milliseconds and changes at a period boundary (3.18) so that no pop is heard.



A late frame tears; a late buffer underruns: a click. Both are cured by writing ahead of the reader and by letting the device clock, not the program clock, decide when the combiner runs.

The SID is the C64's exception: it makes the sound itself from three voices, so it has no buffer and no mixer in memory.

Stages after the model of general knowledge of sound servers (unverified, 3.22); virtio-snd streams as in virtio_snd.h.

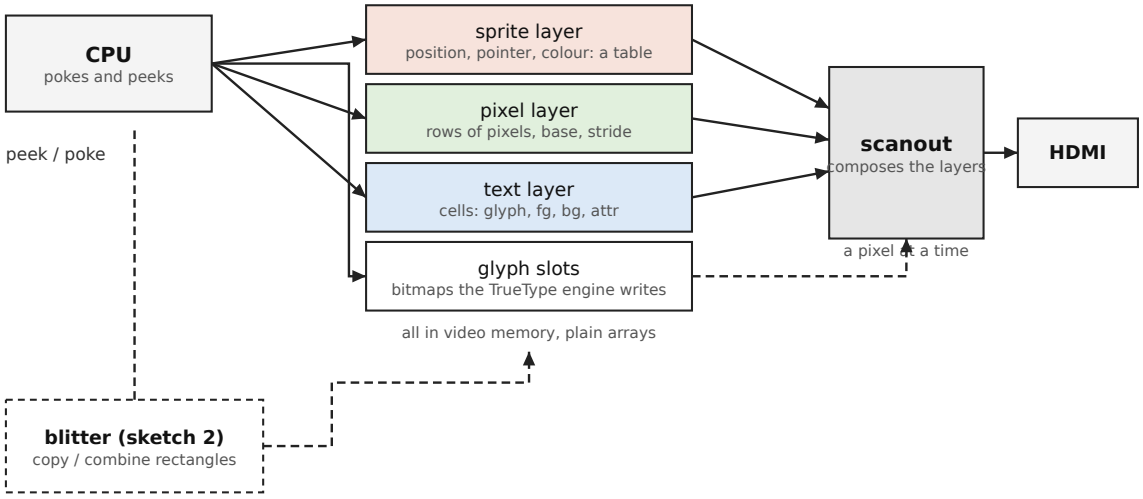
3.21 A design still being worked out: what a mashup of GPU, video memory, C64 and Blit could look like

The author, 2026-11-10: "conclude the graphics chapter: we're still figuring it out, what would a mashup of gpu, vram, c64 and blit look like." **Nothing in this section is decided.** Every statement is tagged *(P)* or *(S)*; the four sketches are the shapes of ideas, put side by side so that the questions at the end can be answered, and none is recommended by the repository.

Two terms first. VRAM (video memory) is memory that belongs to the display device: it sits on the device, behind a window on the bus, and the scanout reads it without crossing the bus; the processor reaches it through the window (slowly: every access is a bus transaction). The alternative is **system memory** that the device reads by DMA as a bus master, as the video device of 3.15.2 does; the framebuffer is then ordinary memory and the scanout competes with the processor for it. A GPU card has VRAM; the first board of the pdp64 has none to speak of (the block RAM is 5.5 megabits and a frame is 8.3 MB, 3.15.2), so on it every layer lives in system memory, and "VRAM" in the sketches below means "memory the device owns the use of", whichever kind it is. **Compositing** is combining several pictures into the one that is shown; the C64's sprites over text over a background were compositing in hardware, as the scanout read them out.

The properties the author has asked for, which each sketch is measured against: **(T)** simple peek/poke text and graphics at once; **(C)** modern colour depth and screen size; **(G)** glyph slots filled by a TrueType engine; **(S)** sprites, colour and sound in the C64's spirit; **(B)** the Blit's bit-blit and windows; **(U)** the GPU's command queue, parallel lanes and shaders. And the other two wishes of the day: anti-tearing sugar (3.8) and the boilerplate goal (3.9).

sketches 1 and 2: layers in video memory, composed by the display hardware as it reads them out



Sketch 1: the CPU writes the arrays; the display hardware does the composing; there is no command queue.
Sketch 2: the same, plus a few registers that move rectangles inside video memory (scrolling, window moves, glyph copies).
Nothing here is decided (3.21); the boxes are the shape of the idea.

Sketch 1: layered scanout, no command queue

3.21.1 Three **layers** (a text-cell layer, a pixel layer and a sprite layer) are plain arrays in video memory that the processor pokes; the display hardware composes them as it reads them out, one pixel at a time, and there is no command queue. *(P)*

What the programmer sees. Three arrays, each at a fixed place, each with a small set of registers (base, size, stride, enable). The text array is the cell array of 3.11, with its glyph slots in another array; the pixel array is a framebuffer; the sprite layer is a table of entries (x, y, shape, colour, enable). A program pokes whichever it wants, in any order. *What is in video memory:* the cells, the glyph slots, the pixels, the sprite table and the sprite shapes. *At scanout:* for each pixel the hardware asks the layers from the top down (sprites, then text, then pixels), looks the cell up to its glyph, takes the glyph's coverage and the colours, and gives the first layer that is not transparent; the sprite layer needs to know only a few entries per row. The composing is done in a **line buffer** (a small memory that holds one row), as 3.15.2 already provides for. *Costs:* a compositor of the order of the 1,500 lookup tables the repository estimates for BLT64, plus a line buffer per layer (an estimate of mine, unmeasured); scanout traffic is the pixel layer's 498 MB/s at 1920 by 1080, 4 bytes, 60 Hz (746 MB/s at 6 bytes), plus about 13 MB/s for the cells and the glyph reads (arithmetic from the examples of 3.11). *Gives up:* anything beyond what the layers can say: no scaling or rotation, no effect that is not in the compositor; every change a program wants to make to the pixel layer is the processor writing pixels. *Properties kept:* T yes (this is the C64 and the Blit's memory-mapped screen); C yes; G yes (the text layer's glyphs); S yes (sprites, and colour per cell; sound is a separate device, 3.1.3); B no (no blitter, windows are the program's job); U no. *Text and graphics split:* healed, the C64's way: a program has text and pixels at once in one frame, with no mode switch. *A game:* sprites and a scrolling text or pixel layer, moved with a few pokes a frame; the C64 game, with a modern screen. *A terminal emulator:* pokes cells (a scroll is a few stores, or, with a base register for the layer, one store); the glyph slots are filled by the TrueType engine. *Sugar:* per-layer flip and a hardware cursor for free (3.8); the fence is the vertical-blank counter. *With virtio across the board:* needs a virtio device type of its own (reading 1 of 3.11); the layers are a device's contents, not the virtio-gpu protocol.

Sketch 2: the same, plus a small blitter and a glyph atlas

3.21.2 Sketch 1 plus a **blitter**, a few registers that copy and combine rectangles inside video memory (the Blit's RasterOp, the Amiga's blitter, BLT64 of 3.10.4), and a glyph atlas that the TrueType engine fills and the text layer indexes. *(P)*

What the programmer sees. Sketch 1, and a handful of blitter registers: source, destination, strides, width, height, raster operation, go. The atlas is the glyph map seen as one picture (3.7): a glyph is drawn from it by the blitter into the pixel layer when a program wants a glyph on the pixel layer at an arbitrary position, and the text layer's cells point at it directly. *In video memory:* sketch 1's contents and the atlas. *At scanout:* as sketch 1. The blitter runs beside it, in the blanking time or contending for the memory. *Costs:* the blitter and its arbitration with scanout; the repository's own estimate is about

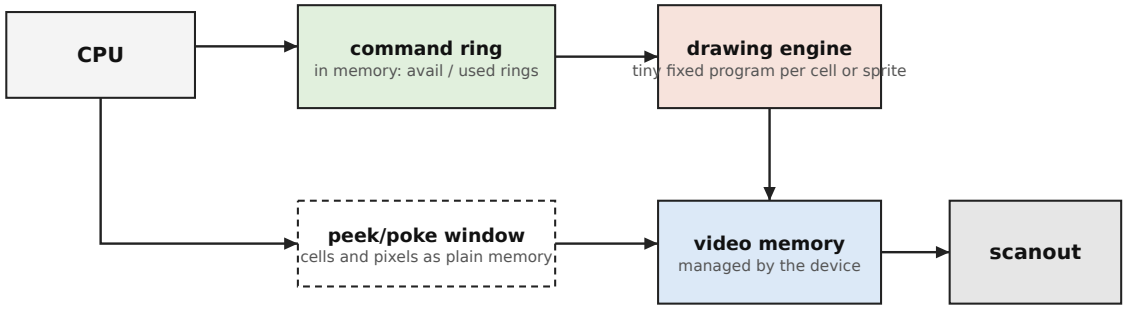
1,500 lookup tables and a line buffer, "nearly all address arithmetic" (3.10 commentary), and a bus-master port on video memory. *Gives up:* the same as sketch 1, except that drawing, scrolling and window moves in the pixel layer no longer cost the processor the pixels. *Properties kept:* T, C, G, S as sketch 1; **B yes** (bit-blit with the raster operations of 3.10.7, and windows as layers or as rectangles in the pixel layer); U no. *Split:* healed as sketch 1, and the pixel layer gains the Blit's operations, so that text on it (a window of a program that draws its own glyphs) is cheap. *A game:* sprites for the actors and the blitter for the background tiles. *A terminal emulator:* the text layer as in sketch 1; the blitter scrolls and moves windows drawn on the pixel layer. *Sugar:* as sketch 1, plus a blit that starts at the vertical blank (the "start-at-vblank" of `architecture-recommendations.md` R-9), which stops a copy from tearing. *With virtio:* as sketch 1, the blitter a feature of the device; it would be a feature bit and a queue if its commands are descriptors rather than registers (the chat chose a three-operand `draw` command, `decisions-pending.md` E3).

```
# sketch 2: scroll the pixel layer up by one text line, then draw a glyph from the atlas (placeholders)
poke(BLT_SRC, FB + LINE_H * STRIDE);  poke(BLT_DST, FB)
poke(BLT_STRIDE, STRIDE);  poke(BLT_W, W);  poke(BLT_H, H - LINE_H)
poke(BLT_OP, STORE);  poke(BLT_GO, 1);  wait_until(peek(BLT_BUSY) == 0)
```

Sketch 3: a command-queue device with a peek/poke window

3.21.3 A device whose main interface is a **ring of commands** in the manner of virtio-gpu, with a tiny shader-like fixed program for drawing glyphs and sprites and video memory managed by the device, and a peek/poke compatibility window on top for the programs that want to poke cells and pixels. *(P)*

sketch 3: commands in a ring, a small drawing engine, and a peek/poke window on top



The program mostly writes commands (put this text, move this sprite); the window is for the cases where a poke is simpler.
The window and the engine both reach video memory, so the device must say which one wins when they touch the same cell.

Nothing here is decided (3.21).

What the programmer sees. Mostly commands: "put this text at this cell", "move this sprite", "fill this rectangle", "present"; and, as a convenience, the window: cells and pixels as plain memory that mirrors the device's. The device allocates its own video memory, so a program does not choose addresses. *In video memory:* what the device decides; the program sees handles. *At scanout:* the device renders the commands into whatever it scans out, at its own pace, and flips at the blank by itself (the sugar is free: a commit is a command). *Costs:* a command processor (a queue engine, as the virtio devices of chapter 2 already are), a drawing engine of fixed programs (the small piece of the GPU that the terminal needs: a textured quad with a glyph and two colours, a sprite), a memory manager for video memory and a consistency rule between the window and the queue; more gates than sketches 1 and 2 by a clear margin (a judgement of mine, unmeasured), and more software in the driver. *Gives up:* the plainness of sketch 1; the program must know the device's model, and two ways to reach one cell must be reconciled. *Properties kept:* T only through the window (and only as well as the window is made); C yes; G yes (the engine reads glyph slots); S yes; B no blit, but "fill" and "copy" commands do its job; **U yes**, a command queue and parallel work, but a *fixed* program, not the general shaders of 3.6. *Split:* healed by the engine, which draws text and pixels from one queue. *A game:* a command per sprite or per tile, one "present" per frame, with little processor work. *A terminal emulator:* one command per changed run of cells; the glyph atlas as in 3.7, but made by the device. *Sugar:* the fence and atomic commit are the natural form (each command completes in order; a commit is a command); versioned buffers are the device's business.

With virtio: the natural fit, a device with queues as in 2.6, and the closest in spirit to virtio-gpu, of which it could be a profile (reading 2 of 3.11).

Sketch 4: virtio-gpu 2D, the sugar in a library

3.21.4 The standard virtio-gpu 2D route of 3.16, with the C64 and Blit conveniences done in software by a library (cells, glyphs, sprites and cursor, blits, flips), and no special hardware. *(P)*

What the programmer sees. A library that offers the peek/poke look: a cell array in memory, `put_char`, sprites as library objects, and a `present` call that does the transfer and the flush for the damaged rectangles. *In video memory:* the host's resources; on the pdp64 the program's memory is the backing (3.16.3). *At scanout:* the host reads its resource when it likes; the library cannot flip at a blank because virtio-gpu has none (3.16); tearing is the host's concern and, on an emulator, usually invisible. *Costs:* nothing in gates; the processor draws everything and a flush copies; bus traffic is the damage rectangles. The glyph slots are a library table, filled from the TrueType engine on the processor. *Gives up:* everything hardware: the hardware cursor is the cursor queue only, there are no sprites except the cursor, no blits, no layers; the C64's "the chip draws it while you do other things" is the library's loop. *Properties kept:* T only through the library (the program pokes memory the library owns, and the library presents it); C only to 32 bits (3.16.4); G yes, in software; S no (the cursor excepted); B in software; U no. *Split:* healed in the library's picture, but each screen is still one pixel framebuffer. *A game:* draws into a back buffer and flushes it, in the PC way of 3.5 (slow, but running now). *A terminal emulator:* the library draws glyphs from a cache and flushes the damaged rows. *Sugar:* damage rectangles and the fence only (3.8, 3.16). *With virtio:* it is the standard (errata 79, 95); sketches 1 to 3 would be added to it as another device or as a profile.

Sketch 5: a c64-mode core as the terminal (a variant)

3.21.5 A small core with the C64 family's chips, into which the host loads the terminal program by DMA (3.18), as a variant or addition to any of the sketches above: it is the terminal, and the other sketches say what the display behind it does. *(P)*

What the programmer sees. The host programmer sees a device that takes messages (the mailbox); the programmer of the small core sees the C64: peek, poke, a screen at a fixed place, colour RAM, glyph slots, sprites and the SID, which is the author's "as simple and pleasurable as the c64 was" at its most literal. *In video memory:* the core's screen memory, colour RAM, glyph slots, sprite shapes. *At scanout:* the VIC (or its extension) composes text, bitmap and sprites as sketch 1 does; the flip is a register of the core's own. *Costs:* a core, its memory, a DMA loader and the chips (3.18, question 7); a second program to write, load and keep secure. *Gives up:* a single, standard programming model on the host's side; the host cannot poke a cell, only send a message. *Properties kept:* T yes, for the core's programmer; C if the chips are widened as the native devices were (3.10); G yes; S yes, with sound; B only if the chip family has a blitter; U no. *Split:* healed, in the core. *A game:* a C64 game, or one written for the core. *A terminal emulator:* the core *is* the terminal; the host sends text. *Sugar:* the VIC's latches and raster interrupt, which the core's program uses; nothing is hidden from it. *With virtio:* the open tension of 3.18, question 6.

The four main sketches side by side (sketch 5 is a variant of any of them)

	S1 layers	S2 layers + blitter	S3 command queue + window	S4 virtio-gpu + library
What the program does	pokes arrays	pokes arrays, starts blits	writes commands (or pokes the window)	calls a library, which sends the commands of 3.16
Who composes	display hardware, at scanout	display hardware, at scanout	the device's engine	the host, or the program
Where glyphs live	slot array in video memory	atlas in video memory	the device's cache of slots	the library's table, in the program's memory
Sprites	hardware layer	hardware layer	engine primitive	no (cursor only)
Blitter	no	yes	fill/copy commands	in software
Commands or pokes	pokes	pokes and a few registers	commands, pokes as a window	commands under a library
Anti-tearing sugar	per-layer latch, cursor, raster	the same plus blit at blank	commit, fence, atomic	damage and fence only
Cost in gates (my estimate, unmeasured)	small: a compositor and line buffers	small to medium: add a blitter	larger: a queue engine, a drawing engine, a memory manager	none

	S1 layers	S2 layers + blitter	S3 command queue + window	S4 virtio-gpu + library
Bus traffic	scanout, plus the pokes	the same, less processor traffic for blits	scanout, plus commands (small)	scanout is the host's; the program writes every pixel
Fits virtio everywhere (errata 95)	needs its own device type	needs its own device type	a profile of virtio-gpu, or its own	is virtio-gpu
Properties T C G S B U	T C G S, no B U	T C G S B, no U	C G S U; T by the window; B by commands	T by library; C only 32-bit; G in software

3.21.6 The five sketches (the four layers-and-queues shapes, and the c64-mode core as a variant) are the author's open design question, and none is decided; a combination (for example, sketch 4 now and sketch 2 or 3 as a second device later) is as open as a choice among them. *(S)*

A leaning, labelled as mine and not a recommendation of the repository. Sketch 4 is what the first iteration already is, and the question the author has put is what the second iteration should be. Of the others, sketch 2 matches the author's words most closely (the Blit's bit-blit and the C64's text, graphics and sprites at once, no more machinery than the problem needs) and is the smallest hardware that makes peek and poke the main interface; sketch 3 is the better fit if the author wants one family with virtio-gpu and the command queue as the main interface; sketch 1 is sketch 2 without the blitter, and the blitter can be added later because it only reads and writes video memory. This leaning assumes the author wants the poke to be the main interface; if he wants commands, sketch 3 is nearer.

The questions the author must answer to choose (each is repeated in 3.23): (1) **Who composes the layers**: the display hardware at scanout (1, 2), the device's engine (3), or software (4)? (2) **Where glyphs live**: a slot array the text layer indexes (1, 2), the device's own cache (3), or a library table (4); and does a cell hold a code or a slot number (3.9, pair 3)? (3) **Do sprites exist**, as a hardware layer, as an engine primitive, or only as the cursor? (4) **Is there a blitter**, and if so as registers or as commands? (5) **Are commands or pokes the main interface**, and what does the other do? (6) **Which virtio shape**: a device type of its own, a profile of virtio-gpu, or the standard device with a library, and so what the "virtio-console" of the author's words is (3.11)? (7) **VRAM or system memory** on the first board and later? (8) **What pixel format**: 48-bit native, with the standard device at 32 bits (3.16.4)?

3.22 What was checked, and what was not

The rule of the project is: check, do not guess. For this chapter:

- **Checked against the Linux headers on this machine:** `virtio_gpu.h` (the command names and structures, the formats, the fence flag, the cursor commands, the maximum scanouts, the feature bits; the absence of any vertical-blank or flip command), `virtio_console.h` and `virtio_ids.h` (the console device's type 3, its configuration and features), `drm.h` and `drm_mode.h` (the existence of the page-flip, atomic, add-framebuffer ioctls and their flags, and the fields of a mode), and `GL/glext.h` (the OpenGL entry points named).
- **Checked against the QEMU sources** (`/home/david/claude/qemu-src/hw/display/virtio-gpu.c`): what `transfer_to_host_2d` and `resource_flush` do and reject; the fence response; the `refresh_rate` field.
- **Checked against 2.11BSD** (`/home/david/claude/facetlang/unixtree/2.11BSD/sys/pdp/cons.h` and `cons.c`): the KL11/DL11 register structure, its address 0177560 and the `DLXCSR_TRDY` bit.
- **Checked against the repository's own records:** the Blit's statements and numbers (the Blit and Tradeoffs papers, as recorded in `docs/kernel-pain-points.md` section 14I); the VT52 `ESC Y` code with its offset of 32 and the "VT100/ANSI" and "xterm-256color" remarks (the author's chats in `c64stuff.txt`, as the assistant of the chat stated them); the author's own terminal chat (`docs/c64-devices-from-chat.md` section 6); the vertical-blank rules (3.10.9, errata 71).
- **Unverified** (general knowledge; no manual of the machine is on this computer): everything about the Commodore 64 chips (addresses, sizes, sprite size and count, colours, the SID's voices and waveforms, the processor's identity) and about the IBM PC adapters (VGA text addresses and attribute layout, mode 13h and the planar modes, the 64 KB window, VBE and its linear framebuffer, the palette); the details of the VT52 and VT100 sequences beyond the records; **what the author's "vt55" refers to** (the repository names the VT52 and VT100 only); the speed of a teletype; which terminal emulators use the GPU; Valve's signed-distance-field paper; the titles, dates and page counts of Michael Abrash's books (the author's "800 page" is his figure), Mode X and the planar-mode details; the group sizes of GPU lanes; variable refresh in its several names; the Vulkan names (no Vulkan header is on this machine); EDID as a data

block; the typical line counts of boilerplate (illustrative throughout and not measured); all gate-count estimates of the sketches (mine).

- **Decided by the author and recorded in `errata.md`:** everything tagged *(D)*; the author's words of 2026-11-10 are in errata 105 and are tagged *(P)* or *(S)*.

3.23 Still open

The chapter's open items, each repeated from the sections above: (1) **what "our virtio-console device" is**: a new virtio device type, virtio-gpu extended, the real virtio-console beside a display device, or the native family (3.11); (2) **the four sketches of 3.21**, and the questions of who composes the layers, where glyphs live, whether sprites exist, whether there is a blitter, whether commands or pokes are the main interface; (3) **whether a cell holds a character code or a slot number**, the glyph slot's layout and the number of slots (3.9 pair 3, 3.11.3); (4) the **anti-tearing sugar** (3.8): which of the candidates, **and whether tear-free flipping is worth hardware if the display side is variable-refresh** (3.8, "Is this still a problem?"), and whether a fence, per-layer latching, variable refresh, versioned buffers are specified; (5) the **boilerplate sugar** (3.9): the default mode, the canonical pixel format, stride and mapping, the colour space and gamma, the units of cells and fonts for HiDPI, a palette or none; (6) the **48-bit pixel against virtio-gpu's 32-bit formats** (3.16.4); (7) a **write-combining** memory type for the framebuffer (3.15); (8) the register-level definitions of `VIC-64X`, `SID-64X` and `BLT64` (3.13, still deferred); (9) the feature subset of virtio-gpu and virtio-snd (3.16.2), and whether OpenBSD has a virtio-snd driver (errata 79); (10) the block-RAM budget and the underrun rule of the line buffer (3.15.3); (11) the hot-plug scope (decisions-pending F23); (12) **the c64 mode** (3.18): which core, unmodified C64 software or terminal firmware only, how video memory is shared, how the core and the host talk, how the loaded image is authenticated and contained, how it relates to the native devices (errata 78) and to virtio across the board (errata 95), the cost, the sound path; (13) **the compositor path of the c64 mode**: exclusive takeover, a second virtio-gpu head and sound stream, a surface for the host compositor, layers at scanout, or a new virtio device type (there is none today); and the hand-over without tear or pop; (14) **whether tear-free flipping is worth hardware if the display side is variable-refresh** (3.8); (15) **the window model** (3.19): which of a plain buffer and a rectangle, drawing as writes, a composing server, downloadable server-side code, no toolkit, windows as files or as virtio queues, and whether the c128 core is the display server; (16) **the audio path** (3.20): where the mixer lives and who drives it, the form of the core's audio (takeover, a virtio-snd stream the host mixes, a fixed-gain mix in the device), the sample rate and format of the SID's output, the fade at hand-over; (17) **the compositor details under the author's preferred path (c)** (3.18.4).

Chapter 4 — Boot, the BIOS, the disk label and the GPT

The machine starts the way the PDP-11 did: a small ROM in the controller chip loads one block from the boot device and jumps to it, and every later stage is an ordinary program, open source, compiled by the project's own compiler. This chapter is written for a reader who has keyed a bootstrap in on a PDP-11 front panel, used a boot ROM, or typed `boot` at the : prompt of Unix Version 7 or 2.11BSD, or who is learning computing from the start, and who has not met anything invented after the 1970s: no BIOS, no UEFI, no partition table, no device tree, no start-up of a second processor, no firmware update. It begins with what the PDP-11 did (the switches, the boot ROM, block 0, the two-stage bootstrap, `ODT`), explains each later idea by contrast with it (the PC's BIOS and boot sector, UEFI, the GPT and the BSD disk label, the hand-off of a description of the machine, starting other cores, starting devices, recovery), walks each stage through in pseudocode, and gives the decisions as numbered statements along the way. The full review of how other systems boot is `../boot.md`. Tags: *(D)* decided; *(P)* proposed; *(S)* unspecified; a fact about an older or another machine that could not be checked on this computer is marked *unverified* and listed in section 4.12.

How to read this chapter. A term is defined where it first appears, as the technical word followed by a plain restatement. Boxes-and-arrows pictures are the kind a textbook draws for a linked list: a box is a piece of memory, a register or a program, an arrow says what is handed on. Code blocks are pseudocode (C-like, `#` starts a comment) unless the text says they are PDP-11 assembly; the PDP-11 code is quoted from the V7M and 2.11BSD sources on this machine (section 4.12), and every pdp64 sketch is **an illustration written for this chapter**: no pdp64 boot ROM, first stage or second stage exists (the QEMU machine loads the kernel directly, 6.12.2), so none of it is tested. The decisions are the numbered statements; the rest is explanation, and where the repository does not say, the text says *not specified*.

4.1 What you already know: starting a PDP-11

A processor that has just been powered on has a problem that no program can solve for it: the program that loads programs has to come from somewhere, and the memory holds nothing it can trust. This is the **bootstrap problem** (the name is the old joke of pulling yourself up by your own bootstraps, which is how it is shortened to "boot"). A PDP-11 solved it in three ways, each smaller than the one before it was.

The front panel. The console of a PDP-11 had rows of switches. You set the address switches to a memory address and pressed LOAD ADDRESS, set the data switches to a word and pressed DEPOSIT (which stores the word and steps the address), repeated that for each word of a short **bootstrap loader** (a program of a few dozen words that reads a bigger program from a device), set the start address and pressed START. This is how machines were brought up, and the manual of the V7M system still says so: its `boot(8)` page tells you to "load block 0 of the boot device by fiddling with the console keys as appropriate for your hardware. If you have no appropriate ROM, some programs suitable for manual use are given below" (section 4.12; the switch mechanics themselves are general knowledge, unverified).

The boot ROM. Later machines kept the loader in a **ROM** (read-only memory: storage that holds its contents without power and is not written during normal operation) on the processor board, one small program for each kind of disk or tape. The V7M sources name the card, the M9312, in a comment on the tape bootstrap (`sas/htboot.s`: "The M9312, which is the only boot for the TS11"); a console switch or a command chose the device and the ROM did the rest. What the ROM did is the whole subject of this chapter in one sentence: it read **block 0** (the first 512-byte **block**, the unit a disk is read in; the first block of the disk is numbered 0) of the boot device into low memory and jumped to it.

Block 0, the primary bootstrap. What lived in block 0 was a **primary bootstrap**: a program small enough to fit in one block. The V7M page says it in one paragraph: "A PDP11 UNIX system is started by a two-stage process. The first is a primary bootstrap which is able to read in relatively small stand-alone programs; the second (called *boot*) is used to read in the system itself. The primary bootstrap must reside in the otherwise unused block zero of the boot device. It can be read in and started by the standard ROM programs, or if necessary by keying in a small startup routine." Here is the head of the

primary bootstrap for the RM02/3 and RP04/5/6 disks, `sys/mdec/hpuboot.s` (285 lines of assembler, measured), which shows what a block-0 program has to do for itself:

```
/ UNIX/v7m RM02/3 and RP04/5/6 block 0 bootstrap
/ Can only boot from unit zero.
/ Assumes disk at standard address (0176700).

core = 28.
.. = [core*2048.]-512.          / 28 K words = 56 KB, less 512 bytes: high in memory

start:
        mov     $..,sp          / the stack is just below the program's new home
        clr     r0
        mov     sp,r1
        cmp     pc,r1           / are we running below the intended origin?
        bhis    2f
1:      mov     (r0)+,(r1)+     / yes: copy this program up, out of the way
        cmp     r1,$end
        blo     1b
        jmp     (sp)
2:      clr     (r0)+           / clear core to make things clean
        cmp     r0,sp
        blo     2b
        mov     $drive,*$hpcs2  / initialize the disk hardware and select the drive type
        ...
        mov     $'\r,r0         / print CRLF and '#' as a prompt
        ...                     / read a pathname from the console, then walk the
                                / inodes from the root (inode 2) to find the file
        jsr     pc,*$0          / the loaded program is entered by a call
```

The ROM put this program at address 0; it copies itself to the top of the first 56 KB so that the kernel can be loaded at 0, clears the rest of memory, starts the disk, prints `#`, reads the name of a file, finds the file by reading the disk's **inodes** (the file system's table entries, one for each file) from the root directory down, loads it, and enters it. The file it loads is the second stage.

The second stage, `boot`. `boot` is an ordinary program, written in C (`sys/sas/boot.c`, 272 lines, measured). The V7M page: "When read into location 0 and executed, *boot* sets up memory management, relocates itself into high memory, and types `Boot` and `:` on the console. Then it reads from the console a device specification followed immediately by a pathname", for example `hp(0,0)unix`: the device, the unit, and the block offset of the file system on it. "*Boot* finds the corresponding file on the given device, loads that file into memory location zero, sets up memory management as required, and calls the program by executing a `trap` instruction." So there were two prompts, `#` from the first stage (type `boot`) and `:` from the second (type `hp(0,0)unix`), and two reasons for the split: block 0 has room for 512 bytes of program, which can find a file by name but cannot also keep a menu, a device table and an error message for every disk; and a bigger program with all of that can be an ordinary compiled file that you can change by copying a new one in.

2.11BSD, the same kernel grown up, is explicit about the limit and about what the stages pass on. Its `boot(8)` page: "a primary bootstrap (limited to 512 bytes) ... must reside in block zero of the boot device (the disklabel resides in block one) ... `disklabel(8)` is normally used to place a copy of the appropriate bootstrap in block zero of new file systems"; copies are kept in `/mdec`. The header of its `hpuboot.s` says "assembled size must be <= 512; if > 494, the 16-byte a.out header must be removed" and that all the options (an alternate name, echoing) "had to be removed to make room". The next stage passes the machine on **in registers**: the comment in its start-up code `pdpstand/srt0.s` reads "the bootstrap passes the cputype through in r0", `r1` holds the controller's register address, `r3` the boot device's major and minor number, and `r2` is a check word that is the complement of the boot options, so that the program can tell an unattended reboot from a person typing. That is the PDP-11 form of what the pdp64 does with `r0`, `r1` and the boot-information block (4.4, 4.6).

`ODT` and the console. When nothing booted, you were not stranded: the processor or the ROM carried a tiny monitor, **ODT** (Octal Debugging Technique, a console program that lets you type an address, see the word there, change it, and start a program at an address: *examine*, *deposit*, *start*). A PDP-11 with a dead disk could still be inspected and a bootstrap keyed in. (`ODT` itself is not in the sources on this machine: it is general knowledge, unverified.)

What a PDP-11 did not have, and what the rest of the chapter explains, each by contrast:

- **No table of partitions on the disk.** V7 compiled the sizes of a disk's regions into the driver (`hp_sizes[8]` in `hp.c`: eight pairs of a size and a first cylinder, with comments like "cyl 44 thru 429"), so changing the layout meant recompiling the kernel (4.5).
- **No description of the machine handed to the kernel.** The kernel was built for the machine, with the devices and their addresses compiled in (the vector table of chapter 6 is that table), and what the bootstrap passed was three or four registers (4.6).
- **No other processors to start,** and no cores held in reset (4.7).
- **No device that needed a program of its own** before it would work, and no way to give it one (4.8).
- **No firmware, no BIOS and no UEFI,** so no question of what to do about them (4.2, 4.3, 4.9).
- **A monitor, but no watchdog:** nothing restarted a hung machine (4.10).

What does not change in the pdp64: a ROM that reads block 0 and jumps; a first stage that fits in a block; a second stage that is an ordinary program and finds the kernel by name; registers that carry the boot device across; a console monitor when nothing boots. What is new is six things, each explained where it first matters: partitions and the GPT (4.5), the boot-information block (4.6), starting other cores (4.7), starting devices (4.8), recovery by watchdog (4.10), and the decision not to build the PC's firmware (4.9).

4.2 What the PC added: the BIOS, the boot sector and UEFI

Everything in this section is the history of the machine that the pdp64's designers did not copy, so it is told briefly and from the PDP-11's point of view. The facts here come from the OpenBSD sources on this machine where they can (section 4.12), and are otherwise general knowledge, marked unverified.

The BIOS. The IBM PC of 1981 (unverified) kept a ROM called the **BIOS** (Basic Input/Output System). It did four different jobs, and it is worth pulling them apart because the name has stuck to all four: (1) **bring-up and self-test**, initialising the chipset, sizing the memory and checking the basics (the power-on self-test); (2) **the first code**: find a boot device, read its first sector (the PC's word for a block, 512 bytes) to a fixed address and jump to it; (3) **a library of services** for software to call, by **software interrupt** (an instruction that behaves like a trap through a vector: the PDP-11's `emt` or `sys` is the same kind of thing), one for the disk, one for the screen, one for the keyboard, one for the memory map; (4) **configuration**, a setup screen and a small battery-backed memory for the clock and the settings. The third job is why a 512-byte PC boot sector can read the rest of a disk: it does not carry a disk driver, it calls the BIOS's. OpenBSD's PC first stage does exactly that: `biosboot.S` calls `INT 0x13`, the disk service, to get the drive's geometry and to read it (read in source, section 4.12). The PDP-11's bootstrap carried its own disk driver (the `hpcs1` and `hpcs2` writes above); the PC's borrowed the firmware's.

The boot sector and its 446 bytes. On the PC the first sector of the disk is the **MBR** (master boot record). It is one block with three parts: the first 446 bytes are the first-stage program, then a **partition table** of four entries of 16 bytes each (a **partition** is a named region of the disk, which lets several operating systems share one disk), then the two-byte signature `0xAA55` that tells the BIOS the block is a boot block. OpenBSD's `sys/disklabel.h` says so in C: `dmbr_boot[446]`, four `dos_partition` entries, `dmbr_sign`, the signature at byte `0x1fe`, and the entry holds a flag byte (`0x80`, "active"), a type byte, and a start and a size of 32 bits each (read in source). The 446 bytes are shared with a partition table because the PC wanted both in one sector, and 446 is what is left. The 32-bit size is why this scheme tops out at 2 TiB for 512-byte sectors (4,294,967,296 sectors of 512 bytes, arithmetic), and four entries is why there are "extended" partitions with a chain of further tables. The chain on an OpenBSD PC is: BIOS, then the MBR's 446 bytes (`mbr.s`, which "finds the first active partition"), then the **partition boot record** (the first block of that partition, `biosboot`: "roughly 512 bytes of code"), then `/boot`, then the kernel: **four stages** where the PDP-11 had two (read in source).

Real mode. The BIOS and the first stages run the processor in **real mode**, the 16-bit mode of the 8086 of 1978 that can address one megabyte, because that is the only mode the processor is in when it starts. Getting out of it into the processor's modern modes is the job of the loader, and is one of the things the pdp64 does not have to do (4.7).

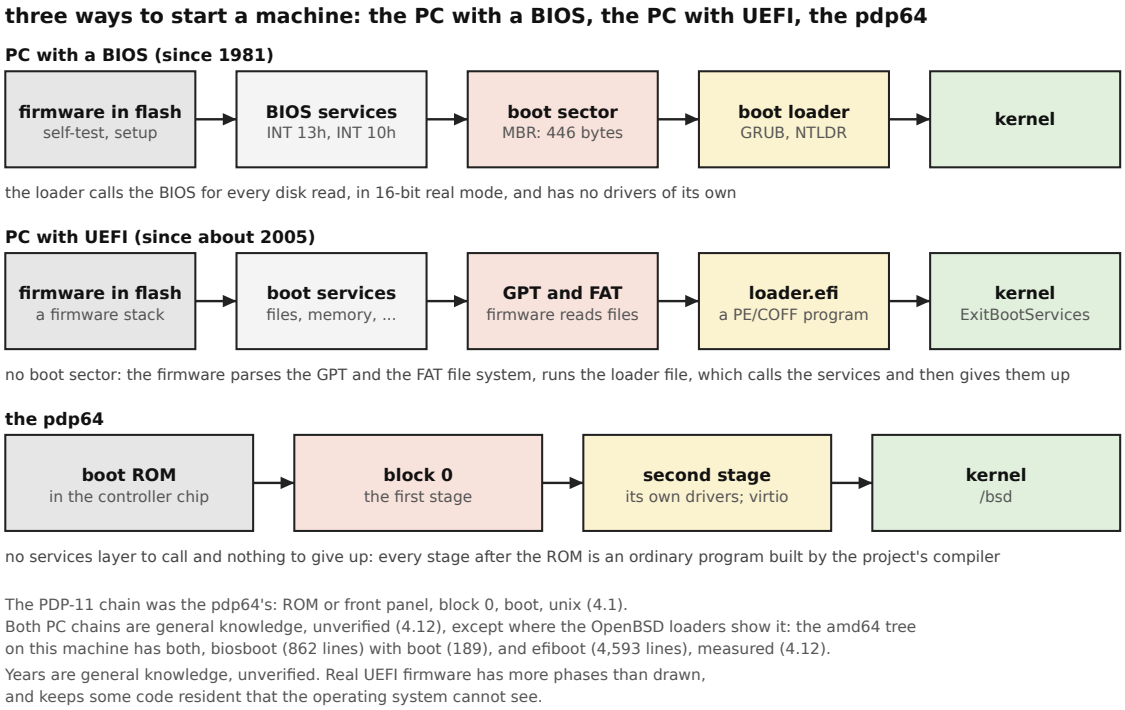
The GPT. The four 32-bit entries ran out, so a new table arrived with UEFI: the **GPT** (GUID partition table, where a **GUID** is a 128-bit identifier, long enough that two made anywhere will not collide). It has a header in the second block, "EFI PART" in its first eight bytes, then an array of entries (128 of them by default, each 128 bytes: a type GUID, a unique GUID, a 64-bit first and last block, attribute bits and a 36-character name), a checksum (a **CRC32**, a 32-bit check value computed over the bytes) over the header and another over the array, and a **backup copy** at the end of the disk, so that

a damaged table can be rebuilt. All values are little-endian (low byte first, the PDP-11's and the pdp64's byte order). All of this is read from OpenBSD's `gpt_header` and `gpt_partition` structures (read in source; the UEFI specification itself was not opened). Block 0 of a GPT disk is a **protective MBR**: an MBR whose table holds one entry of type `0xEE`, "EFI Protective Partition", covering the whole disk, so that an old tool that reads only the MBR sees a disk that is full and does not offer to partition it.

UEFI. The BIOS's successor is a firmware standard, **UEFI** (Unified Extensible Firmware Interface). It does not read a boot sector. The firmware itself understands the GPT and a **FAT** file system (the file system of MS-DOS, kept as the common denominator) on a partition of type "EFI system partition" (`GPT_UUID_EFI_SYSTEM` in OpenBSD's header), finds a **loader** file there, and runs it. The loader is a **PE/COFF** program: the executable file format of Windows (portable executable, from the Common Object File Format), which UEFI requires of its programs, so that a loader is a different kind of file from the Unix `a.out` or ELF files of chapter 1 (OpenBSD's arm64 loader begins "We need to be a PE32+ file for EFI", read in source). The loader receives a table of **boot services** (function pointers: allocate memory, read the memory map, read a file, write to the console, draw on the screen) and a few **runtime services** (the clock, reset, variables, which stay available after boot) and, when it is ready to take over the machine, calls **exit boot services**, which tells the firmware to stop using the memory it kept for itself. OpenBSD's own UEFI loaders call about fifteen of these (counted in 4.9). The loader also receives **configuration tables**, which describe the machine to the kernel: **ACPI** (a standard of tables, with a byte-code interpreter the kernel must carry) or a **device tree** (a text-sourced tree of nodes that names each device and its addresses, flattened into a blob), both of which OpenBSD's loaders look for (`EFI_ACPI_TABLE_GUID` and `FDT_TABLE_GUID`, read in source). Beneath all this the firmware is a large program with phases of its own; the QEMU tree on this machine carries ready-built firmware images of the open-source implementation EDK II (`pc-bios/edk2-*.fd.bz2`), of which the x86-64 code image is 3,653,632 bytes uncompressed (measured; the riscv and aarch64 images are padded flash images of 32 and 64 MiB, which says nothing about the code in them).

The six jobs side by side, from the PDP-11 to the pdp64:

Job	PDP-11	PC with a BIOS	PC with UEFI	pdp64
Bring-up	the processor and its ROM	the BIOS	the firmware	the controller's sequencer; on the FPGA the vendor's memory block
First code	ROM or front panel reads block 0	BIOS reads the MBR	firmware runs a loader file	the boot ROM reads block 0
Services for the loader	none: the bootstrap has its own driver	software interrupts	boot services	none: the loader has its own drivers
Finding the kernel	a name typed at :	partition boot record, `/boot`	the loader on a FAT partition	the second stage reads the label and finds `/bsd`
Telling the kernel about the machine	registers	registers and the BIOS's memory map (unverified)	the memory map and ACPI or a device tree	the boot-information block
Starting other processors	none	INIT and START-UP signals	the same	a write to a start register



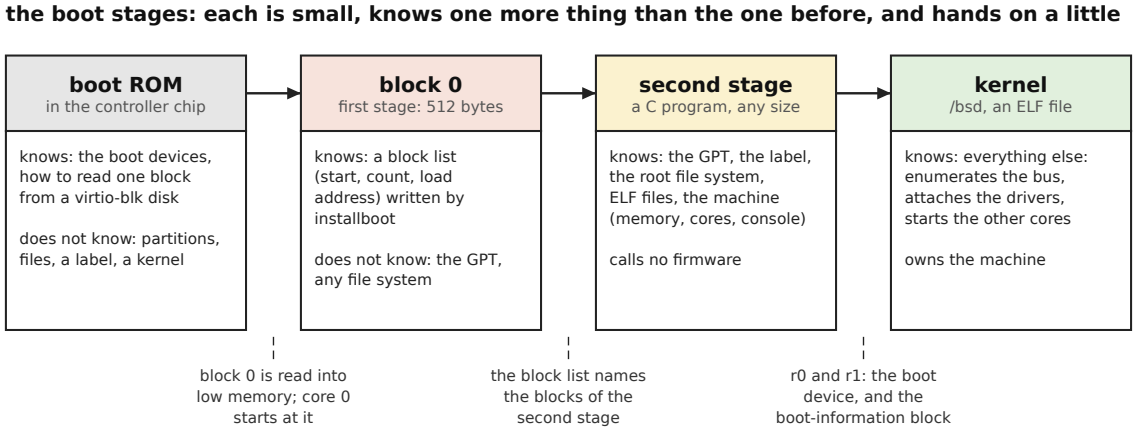
4.3 What takes the place of a BIOS

4.3.1 The pdp64's "BIOS" is the controller chip's boot ROM and its monitor. It brings up the machine, loads the first block, and offers a console monitor when nothing boots. It has no resident services and nothing is called after boot. *(P)*

4.3.2 The boot software is open source and part of the specification: the ROM and monitor, the first-block program, the second-stage loader and the boot tools. *(D)*

4.3.3 Everything in the project, the boot software included, is under the BSD 2-clause licence. *(D)*

Why there are stages. The chain is a ROM, a first block, a second stage and a kernel, and each exists because the one before it is too small to do the next one's job. The ROM sits in the controller chip, so every byte of it costs silicon: it knows how to read one block from the boot devices and nothing else. The first block is one block, 512 bytes, and under a GPT only the 446 of the protective MBR's code area are its own (4.5): it can hold a short list of where the next program lies on the disk and the few instructions that read it. Only the second stage is big enough to understand a file system, the partition table and the label, because it is an ordinary program in C, loaded from the disk, of whatever size it needs. The kernel is then loaded by a program that can find it by name and fill in what it needs to be told. The PDP-11 had the same chain with the partition table missing (the # and : prompts of 4.1). Each stage in the figure knows one more thing than the one before, and hands on a little:



Stage sizes are the reason for the stages: the ROM and the first block are the smallest code that can be made to find the next, and only the second stage is big enough to read a file system. The ROM holds the monitor too (4.10), reached when no block 0 loads.

The kernel is OpenBSD (errata 41 and 47). Where block 0 lands (0x1000 in 4.4) is an address still to be chosen (4.13).

Why open source, and why it is in the specification. The author's words (errata 47, 2026-10-22): "we'll have an open source boot sector boot loader as part of the spec; that is part of the spec that the boot loader/bios is open source." On a PC the firmware that ships (a BIOS, or a UEFI implementation: the UEFI specification itself is published, 4.9) is typically a vendor's build, whose source the owner does not have (unverified), at the bottom of the chain, running before anything the owner can inspect, and kept resident afterwards (4.2). In the pdp64 every stage after the controller's own sequencer is a published source file built by the project's own compiler, and the ROM and monitor of the controller are in the same position, so there is no closed program at the root of the chain of trust. The licence is the BSD 2-clause licence of the whole project (errata 48: "bsd license for everything").

Commentary. **Precedent.** A BIOS did four jobs: hardware bring-up and self-test, the first code (read the boot sector and jump), a runtime library of device services called by software interrupt (the "I/O" in its name), and configuration. UEFI replaced the second and third with a loader file and a set of boot and runtime services, and the other architectures have the same roles under other names: Open Firmware on SPARC and PowerPC, U-Boot with trusted firmware on ARM, OpenSBI on RISC-V, the SRM console on Alpha, the console processor on the VAX, and the boot ROMs and `ODT` monitor on the PDP-11. **Trade-offs.** The pdp64 needs only the first, second and fourth jobs in a small form. The third disappears because every device speaks virtio through one transport (chapter 2), so the loader and the kernel share one driver and the loader needs no firmware to read a disk. The cost is that the second-stage loader carries its own driver for the boot disk, which is small because a virtio device is a handful of structures. **Improvements.** There is no firmware API to specify, no compatibility layer, no resident management code and no closed blob at the bottom of the trust chain. **Benefits.** The failure cases reduce to one: the first block did not load, and the monitor is the recovery. **Modern comparables.** x86 firmware keeps resident system-management code that the operating system cannot see; arm64 and RISC-V servers need trusted firmware and, usually, UEFI beside it. **Impact.** *Kernel:* There are no firmware services to call or to bug-fix. *Compiler:* The ROM, loader and tools are built by the project's own compiler. *Hardware:* A boot ROM of a few hundred bytes to a few kilobytes, with a driver per boot device.

4.4 The reset state and the first block

What happens at power-on. The **controller chip** (the chip that holds the boot ROM, `boot.md`) brings up the memory and holds every core in **reset** (a state in which a core executes nothing and keeps no state). The ROM runs on core 0. On the PDP-11 the equivalent was the processor's own power-up sequence and the ROM or the front panel; here it is a small program in the controller, and it does five things, in this order: find a boot device, read block 0 from it into low memory, check that it looks like a boot block, set core 0's registers, and start core 0 at the block. If it cannot find a device or the block is not a boot block, it runs the monitor (4.10). In pseudocode, the ROM is this:

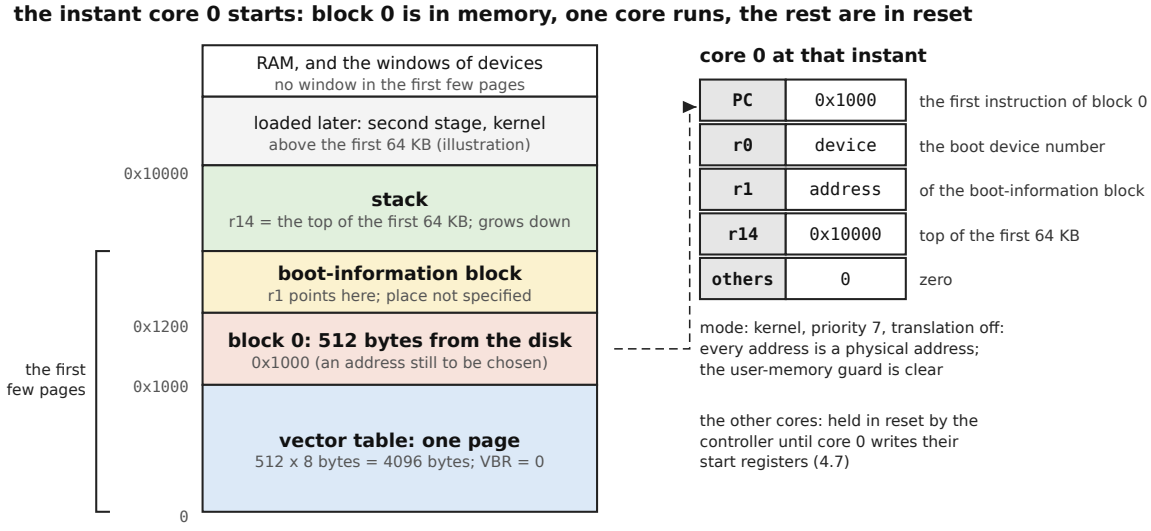
```
rom_entry():                                # core 0; every other core is held in reset
    quick_memory_check()                    # the monitor's short test of the first pages
```

```
    for dev in boot_order:                    # disk or flash, then SD card, then the serial line (4.4.1)
        if not bring_up(dev): continue        # enumerate the bus, find the function, handshake (4.8)
        if not read_block(dev, 0, LOAD):      # LOAD = 0x1000 in this chapter's example (4.4.2)
            continue
        if byte[LOAD + 510] != 0x55 or byte[LOAD + 511] != 0xAA:
            continue                          # not a boot block: try the next device
        info = fill_boot_info(dev)            # memory ranges, cores, root complex, device, console (4.6)
        core0.mode = KERNEL; core0.priority = 7; core0.translation = off; core0.VBR = 0
        core0.guard = clear                   # the user-memory guard
        core0.r0 = number_of(dev)             # which device it came from
        core0.r1 = info                       # the boot-information block
        core0.r14 = 0x10000                   # the top of the first 64 KB of memory
        core0.all_other_registers = 0
        core0.pc = LOAD                       # the first instruction of block 0
        return                                # core 0 now runs the block; the ROM is done
    monitor()                                 # nothing booted: the console monitor (4.10)

read_block(dev, n, address):                  # the standard disk: one virtio-blk request (5.5.1)
    build a three-descriptor chain: request header (type = read, sector = n),
                                    a 512-byte buffer at `address`, a one-byte status
    FENCE; write the queue's doorbell; poll the used ring until the chain returns
    return status == OK
```

The ROM polls (checks in a loop) and does not use interrupts, because no interrupt controller is set up yet. It needs a driver for each kind of boot device, which for the standard disk is the virtio-blk reader of chapter 5: find the function in the bus's configuration space, set up one queue and read one block. Which device comes first, and how the ROM names a device in `r0`, are not specified (the order of decision F10 of `decisions-pending.md` is open after the disk).

The first few pages of memory. At the moment core 0 starts, memory looks like this. The **vector table** (the table of handlers that an exception or an interrupt jumps through, chapter 6) is **one whole page**, 512 entries of 8 bytes, at address 0, because the system register `VBR` (vector base register) is zero at reset; the PDP-11's was a fixed table of a few hundred bytes at 0 to 0377, and its boot programs lived elsewhere. The author's own summary of the PDP-11 way (2026-10-22) was "load 512 byte block 0 then jmp 0", which he called "not precisely true, but directionally true"; here the "jmp 0" becomes a jump to the address block 0 was loaded to, because the first page now holds the vectors. Block 0 is loaded **above** the table, at 0x1000 in this chapter's example (the address is "to be chosen"; `docs/boot.md`), so that the first stage can run without overwriting the vectors. The stack pointer `r14` starts at the top of the first 64 KB of memory, so the first stage has a stack before it has written one instruction. `r1` points at the **boot-information block**, the description of the machine (4.6). The author's rule of errata 102 is that no device window is mapped in **the first few pages**, where "the boot stuff is loaded": how many pages that is, is not decided.



0x1000 and the 64 KB stack top follow the proposal of 4.4 and the figure of chapter 6; how many pages are reserved is not decided (errata 102).
Not to scale. The ROM itself is not in this memory: it sits in the controller chip, outside the address space the kernel uses.
On the PDP-11 the vectors sat at 0 to 0377 and the bootstrap was loaded elsewhere, often high in memory (4.1).

4.4.1 At reset the controller brings up memory, holds every core in reset, and looks for a boot device in a fixed order: disk or flash, then SD card, then the serial line. *(P)*

4.4.2 It reads block 0 of the first device that answers (for the standard disk, a virtio-blk read request) into low memory, checks the `0x55AA` signature in the last two bytes, and starts core 0 at that address. It does not parse a partition table. *(P)*

4.4.3 Core 0 starts in kernel mode at priority 7 with translation off (every address physical), `VBR` zero and the user-memory guard clear; `r0` holds the boot device number, `r1` the address of the boot-information block, `r14` the top of the first 64 KB of memory, and the other registers are zero. *(P)* The first few pages of the physical address space are reserved for the boot code and boot data, and no device window is mapped there (2.3.6, *(D)*); how many pages is not decided. *(P)*

4.4.4 If no boot device answers, or a key is held at power-up, the monitor runs on the serial console. *(P)*

What is not specified here. Where block 0 is loaded (0x1000 is an illustration); how the ROM finds the boot device's windows before any software has assigned them (2.4.1 says the processor assigns windows; the ROM is the first software to do it, and the repository does not say where it puts them or whether it leaves them for the kernel); which of the boot-information block's fields the ROM fills in and which the second stage adds (the natural reading, since the ROM knows the memory it brought up, the cores and the device, is that the ROM fills the block and the second stage adds the command line and the screen, 4.6; not stated); and the number of reserved pages.

Commentary. **Precedent.** The PDP-11 read block 0 of the boot device into low memory and jumped to it, from a boot ROM or a few words keyed in on the front panel; the VAX console processor did the same with the boot block and passed the boot device in registers; the mainframe's IPL record and the Alpha console's boot block are the same idea. In V7M the block-0 program is about 285 lines of PDP-11 assembler that read a pathname and loaded the second stage, `boot`, a C program of 272 lines (4.1; the boot review took the 250-line block-0 program for the second stage). **Trade-offs.** A boot ROM with a driver per boot device is more code in the controller than a "jump to a fixed flash address" scheme, and much less than firmware; the first block must fit in one sector. **Improvements.** The reset state is specified in full, where the earlier machines left it to the console. **Benefits.** The first stage is the same on every board. **Modern comparables.** A PC's BIOS reads one 512-byte sector; ARM boards load a first-stage loader from the boot medium; UEFI runs a file from a FAT partition. **Impact.** *Kernel:* The reset state is fully specified, so the first kernel code makes no guesses. *Compiler:* The first block is 446 bytes of assembler. *Hardware:* The ROM reads one block and starts one core.

4.5 The disk: a GPT around the BSD label

What a partition is, and why V7 had none on the disk. A disk is one long run of blocks, and a computer wants several regions of it: a root file system, a swap area, a place for the users' files. A **partition** is such a region, a named range of blocks. Something has to record where each begins and ends, and there are three places to put the record. V7 put it **in the kernel**:

the disk driver compiles in a table, `hp_sizes[8]` in `hp.c`, eight pairs of a block count and a first cylinder, so changing the layout means editing the driver and building a new kernel; 4.3BSD kept that arrangement. The 1988 4.3BSD-Tahoe put it **on the disk**: a **disk label**, a small structure written in one of the first blocks that holds the disk's geometry and a table of its partitions; its header says "the label is in block 0 or 1, possibly offset from the beginning to leave room for a bootstrap", with the magic number `0x82564557` that marks it as a label. 2.11BSD's block 0 holds the bootstrap and its block 1 the label, the same two places as here. OpenBSD's label is its direct descendant, with the same magic number, **16 partitions** (`MAXPARTITIONS16`) and sector counts beyond 32 bits (fields such as `d_secperunith` hold the high part). The third place is the PC's: a **table in the first sector** (the MBR table of 4.2, then the GPT).

Why the pdp64 has both a GPT and a label. The author's words (errata 50, 2026-10-22): "we will have the gpt wrapper for the disk label." The disk carries a GPT with a protective MBR, so that other operating systems and partitioning tools recognise it, and the operating system's own partition (the OpenBSD type, `GPT_UUID_OPENBSD`) holds the BSD label, which says where the root file system, swap and the rest are. The label alone would have been smaller and would have been unreadable to everything else; the wrapper costs a reader for the GPT in the second stage and the kernel. And the author added (errata 51, "follow the majority as far as boot block offset"): inside the OS partition the label is at **sector 1, offset 0**, the sector that most of OpenBSD's ports choose, leaving sector 0 for a boot block.

The first stage lives in the protective MBR. Block 0 of a GPT disk is the protective MBR, 512 bytes: 446 bytes of code, then the table (one entry, of type `0xEE`), then `0xAA55` (that is, the bytes `0x55` then `0xAA`, which is the `0x55AA` of 4.4.2). So the first stage has 446 bytes for itself. It cannot understand a GPT or a file system in that space, and does not try: it carries a **block list**, three numbers for each piece of the second stage (the first block, the number of blocks, the address to load them at), written into its own code area by the **`installboot`** tool when the system is installed, and it reads those blocks and jumps to the first. This is a pattern the BSD systems already use: OpenBSD's `installboot(8)` page for the sparc says the first-stage program "loads the second stage program from a set of filesystem block numbers that have been hard-coded into it by the installboot program", and on the PC `installboot` patches `biosboot` with the location of `/boot` and "must be re-run whenever `boot(8)` is changed, as its inode may change" (read in source). The boot-code bytes of the MBR are the only place the pdp64 puts block-list data, so a damaged GPT cannot stop the machine reaching the second stage, which can repair it.

```
first_stage():                                # at LOAD (0x1000): 446 bytes of code plus the block list
    # entering state: kernel mode, priority 7, r0 = boot device, r1 = boot-information block, r14 = a stack
    for entry in block_list:                  # entries written by installboot: (start, count, address)
        read_blocks(r0, entry.start, entry.count, entry.address)     # (see the open question below)
        if failed: goto monitor_or_halt       # what to do on failure is not specified
    jump to block_list[0].address             # the second stage, with r0 and r1 unchanged
```

The pseudocode hides a hard question that the repository does not answer: **how the first stage reads blocks.** 4.3.1 says the ROM offers no services after boot, and a virtio-blk read needs a queue, three descriptors, a doorbell and a poll (4.4); can that fit in 446 bytes beside the list? The PDP-11 primary bootstrap carried its own disk driver (the `hpcs1` and `hpcs2` writes of 4.1) and the PC's called the BIOS. The pdp64 has three ways out, none decided: the first stage contains a small polled virtio-blk reader of its own (the PDP-11 way, and consistent with 4.3.1; its size on the pdp64 is unknown: nothing has been assembled); or the ROM leaves a read routine and tells the first stage where it is (a firmware call in miniature, against 4.3.1); or the ROM loads more than block 0 (the whole 8 KB boot area of 4.5.3, say), so the first stage has room to carry the reader. The question is in section 4.13.

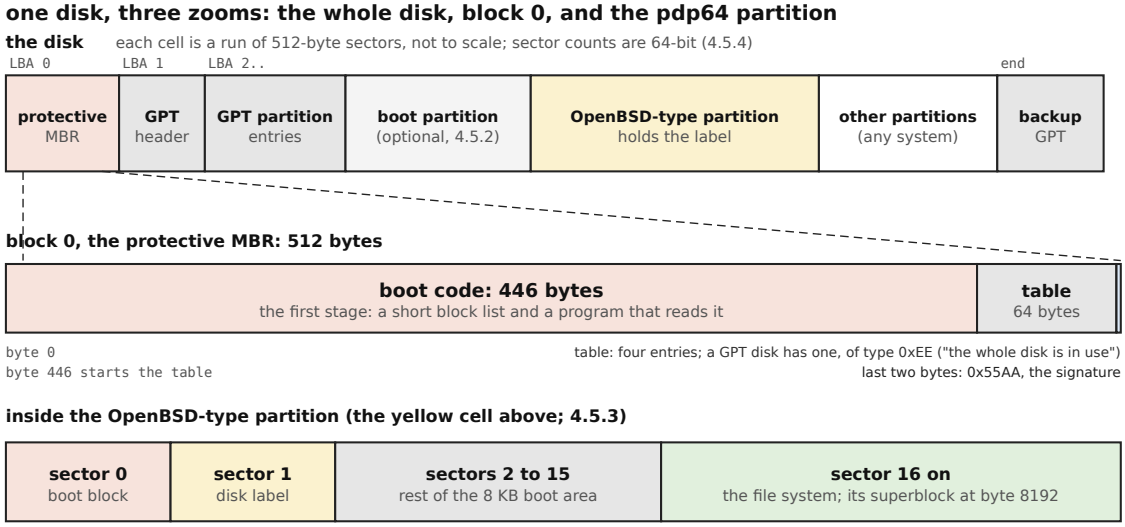
4.5.1 The disk carries a GUID partition table with a protective MBR; the operating system's own partition holds the BSD disk label, which names the root, swap and other partitions. *(D)*

4.5.2 Block 0 is the protective MBR, so the first stage lives in its 446 boot-code bytes, carrying a short block list (start block, count, load address) to the second stage; neither the ROM nor the first stage parses the GPT. *(P)*

4.5.3 Inside the BSD partition, the label is at sector 1, offset 0; the boot area is 8192 bytes at the start of the partition; the file system begins after it, its superblock at offset 8192. *(D)*

Sector (512 bytes) of the BSD partition	Contents
0	the partition's own boot block
1	the disk label, at offset 0
2 to 15	the rest of the 8 KB boot area
16 on	the file system, superblock at offset 8192

4.5.4 Sector counts are 64-bit throughout; the label has 16 partitions and its sizes carry high fields beyond 32 bits. *(D)*



The label (offset 0 of sector 1) has the magic number 0x82564557 and names up to 16 partitions (root, swap, ...); its sizes carry high fields beyond 32 bits. The kernel is a file in the root file system, which the second stage finds.

On the PDP-11 there was no such table: V7 compiled eight (size, first cylinder) pairs into each disk driver (hp_sizes[8] in hp.c). In 2.11BSD block 0 held the 512-byte bootstrap and block 1 the disk label: the same two places as here.

The MBR and GPT layouts are as the OpenBSD header disklabel.h has them (4.12); the 8 KB area and sector 1 are decided (4.5.3). Whether a separate boot partition is needed at all is open: the block list can point into the root file system (4.13).

The **superblock** (the file system's first record: its size, its block size and where its tables are) sits at byte 8192 of the partition, after the 8 KB boot area that the BSD tradition (`BBSIZE`) reserves for the bootstraps, whose header says that the area holds "primary (block 0) and secondary (block 1-15) bootstraps". The second stage can therefore live in the boot area, in a small boot partition, or as a file in the root file system; whether a dedicated boot partition is needed at all is open (4.13), since the block list can point at any of them.

Commentary. **Precedent.** V7 had no partition table on the disk: each driver compiled in an array of eight sizes and starting cylinders (`hp_sizes[8]` in `hp.c`), and changing the layout meant recompiling the kernel. The disk label arrived with 4.3BSD-Tahoe in 1988, in sector 0 or 1 "possibly offset from the beginning to leave room for a bootstrap", with the magic number `0x82564557`; OpenBSD's label is its descendant, with the same magic number, 16 partitions and 64-bit sizes. In the OpenBSD tree seven architecture headers put the label at sector 1 offset 0 (amd64, i386, arm, arm64, riscv64, hppa, powerpc64); Alpha and SPARC64 put it at sector 0. The PC used the MBR, a four-partition table in sector 0 with 446 bytes of boot code, and UEFI requires the GPT, whose header is in the second block, with a backup at the end of the disk and a protective MBR in block 0. **Trade-offs.** The label alone would have been the smallest design and would have been unreadable to other systems' tools; the GPT wrapper costs a GPT reader in the second stage and the kernel (OpenBSD's kernel already has one) and puts the protective MBR in block 0, and gives a disk that other operating systems and partitioning tools recognise. Following the majority on the offsets means the port adds one header of a dozen lines. **Improvements.** The first stage does not depend on a GPT that may be damaged, so a corrupted table cannot stop the machine reaching the second stage, which can repair it. **Benefits.** The label code, `disklabel`, `newfs` and `installboot` come with OpenBSD. **Modern comparables.** BSDs on UEFI machines use a GPT with the label inside an OpenBSD-type partition and a FAT partition for the loader; Linux has no scheme of its own. **Impact.** *Kernel:* The label code, `disklabel`, `newfs` and `installboot` come with OpenBSD. *Compiler:* None. *Hardware:* None.

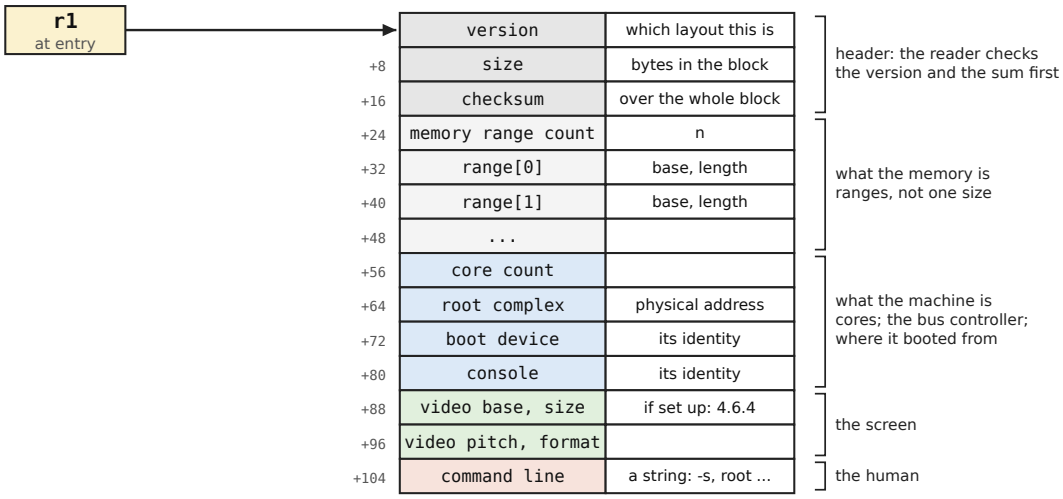
4.6 The second stage and the hand-off

What the second stage is for. The first stage has done one thing: put a bigger program in memory. The second stage is that program, and it does what the PDP-11's `boot` did, with more to read: it finds the disk, reads the partition table, reads the label, finds the root file system, finds the kernel by name in it, loads the kernel into memory, and enters it. Because it is written in C and loaded from disk it can be as large as it needs and be replaced by copying a file. It has its own drivers (the virtio-blk reader, now complete) and no firmware to call. As a sketch:

```
second_stage():                                 # loaded by the first stage; r0 and r1 as at reset
    dev = open_boot_device(r0)                  # the same virtio-blk driver the ROM used, now complete
    gpt = read_gpt(dev)                         # header at block 1: check "EFI PART" and the CRC32;
                                                # if it is bad, use the backup at the end of the disk
    part = first entry whose type GUID is the OpenBSD type
    label = read(dev, part.start + 1)           # sector 1 of the partition (4.5.3)
    check label.magic == 0x82564557 and the label's checksum
    root = the label's root partition ('a'); mount it read-only
    cmdline = prompt("boot> ") or the default   # as the V7 ':' prompt: which kernel, which options
    elf = root.open("/bsd")                     # an ELF file, the toolchain's own object format
    for seg in elf.loadable_segments:
        copy seg.file_bytes to physical address seg.address; zero the rest of seg
    info = r1                                   # the block the ROM began; add what only we know:
    info.cmdline = cmdline
    info.video = framebuffer if we set one up (4.6.4)
    info.checksum = checksum(info)
    jump elf.entry with r0 = boot device, r1 = info     # still kernel mode, priority 7, translation off
```

The hand-off: how the kernel learns about the machine. The PDP-11 had four registers and a kernel built for the machine. A kernel that is to run on any pdp64 needs to be told: how much memory there is and where it lies (ranges, since memory is not always one block), how many cores there are, where the bus's root complex is (so that it can **enumerate**, walk the bus and list what is attached, 2.4), which device it was loaded from, where the console is, and the command line. The way to tell it is a **boot-information block**, a structure in memory whose address is in `r1`. Other machines tell the kernel with larger things: the VAX passed a **restart parameter block** (a record of the machine's state, fixed in layout; in the 4.3BSD source on this machine it is a page in low memory that a restart after a crash runs a dump routine from, and what DEC's own boot program passed is unverified), the Alpha a **hardware restart parameter block** (the same idea), and x86 and ARM servers pass **ACPI tables**, while arm64 and RISC-V pass a **device tree**; an operating system for several of them has to carry a parser for each. The pdp64's hand-off is small because the bus describes its own devices: the block names memory, cores, the root complex, the boot device and console, and the screen if one was set up, and everything else the kernel finds by enumeration.

the boot-information block: a struct the second stage fills in and r1 points at



Each row is one 64-bit word in this drawing (the offsets are bytes). The contents are statements 4.6.2 and 4.6.4; the offsets, the widths and the order drawn are an example, not specified.

Defined once in a grammar from which the headers are generated, so the loader and the kernel cannot disagree (4.6.3). The devices are found by enumerating the bus (2.4), so the block stays short.

The VAX and Alpha passed similar blocks, a restart parameter block each (from the boot review, unverified, 4.12).

4.6.1 The second stage is a program in C that understands the root file system, reads the GPT and the label, loads `/bsd` (an ELF file) and jumps to it with `r0` and `r1` as before and a command line in the boot-information block. *(P)*

4.6.2 The boot-information block names the memory ranges, the core count, the address of the root complex and the identity of the boot device and the console; the kernel enumerates every other device by PCI Express enumeration (section 2.4). *(P)*

4.6.3 The block's layout is defined once, in a grammar from which the headers are generated, and carries a checksum and a version. *(P)*

4.6.4 The boot-information block carries the video device's base address, size, pitch and pixel format when the boot loader has set it up, and nothing about the device's clock or power state is changed behind the kernel: its pixel clock stays up until the kernel's driver owns the device. *(D)*

Statement 4.6.4 exists because of a lesson the PC learned the hard way (the loader sets up a picture so that the boot messages are visible, and the kernel must be able to keep drawing on it until its own driver takes over, without the screen flickering or going blank): the **framebuffer** the loader set up is described in the block, and the device is left as it is.

Commentary. **Precedent.** The VAX passed a restart parameter block, the Alpha a hardware restart parameter block, both fixed-layout records of the machine; the PDP-11 configured the kernel for the machine at build time and probed device addresses; x86 uses ACPI tables, and arm64 and RISC-V use a device tree. **Trade-offs.** A fixed-layout block with a checksum has two failure cases (bad checksum, unknown version), against the parser a device tree or ACPI requires; a device tree has the advantage that OpenBSD's arm64 and riscv64 ports already parse one and attach drivers from it, which could save kernel code if the port reuses many of their drivers. **Improvements.** Because the bus describes its own devices, the block shrinks to memory, cores and the bus controller's address. **Benefits.** The hand-off is a struct; the generated header cannot disagree with the loader. **Modern comparables.** The device tree and ACPI are the two standard answers. **Impact.** *Kernel:* The hand-off is a structure the kernel reads. *Compiler:* The header is generated from one grammar, so it cannot drift from the loader. *Hardware:* None.

4.7 Starting the other cores

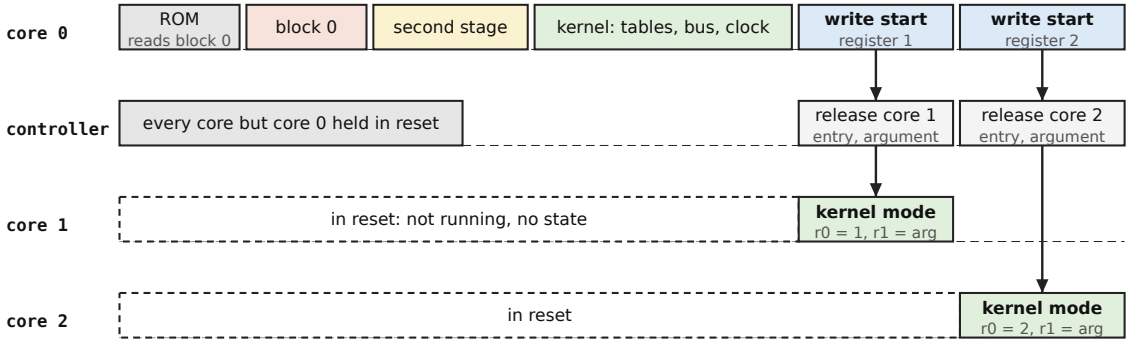
The problem the PDP-11 did not have. A PDP-11 had one processor, so "start the machine" was one thing. A pdp64 may have several **cores** (processors on one chip; chapter 7), and at reset only one of them, core 0, runs the boot code. The others have to be started by the kernel, each with its own stack and its own idea of who it is. On the PC this is a famous mess. The x86 boot core sends special signals through the interrupt controller (INIT and START-UP signals, which Linux's comment calls the INIT, INIT, STARTUP sequence), the START-UP carrying an address below one megabyte, and the other core wakes in the 8086's **real mode** (4.2) at that address, 16 bits and one megabyte, and a hand-written **trampoline** (a short program that takes a processor from a limited mode into its full one, by switching it through two more modes) must walk it into 64-bit mode before it can run the kernel: the Linux `smpboot.c` that does this is 1,528 lines and the real-mode code another 1,258 (measured in the boot review, `boot.md`, from Linux `7be4bc52`). arm64 asks firmware, through a call called PSCI, in 124 lines, or has the waiting cores **spin** (loop on a word in memory) on a **spin table** until the boot core writes an address there, in 127; RISC-V asks the firmware to start a **hart** (its word for a core) in 108 lines or spins in 58. The VAX's console processor started the VAX's other processors (unverified).

The pdp64's rule: one register write. A pdp64 core has no mode ladder, so it can start in its normal mode at any address. The controller holds every core but core 0 in reset; to start core *n*, the kernel writes, to a register block in the controller that belongs to core *n*, the address where the core should begin and one argument, and the controller releases the core into **kernel mode at priority 7** with its core number in `r0` and the argument in `r1`. The kernel's convention for the argument is a pointer to a structure that belongs to that core (its stack, its per-CPU data). A core that sticks is reset by a write to a core-reset register, and started again. In pseudocode, the kernel side and the new core's side:

```
start_secondary_cores():                    # core 0, in the kernel, after its tables are built
    for c in 1 .. core_count - 1:           # core_count comes from the boot-information block
        percpu[c] = allocate(percpu_size); percpu[c].stack = allocate(KSTACK)
        percpu[c].online = 0
        write(start[c].entry, secondary_entry)   # the exact registers are not specified (4.7.1)
        write(start[c].argument, percpu[c])
        write(start[c].go, 1)                    # the controller releases core c into kernel mode
        wait up to a timeout until percpu[c].online == 1
        if not online:
            write(core_reset[c], 1)              # 4.7.2: stop it, and try once more
            ... start it again, or mark core c failed and go on without it
```

```
secondary_entry():                          # core c: kernel mode, priority 7; r0 = c, r1 = percpu[c]
    set VBR and the page-table base to the kernel's, turn translation on, ISYNC    # chapter 6 (6.12)
    set the per-CPU pointer DP from r1; set R14 = percpu.stack top                 # chapter 7 (7.4.2)
    percpu.online = 1                       # core 0 is polling this word
    enable the clock; enter the scheduler's idle loop (WFE until there is work)    # chapter 6, 7.6
```

starting the other cores: one register write each, then the core is a core



time runs to the right (not to scale); priority 7 on entry. The entry address is where the kernel's per-core code starts; the argument is a pointer to that core's own structure. A core that sticks is reset by a write to a core-reset register of the controller and started again (4.7.2).

x86, for comparison (general knowledge and Linux smpboot.c, 4.7): the boot core sends INIT and START-UP signals through the interrupt controller; the other core wakes in 16-bit real mode at an address below 1 MB, and a trampoline must walk it through protected mode into 64-bit mode.

The pdp64 has no such ladder: a core starts in its normal mode at any address. What the start registers look like is proposed, not decided (4.7.1); the register block's address is open (4.13); the SMP chapter (7.8.1) names r0 only, where 4.7.1 loads r1 too.

The cost is small: one register block in the controller, with a start register and a reset hold for each core. The core does not start with translation set up or a stack of its own; the kernel's entry code does that in the first few instructions, as the first kernel does (6.12). Whether a started core begins with translation off and what `MMUCTL` it sees are not specified.

4.7.1 The controller holds every core except core 0 in reset; a write to a core's start register in the controller gives it an entry address and an argument and releases it into kernel mode at priority 7, with its core number in `r0` and the argument in `r1`. *(P)*

4.7.2 A stuck core is reset by a write to a core-reset register. *(P)*

What the author and the other chapters say. This is option 1 of boot decision B2 and of decision R2 of `decisions-pending.md`, which recommend it and leave it proposed (the other options were the x86 signals, a firmware call, a spin table and a table that all cores start from; none was chosen); the SMP chapter states the same rule as 7.8.1 and loads `r0` only, where 4.7.1 loads `r1` too, a difference to settle (4.13). Waiting uses `WFE` and `SEV` (7.6), and a signal from one core to another, an **inter-processor interrupt**, is a write to a per-core register of the interrupt controller (7.8.2).

Commentary. **Precedent.** On x86 the boot core sends INIT and START-UP interrupts through the APIC and each other core wakes in 16-bit real mode, then has to be walked through protected mode into 64-bit mode by a trampoline: the Linux `smpboot.c` is 1,528 lines and the real-mode code another 1,258. arm64 uses a firmware call (PSCI) in 124 lines or a spin table in 127; RISC-V asks the firmware to start a hart in 108 lines or spins in 58. The VAX's console processor started its other processors. **Trade-offs.** The pdp64 has no mode ladder, so a core can start in its normal mode at any address and the whole protocol is one register write, at the cost of a small register block in the controller. **Improvements.** There is no trampoline and no firmware call. **Benefits.** Fewer lines than any system measured and one recovery path, a core reset. **Modern comparables.** PSCI, the SBI hart-state calls, and x86's INIT and START-UP sequence. **Impact.** *Kernel:* Starting a core is a register write and a device start is `INIT`. *Compiler:* None. *Hardware:* A start register and a reset hold per core; an `INIT` bit per device.

4.8 Starting the devices

The INIT bit. On the Unibus every device had a **control and status register** (CSR) with a bit that told the device "initialise yourself" and a bit that said it was ready: you wrote `INIT`, waited for `READY`, and the device was in a known state (the

RK11 of chapter 2 has GO and RDY in its CSR). All the work of bringing the device to life happened behind the register, in the device's own logic. The pdp64 keeps that idea and turns it into a conversation: under virtio (chapter 2, 2.6.3) the driver writes **zero** to the device's `device_status` byte, which resets it, then sets the bits ACKNOWLEDGE, DRIVER, FEATURES_OK and DRIVER_OK in order, each meaning "I have got this far", and the device answers by refusing or accepting the features the driver chose. That is the INIT and READY of the CSR spread over a few steps, and the pseudocode of 2.6.3, `init_device(fn)`, is the pdp64's device start. At boot it is run by the ROM for the boot disk (4.4), by the second stage again for the devices it uses, and by each kernel driver at attach.

Firmware for a device. Some devices contain a small processor of their own and need a program, **firmware**, loaded into them before they work: a network or wireless card is the usual case, a graphics device another. On a PC the driver loads the blob at attach time from a file (Linux's `request_firmware` and OpenBSD's `firmload(9)`, unverified), and the files come in a separate package. The pdp64 proposal is that such a device is given its firmware through a descriptor operation (a request on the device's queue, or a descriptor that gives the device the address and length of an image in memory, with a small header the device checks: its identifier, a version, a length and a checksum), **once the root file system is available**, and that a device the boot depends on needs none: the boot disk, the console and the interrupt controller must work from their built-in logic alone, so that no device waits for firmware that is on the disk it controls.

```
start_device(dev):                          # at boot or at attach
    init_device(dev)                        # the handshake of 2.6.3; reset first
    if dev.needs_firmware:                  # a device the boot depends on never does
        image = read_file(root_fs, firmware_name(dev))      # the file system must be up (4.6)
        check image.header: device id, version, length, checksum
        submit a descriptor chain giving dev the address and length of image
        wait for the device to answer; if it refuses, mark the device failed
```

4.8.1 A device is initialised by writing its INIT bit and waiting for READY; a device that wants firmware is given it through a descriptor operation once the root file system is available, and a device the boot depends on needs no firmware. *(P)*

What the statement predates. 4.8.1 was written when devices were eight registers with an INIT bit and a READY bit (the decision of errata 29, which errata 95 withdrew in favour of virtio); the `device_status` handshake above is how it reads under virtio. How a virtio device is handed firmware is not specified: virtio has no firmware operation of its own, so a request type, or a separate device, would have to be defined (4.13).

Commentary. **Precedent.** PCI Express devices are initialised through their configuration space and per-device driver code; the Unibus's CSR with its INIT bit and nothing more was the whole of it on the PDP-11; the VAX console and some adapters loaded microcode from storage (unverified). **Trade-offs.** A reset bit and a status register make the simplest device, with one failure case, a timeout; a firmware window lets a device be cheap (no flash) and makes an update a file replacement, at the cost of a header format and a device that rejects a wrong image safely; a device whose firmware is on a disk it must itself read cannot start, which is why the boot-critical devices are excluded. **Benefits.** One start convention for every device. **Modern comparables.** `request_firmware` and `firmload(9)`; NVMe's firmware-download command, which needs a working queue first. **Impact.** *Kernel:* Starting a core is a register write and a device start is INIT. *Compiler:* None. *Hardware:* An INIT bit per device.

4.9 Why UEFI is not used

4.9.1 The pdp64 does not use UEFI: its boot is the block-0 route of sections 4.4 and 4.5, its hand-off is the boot-information block, and its devices describe themselves. UEFI firmware is not part of the specification and is not built. *(D)*

4.9.2 The hand-off and the second stage are kept simple enough that a UEFI shim could be written by someone who wanted one. *(D)*

Both statements are the author's decision of 2026-11-10 (errata 109). The record, in his words. On 2026-10-22 he asked, of boot, "look into uefi; is it worth it at all? can't we do what pdp11 used to do, load 512 byte block 0 then jmp 0? (not precisely true, but directionally true)", and asked how UEFI plays into partitions and the GPT; the review (`boot.md`) answered that UEFI was not worth it as the machine's own boot, and sections 4.6.1 and 4.6.2 of the earlier text were the assistant's proposals, tagged *(P)*, never a decision. On 2026-11-10 he asked "i thought we did decide to allow uefi? check all the threads where we discussed it" (no decision to allow UEFI was found in any recorded message), then "or we kept the originnal simple pdp11 boot block method?", and, given three options, chose "option 2, block 0 with a shim possible". So the pdp64 boots

by the block-0 route; a UEFI shim is not built; and the hand-off and second stage are kept simple enough that someone who wanted one could write it.

Is UEFI an open specification, and what does an open-source system do with it? The author asked, after the decision (2026-11-10): "if uefi isn't an open spec how are all the open source os supporting it? it requires binary firmware blobs?" The honest answer, by what could be checked:

- **The specification** is published by the UEFI Forum, the industry body that maintains it, and can be read without charge (general knowledge, unverified: the specification is not on this machine, and whether implementing it needs a membership, a licence or a compliance test is not known here). So it is not a closed specification; the closed part is elsewhere.
- **An open-source implementation exists.** EDK II, the TianoCore project's reference implementation, is under the BSD-2-Clause-Patent licence (read: `/home/david/claude/qemu-src/pc-bios/edk2-licenses.txt`, and the Debian `ovmf` package's `/usr/share/doc/ovmf/copyright`, "Upstream-Name: edk2"). Built images are on this machine (QEMU's `pc-bios/edk2-*.fd.bz2` and `/usr/share/OVMF`, including `OVMF_CODE_4M.secboot.fd`), so an emulated PC boots under open firmware. The source is not on this machine (the `roms/edk2` directory is empty), so nothing about its size or contents was read here.
- **Open-source operating systems write the loader side.** OpenBSD's `efiboot` (read in source), Linux and NetBSD each contain a program or stub that runs on top of firmware that is already on the machine and calls its services (4.9, the table below); they do not ship the firmware. A loader needs no blob.
- **The proprietary blobs are in the firmware shipped on real PCs:** the vendor's firmware builds, and the CPU vendor's initialisation code that brings up memory and the chipset, which many boards include as a binary (unverified general knowledge: not on this machine). That is a property of how PCs are made, not of the UEFI specification.
- **Secure Boot** adds a chain of signed loaders and keys kept in the firmware's variable store, so that firmware runs only a loader that is signed by a key it trusts (unverified; the existence of the `secboot` images above is the only thing seen here).

The author's conclusion (2026-11-10): "I guess leaving the door open to uefe support but having the classic mbr fallback, gpt wrapper is a good choice then." That is a **confirmation of option 2** of errata 109, not a new decision (errata 110). The two routes share one disk: block 0 of the pdp64 disk already has the protective MBR and GPT layout (errata 50, 4.5), so the classic route (the ROM reads block 0, the first stage in its 446 bytes) and a future UEFI route (a firmware, or the shim, reads the GPT and a FAT system partition) read the same disk without repartitioning it.

What a shim is, and what it is not. The author asked, "not sure why firmware and coff are needed for uefi, with virtio native that shuld be minimal, no?" and then, "yes, state the shim's real scope." The answer in two parts. *First, virtio removes most of the driver mass* that a full UEFI firmware carries (a firmware ships drivers for every disk, network card and screen it might meet; the pdp64 has one device interface), and the shim needs only the drivers of the second stage, which already exist. *Second, two things stay,* and they are the shim's real scope:

1. **PE/COFF.** The UEFI specification requires a UEFI application to be a PE32+ image (the 64-bit form of the PE/COFF file of 4.2); this is a property of the **loader programs**, not of the firmware. To run an existing UEFI loader, such as OpenBSD's `efiboot`, it has to be built as a PE/COFF image *for the pdp64*, so the toolchain has to produce one, and a **machine-type code** for the pdp64 (a 16-bit number in the image's header that names the processor: OpenBSD's arm64 loader says `IMAGE_FILE_MACHINE_ARM64 0xaa64`, read in source) would have to be assigned by the owners of the UEFI specification. Both of those are unverified general knowledge (the specification is not on this machine). What the sources on this machine do show is that this need not be a linker feature: OpenBSD's arm64 loader is linked as an ordinary ELF file and wrapped as a PE32+ image by a header written by hand in assembler (`start.S`, 176 lines, "on arm64 we need to do it ourselves") plus a small self-relocator (`self_reloc.c`, 124 lines), with `objcopy` doing the rest (measured, read in source). Whatever the container, the loader is itself a program that must be compiled for the pdp64.
2. **A small services layer.** The loader calls a table of services, and the shim has to provide the few that loaders use: memory allocation and the **memory map**; reading a file from the boot partition; the console and the **graphics output protocol** (a framebuffer to draw on); **exit boot services**; and a runtime clock and reset. All of it over virtio. And a **hand-off** of a device description to the kernel in the form such a loader expects: a device tree or tables.

How many services, counted. OpenBSD's own UEFI loaders in `/home/david/claude/kernel-src/openbsd/sys/arch/*/stand/efiboot` call the following (measured by counting the distinct `BS->`, `RS->` and console calls in the `.c` files; the number after the name is the count of call sites in the amd64 loader):

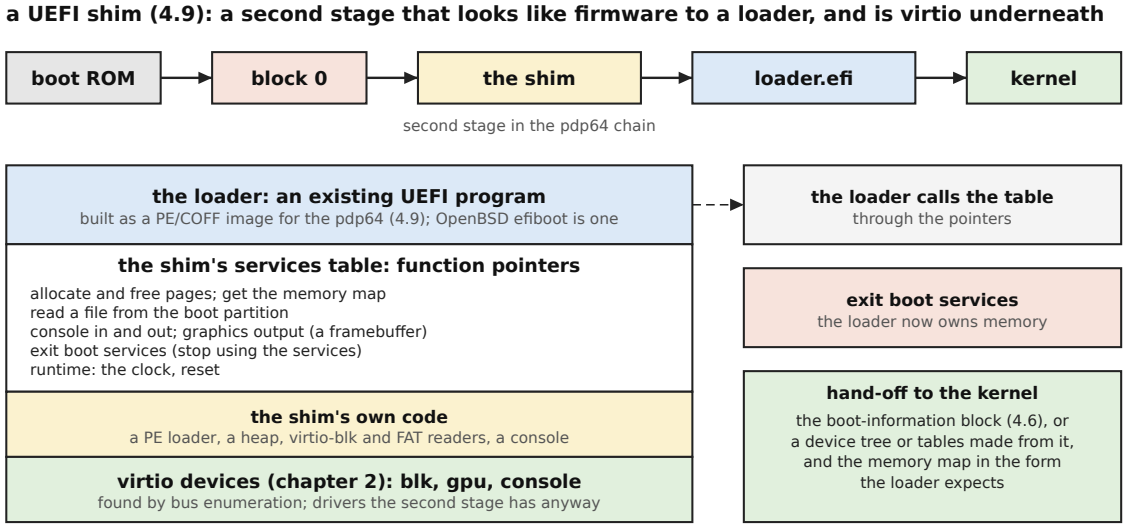
Boot services (`BS->`)	Runtime services (`RS->`)	Console and file protocols (through a handle)
`HandleProtocol` 13, `AllocatePages` 7, `LocateHandle` 5, `LocateProtocol` 2, `LocateHandleBuffer` 2, `GetMemoryMap` 2, `FreePages` 2, `WaitForEvent`, `SetWatchdogTimer`, `FreePool`, `AllocatePool`, `Exit`, `ExitBootServices` (1 each); the arm64 and riscv64 loaders add `SetTimer`, `CreateEvent`, `CloseEvent`	`ResetSystem` 3, `GetVariable` 3, `SetVariable` 2, `GetTime` 1	the text console (`OutputString`, `ReadKeyStroke`, `SetMode`, `QueryMode`, ...), the disk (`ReadBlocks`, `WriteBlocks`), files (`Open`, `Read`, `Write`, `Close`, `SetPosition`, `GetInfo`), the network (`Mtftp`, `Transmit`, `Receive`), the graphics output protocol, the random-number protocol

That is about fifteen to seventeen distinct boot and runtime calls in each loader. It is a count of what **one** loader uses and not a specification of what a shim must offer: another loader would use others, and the person who writes the shim reads the loader. One more fact from the same source shows the calling convention: the loader re-reads the memory map and calls `ExitBootServices` with the map's key, and retries once if the call fails, because the call refuses a stale key (`efiboot.c`, read in source).

A sketch of the shim (an illustration; no such code exists). What the shim does, in C-like pseudocode: the shim is the second stage, loaded by block 0; it builds the services table, loads the loader's PE image, calls it, and answers its calls.

```
shim_entry():                                # the second stage of a pdp64 that wants UEFI loaders
    info = r1; heap = free_ranges_from(info.memory_ranges)
    table = new system_table {
        boot_services = {
            allocate_pages = shim_allocate_pages,   # out of the heap; the loader asks for a page count
            free_pages     = shim_free_pages,
            get_memory_map = shim_get_memory_map,   # the ranges of `info`, plus what the heap handed out
            locate_protocol = shim_locate_protocol, # the console, the graphics output, a file volume
            exit_boot_services = shim_exit_boot_services,
            ... }                                    # the others a given loader uses; unused ones return
"unsupported"
        runtime_services = { get_time = clock_register, reset_system = reset_register, ... }
        configuration_tables = [ (DEVICE_TREE_GUID, make_device_tree(info)) ]   # made from the boot-
information block
    }
    image = read_file(boot_partition, loader_path)  # the path convention is not specified
    entry = pe_load(image)                          # headers, sections, relocations
    status = call entry(image_handle, &table)       # the loader runs, and calls back through the table

shim_exit_boot_services(handle, map_key):
    if map_key != current_map_key: return INVALID_PARAMETER    # the map changed since the loader read it
    stop the shim's console and timer; free the memory of the boot-services data
    keep only the runtime services: the clock and reset are register writes
    return SUCCESS                                 # the loader owns the memory and enters the kernel
```



The loader needs the services table, a PE/COFF image of its own and a machine-type number for the pdp64 that the owners of the UEFI specification would have to assign (unverified, 4.12). The shim is a few services over drivers that already exist; it is not a firmware stack: no PEI, DXE or driver model, no variable store, no setup screen. Nothing is built; this illustrates the size of the job (4.9).

The services named are those OpenBSD's efiboot loaders call (measured, 4.12): about fifteen boot and runtime services in each. Which of them a given loader needs is for the person who writes the shim to find by reading that loader.

The costs, kept honest. A shim is **not** a UEFI firmware stack. The boot review's figure of "hundreds of thousands to millions of lines" for a UEFI firmware (`docs/boot.md`, "Is UEFI worth it?") was never confirmed against a source, and it describes a **full firmware stack** such as EDK II, with its phases, its driver model, its variable store and its drivers for every device; a shim is a different and much smaller thing, and that figure is not used as the cost of one. **No count of the shim's lines is given here, because none can be measured on this machine:** no implementation of the UEFI services is present (the `edk2` and `u-boot` directories under the QEMU tree's `roms/` are empty; only EDK II's built images are, 3,653,632 bytes for the x86-64 code image). The measured sizes are those of the loaders that would run on it (OpenBSD's `efiboot` is 4,593 lines for amd64, 7,195 for arm64, 4,513 for riscv64) and, for what a pdp64 build of one would need beside the loader, the 176 lines of header and 124 of relocator above. The costs that remain:

- **A PE/COFF build and a number.** The toolchain emits a PE/COFF image (small, per the precedent above), and a machine-type number must be obtained (unverified).
- **A loader to port.** The loader has to be ported to the pdp64 as any program is, which is the larger part of the work.
- **A services layer.** It has to be written, tested and kept in step with the loaders it serves.
- **Code to trust before the kernel.** The shim is open source (4.3.2), runs only at boot and, unlike a firmware, is not resident afterwards except for the clock and reset.
- **A platform description.** The shim does not remove the need for one (a device tree or tables), which it must build from the boot-information block.

What it does **not** cost is a closed firmware, a variable store, a setup screen or a second boot path in the machine itself: the machine's own boot stays block 0.

Commentary. **Precedent.** UEFI is a firmware specification of boot services, runtime services, a memory map, the GPT, the PE/COFF executable format and a driver model, implemented usually by the open-source EDK II or by a vendor. Its aim is that one installer boots on any machine that offers it. OpenBSD's arm64 and riscv64 ports use UEFI and a device tree, and its own loaders for them are 7,195 and 4,513 lines; for amd64 the loaders total 15,440 lines, including a BIOS boot block of 862 lines and a second stage of 189 beside the UEFI loader of 4,593 (all measured). **Trade-offs.** UEFI firmware would let existing generic loaders run unchanged, offers a standard memory map and a secure-boot chain, and costs a great deal of firmware, adds failure modes (variable storage, drivers, boot-order state), enlarges the attack surface, and still needs a device tree or ACPI to describe the devices; a shim gives the first without the rest, at the price of a services layer and a PE/COFF build. **Improvements.** The pdp64 has one architecture and one platform description, which is the case UEFI was not built for, and a decision that leaves a door open for someone who wants to carry an existing loader. **Benefits.** Smaller

(principle 1), fewer failures (principle 2), fewer gates and less flash (principle 3), and a shim possible. **Modern comparables.** x86 servers, arm64 servers and RISC-V servers all use UEFI; OpenBSD itself still ships a BIOS-style loader beside the UEFI one on amd64. **Impact.** *Kernel:* No firmware interface, no UEFI services, no memory-map translation in the machine itself; a shim would translate the boot-information block. *Compiler:* No PE/COFF format in the toolchain unless a shim is built; if it is, a header and a relocator. *Hardware:* A smaller flash and no firmware stack.

4.10 Recovery

What to do when nothing boots. A PDP-11 whose disk was dead was not a dead machine: the console's ODT let you examine memory and key a bootstrap in (4.1). The pdp64 keeps the idea and gives it a place. The ROM that reads block 0 (4.4) also contains a **monitor**: a small program that runs on the serial console when no boot device answers, or when a key is held at power-up, and offers a handful of commands: examine and deposit memory and registers, load a program over the serial line, start at an address or from a named device, reset a core, reset the machine. It needs a working serial port and nothing else, so it is also the way to bring up a new board before any boot device works. In pseudocode:

```
monitor():                                  # the ROM's console program; no interrupts, no memory but the
first pages
    loop:
        print("pdp64> ")
        line = read_line(serial)
        case first word of line:
            "x" addr [n]    : print n words at addr                 # examine (the PDP-11's `addr/`)
            "d" addr value  : store value at addr                   # deposit
            "r" [name value]: print or set a register of the stopped core
            "l"             : read a program from the serial line into memory   # load
            "g" addr        : start core 0 at addr (r0, r1 as in 4.4.3)
            "b" device      : try to boot from this device
            "c" n           : write core n's reset register (4.7.2)
            "z"             : write the reset register (a bus-controller write, 2.9.4; what else it resets
is not specified)
            otherwise       : print("?")
```

The command letters and the prompt are an illustration. **Watchdog:** a **watchdog** is a hardware timer that counts down and restarts the machine if software does not write to it ("kick" it) in time; a hung kernel stops kicking and is restarted. The PDP-11 had none; a PC's firmware may have one. It is the simplest recovery of all from a hang, and needs about ten lines in the kernel, as `decisions-pending.md` P7 argues when it asks for a bus-controller reset register and a watchdog **in place of** firmware calls: the monitor's reset command writes that register.

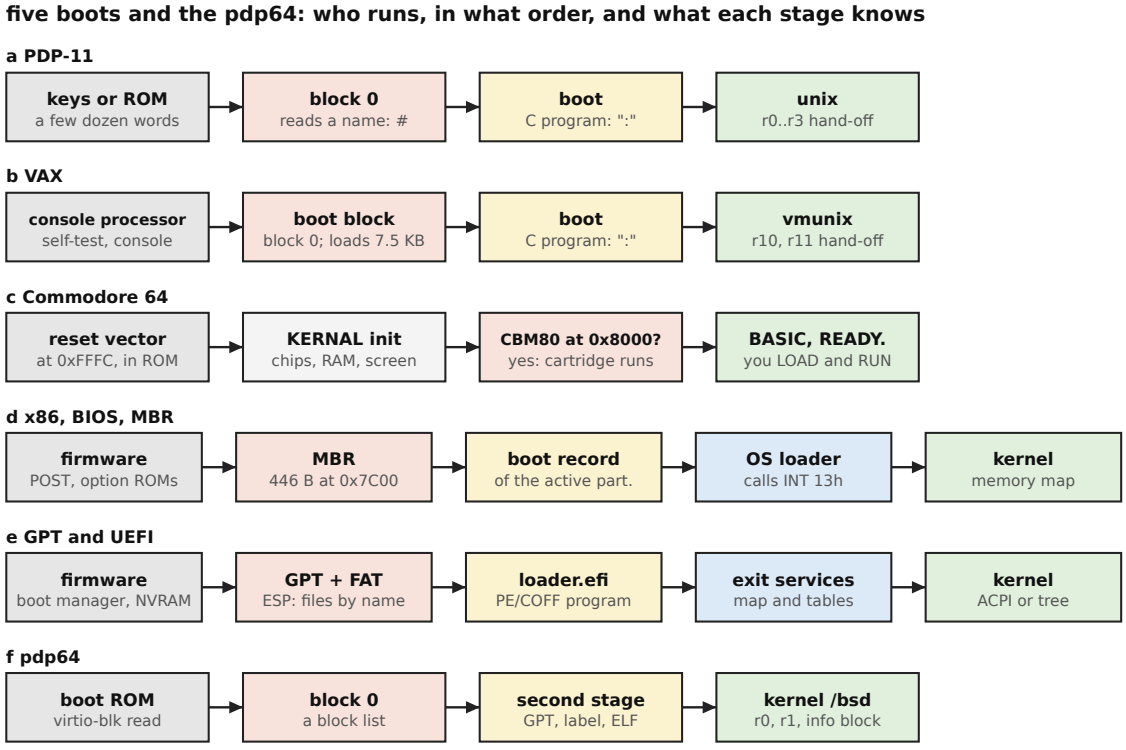
4.10.1 The controller's ROM includes a monitor reached on the serial console when nothing boots: examine and deposit memory and registers, load a program over the serial line, start at an address or from a named device, reset a core, reset the machine. *(P)*

4.10.2 A hardware watchdog restarts the machine if the kernel hangs. *(P)*

Commentary. **Precedent.** The PDP-11's console ODT, a monitor in ROM for examining and depositing memory and starting at an address, and the VAX console processor were the way in when nothing booted. **Trade-offs.** A monitor is a few hundred bytes of ROM and needs a working serial port and nothing else. **Benefits.** The same monitor brings up a new board before any boot device works, and it is the simplest recovery path of all: it needs the least code to write. **Modern comparables.** U-Boot's console and OpenSBI's debug console fill this role; PC firmware has a setup screen. **Impact.** *Kernel:* A monitor is the way in when nothing boots. *Compiler:* None. *Hardware:* A serial port and a few hundred bytes of ROM.

4.11 Five boots side by side

The author asked (2026-11-10): "also boot would compare pdp11 boot, vax boot, c64 boot, the x86 bios boot with mbr, then finally the gpt/uefi". Those five are his referents, in his order, and this section sets them side by side and puts the pdp64 beside them. Each is a numbered list of steps, a chain with what each stage knows (the figure, then the table), and a mark for how much of it could be checked: **(read)** is read in the sources on this machine, **(repo)** is from the repository's documents, and **(unverified)** is general knowledge, the handbooks and specifications not being here.



Sources: a and b read in V7M, 2.11BSD and 4.3BSD; d and e partly read in OpenBSD; c, and the rest of b, d and e, are general knowledge, unverified (4.12).

(a) The PDP-11 boot (4.1).

1. The processor is powered on and does nothing useful by itself; the operator has the console. (unverified)
2. The operator keys in a bootstrap on the front panel, or starts a boot ROM (the M9312 is named in V7M's `htboot.s`) that reads **block 0** of the boot device to low memory and jumps to it. (read: V7M `boot(8)`)
3. Block 0 (`hpuboot.s`) copies itself up, clears memory, starts the disk and prints `#`. (read)
4. The operator types `boot`; the block-0 program finds `/boot` by walking the inodes from the root, loads it and calls it. (read; the page also provides `/boot.bu`, "a backup copy of the bootstrap")
5. `boot` prints `Boot` and `:`; the operator types `hp(0,0)unix`; it loads the file at 0, sets up memory management and enters it with a `trap` instruction. (read)
6. The kernel runs; 2.11BSD's stage passes `r0` cputype, `r1` controller address, `r3` boot device, `r2` a check word. (read)

(b) The VAX boot (the 4.3BSD sources on this machine; the console and DEC's own boot program are not).

1. A **console processor** (on the VAX-11/780 a separate small computer) powers up and runs a self-test; it also provides the console commands and starts the other processors on multiprocessor models. (repo: `boot.md`; unverified otherwise)
2. On a boot command the console, or a boot ROM on the smaller models, reads the boot block (block 0) of the device into memory and starts it with registers describing the device. (repo; read for the registers: 4.3BSD-Tahoe `mdec/raboot.c` takes the Unibus adapter number from `r1`, the controller address from `r2`, the unit from `r3` and the flags and partition from `r5`, and says it "should work with any disk using 750 boot rom")
3. That first-level program (`raboot.c`) "loads next 7.5 Kbytes from boot sector of file system" to address `0x50000`: sectors 1 to 15 of the 8 KB boot area (`BOOTLAST` 15), which is where the pdp64's 8 KB boot area (4.5.3) comes from. (read)
4. That second level is `vaxstand/boot.c`: it takes `r10` (the device) and `r11` (the options), prints `Boot` and `:`, opens a file by name (`vmunix` by default) and enters it. (read)
5. The kernel runs. The restart parameter block, `rpb.h`, is in the 4.3BSD source "a page in (very) low core which runs after a crash", a record for a dump routine; what DEC's boot program (VMB, a name from general knowledge) passed is unverified.

(c) The Commodore 64 boot (nothing about it is in the repository beyond the machine's description in chapter 3; all of it is unverified general knowledge).

1. The 6510 processor, on reset, reads its start address from the two bytes at `0xFFFC`; they lie in the **KERNAL ROM**, the top 8 KB of memory, so the first code is in ROM and nothing is loaded.
2. The KERNAL's cold-start routine initialises the chips, tests the RAM, sets the vectors and the screen.
3. It looks at `0x8000` for the signature `CBM80`: if a cartridge is plugged in and carries it, the cartridge's own start address is called (**cartridge autostart**).
4. Otherwise it enters the **BASIC ROM's** cold start, which prints its banner and `READY.`
5. The user types `LOAD` and `RUN`: the program comes from the datasette or from the 1541 disk drive, a separate machine with its own processor and ROM.

(d) The x86 PC with a BIOS and an MBR (4.2; the OpenBSD loaders are read, the rest is unverified).

1. The processor starts at the **reset vector**, `0xFFFFFFF0`, at the top of the 4 GB space, where the firmware flash is mapped; it is in 16-bit real mode. (unverified)
2. The BIOS runs its self-test (POST), initialises the chipset and memory and runs the **option ROMs** of add-in cards (video, disk). (unverified)
3. It goes through the **boot-device order** and reads the first 512-byte sector, the **MBR**, to `0x7C00`, and checks the signature `0x55AA`; the code in its first 446 bytes runs. (the 446, the table and the signature read in OpenBSD's `disklabel.h`; the address and the order unverified)
4. The MBR code finds the **active** partition in the four-entry table and loads its first sector, the **volume boot record**; OpenBSD's `mbr.s` does this and `biosboot` is the record. (read)
5. That program, using BIOS `INT 13h` for every disk read, loads the next stage, `/boot`, which loads the kernel. (read: `biosboot.s`)
6. The kernel gets the memory map and the machine's description from the loader and from the BIOS. (unverified)

(e) The GPT and UEFI boot (4.2, 4.9; OpenBSD's `efiboot` read).

1. The firmware starts (at the reset vector as above, through phases of its own, unverified) and runs its **boot manager**, which reads the **boot variables** from non-volatile memory (**NVRAM**) to learn which loader to run and from where. (unverified; the loaders read and write variables: `GetVariable`, `SetVariable`, read)
2. The firmware reads the GPT, finds the **EFI system partition** (type `GPT_UUID_EFI_SYSTEM`, read) and its **FAT** file system, and opens the loader file by name.
3. The loader is a **PE/COFF** application (read: `start.S` of the arm64 loader); the firmware loads and runs it with a table of boot services.
4. The loader calls boot services (memory, files, console, graphics: read, counted in 4.9), reads its configuration, and picks up the **memory map** and the ACPI or device-tree table (read: `GetMemoryMap`, `EFI_ACPI_TABLE_GUID`, `FDT_TABLE_GUID`).
5. It loads the kernel and calls **exit boot services** with the map's key (read, with the retry), after which the firmware's boot services are gone.
6. The kernel runs with the memory map and the tables. With **Secure Boot** each step above is a signed image checked against keys in the firmware (unverified).

(f) The pdp64 boot (4.3 to 4.10, a proposal and a decision, no code). The ROM in the controller chip runs on core 0, reads block 0 through a virtio-blk request, checks `0x55AA`, and starts core 0 at it with `r0`, `r1` and `r14` set (4.4); the first stage, in the protective MBR's 446 bytes, reads the blocks on its list (4.5); the second stage reads the GPT, the label and the root file system, fills in the boot-information block and loads `/bsd` (4.6); the kernel enumerates the bus and starts the other cores by register writes (4.7).

The six side by side. A dagger marks a cell that rests on general knowledge, unverified; the last column says which of the five each property resembles.

Property	(a) PDP-11	(b) VAX	(c) C64	(d) x86 BIOS, MBR	(e) GPT, UEFI	(f) pdp64, and which it resembles
Who runs first	the processor, then the operator's keys or a ROM	the console processor†	the 6510 from its reset vector†	the processor at `0xFFFFFFF0`†	the same, then the firmware†	the ROM on core 0: like (a) and (b), without the operator
Where the first code lives	front panel or boot ROM	console ROM and storage†	the KERNAL ROM†	firmware flash†	firmware flash†	a ROM in the controller chip: like (a) and (c)

Property	(a) PDP-11	(b) VAX	(c) C64	(d) x86 BIOS, MBR	(e) GPT, UEFI	(f) pdp64, and which it resembles
How big the first code may be	a few dozen words (keys), a ROM's size; block 0 of 512 bytes	the boot block; the 750 loaded 7.5 KB next	the ROM's 8 KB†	446 bytes in the MBR (read)	the loader file, any size†	the ROM's size; 446 bytes in block 0: like (d)
How it finds the next stage	block 0 reads a name (`boot`) by walking inodes; `boot` reads `hp(0,0)unix` (read)	the boot block loads sectors 1 to 15; `boot` opens a name (read)	it does not: `LOAD` by the user†	the active flag in the table, then `INT 13h` reads (read)	the boot variables and the GPT's type, then a name on FAT†	a block list in block 0, then the GPT, the label and a name: like (b) and (a)
What the hand-off carries	registers `r0` to `r3` (read)	`r10`, `r11`, and others (read)	nothing: the program owns the machine†	registers and the BIOS's memory map†	the memory map and ACPI or a device tree (read)	`r0`, `r1` and the boot-information block: like (a) in registers, like (e) in a description
How it is repaired	keys, `ODT`, the backup `/boot.bu` (read for the last)	the console†	the ROM cannot be damaged by storage; a cartridge†	re-write the MBR from another disk†	the firmware's setup and boot manager†	the monitor, a core reset, a watchdog (4.10): like (a); a damaged GPT does not stop the first stage (4.5)
What is open source	the BSD and V7 stages (the sources here); ROMs†	the BSD stages (read); the console†	the ROMs are Commodore's†	the loaders, in OpenBSD (read); the BIOS is a vendor's†	the specification is published†; EDK II is BSD-2-Clause-Patent (read, 4.9); shipped firmware often not†	everything, by decision (4.3.2): beyond all five
How many stages	3 (ROM or keys, block 0, `boot`) before the kernel	3 (console, boot block, `boot`)	1 (ROM) or 2 with a cartridge†	4 or more (BIOS, MBR, boot record, `/boot`) (read)	3 or more, with firmware phases†	3 (ROM, block 0, second stage): like (a) and (b)
What recovery exists	`ODT` and the front panel†	the console†	reset, a cartridge†	the BIOS setup, a rescue disk†	the setup, rescue media, a fallback loader name†	the monitor and the watchdog (4.10), the same disk readable by a UEFI route (4.9)

The pdp64's boot is, property by property, the PDP-11's and the VAX's: a ROM, a first block, a second stage that finds a file by name, a hand-off in registers. It takes three things from the PC: the protective MBR and its 446 bytes (d), the GPT (e), and, in the boot-information block, a description of the machine (e) that the PDP-11 and the VAX did not pass. It takes from the C64 the ROM that is the first code and cannot be damaged by what is on the storage, and nothing else: the C64 had no second stage, and its user loaded programs by hand. And it differs from all five in that every stage is open source and in that no stage calls a firmware after the ROM.

4.12 What was checked, and what was not

The rule of the project is: check, do not guess. For the facts of this chapter about older and other machines, the handbooks and the specifications are not on this machine. What was available, and what each fact rests on:

- **Checked against the Unix sources on this machine** (`/home/david/claude/facetlang/unixtree`): the V7M `boot(8)` page and `sys/mdec/hpuboot.s` (285 lines: its header, `core = 28.`, the copy up, the clear, the `#` prompt, the pathname read, the inode walk from inode 2, the call `jsr pc,*$0`); `sys/sas/boot.c` (272 lines) and the `Boot` and `:` prompts and `device(unit,offset)` names of the page; the V7M tape bootstrap's mention of the M9312 (`sas/htboot.s`, `dev/ts.c`); V7 `hp.c` and `hp_sizes[8]`; the 2.11BSD `boot(8)` page (the 512-byte limit, block zero, the disklabel in block one, `/mdec`) and the header of its `hpuboot.s` (the 512 and 494 bytes); the registers of its `pdpstand/srt0.s` and `pdpstand/boot.c` (`r0` cputype, `r1` csr, `r3` boot device, `r2` the check word); the 4.3BSD-Tahoe `disklabel.h` (the block 0 or 1 comment, `0x82564557`).
- **Checked against the OpenBSD sources on this machine** (`/home/david/claude/kernel-src/openbsd`, commit `2114d7b7`): `sys/sys/disklabel.h` (the MBR: `DOSPARTOFF` 446, four `dos_partition` entries of a flag, a type, a start and a size, the signature at `0x1fe`,

types 0xee, 0xef, 0xa6; the GPT: "EFI PART", gpt_header, gpt_partition, 128 entries of 128 bytes by default, little-endian, the two GUIDs; DISKMAGIC, MAXPARTITIONS16; the sector for the label); sys/arch/amd64/stand (mbr/mbr.S, 521 lines, finds the first active partition; biosboot/biosboot.S, 862 lines, "roughly 512 bytes of code", calls INT 0x13, patched by installboot; boot/, 189 lines; libsa, 6,319 lines; efiboot, 4,593 lines); sys/arch/arm64/stand/efiboot (7,195 lines, start.S of 176 lines with the PE32+ header and IMAGE_FILE_MACHINE_ARM64 0xaa64, self_reloc.c of 124 lines) and riscv64 (4,513 lines); the counts of the services called by the three efiboot loaders; the ExitBootServices retry; the seven architecture headers of the label sector. The sparc installboot(8) page (the hard-coded block numbers) is in the OpenBSD 4.6 tree under /home/david/claude/facetlang/unixtree/OpenBSD-4.6. The Linux and other counts of 4.7 are those of the boot review (boot.md), measured there in Linux 7be4bc52.

- **Checked for section 4.11:** the 4.3BSD-Tahoe usr/src/sys/mdec/raboot.c (registers, the 7.5 KB, 0x50000, BOOTLAST), usr/src/sys/vaxstand/boot.c (r10, r11, Boot, :, the default vmunix) and usr/src/sys/vax/rpb.h (the comment), under /home/david/claude/facetlang/unixtree; the EDK II licence, BSD-2-Clause-Patent, in qemu-src/pc-bios/edk2-licenses.txt and /usr/share/doc/ovmf/copyright; the Secure-Boot image names in /usr/share/OVMF; OpenBSD's biosboot.S, mbr.S and efiboot for the x86 and UEFI steps marked read. Nothing about the Commodore 64's boot is in the repository: every statement of (c) is general knowledge, unverified, as are the VAX console and VMB, the x86 reset vector and the load address 0x7C00, the UEFI boot manager and NVRAM variables, Secure Boot, and the publication terms of the UEFI specification.
- **Checked on this machine:** the QEMU tree carries EDK II's built firmware images (pc-bios/edk2-x86_64-code.fd.bz2 is 3,653,632 bytes uncompressed) and no source of any UEFI implementation (the edk2 and u-boot directories of roms/ are empty).
- **Checked against the repository:** the statements of this chapter against docs/errata.md (47, 48, 50, 51, 95, 101, 102, 103, 104, 109), docs/boot.md, docs/decisions-pending.md (B2 as R2, P7, F10), chapters 2, 5, 6 and 7 of the manual, and docs/pdp64-system-architecture.md section 9.
- **Unverified** (general knowledge, the specifications and handbooks not being on this machine): the front panel's switches and keys and the bootstrap loader of a few words; that the M9312 held one ROM program per device; what ODT does, and its name; the date of the IBM PC and of UEFI's arrival; the BIOS's four jobs and its other services (video, keyboard, the memory map); real mode and the 1 MB limit; that the PC's x86 start-up signals are INIT and two START-UP signals carrying an address below 1 MB; that the first stage is loaded at a fixed address by the BIOS; the Commodore 64 boot of 4.11 (reset vector 0xFFFC, KERNAL, CBM80 at 0x8000, BASIC, the datasette and the 1541); the VAX console and VMB; the usual meaning of PSCI, the spin table and the SBI hart calls; that the VAX console processor started the other processors; the Alpha and VAX hand-off blocks; that the UEFI specification requires PE32+ images, FAT on the system partition, the services named, and the use of ACPI or a device tree; that a machine-type code for a new processor would be assigned by the owners of the specification; that Linux's request_firmware and OpenBSD's firmload(9) work as described; U-Boot's and OpenSBI's consoles.
- **Decided by the author and recorded in errata.md:** everything stated as a decision with a tag *(D)*; the pseudocode of this chapter is an illustration, and nothing in it is a decision.

4.13 Still open

Decisions the author has asked for, or that follow from the author's, and that are not made: (a) the address block 0 is loaded to (0x1000 is the example; errata 102(a)) and how many pages "the first few" are; (b) how the first stage reads blocks: a small reader of its own in 446 bytes, a routine the ROM leaves, or a larger first stage loaded by the ROM (4.5); (c) how the ROM finds and maps the boot device's windows before any software has (4.4); (d) who fills which fields of the boot-information block: the ROM, the second stage, or both (4.4, 4.6); its layout, order and widths (4.6.3); (e) the partition type of the boot partition, and whether a dedicated boot partition is needed at all, since the first stage's block list can point at the second-stage file in the root file system or at the 8 KB boot area (4.5); (f) the device-tree option: whether the boot-information block is replaced or accompanied by a device tree if the OpenBSD arm64 and riscv64 drivers are reused (4.6); (g) the registers of the core start block, whether a started core begins with translation off, and the difference between 4.7.1 (r0 and r1) and 7.8.1 (r0) (4.7); (h) how a virtio device is given firmware, and whether the INIT and READY words of 4.8.1 are to be restated over virtio (4.8); (i) the order of the other boot media after the virtio-blk disk (F10); (j) the monitor's commands, the serial speed and what is kept in non-volatile memory (boot order, serial speed: boot.md); (k) the shim: the path of the loader it runs, the services it offers, the machine-type number (4.9), none of which is to be settled until someone wants one.

Facts still to check against their own documents: the PDP-11 power-up behaviour, the M9312 boot ROMs and ODT; the VAX and Alpha boot protocols and hand-off blocks; the x86 reset vector and START-UP rules; PSCI, the SBI hart calls and the spin table as specified; what the UEFI specification requires of a loader and of firmware, including the machine-type registry; the size of a UEFI services layer, which only an implementation could give.

Chapter 5 — The device driver model

A driver on the pdp64 is a short ordinary program: it maps a device's windows, negotiates features and sets up queues through the virtio transport, hands the device descriptors in memory and finds them marked used, and because every device speaks virtio the same transport code serves a network card, a display and a sound device, each with a thin class driver on top. The chapter gives the driver writer's view of the memory-mapped registers and of DMA: what a driver sees, the virtio transport and its queue, the life of a driver from probe to detach, how data moves and what the driver need not do, the default devices, how errors and departures are handled, interrupts, and how this maps onto OpenBSD's driver interfaces. The bus the devices sit on, and how they are found, is chapter 2. Tags: *(D)* decided; *(P)* proposed; *(S)* unspecified.

5.1 What a driver sees

5.1.1 A device is a PCI Express function whose windows (the regions its base-address registers describe) a driver maps with the device memory type, strongly ordered. *(D for the memory types; P for windows that start on page boundaries and are whole pages)*

5.1.2 A driver reaches a register with an ordinary load or store; there are no input or output instructions and no port space. *(D)*

5.1.3 Registers are naturally aligned and accessed at their own width; the machine is little-endian throughout and the virtio structures are little-endian, so a driver has no byte-order or byte-lane cases. *(D)*

5.1.4 Accesses to a device page happen in program order; one `FENCE` separates the last write to memory the device will read from the write that notifies the device. *(D)*

Commentary. **Precedent.** A V7 driver is a structure laid over a fixed address: `rk.c` declares `struct device { int rkds; int rker; int rkcs; int rkwc; caddr_t rkba; int rkda; }` and a pointer to it at `0177400`, and reads and writes the fields. **Trade-offs.** A window of whole pages wastes address space and no memory, and lets the kernel give one device to one protection domain. **Improvements.** The register's address is a virtual address the kernel maps, so the driver is position-independent. **Benefits.** There is no accessor layer to learn and nothing to get wrong with endianness or alignment. **Modern comparables.** Linux and BSD wrap register access in an accessor (`readl`, `bus_space_read_8`) to hide port and memory differences and byte order; on the pdp64 those wrappers reduce to a load or a store. **Impact.** *Kernel:* No accessor layer: a register access is a load or a store. *Compiler:* Register access is a `volatile` load or store of the register's width through a pointer to a structure. *Hardware:* A device window is decoded by page address.

5.2 The device interface: virtio

5.2.1 Every standard device is a virtio device or a USB (xHCI) controller; the disk, network, graphics and audio devices are virtio-blk, virtio-net, virtio-gpu (2D) and virtio-snd, over the virtio PCI Express transport, so that one transport driver and a thin class driver per device serve the system and the same devices run in QEMU, `sim64` and the FPGA. *(D)* Any other PCI Express device, an NVMe controller for example, may be attached with its own driver; it is not a standard device. *(D)*

5.2.2 The driver finds a virtio device's structures through the capabilities in its configuration space: the common configuration, the notification area, the interrupt status and the device-specific configuration, each at an offset in one of the device's windows. *(D)* The common configuration has these fields, as the virtio specification lays them out:

Offset	Field	Meaning
`0x00`	`device_feature_select`	which 32 bits of the device's features `device_feature` shows
`0x04`	`device_feature`	the device's feature bits (read-only)
`0x08`	`driver_feature_select`	which 32 bits of the driver's features `driver_feature` sets
`0x0C`	`driver_feature`	the features the driver accepts
`0x10`	`config_msix_vector`	the interrupt vector for a change of configuration

Offset	Field	Meaning
0x12	num_queues	how many queues the device has (read-only)
0x14	device_status	the handshake bits; writing zero resets the device
0x15	config_generation	changes when the device-specific configuration changes
0x16	queue_select	which queue the next fields describe
0x18	queue_size	the number of entries of the selected queue
0x1A	queue_msix_vector	the interrupt vector of the selected queue
0x1C	queue_enable	the driver sets it to 1 to start the queue
0x1E	queue_notify_off	where, in the notification area, the driver notifies this queue
0x20	queue_desc	physical address of the descriptor table
0x28	queue_driver	physical address of the available ring
0x30	queue_device	physical address of the used ring

5.2.3 A transfer is a chain of descriptors in a queue (a virtqueue) in host memory. A descriptor is 16 bytes: a 64-bit buffer address, a 32-bit length, 16 bits of flags (NEXT: the chain continues; WRITE: the device writes the buffer; INDIRECT: the buffer is a table of descriptors) and a 16-bit index of the next descriptor. *(D)*

```
descriptor:  ADDRESS(64)  LENGTH(32)  FLAGS(16)  NEXT(16)
available ring (the driver writes):  flags, idx, ring[ ] of descriptor-chain heads
used ring (the device writes):       flags, idx, ring[ ] of (head id, bytes written)
```

5.2.4 The driver puts the head of a chain in the available ring and advances its index, executes FENCE, and writes the queue's notification address; the device carries the chain out and writes the head and the byte count in the used ring, advances the used index, and may interrupt. Progress is therefore in memory, not in the device. *(D)*

5.2.5 The driver and device agree on features before any queue runs: the driver reads the device's features, writes the subset it accepts, and the device confirms it; the version-1 feature is required. *(D)* The feature subset that the pdp64 accepts for each device, and the platform-ordering feature that the pdp64 must declare, are not specified. *(S)*

5.2.6 A device that carries traffic in two directions (a network card) has one queue for each direction in the one device; a console-like device is the same pattern. *(D)*

5.2.7 Storage is virtio-blk: a virtqueue of requests, each a header (read, write or flush, and the sector), the data buffers and a status byte, the device marking the request done in the used ring. *(D)* An NVMe controller, which keeps a submission queue and a completion queue in host memory with a doorbell register for each (the shape of the virtqueue with a different command encoding), may be attached as any PCI Express device may; it is not the standard disk. *(D)*

Commentary. **Precedent.** The V7 RK11 disk is six registers (drive status, error, control and status, word count, bus address, disk address) with DMA by a word count and a bus address; the 4.3BSD DEUNA Ethernet is four registers plus transmit and receive rings in memory, each ring entry holding a segment length, an address, flags and errors, with an OWN bit that is zero when the descriptor belongs to the driver. The virtqueue is the second design, standardised: the done mark is the used ring instead of a bit in the descriptor. **Trade-offs.** A queue per direction keeps each ring in order and simple; the used ring costs one write per chain. Taking a standard interface rather than a design of the pdp64's own gives up the freedom to shape the registers and gains every driver and every emulated device that already exists. **Improvements.** Because progress lives in the rings and not in a device register, a driver can recover from a departed or reset device with one routine: scan the available ring for chains that are not in the used ring, then read, save, resend or fail them. **Benefits.** A driver is a skeleton shared by all devices. **Modern comparables.** `virtio` rings, NVMe's submission and completion queues, the descriptor-done bit of Intel Ethernet controllers and the command lists of SATA controllers are all rings with a done mark. **Impact.** *Kernel:* One transport driver and a thin driver per class. *Compiler:* The descriptor is a 16-byte aligned C structure, and register offsets are generated constants. *Hardware:* A device is a standard PCI Express function and the virtio interface is the specification's.

5.3 The life of a driver

5.3.1 *Probe.* The kernel enumerates the bus (section 2.4), reads each function's identifiers, and matches the virtio device type (or the USB class) against the drivers it has. *(D)*

5.3.2 *Attach.* The driver resets the device (zero to `device_status`), sets the acknowledge and driver bits, negotiates features, sets the features-ok bit and reads it back, sets up each queue (its size and the three addresses, then `queue_enable`), and sets the driver-ok bit. *(D)*

5.3.3 *Run.* The driver puts the queue in uncached memory, fills descriptors, publishes the chain in the available ring, executes `FENCE`, and notifies the queue; a chain is complete when it appears in the used ring, which the driver sees by polling or by the interrupt. *(D)*

5.3.4 *Fail.* A device that sets its needs-reset bit, or that stops answering, is reset and its queues reprogrammed, and the driver resubmits the chains that are not in the used ring. *(P for the resubmission rule; D for the reset)*

5.3.5 *Detach.* A departure event, or a bus fault from an absent device, tells the driver to stop; it reads, saves, resends or fails the chains still queued. *(P)*

Commentary. **Precedent.** A V7 disk driver is a strategy routine that queues a buffer, a start routine that writes the registers, and an interrupt routine that completes the buffer; 4.3BSD's Unibus drivers add a probe routine that tests whether a device answers at an address and an attach routine, which BSD's autoconfiguration calls in order. **Trade-offs.** The five steps are the same for every device, so the work that differs is the personality: what the requests mean. **Improvements.** Probe reads the bus and not a configuration file, and detach is a defined path because the bus reports departures. **Benefits.** A new device is a new set of requests in a driver that already has the skeleton. **Modern comparables.** OpenBSD's autoconfiguration calls a driver's `match`, `attach` and `detach` functions; Linux's driver model has `probe` and `remove`. **Impact.** *Kernel:* The five steps are the same for every driver, so only the requests differ. *Compiler:* Nothing special. *Hardware:* None beyond the device's own.

5.4 DMA and the hand-off

5.4.1 Buffers and queues are in memory mapped uncached, and the driver does no cache clean or invalidate. *(D)*

5.4.2 A device addresses physical memory directly and the driver gives it physical addresses; until an address-translation unit for devices exists, every device must be able to address all of memory. *(P)*

5.4.3 A buffer that crosses a page is described by several descriptors chained with the `NEXT` flag; the driver does not make buffers contiguous. *(D)*

5.4.4 The device writes the data, then the used-ring entry, then advances the used index; the driver reads the used index and then the data. *(D)*

Commentary. **Precedent.** The Unibus carried 18 address bits, so the VAX and the larger PDP-11s had map registers (`ubalonce` in 4.3BSD) translating device addresses to memory, and the VAX's adapter had data paths a driver had to purge before reading; the 11/70's cache watched DMA writes so that the data appeared coherent. **Trade-offs.** Uncached buffers make the processor's reads of received data slow, in return for a driver that cannot forget a cache operation; a non-coherent device that came later would call for cache-maintenance instructions. No address-translation unit means no protection from a faulty or malicious device. **Benefits.** The sequence build, `FENCE`, notify, complete is the whole of the hand-off. **Modern comparables.** ARM and MIPS systems with non-coherent DMA need a DMA-mapping layer that places cache maintenance; x86 DMA is coherent; `swiotlb` in Linux copies for devices that cannot address all memory. **Impact.** *Kernel:* No cache clean or invalidate in any driver, so none can be forgotten. *Compiler:* None. *Hardware:* DMA goes around the processor's data cache, so the cache needs no snoop port for devices.

5.5 The default devices

5.5.1 The **disk** is virtio-blk (the author's decision of 2026-11-10, errata 101) with read, write and flush at 512-byte sectors. *(D)* The boot ROM reads block 0 with one read request on the device's queue (4.4). *(P)*

5.5.2 The **network card** is virtio-net: a receive queue the host fills with buffers for the device and a transmit queue of frames to send; the device completes each receive buffer with the frame length. *(D)*

5.5.3 The **graphics device** is virtio-gpu in its 2D mode (create a surface, point it at memory, flush a rectangle) and the **audio device** is virtio-snd (streams in queues in shared memory). *(D)*

5.5.4 USB is an xHCI host controller, a PCI Express function, driven by the operating system's USB stack, which enumerates the attached devices. *(P)*

5.5.5 The form of the console, the timer and the entropy source under this interface is not specified: the candidates are a UART-style console, a small fixed timer and interrupt controller that need no enumeration, and virtio's console and entropy devices; the entropy pool is one device shared by all cores. *(S)*

Commentary. **Precedent.** The PDP-11 had a disk with eight registers, a serial line with four, and network controllers with rings. **Trade-offs.** One transport for network, graphics and audio means one driver, and the cost is the feature negotiation

and the queue format of a standard that the pdp64 does not control. An existing USB stack comes with xHCI, at the price of that large stack. **Improvements.** A card of any vendor that speaks the standard works with the drivers already written. **Benefits.** OpenBSD's tree has drivers for several virtio devices (`if_vio`, `vioblk`, `viornd`, `viogpu`, `viocon`) under `sys/dev/pv`, by the listing recorded in `security-proposals.md`; whether it has one for virtio-snd is open. **Modern comparables.** xHCI is the conventional USB controller; a USB-to-Ethernet adapter appears to the host as a network class device. **Impact.** *Kernel:* One transport driver, a thin driver per class. *Compiler:* None. *Hardware:* Standard PCI Express devices; the same virtio devices in the emulator, the simulator and the FPGA.

5.6 Errors, departure and recovery

5.6.1 A request that fails completes with the status of its device class (for a virtio-blk request, a status byte in the last descriptor of its chain); a device that is broken sets its needs-reset bit in `device_status`. *(D)*

5.6.2 A device that is reset leaves its queues in memory untouched; the driver reprograms the queue registers and resubmits the chains that are not in the used ring. *(P)*

5.6.3 A device that leaves the bus leaves its queued work in memory; the driver finds it by comparing the available ring with the used ring. *(P)*

5.6.4 Requests are repeatable where possible (reads, writes of the same data, sends, buffer posts), so that resending an unfinished chain is safe; a request that must not run twice is tracked by its class driver, which can tell "never started" from "started, outcome unknown" only by the request's own status. *(P)*

Commentary. **Precedent.** 4.3BSD's `ubareset()` resets the Unibus adapter and then calls each driver's reset routine, so each driver re-issues what was pending: resending as a design, driver by driver. **Trade-offs.** The cost is a write to the used ring per chain and the ordering rule between data and the used index. **Improvements.** One recovery routine serves a device that fails and is reset, a device replaced by another, and, with a saved queue, a hand-over between cores. **Benefits.** This is the recoverability principle in the driver: fewer failure outcomes (the precise bus fault, not all ones on a removed device) and one path to recover from any of them. **Modern comparables.** NVMe and virtio drivers requeue outstanding requests after a reset; PCI returns all ones for a read of a removed device. **Impact.** *Kernel:* One recovery routine handles a failed, reset, replaced or departed device. *Compiler:* None. *Hardware:* The device writes data before the used entry before the used index.

5.7 Interrupts from devices

5.7.1 A device interrupts by a message-signalled write (MSI-X): the configuration change and each queue have a vector that the driver sets in the common configuration, and the interrupt status structure says why. *(P)*

5.7.2 A driver may mask a device's interrupts and poll the used ring instead. *(D)*

5.7.3 A changeable interrupt target is changed with its vector masked and then read back, or is a single atomic register write. *(D)*

5.7.4 The write that raises an interrupt cannot overtake the data writes that precede it (section 2.5.4). *(D)*

Commentary. **Precedent.** A PDP-11 device asserted one of four bus-request lines and supplied its own vector, set by switches on the card. **Trade-offs.** A message carries its own vector, so no switch and no assignment table are needed; the controller and routing are an open decision (R1). **Benefits.** A driver's interrupt routine reads the used ring to learn what completed. **Modern comparables.** MSI-X is how PCI Express devices interrupt. **Impact.** *Kernel:* An interrupt routine reads the used ring and the status structure. *Compiler:* None. *Hardware:* An MSI-X target in the interrupt controller, and the eight-level priority encoder.

5.8 Drivers in user space, and in OpenBSD

5.8.1 Because a device's windows are whole pages, a driver can run in user space with only those pages and its buffers mapped. *(P)*

5.8.2 Until devices have an address-translation unit, a user-space driver can instruct its device to write anywhere in memory, so it is as trusted as the device. *(P)*

5.8.3 In OpenBSD the register accessors become loads and stores, the DMA mapping interface returns physical addresses with no cache-synchronisation work to do, and the autoconfiguration framework attaches a driver to each function the bus reports. *(P)*

Commentary. **Precedent.** Microkernels and exokernels run drivers in user space where the hardware lets one device be mapped to one process; the PDP-11's Unibus gave every device a fixed address and no protection. **Trade-offs.** Without a translation unit the mapping protects against stray register writes but not against DMA. **Benefits.** A driver can be tested, restarted and replaced without taking the kernel with it. **Modern comparables.** Linux's `VFIO` and `UIO` and DPDK run drivers in user space, relying on an I/O translation unit for protection. **Impact.** *Kernel:* The accessors reduce to loads and stores, the DMA interface to physical addresses, and attach to the functions the bus reports. *Compiler:* None. *Hardware:* Windows of whole pages let the translation unit protect a device.

5.9 Still open

The feature subset of each virtio device and the platform-ordering feature, the form of the console, the timer and the entropy source, the feature subset of virtio-blk (flush, block size, ordering), DMA addressing and protection (R6), errors raised by devices (R7), and the details of interrupt routing (R1). The working documents are `../decisions-pending.md` and `../pdp64-q64-bus.md`, and appendix B.

Chapter 6 — The kernel: modes, exceptions and memory management

An operating system on the pdp64 sees three things: a status word with a mode and a priority, an exception mechanism that writes only registers, and a page table of three, four or five levels with software-managed accessed and dirty bits. This chapter is written for a reader who has written or read the kernel of Unix Version 6 or 7 on the PDP-11, or who is learning computing from the start, and who has not met anything that was invented after the 1970s: it begins with what that kernel did (the status word, the vectors, `trap()`, the memory-management unit with its eight page registers per mode, `mfpi` and `mtpi`, swapping), explains each modern idea by contrast with it (precise exceptions, system registers, page tables, the translation buffer, address-space numbers, direct-mapping windows), walks a system call, a page fault, a timer interrupt, a context switch and the idle loop through in pseudocode, and gives the decisions as numbered statements along the way. It is the design called System v1; the options and the comparisons with other architectures are in `../pdp64-system-architecture.md` (section 15) and `../pdp64-system-design.md`. Tags: *(D)* decided; *(P)* proposed; *(S)* unspecified; a fact about an older or another machine that could not be checked on this computer is marked *unverified* and listed in section 6.14.

How to read this chapter. A term is defined where it first appears, as the technical word followed by a plain restatement. Boxes-and-arrows pictures are the kind a textbook draws for a linked list: a box is a piece of memory or a register, an arrow is an address stored in it. Code blocks are pseudocode (C-like, `#` starts a comment) unless the text says they are PDP-11 assembly or pdp64 assembly; the PDP-11 code is quoted from the V7 and 2.11BSD kernels on this machine (section 6.14), and the pdp64 code is **an illustration written for this chapter**: no pdp64 kernel exists yet (the first operating system is OpenBSD, 6.12.3, and nothing of it is written), so none of it is tested. The decisions are the numbered statements; the rest is explanation, and where the repository does not say, the text says *not specified*.

6.1 What you already know: the kernel of a PDP-11

Think of Unix Version 7 (or 2.11BSD, which is the same kernel grown up) on a PDP-11/45, 70 or 73. Six things about it matter here, because every modern idea in this chapter is one of them, kept or replaced.

The status word and the modes. A PDP-11 has a **processor status word** (PS, also called PSW: one 16-bit register holding the flags, the interrupt priority and the mode of the running program; it sits at the fixed address 0177776, section 6.14). Bits 15-14 are the **current mode** (which privilege the program runs with: kernel, supervisor or user), bits 13-12 the **previous mode** (the mode that was running before the last trap), bits 7-5 the **priority** (the level below which interrupts are held back), bit 4 the **trace** bit (take a trap after every instruction), and bits 3-0 the flags N, Z, V and C. The 2.11BSD header says the same in C (`psl.h`: `PSL_CURMOD` 0xc000, `PSL_PRVMOD` 0x3000, `PSL_IPL` 0xe0, `PSL_T` 0x10, the condition codes in 0xf). In user mode the program may not execute the privileged instructions (`halt`, `reset`, `wait`, a write to the PS priority); each mode has its **own stack pointer** (R6 means a different register in each mode), so a trap that arrives in user mode finds a kernel stack waiting.

Vectors in low memory. A **trap** (the machine transferring control to the kernel because something happened) goes through a **vector**: two words at a fixed low address, the new program counter and the new PS. The hardware pushes the old PS and the old PC on the new mode's stack and loads the pair. This is the V7 table, `l.s` (the octal addresses are added here):

```
        . = ZERO+4
        trap;  br7+0.          / 004  bus error
        trap;  br7+1.          / 010  illegal instruction
        trap;  br7+2.          / 014  bpt, trace
        trap;  br7+3.          / 020  iot
        trap;  br7+4.          / 024  power fail
        trap;  br7+5.          / 030  emulator trap (emt)
        start; br7+6.          / 034  "sys": the system call
        . = ZERO+60
```

```
klin;  br4              / 060  console keyboard
klou;  br4              / 064  console printer
. = ZERO+100
kwlp;  br6              / 100  line clock
```

The first word of each pair is the address of the handler; the second is the new PS, which is `br7` (priority 7: nothing may interrupt) plus a small number that the handler passes on to `trap()` to say which trap it was. The device vectors (the console at 060 and 064, the clock at 0100) carry the device's priority, 4 and 6. A handler ends with `rti` (return from interrupt: pop the PC and PS) or `rtt` (the same, but a pending trace trap is not taken at the first instruction).

`trap()`. The assembly code at the vector (`call` in the V7 `mch` file) saves the registers and calls one C function, `trap(dev, sp, r1, nps, r0, pc, ps)`: its arguments, in the words of the V7 comment, are "the words saved on the system stack by the hardware and software during the trap processing". A system call is one of these traps: the user program executes `sys n` (the instruction `0104400` plus the call number in its low six bits), and `trap()` fetches that instruction back from user memory and uses it as an index (`callp = &sysent[fuiword(pc-1)&077]`).

The memory-management unit. A PDP-11 program uses 16-bit addresses (64 KB), but the physical memory is bigger (18 bits, 256 KB, on the machines of the Unibus; 22 bits on the Q-bus machines and the later ones, section 6.14). The **KT11** (the PDP-11's memory-management unit: the hardware between the processor and memory that turns the program's addresses into memory addresses, a process called **translation**) does it with registers. It has, for each mode, **eight pairs** of **page registers**: a **PAR** (page address register: where the segment starts in physical memory, in units of 64 bytes) and a **PDR** (page descriptor register: how long it is, up to 8 KB, and whether it may be read or written). The top three bits of a program's address pick one of the eight pairs; the other thirteen are the offset into that 8 KB piece. The kernel names the user's registers `UISA` and `UISD` (at 0177640 and 0177600) and has a whole set of its own (`KISA0` at 0172340); with **separate instruction and data space** (a model in which code and data have separate 64 KB address spaces) there are sixteen pairs, eight for each. The V7 function `sureg()` copies a process's eight (or sixteen) saved pairs into the hardware registers at every context switch, and the function `estabur()` fills in the saved pairs from the sizes of the program's text, data and stack, and gives up with `ENOMEM` if they need more than eight segments. A page register describes **base and length** (a start and a size, the oldest form of memory protection); a program's text, data and stack are each one **contiguous** run of memory, handed out by a first-fit allocator on a map (`malloc.c`: "the core map unit is 64 bytes").

Reaching the other space. The kernel's own address space is separate from the user's, so a kernel pointer to user memory means nothing. The instructions `mfpi` and `mtpi` (move from and to the **previous** instruction space; `mfpd` and `mtpd` for data space) take the address from the kernel's registers but use the previous mode's page registers; the copy-in routines of 2.11BSD (`fubyte`, `subyte`) are built from them, with a global named `nofault` set so that an error in the access returns -1 instead of panicking.

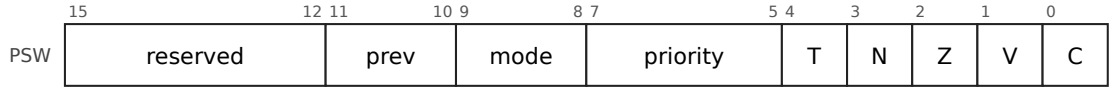
Swapping. A process image was one block of memory; when memory ran short, the whole image was written out to the **swap device** (`swapout()`, called from the scheduler loop `sched()`), and read back whole to run again. There was no concept of part of a program being in memory.

What did not change in the pdp64, and what is new. The pdp64 keeps three modes with a status word you would recognise, a stack pointer per mode, vectors, `RTI`, a priority that the kernel raises and lowers, a previous-mode notion, and an idle instruction. What is new is eight things, each explained where it first matters: the status word and trap entry are stored in **registers** and not pushed on a stack (6.3); an exception is **precise** (6.4); the control registers are not at fixed addresses but are reached by number with `MFSR` and `MTSR` (6.5); the eight page registers become a **tree of page tables in memory** that the hardware **walks** (6.6, 6.7); the answers of the walk are cached in a **translation buffer** that must be kept honest (6.8); that buffer is **tagged with an address-space number** so that a context switch need not empty it (6.8); the upper half of the address space is the kernel's and is reached through its own table, with **direct-mapping windows** for memory and devices (6.9); and `R7` is an ordinary register (errata 100): the PC is not a register at all, and R6 is replaced by `R14`.

6.2 Modes and the status word

The PDP-11's three modes are kept, with the same values: kernel 0, supervisor 1, user 3 (2 is reserved, and loading it into the status word is an illegal instruction). Their positions in the word moved down to make room, and one bit was added. The two layouts side by side:

	PDP-11 (`psl.h`)	pdp64
flags (four bits)	3-0	3-0
trace	4	4
priority	7-5	7-5
current mode	15-14	9-8
previous mode	13-12	11-10
user-memory guard	none	12



The **user-memory guard** (a status bit that says whether the kernel may touch user memory by accident: section 6.10) is the added bit. The **previous mode** exists for the same reason as on the PDP-11: after a trap from user mode it says where the kernel came from, and the instructions that act on behalf of the previous mode (`PROBER` and `PROBEW`, section 6.10) use it. A **privileged instruction** (one that only the kernel may execute, because it could break the protection between programs) outside kernel mode **aborts** (is cancelled before it changes anything, section 6.4) with the privileged-instruction vector, as `halt` and `reset` did on the PDP-11.

6.2.1 There are three modes, with the PDP-11's encoding: kernel 0, supervisor 1, user 3. The supervisor mode exists in the status word and in the banked registers, and no code uses it. *(D)*

6.2.2 The status word holds the flags in bits 0 to 3, the trace bit in bit 4, the interrupt priority in bits 5 to 7, the current mode in bits 8 and 9, the previous mode in bits 10 and 11, and the user-access guard in bit 12; the other bits are zero. *(D)*

6.2.3 `MTPS` in user mode writes only the flags; in kernel mode it writes every writable bit. `SPL #n` is kernel-only. *(D)*

6.2.4 The stack pointer `R14` and the direct-page register `DP` each have one copy per mode; the mode selects which one an instruction names. *(D)*

6.2.5 These instructions abort with the privileged-instruction vector outside kernel mode: `RTI`, `RTT`, `HALT`, `SPL`, `MTASID`, `MFASID`, `TLBI`, `TLBIA`, `TLBSYNC`, `MTDMW`, `MFSR`, `MTSR`, `PROBER`, `PROBEW`, `WFE`, `ISYNC`, `STUA`, `CLUA`, `PUSHX`, `POPX`. *(D)*

6.2.6 These are not privileged: `MTDP` and `MFDP` (in user mode, the thread pointer), `ICINV` and `ICINVA` (a JIT needs them), `FENCE`, `SEV`, `HINT`, `TRAP`, `BPT`, `NOP`, and `MTPS` and `MFPS` with the restriction above. *(D)*

Commentary. **Precedent.** The PDP-11 had kernel, supervisor and user modes, with a status word whose priority and mode fields the pdp64 keeps, and a stack pointer per mode so that a trap arriving in user mode finds a kernel stack without searching for one. In practice the third mode was little used: the 2.11BSD kernel runs its network code in supervisor mode (its `trap()` tests `SUPVMODE(ps)`, checked), and V7 never did. **Trade-offs.** Three modes cost a third copy of two registers and no logic the kernel uses; supervisor mode has no user, and is included because adding a mode would change the status word and every saved copy of it. **Benefits.** Entry from user mode needs no stack switch code; a threaded program's pointer and a kernel's per-CPU pointer are the same register in different modes. `DP` is new to a PDP-11 reader: it is the **direct-page register** (a base register that makes one 4 KB-or-larger region addressable with a short displacement, descended from the 6502's zero page); the kernel uses its copy as a pointer to the structure of the core it is running on (the **per-CPU pointer**: which core am I, and where is its private data), and a user program uses its copy as the pointer to its thread's private data. **Modern comparables.** x86 has four rings of which two are used; AArch64 has exception levels 0 to 3; RISC-V has user, supervisor and machine modes. **Use.** `MFSR` and `MTSR` are the whole of system control (6.5); `PROBER` and `PROBEW` do the work of the `nofault` idiom of V6 and V7. **Impact.** *Kernel:* Banked stack pointers mean a trap from user mode needs no stack switch, and the direct-page copies give a per-CPU pointer in the kernel and a thread pointer in user mode. *Compiler:* User-mode thread-local storage is one `MOVQ d(DP), Rd`. *Hardware:* Three copies of two registers, selected by the mode, and a mode comparison on privileged decode.

6.3 Exceptions, interrupts and traps: how the machine enters the kernel

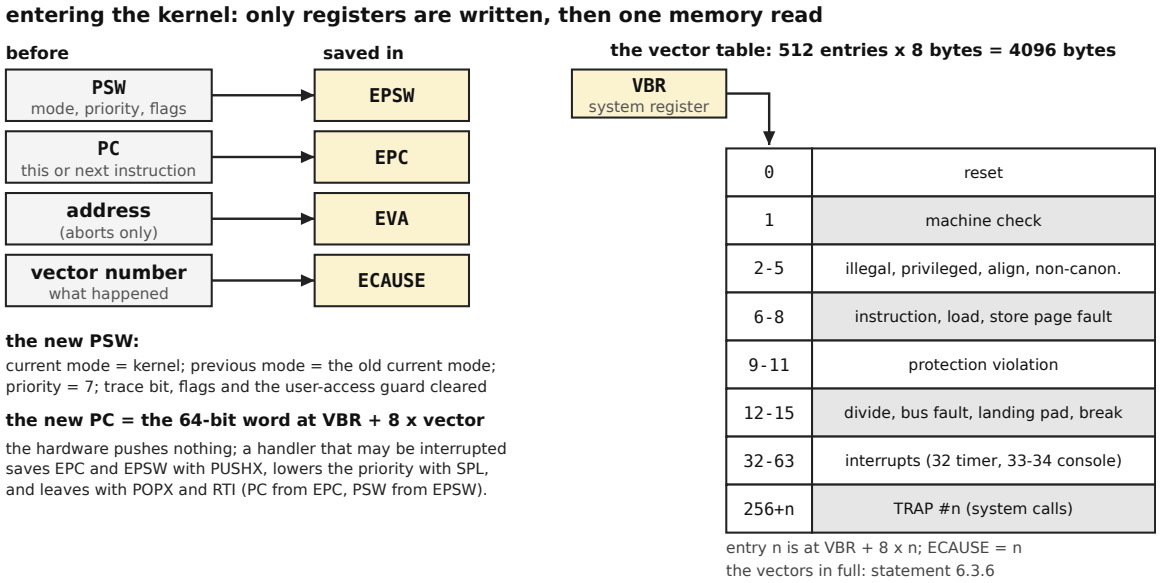
Three words are used for what the PDP-11 called by the single word "trap", and the pdp64 keeps them apart. An **exception** is the machine stopping an instruction because it cannot carry it out (a page that is not in memory, an illegal instruction, a divide by zero): it belongs to the instruction. An **interrupt** is a device asking for attention: it belongs to no instruction, and

is taken between two of them. A **trap**, in the narrow sense used here, is a deliberate entry by an instruction that succeeds (`TRAP #n`, the system call). All three enter the kernel the same way.

The PDP-11's way. The hardware read the new PC and PS from the vector, pushed the old pair on the kernel stack, and ran the handler. That is **entry through memory**: it writes two words to the stack before a single handler instruction runs.

The pdp64's way. Entry writes **only registers**, and then reads one word. Four **system registers** (registers that are not among the general ones and are reached with `MFSR` and `MTSR`, section 6.5) receive the facts: `EPC` (the exception program counter, where to resume), `EPSW` (the old status word), `EVA` (the exception virtual address: the address that caused a fault) and `ECAUSE` (what happened: the vector number). Then the new status word is made, and the new program counter is read from the **vector table**: an array of 8-byte entries, 512 of them, 4096 bytes in all, whose base is held in the register `VBR` (vector base register). In pseudocode, with `kind` saying which of the families of 6.4 this is:

```
enter(vector, kind):
    EPC    = (kind is abort or progress) ? address of this instruction
                                         : address of the next instruction
    EPSW   = PSW
    EVA    = (kind is abort) ? the faulting address : 0
    ECAUSE = vector
    PSW.previous_mode = PSW.current_mode
    PSW.current_mode  = kernel
    PSW.priority      = 7
    PSW.trace = 0 ; PSW.flags = 0 ; PSW.guard = 0        # the guard: section 6.10
    PC = load64_physical(VBR + 8 * vector)               # a bus error here is a machine check
```



Entry differs from the PDP-11's in three ways that a kernel writer feels. **It cannot fault**: writing registers cannot run into a page that is missing, which a push on a kernel stack could (a fault while entering a fault). **It leaves the handler to save what it needs**: the hardware pushes nothing, so a handler that may itself be interrupted saves `EPC` and `EPSW` first (one instruction, `PUSHX`) and a handler that cannot be interrupted need not save them at all. **It is the same for every cause**: the PDP-11 let each vector choose its own priority (V7's trap vectors say `br7` and its device vectors `br4`, `br6`); the pdp64 always enters at priority 7, and the handler lowers it. Because a second exception inside a handler, before it has saved `EPC` and `EPSW`, would overwrite them, the first instructions of every handler and the memory they touch must be such that they cannot fault: this is the reason the kernel's own text and stacks are in pages every address space shares or in a direct-mapping window (6.9), never in pages that may be absent. (That sentence is a consequence of the rules drawn by the author of this chapter; the architecture does not state it as a rule.)

The table. The entries are indexed by the number of the vector, and `ECAUSE` receives that number, so one handler can serve several vectors and tell them apart. They are laid out as follows (the first thirty-two are for the processor itself, thirty-two

to sixty-three for interrupts, and the last 256 are one for each `TRAP` number; compare V7, where every `sys n` shared the single vector at 034 and the number was fetched from the instruction):

6.3.1 Entry to an exception or an interrupt writes only registers: `EPC` receives the address of the faulting instruction (for an abort or a progress exception) or of the next instruction (for a trap or an interrupt), `EPSW` the old status word, `EVA` the faulting address (zero except for aborts), `ECAUSE` the vector number. *(D)*

6.3.2 The new status word has the current mode kernel, the previous mode the old current mode, priority 7, and the trace bit and the flags cleared. *(D)*

6.3.3 The new program counter is the 64-bit word at the physical address `VBR + 8 × vector`; a bus error on that fetch is a machine check that halts. *(D)*

6.3.4 The hardware pushes nothing. A handler that can be interrupted saves `EPC` and `EPSW` with `PUSHX`, lowers the priority with `SPL`, and before returning restores them with `POPX` and executes `RTI`. *(D)*

6.3.5 `RTI` loads the program counter from `EPC` and the status word from `EPSW`; `RTT` does the same and sets the trace bit after the return. *(D)*

6.3.6 The vector table has 512 entries and `ECAUSE` equals the vector number. *(D)*

Vectors	Meaning
0	reset (not used by the QEMU machine)
1	machine check
2, 3, 4, 5	illegal instruction, privileged instruction, misaligned access, non-canonical address
6, 7, 8	instruction, load and store page fault
9, 10, 11	protection violation on instruction, load and store
12, 13	divide by zero, bus fault
14, 15	landing-pad violation, breakpoint
16 to 31	reserved
32 to 63	interrupts (in the virt machine, 32 timer, 33 console input, 34 console output)
64 to 255	reserved (device interrupts are message-signalled, section 5.7)
256 + n	`TRAP #n` and `TRAPL #n`, n from 0 to 255

6.3.7 There are eight interrupt priority levels; the processor takes an interrupt between instructions when its level is greater than the priority in the status word; the timer is at level 6 and the console at level 4. *(D)*

(The two levels are the PDP-11's: V7 and 2.11BSD vector the line clock at priority 6 and the console at priority 4, checked in `scb.s`.) A **message-signalled interrupt** is a device that raises an interrupt by writing a word to a memory address, so that no interrupt wire is needed (section 2.7). How such a write becomes a vector number, and which core receives it, is **not specified** (2.7.3, the interrupt-controller decision R1 of appendix B); the vectors 64 to 255 are reserved for it.

Commentary. **Precedent.** The PDP-11 pushed the program counter and the status word on a stack and loaded new ones from a two-word vector, and its trap vectors conventionally held priority 7, so a PDP-11 handler began with interrupts masked and chose when to lower them (checked in V7 `l.s` and 2.11BSD `scb.s`: every trap vector is `br7+code`); its interrupts took the priority of their device from the vector. The VAX pushed the program counter and the processor status longword and raised the priority to the interrupt's level for an interrupt, leaving it unchanged for an exception. **Trade-offs.** Entering in registers is short and cannot itself fault (no memory is written), and costs the handler an explicit save; it needs the priority raised on entry, because a higher-level interrupt could otherwise arrive before the handler had stored the pair and overwrite it, which is why the entry sets priority 7 for every exception and interrupt, a fixed choice where the PDP-11 let each vector choose. **Improvements.** `PUSHX` and `POPX` make the save and restore one instruction each; the aborts are precise, so a page-fault handler repairs the mapping and executes `RTI` to run the instruction again. **Benefits.** The kernel survey estimates a system-call entry at four or five instructions on paper, against 45 to 70 on RISC kernels; no entry code is needed to find a stack. **Modern comparables.** AArch64 and RISC-V latch the return address and status in registers; x86 pushes five words on the stack; every one of them masks interrupts on entry. **Impact.** *Kernel:* Entry is four or five instructions on paper; priority 7 on entry removes the code that would otherwise guard the saved state. *Compiler:* Handlers written in C need a short assembler stub to save and restore; no other code generation is affected. *Hardware:* Entry writes registers only, so it cannot fault, and the vector fetch is one 64-bit read.

6.4 Faults and restart: what "precise" means

When an instruction cannot finish (its page is absent, say), the machine must stop it, tell the kernel, and later, when the kernel has fixed the cause, **restart** it (run it again from the start). Whether that works depends on what the instruction did before it stopped.

On the PDP-11 it did not always work. An instruction such as `cmp -(sp),-(sp)` changes a register (`sp` goes down by 2, twice) as part of working out its operands, and may then fault on the second operand: if the kernel simply reran it, `sp` would be 4 too low. The hardware therefore kept a small journal of the register changes (the register `MMR1`, on the machines that had it), and the kernel's `backup()` routine read it and undid the changes before restarting; the 2.11BSD `trap()` says that on the 11/40, which had no such register, "the routine backup/mch.s may fail" and the automatic growth of the user stack, which depends on restarting, can then fail too. Floating-point errors were worse: V7 says "the floating exception is an imprecise trap", meaning that by the time it was taken the processor had gone on to later instructions and the saved PC no longer named the instruction at fault.

On the pdp64 the instruction set promises that every exception that belongs to an instruction is **precise**. A **precise exception** (one taken so that the machine looks as if it ran one instruction at a time, in order: every earlier instruction has completed, and the faulting instruction and every later one have changed nothing) is the property that lets the kernel treat a page fault as a subroutine call that the program never noticed. It matters on this machine, and not only as a convenience, because the pdp64 is meant to be implemented both as a simple in-order core (one that runs instructions one after the other, as the PDP-11 did) and as an **out-of-order core** (one that looks ahead in the program, starts later instructions whose inputs are ready before earlier ones have finished, and keeps the results in a buffer until it is certain the earlier instructions did not fault; errata 68). On the second kind, "the instruction that faulted" is, inside the chip, one of dozens in flight at once, some of them later in the program and already half done; the machine has to throw their results away and report the fault as if none of them had started. The architecture is written so that this can be done without a journal: the instruction set has at most one memory operand, and the register update of an auto-increment or auto-decrement mode is held until the instruction **retires** (is certain to complete) and discarded if the instruction aborts, so a handler never undoes anything (6.4.2; `docs/pdp64-isa.md` section 7.2 has the argument, and 7.3 the table of every reference an instruction can make that can fault).

Four families. The kernel sees the cases that the PDP-11 mixed together separately:

Family	The instruction	`EPC` on entry	Restartable
trap	completes	the next instruction	not applicable
abort	annulled, as if it had not begun	the faulting instruction	yes, by `RTI`
progress	partly done, the progress is in registers	the faulting instruction	yes, by `RTI`
machine check	indeterminate	best effort	no

A **progress** exception belongs to the block instructions (`MCPY` and the others of the `MBLK` group, section 9.7): a copy of a megabyte may take 256 page faults, and the instruction keeps its position in its own registers, so that `RTI` re-executes it and it carries on from where it was. A **machine check** is the descendant of the PDP-11's bus error at vector 004: something happened that the machine cannot repair or locate exactly (an uncorrectable memory error, a posted write that timed out), and the kernel's only choice is to stop the process or the machine. The **posted write** (a store that the processor has handed on and no longer waits for) is why: by the time the bus reports the failure the instruction has long since retired.

6.4.1 Faults fall in four categories: traps (the instruction completes), aborts (it is annulled and restarted), progress (a block operation stopped part-way, restarted from its registers) and machine checks (not restartable). *(D)*

6.4.2 The register update of an auto-increment or auto-decrement mode is held until the instruction retires and discarded by an abort, so a handler never undoes anything. *(D)*

6.4.3 A read of non-existent memory is a precise abort with cause `BUSFAULT`; a posted write that times out and an uncorrectable memory error are machine checks. *(D)*

Commentary. **Precedent.** The PDP-11 recorded its register side effects in `MMR1` and left the undo to software; the VAX needed resumable microstate and a status-word bit; the 68000 had no restart path. **Trade-offs.** Software-managed accessed and dirty bits (6.8) cost an extra fault on first touch and first write, which precise exceptions make cheap. The price of precision is in the implementation, not in the kernel: an out-of-order core needs the buffer that holds its results until retirement. **Improvements.** The one-memory-operand rule makes the undo buffer a single entry and row 3 of the fault matrix provably empty, so there is no journal and no undo protocol. **Benefits.** A page-fault handler is the same code for first touch, first write

and demand paging; the `nofault` idiom of V6 and V7 `trap.c` (set the global, do the probe, `longjmp` out of the handler instead of panicking) remains the correct pattern for hardware probing, though `PROBER` and `PROBEW` cover the address-validation case. **Modern comparables.** Current architectures promise precise exceptions for page faults and, for the most part, leave some floating-point and bus errors imprecise as the PDP-11 did (*unverified*); the pdp64 makes the floating-point exceptions flags that never trap. **Use.** A read of a configuration position that held no device is not a fault at all (it returns all ones, 2.4.9), so a bus scan does not take a `BUSFAULT` for every empty slot. **Impact.** *Kernel:* One small page-fault handler for first touch, first write and demand paging; no `backup()` routine and no `MMR1`. *Compiler:* No scheduling restriction arises from faults, because the register update of an addressing mode is held until retirement. *Hardware:* A single-entry undo buffer, a result buffer for the out-of-order implementation, and a retirement stage that decides.

6.5 System registers: `MFSR` and `MTSR`

The PDP-11 reached its control registers by address: the KT11's page registers, the status word and the other control and status registers sit in the top 8 KB of the address space, the I/O page (`KISA6` at 0172354, `UISD` at 0177600, the PS at 0177776), and the kernel used ordinary `mov` instructions to them. The pdp64 cannot do this: its address space is translated (6.6), a load or store to a control register would itself be translated, and an instruction that changes the translation would be changing the mapping of the very access that reads it. Control registers are therefore **system registers**, numbered 0 to 255, and the two instructions `MFSR Rd, name` (move from system register) and `MTSR name, Rs` (move to system register) read and write them. Both are privileged (6.2.5). The names the assembler knows:

6.5.1 The system registers are reached by `MFSR` and `MTSR` with an 8-bit register number. *(D)*

Number	Register	Meaning
0	`PTBR`	physical address of the level-2 page table
1	`EPC`	exception program counter
2	`EVA`	exception virtual address
3	`ECAUSE`	exception cause (the vector number)
4, 5	`DMW0`, `DMW1`	direct-mapping windows (stored; the format is unspecified)
6, 7	`USP`, `SSP`	the user and supervisor stack pointers
8	`EPSW`	exception status word
9	`VBR`	vector base register
10	`MMUCTL`	memory-management control (bit 0 enables translation)
11	`ASID`	address-space identifier
12	`CPUID`	the core number (read-only)
13 and up		reserved (feature and identification registers are proposed here)

(`EVA` and `ECAUSE` are read only, as is `CPUID`; the rest read and write. `PTBR` is called a level-2 table for the three-level form; it holds the top table of whichever number of levels the machine has, 6.6. `USP` is the user's stack pointer, which the kernel needs because in kernel mode `R14` is the kernel's: it takes the place of the PDP-11 idiom `mfpd sp`, and the kernel must read and write it to build a signal frame, copy a process or switch context. `SSP` is the supervisor's and nothing uses it. The register that the proposal of 6.9.1 calls `KTBR` has no number in this table.)

6.5.2 A write to `PTBR`, `ASID` or `MMUCTL`, or an `ICINV`, is guaranteed to take effect only after the next `ISYNC`. *(D)*

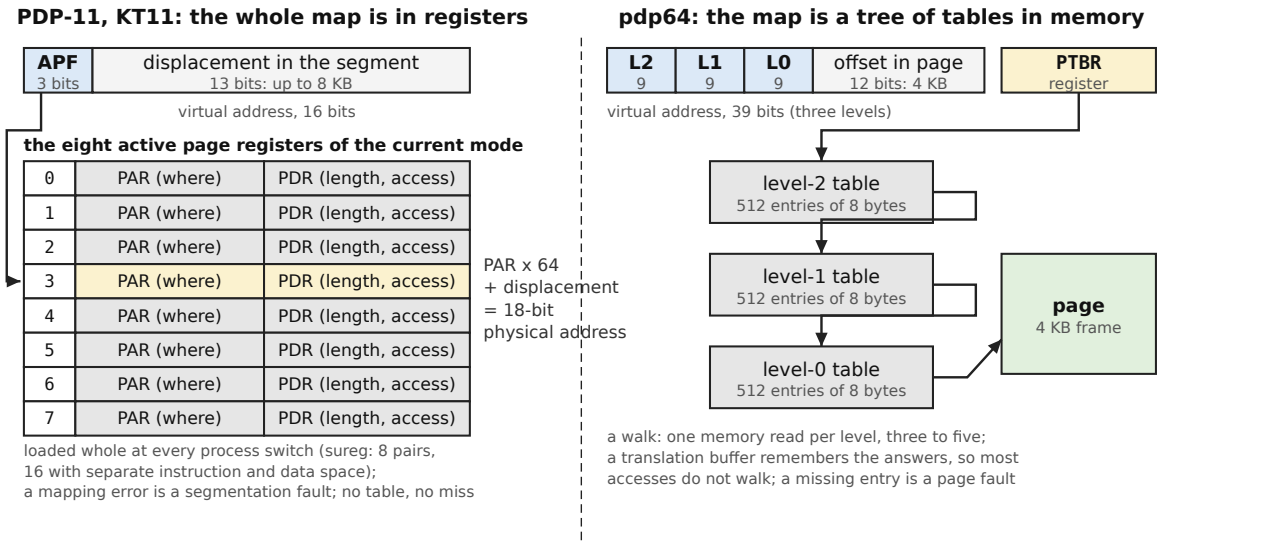
`ISYNC` (instruction synchronisation) is the instruction that makes this safe on a machine that has more than one instruction in progress. A pipelined or out-of-order core has already fetched and started the instructions after the `MTSR`, using the old translation; `ISYNC` waits until every earlier instruction has finished, throws away everything fetched after it, and fetches again under the new rules. The PDP-11 did not need it: an instruction was fetched after the previous one had finished.

Commentary. **Precedent.** The VAX had `MTPR` and `MFPR` with an internal-register number, the same design. The PDP-11 kept its control registers at fixed high addresses and reached them with ordinary moves. **Trade-offs.** A general pair of instructions with 243 free numbers costs two opcodes and no decoding as the registers are added; separate instructions per register would have spent opcode space with every register. **Benefits.** A new register is a number, not an instruction. **Modern comparables.** AArch64's `MRS` and `MSR`, RISC-V's `CSRR` and `CSRW`, PowerPC's `MFSPR` and `MTSPR`; the kernel survey found privileged-register access to be 48% of all the compiler-invisible instructions in four kernels. **Impact.** *Kernel:* A single pair of instructions reaches every system register, so adding a register adds a number and no instruction. *Compiler:* The compiler reaches them through an intrinsic with a constant register number. *Hardware:* A control-register file addressed by an 8-bit number, with a read-only decode for the identification registers.

6.6 From segments to pages: virtual memory

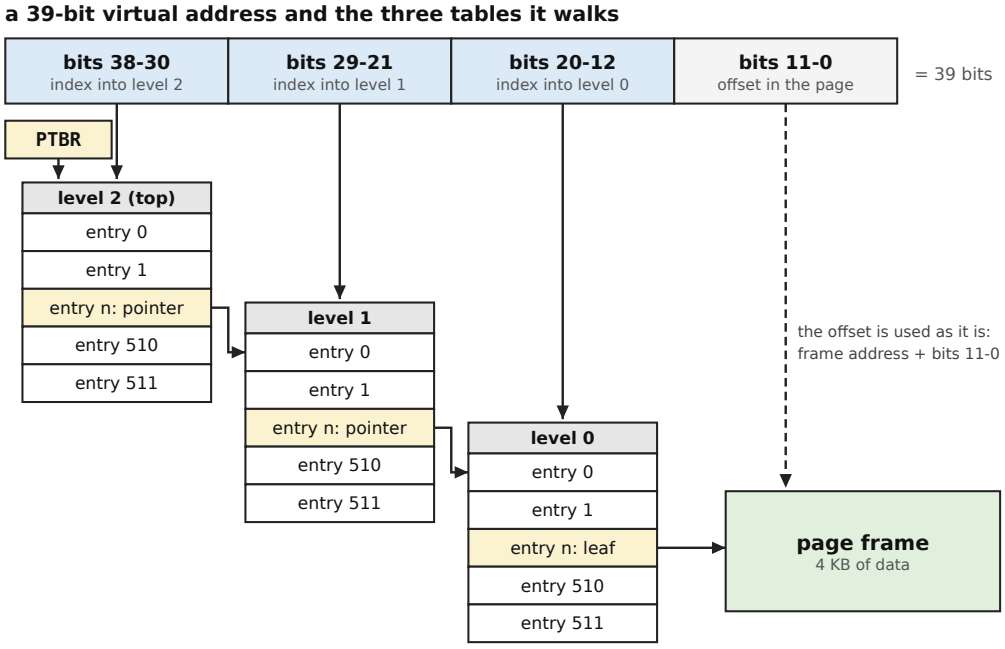
Two kinds of address. A **virtual address** is the number a program uses; a **physical address** is the number the memory chips answer to. The PDP-11 had both, 16 bits against 18 or 22, and the KT11 translated one into the other. A pdp64 program uses 64-bit pointers (stored in 64-bit registers), but only some of the bits count (below), and the physical address is 40 to 64 bits wide (errata 102): 40 bits is one terabyte, and the maximum, 2^{64}, is sixteen exabytes.

What the KT11 did, and what it cost. The figure puts the PDP-11's scheme (left) beside the pdp64's (right). On the left, the three top bits of a 16-bit address pick one of eight page registers, which says where an 8 KB piece starts and how long it is; there is nothing to look up, because the whole map for the running program is in the eight pairs. That costs three things. A process could have at most 64 KB (eight pieces of 8 KB, 128 KB with separate instruction and data space): in 2.11BSD, `estabur()` returns `ENOMEM` for a program that needs a ninth segment. Each of its pieces was one **contiguous** block of physical memory, so memory fragmented, and the allocator found a hole big enough or the kernel swapped something out to make one. And since the map was only eight registers, there was no way to leave a hole in the middle of a program unmapped, to share half a segment, to start running a program before all of it was in memory, or to notice which part of it had been used.



Pages. The remedy, which the VAX had and every machine since has copied, is to cut both address spaces into equal pieces and let any piece of the virtual space sit in any piece of the physical. The virtual piece is a **page** and the physical piece a **frame** (the same size: here 4 KB, 4096 bytes); a virtual address is a **page number** and an **offset** within the page, and translation replaces the page number by a frame number and leaves the offset alone. The map from page numbers to frame numbers is a **page table**: an array indexed by the page number, each element (a **page-table entry**, or PTE, section 6.7) saying where the frame is, who may touch it and whether it is there at all. With pages, a program's memory need not be contiguous; a page can be absent (to be fetched from disk when the program touches it, which is **demand paging**) or shared by two programs; and the kernel can tell which pages are in use.

The tree. A single page table is too large. A 39-bit virtual address with 4 KB pages has 2^{27} pages (134 million); at 8 bytes an entry that is a 1 GB table for every process, nearly all of it empty. The answer is to make the table a **tree**: a table that is exactly one page long, so 4096/8 = 512 entries, which is nine bits of index; some of its entries point at tables of the next level and not at frames. A virtual address is then cut into fields of nine bits, one for each level, over the 12-bit offset: with **three levels** the address has 9 + 9 + 9 + 12 = 39 bits, and the translation is a **walk**:



Each table is 512 entries x 8 bytes = 4 KB. PTBR holds the physical address of the top table.
With four levels bits 47-39 index a level-3 table above these; with five, bits 56-48 index a level-4 table above that.
A leaf found at level 1 maps 2 MB and ends the walk there; a leaf at level 2 maps 1 GB.

```
translate(va, access, mode):                        # three, four or five levels
    if va is not canonical:  fault NONCANONICAL     # bits above the top one must equal it
    table = PTBR                                    # physical address of the top table
    for level = LEVELS-1 down to 0:
        index = (va >> (12 + 9*level)) & 511
        pte   = load64_physical(table + 8*index)    # one memory read per level
        if not pte.V or pte has a reserved bit set: fault PAGE_FAULT
        if pte.R or pte.W or pte.X:                 # a leaf: the walk ends here
            break
        if level == 0:       fault PAGE_FAULT       # a pointer where nothing is left to point to
        table = pte.PPN << 12                       # follow the pointer to the next table
    check the leaf (6.7): permissions for this mode, accessed and dirty (6.8)
    size  = 4 KB << (9*level)                       # level 0: 4 KB; level 1: 2 MB; level 2: 1 GB
    return (pte.PPN << 12) | (va & (size - 1))
```

(The pseudocode is this chapter's rendering of statements 6.6.1, 6.7.1 and 6.7.2, checked against the walker of the QEMU model, `qemu/target-pdp64/mmu.c`; a leaf that is not aligned to its size is also a page fault there.) The cost is plain from the loop: **one memory read per level**, three to five. A program that touches only a few megabytes has a handful of tables, and the **sparse** address space (most of the 512 GB unused) costs nothing, because an entry that is not valid has no table under it. A leaf found above level 0 maps a larger piece (2 MB at level 1, 1 GB at level 2): the **large page** (a page of many times the basic size, mapped by one entry, used for the kernel's own mapping and for big buffers). A leaf above level 0 therefore needs a frame address aligned to its size.

Three, four or five. The author's decision (errata 96) is that the walk may have three levels (39 virtual bits, what System v1 specifies and what the simulator and the emulator implement), four (48 bits) or five (57 bits), with the number of levels discoverable at run time. Each extra level adds nine bits and one more memory read per miss. The author's own referent was the Loongson five-level table (his question, errata 96); RISC-V's Sv39, Sv48 and Sv57 and the VAX are the repository's precedents, named in the commentary. A **canonical** address is one in which all the bits above the top virtual bit equal that bit (so a 39-bit machine accepts a 64-bit pointer only if bits 63 to 38 are all the same): this makes the upper half of the space a copy of the lower half's sign, and rules out a program storing a tag in the unused high bits, which would stop working on a machine with more levels.

6.6.1 Virtual addresses are translated by three, four or five levels of nine bits over 4 KB pages, which gives virtual addresses of 39, 48 or 57 bits. With three levels, address bits 38 to 30 index level 2, 29 to 21 level 1 and 20 to 12 level 0; with four, bits 47 to 39 index level 3 above them; with five, bits 56 to 48 index level 4 above that; `PTBR` holds the physical address of the table of the top level. *(D)*

6.6.1a Software can find the number of levels of the machine at run time, so that a kernel adapts to the implementation without a build option; the register or field that reports it is not specified. *(S)* The simulator and the emulator implement three levels.

What is open about 6.6.1a. The repository's briefing of this question (`docs/decision-log/data/shared/oq/A.md`, item Q2) lists the ways other machines let software find out, each verified in the QEMU source on this machine: x86's `CPUID` leaf 0x80000008 (physical and virtual widths) and the `LA57` bit for five levels; AArch64's `ID_AA64MMFR0` field `PARange`; LoongArch's `CPUCFG1` fields `PALEN` and `VALEN`; and RISC-V's `satp` register, in which software tries to write the mode it wants and reads back whether it stuck. The options it sets out for the pdp64 are a read-only feature register at system register 13 (one `MFSR`, nothing to probe), a write-and-read-back probe in `MMUCTL`, both, a field in the boot-information block (4.6.2) and a user-readable copy. **None of them is decided**, and the briefing's leaning is the assistant's, not the author's; the physical width has the same problem (errata 102(b)).

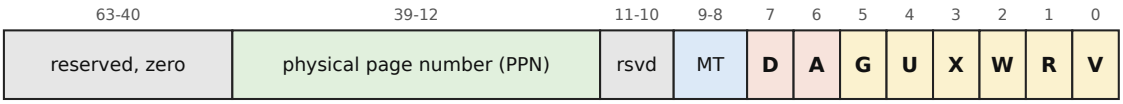
6.6.2 Virtual addresses are 39, 48 or 57 bits, translated by three, four or five levels of nine bits over 4 KB pages (6.6.1) to a physical address of 40 to 64 bits, 40 in the present implementation (section 2.3.6). *(D)*

Commentary. **Precedent.** The PDP-11 had eight segments per mode, each with a base and a length, and no page tables; the VAX had a linear page table per region with a hardware modify bit and no reference bit, so 4.3BSD emulated reference bits in software. **Trade-offs.** Thirty-nine bits is 512 GB, what a three-level table indexes with nine bits per level; each further level adds nine bits (48, then 57) at the price of a longer walk on every TLB miss, and the kernel chooses what the implementation offers. **Improvements.** A leaf at any level makes large pages free to use for the kernel's own mapping. **Benefits.** A walker that only reads is small. **Modern comparables.** The three-level geometry is RISC-V's Sv39 (the modes 8, 9 and 10 of `satp` are Sv39, Sv48 and Sv57, checked in the QEMU source), and x86-64 and AArch64 use four levels (x86-64 five on newer parts, the `LA57` bit); LoongArch describes the width of each level in a register (`PWCL`, checked), and the author asked what a Loongson five-level table would affect (errata 96 and the message it answers). **Impact.** *Kernel:* The same walk code serves three, four and five levels, with the level count read at run time (once the register exists). *Compiler:* Position-independent code uses PC-relative addressing; the canonical form means pointers need no masking and must not carry tags in the high bits. *Hardware:* A walk is one read per level, three to five; the walker is a small state machine that never writes.

6.7 The page-table entry

Each entry is one 64-bit word. The diagram shows every bit, the table says what each does:

a page-table entry: 64 bits, the physical page number in place



V	valid: the entry means something; clear is a page fault
R W X	read, write, execute; all three clear means "pointer to the next table"; W without R is reserved
U	user: user mode may touch it (kernel mode may read and write a user page, never execute one)
G	global: the entry ignores the address-space number
A D	accessed, dirty: set by the kernel, never by the hardware; a clear bit is a page fault
MT	memory type: 0 normal cached, 1 normal uncached, 2 device strongly ordered, 3 device weakly ordered
rsvd	bits 10-11 and 40-63 must be zero; a set bit is a page fault (for 40-bit physical addresses; wider ones are open, 6.7.1)

Bit positions are those of statement 6.7.1; bits 0 to 7 are in the same places as in RISC-V.

An entry has two shapes. A **leaf** is an entry that maps: at least one of `R`, `W`, `X` is set, and the physical page number is the frame the page lives in. A **pointer** is an entry that says "the next table is here": `V` is set, `R`, `W` and `X` are all clear, and the page number

is the next table's. A **valid** bit of zero says "nothing is mapped here", and then the other 63 bits are the kernel's to use for anything, such as the place on the swap device where the page now lives.

Permissions and modes. `R`, `W` and `X` allow reads, writes and instruction fetches. `U` says user mode may use the page: **user mode needs** `U`; kernel mode may read and write a user page (that is what lets the kernel copy to and from user memory directly, section 6.10) but may never **execute** one, so a kernel that has been tricked into jumping to user memory takes a fault instead of running the attacker's code. `W` without `R` is not a valid combination and is a fault. `G` (global) marks a translation the hardware keeps whatever the address space (6.8). Any bit that is not defined must be zero in a valid entry, and a set reserved bit is a page fault: this lets the format grow and catches a kernel that wrote a wild value.

Memory types. Each page also says how the memory system may treat accesses to it: `MT`. **Normal cached** (value 0) is ordinary memory: reads and writes may be kept in the **caches** (small fast memories near the processor that hold copies of recently used memory). **Normal uncached** (1) is memory that must not be cached: a frame buffer that the display hardware reads and the processor writes. The two device types are for the registers of devices: **device strongly ordered** (2) means that every access goes to the device in exactly the order the program wrote it, one at a time, never combined; **device weakly ordered** (3) allows the machine to combine or reorder accesses to the page, for the memory of a device where only the final result matters. (The architecture text gives the four names and their numbers; the glosses of the last two are the usual meanings of the terms and are marked *unverified* in 6.14.) On the PDP-11 the memory type was the address: everything at the top of the address space was a device. The pdp64 has **no address bit that says "device"** (errata 97), so the memory type of the page is what a driver sets when it maps a device, and the window of a device can be anywhere in the physical address space except the first few pages (errata 102, section 2.3.6).

6.7.1 A page-table entry is 64 bits: `V` (valid) in bit 0, `R`, `W`, `X`, `U` and `G` (global) in bits 1 to 5, `A` (accessed) and `D` (dirty) in bits 6 and 7, the memory type `MT` in bits 8 and 9, and the physical page number in bits 12 to 39, in place in the present implementation; all other bits are zero, and a set reserved bit is a page fault. *(D)* The format of the entry for a physical address of up to 64 bits (2.3.6) is open: a page number of 52 bits in place would take bits 12 to 63 and leave no bit above bit 11 for anything else, and whether the entry keeps that shape or changes is the author's question. *(S)*

6.7.2 A non-leaf entry has `V` set and `R`, `W` and `X` clear and points to the next table; a leaf at level 2 maps 1 GB, at level 1 maps 2 MB and at level 0 maps 4 KB, and where the levels exist a leaf at level 3 maps 512 GB and at level 4 maps 256 TB. *(D)*

6.7.3 A user-mode access needs `U`; kernel mode may read and write user pages but may not execute them; `W` without `R` is reserved. *(D)*

6.7.4 The memory type of a page is one of normal cached, normal uncached, device strongly ordered and device weakly ordered. *(D)*

6.7.5 A page-table entry carries the physical page number in place and read, write, execute and user permission bits; accessed and dirty bits are managed by software; one memory type per page selects cached, uncached, strongly ordered or weakly ordered. *(D)*

What is open about 6.7.1. The physical address is 40 to 64 bits (errata 102), and a page number for a 64-bit address needs 52 bits (bits 12 to 63): in place, that takes every bit of the entry above bit 11, and leaves nowhere for a later flag (a protection key, a wider memory type, a no-execute bit separate from `X`). The repository's briefing (`A.md`, item Q3) sets out six shapes, from keeping the page number in place to bit 63, to capping it at 40 bits (a 52-bit physical address, four petabytes through page tables, with the 12 bits above for flags and the windows of 6.9 for anything higher), to shifting the page number down to bit 10 as RISC-V does (checked: RISC-V's number is in bits 10 to 53, flags `N` and `PBMT` in bits 63 to 61), to a 16-byte entry (which would change the nine bits per level, and with them the 39, 48 and 57). **No format is chosen.** The simulator, the QEMU model and `spec/pdp64-system.hbnf` use 40 bits, which is the minimum and fits. They also still compare the memory type of a page with bit 39 of its physical address (the rule of errata 60 and 65, which errata 97 removed): that check belongs to the present implementation, and goes when the implementation catches up.

Commentary. **Precedent.** The VAX entry carried a hardware modify bit and a protection field of four bits; RISC-V's low eight bits are exactly `V`, `R`, `W`, `X`, `U`, `G`, `A` and `D` in the same places (checked in `cpu_bits.h`), which the pdp64's entry shares. **Trade-offs.** One entry shape at every level makes the walker simple; an unaligned large page is rejected as a fault, which costs a check and saves the walker from reasoning about partial frames. A memory type per page, not per access, means the code that writes a frame buffer needs no cooperation to be uncached. **Improvements.** The entry has the physical address in place, so no shifting; a leaf at any level makes large pages free to use for the kernel's own mapping. **Benefits.** A kernel's page-fault

handler is the same code for first touch, first write and demand paging, differing only in the bits it sets. **Use.** The memory type on a page lets a driver map a device register uncached and strongly ordered without a special instruction. **Impact.** *Kernel:* The entry is built and read with shifts and masks that are the same for every level; leaves at any level let the kernel map itself with 1 GB pages. *Compiler:* None. *Hardware:* The physical page number is in place, so no shifting; a translation-buffer entry holds a page number as wide as the implemented physical address, 28 bits at 40.

6.8 Accessed, dirty, the translation buffer and address-space numbers

Accessed and dirty. A kernel that wants to run programs larger than memory must choose pages to throw out. The two facts it needs are whether a page has been used lately (**accessed**, A) and whether it has been written since it was brought in (**dirty**, D: a dirty page must be written to disk before its frame is reused; a clean one can simply be dropped). On the VAX the hardware set a modify bit and the kernel emulated the reference bit; most modern machines have the **walker** (the part of the memory-management hardware that reads the tables) set both bits itself, writing the page-table entry. The pdp64's walker **never writes a page table**: an access to a page with A clear, or a write to a page with D clear, takes the page fault of its kind, and the kernel sets the bit and runs the instruction again. That is **software-managed** accessed and dirty bits (an operating system's own page-fault handler keeps them). The reasons are given in the commentary; the cost is one extra fault the first time each page is touched and one the first time it is written.

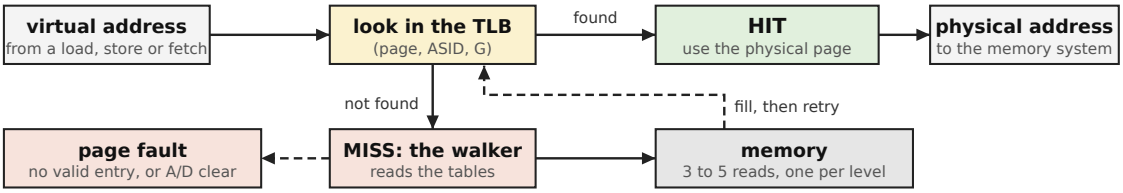
6.8.1 The accessed and dirty bits are managed by software: an access to a page with A clear, or a write to a page with D clear, raises the page fault of its kind and the kernel sets the bit; the hardware walker never writes a page table. *(D)*

The translation buffer. The walk costs a memory read for each level, three to five reads for every load, store and instruction fetch: unaffordable. Every machine with a walk therefore has a **translation buffer** (the TLB, translation lookaside buffer: a small, very fast memory in the processor that keeps the results of recent translations, so that a repeated access to the same page needs no walk). It is a **cache** (a small copy of a larger store, kept for speed) of translations. The PDP-11 had no such thing: its eight page registers *were* the translation, loaded wholesale at each switch, and there was no table to be slow.

the translation buffer (TLB): a cache of finished translations

ASID	G	virtual page	physical page	R W X U	A D MT
7	0	0x00000412	0x0003a1	R W . U	A D 0
7	0	0x00000413	0x0007c2	R . X U	A . 0
-	1	0xFFFFFC001	0x000100	R W X .	A D 0
12	0	0x00000412	0x0009ff	R W . U	A D 0

the same virtual page 0x412 appears twice, for address spaces 7 and 12: that is what the ASID is for; the global entry (G) answers for every address space and has no ASID.



TLBI rN drops the entry for one page (this ASID and the global ones); TLBIA drops all; both reach every core; TLBSYNC waits until all have finished. The size and organisation are the implementation's: the FPGA plan assumes 4-way, 64 entries (section 6.14).

An access looks in the buffer first, by virtual page number and address-space number (below). If an entry matches (a **hit**), the physical page and the permissions come straight out and the access goes on with no memory read for the translation. If none does (a **miss**), the walker reads the tables, and if the walk ends in a valid leaf with the right permissions and the right A and D it **fills** the buffer with the result and the access is retried; if not, the access takes the page fault of 6.3. A miss costs one memory read per level; a program that does not touch more pages than the buffer holds pays only for its first touches. The size and organisation are the implementation's choice; the FPGA plan (section 6.14) assumes a four-way set-associative buffer of 64 entries (**set-associative**: an entry may live in any of a few places selected by the low bits of the page number, which is cheaper than a buffer that can put an entry anywhere).

Keeping it honest. The buffer holds copies. When the kernel changes a page-table entry it must see to it that no stale copy remains, or the old translation will keep working: a page the kernel has unmapped, or made read-only, would still be

reachable. An **invalidate** (a flush, in the older word: removing entries from the buffer) is `TLBI rN`, which drops the translation of the page containing the address in `rN` for the current address space and the global entries, or `TLBIA`, which drops everything. On a machine with several cores each core has its own buffer, so the invalidate must reach all of them: on the pdp64 both instructions are **broadcast** (sent to every core by the hardware) and `TLBSYNC` waits until every core has finished, so that the kernel knows that nobody holds the old translation. Other architectures leave this to software, which sends an inter-processor interrupt to every other core and waits for them to answer (a **shootdown**: a source of deadlocks in the kernels of the BSDs, according to the pain-points report, `docs/kernel-pain-points.md`). Unmapping a page is then three lines, as chapter 7 says (7.9.2):

```
unmap_page(space, va):
    pte_of(space, va) = 0          # change the entry in memory first
    TLBI va                        # drop the old translation, on every core
    TLBSYNC                        # wait until every core has done so
    # now the frame may be reused
```

6.8.2 Translation is off at reset (`MMUCTL` is zero and every address is physical) and is enabled by bit 0 of `MMUCTL`; the `ASID` selects non-global entries. *(D)*

6.8.3 `TLBI rN` invalidates the translation of the page containing the address in `rN` for the current address space and the global entries; `TLBIA` invalidates all; both broadcast to every core and `TLBSYNC` waits for the broadcast to finish. *(D)*

Address-space numbers. When the kernel switches from one process to another the page tables change. If the translation buffer cannot tell which process an entry belongs to, every entry of the old process would answer for the new one, so the whole buffer must be emptied at every switch, and the new process begins with all misses. On the PDP-11 the corresponding cost was `sureg()`, which wrote the new process's eight (or sixteen) pairs into the page registers: sixteen (or thirty-two) register writes at every switch, with nothing to flush. For a table walked from memory the cost is worse, because the emptied buffer must be refilled one miss at a time. The remedy is to **tag** every buffer entry with the number of the address space it came from, the **address-space identifier** (ASID, 16 bits on the pdp64, set with `MTASID`): an entry matches only if its tag equals the current ASID or it is **global** (`G` set, so it answers in every address space, as the kernel's own pages do). Switching is then a write to `PTBR` and to `ASID` and an `ISYNC` (6.5.2): the old process's entries stay in the buffer, harmless, and answer again if that process runs again soon. (The reference executor does not yet do this: its `MTASID` flushes the whole buffer, and the entry of `MFSR` in chapter 9 says that a write to `PTBR`, `ASID` or `MMUCTL` flushes it. That is at odds with the purpose of the tags, and is listed in 6.15; the saving described here is what the tags allow.) The picture of the whole switch is in 6.11.

There are 65,536 numbers, so when they run out the kernel must start again: allocate them in order, and when the counter wraps, invalidate everything (`TLBIA`) and note a new **generation** so that processes that held a number from before are given a fresh one when they next run. That policy is the kernel's; the architecture offers no invalidate by ASID, which is listed as open (6.15).

Commentary. **Precedent.** The VAX had a linear page table per region with a hardware modify bit and no reference bit, so 4.3BSD emulated reference bits in software; x86 added a process-context tag late (the `PCIDE` bit of control register 4, checked in the QEMU source; its width is *unverified*); RISC-V's `satp` carries a 16-bit address-space number (`SATP64_ASID`, checked) and software flushes other cores' buffers; AArch64 has a broadcast invalidate. The ASID and the `G` bit come, in the ISA text, from the Commodore 128's memory-management unit: four sets of map registers selected by one store, and a common region that no map set could hide (`docs/pdp64-isa.md` section 8.1). **Trade-offs.** Software-managed bits cost one extra fault on the first touch of each page and one on the first write, and they make the walker small and make it safe against the chip bugs of hardware management that other architectures have had; a hardware-managed option, advertised in a feature register, is a possible extension. **Benefits.** A walker that only reads is small; an FPGA walker is a state machine of a few states, and a core with no speculation has no page-table side channels. **Modern comparables.** x86-64 and AArch64 use hardware-managed accessed and dirty bits; MIPS refills its translation buffer in software. **Impact.** *Kernel:* The handler sets accessed and dirty bits; `TLBI` broadcasts so there is no shootdown code; a context switch writes two registers. *Compiler:* None. *Hardware:* The walker only reads; a translation-buffer entry carries an ASID tag and a global bit, a comparison on lookup, and (on the FPGA) the buffer is four-way set-associative rather than content-addressable.

6.9 Splitting the space, direct-mapping windows and the layout of an address space

One space or two. On the PDP-11 the kernel had its own address space (its own eight page registers), and the user could not see it at all. A modern kernel instead sits **in the upper part of every process's own address space**, so that it can reach the process's memory directly, and the permission bits keep the user out. That arrangement had a hole: a processor that **speculates** (one that, to go faster, runs instructions ahead before it knows they are allowed, discarding the results if it turns out they were not) could, on some designs, read the kernel's memory before the permission check had caught up, and leave a trace of what it read in the caches where a user program could measure it. (This is the class of flaw that became public in 2018 and is general knowledge here, *unverified*; the repository's reason is stated only as in statement 6.9.1.) The pdp64's answer is that **the user does not walk the kernel's tables at all**:

6.9.1 The translation is split by the top bit of the virtual address (bit 38 with three levels, 47 with four, 56 with five): clear selects the lower half, translated from the table at PTBR (one per address space), and set selects the upper half, translated from the table at the system register KTBR (one for the whole system, the kernel's). In user mode the upper half is never walked: an access to it takes a translation fault without consulting any page table, so no kernel translation is present for a speculating core to read around a permission check. In kernel mode both halves are walked, so the copy routines of 6.10 need no change of root. Entry to an exception changes the mode in hardware and so changes the halves walked; no entry trampoline is mapped in user space. Translation-buffer entries carry the half they came from, and a mode change flushes nothing. The register number of KTBR and the layout of the top-level tables of the two roots are unspecified. *(D)*

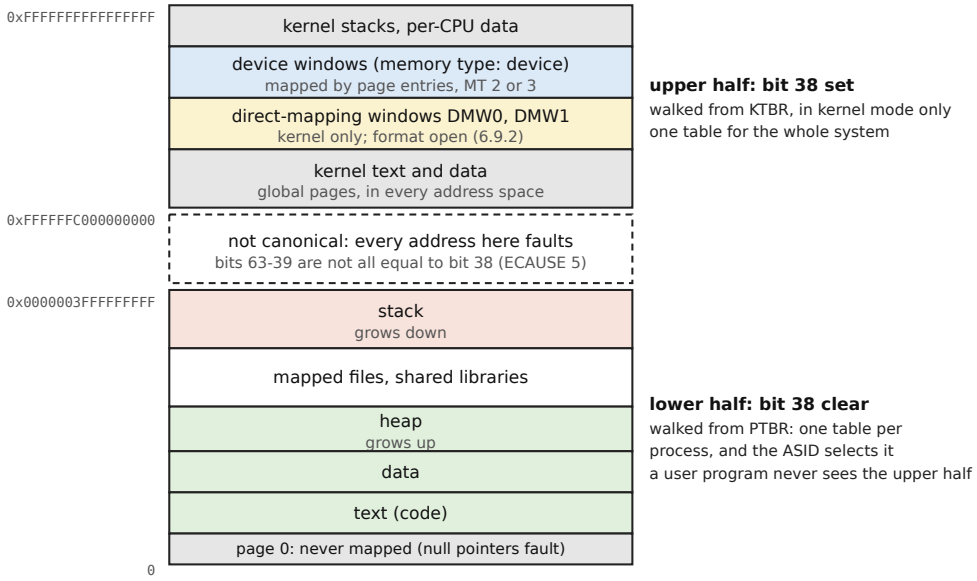
(This is the proposal of research round 7 that the author accepted, errata 71. It is **not yet implemented**: the simulator and the QEMU model have one table base, PTBR, and test only that the high bits equal bit 38; the work needed is a system-register number, the walker and the tests, as the project's hand-over notes say.)

Direct-mapping windows. A kernel must also reach all of physical memory (to write a page it has just allocated, a DMA descriptor, a page table), and the memory of devices. Walking tables for that is slow and uses up translation-buffer entries. A **direct-mapping window** (DMW: a range of virtual addresses that the hardware maps to a range of physical addresses by one addition, with no page table and no buffer entry) does it. The instruction-set text sketches two windows, DMW0 and DMW1, each with a virtual base, a size and a memory type, kernel only, translating as `PA = VA − VirtualBase + PhysicalBase`. Three reasons it gives: the kernel stops taking translation misses on its own text and data; physical memory is reachable without mapping it; and early boot becomes trivial (`docs/pdp64-isa.md` section 8.3). The idea is LoongArch's DMW registers (named in that text), a tidier descendant of MIPS's fixed KSEG0 region (whose behaviour on the real machine is *unverified* here). It is what the author meant by "good thing we already had dmw in the design" (errata 96, 2026-11-09): the answer to petabytes of physical memory and to a deep table is that the kernel need not walk anything to reach them. **The format of the registers is not specified**: the manual stores them and does nothing with them (6.9.2), so the sketch is a sketch, and the statement of the architecture document that the machine comes up with DMW0 covering physical memory is at odds with 6.8.2 (translation is off at reset); the manual follows 6.8.2.

6.9.2 Two direct-mapping windows, DMW0 and DMW1, are stored and their format is unspecified; MMUCTL bit 0 enables translation and is zero at reset, so every address is physical until the kernel turns it on. *(D)*

The layout of a process's address space. Putting the pieces together, a process on a three-level machine sees the picture below. The two halves and the windows are the architecture's; what goes where in each half is the kernel's to choose, and the figure is one example.

the 39-bit virtual space of three levels: one example arrangement



The halves are decided; what goes in them is not specified. Each half is 2^38 bytes = 256 GB with three levels, 128 TB with four, 64 PB with five. The order, the sizes of the regions and the placement of the windows are the kernel's choice; nothing here is tested.

The **user half** holds the program's text, data, heap and stack, in pages with `U` set, reached through the process's own tree (`PTBR`, selected by the `ASID`); address zero is left unmapped so that a null pointer faults. The **kernel half** holds the kernel's text and data (global pages: one copy of the entries answers in every address space), the per-CPU structures (the kernel's `DP` points at them), the kernel stacks, the direct-mapping windows, and the windows of **devices**: a device's registers are mapped by page entries with the device memory type (6.7), at the physical address that PCI Express enumeration found for them (chapter 2); no address bit and no fixed page says "this is a device". Between the two halves lie the addresses that are not canonical; any access to one takes the non-canonical-address fault (vector 5).

Commentary. **Precedent.** The PDP-11 had separate kernel and user spaces and `mfpi`/`mtpi` to cross; the VAX divided its address space into a process half and a system half, the same shape as the split here (unverified: the VAX manual is not on this machine; the 4.3BSD sources are). **Trade-offs.** A kernel table separate from the user's costs a second root register and a second walk to cache; it removes the kernel from the translation that a user program's speculation can reach. Two windows is the number the ISA text sketches; a kernel that wants more regions maps them with large pages. **Improvements.** The kernel stops taking translation misses on its own text and data; physical memory and a petabyte-scale device are reachable with no page table; the first page table is easy to build. **Benefits.** An entry trampoline (the page of kernel code that some architectures map into user space so that an exception has somewhere to land) is not needed. **Modern comparables.** x86 kernels dropped the kernel half from the user's page tables only after 2018 and pay a table switch on every entry; RISC-V and AArch64 offer a second table base register (AArch64 `TTBR0` and `TTBR1`: unverified here). **Use.** The windows serve the copy of a page, a DMA buffer, a page-table page and early boot. **Impact.** *Kernel:* The kernel maps itself once, global, and runs without misses on its own code; physical memory is reachable by addition. *Compiler:* None. *Hardware:* One comparator pair and one adder per window, a second root register, and a half bit in each translation-buffer entry.

6.10 The user-memory guard and system calls

Reaching the user's memory. On the PDP-11 the kernel's pointers and the user's pointers were in different spaces, so reaching user memory took the special instructions `mfpi` and `mtpi` (from the first section), and an accidental use of a user pointer in kernel code failed on its own, because the same number meant something else. On the pdp64 the kernel and user pages share one address space, so a kernel pointer and a user pointer are the same kind of number: the kernel can copy from a user buffer with an ordinary load, and a **buggy kernel** (one that follows a user-supplied pointer without checking) can do it by mistake, which is how many attacks work. The guard is a status-word bit, `UA`: with it clear, a kernel-mode data access to a user page faults; the kernel sets it (`STUA`) only around the copy and clears it (`CLUA`) after, and entry to every exception clears it.

6.10.1 With the guard bit UA clear, a kernel-mode data access to a user page faults; STUA sets it and CLUA clears it; exception entry clears it and EPSW keeps the old value. *(D)*

System calls. A system call is TRAP #n, or TRAPL #n for the long form; it enters through the vector 256 + n, the next instruction becomes EPC, and the call number and arguments travel in registers named by the ABI (appendix D; which registers is not specified there for the system-call convention, and this chapter does not invent it). Compare V7, where sys n entered one vector and trap() fetched the number back from the user's instruction with fuiword, and the arguments were read from the user's stack by fuword.

6.10.2 TRAP #n (n up to 15, two bytes) and TRAPL #n (n up to 255) enter the kernel through vector 256 + n; the system-call number and arguments travel in registers by the ABI. *(D)*

6.10.3 A copy routine opens the guard, copies, and closes it; every error path out of the copy must close it, and entry clearing it bounds the damage if one does not. *(D)*

A copy routine in pseudocode, for the case that a user address is bad:

```
copyin(user_src, kernel_dst, n):
    if PROBER(user_src) == 0 or PROBER(user_src + n - 1) == 0:   # readable in the previous mode?
        return EFAULT                       # PROBER tests, and takes no fault
    STUA                                    # open the guard
    memcpy(kernel_dst, user_src, n)         # may still fault, if the file was truncated under us
    CLUA                                    # close it, on every path out
    return 0
```

Commentary. **Precedent.** The PDP-11 reached user memory with the explicit instructions MFPI and MTPI, since its kernel and user address spaces were separate, so accidental access was impossible; the VAX read user addresses directly after a PROBER check. **Trade-offs.** The guard is a mode bit, accepted because it lives in the status word, which is saved anyway; it makes a stray kernel pointer a clean fault at the cost of an open and a close around every user copy. **Benefits.** A kernel bug that dereferences a user pointer by mistake does not succeed silently. **Modern comparables.** x86 SMAP with STAC and CLAC, AArch64 PAN and RISC-V SUM. **Use.** The user copy routines (copyin, copyout, copyinstr). **Impact.** *Kernel:* Every user copy opens and closes the guard; every error path must close it, and entry clearing it bounds the damage of one that does not. *Compiler:* The copy routines are written in assembler or inline assembler; system-call stubs are one TRAP. *Hardware:* One status-word bit and one check on a kernel-mode data access to a user page.

6.11 The kernel at work: five walks in pseudocode

Everything above is machinery; this section runs it. The pseudocode is **an illustration written for this chapter**, in the style of a V7 kernel, from the statements of this chapter; it is not from a pdp64 kernel (there is none), is not tested, and where it names a table or a function (find_region, alloc_frame, tick) these are the sort of thing any Unix kernel has and are not specified by the architecture. The assembly mnemonics (PUSHX, POPX, SPL, MFSR, MTSR, TLBI, TLBSYNC, MTASID, ISYNC, WFE, RTI) are the real ones.

A system call. The user program puts the call number and the arguments in the registers the ABI names and executes TRAP #0 (or whichever number the system uses); the processor does enter(256 + n, trap) of 6.3, which makes EPC the instruction after the TRAP:

```
vector 256+n  ->  syscall_entry:        # kernel mode, priority 7, R14 is the kernel stack
    PUSHX                               # save EPC and EPSW: one instruction
    save the registers the C code will clobber on the kernel stack
    SPL #0                              # let interrupts in: a system call may take a while
    number = the register the ABI names
    result = sysent[number].handler(args from the registers)     # copyin/copyout use STUA, CLUA
    put result and error in the registers the ABI names
    restore the registers
    SPL #7
    POPX                                # EPC and EPSW back
    RTI                                 # PC = EPC (the instruction after TRAP); PSW = EPSW: user mode again
```

The kernel survey estimates the entry itself (up to the call of the handler) at four or five instructions on paper, against 45 to 70 on the RISC kernels it examined: a paper estimate, not a measurement.

A page fault. A load, store or instruction fetch whose translation is not valid, or whose `A` or `D` bit is clear, aborts with vector 7, 8 or 6 (6.3.6). `EVA` holds the address, `EPC` the faulting instruction, `ECAUSE` the kind:

```
page_fault:                             # vectors 6, 7, 8 (instruction, load, store)
    PUSHX
    save the registers the C code will clobber
    va    = MFSR EVA
    cause = MFSR ECAUSE
    SPL #0                              # we may sleep, reading the page from disk
    region = find_region(current_process, va)
    if region is none:                  # a wild pointer
        send SIGSEGV; goto leave        # (leave delivers the signal on the way out)
    if cause == STORE_FAULT and not region.writable:
        send SIGSEGV; goto leave
    pte = find_pte(current_process.space, va)        # the kernel walks the tree in software, too
    if pte is invalid:                               # the first touch
        frame = alloc_frame()
        fill frame from the file or with zeroes      # may sleep
        pte = make_pte(frame, region.permissions, A = 1, D = (cause == STORE_FAULT))
    else:                                            # valid, but A or D was clear
        pte.A = 1
        if cause == STORE_FAULT: pte.D = 1
    store pte
    TLBI va ; TLBSYNC                                # no stale copy of the old entry on any core
leave:
    SPL #7
    restore the registers
    POPX
    RTI                                              # EPC is the faulting instruction: it runs again
```

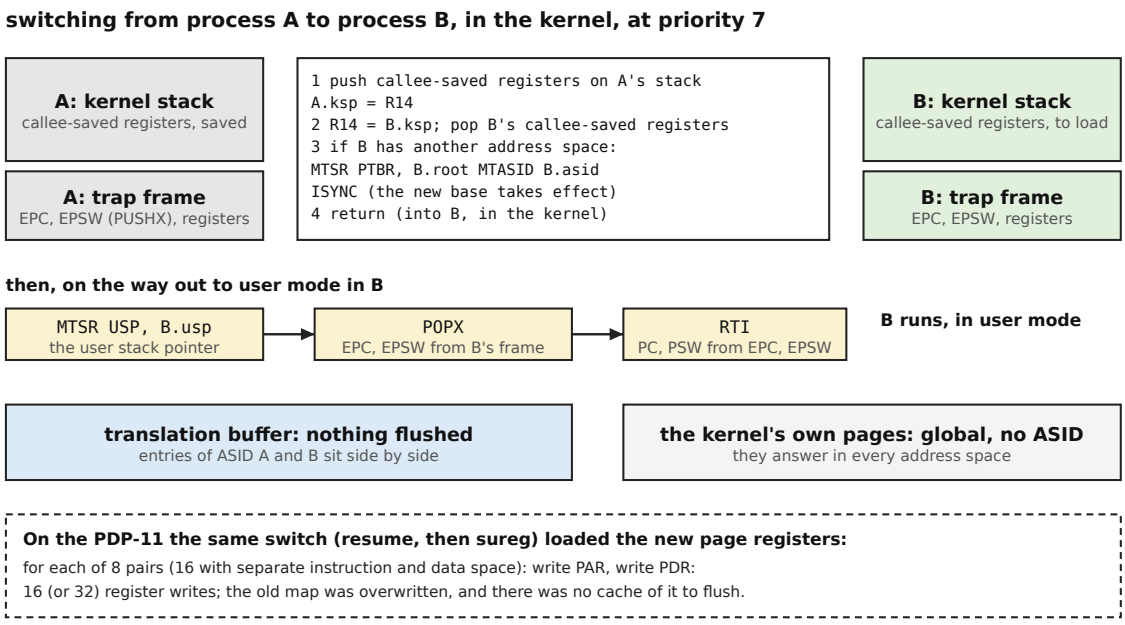
The same handler serves first touch, first write and paging in: the only differences are the bits it sets. The faulting instruction is restarted because the abort was precise (6.4). A page fault of the kernel itself on a user address is the case the copy routines of 6.10 expect: under the proposal of 6.10.4 the handler looks up a **fixup** (a recovery address kept in a table, for each instruction that may fault) for the faulting `EPC` and the copy returns an error.

A timer interrupt. The timer is vector 32 at level 6 (6.3.6, 6.3.7). It is the pdp64's `hardclock`:

```
timer_interrupt:                        # vector 32; entered at priority 7
    PUSHX
    save the registers the C code will clobber
    SPL #6                              # at the device's own level: only a higher one may interrupt
    tick()                              # advance time; charge the running process; run due timeouts
    if --current.time_slice <= 0: need_resched = 1
    SPL #7
    restore the registers
    if need_resched and the interrupted mode (EPSW) was user:
        call switch()                   # V7: "runrun"; see below
    POPX
    RTI
```

(The V7 `call` routine does the same test, `previous mode = user` and `_runrun`, and makes an artificial trap number 12, "give up the cpu", before it returns; checked in `conf/mch_i.s`.)

A context switch. The kernel has decided to run another process. The registers the user program had are already saved in the trap frame on its own kernel stack, so the switch deals only with the kernel's state. The picture shows what is touched, and the code does it in four steps (the V7 equivalent is `swtch()`, `resume()` and `sureg()`):




```
switch_to(old, new):                    # kernel mode, priority 7
    push the callee-saved registers (appendix D) on old's kernel stack
    old.ksp = R14                       # R14 in kernel mode is the kernel stack pointer
    R14     = new.ksp                   # one ordinary move: the stack changes here
    pop the callee-saved registers from new's kernel stack
    if new.space != old.space:
        MTSR PTBR, new.space.root       # the new top-level table
        MTASID new.space.asid           # the new address-space number
        ISYNC                           # both take effect now (6.5.2)
    return                              # to wherever new was when it called switch_to
                                        # (the return address was on the stack just loaded)

return_to_user(p):
    MTSR USP, p.user_sp                 # the user's own stack pointer; R14 is the kernel's
    POPX                                # EPC and EPSW from p's trap frame
    RTI                                 # user mode, user PC
```

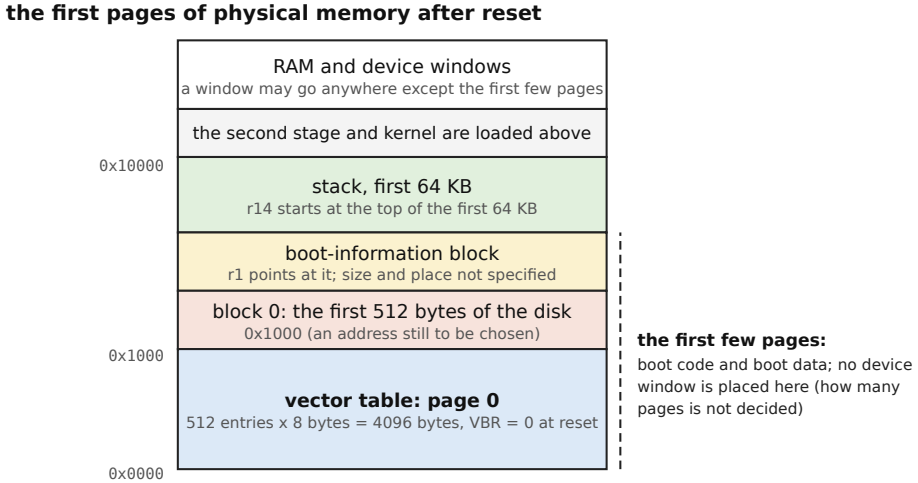
No entry in the translation buffer is touched: the entries of the process that has just stopped stay, tagged with its ASID, and are used again if it is run again soon; the kernel's own entries are global and are used by both. How the kernel saves and restores a thread's **user** `DP` (the thread pointer, a banked register of user mode) when it switches threads is **not specified**: the system-register table (6.5.1) has no number for it, and `MTDP` and `MFDP` name only the current mode's copy. This is listed as open in 6.15.

The idle loop. When no process is runnable the kernel waits for an interrupt. The PDP-11 did it with `wait`, after lowering the priority so that an interrupt could be taken (the V7 and 2.11BSD `idle` routine does exactly that: `SPLLOW`, `wait`, restore). The pdp64's `WFE` (wait for event) is the idle instruction (7.6.3), with the sequence of the proposal 7.6.2, in which `WFE` ends its sleep for an interrupt that is pending even though it is masked, so that there is no gap between testing for work and sleeping:

```
idle():
    loop:
        SPL #7                          # mask everything while we look
        if run_queue is not empty:  SPL #0 ; return
        WFE                             # sleep until an event or an interrupt, masked or not
        SPL #0                          # now take the interrupt that woke us
```

6.12 Reset, the first pages and the first kernel

What the machine contains at power-on. There is no kernel yet; there is a boot ROM (chapter 4) that reads one block into low memory and starts core 0. The architecture fixes that state, and the first few pages of physical memory are laid out as follows (the vector table, the block, the information block and the stack):



(not to scale; the place and size of the boot-information block are not specified)
After reset: kernel mode, priority 7, translation off (every address physical), VBR 0, guard clear.
The PDP-11 kept its vectors at 0 to 0376 octal and booted from elsewhere; here the table is one whole page at 0.

The **vector table** is 512 × 8 = 4096 bytes: exactly one page, at address 0, because `VBR` is zero at reset; the kernel moves it by writing `VBR` (and `ISYNC`). That is where the PDP-11 also had its vectors, but it was a fixed table of a few hundred bytes (V7's `l.s` runs to octal 0114 and beyond with devices), and the boot ROMs lived elsewhere; the pdp64's block 0, the 512 bytes the ROM reads from the disk, goes to **0x1000**, an address still marked "to be chosen" (`docs/boot.md`), so the first stage can run above the table. The **boot-information block** (memory ranges, core count, the address of the root complex, the boot device, the console, a checksum and a version, 4.6.2) is pointed to by `r1`; its size and place are not specified. **`r14`**, the stack pointer, is the top of the first 64 KB of memory. The rule of errata 102 is that no device window is mapped in **the first few pages**; how many that is, is not decided (the briefing, `A.md` item Q1, sets out six options from one page to a megabyte and leaves it to the author), and the 64 KB of `r14` is only the stack's place.

6.12.1 At reset core 0 starts in kernel mode at priority 7 with translation off, the guard clear and `VBR` zero; the kernel sets `VBR`, builds page tables, writes `PTBR`, sets `MMUCTL` bit 0 and executes `ISYNC`. *(D for the QEMU machine; P for the architectural reset, chapter 4)*

6.12.2 The QEMU machine starts at the entry point of the ELF given with `-kernel`, with the kernel stack at the top of memory; the option `kernel-mode=on` turns off the system-call stand-in of user-mode programs so that `TRAP` enters the kernel. *(D)*

6.12.3 The first operating system is OpenBSD, with Unix/32V as the fallback if OpenBSD proves too hard. *(D)*

The first kernel, in outline (illustration):

```
kernel_entry:                           # core 0, kernel mode, priority 7, translation off
    R14 is already a stack (the top of the first 64 KB)
    MTSR VBR, kernel_vector_table       # the table the kernel built or linked
    ISYNC
    read the boot-information block at r1 : memory ranges, core count, root complex
    build the kernel's page tables (the upper half, global pages; section 6.9)
    MTSR PTBR, user_root_for_process_0  # the lower half; ASID 0
    MTSR MMUCTL, 1                      # bit 0: translation on
    ISYNC                               # from here every address is virtual (see below)
    enumerate the bus (section 2.4); start the other cores (7.8); start the clock; run init
```

What is not settled. The instruction after `ISYNC` is fetched through the new tables, so the code that is running must already be reachable at its virtual address; the repository does not say how. The older text said the boot code "runs at physical addresses until the kernel is ready, with no identity-map trampoline"; that is true up to the enabling write, and afterwards only if either the kernel maps its own code at the address it is running at, for that moment, or a direct-mapping window covers

physical memory. The format of the windows is open (6.9.2), so the second way cannot be relied on yet. The question is in 6.15.

Commentary. **Precedent.** The PDP-11 and VAX turned the memory manager on with a bit in a status register and a jump to a mapped address, each needing a transition in the boot code. **Trade-offs.** Translation-off at reset is the same arrangement, and the single bit is a machine-wide mode bit accepted for that reason. **Benefits.** The boot code runs at physical addresses until the kernel is ready. **Modern comparables.** RISC-V's `satp` with a bare mode, AArch64's `SCTLR.M`. **Impact.** *Kernel:* Boot code runs at physical addresses until the kernel is ready; the transition into translation is one write and one `ISYNC`, and what covers the next fetch is the question above. *Compiler:* The first kernel code is position-independent and uses no virtual address until translation is on. *Hardware:* Translation-enable is one bit.

6.13 System instructions, exceptions and interrupts

The statements of 6.2 to 6.8 summarised once more as the vocabulary of the kernel; the entries of the instructions are in chapter 9. This section keeps the statements that the earlier text gave in one place.

6.13.1 There are three modes (kernel, supervisor, user); supervisor is defined and unused. System instructions executed outside kernel mode abort with the privileged-instruction vector. *(D)*

6.13.2 `TRAP #n` (0 to 15, two bytes) and `TRAPL #n` (vectors 0 to 255) enter the kernel; `RTI` returns, and `RTT` returns and sets the trace bit after. *(D)*

6.13.3 An exception or interrupt writes only registers: `EPC` receives the address of the faulting instruction (or the next one, for a trap or interrupt), `EPSW` the old status word, `EVA` the faulting address and `ECAUSE` the vector number; the new program counter is the 64-bit word at `VBR + 8 × vector`. *(D)*

6.13.4 Entry sets the priority to 7 for every exception and every interrupt; the handler lowers it. *(D)*

6.13.5 There are eight interrupt priority levels; `SPL #n` sets the priority without touching the flags. *(D)*

6.13.6 `MFSR` and `MTSR` read and write the system registers by number; `TLBI`, `TLBIA` and `TLBSYNC` invalidate translations (broadcast to other cores); `MTASID` and `MFASID` set the address-space tag; `ICINV` and `ICINVA` invalidate the instruction cache; `PROBER` and `PROBEW` test an address; `MFCPUID` identifies the processor. *(D)*

6.13.7 `UA` in the status word guards user memory: with it clear, a kernel-mode data access to a user page faults; `STUA` sets it and `CLUA` clears it; entry clears it. *(D)*

Commentary. The modes and exceptions are sections 6.2 to 6.4 and the multiprocessor instructions are chapter 7. **Precedent.** The PDP-11 pushed the program counter and status word on a stack and loaded new ones from a two-word vector; its trap vectors conventionally held priority 7. **Trade-offs.** Entering in registers is short and cannot fault, and costs the handler an explicit save; raising the priority on entry closes the window in which an interrupt could overwrite the saved state. **Improvements.** `PUSHX` and `POPX` save the pair in one instruction, and the user-memory guard turns a stray kernel pointer into a clean fault. **Benefits.** The kernel survey estimates system-call entry at four or five instructions against 45 to 70 on RISC kernels (a paper estimate, not a measurement). **Modern comparables.** AArch64 and RISC-V save the return address and status in registers on exception entry; x86 pushes five words; x86's `SMAP` with `STAC`/`CLAC`, AArch64's `PAN` and RISC-V's `SUM` are the user-memory guards. **Use.** `PROBER` and `PROBEW` replace the `nofault` idiom of V6 and V7; `ISYNC` follows a write to the page-table base or the vector base. **Impact.** *Kernel:* This is the kernel's whole instruction vocabulary: `MFSR`, `MTSR`, `TLBI`, `RTI`, `SPL`, `ISYNC`, the probes. *Compiler:* The compiler never emits them except through intrinsics. *Hardware:* The system group takes its sub-opcode from the register field, and the privilege check is one comparison on the current mode.

The instruction entries for this section are in chapter 9, section 9.14: each entry there is complete (syntax, encoding diagram, operation, flags, exceptions, status); the commentary above gives the reasons and the use.

Entry	Mnemonics
No operation and landing pad	`NOP`, `LPAD`
Return from interrupt or exception	`RTI`, `RTT`
Return from subroutine	`RET`
Halt the processor	`HALT`
Breakpoint	`BPT`
Memory fence	`FENCE`
Wait for an event	`WFE`
Send an event	`SEV`

Entry	Mnemonics
Invalidate all translations; wait for translation invalidations; invalidate the whole instruction cache	TLBIA, TLBSYNC, ICINVA
Push and pop the exception state	PUSHX, POPX
Instruction synchronisation	ISYNC
Hint	HINT
Move from and to a system register	MFSR, MTSR
Move to and from the processor status word	MTPS, MFPS
Set the interrupt priority level	SPL
System call and trap	TRAP, TRAPL
Invalidate a translation	TLBI
Move to and from the address-space identifier	MTASID, MFASID
Move to and from the direct pointer	MTDP, MFDP
Move to a direct-mapping window	MTDMW
Read the processor identifier	MFCPUID
Invalidate the instruction cache for an address	ICINV
Probe an address for read or write access	PROBER, PROBEW

6.14 What was checked, and what was not

The rule of the project is: check, do not guess. For the facts of this chapter about older and other machines, the handbooks themselves are not on this machine. What was available, and what each fact rests on:

- **Checked against the Unix sources on this machine** (`/home/david/claude/facetlang/unixtree`: the V7M kernel under `V7M/sys` and 2.11BSD under `2.11BSD/sys`): the V7 vector table (`conf/l.s`: the pairs `trap; br7+n`, the console at 060 and 064 at `br4`, the clock at 0100 at `br6`) and the 2.11BSD table (`pdp/scb.s`); `trap(dev, sp, r1, nps, r0, pc, ps)` and its comment; the system-call dispatch through `sysent[fuiword(pc-1)&077]`; the KT11 register addresses and the processor-status fields in `pdp/seg.h` and `pdp/psl.h` (`UISA`, `UISD`, `KISA0`, `PSL_CURMOD`, `PSL_PRVMOD`, `PSL_IPL`, `PSL_T`); `estabur()` and `sureg()` (eight page-register pairs per mode, sixteen with separate instruction and data space; `ENOMEM` for a ninth segment; the block of 64 bytes and the segment of at most 128 blocks); the first-fit core allocator (`malloc.c`); `swtch()`, `resume()` and `setjmp()`; `mfpi`, `mtpi`, `mfpd`, `mtpd` in `fubyte` and `subyte`, with the `nofault` global; the `wait` of the idle routine; the artificial `T_SWITCHTRAP` of the interrupt exit when `_runrun` is set; the comment that the floating exception is an imprecise trap and that `backup()` may fail on the 11/40; supervisor mode used by the 2.11BSD network (`SUPVMODE`); `sched()`, `swapout()` and `swapin()`.
- **Checked against the QEMU sources** (`/home/david/claude/qemu-src`): RISC-V's `satp` modes 8, 9 and 10 (Sv39, Sv48, Sv57), its 16-bit address-space field, its page-table-entry bits `V R W X U G A D` in bits 0 to 7 and its page number at bit 10; x86's `CPUID` leaf 0x80000008 and the `PCIDE` bit of `CR4`; AArch64's `ID_AA64MMFR0` field `PARange`; LoongArch's `CPUCFG1` fields and `PWCL` register.
- **Checked against the repository:** the statements of this chapter against `docs/pdp64-isa.md` sections 7 and 8, `docs/pdp64-system-architecture.md`, `docs/errata.md` (60, 65, 71, 96, 97, 100, 102, 103) and the walker of the QEMU model (`qemu/target-pdp64/mmu.c`), which implements the walk of 6.6 with three levels, the reserved-bit and alignment checks, and software accessed and dirty bits, and does not implement the split of 6.9.1 or the windows.
- **The repository's sizing, not measured:** the plan for the translation buffer on the FPGA, four-way and 64 entries (`docs/pdp64-system-architecture.md`).
- **Unverified** (general knowledge, the handbooks not being on this machine): the names **APF** and the layouts of the **PAR** and **PDR** (the 2.11BSD code shows the length field and the access and extension bits, not the names); that the KT11 on the original models gave 18-bit physical addresses and 22 bits on the later ones (the 2.11BSD source speaks of "18 bit controllers present in 22-bit q-bus systems", and the Unibus width of 18 bits is in the repository's text); that the I/O page is the top 8 KB of the physical space; the bit positions of the status word apart from those 2.11BSD's header gives (the fixed address 0177776 was checked for the bus chapter, 2.12); `MMR1` as the register that recorded the auto-increment changes (the repository's text says so; the 2.11BSD `backup()` routine is the user of it); the VAX's regions, its modify bit and the absence of a reference bit (repository text); that the VAX divided its address space into a process half and a system half; that MIPS `KSEG0` is a fixed unmapped region; the width of x86's process-context tag; AArch64's two translation-base registers; that the speculative-execution flaw of 2018 is the reason for the split of 6.9.1; the usual meanings of "strongly ordered" and "weakly ordered" for device memory; which instructions the PDP-11 forbade in user mode (`halt`, `reset`, `wait`, writes to the priority); that x86 kernels removed the kernel half from the user's tables only after 2018 (the switch on every entry); that current architectures keep exceptions precise for page faults.

- **Decided by the author and recorded in `errata.md`**: everything stated as a decision with a tag *(D)*; the illustrations in 6.11 and 6.12 are not decisions.

6.15 Still open

Decisions the author has asked for, or that follow from the author's, and that are not made: (a) the register or field that tells software how many levels the machine has and how wide its physical address is (6.6.1a; errata 96, 102(b)); (b) the format of the page-table entry for a physical address wider than 40 bits (6.7.1; errata 102(c)); (c) KTBR: its system-register number and the layout of the top-level tables of the two roots, and whether the user half is the lower half in every case (6.9.1); (d) the format of the direct-mapping windows DMW0 and DMW1, which are stored and have no effect (6.9.2); (e) how many pages "the first few" are, and the address of block 0 (6.12; errata 102(a)); (f) what covers the fetch after the write that turns translation on (6.12).

Smaller items, from the older text and from writing this chapter: the use of supervisor mode; hardware breakpoints and watchdogs; performance counters; machine-check reporting beyond halting; feature and identification registers from register 13 up; a translation-buffer invalidate by address-space tag, and the policy when the 16-bit tags run out; how a thread's user DP is saved and restored by the kernel (6.11); which registers carry the system-call number and arguments (6.10.2: "by the ABI", and appendix D does not give the convention); whether the software-managed accessed and dirty bits are to be joined by a hardware-managed option advertised in a feature register (6.8); and the flush of the whole translation buffer on a write to PTBR, ASID or MMUCTL that the entry of MFSR states and the reference executor performs, against the purpose of the ASID (6.8); and the check of the memory type against bit 39 that the implementation still performs (6.7), which errata 97 removed from the architecture (appendix B).

Chapter 7 — Multiple processors

The pdp64 is built so that adding a second core needs seventeen operations, one per-core register, and nothing a kernel writer has to work around: memory is sequentially consistent, the atomics are one instruction each, the per-CPU pointer is a register that already exists, and the other cores' translation buffers are invalidated by a single broadcast. The chapter collects everything the architecture adds for symmetric multiprocessing, what a kernel does with it, what is left out, and what the device side supplies. The instruction-set text is `../pdp64-isa.md`, section 17. Tags: *(D)* decided; *(P)* proposed.

7.1 The seventeen operations

7.1.1 The instructions that exist for multiple processors fall under one umbrella of seventeen operations: ten atomics and seven others. *(D)*

Group	Operations
Atomics (10)	compare-and-swap (`CAS`), swap (`SWAP`), fetch-and-add (`XADD`), fetch-and-and (`XAND`), fetch-and-or (`XOR_`), fetch-and-exclusive-or (`XXOR`), fetch-and-maximum (`XMAX`, with `XMAXU`), fetch-and-minimum (`XMIN`, with `XMINU`), load-linked (`LL`), store-conditional (`SC`)
Others (7)	`FENCE`, wait-for-event (`WFE`), send-event (`SEV`), `TLBI` (with `TLBIA` as its "all" form), `TLBSYNC`, instruction-cache invalidate (`ICINV`, with `ICINVA`), read the core number (`MFCPUID`)

7.1.2 Every atomic has a variant, `_DB`, that folds a barrier into the instruction, so that a lock acquire or release is one instruction. *(D)*

7.1.3 The atomics live in the atomic escape group and the other operations in the system group; the umbrella is conceptual, not an encoding. *(D)*

7.1.4 Supporting these are the banked direct-page register, the address-space tag, the global-page bit and the banked stack pointer (section 7.4). *(D)*

Commentary. **Precedent.** The PDP-11/70's read-modify-write instructions were indivisible on the Unibus (`DATIP` and `DATO`); the VAX added interlocked instructions (`BBSSI`, `ADAWI` and the queue instructions) and multiprocessor models, and the 4.3BSD kernel on them still used `spl` for exclusion. **Trade-offs.** The set is the union of the three designs found on other machines (compare-and-swap, load-linked and store-conditional, fetch-and-operate), which is more atomic operations than any architecture surveyed, in exchange for nothing a kernel writer has to emulate. **Improvements.** The `_DB` forms were taken from LoongArch: a lock acquire should be one instruction, with no separate fence. **Benefits.** The seven non-atomic operations are the minimum that a kernel cannot synthesise: which core am I, wait and signal, invalidate everywhere, and make the instruction fetch see what the data side wrote. **Modern comparables.** RISC-V's `LR`/`SC` and `AMO` set, AArch64's exclusive loads and stores and its large-system atomics, x86's `LOCK`-prefixed read-modify-write. **Impact.** *Kernel:* Each of the seventeen operations is one instruction in the kernel's lock, idle, shootdown and per-CPU code. *Compiler:* C11 atomics and the kernel's intrinsics map to them directly. *Hardware:* The atomics live in one escape group and the other seven in the system group, so the decoder has two places to look.

7.2 The memory model

7.2.1 Memory is sequentially consistent: the result is as if all cores' accesses happened in one order that respects each program's order. *(D)*

7.2.2 A store buffer is allowed, provided loads snoop it and stores drain in program order. *(D)*

7.2.3 Quad accesses are naturally aligned, so every access is atomic within its own size; no misaligned access can be torn across cores. *(D)*

7.2.4 The machine is in order and does not speculate, so no speculative side channel exists to be mitigated. *(D)*

7.2.5 The ABI says nothing about memory ordering, because with sequential consistency there is nothing to say. *(D)*

7.2.6 Memory is sequentially consistent per byte: a load returns, for each byte, the value written by the last store to that byte in the single order, and an aligned access of N bytes is single-copy atomic. A load that overlaps a buffered store of a different size either waits for the buffer to drain or merges the buffer and memory in one step; it never returns a mixture that no order of the stores could produce. *(D)*

Commentary. **Precedent.** The PDP-11 and VAX ran one processor at a time in the model that mattered, so the question did not arise; ARM's memory model took about a decade to formalise, and the C11 and C++11 models exist largely to paper over the differences between weak architectures. **Trade-offs.** Sequential consistency is the model with no cost to software and the highest cost to a large speculative core; software written against a weak model runs on a strong machine and not the reverse, so the choice is close to irreversible, which is why the choice is made once. If a large out-of-order core is ever built, total store order is the least painful retreat. **Benefits.** None of the barrier code that fills other kernels: the kernel survey found barrier vocabulary in 5,596 lines of Linux code and 3,016 lines of `memory-barriers.txt` documentation, and none of it arises here. **Modern comparables.** x86 and SPARC are total store order; ARM, POWER and RISC-V are weak. **Impact.** *Kernel:* None of the 5,596 lines of ordering code found in Linux, nor the barrier documentation, is needed. *Compiler:* The optimiser may move non-volatile accesses freely, and no fence is inserted for ordering. *Hardware:* In-order with a snooped store buffer; sequential consistency is nearly free on this core and the expensive part only appears on a speculative one.

7.3 The atomics in detail

7.3.1 `LL` takes a reservation on a granule of memory; a write to the granule by any initiator, core or device, clears it; `SC` succeeds exactly when the reservation is intact and otherwise writes nothing. *(D)*

7.3.2 A short `LL`/`SC` sequence must eventually succeed if no other initiator keeps writing the granule. *(D)*

7.3.3 The read-modify-write atomics are single-copy atomic with respect to every initiator; the memory system may build them from an exclusive access with hardware retry. *(D)*

7.3.4 A device that writes memory shared with a core takes part by clearing reservations. *(D)*

Commentary. **Precedent.** A bus lock, the Unibus's `DATIP`/`DATO` pair, is simple and a bottleneck as cores multiply; a monitor maps onto the exclusive access of AXI and the memory systems of FPGAs and modern chips. **Trade-offs.** A monitor needs forward-progress rules and, under contention, can livelock if not specified; RISC-V's guarantee of eventual success for constrained sequences is the model. The pain-points report records that the kernels' simple lock does not scale (OpenBSD's ticket kernel lock measured 155% worse than an Anderson lock in one developer's test) and that Linux moved from test-and-set through ticket locks to queued locks; the instruction set provides the primitives and the lock design is the kernel's. **Benefits.** A kernel's lock needs one instruction; lock-free queues need compare-and-swap. **Modern comparables.** RISC-V `LR`/`SC` with its constrained-loop guarantee, AArch64 load-exclusive and `CAS`, x86 `LOCK CMPXCHG`. **Use.** `CAS` for lock-free structures and lock words; `XADD` for counters and ticket numbers; `LL`/`SC` for everything else. **Impact.** *Kernel:* A lock is one `_DB` instruction. *Compiler:* A `LL`/`SC` loop must be emitted in the constrained form, with no other memory access between the two. *Hardware:* A reservation granule tag in the cache controller and an exclusive-access path to memory.

7.4 Per-CPU state and cheap context

7.4.1 `MFCPUID` returns the number of the core that executes it. It cannot be synthesised and per-CPU data needs it. *(D)*

7.4.2 `DP`, the direct-page register, has a copy per mode; in kernel mode it is the per-CPU pointer and a field of the per-CPU structure is one four-byte instruction, `MOVQ d(DP), Rd`. *(D)*

7.4.3 The address-space tag makes a context switch constant-time, and global page-table entries and the direct-mapping windows keep the kernel's own mappings out of the invalidation path. *(D)*

7.4.4 The stack pointer is banked by mode, so an interrupt arriving in user mode on any core finds that core's kernel stack without a search. *(D)*

Commentary. **Precedent.** Every SMP kernel builds a per-CPU base by hand: x86 uses the `GS` base and a `swapgs` instruction, ARM uses `TPIDR_EL1`, RISC-V the supervisor scratch register, MIPS reserves two registers; the kernel survey found seven architectures paying for the problem of having no free register at the moment of entry. **Improvements.** `DP` serves code density and is also what both an SMP kernel and a threaded C runtime need most, in the mode where each needs it, and the two copies do not conflict. **Benefits.** No `swapgs`, no reserved scratch register, no code spent finding the CPU. **Modern comparables.** `GS` and `TPIDR` as above. **Impact.** *Kernel:* Per-CPU data is one four-byte `MOVQ d(DP), Rd`, with no `swapgs` and no

reserved register. *Compiler:* The compiler addresses per-CPU variables relative to `DP`. *Hardware:* Three copies of `DP`, one per mode.

7.5 Translation and instruction-cache coherence

7.5.1 `TLBI` and `TLBIA` broadcast to all cores; `TLBSYNC` waits until the broadcast has completed everywhere. *(D)*

7.5.2 Instruction caches are not coherent with data caches; `ICINV` invalidates a line and `ICINVA` all of it, and neither is privileged, because a JIT needs them. *(D)*

7.5.3 A write to the page-table base, the address-space tag, the translation-enable bit or the vector base takes effect only after the next `ISYNC`. *(D)*

Commentary. **Precedent.** x86 flushes remote translation buffers with an inter-processor interrupt and a software handshake, which is slow and intricate; AMD added a broadcast invalidate only late. **Trade-offs.** A broadcast is simpler in hardware and far faster for two to four cores; it needs the completion barrier, because a broadcast one cannot wait for is no use. Making instruction caches coherent would cost a snoop port for a case that almost never happens. **Improvements.** The kernel's own mappings are global or window-mapped and never part of a shootdown. **Benefits.** The pain-points report finds that remote translation-buffer shootdown, a source of deadlocks in the BSDs and of much code in Linux, does not arise. **Modern comparables.** ARM's broadcast `TLBI` and `DSB`; RISC-V's `SFENCE.VMA` still needs software shootdown. **Impact.** *Kernel:* A shootdown is `TLBI` then `TLBSYNC`; a module or JIT write is `ICINV` then `ISYNC`. *Compiler:* A JIT or a trampoline generator must call `ICINV` after writing code. *Hardware:* A broadcast on the interconnect and a counter for completion; the instruction cache has an invalidate port but no snoop port.

7.6 Waiting and signalling

7.6.1 `WFE` sleeps until another core executes `SEV` or an interrupt arrives, and `HINT #1` is the spin-wait pause. *(D)*

7.6.2 `WFE` ends its sleep when an interrupt is pending even if the priority masks it, so that the idle loop is `SPL #7`, check for work, `WFE`, `SPL #0`, with no gap. *(P)*

7.6.3 On one core `WFE` is the idle instruction, and `WAIT` is its assembler alias. *(D)*

Commentary. **Precedent.** x86's `PAUSE` tells a spinning core to back off; ARM's `WFE` lets it stop until an event. V7's idle routine lowered the priority and executed `wait` in a loop that rechecked the run queue after every interrupt, so a missed wake-up cost one clock tick. **Trade-offs.** On an FPGA a spinning core consumes memory bandwidth the other cores want, which is the reason to prefer a real wait. **Benefits.** An idle core costs nothing and wakes on a signal. **Modern comparables.** ARM `WFE` and `SEV`, RISC-V `WFI`. **Impact.** *Kernel:* An idle core waits on `WFE` and costs no bandwidth. *Compiler:* Spin-wait loops use `HINT #1`. *Hardware:* `WFE` stalls or gates the core's clock until an event or an interrupt.

7.7 Cache coherence

7.7.1 Between processors, coherence is transparent to software, by snooping with three states (modified, shared, invalid), for the four cores of the first device. *(D)*

7.7.2 A DMA device is the opposite case: the buffers are uncached and the hand-off is explicit, because snooping the DMA path is expensive and DMA transfers are large, infrequent and have an explicit hand-off. *(D)*

7.7.3 The design supports at least four cores; sixteen or more is the preferred ceiling that no decision may preclude; the initial device has four cores, which it is hoped will fit the FPGA. *(D)*

7.7.4 The protocol for more than four coherent cores is open. The leaning of the SMP decisions is MESI with a snoop filter inside clusters of four and a directory between clusters, with dedicated cores as non-coherent partners that use uncached rings; the architecture promises only that memory is coherent and sequentially consistent. *(P)*

Commentary. **Trade-offs.** MESI would add an exclusive state to avoid a bus transaction when writing a clean line nobody else holds; at four cores the benefit is small and the state machine larger; a directory scales further and is not needed below about eight cores. Software-managed coherence is cheaper in hardware and is where concurrency bugs live, since every missing flush is a race that reproduces once a week. **Benefits.** The instruction set owes nothing to the coherence protocol, which keeps the two separate. **Modern comparables.** x86 snoops with MESIF or MOESI and directories across sockets; ARM's coherent interconnects use directories or snoop filters. **Impact.** *Kernel:* No coherence code, in either direction between cores. *Compiler:* None. *Hardware:* A snoop port on each data cache, about 500 lookup tables and a small state machine for three states, plus about 600 per core.

7.8 Starting and signalling the other cores

7.8.1 The controller holds every core but core 0 in reset; a write to a per-core start register gives a core an entry address and an argument and releases it into kernel mode at priority 7, with its number in `r0`. *(P)*

7.8.2 An inter-processor interrupt is a write to a per-core register in the interrupt controller; no APIC-like machinery is needed. *(P)*

7.8.3 The routing of device interrupts to cores, and a shared second-level cache or a coherent interconnect, are device-side and unspecified. *(P)*

7.8.4 Each core has a read-only identification register giving its class and capacity, defined from the first system although every core is alike, and a stop-and-drain register that a kernel writes to take a core offline: the core finishes its outstanding bus transactions and stops. *(D)*

7.8.5 In-order and out-of-order cores may share a die if software can tell which is which, which the class register of 7.8.4 reports; the memory model and the semantics of every instruction are identical on every core, and a core class may differ in speed and never in what a program can observe. *(D)*

7.8.6 At least one core is a secure core: every timing channel is removed (every instruction and every cache fetch takes a time independent of its data), it has private cache lines and takes no periodic timer tick, and it is stopped and preempted by an inter-processor interrupt. *(D)*

7.8.7 The author's recommendation is dedicated cores for graphics, for the network and for audio, then general-purpose cores. *(P)*

Commentary. **Precedent.** An x86 core wakes in 16-bit real mode and has to be walked through three modes by a trampoline (1,528 lines of start-up code and 1,258 of real-mode code in Linux); arm64 uses a firmware call in 124 lines or a spin table in 127; RISC-V 108 or 58; the VAX's console processor started the others. **Benefits.** A core can start in its normal mode at any address, so the whole protocol is one register write. **Modern comparables.** x86's INIT and START-UP interrupts, ARM's PSCI, RISC-V's hart-start call. **Impact.** *Kernel:* Starting a core is one register write, and a stuck core is reset by another. *Compiler:* None. *Hardware:* A start register and a reset-hold per core in the controller.

7.9 A kernel's view

7.9.1 The kernel starts a core by writing its start register, keeps its per-CPU structure at the address in `dp`, and takes a lock with one `_db` atomic. *(P)*

7.9.2 It unmaps a page by changing the entry, issuing `tlbi`, and waiting with `tlbsync`. *(D)*

7.9.3 It makes new instructions visible by writing them, issuing `icinv` and executing `isync`. *(D)*

7.9.4 It idles a core with `wfe` and wakes it with `sev` or an inter-processor interrupt. *(P)*

Commentary. Each item is one instruction or one register write, and each is a place where other architectures need a trampoline, a handshake or a fence.

7.10 What was avoided

7.10.1 The architecture has no weak ordering, no speculative execution, no hardware transactional memory, no software-managed coherence between processors, no coherent instruction caches, no inter-processor-interrupt shootdown as the main mechanism, and no memory-model-dependent ABI. *(D)*

Commentary. **Trade-offs.** Intel's transactional memory produced a decade of errata and side channels for a feature few programs used; compare-and-swap and `ll`/`sc` suffice. Each avoided item is a tax on every concurrent program, or on every implementation, in exchange for performance the small in-order core cannot use. **Impact.** *Kernel:* Each avoided feature is code a kernel does not have to write: no barriers, no software coherence, no shootdown interrupts. *Compiler:* Each is analysis the compiler does not have to do: no memory-model-dependent code generation. *Hardware:* Each is a block not built: no speculation, no transactional memory, no coherent instruction cache.

7.11 What the model does not give for free, and the road to out of order

7.11.1 A simple memory model does not make an atomic implementation simple by itself: cache coherence, exclusive ownership, invalidation, ordering and the interaction with interrupts still have to be designed and tested.

7.11.2 Out-of-order execution is not ruled out: the first implementations are in order, and the architecture is kept ready for a large out-of-order design. *(D)*

7.11.3 Sequential consistency on a speculative core needs loads that are replayed when another core intervenes; the readiness review calls this awkward but defined, and names the flag rules, the reservation rules and the `ISYNC` and `HINT` instructions as what a speculative core needs. *(D)*

Commentary. **Precedent.** One instruction set often spans small in-order and large out-of-order cores: ARM's small and large cores, RISC-V's Rocket and BOOM, the x86 Pentium and P6, Alpha's 21064 and 21264. **Trade-offs.** What the pdp64 gains by refusing speculation is a smaller design with fewer error cases and none of the speculative side channels; what it gives up is the performance of an aggressive core, which an FPGA or a small chip could not build anyway. The readiness document (`../pdp64-ooo-readiness.md`) and the hardware-vulnerabilities report give the rules a future core would follow. **Benefits.** The decision is reversible in the direction that matters: total store order can be the retreat for a large core. **Impact.** *Kernel:* None. *Compiler:* None. *Hardware:* A speculative core would need loads that replay when another core intervenes, and the flag and reservation rules keep that path open.

7.12 The atomic and ordering instructions

7.12.1 The atomic escape provides `CAS`, `SWAP`, `XADD`, `XAND`, `XOR_`, `XXOR`, `XMAX`, `XMIN`, `XMAXU` and `XMINU`, each with a `_DB` variant that adds a barrier, and `LL` and `SC`. *(opcode `0x11`)*

7.12.2 `LL` takes a reservation on a granule of memory; any write to it by any initiator clears it; `SC` succeeds exactly when the reservation is intact. A short `LL`/`SC` sequence must eventually succeed if no other initiator keeps writing the granule. *(D)*

7.12.3 The read-modify-write atomics are atomic with respect to every initiator, cores and devices alike. *(D)*

7.12.4 `FENCE` orders device accesses; `WFE` and `SEV` wait for and signal an event; `ISYNC` makes earlier system-register writes take effect; `HINT #n` has no architectural effect (1 pause, 2 speculation barrier, others reserved). *(D)*

Commentary. **Precedent.** The PDP-11's read-modify-write instructions were indivisible on the Unibus (the `DATIP`/`DATO` pair); the VAX had interlocked queue and bit instructions. **Trade-offs.** A bus lock is simple and does not scale as cores multiply; a monitor (reservation) maps onto the memory systems that FPGAs and modern chips use, at the cost of forward-progress rules. **Improvements.** The pdp64 offers all three designs found on other machines (load-linked/store-conditional, compare-and-swap, fetch-and-operate), more atomic operations than any architecture surveyed. **Benefits.** A kernel lock needs one instruction. **Modern comparables.** RISC-V `LR`/`SC` and the `AMO` set; AArch64 load-exclusive and its large-system atomics; x86 `LOCK`-prefixed read-modify-write. **Use.** `CAS` for lock-free structures; `XADD` for counters; `LL`/`SC` for anything else; `WFE` for the idle loop. **Impact.** *Kernel:* A lock is one instruction, and the `_DB` forms need no separate fence. *Compiler:* C11 atomics map one-to-one, no ordering fences are emitted under sequential consistency, and a `LL`/`SC` loop must be generated in the constrained form (short, no other memory access inside). *Hardware:* The reservation is a granule tag in the cache controller, and the read-modify-write atomics use an exclusive access with hardware retry.

The instruction entries for this section are in chapter 9, section 9.13: each entry there is complete (syntax, encoding diagram, operation, flags, exceptions, status); the commentary above gives the reasons and the use.

Entry	Mnemonics
Compare and swap	`CAS`, `CASDB`
Atomic exchange	`SWAP`, `SWAPDB`
Atomic fetch and add	`XADD`, `XADDDB`
Atomic fetch and and, or, xor	`XAND`, `XANDDB`, `XOR`, `XOR_DB`, `XXOR`, `XXORDB`
Atomic fetch and minimum or maximum	`XMAX`, `XMAXDB`, `XMIN`, `XMINDB`, `XMAXU`, `XMAXUDB`, `XMINU`, `XMINUDB`
Load-linked and store-conditional	`LL`, `SC`

7.13 Still open

The mapping of the bus's atomics onto PCI Express (the transport of decision R4 is settled by errata 80), the interrupt controller and routing (R1), starting the other cores (R2, B2), how coherence scales beyond four cores (R3), the wake rule of `WFE` (P1), and lock scaling above a few cores, which is a kernel design question the instruction set does not settle.

Chapter 8 — Vector work, AI and array languages: the device route

Wide data-parallel work, whether neural-network inference or the primitives of an array language, is done by memory-mapped devices with their own memory and driven by descriptors, not by SIMD registers in the processor. There is no vector register file and no AVX2-style instruction set in the pdp64. The chapter states that decision, the reason for it, and how it serves AI and large-language-model inference and the array languages APL and K. Graphics has its own chapter (chapter 3), since its devices are the first a user sees. It is the route ARM-based systems take for their neural-network and graphics engines, and the one the PDP-11 took for its attached array processors. The discussion of the alternatives is appendix B (S1 to S6); the accelerator designs are `../pdp64-lb64-accelerator.md` and section 12 of `../pdp64-fpga-plan.md`. Tags: *(D)* decided; *(P)* proposed.

8.1 The decision

8.1.1 The processor has no vector register file and no SIMD instruction set of the SSE, AVX2 or NEON kind. *(D)*

8.1.2 Wide work is described to a device by descriptors in memory: an operation, the addresses of the arrays and their length. *(D)*

8.1.3 The processor owes such a device what it owes every device: a memory type per page, a FENCE, and atomics that reach device memory; the instruction set is unchanged. *(D)*

8.1.4 Packed-lane operations in the ordinary integer registers (a handful of instructions for bytes, halves and words in a 64-bit register) are not part of this decision; they remain a possibility to be settled by measurement. *(P)*

Commentary. **Precedent.** The PDP-11's heavy numerical work went to attached array processors driven by library calls (the Floating Point Systems AP-120B attached to PDP-11 hosts, by the one brochure checked), and the VAX gave its 6000 series a vector extension of 16 registers of 64 elements, supported by VMS but little used by Unix. x86 went from MMX through SSE, AVX and AVX2 (256-bit) to AVX-512, a new width and a new set of state each time; ARM has NEON in the processor and scalable vectors beyond, and attaches its neural-network and graphics engines as memory-mapped devices; RISC-V has a scalable vector extension. **Trade-offs.** A device needs a setup (a descriptor) and a completion (a status), so small arrays are faster on the processor and large ones on the device; a SIMD register file has no setup and a cost in state, opcodes and width that grows with every generation. **Improvements.** The decision removes the questions that a vector register file raises: whether a context switch saves the vector state (x86's lazy saving of floating-point state was a security hole and Linux went back to eager restore), whether the kernel may use the registers, how wide they are, and what the calling convention does with them. **Benefits.** A processor that stays small and in order and that gains graphics and inference by adding devices. **Modern comparables.** ARM's Ethos neural engines and Mali graphics engines, Apple's matrix coprocessor reached through a library, Google's TPU on PCI Express. **Impact.** *Kernel:* no vector state to save at a context switch and no rule about the kernel using vector registers; a device is one more slot, mapped like any other. *Compiler:* no auto-vectoriser targets the device; the library and the language runtime call it at the level of an operation on an array. *Hardware:* no second register file, no wide load and store path, no opcodes spent; the processor's cost does not grow with the workload.

8.2 Why not SIMD in the processor

8.2.1 A SIMD register file is a second wide register file, which the instruction set deliberately avoided (floating point shares the general registers), and it brings wide loads and stores entangled with the alignment rules and a large demand on a first-page opcode map that is 58 of 64 full. *(D)*

8.2.2 A fixed-width design (128-bit NEON, 256-bit AVX2) bakes the width into every instruction, so a wider generation needs new instructions, and old binaries cannot use the new width. *(D)*

8.2.3 A scalable design (ARM SVE, RISC-V V) avoids that and is the most complex to specify and to implement; it is the only in-processor form that would suit this project's range from a small FPGA core to a possible large one. *(P)*

8.2.4 The compiled corpus shows little vectorisable code at the size-optimising setting; whether it shows more at higher optimisation is a measurement to make before any in-processor form is reconsidered. *(P)*

Commentary. **Precedent.** x86's history of SSE, AVX, AVX2 and AVX-512 is the argument against a fixed width: each added instructions and state, and AVX-512 was withdrawn from some client chips and later proposed to be unified. **Trade-offs.** A scalable vector extension would let one binary run on a small and a large implementation, at the price of the largest specification in the architecture and a compiler model that must handle an unknown vector length. **Improvements.** Moving the work to a device makes the length a descriptor field, which is the scalable idea without a register file. **Benefits.** The block RAM of a small FPGA, the scarce resource of the plan, is not spent on vector registers. **Modern comparables.** AVX2 on x86, NEON on ARM, SVE and the RISC-V vector extension. **Impact.** *Kernel:* nothing to save and no lazy-save hazard. *Compiler:* no vector types in the ABI and no vectorisation pass for the processor. *Hardware:* no area for a register file or wide data paths; the FPGA's memory goes to the caches and the device buffers.

8.3 AI and large-language-model inference

8.3.1 The inference accelerator, LB64, evaluates the layers of a binary-weight language model as sign-flip accumulates with no multipliers, taking a chain of descriptors and writing a status back on completion. *(D)*

8.3.2 A layer is two descriptors, the second accumulating into the first's result for the residual path, and a transformer block is a chain; the processor writes the chain once and is then idle. *(D)*

8.3.3 The work is partitioned between small engines, each doing one thing with its own local memory: LB64 for the projections and the feed-forward layers, a floating-point multiply-accumulate engine for the attention products, a vector engine for normalisation, softmax, activation and rotary position encoding, and the processor for the descriptor chains, sampling and the tokeniser. *(P)*

8.3.4 The accelerator has no instruction set: its computation has exactly one shape, so a programmable sequencer would spend its area on flexibility the work does not use. *(D)*

8.3.5 The processor keeps `POPCNT` and `XNORPOP` for the binary operations that small models and bitsets use; they are not what the accelerator uses. *(D)*

Commentary. **Precedent.** A descriptor chain is how every competent DMA engine since the PDP-11's RK11 has worked, and the partition into small single-purpose engines fed by chains is the idea of the 144-processor GA144 chip applied to a workload that suits it. **Trade-offs.** The accelerator is fast at what it does and does nothing else: attention is not helped, the element-wise work needs another engine, and a training accelerator is a different design again. The accelerator's descriptor is larger than the 32-byte descriptor of chapter 5, so the two must be reconciled: the proposal is that the standard descriptor's address points at a device-defined work block, which can chain, and the ring carries pointers to work blocks. **Benefits.** The lane count is set by the memory bandwidth and not by the amount of logic, so a small board can run a model the processor alone could not. **Modern comparables.** ARM's Ethos engines, Google's TPU, and the many inference accelerators on small boards. **Impact.** *Kernel:* a driver builds a chain and starts it; the kernel owns no model state and the only scheduling is of the device. *Compiler:* none; inference frameworks emit descriptors. *Hardware:* adders and sign-flip logic and no multipliers in LB64; the multiply-accumulate engine needs multiplier blocks; the memory interface sets the size.

8.4 Array languages: APL and K

8.4.1 Array languages express computation as operations on whole arrays: each primitive (add, multiply, compare, reduce, scan, index, where, sort) is one operation over contiguous data.

8.4.2 A device that takes a descriptor naming an operation, the addresses of the arrays and a length is the natural implementation of those primitives: one descriptor per primitive, the length in the descriptor. *(P)*

8.4.3 The interpreter stays on the processor and sends a primitive to the device when the array is long enough to repay the setup; short arrays run on the processor. *(P)*

8.4.4 There is no vector length in the language and none in a register: the length is a field of the descriptor, so one implementation serves any array size. *(P)*

8.4.5 A vector engine for such primitives would provide element-wise arithmetic and comparison, selection, reductions, scans, gather and scatter, compress (`where`) and index generation, on integer, floating-point and bit-array types. *(P)*

Commentary. **Precedent.** Iverson's APL made whole-array operations the language, and its descendants J and Arthur Whitney's K (the language of the kdb+ database) are interpreters whose speed comes from running long loops over contiguous arrays in a few tight kernels; the Cray vector machines were built around the same shape of loop, and DEC offered APL for the PDP-11 (not confirmed in the sources read). **Trade-offs.** Offloading a primitive has a fixed cost, so the benefit is for arrays above a size threshold; reductions and scans carry a dependence along the array that the device must support directly; sorting and gather are bandwidth-limited. **Improvements.** The language supplies the parallelism: the programmer says "add these two arrays", so there is no vectorising compiler to find it, which is the hardest part of making an ordinary language fast on SIMD hardware. The interpretive overhead is paid once per primitive, not once per element, and the device streams the data at memory bandwidth. **Benefits.** Vector languages are easier to support (a small interpreter with a library of primitives that map onto descriptors) and more efficient (the per-element work is hardware, and the per-primitive work is one descriptor), and the same device that serves the array languages serves the elementwise work of inference (section 8.3) and the lane-wise work of graphics. **Modern comparables.** K and kdb+ interpreters use AVX2 intrinsics heavily on x86 *(not confirmed in the sources read)*; array-language compilers for GPUs (Futhark, Co-dfns) target accelerators through offload, the same shape as this. **Impact.** *Kernel:* a language runtime that drives a device from user space needs the device's page mapped into its address space, with the protection caveat of chapter 5 (without a translation unit a user-space driver can direct its device to write anywhere). *Compiler:* none; the interpreter or compiler for the array language lowers primitives to descriptors, and a conventional compiler never sees a vector. *Hardware:* a streaming engine with a small set of element-wise, reduce, scan and gather units behind the ring engine, sized to the memory bandwidth.

8.5 Number formats

8.5.1 The processor handles `binary64`, `binary32` and `binary16` as scalar floating point; other formats (bfloat16, 8-bit integers with 32-bit accumulation, binary weights, 8-bit floating point) are the device's. *(D for the processor; P for the devices)*

8.5.2 Each vector device names its operations and formats in its identification block and feature bits, so a library can choose by asking. *(P)*

Commentary. **Trade-offs.** A format fixed in the processor's instruction set is a format that cannot change when the machine-learning field changes it, as it has several times; a format in a device is replaced by replacing the device. **Modern comparables.** x86 has half precision, bfloat16 and `VNNI` dot products in the instruction set; ARM has `BF16`, `SDOT` and matrix extensions; accelerators handle the rest. **Impact.** *Kernel:* none. *Compiler:* only the scalar formats; the machine-learning formats are descriptor fields. *Hardware:* the devices' multiply-accumulate units, not the processor's, are built for the formats in use.

8.6 Driving a vector device

8.6.1 A program fills a descriptor naming the operation and the arrays, executes `FENCE`, and writes the doorbell; the device processes the descriptor, writes the results and a status, and clears the ownership bit; the program polls or takes the interrupt. *(P)*

8.6.2 A chain of descriptors runs without the processor, and a work block that is a chain is how a transformer layer or a sequence of array primitives is expressed. *(P)*

8.6.3 The buffers are uncached, as for every device, so the processor's reads of results are slow; a large result is left in the device's memory or read back by the next descriptor. *(P)*

Commentary. This is the same driver skeleton as for a disk (chapter 5), with an operation field that means "add these arrays". **Trade-offs.** The slow uncached readback is the cost of the explicit hand-off; an interpreter that needs many small results back on the processor should leave them in device memory. **Impact.** *Kernel:* one driver skeleton. *Compiler:* none. *Hardware:* one ring engine shared with every device.

8.7 Still open

The reconciliation of the accelerators' descriptors with the standard descriptor, the formats and operation set of the vector engine, the target workloads, and the evidence that would reopen the question of packed integer lanes: compile the corpus

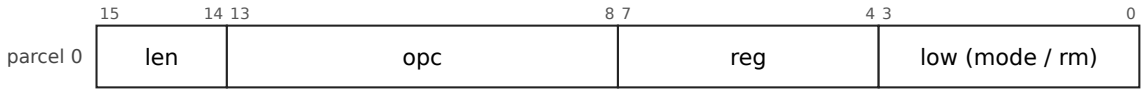
at higher optimisation and count vectorised loops, measure the share of time in memory and string routines, and estimate the block RAM cost (appendix B, S1 to S6).

Chapter 9 — The instruction set

The instruction set is built around one idea: the length of an instruction is known from its first two bits, so the cheapest instructions are two bytes and nothing is longer than six. The chapter covers how the 64 opcodes are divided, the properties and principles that hold throughout, the registers, the format and the addressing modes, each group of instructions, faults, restart and memory management, position-independent code, what is rejected, the idioms of the PDP-11 and VAX that are restored, lost or replaced, the comparison with the PDP-11 and VAX, what is taken from LoongArch, and what is open. Every instruction has its entry (syntax, encoding drawing, operation, flags, exceptions) in the section of its group below; the atomic instructions are in section 7.12 and the system instructions in section 6.13; the reference tables hold the second page, the aliases, the constants and the lists of mnemonics and opcodes. The full text with every encoding is `../pdp64-isa.md`; every number here comes from the grammars that the tools are generated from.

9.1 How the opcode space is divided

9.1.1 Every instruction begins with a 16-bit parcel with four fields: `len` (2 bits), `opcode` (6 bits), `reg` (4 bits) and `mode` or `rm` (4 bits).



9.1.2 There are 64 opcodes on the first page. An operation gets a top-level opcode if its common case is register to register, because only then can it be two bytes; every other operation lives in an escape group and costs nothing for it.

9.1.3 Six escape groups, `001100 (0x0C)` to `010001 (0x11)`, read a sub-opcode from the second parcel: `BESC` (bits, fields, rotates, UTF-8), `MBLK` (block memory), `AESC` (divide, remainder, absolute, minimum, maximum, set), `MESC` (unaligned access, ALU to memory), `FESC` (fused multiply-add, binary32 and binary16, divide, square root, classify) and `ATOM` (atomics).

9.1.4 Three groups, `010010 (0x12)` to `010100 (0x14)`, take their sub-opcode from the register field: `XFER` (indirect jump and call, tag dispatch), `SOP` (single-operand instructions, jump, call, push, pop) and `SYS` (system control).

9.1.5 Every other run of opcodes is one kind, and its low bits select the operation directly: twelve moves (`000000 (0x00)-001011 (0x0B)`), thirteen ALU operations (`010110 (0x16)-100010 (0x22)`), four floating-point operations (`100100 (0x24)-100111 (0x27)`), three compare-and-branch forms (`101000 (0x28)-101010 (0x2A)`) and sixteen branches (`110000 (0x30)-111111 (0x3F)`).

9.1.6 Fifty-eight of the sixty-four opcodes are assigned, one ALU slot (`100011 (0x23)`) is spare and five (`101011 (0x2B)-101111 (0x2F)`) are reserved.

9.1.7 When `len` is `11`, the instruction is a single parcel from a second page of 64 opcodes, 45 defined and 19 reserved, each an exact abbreviation of a first-page instruction.

Commentary. **Precedent.** The PDP-11 partitioned its opcodes by the addressing modes of the first word; the VAX by operand specifiers parsed in sequence; x86 by prefixes, opcode, ModR/M and SIB read one after another. Each step down that list costs more front-end logic before the address of the next instruction is known. **Trade-offs.** Putting an operation in an escape group makes its shortest form four bytes, since the sub-opcode lives in the second parcel, so the allocation is a frequency sort, and five groups sit in escapes for that reason (divide and remainder, rotate by a register count, floating-point divide and square root, and the read-modify-write memory forms), where the cost is nothing or almost nothing. **Improvements.** The six second-parcel escapes sit together so that "read a sub-opcode" is one range comparison on bits 13 to 8; the three register-field escapes sit together so that one range compare enables the mux that already feeds the register file's address inputs; the other runs are contiguous so their low bits are wired straight to the function selects. **Benefits.** The decoder is a few comparators and wires, and a new instruction usually costs a sub-opcode, not an opcode. **Modern comparables.** RISC-V groups its major opcodes by format and AArch64 by top-level encoding class; the pdp64 groups by the one property that matters most to its front end, whether the sub-opcode needs the second parcel. **Impact.** *Kernel:*

Nothing in the kernel depends on the opcode map except the privileged group, which sits in one place. *Compiler:* The instruction selector gets the two-byte form for every register-to-register operation and must charge four bytes for anything in an escape group, which its cost model has to know. *Hardware:* The decoder is a few range comparisons and wires: six escape groups are one compare on bits 13 to 8, three register-field escapes are one compare that enables a mux already feeding the register file, and every run's low bits go straight to its function select.

9.2 Properties and principles

9.2.1 The length of an instruction is a function of its `len` field alone: 0, 1 or 2 extension parcels, for 2, 4 or 6 bytes. *(D)*

9.2.2 At most one operand of an instruction may be a memory reference; the other, where present, is a register. The block operations are the one admitted exception. *(D)*

9.2.3 No instruction is longer than six bytes. *(D)*

9.2.4 A register is always written in full: narrow loads sign- or zero-extend, and there is no partial-register write. *(D)*

9.2.5 Anything that changes what an instruction means is, by default, visible state set by an instruction that names it, not hidden state; whether a mode bit is worth its cost is decided case by case (errata 83). Compatibility is by conversion instructions where that is cheap. *(D)*

9.2.6 Every instruction can be restarted after a fault, and exceptions are precise; the machine has no hidden instruction state. *(D)*

9.2.7 Floating point is done in the general registers; there is no separate floating-point register file. *(D)*

9.2.8 Memory is sequentially consistent between processors. *(D)*

9.2.9 The ALU has one width, 64 bits; sizes appear only on loads, stores and conversions. *(D)*

9.2.10 Addresses are canonical: 39, 48 or 57 bits of virtual address, according to the three, four or five levels of the page table (6.6.1), and a physical address of 40 to 64 bits, 40 in the present implementation (2.3.6). *(D)*

9.2.11 Instructions are aligned to two bytes; quad data accesses are naturally aligned, and the explicit unaligned forms exist for the cases that need them. *(D)*

9.2.12 An instruction does what its name says and nothing else; no flag is set that its entry does not list. *(D)*

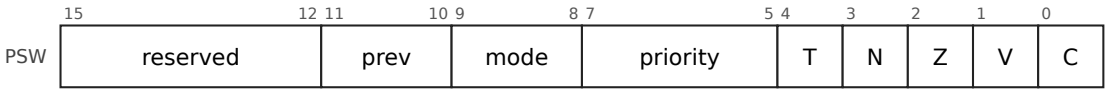
Commentary. **Precedent.** 9.2.2 is x86's ModR/M idea made architectural; it is also what lets the exception machinery be small (the PDP-11 recorded at most two register side effects per instruction because it had two specifiers; with one memory operand there is at most one). 9.2.6 exists because the Motorola 68000, which was very nearly this machine, could not restart an instruction after a bus error and so could not do demand paging until its successor retrofitted a fault model. 9.2.4 is the rule x86-64 adopted for 32-bit writes and failed to apply to the 8- and 16-bit forms. 9.2.7 is the arrangement RISC-V standardised as Zfinx. **Trade-offs.** 9.2.9 means narrow arithmetic is done wide and converted, which costs an extension instruction after some narrow operations; whether that cost is large enough to justify 32-bit forms of add and subtract is an open question that the compiled corpus can answer (appendix B). 9.2.8 is the cheapest model on a small in-order core and the most expensive on a large speculative one. 9.2.10 lets the page table have three levels, as RISC-V's Sv39 does, or four or five as x86-64, AArch64 and Sv48 and Sv57 do, at a longer walk for each level. **Improvements.** The rules are soft defaults: each says what it protects and what breaking it costs. **Benefits.** Together they make the machine small, restartable and easy for a compiler; the measured code size is in chapter 1. **Modern comparables.** AArch64 and RISC-V have fixed-length register-to-register encodings and weak memory models; x86-64 has memory operands and total store order; the pdp64 takes memory operands, strong ordering and a bounded fault surface. **Impact.** *Kernel:* Restartable instructions and precise exceptions mean a page-fault handler repairs the mapping and returns, with no instruction-specific cases, and sequential consistency means no barrier code. *Compiler:* One memory operand keeps instruction selection and register allocation simple; a register written in full removes partial-register merges; the single ALU width costs an extension instruction after some narrow arithmetic. *Hardware:* One address generator, at most one register side effect, a single-entry undo buffer, no byte-lane multiplexing and a two-bit length are what keep the die small.

9.3 State: registers and flags

9.3.1 There are sixteen 64-bit general registers, `R0` to `R15`. `R14` is the stack pointer `SP`, with three copies (kernel, supervisor, user) chosen by the current mode; `R13` is the frame pointer by convention only; the others carry no meaning to the hardware. *(D)*

9.3.2 The program counter is not a general register, and `R7` is an ordinary general register, not a view of the program counter; the program counter is read with `LEA 0(PC), Rn`. *(D)*

9.3.3 The processor status word is 16 bits. *(D)*



(Bit positions as in System v1: `C` 0, `V` 1, `Z` 2, `N` 3, `T` 4, priority 5-7, current mode 8-9, previous mode 10-11, `UA` 12.)

9.3.4 The flags form two groups, `C` and `{N, Z, V}`, and no instruction writes part of `{N, Z, V}` or writes it conditionally. Moves, loads, stores and the compare-and-branch instructions write no flags; shifts write the same flags for every count; rotates and the funnel shift write none; the by-one left shift sets `V` to `N` xor `C`. *(D)*

9.3.5 `DP`, the direct-page base, has three copies by mode. It makes `d(DP)` the shortest address form (a 12-bit displacement in four bytes); in kernel mode it is the per-CPU pointer, and in user mode the thread pointer. *(D)*

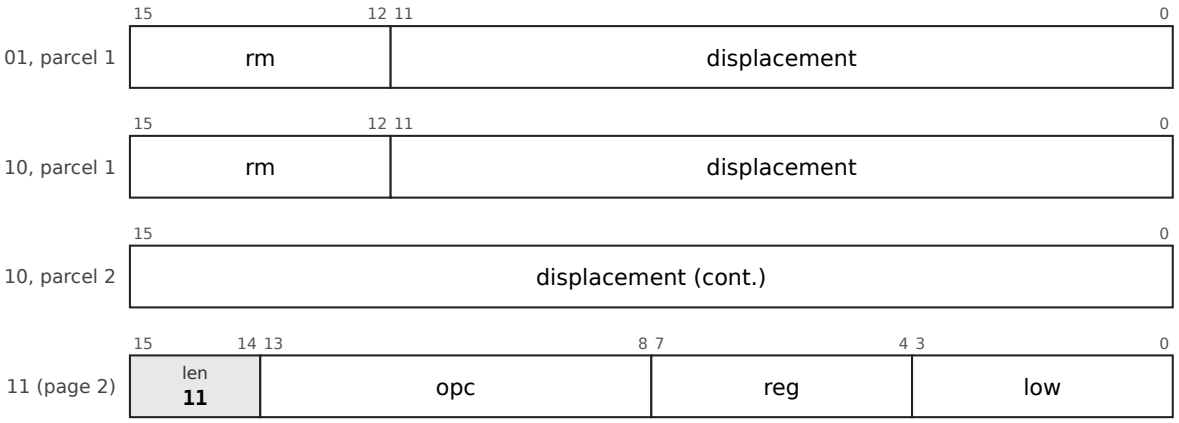
9.3.6 Other special registers: `FPSR` (five sticky floating-point flags), `ASID`, `PTBR`, `DMW0` and `DMW1`, `EPC`, `EPSW`, `EVA`, `ECAUSE`, `VBR`, `MMUCTL`; the system registers are read and written by `MFSR` and `MTSR` with an 8-bit register number. *(D)*

9.3.7 A bit `DIT` in the status word (its position is unspecified) promises data-independent timing: while it is set no data-dependent timing effect of any kind remains, neither in instruction latency (divide, `MSCN`, `MCMP` and floating point included), nor in frequency or power management (a core that scales frequency holds it steady while the bit is set, or says in a feature register that it cannot), nor from any data-dependent prefetcher or value predictor (a core may have one, but it is off while the bit is set). User code may set it for its own use. *(D)*

Commentary. **Precedent.** The PDP-11's PC and SP were ordinary registers and its flags were set by nearly every instruction. The VAX put the PC in R15. **Trade-offs.** Keeping the PC out of the register file means a jump to a computed address uses `JMP`, not a write to a register, so that writing `SP` has no control-flow consequence and the hazard logic treats it like any other destination. Condition codes are kept (rather than a flagless design) because compare-and-branch and flag-setting arithmetic make for dense code; the flag rules make them cheap for an out-of-order implementation, because the compiled corpus shows that 68% of instructions write a flag and only 1.5% read one. **Improvements.** The status word carries a user-memory guard bit (9.14) and the per-mode `DP` serves two jobs, the per-CPU pointer in the kernel and the thread pointer in user mode. **Benefits.** An exception handler finds a kernel stack and a per-CPU pointer without first searching for them; a threaded program's thread-local variable is one instruction. **Modern comparables.** x86-64 keeps `FS` and `GS` base registers for the same two jobs; AArch64 has `TPIDR_EL0` and `TPIDR_EL1`; RISC-V uses `tp` and the supervisor scratch register. **Impact.** *Kernel:* Banked stack and direct-page registers let entry find a kernel stack and a per-CPU pointer with no code, and the status word is saved by one `PUSHX`. *Compiler:* The flag rules decide when a compare may fuse with its branch (1,893 pairs stay separate because a second branch reads the flags) and what the compiler may move between a compare and a branch. *Hardware:* Two flag groups need one or two rename targets, not one per bit; the register file is 16 by 64 with two read ports and one write (about 250 lookup tables of distributed memory); the banked copies are a three-way mux.

9.4 The instruction format

9.4.1 The first parcel holds `len`, the opcode and two four-bit fields; `len` is the number of extension parcels (0, 1 or 2) when it is less than 3. *(D)*



9.4.2 Instructions are parcel-aligned and little-endian; an instruction may cross any boundary of 8 bytes, a cache line or a page. *(D)*

9.4.3 Most instructions are two or four bytes: register-to-register work and the whole conversion set is one parcel; any memory operand within ±2 KB is two; displacements to ±128 MB, 32-bit immediates and branches to ±4 GB are three. *(D)*

9.4.4 Where an addressing mode names no base register (immediate, absolute, PC-relative), the register nibble of the second parcel carries payload, so those modes reach 16 bits at four bytes and 32 bits at six. *(D)*

9.4.5 A constant wider than 32 bits is loaded from a literal pool with `MOVQ =K, Rd`, written with an explicit `=`; nothing is pooled silently. *(D)*

Commentary. **Precedent.** The PDP-11 had the length in the first word, but as a function of both of its six-bit specifiers; UTF-8 made "the leading unit says how long" famous, but what matters here is that length is the *primary* partition, so the payload is budgeted by what each form needs and not carved from a fixed width. **Trade-offs.** Byte-granular encoding, as in x86, buys perhaps a tenth of the code size and costs a full alignment rotator and cross-line splice logic in the fetch path on a 64-bit datapath; the pdp64 refuses it. Forcing an instruction to stay inside an eight-byte window would simplify fetch and would cost 5% to 10% of code size in padding. **Improvements.** The `len` code `11` selects a second page of one-parcel abbreviations; the measured average is near three bytes per instruction. **Benefits.** The front end is a two-bit counter, a two-deep parcel buffer and no feedback. **Modern comparables.** Thumb-2 and RISC-V's compressed instructions mix 16- and 32-bit forms too, with the length read from the low bits. **Impact.** *Kernel:* Page-2 forms make kernel text smaller with no source change. *Compiler:* The assembler picks the shortest form, branch relaxation is a length choice, and the cost model knows only 2, 4 and 6 bytes. *Hardware:* The fetch unit is a two-bit down-counter and a two-deep parcel buffer with no feedback.

9.5 Addressing modes

9.5.1 Four bits select one of eight modes, present at four bytes and longer. *(D)*

Mode	Syntax	Effect
`0000`	`d(Rn)`	base plus displacement
`0001`	`d(Rn)(Rx*s)`	scaled index, `s` in {1, 2, 4, 8}
`0010`	`d(PC)`	PC-relative; the base is the first parcel of the instruction
`0011`	`d(DP)`	direct page
`0100`	`#imm`	immediate, sign-extended
`0101`	`@abs`	absolute, sign-extended
`0110`	`(Rn)+d`	post-increment by `d` (default: the operand size)
`0111`	`-(Rn)` or `(Rn)-d`	pre-decrement by the operand size, or by `d`

9.5.2 The deferred modes of the PDP-11 are removed; indirection is done with `JMPI`, `CALLI` and a load. *(D)*

9.5.3 At most one register side effect occurs per instruction, and it is committed at retirement; a fault discards it. *(D)*

Commentary. **Precedent.** The PDP-11 had eight modes for each of eight registers, with a deferred flavour of each, and it made `*p++` and `*--p` single instructions. **Trade-offs.** Dropping the deferred modes removes the case where a fault occurs after a register has been updated at an address that itself came from a load; keeping the strided autoincrement costs

nothing, because the undo information was already a delta (the PDP-11's `MMR1` recorded an amount for the same reason). **Improvements.** The increment is a stride, so walking an array of 16-byte records is one instruction; the PC-relative modes count from the instruction itself, so a branch needs no adjustment and `LEA` reads the PC. **Benefits.** Address generation is one adder pass, never load-then-add, which is the difference between one cycle and a long path on an FPGA. **Modern comparables.** AArch64 has pre- and post-indexed forms with an immediate; x86 has scaled index with a base; RISC-V has base plus immediate only. **Impact.** *Kernel:* A strided autoincrement walks a table of records, and `d(DP)` reaches the per-CPU structure in one instruction. *Compiler:* The modes match C idioms (`*p++`, `a[i]`) one-to-one, and dropping the deferred modes removes the cases the selector would otherwise have to consider. *Hardware:* Address generation is one adder pass, never load-then-add, and the PC-relative base is the first parcel, so no adjustment is needed.

9.6 Moves, conversions and address formation

The instructions. Every instruction has an entry in the section of its group, laid out as in a processor reference manual: its mnemonics, its syntax, its encoding (a drawing of the parcels and the numbers), its operation in pseudocode, the flags it writes, the exceptions it can take, and where the manual explains the matter it touches (*Cf.*). Every instruction, the atomic and the system instructions included, has its official entry below (sections 9.13 and 9.14 for those two groups); the chapters on the kernel and on several processors give the reasons, and an entry may point back to them for a fuller explanation. In the pseudocode, ← is assignment, = and ≠ compare, × and ÷ multiply and divide, and `M[a]` is memory at address `a`. Numbers that are encodings (opcodes, sub-opcodes, field values) are written in binary with the hexadecimal value in brackets behind, as `010011 (0x13)`. Masks are binary too, with a subscript 2 to mark the base (`Rn & 111`$_2$ keeps the low three bits). The entries are generated from `spec/pdp64-mnemonics.hbnf`, the one place where the numbers of the architecture are written, so an encoding here cannot disagree with the assembler or the simulators; the words (operation, flags, exceptions) are checked against the reference executor, `common/pdp64exec.c`, which `sim64` and QEMU share, and the Status line of each entry says whether the executor implements the instruction. The one-parcel second page, the aliases, the constants and the lists of the mnemonics and the opcodes are in the reference tables.

The stories. An instruction that is not common (the PDP-11 core and the operations every architecture has carry none) is followed by *Why it is here*: the machine the idea came from and the reason it was chosen, with the measurements that bear on it. Where no reason is recorded, the paragraph says so. The documents behind the paragraphs are listed in appendix F.

Notation. `Rd` is the destination register (the `reg` field of parcel 0), `Rs` the first source (`low`), `Rt` the second (`t` of parcel 1), `Ru` the third (`x`); the assembler writes the destination last. `rm` is a general operand: a register at `len = 00`, or a memory operand, an immediate or an absolute address by one of the eight addressing modes (section 9.5). `EA(rm)` is its effective address. `M[a]` is memory, `M8[a]` one byte. `sext` and `zext` sign- and zero-extend. A `len = 01` instruction is two parcels (four bytes) and a `len = 10` instruction three (six bytes); `len = 11` is page 2. `N Z V C` are the condition flags. In the drawings the bits of a parcel are numbered from 15 at the left, a shaded box is a fixed value, and a white box is a field that varies.

Cf. section 6.7 for the memory types that decide whether an access is cached, and section 6.4 for what a faulting load leaves behind.

9.6.1 `MOVQ`, `MOVL`, `MOVW` and `MOVB` load and store 64, 32, 16 and 8 bits; the loads sign-extend. `MOVZL`, `MOVZW` and `MOVZB` load with zero extension. *(opcodes `000000 (0x00)`-`001010 (0x0A)`)*

9.6.2 With a register operand, the same opcodes are the conversion set: `MOVB R1, R2` sign-extends the low byte, `MOVZW R1, R2` zero-extends the low word.

9.6.3 `LEA` forms an effective address; `LEA 0(PC), Rn` reads the program counter. *(opcode `001011 (0x0B)`)*

9.6.4 Moves, loads and stores write no flags. *(D)*

9.6.5 The explicit unaligned forms `MOVUB`, `MOVUW`, `MOVUL` and `MOVUQ` exist in both directions, in the memory escape. *(D)*

9.6.6 Page 2 abbreviates the common cases to one parcel: loads and stores through `(rn)`, loads and stores at small offsets from `SP`, `LEA` from `SP`, and `MOVQ #imm, rd` with a four-bit immediate. *(D)*

Commentary. **Precedent.** The PDP-11 had `MOV` and `MOVB` with a byte sign-extended into a register; the VAX had a sized `MOV` for every width. **Trade-offs.** Keeping the sized loads costs four extra opcodes against a quad-only design, but without `MOVL` a struct field load becomes two `MOVWS`, a shift and an or, and ELF, timestamps and most on-disk and wire structures are full of 32- and 16-bit fields. **Improvements.** There is no separate `CVT` group: the conversions are the moves with a register source, so they cost nothing. **Benefits.** No partial-register state anywhere. **Modern comparables.** AArch64 and RISC-V

have the same sized loads with sign and zero extension; x86-64 keeps 8- and 16-bit register writes that preserve the upper bits, which makes narrow writes depend on the old value. **Use.** `LEA 0(PC)` is how a position-independent program finds itself; the unaligned forms are for packed on-disk and network structures. **Impact.** *Kernel:* `LEA 0(PC)` lets position-independent kernel code find itself; the unaligned forms read packed on-disk and network structures. *Compiler:* Sized loads avoid the shift, mask and or of a struct-field read, and the conversions are moves, so no separate conversion selection exists. *Hardware:* No partial-register write anywhere; the conversions reuse the load-extend path.

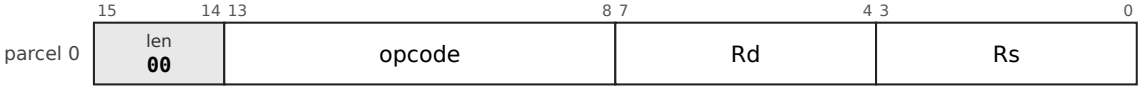
MOV: Move, load and store (64, 32, 16 and 8 bits)

Mnemonics: `MOVQ`, `MOVL`, `MOVW`, `MOVB`

Syntax

```
MOVQ rm, Rd        ; load
MOVQ Rs, rm        ; store
MOVL, MOVW, MOVB likewise
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
`MOVQ`	`000000 (0x00)`, `000001 (0x01)`
`MOVL`	`000010 (0x02)`, `000011 (0x03)`
`MOVW`	`000100 (0x04)`, `000101 (0x05)`
`MOVB`	`000110 (0x06)`, `000111 (0x07)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
size = 8 (MOVQ), 4 (MOVL), 2 (MOVW) or 1 (MOVB) bytes
load :  Rd ← sext(M[EA(rm)], size)          ; len = 00: Rd ← sext(Rs[8×size-1:0])
store:  M[EA(rm)] ← Rs[8×size-1:0]          ; len = 00: Rd ← sext(Rs[8×size-1:0])
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

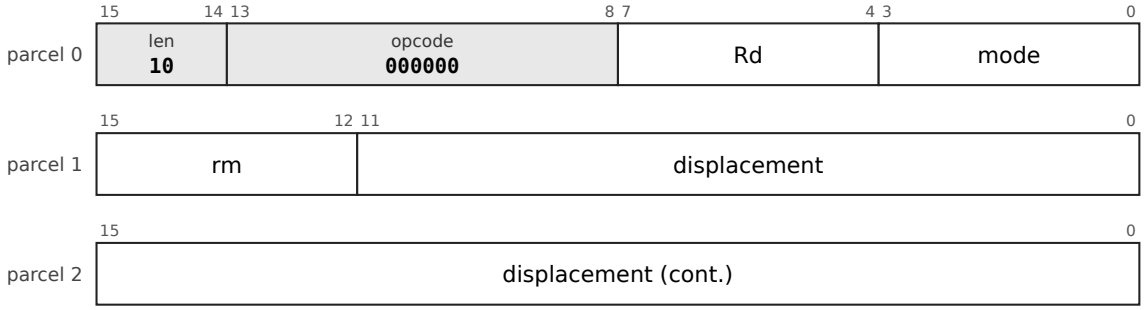
A register is always written in full: a narrow load sign-extends to 64 bits and there is no partial-register write anywhere. At `len = 00` the same opcodes are the conversion set: `MOVB R1, R2` sign-extends the low byte of `R1` into `R2`. Page 2 abbreviates the loads and stores through `(rn)`, at small offsets from `SP` and from a register, and `MOVQ #imm, rd` with a 4-bit immediate (see the page-2 table).

Cf. 6.7 for the memory type that decides whether the access is cached; 6.4 for what a faulting access leaves behind.

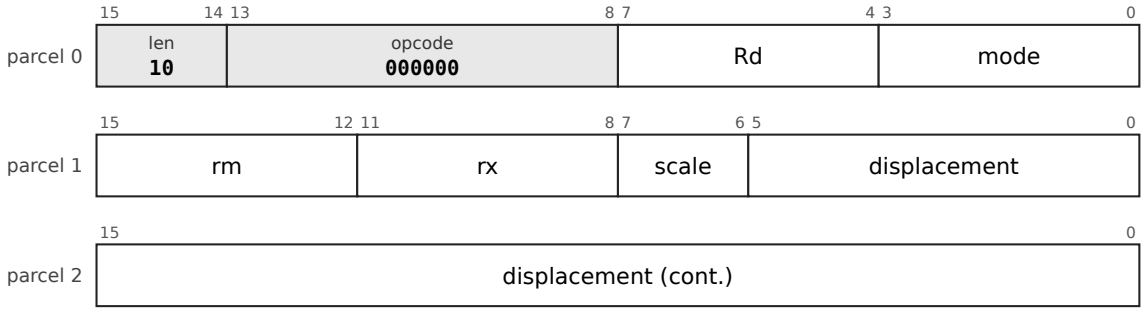
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MOVQ

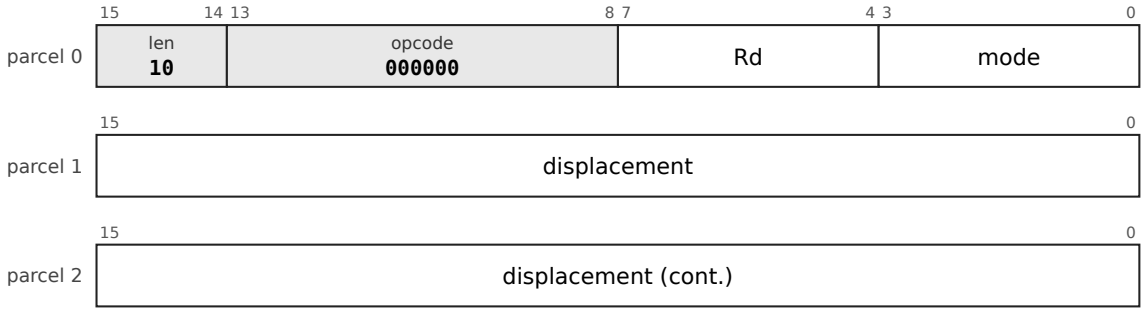
MOVQ, opcode 000000, three parcels (modes base, postinc, predec) (6 bytes)



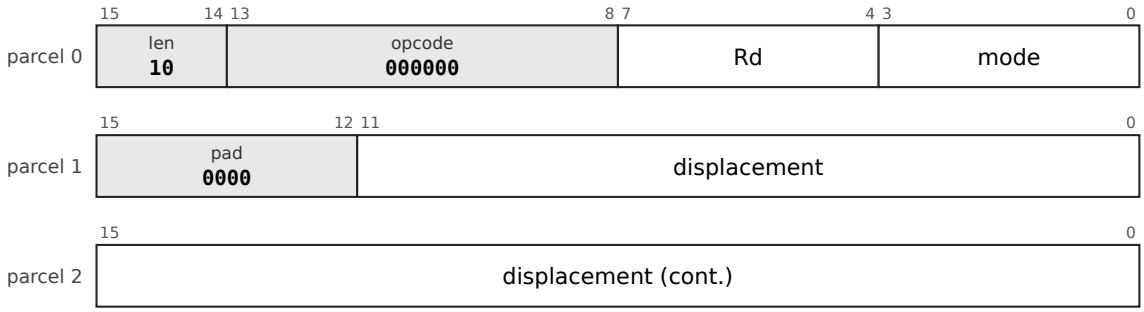
MOVQ, opcode 000000, three parcels (modes index) (6 bytes)



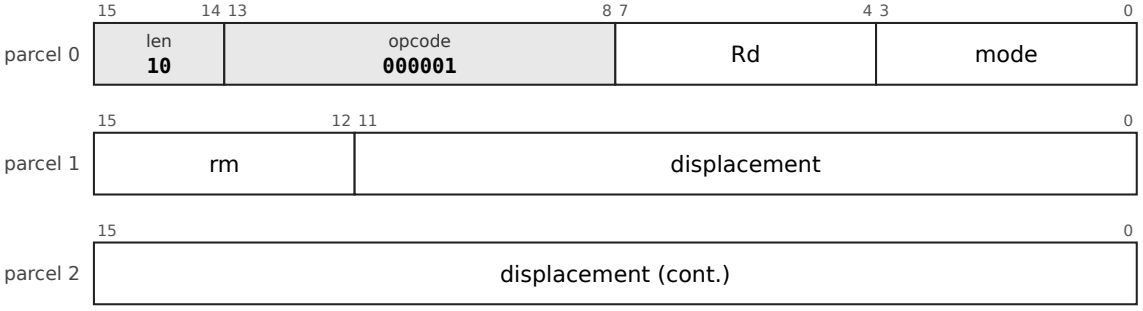
MOVQ, opcode 000000, three parcels (modes pcrel, imm, abs) (6 bytes)



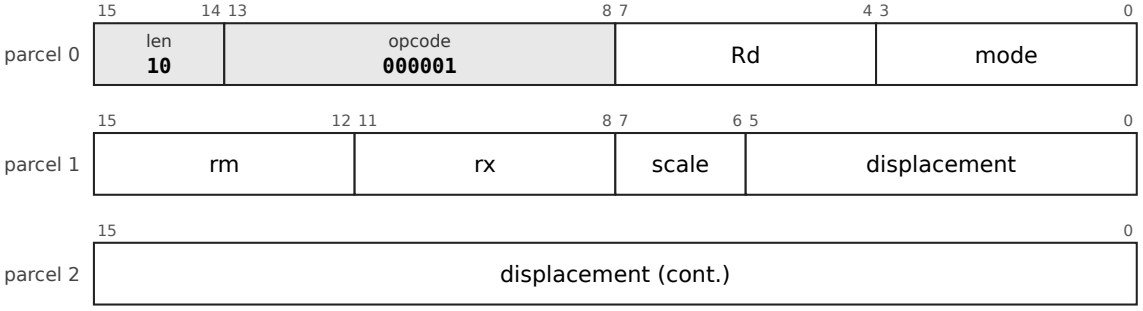
MOVQ, opcode 000000, three parcels (modes dp) (6 bytes)



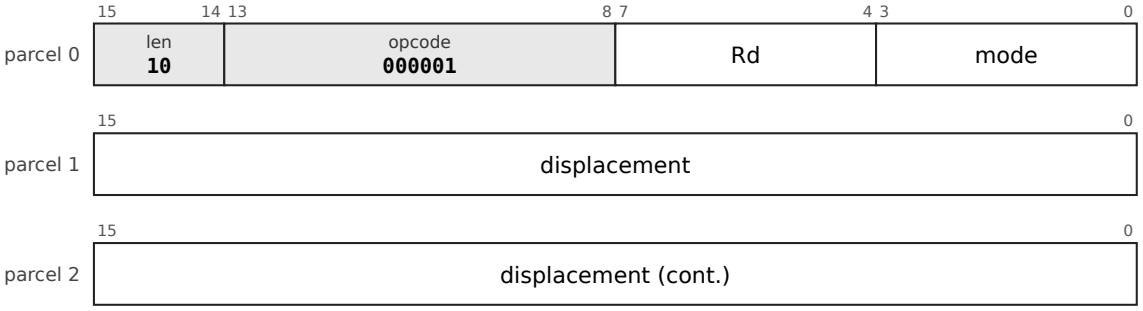
MOVQ, opcode 000001, three parcels (modes base, postinc, predec) (6 bytes)



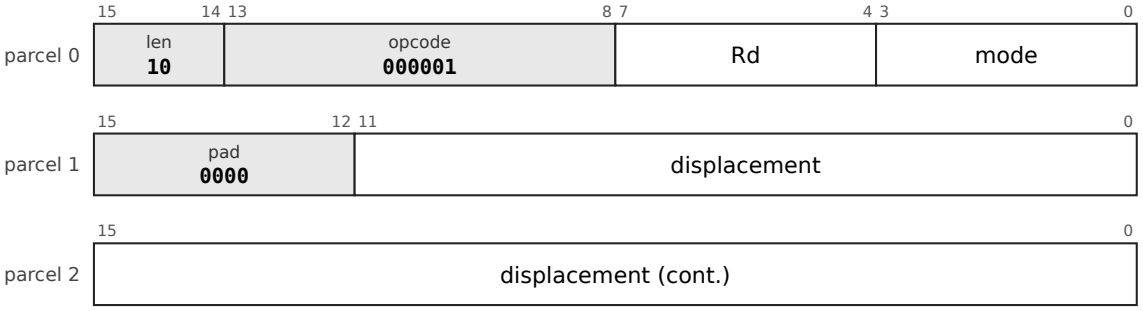
MOVQ, opcode 000001, three parcels (modes index) (6 bytes)



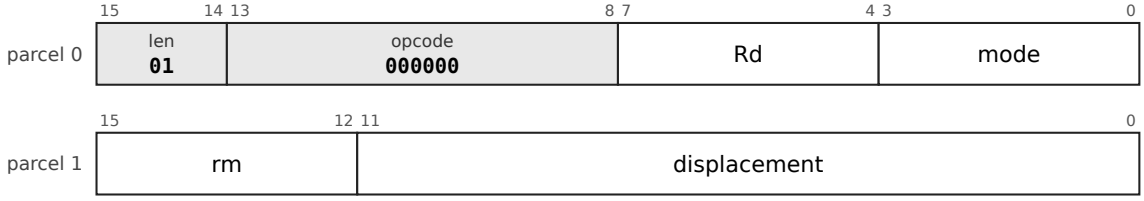
MOVQ, opcode 000001, three parcels (modes pcrel, imm, abs) (6 bytes)



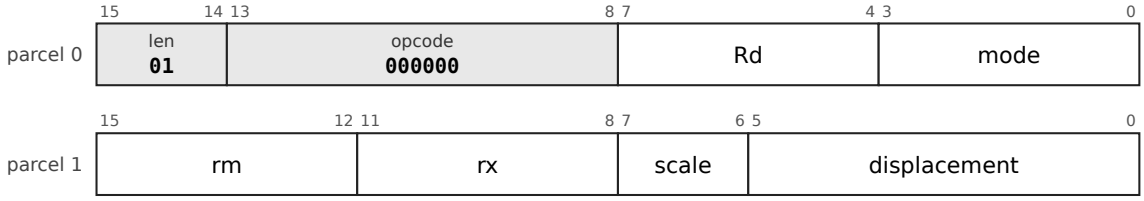
MOVQ, opcode 000001, three parcels (modes dp) (6 bytes)



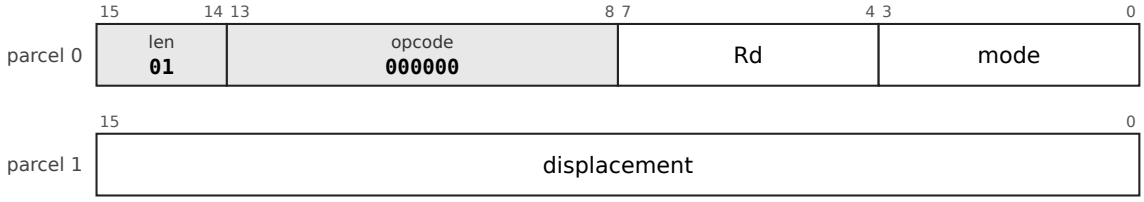
MOVQ, opcode 000000, two parcels (modes base, postinc, predec) (4 bytes)



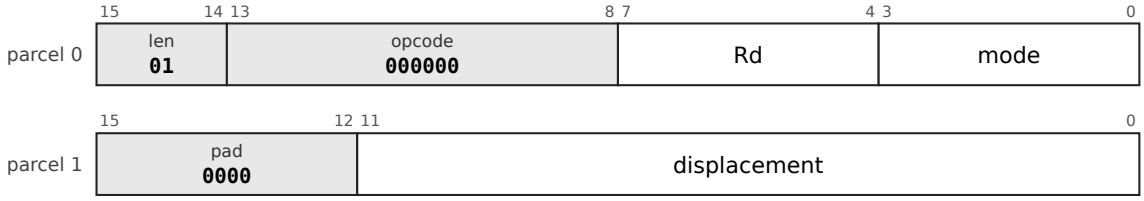
MOVQ, opcode 000000, two parcels (modes index) (4 bytes)



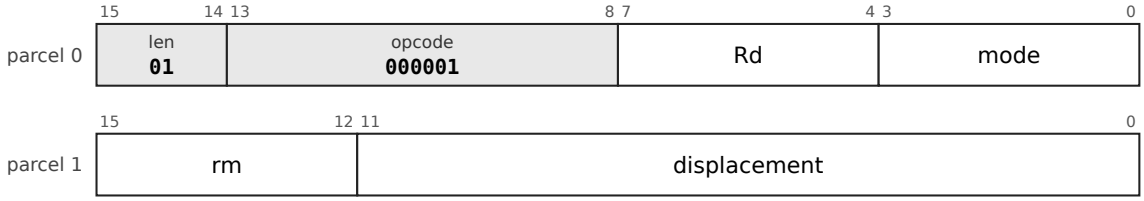
MOVQ, opcode 000000, two parcels (modes pcrel, imm, abs) (4 bytes)



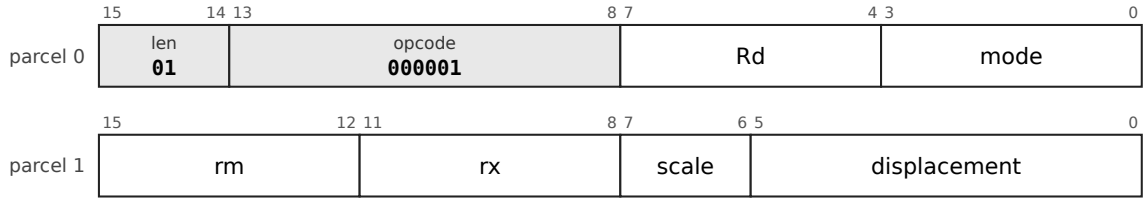
MOVQ, opcode 000000, two parcels (modes dp) (4 bytes)



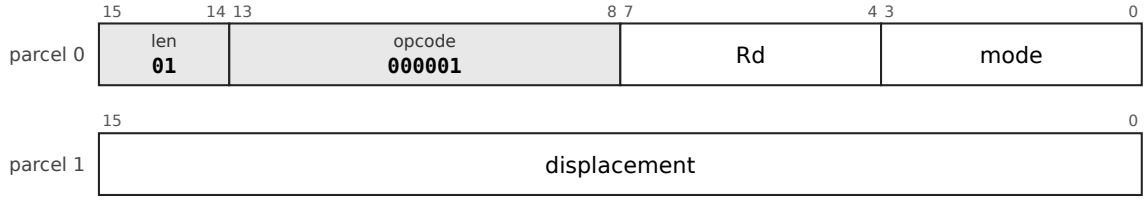
MOVQ, opcode 000001, two parcels (modes base, postinc, predec) (4 bytes)



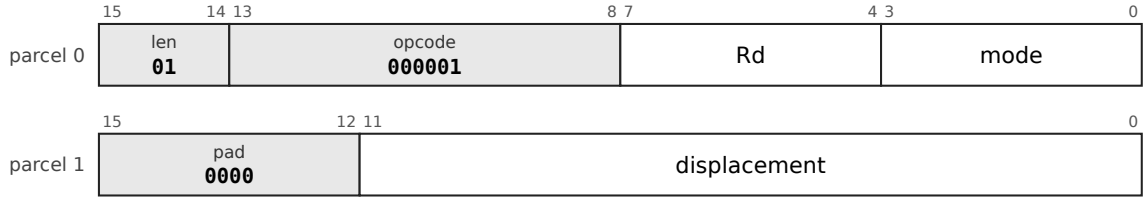
MOVQ, opcode 000001, two parcels (modes index) (4 bytes)



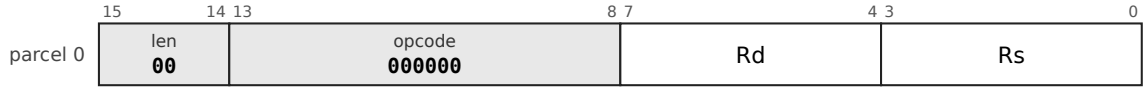
MOVQ, opcode 000001, two parcels (modes pcrel, imm, abs) (4 bytes)



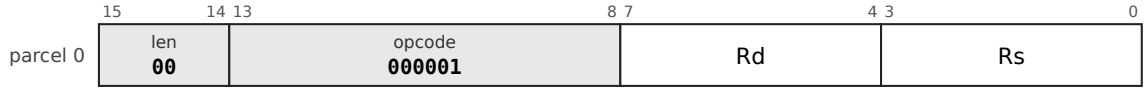
MOVQ, opcode 000001, two parcels (modes dp) (4 bytes)



MOVQ, opcode 000000, one parcel (2 bytes)



MOVQ, opcode 000001, one parcel (2 bytes)



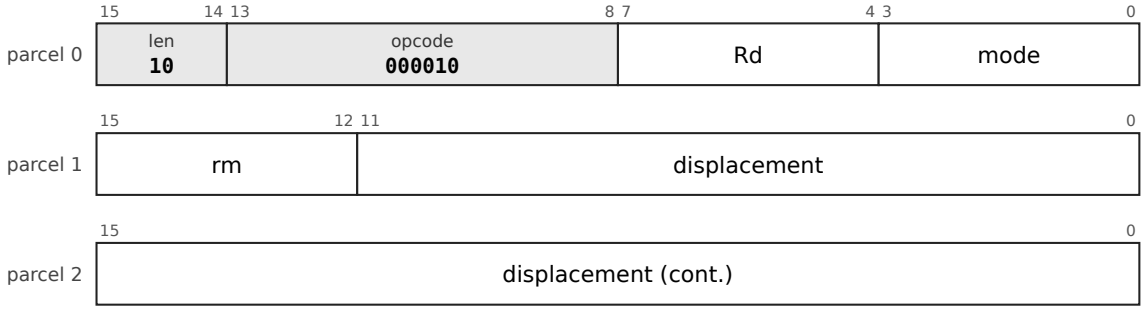

```
MOVQ rm, Rd
MOVQ Rs, rm
```

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)

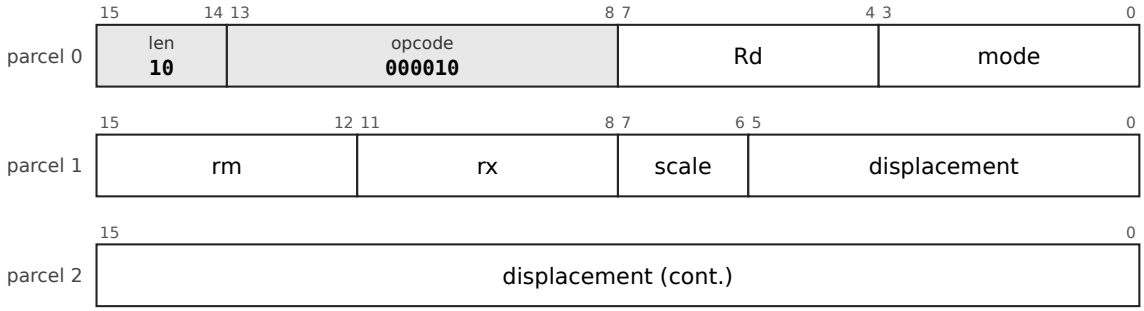
Operation, flags and exceptions as for the entry MOV above; 18 layout forms, each a chart of its own (above).

MOVL

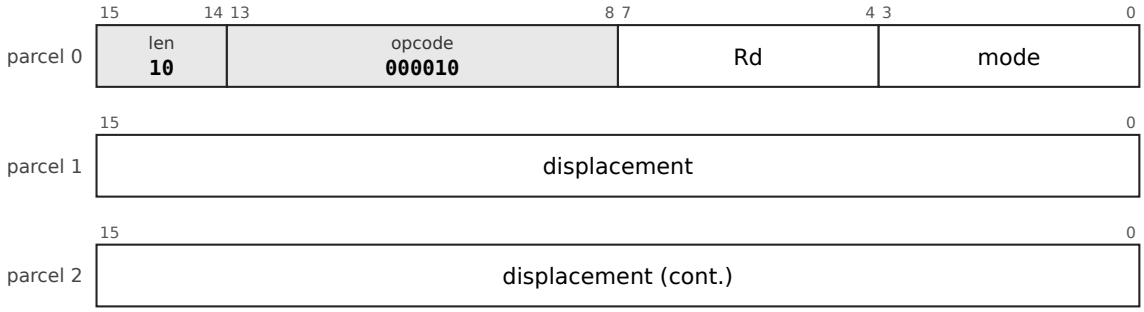
MOVL, opcode 000010, three parcels (modes base, postinc, predec) (6 bytes)



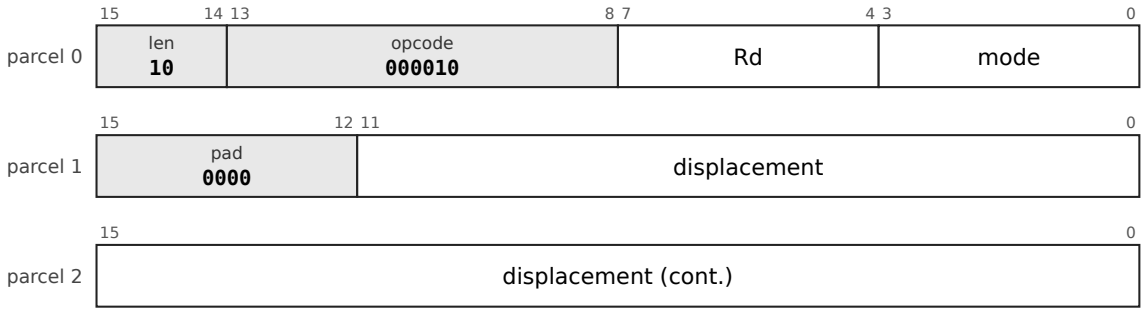
MOVL, opcode 000010, three parcels (modes index) (6 bytes)



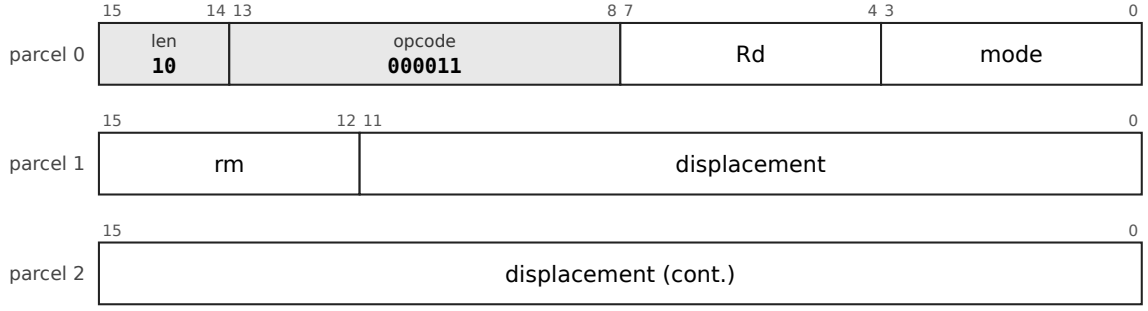
MOVL, opcode 000010, three parcels (modes pcrel, imm, abs) (6 bytes)



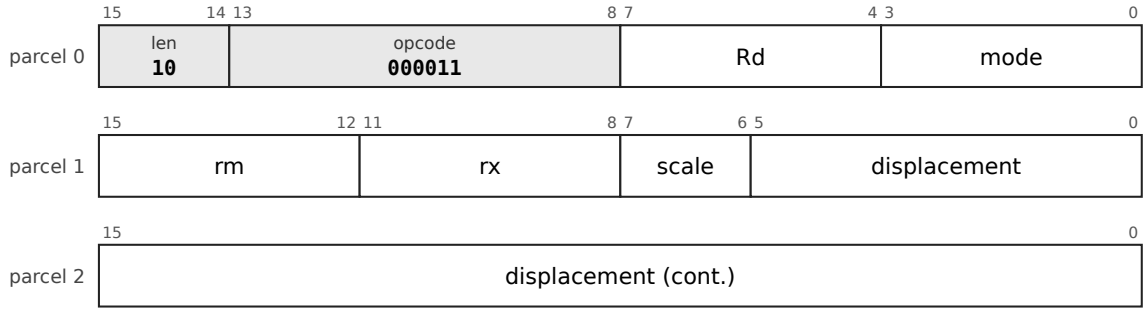
MOVL, opcode 000010, three parcels (modes dp) (6 bytes)



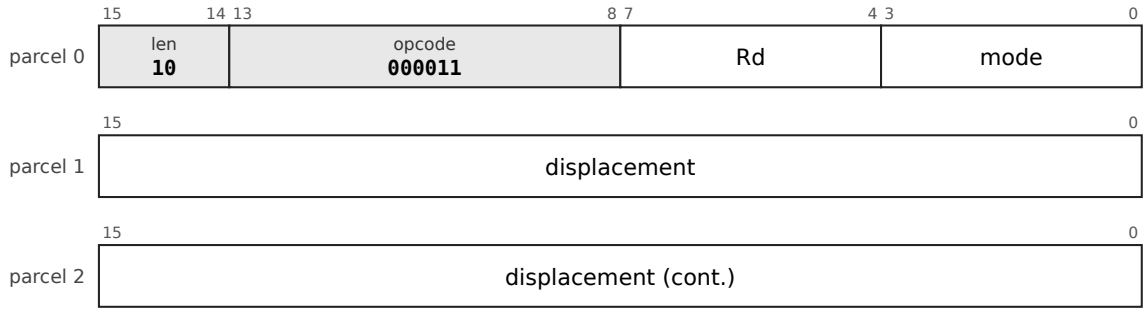
MOVL, opcode 000011, three parcels (modes base, postinc, predec) (6 bytes)



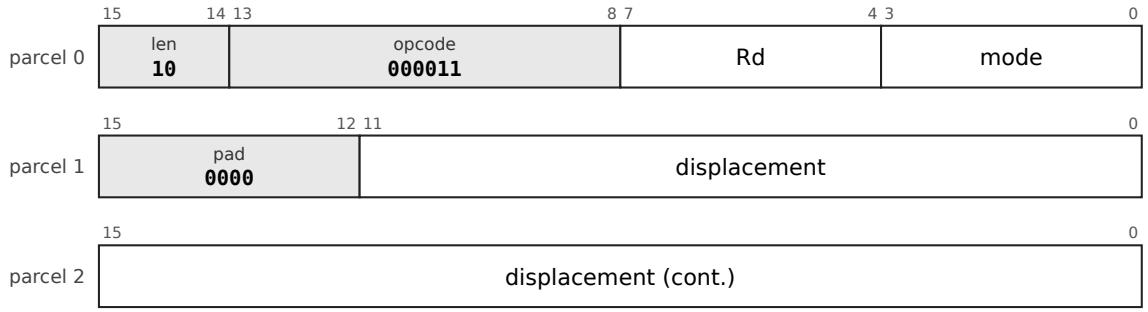
MOVL, opcode 000011, three parcels (modes index) (6 bytes)



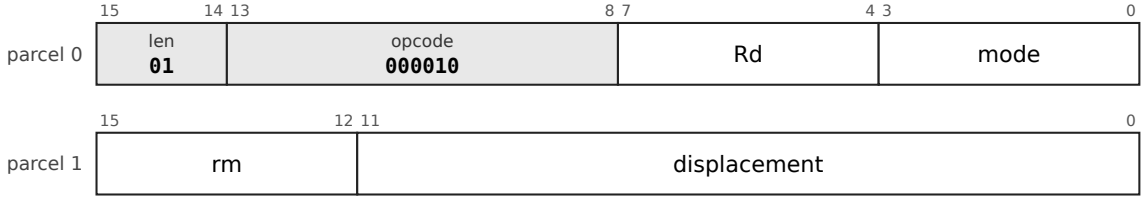
MOVL, opcode 000011, three parcels (modes pcrel, imm, abs) (6 bytes)



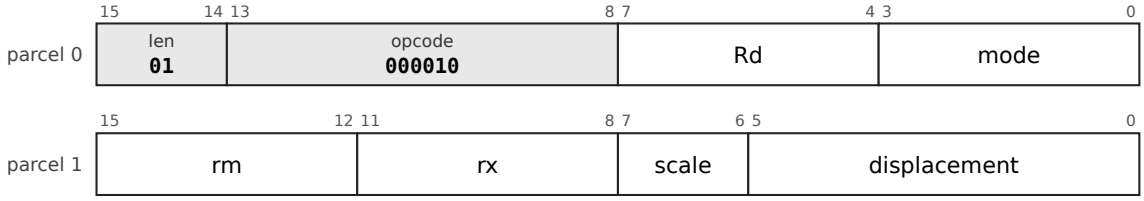
MOVL, opcode 000011, three parcels (modes dp) (6 bytes)



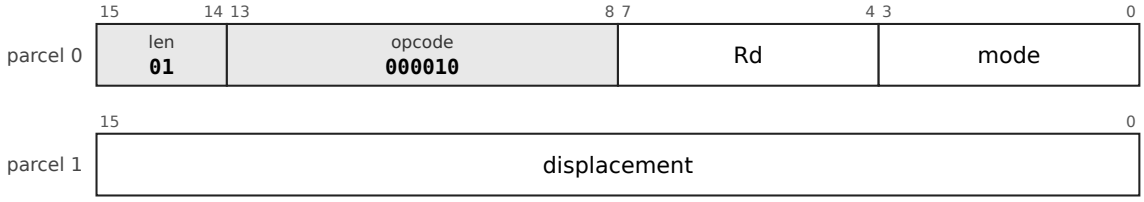
MOVL, opcode 000010, two parcels (modes base, postinc, predec) (4 bytes)



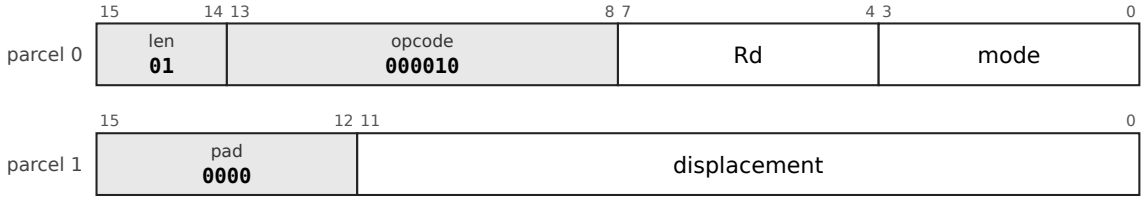
MOVL, opcode 000010, two parcels (modes index) (4 bytes)



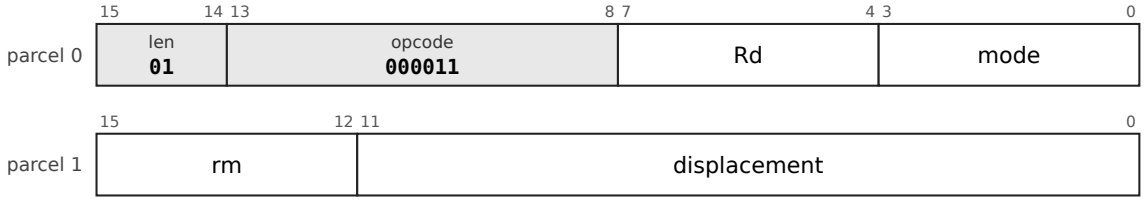
MOVL, opcode 000010, two parcels (modes pcrel, imm, abs) (4 bytes)



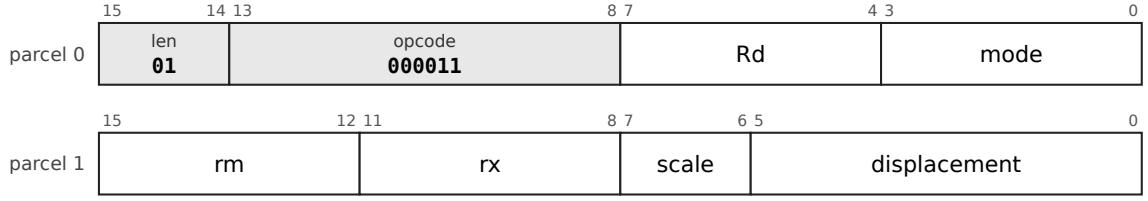
MOVL, opcode 000010, two parcels (modes dp) (4 bytes)



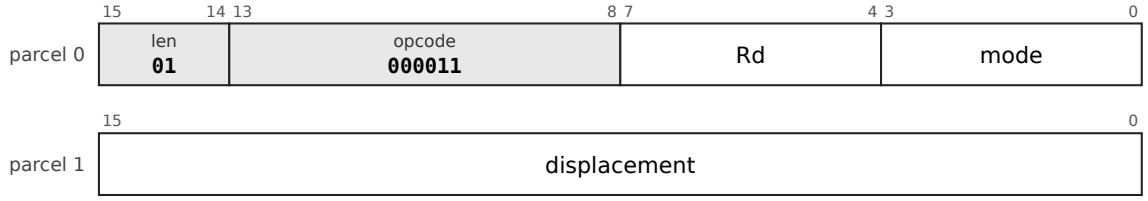
MOVL, opcode 000011, two parcels (modes base, postinc, predec) (4 bytes)



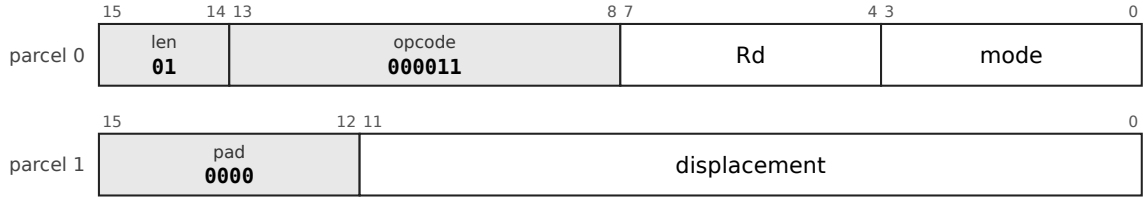
MOVL, opcode 000011, two parcels (modes index) (4 bytes)



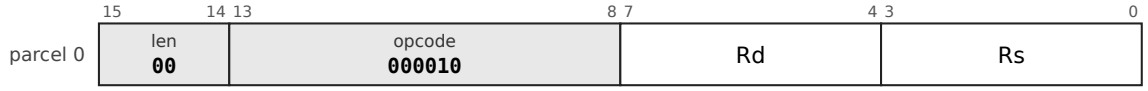
MOVL, opcode 000011, two parcels (modes pcrel, imm, abs) (4 bytes)



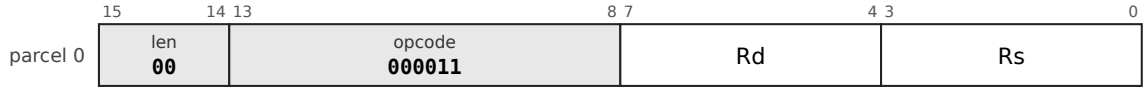
MOVL, opcode 000011, two parcels (modes dp) (4 bytes)



MOVL, opcode 000010, one parcel (2 bytes)



MOVL, opcode 000011, one parcel (2 bytes)



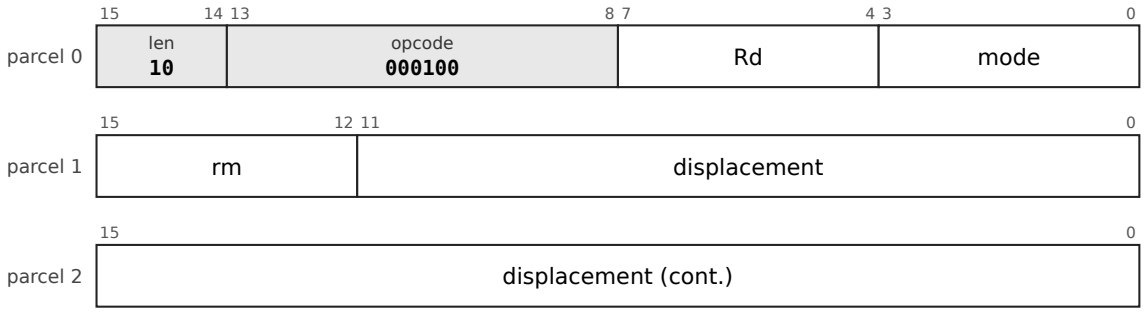

```
MOVL, MOVW, MOVB likewise
```

Mnemonic	Opcode
MOVL	000010 (0x02), 000011 (0x03)

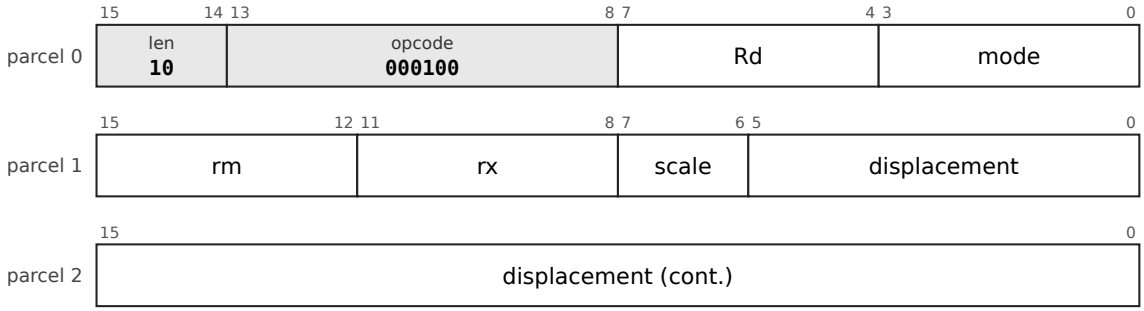
Operation, flags and exceptions as for the entry MOV above; 18 layout forms, each a chart of its own (above).

MOVW

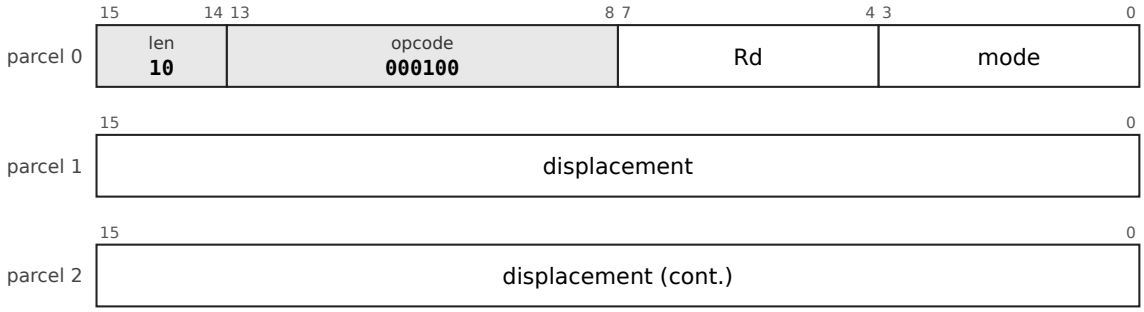
MOVW, opcode 000100, three parcels (modes base, postinc, predec) (6 bytes)



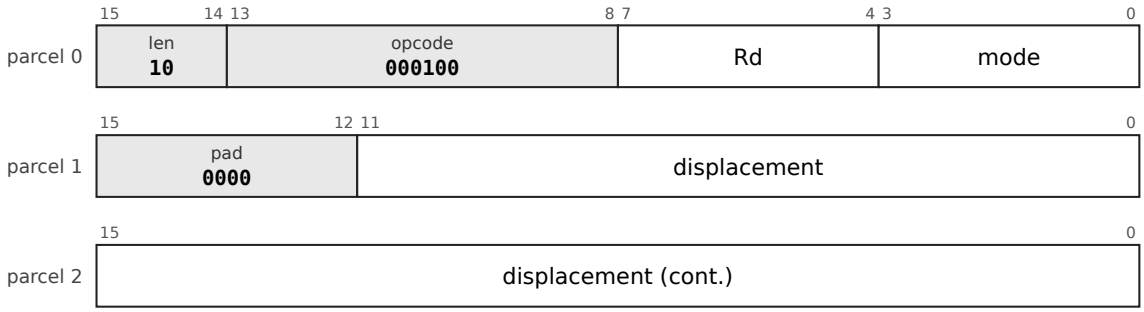
MOVW, opcode 000100, three parcels (modes index) (6 bytes)



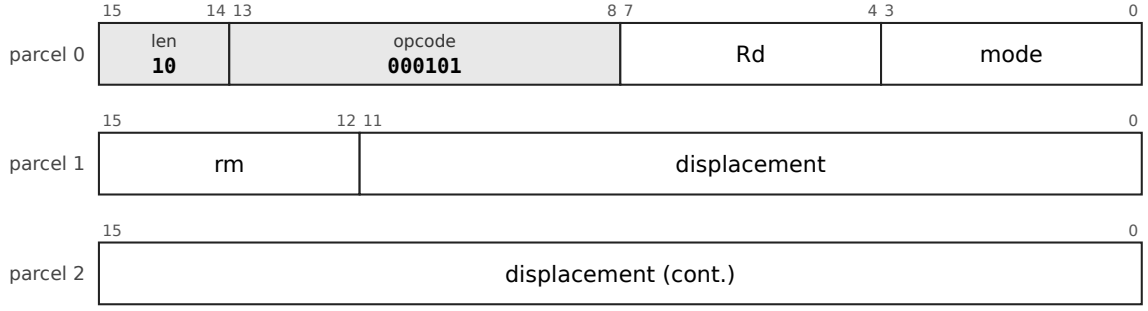
MOVW, opcode 000100, three parcels (modes pcrel, imm, abs) (6 bytes)



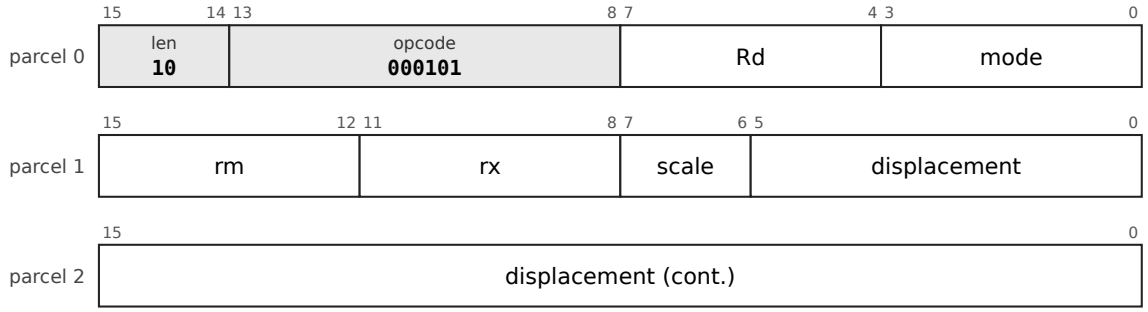
MOVW, opcode 000100, three parcels (modes dp) (6 bytes)



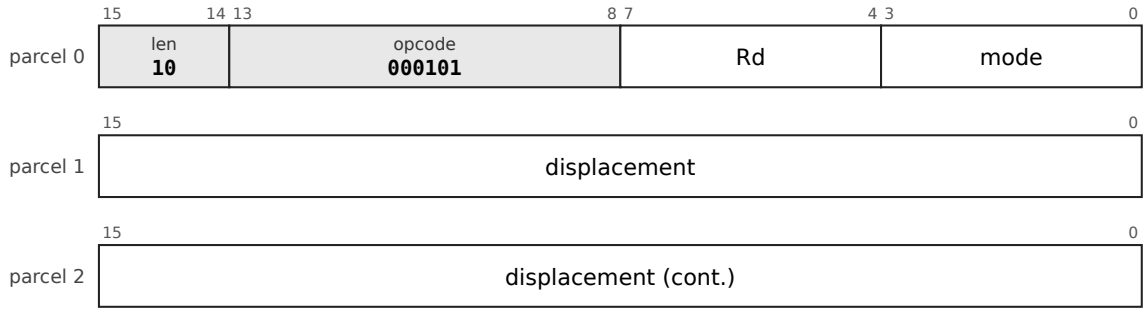
MOVW, opcode 000101, three parcels (modes base, postinc, predec) (6 bytes)



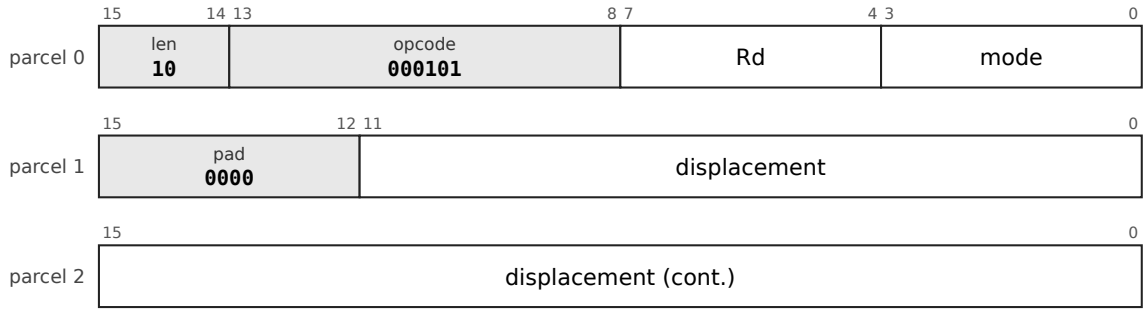
MOVW, opcode 000101, three parcels (modes index) (6 bytes)



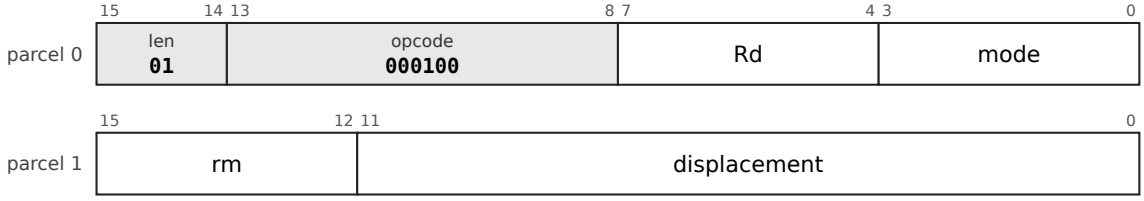
MOVW, opcode 000101, three parcels (modes pcrel, imm, abs) (6 bytes)



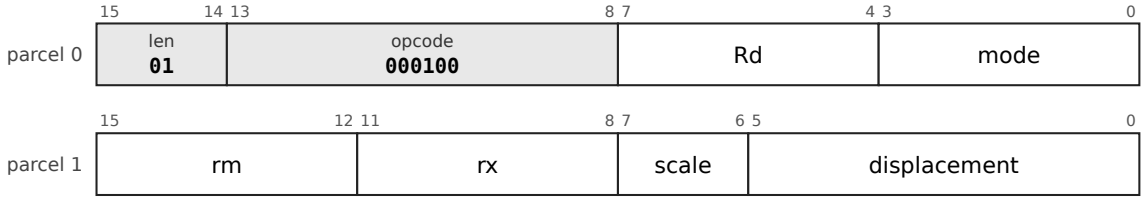
MOVW, opcode 000101, three parcels (modes dp) (6 bytes)



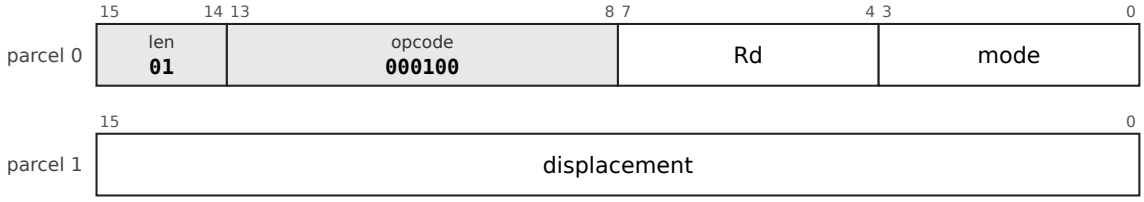
MOVW, opcode 000100, two parcels (modes base, postinc, predec) (4 bytes)



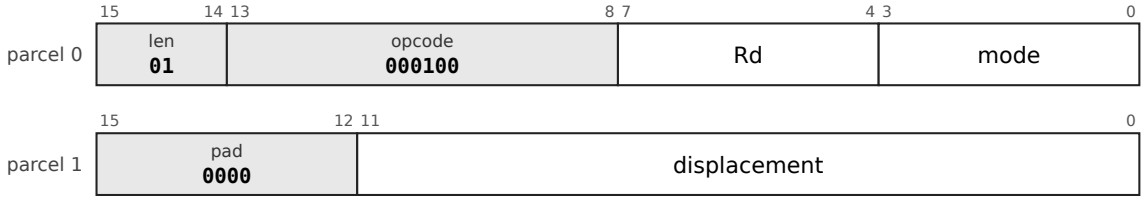
MOVW, opcode 000100, two parcels (modes index) (4 bytes)



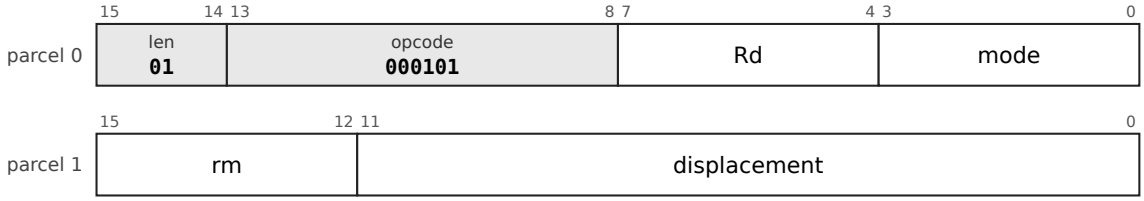
MOVW, opcode 000100, two parcels (modes pcrel, imm, abs) (4 bytes)



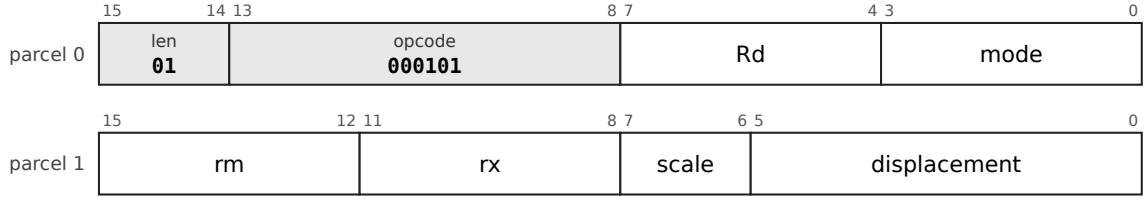
MOVW, opcode 000100, two parcels (modes dp) (4 bytes)



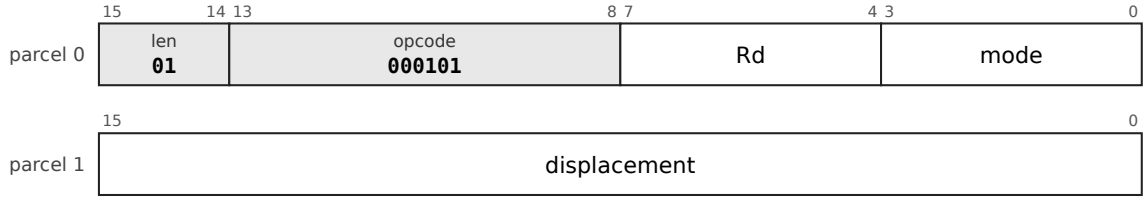
MOVW, opcode 000101, two parcels (modes base, postinc, predec) (4 bytes)



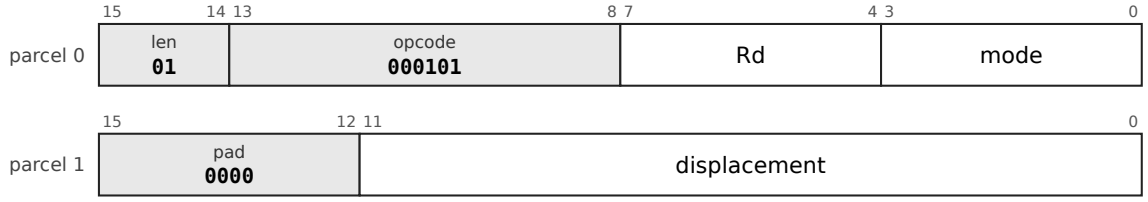
MOVW, opcode 000101, two parcels (modes index) (4 bytes)



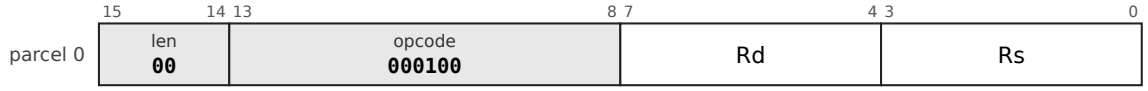
MOVW, opcode 000101, two parcels (modes pcrel, imm, abs) (4 bytes)



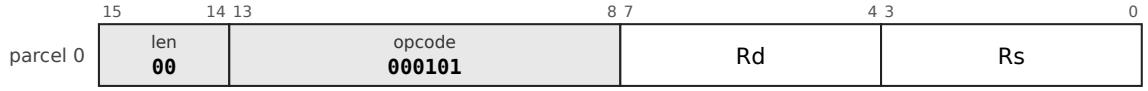
MOVW, opcode 000101, two parcels (modes dp) (4 bytes)



MOVW, opcode 000100, one parcel (2 bytes)



MOVW, opcode 000101, one parcel (2 bytes)

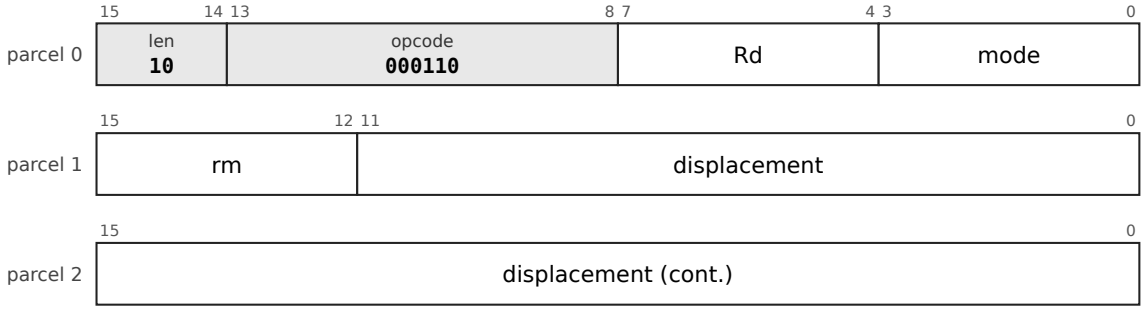


Mnemonic	Opcode
MOVW	000100 (0x04), 000101 (0x05)

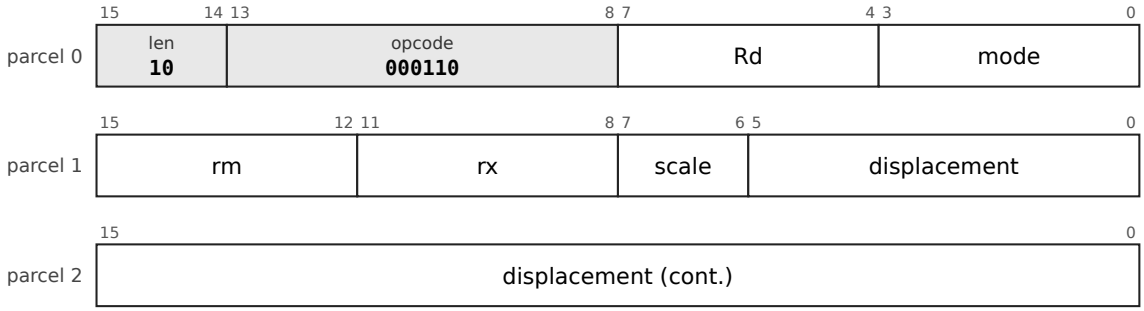
Operation, flags and exceptions as for the entry MOV above; 18 layout forms, each a chart of its own (above).

MOVB

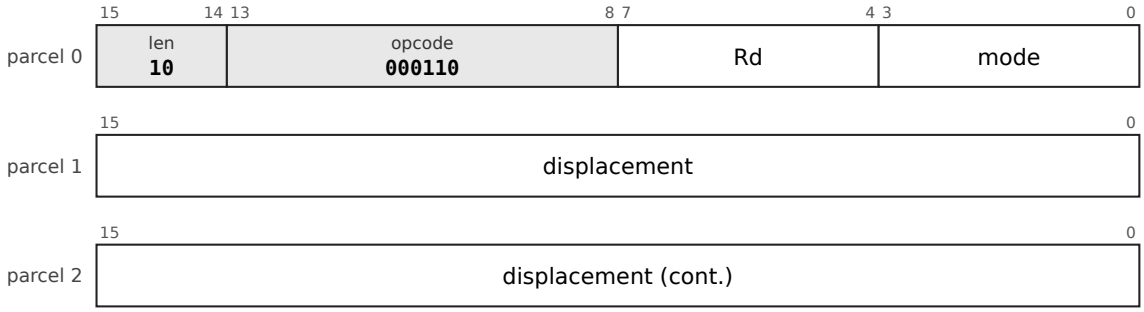
MOVB, opcode 000110, three parcels (modes base, postinc, predec) (6 bytes)



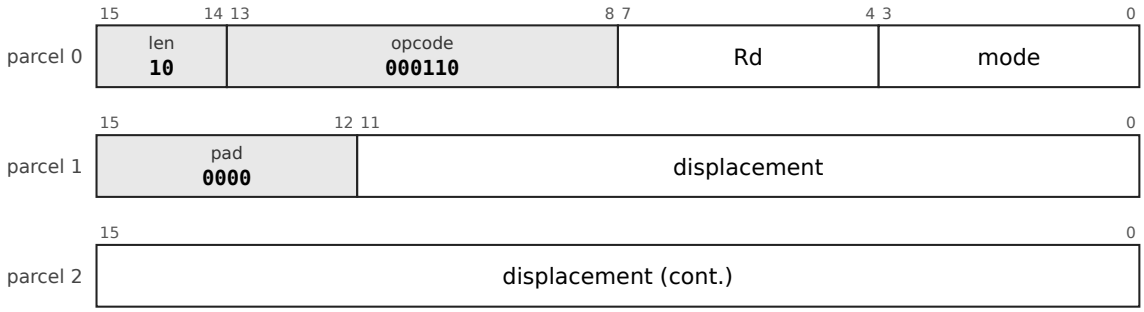
MOVB, opcode 000110, three parcels (modes index) (6 bytes)



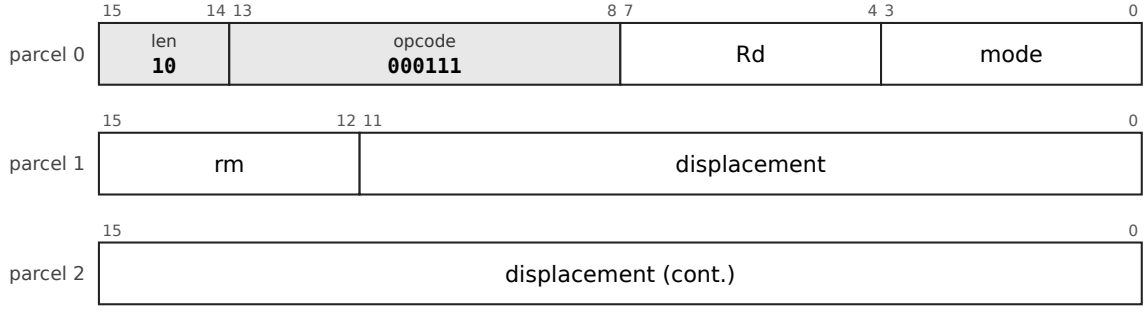
MOVB, opcode 000110, three parcels (modes pcrel, imm, abs) (6 bytes)



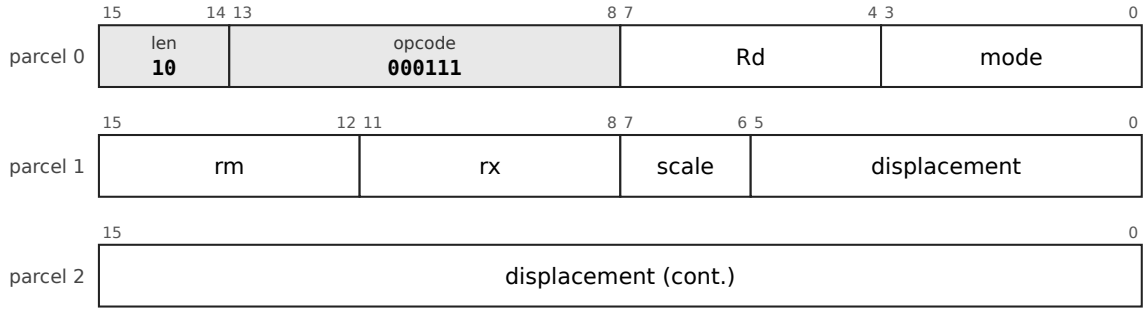
MOVB, opcode 000110, three parcels (modes dp) (6 bytes)



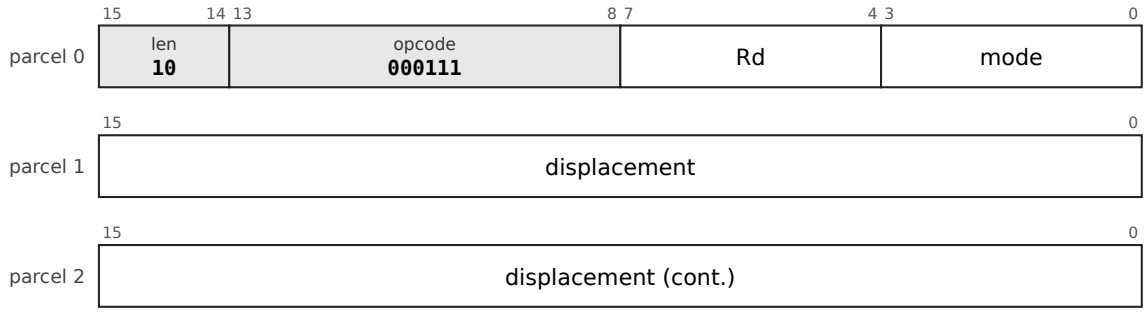
MOVB, opcode 000111, three parcels (modes base, postinc, predec) (6 bytes)



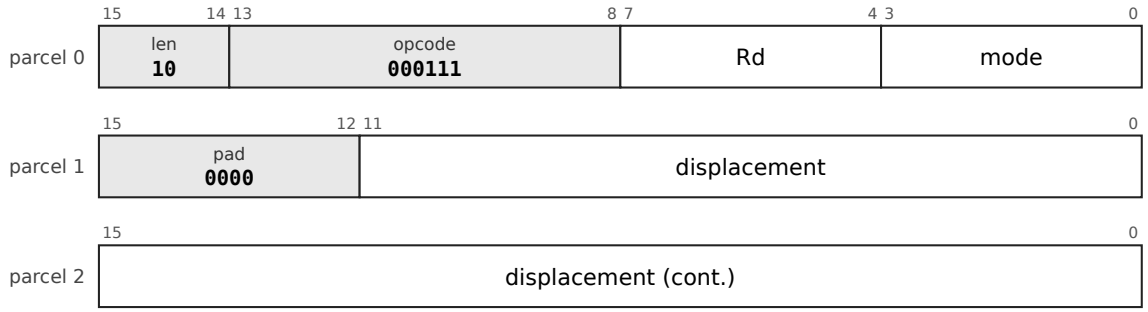
MOVB, opcode 000111, three parcels (modes index) (6 bytes)



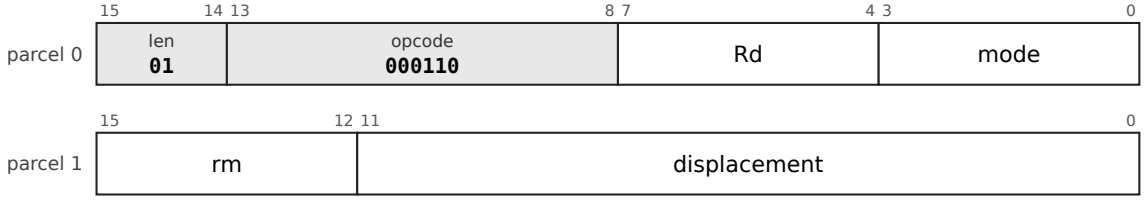
MOVB, opcode 000111, three parcels (modes pcrel, imm, abs) (6 bytes)



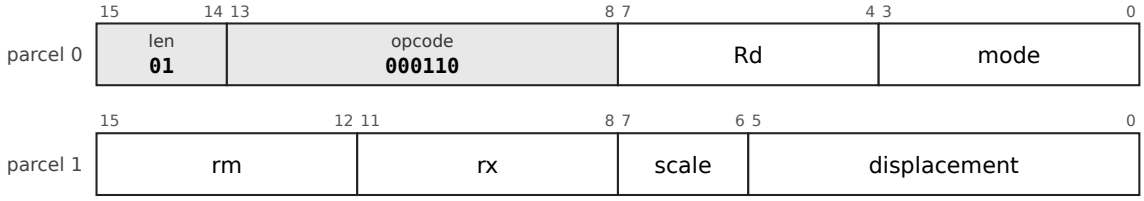
MOVB, opcode 000111, three parcels (modes dp) (6 bytes)



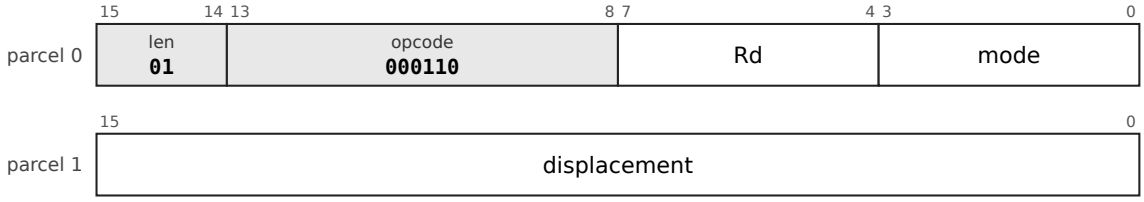
MOVB, opcode 000110, two parcels (modes base, postinc, predec) (4 bytes)



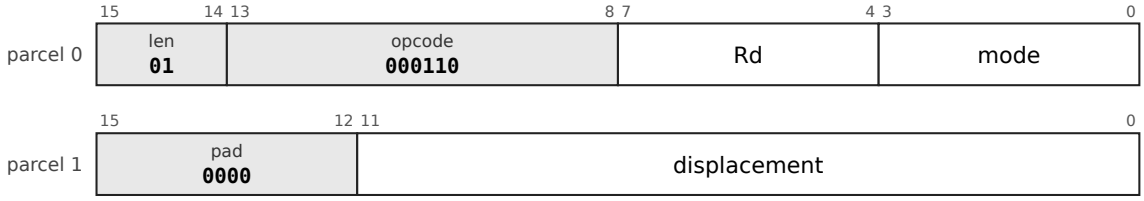
MOVB, opcode 000110, two parcels (modes index) (4 bytes)



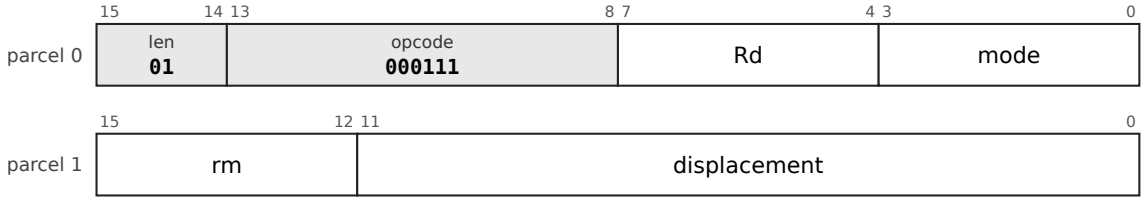
MOVB, opcode 000110, two parcels (modes pcrel, imm, abs) (4 bytes)



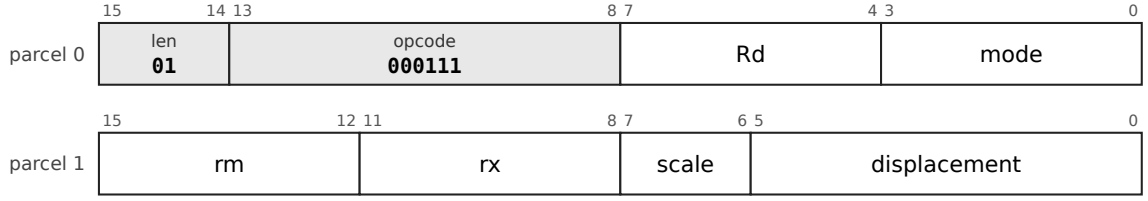
MOVB, opcode 000110, two parcels (modes dp) (4 bytes)



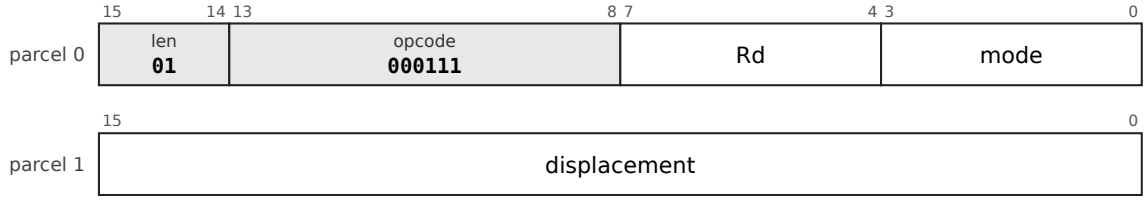
MOVB, opcode 000111, two parcels (modes base, postinc, predec) (4 bytes)



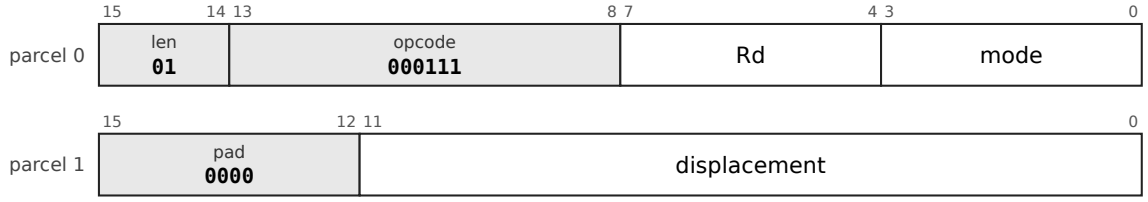
MOVB, opcode 000111, two parcels (modes index) (4 bytes)



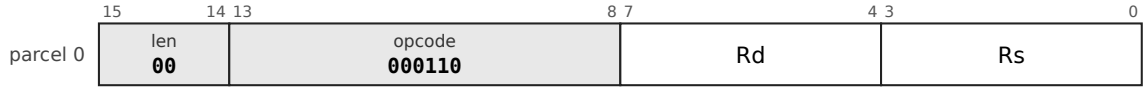
MOVB, opcode 000111, two parcels (modes pcrel, imm, abs) (4 bytes)



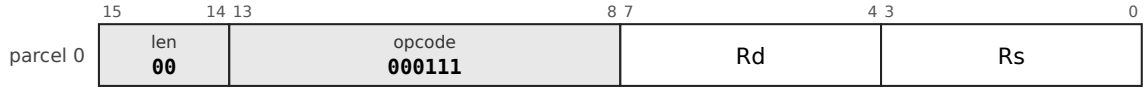
MOVB, opcode 000111, two parcels (modes dp) (4 bytes)



MOVB, opcode 000110, one parcel (2 bytes)



MOVB, opcode 000111, one parcel (2 bytes)



Mnemonic	Opcode
MOVB	000110 (0x06), 000111 (0x07)

Operation, flags and exceptions as for the entry MOV above; 18 layout forms, each a chart of its own (above).

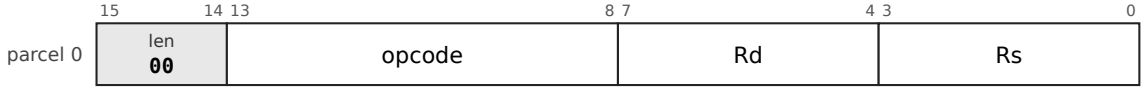
MOVZ: Move zero-extended (32, 16 and 8 bits)

Mnemonics: MOVZL, MOVZW, MOVZB

Syntax

```
MOVZL rm, Rd
MOVZW rm, Rd
MOVZB rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← zext(M[EA(rm)], size)      ; len = 00: Rd ← zext(Rs[8×size-1:0])
```

Flags None written.

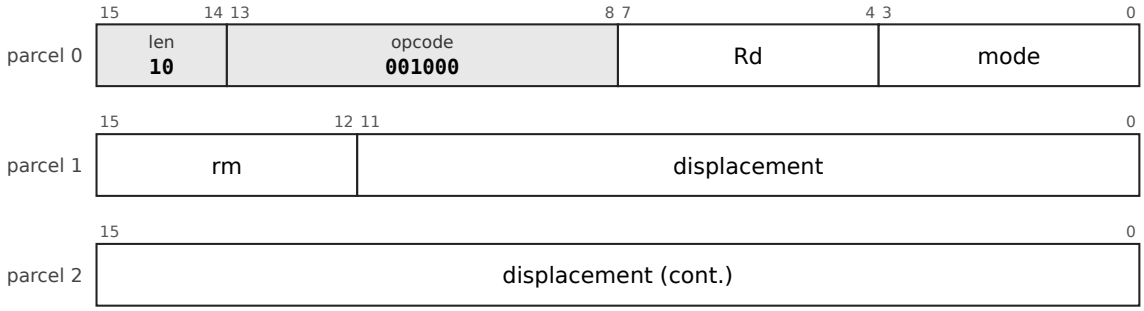
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

There is no zero-extending 64-bit move (it would be `MOVQ`). At `len = 00` these are the zero-extending half of the conversion set.

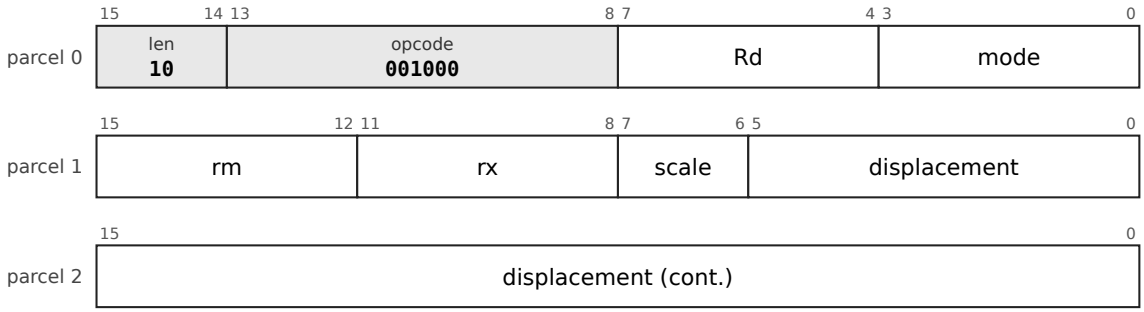
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MOVZL

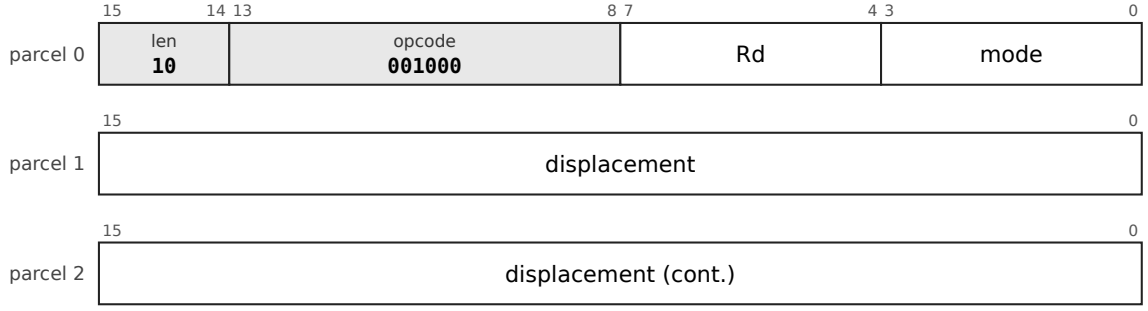
MOVZL, opcode 001000, three parcels (modes base, postinc, predec) (6 bytes)



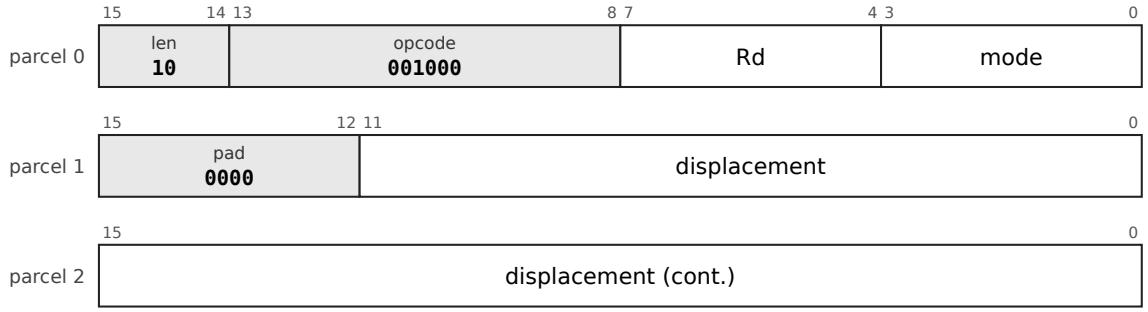
MOVZL, opcode 001000, three parcels (modes index) (6 bytes)



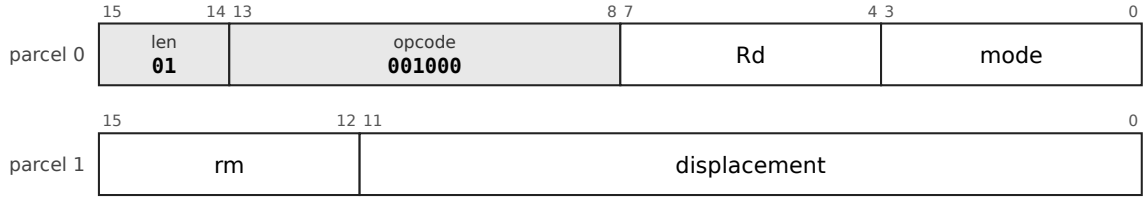
MOVZL, opcode 001000, three parcels (modes pcrel, imm, abs) (6 bytes)



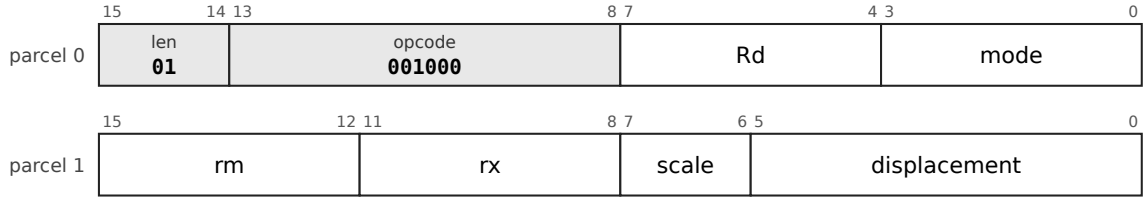
MOVZL, opcode 001000, three parcels (modes dp) (6 bytes)



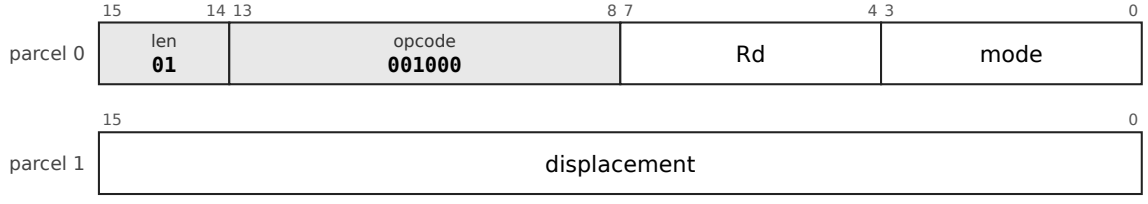
MOVZL, opcode 001000, two parcels (modes base, postinc, predec) (4 bytes)



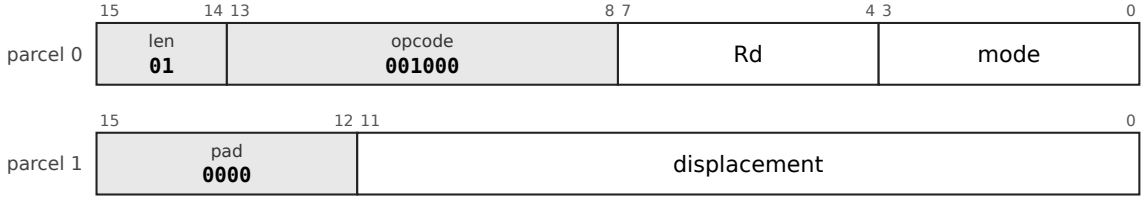
MOVZL, opcode 001000, two parcels (modes index) (4 bytes)



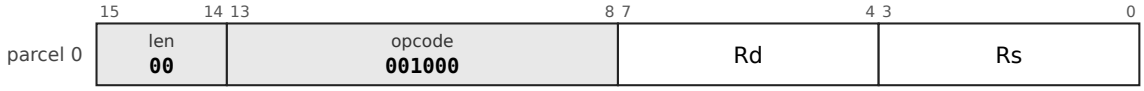
MOVZL, opcode 001000, two parcels (modes pcrel, imm, abs) (4 bytes)



MOVZL, opcode 001000, two parcels (modes dp) (4 bytes)



MOVZL, opcode 001000, one parcel (2 bytes)



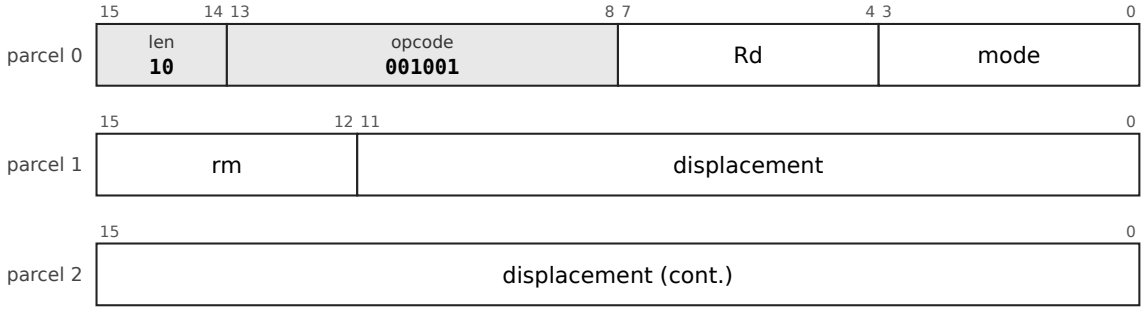

```
MOVZL rm, Rd
```

Mnemonic	Opcode
MOVZL	001000 (0x08)

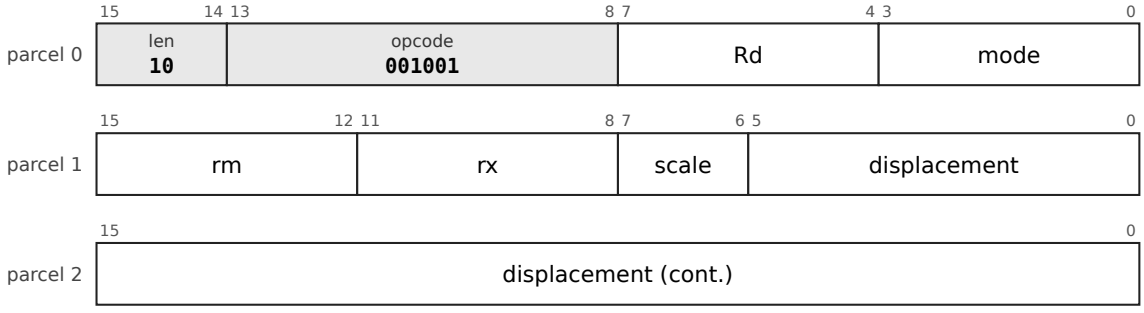
Operation, flags and exceptions as for the entry MOVZ above; 9 layout forms, each a chart of its own (above).

MOVZW

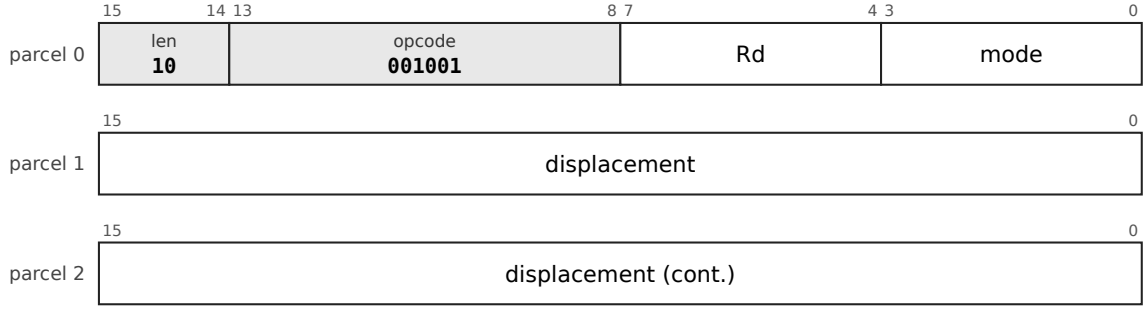
MOVZW, opcode 001001, three parcels (modes base, postinc, predec) (6 bytes)



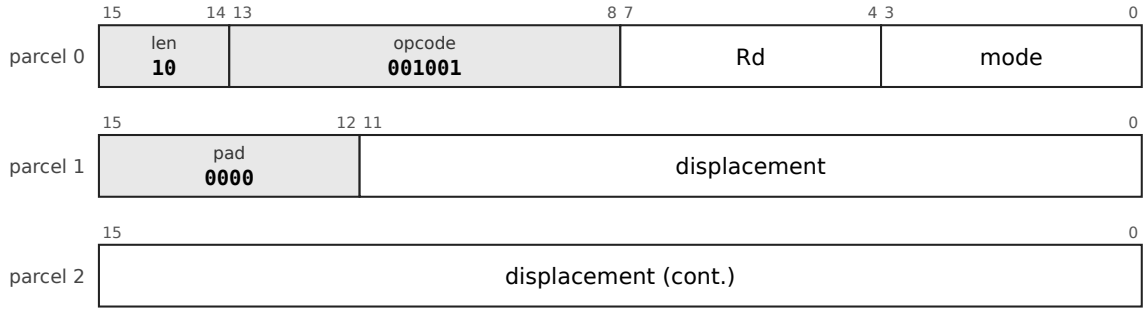
MOVZW, opcode 001001, three parcels (modes index) (6 bytes)



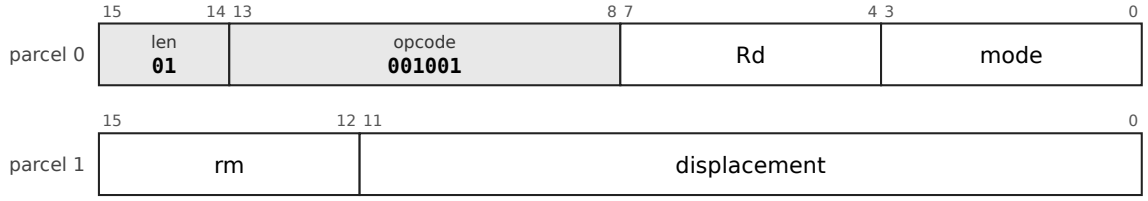
MOVZW, opcode 001001, three parcels (modes pcrel, imm, abs) (6 bytes)



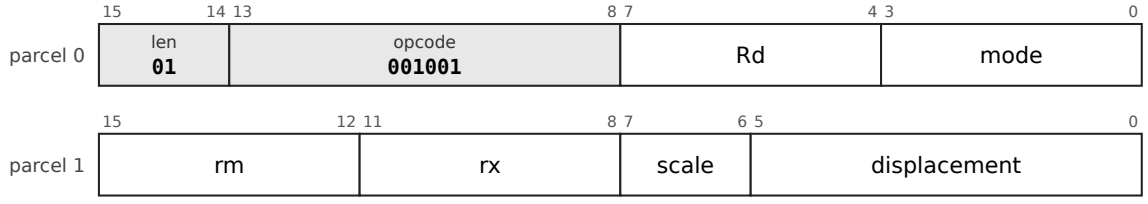
MOVZW, opcode 001001, three parcels (modes dp) (6 bytes)



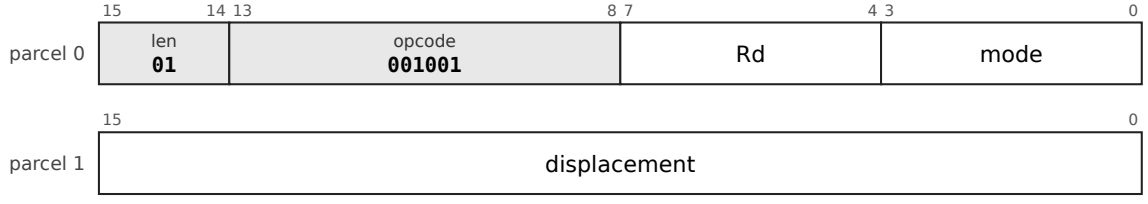
MOVZW, opcode 001001, two parcels (modes base, postinc, predec) (4 bytes)



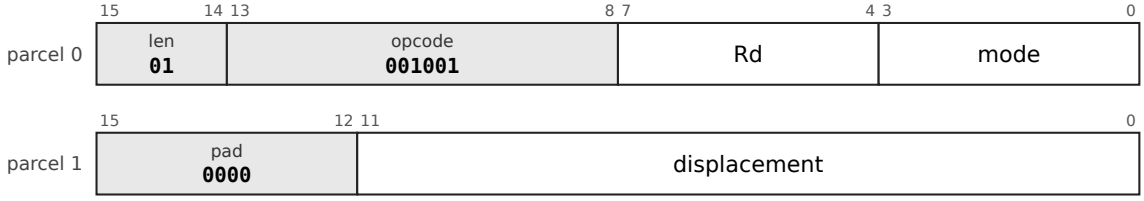
MOVZW, opcode 001001, two parcels (modes index) (4 bytes)



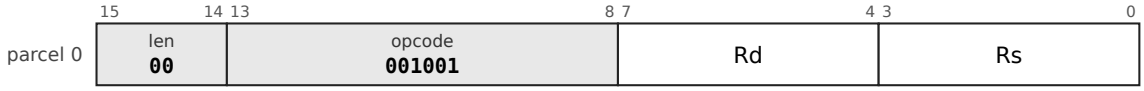
MOVZW, opcode 001001, two parcels (modes pcrel, imm, abs) (4 bytes)



MOVZW, opcode 001001, two parcels (modes dp) (4 bytes)



MOVZW, opcode 001001, one parcel (2 bytes)



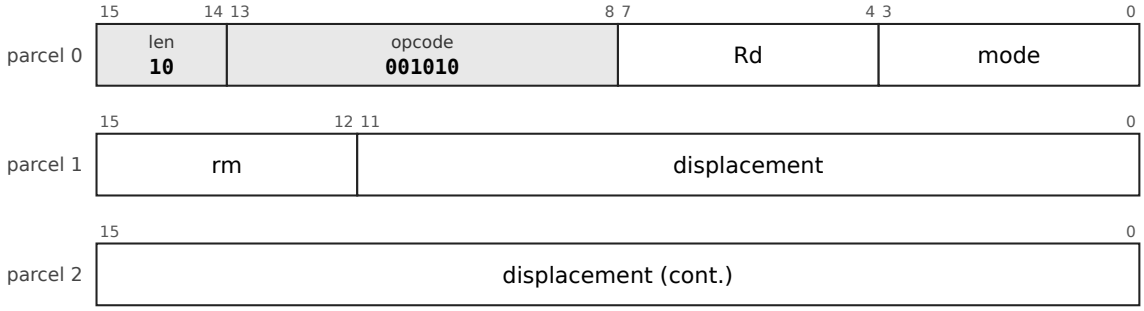

```
MOVZW rm, Rd
```

Mnemonic	Opcode
MOVZW	001001 (0x09)

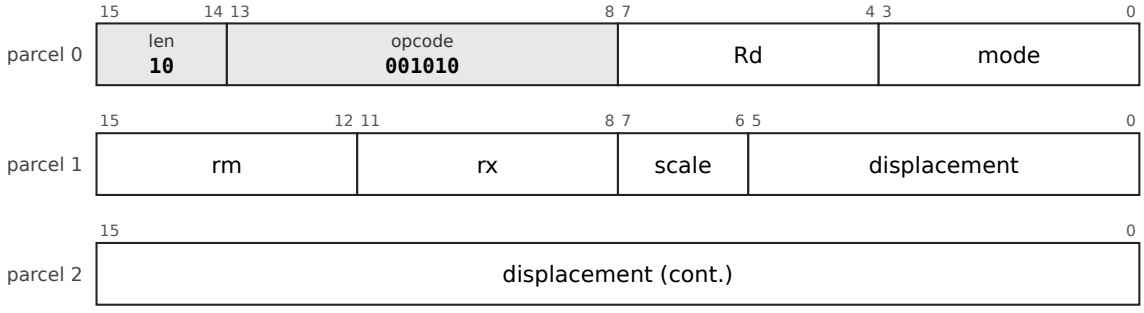
Operation, flags and exceptions as for the entry MOVZ above; 9 layout forms, each a chart of its own (above).

MOVZB

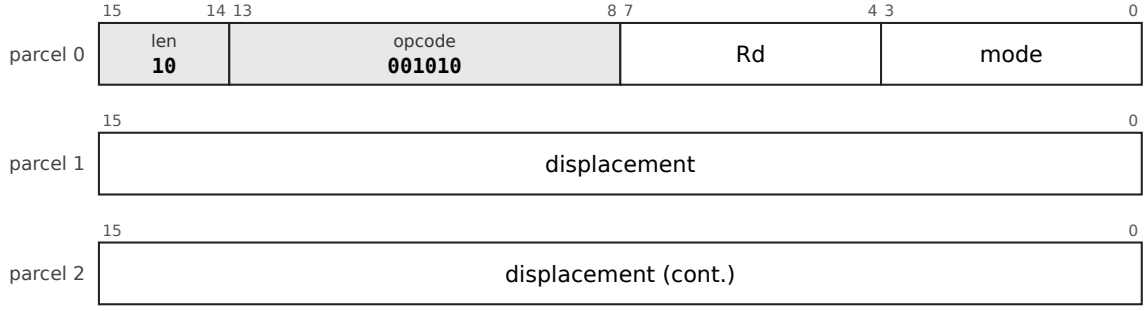
MOVZB, opcode 001010, three parcels (modes base, postinc, predec) (6 bytes)



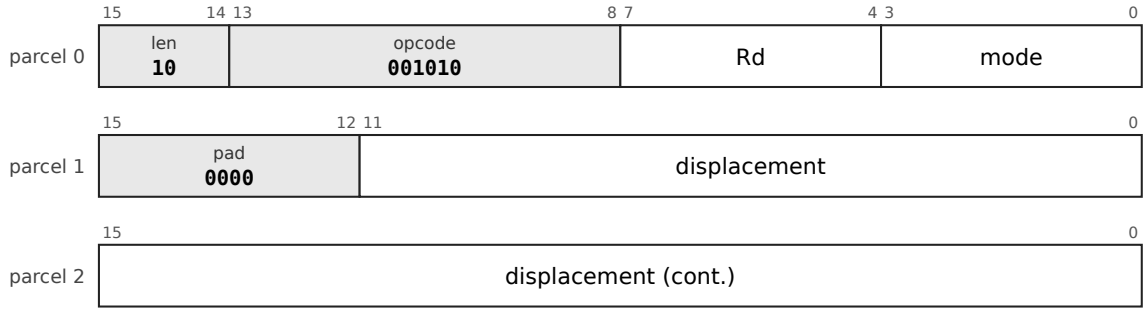
MOVZB, opcode 001010, three parcels (modes index) (6 bytes)



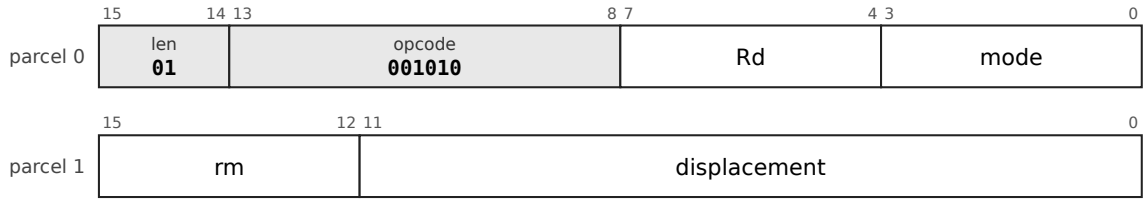
MOVZB, opcode 001010, three parcels (modes pcrel, imm, abs) (6 bytes)



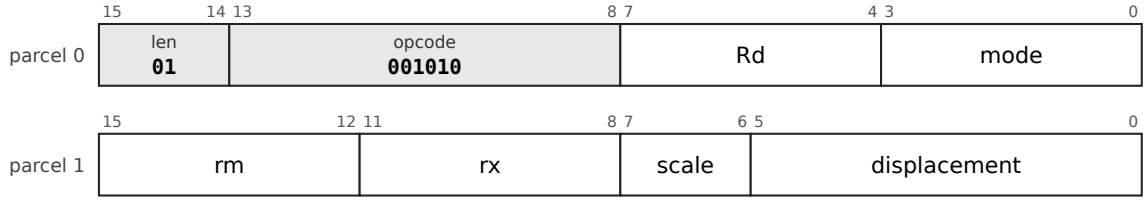
MOVZB, opcode 001010, three parcels (modes dp) (6 bytes)



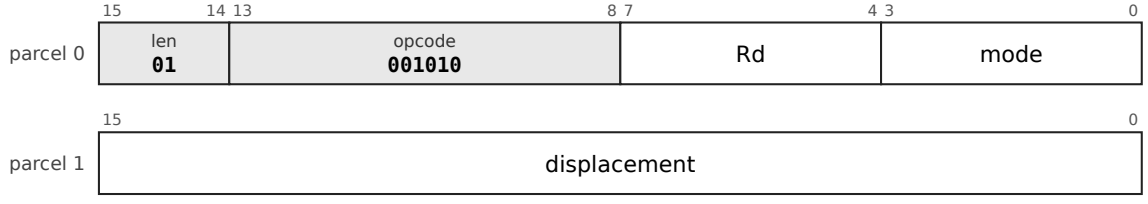
MOVZB, opcode 001010, two parcels (modes base, postinc, predec) (4 bytes)



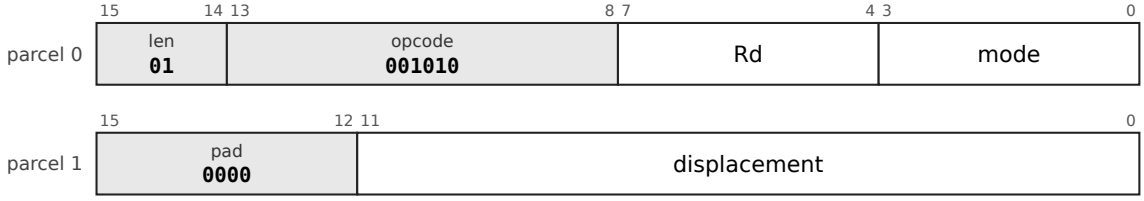
MOVZB, opcode 001010, two parcels (modes index) (4 bytes)



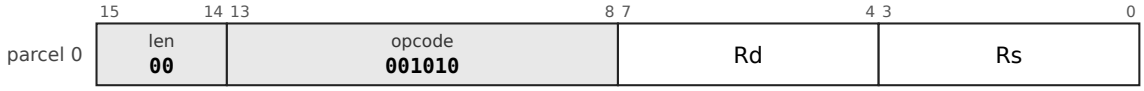
MOVZB, opcode 001010, two parcels (modes pcrel, imm, abs) (4 bytes)



MOVZB, opcode 001010, two parcels (modes dp) (4 bytes)



MOVZB, opcode 001010, one parcel (2 bytes)



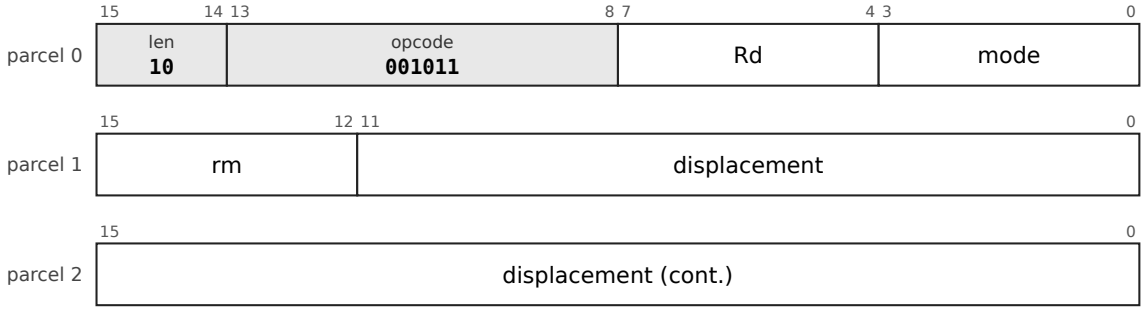

```
MOVZB rm, Rd
```

Mnemonic	Opcode
MOVZB	001010 (0x0A)

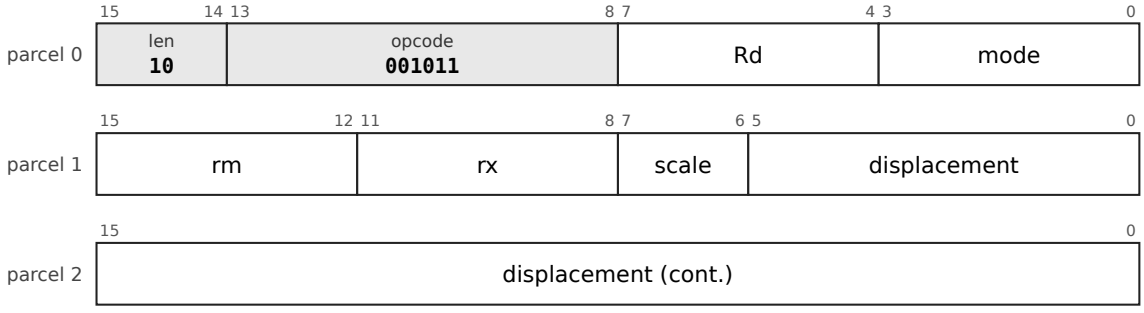
Operation, flags and exceptions as for the entry MOVZ above; 9 layout forms, each a chart of its own (above).

LEA: Load effective address

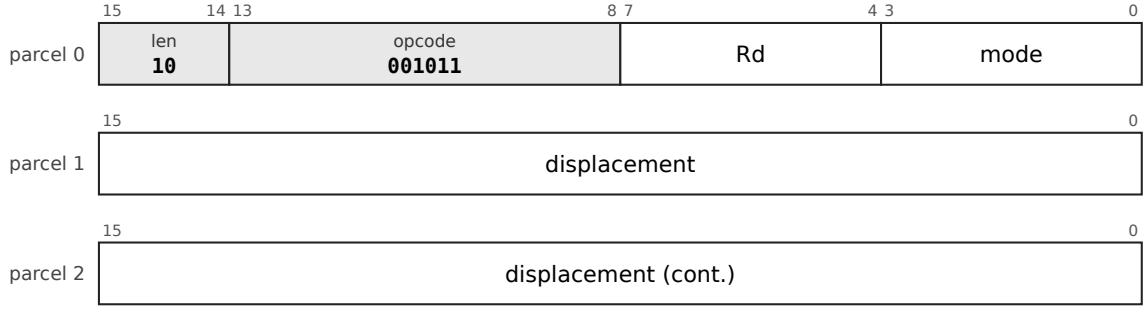
LEA, opcode 001011, three parcels (modes base, postinc, predec) (6 bytes)



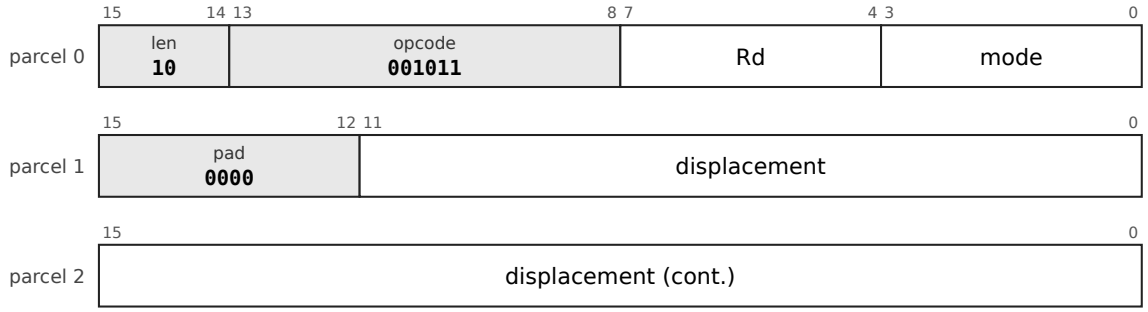
LEA, opcode 001011, three parcels (modes index) (6 bytes)



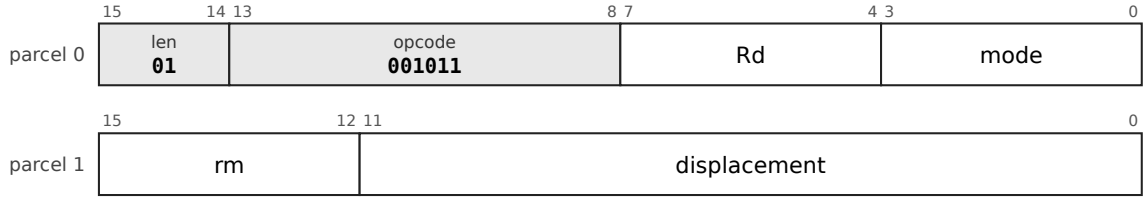
LEA, opcode 001011, three parcels (modes pcrel, imm, abs) (6 bytes)



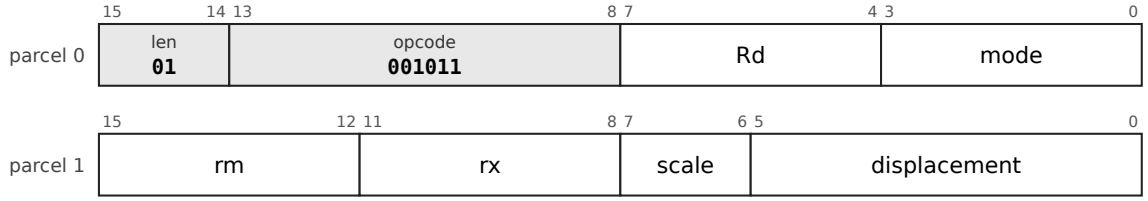
LEA, opcode 001011, three parcels (modes dp) (6 bytes)



LEA, opcode 001011, two parcels (modes base, postinc, predec) (4 bytes)



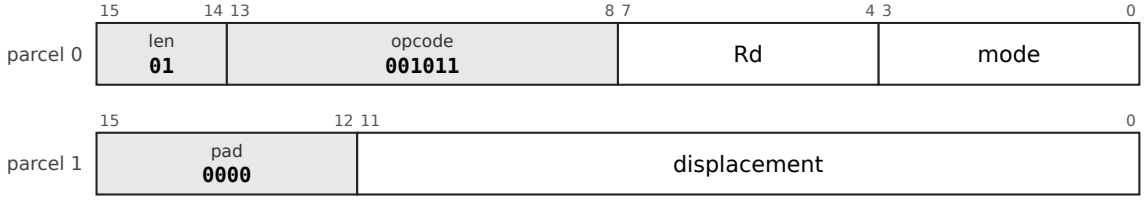
LEA, opcode 001011, two parcels (modes index) (4 bytes)



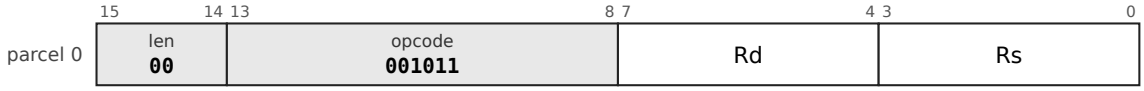
LEA, opcode 001011, two parcels (modes pcrel, imm, abs) (4 bytes)



LEA, opcode 001011, two parcels (modes dp) (4 bytes)



LEA, opcode 001011, one parcel (2 bytes)

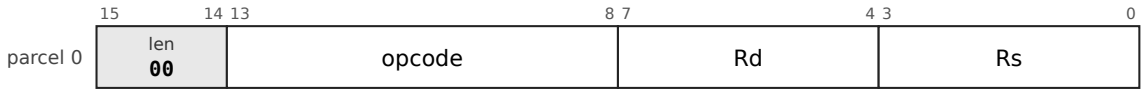


Mnemonics: `LEA`

Syntax

```
LEA d(Rn), Rd
LEA d(PC), Rd
LEA d(Rn)(Rx×s), Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
`LEA`	`001011 (0x0B)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← EA(rm)          ; no memory access
```

Flags None written.

Exceptions The operand must name memory; `len = 00` (a register) and the immediate mode are reserved. No memory access is made, so the instruction takes no memory fault. A reserved encoding aborts with the illegal-instruction vector.

`LEA 0(PC), Rd` is how the program counter is read: it puts the address of the `LEA` itself in `Rd`. At `len = 10` it forms any address within ±2 GB of itself PC-relatively in six bytes. The base of every PC-relative displacement is the address of the first parcel of the instruction.

Why it is here. The PDP-11 and the VAX made the program counter a register (R7, R15). Here the program counter is kept out of the register file, because every register write would then be a potential branch, and a read-only view of it (R7 as a view of the PC) is refused, because it would fix for ever *which* PC is meant (ARM32 is stuck with `PC+8`). `LEA 0(PC), Rn` gives the capability without the commitment: the address of the instruction itself, in one instruction. The same instruction forms any address PC-relatively, which is what makes position-independent code free (section 9.16).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

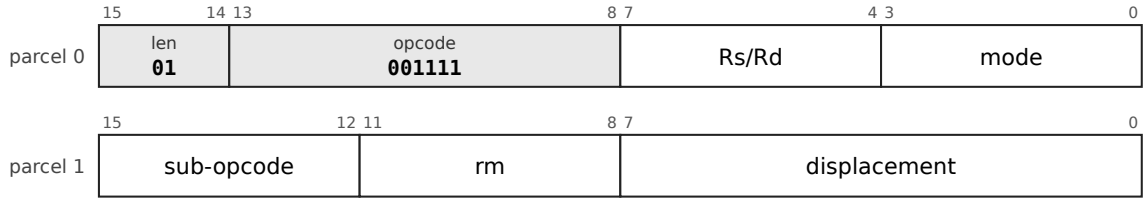
MOVU: Unaligned load and store

Mnemonics: `MOVUB`, `MOVUW`, `MOVUL`, `MOVUQ`

Syntax

```
MOVUQ rm, Rd        ; load, any alignment
MOVUQ Rs, rm        ; store, any alignment
MOVUW, MOVUL, MOVUB likewise
```

Encoding



The chart is the layout `r20-mesc-1`, Memory escape (MESC), two parcels (`len = 01`): Addressing modes (`low`) that use this layout: `base` 0000, `index` 0001, `pcrel` 0010, `dp` 0011, `imm` 0100, `abs` 0101, `postinc` 0110, `predec` 0111. `low` of parcel 0 is an addressing mode; parcel 1 holds the sub-opcode, `rm` and the first 8 bits of the displacement.

Mnemonic	Opcode	Sub-opcode
`MOVUB`	`001111 (0x0F)`	`0000 (0x0)`
`MOVUW`	`001111 (0x0F)`	`0010 (0x2)`
`MOVUL`	`001111 (0x0F)`	`0100 (0x4)`
`MOVUQ`	`001111 (0x0F)`	`0110 (0x6)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mesc-1`, `r20-mesc-2`.

Operation

```
size = 1, 2, 4 or 8 bytes
load :  Rd ← sext(M[EA(rm)], size)           ; the access may cross an alignment boundary
store:  M[EA(rm)] ← Rs[8×size-1:0]
```

Flags None written.

Exceptions Page and protection faults and bus errors as for any memory operand; **no alignment fault**.

The ordinary moves fault on a misaligned address (the alignment fault is the tag check of section 9.5); these are the explicit forms for data that is not aligned, in `MESC`.

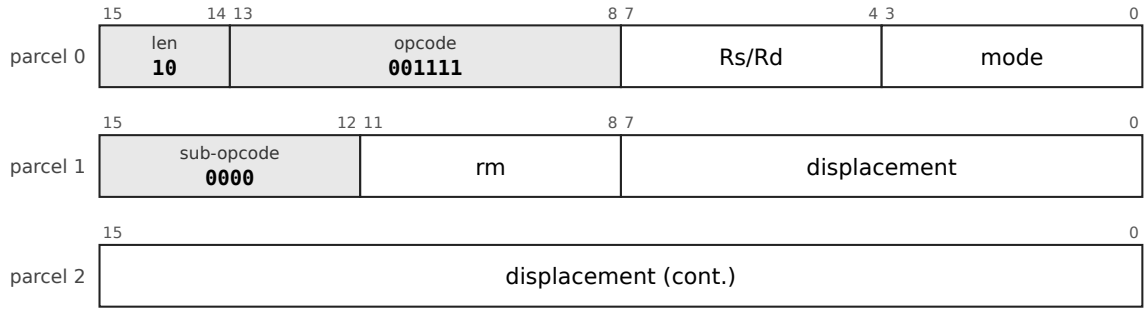
Why it is here. Data alignment is architectural on this machine (9.2.11): an ordinary quad access to an address that is not a multiple of 8 aborts, and that abort doubles as the tag check for tagged pointers; a load form that ignores low pointer bits is rejected for that reason. Packed on-disk and network structures still need unaligned access, so `MESC` carries explicit unaligned forms rather than weakening the rule for every load. No further reason is recorded for these four.

Cf. 6.6, 6.4.

Status specified; **not yet implemented** in the reference executor.

MOVUB

MOVUB, opcode 001111, three parcels, sub-opcode 0000 (modes base, index, pcrel, dp, imm,



MOVUB, opcode 001111, two parcels, sub-opcode 0000 (modes base, index, pcrel, dp, imm, a

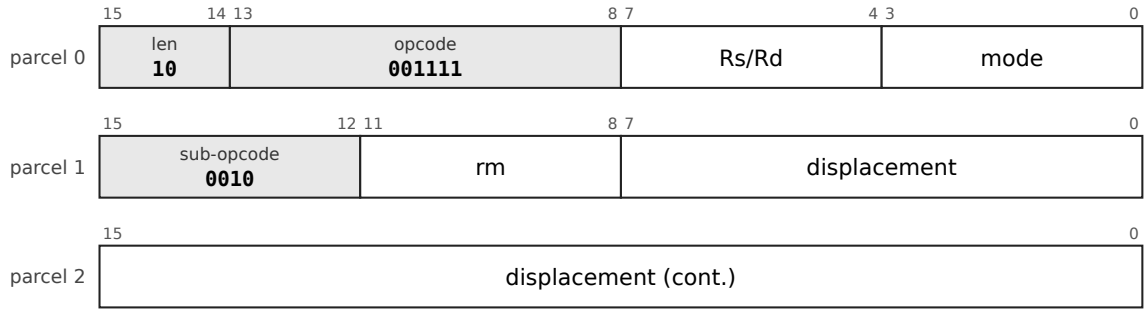


Mnemonic	Opcode	Sub-opcode
MOVUB	001111 (0x0F)	0000 (0x0)

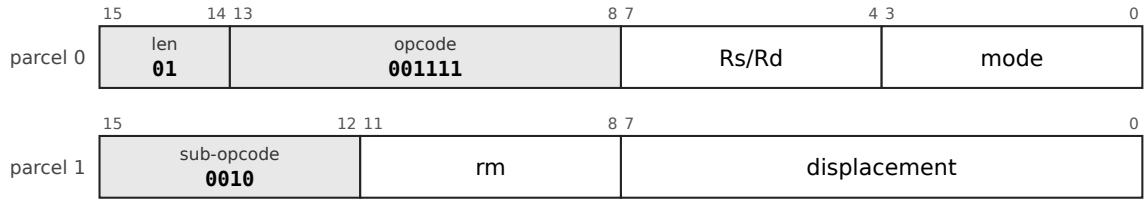
Operation, flags and exceptions as for the entry MOVU above; 2 layout forms, each a chart of its own (above).

MOVUW

MOVUW, opcode 001111, three parcels, sub-opcode 0010 (modes base, index, pcrel, dp, imm



MOVUW, opcode 001111, two parcels, sub-opcode 0010 (modes base, index, pcrel, dp, imm, a



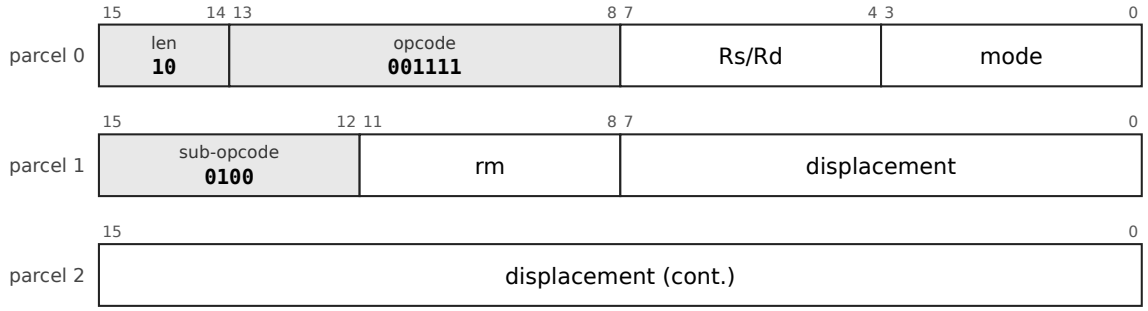

```
MOVUW, MOVUL, MOVUB likewise
```

Mnemonic	Opcode	Sub-opcode
MOVUW	001111 (0x0F)	0010 (0x2)

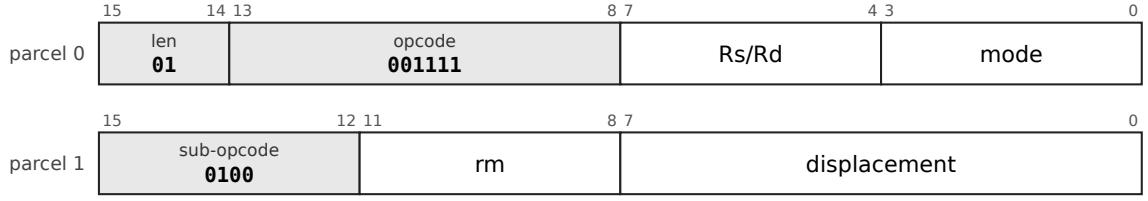
Operation, flags and exceptions as for the entry MOVU above; 2 layout forms, each a chart of its own (above).

MOVUL

MOVUL, opcode 001111, three parcels, sub-opcode 0100 (modes base, index, pcrel, dp, imm,



MOVUL, opcode 001111, two parcels, sub-opcode 0100 (modes base, index, pcrel, dp, imm, a

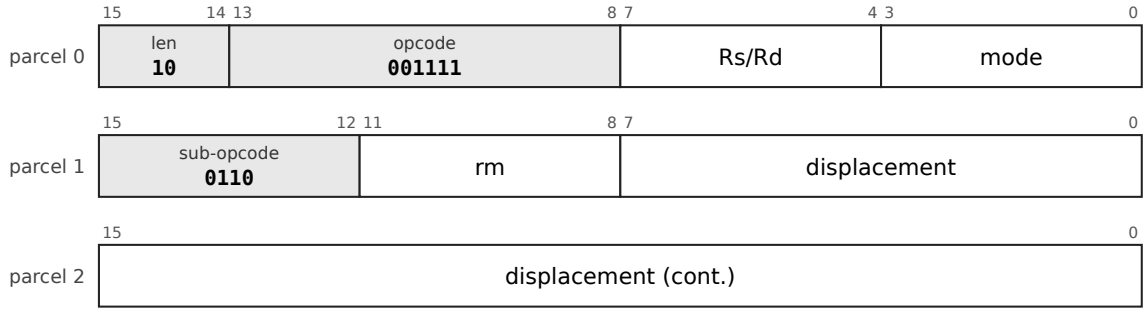


Mnemonic	Opcode	Sub-opcode
MOVUL	001111 (0x0F)	0100 (0x4)

Operation, flags and exceptions as for the entry MOVU above; 2 layout forms, each a chart of its own (above).

MOVUQ

MOVUQ, opcode 001111, three parcels, sub-opcode 0110 (modes base, index, pcrel, dp, imm



MOVUQ, opcode 001111, two parcels, sub-opcode 0110 (modes base, index, pcrel, dp, imm, a




```
MOVUQ rm, Rd
MOVUQ Rs, rm
```

Mnemonic	Opcode	Sub-opcode
MOVUQ	001111 (0x0F)	0110 (0x6)

Operation, flags and exceptions as for the entry MOVU above; 2 layout forms, each a chart of its own (above).

9.7 Integer arithmetic

Cf. section 6.3 for the divide-by-zero vector (DIV, DIVU, MOD, MODU).

9.7.1 ADD, SUB, CMP, MUL and MULH are two bytes with register operands; MULHU (high half of an unsigned product), DIV, DIVU, MOD and MODU are in the arithmetic escape. *(D)*

9.7.2 CMP is written `CMP Rd, rm`, register first, and computes `Rd - rm`. *(D)*

9.7.3 ADD sets C on a carry; SUB and CMP set C on a borrow, the PDP-11 sense, so BHIS and BCC mean "no borrow", unsigned greater or equal. *(D)*

9.7.4 ADC and SBC add and subtract the carry; INC, DEC, NEG, CLR and TST are single-operand forms that work on a register or memory. *(opcode group SOP)*

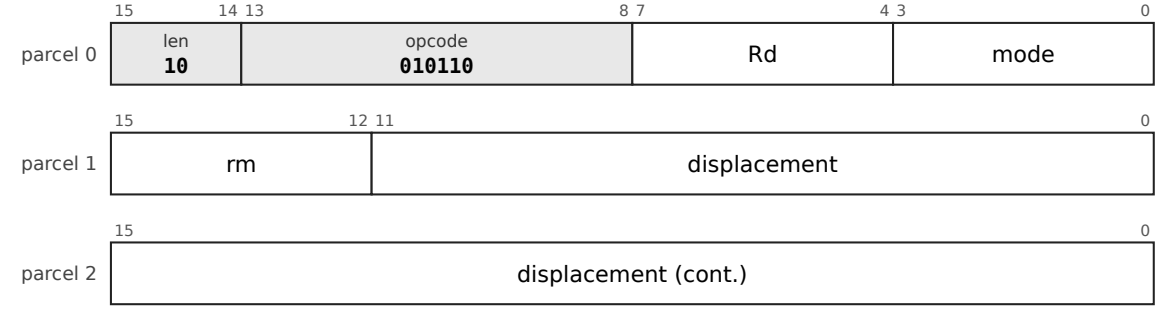
9.7.5 `SET cc, Rd` (write 1 if the condition holds, else 0) is in the arithmetic escape. *(D)*

9.7.6 ABS, MIN, MINU, MAX and MAXU are assembler aliases, not native instructions: ABS is the absolute value, and the others the smaller or the larger of two registers, signed or unsigned, with N and Z from the result, C unchanged, and V cleared (set by ABS on the most negative number). *(D)* Their expansion is open *(P)*: the flag-driven conditional select has no form yet, and a candidate over the branch-free MASKEQZ and MASKNEZ (a comparison, SET into a scratch register, the two masks and BIS) needs a scratch register and changes C. The present assembler still encodes the five words as sub-opcodes of the arithmetic escape; their dictionary entries show that encoding as the build's, not the design's. *(D)*

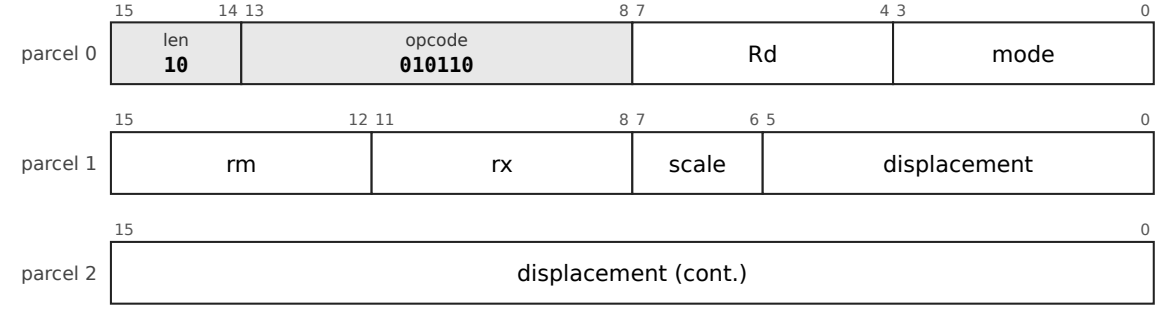
Commentary. **Precedent.** ADD, SUB, CMP, BIT, BIS, BIC and the single-operand set are the PDP-11's, unchanged in name and meaning except for the order of CMP's operands. **Trade-offs.** The PDP-11 wrote `CMP a, b` to compute `a - b`; since the destination of an ALU operation here must be a register, a literal `src, dst` reading would force every comparison against an immediate or memory to be reversed, so the register comes first. Divide is in the escape group, at four bytes, because compilers turn division by a constant into MULH plus a shift, so a real divide is rare. **Improvements.** The high-half multiply that bignum code needs is present; a second unsigned multiply is the same instruction as the first and is not provided. **Benefits.** SET replaces a branch around a move for the very common "store the truth value of a comparison", and absolute value, minimum and maximum (as aliases) remove a compare-and-branch from small loops. **Modern comparables.** RISC-V has MULH, MULHU and MULHSU, and AArch64 SMULH and UMULH; x86 has a one-operand multiply that produces both halves; AArch64's conditional select and set instructions are the nearest to SET. **Use.** MULH and MULHU are for multi-precision arithmetic and for division by constants; ADC and SBC carry a multi-word add or subtract; MIN, MAX and ABS (aliases) appear under a tenth of a percent of compiled code but each replaces a branch. **Impact.** *Kernel:* ADC and SBC carry multiword arithmetic for the kernel's 128-bit counters and for the bignum code in the base system. *Compiler:* MULH turns division by a constant into a multiply and a shift; SET materialises a comparison without a branch; CMP takes the register first, as the compiler orders its operands. *Hardware:* One adder on the carry chain; on an FPGA the multiplier is built from two to four multiplier blocks over four to eight cycles; the divide is a sequencer.

ADD: Add

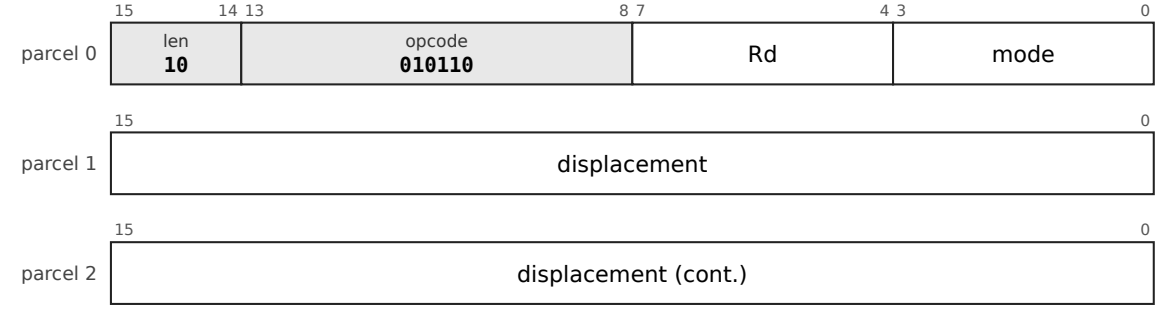
ADD, opcode 010110, three parcels (modes base, postinc, predec) (6 bytes)



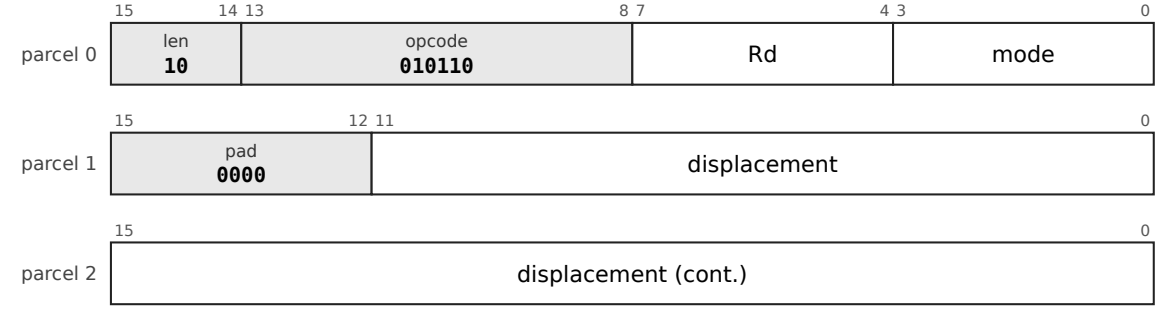
ADD, opcode 010110, three parcels (modes index) (6 bytes)



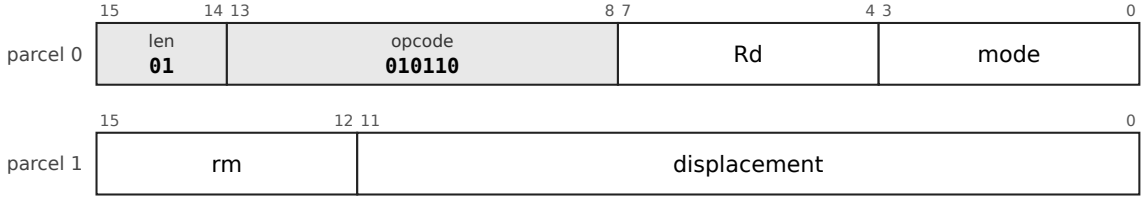
ADD, opcode 010110, three parcels (modes pcrel, imm, abs) (6 bytes)



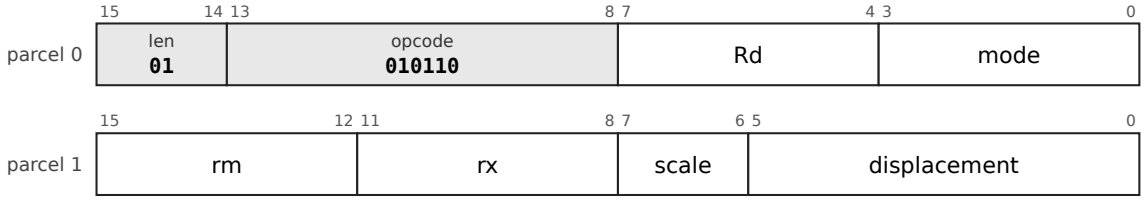
ADD, opcode 010110, three parcels (modes dp) (6 bytes)



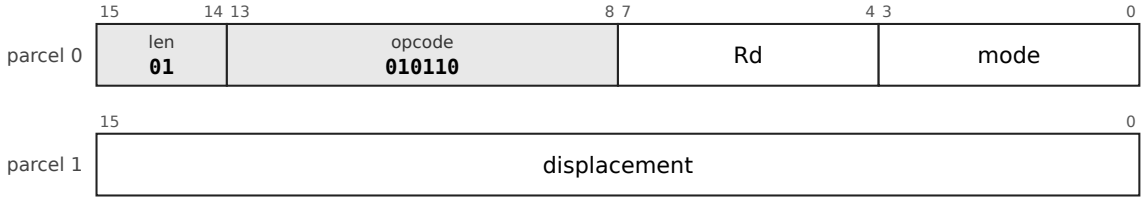
ADD, opcode 010110, two parcels (modes base, postinc, predec) (4 bytes)



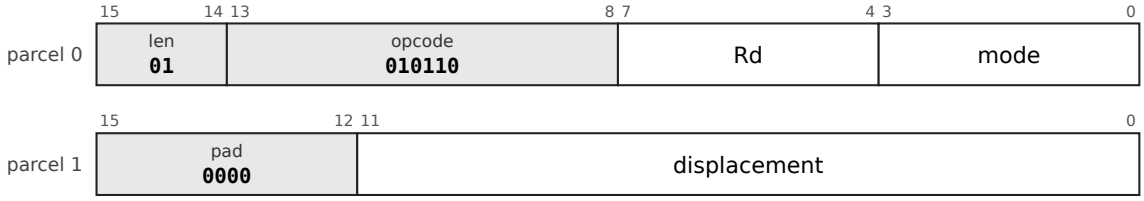
ADD, opcode 010110, two parcels (modes index) (4 bytes)



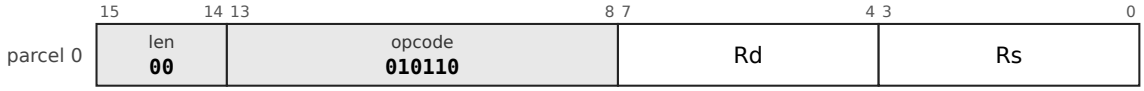
ADD, opcode 010110, two parcels (modes pcrel, imm, abs) (4 bytes)



ADD, opcode 010110, two parcels (modes dp) (4 bytes)



ADD, opcode 010110, one parcel (2 bytes)

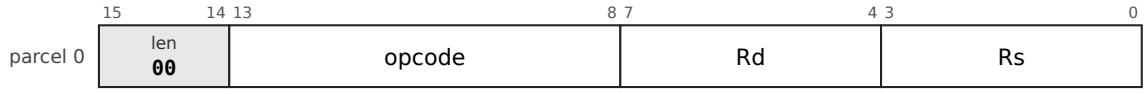


Mnemonics: `ADD`

Syntax

```
ADD rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
ADD	010110 (0x16)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← Rd + rm
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a carry out of bit 63.

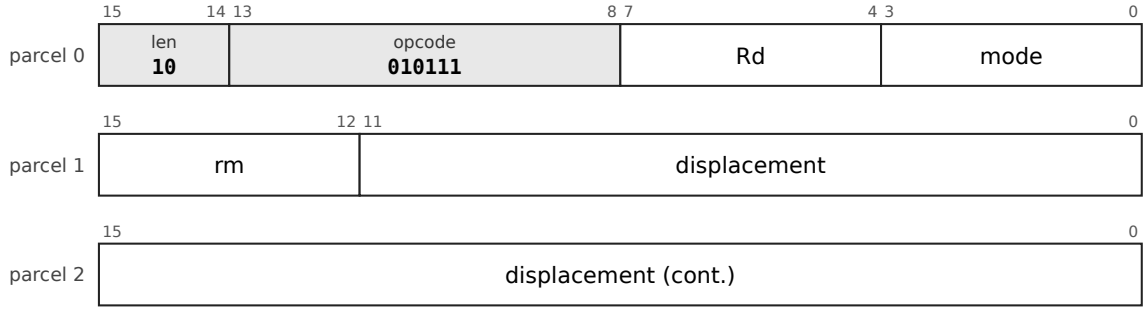
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

All ALU operations are 64 bits, with no size field. Page 2 has `ADD #imm, rd` with a 4-bit immediate and `ADD #(8*n), SP`.

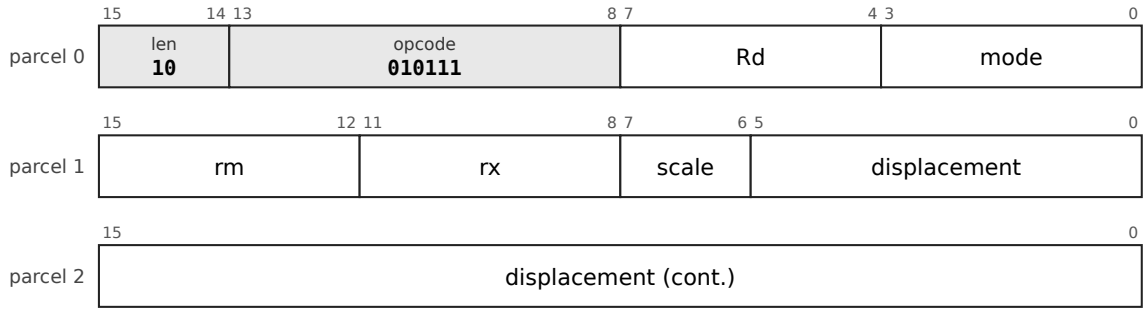
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

SUB: Subtract

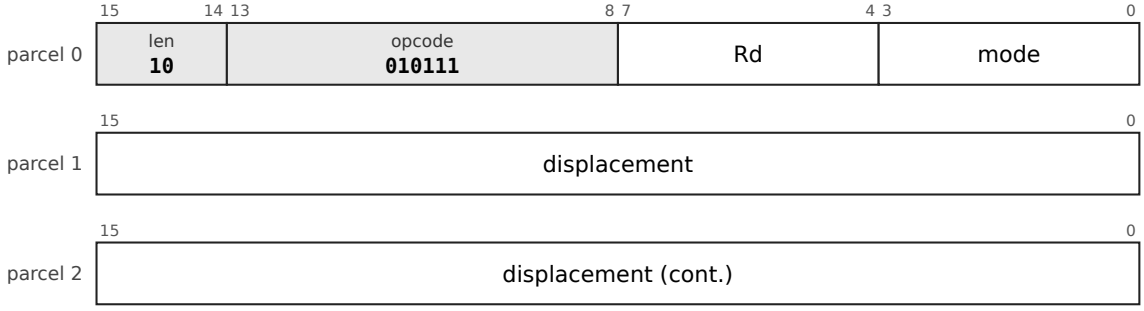
SUB, opcode 010111, three parcels (modes base, postinc, predec) (6 bytes)



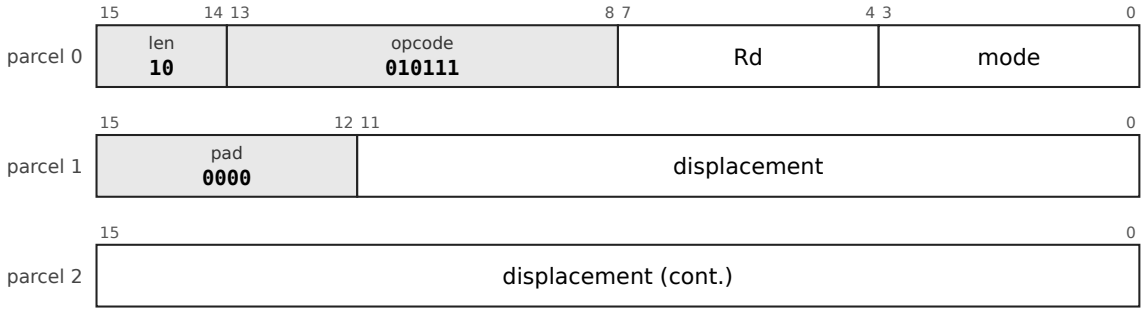
SUB, opcode 010111, three parcels (modes index) (6 bytes)



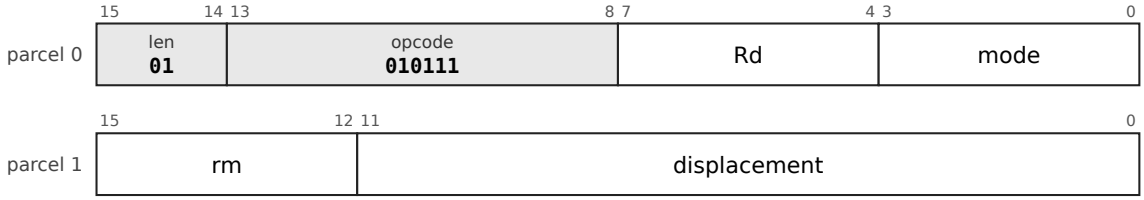
SUB, opcode 010111, three parcels (modes pcrel, imm, abs) (6 bytes)



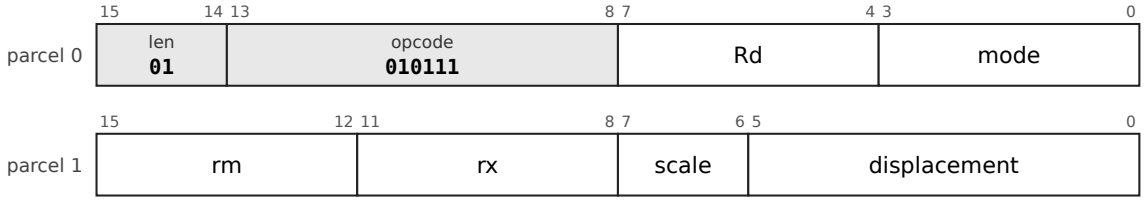
SUB, opcode 010111, three parcels (modes dp) (6 bytes)



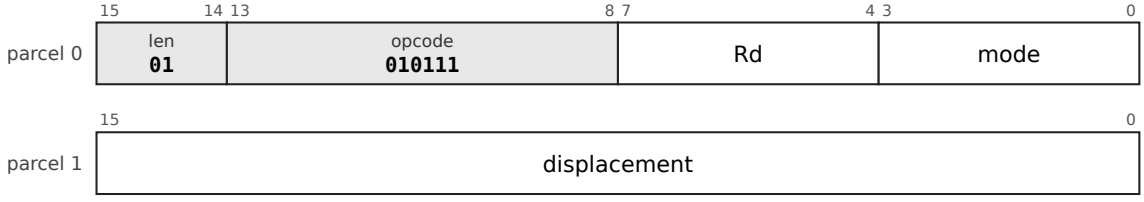
SUB, opcode 010111, two parcels (modes base, postinc, predec) (4 bytes)



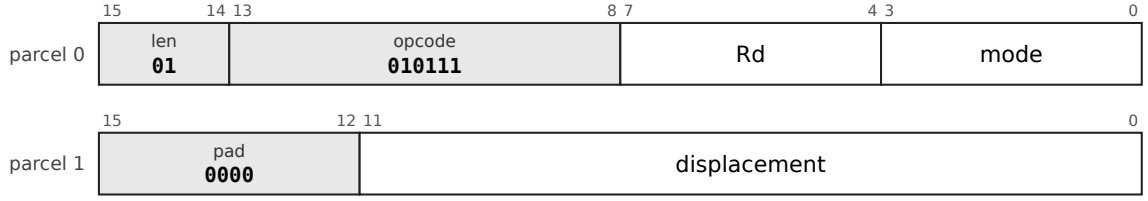
SUB, opcode 010111, two parcels (modes index) (4 bytes)



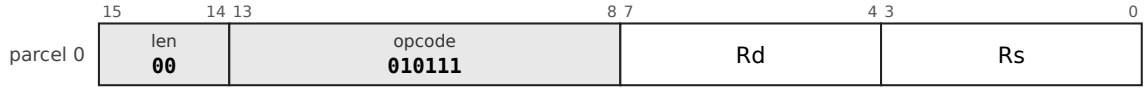
SUB, opcode 010111, two parcels (modes pcrel, imm, abs) (4 bytes)



SUB, opcode 010111, two parcels (modes dp) (4 bytes)



SUB, opcode 010111, one parcel (2 bytes)

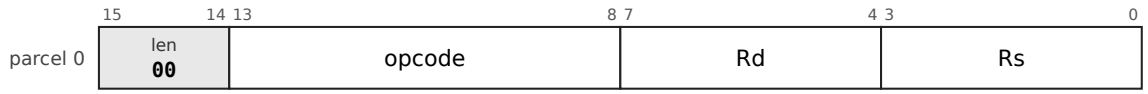


Mnemonics: `SUB`

Syntax

```
SUB rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
SUB	010111 (0x17)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← Rd - rm
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

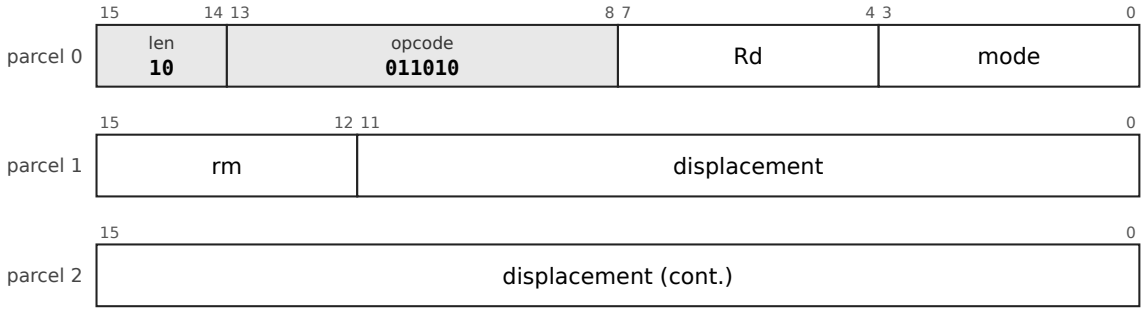
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Page 2 has `SUB #(8*n), SP`.

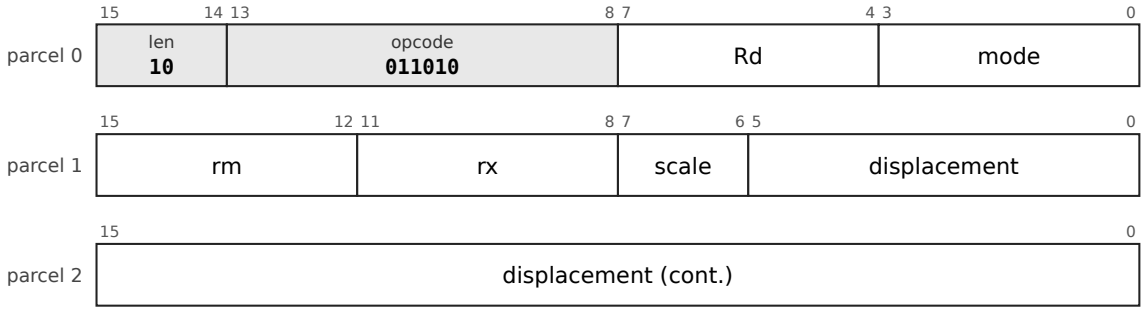
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

CMP: Compare

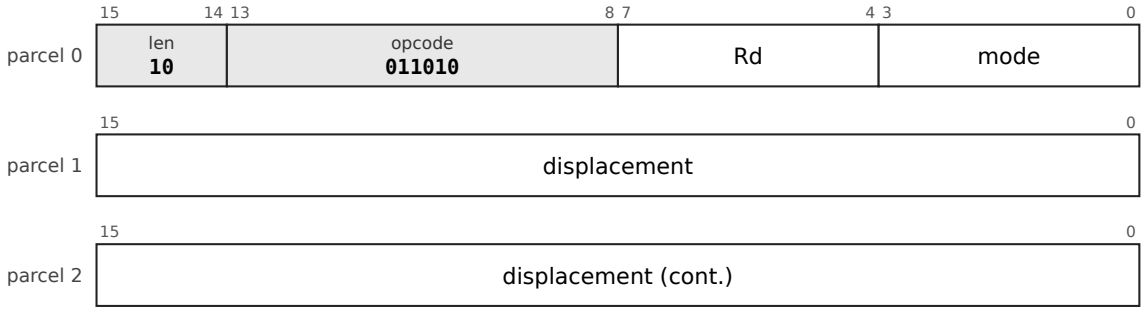
CMP, opcode 011010, three parcels (modes base, postinc, predec) (6 bytes)



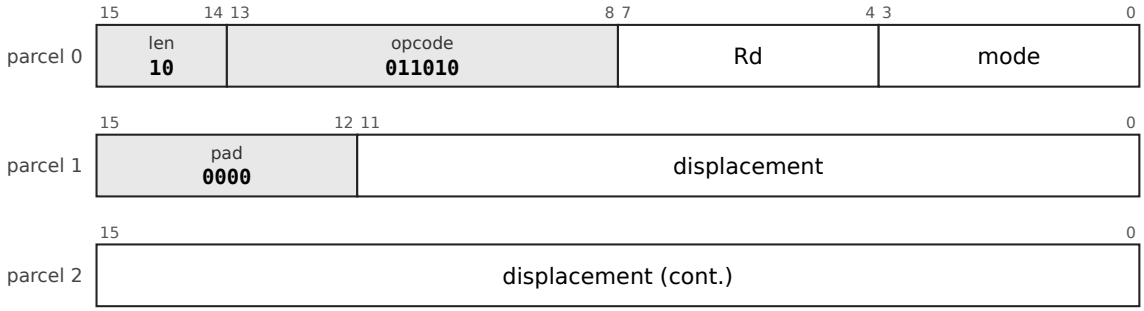
CMP, opcode 011010, three parcels (modes index) (6 bytes)



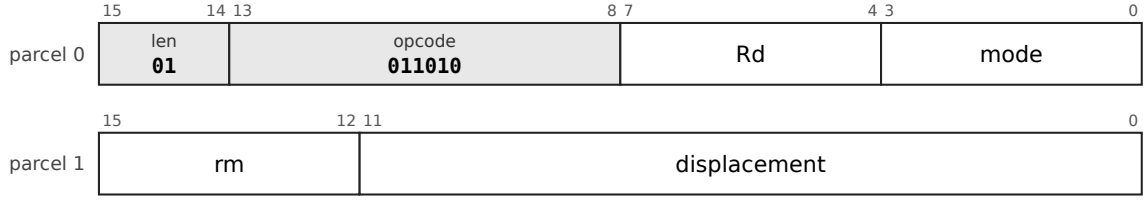
CMP, opcode 011010, three parcels (modes pcrel, imm, abs) (6 bytes)



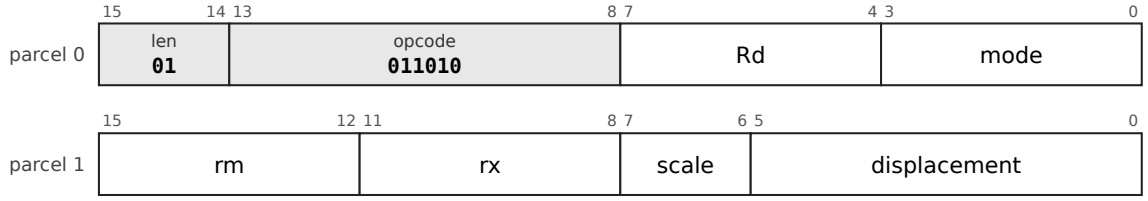
CMP, opcode 011010, three parcels (modes dp) (6 bytes)



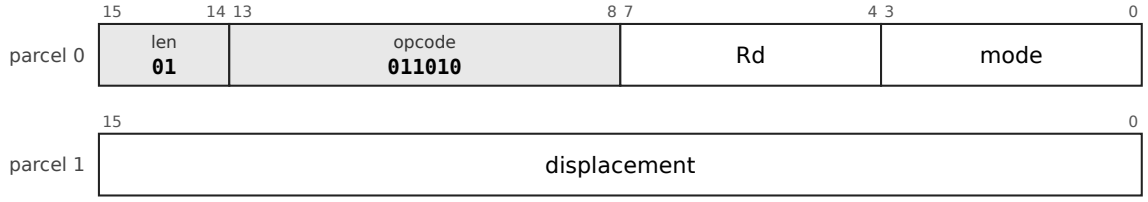
CMP, opcode 011010, two parcels (modes base, postinc, predec) (4 bytes)



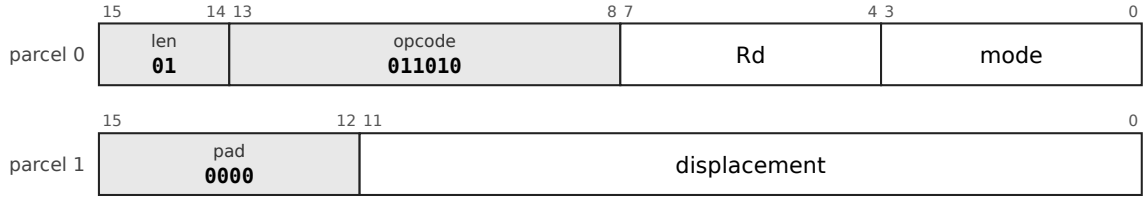
CMP, opcode 011010, two parcels (modes index) (4 bytes)



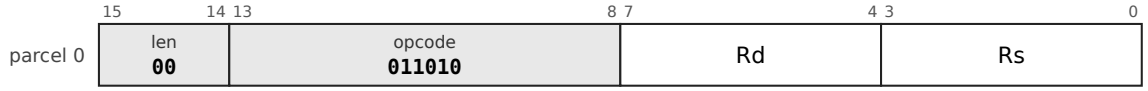
CMP, opcode 011010, two parcels (modes pcrel, imm, abs) (4 bytes)



CMP, opcode 011010, two parcels (modes dp) (4 bytes)



CMP, opcode 011010, one parcel (2 bytes)

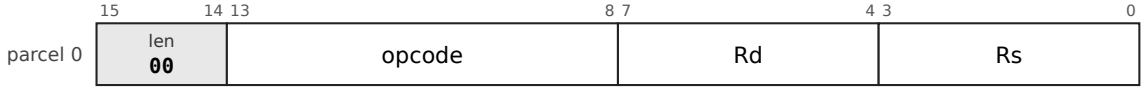


Mnemonics: `CMP`

Syntax

```
CMP Rd, rm        ; register first
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
`CMP`	`011010 (0x1A)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
tmp ← Rd - rm         ; the result is discarded
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

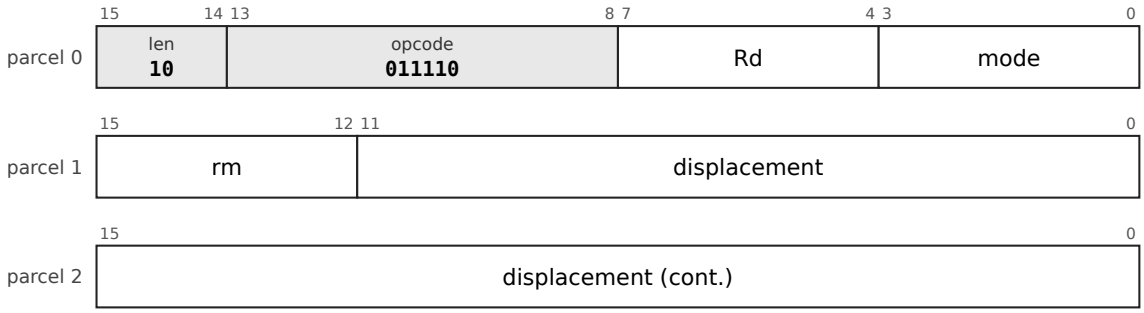
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The operands are in the order register, then the other operand, the reverse of the PDP-11 `CMP src,dst`: `CMP Rd, rm` computes `Rd - rm`, so a compare of a register against an immediate or memory operand does not reverse its branch condition. After `CMP`, `BHIS`/`BCC` mean unsigned `>=` (no borrow) and `BLO`/`BCS` mean a borrow occurred.

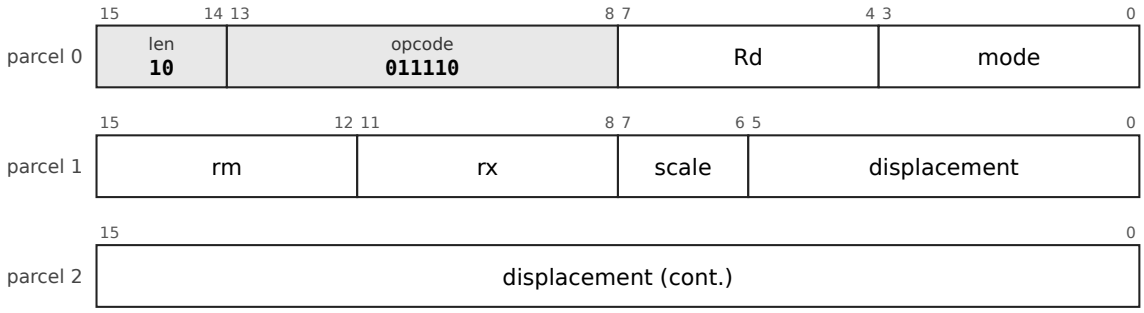
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MUL: Multiply, low 64 bits

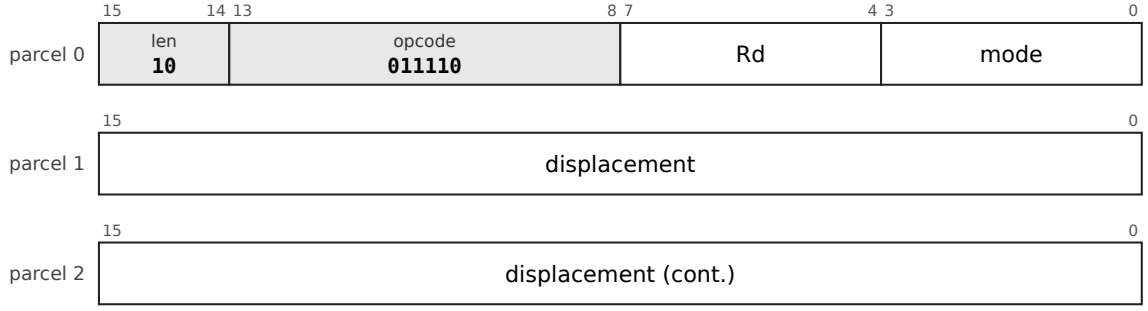
MUL, opcode 011110, three parcels (modes base, postinc, predec) (6 bytes)



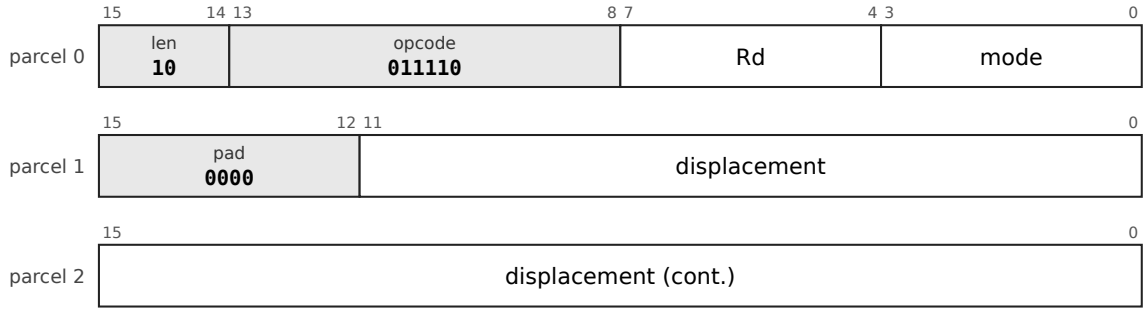
MUL, opcode 011110, three parcels (modes index) (6 bytes)



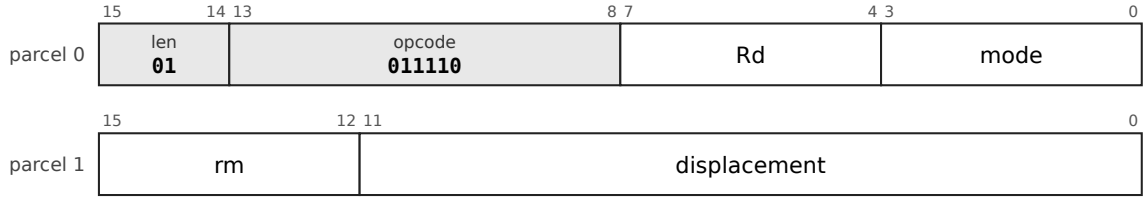
MUL, opcode 011110, three parcels (modes pcrel, imm, abs) (6 bytes)



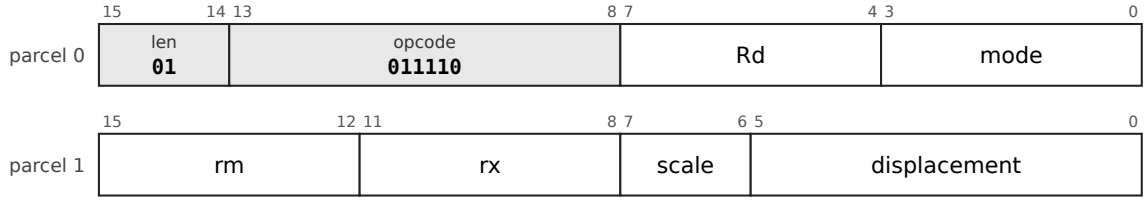
MUL, opcode 011110, three parcels (modes dp) (6 bytes)



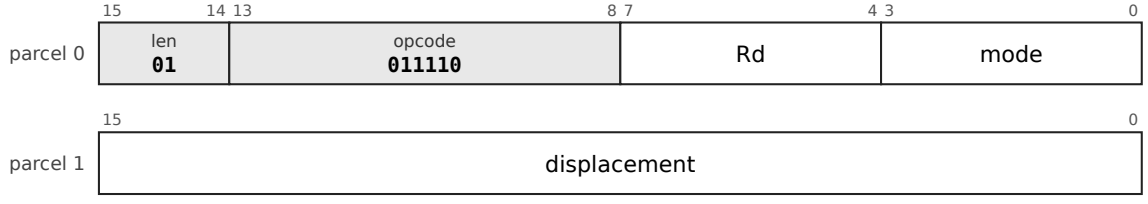
MUL, opcode 011110, two parcels (modes base, postinc, predec) (4 bytes)



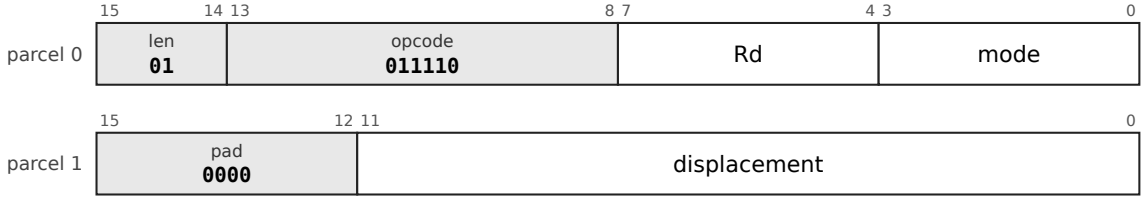
MUL, opcode 011110, two parcels (modes index) (4 bytes)



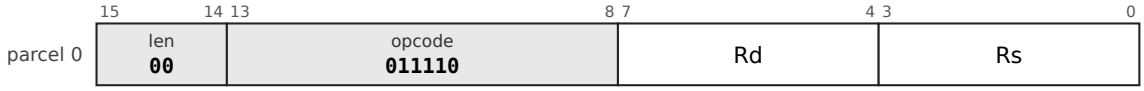
MUL, opcode 011110, two parcels (modes pcrel, imm, abs) (4 bytes)



MUL, opcode 011110, two parcels (modes dp) (4 bytes)



MUL, opcode 011110, one parcel (2 bytes)

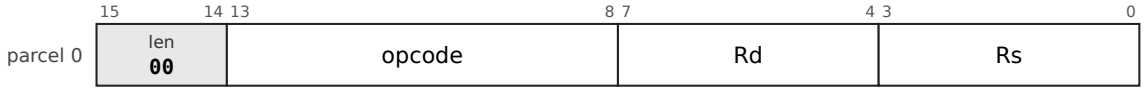


Mnemonics: `MUL` (aliases: `MULU`)

Syntax

```
MUL rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
`MUL`	`011110 (0x1E)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← (Rd × rm)[63:0]        ; sign-agnostic
```

Flags `N` and `Z` from the result, `V` set when the signed 128-bit product does not fit in 64 bits, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

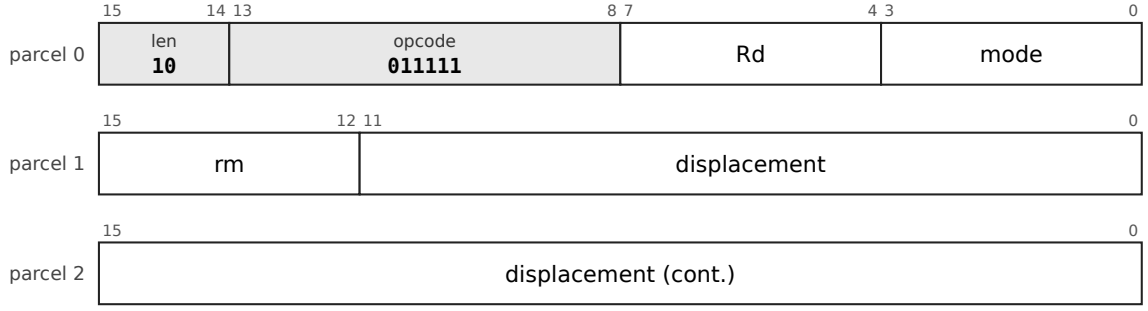
The low 64 bits of a two's-complement product are the same whether the operands are read as signed or unsigned, so there is one low-half multiply. The mnemonic `MULU` is an alias of `MUL`. The high half is `MULH` (signed) and `MULHU` (unsigned).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

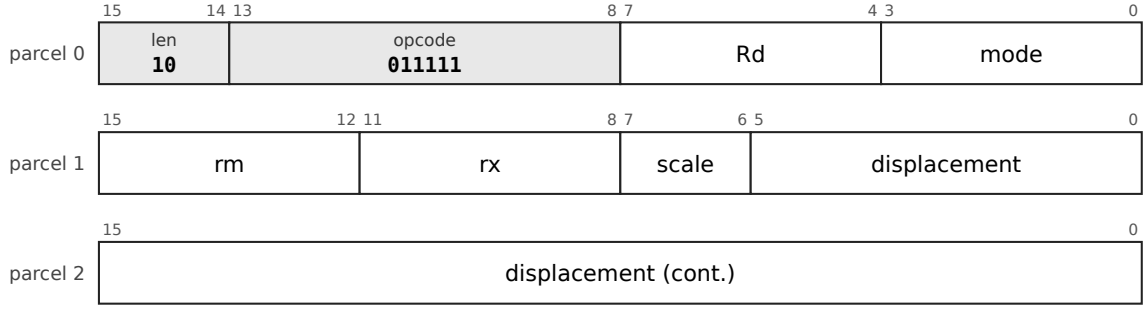
`MULU` is an alias of `MUL`: the assembler writes it as that word.

MULH: Multiply, high 64 bits, signed

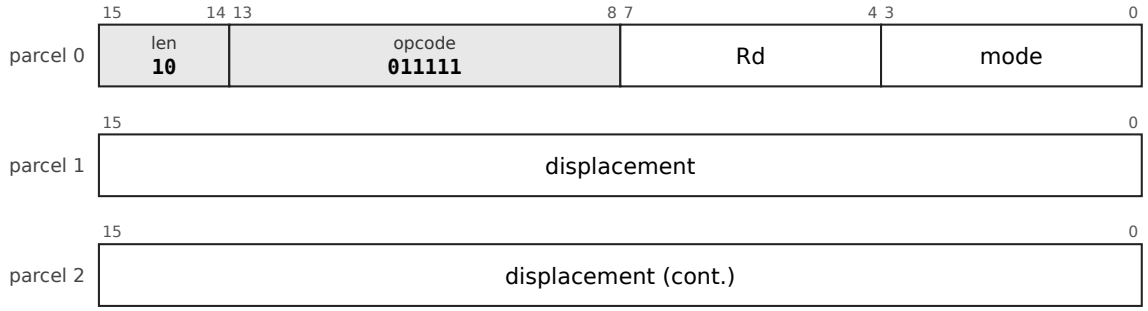
MULH, opcode 011111, three parcels (modes base, postinc, predec) (6 bytes)



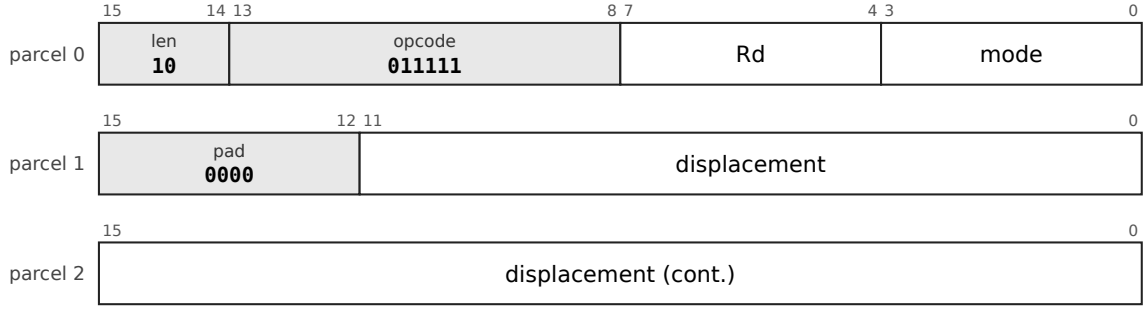
MULH, opcode 011111, three parcels (modes index) (6 bytes)



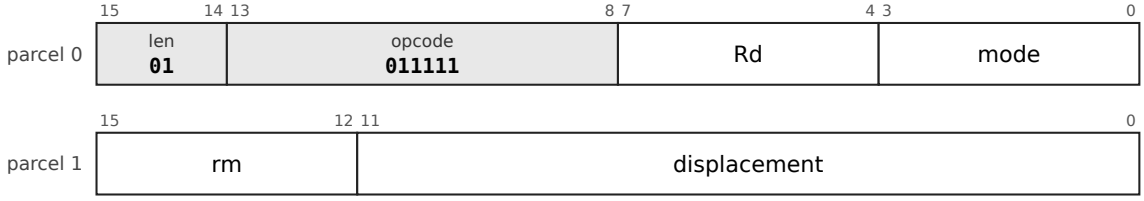
MULH, opcode 011111, three parcels (modes pcrel, imm, abs) (6 bytes)



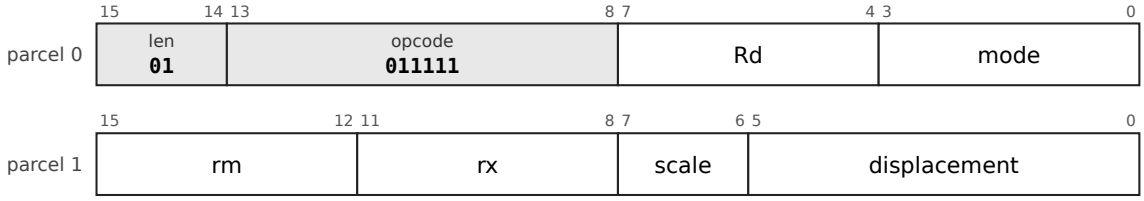
MULH, opcode 011111, three parcels (modes dp) (6 bytes)



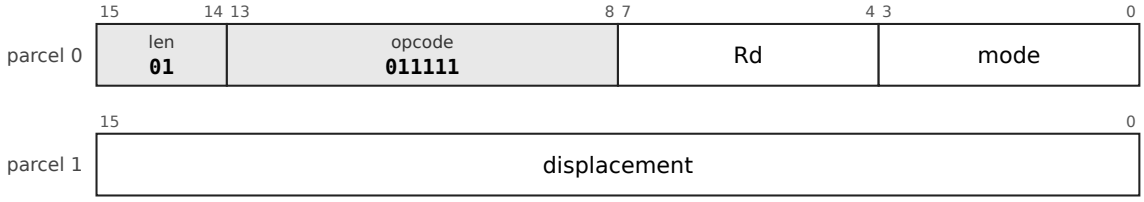
MULH, opcode 011111, two parcels (modes base, postinc, predec) (4 bytes)



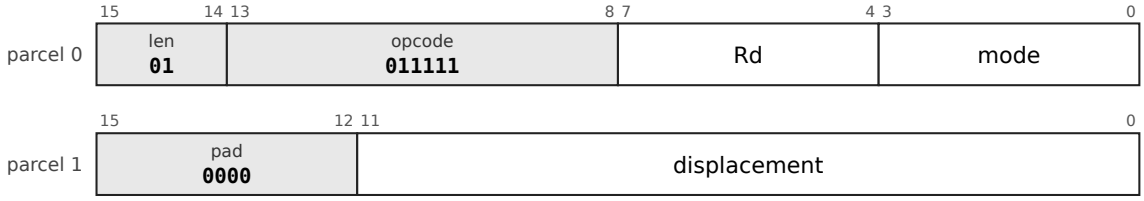
MULH, opcode 011111, two parcels (modes index) (4 bytes)



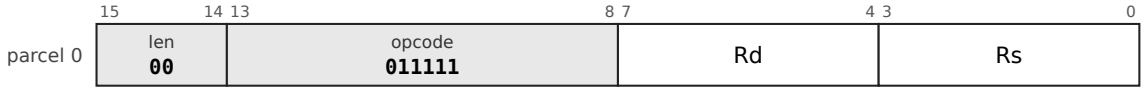
MULH, opcode 011111, two parcels (modes pcrel, imm, abs) (4 bytes)



MULH, opcode 011111, two parcels (modes dp) (4 bytes)



MULH, opcode 011111, one parcel (2 bytes)

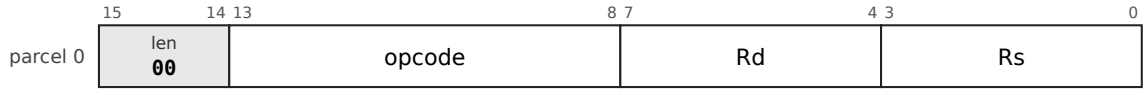


Mnemonics: `MULH`

Syntax

```
MULH rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MULH	011111 (0x1F)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← (sext(Rd) × sext(rm))[127:64]
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

With `MULH`, division by a constant is a multiply by a reciprocal and a shift, which is what compilers emit.

Why it is here. The low 64 bits of a two's-complement product are identical signed or unsigned, so a second multiply (`MULU`) would duplicate `MUL`; what the architecture needs in that place is the *high* half of the product. Without it a 64 by 64 to 128-bit multiply, every bignum multiply, 128-bit fixed point and divide-by-a-constant by multiplying by the reciprocal are impossible. `MULH` is that instruction. It is also why a divide is rare in compiled code: compilers turn division by a constant into `MULH` and a shift.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

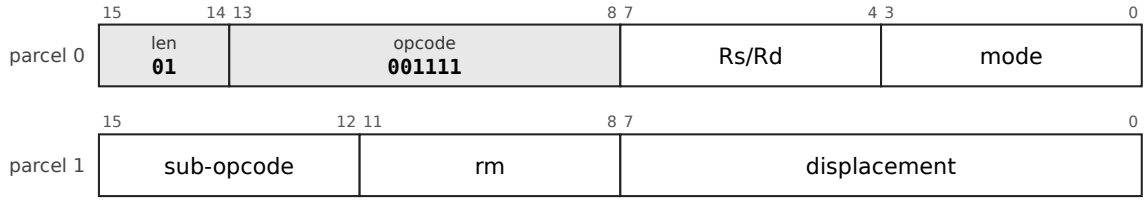
ADDM: ALU operation with a memory destination

Mnemonics: `ADDM`, `SUBM`, `BICM`, `BISM`

Syntax

```
ADDM Rs, rm
SUBM Rs, rm
BICM Rs, rm
BISM Rs, rm
```

Encoding



The chart is the layout `r20-mesc-1`, Memory escape (MESC), two parcels (`len = 01`): Addressing modes (`low`) that use this layout: `base` 0000, `index` 0001, `pcrel` 0010, `dp` 0011, `imm` 0100, `abs` 0101, `postinc` 0110, `predec` 0111. `low` of parcel 0 is an addressing mode; parcel 1 holds the sub-opcode, `rm` and the first 8 bits of the displacement.

Mnemonic	Opcode	Sub-opcode
ADDM	001111 (0x0F)	1000 (0x8)

Mnemonic	Opcode	Sub-opcode
SUBM	001111 (0x0F)	1001 (0x9)
BICM	001111 (0x0F)	1010 (0xA)
BISM	001111 (0x0F)	1011 (0xB)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-mesc-1, r20-mesc-2.

Operation

```
ADDM: M[EA] ← M[EA] + Rs
SUBM: M[EA] ← M[EA] - Rs
BICM: M[EA] ← M[EA] AND NOT Rs
BISM: M[EA] ← M[EA] OR Rs
```

Flags As for ADD, SUB, BIC and BIS.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. Write permission is checked during the read access, so the write cannot fault after the read has succeeded.

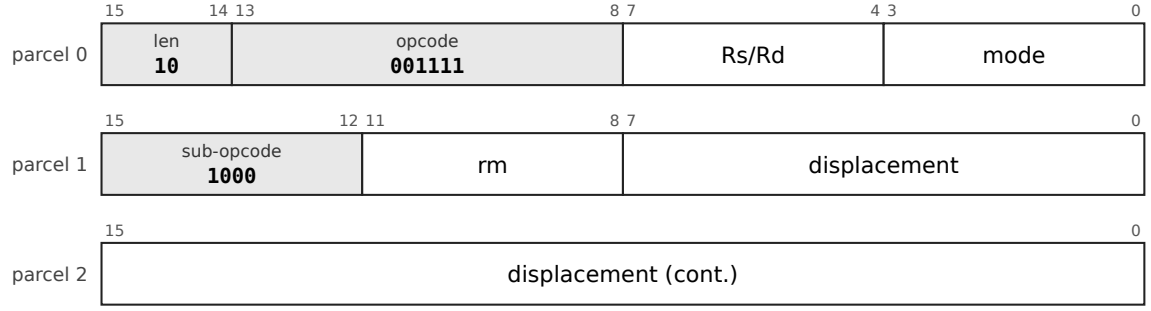
Parcel 0's low nibble is a mode, as in the ordinary memory format, so these are MESC forms with a memory destination (len >= 01). INC, DEC, NEG, COM and CLR on a memory operand are in SOP.

Why it is here. These are the read-modify-write forms with a memory destination. An instruction with a memory destination is at least four bytes anyway, so they live in MESC at no cost; the opcode map follows one rule, that a top-level opcode is for an operation whose common case is register to register. They cover the case where the second operand is a register, for example BISM Rmask, flags(DP). The compiled corpus has 265 sites (1.6 KB), so they earn their place in the escape group but not in the opcode space.

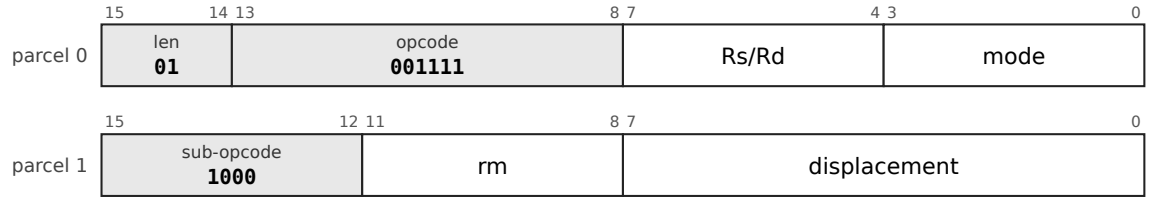
Status specified; **not yet implemented** in the reference executor.

ADDM

ADDM, opcode 001111, three parcels, sub-opcode 1000 (modes base, index, pcrel, dp, imm, a



ADDM, opcode 001111, two parcels, sub-opcode 1000 (modes base, index, pcrel, dp, imm, ab



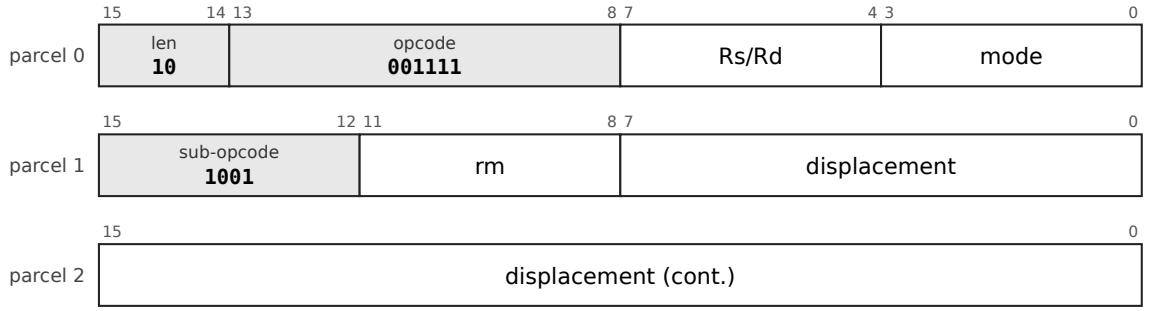

```
ADDM Rs, rm
```

Mnemonic	Opcode	Sub-opcode
ADDM	001111 (0x0F)	1000 (0x8)

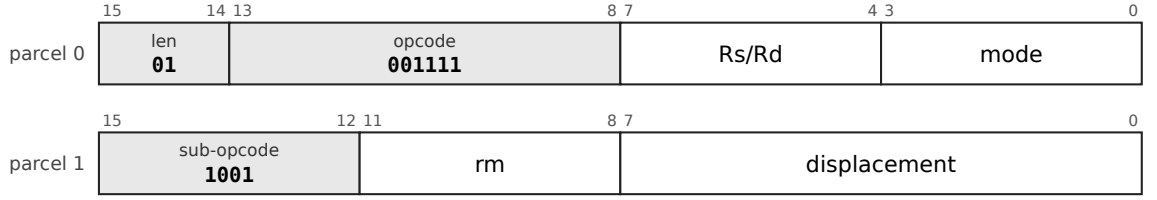
Operation, flags and exceptions as for the entry ADDM above; 2 layout forms, each a chart of its own (above).

SUBM

SUBM, opcode 001111, three parcels, sub-opcode 1001 (modes base, index, pcrel, dp, imm, a



SUBM, opcode 001111, two parcels, sub-opcode 1001 (modes base, index, pcrel, dp, imm, ab



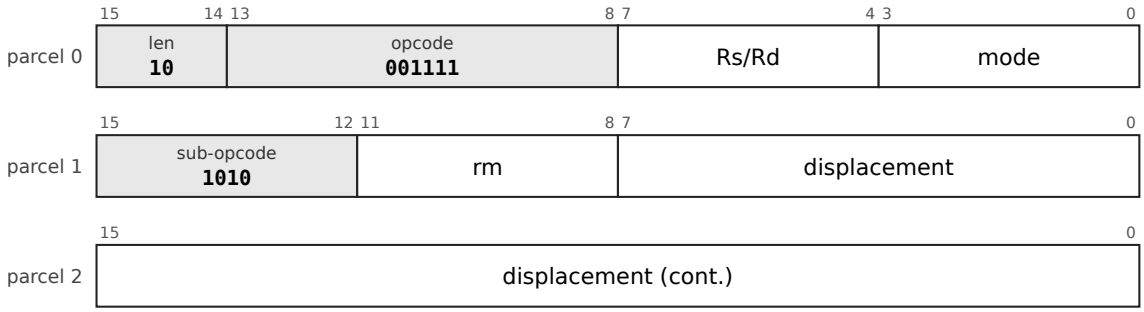

```
SUBM Rs, rm
```

Mnemonic	Opcode	Sub-opcode
SUBM	001111 (0x0F)	1001 (0x9)

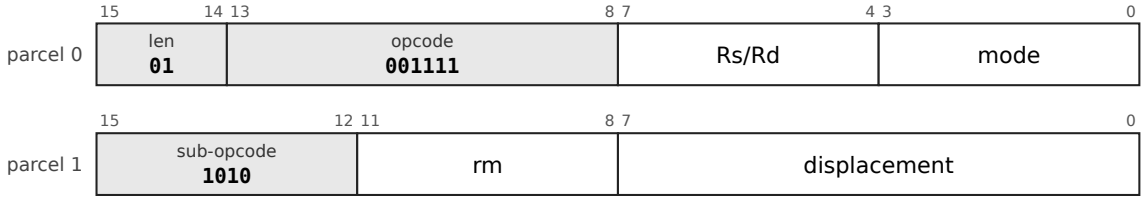
Operation, flags and exceptions as for the entry ADDM above; 2 layout forms, each a chart of its own (above).

BICM

BICM, opcode 001111, three parcels, sub-opcode 1010 (modes base, index, pcrel, dp, imm, a



BICM, opcode 001111, two parcels, sub-opcode 1010 (modes base, index, pcrel, dp, imm, abs



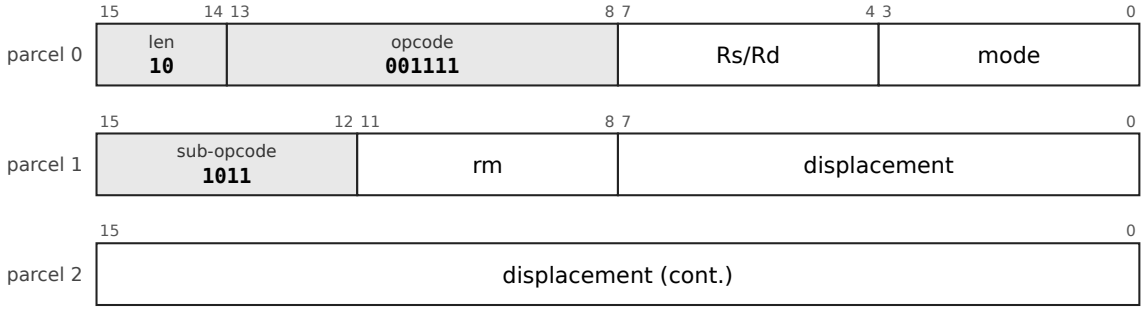

```
BICM Rs, rm
```

Mnemonic	Opcode	Sub-opcode
BICM	001111 (0x0F)	1010 (0xA)

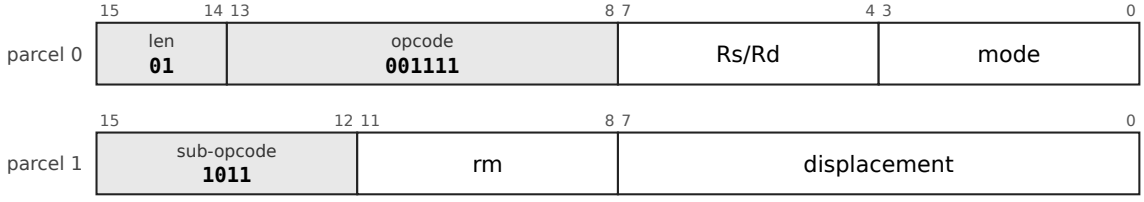
Operation, flags and exceptions as for the entry ADDM above; 2 layout forms, each a chart of its own (above).

BISM

BISM, opcode 001111, three parcels, sub-opcode 1011 (modes base, index, pcrel, dp, imm, a



BISM, opcode 001111, two parcels, sub-opcode 1011 (modes base, index, pcrel, dp, imm, abs



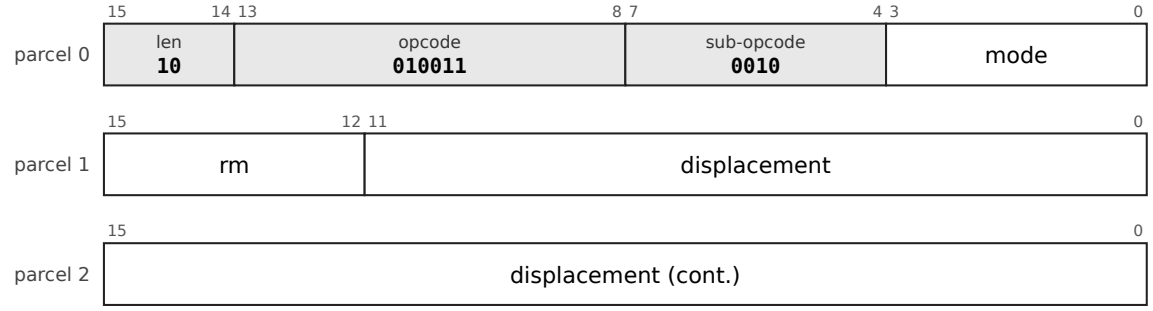

```
BISM Rs, rm
```

Mnemonic	Opcode	Sub-opcode
BISM	001111 (0x0F)	1011 (0xB)

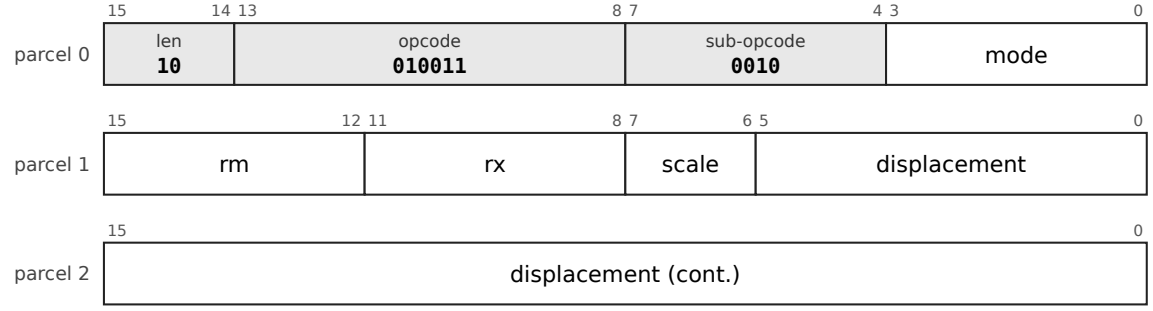
Operation, flags and exceptions as for the entry ADDM above; 2 layout forms, each a chart of its own (above).

INC: Increment

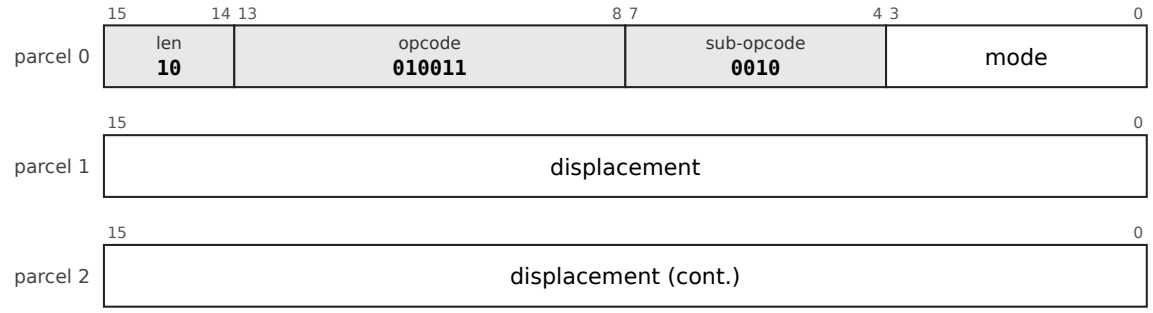
INC, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



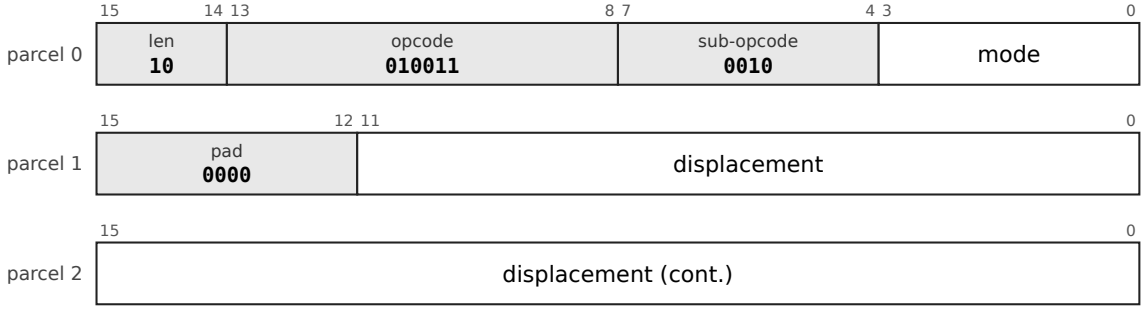
INC, opcode 010011, three parcels (modes index) (6 bytes)



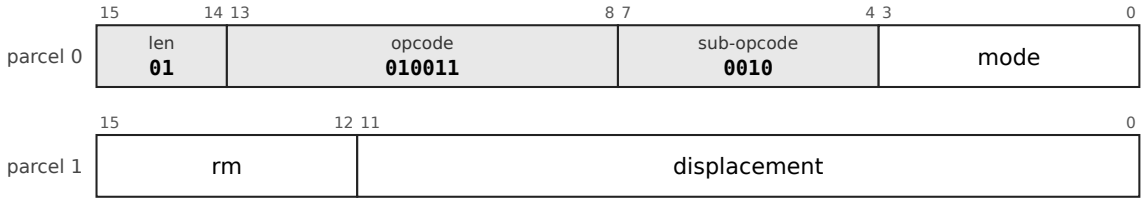
INC, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



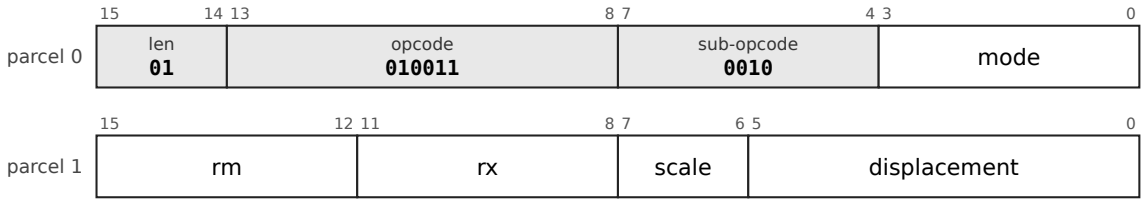
INC, opcode 010011, three parcels (modes dp) (6 bytes)



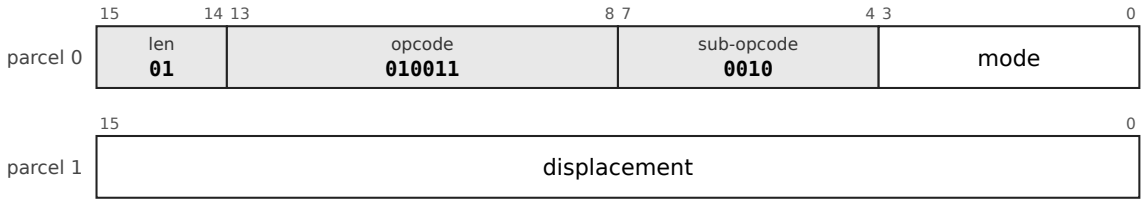
INC, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



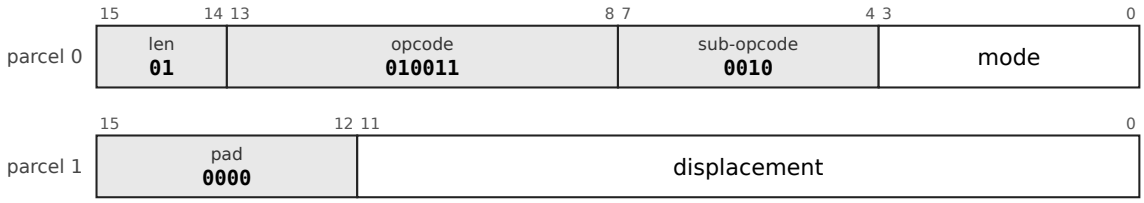
INC, opcode 010011, two parcels (modes index) (4 bytes)



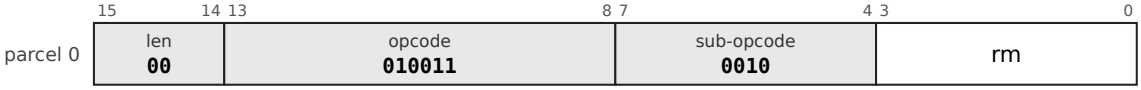
INC, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



INC, opcode 010011, two parcels (modes dp) (4 bytes)



INC, opcode 010011, one parcel (2 bytes)

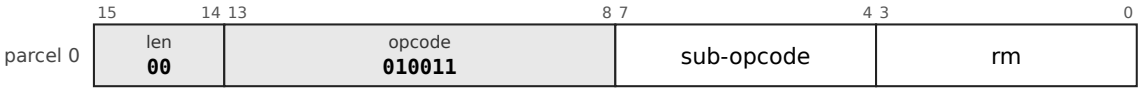


Mnemonics: `INC`

Syntax

```
INC rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`INC`	`010011 (0x13)`	`0010 (0x2)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
rm ← rm + 1
```

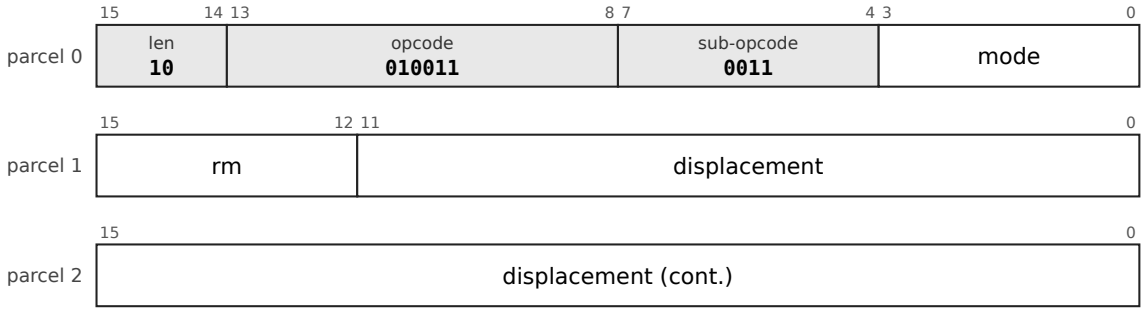
Flags `N`, `Z` and `V` as for `ADD`; `C` **unchanged**.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

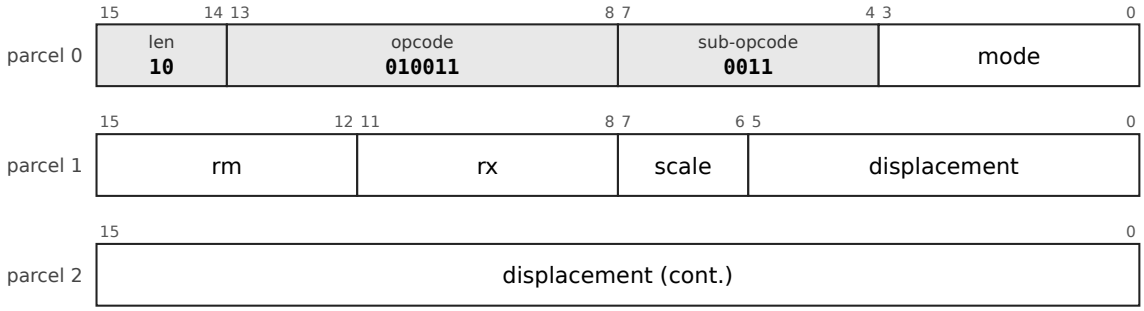
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

DEC: Decrement

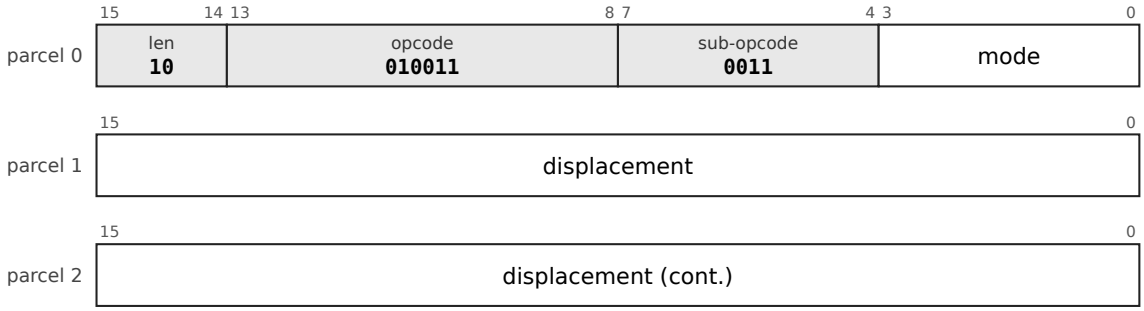
DEC, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



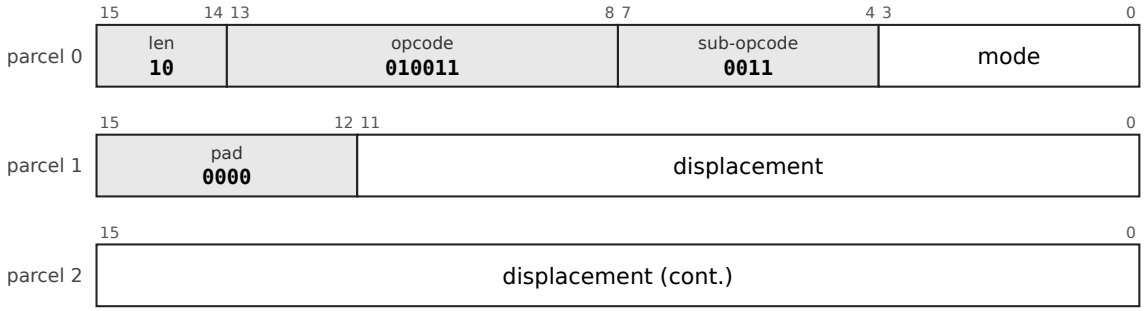
DEC, opcode 010011, three parcels (modes index) (6 bytes)



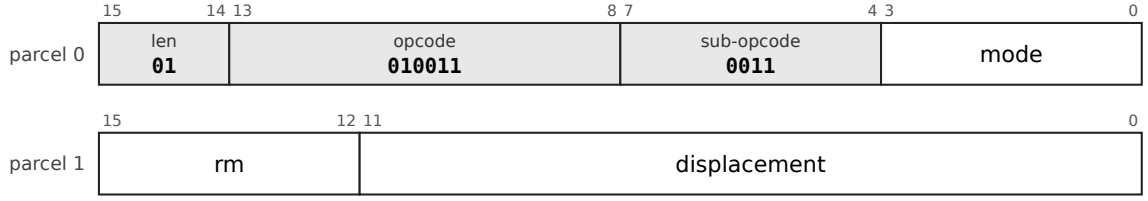
DEC, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



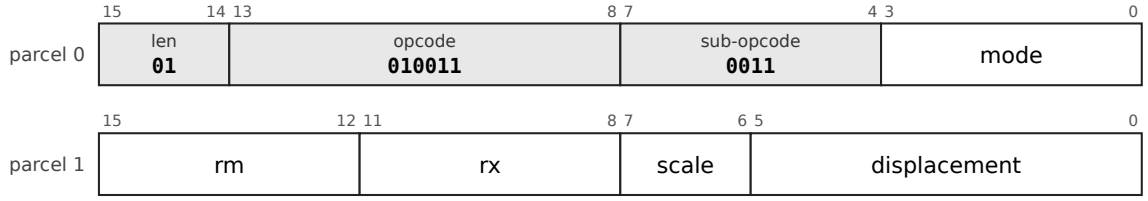
DEC, opcode 010011, three parcels (modes dp) (6 bytes)



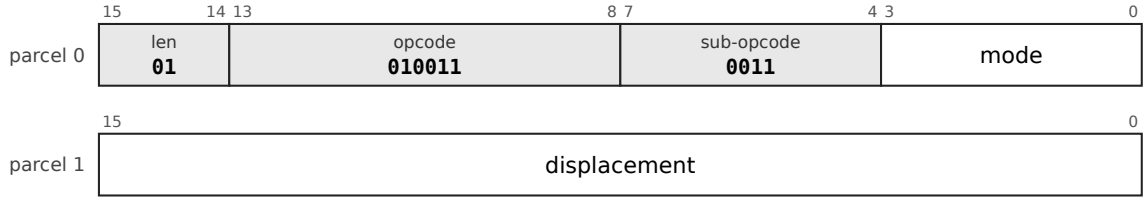
DEC, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



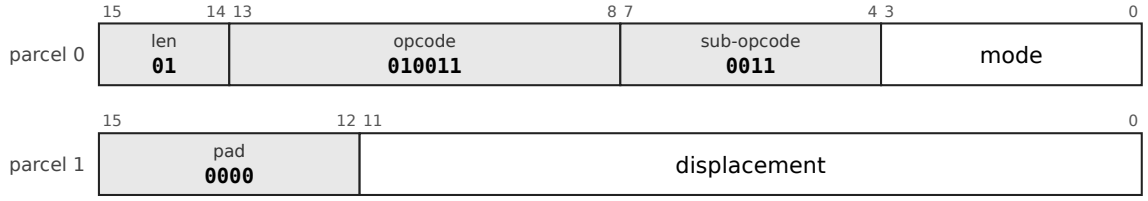
DEC, opcode 010011, two parcels (modes index) (4 bytes)



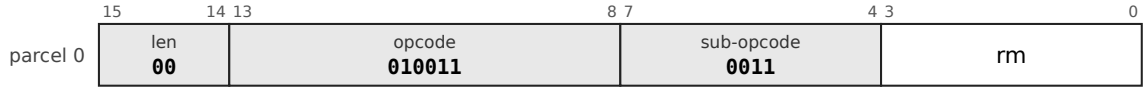
DEC, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



DEC, opcode 010011, two parcels (modes dp) (4 bytes)



DEC, opcode 010011, one parcel (2 bytes)



Mnemonics: `DEC`

Syntax

```
DEC rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
DEC	010011 (0x13)	0011 (0x3)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
rm ← rm - 1
```

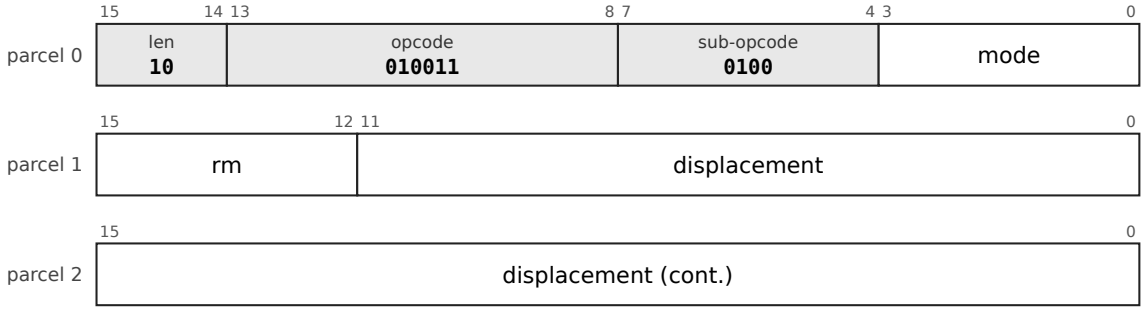
Flags `N`, `Z` and `V` as for `SUB`; `C` **unchanged**.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

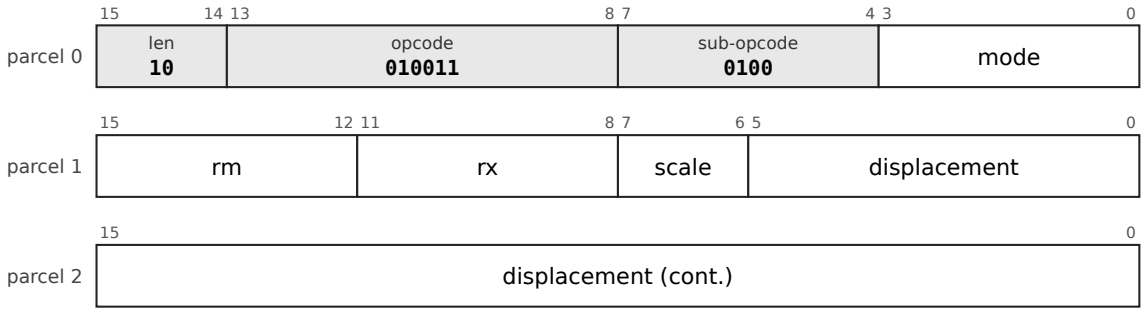
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

NEG: Negate

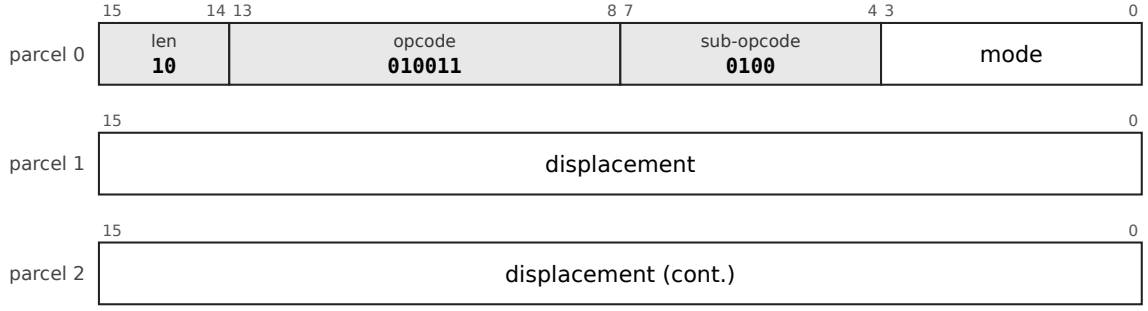
NEG, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



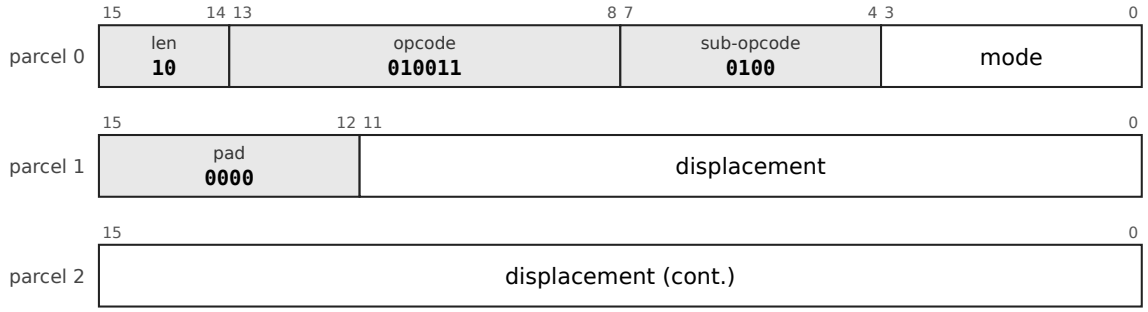
NEG, opcode 010011, three parcels (modes index) (6 bytes)



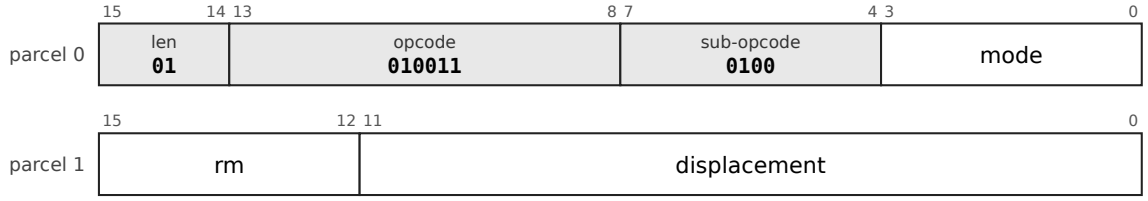
NEG, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



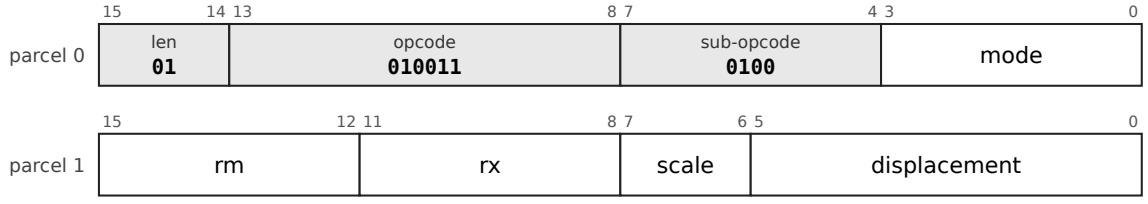
NEG, opcode 010011, three parcels (modes dp) (6 bytes)



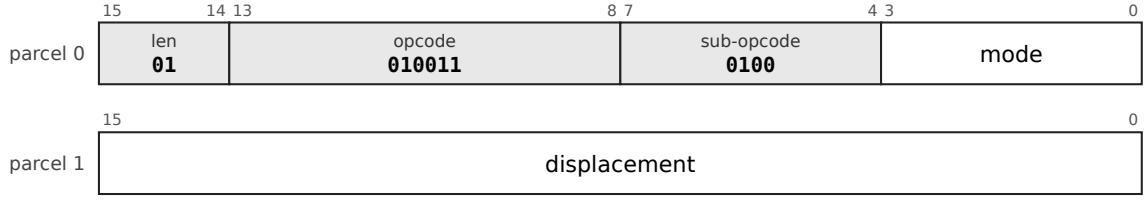
NEG, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



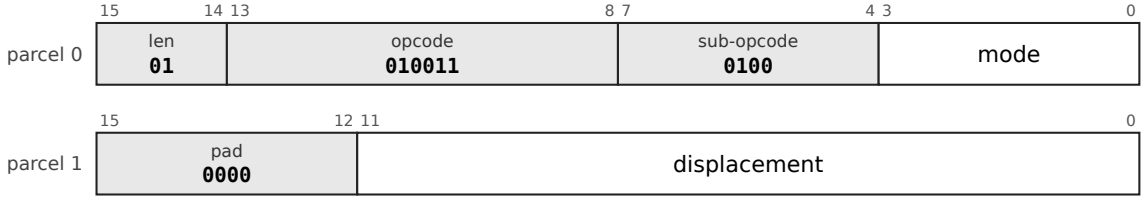
NEG, opcode 010011, two parcels (modes index) (4 bytes)



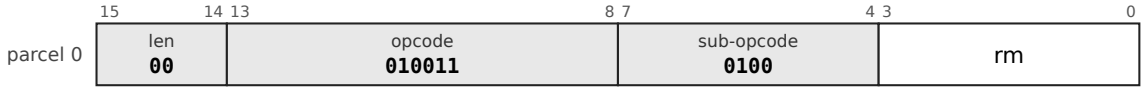
NEG, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



NEG, opcode 010011, two parcels (modes dp) (4 bytes)



NEG, opcode 010011, one parcel (2 bytes)

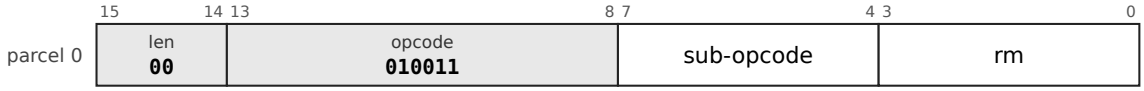


Mnemonics: `NEG`

Syntax

```
NEG rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`NEG`	`010011 (0x13)`	`0100 (0x4)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
rm ← 0 - rm
```

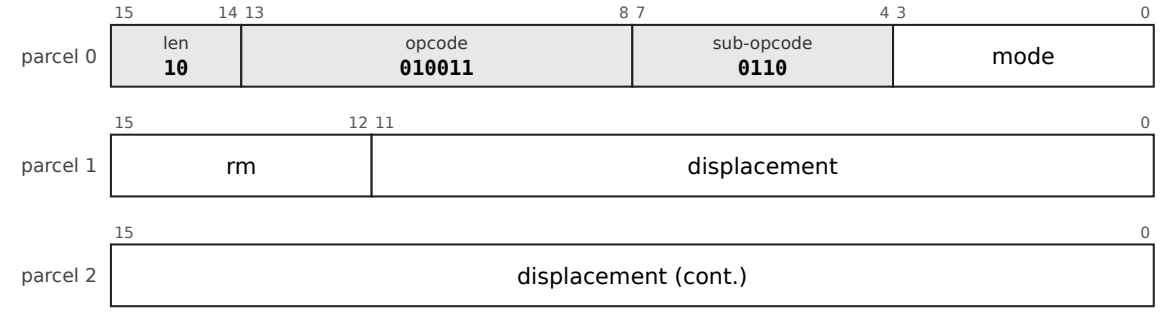
Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger. (`C` = 1 unless the operand was 0.)

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

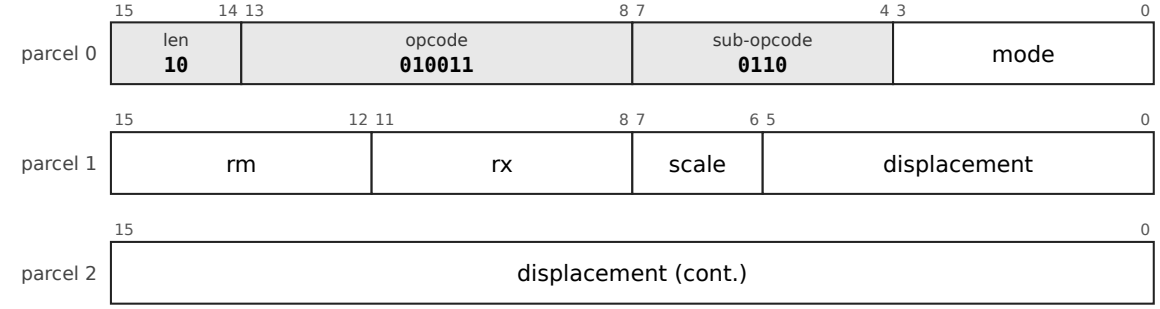
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

ADC: Add carry

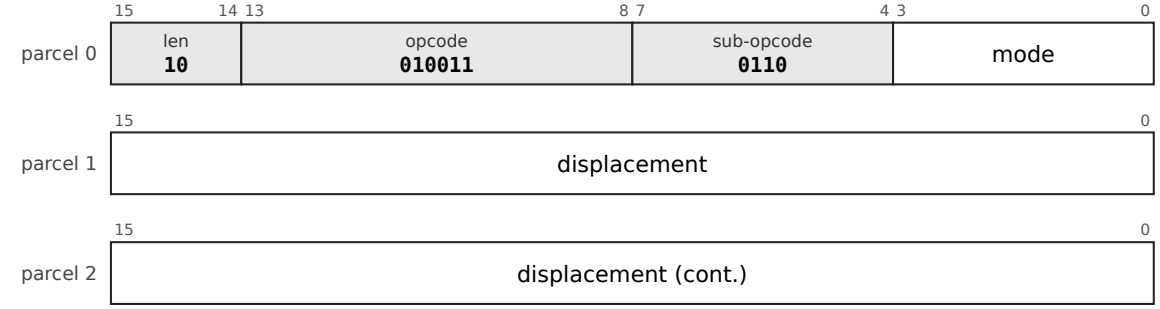
ADC, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



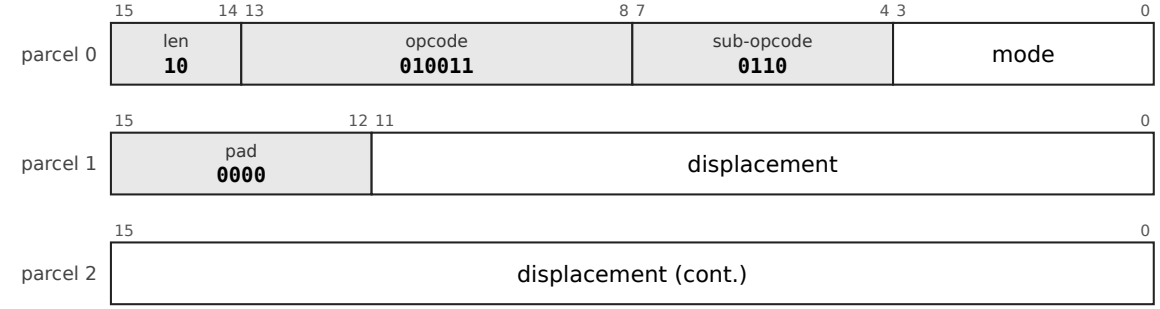
ADC, opcode 010011, three parcels (modes index) (6 bytes)



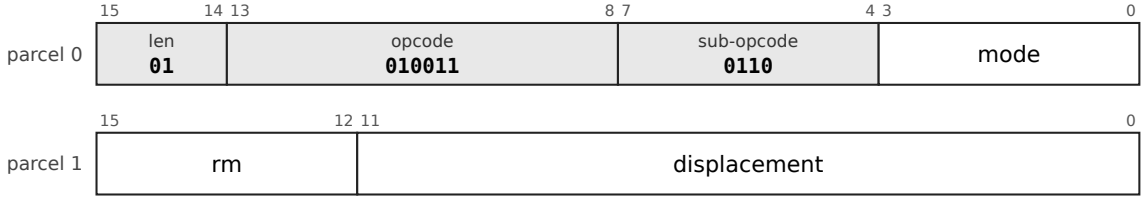
ADC, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



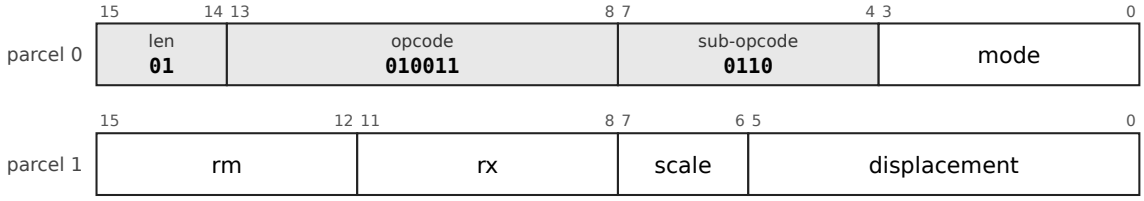
ADC, opcode 010011, three parcels (modes dp) (6 bytes)



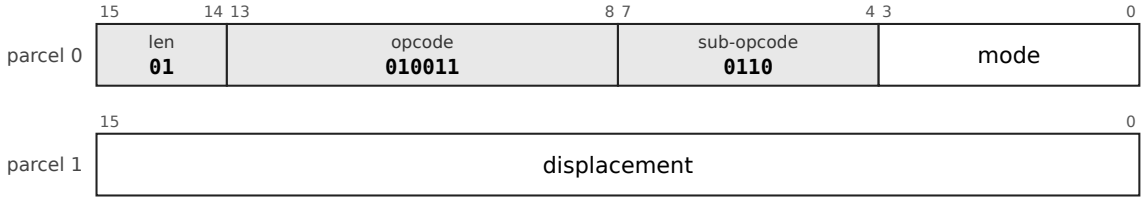
ADC, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



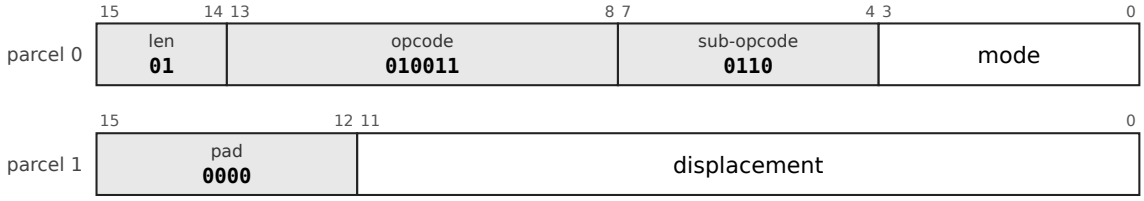
ADC, opcode 010011, two parcels (modes index) (4 bytes)



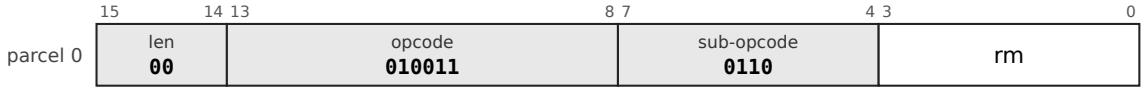
ADC, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



ADC, opcode 010011, two parcels (modes dp) (4 bytes)



ADC, opcode 010011, one parcel (2 bytes)

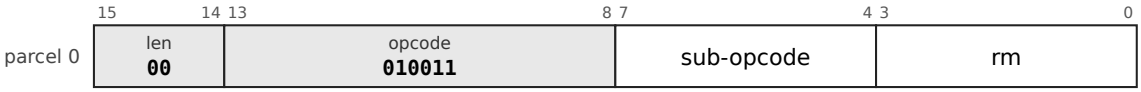


Mnemonics: `ADC`

Syntax

```
ADC rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
ADC	010011 (0x13)	0110 (0x6)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
rm ← rm + C
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a carry out of bit 63.

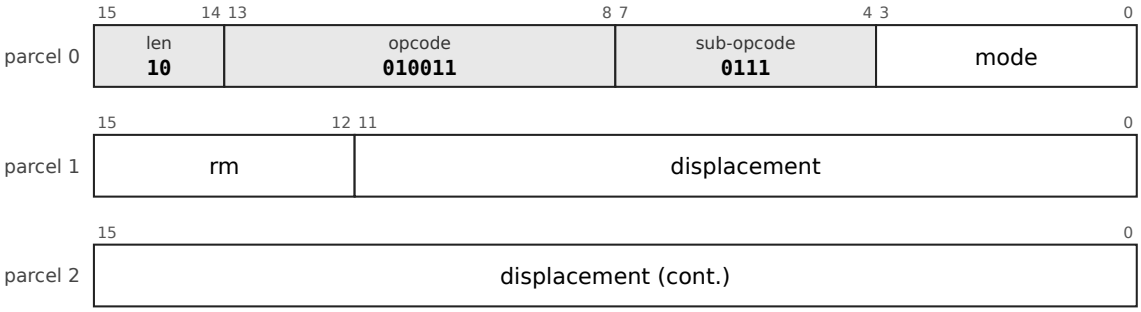
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

With `ADD`, multi-word addition.

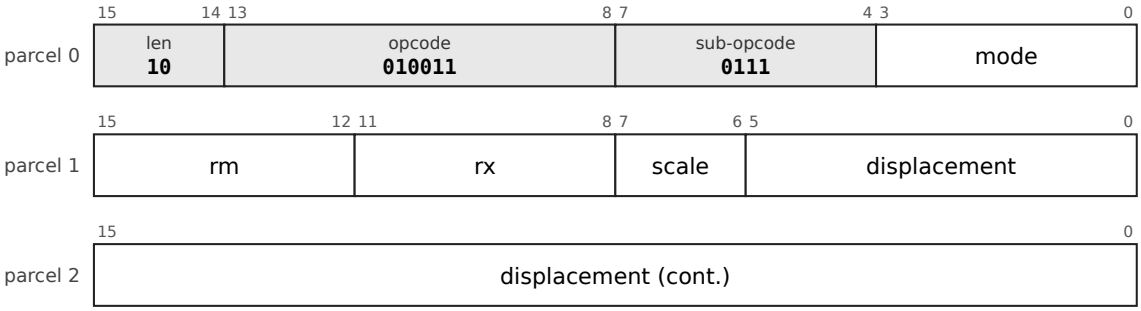
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

SBC: Subtract carry

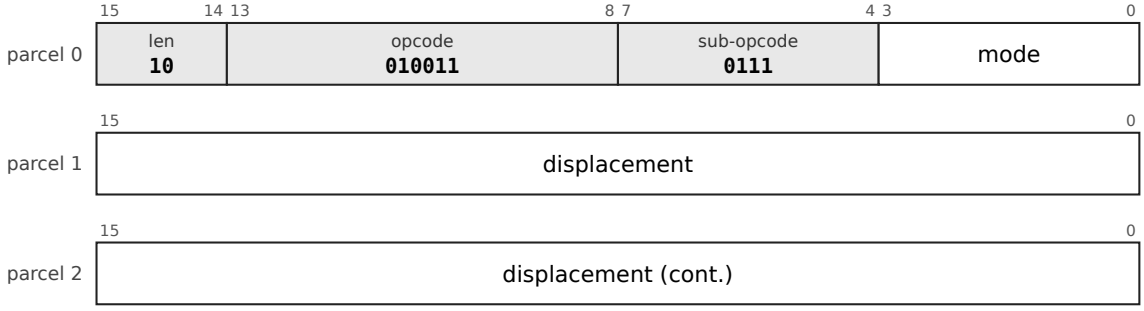
SBC, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



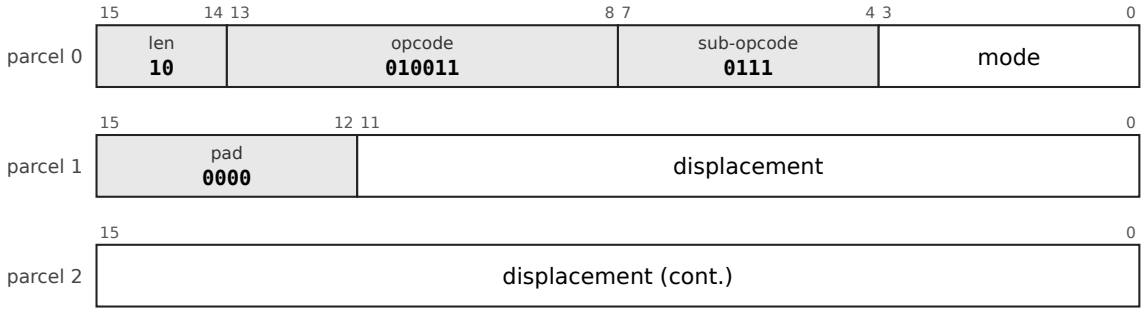
SBC, opcode 010011, three parcels (modes index) (6 bytes)



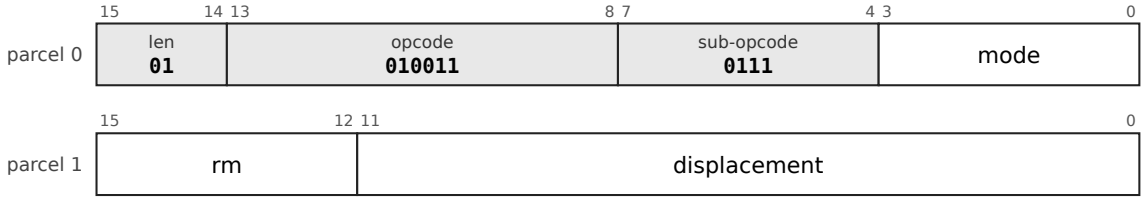
SBC, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



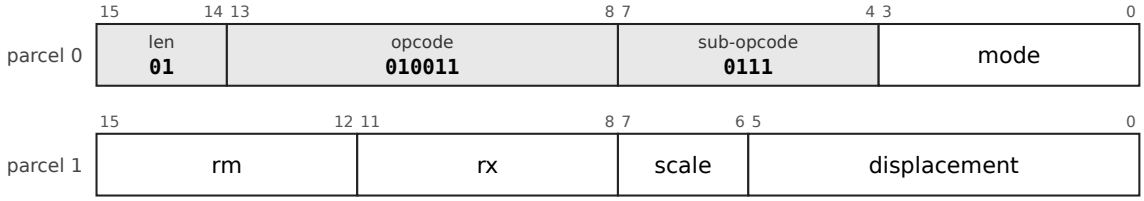
SBC, opcode 010011, three parcels (modes dp) (6 bytes)



SBC, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



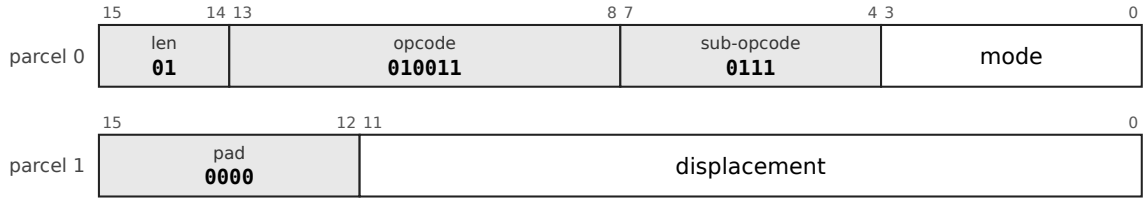
SBC, opcode 010011, two parcels (modes index) (4 bytes)



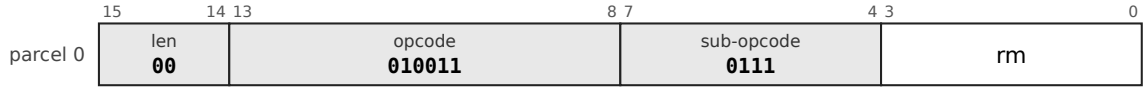
SBC, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



SBC, opcode 010011, two parcels (modes dp) (4 bytes)



SBC, opcode 010011, one parcel (2 bytes)

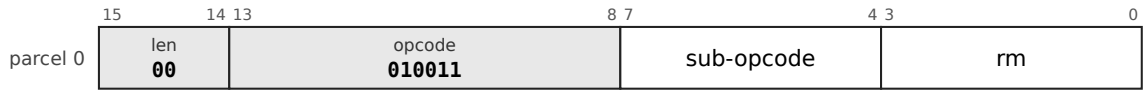


Mnemonics: `SBC`

Syntax

```
SBC rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`SBC`	`010011 (0x13)`	`0111 (0x7)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
rm ← rm - C
```

Flags `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

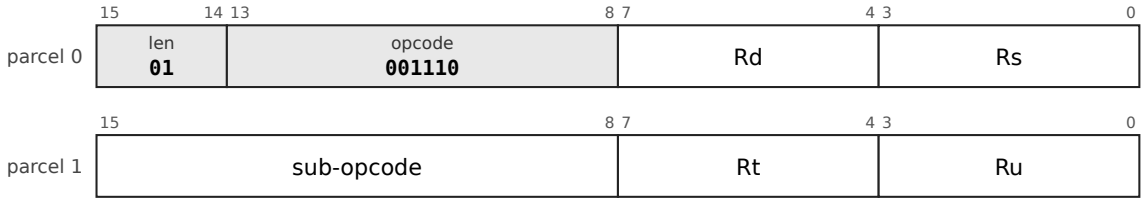
DIV: Divide and remainder, signed and unsigned

Mnemonics: `DIV`, `DIVU`, `MOD`, `MODU`

Syntax

```
DIV Ra, Rb, Rd
DIVU Ra, Rb, Rd
MOD Ra, Rb, Rd
MODU Ra, Rb, Rd
```

Encoding



The chart is the layout `r20-aesc-1`, Arithmetic (AESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
DIV	001110 (0x0E)	00000000 (0x00)
DIVU	001110 (0x0E)	00000001 (0x01)
MOD	001110 (0x0E)	00000010 (0x02)
MODU	001110 (0x0E)	00000011 (0x03)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-aesc-1`, `r20-aesc-2`.

Operation

```
if Rb = 0 then divide-by-zero abort
DIV : Rd ← Ra ÷ Rb  (signed, rounded toward zero; INT_MIN ÷ -1 = INT_MIN)
DIVU: Rd ← Ra ÷ Rb  (unsigned)
MOD : Rd ← Ra mod Rb (signed, the sign of Ra; any value mod -1 = 0)
MODU: Rd ← Ra mod Rb (unsigned)
```

Flags None written.

Exceptions Division by zero aborts with the divide-by-zero vector (cause `DIVZERO`). There is no overflow trap: `INT_MIN / -1` returns `INT_MIN`.

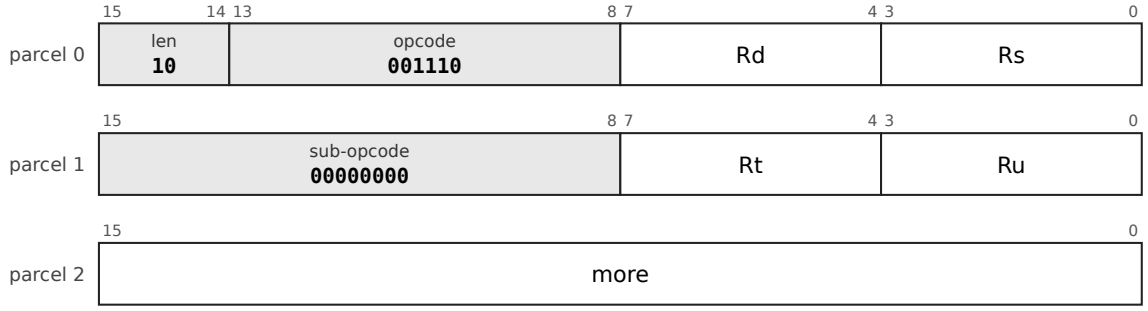
Division by a constant is a `MULH` and a shift, so a real divide is rare; this is why divide has no two-byte form.

Cf. 6.3 (the divide-by-zero vector).

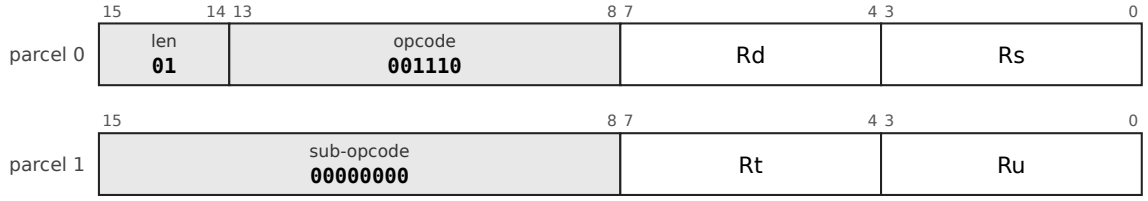
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

DIV

DIV, opcode 001110, three parcels, sub-opcode 00000000 (6 bytes)



DIV, opcode 001110, two parcels, sub-opcode 00000000 (4 bytes)



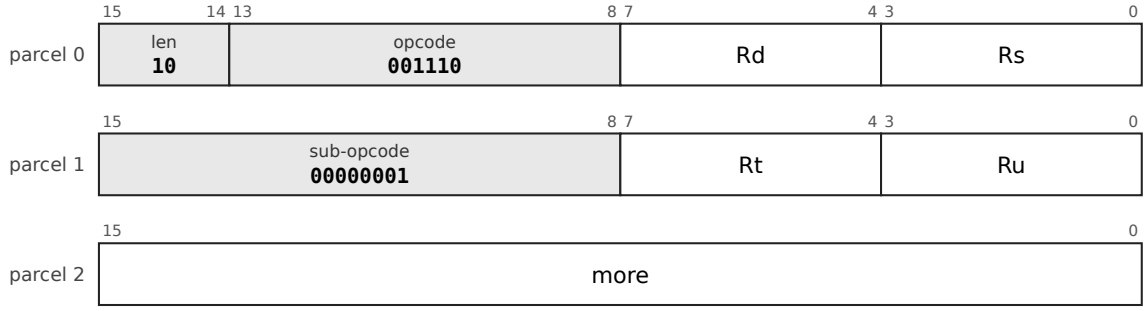

```
DIV Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
DIV	001110 (0x0E)	00000000 (0x00)

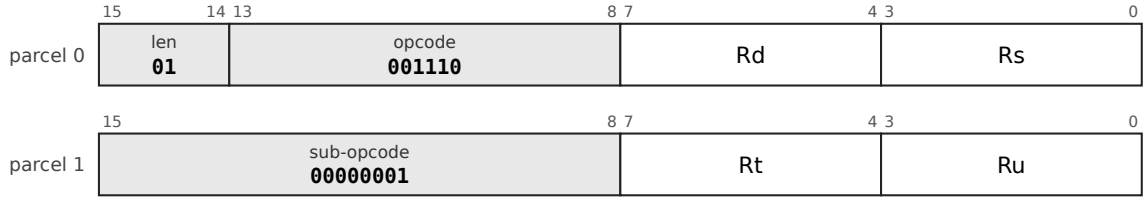
Operation, flags and exceptions as for the entry DIV above; 2 layout forms, each a chart of its own (above).

DIVU

DIVU, opcode 001110, three parcels, sub-opcode 00000001 (6 bytes)



DIVU, opcode 001110, two parcels, sub-opcode 00000001 (4 bytes)



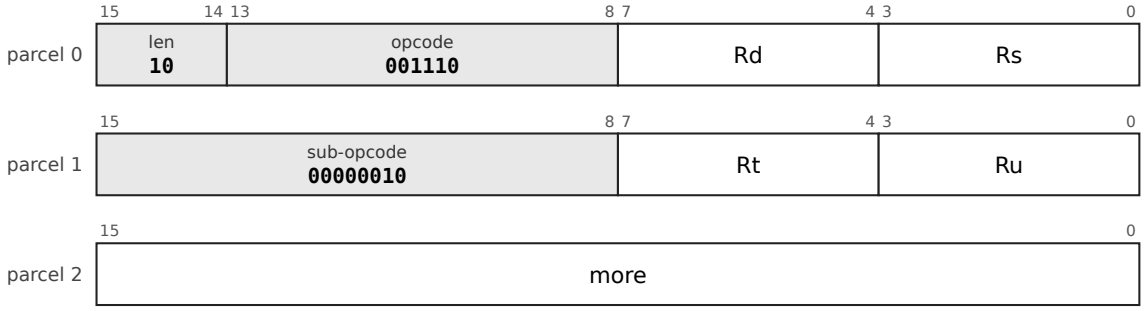

```
DIVU Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
DIVU	001110 (0x0E)	00000001 (0x01)

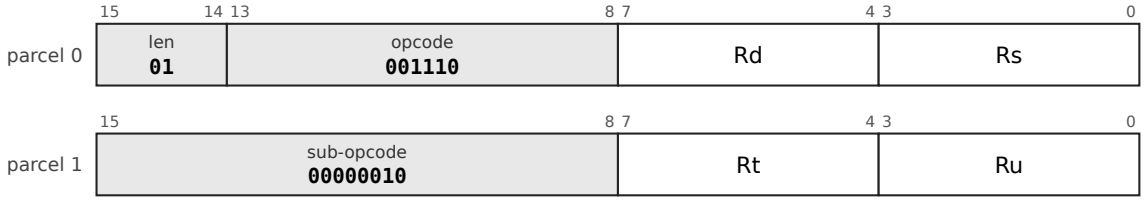
Operation, flags and exceptions as for the entry DIV above; 2 layout forms, each a chart of its own (above).

MOD

MOD, opcode 001110, three parcels, sub-opcode 00000010 (6 bytes)



MOD, opcode 001110, two parcels, sub-opcode 00000010 (4 bytes)



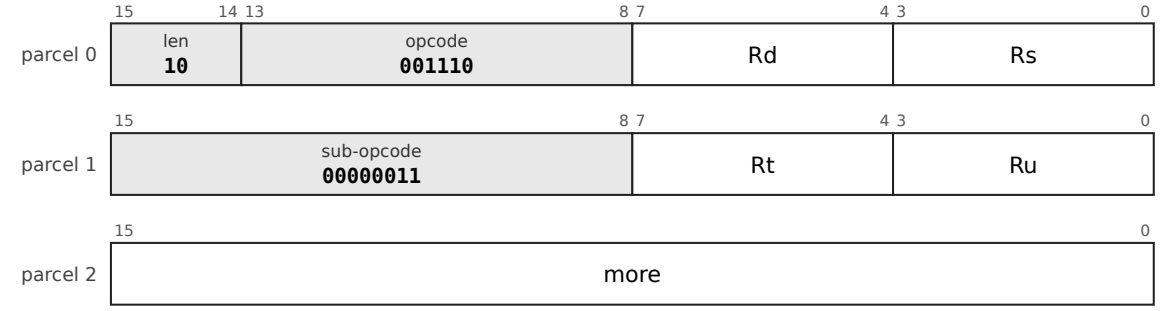

```
MOD Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
MOD	001110 (0x0E)	00000010 (0x02)

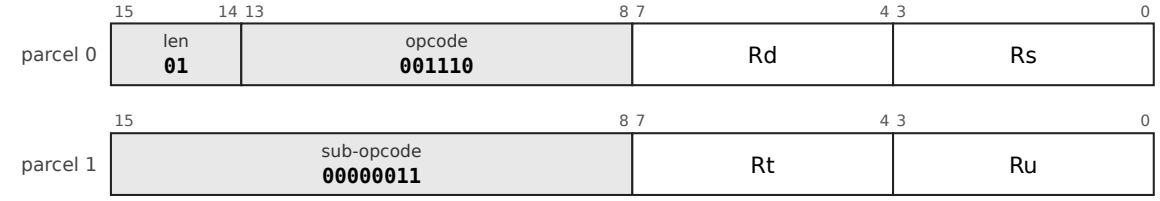
Operation, flags and exceptions as for the entry DIV above; 2 layout forms, each a chart of its own (above).

MODU

MODU, opcode 001110, three parcels, sub-opcode 00000011 (6 bytes)



MODU, opcode 001110, two parcels, sub-opcode 00000011 (4 bytes)



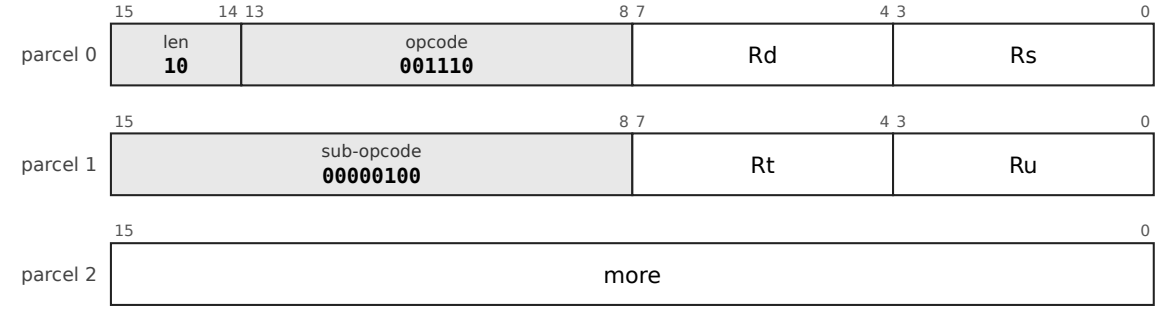

```
MODU Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
MODU	001110 (0x0E)	00000011 (0x03)

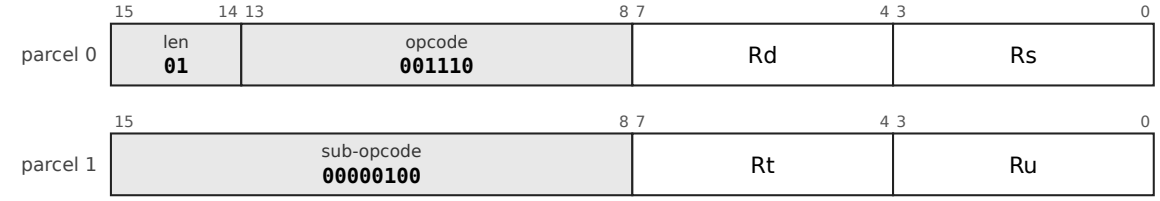
Operation, flags and exceptions as for the entry DIV above; 2 layout forms, each a chart of its own (above).

MULHU: Multiply, high 64 bits, unsigned

MULHU, opcode 001110, three parcels, sub-opcode 00000100 (6 bytes)



MULHU, opcode 001110, two parcels, sub-opcode 00000100 (4 bytes)

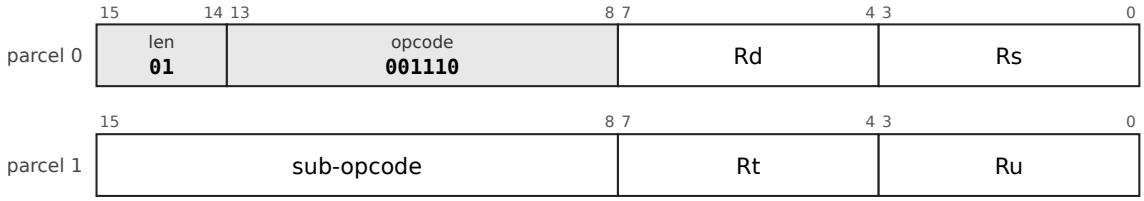


Mnemonics: `MULHU`

Syntax

```
MULHU Ra, Rb, Rd
```

Encoding



The chart is the layout `r20-aesc-1`, Arithmetic (AESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
MULHU	001110 (0x0E)	00000100 (0x04)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-aesc-1`, `r20-aesc-2`.

Operation

```
Rd ← (Ra × Rb)[127:64]    ; unsigned
```

Flags None written.

Exceptions None.

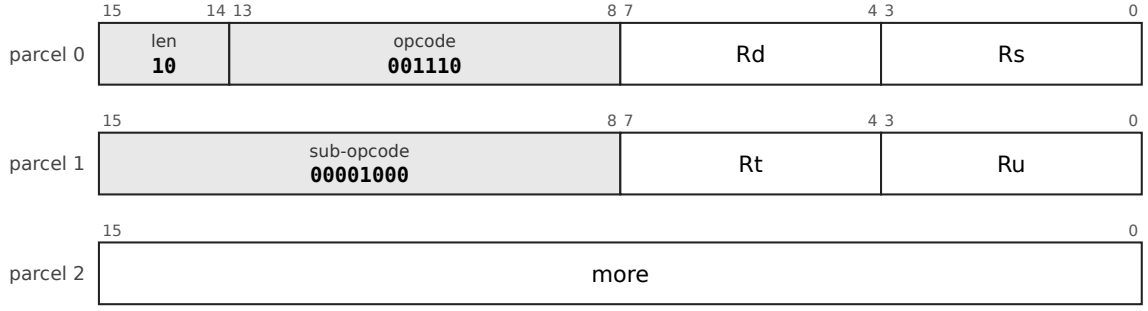
MULHSU (signed by unsigned) is reserved and not committed.

Why it is here. The unsigned partner of MULH, for the same reason (the high half of the product). It is in AESC, where arithmetic belongs. MULHSU, signed times unsigned for a bignum's top limb, is reserved because it can be built from MULH with one correction.

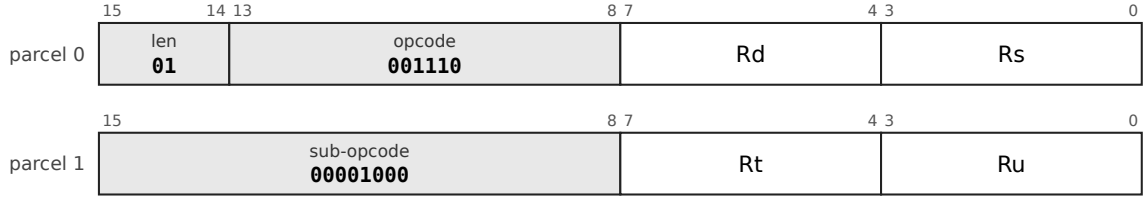
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

ABS: Absolute value (assembler alias)

ABS, opcode 001110, three parcels, sub-opcode 00001000 (6 bytes)



ABS, opcode 001110, two parcels, sub-opcode 00001000 (4 bytes)



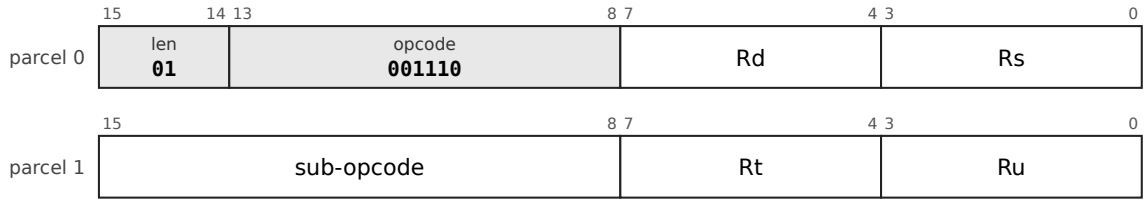
Mnemonics: `ABS`

Alias *(D)* `ABS` is an assembler alias, not a native instruction (the author's decision, errata 98). Meaning: `Rd <- |Rs|`, with the flags below, which the alias must keep or the decision that defines it must change. Expansion: open *(P)*; the flag-driven conditional select (`CMOV`, `CSEL`) is on hold and has no form in the records (ISA section 12 item 15); the only select the records define is the branch-free `MASKEQZ`/`MASKNEZ` (errata 85), which reads a register and no flag. A candidate over it, not decided: compare the operands, `SET` the condition into a scratch register, then `MASKEQZ`, `MASKNEZ` and `BIS` (the constant-time select `d = c ? a : b` of the `MASKEQZ` entry) for the minimum and maximum, and the same around the negated value for `ABS`; it needs a scratch register and leaves `C` as the comparison set it, which the native word does not do, so the flags would change. Which expansion, and whether the select the author meant is this one, is the author's question. The present assembler still encodes the word as the arithmetic-escape sub-opcode of the chart below.

Syntax

```
ABS Rs, Rd
```

Encoding (the present encoding of the build, as the grammar gives it; not the design)



The chart is the layout `r20-aesc-1`, Arithmetic (AESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
ABS	001110 (0x0E)	00001000 (0x08)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-aesc-1`, `r20-aesc-2`.

Operation

```
Rd ← |Rs|
```

Flags N and Z from the result, V set if Rs is the most negative value (the result is then still negative), C unchanged.

Exceptions None.

Why it is here. Integer absolute value is the counterpart of FABS. It was added as a native sub-opcode and the author then decided (2026-11-10) that it is an assembler alias over the conditional select, which is not yet defined. The cost of the native word was about 150 to 250 lookup tables, no new operation by the folding rule (integer against float and signed against unsigned are qualifiers), and the compiler's work of recognising `x < 0 ? -x : x`, which is the real cost. The compiled corpus has 18 uses of `abs` and 5 of its unsigned form, under 0.1% of code, so the case is clarity and the idiom more than size.

Status by decision an assembler alias (errata 98); the present assembler, simulator, QEMU and LLVM still encode and execute the word as the arithmetic-escape sub-opcode shown above, so the word is implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MINMAX: Minimum and maximum, signed and unsigned (assembler aliases)

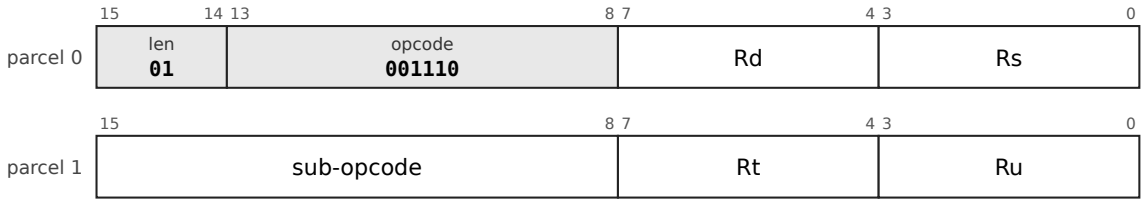
Mnemonics: MIN, MINU, MAX, MAXU

Alias *(D)* MIN, MINU, MAX and MAXU are assembler aliases, not native instructions (the author's decision, errata 98). Meaning: the smaller or the larger of Ra and Rb, signed for MIN and MAX and unsigned for MINU and MAXU, with the flags below, which an alias must keep or the decision that defines it must change. Expansion: open *(P)*; the flag-driven conditional select (CMOV, CSEL) is on hold and has no form in the records (ISA section 12 item 15); the only select the records define is the branch-free MASKEQZ/MASKNEZ (errata 85), which reads a register and no flag. A candidate over it, not decided: compare the operands, SET the condition into a scratch register, then MASKEQZ, MASKNEZ and BIS (the constant-time select `d = c ? a : b` of the MASKEQZ entry) for the minimum and maximum, and the same around the negated value for ABS; it needs a scratch register and leaves C as the comparison set it, which the native word does not do, so the flags would change. Which expansion, and whether the select the author meant is this one, is the author's question. The present assembler still encodes each word as an arithmetic-escape sub-opcode, as the chart below shows.

Syntax

```
MIN Ra, Rb, Rd
MINU Ra, Rb, Rd
MAX Ra, Rb, Rd
MAXU Ra, Rb, Rd
```

Encoding (the present encoding of the build, as the grammar gives it; not the design)



The chart is the layout `r20-aesc-1`, Arithmetic (AESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
MIN	001110 (0x0E)	00010000 (0x10)
MINU	001110 (0x0E)	00010001 (0x11)
MAX	001110 (0x0E)	00010010 (0x12)
MAXU	001110 (0x0E)	00010011 (0x13)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-aesc-1`, `r20-aesc-2`.

Operation

```
MIN : Rd ← smaller of Ra, Rb (signed)        MINU: unsigned
MAX : Rd ← larger of Ra, Rb (signed)         MAXU: unsigned
```

Flags N and Z from the result, V = 0, C unchanged.

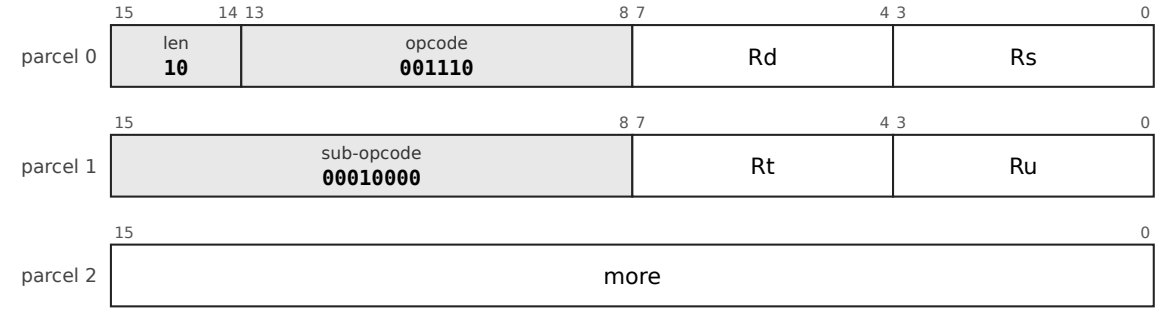
Exceptions None.

Why it is here. The integer counterparts of FMIN and FMAX, provided with ABS; the author decided (2026-11-10) that they are assembler aliases over the conditional select, which is not yet defined. By the folding rule they add no operation, and the native hardware reused the subtract-and-compare that CMP already does plus a 64-bit select. The corpus has 171 uses of MIN and 52 of MAX across 95 files, under 0.1% of code in all. A general conditional select is the alternative; it is open (9.21) for want of a speed model.

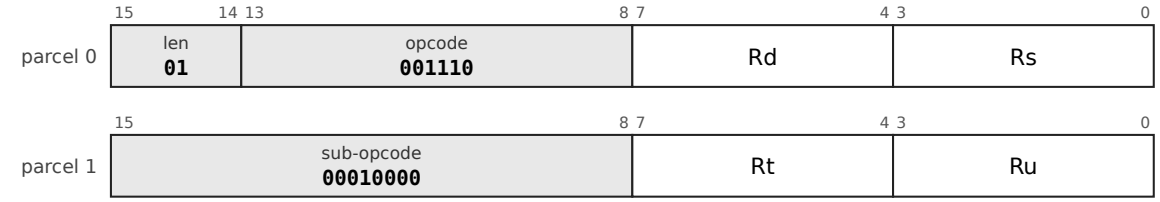
Status by decision an assembler alias (errata 98); the present assembler, simulator, QEMU and LLVM still encode and execute the word as the arithmetic-escape sub-opcode shown above, so the word is implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MIN

MIN, opcode 001110, three parcels, sub-opcode 00010000 (6 bytes)



MIN, opcode 001110, two parcels, sub-opcode 00010000 (4 bytes)



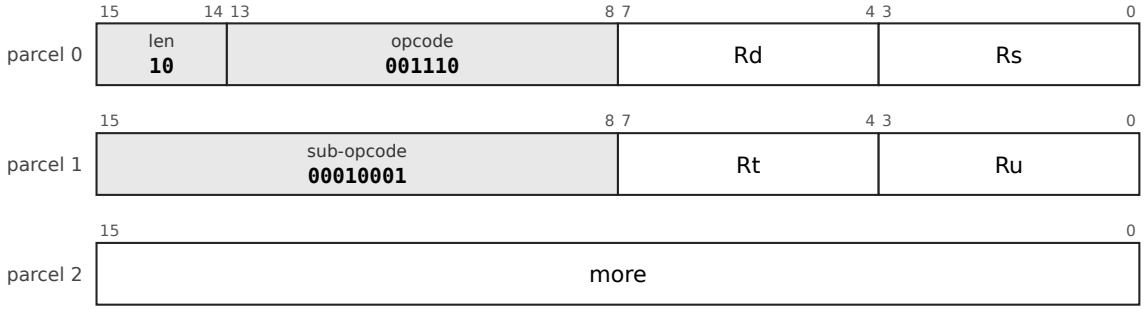

```
MIN Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
MIN	001110 (0x0E)	00010000 (0x10)

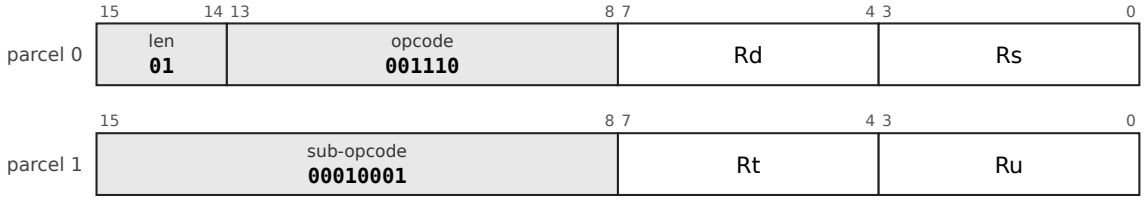
Operation, flags and exceptions as for the entry MINMAX above; 2 layout forms, each a chart of its own (above).

MINU

MINU, opcode 001110, three parcels, sub-opcode 00010001 (6 bytes)



MINU, opcode 001110, two parcels, sub-opcode 00010001 (4 bytes)



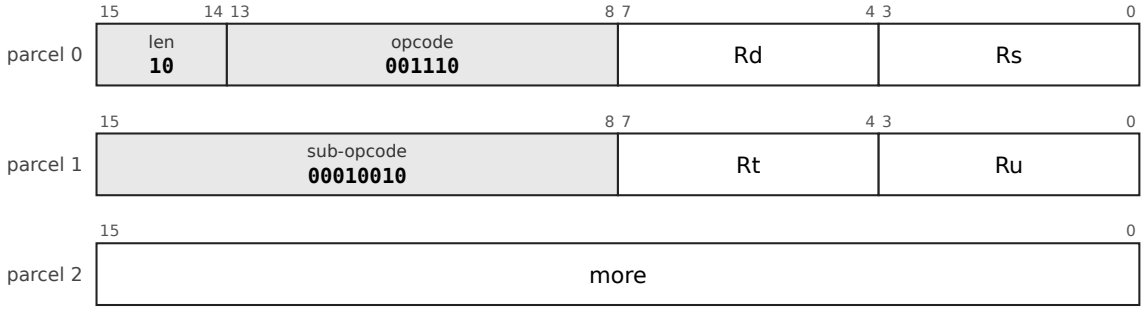

```
MINU Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
MINU	001110 (0x0E)	00010001 (0x11)

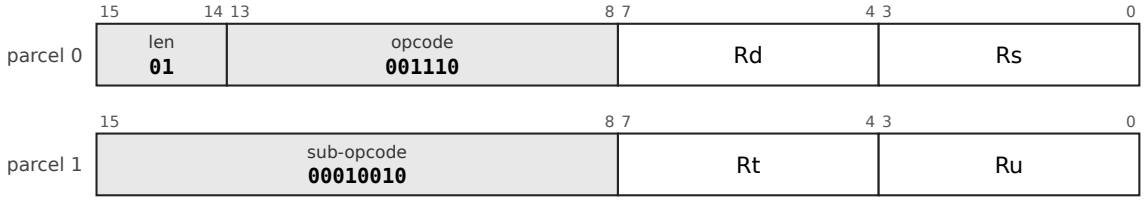
Operation, flags and exceptions as for the entry MINMAX above; 2 layout forms, each a chart of its own (above).

MAX

MAX, opcode 001110, three parcels, sub-opcode 00010010 (6 bytes)



MAX, opcode 001110, two parcels, sub-opcode 00010010 (4 bytes)



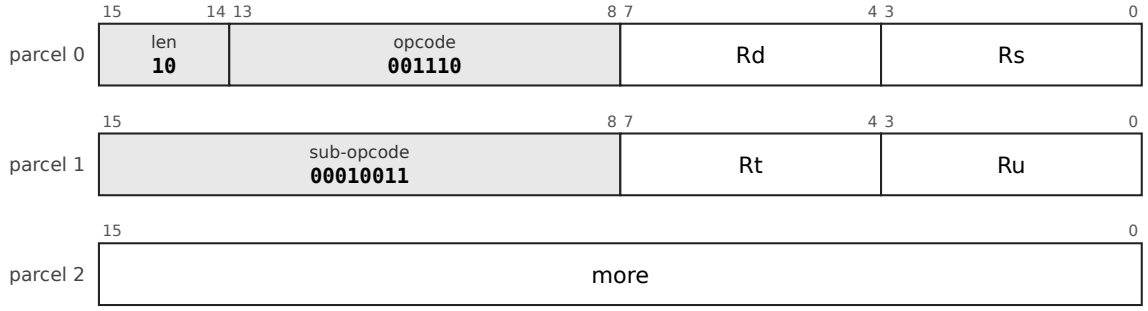

```
MAX Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
MAX	001110 (0x0E)	00010010 (0x12)

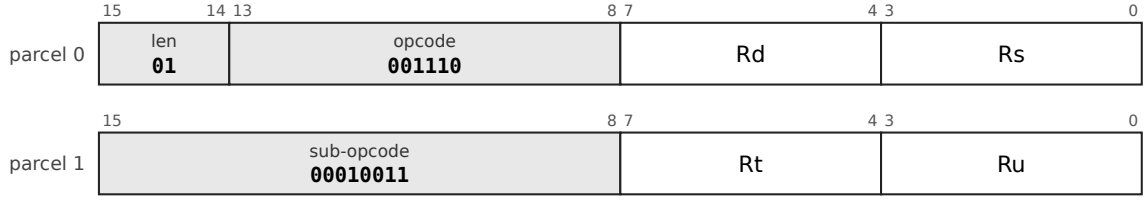
Operation, flags and exceptions as for the entry MINMAX above; 2 layout forms, each a chart of its own (above).

MAXU

MAXU, opcode 001110, three parcels, sub-opcode 00010011 (6 bytes)



MAXU, opcode 001110, two parcels, sub-opcode 00010011 (4 bytes)



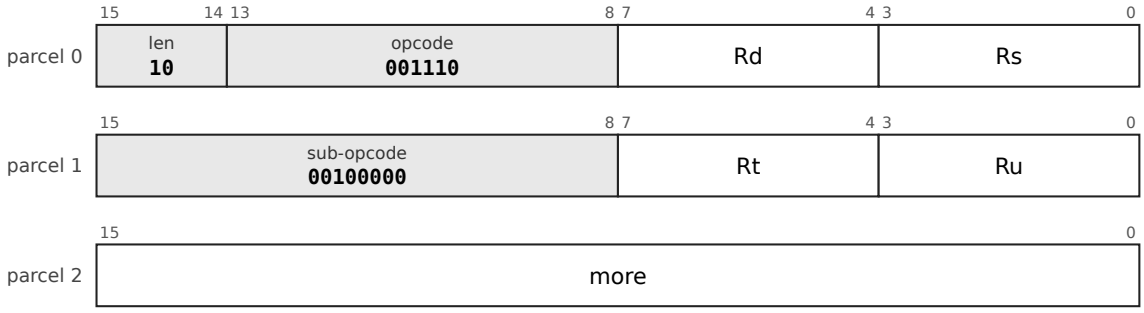

```
MAXU Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
MAXU	001110 (0x0E)	00010011 (0x13)

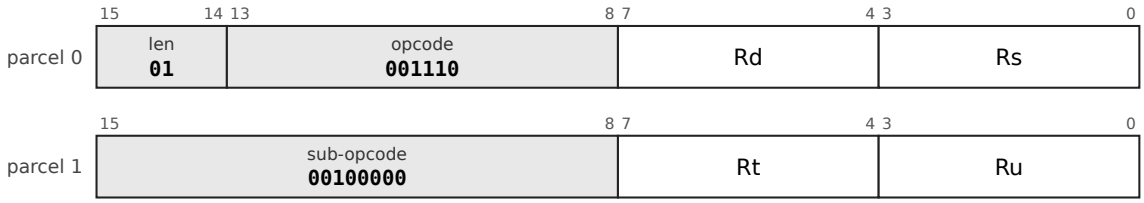
Operation, flags and exceptions as for the entry MINMAX above; 2 layout forms, each a chart of its own (above).

SET: Set a register from a condition

SET, opcode 001110, three parcels, sub-opcode 00100000 (6 bytes)



SET, opcode 001110, two parcels, sub-opcode 00100000 (4 bytes)

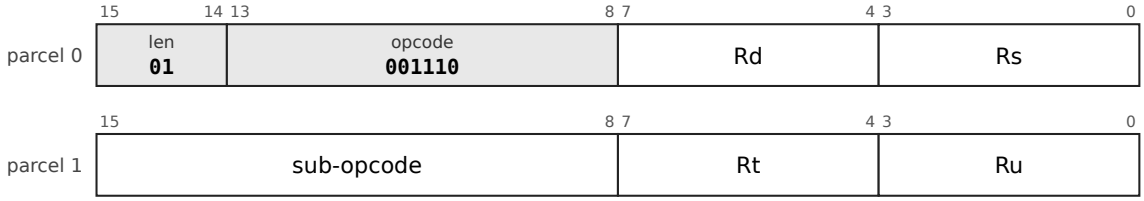


Mnemonics: `SET`

Syntax

```
SET cc, Rd        ; for example SET EQ, R1
```

Encoding



The chart is the layout `r20-aesc-1`, Arithmetic (AESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
`SET`	`001110 (0x0E)`	`00100000 (0x20)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-aesc-1`, `r20-aesc-2`.

Operation

```
Rd ← 1 if condition cc holds on the current flags, else 0
```

Flags None written.

Exceptions None.

The condition is the `x` field of parcel 1, numbered as the low nibble of the branch opcodes (`al` 0, `ne` 1, `eq` 2, ... `los` 14).

Why it is here. The first instruction written with a condition word: one instruction, `SET cc, Rd`, rather than one opcode per condition, with the condition in a field numbered like the low nibble of the branch opcodes. It turns a comparison used

as a value into 0 or 1 without a branch. gcc `-Os` for amd64 emits 673 `set<cc>` on the corpus, and `SET` would remove about six bytes per site, around 0.4% of code; the compiler's first pass turns such a comparison into branches before its second pass sees it, so the saving is not reached in compiled code.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

9.8 Logic, shifts and rotates

9.8.1 `AND`, `BIC` (bit clear), `BIS` (bit set, the or), `XOR` and `BIT` (test) are two-byte ALU operations; `COM` is one's complement. *(opcodes `011000 (0x18)`, `011001 (0x19)`, `011011 (0x1B)`, `011100 (0x1C)`, `011101 (0x1D)`)*

9.8.2 `SHL`, `SHR` and `SAR` shift by a count taken from `rm` (a register, an immediate, or memory); `ASL` and `ASR` are the by-one forms in two bytes. *(D)*

9.8.3 A shift count of 64 or more shifts the value entirely away; `C` receives the last bit that left. *(D)*

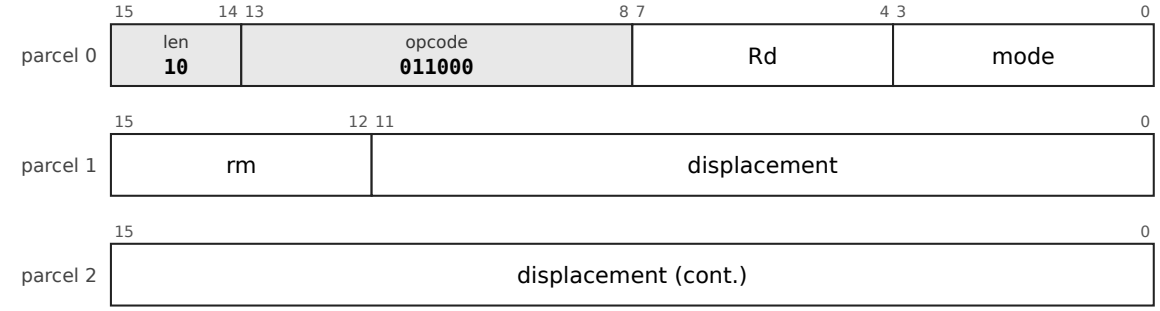
9.8.4 `ROR` rotates right by a register count and `RORI` by an immediate; `ROL Rd, Rs, #n` assembles to `RORI` with `64 - n`; `RORI32` rotates the low 32 bits and sign-extends. Rotates write no flags. *(D)*

9.8.5 `DSHR` is a double-width funnel shift. It writes no flags. *(D)*

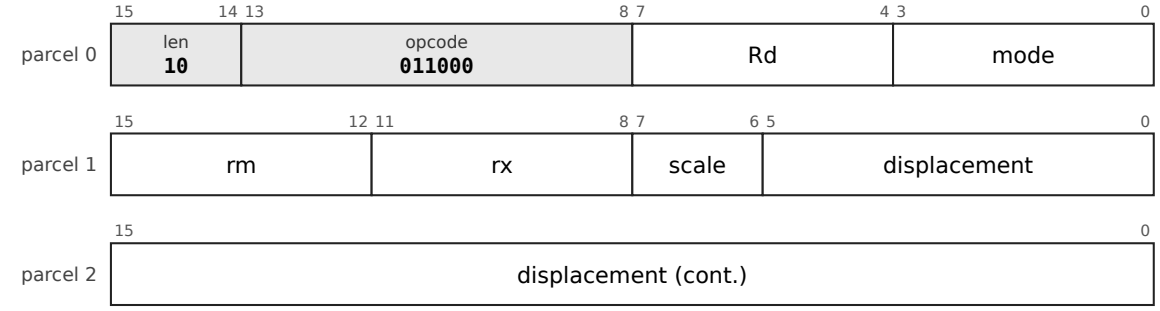
Commentary. **Precedent.** The PDP-11 had `ASL`, `ASR`, `ROL` and `ROR` by one through the carry; x86's `SHRD` and ARM's `EXTR` are the funnel shifts. **Trade-offs.** One rotate direction is enough, so there is no by-one `ROL` in a two-byte slot; rotates are not through the carry because a carry chain is fragile when the carry flag is shared. **Improvements.** The shift count comes from `rm`, not from a fixed register like x86's `CL`, which costs a register shuffle in every hand-written loop. **Benefits.** Multi-precision shifting is `DSHR`, two instructions for any distance. **Modern comparables.** RISC-V and AArch64 have rotates by register and by immediate; x86 has `ROL`/`ROR` through the carry as well. **Use.** `RORI` and `RORI32` are for hashes and ciphers, which rotate by constants; `DSHR` shifts a bignum. **Impact.** *Kernel:* Bit tests and mask operations on the status and page-table words are single instructions. *Compiler:* The shift count may be any register, immediate or memory operand, so the compiler never has to move a count into a fixed register; hashes and ciphers rotate by constants. *Hardware:* The barrel shifter (about 450 lookup tables) serves shifts and rotates, and writing no flag for rotates removes a result path.

BIC: Bit clear

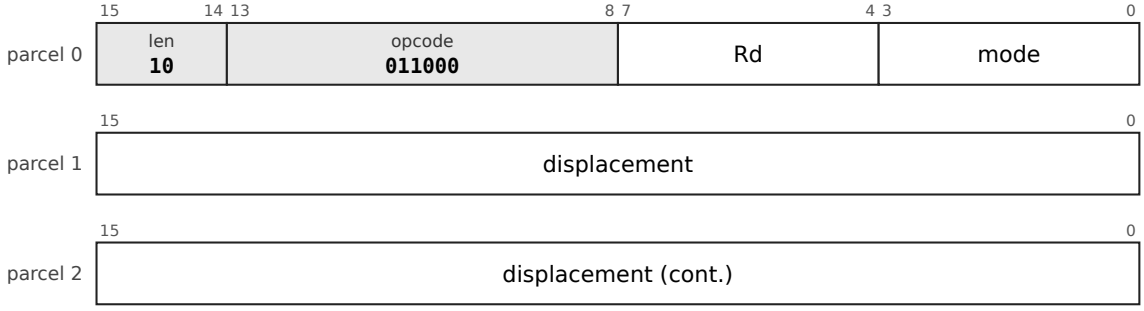
BIC, opcode 011000, three parcels (modes base, postinc, predec) (6 bytes)



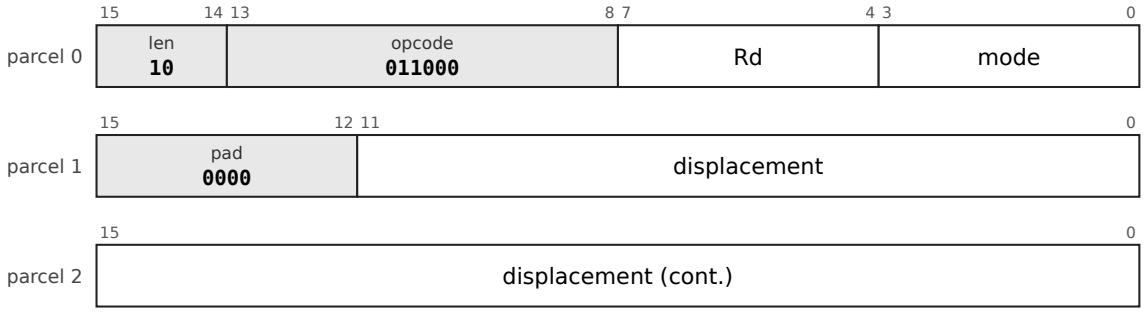
BIC, opcode 011000, three parcels (modes index) (6 bytes)



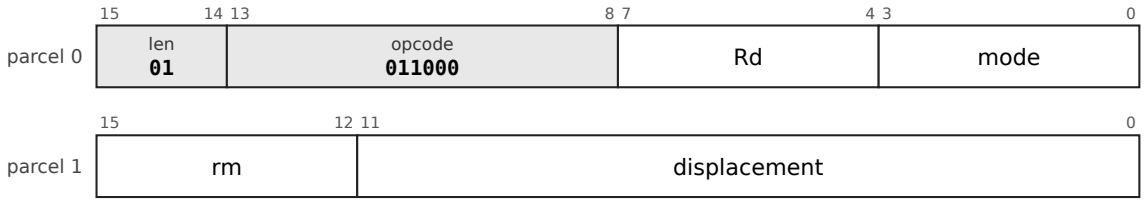
BIC, opcode 011000, three parcels (modes pcrel, imm, abs) (6 bytes)



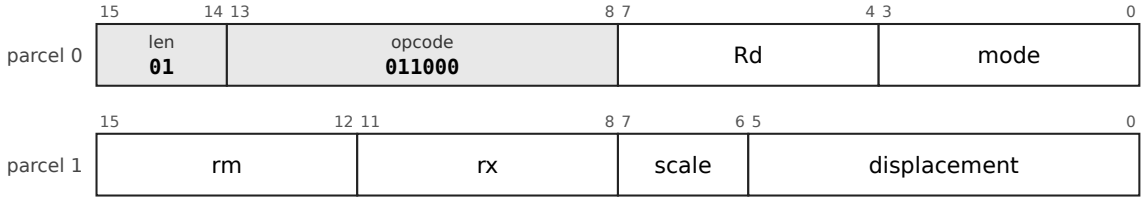
BIC, opcode 011000, three parcels (modes dp) (6 bytes)



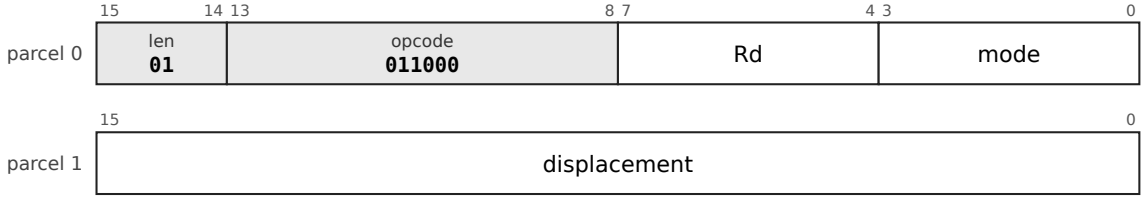
BIC, opcode 011000, two parcels (modes base, postinc, predec) (4 bytes)



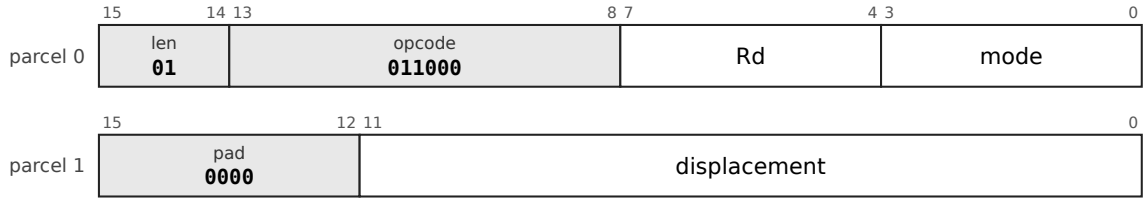
BIC, opcode 011000, two parcels (modes index) (4 bytes)



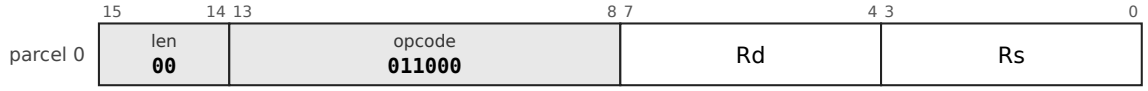
BIC, opcode 011000, two parcels (modes pcrel, imm, abs) (4 bytes)



BIC, opcode 011000, two parcels (modes dp) (4 bytes)



BIC, opcode 011000, one parcel (2 bytes)

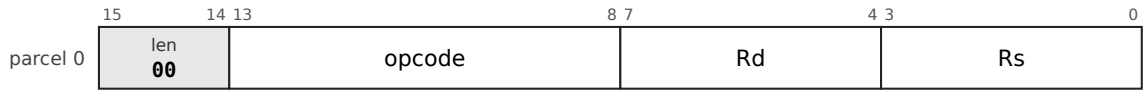


Mnemonics: `BIC`

Syntax

```
BIC rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
`BIC`	`011000 (0x18)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← Rd AND NOT rm
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

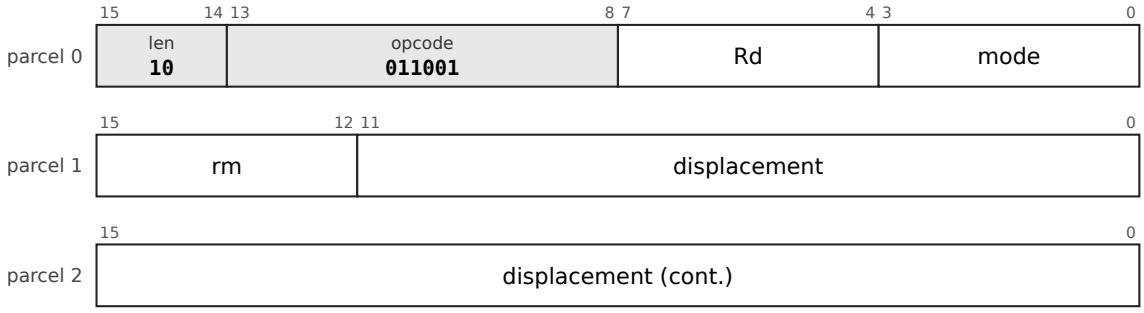
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The assembler also accepts `BIC #imm, rm` as `AND #~imm, rm`.

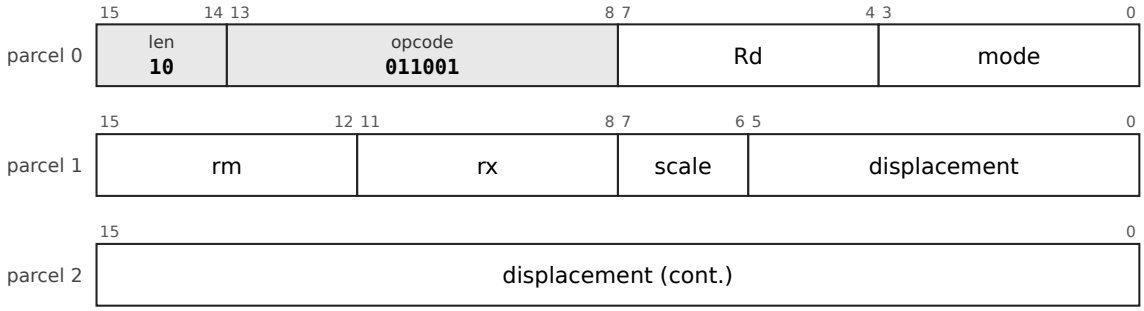
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

BIS: Bit set (inclusive or)

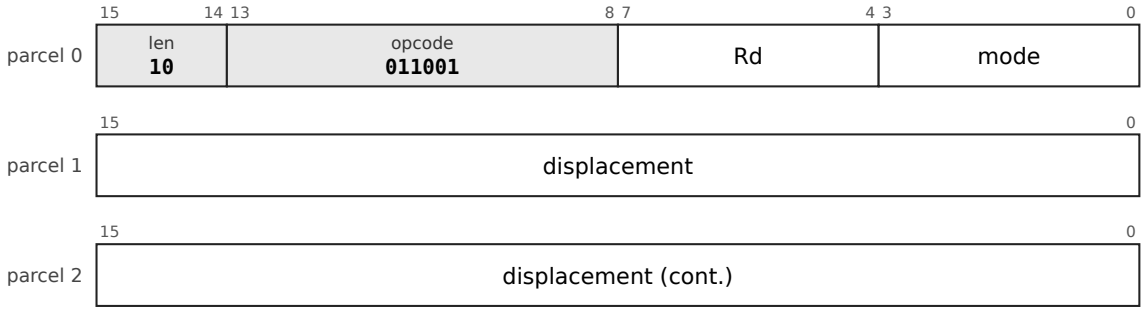
BIS, opcode 011001, three parcels (modes base, postinc, predec) (6 bytes)



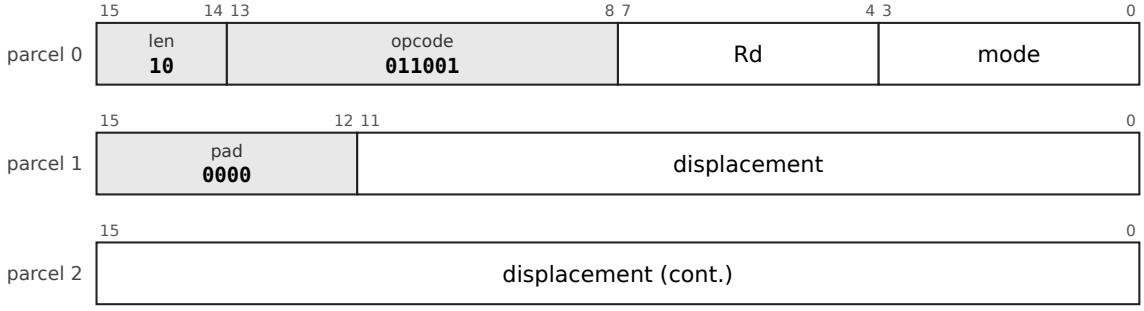
BIS, opcode 011001, three parcels (modes index) (6 bytes)



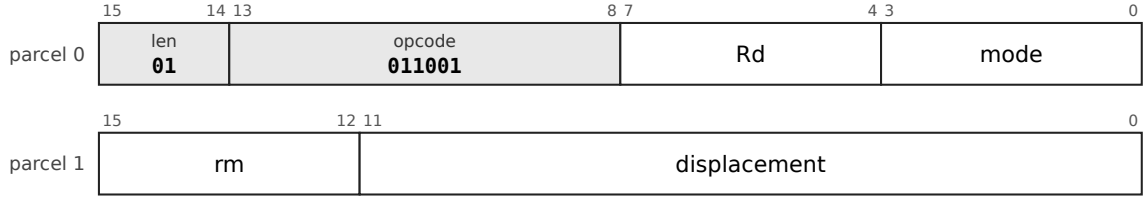
BIS, opcode 011001, three parcels (modes pcrel, imm, abs) (6 bytes)



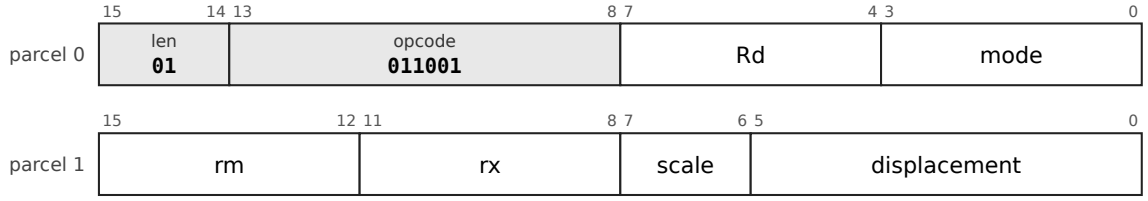
BIS, opcode 011001, three parcels (modes dp) (6 bytes)



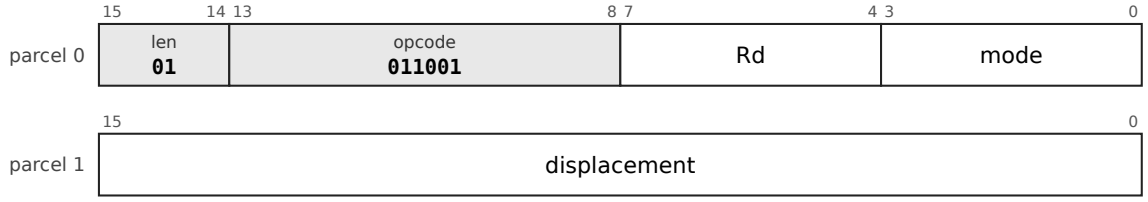
BIS, opcode 011001, two parcels (modes base, postinc, predec) (4 bytes)



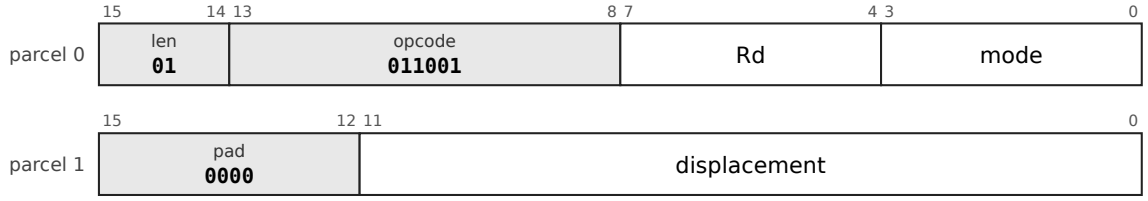
BIS, opcode 011001, two parcels (modes index) (4 bytes)



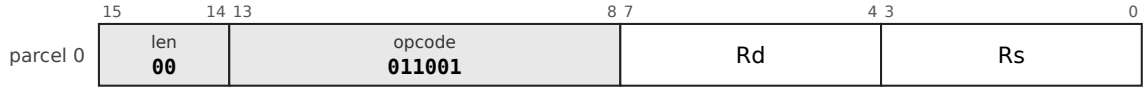
BIS, opcode 011001, two parcels (modes pcrel, imm, abs) (4 bytes)



BIS, opcode 011001, two parcels (modes dp) (4 bytes)



BIS, opcode 011001, one parcel (2 bytes)

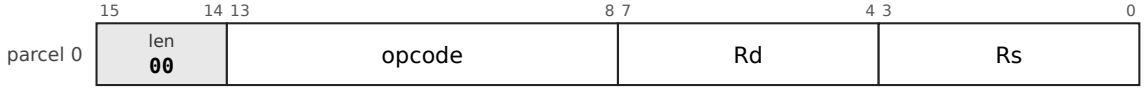


Mnemonics: `BIS`

Syntax

```
BIS rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
`BIS`	`011001 (0x19)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← Rd OR rm
```

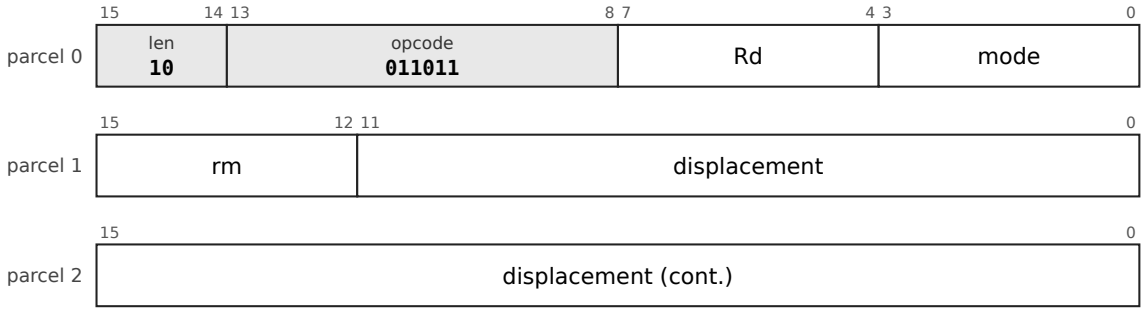
Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

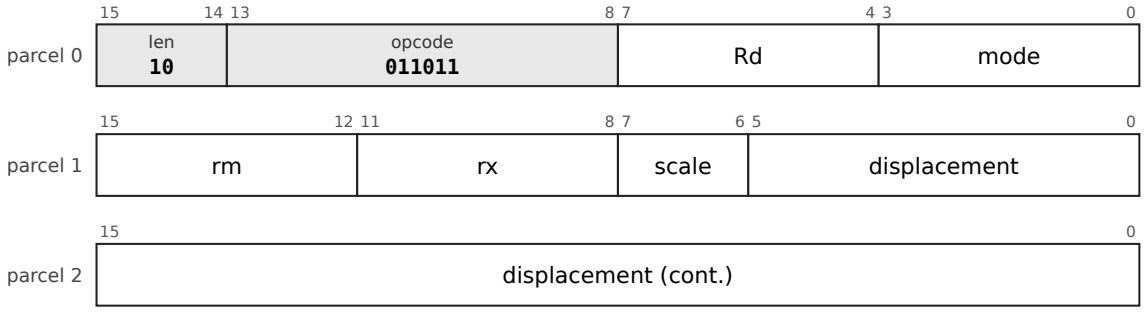
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

BIT: Bit test

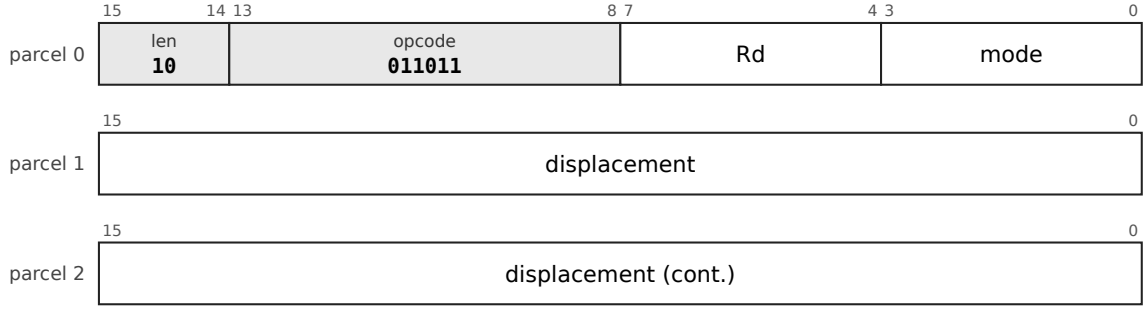
BIT, opcode 011011, three parcels (modes base, postinc, predec) (6 bytes)



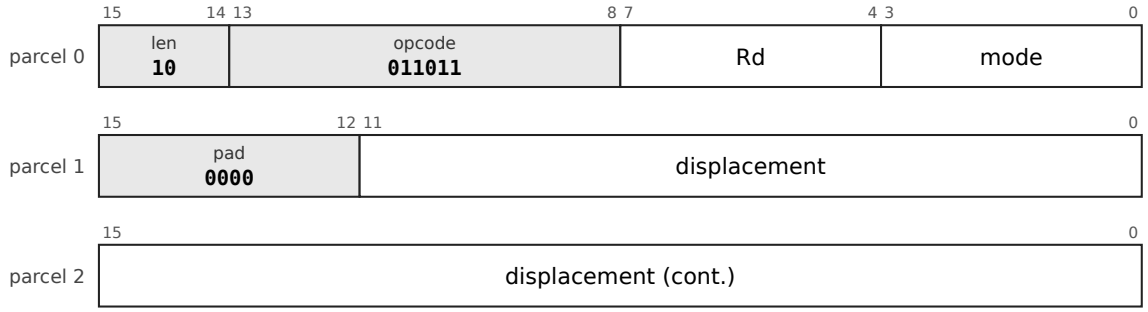
BIT, opcode 011011, three parcels (modes index) (6 bytes)



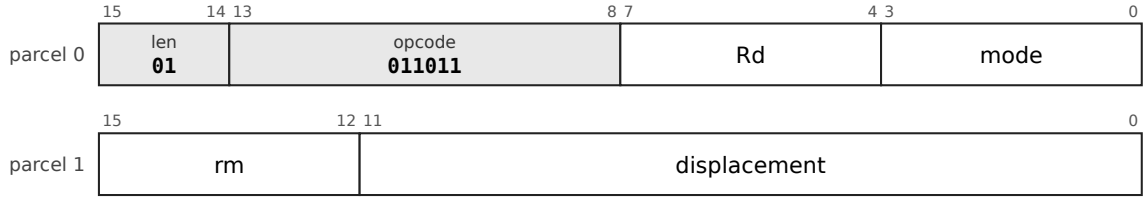
BIT, opcode 011011, three parcels (modes pcrel, imm, abs) (6 bytes)



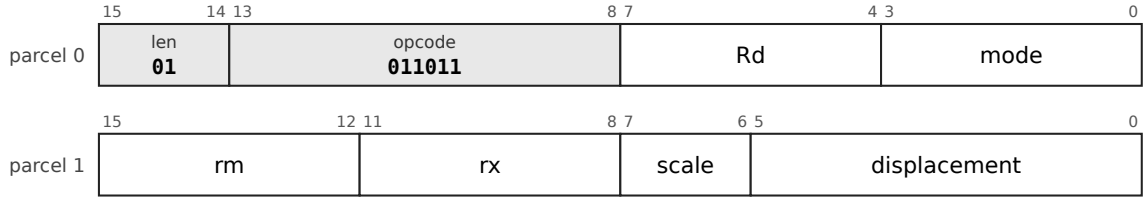
BIT, opcode 011011, three parcels (modes dp) (6 bytes)



BIT, opcode 011011, two parcels (modes base, postinc, predec) (4 bytes)



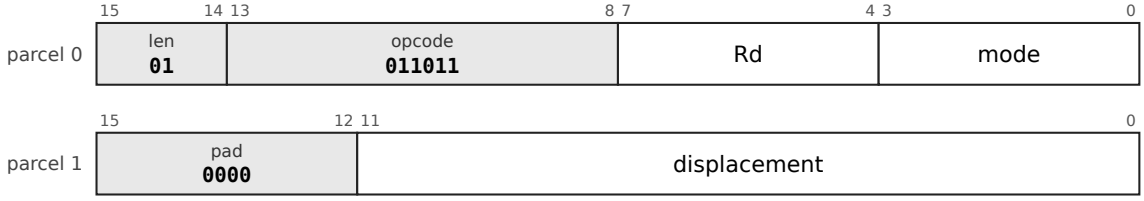
BIT, opcode 011011, two parcels (modes index) (4 bytes)



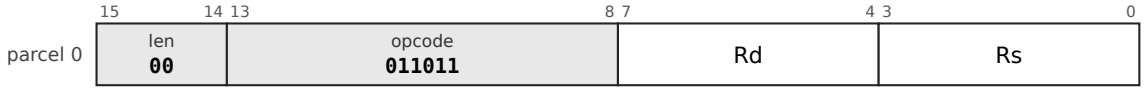
BIT, opcode 011011, two parcels (modes pcrel, imm, abs) (4 bytes)



BIT, opcode 011011, two parcels (modes dp) (4 bytes)



BIT, opcode 011011, one parcel (2 bytes)

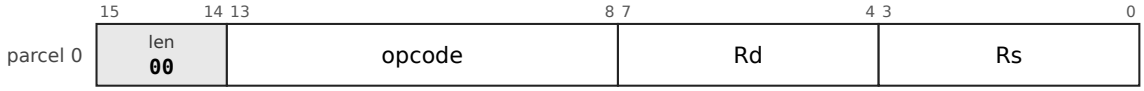


Mnemonics: `BIT`

Syntax

```
BIT rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
`BIT`	`011011 (0x1B)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
tmp ← Rd AND rm         ; the result is discarded
```

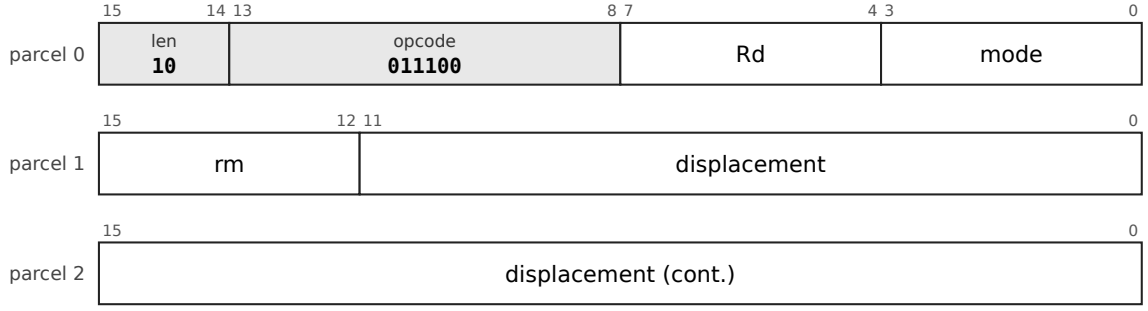
Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

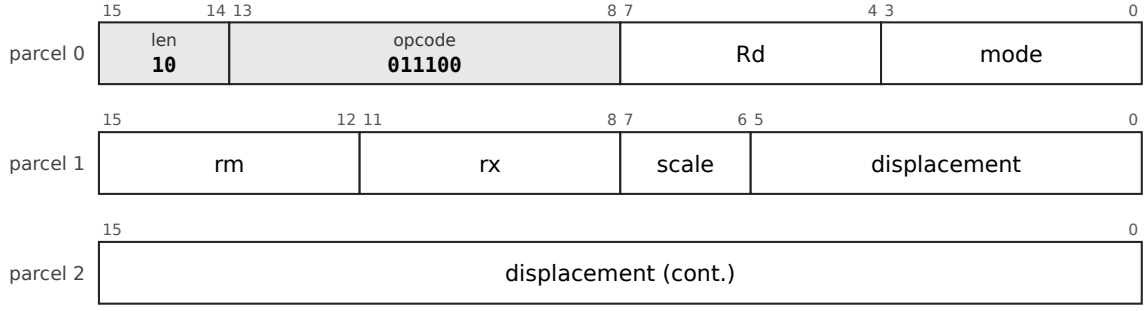
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

XOR: Exclusive or

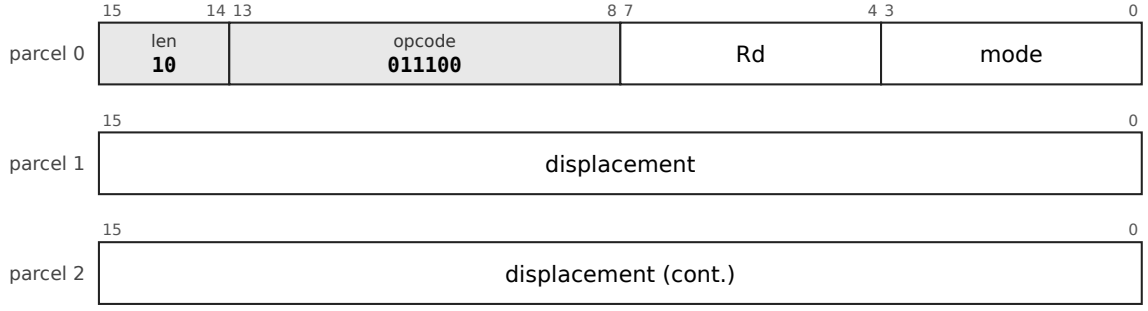
XOR, opcode 011100, three parcels (modes base, postinc, predec) (6 bytes)



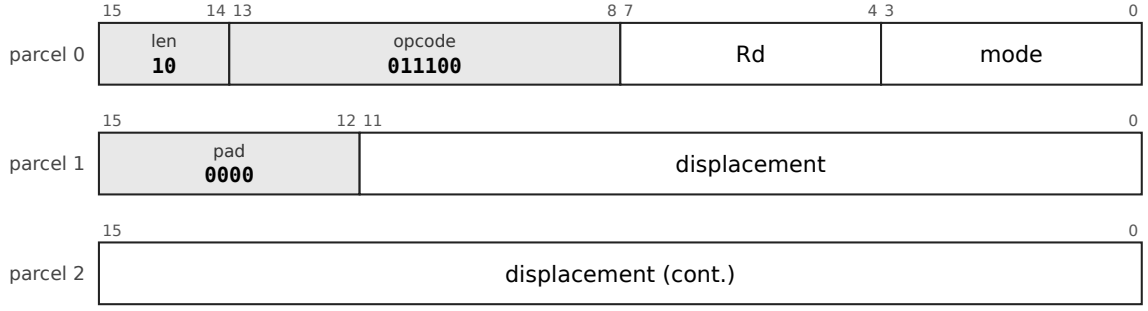
XOR, opcode 011100, three parcels (modes index) (6 bytes)



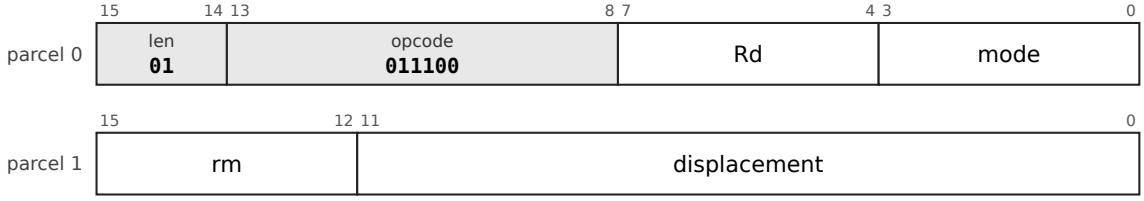
XOR, opcode 011100, three parcels (modes pcrel, imm, abs) (6 bytes)



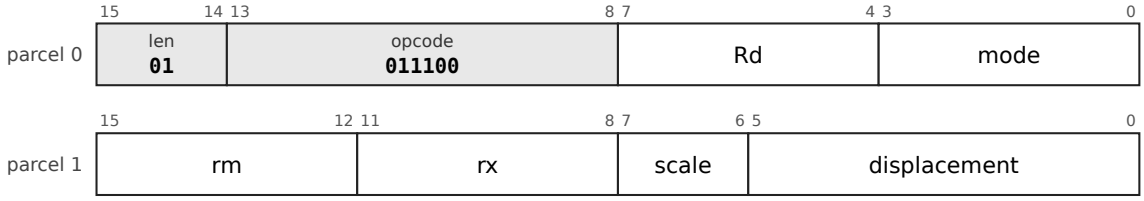
XOR, opcode 011100, three parcels (modes dp) (6 bytes)



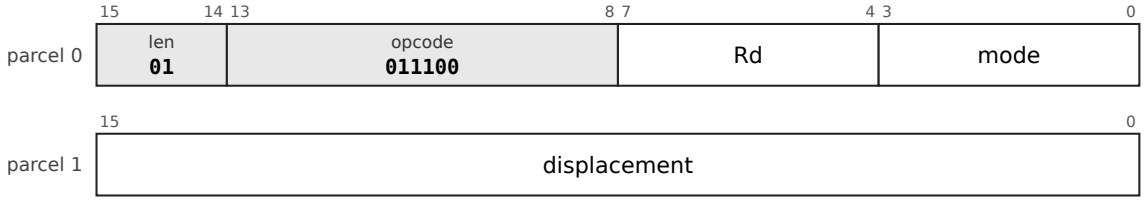
XOR, opcode 011100, two parcels (modes base, postinc, predec) (4 bytes)



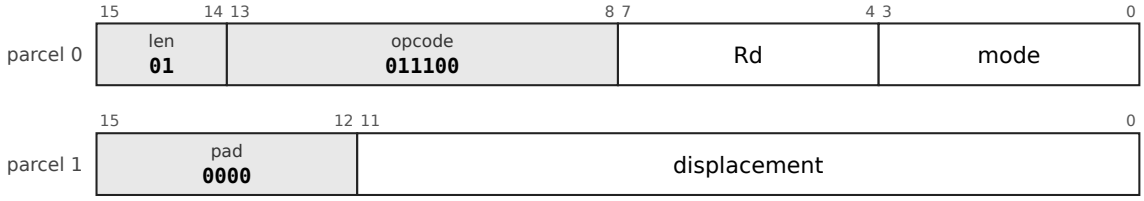
XOR, opcode 011100, two parcels (modes index) (4 bytes)



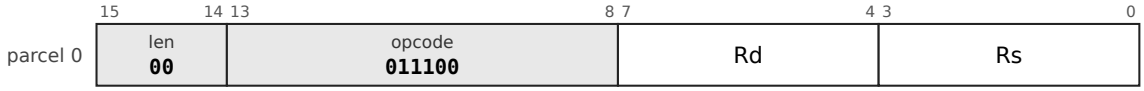
XOR, opcode 011100, two parcels (modes pcrel, imm, abs) (4 bytes)



XOR, opcode 011100, two parcels (modes dp) (4 bytes)



XOR, opcode 011100, one parcel (2 bytes)

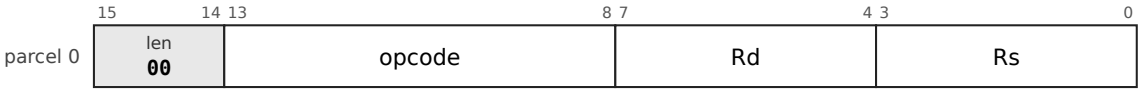


Mnemonics: `XOR`

Syntax

```
XOR rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
XOR	011100 (0x1C)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← Rd XOR rm
```

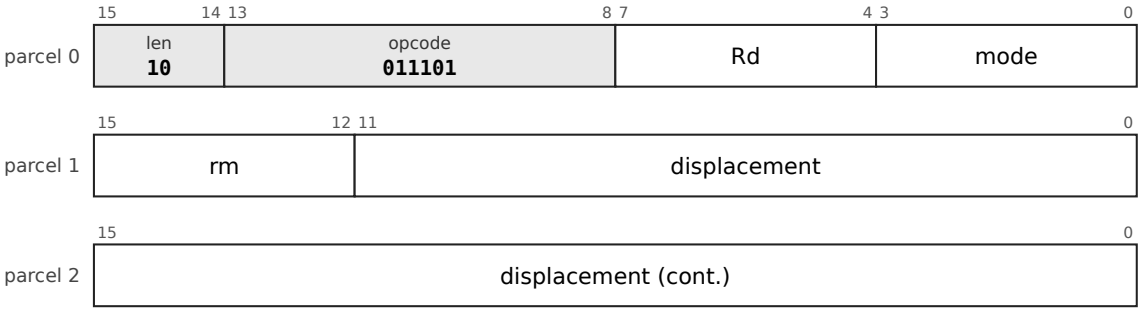
Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

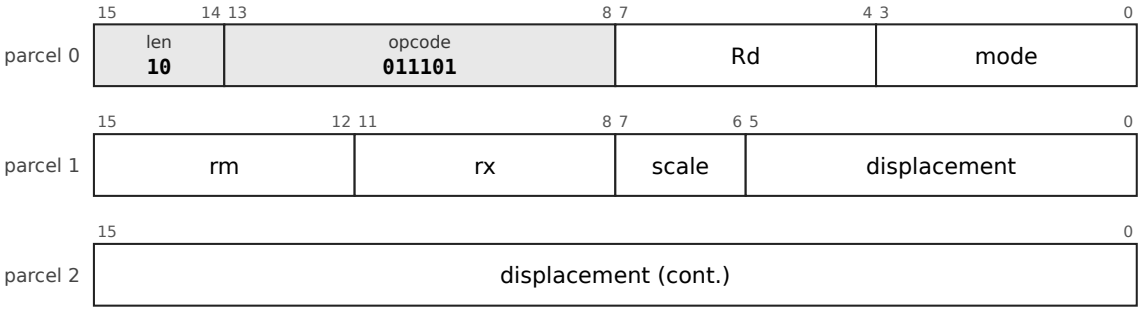
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

AND: And

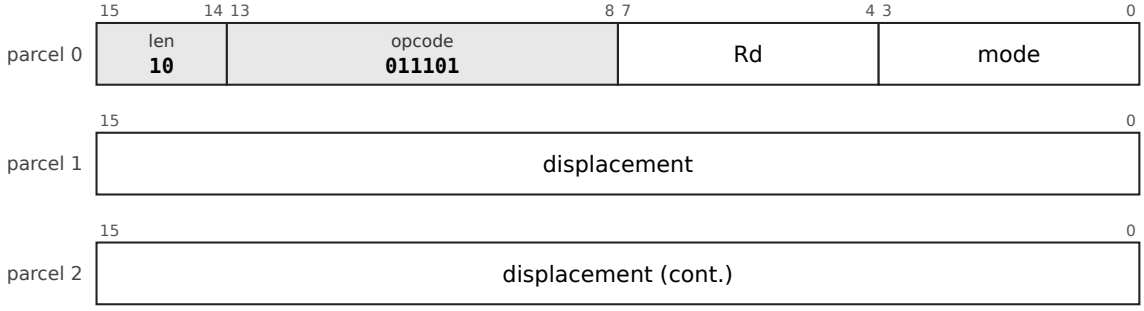
AND, opcode 011101, three parcels (modes base, postinc, predec) (6 bytes)



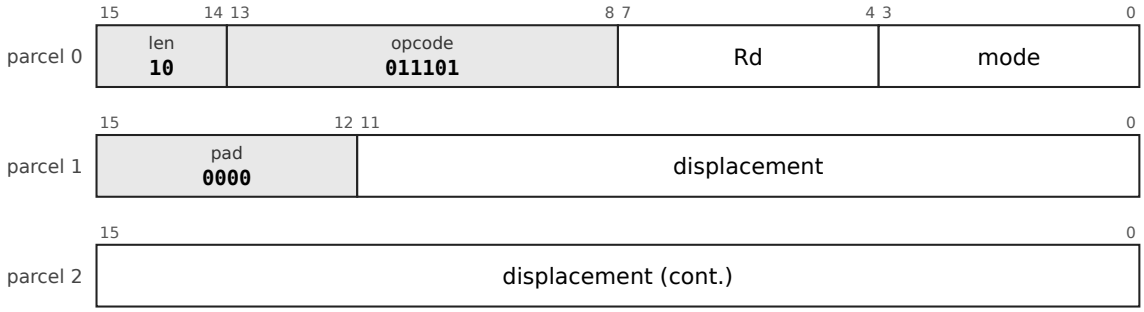
AND, opcode 011101, three parcels (modes index) (6 bytes)



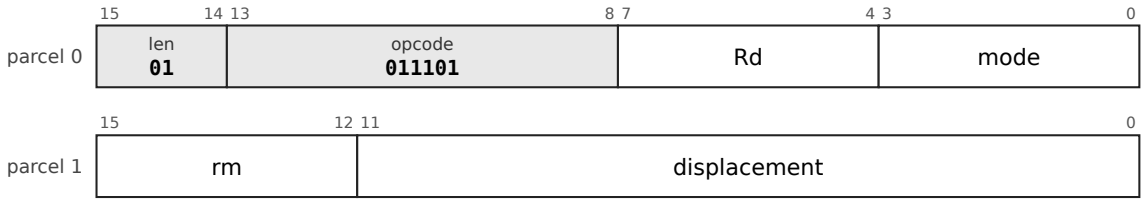
AND, opcode 011101, three parcels (modes pcrel, imm, abs) (6 bytes)



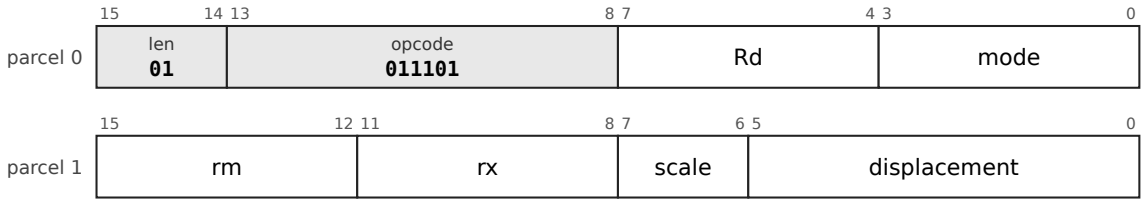
AND, opcode 011101, three parcels (modes dp) (6 bytes)



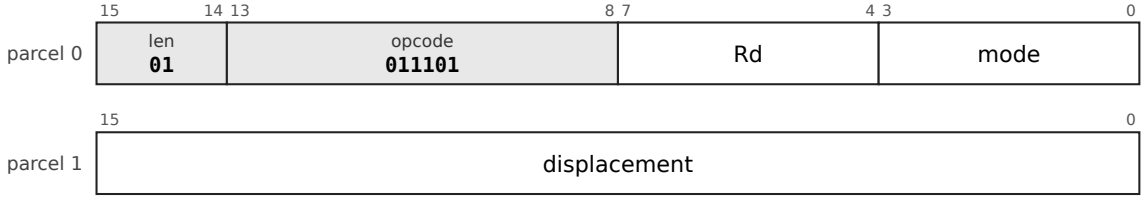
AND, opcode 011101, two parcels (modes base, postinc, predec) (4 bytes)



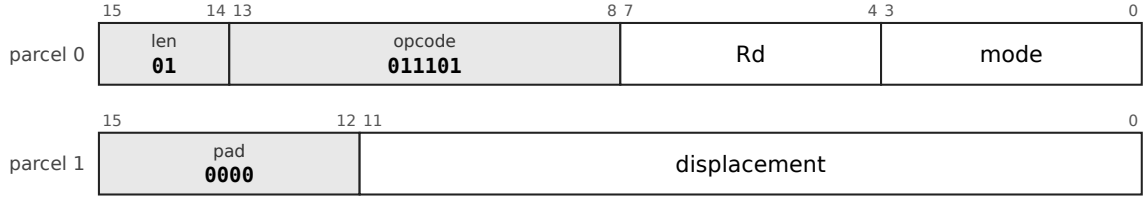
AND, opcode 011101, two parcels (modes index) (4 bytes)



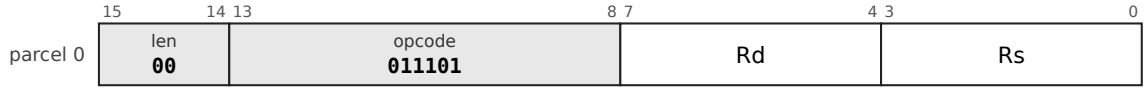
AND, opcode 011101, two parcels (modes pcrel, imm, abs) (4 bytes)



AND, opcode 011101, two parcels (modes dp) (4 bytes)



AND, opcode 011101, one parcel (2 bytes)

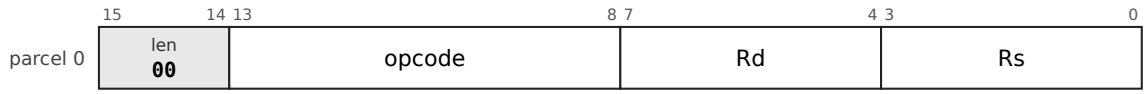


Mnemonics: `AND`

Syntax

```
AND rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
AND	011101 (0x1D)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← Rd AND rm
```

Flags `N` and `Z` from the result, `V` cleared, `C` unchanged.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

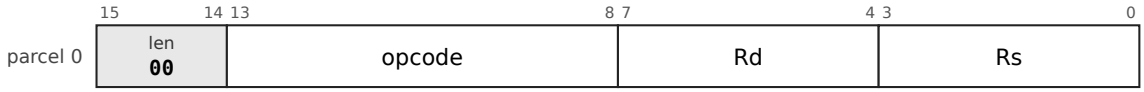
SHIFT: Shift left, shift right logical, shift right arithmetic

Mnemonics: `SHL`, `SHR`, `SAR`

Syntax

```
SHL rm, Rd
SHR rm, Rd
SAR rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
n ← rm                                  ; the count: a register, an immediate, or memory
SHL: Rd ← (n ≥ 64) ? 0 : Rd << n
SHR: Rd ← (n ≥ 64) ? 0 : Rd >>u n
SAR: Rd ← (n ≥ 64) ? (Rd < 0 ? -1 : 0) : Rd >>s n
```

Flags `N` and `Z` from the result, `V` cleared, `C` the last bit shifted out (bit 63 for `SHL`, bit 0 for `SHR` and `SAR`). **The same flags are written for every count**: a count of zero gives `N` and `Z` from the unchanged operand, `V` = 0 and `C` = 0.

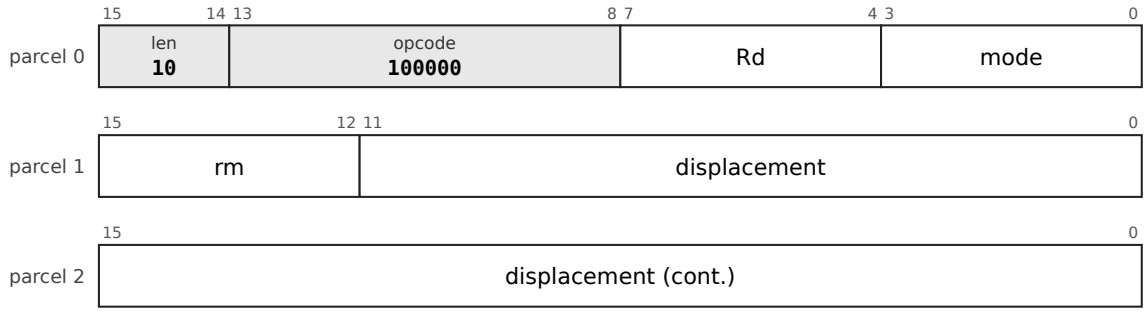
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The count is read as an unsigned 64-bit value, so a count of 64 or more shifts the value entirely away. Page 2 has the three shifts by an immediate 0 to 15. `ASL` and `ASR` (in `SOP`) are the by-one forms in two bytes; rotates are in `BESC`.

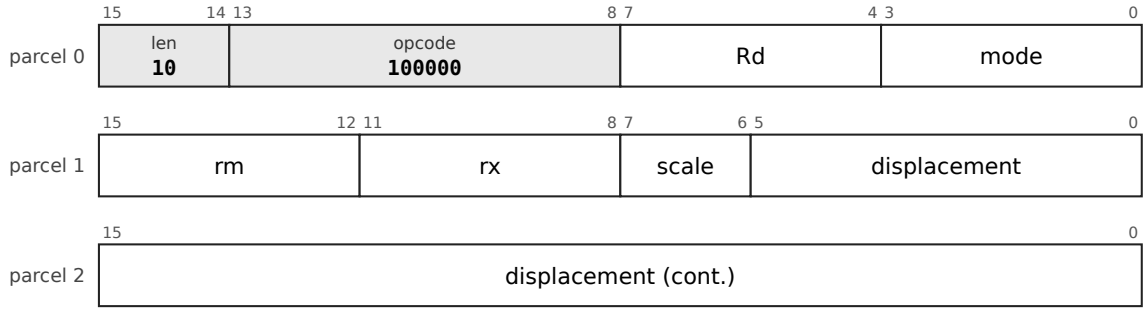
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

SHL

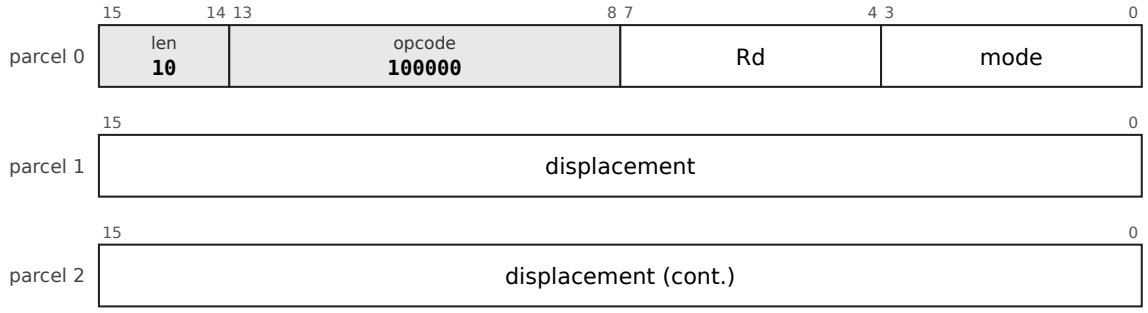
SHL, opcode 100000, three parcels (modes base, postinc, predec) (6 bytes)



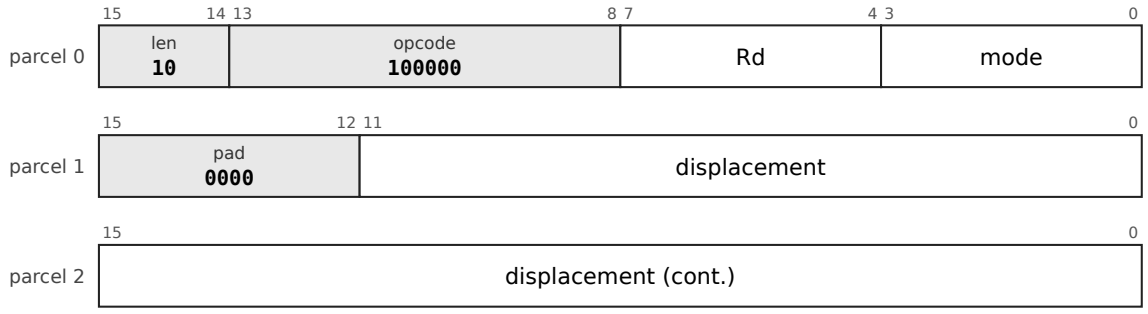
SHL, opcode 100000, three parcels (modes index) (6 bytes)



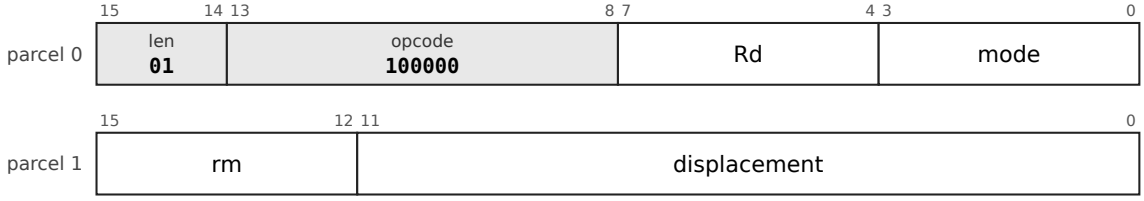
SHL, opcode 100000, three parcels (modes pcrel, imm, abs) (6 bytes)



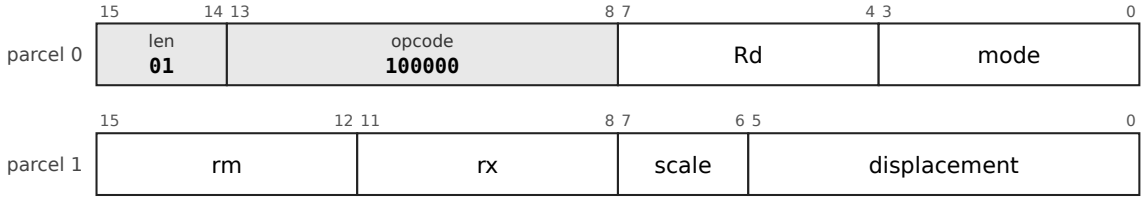
SHL, opcode 100000, three parcels (modes dp) (6 bytes)



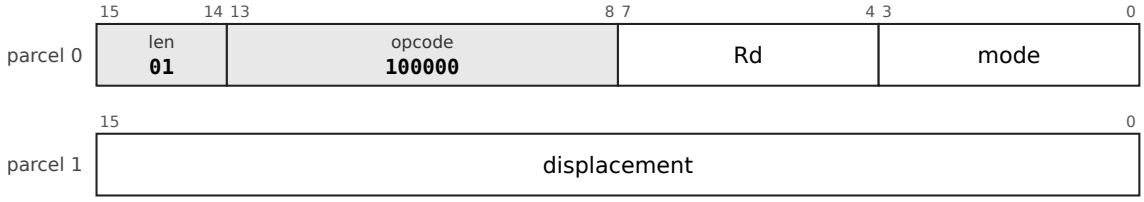
SHL, opcode 100000, two parcels (modes base, postinc, predec) (4 bytes)



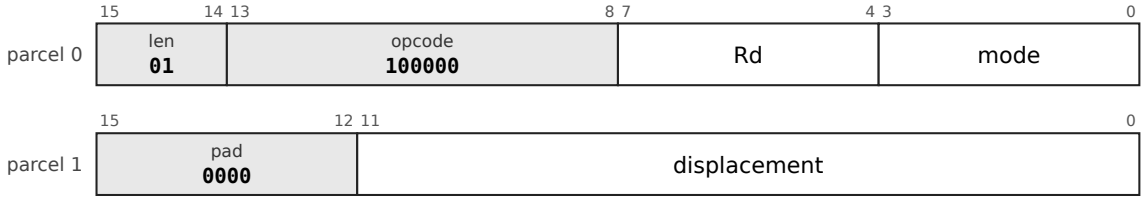
SHL, opcode 100000, two parcels (modes index) (4 bytes)



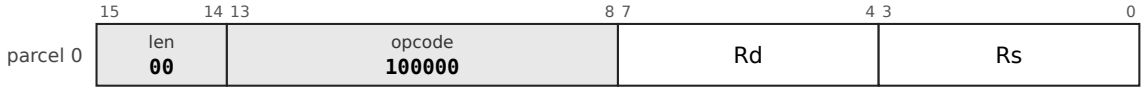
SHL, opcode 100000, two parcels (modes pcrel, imm, abs) (4 bytes)



SHL, opcode 100000, two parcels (modes dp) (4 bytes)



SHL, opcode 100000, one parcel (2 bytes)



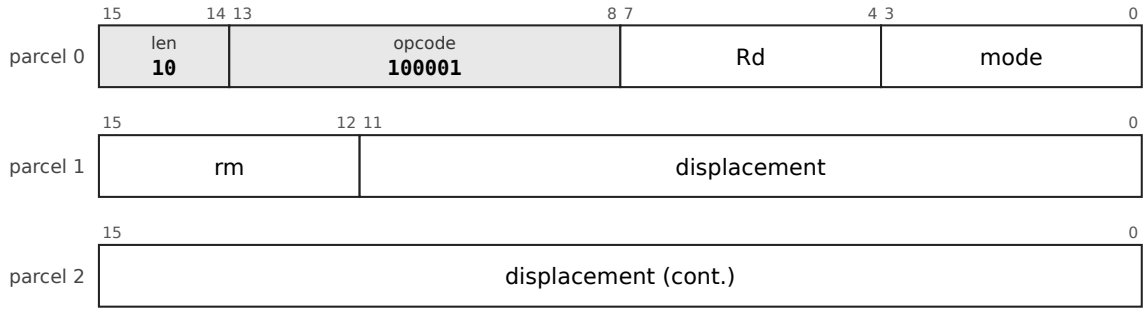

```
SHL rm, Rd
```

Mnemonic	Opcode
SHL	100000 (0x20)

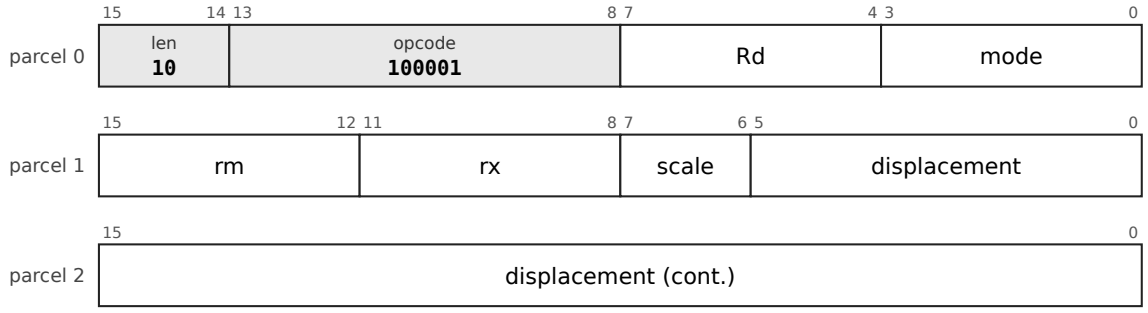
Operation, flags and exceptions as for the entry SHIFT above; 9 layout forms, each a chart of its own (above).

SHR

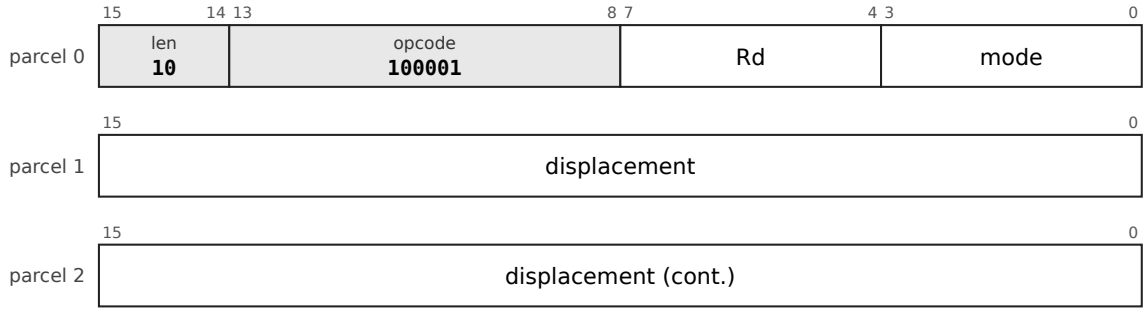
SHR, opcode 100001, three parcels (modes base, postinc, predec) (6 bytes)



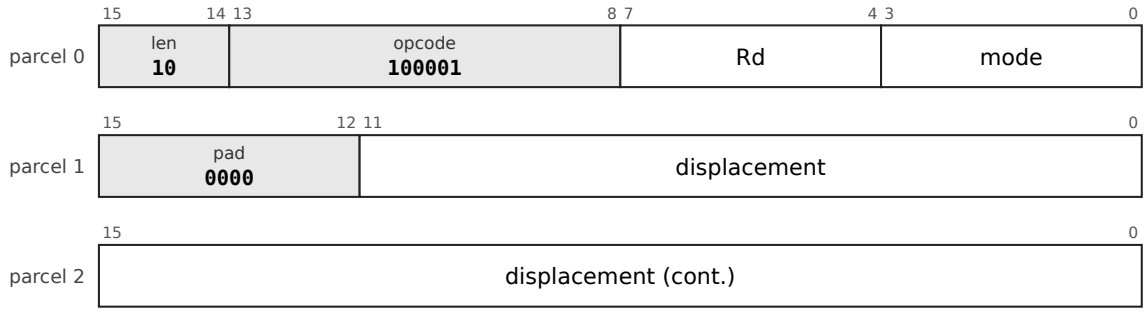
SHR, opcode 100001, three parcels (modes index) (6 bytes)



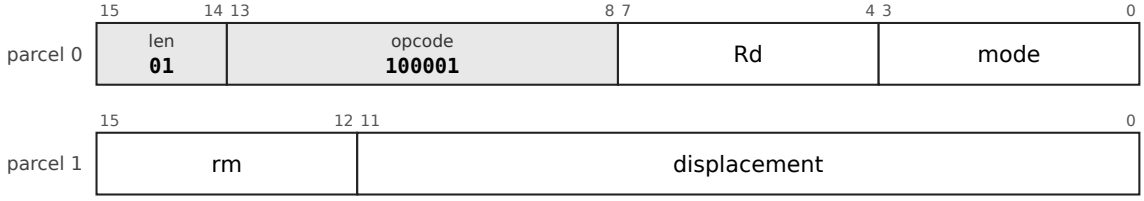
SHR, opcode 100001, three parcels (modes pcrel, imm, abs) (6 bytes)



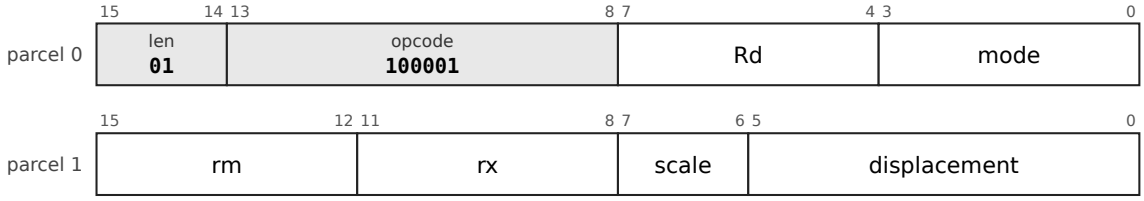
SHR, opcode 100001, three parcels (modes dp) (6 bytes)



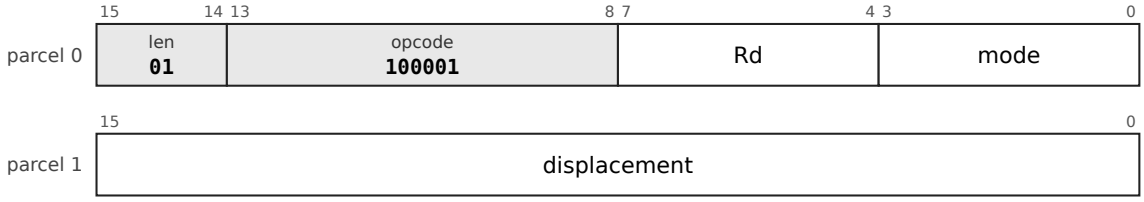
SHR, opcode 100001, two parcels (modes base, postinc, predec) (4 bytes)



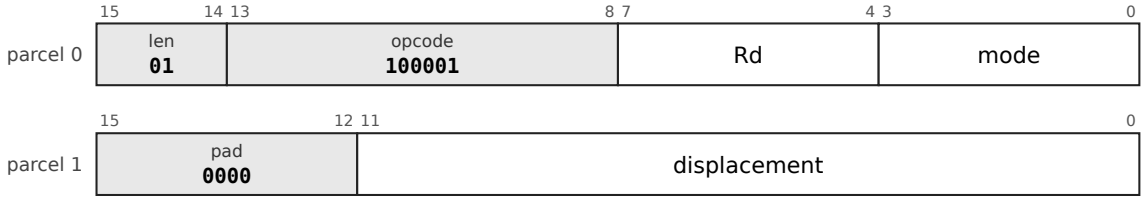
SHR, opcode 100001, two parcels (modes index) (4 bytes)



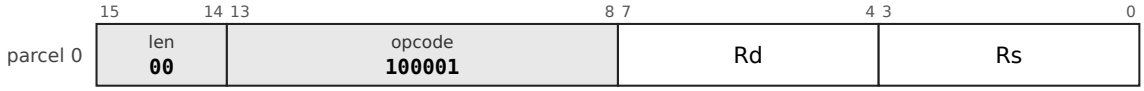
SHR, opcode 100001, two parcels (modes pcrel, imm, abs) (4 bytes)



SHR, opcode 100001, two parcels (modes dp) (4 bytes)



SHR, opcode 100001, one parcel (2 bytes)



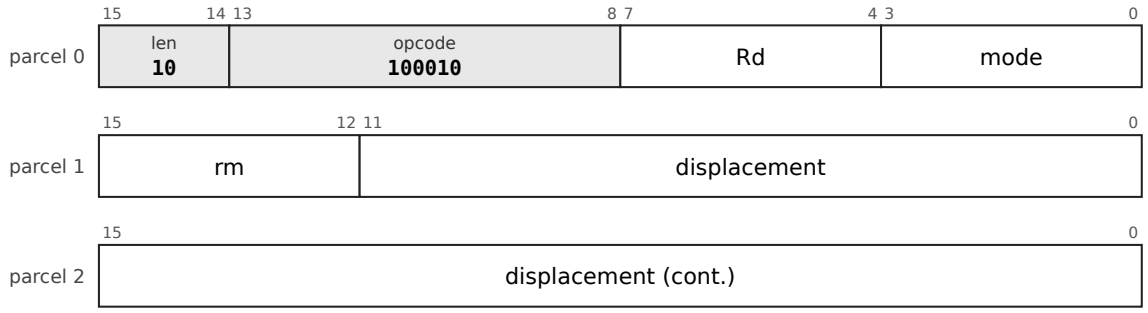

```
SHR rm, Rd
```

Mnemonic	Opcode
SHR	100001 (0x21)

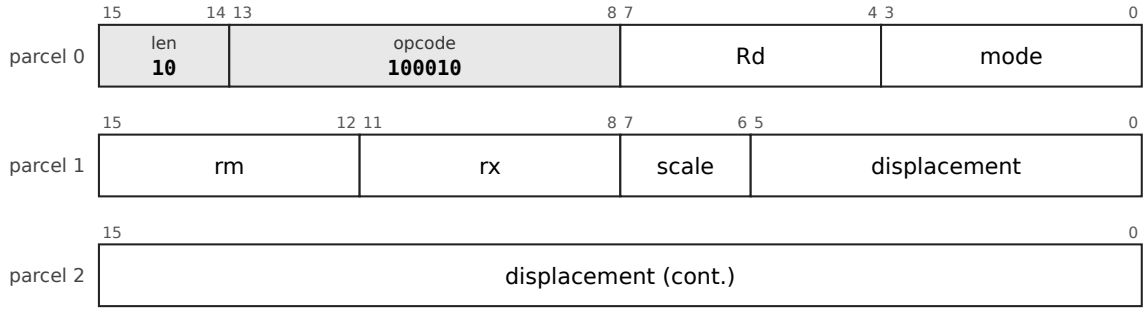
Operation, flags and exceptions as for the entry SHIFT above; 9 layout forms, each a chart of its own (above).

SAR

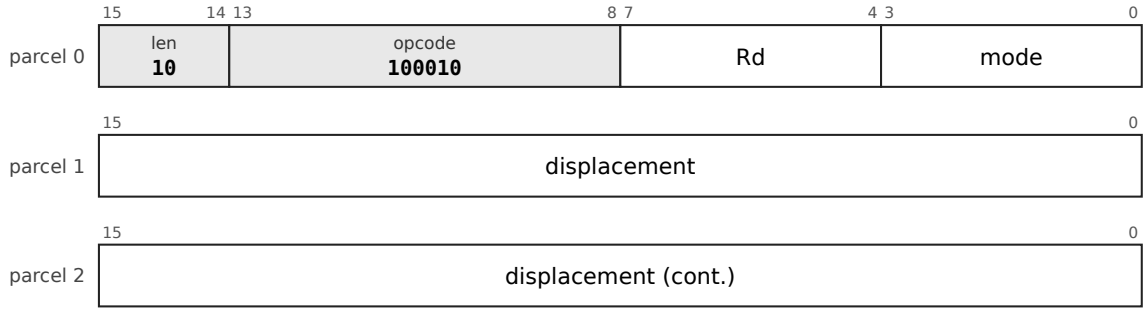
SAR, opcode 100010, three parcels (modes base, postinc, predec) (6 bytes)



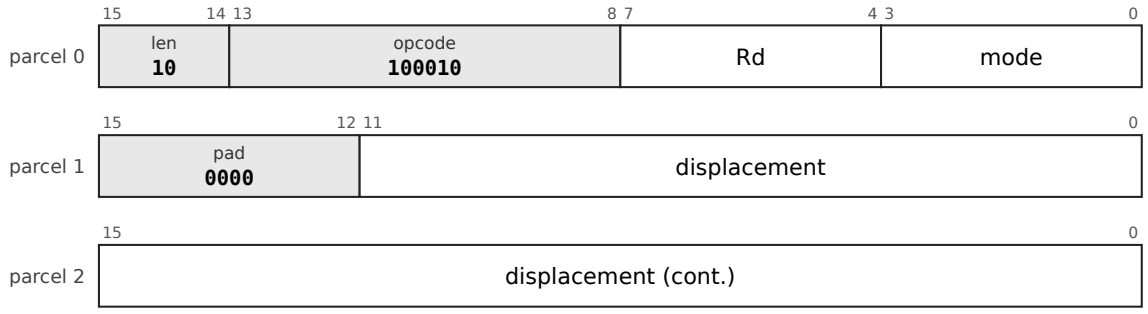
SAR, opcode 100010, three parcels (modes index) (6 bytes)



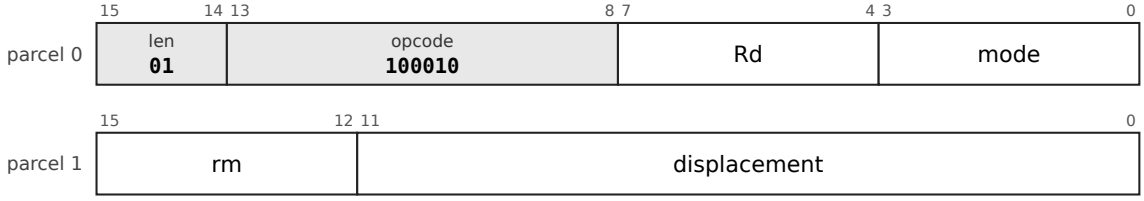
SAR, opcode 100010, three parcels (modes pcrel, imm, abs) (6 bytes)



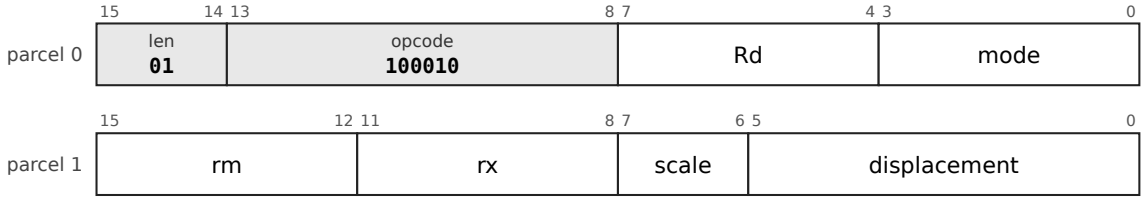
SAR, opcode 100010, three parcels (modes dp) (6 bytes)



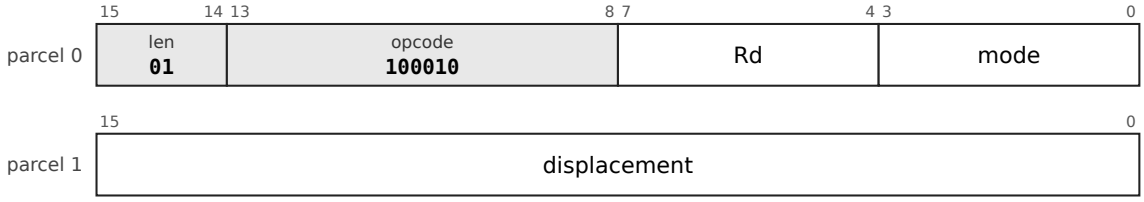
SAR, opcode 100010, two parcels (modes base, postinc, predec) (4 bytes)



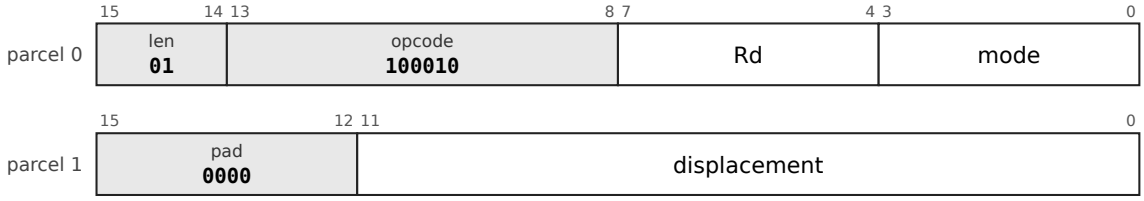
SAR, opcode 100010, two parcels (modes index) (4 bytes)



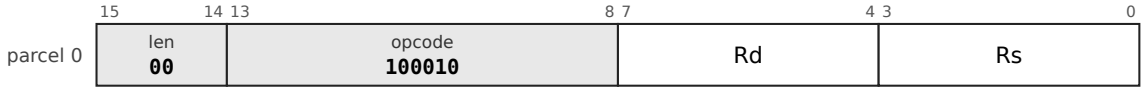
SAR, opcode 100010, two parcels (modes pcrel, imm, abs) (4 bytes)



SAR, opcode 100010, two parcels (modes dp) (4 bytes)



SAR, opcode 100010, one parcel (2 bytes)



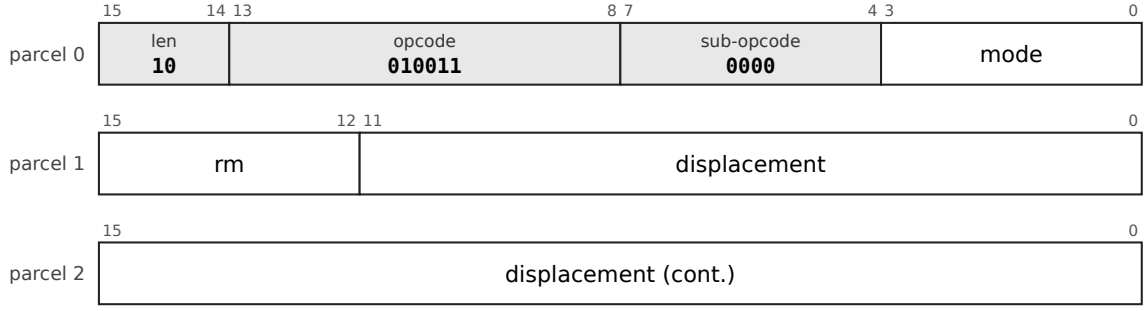

```
SAR rm, Rd
```

Mnemonic	Opcode
SAR	100010 (0x22)

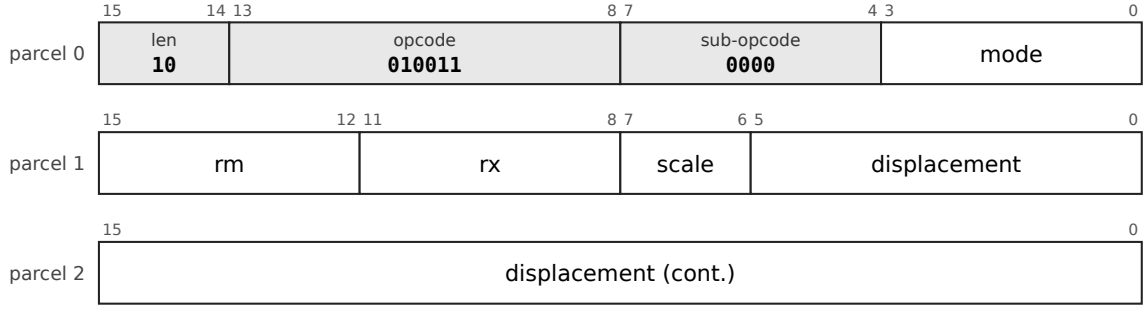
Operation, flags and exceptions as for the entry SHIFT above; 9 layout forms, each a chart of its own (above).

CLR: Clear

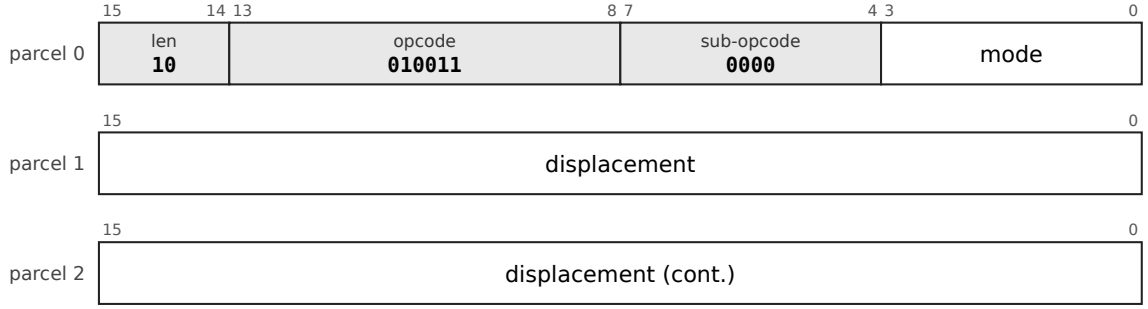
CLR, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



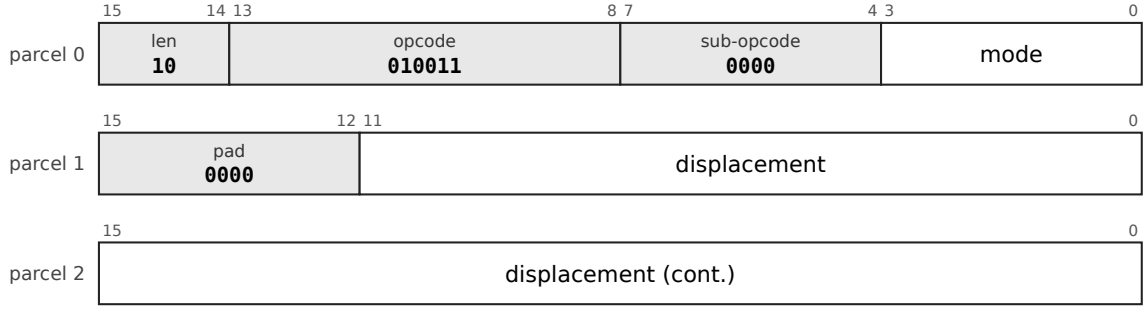
CLR, opcode 010011, three parcels (modes index) (6 bytes)



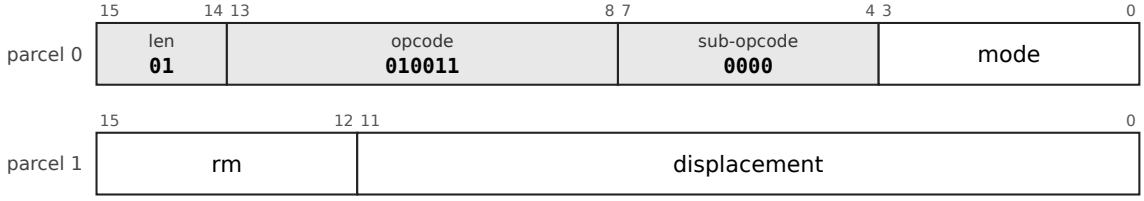
CLR, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



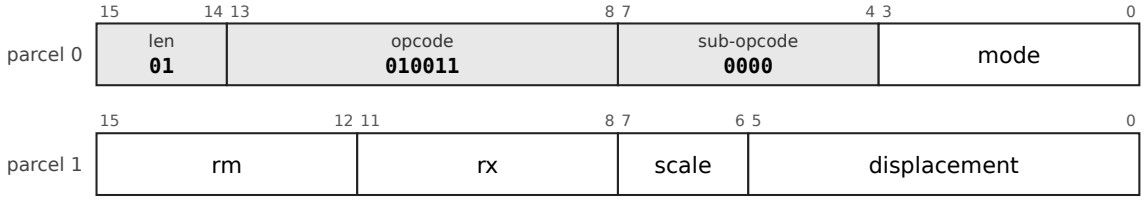
CLR, opcode 010011, three parcels (modes dp) (6 bytes)



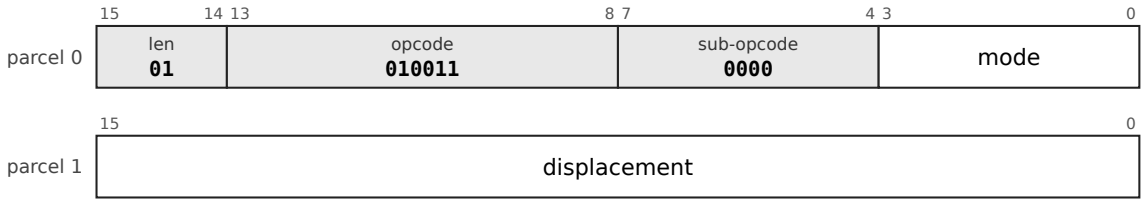
CLR, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



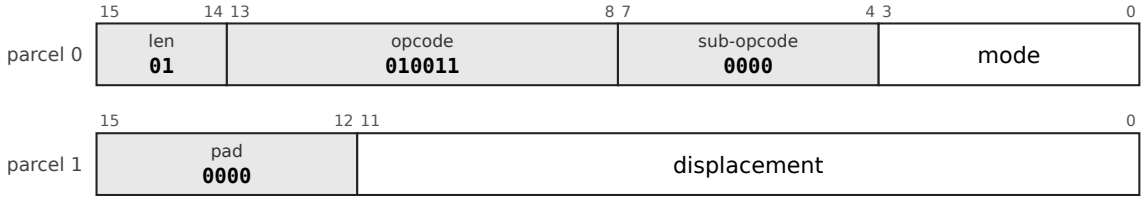
CLR, opcode 010011, two parcels (modes index) (4 bytes)



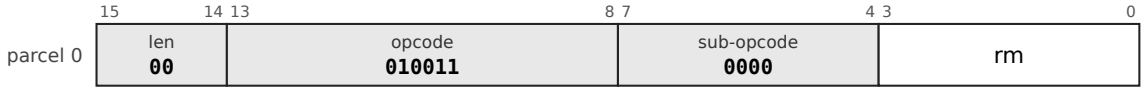
CLR, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



CLR, opcode 010011, two parcels (modes dp) (4 bytes)



CLR, opcode 010011, one parcel (2 bytes)

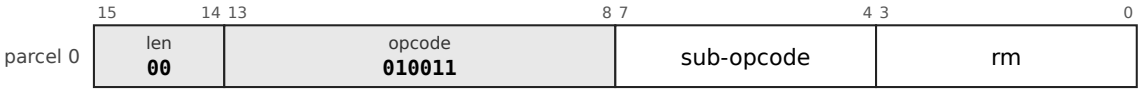


Mnemonics: `CLR`

Syntax

```
CLR rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
CLR	010011 (0x13)	0000 (0x0)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
rm ← 0
```

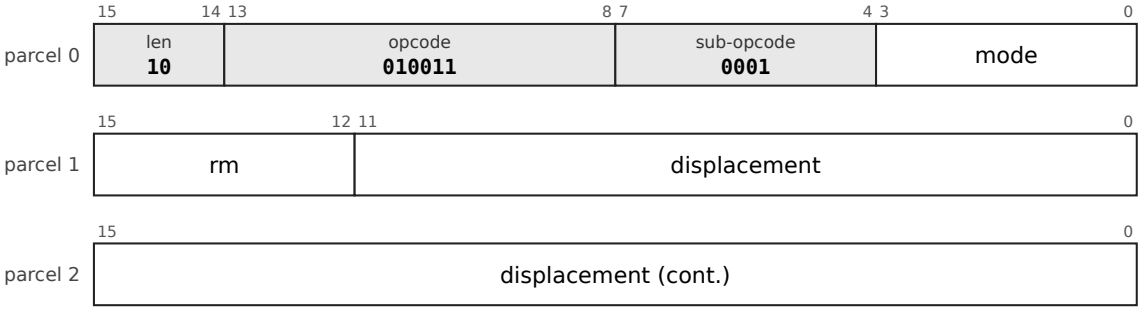
Flags N = 0, Z = 1, V = 0, C = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

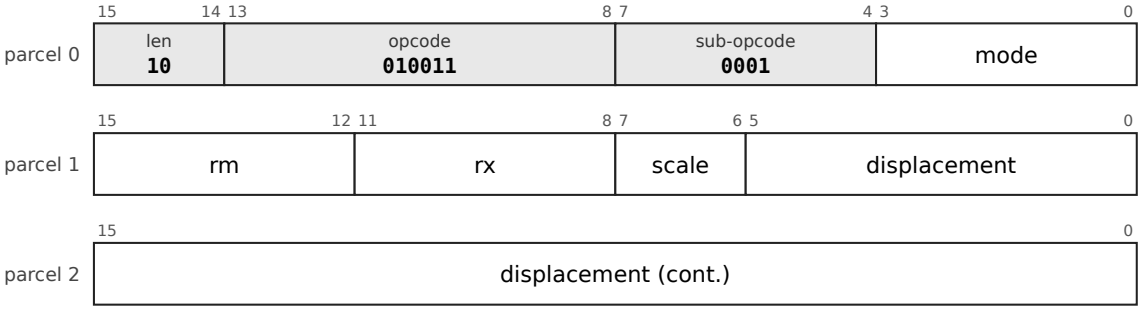
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

COM: Complement

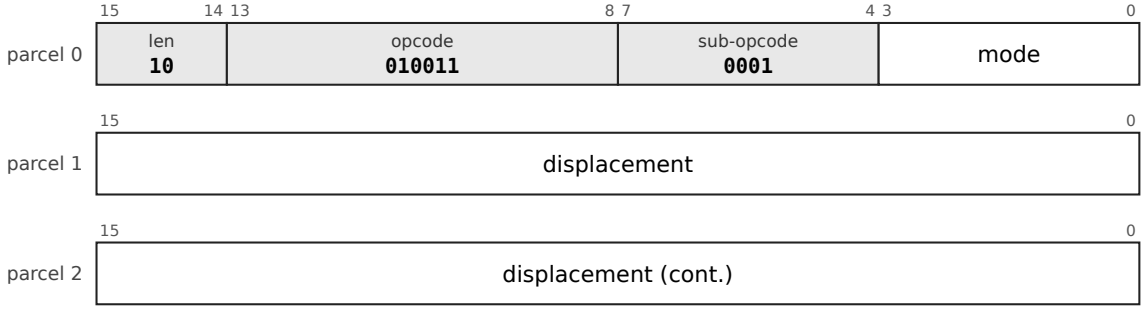
COM, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



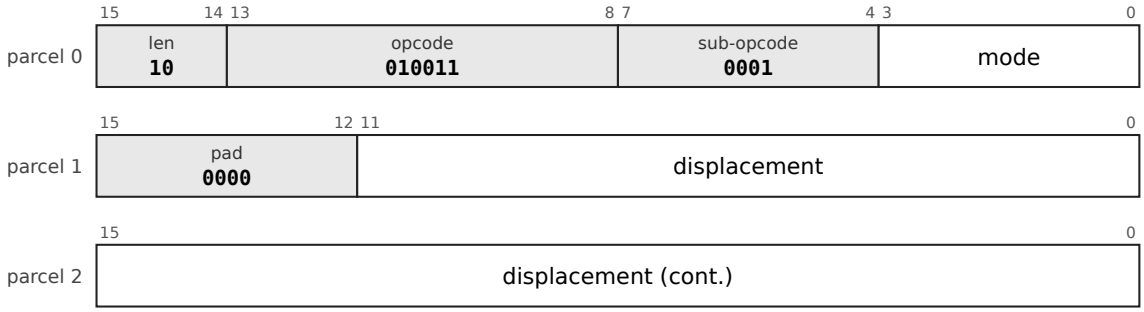
COM, opcode 010011, three parcels (modes index) (6 bytes)



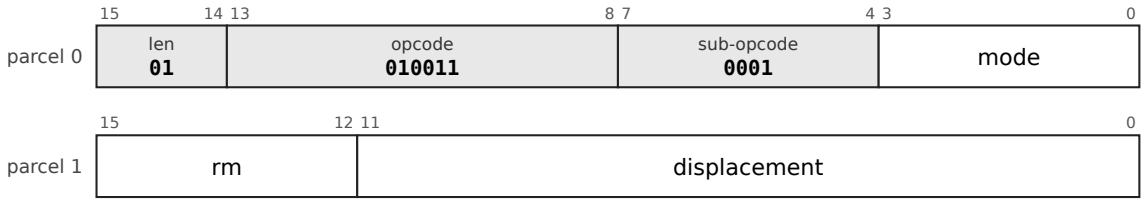
COM, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



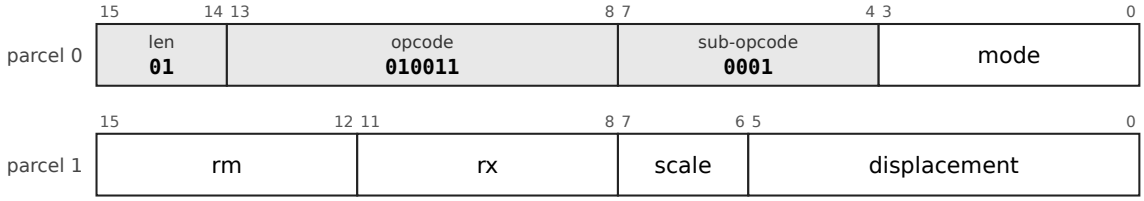
COM, opcode 010011, three parcels (modes dp) (6 bytes)



COM, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



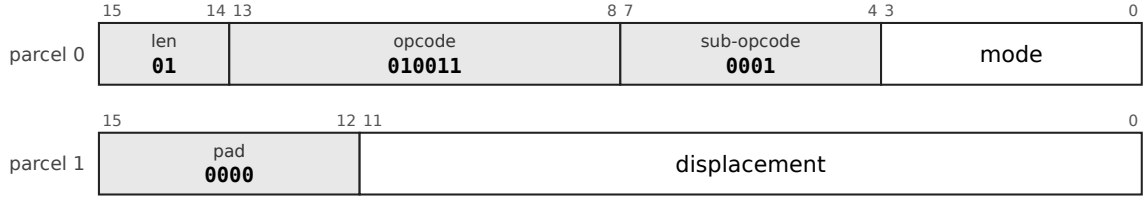
COM, opcode 010011, two parcels (modes index) (4 bytes)



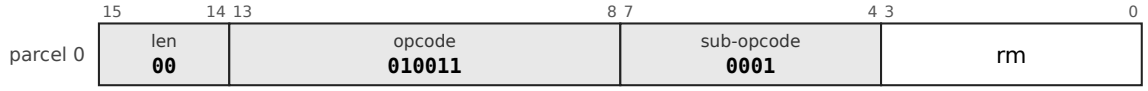
COM, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



COM, opcode 010011, two parcels (modes dp) (4 bytes)



COM, opcode 010011, one parcel (2 bytes)

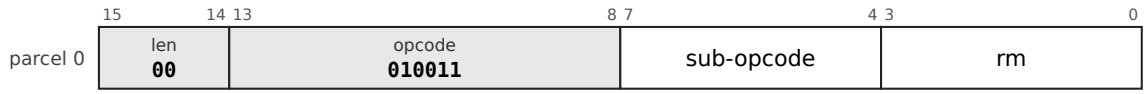


Mnemonics: `COM`

Syntax

```
COM rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`COM`	`010011 (0x13)`	`0001 (0x1)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
rm ← NOT rm
```

Flags `N` and `Z` from the result, `V` = 0, `C` = 1 (the PDP-11 rule).

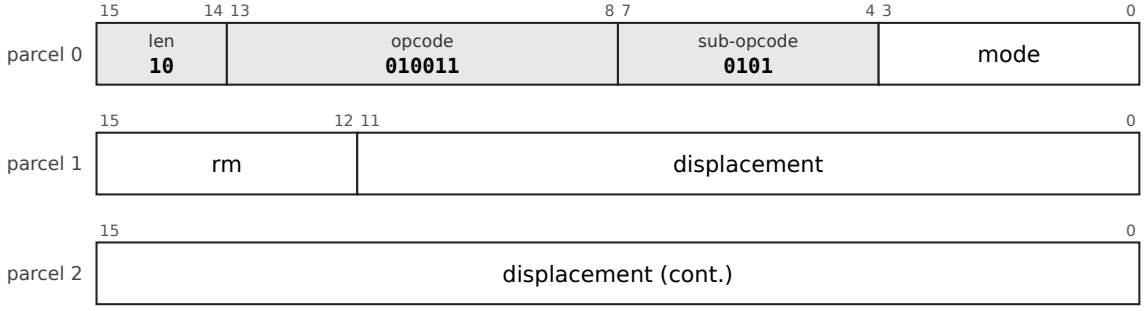
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

A two-byte instruction for a register operand.

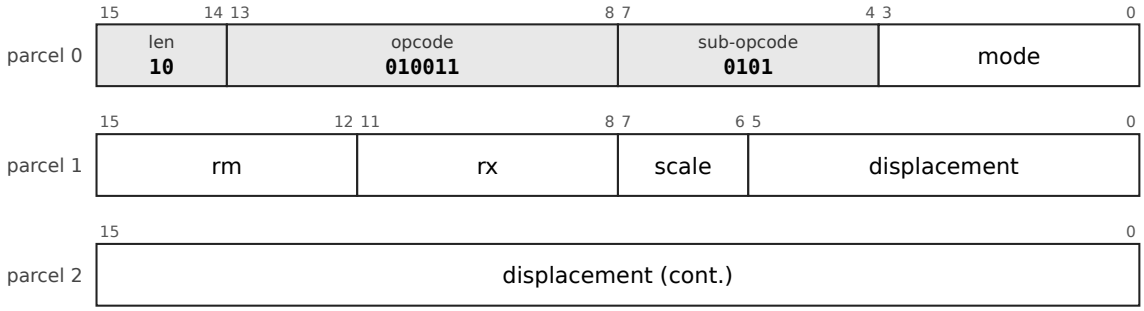
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

TST: Test

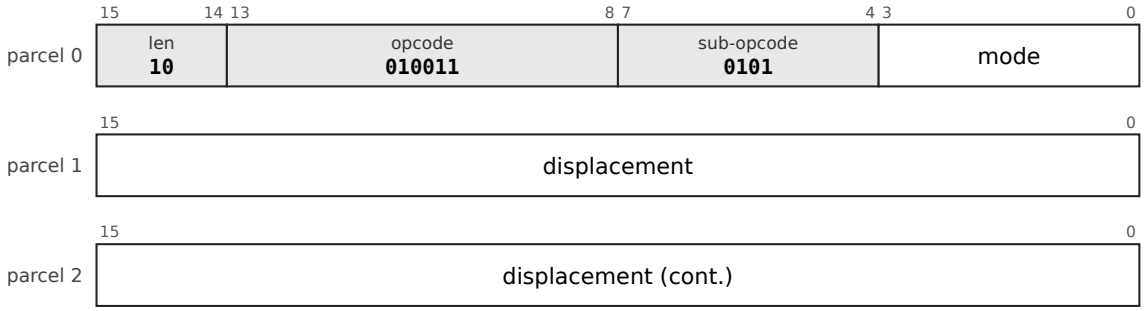
TST, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



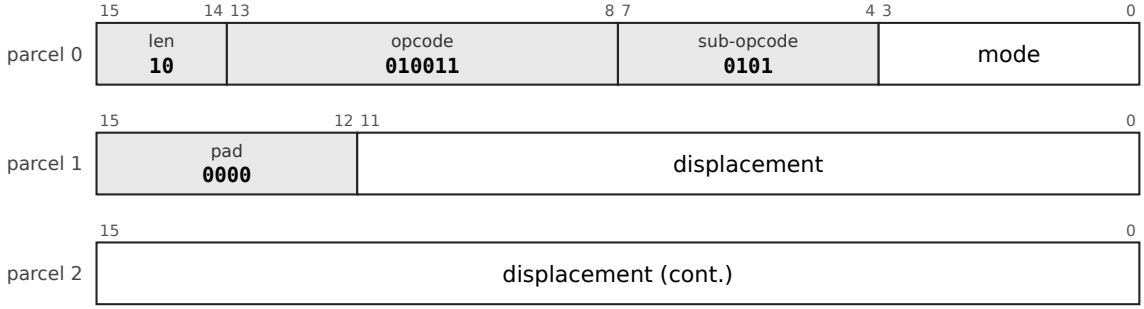
TST, opcode 010011, three parcels (modes index) (6 bytes)



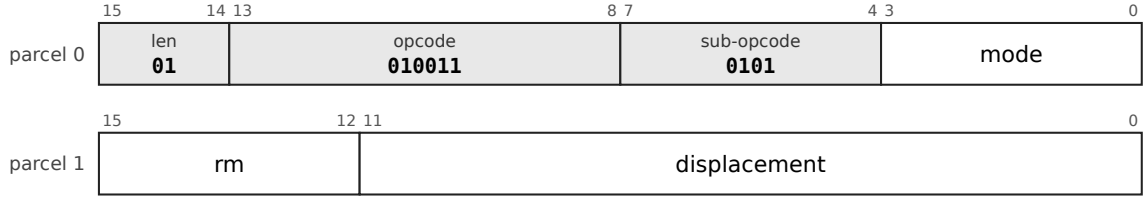
TST, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



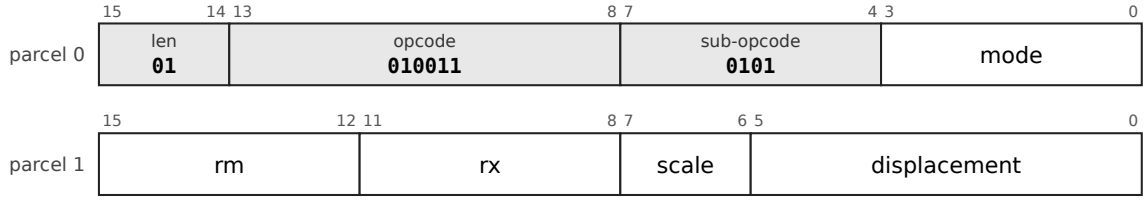
TST, opcode 010011, three parcels (modes dp) (6 bytes)



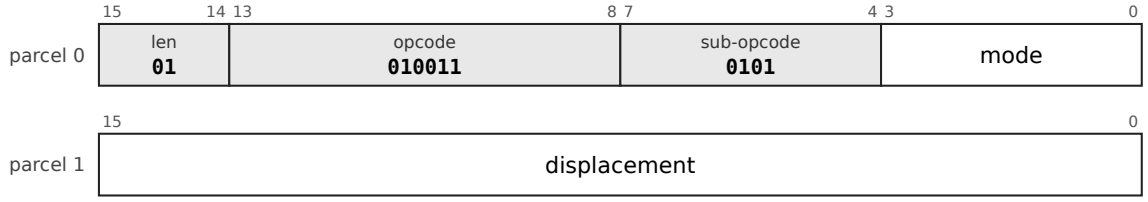
TST, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



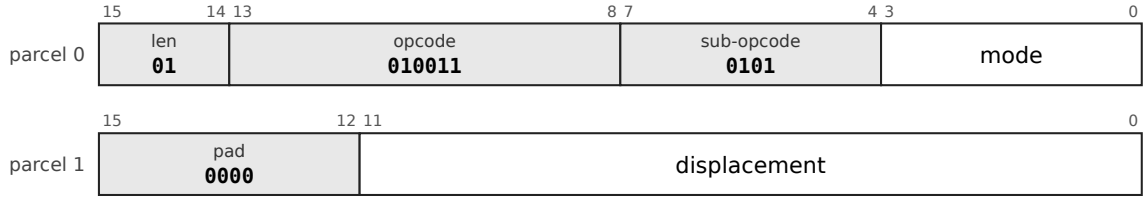
TST, opcode 010011, two parcels (modes index) (4 bytes)



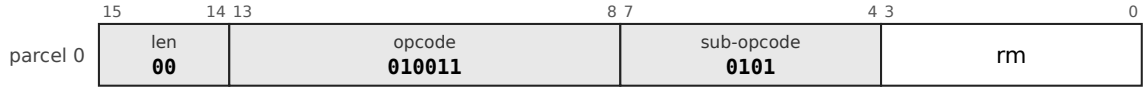
TST, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



TST, opcode 010011, two parcels (modes dp) (4 bytes)



TST, opcode 010011, one parcel (2 bytes)



Mnemonics: TST

Syntax

```
TST rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
TST	010011 (0x13)	0101 (0x5)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
set flags from rm; nothing is written
```

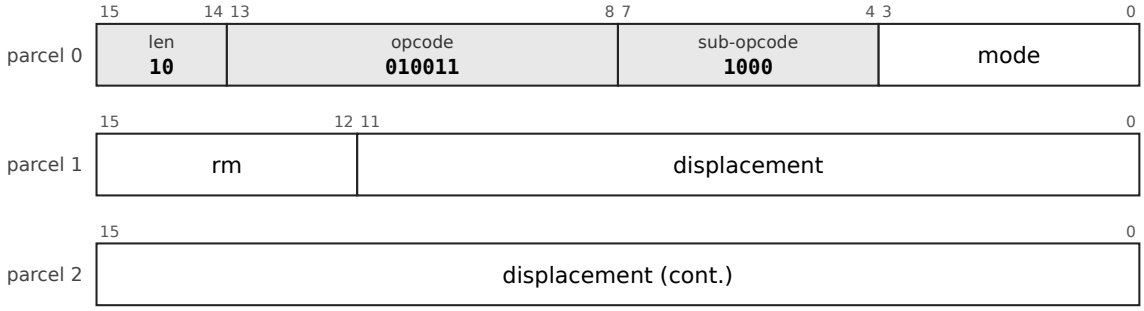
Flags `N` and `Z` from the operand, `V` = 0, `C` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

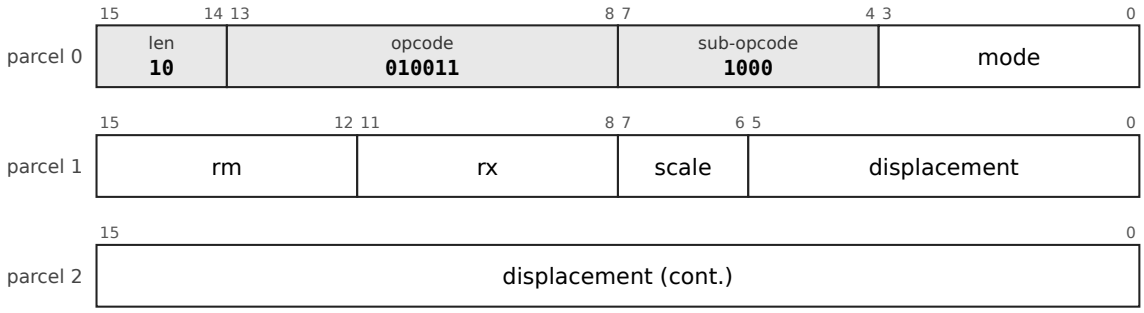
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

ASL: Arithmetic shift left by one

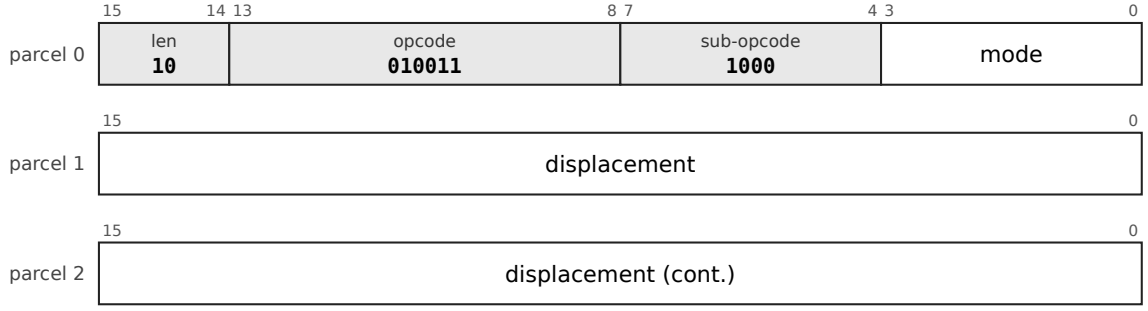
ASL, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



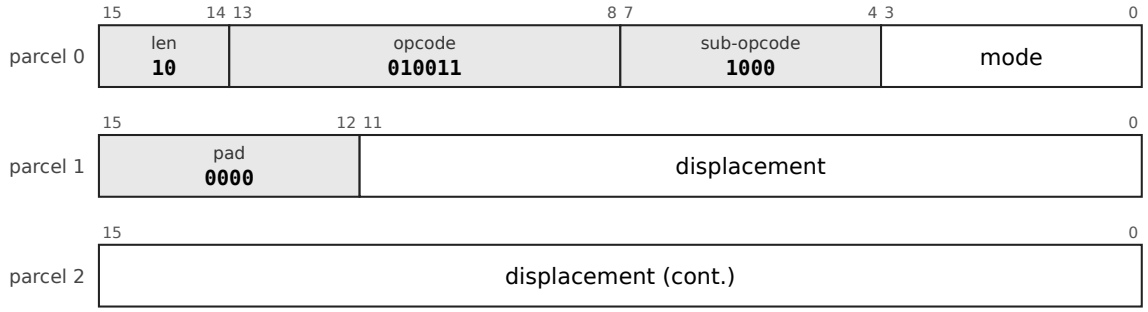
ASL, opcode 010011, three parcels (modes index) (6 bytes)



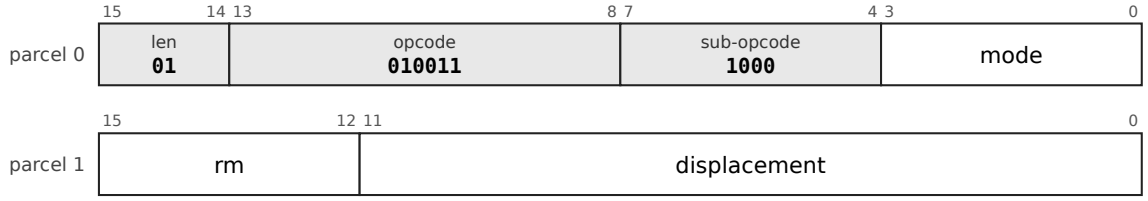
ASL, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



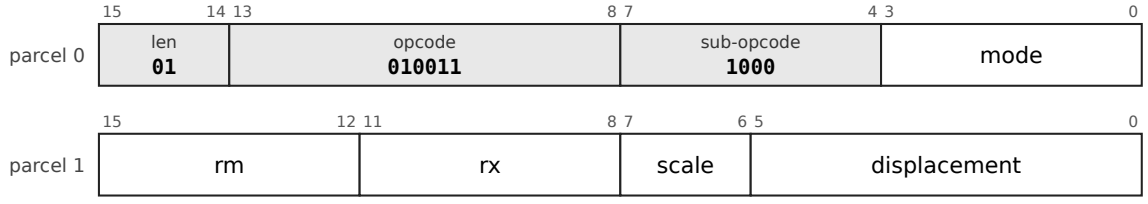
ASL, opcode 010011, three parcels (modes dp) (6 bytes)



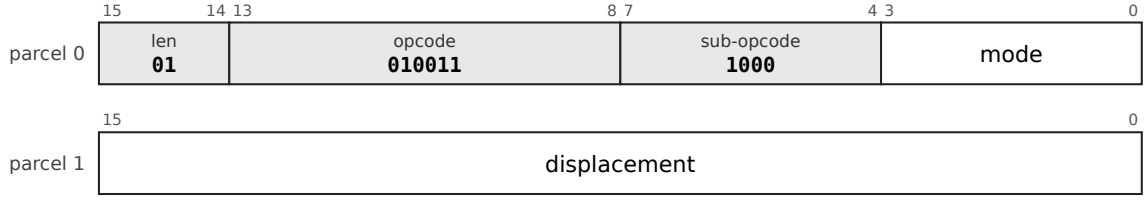
ASL, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



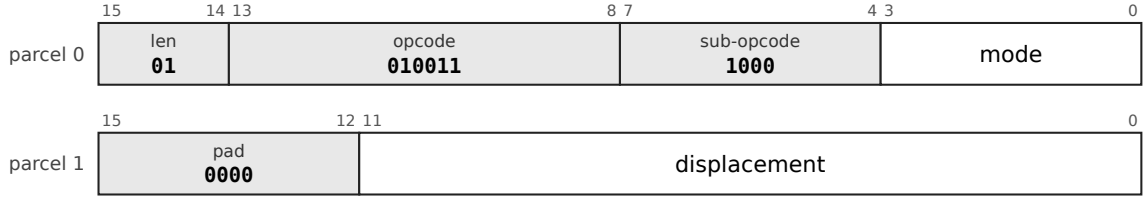
ASL, opcode 010011, two parcels (modes index) (4 bytes)



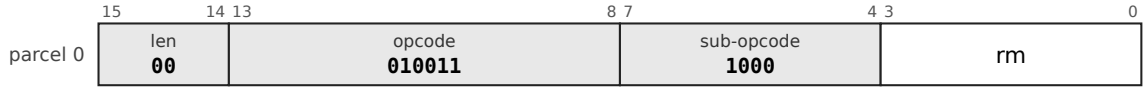
ASL, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



ASL, opcode 010011, two parcels (modes dp) (4 bytes)



ASL, opcode 010011, one parcel (2 bytes)

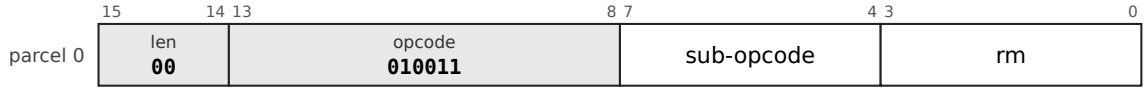


Mnemonics: ASL

Syntax

```
ASL rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
ASL	010011 (0x13)	1000 (0x8)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

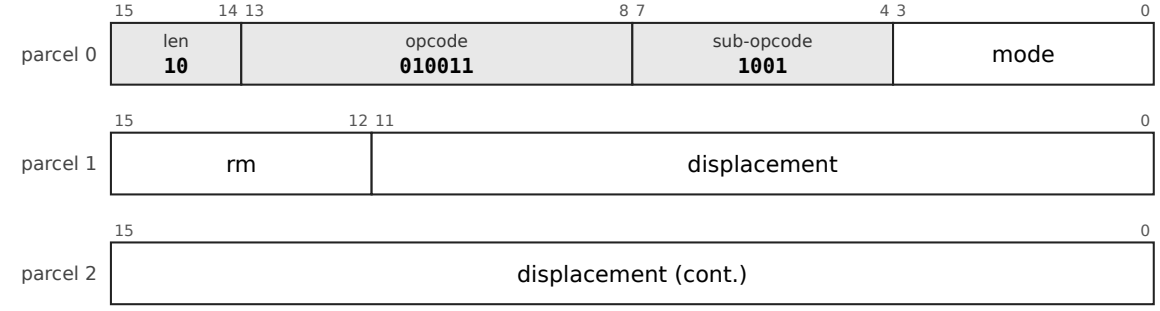
Operation

```
C ← rm[63] ; rm ← rm << 1
```

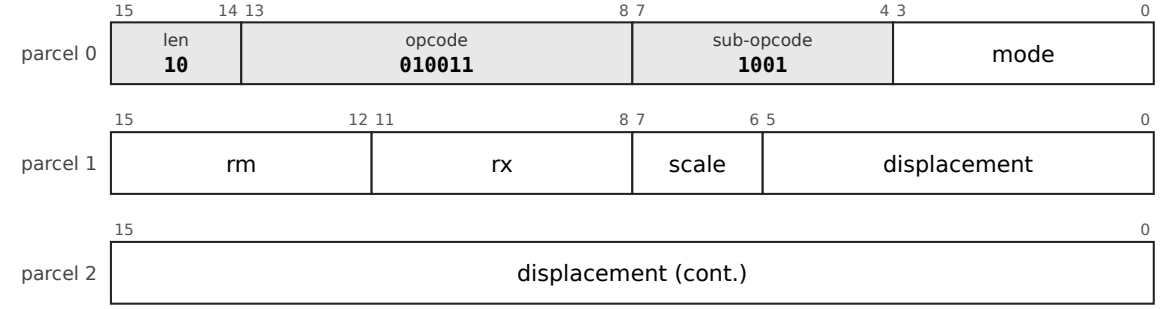
Flags N and Z from the result, C the bit shifted out of bit 63, V = N xor C (set if the sign changed: the PDP-11 rule).
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.
The by-one form of SHL in two bytes.
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

ASR: Arithmetic shift right by one

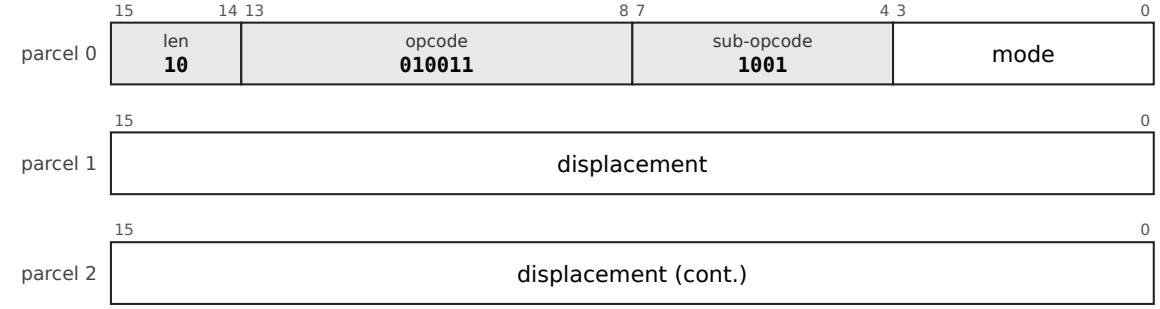
ASR, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



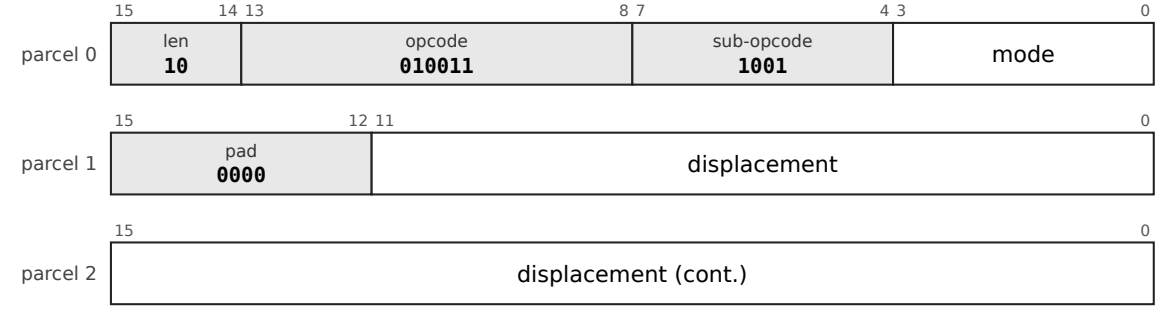
ASR, opcode 010011, three parcels (modes index) (6 bytes)



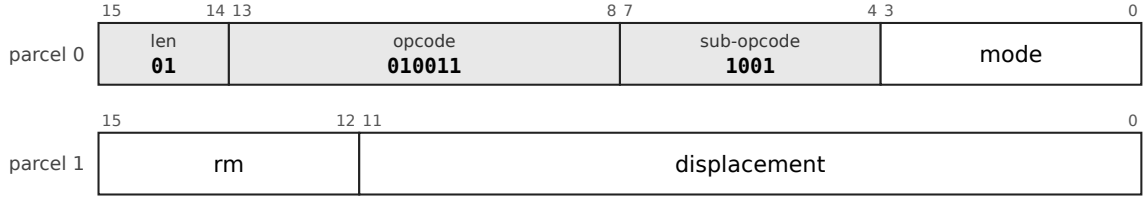
ASR, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



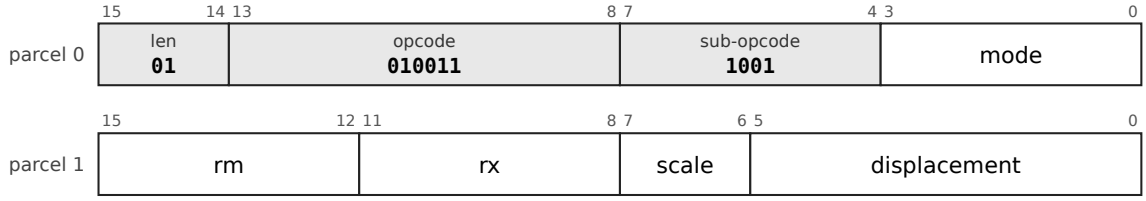
ASR, opcode 010011, three parcels (modes dp) (6 bytes)



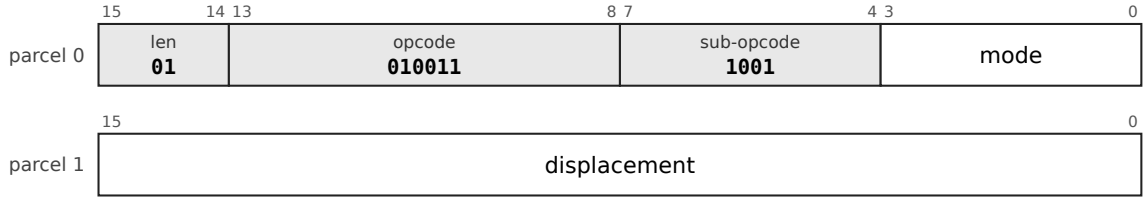
ASR, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



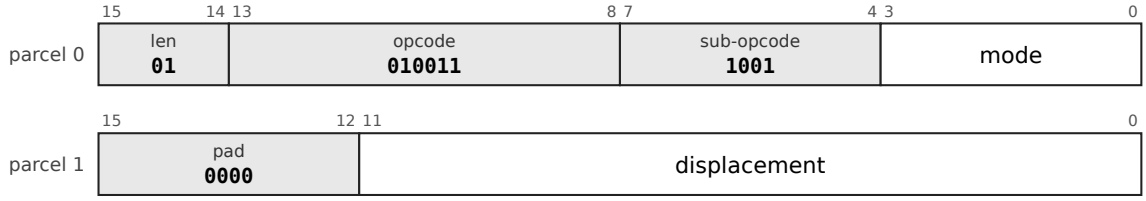
ASR, opcode 010011, two parcels (modes index) (4 bytes)



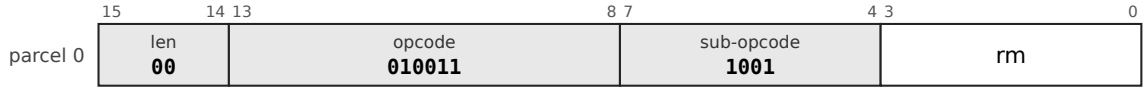
ASR, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



ASR, opcode 010011, two parcels (modes dp) (4 bytes)



ASR, opcode 010011, one parcel (2 bytes)

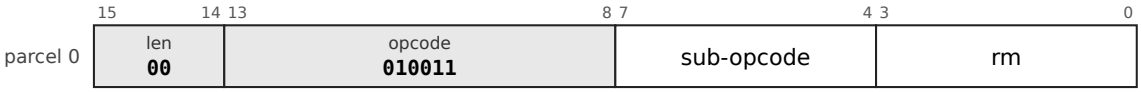


Mnemonics: `ASR`

Syntax

```
ASR rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
ASR	010011 (0x13)	1001 (0x9)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
C ← rm[0] ; rm ← rm >>s 1
```

Flags N and Z from the result, C the bit shifted out of bit 0, V = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The by-one form of SAR in two bytes.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

9.9 Bits, bytes and text

9.9.1 The bit-and-field escape provides CLZ, CTZ, CLO, CTO (count leading and trailing zeros and ones), POPCNT, BSWAP, BSWAPW, BSWAPL and BITREV. *(opcode `001100 (0x0C)`)*

9.9.2 EXTB and EXTZB extract a bitfield at a position and width, sign- or zero-extended, and INSB inserts one; the position and width are in the third parcel, so these are six bytes. *(D)*

9.9.3 XNORPOP computes the population count of the exclusive-nor of two registers. *(D)*

9.9.4 CLMULL and CLMULH are the low and high halves of a carry-less multiply; no polynomial is built into the architecture. *(D)*

9.9.5 UTF8LEN, UTF8DEC and UTF8ENC find the length of, decode and encode a UTF-8 sequence, and reject overlong forms, surrogates, out-of-range values and bad continuation bytes. *(D)*

Commentary. **Precedent.** The PDP-11 had SWAB; the VAX had bit-field instructions and a table-driven CRC; x86 added POPCNT, LZCNT and PCLMULQDQ late, and committed CRC32 to one polynomial. **Trade-offs.** A bit unit is one shared structure: POPCNT, XNORPOP and a garbage collector's mark-bitmap scan all want the same 64-input adder tree, and CLZ and its relatives want a priority encoder over the same operand. **Improvements.** A carry-less multiply stands in for a CRC instruction: no polynomial is baked in, and the same instruction serves CRC, the Galois-counter-mode authentication used in TLS, and hashing. Every UTF-8 security hole of the past is three comparators, which an instruction cannot omit and software often does. **Benefits.** The bit unit and the UTF-8 decoder together cost less than 5% of the die. **Modern comparables.** RISC-V's bit-manipulation and carry-less-multiply extensions, AArch64's CLZ, RBIT, REV and PMULL, and x86's PCLMULQDQ. **Use.** POPCNT for bitsets and Hamming distance; XNORPOP for binary-weight neural networks and bitset intersection; CLMUL for CRC, GCM and fingerprinting; UTF8* for text validation in parsers and kernels. **Impact.** *Kernel:* CTZ and CLZ find the first set bit in a run-queue or free-page bitmap; POPCNT counts bits in bitmaps; BSWAP converts network byte order. *Compiler:* The compiler maps `__builtin_clz`, `__builtin_popcount` and `__builtin_bswap` directly, and a carry-less multiply is the primitive for CRC and hashing code. *Hardware:* One bit unit holds a shared 64-input adder tree and a priority encoder, and the UTF-8 comparators are about 300 lookup tables.

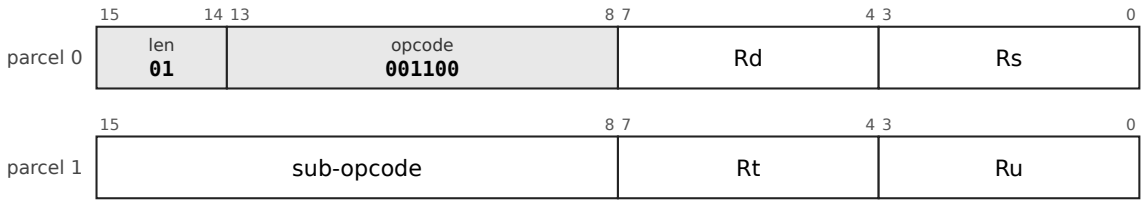
COUNT: Count leading and trailing zeros and ones

Mnemonics: CLZ, CTZ, CLO, CTO

Syntax

```
CLZ Rs, Rd
CTZ Rs, Rd
CLO Rs, Rd
CTO Rs, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
`CLZ`	`001100 (0x0C)`	`00000000 (0x00)`
`CTZ`	`001100 (0x0C)`	`00000001 (0x01)`
`CLO`	`001100 (0x0C)`	`00000010 (0x02)`
`CTO`	`001100 (0x0C)`	`00000011 (0x03)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2`.

Operation

```
CLZ: Rd ← number of leading zero bits of Rs        (64 if Rs = 0)
CTZ: Rd ← number of trailing zero bits of Rs       (64 if Rs = 0)
CLO: Rd ← number of leading one bits of Rs         (64 if Rs = all ones)
CTO: Rd ← number of trailing one bits of Rs        (64 if Rs = all ones)
```

Flags None written.

Exceptions None.

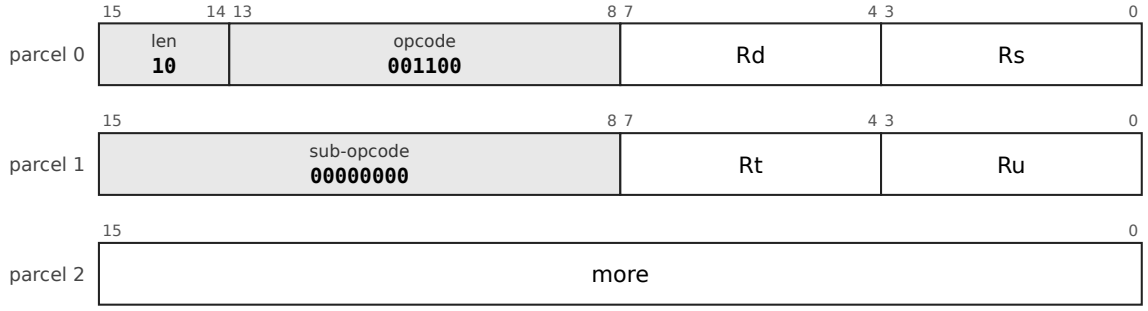
CTZ is the VAX FFS. Bit manipulation is core on a machine that does filesystem and bignum work, not an extension.

Why it is here. BESC is the bit-manipulation group: bit manipulation is not an extension on a machine that does filesystem work, bignum arithmetic and the like. The counts are the VAX FFS and FFC idiom (find first set, find first clear). LoongArch is the model for putting them in the base: it has CLO, CLZ, CTO, CTZ there, which is the decisive argument for giving BESC a top-level opcode. The compiler emits none of them, since C has no operator for them; the corpus shows zero uses.

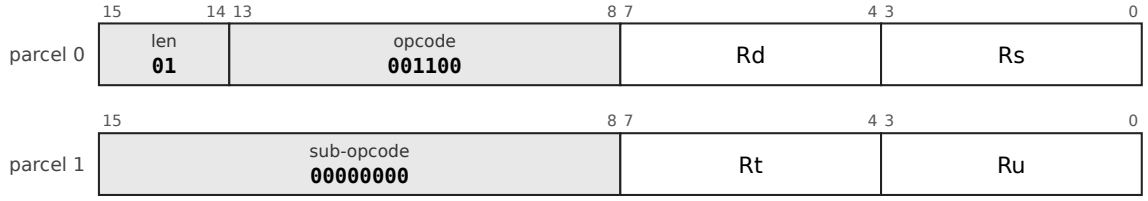
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

CLZ

CLZ, opcode 001100, three parcels, sub-opcode 00000000 (6 bytes)



CLZ, opcode 001100, two parcels, sub-opcode 00000000 (4 bytes)



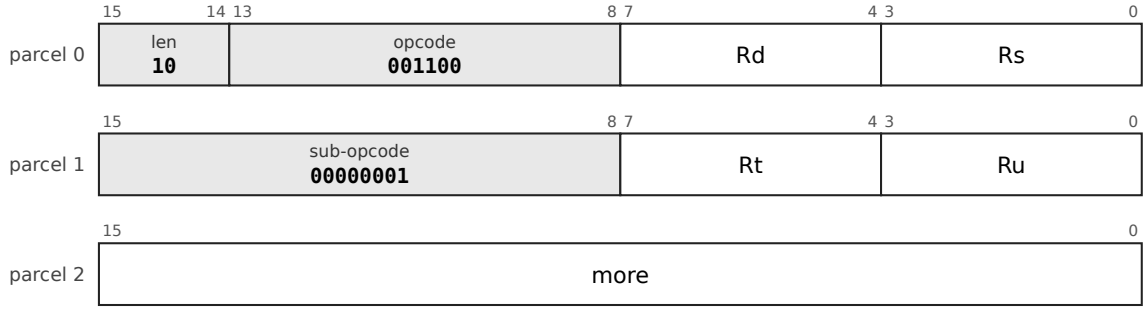

```
CLZ Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
CLZ	001100 (0x0C)	00000000 (0x00)

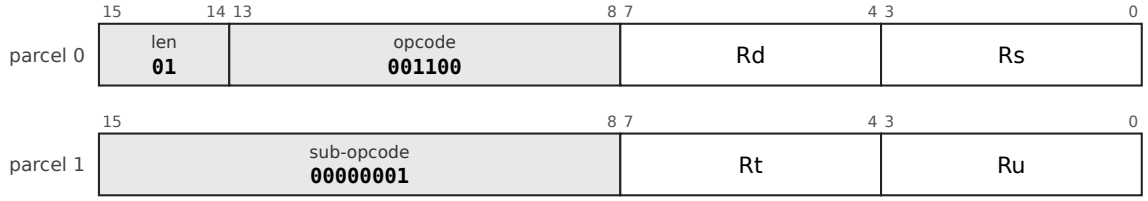
Operation, flags and exceptions as for the entry COUNT above; 2 layout forms, each a chart of its own (above).

CTZ

CTZ, opcode 001100, three parcels, sub-opcode 00000001 (6 bytes)



CTZ, opcode 001100, two parcels, sub-opcode 00000001 (4 bytes)



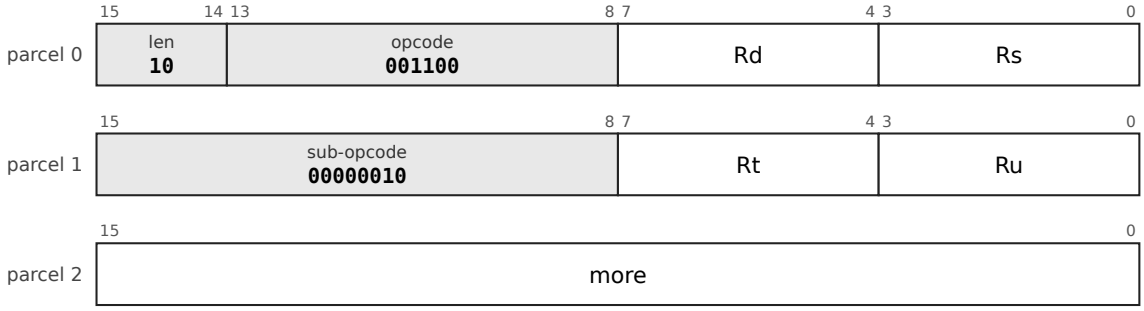

```
CTZ Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
CTZ	001100 (0x0C)	00000001 (0x01)

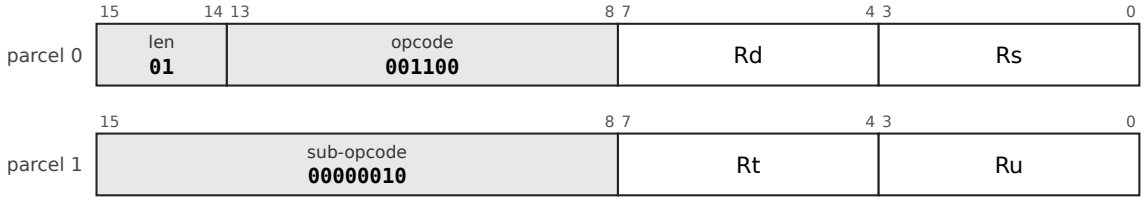
Operation, flags and exceptions as for the entry COUNT above; 2 layout forms, each a chart of its own (above).

CLO

CLO, opcode 001100, three parcels, sub-opcode 00000010 (6 bytes)



CLO, opcode 001100, two parcels, sub-opcode 00000010 (4 bytes)



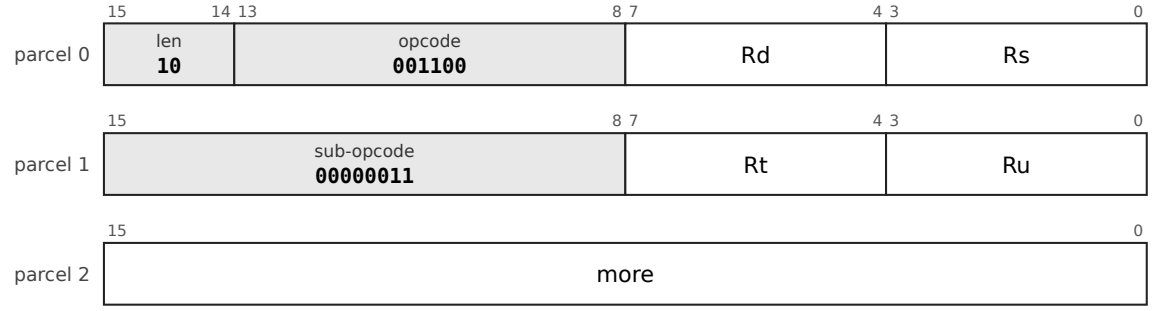

```
CLO Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
CLO	001100 (0x0C)	00000010 (0x02)

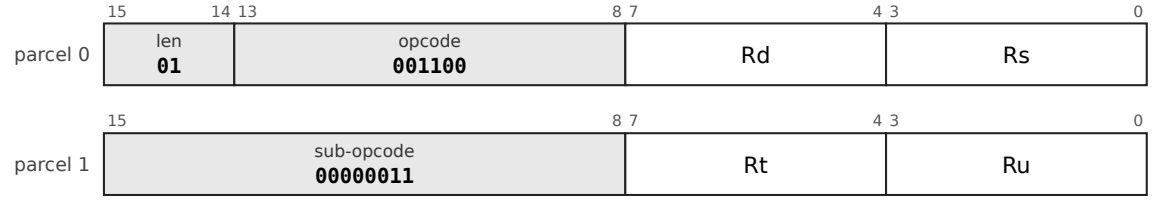
Operation, flags and exceptions as for the entry COUNT above; 2 layout forms, each a chart of its own (above).

CTO

CTO, opcode 001100, three parcels, sub-opcode 00000011 (6 bytes)



CTO, opcode 001100, two parcels, sub-opcode 00000011 (4 bytes)



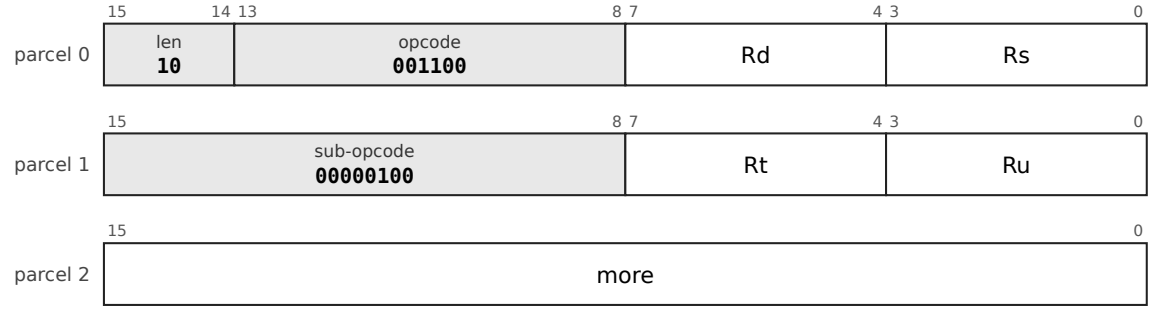

```
CTO Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
CTO	001100 (0x0C)	00000011 (0x03)

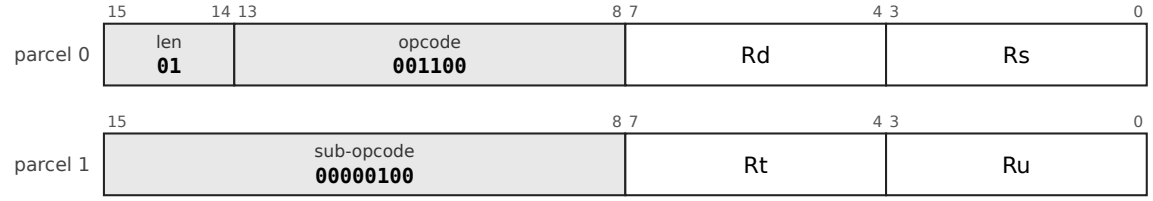
Operation, flags and exceptions as for the entry COUNT above; 2 layout forms, each a chart of its own (above).

POPCNT: Population count

POPCNT, opcode 001100, three parcels, sub-opcode 00000100 (6 bytes)



POPCNT, opcode 001100, two parcels, sub-opcode 00000100 (4 bytes)

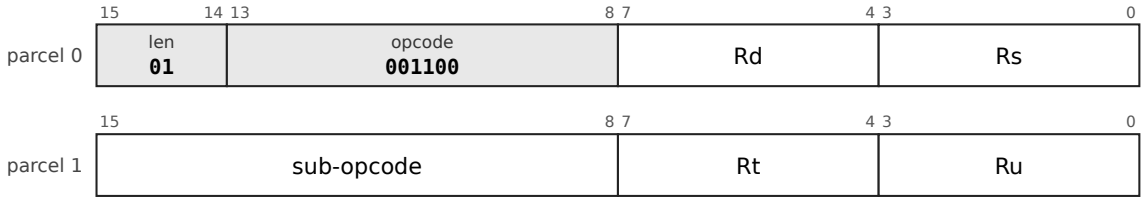


Mnemonics: POPCNT

Syntax

```
POPCNT Rs, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
POPCNT	001100 (0x0C)	00000100 (0x04)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2`.

Operation

```
Rd ← number of one bits in Rs
```

Flags None written.

Exceptions None.

Why it is here. One of the counts of `BESC`; it is also the building block of `XNORPOP`. A compiler needs `__builtin_popcount` to emit it, so the corpus shows no use (see `COUNT`).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

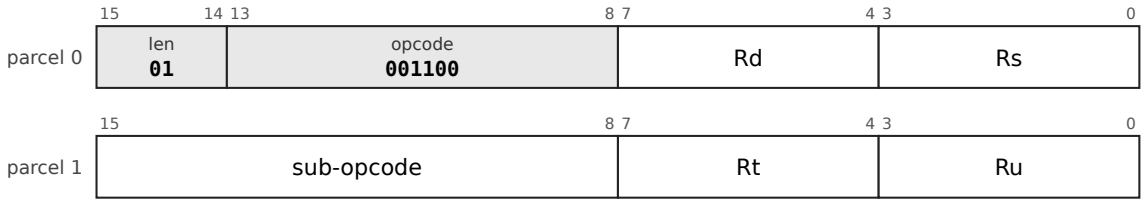
BSWAP: Byte swap

Mnemonics: `BSWAP`, `BSWAPW`, `BSWAPL`

Syntax

```
BSWAP Rs, Rd
BSWAPW Rs, Rd
BSWAPL Rs, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
BSWAP	001100 (0x0C)	00000101 (0x05)
BSWAPW	001100 (0x0C)	00000110 (0x06)
BSWAPL	001100 (0x0C)	00000111 (0x07)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2`.

Operation

```
BSWAP : Rd ← Rs with all 8 bytes reversed
BSWAPW: Rd ← Rs with the bytes of each 16-bit word reversed
BSWAPL: Rd ← Rs with the bytes of each 32-bit long reversed
```

Flags None written.

Exceptions None.

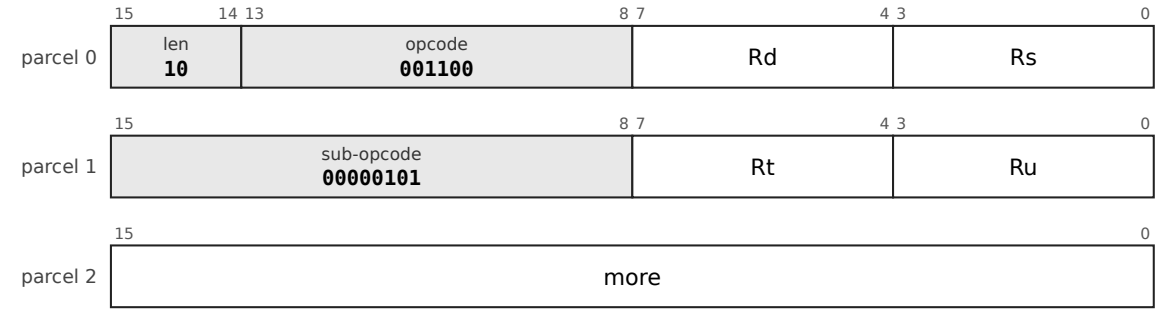
The generalisation of the PDP-11 `SWAB` to 64 bits.

Why it is here. The PDP-11 `SWAB` swapped the bytes of a word; `BSWAP` generalises it to 64 bits, with word and long variants. `SWAB` is therefore subsumed, not lost (9.18).

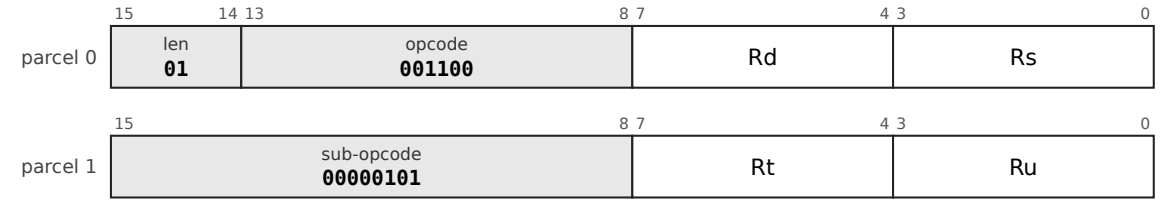
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

BSWAP

BSWAP, opcode 001100, three parcels, sub-opcode 00000101 (6 bytes)



BSWAP, opcode 001100, two parcels, sub-opcode 00000101 (4 bytes)



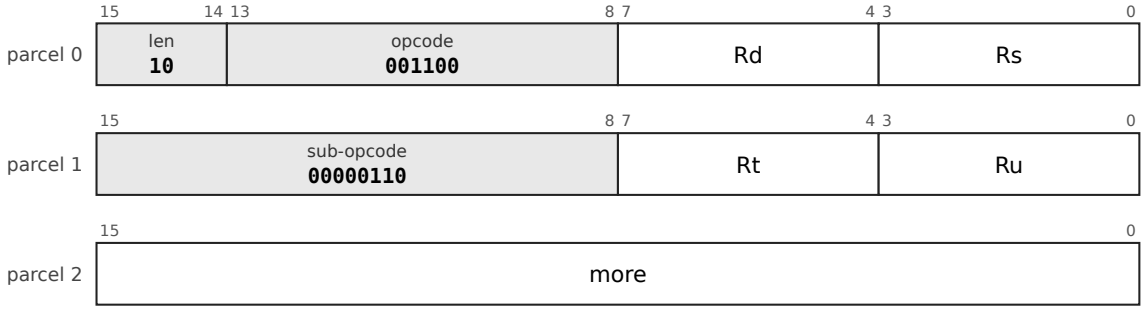

```
BSWAP Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
BSWAP	001100 (0x0C)	00000101 (0x05)

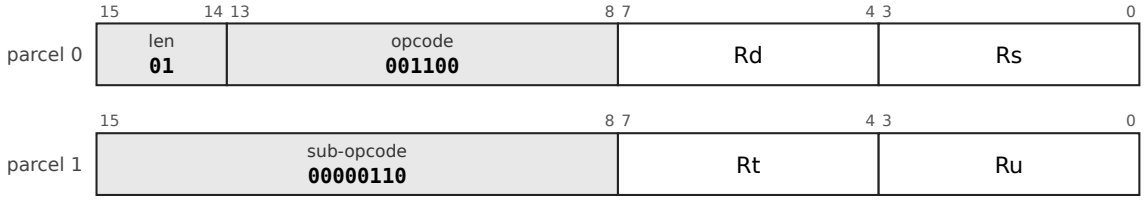
Operation, flags and exceptions as for the entry BSWAP above; 2 layout forms, each a chart of its own (above).

BSWAPW

BSWAPW, opcode 001100, three parcels, sub-opcode 00000110 (6 bytes)



BSWAPW, opcode 001100, two parcels, sub-opcode 00000110 (4 bytes)



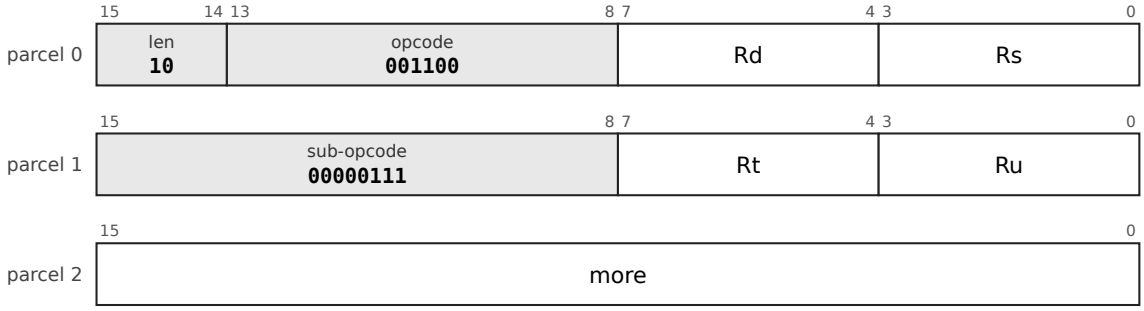

```
BSWAPW Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
BSWAPW	001100 (0x0C)	00000110 (0x06)

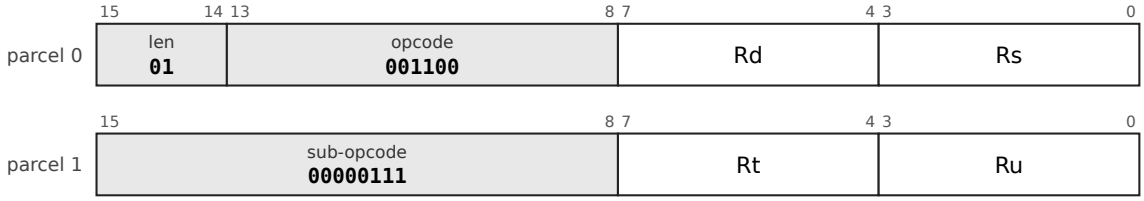
Operation, flags and exceptions as for the entry BSWAP above; 2 layout forms, each a chart of its own (above).

BSWAPL

BSWAPL, opcode 001100, three parcels, sub-opcode 00000111 (6 bytes)



BSWAPL, opcode 001100, two parcels, sub-opcode 00000111 (4 bytes)



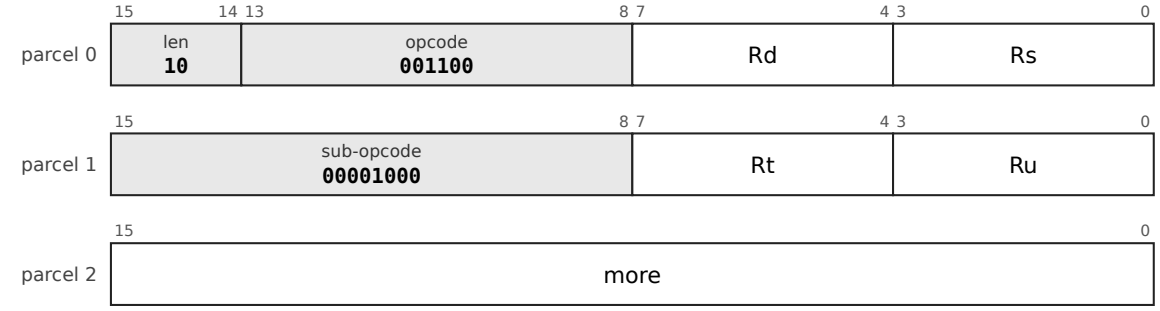

```
BSWAPL Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
BSWAPL	001100 (0x0C)	00000111 (0x07)

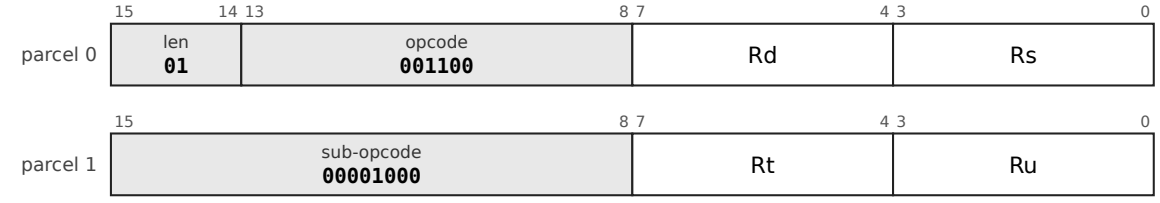
Operation, flags and exceptions as for the entry BSWAP above; 2 layout forms, each a chart of its own (above).

BITREV: Reverse the bits

BITREV, opcode 001100, three parcels, sub-opcode 00001000 (6 bytes)



BITREV, opcode 001100, two parcels, sub-opcode 00001000 (4 bytes)

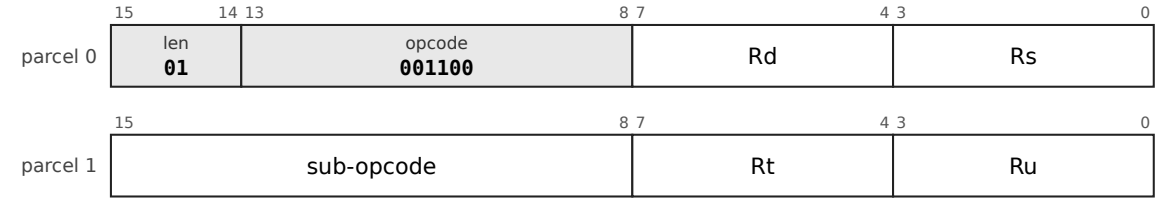


Mnemonics: `BITREV`

Syntax

```
BITREV Rs, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
BITREV	001100 (0x0C)	00001000 (0x08)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2`.

Operation

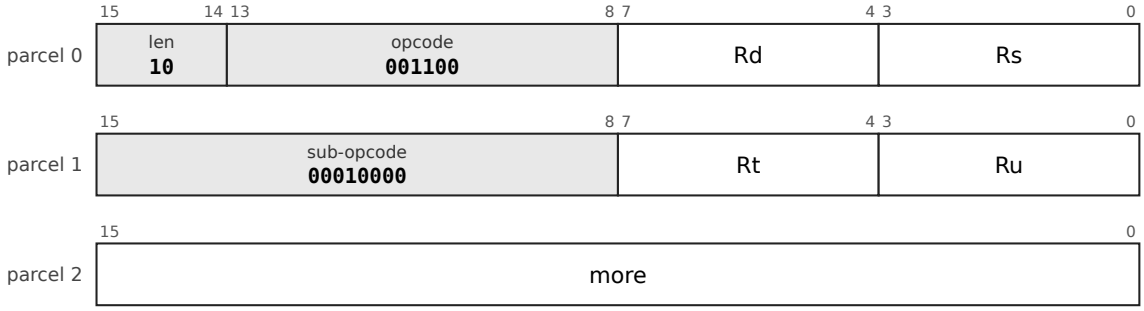
```
Rd ← Rs with bit i moved to bit 63-i
```

Flags None written.

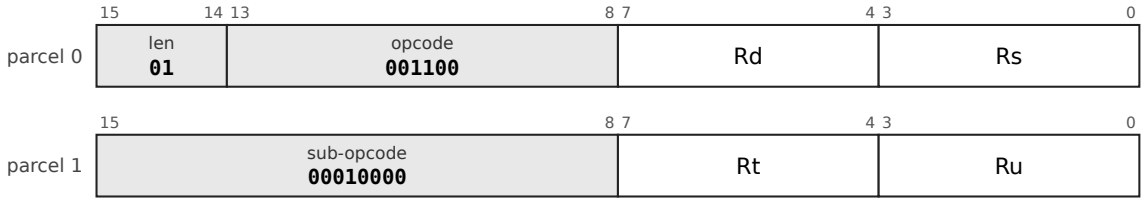
Exceptions None.
Why it is here. Part of the BESC group, borrowed with the other LoongArch bit operations (REVB, REVH, BITREV). No workload from the corpus is recorded.
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

ROR: Rotate right by a register count

ROR, opcode 001100, three parcels, sub-opcode 00010000 (6 bytes)



ROR, opcode 001100, two parcels, sub-opcode 00010000 (4 bytes)

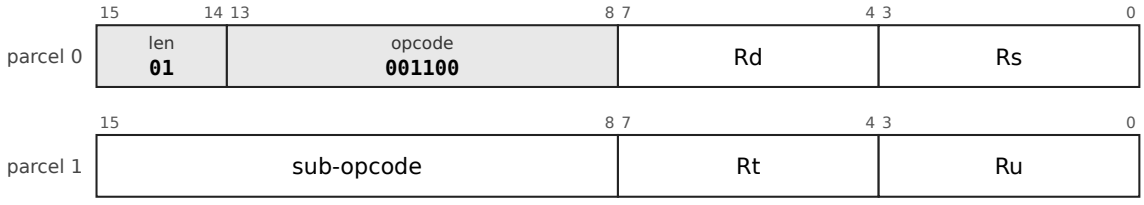


Mnemonics: ROR

Syntax

```
ROR Ra, Rb, Rd
```

Encoding



The chart is the layout r20-besc-1, Bits and fields (BESC), two parcels (len = 01): From len = 01. In this group low is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
ROR	001100 (0x0C)	00010000 (0x10)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-besc-1, r20-besc-2.

Operation

```
Rd ← Ra rotated right by (Rb mod 64)
```

Flags None written (rotates write no flags).

Exceptions None.

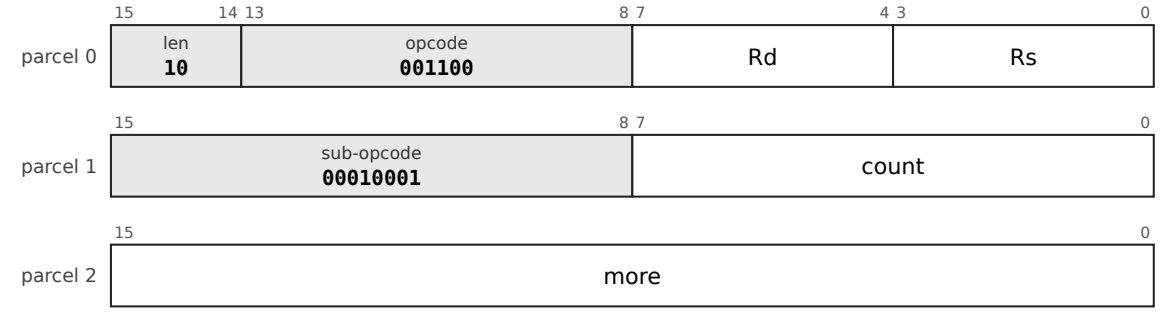
There is no rotate left instruction: `ROL rs, #n, rd` is an assembler spelling of `RORI rs, #(64-n), rd`. No rotate is through carry; multi-word shifting uses `DSHR`.

Why it is here. One rotate direction suffices and `ROR` is the one to keep: `ROL x, #n` is exactly `ROR x, #(64-n)`, and a funnel shifter yields `ROR` directly. No rotate is through carry (see `DSHR`). `ROR` is in `BESC`, not the ALU group: cryptographic code rotates by compile-time constants, so a four-byte immediate form is needed either way, and the register-count form has almost no users. Rotates write no flags.

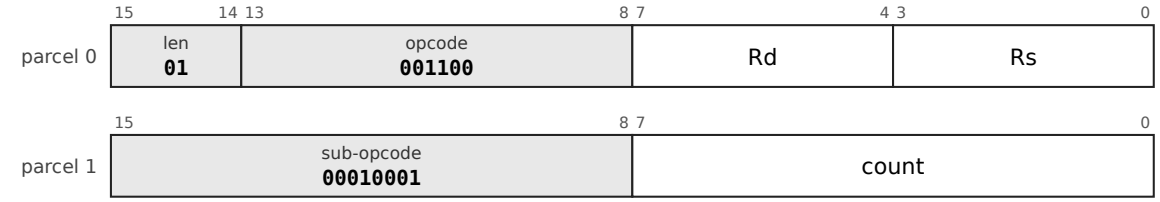
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

RORI: Rotate right by an immediate

RORI, opcode 001100, three parcels, sub-opcode 00010001 (6 bytes)



RORI, opcode 001100, two parcels, sub-opcode 00010001 (4 bytes)

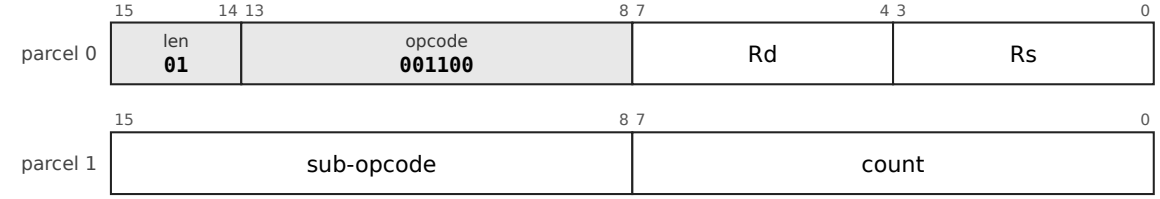


Mnemonics: `RORI` (aliases: `ROL`)

Syntax

```
RORI Rs, #n, Rd
ROL Rs, #n, Rd        ; assembles to RORI Rs, #(64-n), Rd
```

Encoding



The chart is the layout `r20-besc-1-2`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
RORI	001100 (0x0C)	00010001 (0x11)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1-2`, `r20-besc-2-2`.

Operation

```
Rd ← Rs rotated right by (n mod 64)
```

Flags None written.

Exceptions None.

Four bytes: the count is in the low byte of parcel 1.

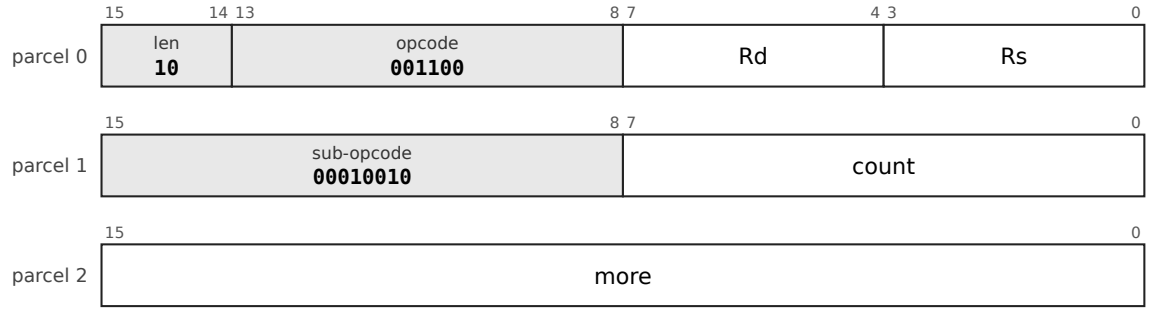
Why it is here. It has its own mnemonic because an immediate count and a register count are different encodings. `ROL Rs, #n, Rd` is accepted by the assembler as `RORI Rs, #(64-n), Rd`: the complement costs nothing at assembly time, so there is no reason to make programmers do it, and a register-count `ROL` is rejected so that one mnemonic stays one instruction.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

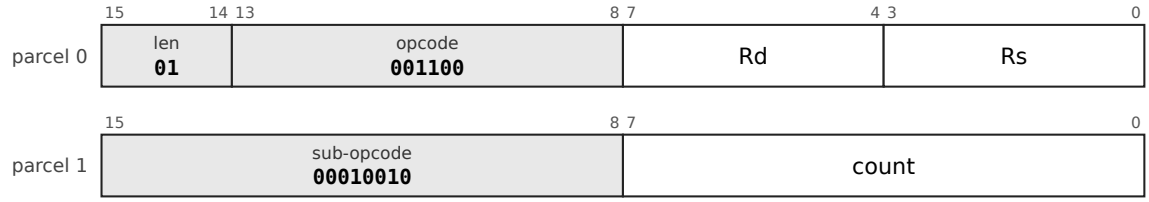
`ROL` is an alias of `RORI`: the assembler writes it as that word.

RORI32: Rotate right by an immediate, 32 bits

RORI32, opcode 001100, three parcels, sub-opcode 00010010 (6 bytes)



RORI32, opcode 001100, two parcels, sub-opcode 00010010 (4 bytes)

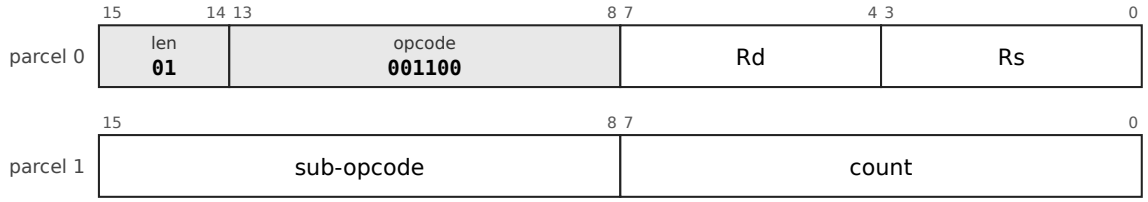


Mnemonics: `RORI32`

Syntax

```
RORI32 Rs, #n, Rd
```

Encoding



The chart is the layout `r20-besc-1-2`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
RORI32	001100 (0x0C)	00010010 (0x12)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1-2`, `r20-besc-2-2`.

Operation

```
Rd ← sext32(ror32(Rs[31:0], n mod 32))
```

Flags None written.

Exceptions None.

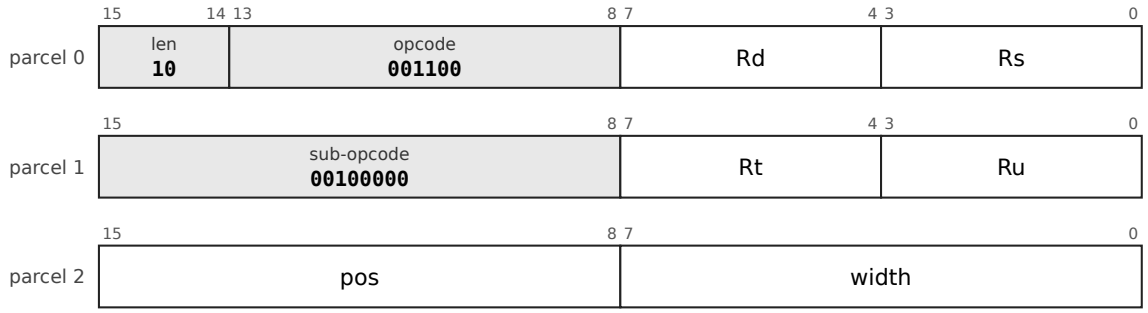
The 32-bit rotate with the result sign-extended like every 32-bit result; the primitive of most hash functions and ciphers.

Why it is here. A 32-bit rotate by a constant, the shape of crypto and hash inner loops, estimated together with `CBCC` at 22 KB (1.3%) of the compiled image. The result is sign-extended like the rest of the 32-bit operations, so that no instruction is needed after it. It removes 44 `MOVL` instructions in the corpus (the demand analysis already removes nearly all of them).

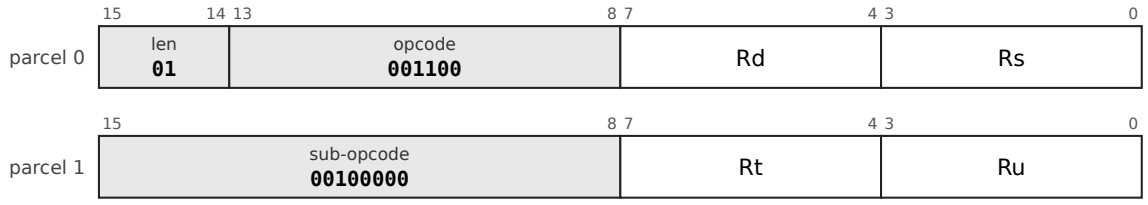
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

DSHR: Double-width funnel shift right

DSHR, opcode 001100, three parcels, sub-opcode 00100000 (6 bytes)



DSHR, opcode 001100, two parcels, sub-opcode 00100000 (4 bytes)

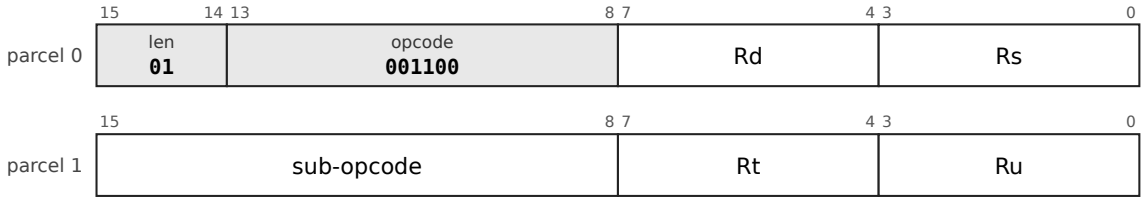


Mnemonics: `DSHR`

Syntax

```
DSHR Ra, Rb, #n, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
DSHR	001100 (0x0C)	00100000 (0x20)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2-3`.

Operation

```
t ← Ra:Rb                    ; 128 bits, Ra the high half
Rd ← (t >> n)[63:0]          ; n = 0..63
```

Flags None written.

Exceptions None.

x86 SHRD, ARM EXTR. Six bytes (the count is in parcel 2). A 128-bit left shift by k is `DSHR hi, hi, lo, #(64-k)` then `SHL #k, lo`.

Why it is here. A double-width funnel shift (x86 SHRD, ARM EXTR), the right primitive for multi-word shifting. Two instructions shift a 128-bit value by any distance, where rotate-through-carry takes one bit per instruction, and that is why no rotate is through carry. It also subsumes ROR when both sources are the same register. It leaves every flag alone.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

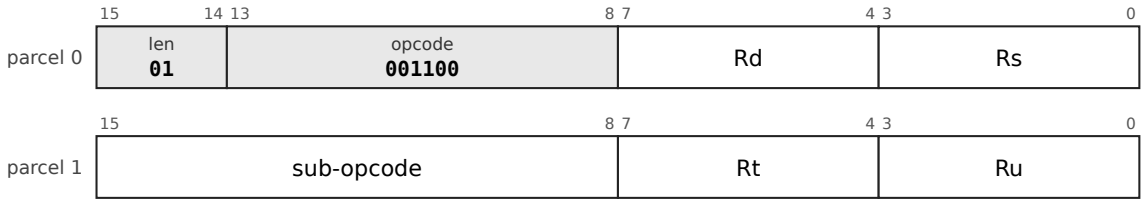
EXTB: Extract a bit field

Mnemonics: EXTB, EXTZB

Syntax

```
EXTB Rs, #pos, #width, Rd        ; sign-extended
EXTZB Rs, #pos, #width, Rd       ; zero-extended
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
EXTB	001100 (0x0C)	00100001 (0x21)

Mnemonic	Opcode	Sub-opcode
EXTZB	001100 (0x0C)	00100010 (0x22)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2-3`.

Operation

```
Rd ← ext((Rs >> pos)[width-1:0])          ; sext for EXTB, zext for EXTZB; width 0 or > 64 means 64
```

Flags None written.

Exceptions None.

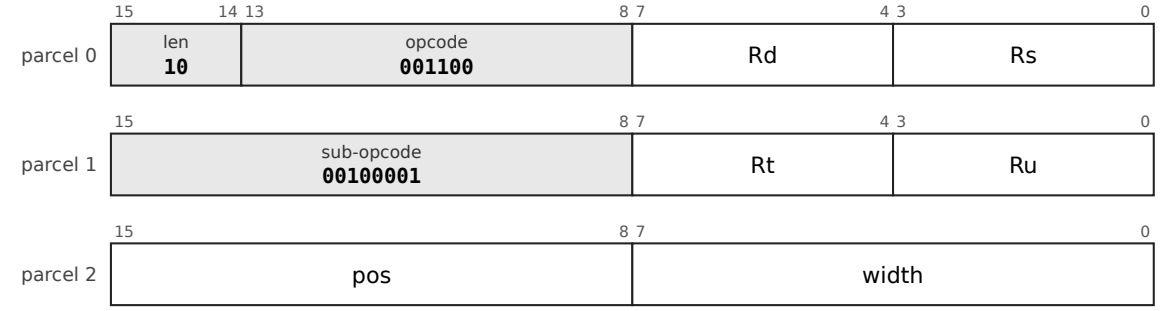
Six bytes (position and width are in parcel 2), so these exist at `len = 10` only.

Why it is here. Bit-field extract and insert are the VAX `EXTV`, `EXTZV` and `INSV` idiom. The compiler does not use them: the shift-pair extraction pattern occurs 151 times in 26 files of the corpus, about 600 bytes at best (0.03%), and there are four true bit-field extracts in all. They are for hand-written protocol and bit-stream code.

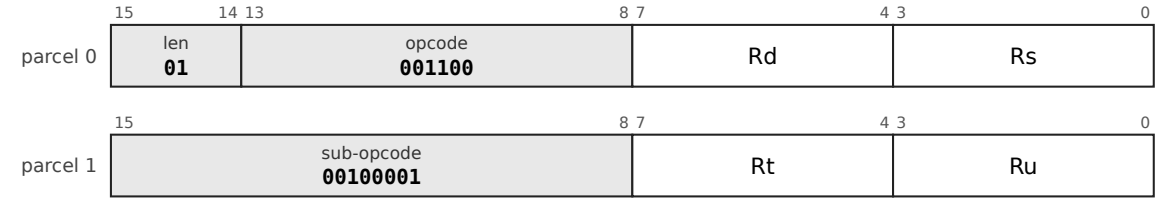
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

EXTB

EXTB, opcode 001100, three parcels, sub-opcode 00100001 (6 bytes)



EXTB, opcode 001100, two parcels, sub-opcode 00100001 (4 bytes)



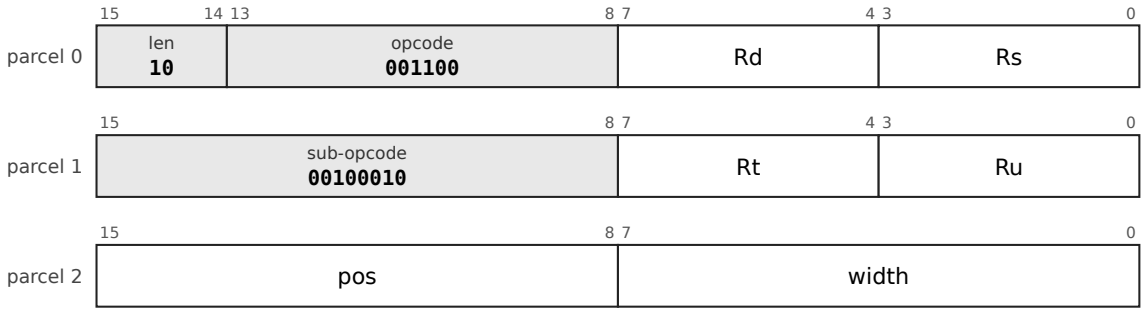

```
EXTB Rs, #pos, #width, Rd
```

Mnemonic	Opcode	Sub-opcode
EXTB	001100 (0x0C)	00100001 (0x21)

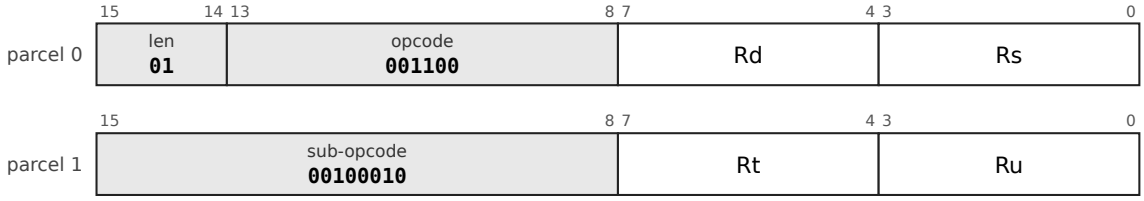
Operation, flags and exceptions as for the entry EXTB above; 2 layout forms, each a chart of its own (above).

EXTZB

EXTZB, opcode 001100, three parcels, sub-opcode 00100010 (6 bytes)



EXTZB, opcode 001100, two parcels, sub-opcode 00100010 (4 bytes)



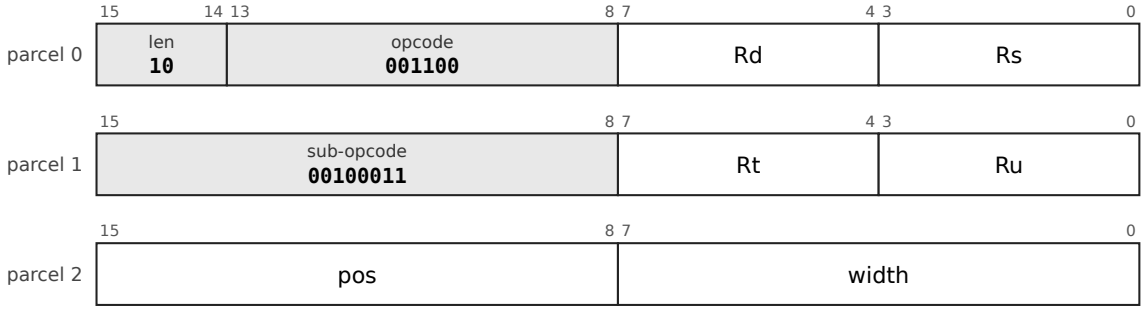

```
EXTZB Rs, #pos, #width, Rd
```

Mnemonic	Opcode	Sub-opcode
EXTZB	001100 (0x0C)	00100010 (0x22)

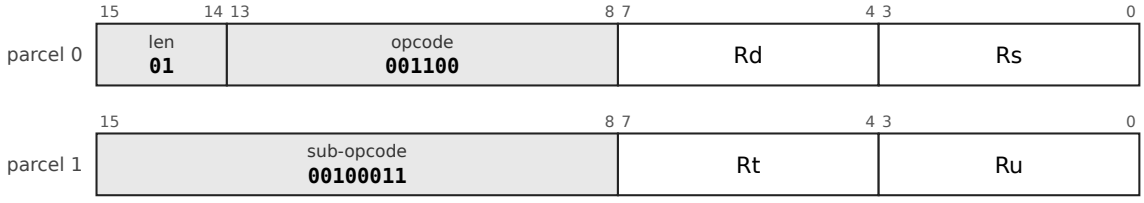
Operation, flags and exceptions as for the entry EXTB above; 2 layout forms, each a chart of its own (above).

INSB: Insert a bit field

INSB, opcode 001100, three parcels, sub-opcode 00100011 (6 bytes)



INSB, opcode 001100, two parcels, sub-opcode 00100011 (4 bytes)

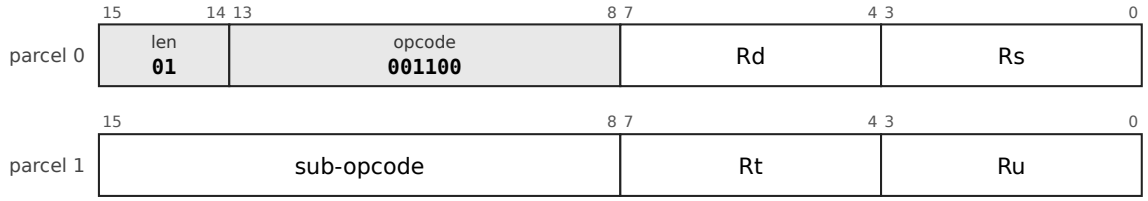


Mnemonics: `INSB`

Syntax

```
INSB Rs, #pos, #width, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
INSB	001100 (0x0C)	00100011 (0x23)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2-3`.

Operation

```
mask ← ((1 << width) - 1) << pos
Rd ← (Rd AND NOT mask) OR ((Rs << pos) AND mask)
```

Flags None written.

Exceptions None.

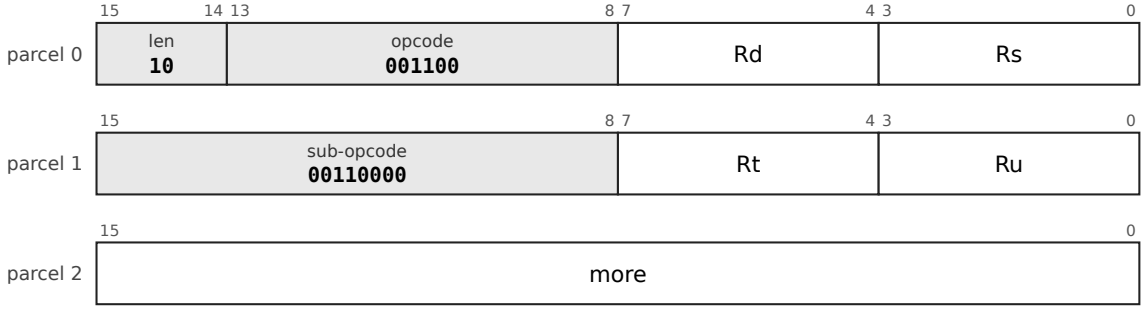
Six bytes. `Rd` is both source and destination.

Why it is here. The VAX `INSV` idiom, with `EXTB`. No use in the compiled corpus is recorded.

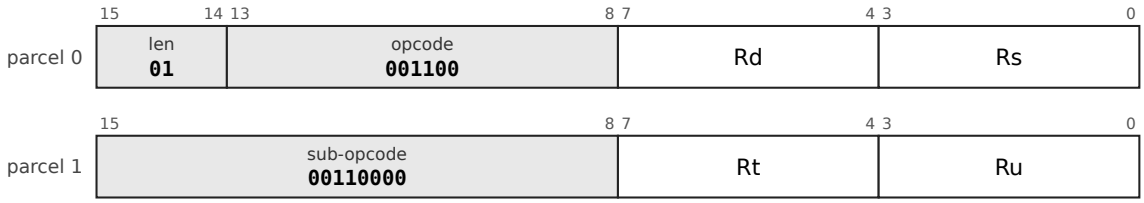
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

XNORPOP: Population count of the equality of two words

XNORPOP, opcode 001100, three parcels, sub-opcode 00110000 (6 bytes)



XNORPOP, opcode 001100, two parcels, sub-opcode 00110000 (4 bytes)

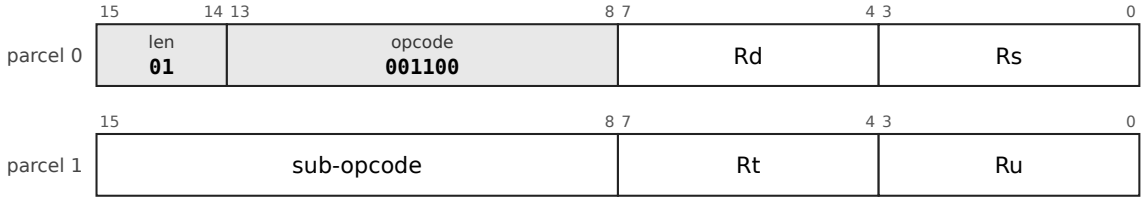


Mnemonics: `XNORPOP`

Syntax

```
XNORPOP Ra, Rb, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
`XNORPOP`	`001100 (0x0C)`	`00110000 (0x30)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2`.

Operation

```
Rd ← popcount(NOT (Ra XOR Rb))
```

Flags None written.

Exceptions None.

For binary-activation networks, Hamming distance and bitset intersection cardinality. It does **not** serve sub-1-bit language models (section 8.3).

Why it is here. It serves binary-activation networks, Hamming distance, bitset intersection counts and counting a collector's mark bitmap. It does not serve sub-1-bit language models: their activations (in the LittleBit work) are FP16 and the inner loop is a conditional add, not a popcount. The sub-1-bit kernel is bandwidth-bound and belongs to a bus-mastering accelerator, not an opcode (chapter 8).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

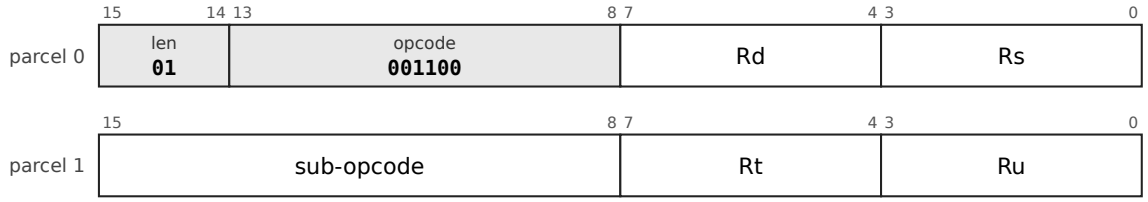
CLMUL: Carry-less multiply

Mnemonics: `CLMULL`, `CLMULH`

Syntax

```
CLMULL Ra, Rb, Rd
CLMULH Ra, Rb, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
`CLMULL`	`001100 (0x0C)`	`00110001 (0x31)`
`CLMULH`	`001100 (0x0C)`	`00110010 (0x32)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2`.

Operation

```
p ← carry-less (polynomial over GF(2)) product of Ra and Rb      ; 128 bits
CLMULL: Rd ← p[63:0]
CLMULH: Rd ← p[127:64]
```

Flags None written.

Exceptions None.

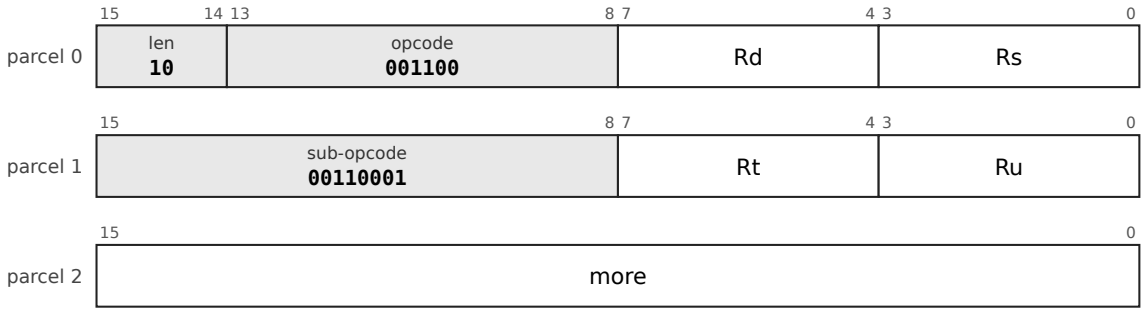
The primitive under every CRC (with a Barrett reduction), GF(2^k) arithmetic and GCM; no polynomial is committed to the architecture.

Why it is here. Carry-less multiply is the primitive beneath every CRC polynomial (a CRC is a carry-less multiply plus a Barrett reduction), GF(2^k) erasure codes and AES-GCM. It is chosen over a `CRC` instruction because x86 and ARM each committed a polynomial to the architecture; there are no `CRC32` or `CRC32C` words.

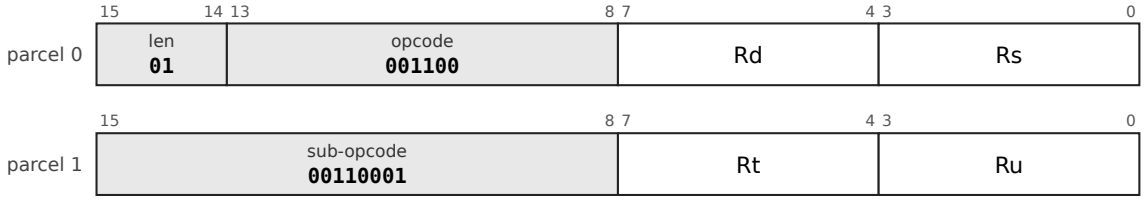
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

CLMULL

CLMULL, opcode 001100, three parcels, sub-opcode 00110001 (6 bytes)



CLMULL, opcode 001100, two parcels, sub-opcode 00110001 (4 bytes)



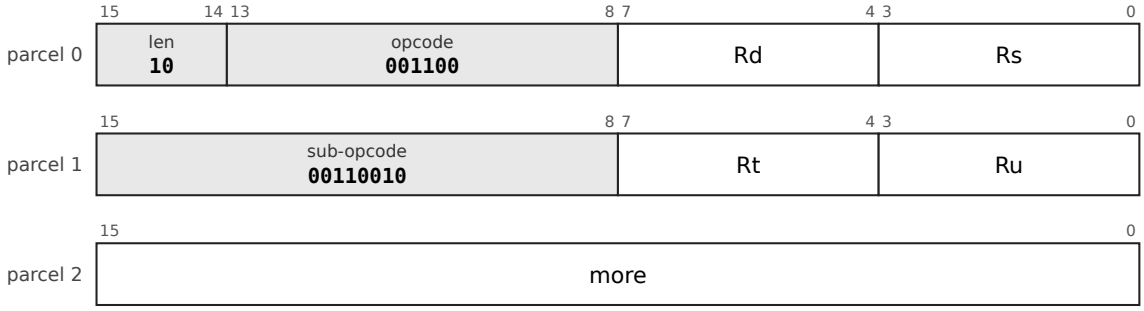

```
CLMULL Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
CLMULL	001100 (0x0C)	00110001 (0x31)

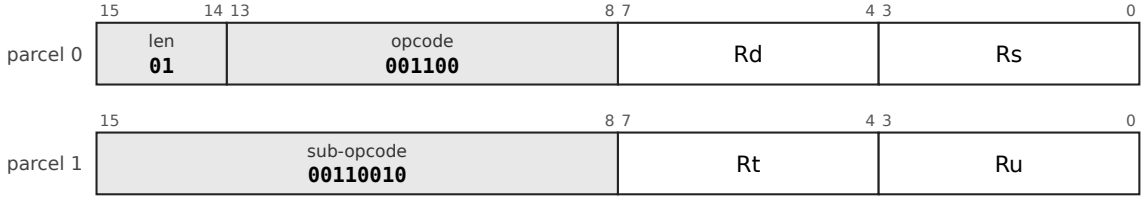
Operation, flags and exceptions as for the entry CLMUL above; 2 layout forms, each a chart of its own (above).

CLMULH

CLMULH, opcode 001100, three parcels, sub-opcode 00110010 (6 bytes)



CLMULH, opcode 001100, two parcels, sub-opcode 00110010 (4 bytes)



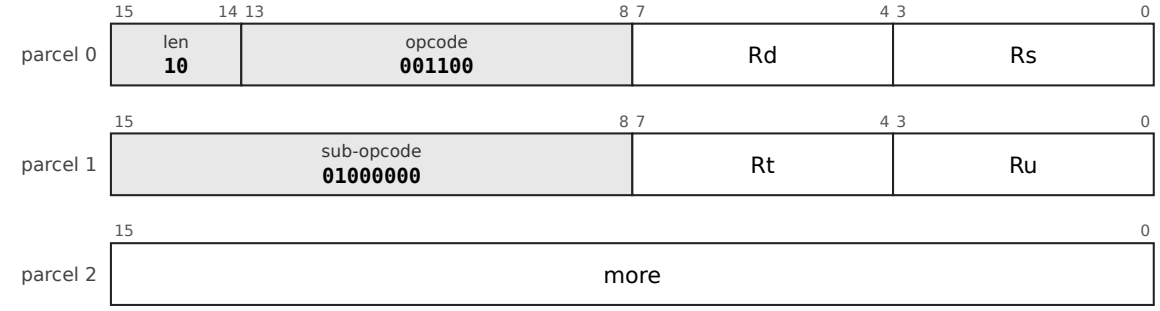

```
CLMULH Ra, Rb, Rd
```

Mnemonic	Opcode	Sub-opcode
CLMULH	001100 (0x0C)	00110010 (0x32)

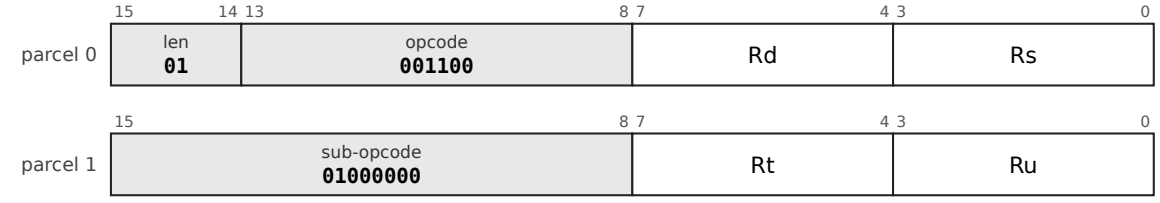
Operation, flags and exceptions as for the entry CLMUL above; 2 layout forms, each a chart of its own (above).

UTF8LEN: Length of a UTF-8 sequence from its leading byte

UTF8LEN, opcode 001100, three parcels, sub-opcode 01000000 (6 bytes)



UTF8LEN, opcode 001100, two parcels, sub-opcode 01000000 (4 bytes)

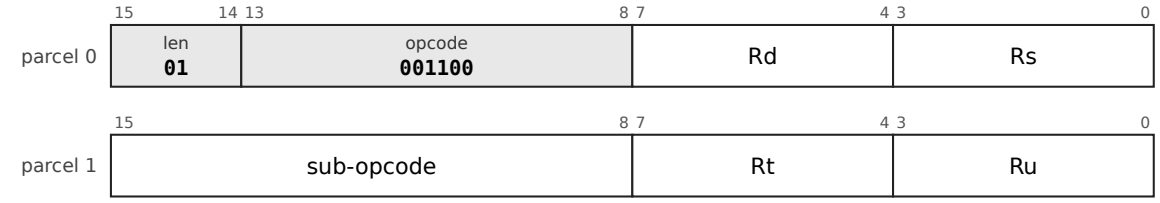


Mnemonics: `UTF8LEN`

Syntax

```
UTF8LEN Rs, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
UTF8LEN	001100 (0x0C)	01000000 (0x40)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2`.

Operation

```
Rd ← length (1 to 4) implied by the leading byte Rs[7:0]; 0 if it cannot begin a valid sequence
```

Flags None written.

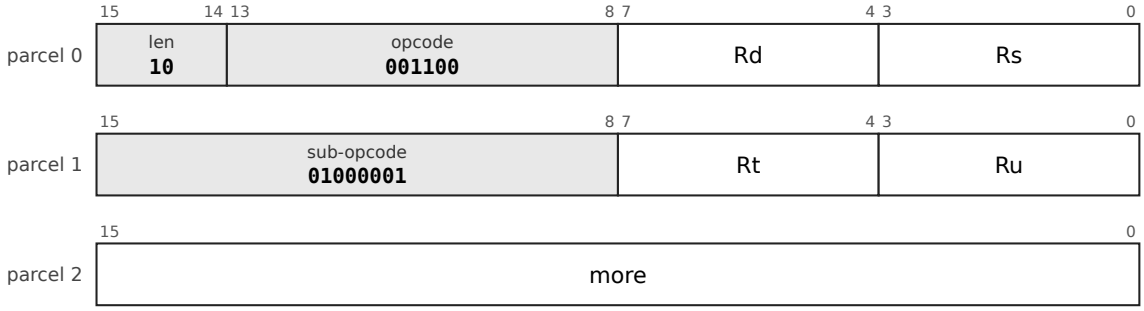
Exceptions None.

Why it is here. Most of what UTF-8 needs is general bit manipulation, but the validity rules are where the bugs are, so the group has three instructions. `UTF8LEN` is the one that is nearly free: `CLO` on the leading byte, mapped to 1, 2, 3 or 4.

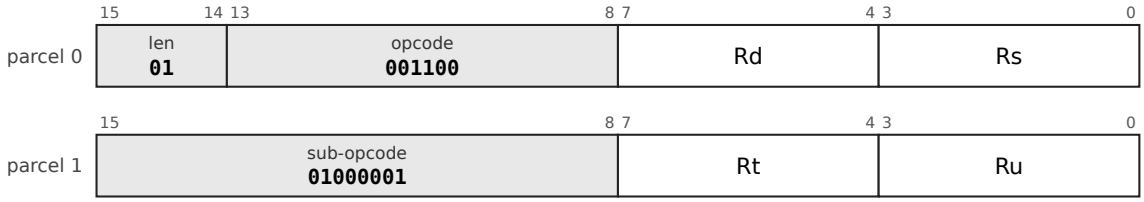
Status specified; **not yet implemented** in the reference executor.

UTF8DEC: Decode a UTF-8 sequence

UTF8DEC, opcode 001100, three parcels, sub-opcode 01000001 (6 bytes)



UTF8DEC, opcode 001100, two parcels, sub-opcode 01000001 (4 bytes)

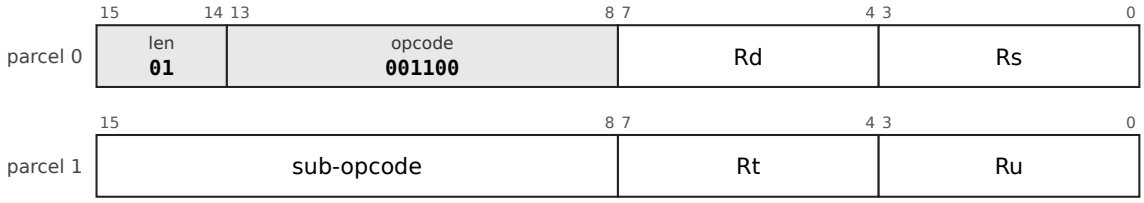


Mnemonics: `UTF8DEC`

Syntax

```
UTF8DEC Rs, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
`UTF8DEC`	`001100 (0x0C)`	`01000001 (0x41)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-besc-1`, `r20-besc-2`.

Operation

```
Rs holds up to 4 bytes, leading byte in bits 7..0
Rd ← the scalar value
V  ← 1 if the sequence is not a valid encoding
```

Flags `V` set for a bad continuation byte, an overlong form, a surrogate (U+D800 to U+DFFF), a value above U+10FFFF, or a leading byte of `11000000 (0xC0)`, `11000001 (0xC1)` or `11110101 (0xF5)` to `11111111 (0xFF)`; the length consumed is reported in a scratch field of the status word (details in the instruction-set text, section 4.6.1).

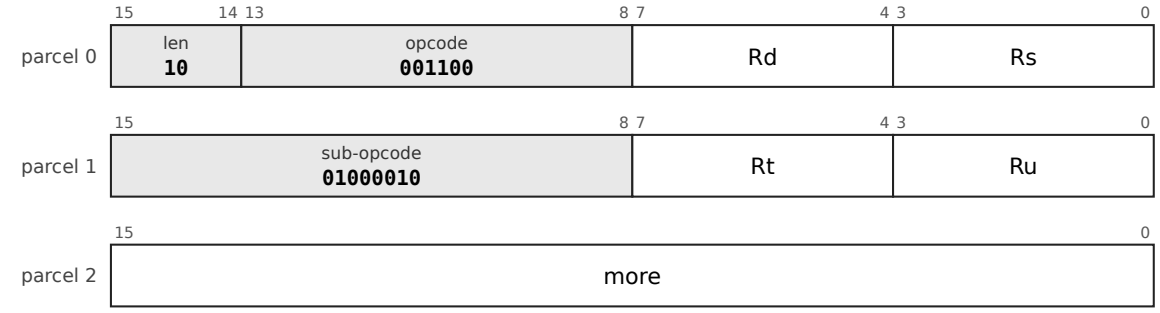
Exceptions None.

Why it is here. Not for speed but for correctness: overlong forms, surrogates, values above U+10FFFF and the bytes that can never begin a sequence are each a historical security hole, each is three comparators in hardware, and software implementations get them wrong or omit them for speed (about 300 lookup tables).

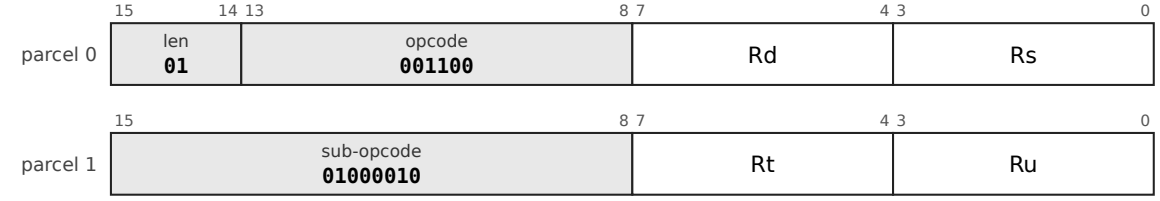
Status specified; **not yet implemented** in the reference executor.

UTF8ENC: Encode a Unicode scalar value

UTF8ENC, opcode 001100, three parcels, sub-opcode 01000010 (6 bytes)



UTF8ENC, opcode 001100, two parcels, sub-opcode 01000010 (4 bytes)

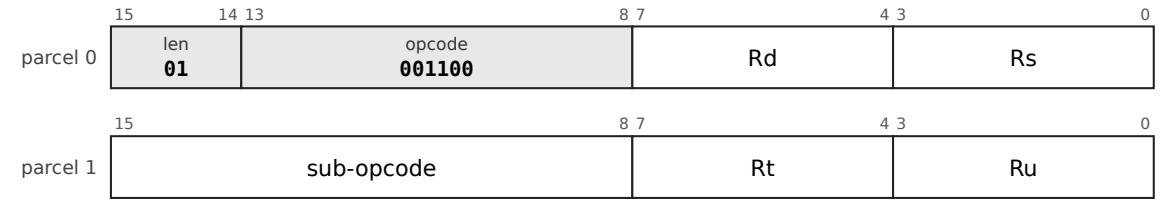


Mnemonics: `UTF8ENC`

Syntax

```
UTF8ENC Rs, Rd
```

Encoding



The chart is the layout `r20-besc-1`, Bits and fields (BESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
UTF8ENC	001100 (0x0C)	01000010 (0x42)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-besc-1, r20-besc-2.

Operation

```
Rd ← the UTF-8 bytes of the scalar value Rs, leading byte in bits 7..0
V  ← 1 if Rs is not a Unicode scalar value
```

Flags V set if Rs is not a Unicode scalar value (a surrogate or above U+10FFFF).

Exceptions None.

Why it is here. The encoding half of the UTF-8 pair; it refuses surrogates and values above U+10FFFF so that an encoder cannot produce what a decoder must reject.

Status specified; **not yet implemented** in the reference executor.

9.10 Branches and control flow

Cf. section 6.13 for TRAP, RTI and RTT, and section 6.3 for how a trap enters the kernel; section 9.16 for CALLI and JMPI in position-independent code.

9.10.1 There is an unconditional branch BR and fourteen conditional branches, two bytes with an 8-bit displacement, four with 16 bits, six with 32 bits, counting two-byte parcels from the first parcel of the branch: ±256 bytes, ±64 KB, ±4 GB. *(opcodes 110000 (0x30)-111110 (0x3E))*

9.10.2 The conditions are ne, eq, pl, mi, vc, vs, cc/hs, cs/lo, ge, lt, gt, le, hi and los.

9.10.3 SOB Rn, dst subtracts one from Rn and branches if the result is nonzero, forward or backward. *(opcode 111111 (0x3F))*

9.10.4 CBCC compares a register with a 5-bit immediate and branches (four bytes, ±1024 parcels); CBW takes a 16-bit immediate and a 16-bit displacement (six bytes); CBRR compares two registers and branches. None writes the flags. *(D)*

9.10.5 JMP and CALL take any address mode; RET returns; JMPI and CALLI jump and call through a pointer in memory; JTAG dispatches on the tag in the low three bits of a register; BTAG and BNTAG branch on a tag. *(D)*

9.10.6 A call pushes the return address on the stack; there is no link register. *(D)*

Commentary. **Precedent.** The complete PDP-11 branch set, which the long-branch problem of the PDP-11 (±256 bytes) made awkward, here reaches ±4 GB with no long-branch opcode and no trampoline, because the displacement widens with the length. SOB was the PDP-11's, which could only branch backward. **Trade-offs.** A compare-and-branch that writes no flags cannot feed a second branch that reads them, so a three-way compare is compare-then-branch; the compiler fuses only where the flags are dead afterwards, which costs about 0.4% in code size. **Improvements.** The tag branches serve a Lisp or other tagged runtime in one instruction. **Benefits.** Branch relaxation in the assembler is choosing a length. **Modern comparables.** RISC-V and MIPS are compare-and-branch machines with no flags; AArch64 has CBZ and TBZ; x86 uses flags like the pdp64's branches. **Use.** JMPI and CALLI are the shared-library call and the virtual call; JTAG is the dispatch of a dynamic-language interpreter. **Impact.** *Kernel:* JMPI, CALLI and JTAG are the shared-library call, the virtual call and the dispatch of an interpreter; SOB and CBCC are the loops of the kernel's inner routines. *Compiler:* Branch relaxation is choosing a length; fusing a compare and a branch is legal only where the flags are dead afterwards, so the compiler needs a flag-liveness analysis. *Hardware:* The next-PC mux takes the load-return path for the indirect forms; the compare-and-branch compare runs in the branch unit beside the ALU; displacements count parcels, so no shifter is needed.

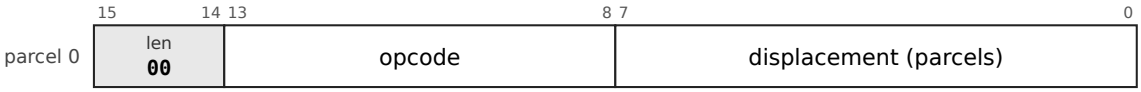
BCC: Branch on condition

Mnemonics: BR, BNE, BEQ, BPL, BMI, BVC, BVS, BCC, BCS, BGE, BLT, BGT, BLE, BHI, BLOS (aliases: BHIS, BLO)

Syntax

```
BR target
BNE target
BEQ target ... and the rest
```

Encoding



The chart is the layout `r20-br-0`, Conditional branches, one parcel (`len = 00`): The displacement is signed and counts 2-byte parcels from the first parcel of the instruction.

Mnemonic	Opcode
BR	110000 (0x30)
BNE	110001 (0x31)
BEQ	110010 (0x32)
BPL	110011 (0x33)
BMI	110100 (0x34)
BVC	110101 (0x35)
BVS	110110 (0x36)
BCC	110111 (0x37)
BCS	111000 (0x38)
BGE	111001 (0x39)
BLT	111010 (0x3A)
BGT	111011 (0x3B)
BLE	111100 (0x3C)
BHI	111101 (0x3D)
BLOS	111110 (0x3E)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-br-0`, `r20-br-1`, `r20-br-2`.

Operation

```
if cond(c) then PC ← address of this instruction + 2 × disp
```

Flags None written.

Exceptions None; a branch to an unmapped or non-executable address faults on the fetch of the target.

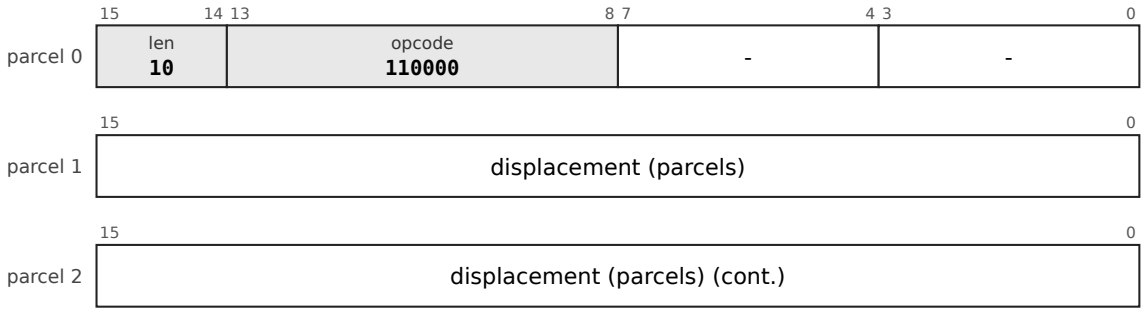
Condition numbers: `al` 0 (`BR`), `ne` 1, `eq` 2, `pl` 3, `mi` 4, `vc` 5, `vs` 6, `cc`/`hs` 7, `cs`/`lo` 8, `ge` 9, `lt` 10, `gt` 11, `le` 12, `hi` 13, `los` 14. Conditions on the flags: `ne` `!Z`; `eq` `Z`; `pl` `!N`; `mi` `N`; `vc` `!V`; `vs` `V`; `cc` `!C`; `cs` `C`; `ge` `N == V`; `lt` `N != V`; `gt` `!Z and N == V`; `le` `Z or N != V`; `hi` `!C and !Z`; `los` `C or Z`. The displacement counts 2-byte parcels; the branch reaches ±4 GB with no long-branch instruction.

Cf. 6.13 for `TRAP`, `RTI` and `RTT`.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

BR

BR, opcode 110000, three parcels (6 bytes)



BR, opcode 110000, two parcels (4 bytes)



BR, opcode 110000, one parcel (2 bytes)



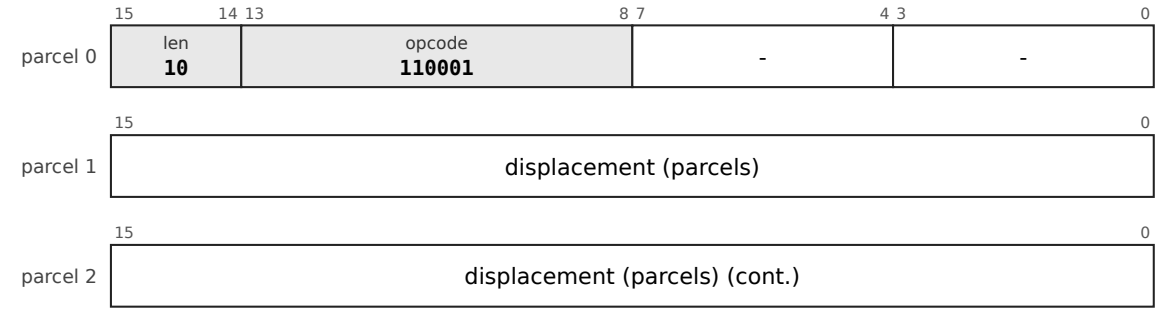

```
BR target
```

Mnemonic	Opcode
BR	110000 (0x30)

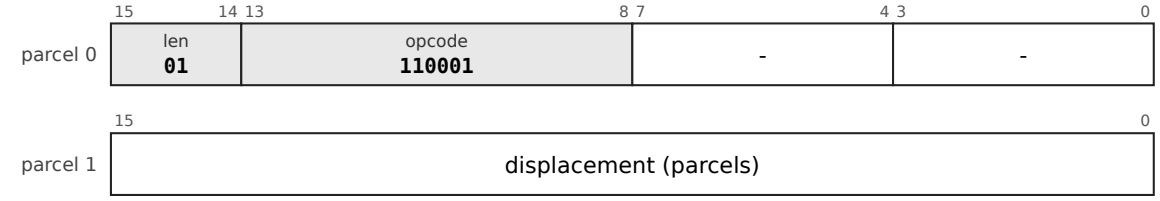
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BNE

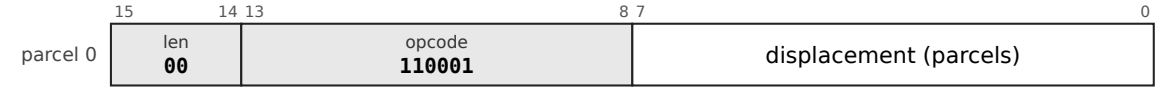
BNE, opcode 110001, three parcels (6 bytes)



BNE, opcode 110001, two parcels (4 bytes)



BNE, opcode 110001, one parcel (2 bytes)



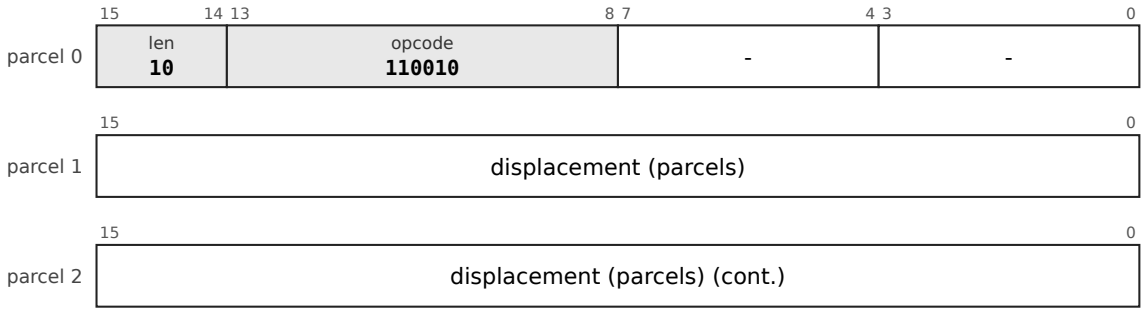

```
BNE target
```

Mnemonic	Opcode
BNE	110001 (0x31)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BEQ

BEQ, opcode 110010, three parcels (6 bytes)



BEQ, opcode 110010, two parcels (4 bytes)



BEQ, opcode 110010, one parcel (2 bytes)



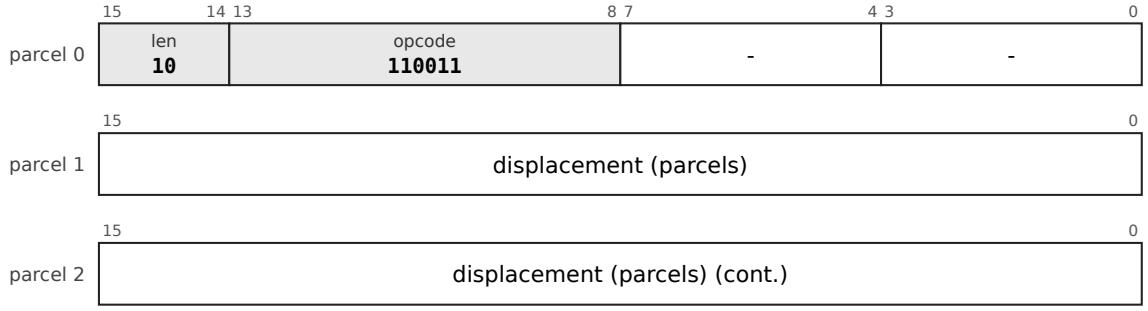

```
BEQ target ... and the rest
```

Mnemonic	Opcode
BEQ	110010 (0x32)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BPL

BPL, opcode 110011, three parcels (6 bytes)



BPL, opcode 110011, two parcels (4 bytes)



BPL, opcode 110011, one parcel (2 bytes)

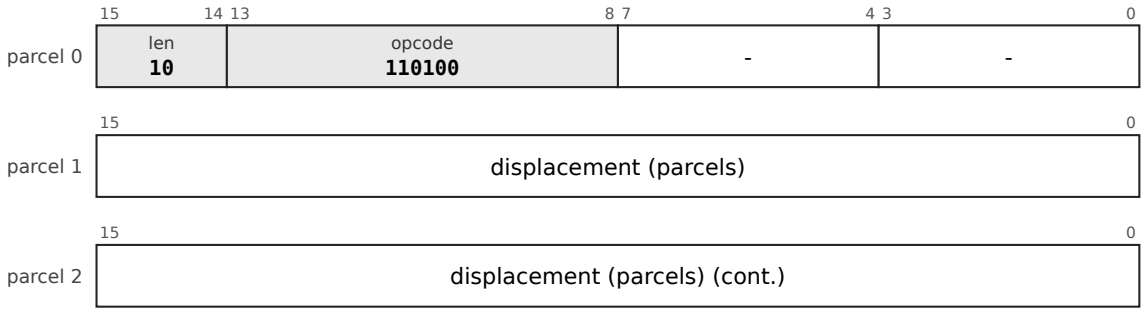


Mnemonic	Opcode
BPL	110011 (0x33)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BMI

BMI, opcode 110100, three parcels (6 bytes)



BMI, opcode 110100, two parcels (4 bytes)



BMI, opcode 110100, one parcel (2 bytes)

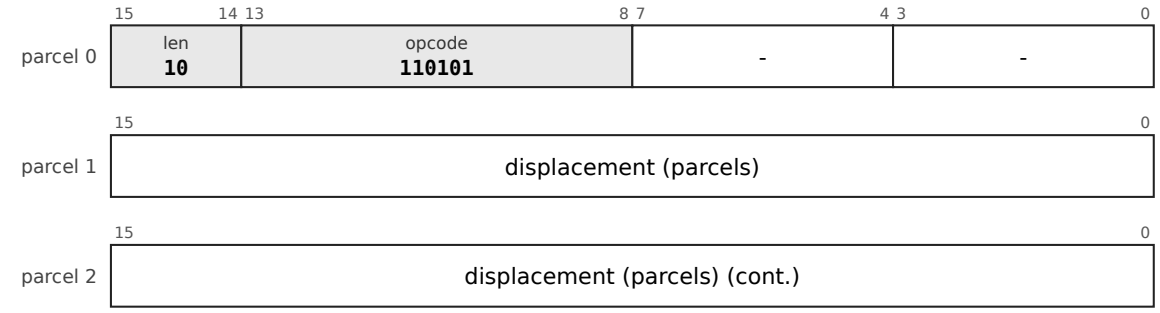


Mnemonic	Opcode
BMI	110100 (0x34)

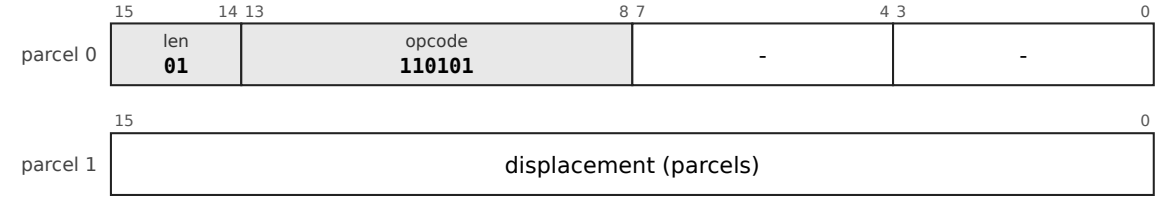
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BVC

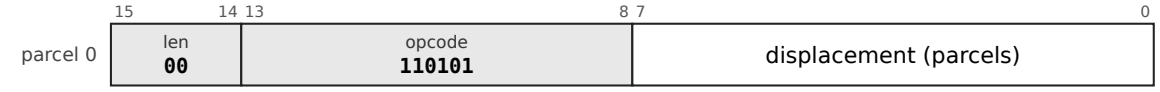
BVC, opcode 110101, three parcels (6 bytes)



BVC, opcode 110101, two parcels (4 bytes)



BVC, opcode 110101, one parcel (2 bytes)

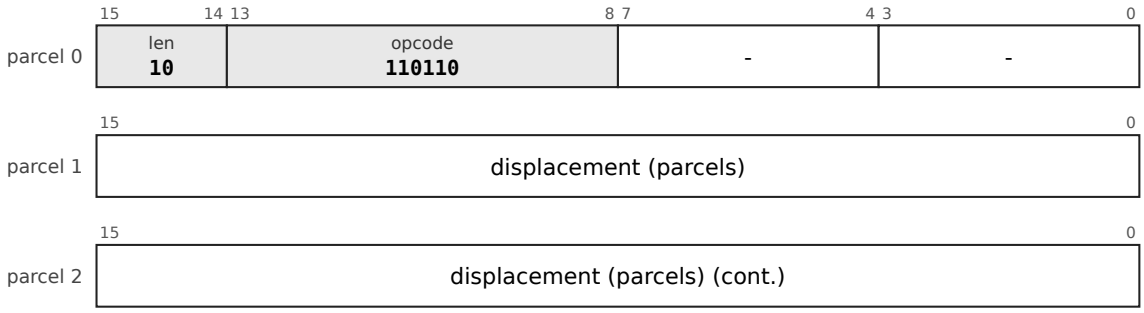


Mnemonic	Opcode
BVC	110101 (0x35)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BVS

BVS, opcode 110110, three parcels (6 bytes)



BVS, opcode 110110, two parcels (4 bytes)



BVS, opcode 110110, one parcel (2 bytes)

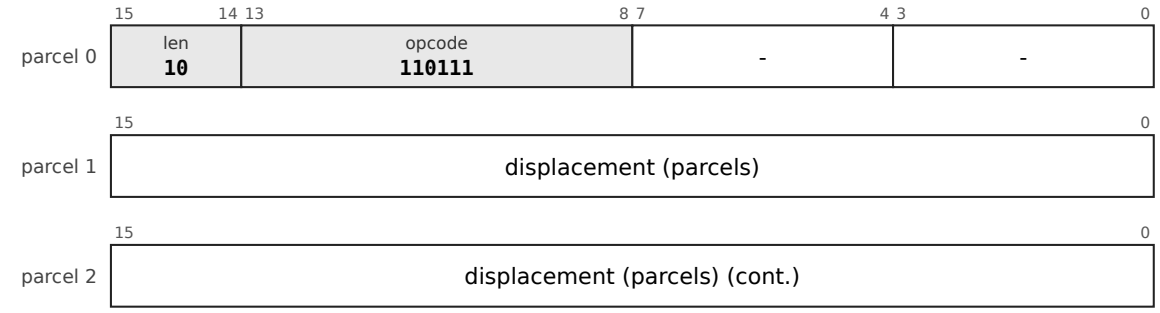


Mnemonic	Opcode
BVS	110110 (0x36)

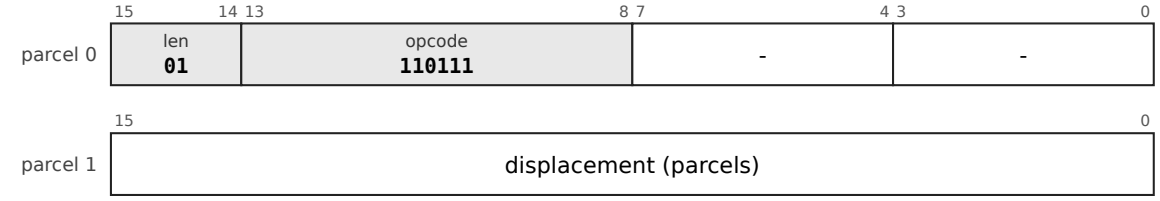
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BCC

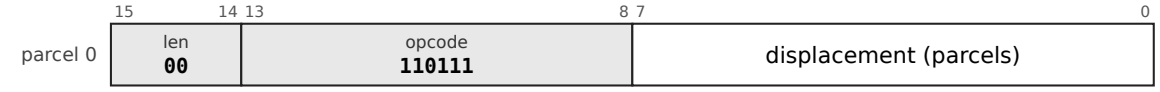
BCC, opcode 110111, three parcels (6 bytes)



BCC, opcode 110111, two parcels (4 bytes)



BCC, opcode 110111, one parcel (2 bytes)

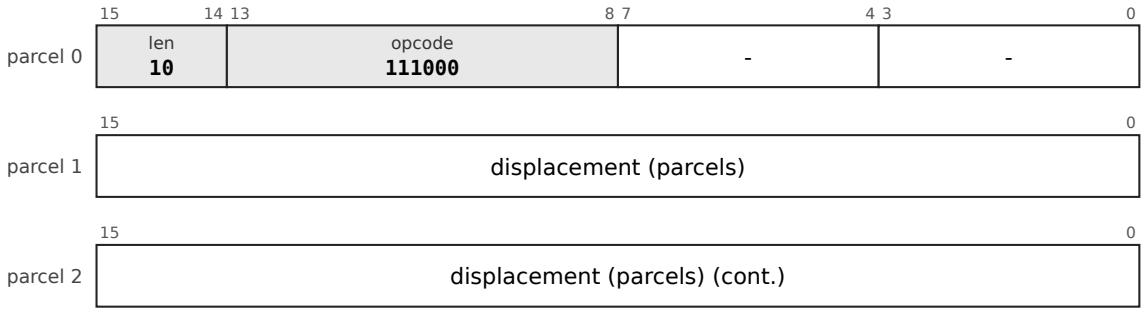


Mnemonic	Opcode
BCC	110111 (0x37)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BCS

BCS, opcode 111000, three parcels (6 bytes)



BCS, opcode 111000, two parcels (4 bytes)



BCS, opcode 111000, one parcel (2 bytes)

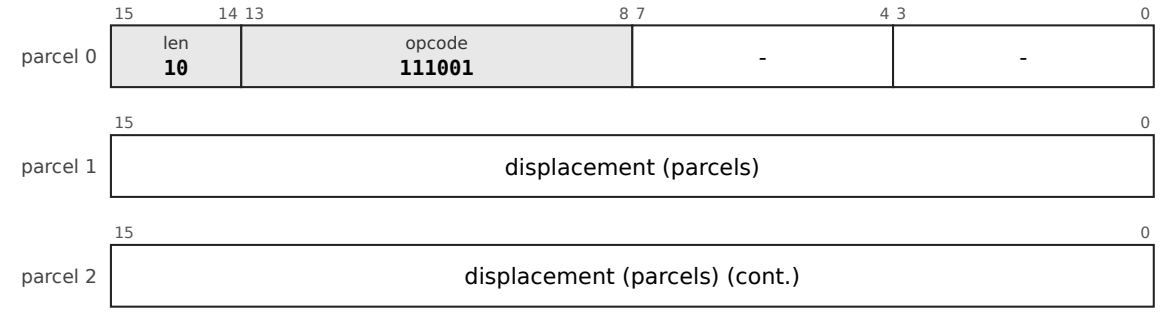


Mnemonic	Opcode
BCS	111000 (0x38)

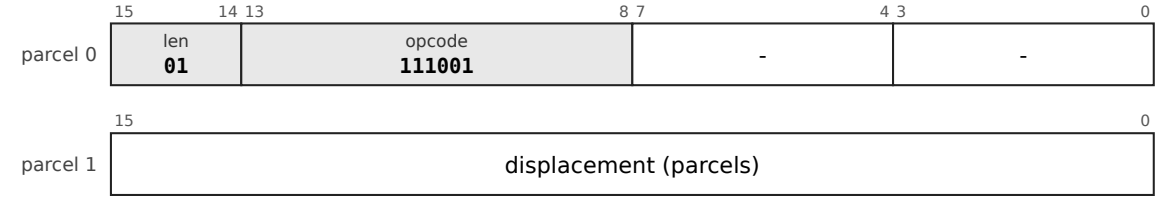
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BGE

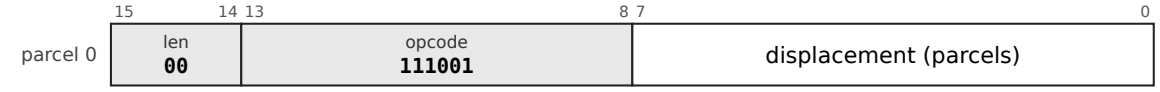
BGE, opcode 111001, three parcels (6 bytes)



BGE, opcode 111001, two parcels (4 bytes)



BGE, opcode 111001, one parcel (2 bytes)

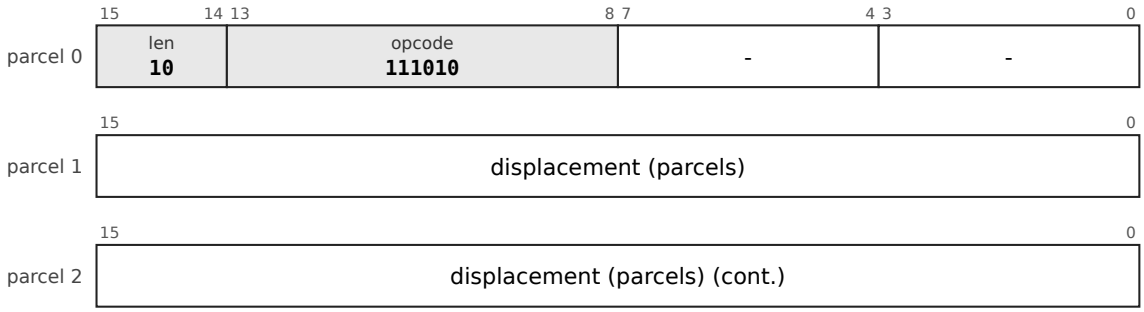


Mnemonic	Opcode
BGE	111001 (0x39)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BLT

BLT, opcode 111010, three parcels (6 bytes)



BLT, opcode 111010, two parcels (4 bytes)



BLT, opcode 111010, one parcel (2 bytes)

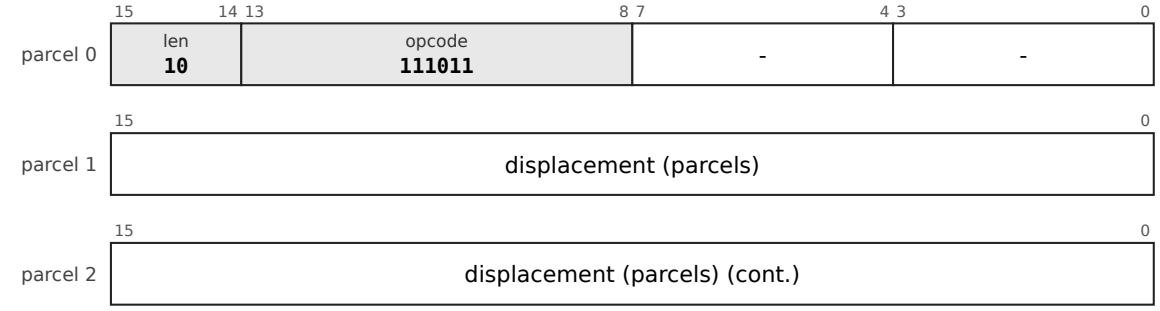


Mnemonic	Opcode
BLT	111010 (0x3A)

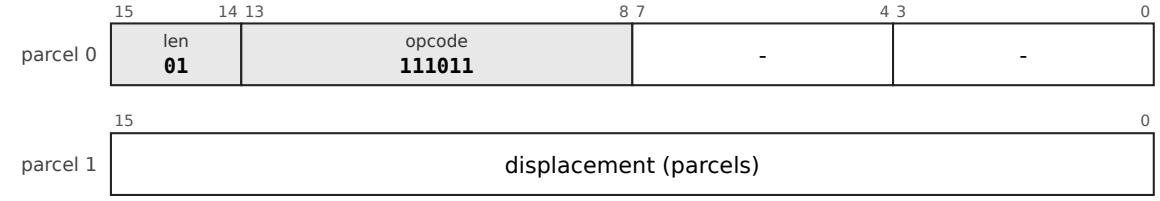
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BGT

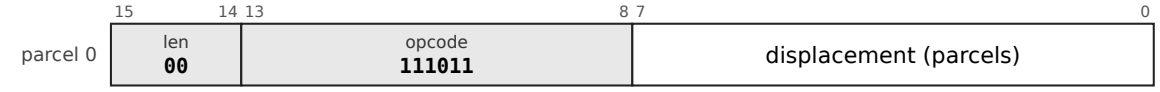
BGT, opcode 111011, three parcels (6 bytes)



BGT, opcode 111011, two parcels (4 bytes)



BGT, opcode 111011, one parcel (2 bytes)

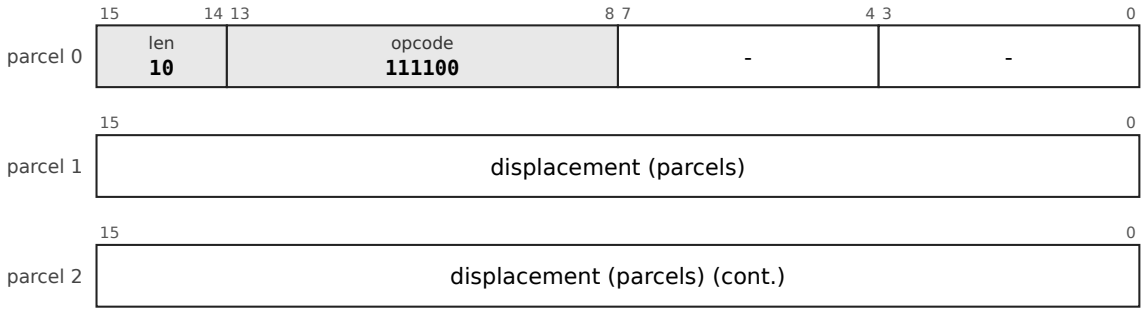


Mnemonic	Opcode
BGT	111011 (0x3B)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BLE

BLE, opcode 111100, three parcels (6 bytes)



BLE, opcode 111100, two parcels (4 bytes)



BLE, opcode 111100, one parcel (2 bytes)

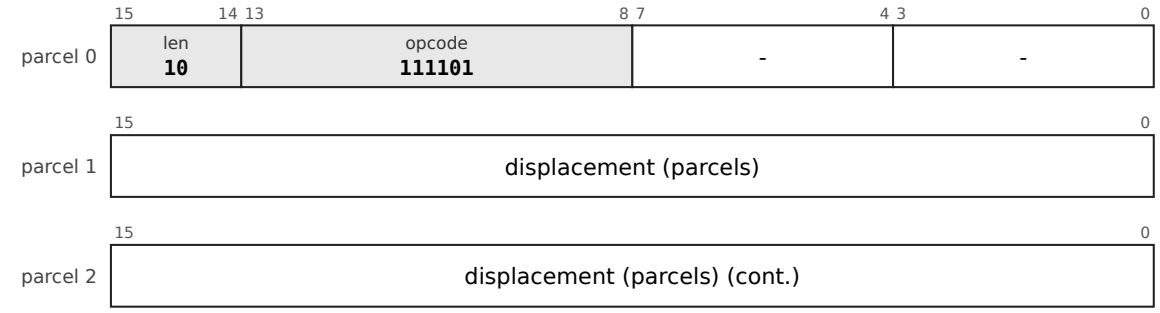


Mnemonic	Opcode
BLE	111100 (0x3C)

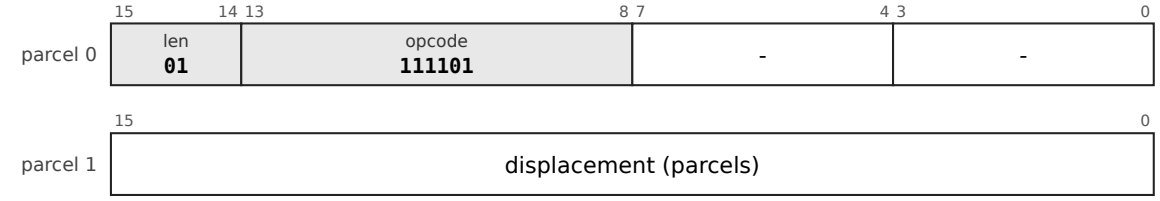
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BHI

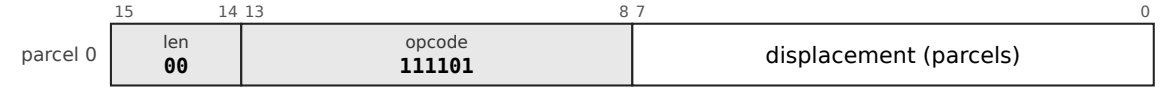
BHI, opcode 111101, three parcels (6 bytes)



BHI, opcode 111101, two parcels (4 bytes)



BHI, opcode 111101, one parcel (2 bytes)

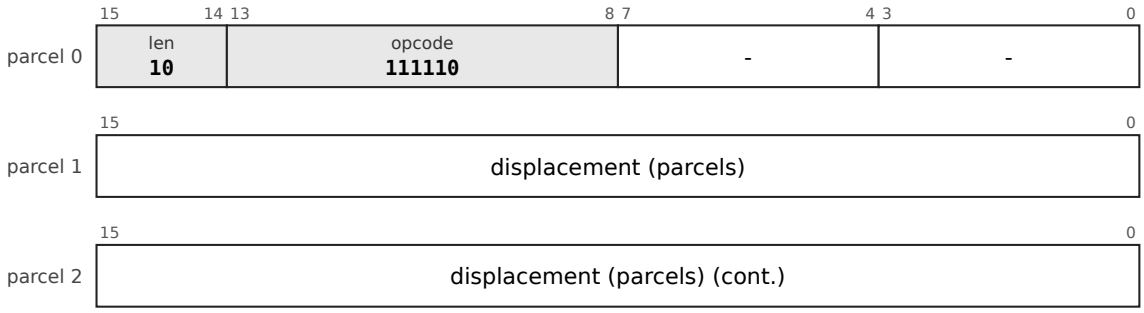


Mnemonic	Opcode
BHI	111101 (0x3D)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BLOS

BLOS, opcode 111110, three parcels (6 bytes)



BLOS, opcode 111110, two parcels (4 bytes)



BLOS, opcode 111110, one parcel (2 bytes)



Mnemonic	Opcode
BLOS	111110 (0x3E)

Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

BHIS

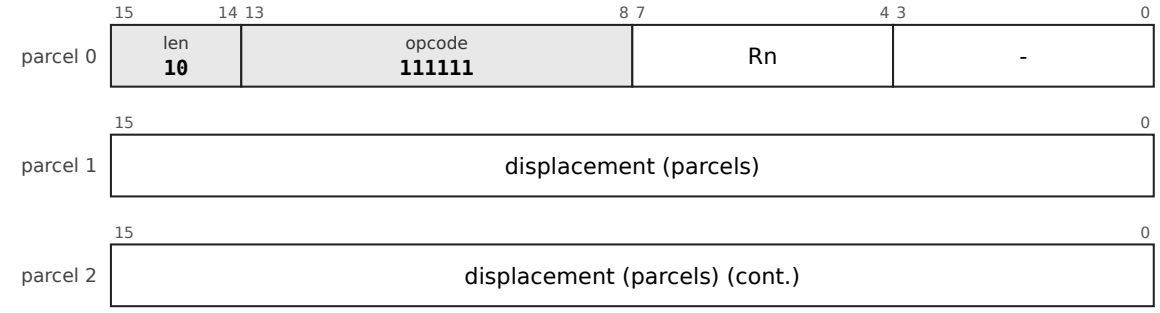
An alias of BCC: the assembler writes it as that word.

BLO

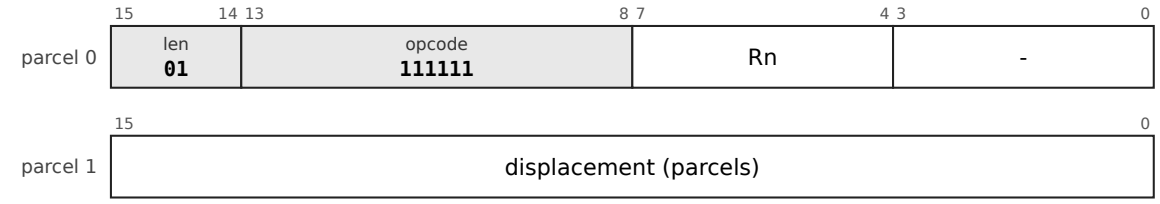
An alias of BCS: the assembler writes it as that word.

SOB: Subtract one and branch

SOB, opcode 111111, three parcels (6 bytes)



SOB, opcode 111111, two parcels (4 bytes)

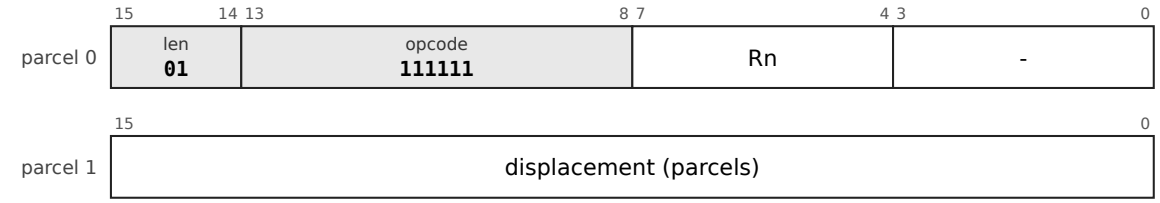


Mnemonics: `SOB`

Syntax

```
SOB Rn, target
```

Encoding



The chart is the layout `r20-sob-1`, SOB, two parcels (`len = 01`): The displacement is signed and counts parcels.

Mnemonic	Opcode
`SOB`	`111111 (0x3F)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sob-1`, `r20-sob-2`.

Operation

```
Rn ← Rn - 1
if Rn ≠ 0 then PC ← address of this instruction + 2 × disp
```

Flags None written.

Exceptions None.

From the PDP-11, where it could branch only backward; here the signed displacement goes either way. Four bytes at least.

Why it is here. Taken from the PDP-11, at the branch group's one free opcode. The PDP-11's could only branch backward (a 6-bit unsigned displacement); this one branches either way. Against the `cbnz` idiom of other architectures it does not save

bytes (DEC plus BNE is also four) but it saves an instruction issue, a cycle per iteration on a machine that does not pipeline: same size, one instruction fewer (static `dec; bne` pairs exist).

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

CBCC: Compare and branch on condition

Mnemonics: CBNE, CBEQ, CBPL, CBMI, CBVC, CBVS, CBCC, CBCS, CBGE, CBLT, CBGT, CBLE, CBHI, CBLOS (aliases: CBHIS, CBLO)

Syntax

```
CBNE Rd, #imm, target
CBEQ Ra, Rb, target ... and the rest
```

Encoding



The chart is the layout `r20-cb-1`, Compare and branch, two parcels (`len = 01`): `low` is the condition; the immediate and the displacement are signed.

Mnemonic	Opcode	low
CBNE	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0001 (0x1), 0001 (0x1), 0001 (0x1)
CBEQ	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0010 (0x2), 0010 (0x2), 0010 (0x2)
CBPL	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0011 (0x3), 0011 (0x3), 0011 (0x3)
CBMI	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0100 (0x4), 0100 (0x4), 0100 (0x4)
CBVC	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0101 (0x5), 0101 (0x5), 0101 (0x5)
CBVS	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0110 (0x6), 0110 (0x6), 0110 (0x6)
CBCC	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0111 (0x7), 0111 (0x7), 0111 (0x7)
CBCS	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1000 (0x8), 1000 (0x8), 1000 (0x8)
CBGE	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1001 (0x9), 1001 (0x9), 1001 (0x9)
CBLT	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1010 (0xA), 1010 (0xA), 1010 (0xA)
CBGT	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1011 (0xB), 1011 (0xB), 1011 (0xB)
CBLE	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1100 (0xC), 1100 (0xC), 1100 (0xC)
CBHI	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1101 (0xD), 1101 (0xD), 1101 (0xD)
CBLOS	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1110 (0xE), 1110 (0xE), 1110 (0xE)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-cb-1`, `r20-cb-1-2`, `r20-cb-2`, `r20-cb-2-2`, `r20-cb-2-3`.

Operation

```
tmp ← Rd - (imm or Rb)         ; as CMP, for the decision only
if cond(c) on that comparison then PC ← address of this instruction + 2 × disp
```

Flags None written: the instruction computes the comparison for its own decision and leaves `N Z V C` as they were.

Exceptions None.

Three encodings, chosen by the assembler: CBCC (`101000 (0x28)`, a 5-bit signed immediate -16 to 15, a displacement of ±1024 parcels at four bytes or 27 bits at six), CBW (`101001 (0x29)`, a 16-bit immediate and a 16-bit displacement, six bytes) and CBRR (`101010 (0x2A)`, two registers, ±2,048 parcels at four bytes or 28 bits at six). There is no `len = 00` form. A target in another section is refused.

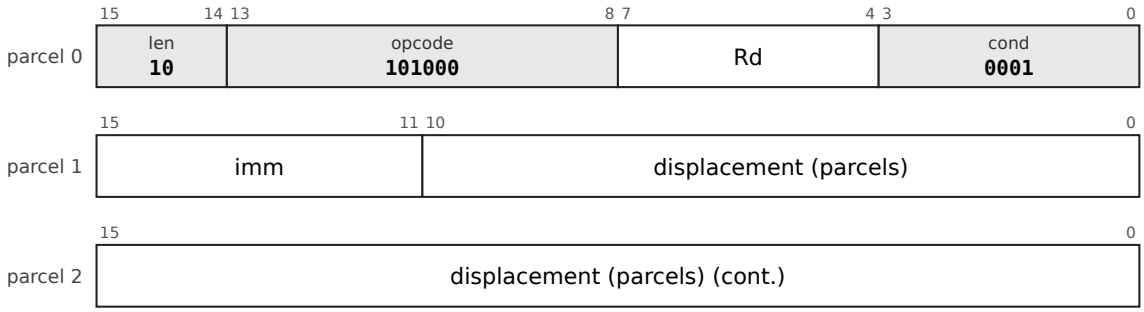
Why it is here. A fused compare-and-branch in the reserved opcode `101000 (0x28)`, estimated with RORI32 at 22 KB (1.3%) of the image. The wide-immediate CBCC removes 11,735 instructions (3.4%) and 1,828 bytes; CBRR, the register-register compare-and-branch, removes 4,631 instructions (1.4%) and 142 bytes. It is not a replacement for flags: LoongArch and RISC-V

chose fused compare-and-branch instead of flags, and this architecture declines that (9.20) because it needs a comparator in the branch unit and a third read port; here the fused form is added beside the flags. CBCC writes no flags.

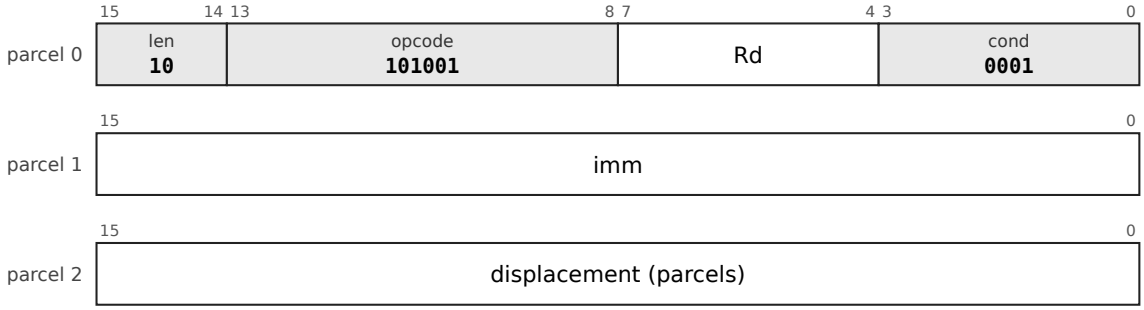
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

CBNE

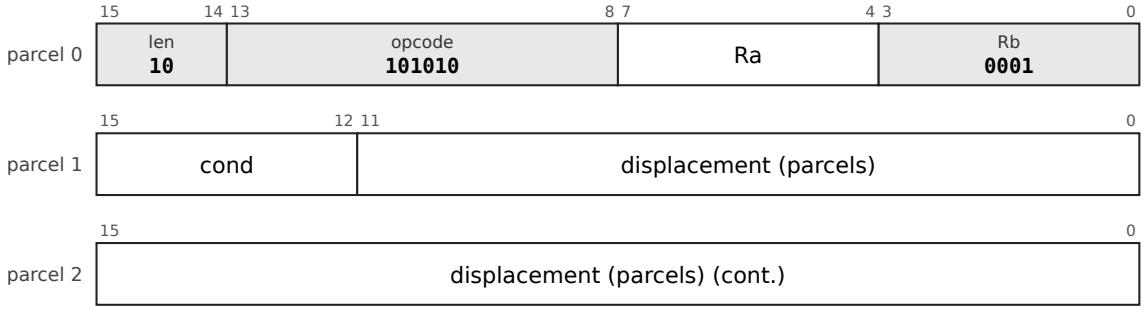
CBNE, opcode 101000, three parcels (6 bytes)



CBNE, opcode 101001, three parcels (6 bytes)



CBNE, opcode 101010, three parcels (6 bytes)



CBNE, opcode 101000, two parcels (4 bytes)



CBNE, opcode 101010, two parcels (4 bytes)



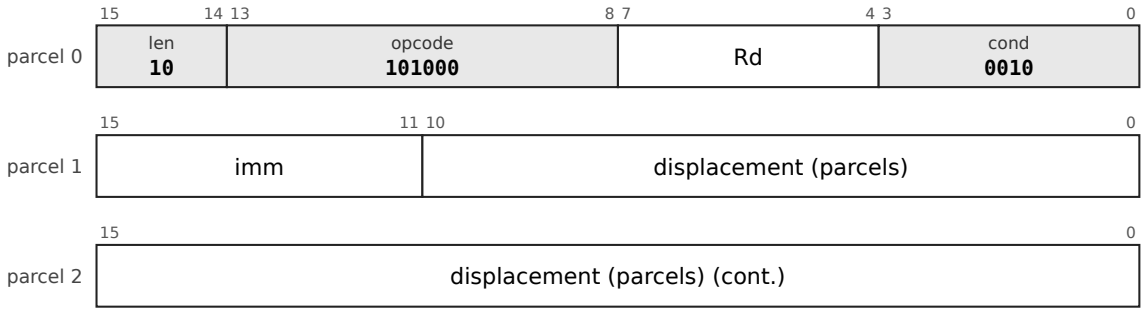

```
CBNE Rd, #imm, target
```

Mnemonic	Opcode	low
CBNE	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0001 (0x1), 0001 (0x1), 0001 (0x1)

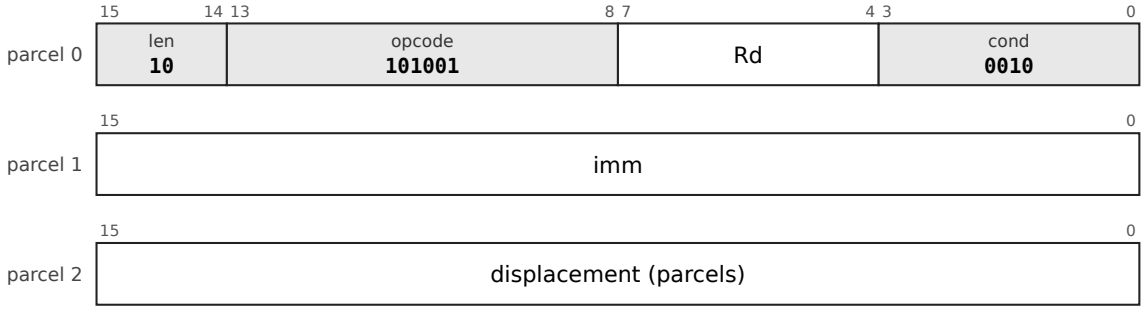
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBEQ

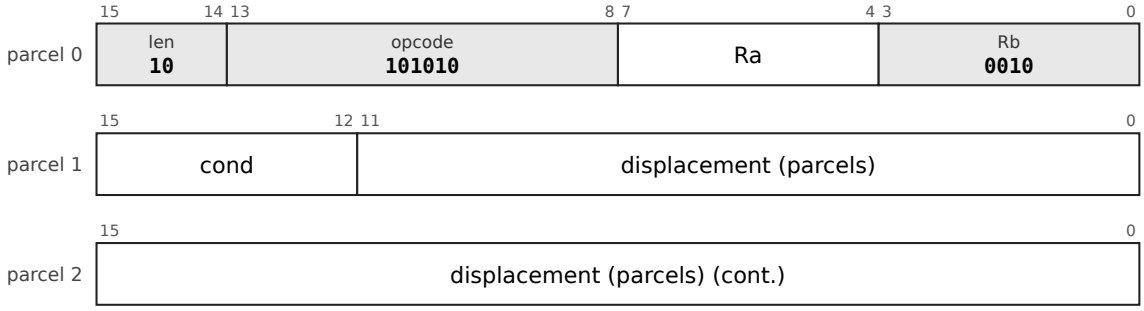
CBEQ, opcode 101000, three parcels (6 bytes)



CBEQ, opcode 101001, three parcels (6 bytes)



CBEQ, opcode 101010, three parcels (6 bytes)



CBEQ, opcode 101000, two parcels (4 bytes)



CBEQ, opcode 101010, two parcels (4 bytes)



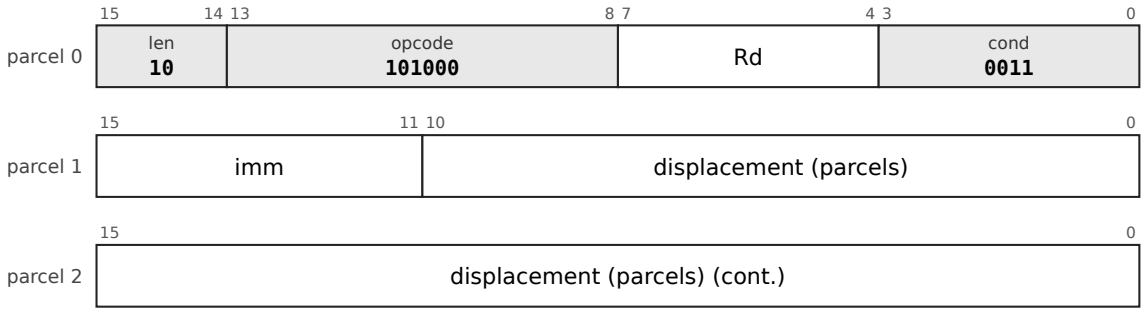

```
CBEQ Ra, Rb, target ... and the rest
```

Mnemonic	Opcode	low
CBEQ	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0010 (0x2), 0010 (0x2), 0010 (0x2)

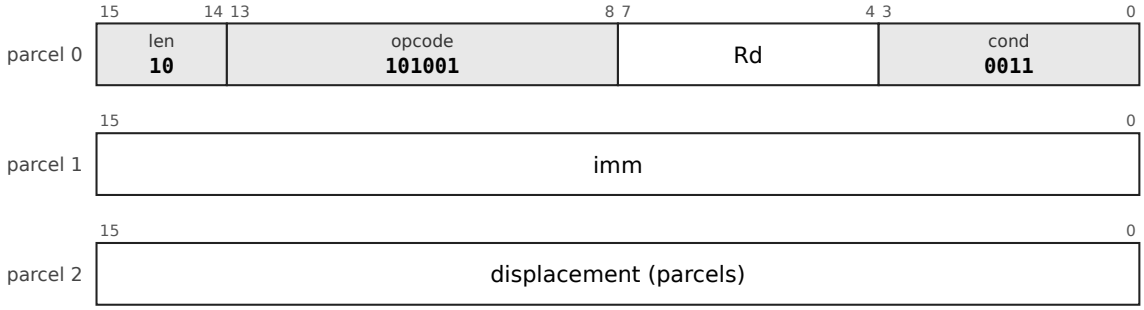
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBPL

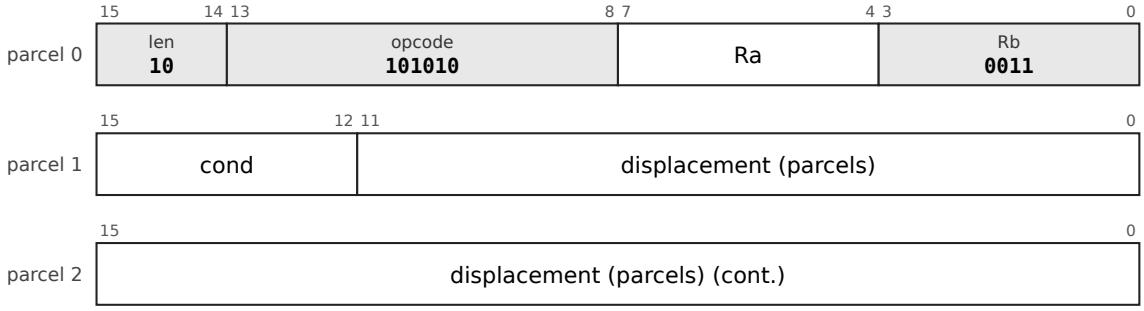
CBPL, opcode 101000, three parcels (6 bytes)



CBPL, opcode 101001, three parcels (6 bytes)



CBPL, opcode 101010, three parcels (6 bytes)



CBPL, opcode 101000, two parcels (4 bytes)



CBPL, opcode 101010, two parcels (4 bytes)

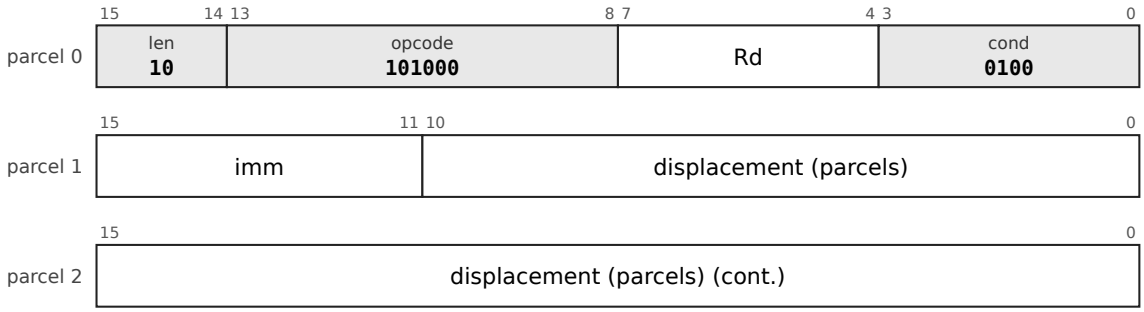


Mnemonic	Opcode	low
CBPL	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0011 (0x3), 0011 (0x3), 0011 (0x3)

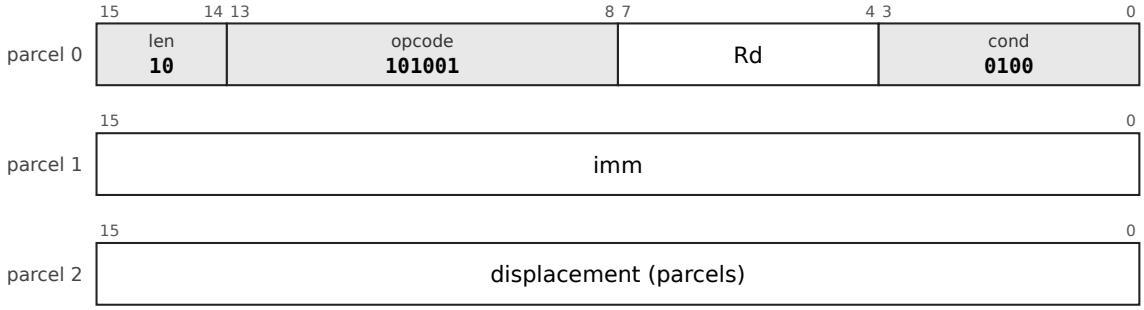
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBMI

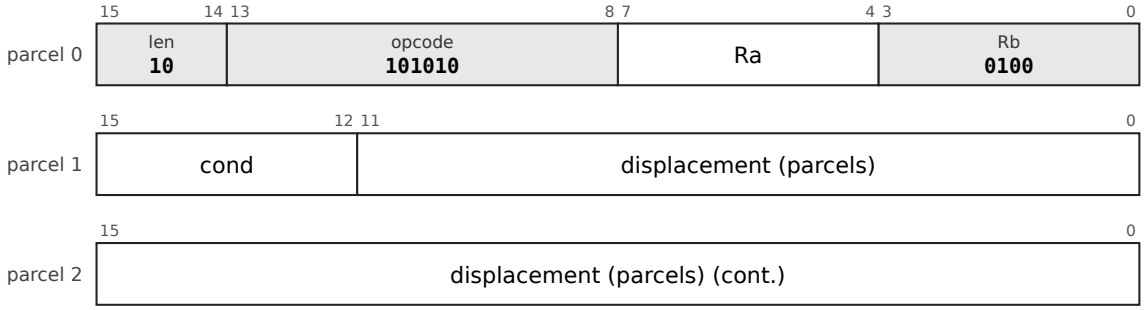
CBMI, opcode 101000, three parcels (6 bytes)



CBMI, opcode 101001, three parcels (6 bytes)



CBMI, opcode 101010, three parcels (6 bytes)



CBMI, opcode 101000, two parcels (4 bytes)



CBMI, opcode 101010, two parcels (4 bytes)

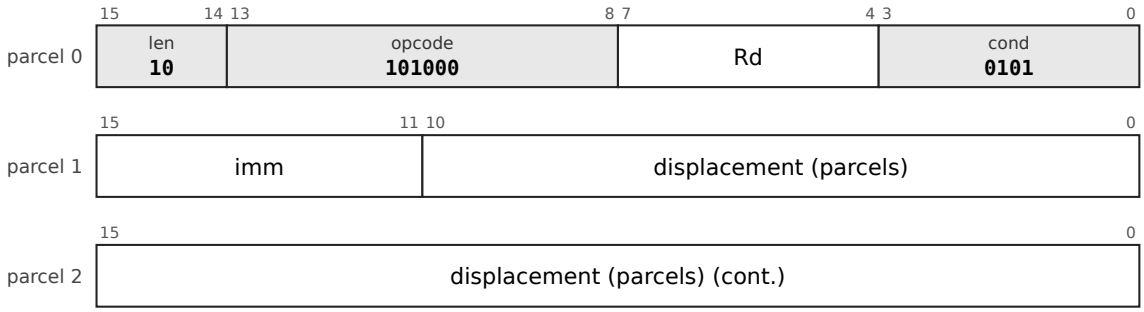


Mnemonic	Opcode	low
CBMI	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0100 (0x4), 0100 (0x4), 0100 (0x4)

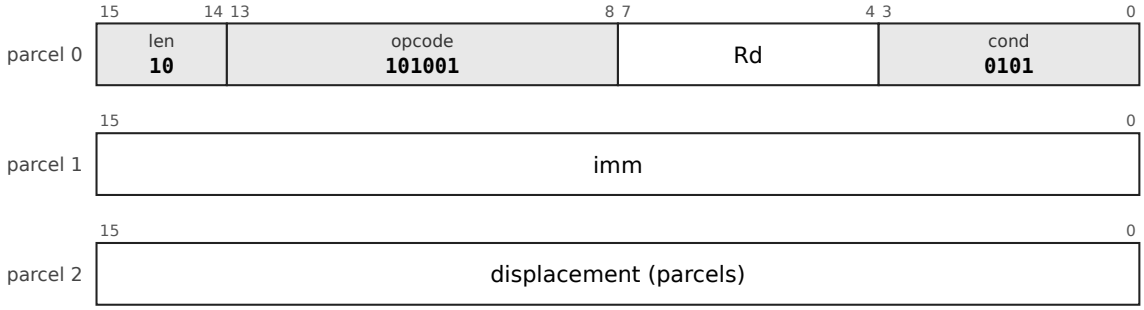
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBVC

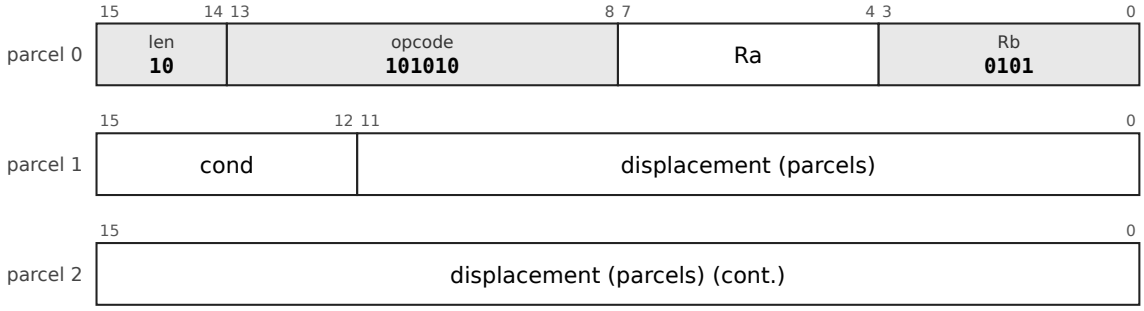
CBVC, opcode 101000, three parcels (6 bytes)



CBVC, opcode 101001, three parcels (6 bytes)



CBVC, opcode 101010, three parcels (6 bytes)



CBVC, opcode 101000, two parcels (4 bytes)



CBVC, opcode 101010, two parcels (4 bytes)

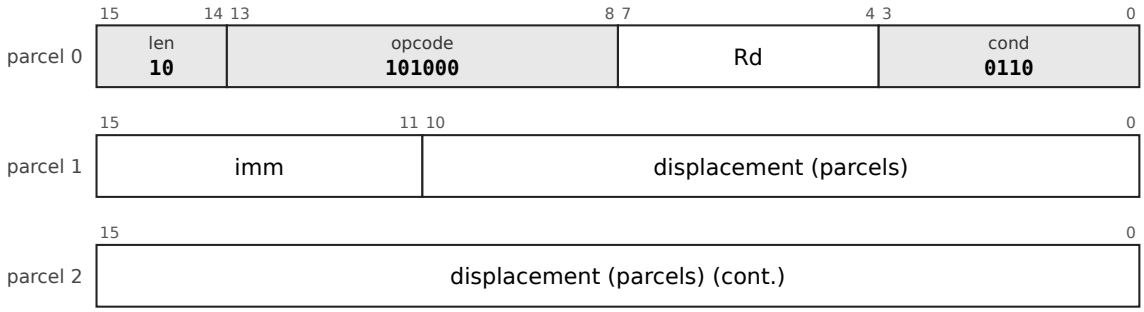


Mnemonic	Opcode	low
CBVC	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0101 (0x5), 0101 (0x5), 0101 (0x5)

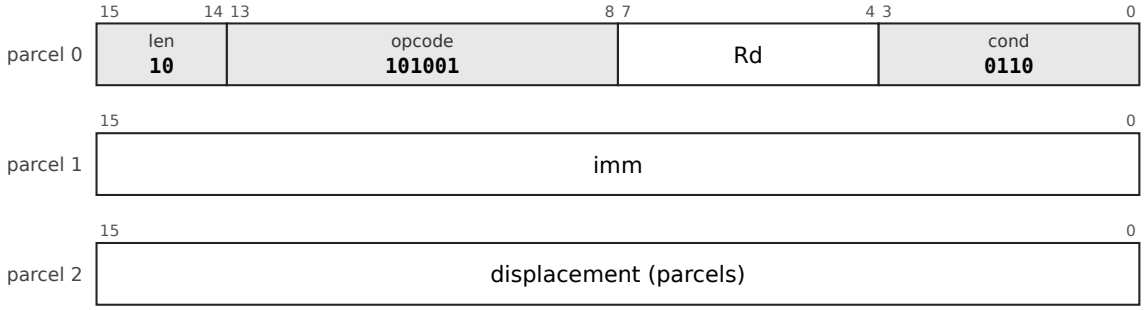
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBVS

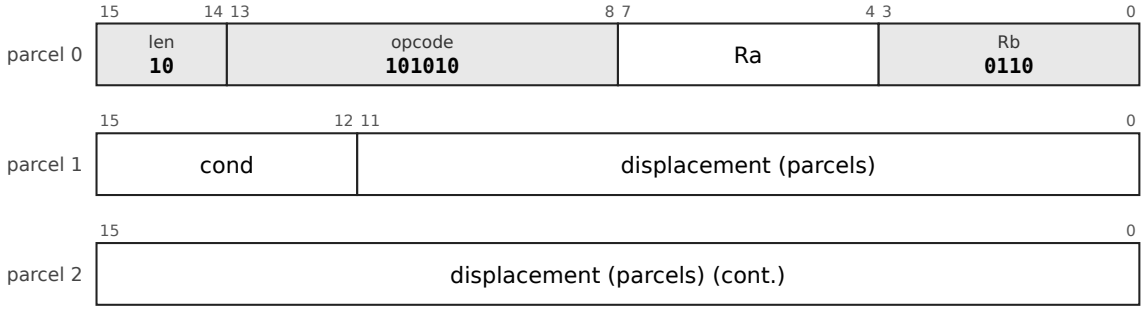
CBVS, opcode 101000, three parcels (6 bytes)



CBVS, opcode 101001, three parcels (6 bytes)



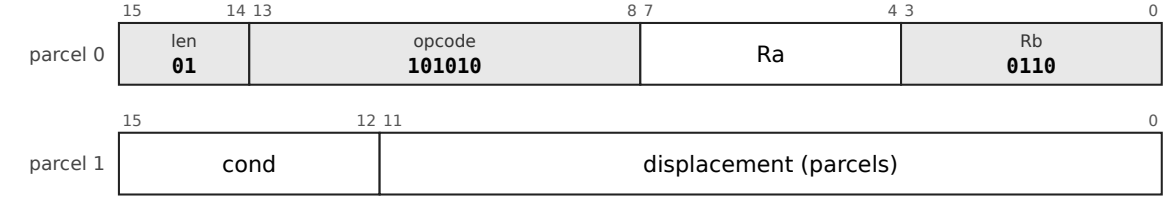
CBVS, opcode 101010, three parcels (6 bytes)



CBVS, opcode 101000, two parcels (4 bytes)



CBVS, opcode 101010, two parcels (4 bytes)

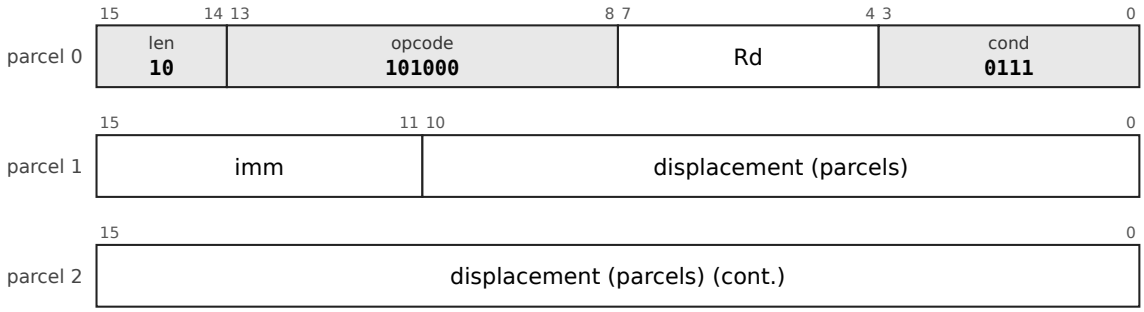


Mnemonic	Opcode	low
CBVS	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0110 (0x6), 0110 (0x6), 0110 (0x6)

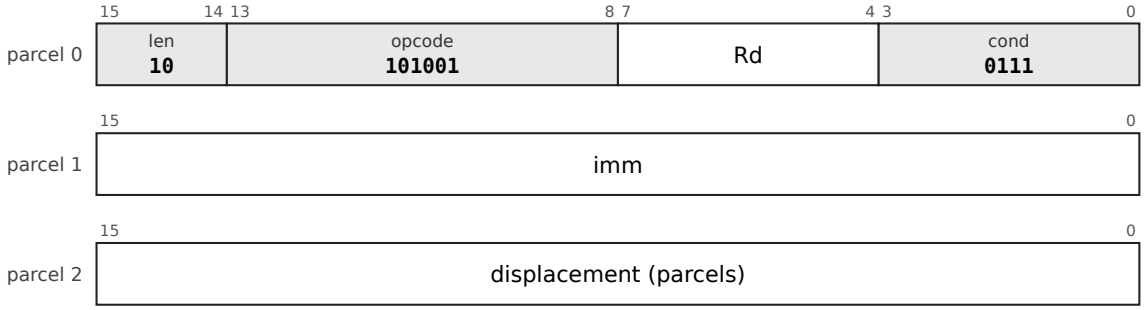
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBCC

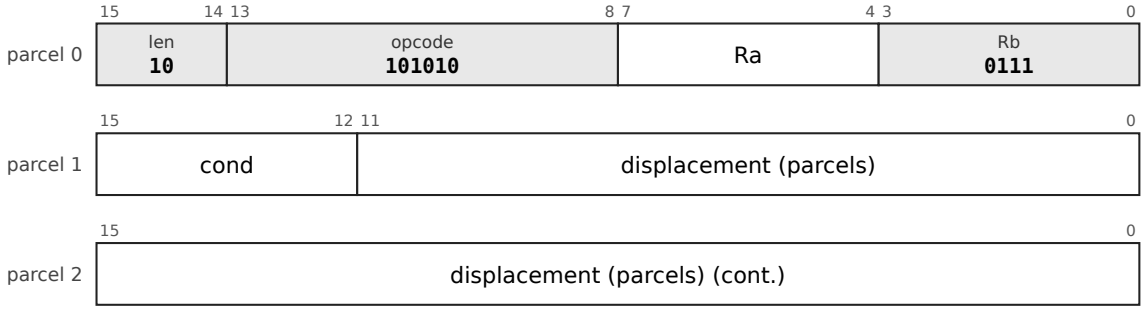
CBCC, opcode 101000, three parcels (6 bytes)



CBCC, opcode 101001, three parcels (6 bytes)



CBCC, opcode 101010, three parcels (6 bytes)



CBCC, opcode 101000, two parcels (4 bytes)



CBCC, opcode 101010, two parcels (4 bytes)

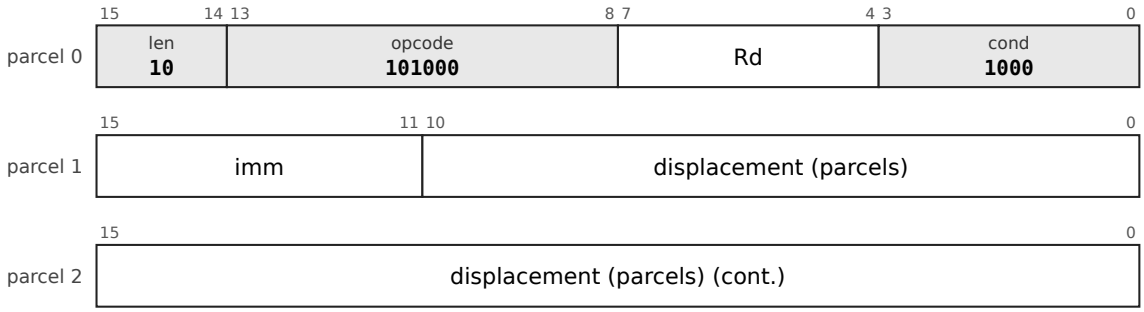


Mnemonic	Opcode	low
CBCC	101000 (0x28), 101001 (0x29), 101010 (0x2A)	0111 (0x7), 0111 (0x7), 0111 (0x7)

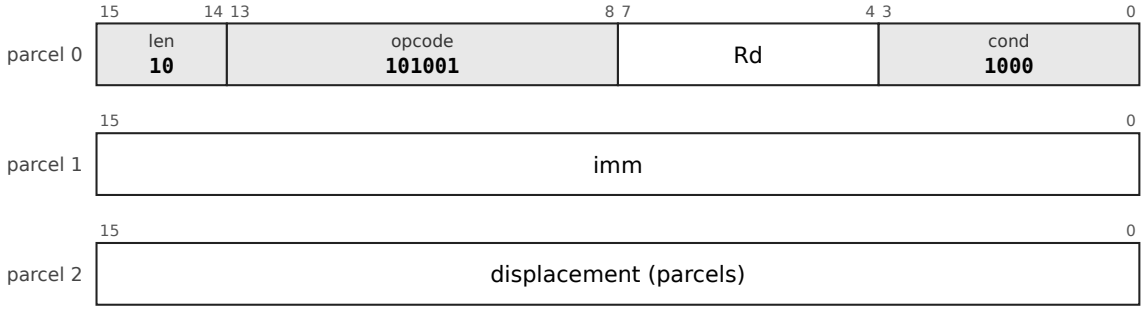
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBCS

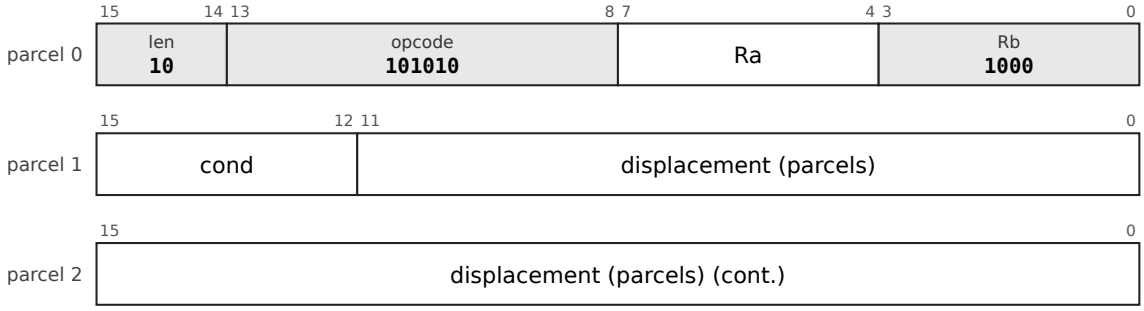
CBCS, opcode 101000, three parcels (6 bytes)



CBCS, opcode 101001, three parcels (6 bytes)



CBCS, opcode 101010, three parcels (6 bytes)



CBCS, opcode 101000, two parcels (4 bytes)



CBCS, opcode 101010, two parcels (4 bytes)

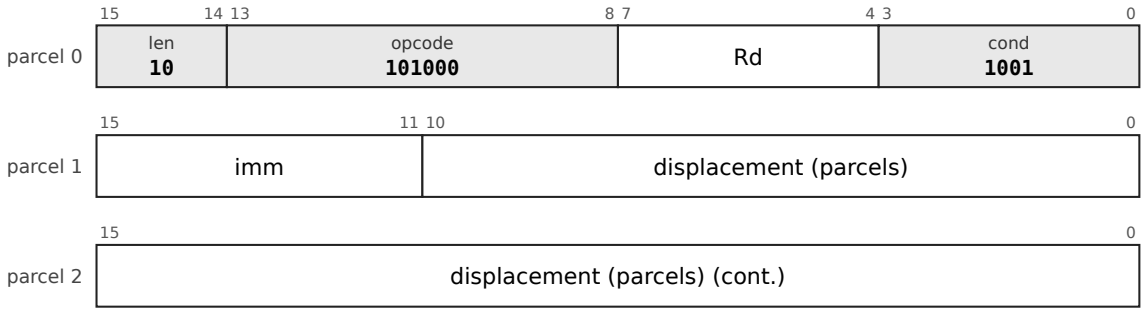


Mnemonic	Opcode	low
CBCS	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1000 (0x8), 1000 (0x8), 1000 (0x8)

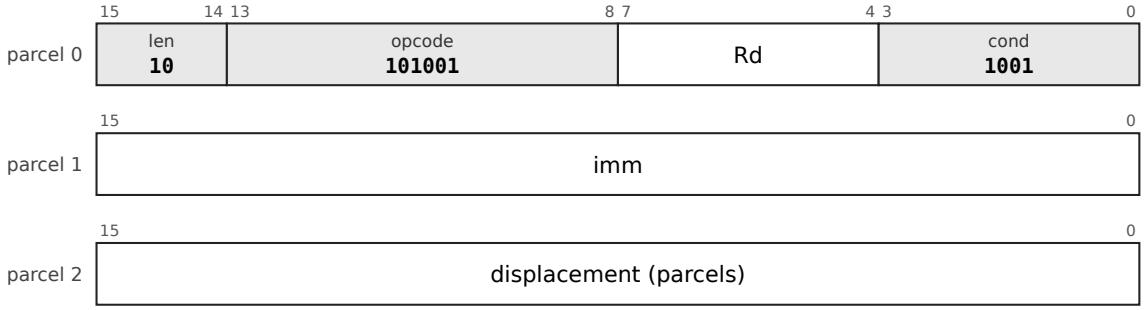
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBGE

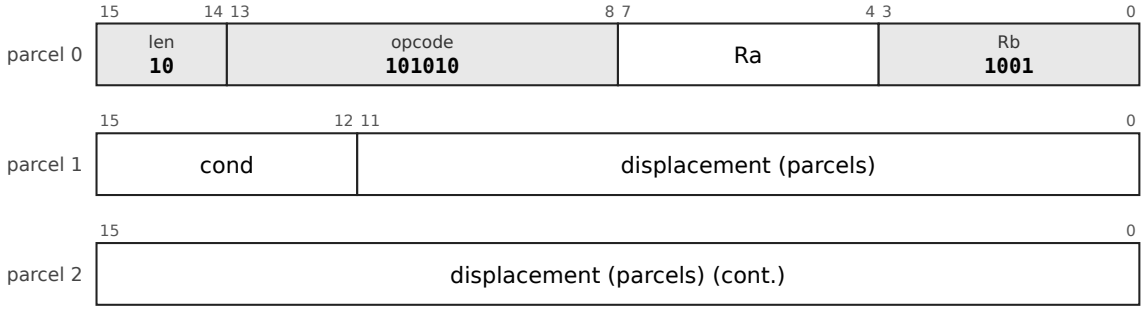
CBGE, opcode 101000, three parcels (6 bytes)



CBGE, opcode 101001, three parcels (6 bytes)



CBGE, opcode 101010, three parcels (6 bytes)



CBGE, opcode 101000, two parcels (4 bytes)



CBGE, opcode 101010, two parcels (4 bytes)

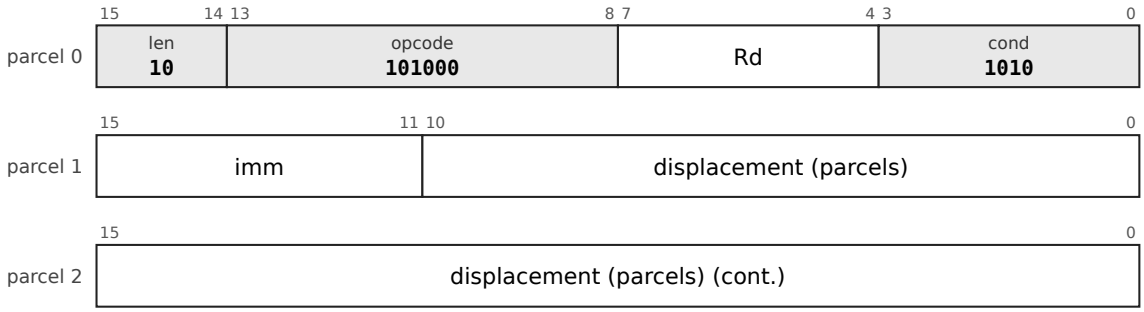


Mnemonic	Opcode	low
CBGE	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1001 (0x9), 1001 (0x9), 1001 (0x9)

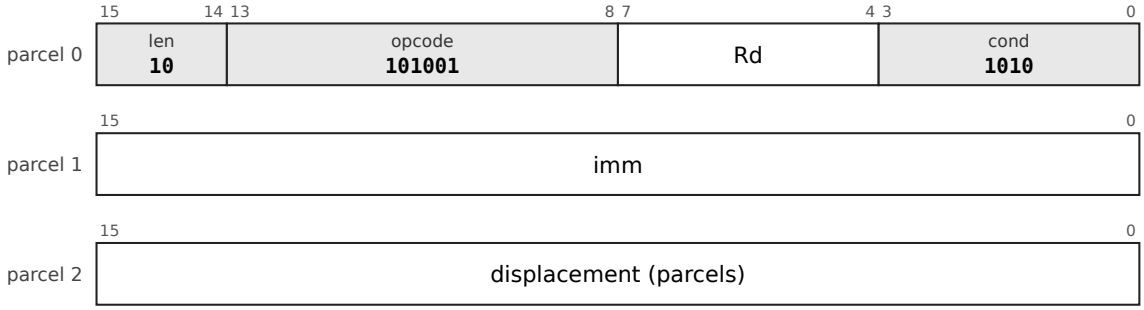
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBLT

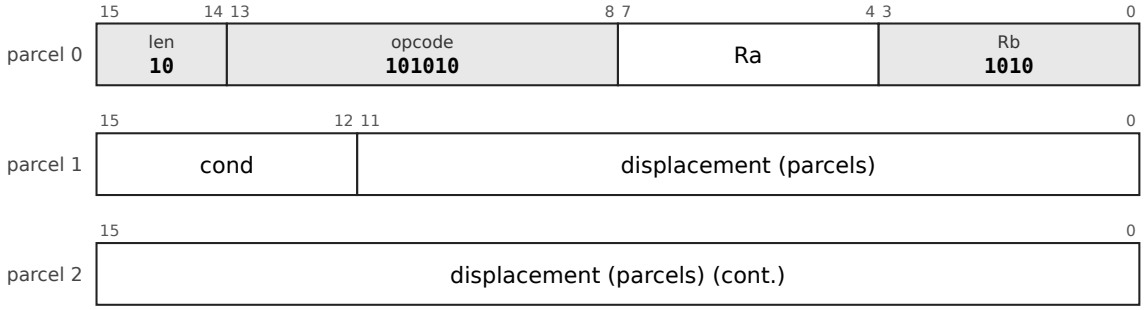
CBLT, opcode 101000, three parcels (6 bytes)



CBLT, opcode 101001, three parcels (6 bytes)



CBLT, opcode 101010, three parcels (6 bytes)



CBLT, opcode 101000, two parcels (4 bytes)



CBLT, opcode 101010, two parcels (4 bytes)

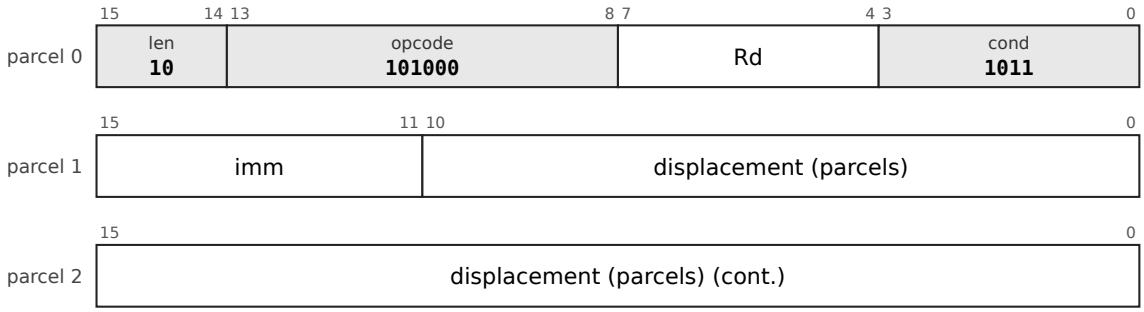


Mnemonic	Opcode	low
CBLT	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1010 (0xA), 1010 (0xA), 1010 (0xA)

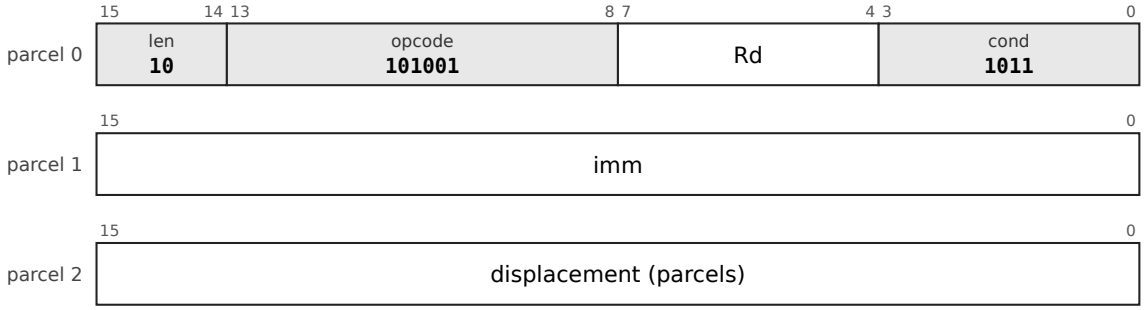
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBGT

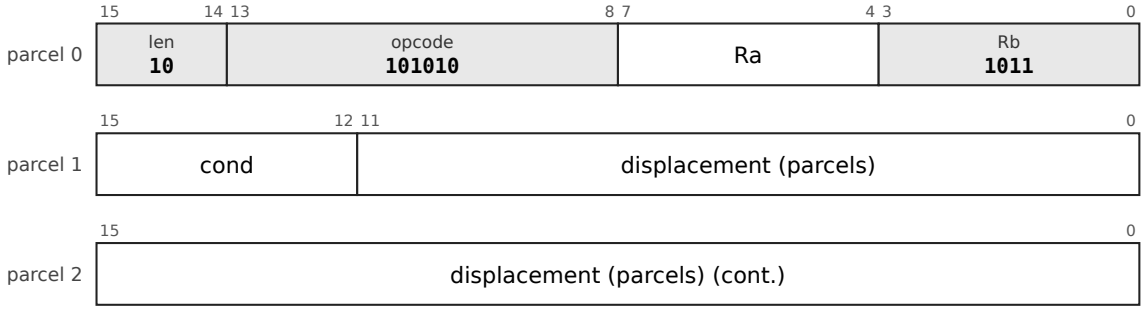
CBGT, opcode 101000, three parcels (6 bytes)



CBGT, opcode 101001, three parcels (6 bytes)



CBGT, opcode 101010, three parcels (6 bytes)



CBGT, opcode 101000, two parcels (4 bytes)



CBGT, opcode 101010, two parcels (4 bytes)

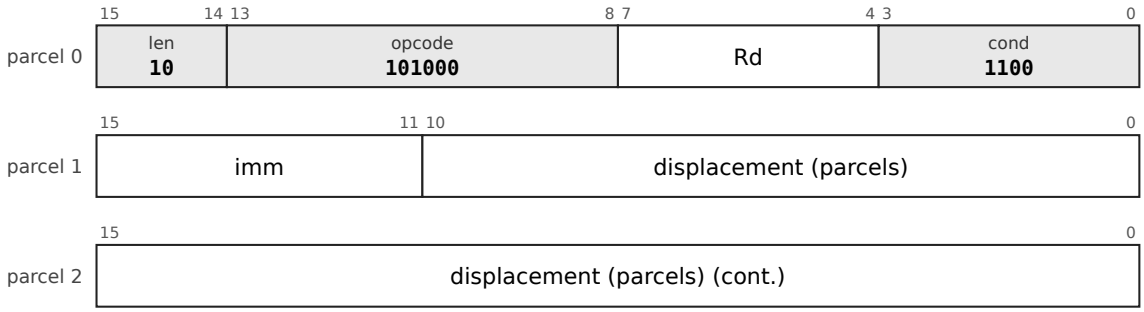


Mnemonic	Opcode	low
CBGT	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1011 (0xB), 1011 (0xB), 1011 (0xB)

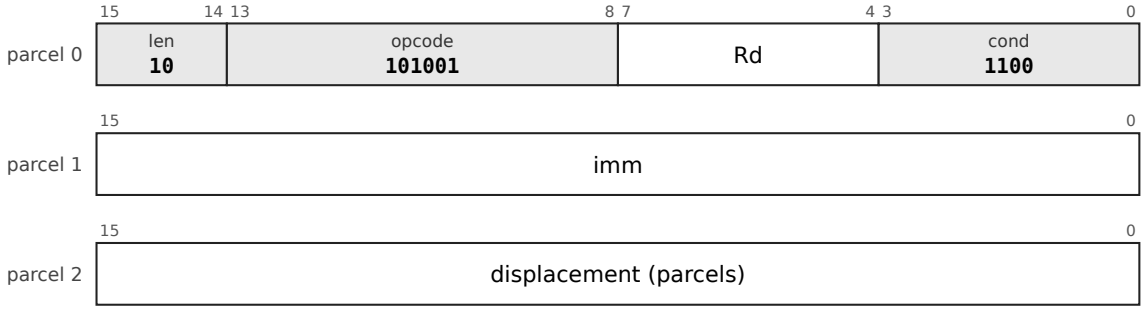
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBLE

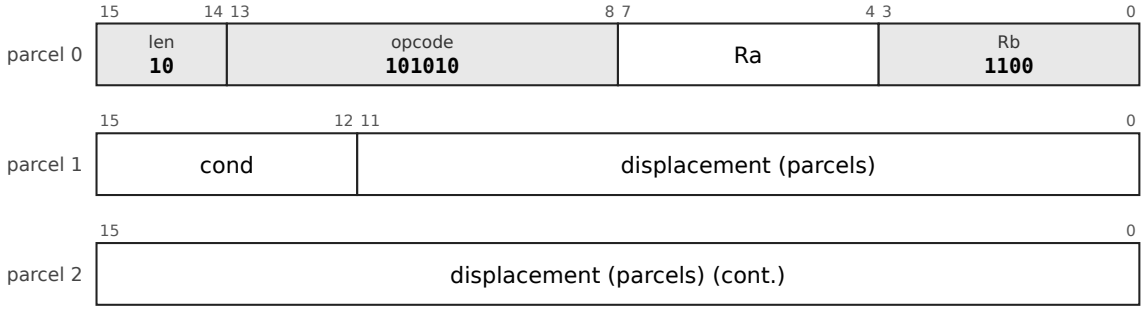
CBLE, opcode 101000, three parcels (6 bytes)



CBLE, opcode 101001, three parcels (6 bytes)



CBLE, opcode 101010, three parcels (6 bytes)



CBLE, opcode 101000, two parcels (4 bytes)



CBLE, opcode 101010, two parcels (4 bytes)

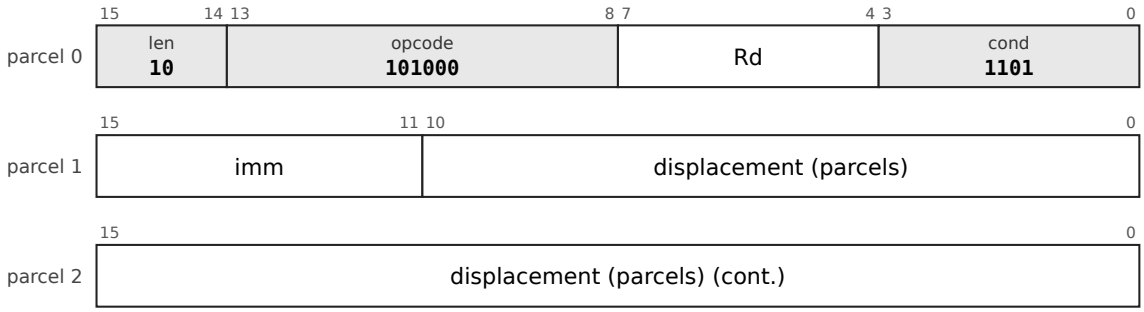


Mnemonic	Opcode	low
CBLE	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1100 (0xC), 1100 (0xC), 1100 (0xC)

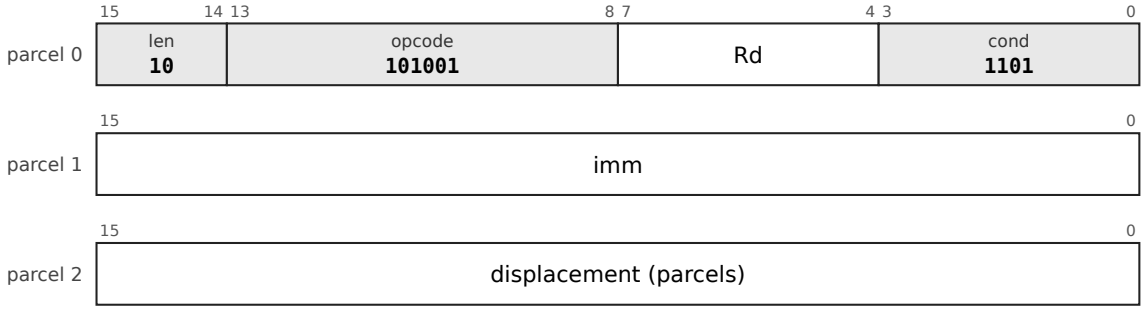
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBHI

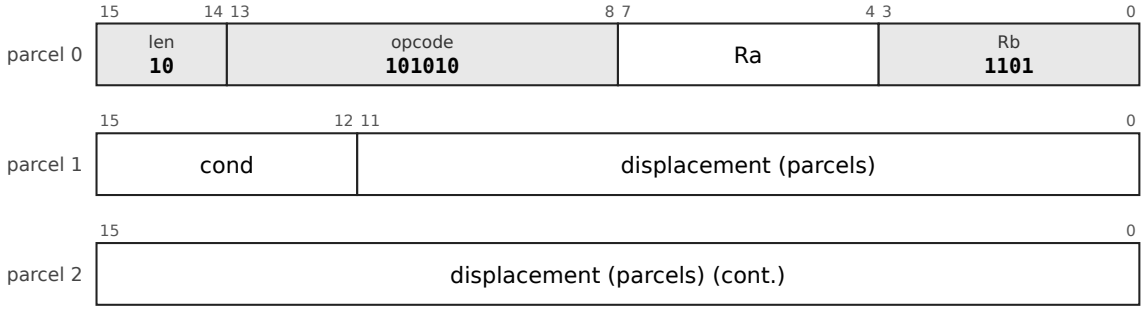
CBHI, opcode 101000, three parcels (6 bytes)



CBHI, opcode 101001, three parcels (6 bytes)



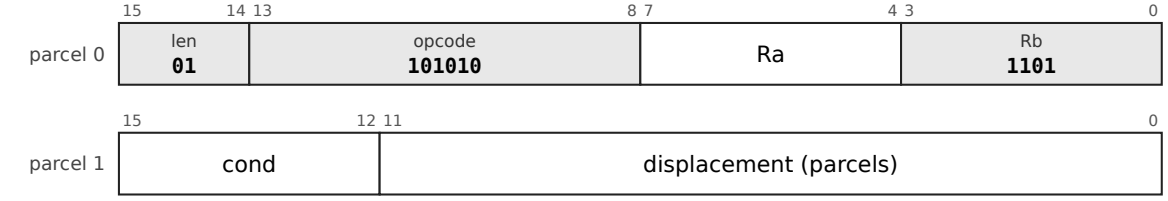
CBHI, opcode 101010, three parcels (6 bytes)



CBHI, opcode 101000, two parcels (4 bytes)



CBHI, opcode 101010, two parcels (4 bytes)

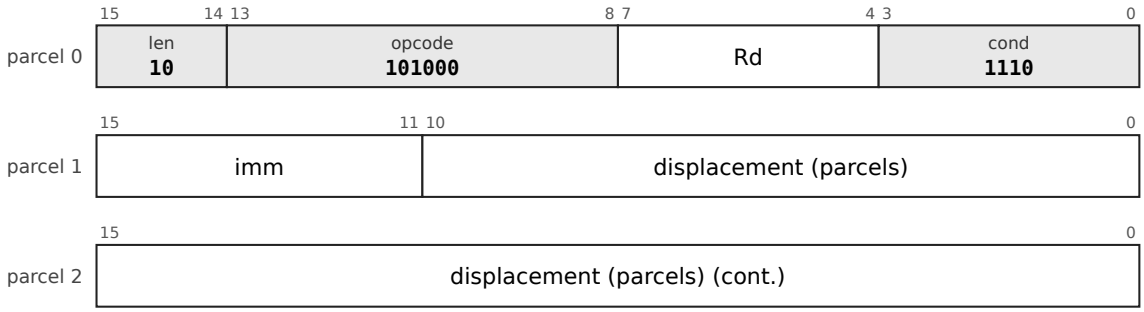


Mnemonic	Opcode	low
CBHI	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1101 (0xD), 1101 (0xD), 1101 (0xD)

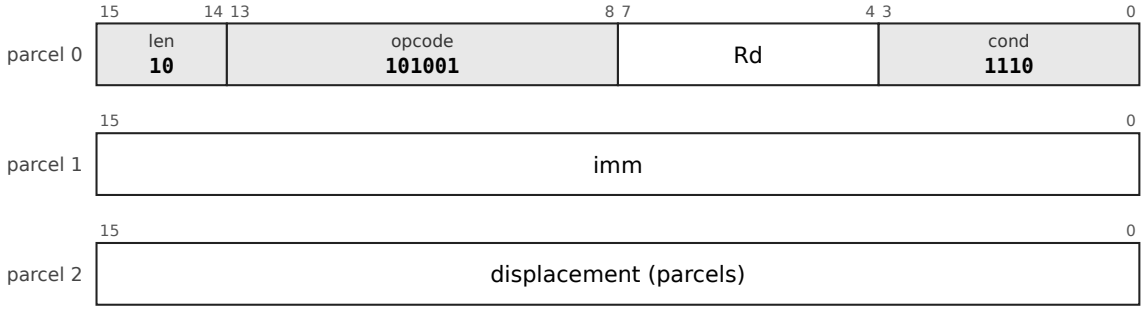
Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBLOS

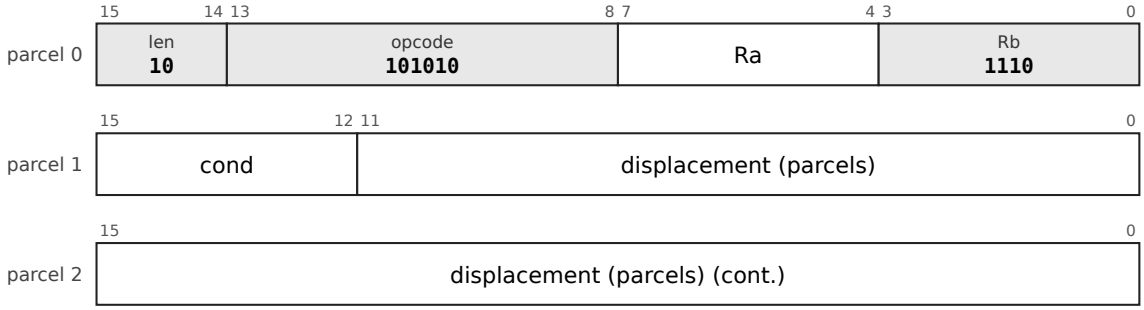
CBLOS, opcode 101000, three parcels (6 bytes)



CBLOS, opcode 101001, three parcels (6 bytes)



CBLOS, opcode 101010, three parcels (6 bytes)



CBLOS, opcode 101000, two parcels (4 bytes)



CBLOS, opcode 101010, two parcels (4 bytes)



Mnemonic	Opcode	low
CBLOS	101000 (0x28), 101001 (0x29), 101010 (0x2A)	1110 (0xE), 1110 (0xE), 1110 (0xE)

Operation, flags and exceptions as for the entry CBCC above; 5 layout forms, each a chart of its own (above).

CBHIS
An alias of CBCC: the assembler writes it as that word.

CBLO
An alias of CBCS: the assembler writes it as that word.

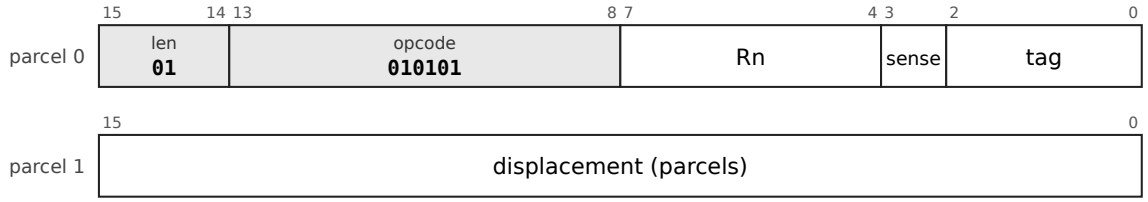
BTAG: Branch on pointer tag

Mnemonics: BTAG, BNTAG

Syntax

```
BTAG Rn, #tag, target
BNTAG Rn, #tag, target
```

Encoding



The chart is the layout r20-tag-1, Tag branches, two parcels (len = 01): sense is 0 for BTAG and 1 for BNTAG; tag is the 3-bit tag.

Mnemonic	Opcode
BTAG	010101 (0x15)
BNTAG	010101 (0x15)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-tag-1, r20-tag-2.

Operation

```
BTAG : if (Rn & 111₂) = tag then PC ← address of this instruction + 2 × disp
BNTAG: if (Rn & 111₂) ≠ tag then PC ← address of this instruction + 2 × disp
```

Flags None written.

Exceptions None.

For runtimes that keep a 3-bit type tag in the low bits of a pointer (lowtag). Four bytes at least; the displacement is 16 bits (32 at len = 10).

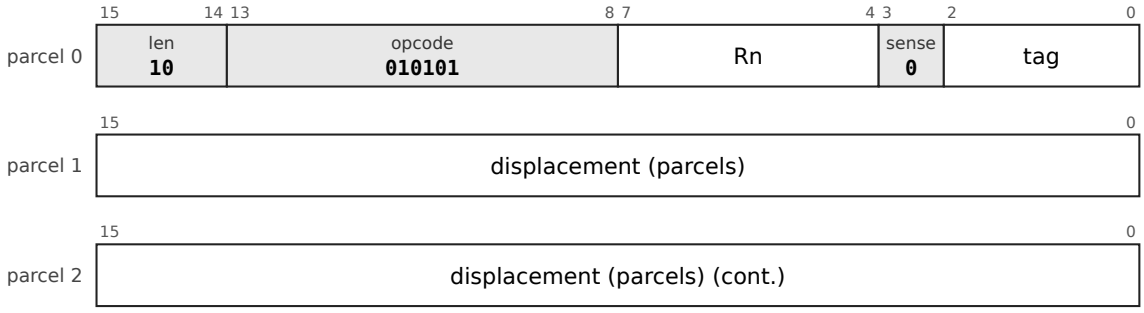
Why it is here. The tag branches BTAG and BNTAG and the table dispatch JTAG make tagged pointers cheap to use rather than merely possible: three bits of tag and one of sense fit the low nibble exactly, so parcel 1 is a full displacement. Because quad-

aligned objects have zero in the low three bits, the tag is free, and an unaligned access is a fault. The compiler for C never emits them (the corpus shows none); they are for Lisp and other tagged runtimes.

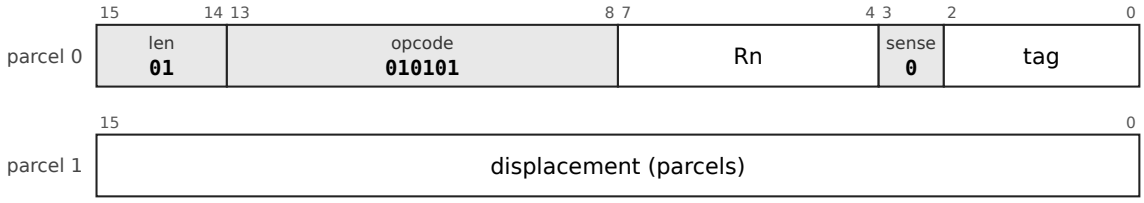
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

BTAG

BTAG, opcode 010101, three parcels (6 bytes)



BTAG, opcode 010101, two parcels (4 bytes)



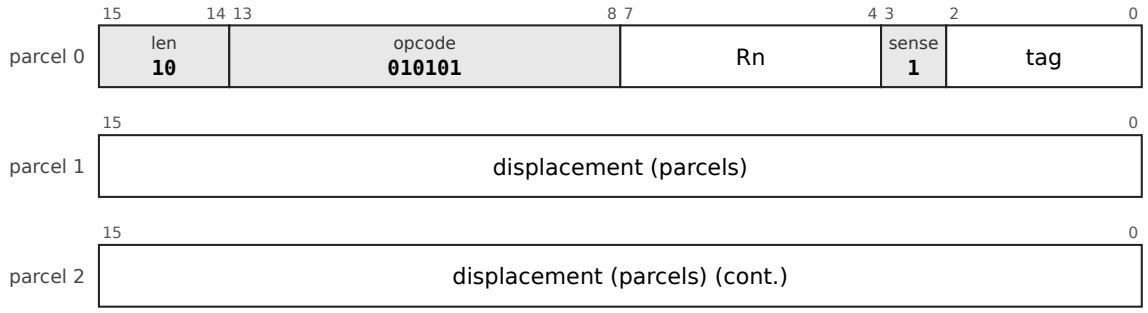

```
BTAG Rn, #tag, target
```

Mnemonic	Opcode
BTAG	010101 (0x15)

Operation, flags and exceptions as for the entry BTAG above; 2 layout forms, each a chart of its own (above).

BNTAG

BNTAG, opcode 010101, three parcels (6 bytes)



BNTAG, opcode 010101, two parcels (4 bytes)



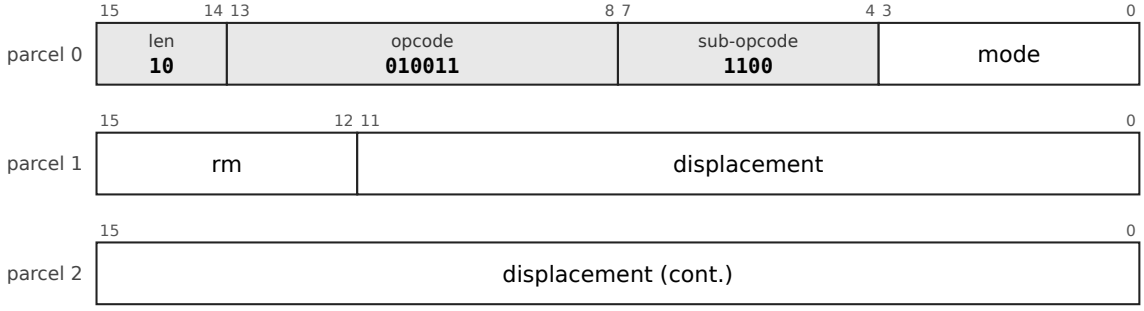

```
BNTAG Rn, #tag, target
```

Mnemonic	Opcode
BNTAG	010101 (0x15)

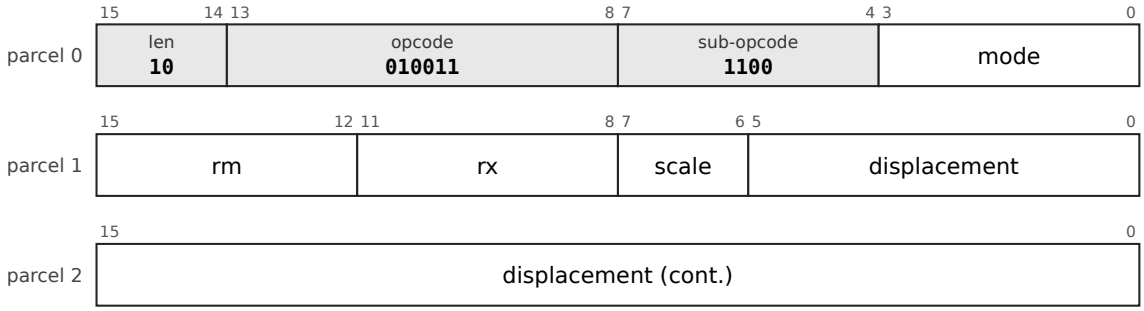
Operation, flags and exceptions as for the entry BTAG above; 2 layout forms, each a chart of its own (above).

JMP: Jump

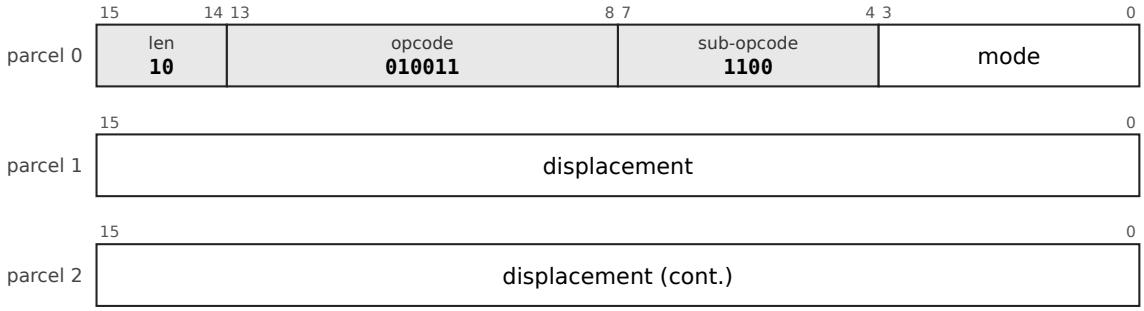
JMP, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



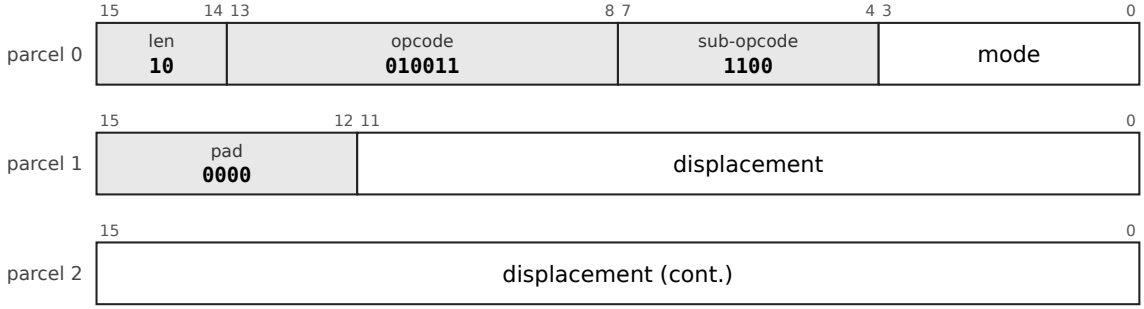
JMP, opcode 010011, three parcels (modes index) (6 bytes)



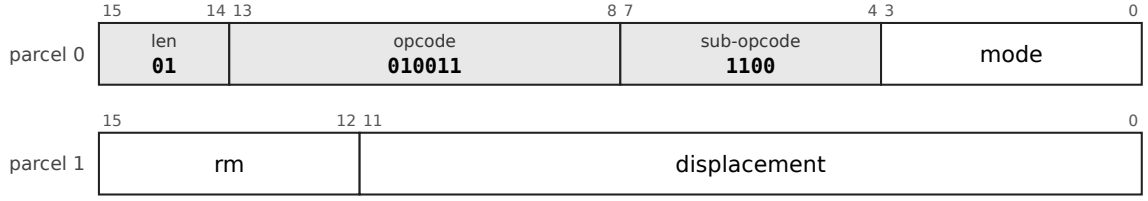
JMP, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



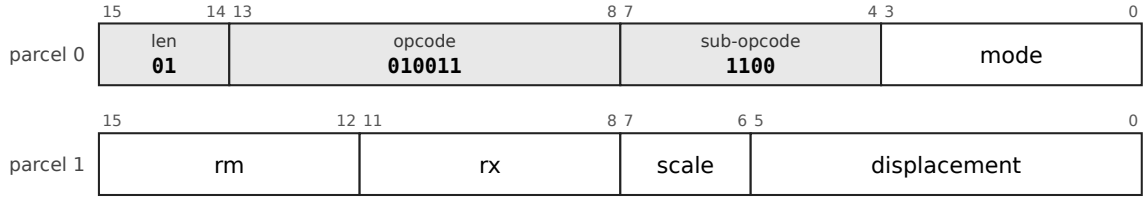
JMP, opcode 010011, three parcels (modes dp) (6 bytes)



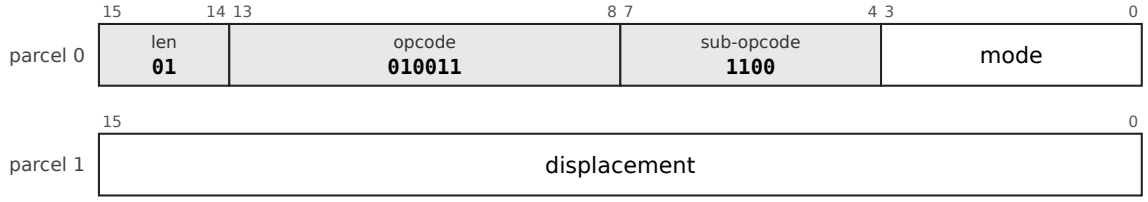
JMP, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



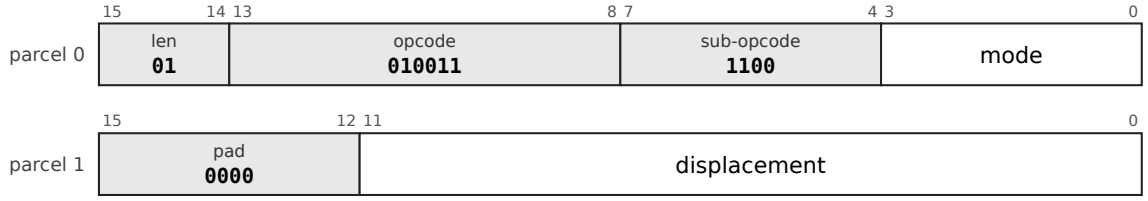
JMP, opcode 010011, two parcels (modes index) (4 bytes)



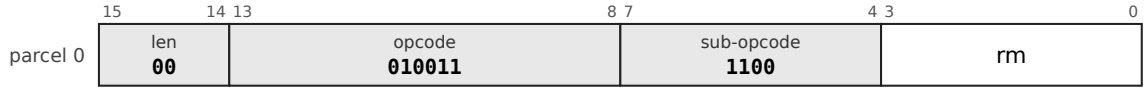
JMP, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



JMP, opcode 010011, two parcels (modes dp) (4 bytes)



JMP, opcode 010011, one parcel (2 bytes)

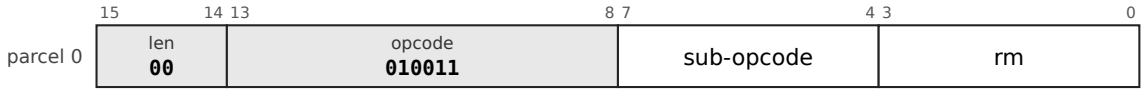


Mnemonics: `JMP`

Syntax

```
JMP rm        ; a register, or an address by any memory mode
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
JMP	010011 (0x13)	1100 (0xC)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

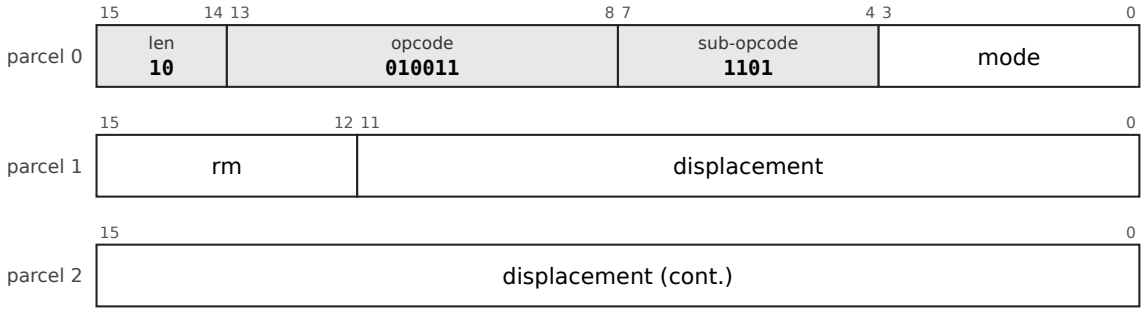
Operation

```
PC ← (register operand) ? Rm : EA(rm)
```

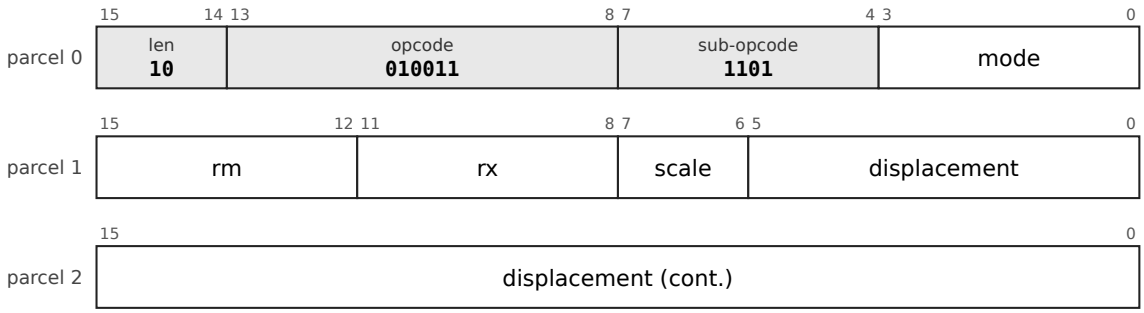
Flags None written.
Exceptions A jump to an unmapped or non-executable address faults on the fetch of the target.
A switch is a `CMP` against the limit, a `BHIS` to the default and a `JMP` through a scaled index.
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

CALL: Call

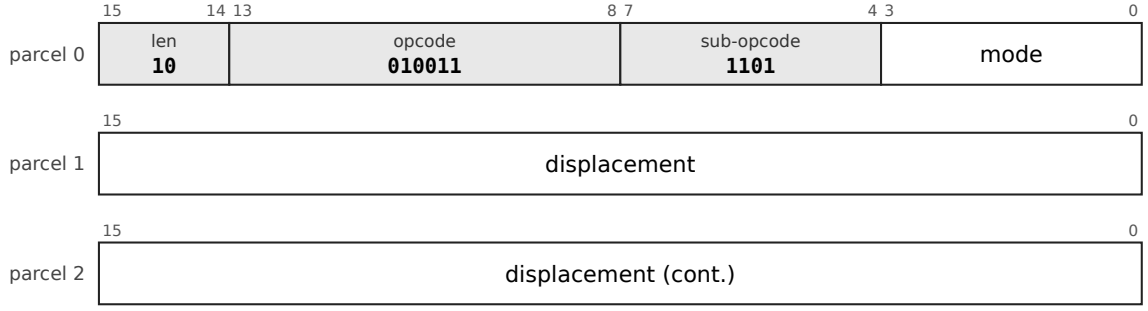
CALL, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



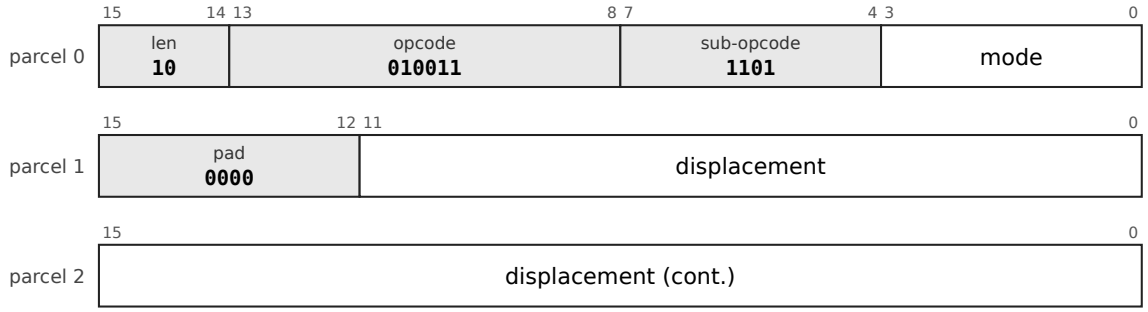
CALL, opcode 010011, three parcels (modes index) (6 bytes)



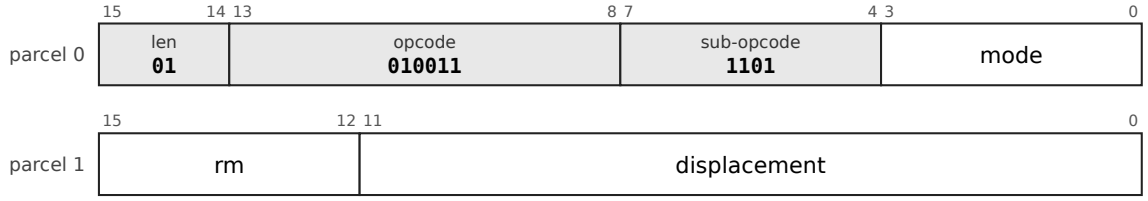
CALL, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



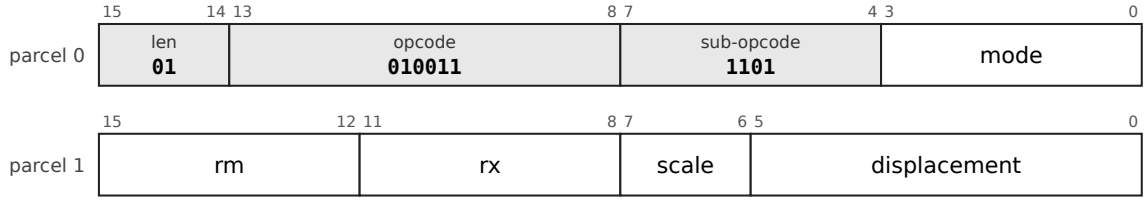
CALL, opcode 010011, three parcels (modes dp) (6 bytes)



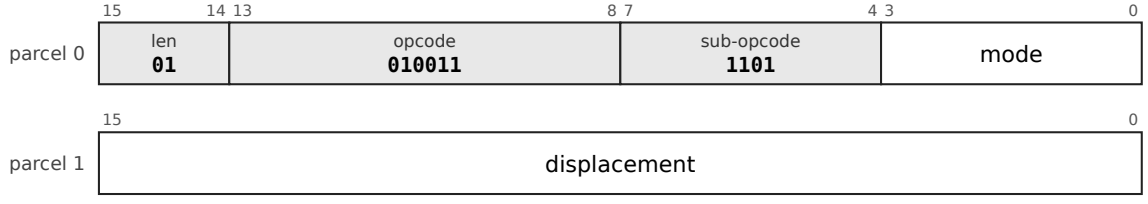
CALL, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



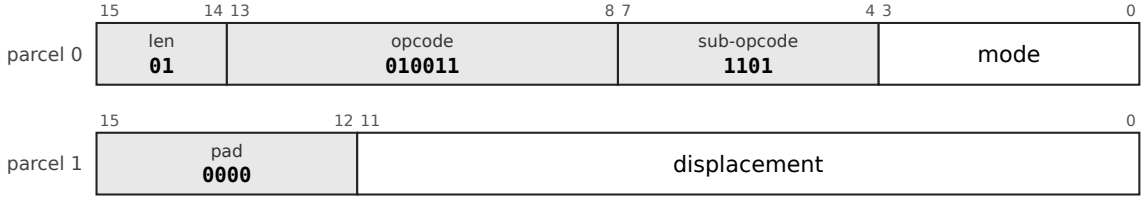
CALL, opcode 010011, two parcels (modes index) (4 bytes)



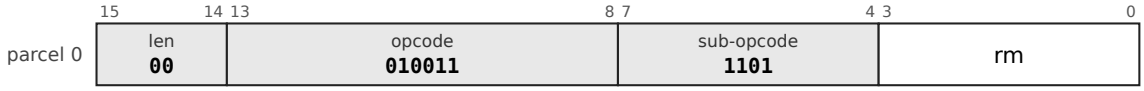
CALL, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



CALL, opcode 010011, two parcels (modes dp) (4 bytes)



CALL, opcode 010011, one parcel (2 bytes)

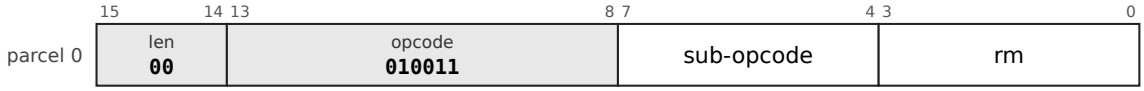


Mnemonics: `CALL`

Syntax

```
CALL rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`CALL`	`010011 (0x13)`	`1101 (0xD)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
t ← (register operand) ? Rm : EA(rm)
SP ← SP - 8 ; M[SP] ← address of the next instruction
PC ← t
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The return address is on the data stack where every unwinder and debugger expects it. `RET` pops it.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

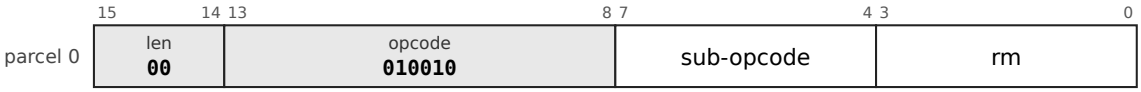
JMPI: Jump and call through memory

Mnemonics: `JMPI`, `CALLI`

Syntax

```
JMPI rm        ; PC ← M[EA]
CALLI rm       ; push the return address, then PC ← M[EA]
```

Encoding



The chart is the layout `r20-xfer-0`, Indirect transfers (XFER), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
JMPI	010010 (0x12)	0000 (0x0)
CALLI	010010 (0x12)	0001 (0x1)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-xfer-0`, `r20-xfer-1`, `r20-xfer-1-2`, `r20-xfer-1-3`, `r20-xfer-1-4`, `r20-xfer-2`, `r20-xfer-2-2`, `r20-xfer-2-3`, `r20-xfer-2-4`.

Operation

```
JMPI : PC ← M[EA(rm)]
CALLI: t ← M[EA(rm)] ; SP ← SP - 8 ; M[SP] ← address of the next instruction ; PC ← t
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. For `CALLI` the order is specified: load the target, then push, then transfer; a fault on the load leaves nothing changed.

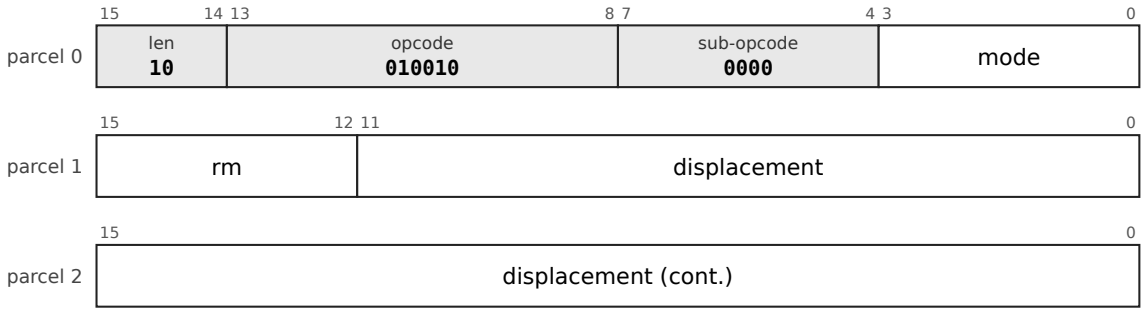
The one deferred reference the architecture admits, for control transfer only: the PLT stub (`JMPI got_slot(PC)`) and virtual-method dispatch (`CALLI vtbl_off(Robj)`). It needs a memory operand, so it is four bytes at least.

Why it is here. Exactly one deferred reference, for control transfer only, in `XFER`: the PLT stub of every dynamically linked program (`JMPI got_slot(PC)`) and virtual-method dispatch (`CALLI vtbl_off(Robj)`), each two instructions and a scratch register without it. It is admitted because it breaks none of the reasons deferred modes are removed (no register is side-effected, the reference is in the branch path and not the operand path, a fault is clean by ordering). Jump tables through `JMPI` are not smaller, because the table entries are 8 bytes against 8 for a case in a compare chain.

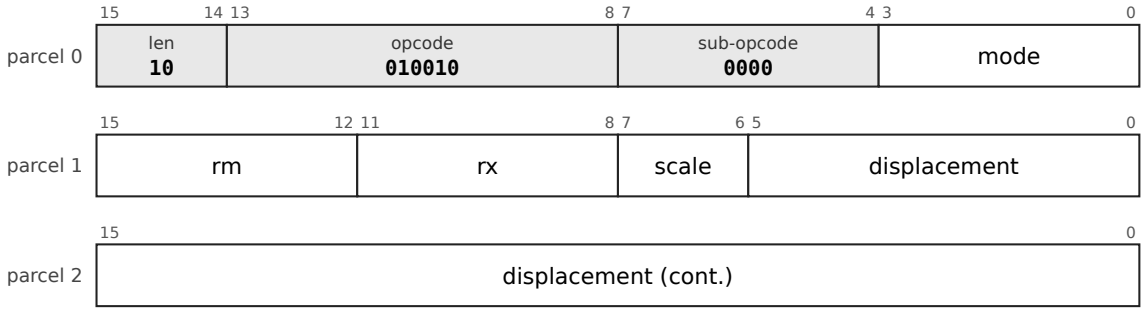
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

JMPI

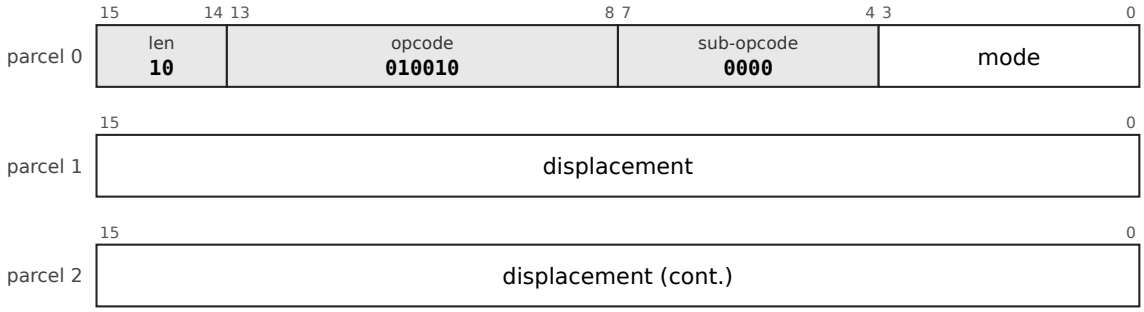
JMPI, opcode 010010, three parcels (modes base, postinc, predec) (6 bytes)



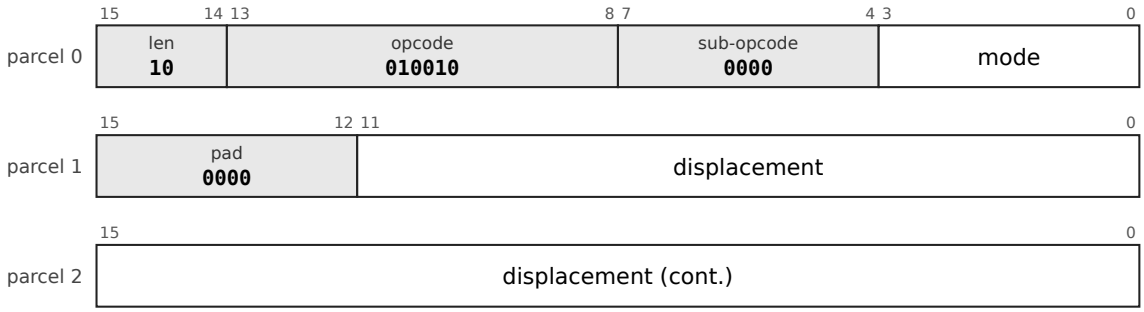
JMPI, opcode 010010, three parcels (modes index) (6 bytes)



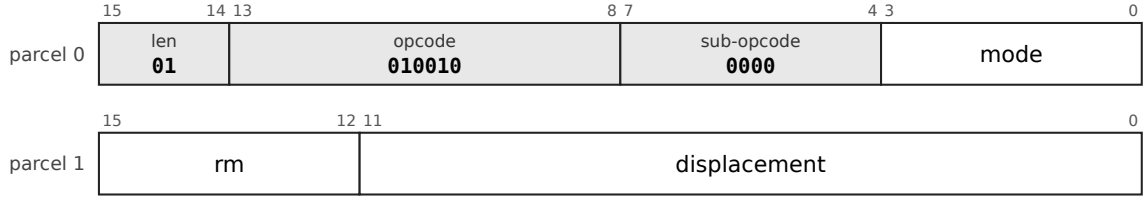
JMPI, opcode 010010, three parcels (modes pcrel, imm, abs) (6 bytes)



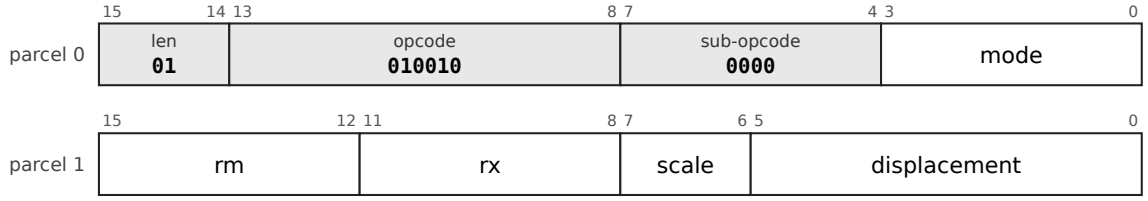
JMPI, opcode 010010, three parcels (modes dp) (6 bytes)



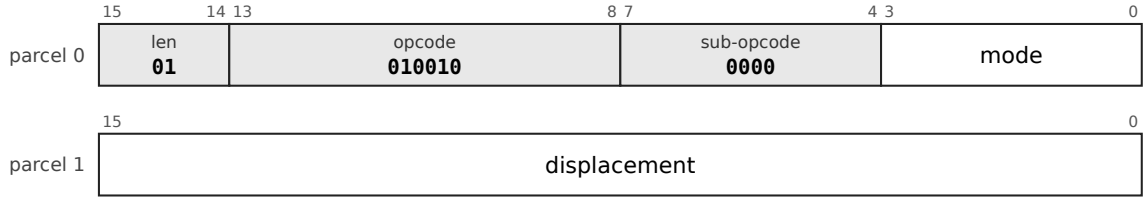
JMPI, opcode 010010, two parcels (modes base, postinc, predec) (4 bytes)



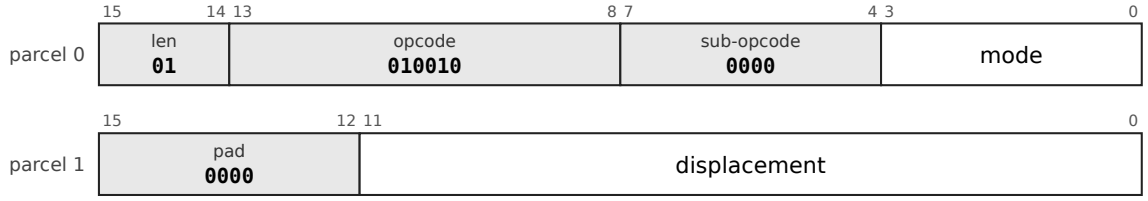
JMPI, opcode 010010, two parcels (modes index) (4 bytes)



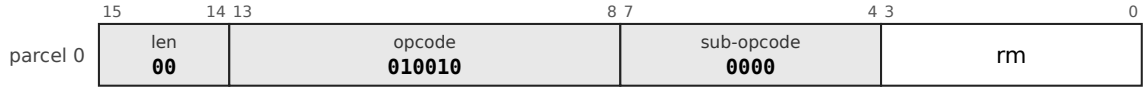
JMPI, opcode 010010, two parcels (modes pcrel, imm, abs) (4 bytes)



JMPI, opcode 010010, two parcels (modes dp) (4 bytes)



JMPI, opcode 010010, one parcel (2 bytes)



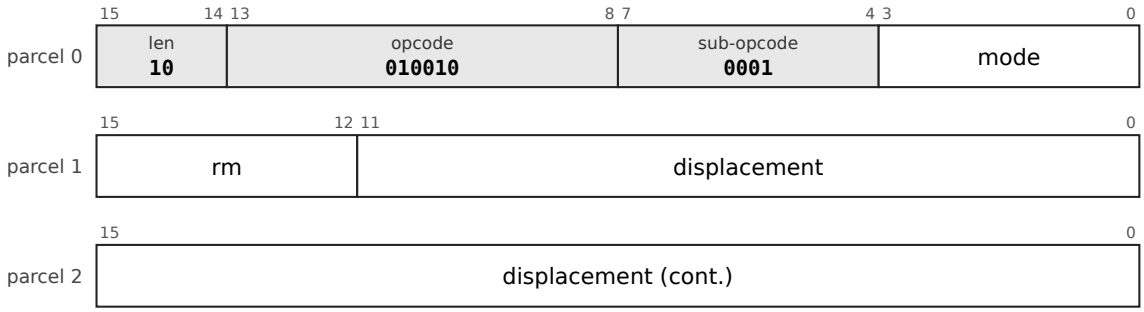

```
JMPI rm
```

Mnemonic	Opcode	reg
JMPI	010010 (0x12)	0000 (0x0)

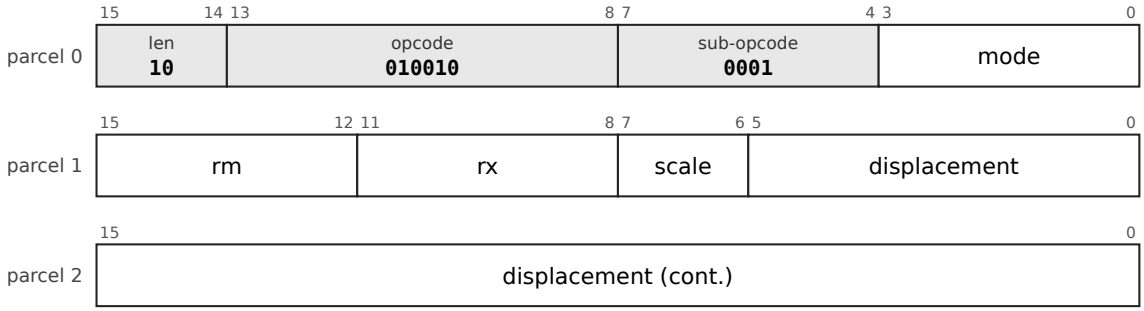
Operation, flags and exceptions as for the entry JMPI above; 9 layout forms, each a chart of its own (above).

CALLI

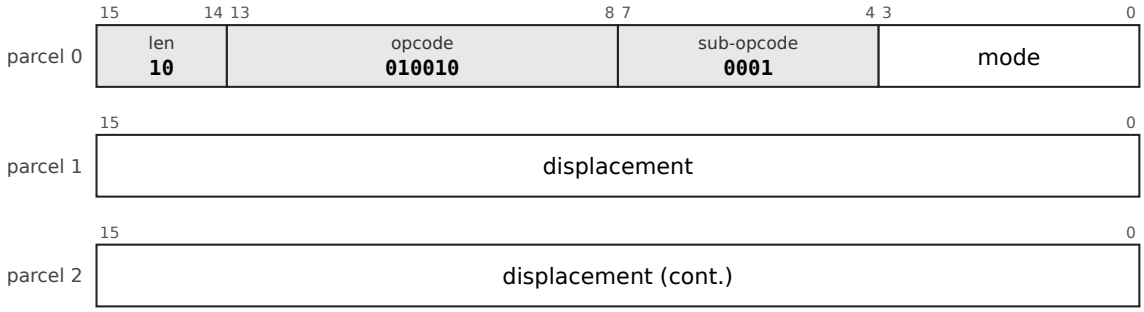
CALLI, opcode 010010, three parcels (modes base, postinc, predec) (6 bytes)



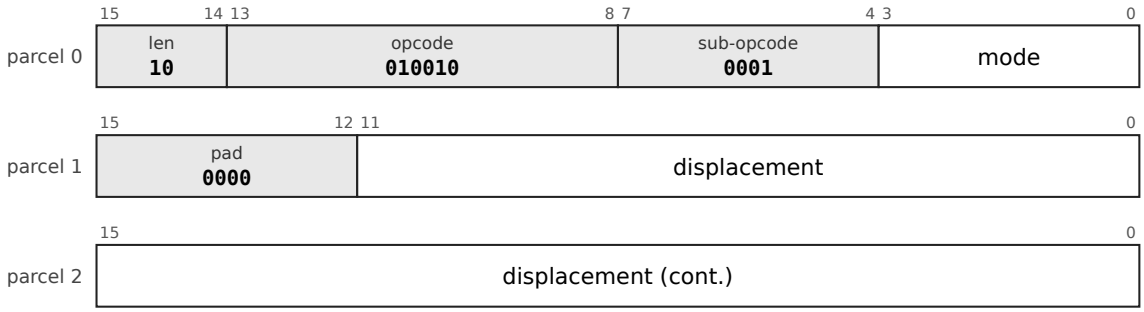
CALLI, opcode 010010, three parcels (modes index) (6 bytes)



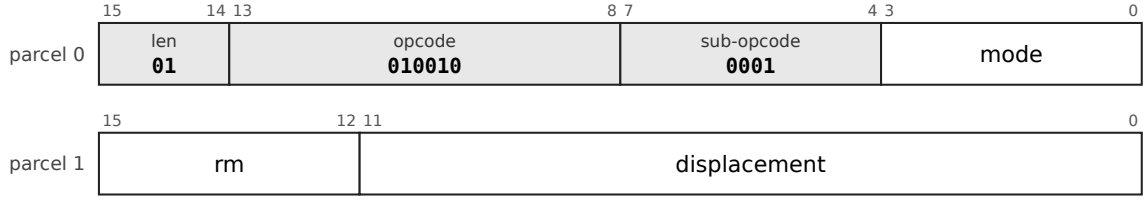
CALLI, opcode 010010, three parcels (modes pcrel, imm, abs) (6 bytes)



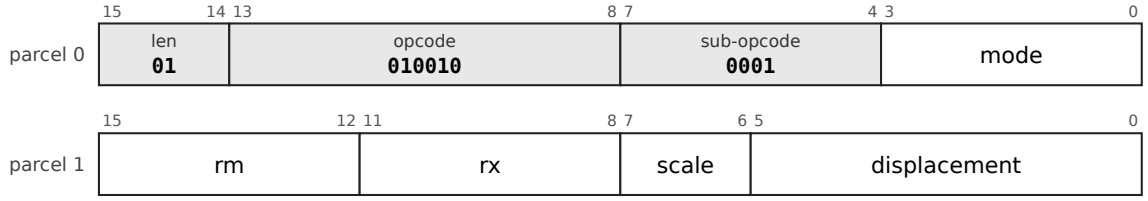
CALLI, opcode 010010, three parcels (modes dp) (6 bytes)



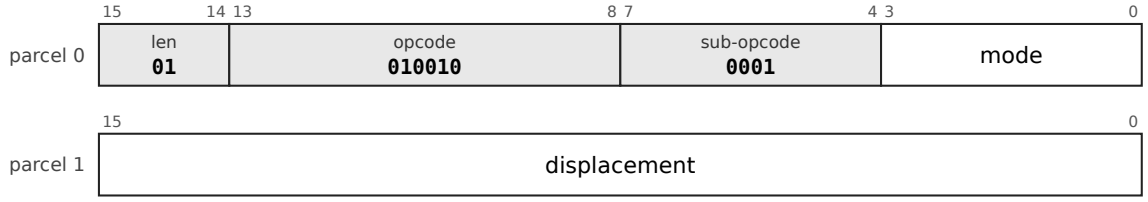
CALLI, opcode 010010, two parcels (modes base, postinc, predec) (4 bytes)



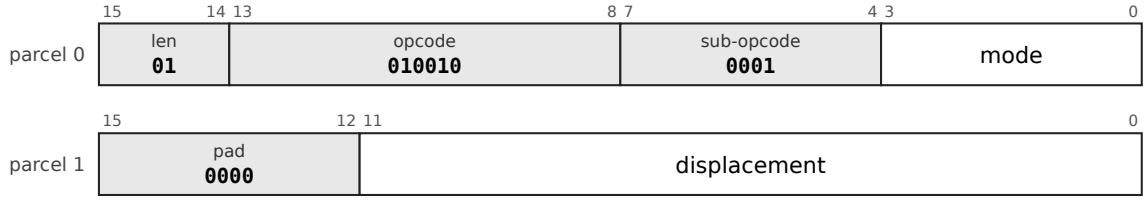
CALLI, opcode 010010, two parcels (modes index) (4 bytes)



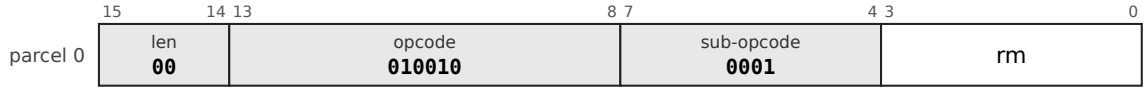
CALLI, opcode 010010, two parcels (modes pcrel, imm, abs) (4 bytes)



CALLI, opcode 010010, two parcels (modes dp) (4 bytes)



CALLI, opcode 010010, one parcel (2 bytes)



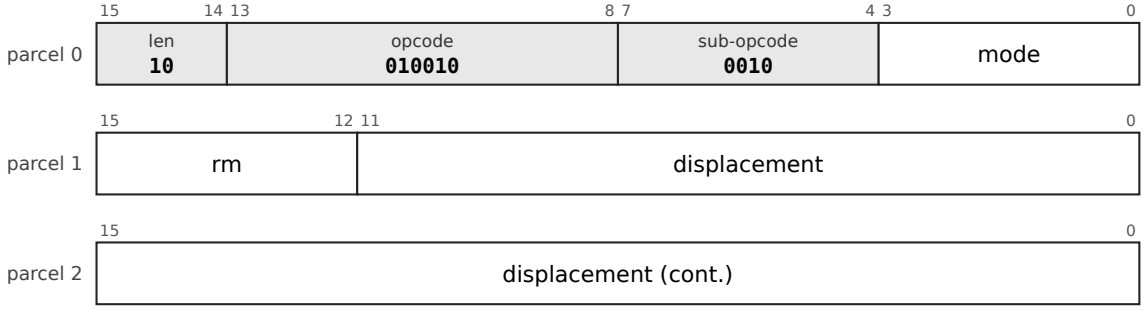

```
CALLI rm
```

Mnemonic	Opcode	reg
CALLI	010010 (0x12)	0001 (0x1)

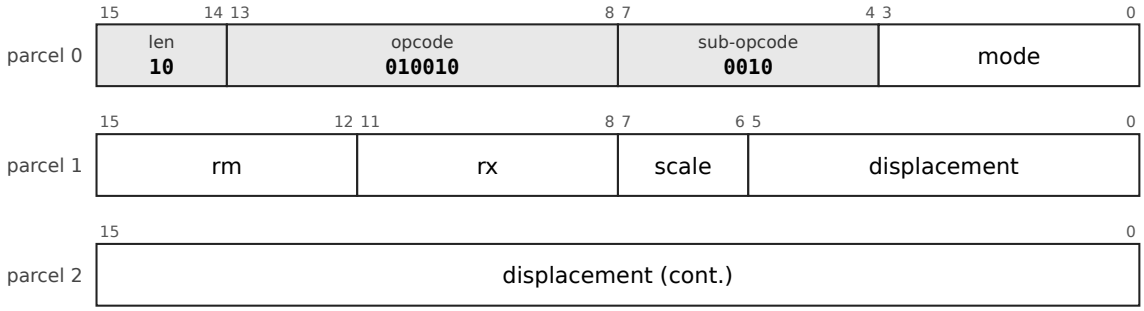
Operation, flags and exceptions as for the entry JMPI above; 9 layout forms, each a chart of its own (above).

JTAG: Jump through a tag table

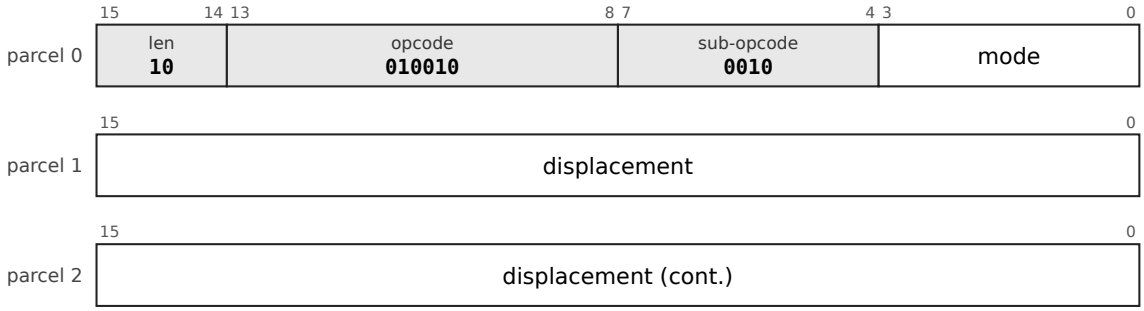
JTAG, opcode 010010, three parcels (modes base, postinc, predec) (6 bytes)



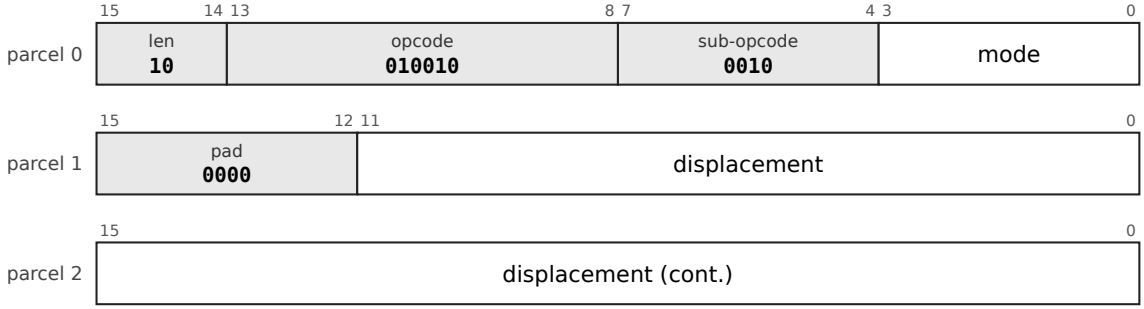
JTAG, opcode 010010, three parcels (modes index) (6 bytes)



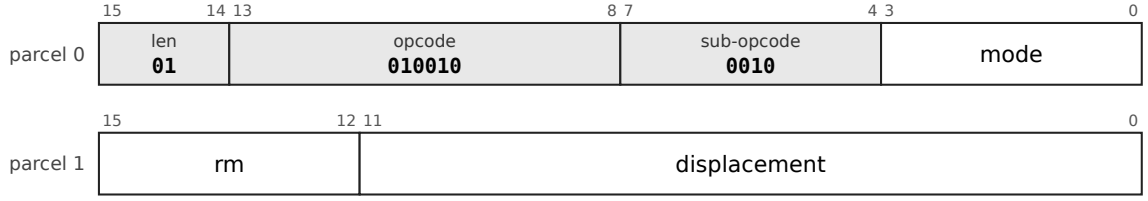
JTAG, opcode 010010, three parcels (modes pcrel, imm, abs) (6 bytes)



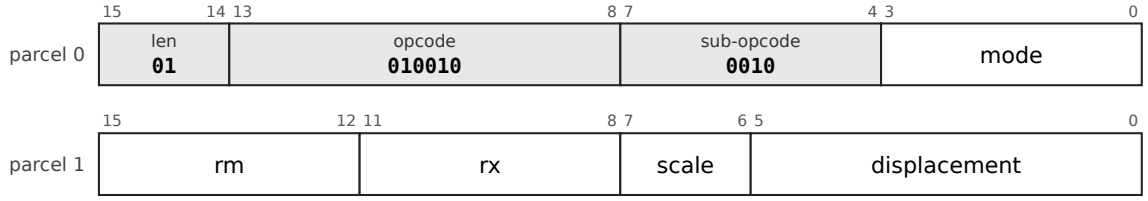
JTAG, opcode 010010, three parcels (modes dp) (6 bytes)



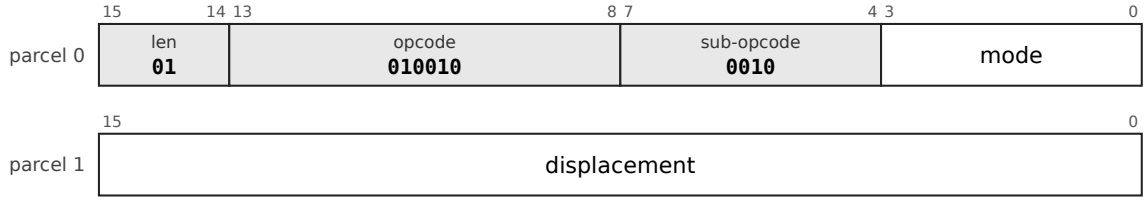
JTAG, opcode 010010, two parcels (modes base, postinc, predec) (4 bytes)



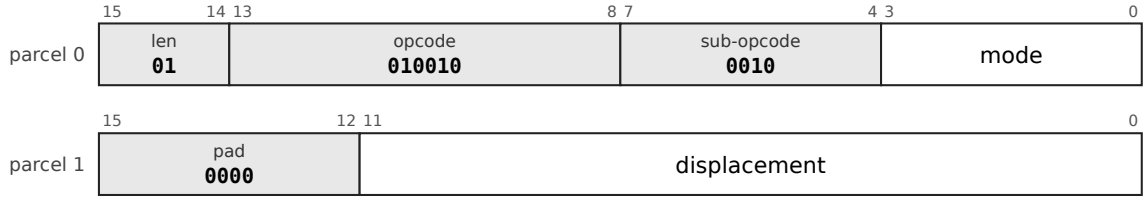
JTAG, opcode 010010, two parcels (modes index) (4 bytes)



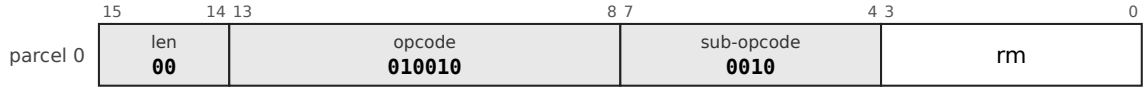
JTAG, opcode 010010, two parcels (modes pcrel, imm, abs) (4 bytes)



JTAG, opcode 010010, two parcels (modes dp) (4 bytes)



JTAG, opcode 010010, one parcel (2 bytes)

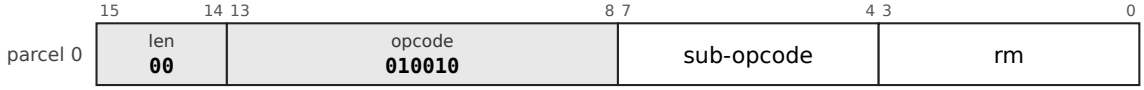


Mnemonics: `JTAG`

Syntax

```
JTAG d(Rb)(Rx×s)
```

Encoding



The chart is the layout `r20-xfer-0`, Indirect transfers (XFER), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
JTAG	010010 (0x12)	0010 (0x2)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-xfer-0`, `r20-xfer-1`, `r20-xfer-1-2`, `r20-xfer-1-3`, `r20-xfer-1-4`, `r20-xfer-2`, `r20-xfer-2-2`, `r20-xfer-2-3`, `r20-xfer-2-4`.

Operation

```
PC ← M[Rb + d + 8 × (Rx & 111₂)]
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Legal only with the scaled-index mode (`0001`). Dispatches on the low three bits of a tagged pointer through a table of eight entries; the 3-bit tag cannot index out of range.

Why it is here. The table-dispatch partner of BTAG. It is legal only in the scaled-index mode, the one mode with a free register slot for the tagged value. It replaces the PDP-11 dispatch-table idiom `@(Rn)+` better, because it needs no pointer register and cannot index out of range.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

9.11 The stack and block operations

Cf. section 6.4 for restarting a block operation after a fault (progress is held in registers) and section 6.6 for the page faults it can take; section 7.2 for how block operations order their accesses.

9.11.1 PUSH and POP move one register; PUSHM and POPM move a set named by a mask (a short mask form is in page 2). A mask containing R14 is reserved. *(D)*

9.11.2 PUSHX and POPX save and restore the exception state EPSW and EPC in one instruction each; both words are checked before either is accessed, so an abort leaves nothing changed. *(D)*

9.11.3 The block operations MCPY, MMOV (overlap-safe), MSET, MCMP, MSCN and MSCNN take a destination pointer, a source pointer or value and a count in registers; MVAL validates a buffer as UTF-8. *(opcode `001101 (0x0D)`)*

9.11.4 A block operation keeps its progress in its registers: when it completes, the count is zero and the pointers are one past the end; when it stops early or is interrupted, they point where it stopped, and it restarts by running again. *(D)*

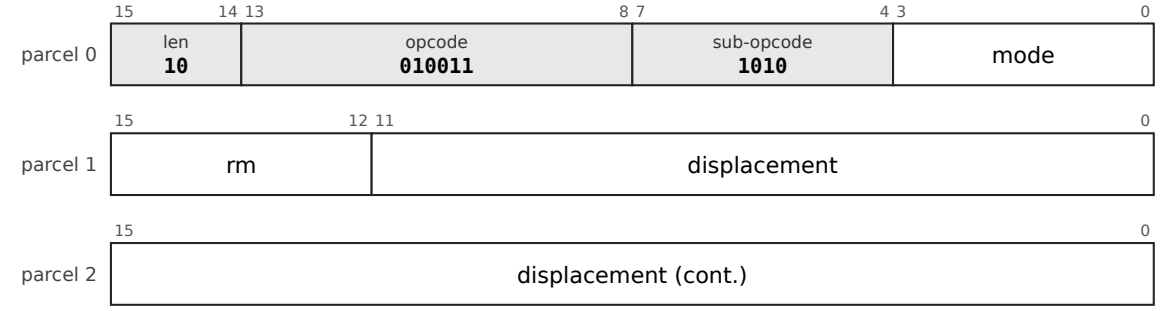
9.11.5 An implementation may move more than a byte per step, provided the registers are consistent with byte-at-a-time progress wherever an exception can be taken. *(D)*

Commentary. **Precedent.** The VAX's MOVC5 is the canonical argument against such instructions: its operands could be memory, its architectural definition needed the original values to survive, and so it required hidden state and a first-part-done bit in the status word to resume. The PDP-10 had a block transfer, and x86's REP MOVS keeps its progress in registers as the pdp64's do. **Trade-offs.** This is the one admitted exception to the one-memory-operand rule, accepted because its mechanism, hidden state, is absent: the state is the registers, which are already architectural. A large copy can evict the whole data cache, and a non-temporal variant is an open question. **Improvements.** ARM reached the same design in 2022 with FEAT_MOPS, whose specification says the registers are updated so that progress is not lost. **Benefits.** The block-move sequencer is three counters and a byte-lane selector, about 1,500 transistors. **Modern comparables.** x86 REP MOVSB and REP STOSB with fast-string microcode; AArch64 CPYP/CPYM/CPYE and SETP/SETM/SETE; s390 MVC. **Use.** `memcpy`, `memmove`, `memset`, `memcmp` and `memchr` in the C library; MVAL in parsers and kernels. **Impact.** *Kernel:* PUSHM, POPM, PUSHX and POPX shorten entry, exit and the context switch, and

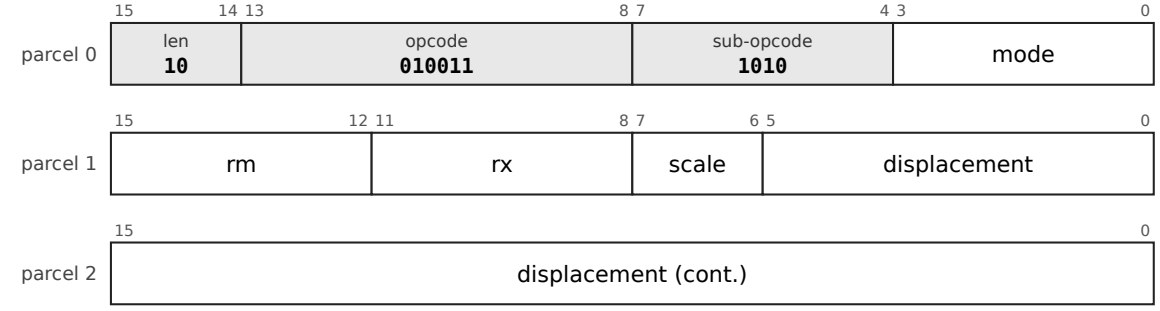
the block operations are `copyin`, `copyout`, page clearing and `memcpy`. *Compiler:* A prologue and an epilogue are one instruction each, and `memcpy` and `memset` lower to the block operations. *Hardware:* The block-move sequencer is three counters and a byte-lane selector, about 1,500 transistors, with no hidden state.

PUSHM: Push registers by mask

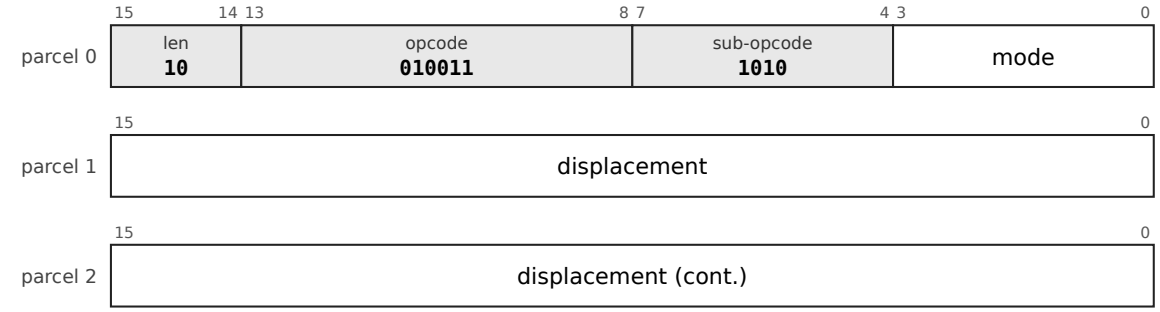
PUSHM, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



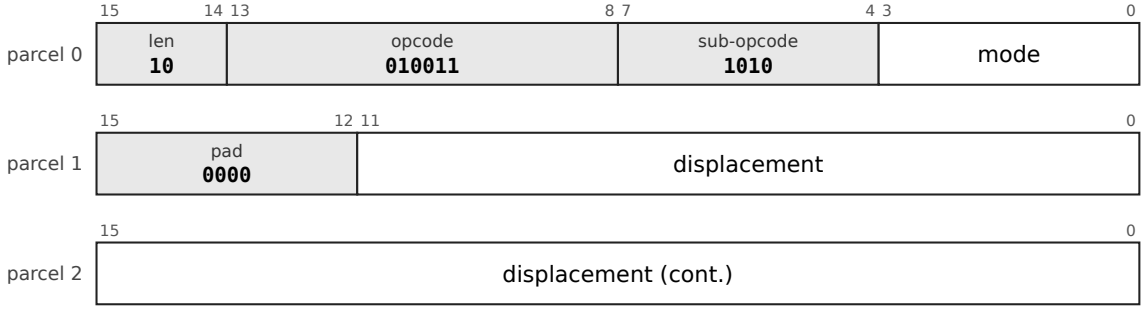
PUSHM, opcode 010011, three parcels (modes index) (6 bytes)



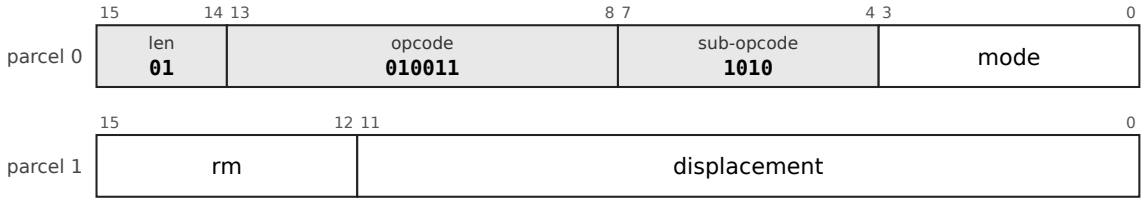
PUSHM, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



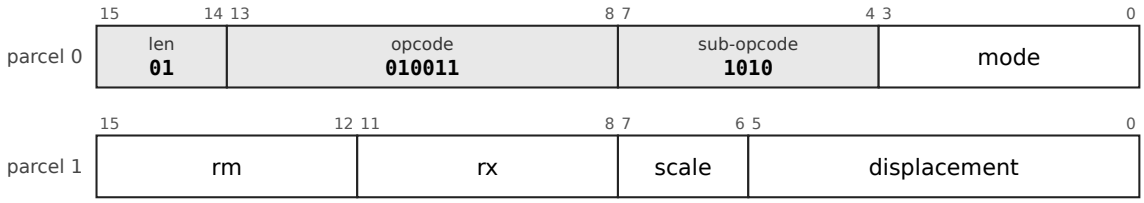
PUSHM, opcode 010011, three parcels (modes dp) (6 bytes)



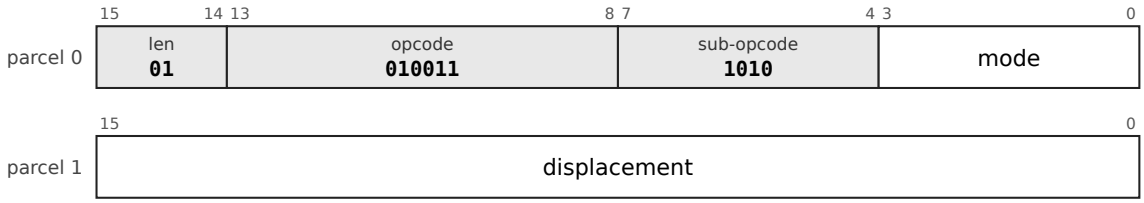
PUSHM, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



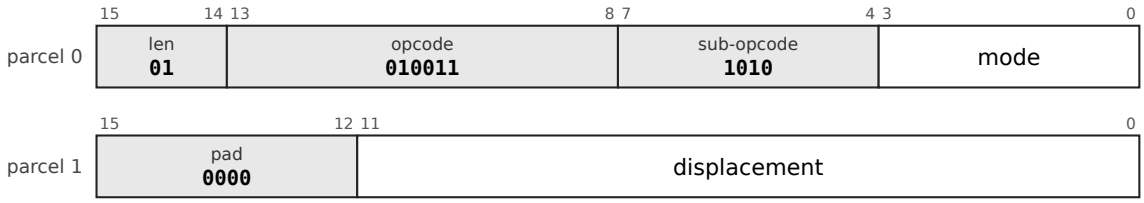
PUSHM, opcode 010011, two parcels (modes index) (4 bytes)



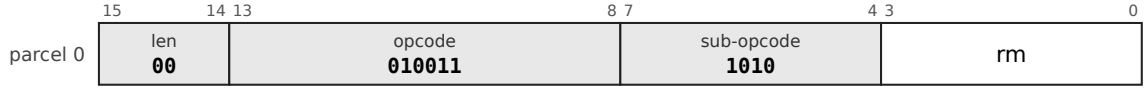
PUSHM, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



PUSHM, opcode 010011, two parcels (modes dp) (4 bytes)



PUSHM, opcode 010011, one parcel (2 bytes)

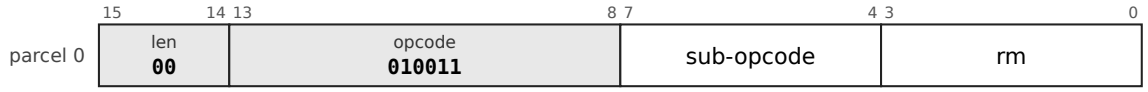


Mnemonics: PUSHM

Syntax

```
PUSHM #mask        ; a 16-bit immediate, bit n is Rn
```

Encoding



The chart is the layout r20-sop-0, Single operand (SOP), one parcel (len = 00): The reg field is the sub-opcode; the operand is rm with its mode, at any length.

Mnemonic	Opcode	reg
PUSHM	010011 (0x13)	1010 (0xA)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4.

Operation

```
for n = 0 to 15, if mask bit n:
    SP ← SP - 8 ; M[SP] ← Rn
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. A mask that includes R14 (SP) is reserved.

Four bytes (mode 0100, immediate). The registers are pushed in ascending order, so POPM with the same mask restores them. Page 2 has an 8-bit mask for R8 to R15.

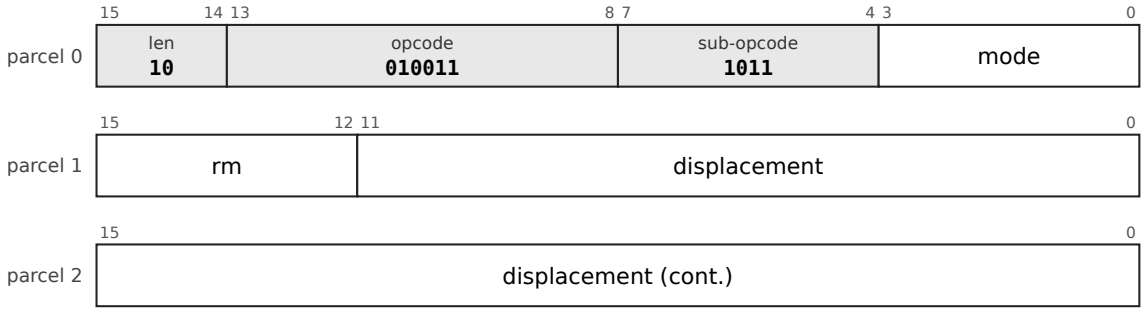
Why it is here. The VAX procedure entry mask was the useful half of CALLS; PUSHM and POPM keep that half without the frame (hardware call frames ran slower than the sequence on later VAXes). They are in SOP because a 16-bit mask does not fit beside MBLK's sub-opcode; that uses SOP's last two free slots. The second page has 8-bit forms for R8 to R15, because all 2,392 push sites and 2,392 pop sites in the measured corpus use a mask within R8 to R15.

Cf. 6.4 for the restart rule.

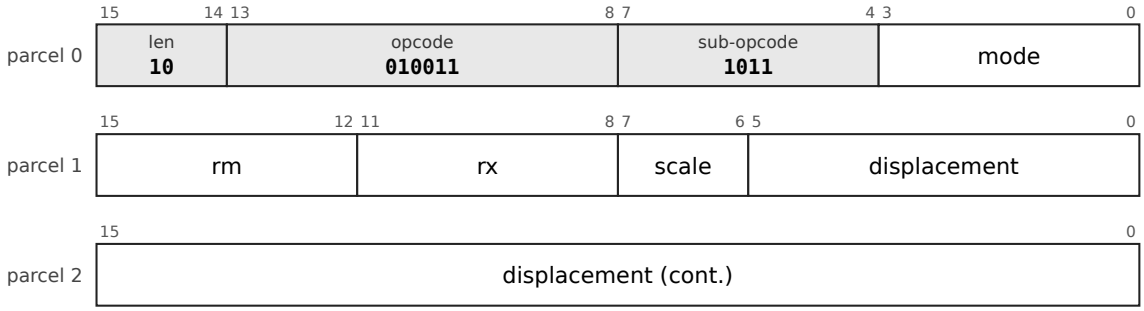
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

POPM: Pop registers by mask

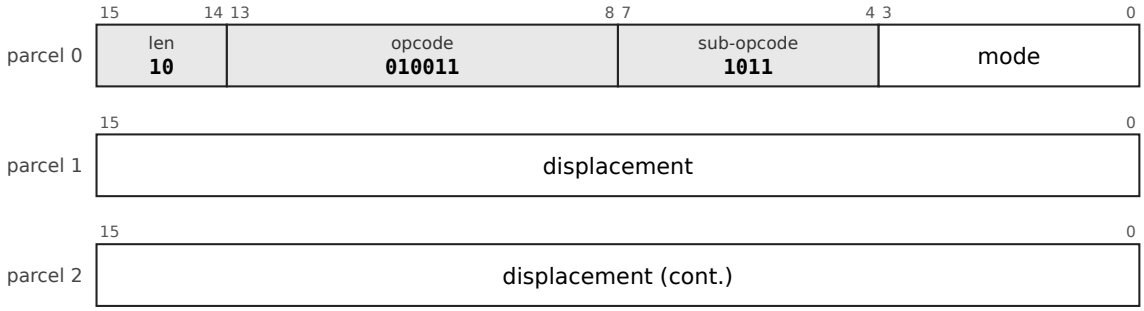
POPM, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



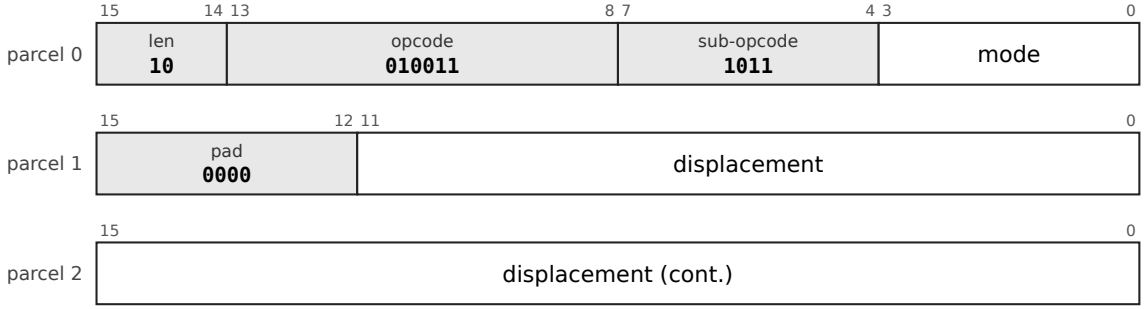
POPM, opcode 010011, three parcels (modes index) (6 bytes)



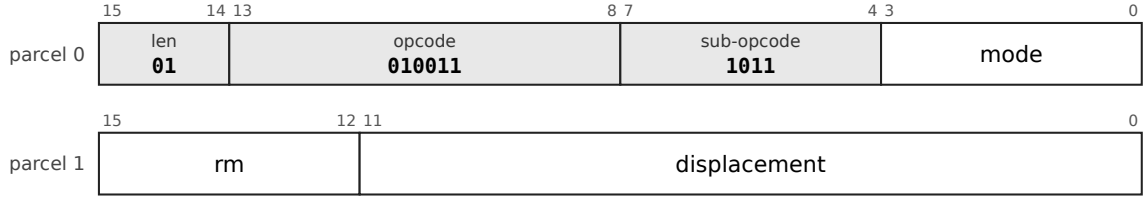
POPM, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



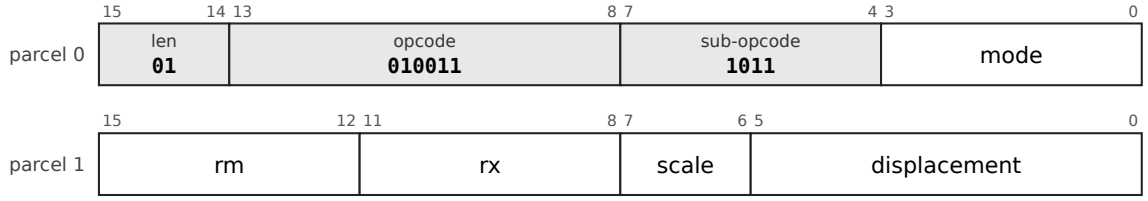
POPM, opcode 010011, three parcels (modes dp) (6 bytes)



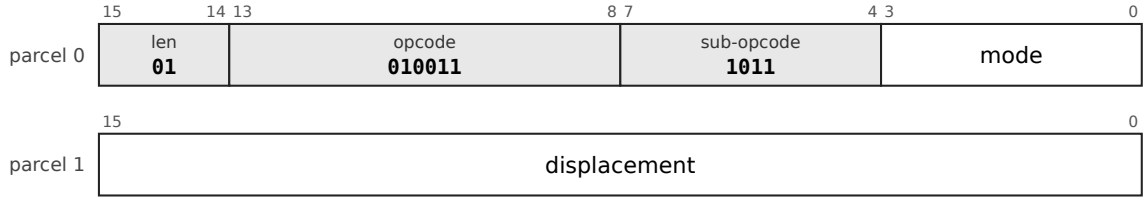
POPM, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



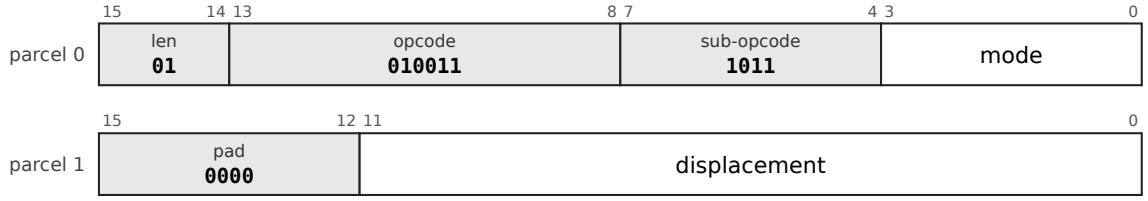
POPM, opcode 010011, two parcels (modes index) (4 bytes)



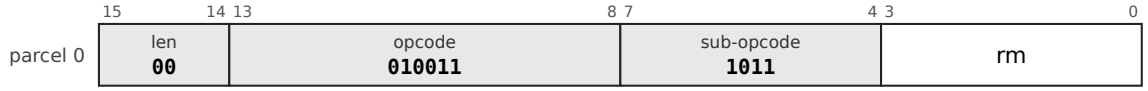
POPM, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



POPM, opcode 010011, two parcels (modes dp) (4 bytes)



POPM, opcode 010011, one parcel (2 bytes)



Mnemonics: `POPM`

Syntax

```
POPM #mask
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
POPM	010011 (0x13)	1011 (0xB)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
for n = 15 down to 0, if mask bit n:
    Rn ← M[SP] ; SP ← SP + 8
```

Flags None written.

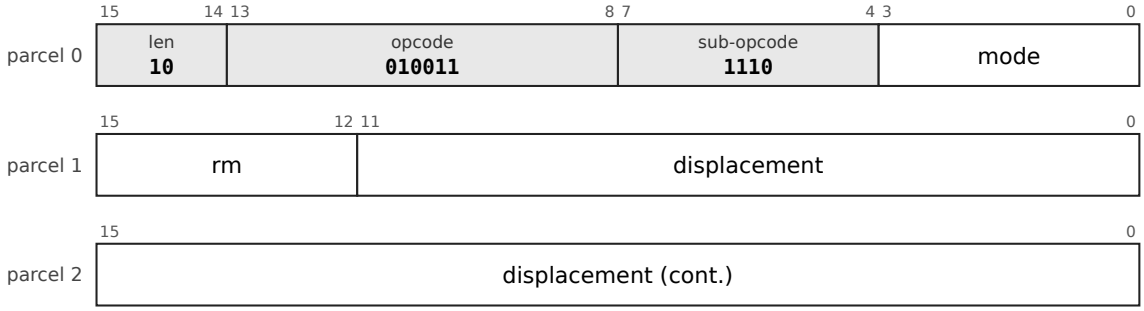
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The mirror of PUSHM; four bytes, with an 8-bit-mask form on page 2.

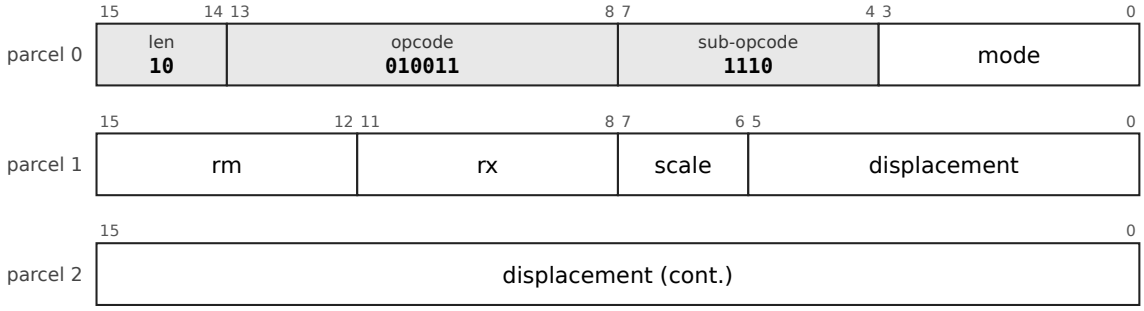
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

PUSH: Push

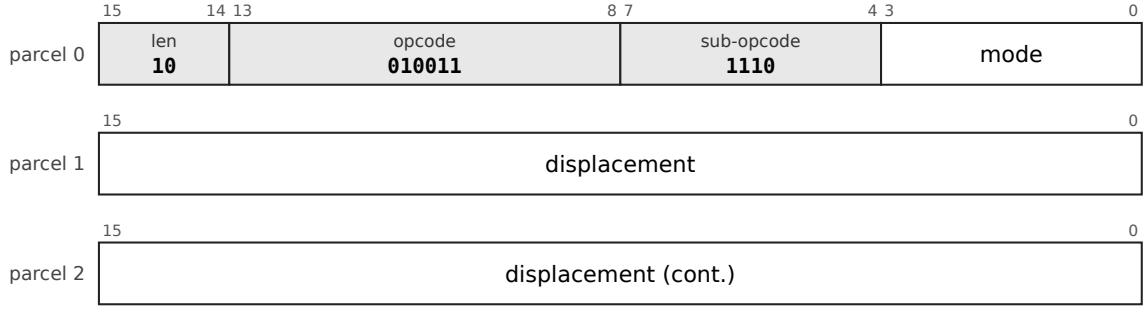
PUSH, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



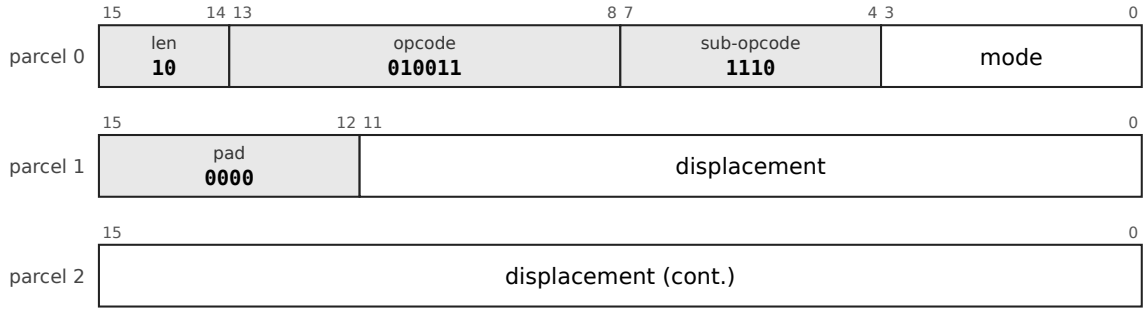
PUSH, opcode 010011, three parcels (modes index) (6 bytes)



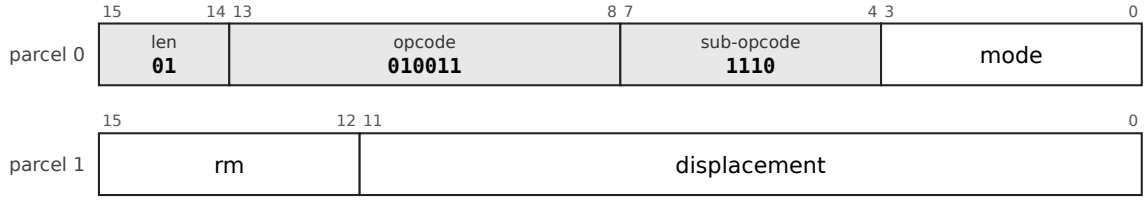
PUSH, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



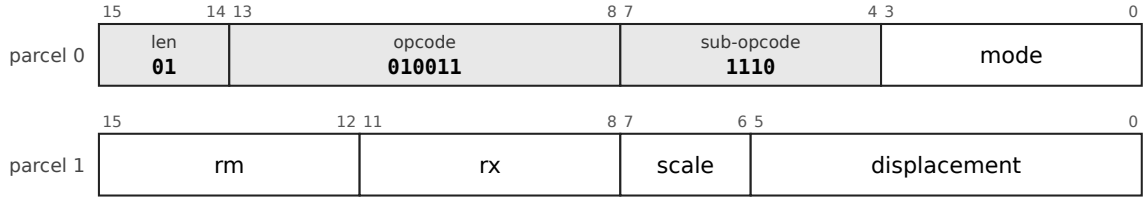
PUSH, opcode 010011, three parcels (modes dp) (6 bytes)



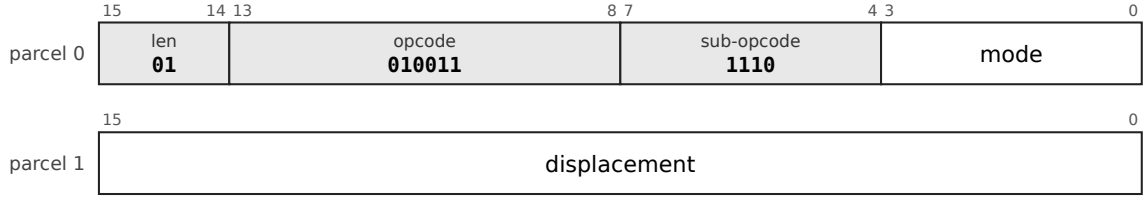
PUSH, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



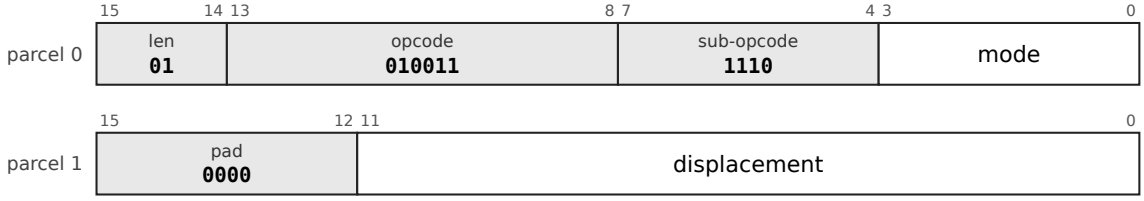
PUSH, opcode 010011, two parcels (modes index) (4 bytes)



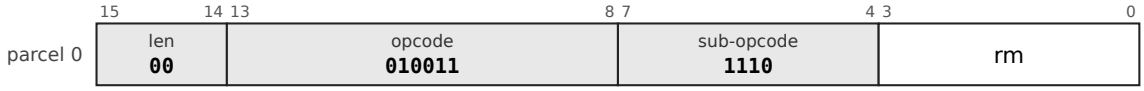
PUSH, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



PUSH, opcode 010011, two parcels (modes dp) (4 bytes)



PUSH, opcode 010011, one parcel (2 bytes)

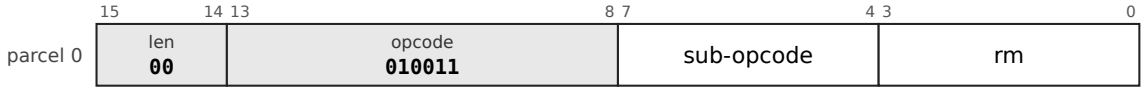


Mnemonics: `PUSH`

Syntax

```
PUSH rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`PUSH`	`010011 (0x13)`	`1110 (0xE)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
v ← rm ; SP ← SP - 8 ; M[SP] ← v
```

Flags None written.

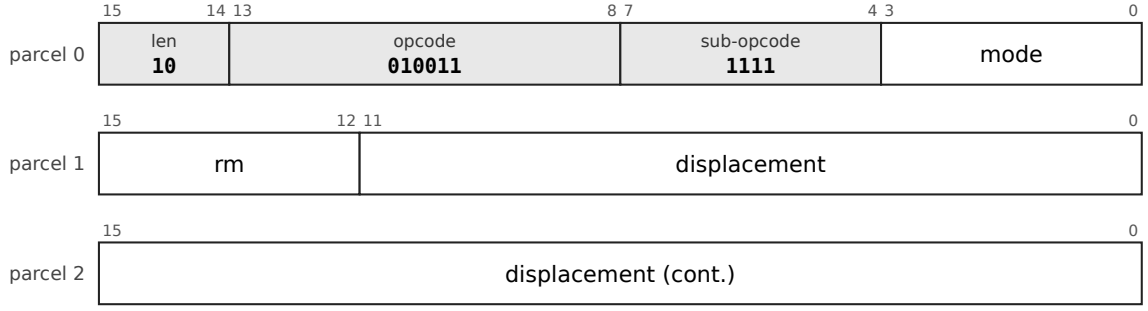
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

A one-parcel instruction for a register operand.

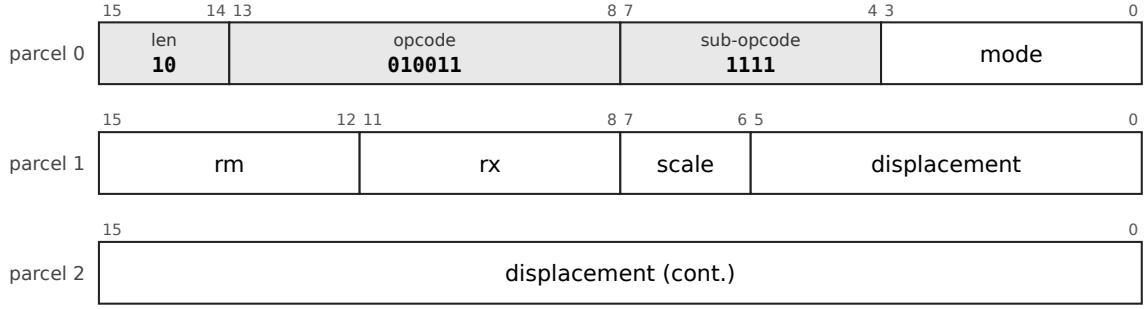
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

POP: Pop

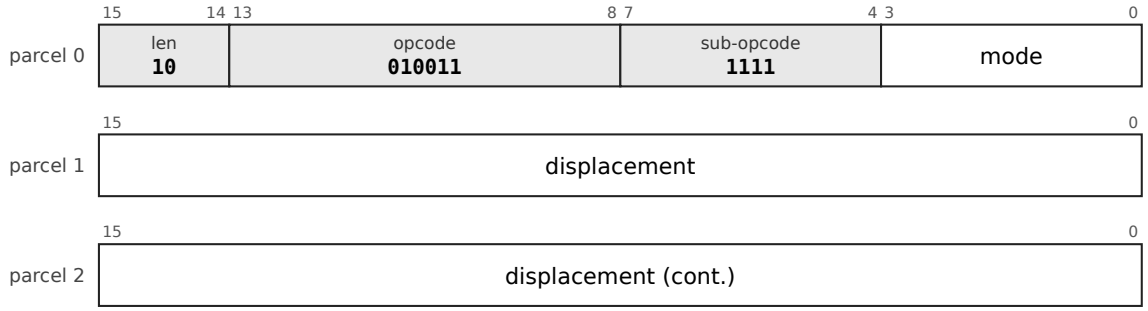
POP, opcode 010011, three parcels (modes base, postinc, predec) (6 bytes)



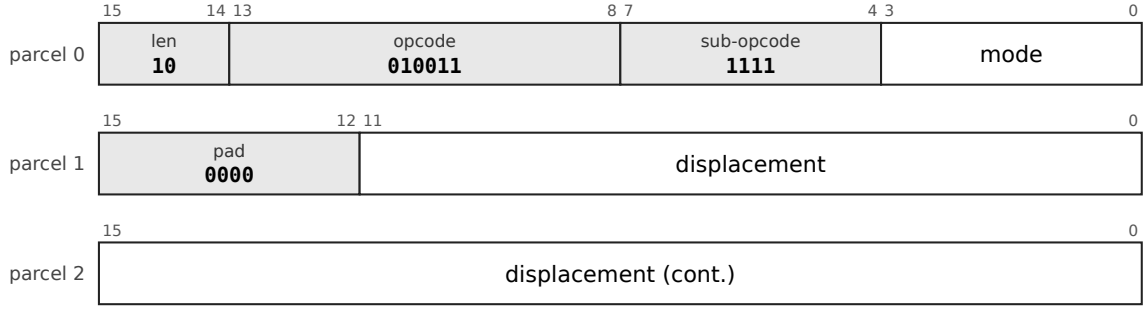
POP, opcode 010011, three parcels (modes index) (6 bytes)



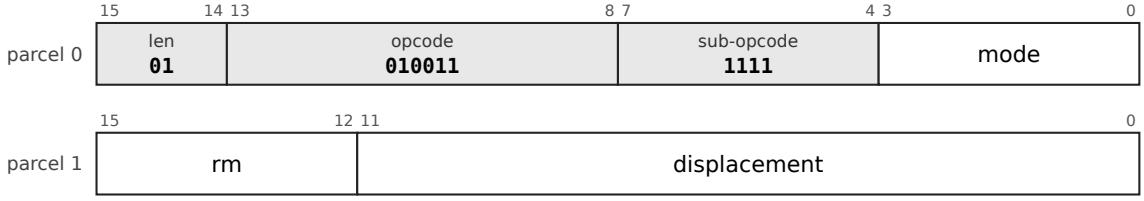
POP, opcode 010011, three parcels (modes pcrel, imm, abs) (6 bytes)



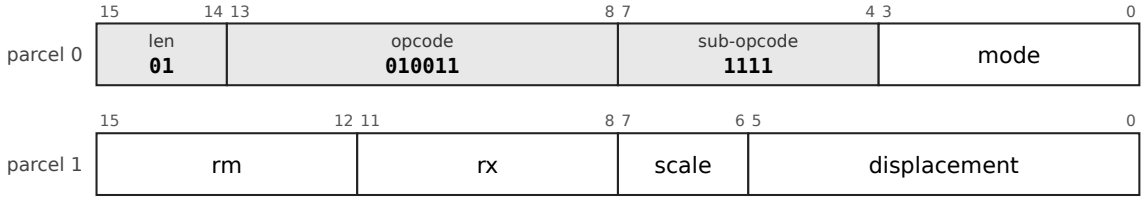
POP, opcode 010011, three parcels (modes dp) (6 bytes)



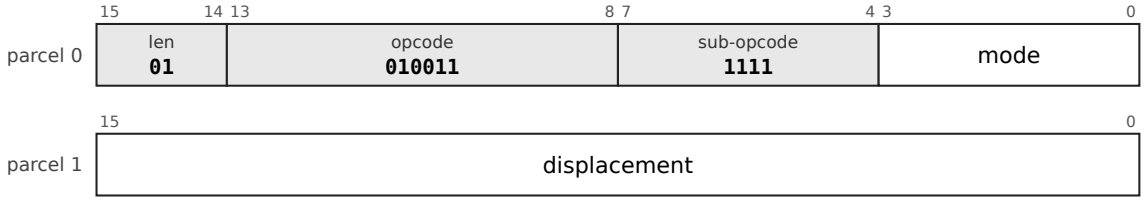
POP, opcode 010011, two parcels (modes base, postinc, predec) (4 bytes)



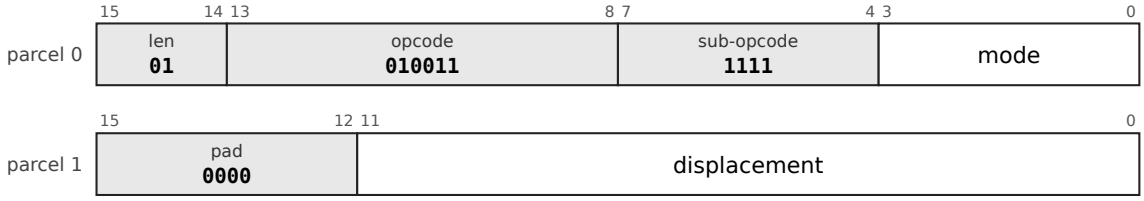
POP, opcode 010011, two parcels (modes index) (4 bytes)



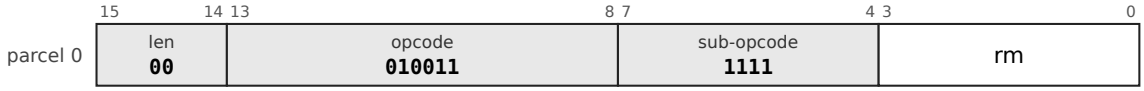
POP, opcode 010011, two parcels (modes pcrel, imm, abs) (4 bytes)



POP, opcode 010011, two parcels (modes dp) (4 bytes)



POP, opcode 010011, one parcel (2 bytes)

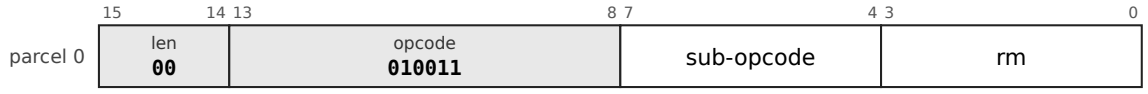


Mnemonics: `POP`

Syntax

```
POP rm
```

Encoding



The chart is the layout `r20-sop-0`, Single operand (SOP), one parcel (`len = 00`): The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
POP	010011 (0x13)	1111 (0xF)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sop-0`, `r20-sop-1`, `r20-sop-1-2`, `r20-sop-1-3`, `r20-sop-1-4`, `r20-sop-2`, `r20-sop-2-2`, `r20-sop-2-3`, `r20-sop-2-4`.

Operation

```
v ← M[SP] ; SP ← SP + 8 ; rm ← v
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

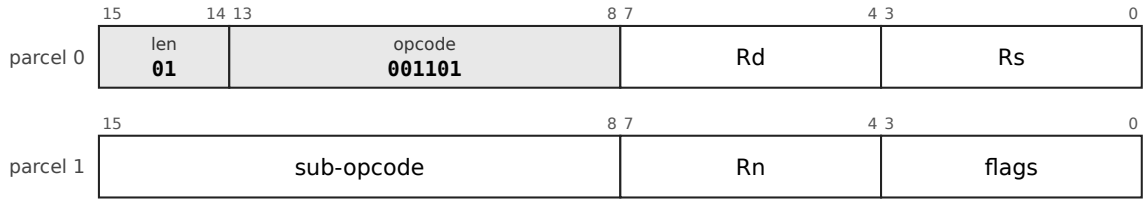
MCPY: Copy a block of bytes

Mnemonics: `MCPY`, `MMOV`

Syntax

```
MCPY Rd, Rs, Rn        ; destination pointer, source pointer, count
MMOV Rd, Rs, Rn        ; correct for overlap
```

Encoding



The chart is the layout `r20-mblk-1`, Block memory (MBLK), two parcels (`len = 01`): `reg` is the destination pointer, `low` the source pointer (or the value register), `Rn` the count register.

Mnemonic	Opcode	Sub-opcode
MCPY	001101 (0x0D)	00000000 (0x00)
MMOV	001101 (0x0D)	00000001 (0x01)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mblk-1`, `r20-mblk-2`.

Operation

```
MCPY:                          ; ascending; overlap is undefined
  while Rn ≠ 0:  M8[Rd] ← M8[Rs] ; Rd ← Rd + 1 ; Rs ← Rs + 1 ; Rn ← Rn - 1
MMOV: as MCPY, but the direction is chosen from the pointers so that an overlapping copy is correct
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. The three registers are updated as work completes, so a fault leaves a consistent state and the instruction restarts from its registers.

The one granted exception to the one-memory-operand rule. All three registers are architectural state: on completion `Rn` is 0 and `Rd` and `Rs` point one past the end. An implementation may move more than a byte per step, so long as the register state at every point where an exception can be taken is consistent with a byte-at-a-time copy. The accesses made inside one block operation are unordered with respect to each other.

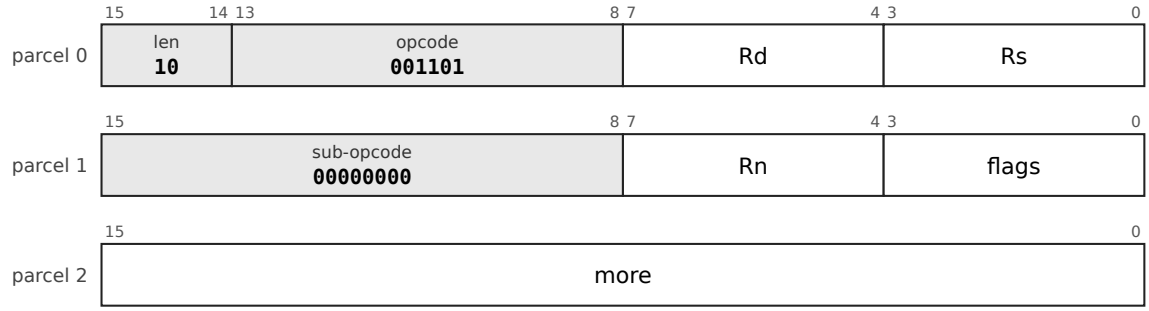
Why it is here. The block operations are the one granted exception to the one-memory-operand rule (an exception to a principle is allowed where there is a clear benefit), made safe by keeping the progress in architectural registers: the VAX `MOVC5` kept it in hidden microstate, which is why it is rejected, and ARM reached the same design in 2022 with `FEAT_MOPS`. The names `MCPY`, `MMOV` and `MSET` follow the memory operations of IBM System z; `MCMP` and `MSCN` complete the set. `MCPY` replaces the PDP-11 `MOV (R1)+,(R2)+` loop, and the `rep movsb` of x86, with one restartable instruction.

Cf. 6.4 (restart, progress faults); 7.2 (ordering of the accesses).

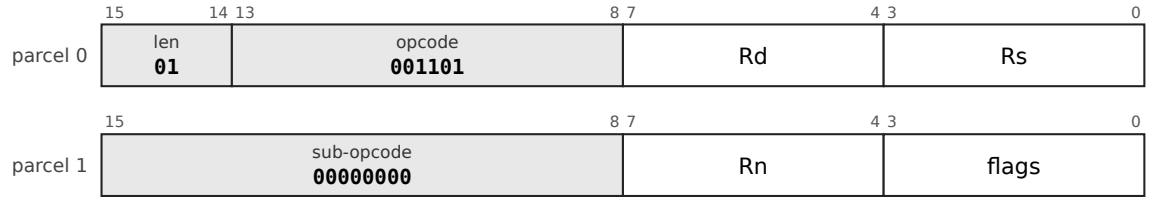
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MCPY

MCPY, opcode 001101, three parcels, sub-opcode 00000000 (6 bytes)



MCPY, opcode 001101, two parcels, sub-opcode 00000000 (4 bytes)



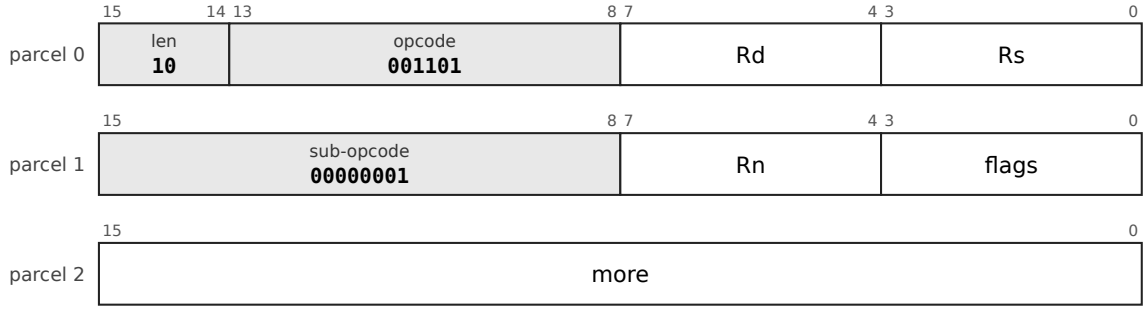

```
MCPY Rd, Rs, Rn
```

Mnemonic	Opcode	Sub-opcode
MCPY	001101 (0x0D)	00000000 (0x00)

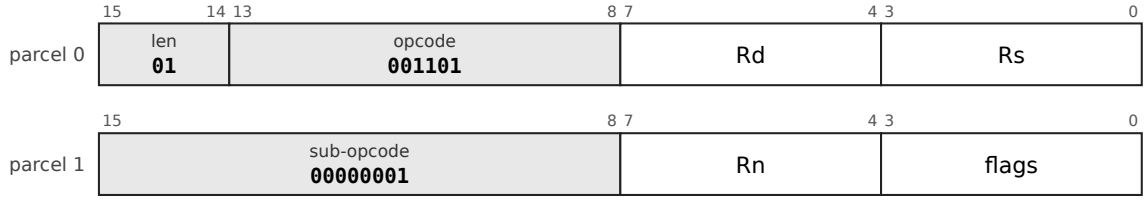
Operation, flags and exceptions as for the entry MCPY above; 2 layout forms, each a chart of its own (above).

MMOV

MMOV, opcode 001101, three parcels, sub-opcode 00000001 (6 bytes)



MMOV, opcode 001101, two parcels, sub-opcode 00000001 (4 bytes)



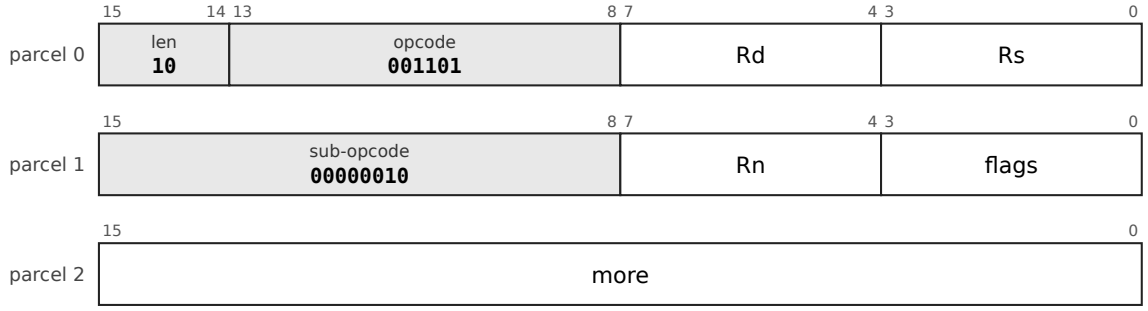

```
MMOV Rd, Rs, Rn
```

Mnemonic	Opcode	Sub-opcode
MMOV	001101 (0x0D)	00000001 (0x01)

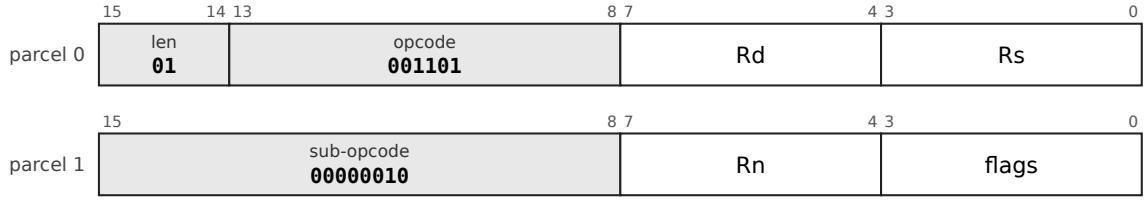
Operation, flags and exceptions as for the entry MCPY above; 2 layout forms, each a chart of its own (above).

MSET: Fill a block of bytes

MSET, opcode 001101, three parcels, sub-opcode 00000010 (6 bytes)



MSET, opcode 001101, two parcels, sub-opcode 00000010 (4 bytes)

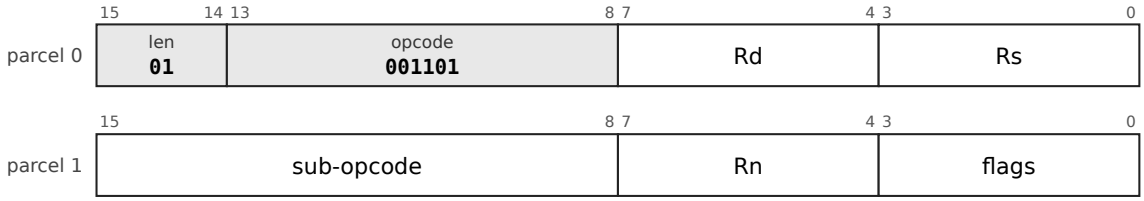


Mnemonics: `MSET`

Syntax

```
MSET Rd, Rv, Rn
```

Encoding



The chart is the layout `r20-mblk-1`, Block memory (MBLK), two parcels (`len = 01`): `reg` is the destination pointer, `low` the source pointer (or the value register), `Rn` the count register.

Mnemonic	Opcode	Sub-opcode
MSET	001101 (0x0D)	00000010 (0x02)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mblk-1`, `r20-mblk-2`.

Operation

```
while Rn ≠ 0:  M8[Rd] ← Rv[7:0] ; Rd ← Rd + 1 ; Rn ← Rn - 1
```

Flags None written.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

`MSET` with `Rv = 0` is the range-zeroing instruction: `bzero` and the clear of `calloc` are one instruction.

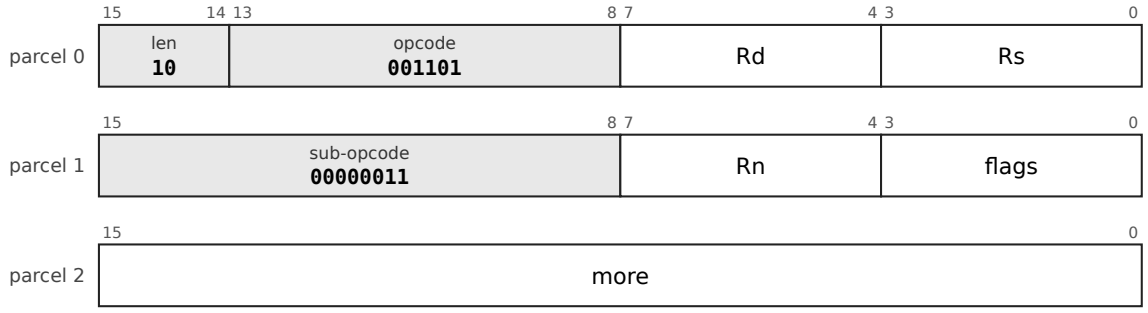
Why it is here. With a zero value it is the range-zeroing instruction, so `bzero` and the clearing of `calloc` need no separate opcode.

Cf. 6.4.

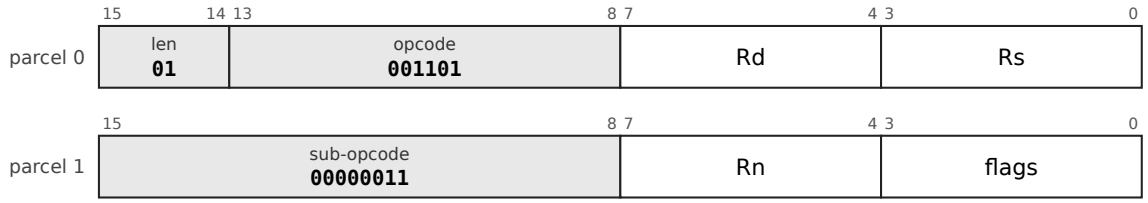
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MCMP: Compare two blocks of bytes

MCMP, opcode 001101, three parcels, sub-opcode 00000011 (6 bytes)



MCMP, opcode 001101, two parcels, sub-opcode 00000011 (4 bytes)

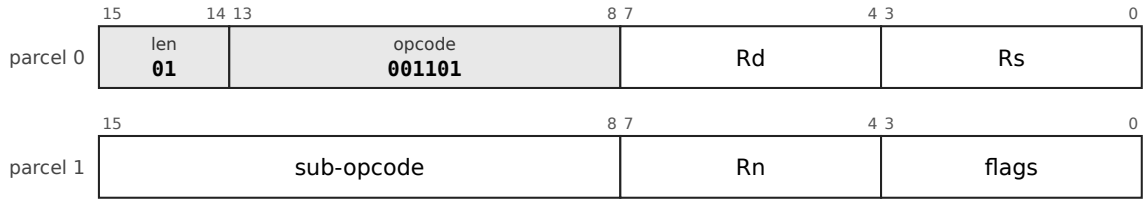


Mnemonics: `MCMP`

Syntax

```
MCMP Rd, Rs, Rn
```

Encoding



The chart is the layout `r20-mblk-1`, Block memory (MBLK), two parcels (`len = 01`): `reg` is the destination pointer, `low` the source pointer (or the value register), `Rn` the count register.

Mnemonic	Opcode	Sub-opcode
`MCMP`	`001101 (0x0D)`	`00000011 (0x03)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mblk-1`, `r20-mblk-2`.

Operation

```
while Rn ≠ 0:
    x ← M8[Rd] ; y ← M8[Rs]
    if x ≠ y then  flags ← flags of (x - y) ; stop      ; pointers left AT the differing byte
    Rd ← Rd + 1 ; Rs ← Rs + 1 ; Rn ← Rn - 1
if equal:  N = 0, Z = 1, V = 0, C = 0
```

Flags From the first differing byte as for `CMP` of the two bytes (`C` set if `x < y` unsigned, `N` and `V` from the signed difference); for equal blocks `N` = 0, `Z` = 1, `V` = 0, `C` = 0.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

When the instruction stops at a difference, `Rd` and `Rs` point at the differing bytes and `Rn` is the count remaining, so the position of the difference is in the registers.

Why it is here. Seven of the VAX's ten character-string instructions are present (`MOVC3`, `CMPC3`, `LOCC` and `SKPC` among them); the rejected ones are `EDITPC` and packed decimal. `MCMP` is the `CMPC3` equivalent.

Cf. 6.4.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

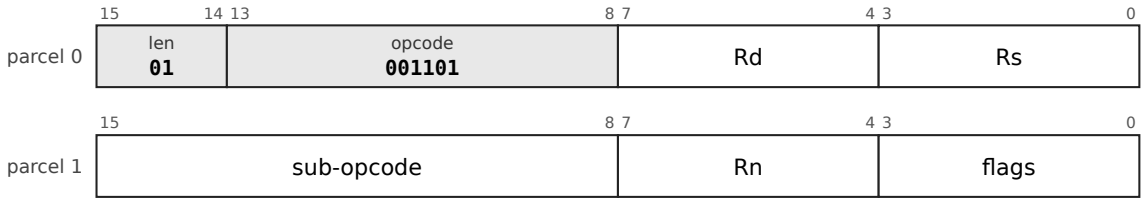
MSCN: Scan a block for a byte, or for a byte not equal to it

Mnemonics: `MSCN`, `MSCNN`

Syntax

```
MSCN Rd, Rv, Rn
MSCNN Rd, Rv, Rn
```

Encoding



The chart is the layout `r20-mblk-1`, Block memory (MBLK), two parcels (`len = 01`): `reg` is the destination pointer, `low` the source pointer (or the value register), `Rn` the count register.

Mnemonic	Opcode	Sub-opcode
`MSCN`	`001101 (0x0D)`	`00000100 (0x04)`
`MSCNN`	`001101 (0x0D)`	`00000101 (0x05)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mblk-1`, `r20-mblk-2`.

Operation

```
while Rn ≠ 0:
    hit ← (M8[Rd] = Rv[7:0])            ; MSCNN: hit ← (M8[Rd] ≠ Rv[7:0])
    if hit then  Z = 1 ; stop             ; Rd points AT the byte found
    Rd ← Rd + 1 ; Rn ← Rn - 1
not found: Z = 0
```

Flags All four flags are written: `Z` = 1 if the byte was found, else 0; `N`, `V` and `C` are cleared.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

`MSCN` is `memchr`; with an explicit count it gives `strnlen`, and composing it with `MCPY` gives a bounded `strcpy` in two instructions. There are no NUL-terminated string instructions.

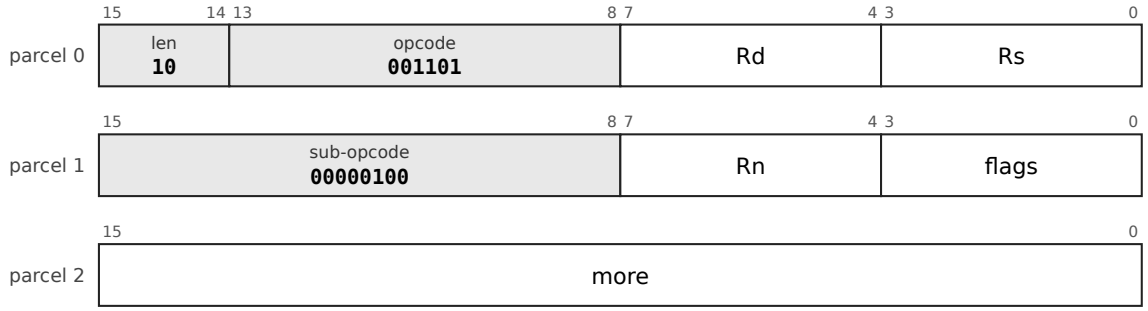
Why it is here. `MSCN` is `LOCC` and `memchr`; `MSCNN` fills the `SKPC` gap (the first byte that is *not* equal). Both cost no opcode and almost no hardware. With an explicit count `MSCN` gives `strnlen`, and with `MCPY` a bounded `strcpy` in two instructions. NUL-terminated string instructions (the System/z `MVST`, `SRST`, `CLST`) are rejected, because they encode an unbounded read, the largest single source of memory-safety failures in C (9.17).

Cf. 6.4.

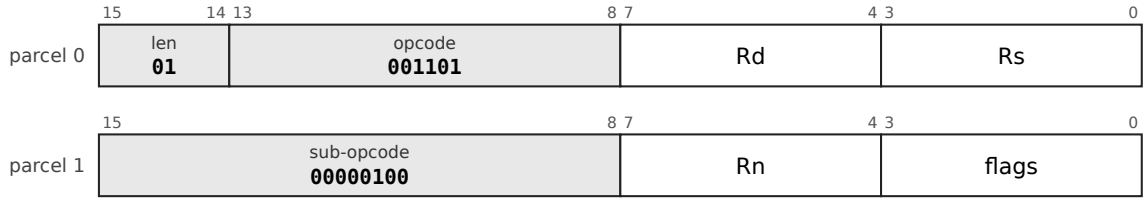
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MSCN

MSCN, opcode 001101, three parcels, sub-opcode 00000100 (6 bytes)



MSCN, opcode 001101, two parcels, sub-opcode 00000100 (4 bytes)



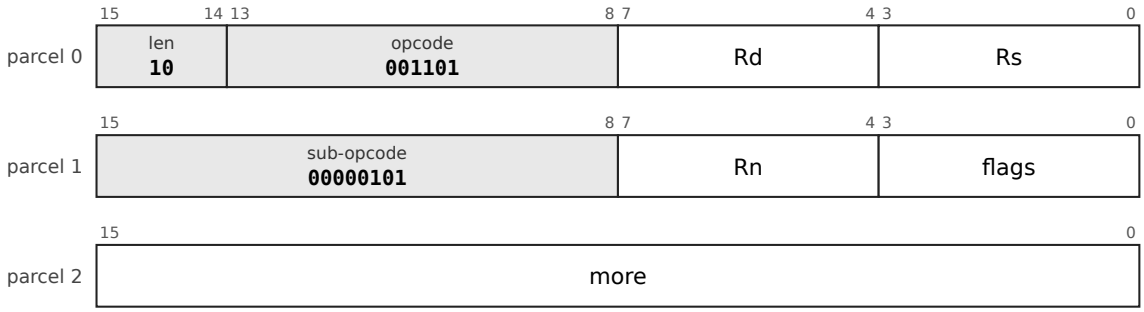

```
MSCN Rd, Rv, Rn
```

Mnemonic	Opcode	Sub-opcode
MSCN	001101 (0x0D)	00000100 (0x04)

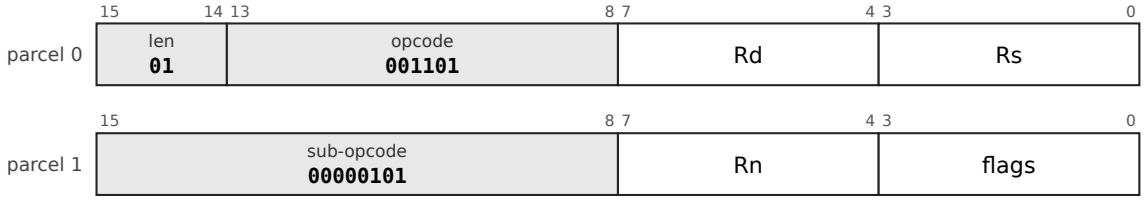
Operation, flags and exceptions as for the entry MSCN above; 2 layout forms, each a chart of its own (above).

MSCNN

MSCNN, opcode 001101, three parcels, sub-opcode 00000101 (6 bytes)



MSCNN, opcode 001101, two parcels, sub-opcode 00000101 (4 bytes)



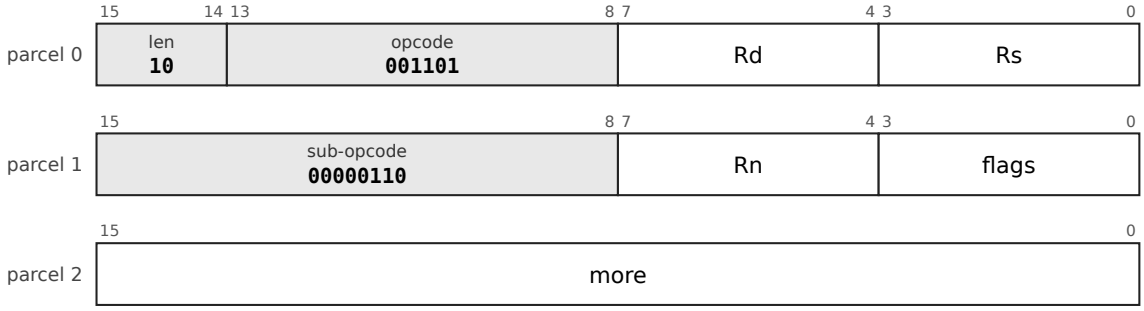

```
MSCNN Rd, Rv, Rn
```

Mnemonic	Opcode	Sub-opcode
MSCNN	001101 (0x0D)	00000101 (0x05)

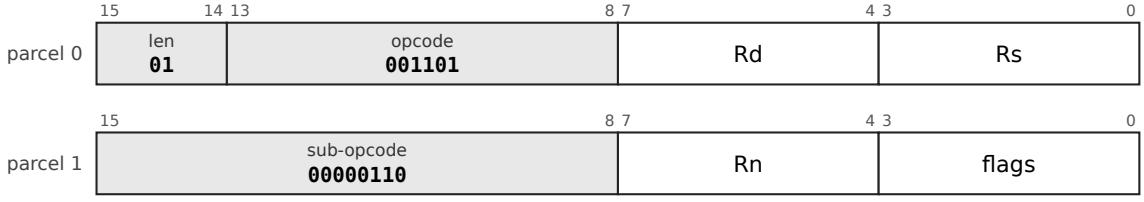
Operation, flags and exceptions as for the entry MSCN above; 2 layout forms, each a chart of its own (above).

MVAL: Validate a block as UTF-8

MVAL, opcode 001101, three parcels, sub-opcode 00000110 (6 bytes)



MVAL, opcode 001101, two parcels, sub-opcode 00000110 (4 bytes)

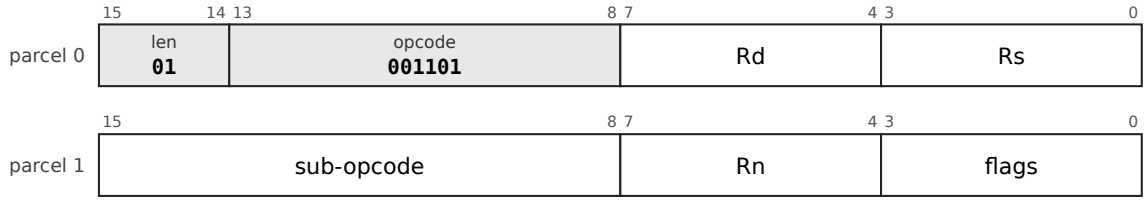


Mnemonics: MVAL

Syntax

```
MVAL Rd, Rn
```

Encoding



The chart is the layout `r20-mblk-1`, Block memory (MBLK), two parcels (`len = 01`): `reg` is the destination pointer, `low` the source pointer (or the value register), `Rn` the count register.

Mnemonic	Opcode	Sub-opcode
`MVAL`	`001101 (0x0D)`	`00000110 (0x06)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mblk-1`, `r20-mblk-2`.

Operation

```
scan Rn bytes at Rd; stop at the first byte that cannot be part of a valid UTF-8 sequence (the rules of
UTF8DEC)
```

Flags On completion `Rn` is 0 and the buffer is valid; otherwise `Rd` points at the offending sequence and `Rn` counts the bytes remaining (details in the instruction-set text, section 4.7.1).

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The `UTF8DEC` comparators wired to the block sequencer.

Why it is here. The UTF-8 validity comparators already built for `UTF8DEC`, wired to the `MBLK` streaming sequencer already built for `MCPY`, so it costs no opcode and almost no hardware. The alternative, a SWAR loop (SIMD within a register), runs at about two to three cycles a byte and is respectable, which is why the justification is cost and not throughput.

Cf. 6.4.

Status specified; **not yet implemented** in the reference executor.

9.12 Floating point

9.12.1 Floating point uses the general registers: a `binary64` value fills one register, and the instruction, not the register, decides how its bits are read. *(D)*

9.12.2 `FADD`, `FSUB`, `FMUL` and `FCMP` are two-byte instructions; `FDIV`, `FSQRT`, the fused multiply-add family, `FABS`, `FMIN`, `FMAX`, `FCLASS`, the roundings and the `binary32` forms are in the floating-point escape; `binary16` has seventeen operations and conversions. *(D)*

9.12.3 The five IEEE flags (invalid, divide by zero, overflow, underflow, inexact) accumulate in `FPSR`; no floating-point operation traps; `FSFLAGS` reads and clears them. *(D)*

9.12.4 Every NaN that an operation creates is the canonical quiet NaN. *(D)*

9.12.5 There are no rounding-mode or precision mode bits: the precision is in the opcode, and so is the rounding mode wherever it is not nearest-even. *(D)*

9.12.6 There are no transcendental instructions. *(D)*

Commentary. **Precedent.** The PDP-11's FP11 had `AC0` to `AC5` and mode bits for single and double precision; the VAX had F, D, G and H formats. **Trade-offs.** An integer operation applied to a register holding a `binary64` destroys it, with no trap and no tag; that is intended and the ABI states it. A separate register file would cost a second set of read ports and a floating-point context; on a small implementation the register file is among the largest blocks. **Improvements.** The fused

multiply-add is the largest single addition to the datapath (about 9,000 transistors) and the least negotiable; the IEEE exception flags are recorded without traps, which is what C99 `<fenv.h>` needs. **Benefits.** No transfer instructions between files and a single calling convention. **Modern comparables.** RISC-V Zfinx and Zdinx use the integer register file for the same reason; AArch64 and x86 have separate vector and floating-point files. **Use.** FCLASS begins every library maths function; FMA is the building block of accurate polynomial evaluation. **Impact.** *Kernel:* There is no separate floating-point state to save at a context switch beyond the registers already saved, and the sticky flags go with the status word. *Compiler:* One calling convention for integers and floats, fused multiply-add contraction, and no rounding-mode state for the compiler to track. *Hardware:* No second register file (one of the largest blocks of a small floating-point unit); the floating-point unit with fused multiply-add is about 34,000 transistors.

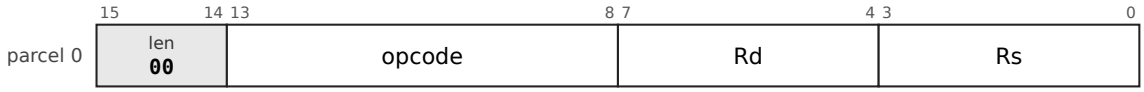
FADD: Floating add, subtract and multiply, binary64

Mnemonics: FADD, FSUB, FMUL (aliases: FADDD, FSUBD, FMULD)

Syntax

```
FADD rm, Rd
FSUB rm, Rd
FMUL rm, Rd
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
Rd ← Rd + rm   (FADD)
Rd ← Rd - rm   (FSUB)
Rd ← Rd × rm   (FMUL)       ; IEEE 754 binary64, round to nearest even
```

Flags None written. The five IEEE exception flags are ORed into FPSR; no instruction traps.

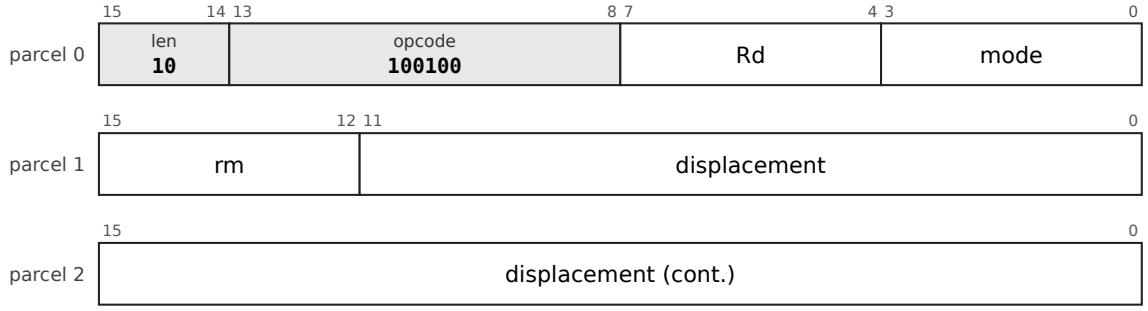
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. Floating-point exceptions never trap.

Floating-point values live in the general registers; there is no separate register file and no mode bit. A NaN result is the canonical quiet NaN `0x7FF8000000000000`. The alternative spellings FADDD, FSUBD, FMULD are aliases.

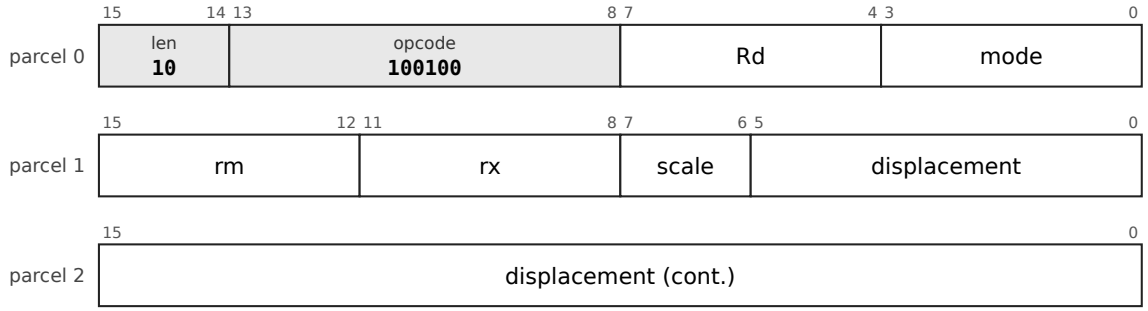
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

FADD

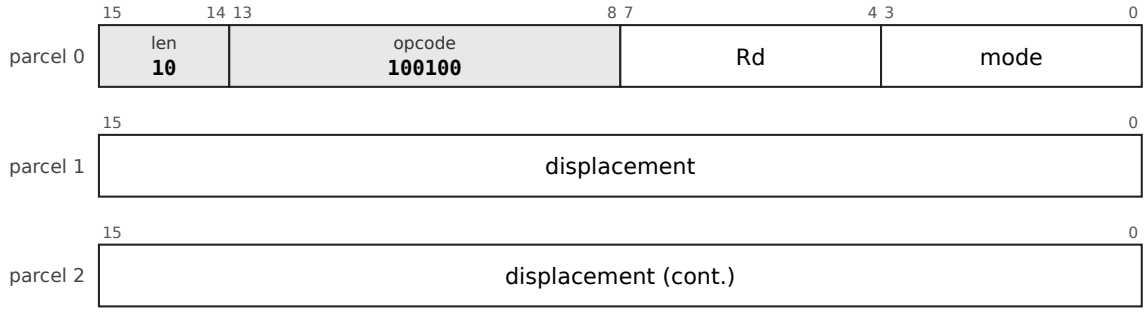
FADD, opcode 100100, three parcels (modes base, postinc, predec) (6 bytes)



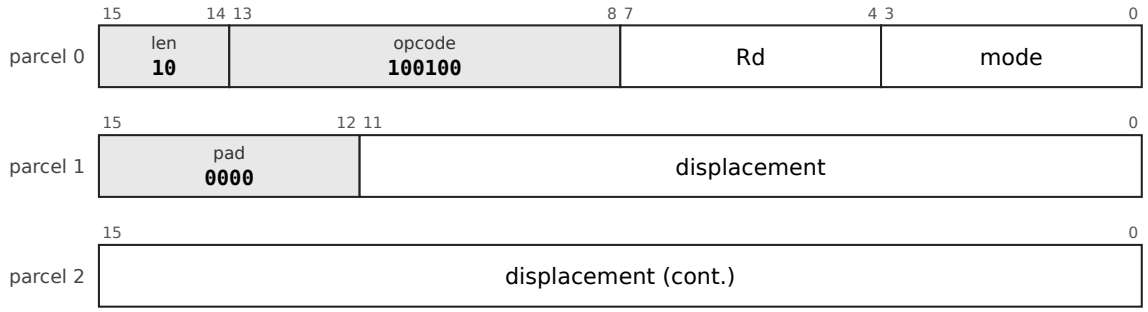
FADD, opcode 100100, three parcels (modes index) (6 bytes)



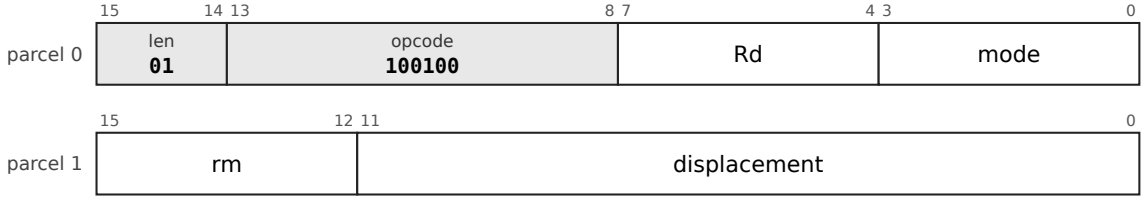
FADD, opcode 100100, three parcels (modes pcrel, imm, abs) (6 bytes)



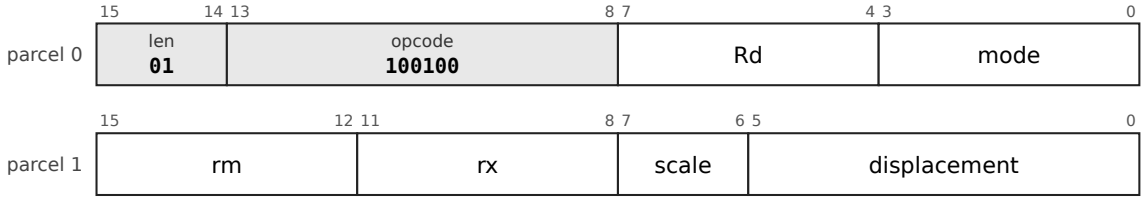
FADD, opcode 100100, three parcels (modes dp) (6 bytes)



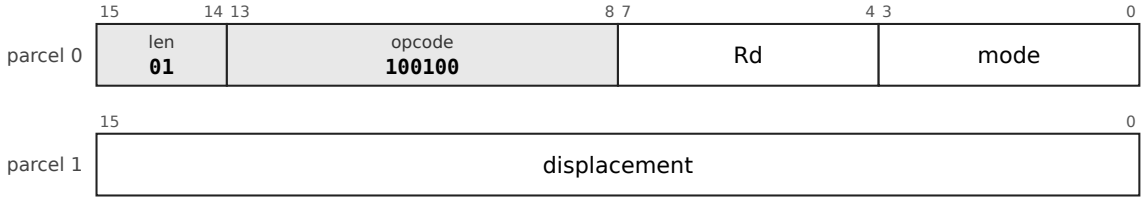
FADD, opcode 100100, two parcels (modes base, postinc, predec) (4 bytes)



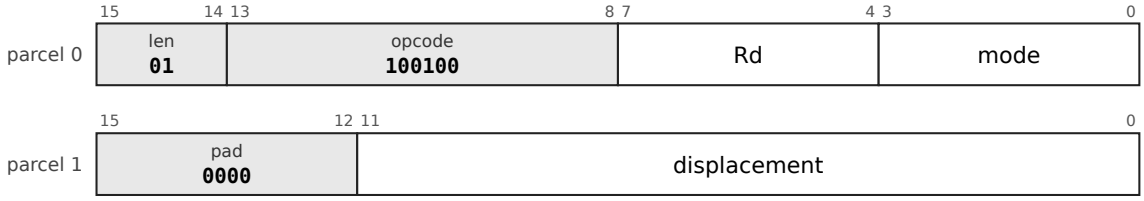
FADD, opcode 100100, two parcels (modes index) (4 bytes)



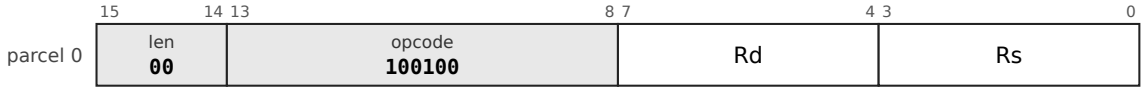
FADD, opcode 100100, two parcels (modes pcrel, imm, abs) (4 bytes)



FADD, opcode 100100, two parcels (modes dp) (4 bytes)



FADD, opcode 100100, one parcel (2 bytes)



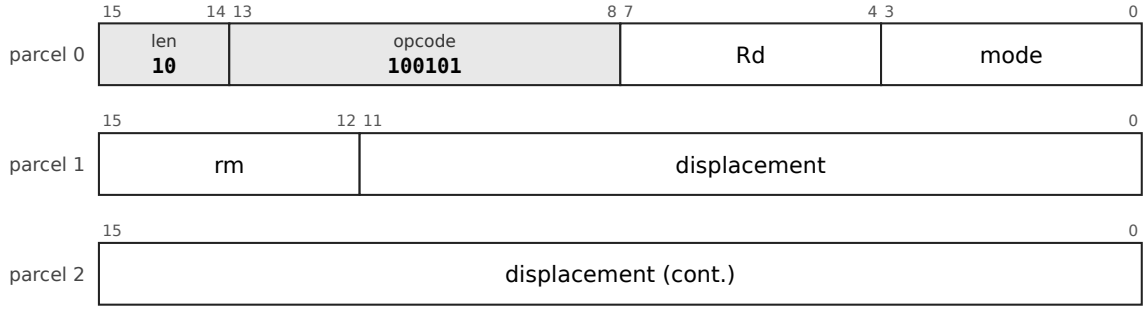

```
FADD rm, Rd
```

Mnemonic	Opcode
FADD	100100 (0x24)

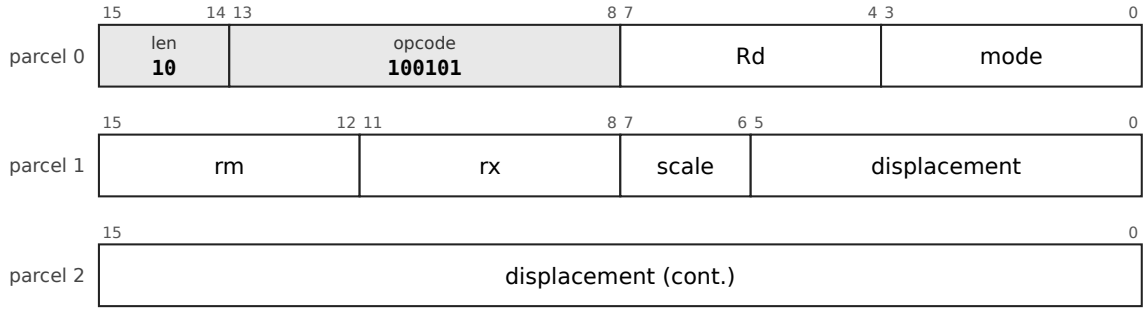
Operation, flags and exceptions as for the entry FADD above; 9 layout forms, each a chart of its own (above).

FSUB

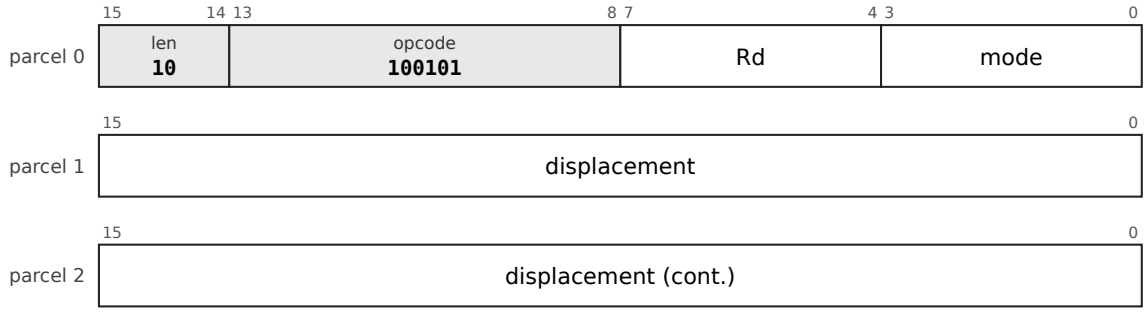
FSUB, opcode 100101, three parcels (modes base, postinc, predec) (6 bytes)



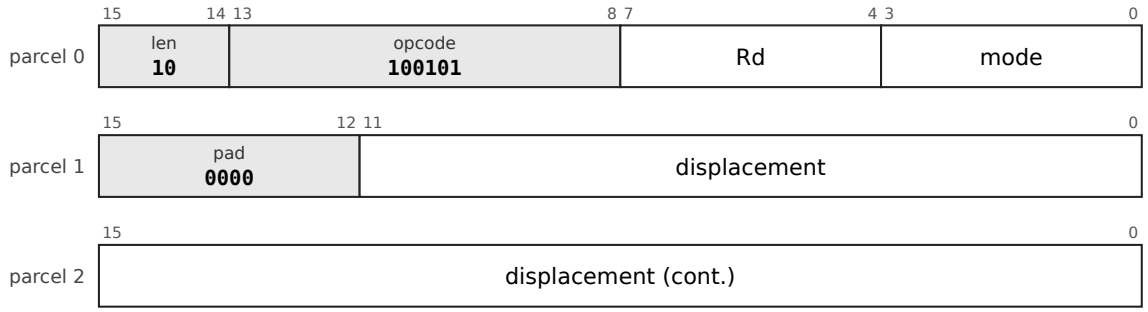
FSUB, opcode 100101, three parcels (modes index) (6 bytes)



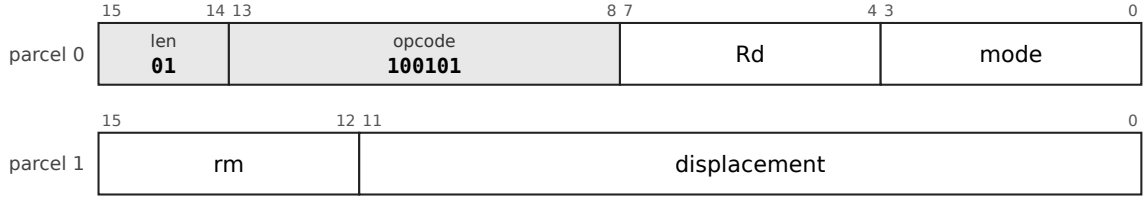
FSUB, opcode 100101, three parcels (modes pcrel, imm, abs) (6 bytes)



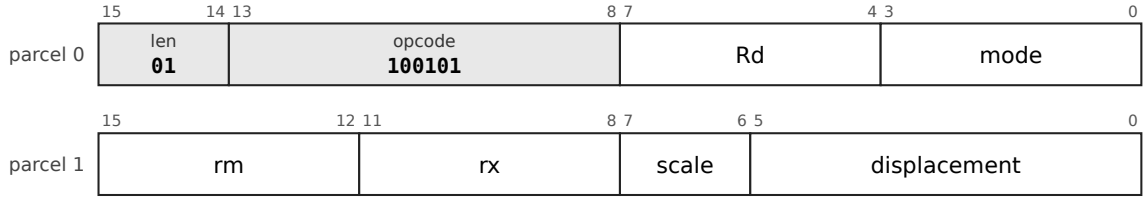
FSUB, opcode 100101, three parcels (modes dp) (6 bytes)



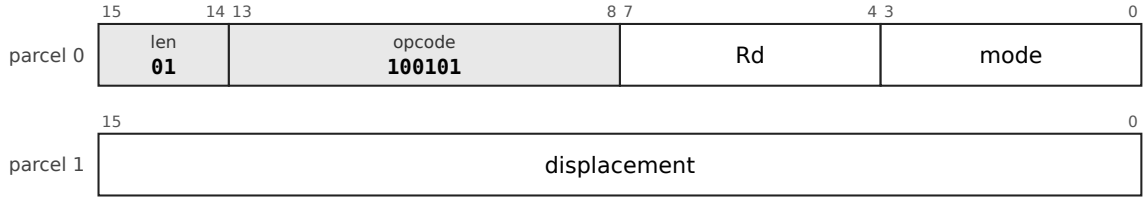
FSUB, opcode 100101, two parcels (modes base, postinc, predec) (4 bytes)



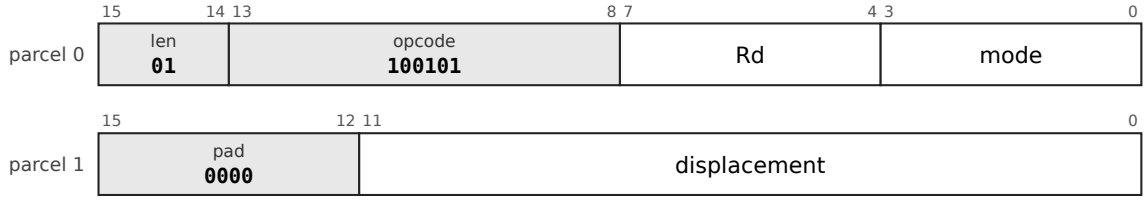
FSUB, opcode 100101, two parcels (modes index) (4 bytes)



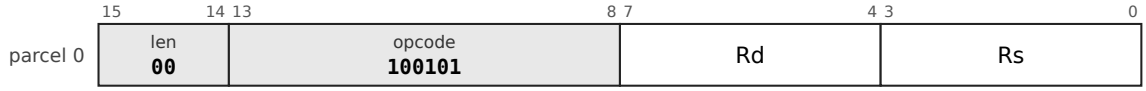
FSUB, opcode 100101, two parcels (modes pcrel, imm, abs) (4 bytes)



FSUB, opcode 100101, two parcels (modes dp) (4 bytes)



FSUB, opcode 100101, one parcel (2 bytes)



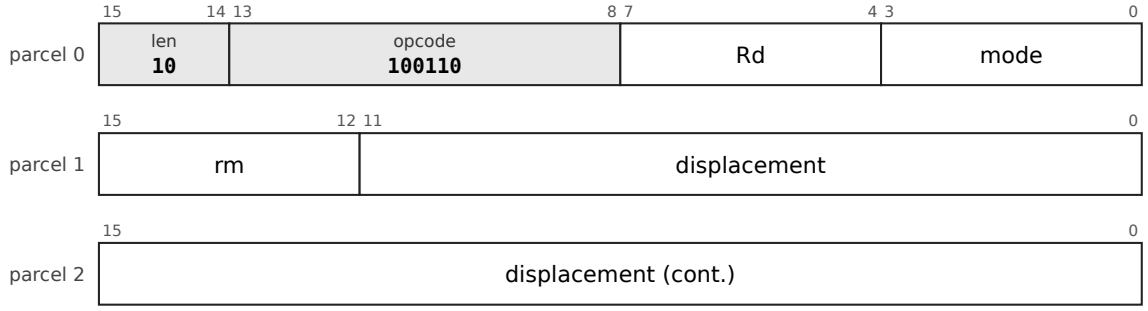

```
FSUB rm, Rd
```

Mnemonic	Opcode
FSUB	100101 (0x25)

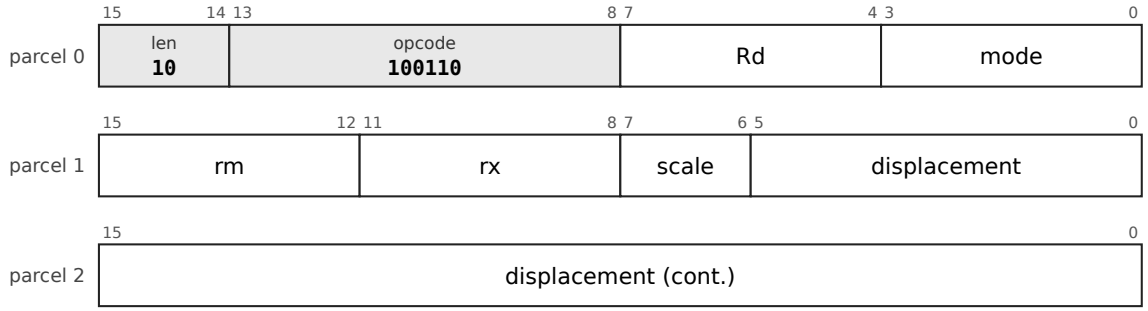
Operation, flags and exceptions as for the entry FADD above; 9 layout forms, each a chart of its own (above).

FMUL

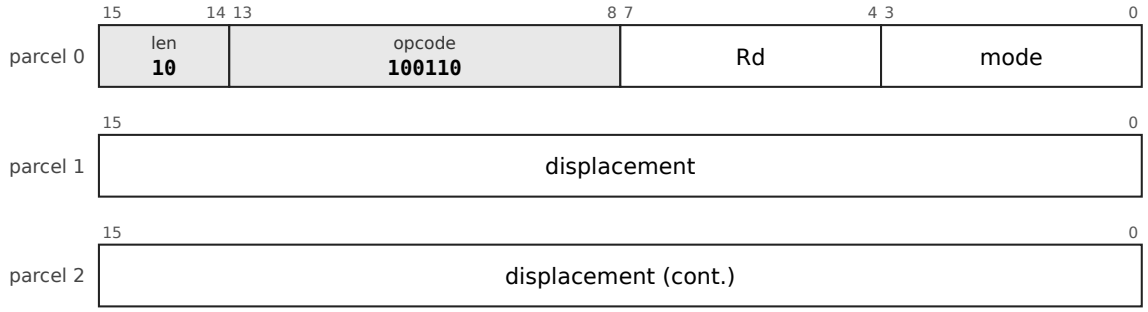
FMUL, opcode 100110, three parcels (modes base, postinc, predec) (6 bytes)



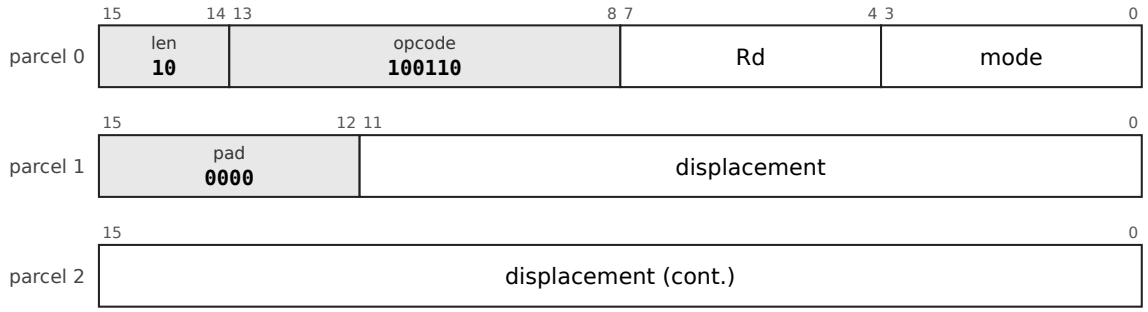
FMUL, opcode 100110, three parcels (modes index) (6 bytes)



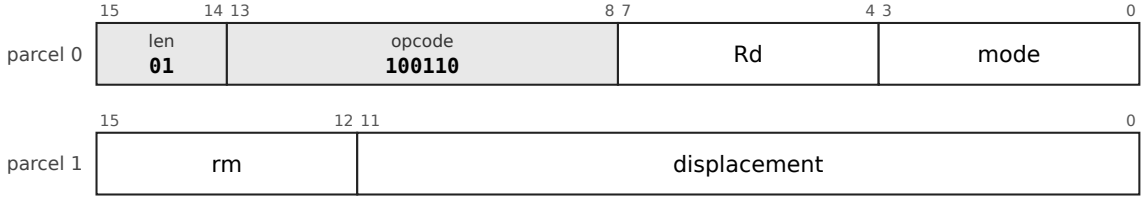
FMUL, opcode 100110, three parcels (modes pcrel, imm, abs) (6 bytes)



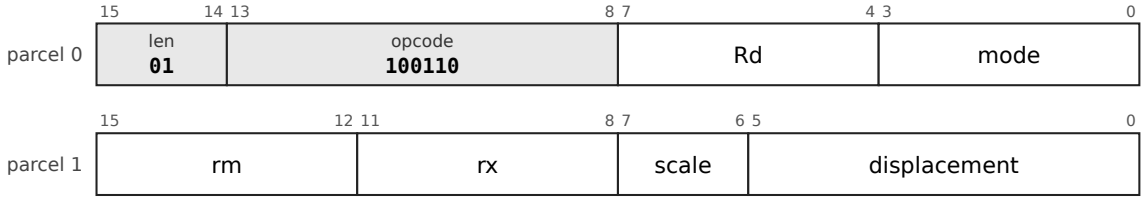
FMUL, opcode 100110, three parcels (modes dp) (6 bytes)



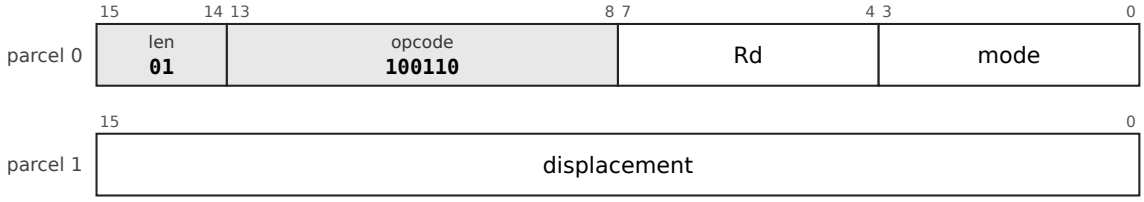
FMUL, opcode 100110, two parcels (modes base, postinc, predec) (4 bytes)



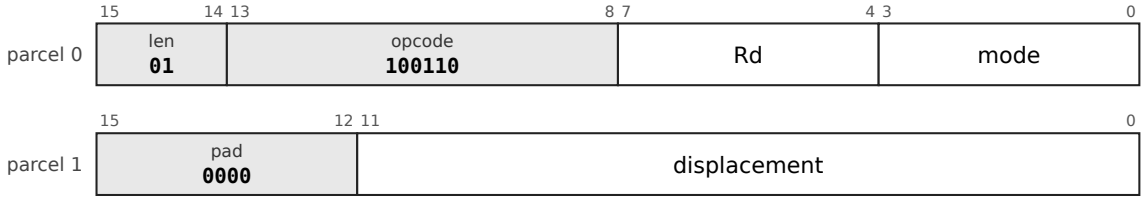
FMUL, opcode 100110, two parcels (modes index) (4 bytes)



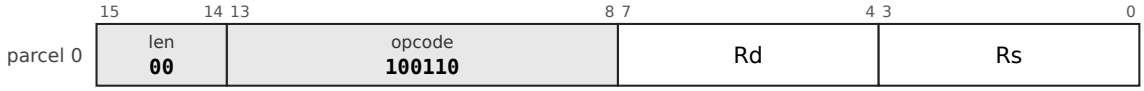
FMUL, opcode 100110, two parcels (modes pcrel, imm, abs) (4 bytes)



FMUL, opcode 100110, two parcels (modes dp) (4 bytes)



FMUL, opcode 100110, one parcel (2 bytes)




```
FMUL rm, Rd
```

Mnemonic	Opcode
FMUL	100110 (0x26)

Operation, flags and exceptions as for the entry FADD above; 9 layout forms, each a chart of its own (above).

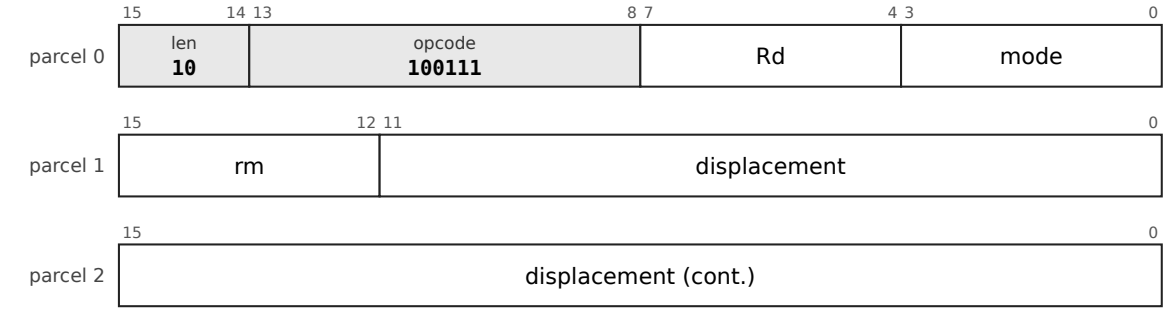
FADDD
An alias of FADD: the assembler writes it as that word.

FSUBD
An alias of FSUB: the assembler writes it as that word.

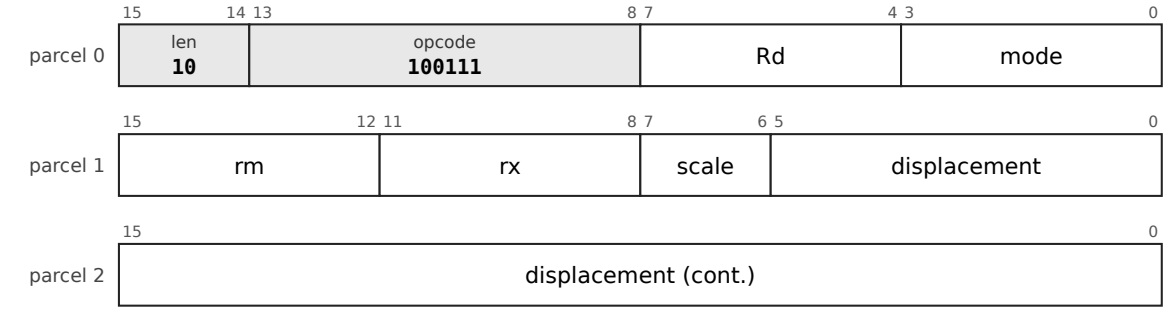
FMULD
An alias of FMUL: the assembler writes it as that word.

FCMP: Floating compare, binary64

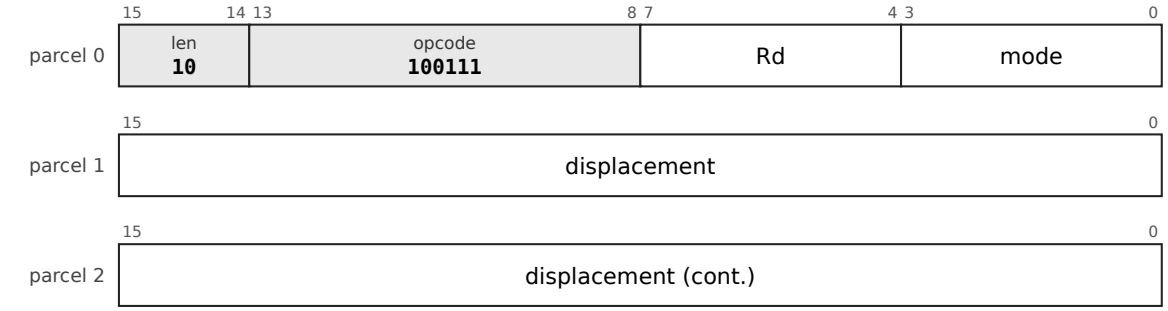
FCMP, opcode 100111, three parcels (modes base, postinc, predec) (6 bytes)



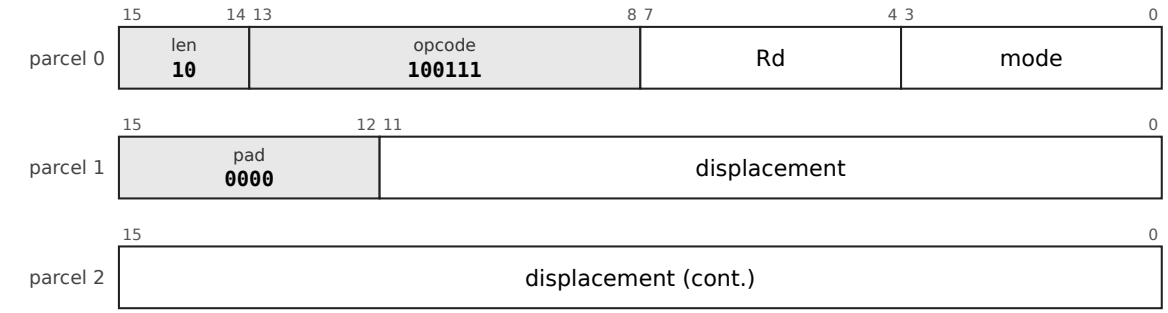
FCMP, opcode 100111, three parcels (modes index) (6 bytes)



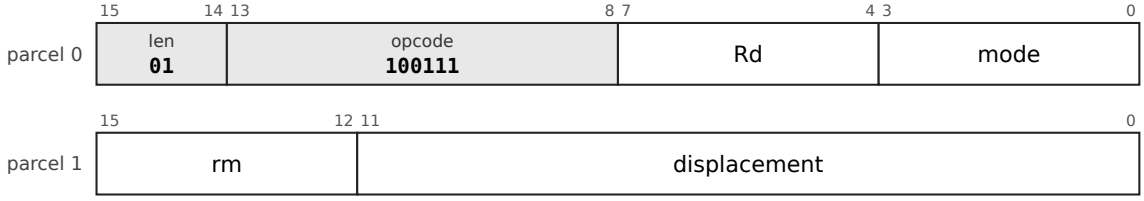
FCMP, opcode 100111, three parcels (modes pcrel, imm, abs) (6 bytes)



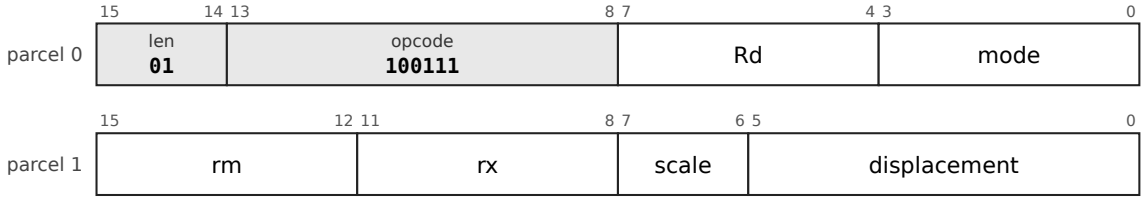
FCMP, opcode 100111, three parcels (modes dp) (6 bytes)



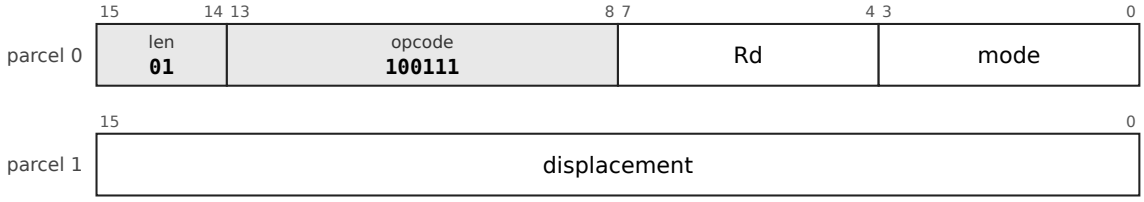
FCMP, opcode 100111, two parcels (modes base, postinc, predec) (4 bytes)



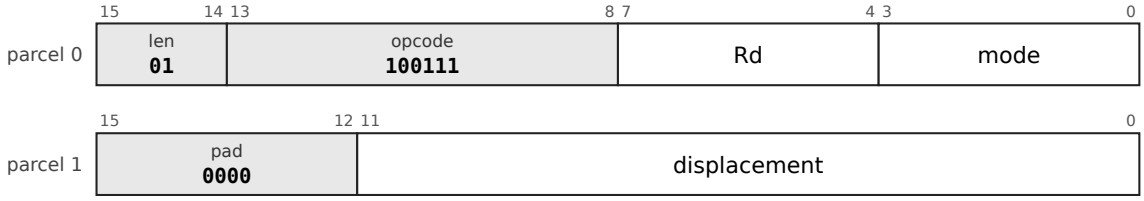
FCMP, opcode 100111, two parcels (modes index) (4 bytes)



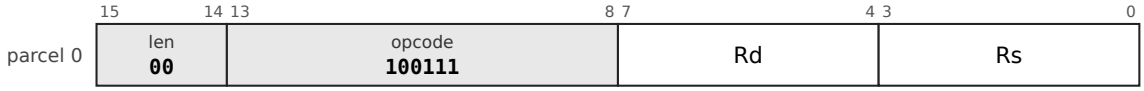
FCMP, opcode 100111, two parcels (modes pcrel, imm, abs) (4 bytes)



FCMP, opcode 100111, two parcels (modes dp) (4 bytes)



FCMP, opcode 100111, one parcel (2 bytes)

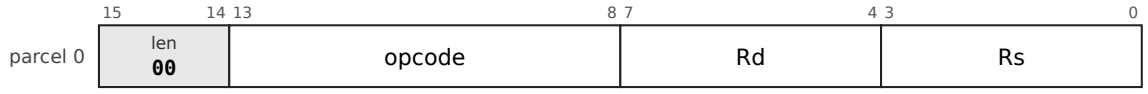


Mnemonics: `FCMP` (aliases: `FCMPD`)

Syntax

```
FCMP Rd, rm        ; register first
```

Encoding



The chart is the layout `r20-mem-0`, Moves, address formation and the ALU, one parcel (`len = 00`): `reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
FCMP	100111 (0x27)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-mem-0`, `r20-mem-1`, `r20-mem-1-2`, `r20-mem-1-3`, `r20-mem-1-4`, `r20-mem-2`, `r20-mem-2-2`, `r20-mem-2-3`, `r20-mem-2-4`.

Operation

```
N ← Rd < rm ; Z ← Rd = rm ; V ← unordered ; C ← 0
```

Flags `N` set if `Rd < rm`, `Z` if equal, `V` if the operands are unordered (either is a NaN), `C` cleared. A signalling NaN raises `NV` in `FPSR`.

Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The signed branches do not treat the unordered case cleanly (`BLT` after an unordered compare takes `N xor V` = 1); this is an open item of the instruction set.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

`FCMPD` is an alias of `FCMP`: the assembler writes it as that word.

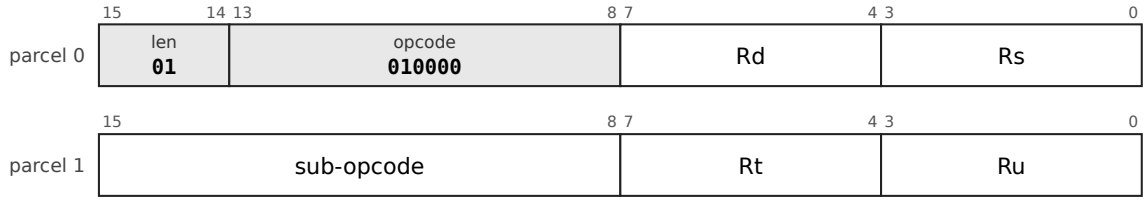
FDIV: Floating divide and square root, binary64

Mnemonics: `FDIV`, `FSQRT` (aliases: `FDIVD`, `FSQRTD`)

Syntax

```
FDIV Rs, Rd
FSQRT Rs, Rd
```

Encoding



The chart is the layout `r20-fesc-1`, Floating point (FESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FDIV	010000 (0x10)	00000000 (0x00)
FSQRT	010000 (0x10)	00000001 (0x01)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-fesc-1`, `r20-fesc-2`.

Operation

```
FDIV : Rd ← Rd ÷ Rs
FSQRT: Rd ← sqrt(Rs)
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps.

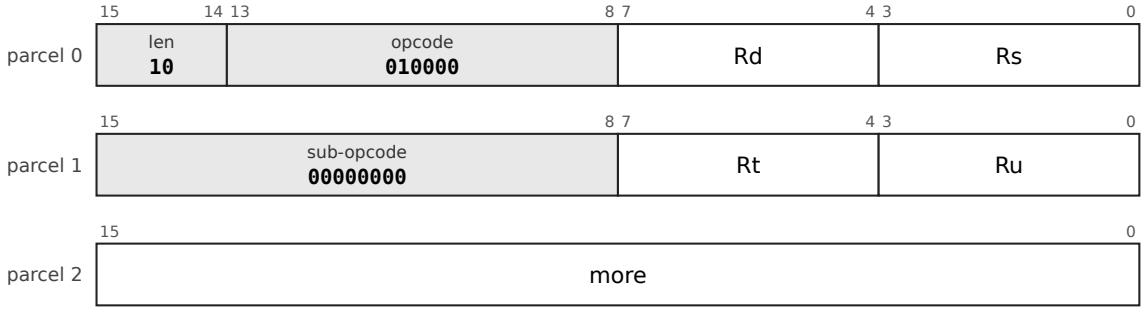
Exceptions None.

FDIV and FSQRT are multi-cycle sequencer operations and rarely appear in a two-byte form, so they are in FESC. The alternative spellings FDIVD, FSQRTD are aliases.

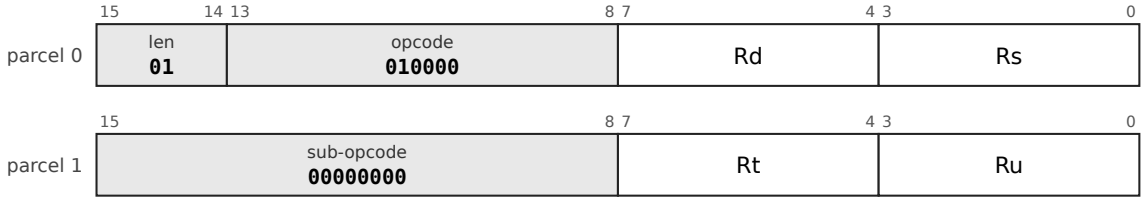
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

FDIV

FDIV, opcode 010000, three parcels, sub-opcode 00000000 (6 bytes)



FDIV, opcode 010000, two parcels, sub-opcode 00000000 (4 bytes)



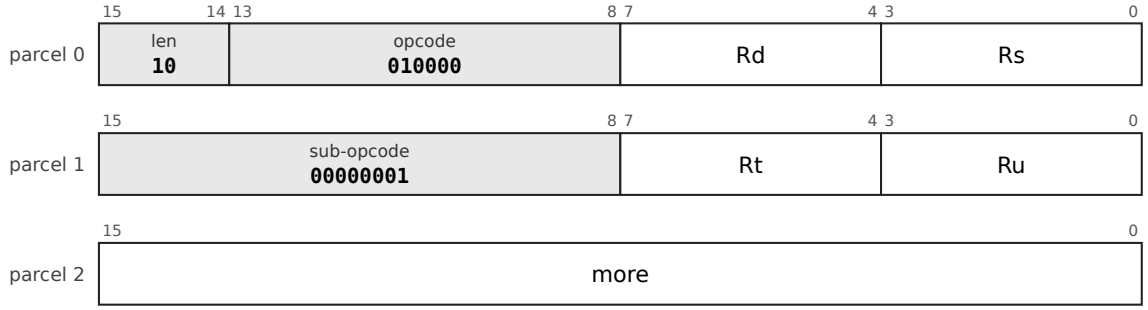

```
FDIV Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FDIV	010000 (0x10)	00000000 (0x00)

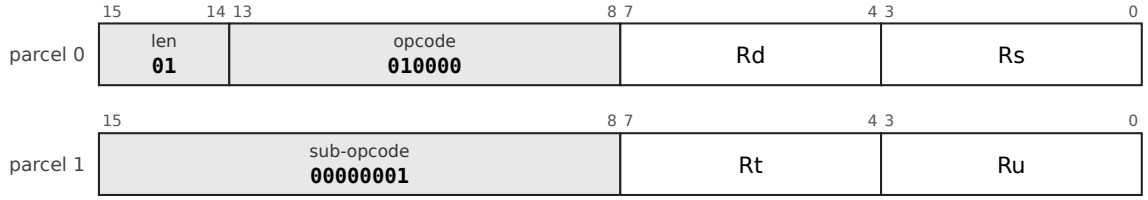
Operation, flags and exceptions as for the entry FDIV above; 2 layout forms, each a chart of its own (above).

FSQRT

FSQRT, opcode 010000, three parcels, sub-opcode 00000001 (6 bytes)



FSQRT, opcode 010000, two parcels, sub-opcode 00000001 (4 bytes)




```
FSQRT Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FSQRT	010000 (0x10)	00000001 (0x01)

Operation, flags and exceptions as for the entry FDIV above; 2 layout forms, each a chart of its own (above).

FDIVD

An alias of FDIV: the assembler writes it as that word.

FSQRTD

An alias of FSQRT: the assembler writes it as that word.

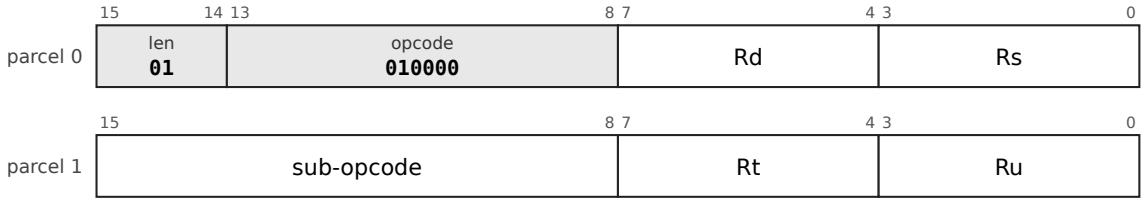
FSINGLE: Floating arithmetic, binary32

Mnemonics: FADD32, FSUB32, FMUL32, FDIV32, FSQRT32, FCMP32 (aliases: FADDS, FSUBS, FMULS, FDIVS, FSQRTS, FCMPS)

Syntax

```
FADD32 Rs, Rd
FSUB32 Rs, Rd
FMUL32 Rs, Rd
FDIV32 Rs, Rd
FSQRT32 Rs, Rd
FCMP32 Rd, Rs
```

Encoding



The chart is the layout `r20-fesc-1`, Floating point (FESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FADD32	010000 (0x10)	00001000 (0x08)
FSUB32	010000 (0x10)	00001001 (0x09)
FMUL32	010000 (0x10)	00001010 (0x0A)
FDIV32	010000 (0x10)	00001011 (0x0B)
FSQRT32	010000 (0x10)	00001100 (0x0C)
FCMP32	010000 (0x10)	00001101 (0x0D)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-fesc-1`, `r20-fesc-2`.

Operation

```
Rd ← Rd op Rs  (FADD32 FSUB32 FMUL32 FDIV32); Rd ← sqrt(Rs) (FSQRT32)
FCMP32: flags as FCMP, comparing Rd with Rs
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps. `FCMP32` writes `N Z V C` as `FCMP` does.

Exceptions None.

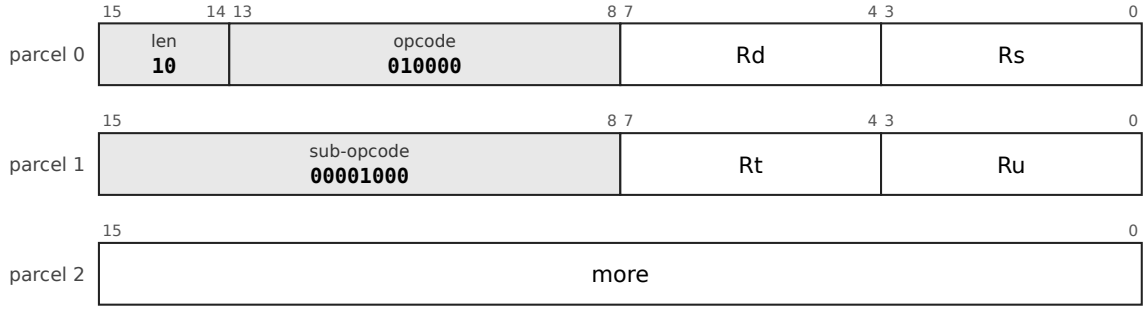
A `binary32` lives in the low 32 bits of a register, sign-extended to 64 like every 32-bit result. Results are correctly rounded to nearest even. The alternative spellings `FADDS`, `FSUBS`, ... are aliases.

Why it is here. `binary64` has first-class opcodes because on a 64-bit machine it is the common case; `binary32` is in `FESC` with the same operations, held in the low 32 bits of a register and sign-extended like every other 32-bit value (see also the ABI). The naming rule is an `F` in front of the integer mnemonic (`FADD`) and a size suffix for the other formats (`FADD16`); the older spellings (`FADDS` and so on) are aliases.

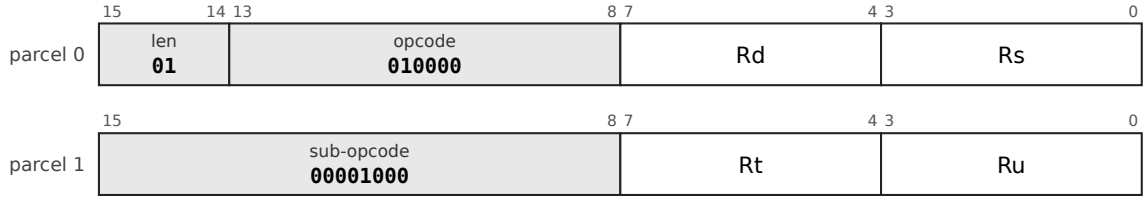
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

FADD32

FADD32, opcode 010000, three parcels, sub-opcode 00001000 (6 bytes)



FADD32, opcode 010000, two parcels, sub-opcode 00001000 (4 bytes)



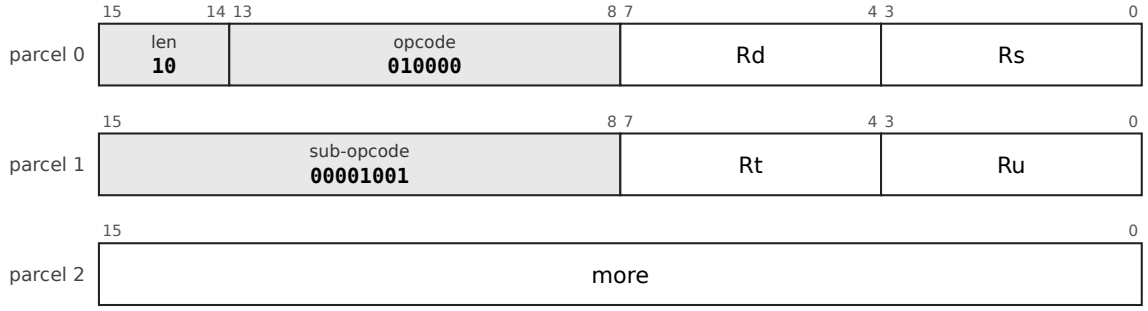

```
FADD32 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FADD32	010000 (0x10)	00001000 (0x08)

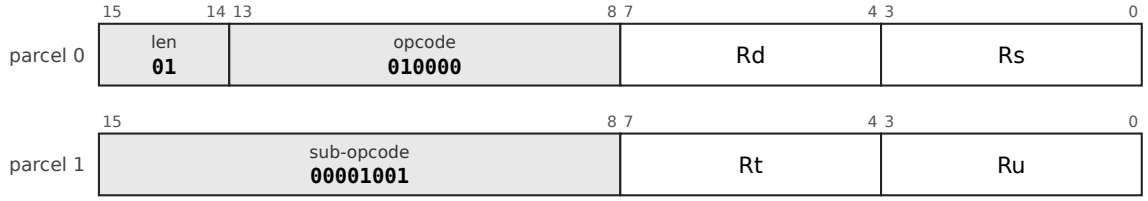
Operation, flags and exceptions as for the entry FSINGLE above; 2 layout forms, each a chart of its own (above).

FSUB32

FSUB32, opcode 010000, three parcels, sub-opcode 00001001 (6 bytes)



FSUB32, opcode 010000, two parcels, sub-opcode 00001001 (4 bytes)



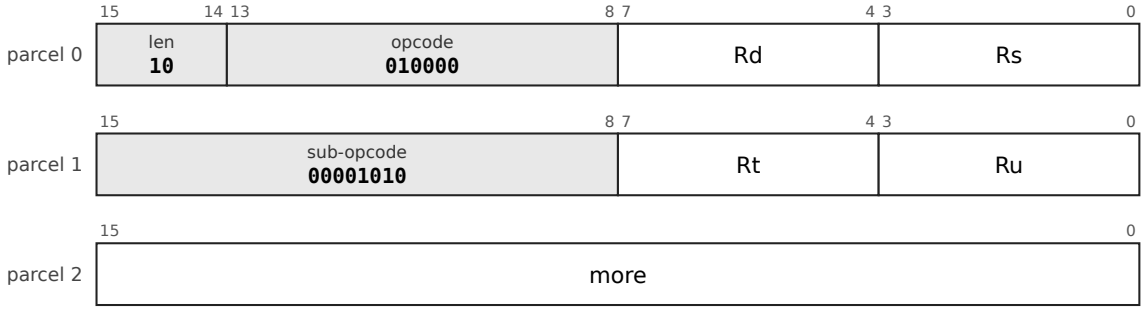

```
FSUB32 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FSUB32	010000 (0x10)	00001001 (0x09)

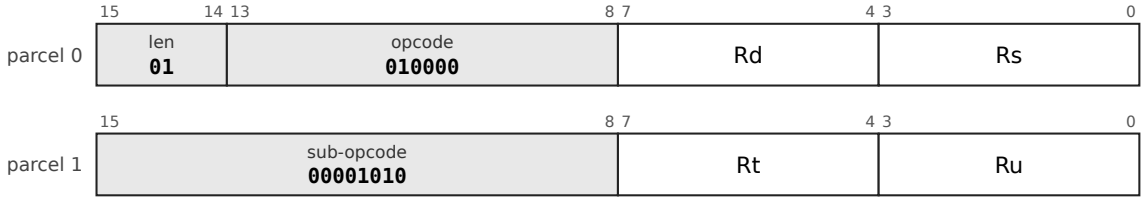
Operation, flags and exceptions as for the entry FSINGLE above; 2 layout forms, each a chart of its own (above).

FMUL32

FMUL32, opcode 010000, three parcels, sub-opcode 00001010 (6 bytes)



FMUL32, opcode 010000, two parcels, sub-opcode 00001010 (4 bytes)



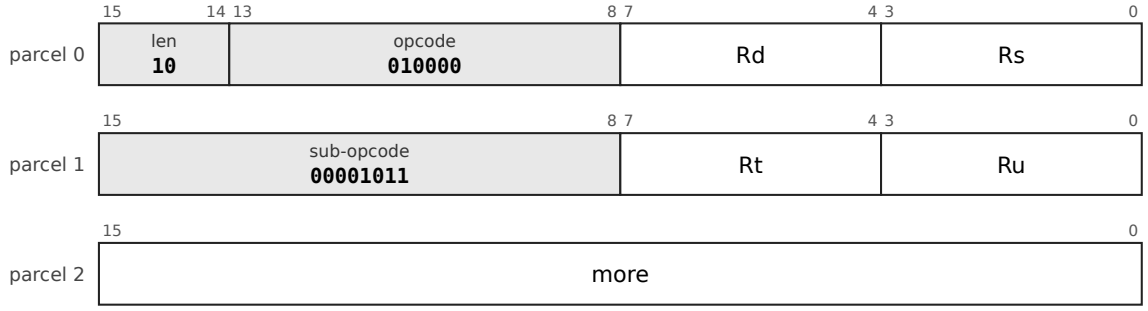

```
FMUL32 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FMUL32	010000 (0x10)	00001010 (0x0A)

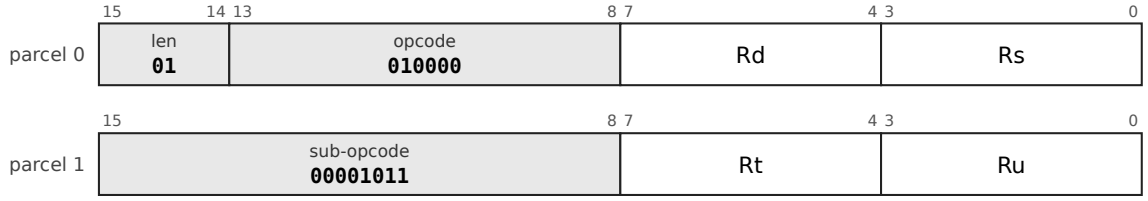
Operation, flags and exceptions as for the entry FSINGLE above; 2 layout forms, each a chart of its own (above).

FDIV32

FDIV32, opcode 010000, three parcels, sub-opcode 00001011 (6 bytes)



FDIV32, opcode 010000, two parcels, sub-opcode 00001011 (4 bytes)



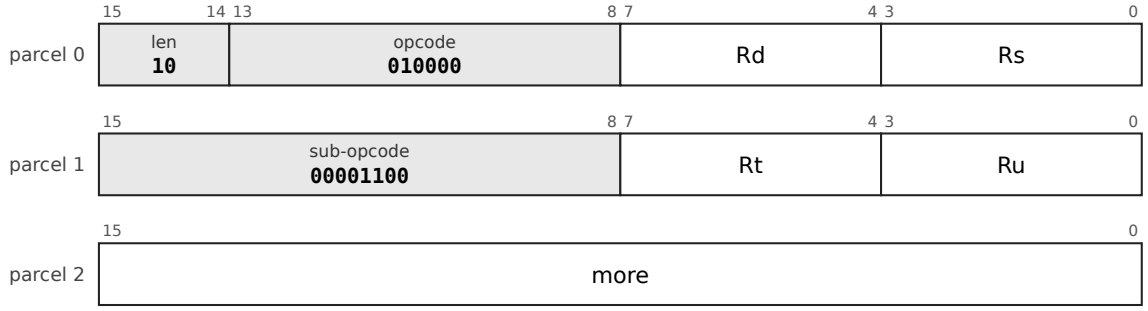

```
FDIV32 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FDIV32	010000 (0x10)	00001011 (0x0B)

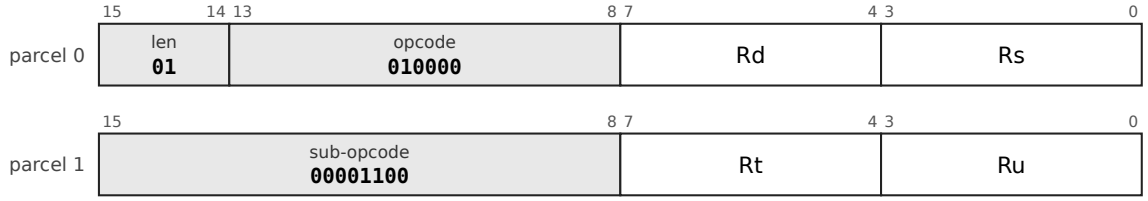
Operation, flags and exceptions as for the entry FSINGLE above; 2 layout forms, each a chart of its own (above).

FSQRT32

FSQRT32, opcode 010000, three parcels, sub-opcode 00001100 (6 bytes)



FSQRT32, opcode 010000, two parcels, sub-opcode 00001100 (4 bytes)



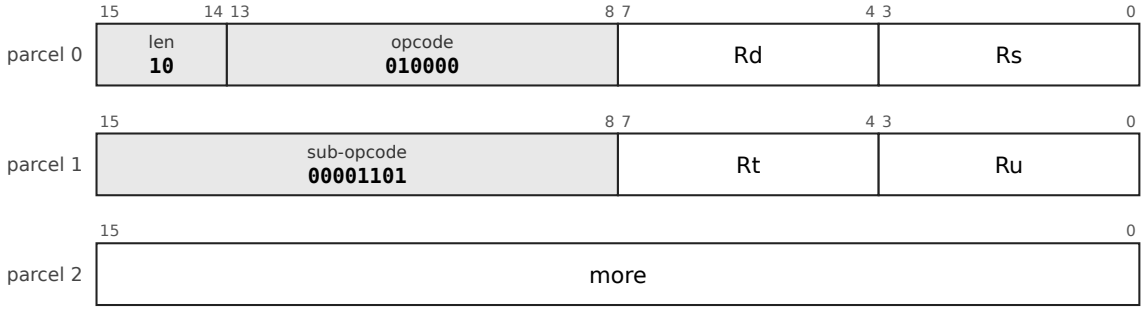

```
FSQRT32 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FSQRT32	010000 (0x10)	00001100 (0x0C)

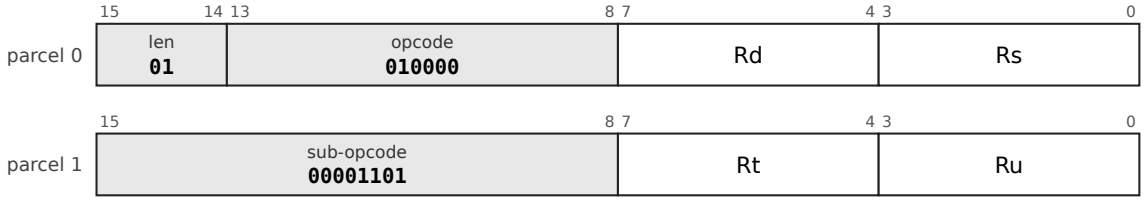
Operation, flags and exceptions as for the entry FSINGLE above; 2 layout forms, each a chart of its own (above).

FCMP32

FCMP32, opcode 010000, three parcels, sub-opcode 00001101 (6 bytes)



FCMP32, opcode 010000, two parcels, sub-opcode 00001101 (4 bytes)




```
FCMP32 Rd, Rs
```

Mnemonic	Opcode	Sub-opcode
FCMP32	010000 (0x10)	00001101 (0x0D)

Operation, flags and exceptions as for the entry FSINGLE above; 2 layout forms, each a chart of its own (above).

FADDS
An alias of FADD32: the assembler writes it as that word.

FSUBS
An alias of FSUB32: the assembler writes it as that word.

FMULS
An alias of FMUL32: the assembler writes it as that word.

FDIVS
An alias of FDIV32: the assembler writes it as that word.

FSQRTS
An alias of FSQRT32: the assembler writes it as that word.

FCMPS
An alias of FCMP32: the assembler writes it as that word.

FMADD: Fused multiply-add, binary64

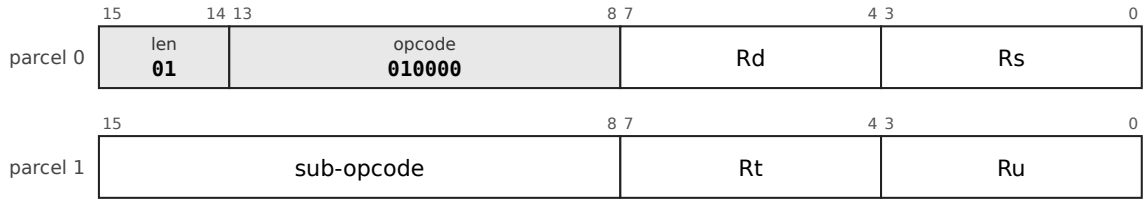
Mnemonics: FMADD, FMSUB, FNMADD, FNMSUB (aliases: FMADDD, FMSUBD, FNMADDD, FNMSUBD)

Syntax

```
FMADD Rs, Rt, Ru, Rd
FMSUB Rs, Rt, Ru, Rd
```

```
FNMADD Rs, Rt, Ru, Rd
FNMSUB Rs, Rt, Ru, Rd
```

Encoding



The chart is the layout `r20-fesc-1`, Floating point (FESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FMADD	010000 (0x10)	00010000 (0x10)
FMSUB	010000 (0x10)	00010001 (0x11)
FNMADD	010000 (0x10)	00010010 (0x12)
FNMSUB	010000 (0x10)	00010011 (0x13)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-fesc-1`, `r20-fesc-2`.

Operation

```
FMADD : Rd ← Rs×Rt + Ru
FMSUB : Rd ← Rs×Rt - Ru
FNMADD: Rd ← -(Rs×Rt) - Ru
FNMSUB: Rd ← -(Rs×Rt) + Ru          ; each rounded once
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps.

Exceptions None.

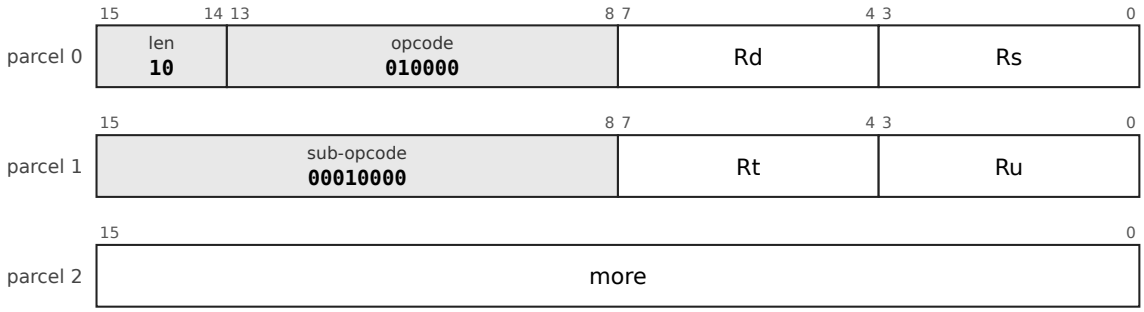
Four register operands are available at `len = 01`. The alternative spellings FMADDD, ... are aliases.

Why it is here. The whole fused multiply-add family is in FESC, because its absence would be the single worst gap in the floating-point set: one rounding instead of two, and the inner step of dot products, polynomial evaluation and every matrix kernel. FESC gives it four register operands. It replaces the VAX POLY, which was the best answer before fused multiply-add.

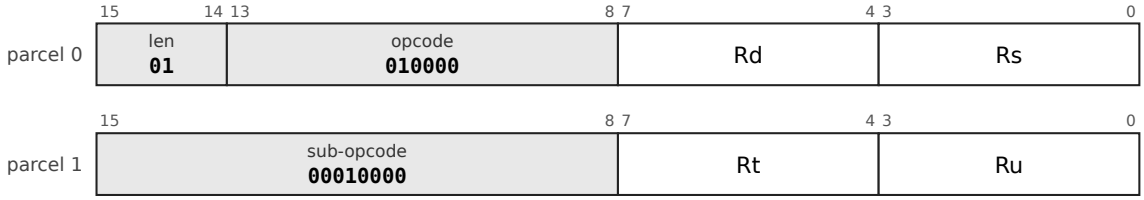
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

FMADD

FMADD, opcode 010000, three parcels, sub-opcode 00010000 (6 bytes)



FMADD, opcode 010000, two parcels, sub-opcode 00010000 (4 bytes)



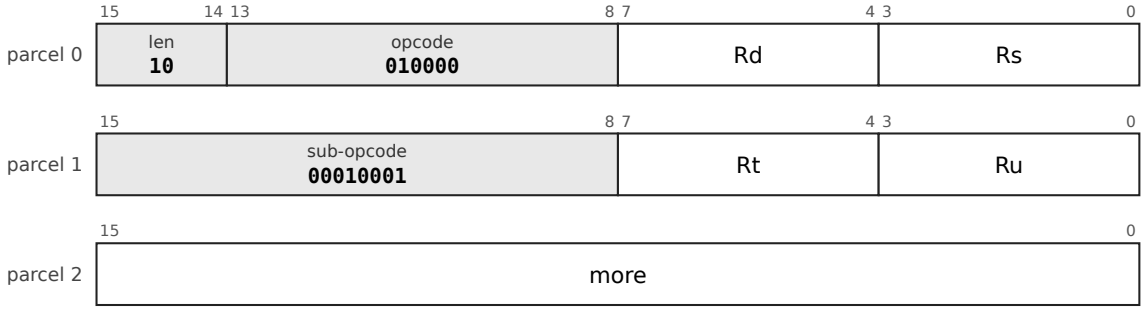

```
FMADD Rs, Rt, Ru, Rd
```

Mnemonic	Opcode	Sub-opcode
FMADD	010000 (0x10)	00010000 (0x10)

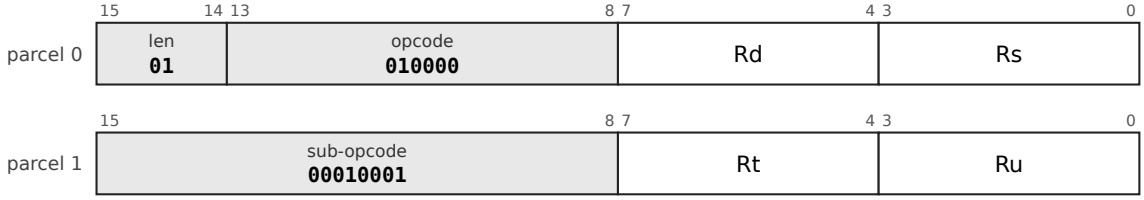
Operation, flags and exceptions as for the entry FMADD above; 2 layout forms, each a chart of its own (above).

FMSUB

FMSUB, opcode 010000, three parcels, sub-opcode 00010001 (6 bytes)



FMSUB, opcode 010000, two parcels, sub-opcode 00010001 (4 bytes)



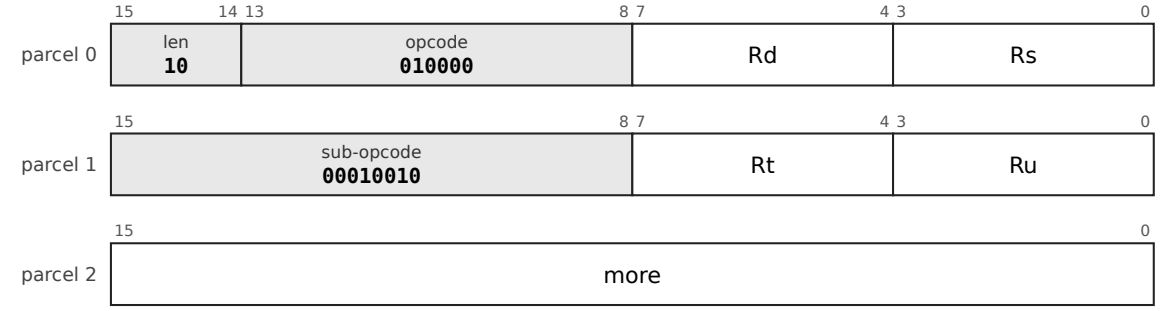

```
FMSUB Rs, Rt, Ru, Rd
```

Mnemonic	Opcode	Sub-opcode
FMSUB	010000 (0x10)	00010001 (0x11)

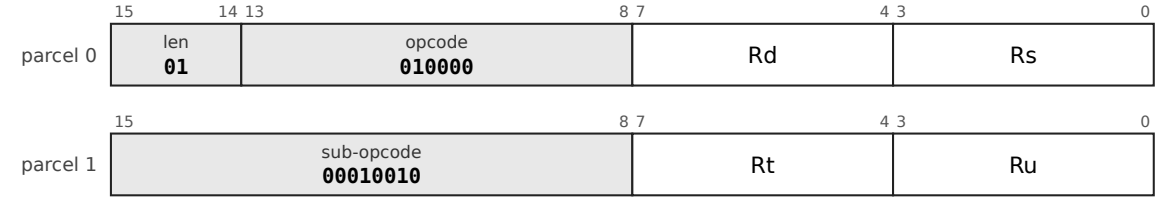
Operation, flags and exceptions as for the entry FMADD above; 2 layout forms, each a chart of its own (above).

FNMADD

FNMADD, opcode 010000, three parcels, sub-opcode 00010010 (6 bytes)



FNMADD, opcode 010000, two parcels, sub-opcode 00010010 (4 bytes)



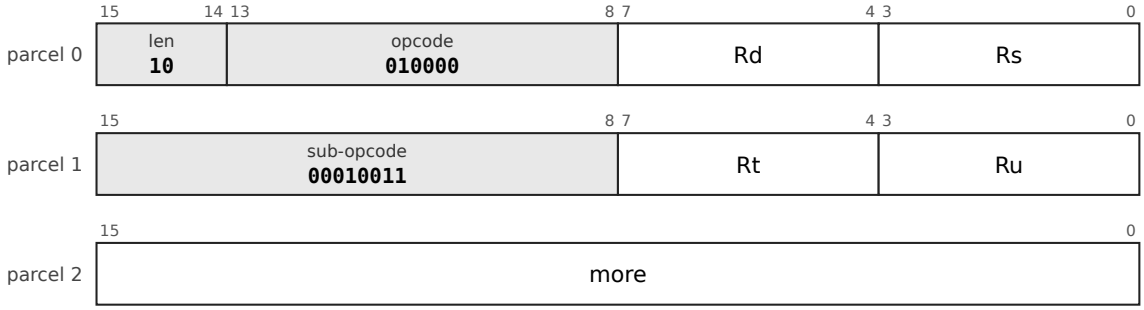

```
FNMADD Rs, Rt, Ru, Rd
```

Mnemonic	Opcode	Sub-opcode
FNMADD	010000 (0x10)	00010010 (0x12)

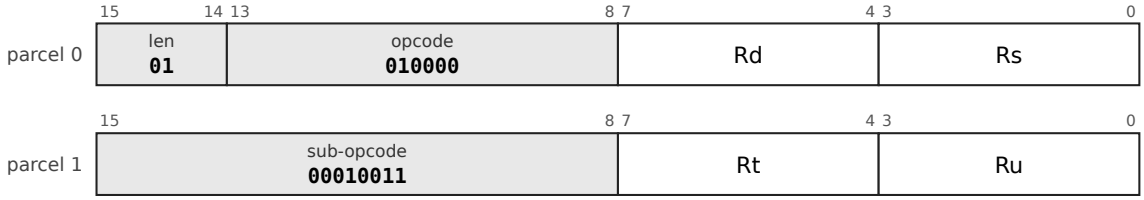
Operation, flags and exceptions as for the entry FMADD above; 2 layout forms, each a chart of its own (above).

FNMSUB

FNMSUB, opcode 010000, three parcels, sub-opcode 00010011 (6 bytes)



FNMSUB, opcode 010000, two parcels, sub-opcode 00010011 (4 bytes)




```
FNMSUB Rs, Rt, Ru, Rd
```

Mnemonic	Opcode	Sub-opcode
FNMSUB	010000 (0x10)	00010011 (0x13)

Operation, flags and exceptions as for the entry FMADD above; 2 layout forms, each a chart of its own (above).

FMADDD

An alias of FMADD: the assembler writes it as that word.

FMSUBD

An alias of FMSUB: the assembler writes it as that word.

FNMADDD

An alias of FNMADD: the assembler writes it as that word.

FNMSUBD

An alias of FNMSUB: the assembler writes it as that word.

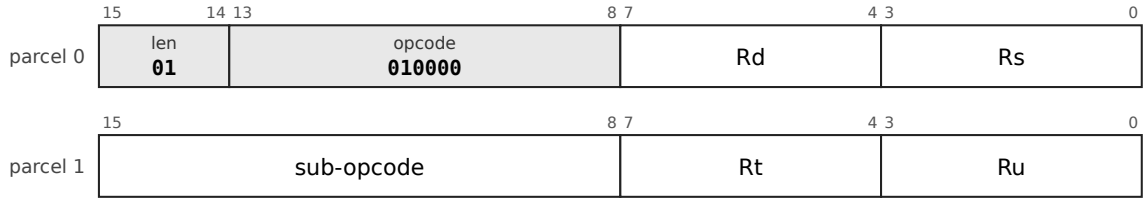
FMADD32: Fused multiply-add, binary32

Mnemonics: FMADD32, FMSUB32, FNMADD32, FNMSUB32 (aliases: FMADDS, FMSUBS, FNMADDS, FNMSUBS)

Syntax

```
FMADD32 Rs, Rt, Ru, Rd
FMSUB32 ...
FNMADD32 ...
FNMSUB32 ...
```

Encoding



The chart is the layout `r20-fesc-1`, Floating point (FESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FMADD32	010000 (0x10)	00011000 (0x18)
FMSUB32	010000 (0x10)	00011001 (0x19)
FNMADD32	010000 (0x10)	00011010 (0x1A)
FNMSUB32	010000 (0x10)	00011011 (0x1B)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-fesc-1`, `r20-fesc-2`.

Operation

```
As FMADD, in binary32, rounded once
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

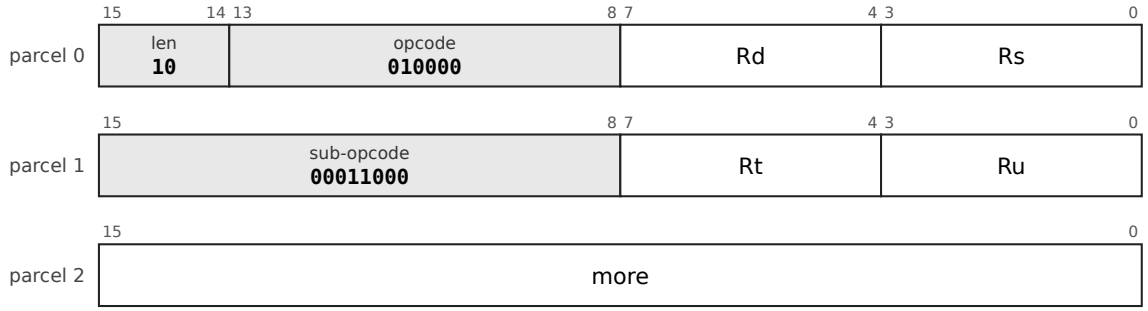
Exceptions None.

Why it is here. The `binary32` forms of the fused multiply-add family.

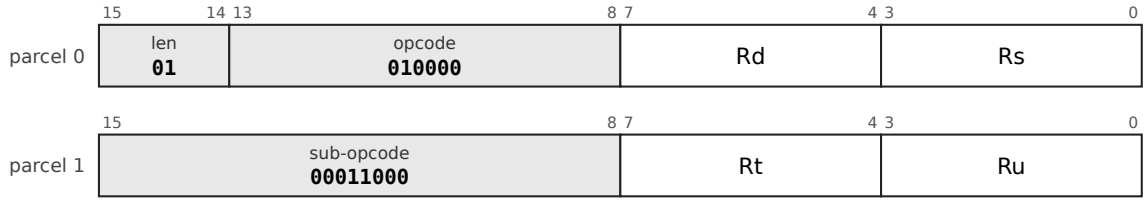
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

FMADD32

FMADD32, opcode 010000, three parcels, sub-opcode 00011000 (6 bytes)



FMADD32, opcode 010000, two parcels, sub-opcode 00011000 (4 bytes)



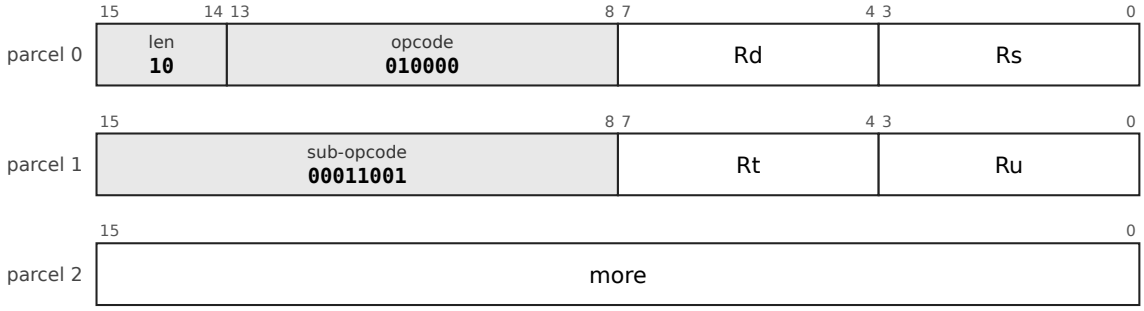

```
FMADD32 Rs, Rt, Ru, Rd
```

Mnemonic	Opcode	Sub-opcode
FMADD32	010000 (0x10)	00011000 (0x18)

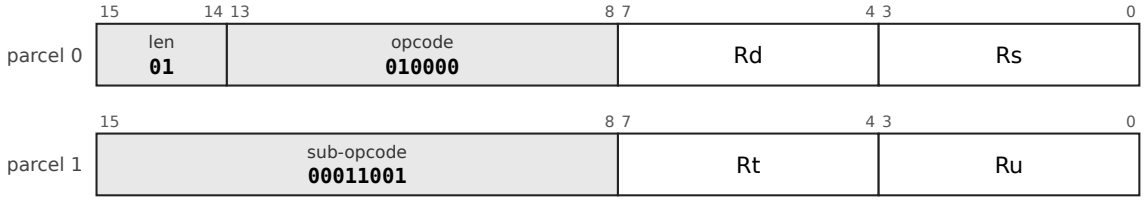
Operation, flags and exceptions as for the entry FMADD32 above; 2 layout forms, each a chart of its own (above).

FMSUB32

FMSUB32, opcode 010000, three parcels, sub-opcode 00011001 (6 bytes)



FMSUB32, opcode 010000, two parcels, sub-opcode 00011001 (4 bytes)



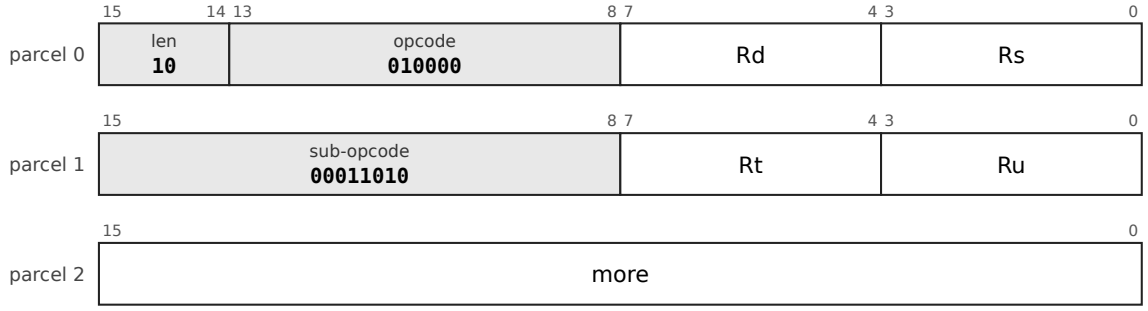

```
FMSUB32 ...
```

Mnemonic	Opcode	Sub-opcode
FMSUB32	010000 (0x10)	00011001 (0x19)

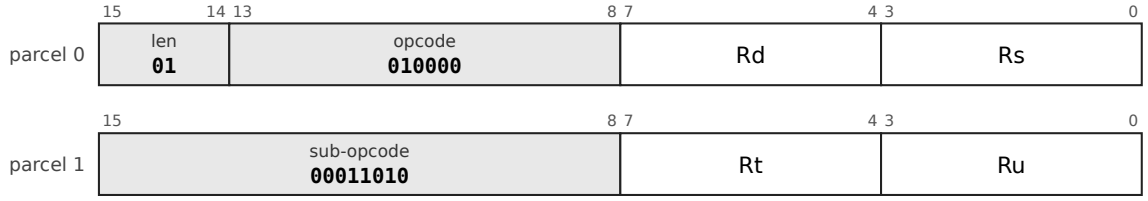
Operation, flags and exceptions as for the entry FMADD32 above; 2 layout forms, each a chart of its own (above).

FNMADD32

FNMADD32, opcode 010000, three parcels, sub-opcode 00011010 (6 bytes)



FNMADD32, opcode 010000, two parcels, sub-opcode 00011010 (4 bytes)



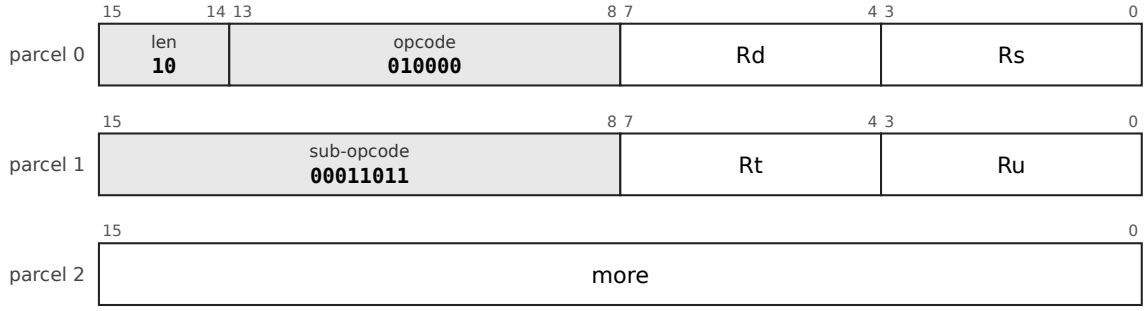

```
FNMADD32 ...
```

Mnemonic	Opcode	Sub-opcode
FNMADD32	010000 (0x10)	00011010 (0x1A)

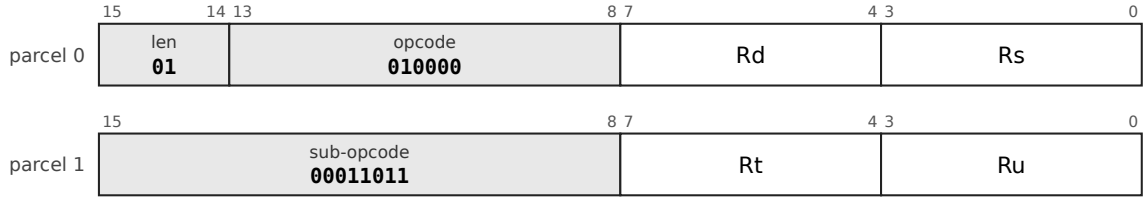
Operation, flags and exceptions as for the entry FMADD32 above; 2 layout forms, each a chart of its own (above).

FNMSUB32

FNMSUB32, opcode 010000, three parcels, sub-opcode 00011011 (6 bytes)



FNMSUB32, opcode 010000, two parcels, sub-opcode 00011011 (4 bytes)




```
FNMSUB32 ...
```

Mnemonic	Opcode	Sub-opcode
FNMSUB32	010000 (0x10)	00011011 (0x1B)

Operation, flags and exceptions as for the entry FMADD32 above; 2 layout forms, each a chart of its own (above).

FMADDS
An alias of FMADD32: the assembler writes it as that word.

FMSUBS
An alias of FMSUB32: the assembler writes it as that word.

FNMADDS
An alias of FNMADD32: the assembler writes it as that word.

FNMSUBS
An alias of FNMSUB32: the assembler writes it as that word.

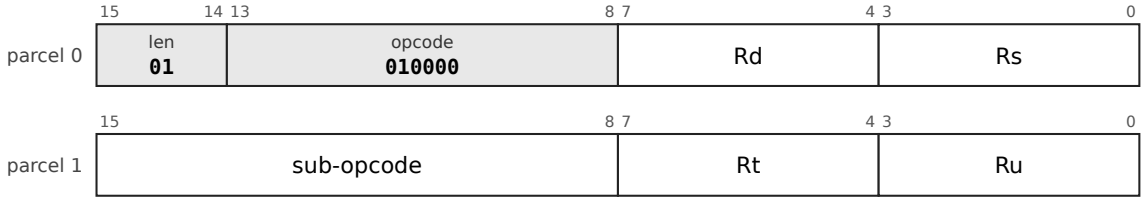
FSIGN: Sign operations on the bits

Mnemonics: FNEG, FABS, FCOPYSIGN

Syntax

```
FNEG Rs, Rd
FABS Rs, Rd
FCOPYSIGN Rs, Rt, Rd
```

Encoding



The chart is the layout r20-fesc-1, Floating point (FESC), two parcels (len = 01): From len = 01. In this group low is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FNEG	010000 (0x10)	00100000 (0x20)
FABS	010000 (0x10)	00100001 (0x21)
FCOPYSIGN	010000 (0x10)	00100010 (0x22)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-fesc-1, r20-fesc-2.

Operation

```
FNEG     : Rd ← Rs with the sign bit flipped
FABS     : Rd ← Rs with the sign bit cleared
FCOPYSIGN: Rd ← magnitude of Rs, sign of Rt
```

Flags None. These operate on the bits, do not canonicalise a NaN and raise no exception flag.

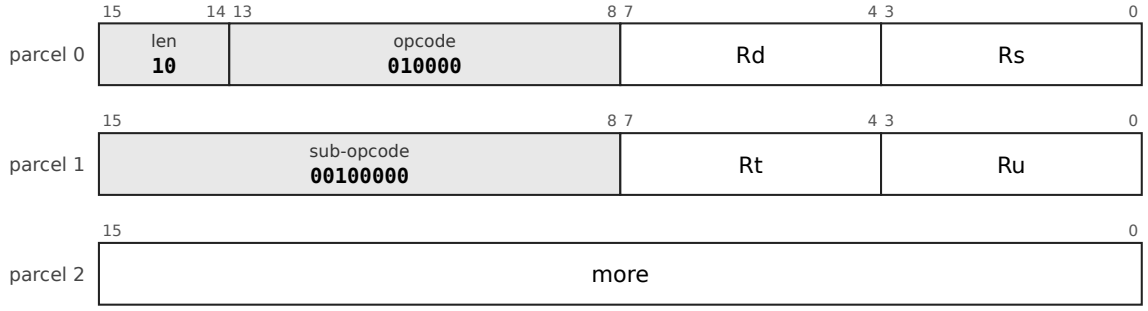
Exceptions None.

Why it is here. Sign and selection operations (negate, absolute value and the like) work on the bits, do not canonicalise a NaN and raise no flag, so they cost almost nothing. The PowerPC 601 floating-point set has the same operations (stated from memory, not checked against a manual). No separate reason is recorded for these three.

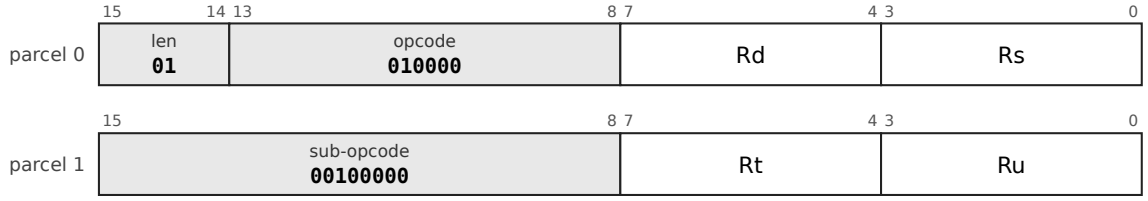
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

FNEG

FNEG, opcode 010000, three parcels, sub-opcode 00100000 (6 bytes)



FNEG, opcode 010000, two parcels, sub-opcode 00100000 (4 bytes)



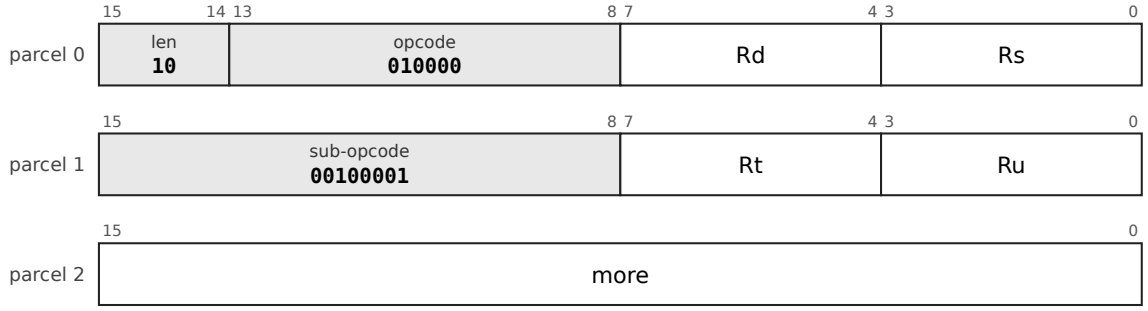

```
FNEG Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FNEG	010000 (0x10)	00100000 (0x20)

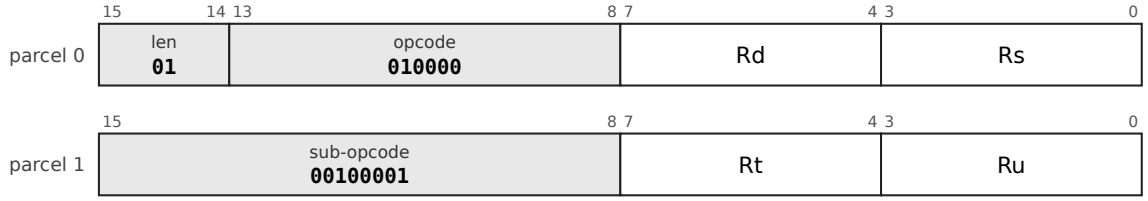
Operation, flags and exceptions as for the entry FSIGN above; 2 layout forms, each a chart of its own (above).

FABS

FABS, opcode 010000, three parcels, sub-opcode 00100001 (6 bytes)



FABS, opcode 010000, two parcels, sub-opcode 00100001 (4 bytes)



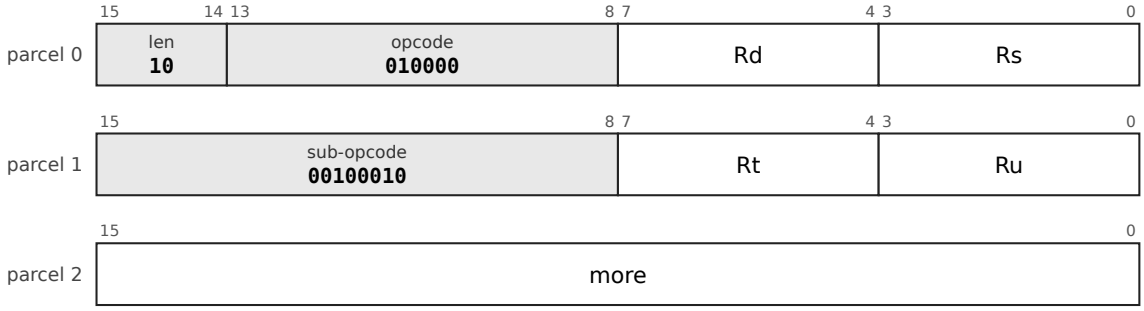

```
FABS Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FABS	010000 (0x10)	00100001 (0x21)

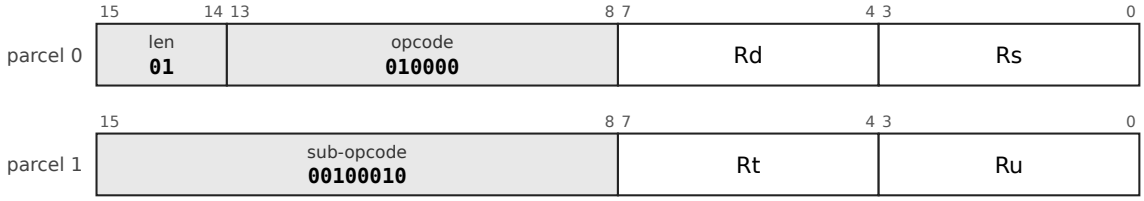
Operation, flags and exceptions as for the entry FSIGN above; 2 layout forms, each a chart of its own (above).

FCOPYSIGN

FCOPYSIGN, opcode 010000, three parcels, sub-opcode 00100010 (6 bytes)



FCOPYSIGN, opcode 010000, two parcels, sub-opcode 00100010 (4 bytes)




```
FCOPYSIGN Rs, Rt, Rd
```

Mnemonic	Opcode	Sub-opcode
FCOPYSIGN	010000 (0x10)	00100010 (0x22)

Operation, flags and exceptions as for the entry FSIGN above; 2 layout forms, each a chart of its own (above).

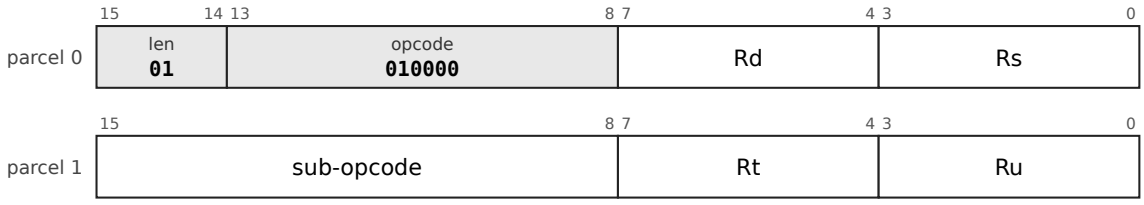
FMIN: Floating minimum and maximum, binary64

Mnemonics: `FMIN`, `FMAX` (aliases: `FMIND`, `FMAXD`)

Syntax

```
FMIN Rs, Rt, Rd
FMAX Rs, Rt, Rd
```

Encoding



The chart is the layout `r20-fesc-1`, Floating point (FESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FMIN	010000 (0x10)	00100011 (0x23)
FMAX	010000 (0x10)	00100100 (0x24)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-fesc-1`, `r20-fesc-2`.

Operation

```
if Rs is a NaN then Rd ← Rt ; else if Rt is a NaN then Rd ← Rs ; else Rd ← smaller (FMIN) or larger (FMAX)
if both are NaNs the result is the canonical quiet NaN
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

Exceptions None.

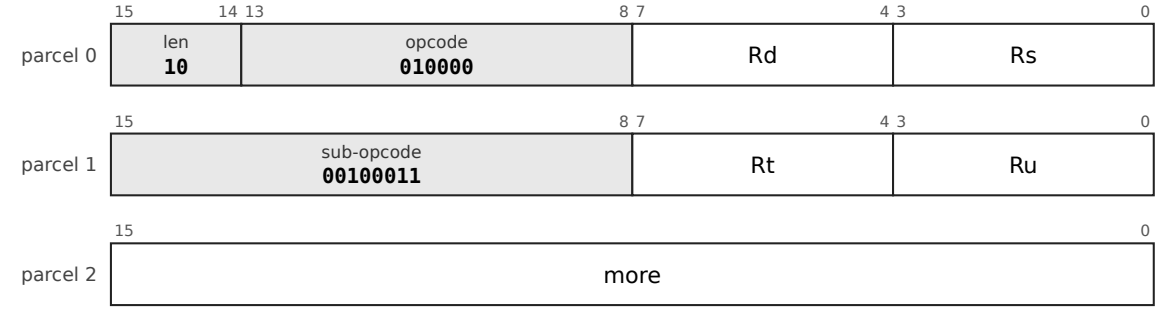
The alternative spellings `FMIND`, `FMAXD` are aliases.

Why it is here. The floating counterparts of the integer `MIN` and `MAX`. A NaN operand loses to a number and two NaNs give the canonical quiet NaN.

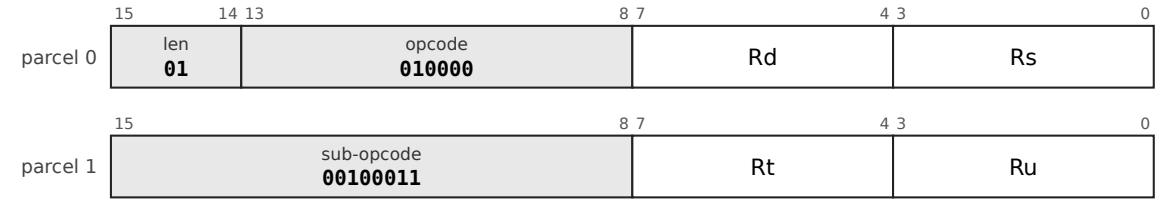
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

FMIN

FMIN, opcode 010000, three parcels, sub-opcode 00100011 (6 bytes)



FMIN, opcode 010000, two parcels, sub-opcode 00100011 (4 bytes)



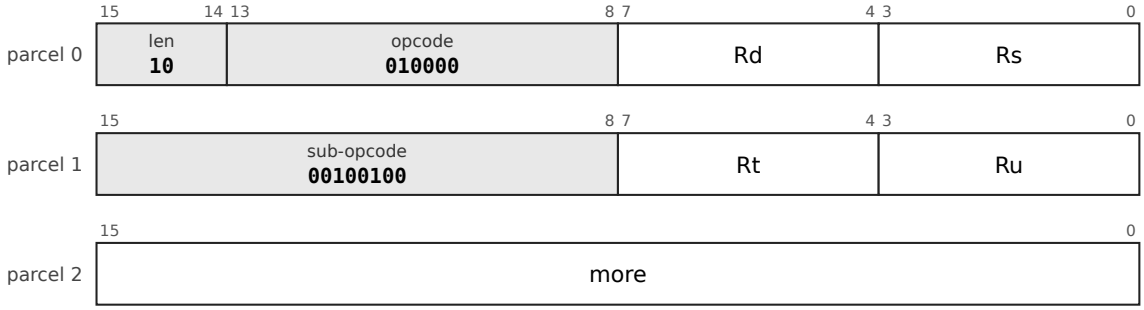

```
FMIN Rs, Rt, Rd
```

Mnemonic	Opcode	Sub-opcode
`FMIN`	`010000 (0x10)`	`00100011 (0x23)`

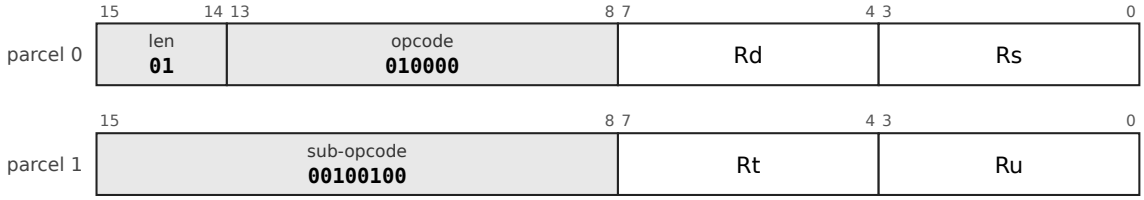
Operation, flags and exceptions as for the entry FMIN above; 2 layout forms, each a chart of its own (above).

FMAX

FMAX, opcode 010000, three parcels, sub-opcode 00100100 (6 bytes)



FMAX, opcode 010000, two parcels, sub-opcode 00100100 (4 bytes)




```
FMAX Rs, Rt, Rd
```

Mnemonic	Opcode	Sub-opcode
FMAX	010000 (0x10)	00100100 (0x24)

Operation, flags and exceptions as for the entry FMIN above; 2 layout forms, each a chart of its own (above).

FMIND

An alias of FMIN: the assembler writes it as that word.

FMAXD

An alias of FMAX: the assembler writes it as that word.

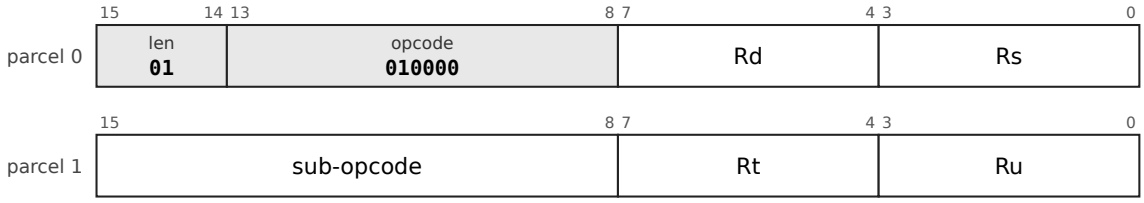
FCLASS: Classify a floating-point value

Mnemonics: FCLASS, FCLASS32, FCLASS16 (aliases: FCLASSD, FCLASSS)

Syntax

```
FCLASS Rs, Rd
FCLASS32 Rs, Rd
FCLASS16 Rs, Rd
```

Encoding



The chart is the layout r20-fesc-1, Floating point (FESC), two parcels (len = 01): From len = 01. In this group low is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FCLASS	010000 (0x10)	00101000 (0x28)
FCLASS32	010000 (0x10)	00101001 (0x29)
FCLASS16	010000 (0x10)	01011100 (0x5C)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-fesc-1`, `r20-fesc-2`.

Operation

```
Rd ← a bit vector with exactly one bit set:
bit 0 -infinity      bit 1 -normal      bit 2 -subnormal   bit 3 -zero
bit 4 +zero          bit 5 +subnormal   bit 6 +normal      bit 7 +infinity
bit 8 signalling NaN bit 9 quiet NaN
```

Flags None.

Exceptions None.

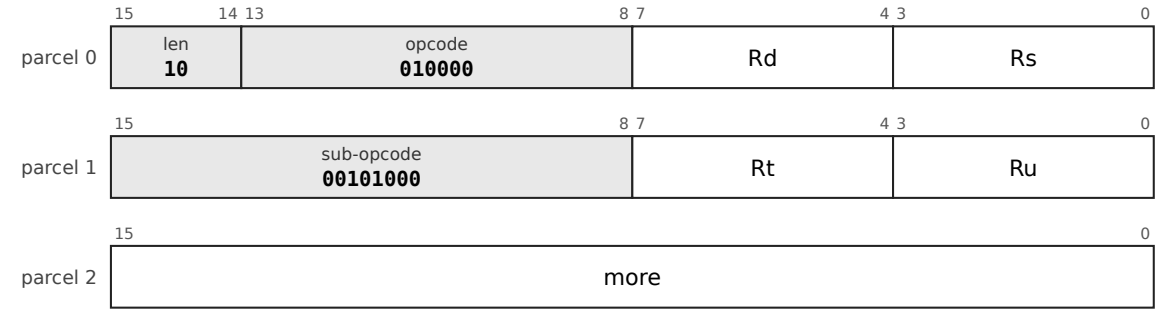
Every libm function begins with this dispatch. The alternative spellings `FCLASSD`, `FCLASSS` are aliases.

Why it is here. Every libm function begins with this dispatch: one instruction returns which of ten kinds a value is (signalling or quiet NaN, infinity, normal, subnormal, zero, each signed), where software needs several compares and masks. It is justified by a workload, library mathematics, and not by compiled C, as the tag branches are. Nothing in the corpus emits it, because the compiler does not use it.

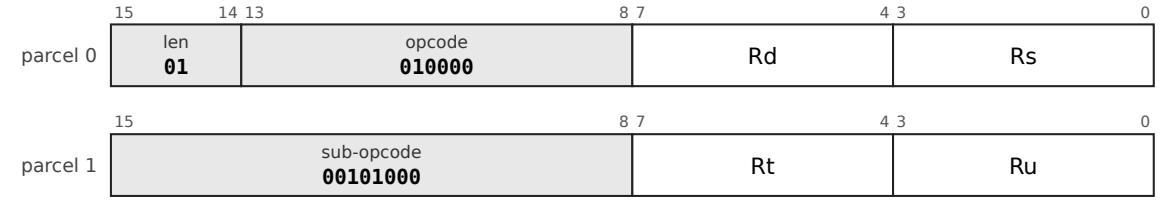
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

FCLASS

FCLASS, opcode 010000, three parcels, sub-opcode 00101000 (6 bytes)



FCLASS, opcode 010000, two parcels, sub-opcode 00101000 (4 bytes)



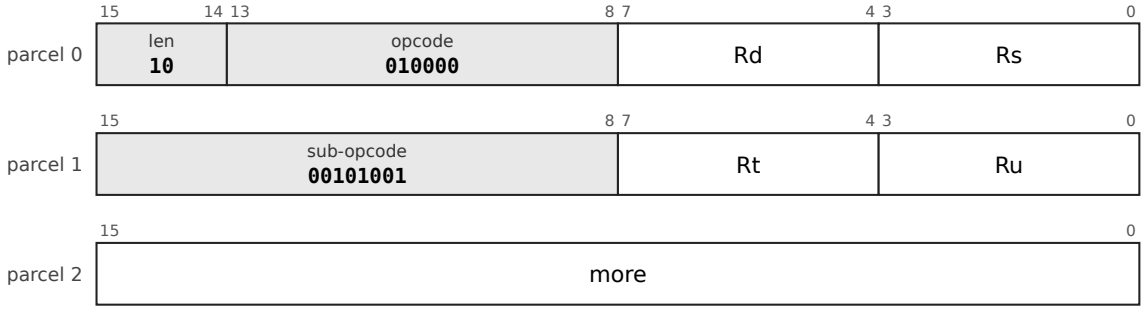

```
FCLASS Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCLASS	010000 (0x10)	00101000 (0x28)

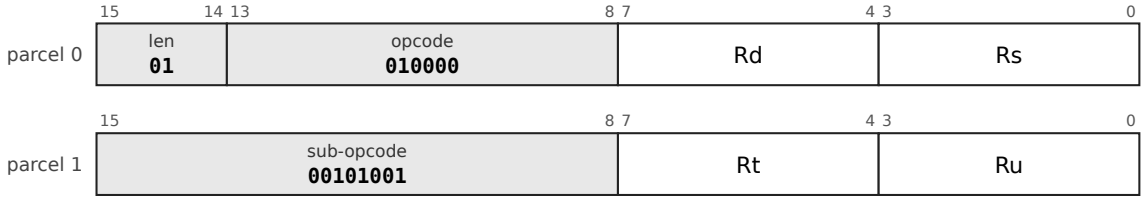
Operation, flags and exceptions as for the entry FCLASS above; 2 layout forms, each a chart of its own (above).

FCLASS32

FCLASS32, opcode 010000, three parcels, sub-opcode 00101001 (6 bytes)



FCLASS32, opcode 010000, two parcels, sub-opcode 00101001 (4 bytes)



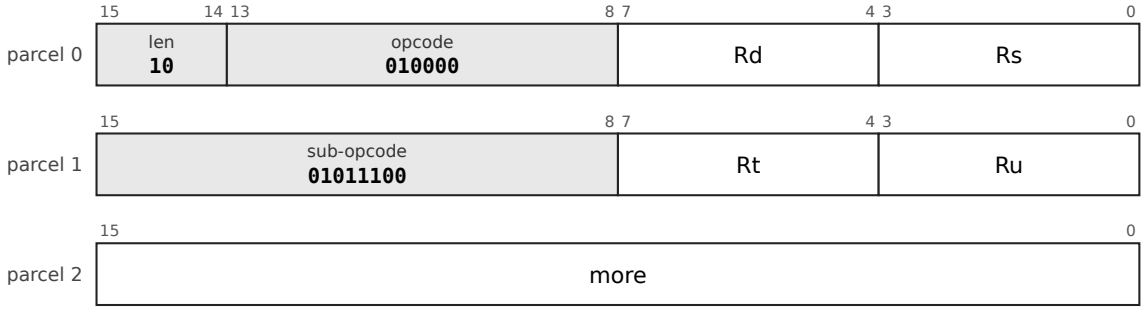

```
FCLASS32 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCLASS32	010000 (0x10)	00101001 (0x29)

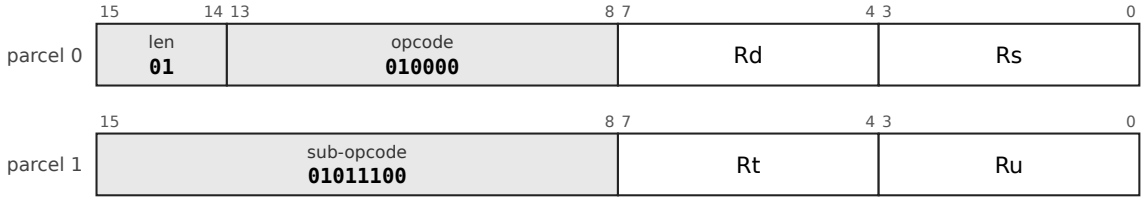
Operation, flags and exceptions as for the entry FCLASS above; 2 layout forms, each a chart of its own (above).

FCLASS16

FCLASS16, opcode 010000, three parcels, sub-opcode 01011100 (6 bytes)



FCLASS16, opcode 010000, two parcels, sub-opcode 01011100 (4 bytes)




```
FCLASS16 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCLASS16	010000 (0x10)	01011100 (0x5C)

Operation, flags and exceptions as for the entry FCLASS above; 2 layout forms, each a chart of its own (above).

FCLASSD
An alias of FCLASS: the assembler writes it as that word.

FCLASSS
An alias of FCLASS32: the assembler writes it as that word.

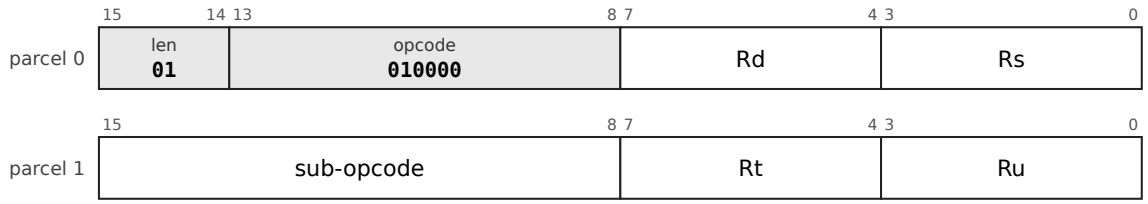
FRINT: Round to an integral value, binary64

Mnemonics: FRINT, FRINTZ, FRINTM, FRINTP, FRINTA (aliases: FRINTD, FRINTDZ, FRINTDM, FRINTDP, FRINTDA)

Syntax

```
FRINT Rs, Rd
FRINTZ Rs, Rd
FRINTM Rs, Rd
FRINTP Rs, Rd
FRINTA Rs, Rd
```

Encoding



The chart is the layout r20-fesc-1, Floating point (FESC), two parcels (len = 01): From len = 01. In this group low is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FRINT	010000 (0x10)	00110000 (0x30)
FRINTZ	010000 (0x10)	00110001 (0x31)
FRINTM	010000 (0x10)	00110010 (0x32)
FRINTP	010000 (0x10)	00110011 (0x33)
FRINTA	010000 (0x10)	00110100 (0x34)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-fesc-1, r20-fesc-2.

Operation

```
FRINT : Rd ← Rs rounded to nearest, ties to even
FRINTZ: toward zero         FRINTM: toward -infinity
FRINTP: toward +infinity    FRINTA: to nearest, ties away from zero (C round())
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps.

Exceptions None.

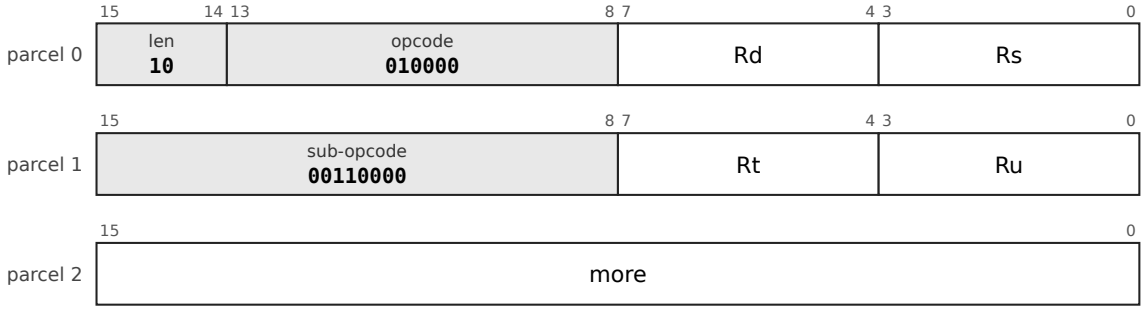
The rounding mode is in the opcode, never in a mode bit. The alternative spellings FRINTD, FRINTDZ, ... are aliases.

Why it is here. Rounding to an integral value in each direction (nearest even, toward zero, down, up) is in the set, and FRINTA, ties away from zero, serves C's round(), which is common in libm; it is a qualifier of the round operation and so costs no new operation.

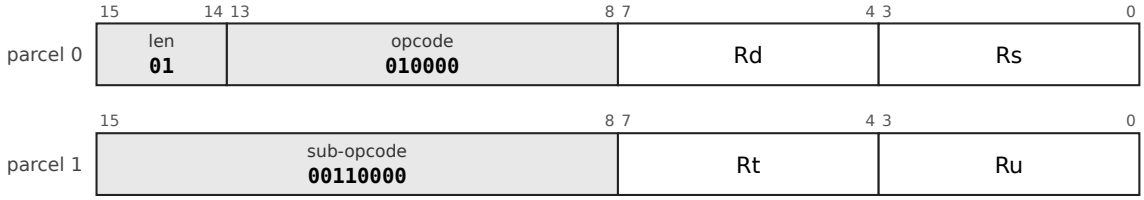
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

FRINT

FRINT, opcode 010000, three parcels, sub-opcode 00110000 (6 bytes)



FRINT, opcode 010000, two parcels, sub-opcode 00110000 (4 bytes)



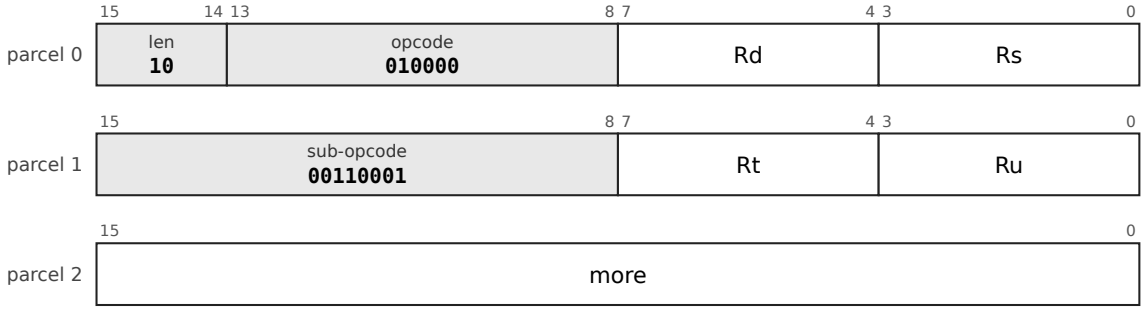

```
FRINT Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FRINT	010000 (0x10)	00110000 (0x30)

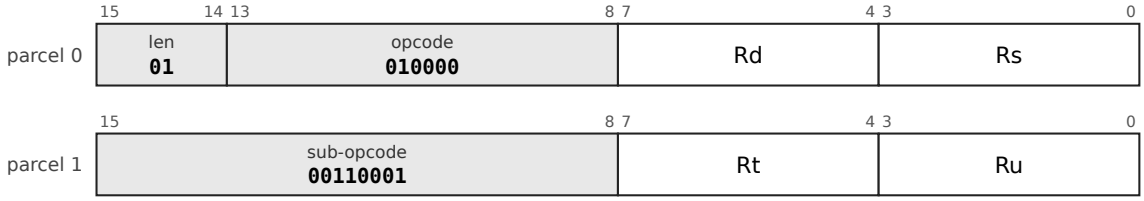
Operation, flags and exceptions as for the entry FRINT above; 2 layout forms, each a chart of its own (above).

FRINTZ

FRINTZ, opcode 010000, three parcels, sub-opcode 00110001 (6 bytes)



FRINTZ, opcode 010000, two parcels, sub-opcode 00110001 (4 bytes)



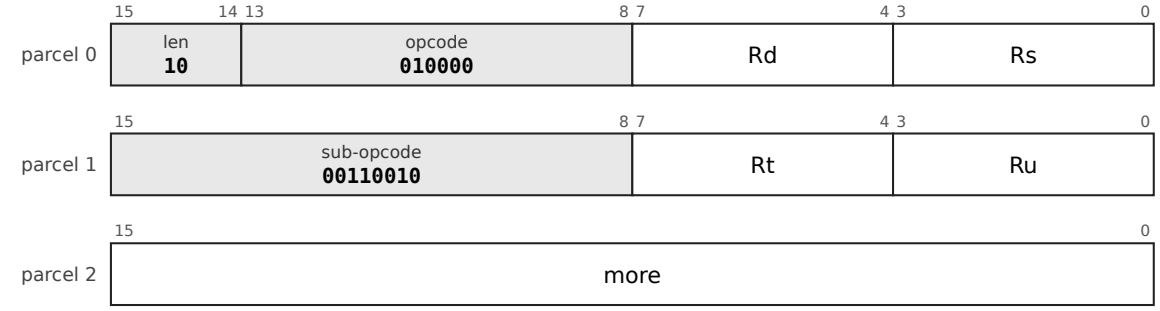

```
FRINTZ Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FRINTZ	010000 (0x10)	00110001 (0x31)

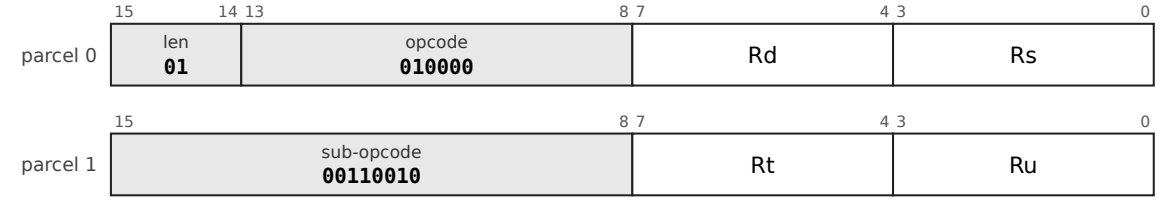
Operation, flags and exceptions as for the entry FRINT above; 2 layout forms, each a chart of its own (above).

FRINTM

FRINTM, opcode 010000, three parcels, sub-opcode 00110010 (6 bytes)



FRINTM, opcode 010000, two parcels, sub-opcode 00110010 (4 bytes)



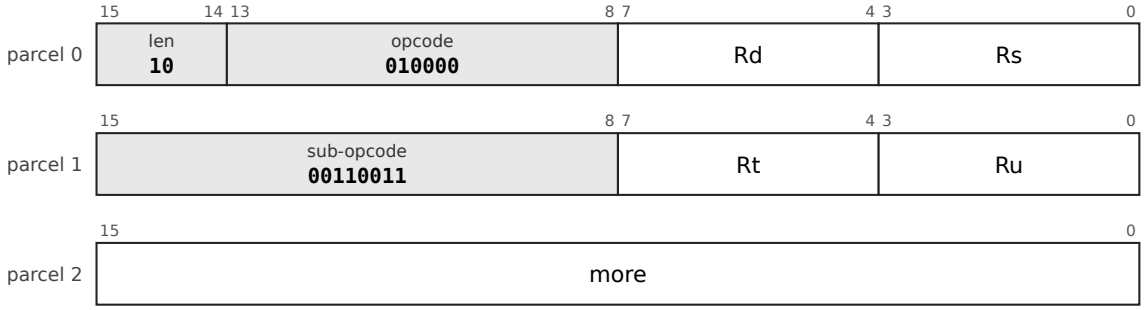

```
FRINTM Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FRINTM	010000 (0x10)	00110010 (0x32)

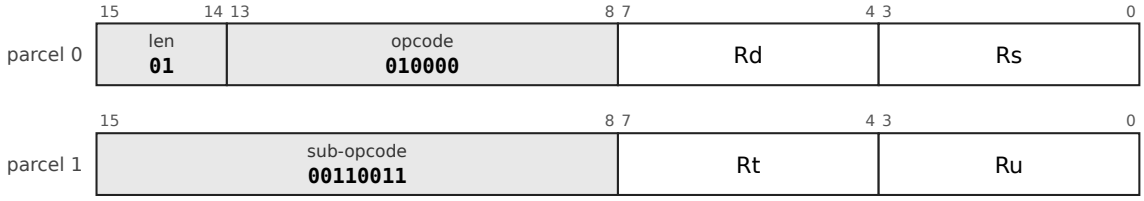
Operation, flags and exceptions as for the entry FRINT above; 2 layout forms, each a chart of its own (above).

FRINTP

FRINTP, opcode 010000, three parcels, sub-opcode 00110011 (6 bytes)



FRINTP, opcode 010000, two parcels, sub-opcode 00110011 (4 bytes)



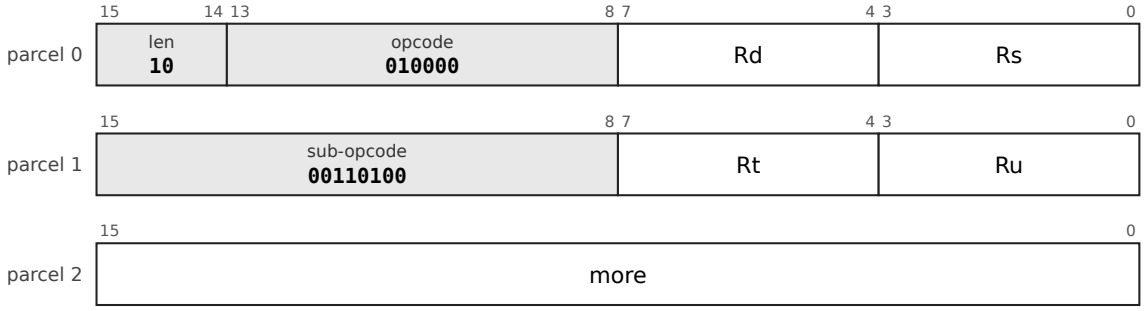

```
FRINTP Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FRINTP	010000 (0x10)	00110011 (0x33)

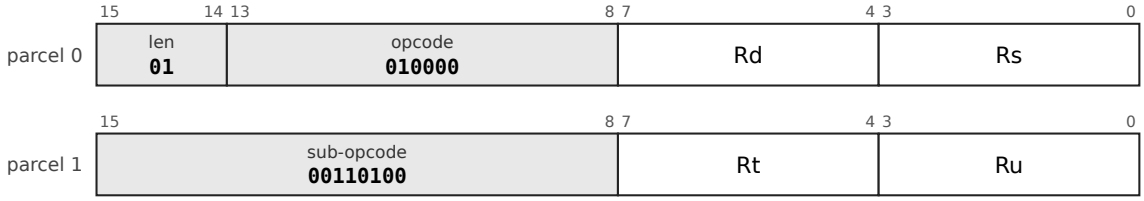
Operation, flags and exceptions as for the entry FRINT above; 2 layout forms, each a chart of its own (above).

FRINTA

FRINTA, opcode 010000, three parcels, sub-opcode 00110100 (6 bytes)



FRINTA, opcode 010000, two parcels, sub-opcode 00110100 (4 bytes)




```
FRINTA Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FRINTA	010000 (0x10)	00110100 (0x34)

Operation, flags and exceptions as for the entry FRINT above; 2 layout forms, each a chart of its own (above).

FRINTD
An alias of FRINT: the assembler writes it as that word.

FRINTDZ
An alias of FRINTZ: the assembler writes it as that word.

FRINTDM
An alias of FRINTM: the assembler writes it as that word.

FRINTDP
An alias of FRINTP: the assembler writes it as that word.

FRINTDA
An alias of FRINTA: the assembler writes it as that word.

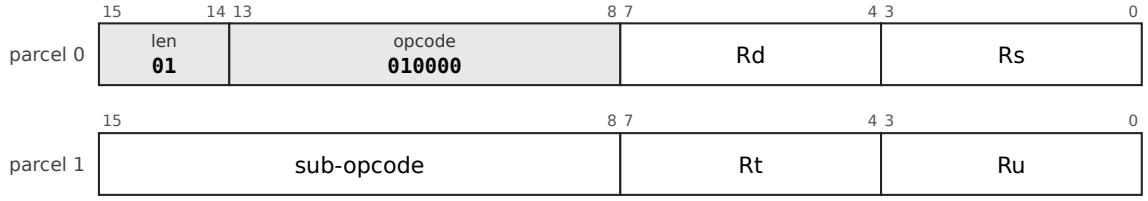
FCVT: Floating conversions

Mnemonics: FCVTDQ, FCVTDQU, FCVTQD, FCVTQUD, FCVTDS, FCVTSD, FCVTSL, FCVTLS

Syntax

```
FCVTDQ Rs, Rd        ; binary64 to signed quad
FCVTDQU Rs, Rd       ; binary64 to unsigned quad
FCVTQD Rs, Rd        ; signed quad to binary64
FCVTQUD Rs, Rd       ; unsigned quad to binary64
FCVTDS Rs, Rd        ; binary64 to binary32
FCVTSD Rs, Rd        ; binary32 to binary64
FCVTSL Rs, Rd        ; binary32 to signed 32-bit integer
FCVTLS Rs, Rd        ; signed 32-bit integer to binary32
```

Encoding



The chart is the layout r20-fesc-1, Floating point (FESC), two parcels (len = 01): From len = 01. In this group low is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FCVTDQ	010000 (0x10)	00111000 (0x38)
FCVTDQU	010000 (0x10)	00111110 (0x3E)
FCVTQD	010000 (0x10)	00111001 (0x39)
FCVTQUD	010000 (0x10)	00111111 (0x3F)
FCVTDS	010000 (0x10)	00111010 (0x3A)
FCVTSD	010000 (0x10)	00111011 (0x3B)
FCVTSL	010000 (0x10)	00111100 (0x3C)
FCVTLS	010000 (0x10)	00111101 (0x3D)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-fesc-1, r20-fesc-2.

Operation

```
float to integer: truncate toward zero, then saturate; a NaN gives the largest value
integer to float: round to nearest even
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps. A NaN or an out-of-range value raises NV and saturates.

Exceptions None.

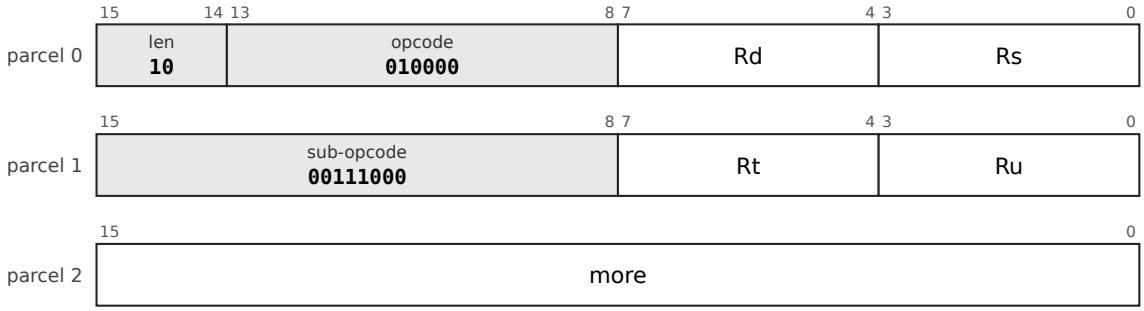
Integer results are 64-bit (FCVTDQ, FCVTDQU) or 32-bit sign-extended (FCVTSL). Unsigned conversion of a negative value is invalid and gives 0.

Why it is here. FCVTDQU and FCVTQUD convert unsigned 64-bit integers to and from binary64. C needs both, and without them each takes a branch and several instructions.

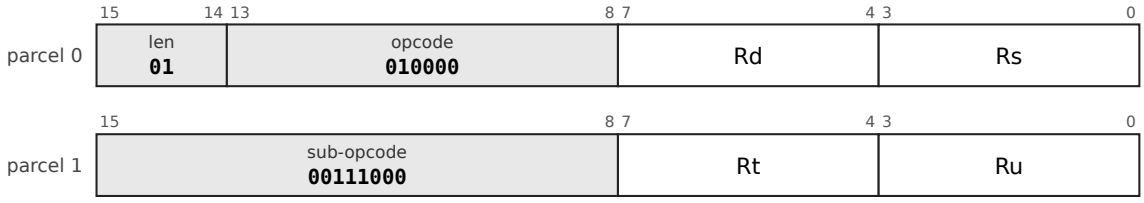
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

FCVTDQ

FCVTDQ, opcode 010000, three parcels, sub-opcode 00111000 (6 bytes)



FCVTDQ, opcode 010000, two parcels, sub-opcode 00111000 (4 bytes)



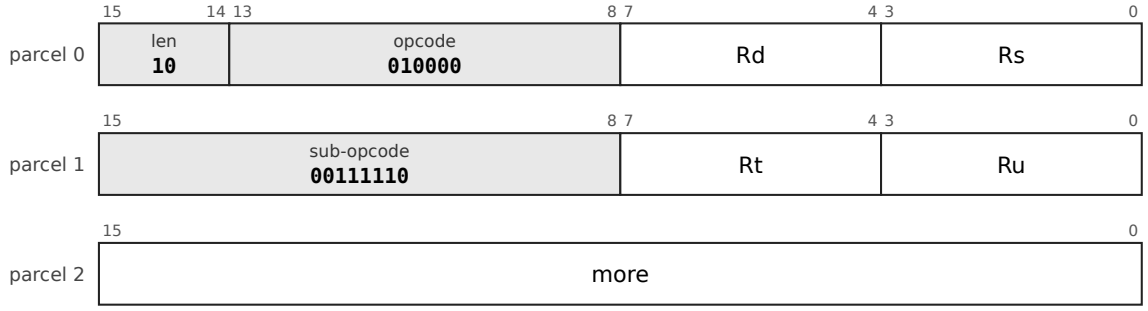

```
FCVTDQ Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTDQ	010000 (0x10)	00111000 (0x38)

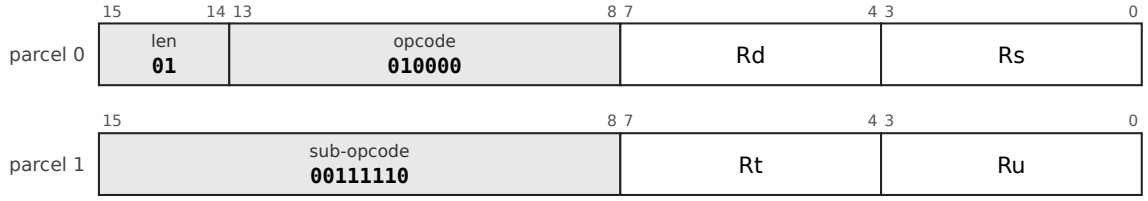
Operation, flags and exceptions as for the entry FCVT above; 2 layout forms, each a chart of its own (above).

FCVTDQU

FCVTDQU, opcode 010000, three parcels, sub-opcode 00111110 (6 bytes)



FCVTDQU, opcode 010000, two parcels, sub-opcode 00111110 (4 bytes)



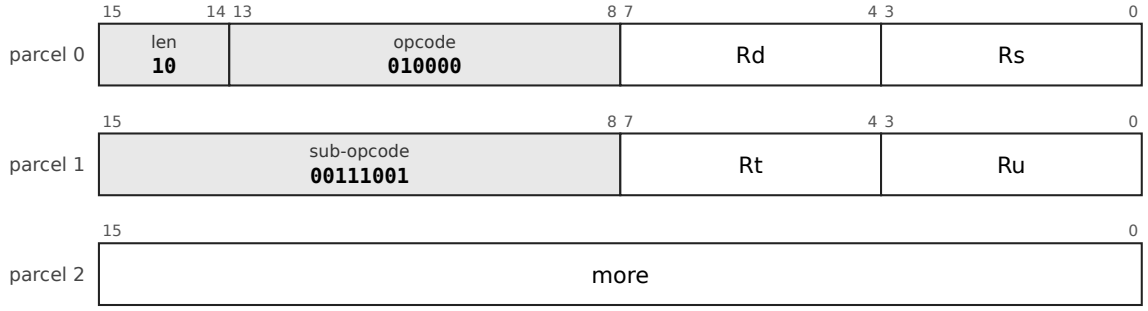

```
FCVTDQU Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTDQU	010000 (0x10)	00111110 (0x3E)

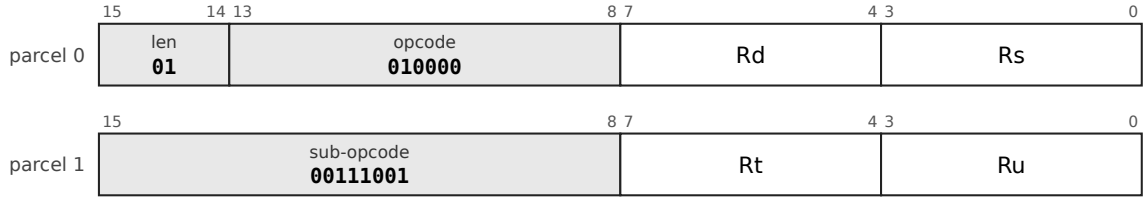
Operation, flags and exceptions as for the entry FCVT above; 2 layout forms, each a chart of its own (above).

FCVTQD

FCVTQD, opcode 010000, three parcels, sub-opcode 00111001 (6 bytes)



FCVTQD, opcode 010000, two parcels, sub-opcode 00111001 (4 bytes)



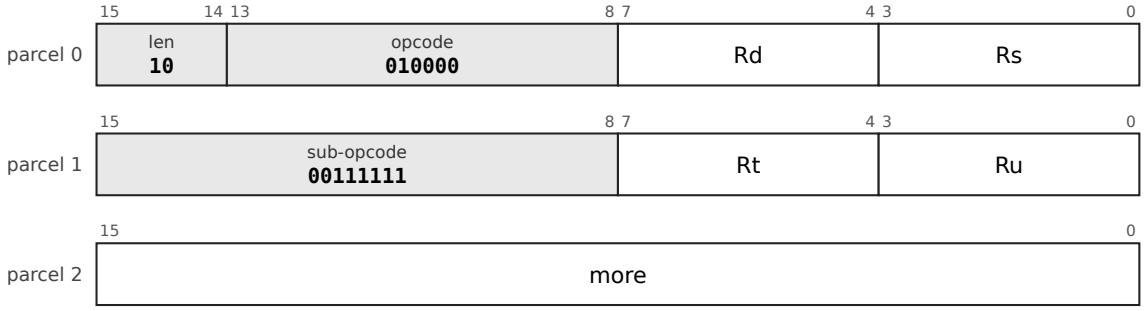

```
FCVTQD Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTQD	010000 (0x10)	00111001 (0x39)

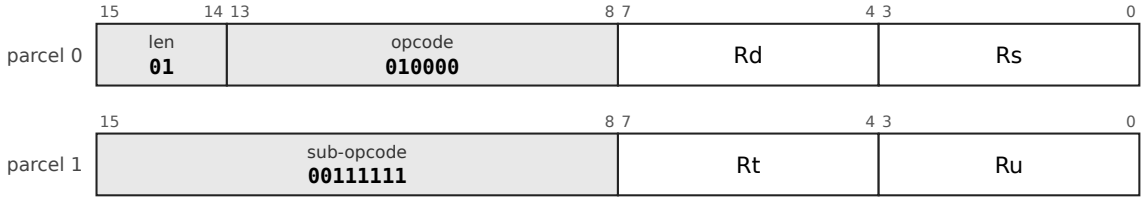
Operation, flags and exceptions as for the entry FCVT above; 2 layout forms, each a chart of its own (above).

FCVTQUD

FCVTQUD, opcode 010000, three parcels, sub-opcode 00111111 (6 bytes)



FCVTQUD, opcode 010000, two parcels, sub-opcode 00111111 (4 bytes)



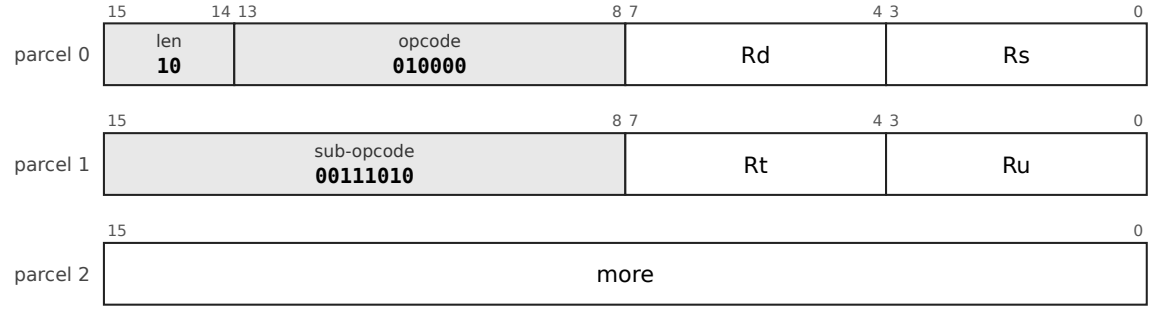

```
FCVTQUD Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTQUD	010000 (0x10)	00111111 (0x3F)

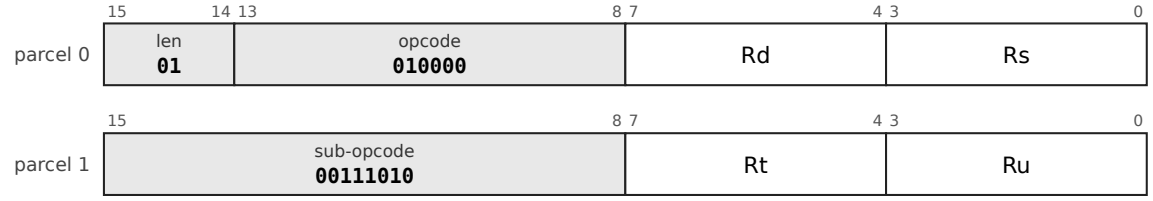
Operation, flags and exceptions as for the entry FCVT above; 2 layout forms, each a chart of its own (above).

FCVTDS

FCVTDS, opcode 010000, three parcels, sub-opcode 00111010 (6 bytes)



FCVTDS, opcode 010000, two parcels, sub-opcode 00111010 (4 bytes)



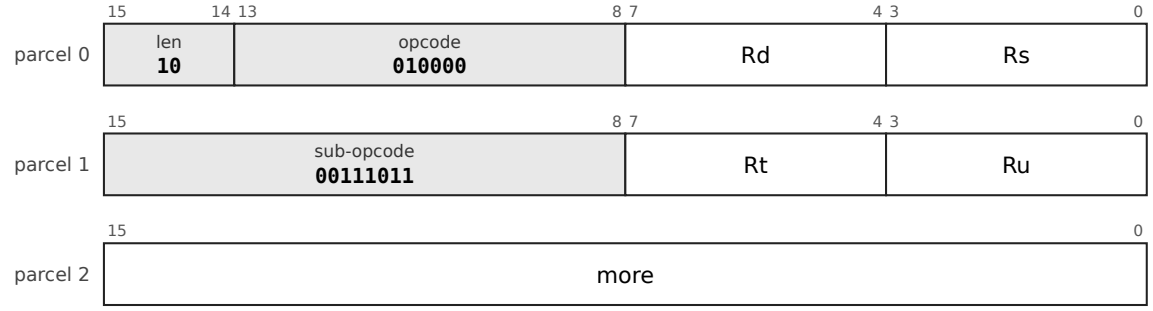

```
FCVTDS Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTDS	010000 (0x10)	00111010 (0x3A)

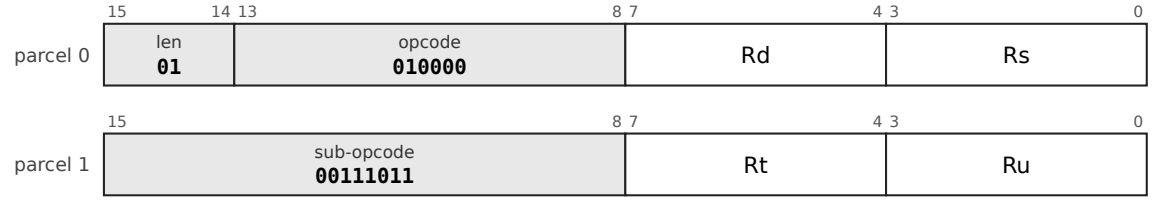
Operation, flags and exceptions as for the entry FCVT above; 2 layout forms, each a chart of its own (above).

FCVTSD

FCVTSD, opcode 010000, three parcels, sub-opcode 00111011 (6 bytes)



FCVTSD, opcode 010000, two parcels, sub-opcode 00111011 (4 bytes)



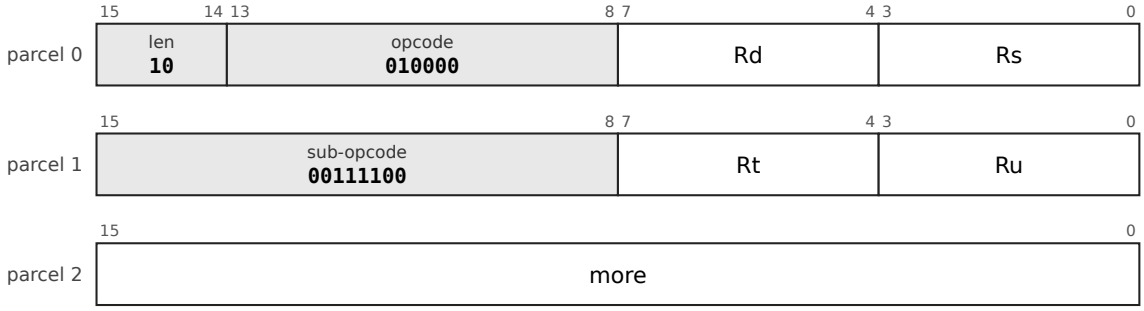

```
FCVTSD Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTSD	010000 (0x10)	00111011 (0x3B)

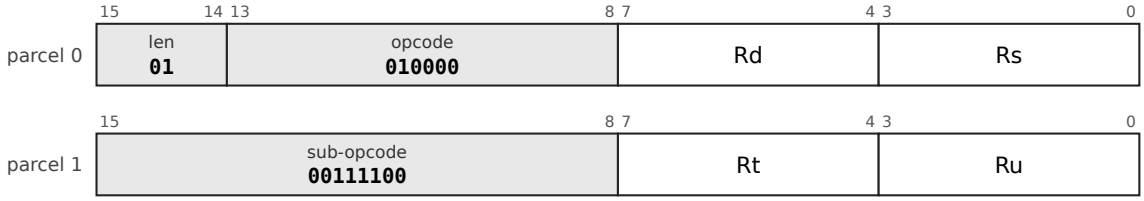
Operation, flags and exceptions as for the entry FCVT above; 2 layout forms, each a chart of its own (above).

FCVTSL

FCVTSL, opcode 010000, three parcels, sub-opcode 00111100 (6 bytes)



FCVTSL, opcode 010000, two parcels, sub-opcode 00111100 (4 bytes)



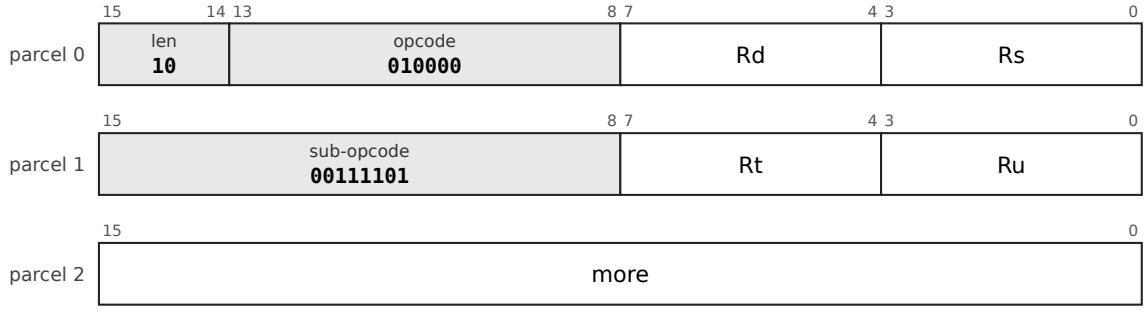

```
FCVTSL Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTSL	010000 (0x10)	00111100 (0x3C)

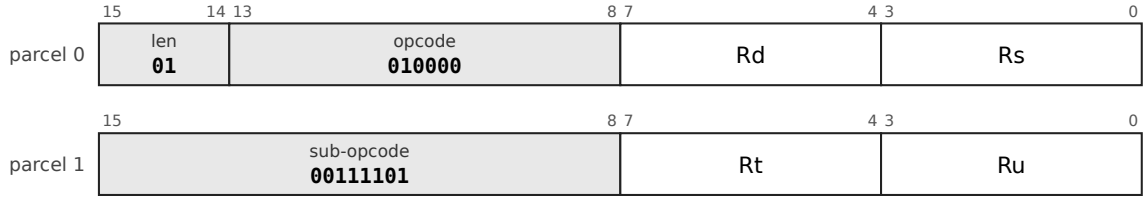
Operation, flags and exceptions as for the entry FCVT above; 2 layout forms, each a chart of its own (above).

FCVTLS

FCVTLS, opcode 010000, three parcels, sub-opcode 00111101 (6 bytes)



FCVTLS, opcode 010000, two parcels, sub-opcode 00111101 (4 bytes)



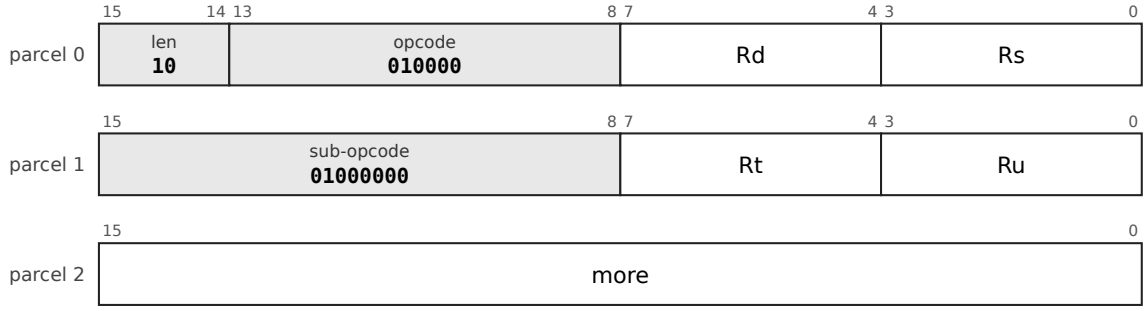

```
FCVTLS Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTLS	010000 (0x10)	00111101 (0x3D)

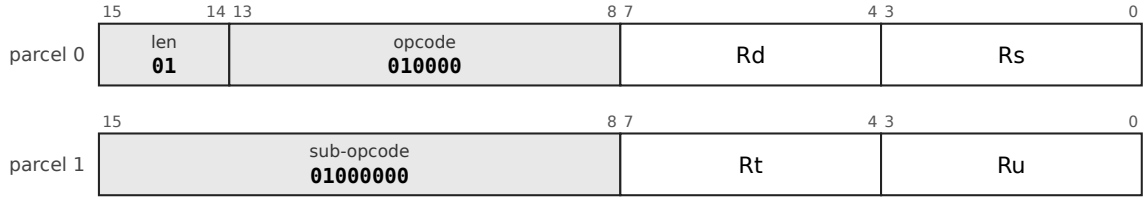
Operation, flags and exceptions as for the entry FCVT above; 2 layout forms, each a chart of its own (above).

FSFLAGS: Swap the floating-point exception flags

FSFLAGS, opcode 010000, three parcels, sub-opcode 01000000 (6 bytes)



FSFLAGS, opcode 010000, two parcels, sub-opcode 01000000 (4 bytes)

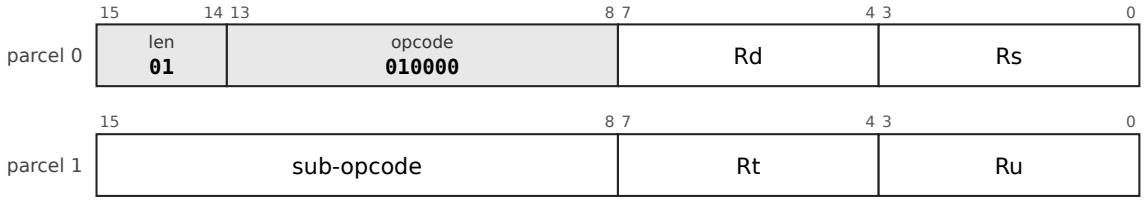


Mnemonics: FSFLAGS

Syntax

```
FSFLAGS Rs, Rd
```

Encoding



The chart is the layout `r20-fesc-1`, Floating point (FESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FSFLAGS	010000 (0x10)	01000000 (0x40)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-fesc-1`, `r20-fesc-2`.

Operation

```
old ← FPSR
FPSR ← Rs[4:0]
Rd ← old
```

Flags None.

Exceptions None.

`FPSR` is the sticky five-bit register (`NX` bit 0, `UF` 1, `OF` 2, `DZ` 3, `NV` 4). C99 `<fenv.h>` is built on this one instruction.

Why it is here. IEEE 754 requires five status flags, and C99 `<fenv.h>` needs software access to them. The architecture has a sticky five-bit `FPSR`, laid out as RISC-V's `fflags`, written by every operation that can raise a flag and never trapping, and one swap instruction to read and write it. It is a record of what happened, not a mode, so the rule against mode bits is untouched.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

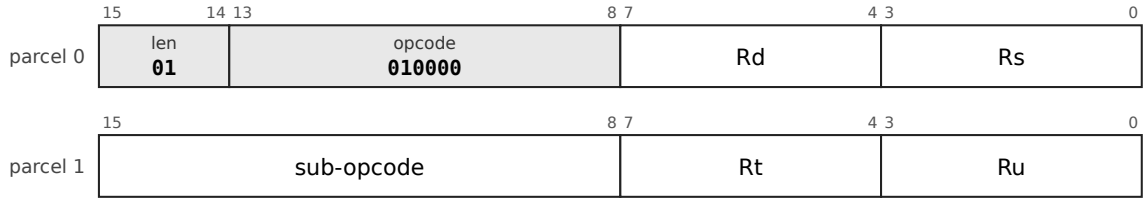
FHALF: Floating arithmetic, binary16

Mnemonics: `FADD16`, `FSUB16`, `FMUL16`, `FDIV16`, `FSQRT16`, `FCMP16`, `FMADD16`, `FMSUB16`, `FNMADD16`, `FNMSUB16`

Syntax

```
FADD16 Rs, Rd
FSUB16 Rs, Rd
FMUL16 Rs, Rd
FDIV16 Rs, Rd
FSQRT16 Rs, Rd
FCMP16 Rd, Rs
FMADD16 Rs, Rt, Ru, Rd ...
```

Encoding



The chart is the layout `r20-fesc-1`, Floating point (FESC), two parcels (`len = 01`): From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FADD16	010000 (0x10)	01010000 (0x50)
FSUB16	010000 (0x10)	01010001 (0x51)
FMUL16	010000 (0x10)	01010010 (0x52)
FDIV16	010000 (0x10)	01010011 (0x53)
FSQRT16	010000 (0x10)	01010100 (0x54)
FCMP16	010000 (0x10)	01010101 (0x55)
FMADD16	010000 (0x10)	01011000 (0x58)
FMSUB16	010000 (0x10)	01011001 (0x59)
FNMADD16	010000 (0x10)	01011010 (0x5A)
FNMSUB16	010000 (0x10)	01011011 (0x5B)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-fesc-1`, `r20-fesc-2`.

Operation

```
as the binary32 words, in binary16: Rd ← Rd op Rs ; the fused forms take Rs, Rt, Ru, Rd
a half lives in the low 16 bits of a register, sign-extended to 64
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps. `FCMP16` writes `N Z V C` as `FCMP`.

Exceptions None.

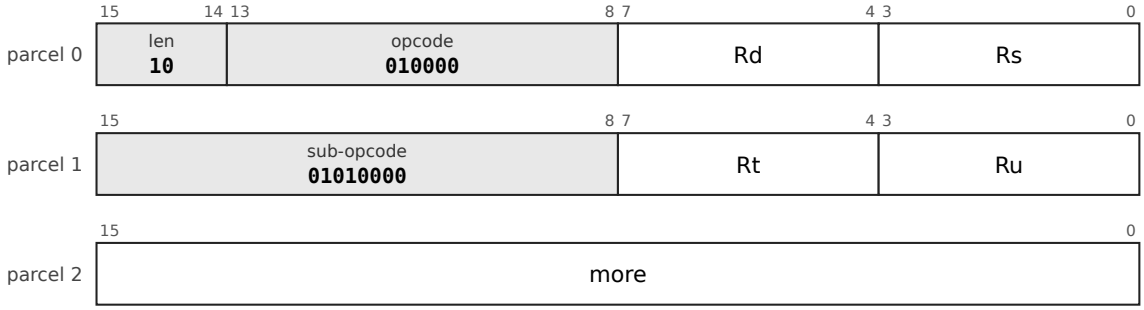
Results are correctly rounded to nearest even, tininess judged after rounding; a NaN result is the canonical `0x7E00`. Half `FMIN`, `FMAX`, `FRINT*`, bfloat16 and conversions to and from 64-bit integers are open items.

Why it is here. Half precision began as a GPU format (NVIDIA's `half`, 2002), spread to machine learning and image formats, and is in CPUs (x86 `F16C` and AVX-512 FP16, ARMv8.2, RISC-V `Zfh` and `Zfhmin`); this account is from memory and not checked against a manual. Two levels are possible: conversions only (about 500 to 1,000 lookup tables, no new operation by the folding rule) and full arithmetic (about 2,000 to 3,500 tables; estimates), and the existing double datapath can compute half correctly rounded. The architecture has both conversions and arithmetic. The leftovers (half `FMIN`, `FRINT*`, bfloat16, 64-bit integer conversions) are open items. 16-bit *integer* arithmetic is not provided: the ALU has one width (9.2.9).

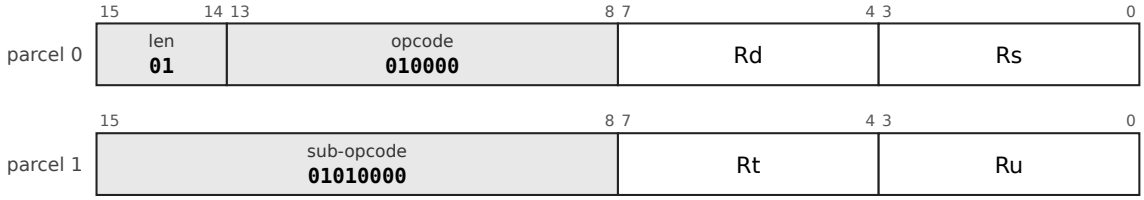
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

FADD16

FADD16, opcode 010000, three parcels, sub-opcode 01010000 (6 bytes)



FADD16, opcode 010000, two parcels, sub-opcode 01010000 (4 bytes)



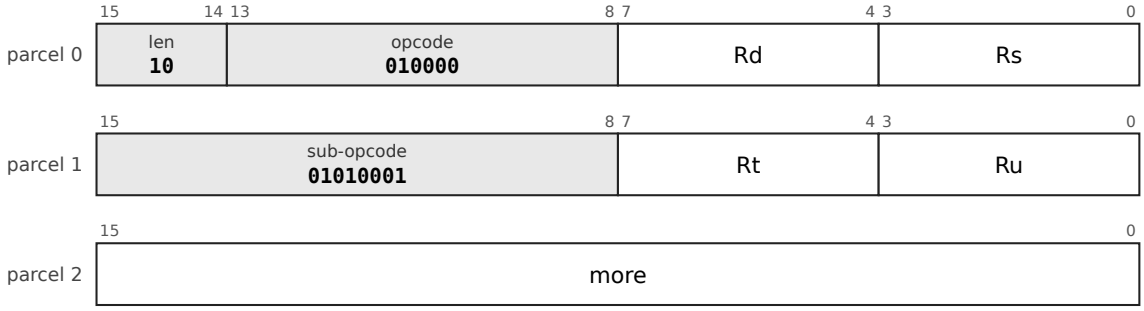

```
FADD16 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FADD16	010000 (0x10)	01010000 (0x50)

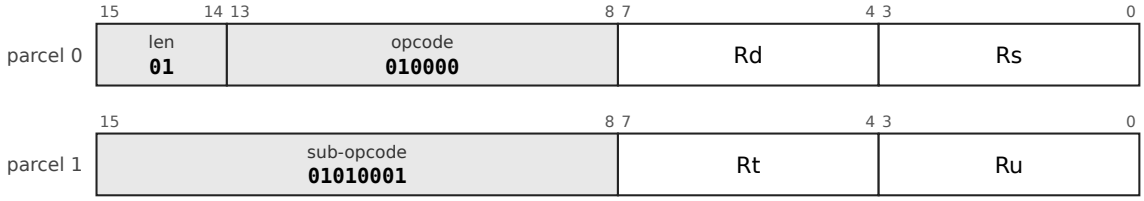
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FSUB16

FSUB16, opcode 010000, three parcels, sub-opcode 01010001 (6 bytes)



FSUB16, opcode 010000, two parcels, sub-opcode 01010001 (4 bytes)



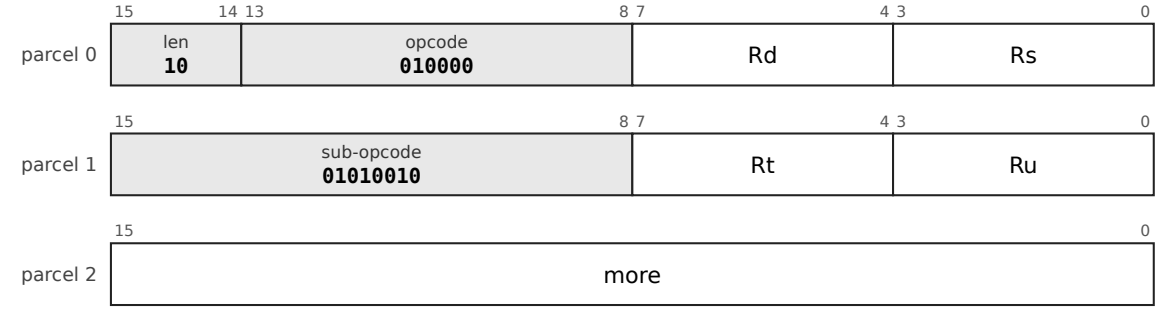

```
FSUB16 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FSUB16	010000 (0x10)	01010001 (0x51)

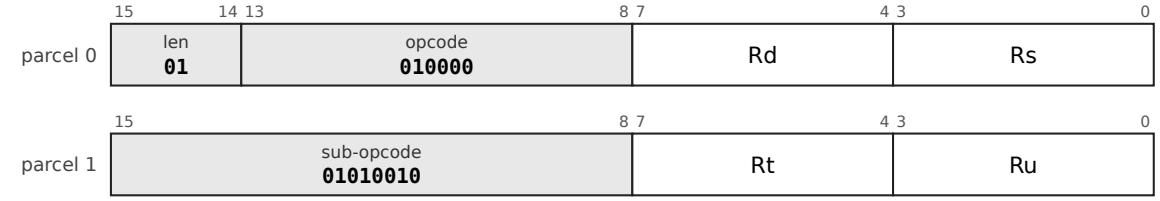
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FMUL16

FMUL16, opcode 010000, three parcels, sub-opcode 01010010 (6 bytes)



FMUL16, opcode 010000, two parcels, sub-opcode 01010010 (4 bytes)



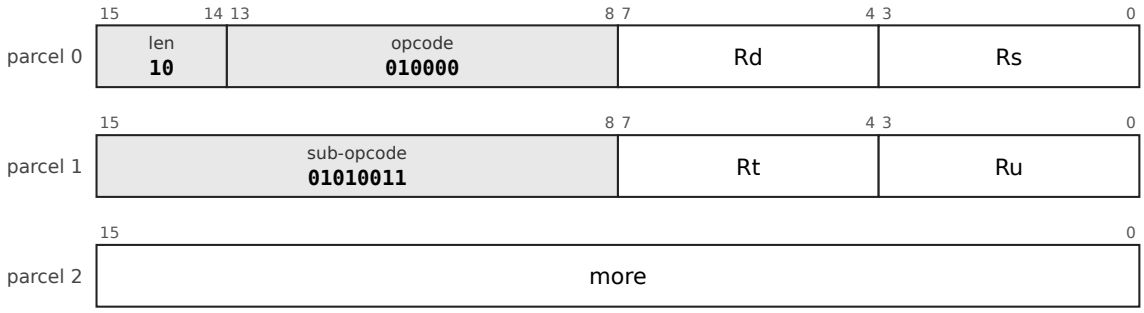

```
FMUL16 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FMUL16	010000 (0x10)	01010010 (0x52)

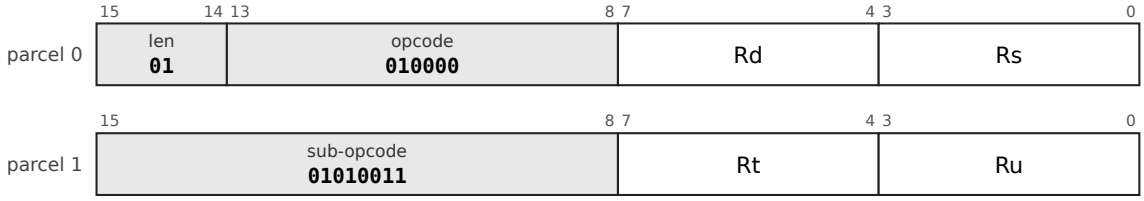
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FDIV16

FDIV16, opcode 010000, three parcels, sub-opcode 01010011 (6 bytes)



FDIV16, opcode 010000, two parcels, sub-opcode 01010011 (4 bytes)



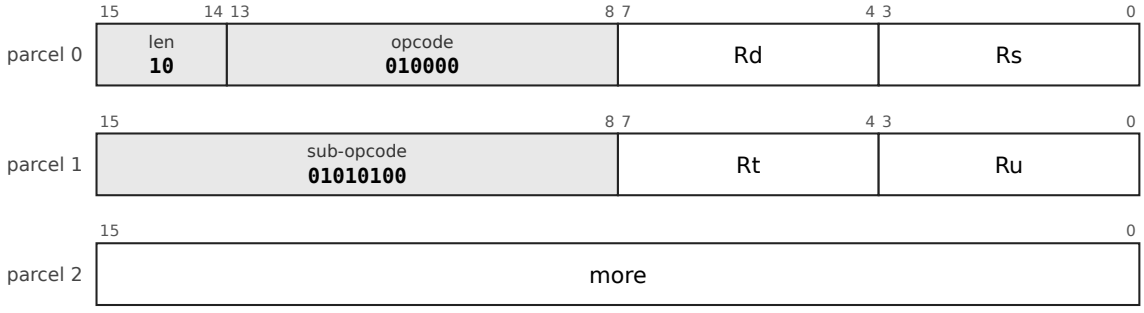

```
FDIV16 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FDIV16	010000 (0x10)	01010011 (0x53)

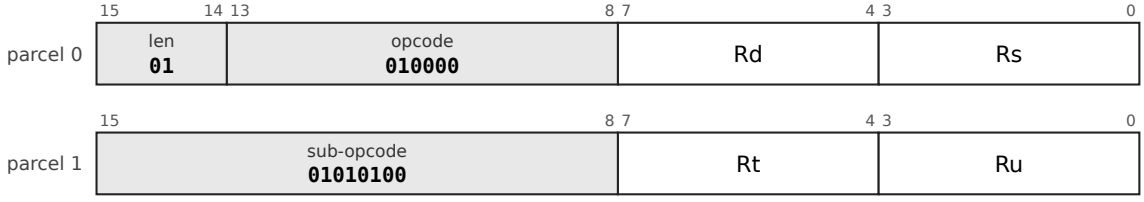
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FSQRT16

FSQRT16, opcode 010000, three parcels, sub-opcode 01010100 (6 bytes)



FSQRT16, opcode 010000, two parcels, sub-opcode 01010100 (4 bytes)



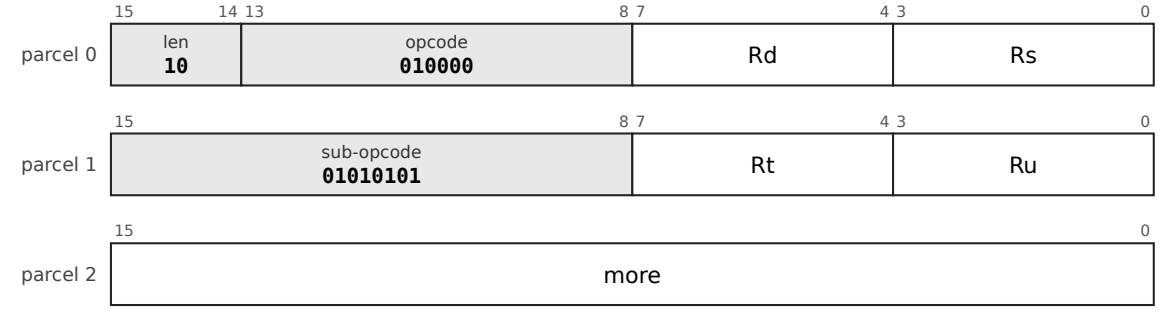

```
FSQRT16 Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FSQRT16	010000 (0x10)	01010100 (0x54)

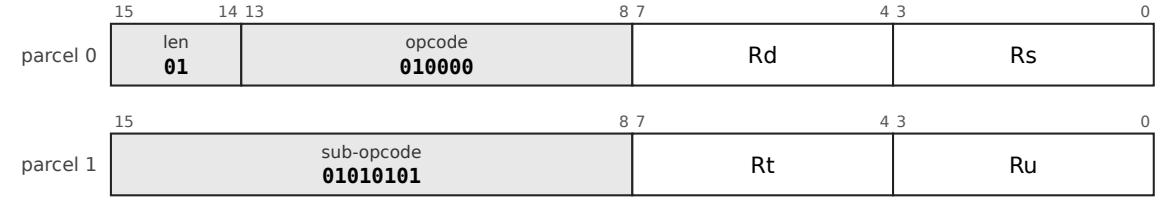
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FCMP16

FCMP16, opcode 010000, three parcels, sub-opcode 01010101 (6 bytes)



FCMP16, opcode 010000, two parcels, sub-opcode 01010101 (4 bytes)



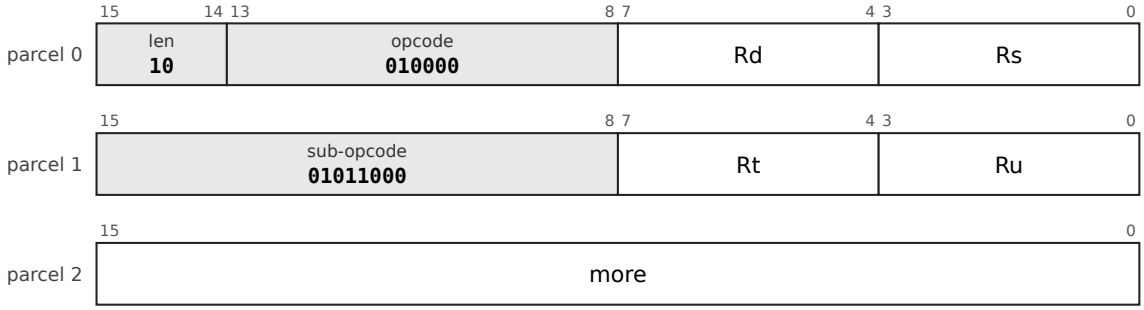

```
FCMP16 Rd, Rs
```

Mnemonic	Opcode	Sub-opcode
FCMP16	010000 (0x10)	01010101 (0x55)

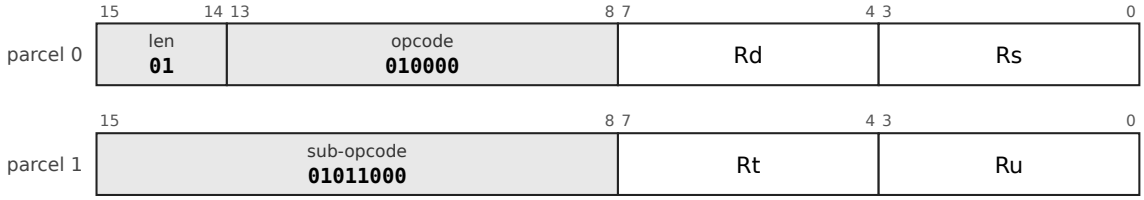
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FMADD16

FMADD16, opcode 010000, three parcels, sub-opcode 01011000 (6 bytes)



FMADD16, opcode 010000, two parcels, sub-opcode 01011000 (4 bytes)



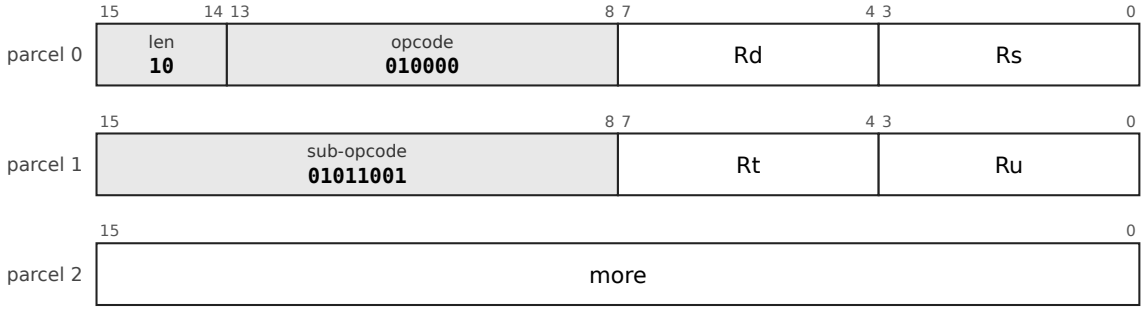

```
FMADD16 Rs, Rt, Ru, Rd ...
```

Mnemonic	Opcode	Sub-opcode
FMADD16	010000 (0x10)	01011000 (0x58)

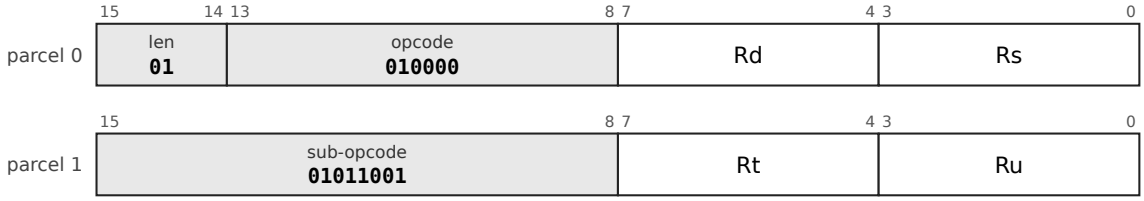
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FMSUB16

FMSUB16, opcode 010000, three parcels, sub-opcode 01011001 (6 bytes)



FMSUB16, opcode 010000, two parcels, sub-opcode 01011001 (4 bytes)

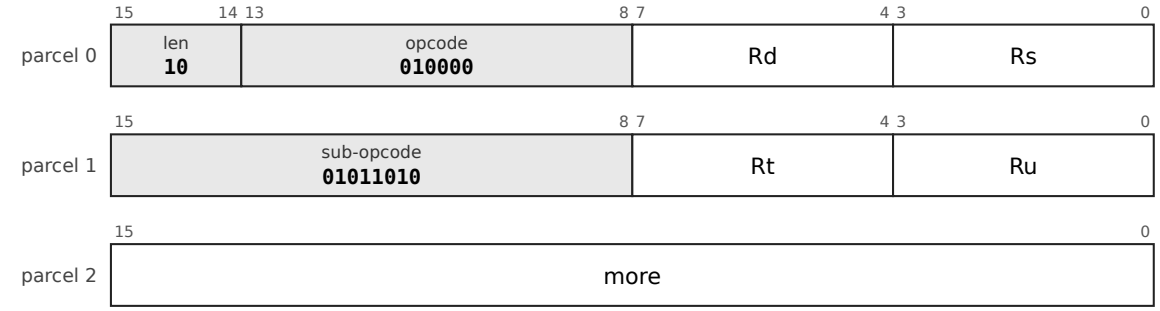


Mnemonic	Opcode	Sub-opcode
FMSUB16	010000 (0x10)	01011001 (0x59)

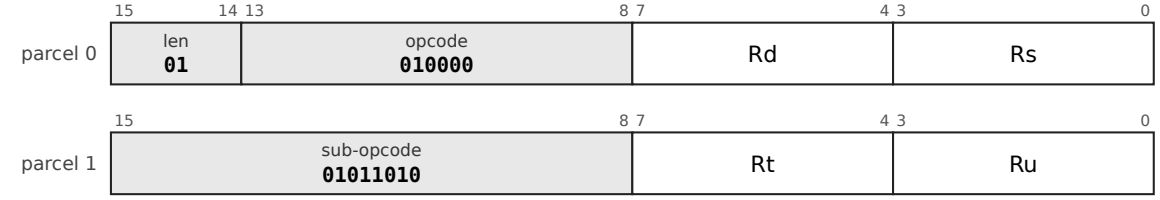
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FNMADD16

FNMADD16, opcode 010000, three parcels, sub-opcode 01011010 (6 bytes)



FNMADD16, opcode 010000, two parcels, sub-opcode 01011010 (4 bytes)

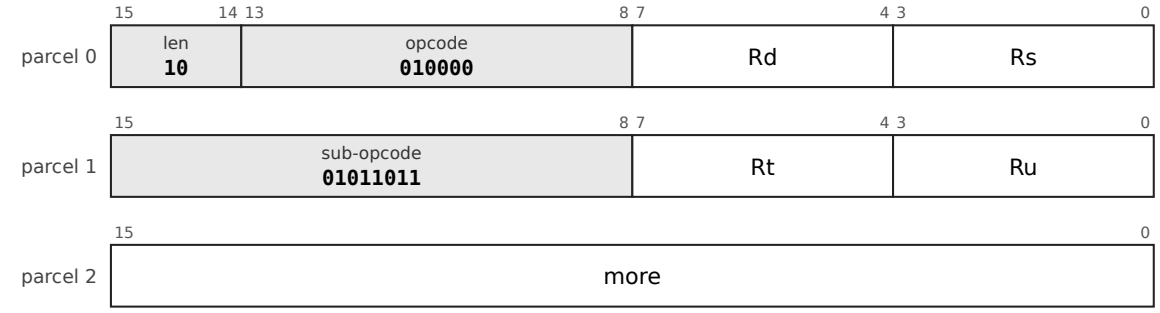


Mnemonic	Opcode	Sub-opcode
FNMADD16	010000 (0x10)	01011010 (0x5A)

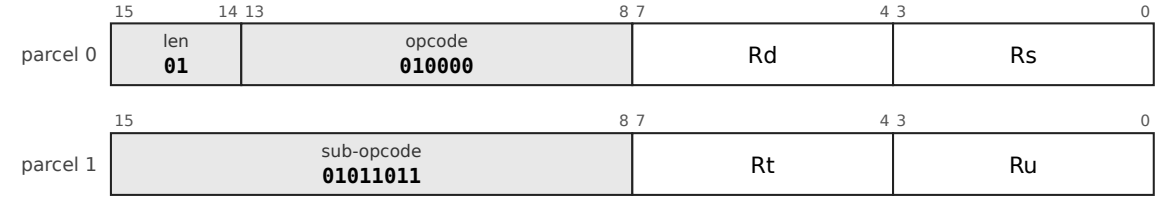
Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

FNMSUB16

FNMSUB16, opcode 010000, three parcels, sub-opcode 01011011 (6 bytes)



FNMSUB16, opcode 010000, two parcels, sub-opcode 01011011 (4 bytes)



Mnemonic	Opcode	Sub-opcode
FNMSUB16	010000 (0x10)	01011011 (0x5B)

Operation, flags and exceptions as for the entry FHALF above; 2 layout forms, each a chart of its own (above).

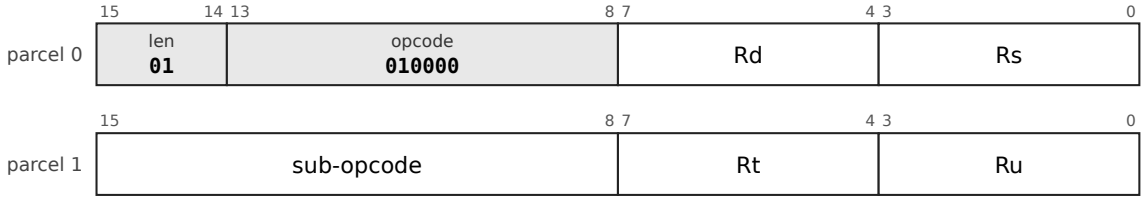
FCVTH: Conversions to and from binary16

Mnemonics: FCVTSH, FCVTHS, FCVTDH, FCVTHD, FCVTHL, FCVTLH

Syntax

```
FCVTSH Rs, Rd        ; binary32 to half
FCVTHS Rs, Rd        ; half to binary32
FCVTDH Rs, Rd        ; binary64 to half
FCVTHD Rs, Rd        ; half to binary64
FCVTHL Rs, Rd        ; half to signed 32-bit integer, toward zero
FCVTLH Rs, Rd        ; signed 32-bit integer to half
```

Encoding



The chart is the layout r20-fesc-1, Floating point (FESC), two parcels (len = 01): From len = 01. In this group low is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FCVTSH	010000 (0x10)	01100000 (0x60)
FCVTHS	010000 (0x10)	01100001 (0x61)
FCVTDH	010000 (0x10)	01100010 (0x62)
FCVTHD	010000 (0x10)	01100011 (0x63)
FCVTHL	010000 (0x10)	01100100 (0x64)
FCVTLH	010000 (0x10)	01100101 (0x65)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-fesc-1, r20-fesc-2.

Operation

```
narrowing: round to nearest even ; widening: exact
FCVTHL of a NaN or an infinity is invalid and saturates; truncating a fraction is inexact
```

Flags No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps.

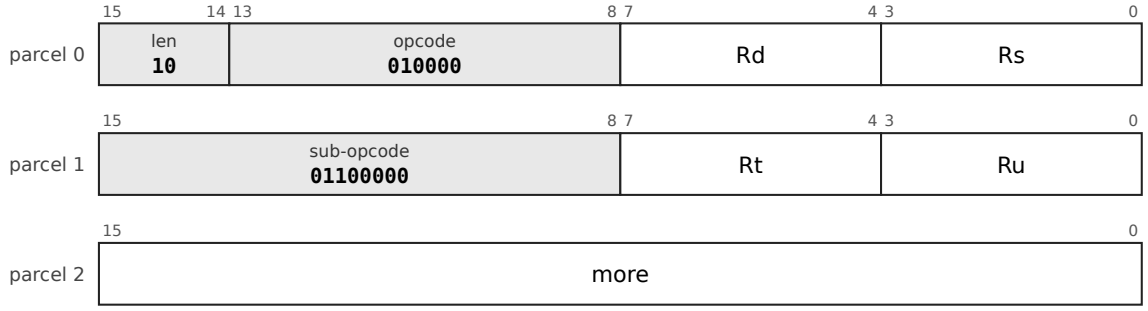
Exceptions None.

Why it is here. The conversion half of the binary16 support (see FHALF). The conversions are the part machine-learning code needs most, because a half is normally loaded and stored with MOVW, converted, computed in single or double, and converted back.

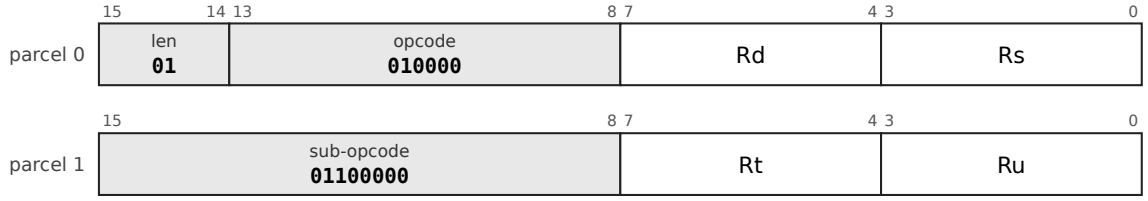
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

FCVTSH

FCVTSH, opcode 010000, three parcels, sub-opcode 01100000 (6 bytes)



FCVTSH, opcode 010000, two parcels, sub-opcode 01100000 (4 bytes)



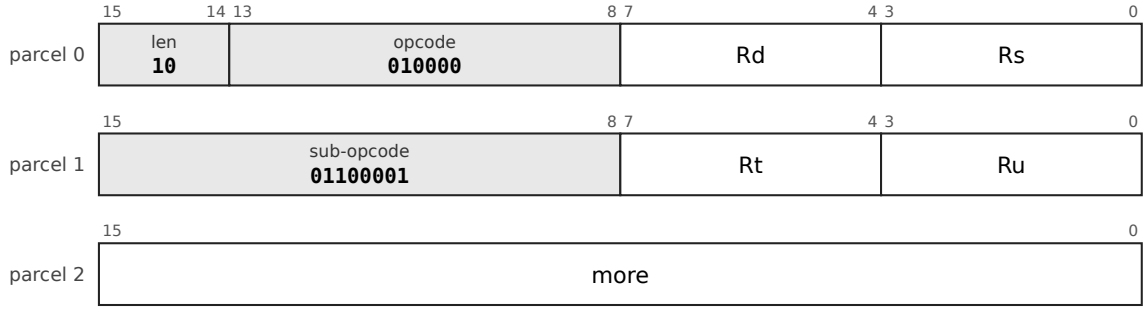

```
FCVTSH Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTSH	010000 (0x10)	01100000 (0x60)

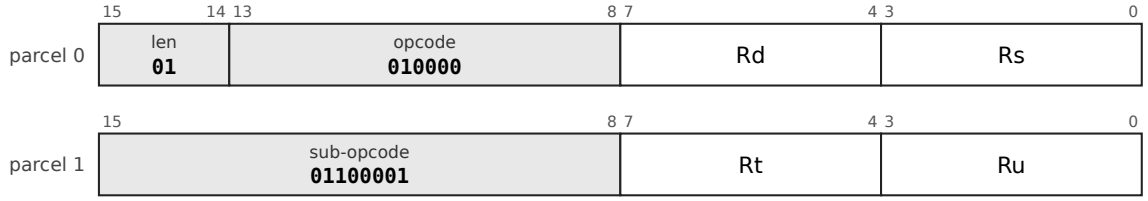
Operation, flags and exceptions as for the entry FCVTH above; 2 layout forms, each a chart of its own (above).

FCVTHS

FCVTHS, opcode 010000, three parcels, sub-opcode 01100001 (6 bytes)



FCVTHS, opcode 010000, two parcels, sub-opcode 01100001 (4 bytes)



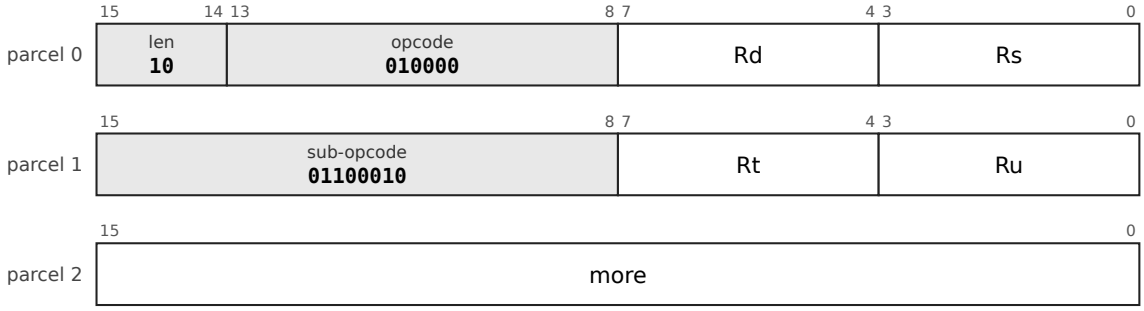

```
FCVTHS Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTHS	010000 (0x10)	01100001 (0x61)

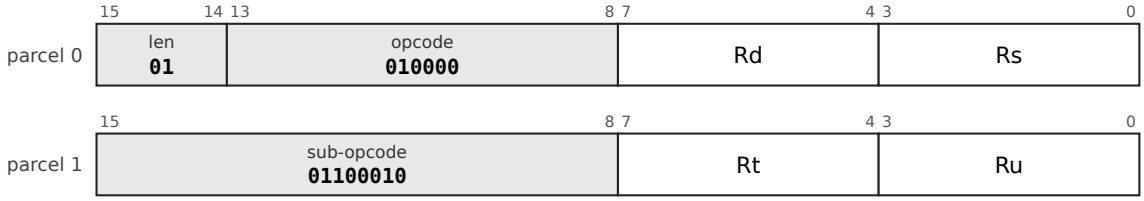
Operation, flags and exceptions as for the entry FCVTH above; 2 layout forms, each a chart of its own (above).

FCVTDH

FCVTDH, opcode 010000, three parcels, sub-opcode 01100010 (6 bytes)



FCVTDH, opcode 010000, two parcels, sub-opcode 01100010 (4 bytes)



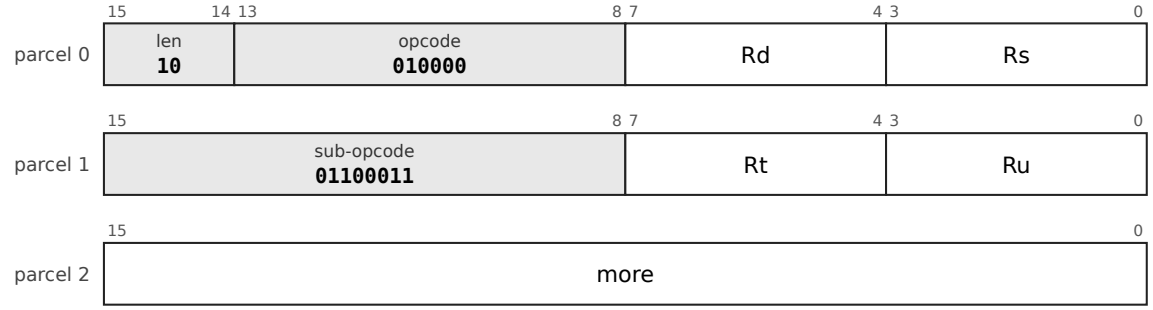

```
FCVTDH Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTDH	010000 (0x10)	01100010 (0x62)

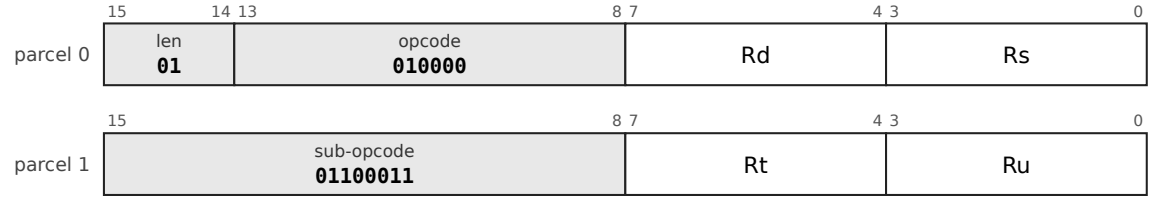
Operation, flags and exceptions as for the entry FCVTH above; 2 layout forms, each a chart of its own (above).

FCVTHD

FCVTHD, opcode 010000, three parcels, sub-opcode 01100011 (6 bytes)



FCVTHD, opcode 010000, two parcels, sub-opcode 01100011 (4 bytes)



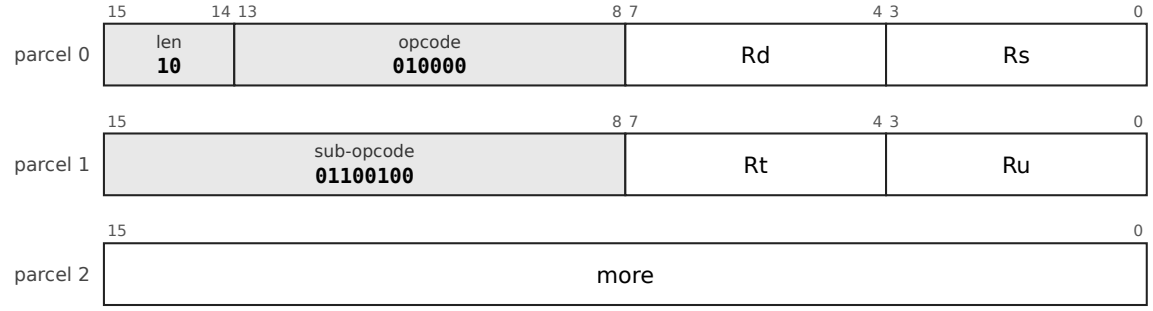

```
FCVTHD Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTHD	010000 (0x10)	01100011 (0x63)

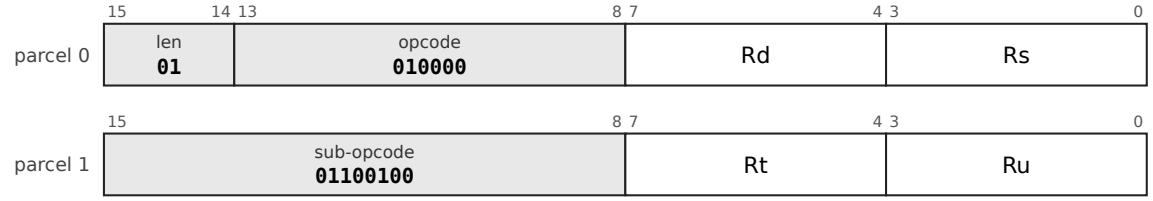
Operation, flags and exceptions as for the entry FCVTH above; 2 layout forms, each a chart of its own (above).

FCVTHL

FCVTHL, opcode 010000, three parcels, sub-opcode 01100100 (6 bytes)



FCVTHL, opcode 010000, two parcels, sub-opcode 01100100 (4 bytes)



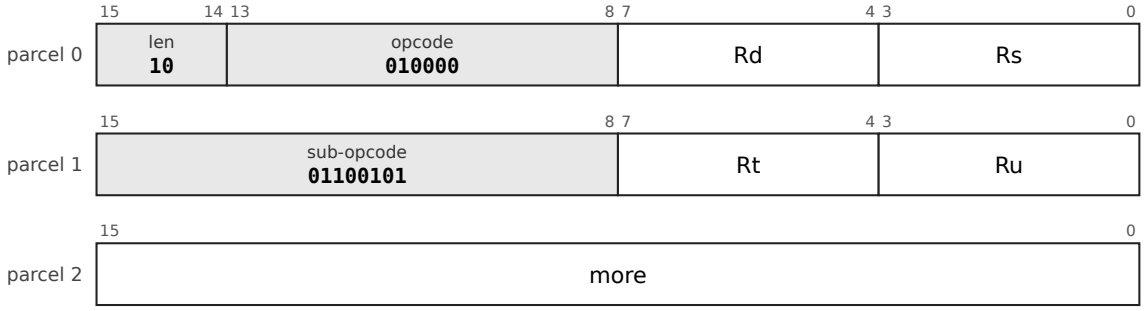

```
FCVTHL Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTHL	010000 (0x10)	01100100 (0x64)

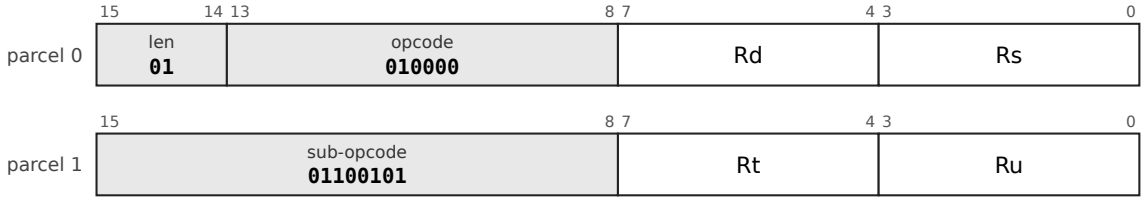
Operation, flags and exceptions as for the entry FCVTH above; 2 layout forms, each a chart of its own (above).

FCVTLH

FCVTLH, opcode 010000, three parcels, sub-opcode 01100101 (6 bytes)



FCVTLH, opcode 010000, two parcels, sub-opcode 01100101 (4 bytes)




```
FCVTLH Rs, Rd
```

Mnemonic	Opcode	Sub-opcode
FCVTLH	010000 (0x10)	01100101 (0x65)

Operation, flags and exceptions as for the entry FCVTH above; 2 layout forms, each a chart of its own (above).

9.13 Atomics and ordering

Cf. chapter 7, section 7.12 (the atomic and ordering instructions), section 7.3 (the atomics in detail) and section 7.2 (the memory model).

9.13.1 The atomic escape ATOM (opcode 010001 (0x11)) holds CAS, SWAP, XADD, XAND, XOR_, XXOR, XMAX, XMIN, XMAXU and XMINU, each with a _DB variant that adds a barrier, and LL and SC; FENCE, WFE, SEV, ISYNC and HINT are in SYS. Their semantics, ordering rules and the reservation monitor are in chapter 7. *(opcode 010001 (0x11))*

9.13.2 CAS operates on 4 or 8 bytes; an 8-byte compare-and-swap may span two adjacent 4-byte fields, because the usual lock-and-count fast path of an operating system is exactly that (an 8-byte whole over two 4-byte members). *(D)* The encoding of the CAS size is unspecified. *(S)*

Commentary. The official entry of each instruction is below, in this section; chapter 7 gives the reasons, the memory model and the use. **Impact.** *Kernel:* the entries below are all a kernel needs to write its locks; chapter 7 explains how they compose. *Compiler:* a lock acquire is one _DB instruction. *Hardware:* a reservation monitor beside the data cache.

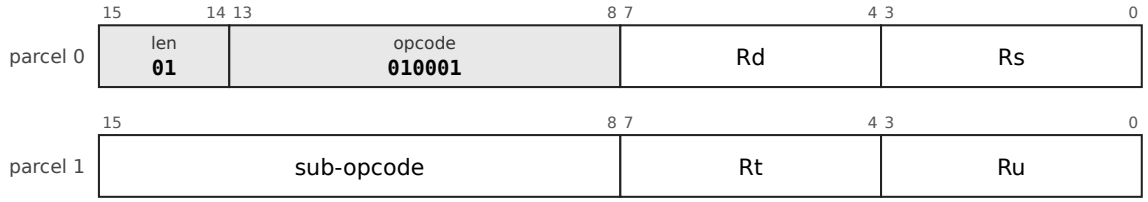
CAS: Compare and swap

Mnemonics: CAS, CASDB

Syntax

```
CAS expected, new, (address)       ; operand placement and size encoding unspecified; size is 4 or 8 bytes
CASDB ...                           ; with a full barrier
```

Encoding



The chart is the layout `r20-atom-1`, Atomics (ATOM), two parcels (`len = 01`): From `len = 01`. The placement of the operands is not specified.

Mnemonic	Opcode	Sub-opcode
CAS	010001 (0x11)	00000000 (0x00)
CASDB	010001 (0x11)	00000001 (0x01)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-atom-1`, `r20-atom-2`.

Operation

```
old ← M[address]                          ; 4 or 8 bytes, naturally aligned
if old = expected then M[address] ← new
result ← old          ; the program compares result with expected to learn whether the swap happened
```

Flags Not specified beyond “none for the read-modify-write forms”; `SC` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified; the placement of the operands of each form is unspecified, and the forms below name the roles.

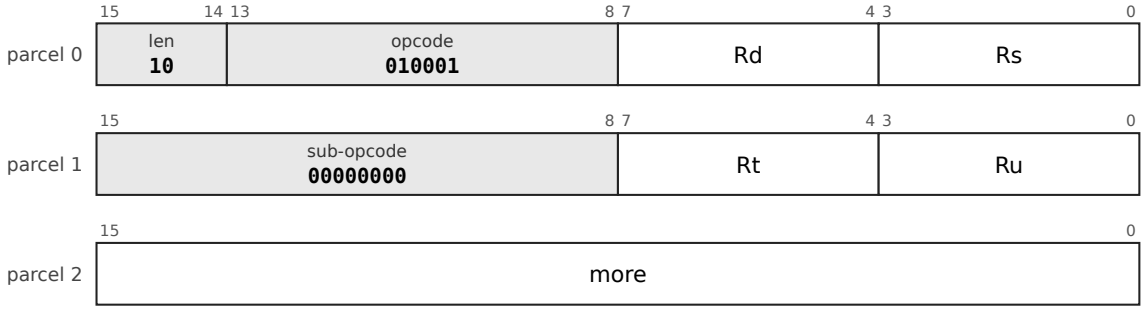
Why it is here. The barrier-fused variants are taken from LoongArch’s pragmatism: a lock acquire is one instruction rather than an atomic followed by a `FENCE`, and that pair is in every critical section of the system. With the other multiprocessor operations the atomics make the seventeen-operation SMP umbrella (7.1). They are defined on a reservation monitor and not on a bus lock (2.5).

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

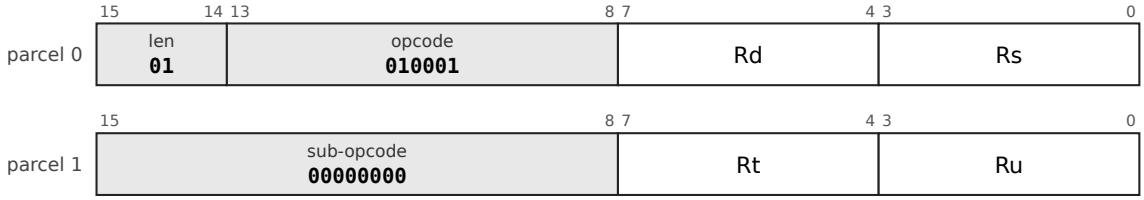
Status specified; **not yet implemented** in the reference executor.

CAS

CAS, opcode 010001, three parcels, sub-opcode 00000000 (6 bytes)



CAS, opcode 010001, two parcels, sub-opcode 00000000 (4 bytes)



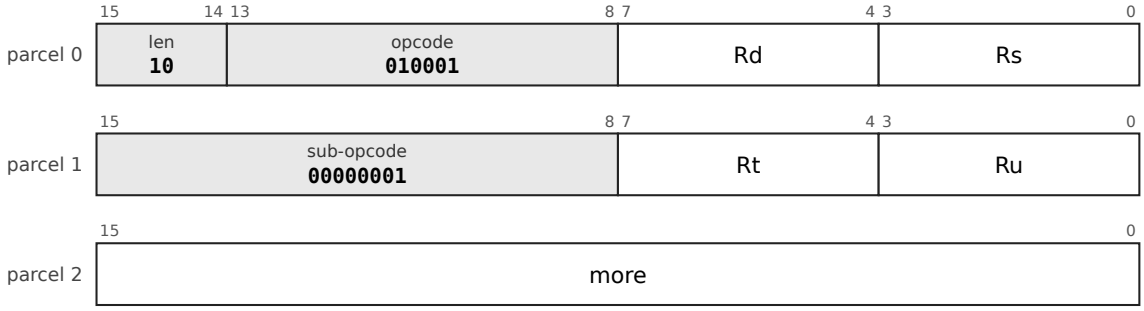

```
CAS expected, new, (address)
```

Mnemonic	Opcode	Sub-opcode
CAS	010001 (0x11)	00000000 (0x00)

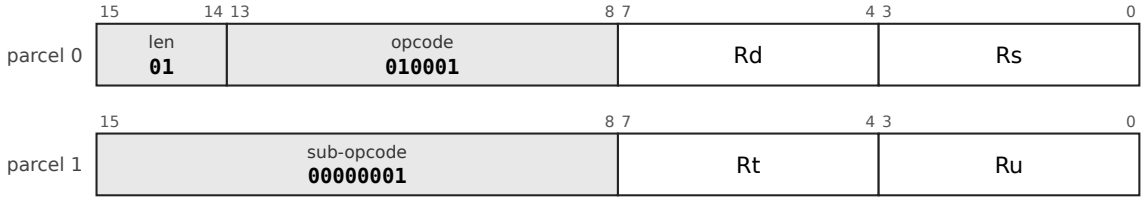
Operation, flags and exceptions as for the entry CAS above; 2 layout forms, each a chart of its own (above).

CASDB

CASDB, opcode 010001, three parcels, sub-opcode 00000001 (6 bytes)



CASDB, opcode 010001, two parcels, sub-opcode 00000001 (4 bytes)




```
CASDB ...
```

Mnemonic	Opcode	Sub-opcode
CASDB	010001 (0x11)	00000001 (0x01)

Operation, flags and exceptions as for the entry CAS above; 2 layout forms, each a chart of its own (above).

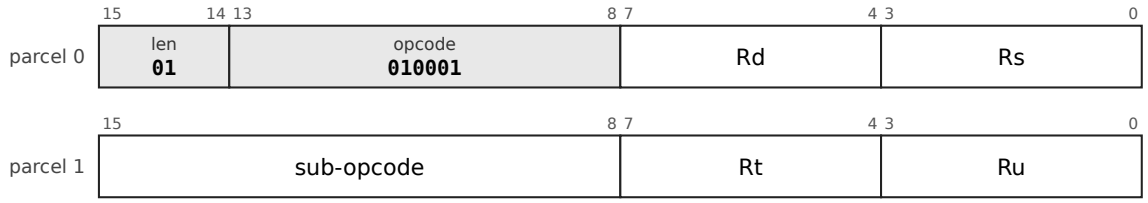
SWAP: Atomic exchange

Mnemonics: SWAP, SWAPDB

Syntax

```
SWAP value, (address)
SWAPDB ...
```

Encoding



The chart is the layout r20-atom-1, Atomics (ATOM), two parcels (len = 01): From len = 01. The placement of the operands is not specified.

Mnemonic	Opcode	Sub-opcode
SWAP	010001 (0x11)	00000010 (0x02)
SWAPDB	010001 (0x11)	00000011 (0x03)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-atom-1, r20-atom-2.

Operation

```
old ← M[address] ; M[address] ← value ; result ← old
```

Flags Not specified beyond "none for the read-modify-write forms"; SC reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified; the placement of the operands of each form is unspecified, and the forms below name the roles.

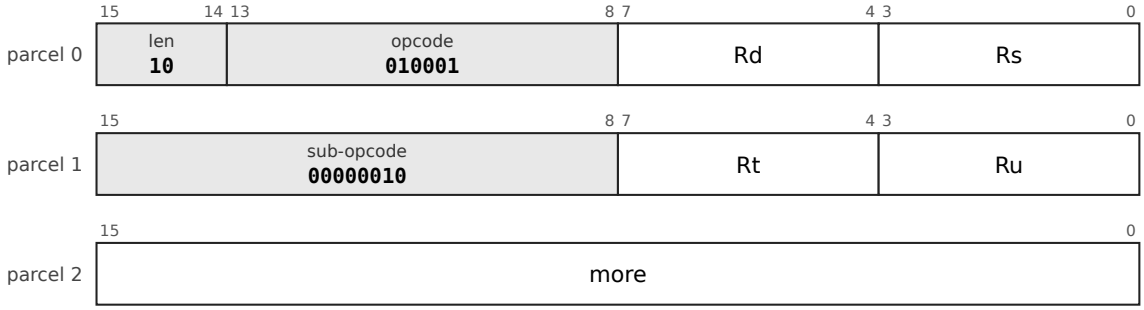
Why it is here. One of the read-modify-write atomics of the ATOM group (see CAS). No separate reason is recorded for it.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

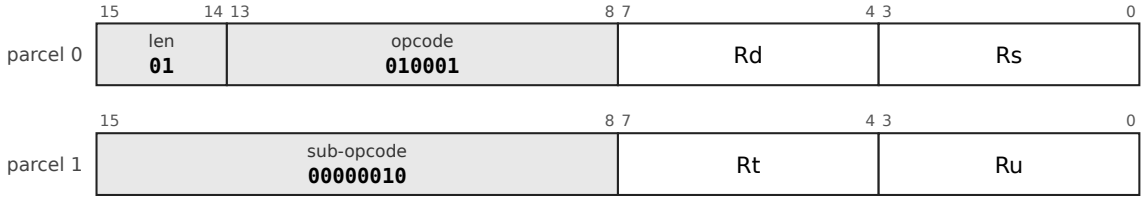
Status specified; **not yet implemented** in the reference executor.

SWAP

SWAP, opcode 010001, three parcels, sub-opcode 00000010 (6 bytes)



SWAP, opcode 010001, two parcels, sub-opcode 00000010 (4 bytes)



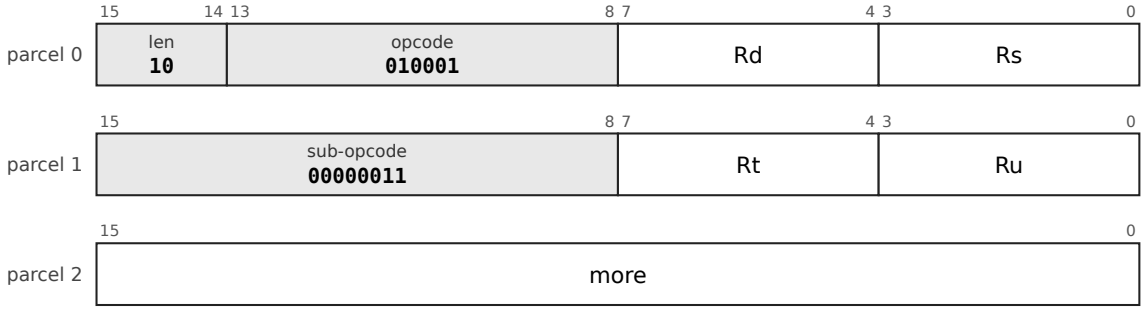

```
SWAP value, (address)
```

Mnemonic	Opcode	Sub-opcode
SWAP	010001 (0x11)	00000010 (0x02)

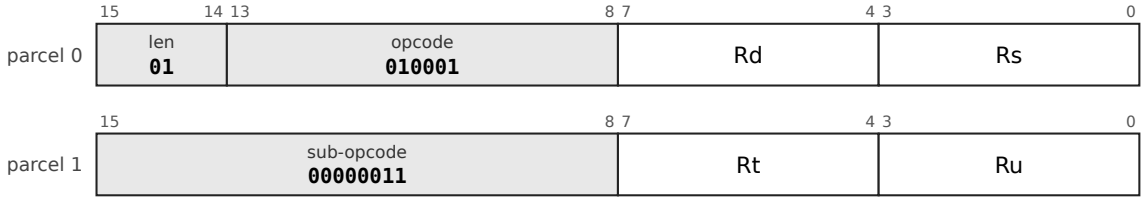
Operation, flags and exceptions as for the entry SWAP above; 2 layout forms, each a chart of its own (above).

SWAPDB

SWAPDB, opcode 010001, three parcels, sub-opcode 00000011 (6 bytes)



SWAPDB, opcode 010001, two parcels, sub-opcode 00000011 (4 bytes)




```
SWAPDB ...
```

Mnemonic	Opcode	Sub-opcode
SWAPDB	010001 (0x11)	00000011 (0x03)

Operation, flags and exceptions as for the entry SWAP above; 2 layout forms, each a chart of its own (above).

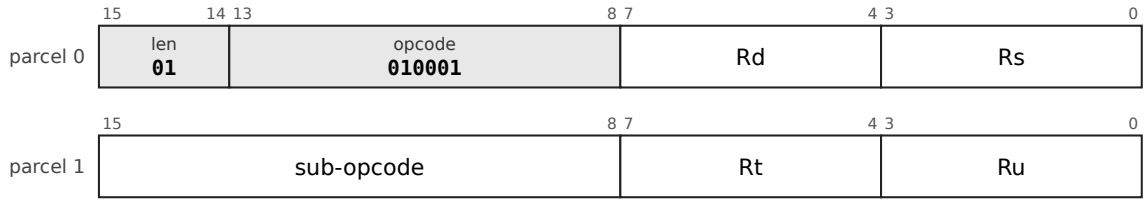
XADD: Atomic fetch and add

Mnemonics: XADD, XADDDB

Syntax

```
XADD value, (address)
XADDDB ...
```

Encoding



The chart is the layout r20-atom-1, Atomics (ATOM), two parcels (len = 01): From len = 01. The placement of the operands is not specified.

Mnemonic	Opcode	Sub-opcode
XADD	010001 (0x11)	00000100 (0x04)
XADDDB	010001 (0x11)	00000101 (0x05)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-atom-1, r20-atom-2.

Operation

```
old ← M[address] ; M[address] ← old + value ; result ← old
```

Flags Not specified beyond "none for the read-modify-write forms"; SC reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified; the placement of the operands of each form is unspecified, and the forms below name the roles.

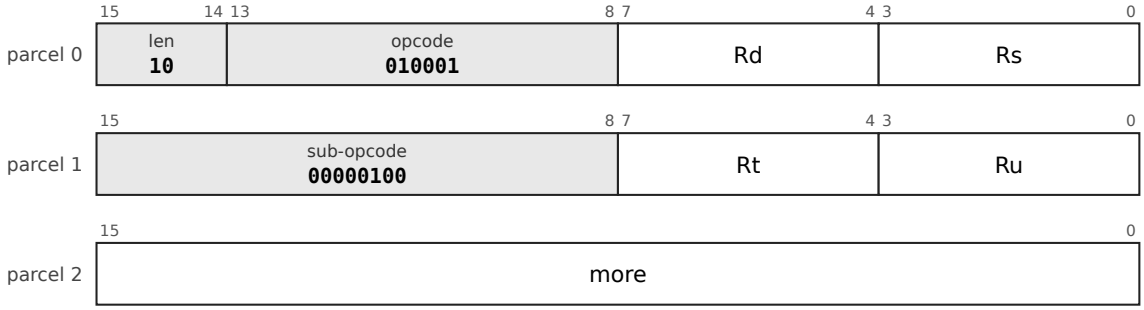
Why it is here. One of the fetch-and-operate atomics of ATOM (see CAS), in the place of the PDP-11's read-modify-write instructions, which were indivisible on the Unibus through the DATIP/DATO pair (7.1).

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

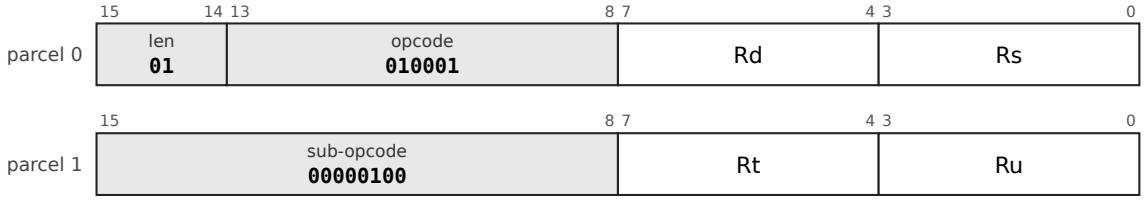
Status specified; **not yet implemented** in the reference executor.

XADD

XADD, opcode 010001, three parcels, sub-opcode 00000100 (6 bytes)



XADD, opcode 010001, two parcels, sub-opcode 00000100 (4 bytes)



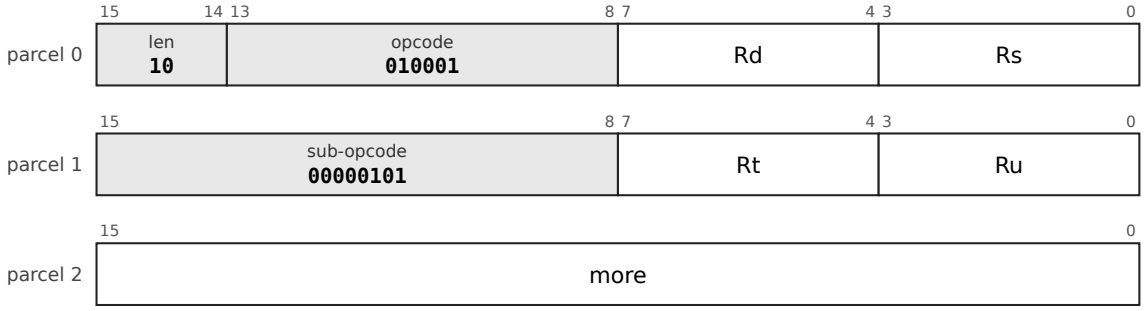

```
XADD value, (address)
```

Mnemonic	Opcode	Sub-opcode
XADD	010001 (0x11)	00000100 (0x04)

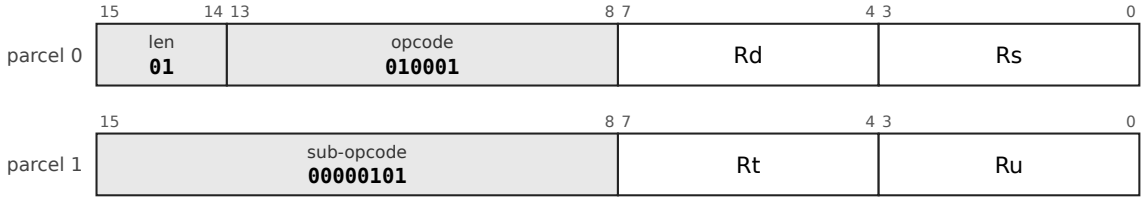
Operation, flags and exceptions as for the entry XADD above; 2 layout forms, each a chart of its own (above).

XADDDB

XADDDB, opcode 010001, three parcels, sub-opcode 00000101 (6 bytes)



XADDDB, opcode 010001, two parcels, sub-opcode 00000101 (4 bytes)




```
XADDDB ...
```

Mnemonic	Opcode	Sub-opcode
XADDDB	010001 (0x11)	00000101 (0x05)

Operation, flags and exceptions as for the entry XADD above; 2 layout forms, each a chart of its own (above).

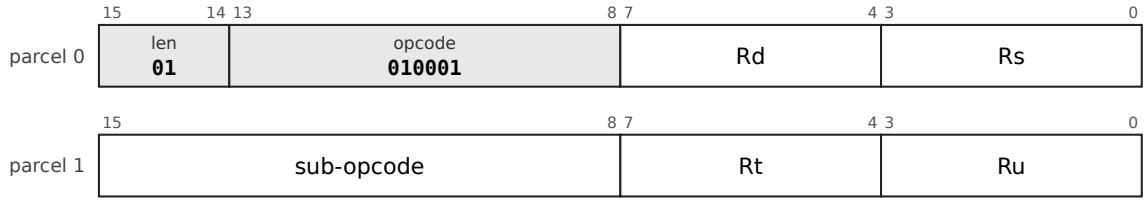
XAND: Atomic fetch and and, or, xor

Mnemonics: XAND, XANDDB, XOR, XOR_DB, XXOR, XXORDB

Syntax

```
XAND value, (address)
XOR_ value, (address)       ; the trailing underscore keeps the mnemonic distinct from XOR
XXOR value, (address)
each with a DB form
```

Encoding



The chart is the layout r20-atom-1, Atomics (ATOM), two parcels (len = 01): From len = 01. The placement of the operands is not specified.

Mnemonic	Opcode	Sub-opcode
XAND	010001 (0x11)	00000110 (0x06)
XANDDB	010001 (0x11)	00000111 (0x07)
XOR_	010001 (0x11)	00001000 (0x08)
XOR_DB	010001 (0x11)	00001001 (0x09)
XXOR	010001 (0x11)	00001010 (0x0A)
XXORDB	010001 (0x11)	00001011 (0x0B)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-atom-1, r20-atom-2.

Operation

```
old ← M[address]
XAND: M[address] ← old AND value
XOR_: M[address] ← old OR value
XXOR: M[address] ← old XOR value
result ← old
```

Flags Not specified beyond "none for the read-modify-write forms"; SC reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified; the placement of the operands of each form is unspecified, and the forms below name the roles.

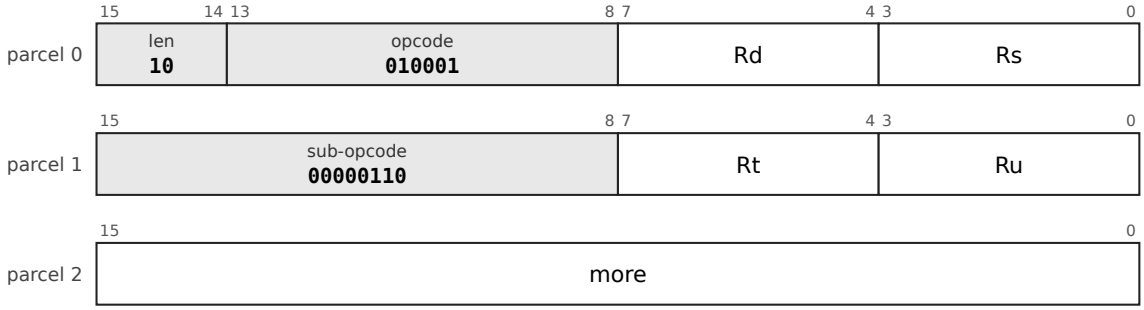
Why it is here. The fetch-and-and, or and xor atomics of ATOM (see CAS). No separate reason is recorded for them.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

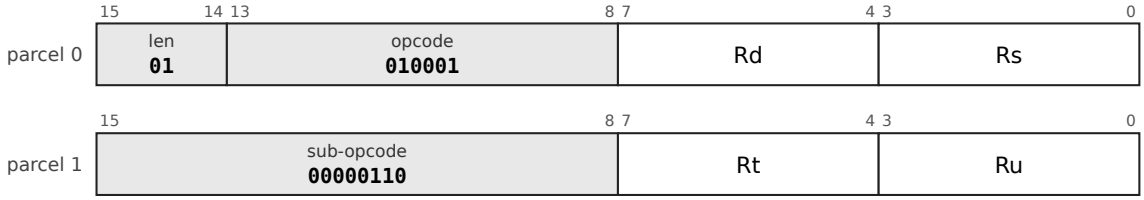
Status specified; **not yet implemented** in the reference executor.

XAND

XAND, opcode 010001, three parcels, sub-opcode 00000110 (6 bytes)



XAND, opcode 010001, two parcels, sub-opcode 00000110 (4 bytes)



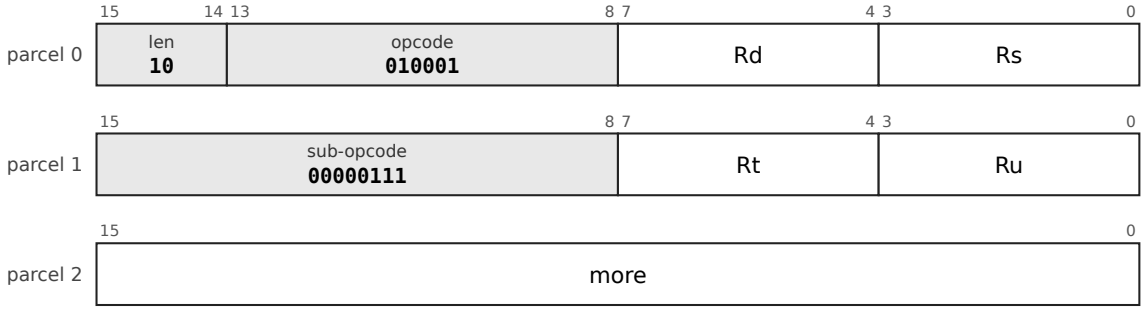

```
XAND value, (address)
```

Mnemonic	Opcode	Sub-opcode
XAND	010001 (0x11)	00000110 (0x06)

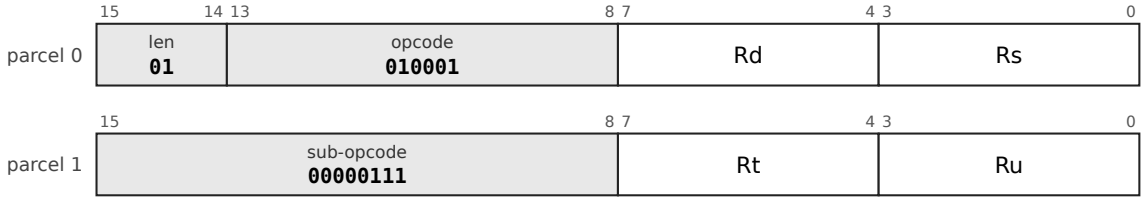
Operation, flags and exceptions as for the entry XAND above; 2 layout forms, each a chart of its own (above).

XANDDB

XANDDB, opcode 010001, three parcels, sub-opcode 00000111 (6 bytes)



XANDDB, opcode 010001, two parcels, sub-opcode 00000111 (4 bytes)

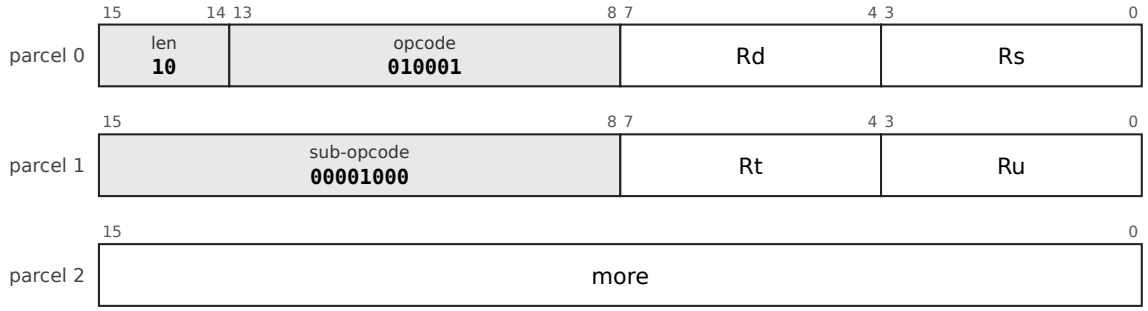


Mnemonic	Opcode	Sub-opcode
XANDDB	010001 (0x11)	00000111 (0x07)

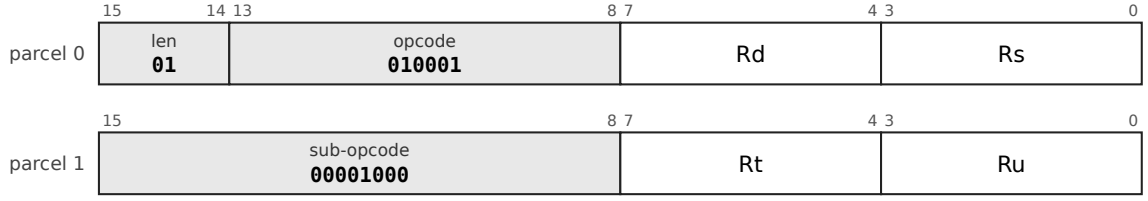
Operation, flags and exceptions as for the entry XAND above; 2 layout forms, each a chart of its own (above).

XOR_

XOR_, opcode 010001, three parcels, sub-opcode 00001000 (6 bytes)



XOR_, opcode 010001, two parcels, sub-opcode 00001000 (4 bytes)



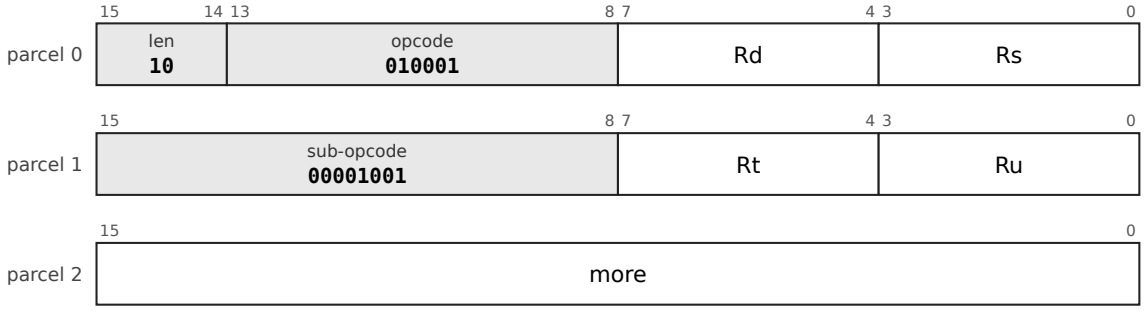

```
XOR_ value, (address)
```

Mnemonic	Opcode	Sub-opcode
XOR_	010001 (0x11)	00001000 (0x08)

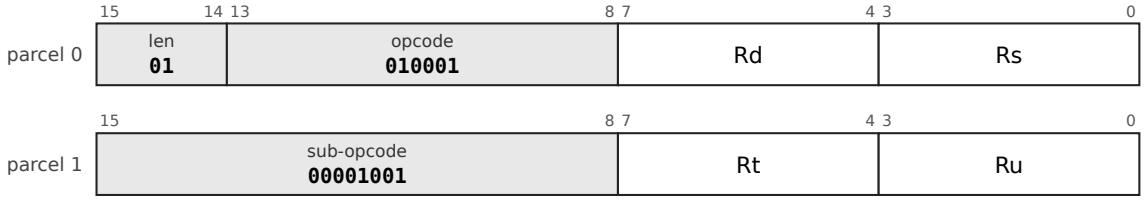
Operation, flags and exceptions as for the entry XAND above; 2 layout forms, each a chart of its own (above).

XOR_DB

XOR_DB, opcode 010001, three parcels, sub-opcode 00001001 (6 bytes)



XOR_DB, opcode 010001, two parcels, sub-opcode 00001001 (4 bytes)

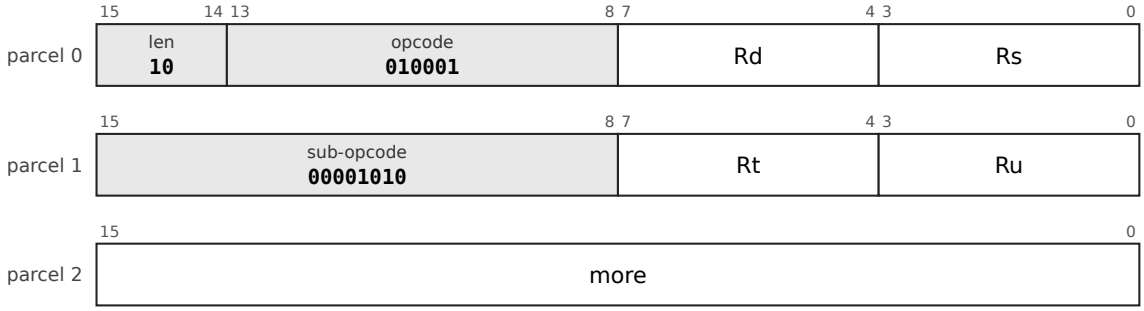


Mnemonic	Opcode	Sub-opcode
XOR_DB	010001 (0x11)	00001001 (0x09)

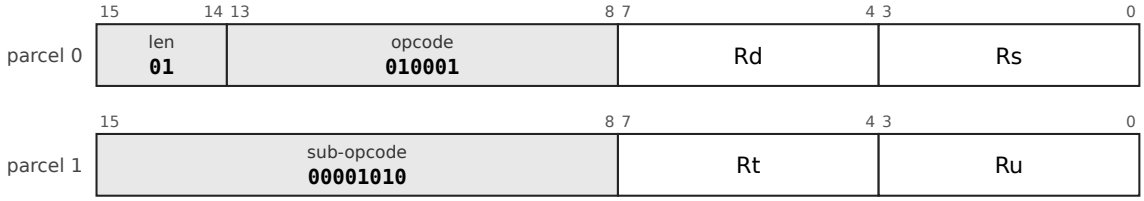
Operation, flags and exceptions as for the entry XAND above; 2 layout forms, each a chart of its own (above).

XXOR

XXOR, opcode 010001, three parcels, sub-opcode 00001010 (6 bytes)



XXOR, opcode 010001, two parcels, sub-opcode 00001010 (4 bytes)



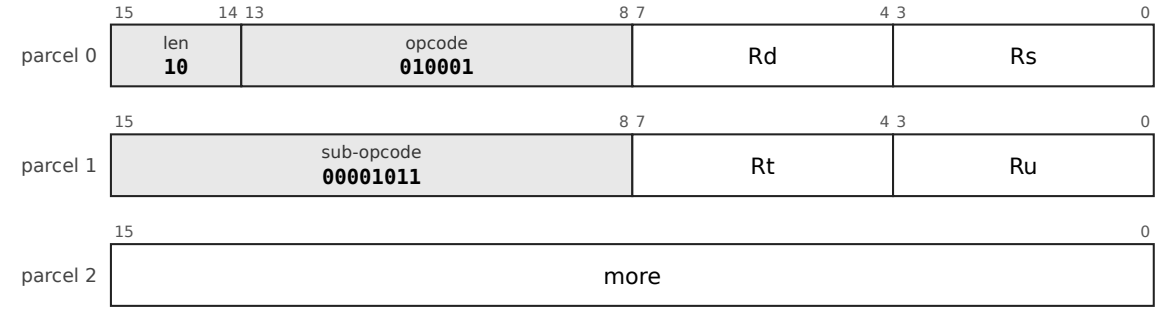

```
XXOR value, (address)
```

Mnemonic	Opcode	Sub-opcode
XXOR	010001 (0x11)	00001010 (0x0A)

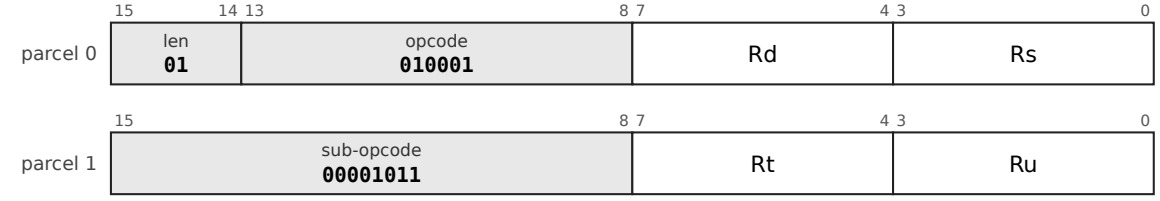
Operation, flags and exceptions as for the entry XAND above; 2 layout forms, each a chart of its own (above).

XXORDB

XXORDB, opcode 010001, three parcels, sub-opcode 00001011 (6 bytes)



XXORDB, opcode 010001, two parcels, sub-opcode 00001011 (4 bytes)



Mnemonic	Opcode	Sub-opcode
XXORDB	010001 (0x11)	00001011 (0x0B)

Operation, flags and exceptions as for the entry XAND above; 2 layout forms, each a chart of its own (above).

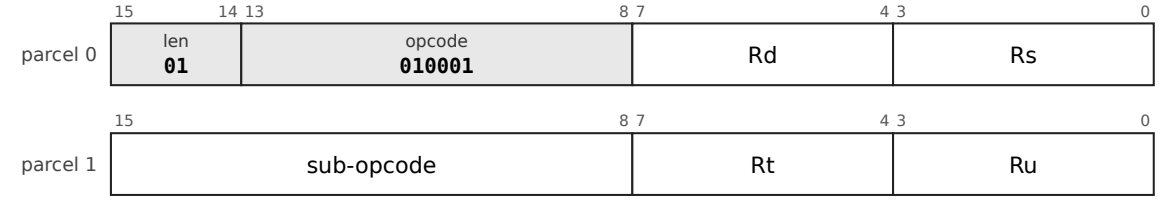
XMAX: Atomic fetch and minimum or maximum

Mnemonics: XMAX, XMAXDB, XMIN, XMINDB, XMAXU, XMAXUDB, XMINU, XMINUDB

Syntax

```
XMAX value, (address)
XMIN ..., XMAXU ..., XMINU ...
each with a DB form
```

Encoding



The chart is the layout r20-atom-1, Atomics (ATOM), two parcels (len = 01): From len = 01. The placement of the operands is not specified.

Mnemonic	Opcode	Sub-opcode
XMAX	010001 (0x11)	00001100 (0x0C)
XMAXDB	010001 (0x11)	00001101 (0x0D)
XMIN	010001 (0x11)	00001110 (0x0E)
XMINDB	010001 (0x11)	00001111 (0x0F)
XMAXU	010001 (0x11)	00010000 (0x10)
XMAXUDB	010001 (0x11)	00010001 (0x11)

Mnemonic	Opcode	Sub-opcode
`XMINU`	`010001 (0x11)`	`00010010 (0x12)`
`XMINUDB`	`010001 (0x11)`	`00010011 (0x13)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-atom-1`, `r20-atom-2`.

Operation

```
old ← M[address] ; M[address] ← max(old, value) (XMAX, XMAXU) or min (XMIN, XMINU) ; result ← old
signed for XMAX and XMIN, unsigned for the U forms
```

Flags Not specified beyond "none for the read-modify-write forms"; `sc` reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

The sub-opcodes and the semantics of this group are specified; the placement of the operands of each form is unspecified, and the forms below name the roles.

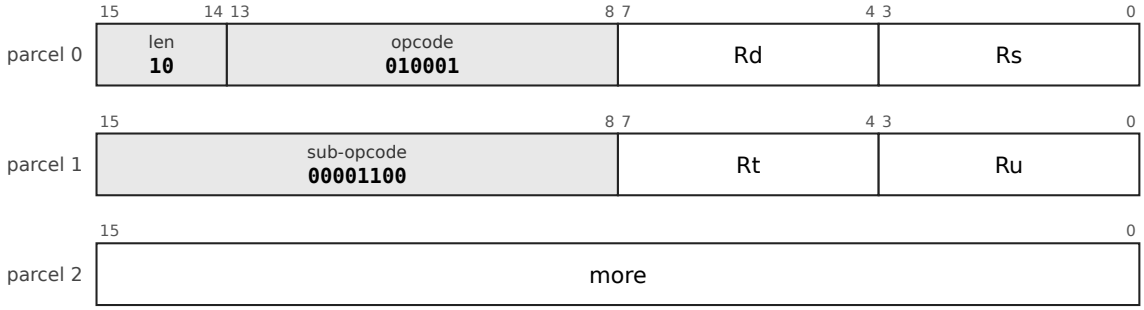
Why it is here. The fetch-and-minimum and maximum atomics, borrowed from LoongArch's richer atomic set: each is a couple of hundred lookup tables and saves a compare-and-swap retry loop.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

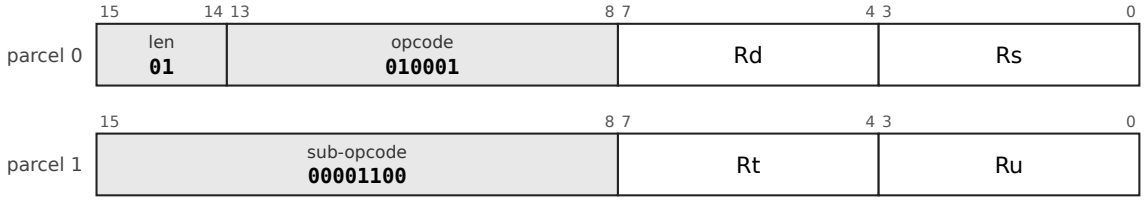
Status specified; **not yet implemented** in the reference executor.

XMAX

XMAX, opcode 010001, three parcels, sub-opcode 00001100 (6 bytes)



XMAX, opcode 010001, two parcels, sub-opcode 00001100 (4 bytes)



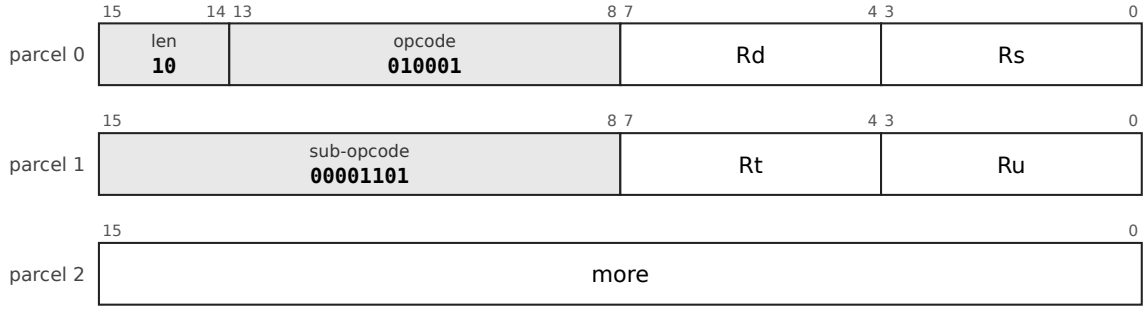

```
XMAX value, (address)
```

Mnemonic	Opcode	Sub-opcode
`XMAX`	`010001 (0x11)`	`00001100 (0x0C)`

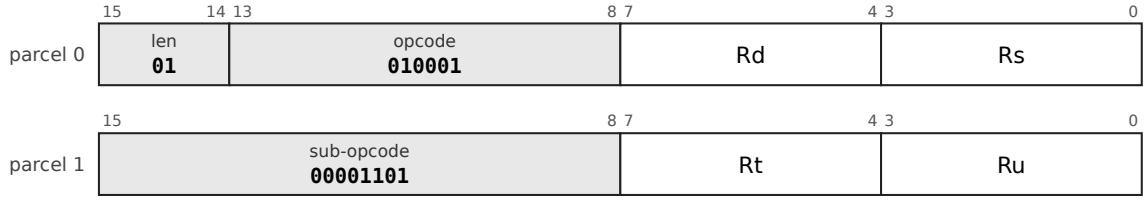
Operation, flags and exceptions as for the entry XMAX above; 2 layout forms, each a chart of its own (above).

XMAXDB

XMAXDB, opcode 010001, three parcels, sub-opcode 00001101 (6 bytes)



XMAXDB, opcode 010001, two parcels, sub-opcode 00001101 (4 bytes)

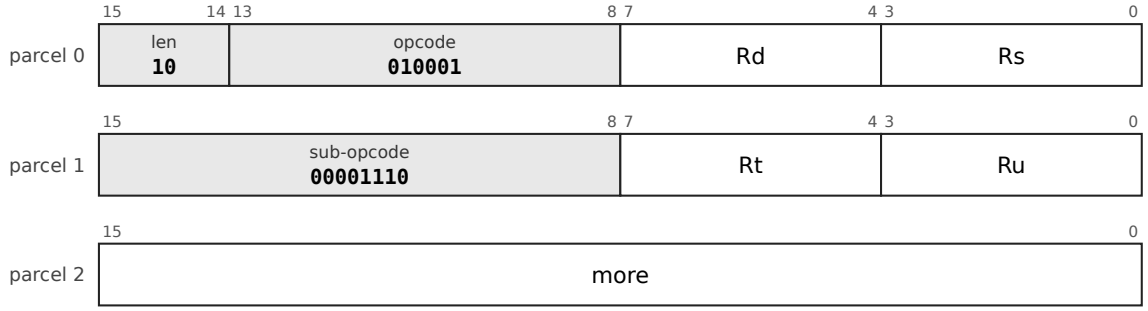


Mnemonic	Opcode	Sub-opcode
XMAXDB	010001 (0x11)	00001101 (0x0D)

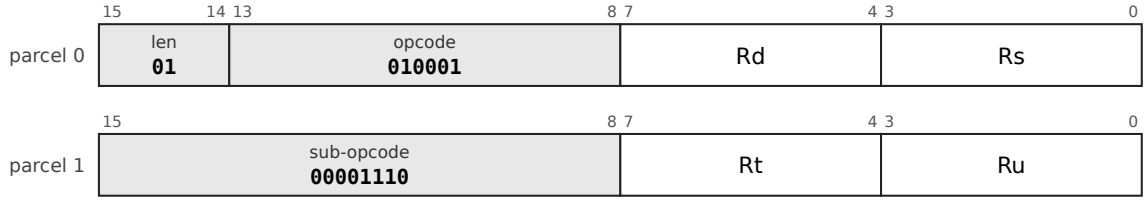
Operation, flags and exceptions as for the entry XMAX above; 2 layout forms, each a chart of its own (above).

XMIN

XMIN, opcode 010001, three parcels, sub-opcode 00001110 (6 bytes)



XMIN, opcode 010001, two parcels, sub-opcode 00001110 (4 bytes)



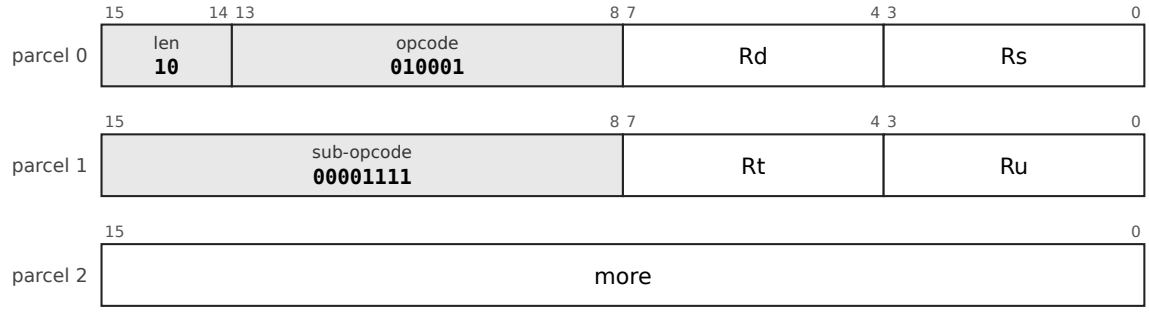

```
XMIN ..., XMAXU ..., XMINU ...
```

Mnemonic	Opcode	Sub-opcode
XMIN	010001 (0x11)	00001110 (0x0E)

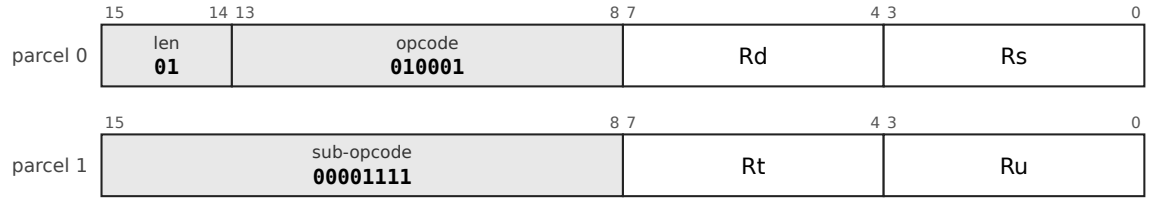
Operation, flags and exceptions as for the entry XMAX above; 2 layout forms, each a chart of its own (above).

XMINDB

XMINDB, opcode 010001, three parcels, sub-opcode 00001111 (6 bytes)



XMINDB, opcode 010001, two parcels, sub-opcode 00001111 (4 bytes)

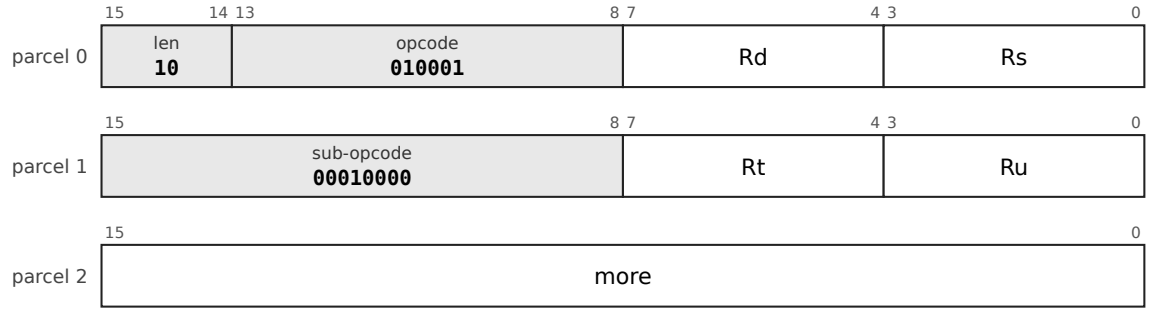


Mnemonic	Opcode	Sub-opcode
XMINDB	010001 (0x11)	00001111 (0x0F)

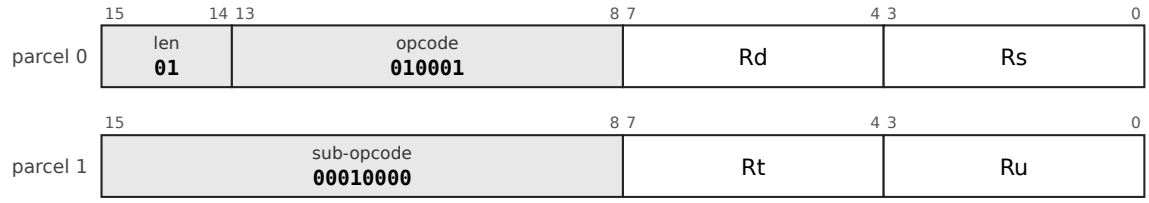
Operation, flags and exceptions as for the entry XMAX above; 2 layout forms, each a chart of its own (above).

XMAXU

XMAXU, opcode 010001, three parcels, sub-opcode 00010000 (6 bytes)



XMAXU, opcode 010001, two parcels, sub-opcode 00010000 (4 bytes)

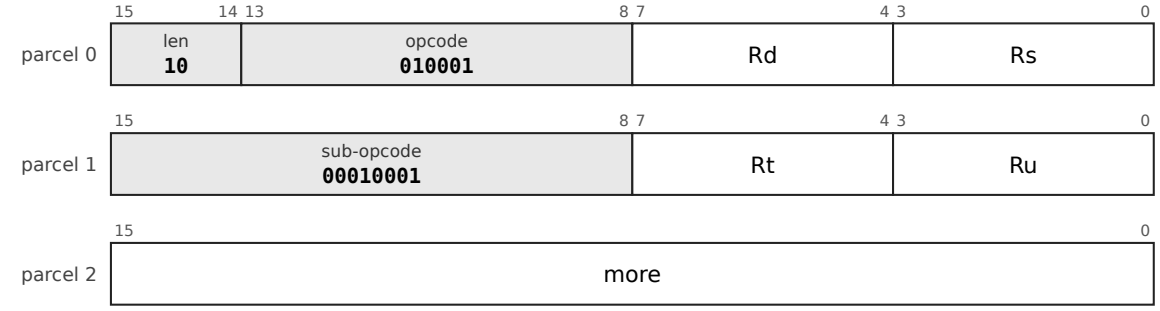


Mnemonic	Opcode	Sub-opcode
XMAXU	010001 (0x11)	00010000 (0x10)

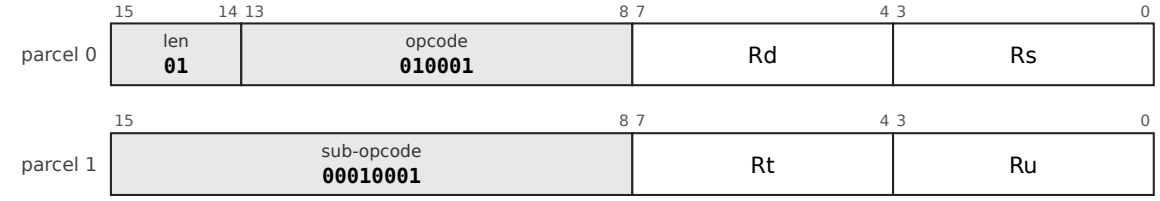
Operation, flags and exceptions as for the entry XMAX above; 2 layout forms, each a chart of its own (above).

XMAXUDB

XMAXUDB, opcode 010001, three parcels, sub-opcode 00010001 (6 bytes)



XMAXUDB, opcode 010001, two parcels, sub-opcode 00010001 (4 bytes)

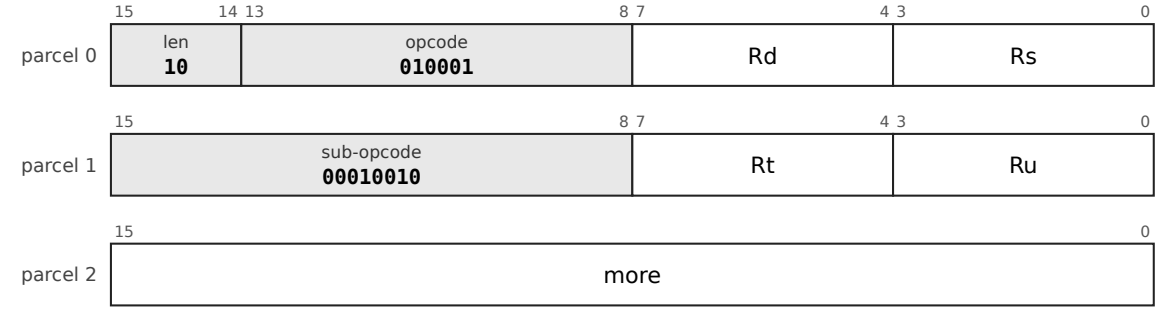


Mnemonic	Opcode	Sub-opcode
XMAXUDB	010001 (0x11)	00010001 (0x11)

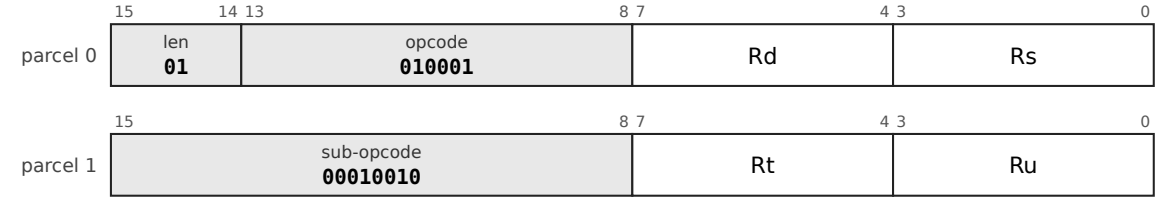
Operation, flags and exceptions as for the entry XMAX above; 2 layout forms, each a chart of its own (above).

XMINU

XMINU, opcode 010001, three parcels, sub-opcode 00010010 (6 bytes)



XMINU, opcode 010001, two parcels, sub-opcode 00010010 (4 bytes)

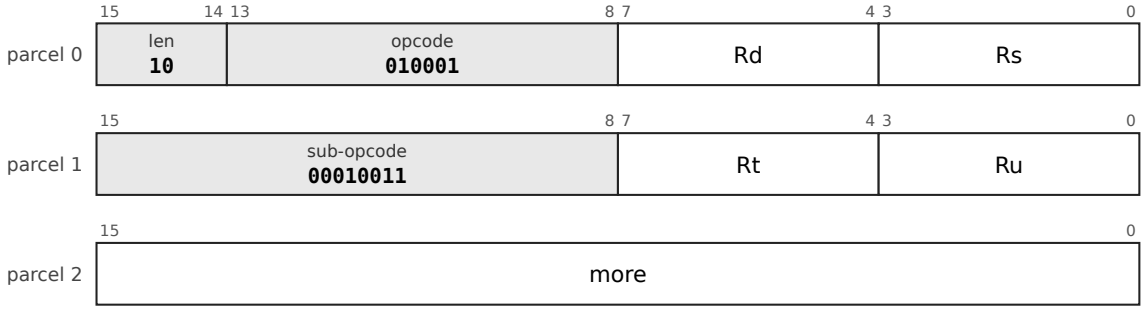


Mnemonic	Opcode	Sub-opcode
XMINU	010001 (0x11)	00010010 (0x12)

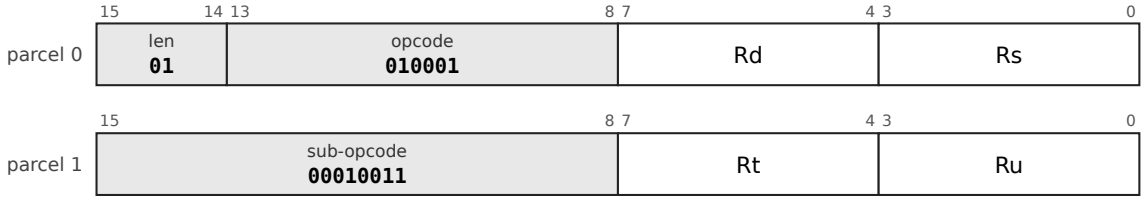
Operation, flags and exceptions as for the entry XMAX above; 2 layout forms, each a chart of its own (above).

XMINUDB

XMINUDB, opcode 010001, three parcels, sub-opcode 00010011 (6 bytes)



XMINUDB, opcode 010001, two parcels, sub-opcode 00010011 (4 bytes)



Mnemonic	Opcode	Sub-opcode
XMINUDB	010001 (0x11)	00010011 (0x13)

Operation, flags and exceptions as for the entry XMAX above; 2 layout forms, each a chart of its own (above).

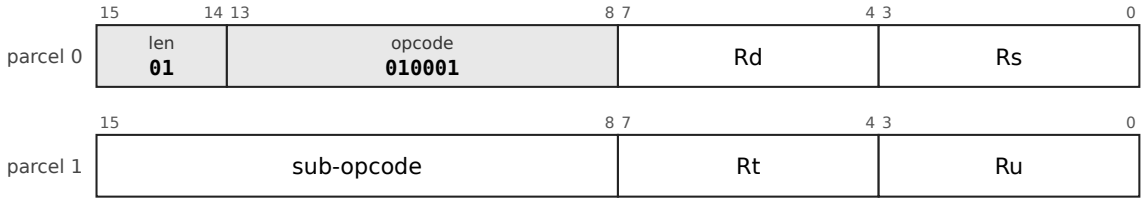
LLSC: Load-linked and store-conditional

Mnemonics: LL, SC

Syntax

```
LL (address), Rd
SC value, (address), result
```

Encoding



The chart is the layout r20-atom-1, Atomics (ATOM), two parcels (len = 01): From len = 01. The placement of the operands is not specified.

Mnemonic	Opcode	Sub-opcode
LL	010001 (0x11)	00100000 (0x20)
SC	010001 (0x11)	00100001 (0x21)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-atom-1, r20-atom-2.

Operation

```
LL: Rd ← M[address] ; take a reservation on the granule containing address
SC: if the reservation is intact then M[address] ← value ; result ← 0 (success)
    else nothing is written ; result ← 1 (failure)
a write to the granule by any initiator, core or device, clears the reservation
```

Flags Not specified beyond "none for the read-modify-write forms"; SC reports success in its result register.

Exceptions Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

A short LL/SC sequence must eventually succeed if no other initiator keeps writing the granule (the RISC-V guarantee for constrained sequences). The sub-opcodes and the semantics of this group are specified; the placement of the operands of each form is unspecified, and the forms below name the roles. The result convention shown for SC is the usual one and is unspecified.

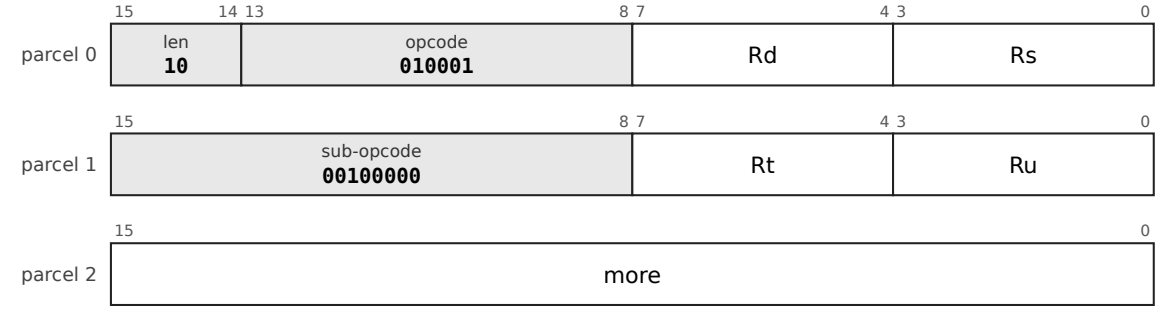
Why it is here. Load-linked and store-conditional, with the reservation a monitor, because a monitor maps onto the exclusive access of AXI and of FPGA and modern memory systems, where the Unibus's locked read-modify-write (DATIP/DATO) does not scale as cores multiply. A short sequence must eventually succeed if no other initiator keeps writing the granule, the RISC-V rule.

Cf. 7.12 (the atomic instructions), 7.3 (the atomics in detail: the reservation monitor), 7.2 (the memory model).

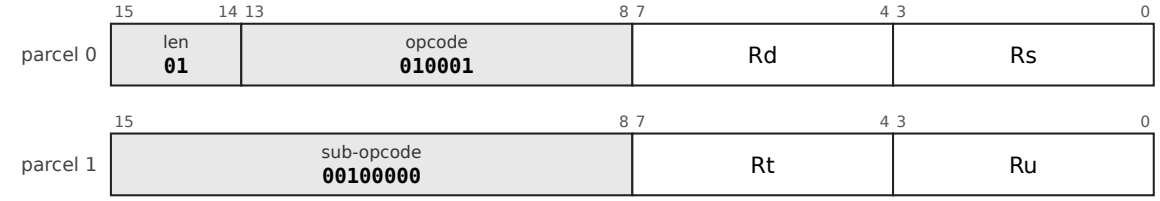
Status specified; **not yet implemented** in the reference executor.

LL

LL, opcode 010001, three parcels, sub-opcode 00100000 (6 bytes)



LL, opcode 010001, two parcels, sub-opcode 00100000 (4 bytes)



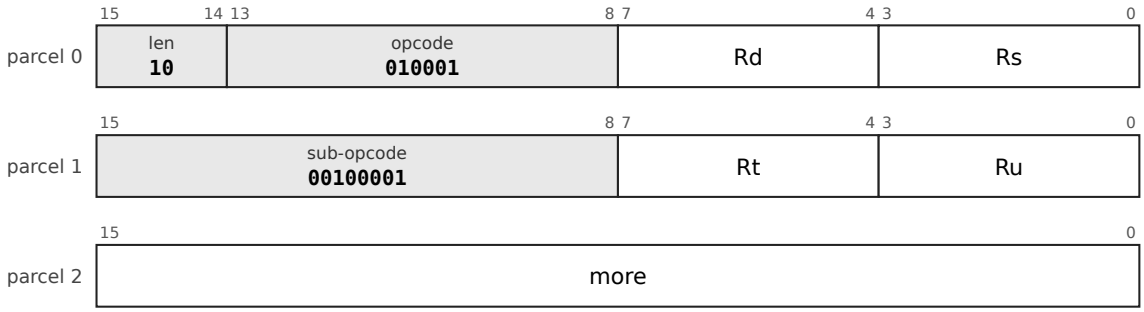

```
LL (address), Rd
```

Mnemonic	Opcode	Sub-opcode
LL	010001 (0x11)	00100000 (0x20)

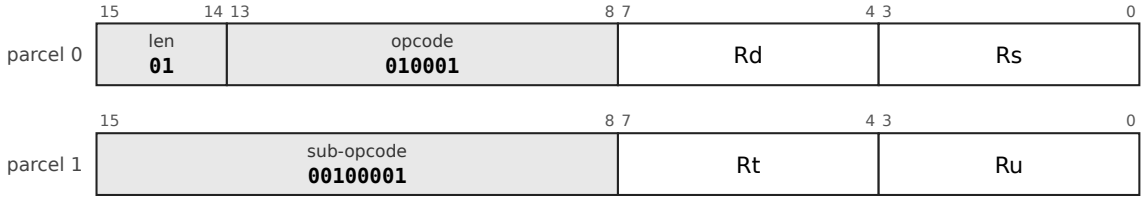
Operation, flags and exceptions as for the entry LLSC above; 2 layout forms, each a chart of its own (above).

SC

SC, opcode 010001, three parcels, sub-opcode 00100001 (6 bytes)



SC, opcode 010001, two parcels, sub-opcode 00100001 (4 bytes)




```
SC value, (address), result
```

Mnemonic	Opcode	Sub-opcode
SC	010001 (0x11)	00100001 (0x21)

Operation, flags and exceptions as for the entry LLSC above; 2 layout forms, each a chart of its own (above).

9.14 System instructions, exceptions and interrupts

Cf. chapter 6, section 6.13 (system instructions, exceptions and interrupts), section 6.3 (exceptions and interrupts) and section 6.5 (system registers).

9.14.1 The system escape SYS (opcode 010100 (0x14)) holds the privileged and system instructions: TRAP, TRAPL, RTI, RTT, RET, HALT, BPT, MTPS, MFPS, SPL, MFSR, MTSR, MTDP, MFDP, MFCPUID, PROBER, PROBEW, PUSHX, POPX, and the translation and cache maintenance instructions (TLBI, TLBIA, TLBSYNC, MTASID, MFASID, ICINV, ICINVA). The user-memory guard instructions STUA and CLUA are described with them. *(opcode 010100 (0x14))*

Commentary. The official entry of each instruction is below, in this section; the kernel chapter gives the modes, the entry sequence, the system registers and the reasons. **Impact.** *Kernel:* the entries below are the complete list of what the kernel may execute that user code may not, and of what both may; chapter 6 explains how they are used. *Compiler:* none, apart from TRAP in system-call stubs. *Hardware:* a class decode and one privilege check.

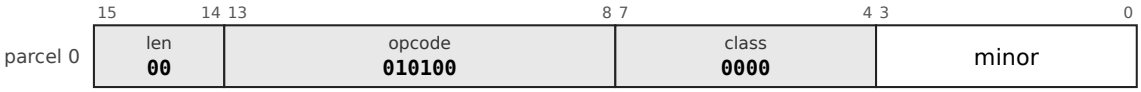
NOP: No operation and landing pad

Mnemonics: NOP, LPAD

Syntax

```
NOP
LPAD
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
NOP	010100 (0x14)	0000 (0x0)	0000 (0x0)
LPAD	010100 (0x14)	0000 (0x0)	1001 (0x9)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

Operation

```
nothing
```

Flags None written.

Exceptions None.

`LPAD` marks a legal target of an indirect jump or call; no instruction enforces it in ABI version 1.

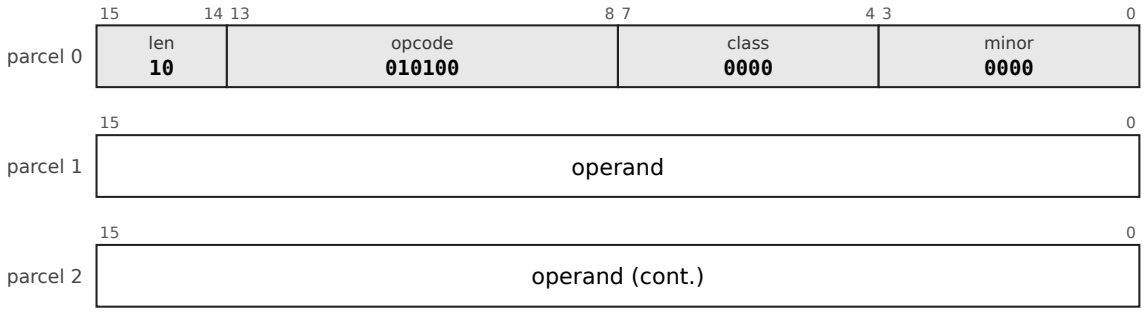
Why it is here. `NOP` is common and has no story; `LPAD` is the landing pad. Landing pads mark the legal targets of indirect jumps and calls so that jump-oriented attacks can be stopped at the transfer, as Intel's indirect-branch tracking and ARM's branch target identification do and OpenBSD enforces. ABI version 1 leaves them off (an open item of the instruction set, 9.21), and the assembler provides `.lpad`; the kernel pain-points and mitigations reports record why deciding later is more expensive (appendix A, section A.1).

Cf. Appendix A, section A.1 (landing pads).

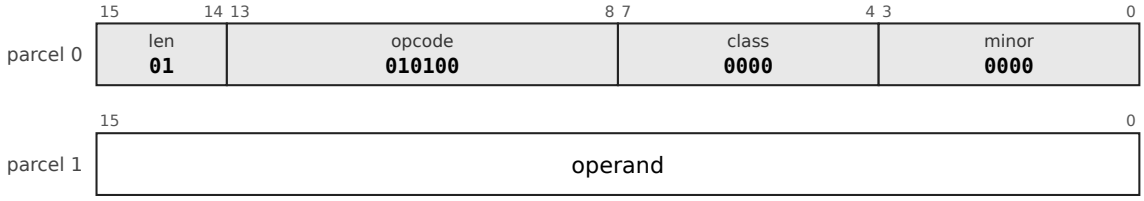
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

NOP

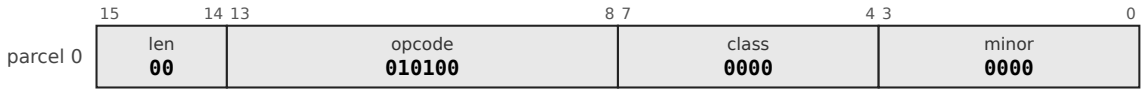
NOP, opcode 010100, three parcels (6 bytes)



NOP, opcode 010100, two parcels (4 bytes)



NOP, opcode 010100, one parcel (2 bytes)



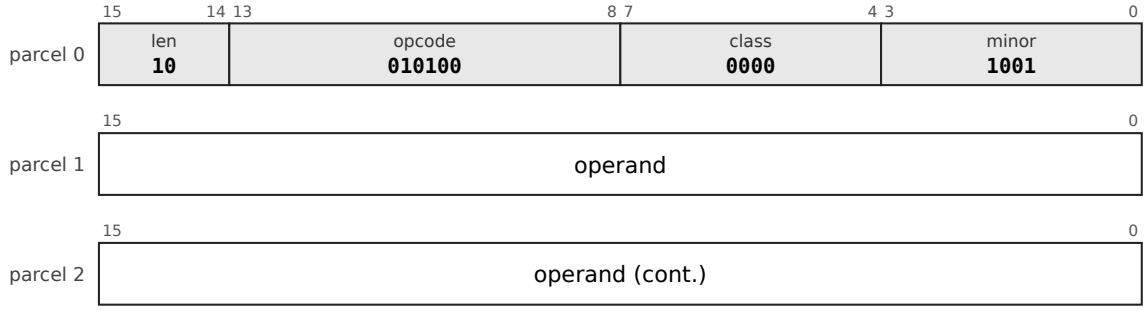

```
NOP
```

Mnemonic	Opcode	reg	low
NOP	010100 (0x14)	0000 (0x0)	0000 (0x0)

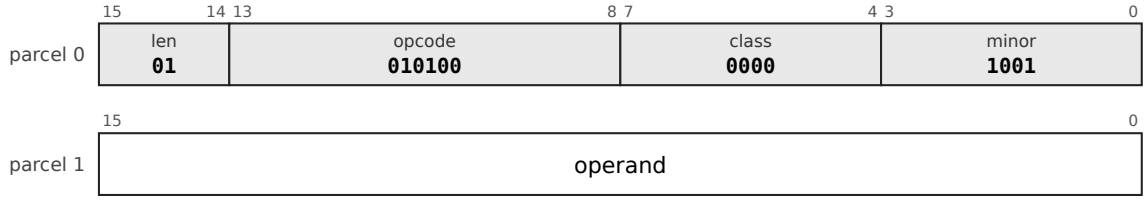
Operation, flags and exceptions as for the entry NOP above; 3 layout forms, each a chart of its own (above).

LPAD

LPAD, opcode 010100, three parcels (6 bytes)



LPAD, opcode 010100, two parcels (4 bytes)



LPAD, opcode 010100, one parcel (2 bytes)




```
LPAD
```

Mnemonic	Opcode	reg	low
LPAD	010100 (0x14)	0000 (0x0)	1001 (0x9)

Operation, flags and exceptions as for the entry NOP above; 3 layout forms, each a chart of its own (above).

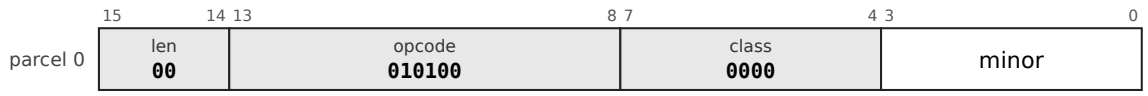
RTI: Return from interrupt or exception

Mnemonics: `RTI`, `RTT`

Syntax

```
RTI
RTT
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
RTI	010100 (0x14)	0000 (0x0)	0001 (0x1)
RTT	010100 (0x14)	0000 (0x0)	0010 (0x2)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

Operation

```
PC ← EPC ; PSW ← EPSW           ; a PSW with the reserved mode 2 in either mode field aborts
RTT: and then set the trace bit T after the return
```

Flags Replaced by `EPSW`, including the flags.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

`RTT` sets the trace bit *after* the return rather than before, which is what lets a debugger single-step out of its own trap handler.

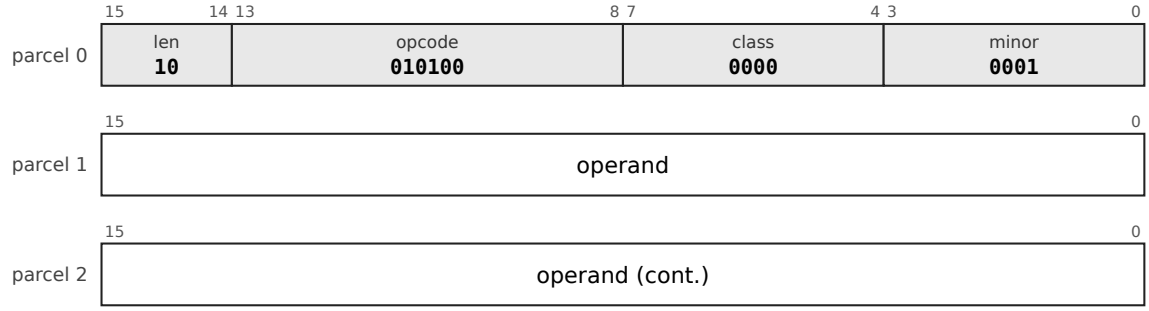
Why it is here. `RTT` looks redundant against `RTI`, and stays: it sets the trace bit *after* the return rather than before, which is what lets a debugger single-step out of its own trap handler.

Cf. 6.3 (exceptions and interrupts), 6.13.

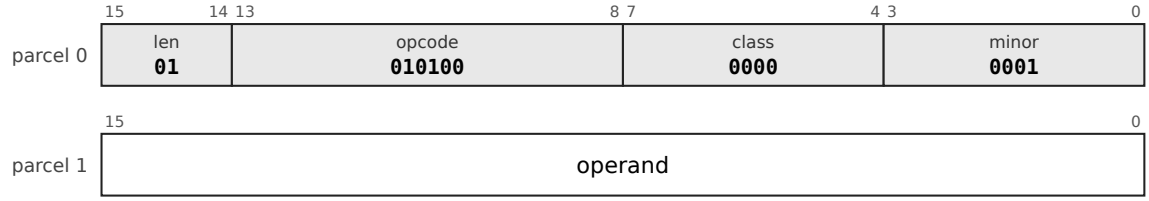
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

RTI

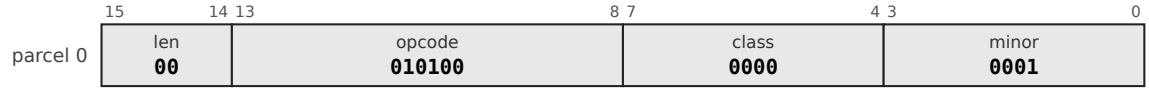
RTI, opcode 010100, three parcels (6 bytes)



RTI, opcode 010100, two parcels (4 bytes)



RTI, opcode 010100, one parcel (2 bytes)



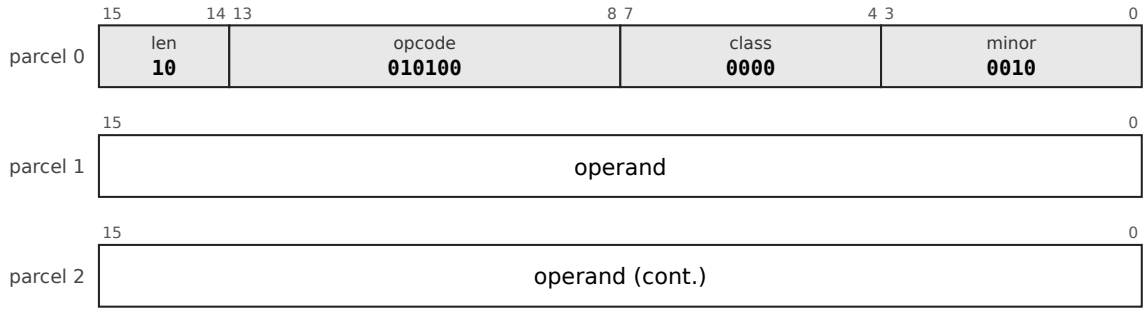

```
RTI
```

Mnemonic	Opcode	reg	low
RTI	010100 (0x14)	0000 (0x0)	0001 (0x1)

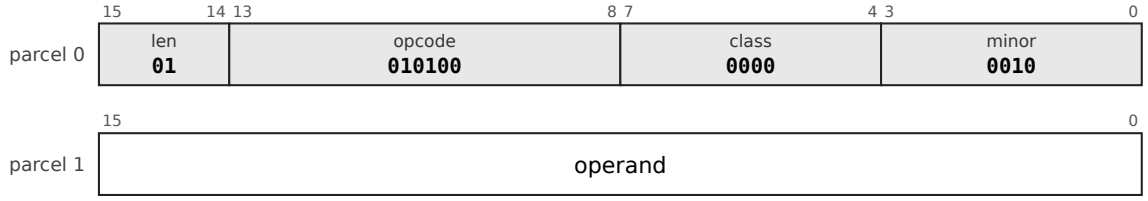
Operation, flags and exceptions as for the entry RTI above; 3 layout forms, each a chart of its own (above).

RTT

RTT, opcode 010100, three parcels (6 bytes)



RTT, opcode 010100, two parcels (4 bytes)



RTT, opcode 010100, one parcel (2 bytes)



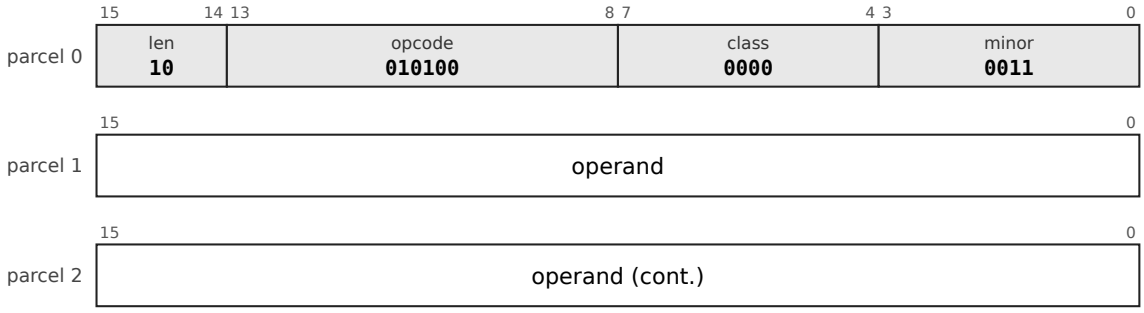

```
RTT
```

Mnemonic	Opcode	reg	low
RTT	010100 (0x14)	0000 (0x0)	0010 (0x2)

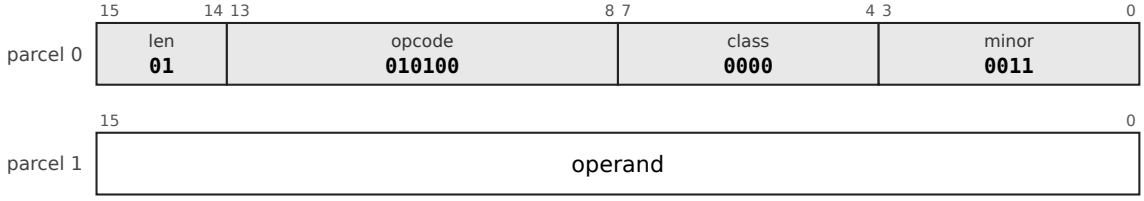
Operation, flags and exceptions as for the entry RTI above; 3 layout forms, each a chart of its own (above).

RET: Return from subroutine

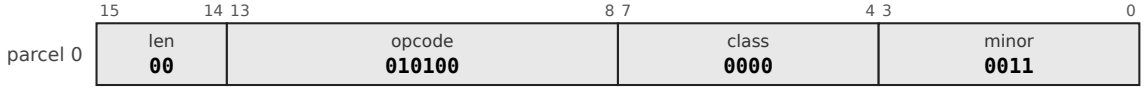
RET, opcode 010100, three parcels (6 bytes)



RET, opcode 010100, two parcels (4 bytes)



RET, opcode 010100, one parcel (2 bytes)



Mnemonics: `RET`

Syntax

```
RET
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
RET	010100 (0x14)	0000 (0x0)	0011 (0x3)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

Operation

```
PC ← M[SP] ; SP ← SP + 8
```

Flags None written.

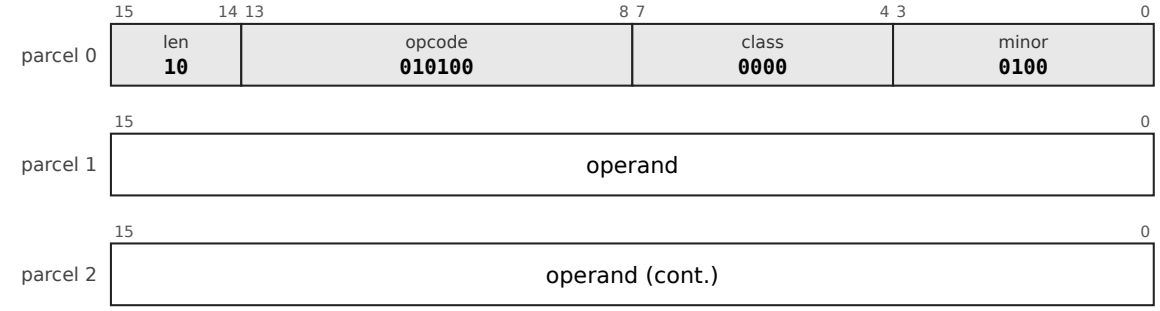
Exceptions Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Not privileged, though it is in the system escape: it is the two-byte partner of `CALL`.

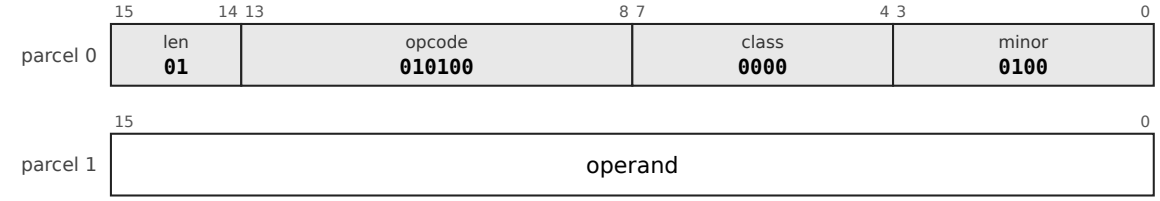
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

HALT: Halt the processor

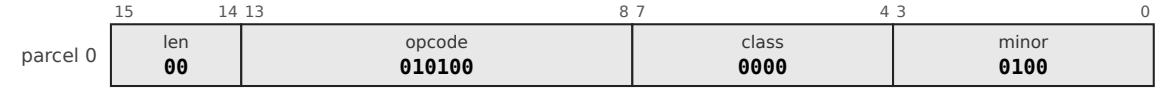
HALT, opcode 010100, three parcels (6 bytes)



HALT, opcode 010100, two parcels (4 bytes)



HALT, opcode 010100, one parcel (2 bytes)

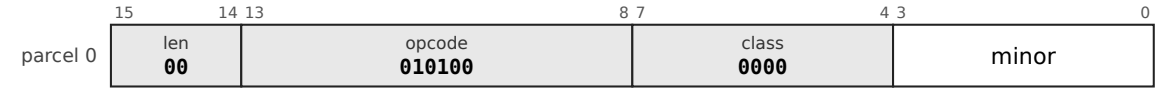


Mnemonics: `HALT`

Syntax

```
HALT
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
HALT	010100 (0x14)	0000 (0x0)	0100 (0x4)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

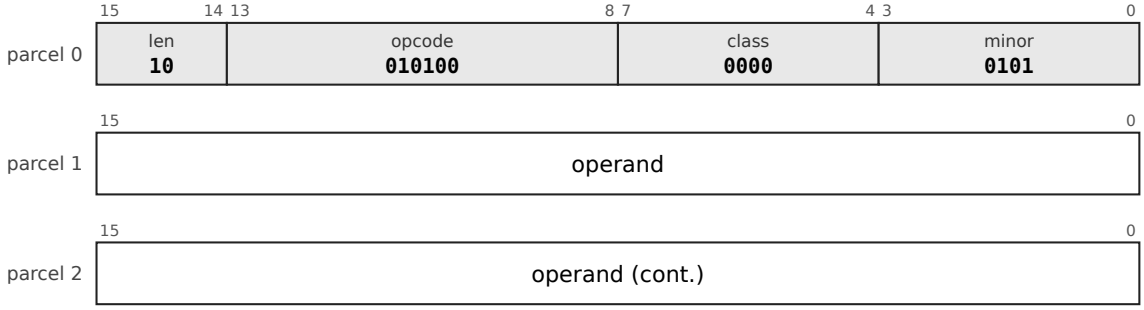
Operation

```
stop the processor
```

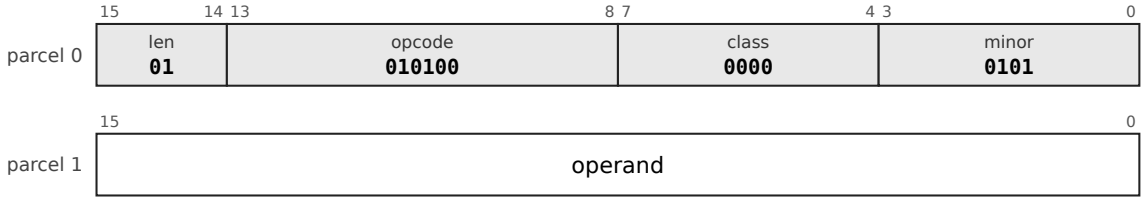
Flags None written.
Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.
Cf. 6.12.
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

BPT: Breakpoint

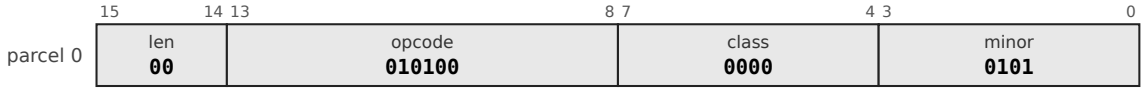
BPT, opcode 010100, three parcels (6 bytes)



BPT, opcode 010100, two parcels (4 bytes)



BPT, opcode 010100, one parcel (2 bytes)



Mnemonics: `BPT`

Syntax

```
BPT
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
BPT	010100 (0x14)	0000 (0x0)	0101 (0x5)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

Operation

```
take the breakpoint vector with EPC = the address of the next instruction
```

Flags None written.

Exceptions Enters the kernel by the `BPT` vector.

A trap like any other, but kept as its own two-byte instruction because a debugger patches it over an existing instruction, so it must be the shortest instruction the machine has.

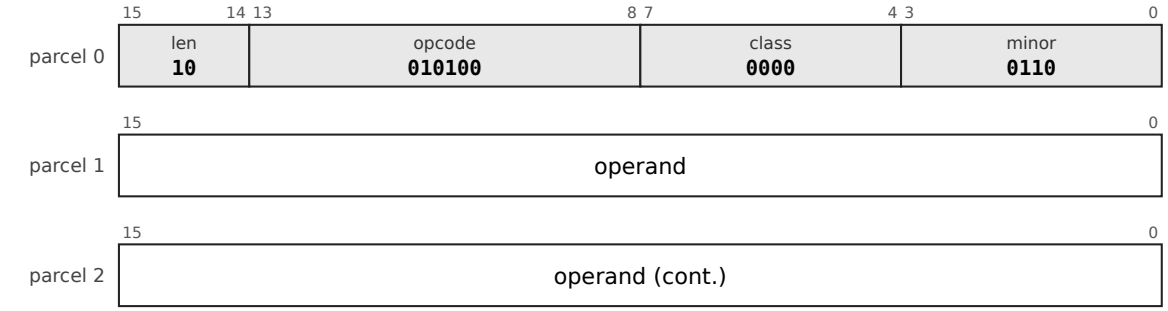
Why it is here. `BPT` is a trap vector, but a debugger patches it over an existing instruction, so it must be the shortest instruction the machine has; that is why it is kept although `IOT` and `EMT` are not.

Cf. 6.3.

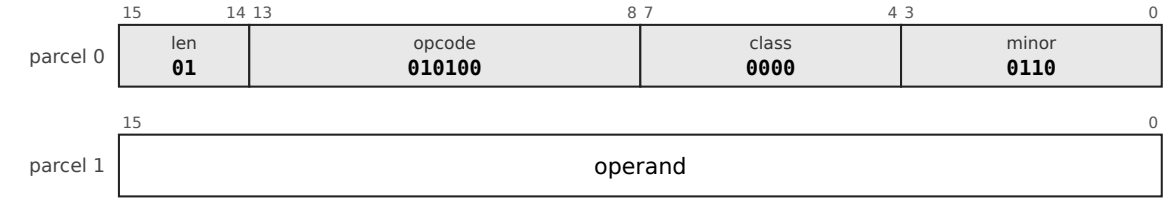
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

FENCE: Memory fence

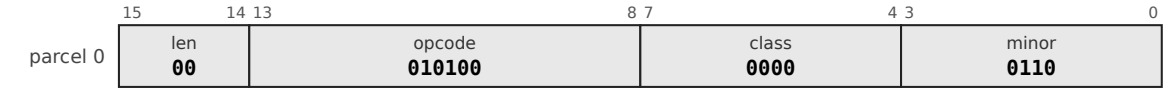
FENCE, opcode 010100, three parcels (6 bytes)



FENCE, opcode 010100, two parcels (4 bytes)



FENCE, opcode 010100, one parcel (2 bytes)

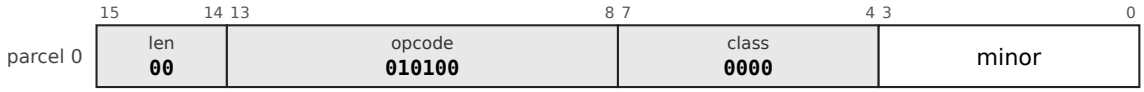


Mnemonics: `FENCE`

Syntax

```
FENCE
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
FENCE	010100 (0x14)	0000 (0x0)	0110 (0x6)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

Operation

```
order the memory accesses before it against those after it, for every initiator that observes them
```

Flags None written.

Exceptions None.

Owed to devices (one `FENCE` separates writing a device's registers from setting its go bit) and to other cores.

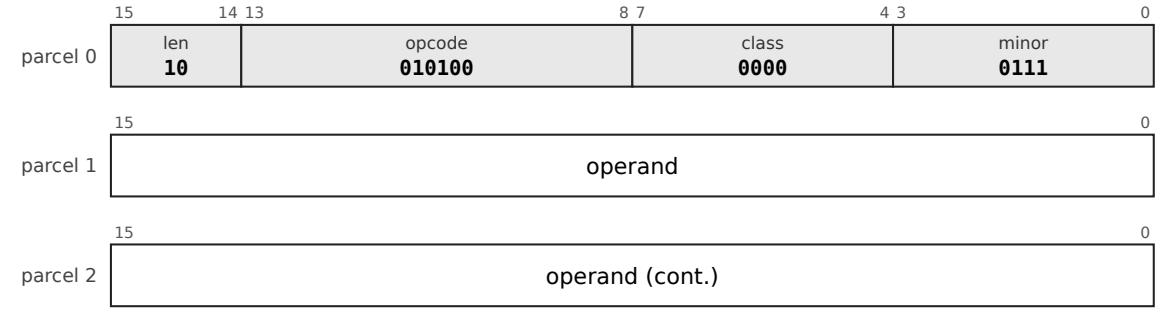
Why it is here. The memory model is sequentially consistent, so `FENCE` is needed only for what is outside the model: devices and the framebuffer (one `FENCE` between writing a device's registers and setting its go bit, section 3.15) and instruction streams. The `_DB` barrier forms of the atomics exist so that a `FENCE` is not needed in a lock acquire.

Cf. 7.2 (the memory model), 2.2 (what the processor owes the bus).

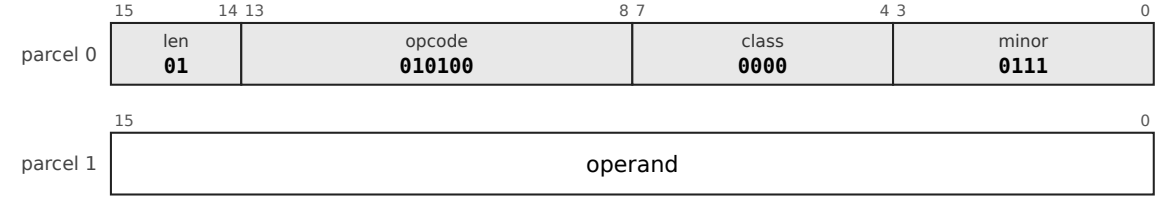
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

WFE: Wait for an event

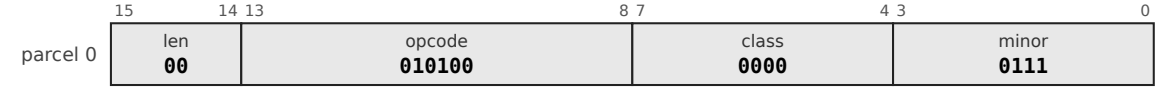
WFE, opcode 010100, three parcels (6 bytes)



WFE, opcode 010100, two parcels (4 bytes)



WFE, opcode 010100, one parcel (2 bytes)

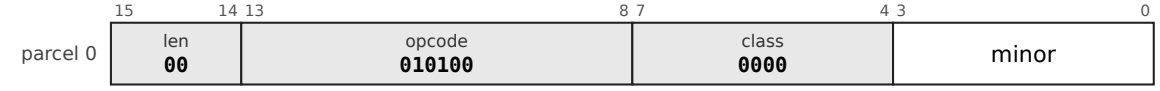


Mnemonics: `WFE` (aliases: `WAIT`)

Syntax

```
WFE
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	`reg`	`low`
`WFE`	`010100 (0x14)`	`0000 (0x0)`	`0111 (0x7)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

Operation

```
sleep until an interrupt is pending or another core executes SEV
```

Flags None written.

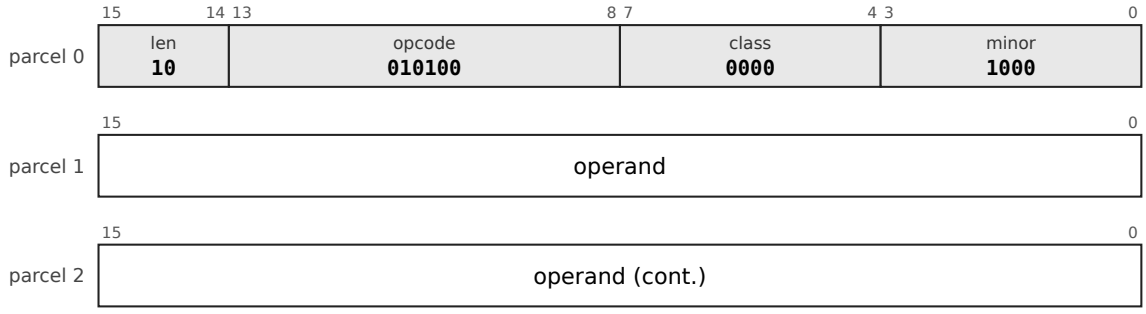
Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

`WAIT` is an assembler alias. Whether `WFE` wakes on a pending but masked interrupt (the idle-loop question) is open.

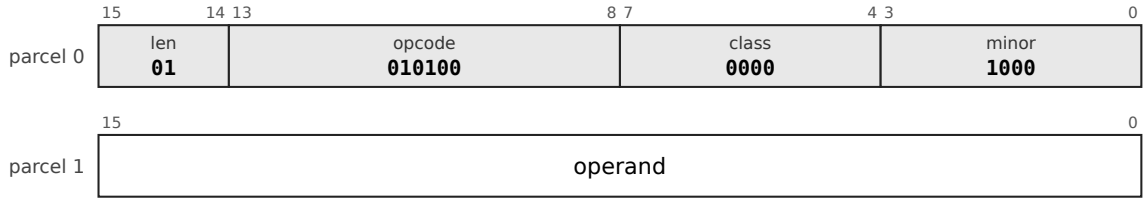
Why it is here. `WAIT` was a PDP-11 instruction (sleep until an interrupt); `WFE` subsumes it: it sleeps until an interrupt or another core's `SEV`, and on a uniprocessor it behaves exactly as `WAIT` did. `WFE` and `SEV` belong to the multiprocessor operations (7.6). The wake rule (whether a pending but masked interrupt wakes it) is open item P1.
Cf. 7.6 (waiting and signalling).
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).
`WAIT` is an alias of `WFE`: the assembler writes it as that word.

SEV: Send an event

SEV, opcode 010100, three parcels (6 bytes)



SEV, opcode 010100, two parcels (4 bytes)



SEV, opcode 010100, one parcel (2 bytes)



Mnemonics: `SEV`

Syntax

```
SEV
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
SEV	010100 (0x14)	0000 (0x0)	1000 (0x8)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

Operation

```
wake every core waiting in WFE
```

Flags None written.

Exceptions None.

Why it is here. The signalling half of `WFE`, so that a core waiting for a lock can sleep instead of spinning and be woken by the core that releases it.

Cf. 7.6.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

TLBA: Invalidate all translations; wait for translation invalidations; invalidate the whole instruction cache

Mnemonics: `TLBIA`, `TLBSYNC`, `ICINVA`

Syntax

```
TLBIA
TLBSYNC
ICINVA
```

Encoding



The chart is the layout `r20-sysn-0`, System, no operand, one parcel (`len = 00`): The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	`reg`	`low`
`TLBIA`	`010100 (0x14)`	`0000 (0x0)`	`1010 (0xA)`
`TLBSYNC`	`010100 (0x14)`	`0000 (0x0)`	`1011 (0xB)`
`ICINVA`	`010100 (0x14)`	`0000 (0x0)`	`1100 (0xC)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysn-0`, `r20-sysn-1`, `r20-sysn-2`.

Operation

```
TLBIA   : invalidate every translation-buffer entry on this core and, by broadcast, on the others
TLBSYNC : wait until the invalidations this core started have completed on every core
ICINVA  : invalidate the whole instruction cache
```

Flags None written.

Exceptions Privileged (`TLBIA`, `TLBSYNC`); the reference executor treats `ICINVA` as a no-op.

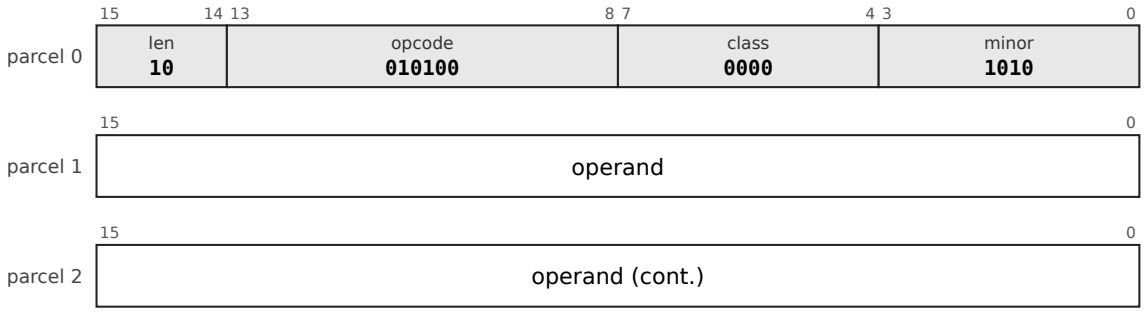
Why it is here. `TLBI` and `TLBIA` broadcast, `TLBSYNC` waits for the other cores, and `ICINV` and `ICINVA` invalidate the instruction cache: a Lisp runtime that compiles a function cannot work without an instruction-cache invalidate.

Cf. 7.5 (translation and instruction-cache coherence).

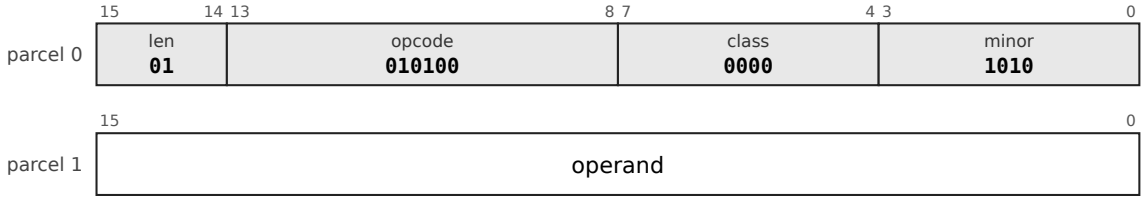
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

TLBIA

TLBIA, opcode 010100, three parcels (6 bytes)



TLBIA, opcode 010100, two parcels (4 bytes)



TLBIA, opcode 010100, one parcel (2 bytes)



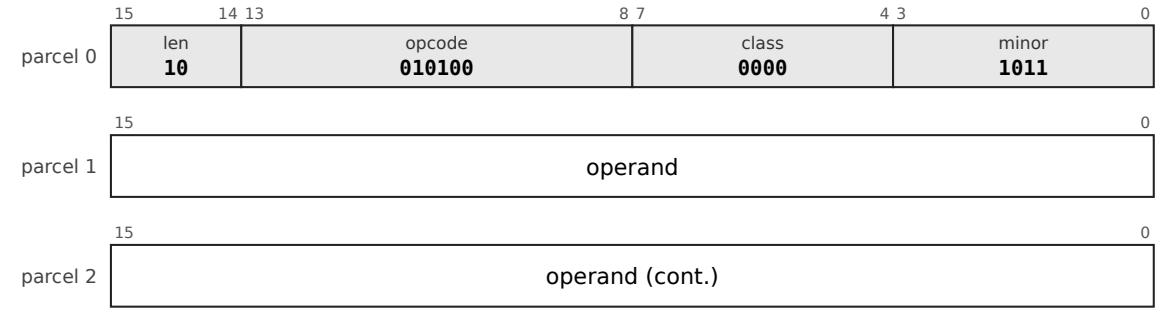

```
TLBIA
```

Mnemonic	Opcode	reg	low
TLBIA	010100 (0x14)	0000 (0x0)	1010 (0xA)

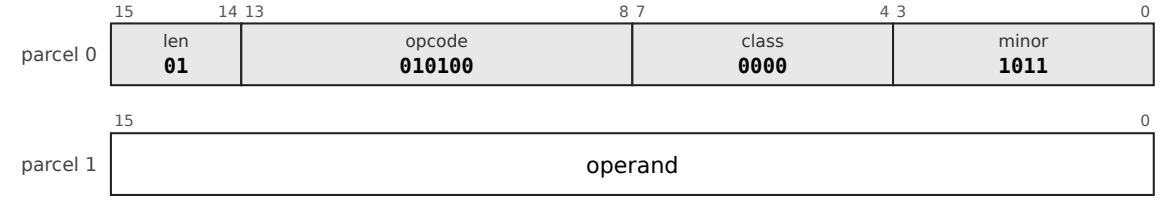
Operation, flags and exceptions as for the entry TLBA above; 3 layout forms, each a chart of its own (above).

TLBSYNC

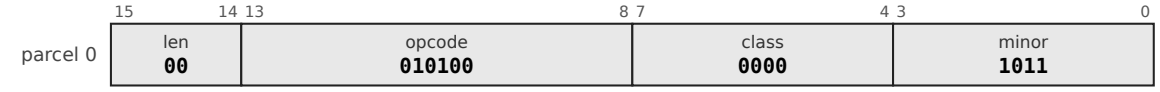
TLBSYNC, opcode 010100, three parcels (6 bytes)



TLBSYNC, opcode 010100, two parcels (4 bytes)



TLBSYNC, opcode 010100, one parcel (2 bytes)



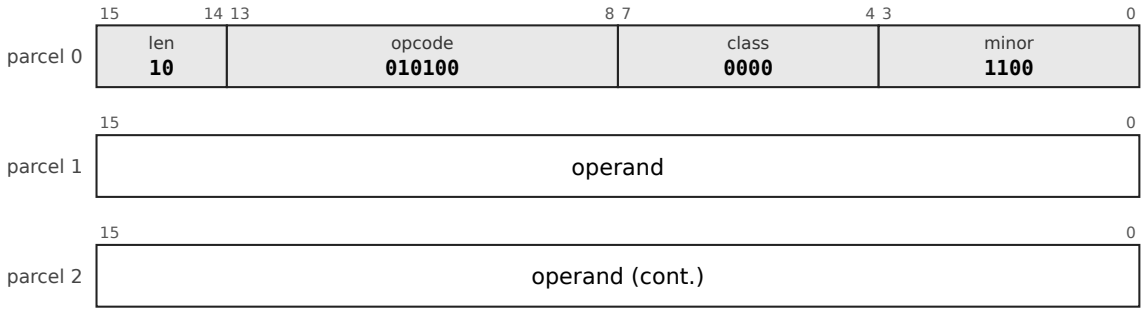

```
TLBSYNC
```

Mnemonic	Opcode	reg	low
TLBSYNC	010100 (0x14)	0000 (0x0)	1011 (0xB)

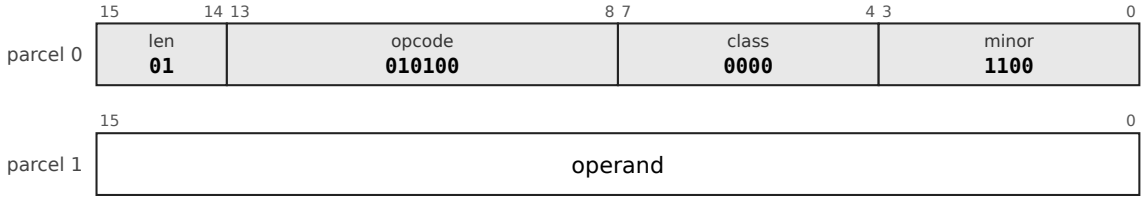
Operation, flags and exceptions as for the entry TLBA above; 3 layout forms, each a chart of its own (above).

ICINVA

ICINVA, opcode 010100, three parcels (6 bytes)



ICINVA, opcode 010100, two parcels (4 bytes)



ICINVA, opcode 010100, one parcel (2 bytes)




```
ICINVA
```

Mnemonic	Opcode	reg	low
ICINVA	010100 (0x14)	0000 (0x0)	1100 (0xC)

Operation, flags and exceptions as for the entry TLBA above; 3 layout forms, each a chart of its own (above).

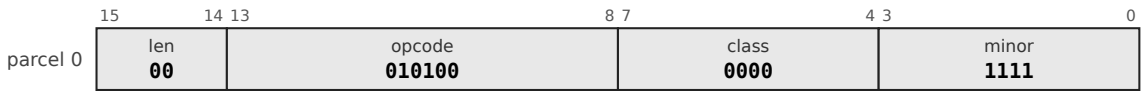
PUSHX: Push and pop the exception state

Mnemonics: PUSHX, POPX

Syntax

```
PUSHX
POPX
```

Encoding



The chart is the layout r20-sysx-0, System, SYSX, one parcel (len = 00): Class 0, minor 15: the sub-opcode is in parcel 1.

Mnemonic	Opcode	reg	low	Sub-opcode
PUSHX	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000000 (0x00)
POPX	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000001 (0x01)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-sysx-0, r20-sysx-1.

Operation

```
PUSHX: SP ← SP - 16 ; M[SP] ← EPSW ; M[SP + 8] ← EPC
POPX : EPSW ← M[SP] ; EPC ← M[SP + 8] ; SP ← SP + 16
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. SP must be 8-byte aligned (a misaligned-access abort); both words are checked before either is touched.

Four bytes (the SYSX escape, sub-opcodes 0 and 1) and two bytes on page 2. For a handler that wants to be interrupted: it can save the exception state and lower the priority.

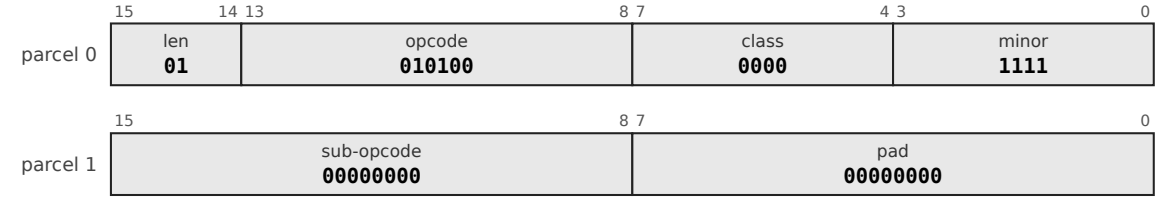
Why it is here. A handler that can be interrupted must save EPC and EPSW before it lowers the priority; with only MFSR that costs two moves and two stores (sixteen bytes) on every such kernel entry, and the same to restore. The instructions may not show up high in the instruction count, but they save bytes and cycles in a tight, very often used kernel loop. Page 2 has the two-byte forms.

Cf. 6.3, 6.13.

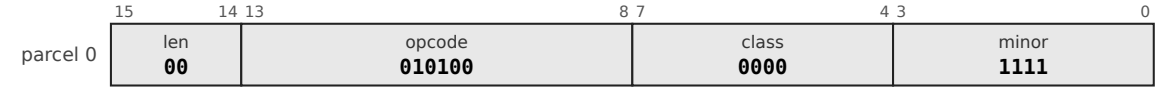
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

PUSHX

PUSHX, opcode 010100, two parcels, sub-opcode 00000000 (4 bytes)



PUSHX, opcode 010100, one parcel, sub-opcode 00000000 (2 bytes)



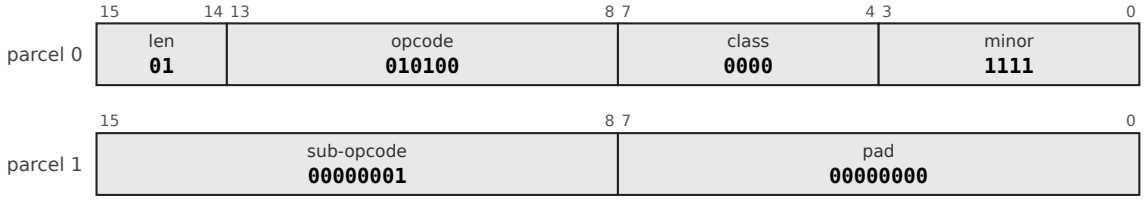

```
PUSHX
```

Mnemonic	Opcode	reg	low	Sub-opcode
PUSHX	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000000 (0x00)

Operation, flags and exceptions as for the entry PUSHX above; 2 layout forms, each a chart of its own (above).

POPX

POPX, opcode 010100, two parcels, sub-opcode 00000001 (4 bytes)



POPX, opcode 010100, one parcel, sub-opcode 00000001 (2 bytes)



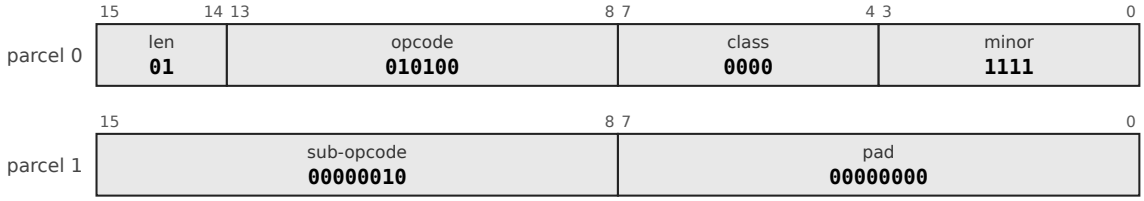

```
POPX
```

Mnemonic	Opcode	reg	low	Sub-opcode
POPX	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000001 (0x01)

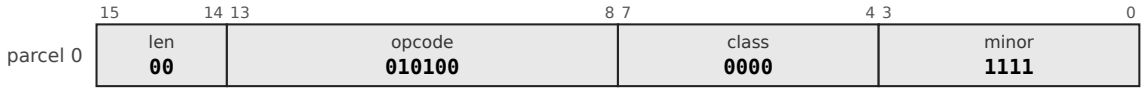
Operation, flags and exceptions as for the entry PUSHX above; 2 layout forms, each a chart of its own (above).

ISYNC: Instruction synchronisation

ISYNC, opcode 010100, two parcels, sub-opcode 00000010 (4 bytes)



ISYNC, opcode 010100, one parcel, sub-opcode 00000010 (2 bytes)

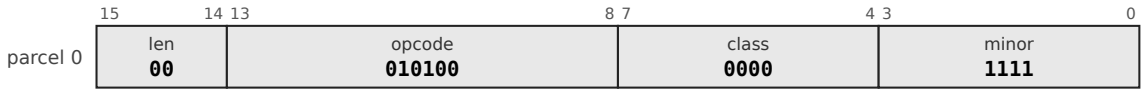


Mnemonics: `ISYNC`

Syntax

```
ISYNC
```

Encoding



The chart is the layout `r20-sysx-0`, System, SYSX, one parcel (`len = 00`): Class 0, minor 15: the sub-opcode is in parcel 1.

Mnemonic	Opcode	reg	low	Sub-opcode
ISYNC	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000010 (0x02)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysx-0`, `r20-sysx-1`.

Operation

```
make earlier writes to PTBR, ASID, MMUCTL and VBR, and earlier ICINVs, take effect for the instructions that
follow
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. A non-zero pad byte is an illegal instruction.

The architecture promises the effect only after the next `ISYNC`; an in-order implementation may make it immediate, and software must not rely on that.

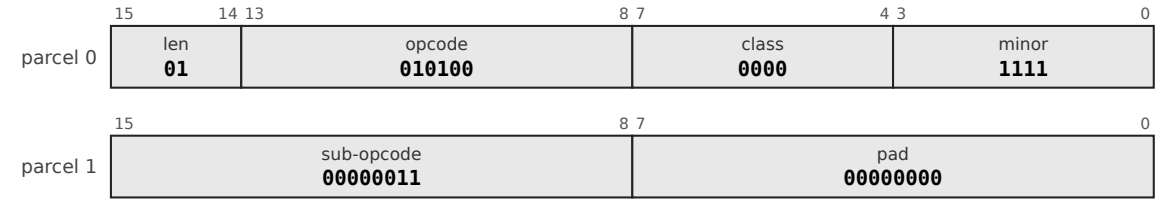
Why it is here. Taken from the out-of-order readiness review: it makes earlier writes to `PTBR`, `ASID`, `MMUCTL` and `VBR` take effect for what follows, so an in-order machine may make them immediate and a speculative one has a place to serialise. It serves the aim that the architecture is efficient on both in-order and out-of-order cores (1.4.4).

Cf. 7.5, 6.5.

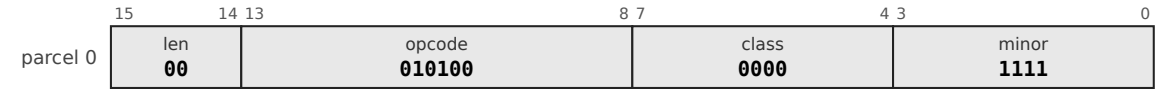
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

HINT: Hint

HINT, opcode 010100, two parcels, sub-opcode 00000011 (4 bytes)



HINT, opcode 010100, one parcel, sub-opcode 00000011 (2 bytes)

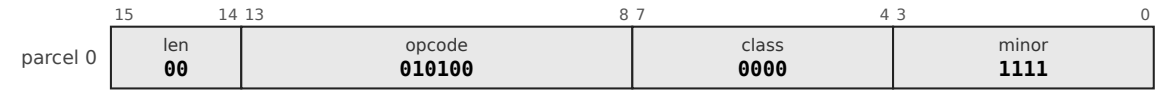


Mnemonics: `HINT`

Syntax

```
HINT #n        ; n = 0 to 255
```

Encoding



The chart is the layout `r20-sysx-0`, System, SYSX, one parcel (`len = 00`): Class 0, minor 15: the sub-opcode is in parcel 1.

Mnemonic	Opcode	reg	low	Sub-opcode
HINT	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000011 (0x03)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysx-0`, `r20-sysx-1`.

Operation

```
no architectural effect
```

Flags None written.

Exceptions None.

Defined codes: 0 a plain no-op hint, 1 pause (spin-wait), 2 speculation barrier. Every other code is a reserved hint that executes as a no-op; software shall not use it. Not privileged.

Why it is here. Defined with `ISYNC`: a no-op with a byte of code, defined for `0` (plain), `1` (pause, for spin-wait loops) and `2` (speculation barrier), every other code a reserved hint that executes as a no-op so that a later machine can give it a meaning without breaking an old binary.

Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

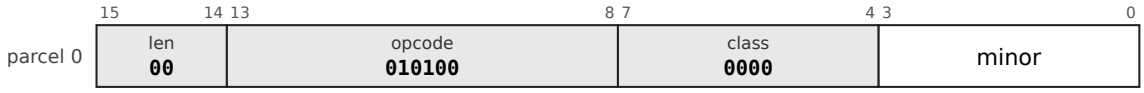
MFSR: Move from and to a system register

Mnemonics: `MFSR`, `MTSR`

Syntax

```
MFSR Rd, name
MTSR name, Rs
```

Encoding



The chart is the layout `r20-sysreg-0`, System registers (MFSR, MTSR), one parcel (`len = 00`): Class 0: the system register number is in parcel 1.

Mnemonic	Opcode	`reg`	`low`
`MFSR`	`010100 (0x14)`	`0000 (0x0)`	`1101 (0xD)`
`MTSR`	`010100 (0x14)`	`0000 (0x0)`	`1110 (0xE)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sysreg-0`, `r20-sysreg-1`.

Operation

```
MFSR: Rd ← system register `name`
MTSR: system register `name` ← Rs        ; read-only registers are an illegal instruction
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. An unassigned system register is an illegal instruction.

The register is chosen by name (a number in parcel 1); the instruction is four bytes. A write to `PTBR`, `ASID` or `MMUCTL` flushes the translation buffer.

Name	Number	Access
`PTBR`	0	read and write
`EPC`	1	read and write
`EVA`	2	read only
`ECAUSE`	3	read only
`DMW0`	4	read and write
`DMW1`	5	read and write
`USP`	6	read and write
`SSP`	7	read and write
`EPSW`	8	read and write

Name	Number	Access
VBR	9	read and write
MMUCTL	10	read and write
ASID	11	read and write
CPUID	12	read only

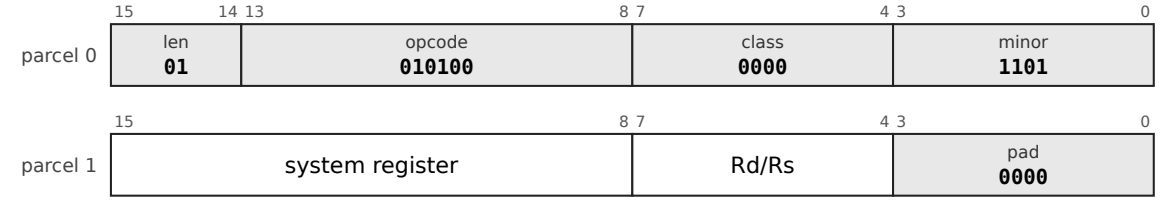
Why it is here. A paging operating system must install a page table and learn why it was entered, which needs instructions to write PTBR and read EPC, EVA and ECAUSE. One general pair reaches all of the system registers by number. The user and supervisor stack pointers are in the set because any kernel must read and write the user stack pointer to build a signal frame, copy a process or switch context.

Cf. 6.5 (system registers).

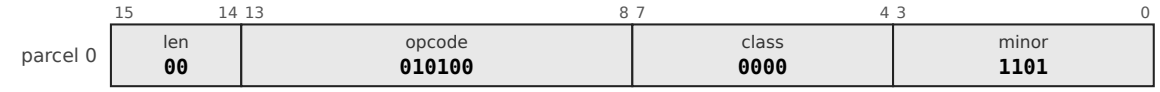
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MFSR

MFSR, opcode 010100, two parcels (4 bytes)



MFSR, opcode 010100, one parcel (2 bytes)



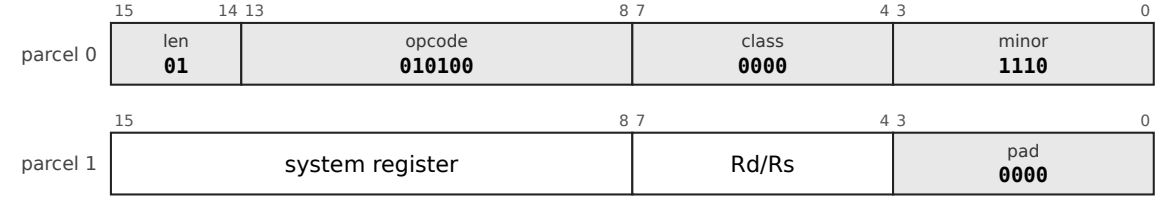

```
MFSR Rd, name
```

Mnemonic	Opcode	reg	low
MFSR	010100 (0x14)	0000 (0x0)	1101 (0xD)

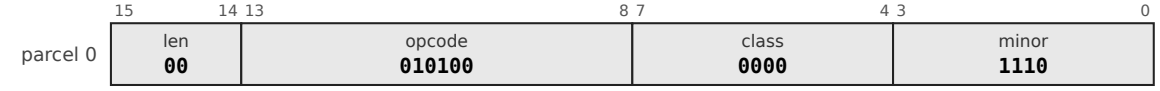
Operation, flags and exceptions as for the entry MFSR above; 2 layout forms, each a chart of its own (above).

MTSR

MTSR, opcode 010100, two parcels (4 bytes)



MTSR, opcode 010100, one parcel (2 bytes)




```
MTSR name, Rs
```

Mnemonic	Opcode	reg	low
MTSR	010100 (0x14)	0000 (0x0)	1110 (0xE)

Operation, flags and exceptions as for the entry MFSR above; 2 layout forms, each a chart of its own (above).

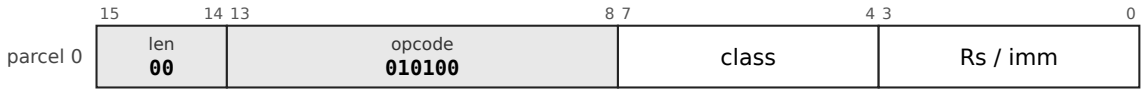
MTPS: Move to and from the processor status word

Mnemonics: MTPS, MFPS

Syntax

```
MTPS Rs
MFPS Rd
```

Encoding



The chart is the layout r20-sys-0, System, one parcel (len = 00): The reg field is the class; the other classes take a register or a 4-bit immediate from low.

Mnemonic	Opcode	reg
MTPS	010100 (0x14)	0001 (0x1)
MFPS	010100 (0x14)	0010 (0x2)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-sys-0, r20-sys-1, r20-sys-2.

Operation

```
MTPS: PSW ← Rs          ; in user mode only the flags N Z V C are written
MFPS: Rd ← PSW
```

Flags MTPS writes the flags from Rs; MFPS writes none.

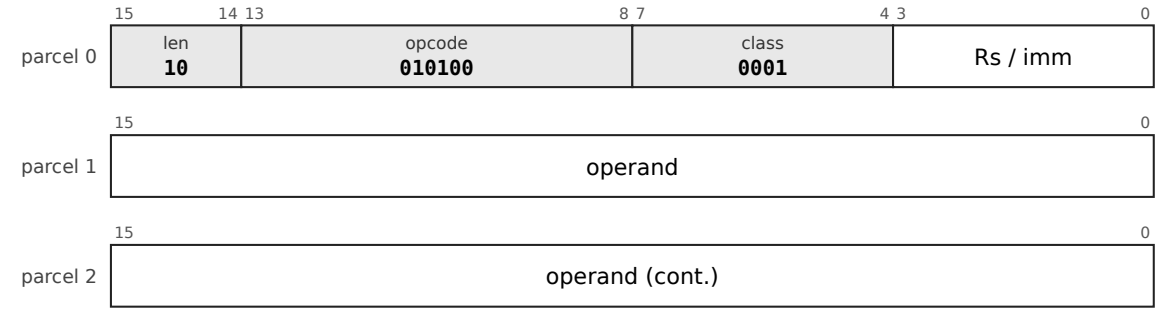
Exceptions A MTPS value with the reserved mode 2 in a mode field aborts.

Cf. 6.2 (modes and the status word).

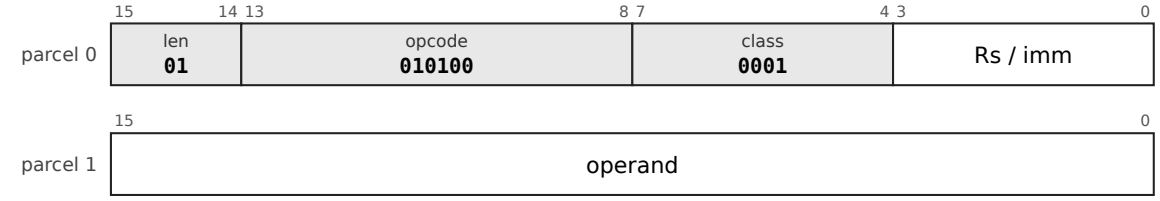
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

MTPS

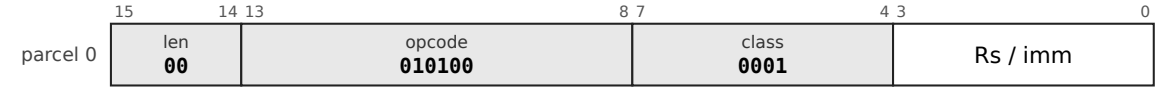
MTPS, opcode 010100, three parcels (6 bytes)



MTPS, opcode 010100, two parcels (4 bytes)



MTPS, opcode 010100, one parcel (2 bytes)



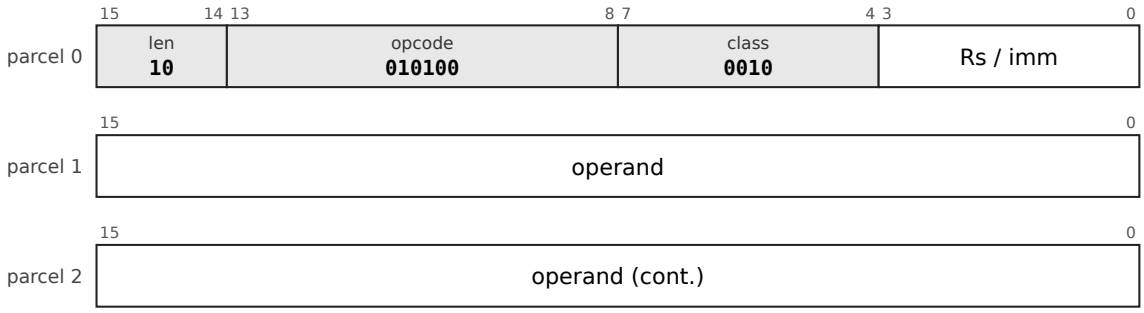

```
MTPS Rs
```

Mnemonic	Opcode	reg
MTPS	010100 (0x14)	0001 (0x1)

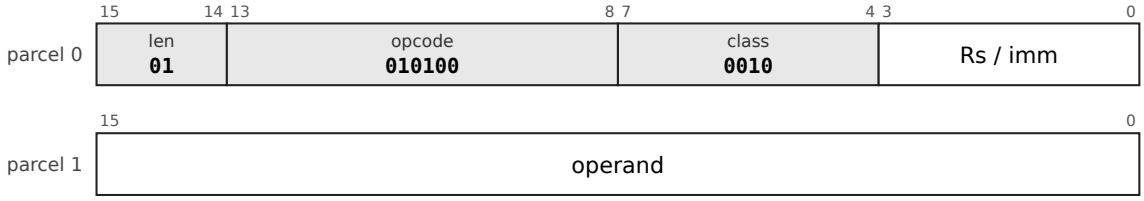
Operation, flags and exceptions as for the entry MTPS above; 3 layout forms, each a chart of its own (above).

MFPS

MFPS, opcode 010100, three parcels (6 bytes)



MFPS, opcode 010100, two parcels (4 bytes)



MFPS, opcode 010100, one parcel (2 bytes)



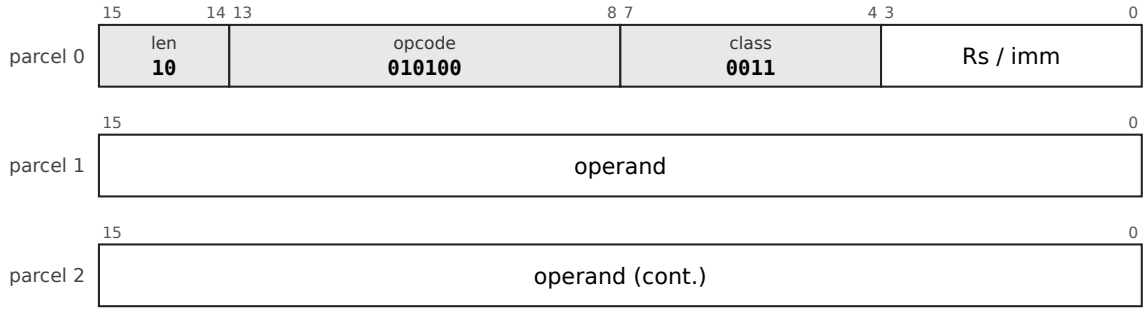

```
MFPS Rd
```

Mnemonic	Opcode	reg
MFPS	010100 (0x14)	0010 (0x2)

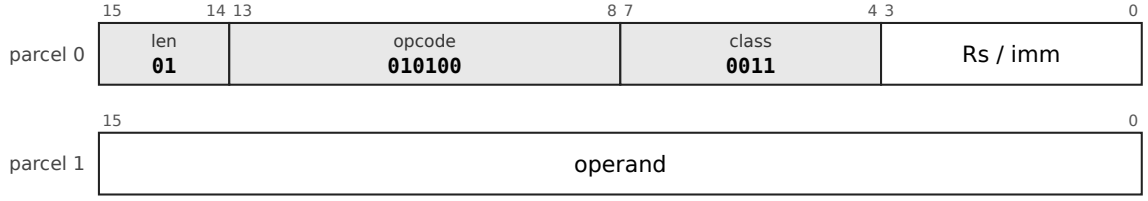
Operation, flags and exceptions as for the entry MTPS above; 3 layout forms, each a chart of its own (above).

SPL: Set the interrupt priority level

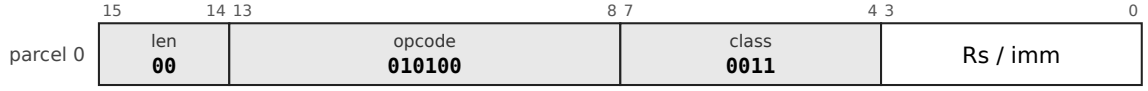
SPL, opcode 010100, three parcels (6 bytes)



SPL, opcode 010100, two parcels (4 bytes)



SPL, opcode 010100, one parcel (2 bytes)

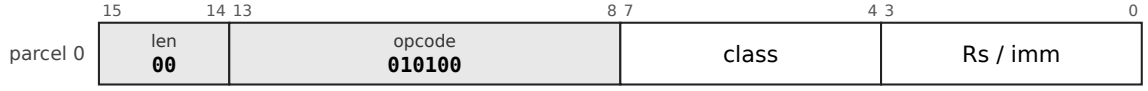


Mnemonics: `SPL`

Syntax

```
SPL #n        ; n = 0 to 7
```

Encoding



The chart is the layout `r20-sys-0`, System, one parcel (`len = 00`): The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	`reg`
`SPL`	`010100 (0x14)`	`0011 (0x3)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sys-0`, `r20-sys-1`, `r20-sys-2`.

Operation

```
PSW.priority ← n
```

Flags None written. The condition flags are untouched.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. A priority above 7 is an illegal instruction.

Kept despite MTPS writing the whole word because it sets the priority alone: a kernel raising priority around a critical section must not clobber the flags.

Why it is here. SPL stays although MTPS writes the whole status word: it sets the priority alone, leaving the condition codes untouched, so a kernel raising priority around a critical section does not clobber flags. It is the PDP-11 instruction of the same name.

Cf. 6.3.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

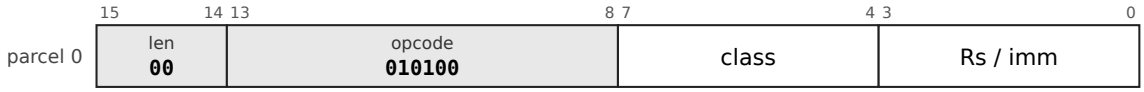
TRAP: System call and trap

Mnemonics: TRAP, TRAPL

Syntax

```
TRAP #n        ; n = 0 to 15, two bytes
TRAPL #n       ; n = 0 to 255, four bytes
```

Encoding



The chart is the layout r20-sys-0, System, one parcel (len = 00): The reg field is the class; the other classes take a register or a 4-bit immediate from low.

Mnemonic	Opcode	reg
TRAP	010100 (0x14)	0100 (0x4)
TRAPL	010100 (0x14)	1111 (0xF)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-sys-0, r20-sys-1, r20-sys-2, r20-sys-1-3.

Operation

```
enter the kernel by vector (TRAP_BASE + n), with EPC = the address of the next instruction
```

Flags None written.

Exceptions Enters the kernel; not privileged.

The replacement for the PDP-11 IOT and EMT. TRAPL reaches the vectors 0 to 255 with n in the high byte of parcel 1.

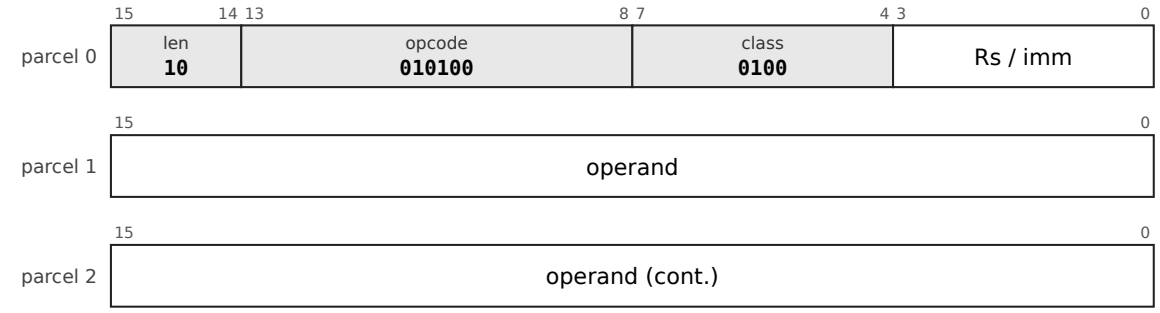
Why it is here. TRAPL exists so that the vector range of 256 and the two-byte common case can both be kept: TRAP #n for n below 16 is two bytes and the long form reaches 256. IOT and EMT, which were TRAP at two particular vectors on the PDP-11, are not provided.

Cf. 6.3, 6.10 (system calls).

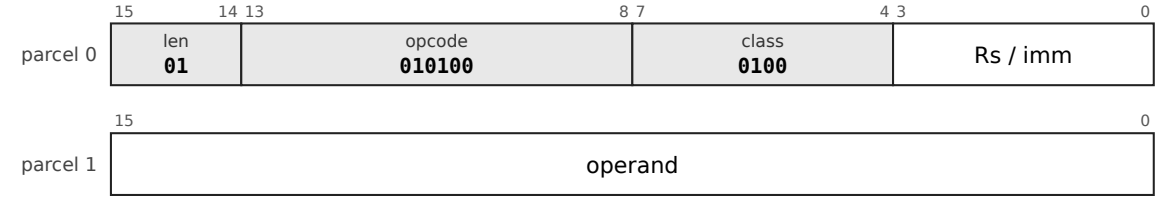
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

TRAP

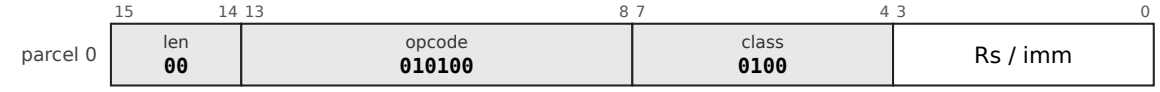
TRAP, opcode 010100, three parcels (6 bytes)



TRAP, opcode 010100, two parcels (4 bytes)



TRAP, opcode 010100, one parcel (2 bytes)



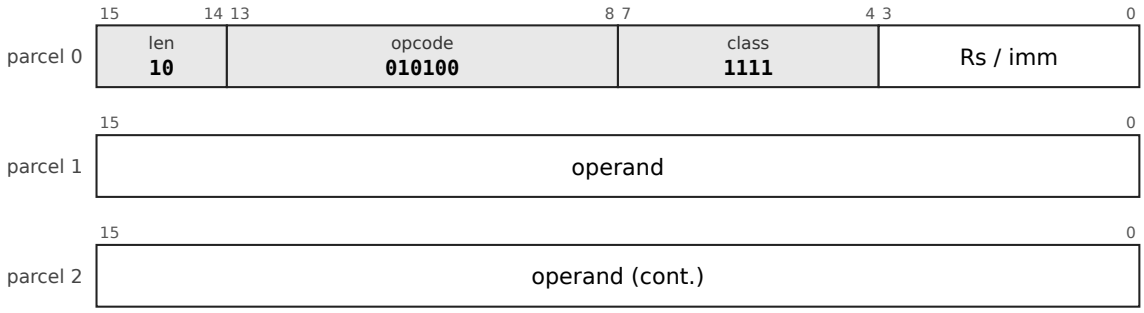

```
TRAP #n
```

Mnemonic	Opcode	reg
TRAP	010100 (0x14)	0100 (0x4)

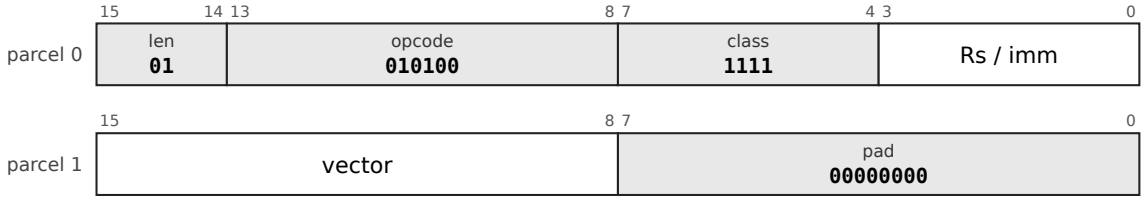
Operation, flags and exceptions as for the entry TRAP above; 3 layout forms, each a chart of its own (above).

TRAPL

TRAPL, opcode 010100, three parcels (6 bytes)



TRAPL, opcode 010100, two parcels (4 bytes)



TRAPL, opcode 010100, one parcel (2 bytes)



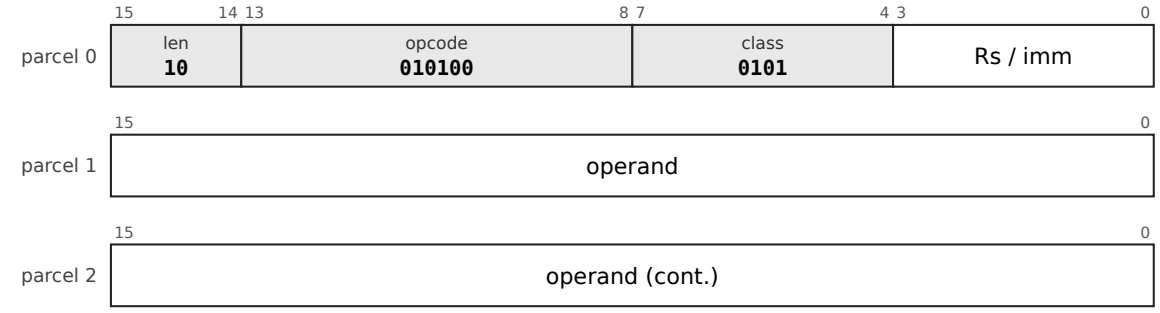

```
TRAPL #n
```

Mnemonic	Opcode	reg
TRAPL	010100 (0x14)	1111 (0xF)

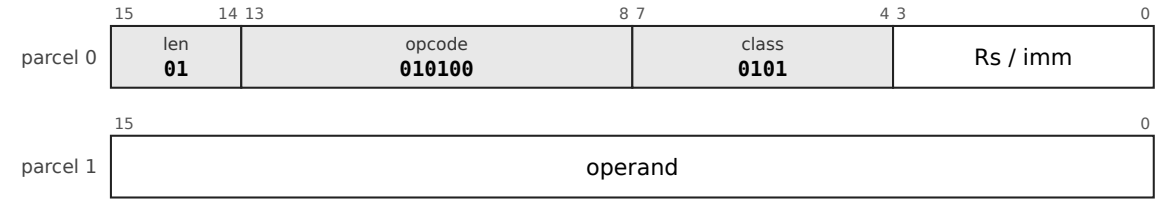
Operation, flags and exceptions as for the entry TRAP above; 3 layout forms, each a chart of its own (above).

TLBI: Invalidate a translation

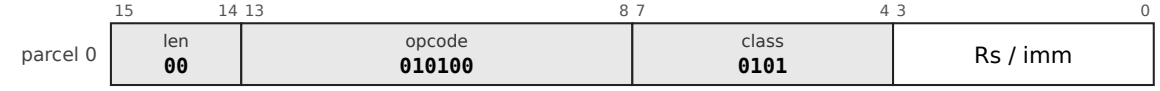
TLBI, opcode 010100, three parcels (6 bytes)



TLBI, opcode 010100, two parcels (4 bytes)



TLBI, opcode 010100, one parcel (2 bytes)

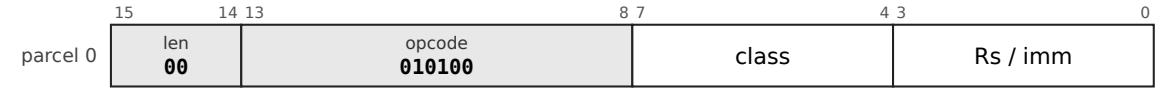


Mnemonics: `TLBI`

Syntax

```
TLBI Rs
```

Encoding



The chart is the layout `r20-sys-0`, System, one parcel (`len = 00`): The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	`reg`
`TLBI`	`010100 (0x14)`	`0101 (0x5)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sys-0`, `r20-sys-1`, `r20-sys-2`.

Operation

```
invalidate the translation-buffer entry for the page containing the address in Rs, on this core and by
broadcast on the others
```

Flags None written.
Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

Why it is here. It broadcasts: a translation changed by one core must be invalidated on all of them, and `TLBSYNC` waits for it.
Cf. 7.5, 6.8.
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

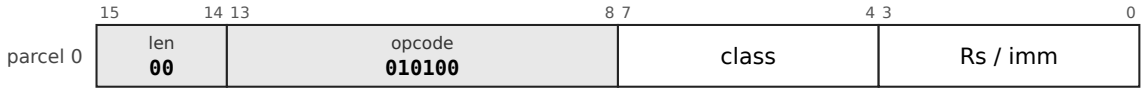
MTASID: Move to and from the address-space identifier

Mnemonics: `MTASID`, `MFASID`

Syntax

```
MTASID Rs
MFASID Rd
```

Encoding



The chart is the layout `r20-sys-0`, System, one parcel (`len = 00`): The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	`reg`
`MTASID`	`010100 (0x14)`	`0110 (0x6)`
`MFASID`	`010100 (0x14)`	`0111 (0x7)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sys-0`, `r20-sys-1`, `r20-sys-2`.

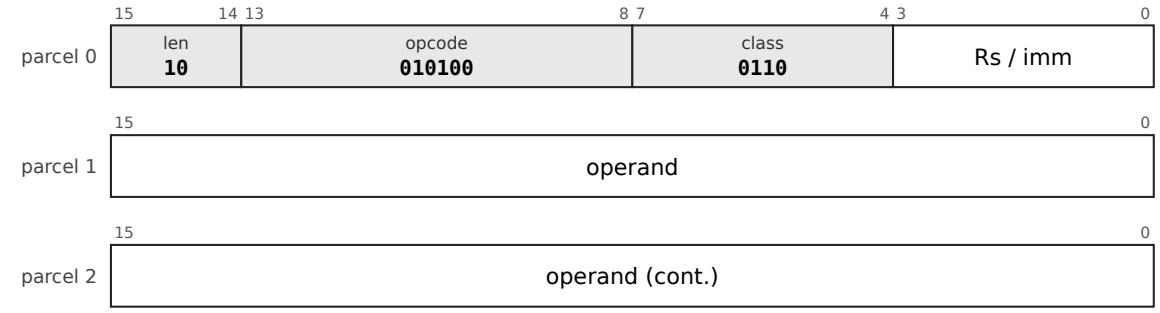
Operation

```
MTASID: ASID ← Rs[15:0]          ; the translation buffer is flushed in the reference executor
MFASID: Rd ← ASID
```

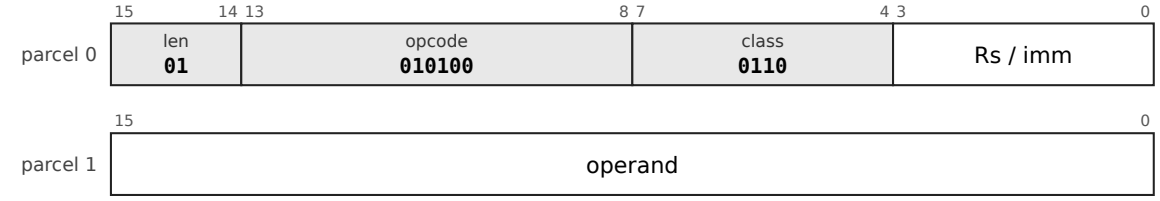
Flags None written.
Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.
The `ASID` tags translation-buffer entries so a context switch is one store.
Why it is here. The `ASID` tags translation-buffer entries so a context switch is one store and no flush. No further reason is recorded.
Cf. 6.8.
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MTASID

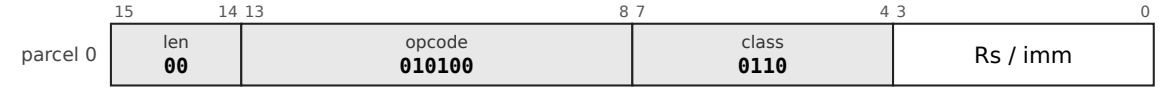
MTASID, opcode 010100, three parcels (6 bytes)



MTASID, opcode 010100, two parcels (4 bytes)



MTASID, opcode 010100, one parcel (2 bytes)



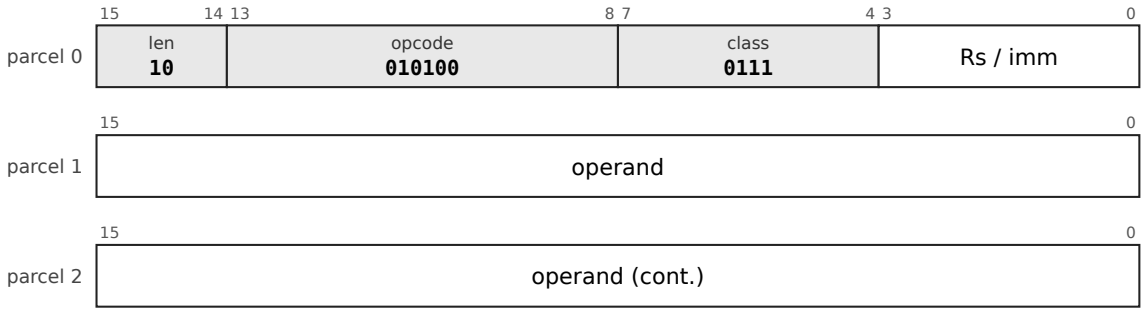

```
MTASID Rs
```

Mnemonic	Opcode	reg
MTASID	010100 (0x14)	0110 (0x6)

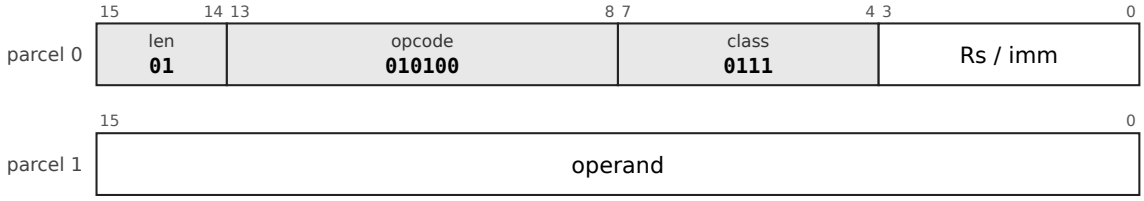
Operation, flags and exceptions as for the entry MTASID above; 3 layout forms, each a chart of its own (above).

MFASID

MFASID, opcode 010100, three parcels (6 bytes)



MFASID, opcode 010100, two parcels (4 bytes)



MFASID, opcode 010100, one parcel (2 bytes)




```
MFASID Rd
```

Mnemonic	Opcode	reg
MFASID	010100 (0x14)	0111 (0x7)

Operation, flags and exceptions as for the entry MTASID above; 3 layout forms, each a chart of its own (above).

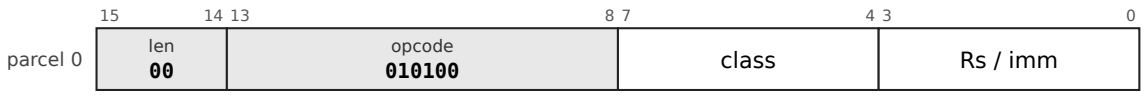
MTDP: Move to and from the direct pointer

Mnemonics: `MTDP`, `MFDP`

Syntax

```
MTDP Rs
MFDP Rd
```

Encoding



The chart is the layout `r20-sys-0`, System, one parcel (`len = 00`): The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	reg
MTDP	010100 (0x14)	1000 (0x8)
MFDP	010100 (0x14)	1001 (0x9)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sys-0`, `r20-sys-1`, `r20-sys-2`.

Operation

```
MTDP: DP ← Rs
MFDP: Rd ← DP
```

Flags None written.

Exceptions None.

`DP` is the thread pointer in user mode and the per-CPU pointer in kernel mode; there is one copy per mode.

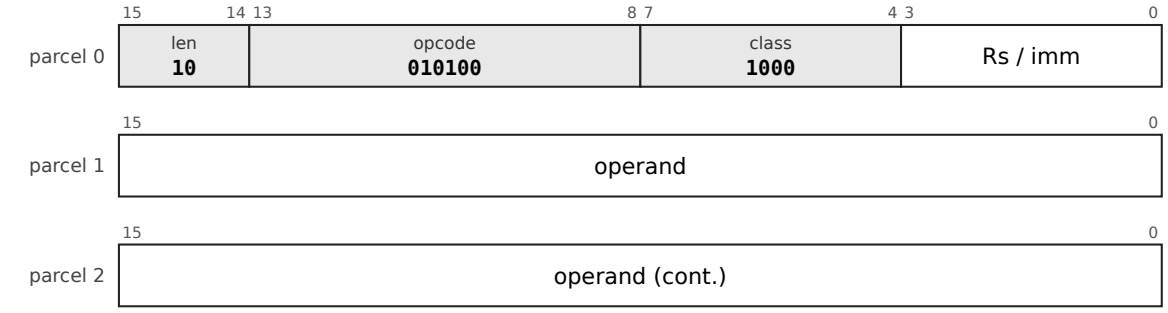
Why it is here. `DP` descends from the 6502's zero page and is kept for code density (a 12-bit displacement in four bytes); it is also the thread pointer in user mode and, in kernel mode, the per-CPU pointer that an SMP kernel wants (7.4). `MTDP` and `MFDP` read and write the current mode's copy.

Cf. 7.4 (per-CPU state).

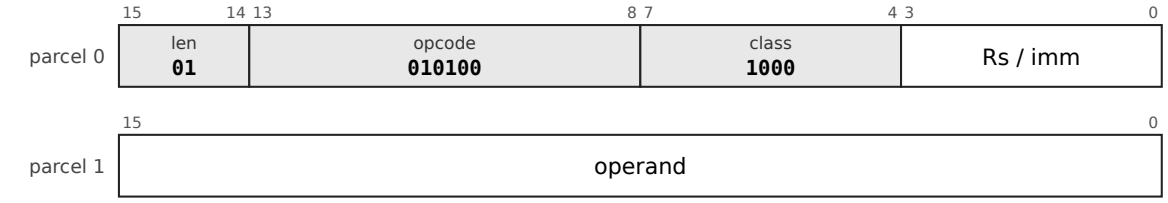
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MTDP

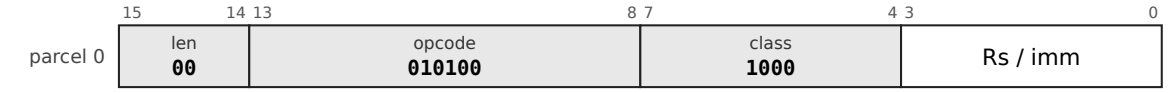
MTDP, opcode 010100, three parcels (6 bytes)



MTDP, opcode 010100, two parcels (4 bytes)



MTDP, opcode 010100, one parcel (2 bytes)



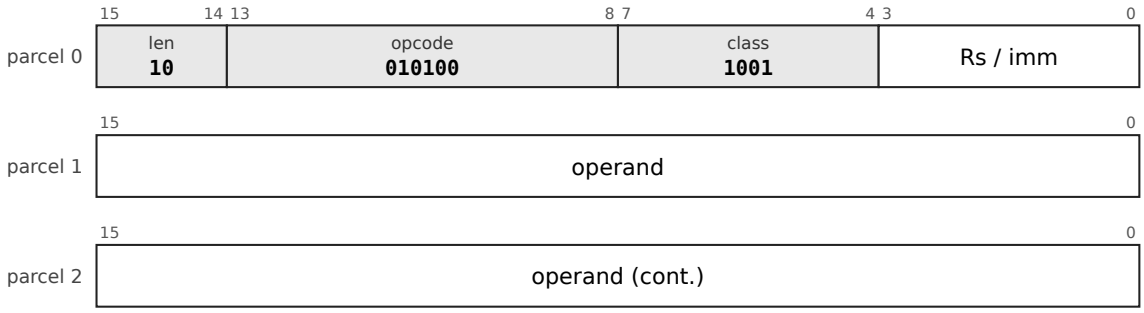

```
MTDP Rs
```

Mnemonic	Opcode	reg
MTDP	010100 (0x14)	1000 (0x8)

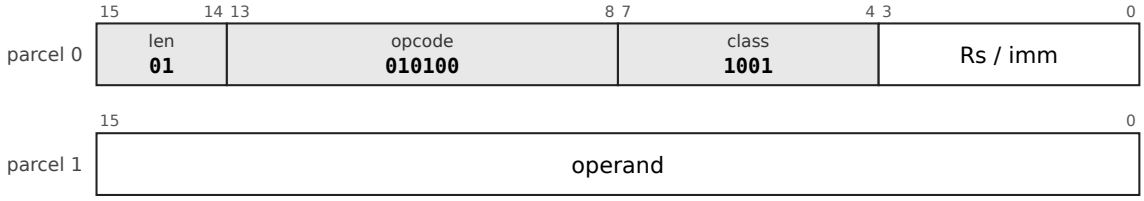
Operation, flags and exceptions as for the entry MTDP above; 3 layout forms, each a chart of its own (above).

MFDP

MFDP, opcode 010100, three parcels (6 bytes)



MFDP, opcode 010100, two parcels (4 bytes)



MFDP, opcode 010100, one parcel (2 bytes)



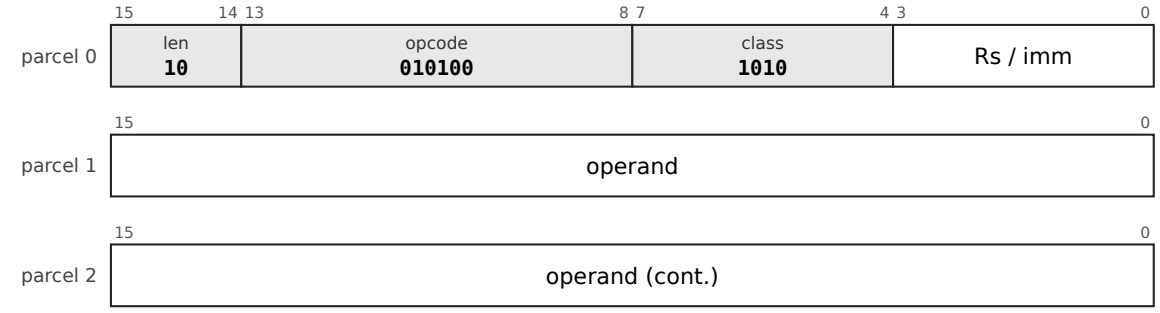

```
MFDP Rd
```

Mnemonic	Opcode	reg
MFDP	010100 (0x14)	1001 (0x9)

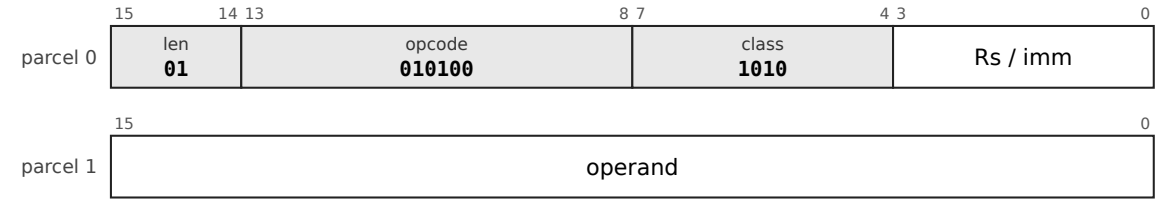
Operation, flags and exceptions as for the entry MTDP above; 3 layout forms, each a chart of its own (above).

MTDMW: Move to a direct-mapping window

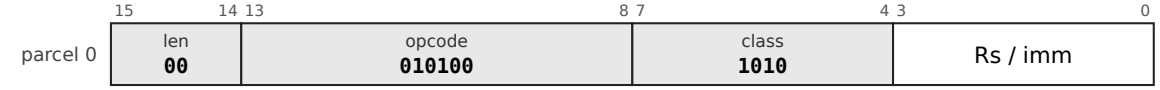
MTDMW, opcode 010100, three parcels (6 bytes)



MTDMW, opcode 010100, two parcels (4 bytes)



MTDMW, opcode 010100, one parcel (2 bytes)

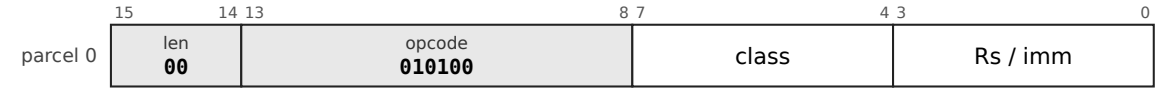


Mnemonics: `MTDMW`

Syntax

```
MTDMW Rs
```

Encoding



The chart is the layout `r20-sys-0`, System, one parcel (`len = 00`): The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	`reg`
`MTDMW`	`010100 (0x14)`	`1010 (0xA)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sys-0`, `r20-sys-1`, `r20-sys-2`.

Operation

```
DMW0 ← Rs          ; storage only: the operand names no window and the format of the windows is unspecified
```

Flags None written.

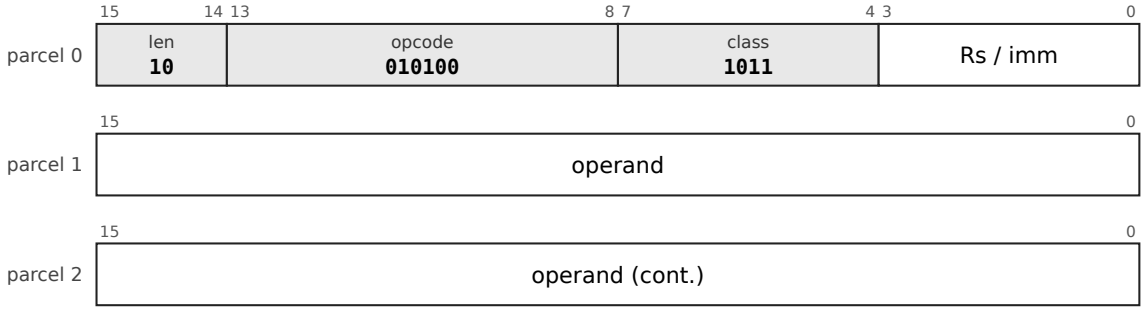
Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

The direct-mapping windows have no defined format.

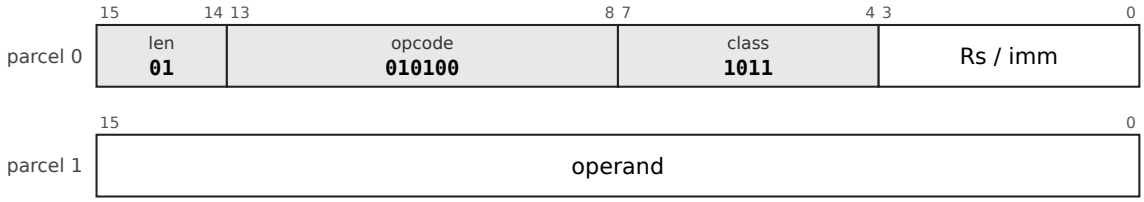
Why it is here. Taken from LoongArch's direct-mapping windows, which stop the kernel taking translation-buffer misses on its own text. The windows are deferred: the instruction is storage only.
Cf. 6.9.
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

MFCPUID: Read the processor identifier

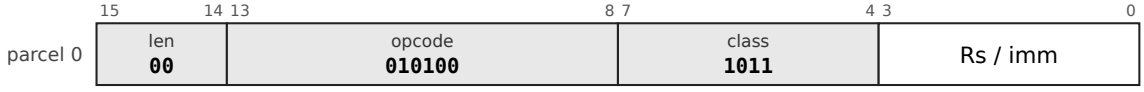
MFCPUID, opcode 010100, three parcels (6 bytes)



MFCPUID, opcode 010100, two parcels (4 bytes)



MFCPUID, opcode 010100, one parcel (2 bytes)

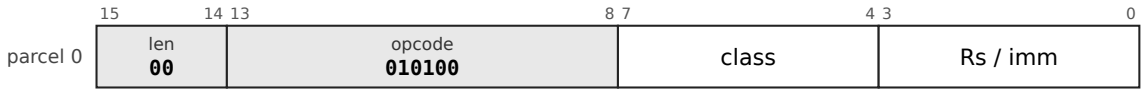


Mnemonics: `MFCPUID`

Syntax

```
MFCPUID Rd
```

Encoding



The chart is the layout `r20-sys-0`, System, one parcel (`len = 00`): The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	`reg`
`MFCPUID`	`010100 (0x14)`	`1011 (0xB)`

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `r20-sys-0`, `r20-sys-1`, `r20-sys-2`.

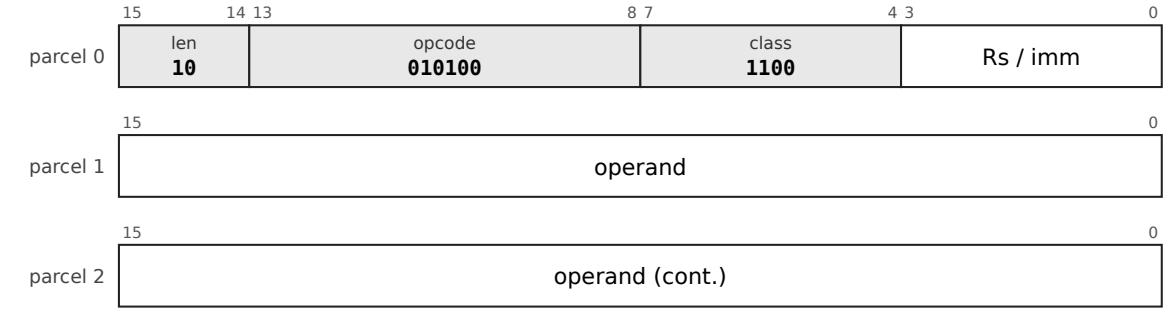
Operation

```
Rd ← CPUID
```

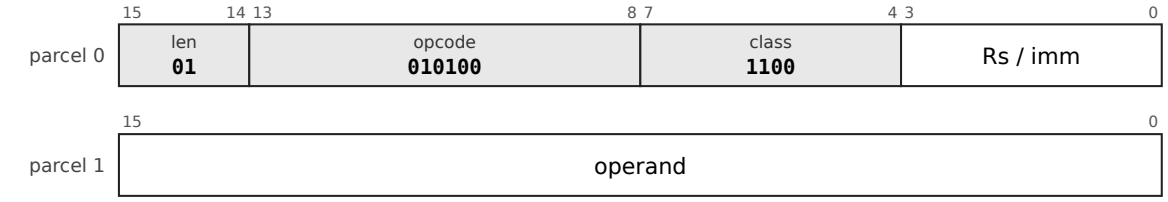
Flags None written.
Exceptions None.
The index of this core, for per-CPU data.
Why it is here. Part of the multiprocessor operations, because per-CPU data needs each core to know which one it is.
Cf. 7.4, 7.8.
Status implemented in the reference executor (`common/pdp64exec.c`, shared by `sim64` and QEMU).

ICINV: Invalidate the instruction cache for an address

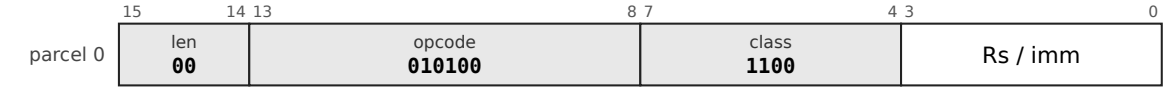
ICINV, opcode 010100, three parcels (6 bytes)



ICINV, opcode 010100, two parcels (4 bytes)



ICINV, opcode 010100, one parcel (2 bytes)

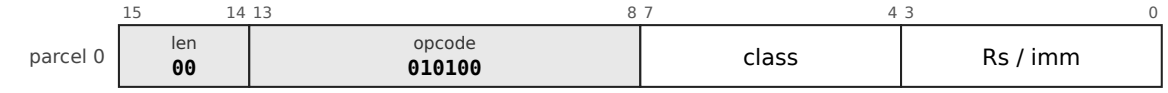


Mnemonics: `ICINV`

Syntax

```
ICINV Rs
```

Encoding



The chart is the layout `r20-sys-0`, System, one parcel (`len = 00`): The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	reg
ICINV	010100 (0x14)	1100 (0xC)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-sys-0, r20-sys-1, r20-sys-2.

Operation

```
invalidate the instruction-cache line containing the address in Rs
```

Flags None written.

Exceptions The reference executor treats it as a no-op; privilege as for the other cache-maintenance instructions.

Why it is here. Instruction-cache invalidation, by address (ICINV) and whole (ICINVA). The reason is the one given for TLBA: a runtime that compiles a function cannot work without it.

Cf. 7.5.

Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

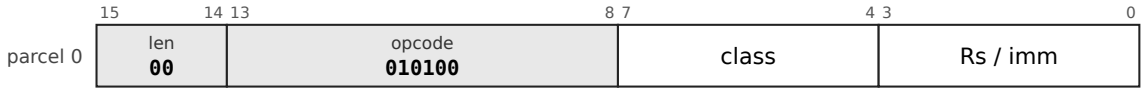
PROBE: Probe an address for read or write access

Mnemonics: PROBER, PROBEW

Syntax

```
PROBER Rd, Rs
PROBEW Rd, Rs
```

Encoding



The chart is the layout r20-sys-0, System, one parcel (len = 00): The reg field is the class; the other classes take a register or a 4-bit immediate from low.

Mnemonic	Opcode	reg
PROBER	010100 (0x14)	1101 (0xD)
PROBEW	010100 (0x14)	1110 (0xE)

All the layouts of this entry (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): r20-sys-0, r20-sys-1-2, r20-sys-2.

Operation

```
Rd ← 1 if the address in Rs is readable (PROBER) or writable (PROBEW) in the previous mode, else 0
no fault is taken
```

Flags None written.

Exceptions Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

Taken from the VAX PROBER and PROBEW: the kernel validates a user address before using it.

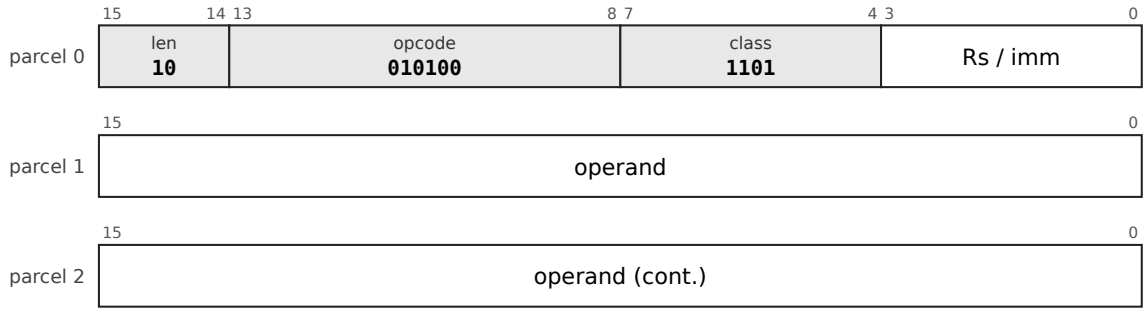
Why it is here. Taken from the VAX PROBER and PROBEW: the kernel validates a user address before using it, and it replaces the nofault global of the V7 kernel for the address-validation case.

Cf. 6.10 (the user-memory guard), 6.6.

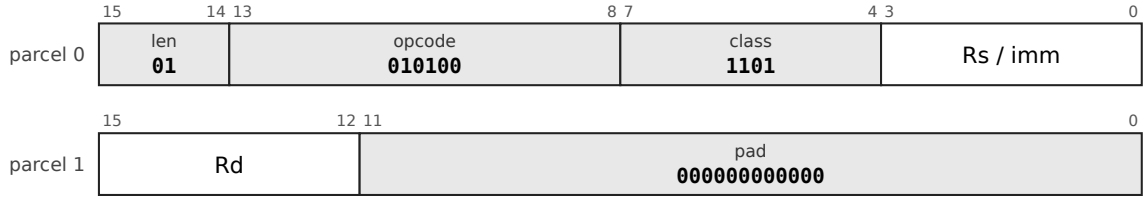
Status implemented in the reference executor (common/pdp64exec.c, shared by sim64 and QEMU).

PROBER

PROBER, opcode 010100, three parcels (6 bytes)



PROBER, opcode 010100, two parcels (4 bytes)



PROBER, opcode 010100, one parcel (2 bytes)



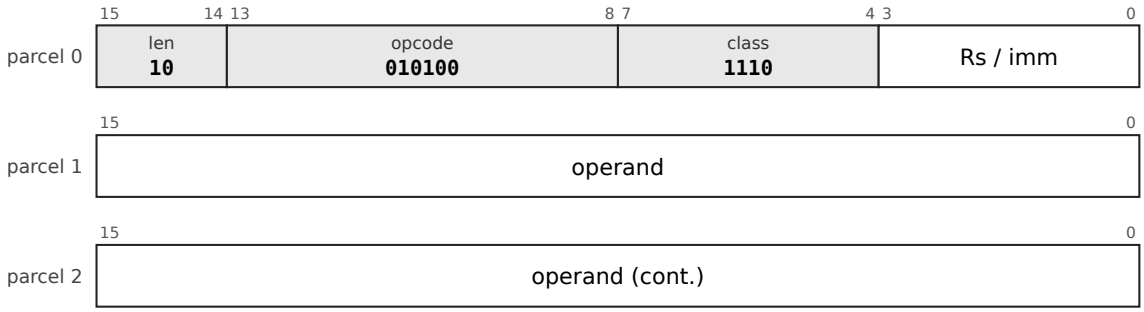

```
PROBER Rd, Rs
```

Mnemonic	Opcode	reg
PROBER	010100 (0x14)	1101 (0xD)

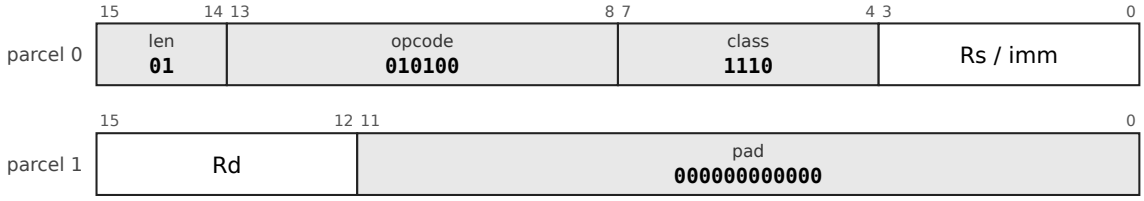
Operation, flags and exceptions as for the entry PROBE above; 3 layout forms, each a chart of its own (above).

PROBEW

PROBEW, opcode 010100, three parcels (6 bytes)



PROBEW, opcode 010100, two parcels (4 bytes)



PROBEW, opcode 010100, one parcel (2 bytes)




```
PROBEW Rd, Rs
```

Mnemonic	Opcode	reg
PROBEW	010100 (0x14)	1110 (0xE)

Operation, flags and exceptions as for the entry PROBE above; 3 layout forms, each a chart of its own (above).

9.15 Faults, restart and memory management

Cf. chapter 6, section 6.4 (faults and restart) and sections 6.6 to 6.9 (memory management).

9.15.1 A fault is a trap, an abort, a progress stop in a block operation or a machine check, and every instruction either completes or is annulled and restarted with no state to undo (MCPY and its kin keep their progress in registers); the page-table format, the memory types and the translation registers are in the kernel chapter. *(D)*

Commentary. The instruction-set consequence is the restart rule: the register update of an auto-increment mode is held until the instruction retires. **Impact.** *Kernel:* see 6.4. *Compiler:* no scheduling restriction arises from faults. *Hardware:* a single-entry undo buffer and a read-only walker.

9.16 Position-independent code

9.16.1 Position-independent code costs nothing extra on this machine: every access to code or data in the same module is PC-relative by an ordinary addressing mode, so there is no GOT register and no prologue to set one up. *(D)*

9.16.2 A call to an imported function is `CALLI got_slot(PC)` and a PLT stub is the single instruction `JMPI got_slot(PC)`; an imported variable's address is one `MOVQ got_slot(PC), Rd`. *(D)*

9.16.3 A position-independent access to a global is no longer than an absolute one: four bytes within 32 KB and six within 2 GB, against six for a 32-bit absolute address. *(D)*

9.16.4 The thread pointer is DP, so the local-exec thread-local model is one four-byte `MOVQ tls_offset(DP), Rd`. *(D)*

9.16.5 A branch at the middle length reaches 4 GB, so no linker veneers are needed for any binary smaller than that; the assembler's and the linker's branch relaxation both reduce to choosing a length. *(D)*

9.16.6 `R12` is the linker's scratch register. The ABI (appendix D) decides that imports are called without a PLT stub; lazy against eager binding is open. *(D for `R12` and no stub; open for binding)*

Commentary. **Precedent.** On i386 a position-independent executable reserved `%ebx` (one of eight registers) for the GOT base and set it in every prologue, which cost a few percent of size and speed; x86-64 fixed that with RIP-relative addressing, and AArch64 needs linker veneers because its branches reach only 128 MB. **Trade-offs.** The two-byte PC-relative form is the shortest, so position-independent data access is slightly smaller than absolute access; the price is a GOT load per imported call site where a PLT stub would cost a load per function. **Improvements.** Position-independence is a side effect of three decisions made for other reasons: PC-relative as an ordinary mode (adopted because the PC left the register file), variable-length encoding (for density) and `DP` (a 6502 zero-page descendant kept for density), which is exactly the thread pointer. **Benefits.** OpenBSD builds everything position-independent by default, and here that costs nothing. **Modern comparables.** x86-64 RIP-relative addressing; AArch64 `ADRP` with a GOT load and veneers for far calls. **Impact.** *Kernel:* a loader applies only relocations for data pointers, and write-xor-execute and randomisation come with no code penalty. *Compiler:* `-fPIE` is the default and emits the same instructions as `-fno-pic` for local data; no reserved register, no per-function setup. *Hardware:* nothing; the PC-relative mode is in the address adder already.

9.17 What was rejected

9.17.1 The architecture refuses, by design, every feature that adds hidden state, an unbounded instruction, a second operand path or a mode bit that changes what other instructions mean. *(D)*

9.17.2 From the PDP-11 and VAX it refuses general memory-to-memory operands, deferred addressing as a general form (only `JMPI` and `CALLI` are deferred), unbounded instructions with hidden microstate (`MOVC5`, `EDITPC`), packed decimal, hardware call frames (`CALLS`, `CALLG`), the PC as a general register, multiple floating-point formats, `POLY`, the `IOT`, `EMT`, `WAIT` and `RESET` instructions, a distinct low-half unsigned multiply, and the PDP-11's compare operand order. *(D)*

9.17.3 From x86 it refuses partial-register writes that preserve upper bits, operand-size and `REX` prefixes, segmentation, flag writes that vary by instruction, implicit registers for multiply, divide and shift count, the x87 stack, transcendental instructions and byte-granular instruction fetch. *(D)*

9.17.4 From the RISC family it refuses a register hardwired to zero (a sixteenth of a 16-register file), fused compare-and-branch in place of flags, branch delay slots, register windows and software-refilled translation buffers. *(D)*

9.17.5 It also refuses features of its own invention: NUL-terminated string instructions (`MSCN` with an explicit maximum gives a bounded `strlen`), `CRC32` words (carry-less multiply is the primitive under every CRC), reciprocal-estimate instructions, NaN payload propagation, a 32-bit profile, an eight-byte instruction alignment, `MOVT` and `MOVK` partial-immediate inserts, hardware tag checking, store-with-write-barrier, a self-synchronising encoding, and a scalar instruction for sub-1-bit inference (the accelerator of chapter 8 does it). *(D)*

Commentary. **Precedent.** The VAX's `EDITPC` is a bytecode interpreter in microcode, and DEC needed a separate control chip for the PDP-11's commercial instruction set and shipped none; the VAX's hardware call frames ran slower than the equivalent sequence on later models; ARM32's `PC+8` and MIPS's delay slot expose one implementation's pipeline forever. **Trade-offs.** Each refusal gives up a real convenience: the lost `MOV (R1)+,(R2)+` copy loop, the byte-wide `ADDB`, the inline-argument `JSR`. Section 9.18 says what replaces each. **Improvements.** The common pattern is that a refused feature was either subsumed by a more general one (`MCPY` for the copy loop) or paid its cost in every implementation for a few users (`EDITPC`). **Benefits.** An instruction set a small in-order core can implement completely and a compiler can target without exceptions. **Modern comparables.** RISC-V and AArch64 also refuse memory-to-memory operands and delay slots; AArch64 keeps flags where RISC-V refuses them. **Impact.** *Kernel:* no instruction has a restart state except the three that keep their progress in architectural registers, so a fault is always restartable. *Compiler:* no special cases: every operand rule is uniform, one memory operand and the rest registers. *Hardware:* no microcode engine, no second operand path, no hidden-state latch, no alignment rotator in the fetch path.

9.18 Idioms: restored, lost and replaced

9.18.1 Several idioms that the PDP-11 and VAX had and later RISC designs dropped are restored: subtract-one-and-branch (`SOB`, here in both directions), probe an address without faulting (`PROBER`, `PROBEW`), bit-field extract and insert, find-first-set (`CTZ`,

clz), byte swap, block move, fill, compare and scan with their progress held in registers, the register-save mask (pushm, popm, the useful half of calls), and carry-less multiply. *(D)*

9.18.2 The VAX's counted-string instructions are largely present: movc3 is mcpy, cmpc3 is mcmp, locc is mscn, skpc is mscnn, and movc5 is mcpy plus mset. Table-translated move, table scans and substring search are absent because a per-byte table lookup stops a block operation from streaming. *(D)*

9.18.3 Deliberately lost, with their replacements: memory-to-memory mov (replaced by mcpy); autoincrement-deferred and the dispatch-table idiom (replaced by jtag for tag dispatch and a scaled-index jmp for the rest); jsr Rn and rts Rn with inline arguments and cheap coroutines (replaced by call and ret with the return address on the stack where every unwinder and debugger expects it); the VAX case and index instructions (replaced by a compare, branch and scaled jmp); byte and word ALU operations (movzb, the operation, movb: three instructions where the PDP-11 had one); and the flag set and clear instructions. *(D)*

9.18.4 Subsumed, so not losses: sxt (the conversion moves), swab (bswap), mark (the convention it served is gone), ash and ashc (the shifts take a register count), mfpi and mtpi (prober with the previous-mode bits and the user-memory guard), mulu (identical to mul in the low half), and rol (an assembler alias for ror). *(D)*

9.18.5 Measurements decide what is included: the compiler uses set cc, rd (about 0.4% of code), not integer abs, min or max (under 0.1%), not the bit-field instructions (0.03%) and not the auto-increment modes (0.1%); the second opcode page is about 7% of the instructions emitted. *(D)*

Commentary. **Precedent.** The PDP-11's jsr Rn put the return address in a register of the caller's choosing, which made coroutines a pair of jsrs and inline arguments free; the VAX replaced it with calls and a frame, which was slower than the code it replaced. **Trade-offs.** The most frequently felt loss in hand-written code is the byte-wide read-modify-write; the compiler never notices it. A bounds-checked index for Pascal and Ada is a possible addition. **Improvements.** sob branches either way, with four reaches; the block operations keep their progress in registers, which is what the VAX got wrong, so a fault restarts for free. **Benefits.** The code a compiler emits is the code the table above measures, and the extras are for the people who write assembler. **Modern comparables.** x86 keeps loop and the string instructions with rep; AArch64 has bit-field moves and no memory-to-memory forms; RISC-V has none of them in the base. **Impact.** *Kernel:* prober and probew replace the nofault global in the user-copy routines. *Compiler:* idioms the compiler never emits are not built for it, so the compiler's instruction selection stays small. *Hardware:* each restored idiom is a few hundred gates of existing datapath, and each lost one is a decode that never existed.

9.19 Side by side: the PDP-11, the VAX and the pdp64

9.19.1 The word is 64 bits against the PDP-11's 16 and the VAX's 32, the registers are 16 as on the VAX, the virtual address is 39, 48 or 57 bits with 40 physical in the present implementation, and the instruction count is 58 assigned opcodes plus 45 on the second page, against the VAX's 304. *(D)*

9.19.2 The program counter is not a register, there is no argument pointer, and the frame pointer and the linker scratch are ordinary software conventions. *(D)*

9.19.3 An instruction has one or two operands (three for the block operations) and at most one memory operand, against the PDP-11's two and the VAX's six. *(D)*

9.19.4 The condition codes are the same four, n, z, v and c; floating point is in the base and in the general registers, with three IEEE formats (half, single, double) and no mode bits that change the meaning of other instructions, where the PDP-11 had fps.fd, fl and ft and the VAX had rounding bits in the status word. *(D)*

9.19.5 An instruction is two, four or six bytes (one to three parcels of 16 bits) with a one-parcel second page, against the PDP-11's two to six and the VAX's one to more than fifty. *(D)*

Commentary. **Precedent.** The PDP-11 and VAX instruction formats, with their encodings drawn side by side, are in the instruction-set text, section 13. **Trade-offs.** The pdp64 keeps what made the PDP-11 pleasant, orthogonal addressing and a small register set, and what made the VAX powerful, flags, a register-save mask and string operations, and drops what made both expensive: memory-to-memory operands and unbounded instructions. **Improvements.** The compiler's code averages about 3.3 bytes an instruction, measured on the fair corpus before the second page. **Benefits.** A processor that is smaller than either predecessor while running 64-bit code. **Modern comparables.** The closest family is AArch64 in the number of registers and addressing modes and RISC-V compressed in density; neither has flags and a variable length

together. **Impact.** *Kernel:* the exception model is the PDP-11's with the VAX's per-mode stacks. *Compiler:* one memory operand makes instruction selection a register-allocation problem, as on a RISC. *Hardware:* a decoder that reads two bits to find the length, and no microcode.

9.20 What was taken from LoongArch, and what was not

9.20.1 LoongArch64 is the most recent clean-room instruction set designed without an ideological commitment: it borrowed from MIPS, RISC-V and ARM as each had the better answer, which is the posture of the pdp64.

9.20.2 Adopted from it: bit manipulation in the base (`CLO`, `CLZ`, `CTZ`, bit-field moves, byte reverse), atomics with the barrier folded in (the `_DB` variants), a richer atomic set (fetch-and-minimum and maximum), and memory type as a page attribute. *(D)*

9.20.3 Declined, with reasons: `R0` hardwired to zero (a sixteenth of a 16-register file, and `CLR` and `NEG` are one parcel already); no condition codes (a third read port in the branch unit against a four-bit latch); `FCSR` rounding-mode bits (hidden state that changes other instructions); four PC-relative address-formation instructions (one `LEA` carries 32 bits); 32 registers (this encoding has no bit to spare for the field); and separate vector register files (chapter 8). *(D)*

9.20.4 Under consideration: separate floating-point condition flags, software-visible translation-buffer manipulation (to pin an entry for a real-time kernel), and a conditional select for constant-time code. *(P)*

Commentary. **Precedent.** LoongArch has a fixed 32-bit encoding with room for two register fields and a displacement, so it can afford compare-and-branch and needs four instructions to form an address; this machine's variable length reverses both trade-offs. **Trade-offs.** The direct-mapping windows LoongArch uses to avoid translation-buffer misses on kernel text are deferred: `DMW0` and `DMW1` are stored without effect (6.9.2). **Improvements.** The same idea is right or wrong depending on the encoding around it, and each adoption is judged against this encoding. **Benefits.** Borrowing the tested parts saves design time and avoids their bugs. **Modern comparables.** LoongArch itself; the RISC-V bit-manipulation extensions (Zbb, Zbs) do much of the same. **Impact.** *Kernel:* atomics with barriers make a lock acquire one instruction. *Compiler:* bit operations in the base mean the compiler can use them without a feature test. *Hardware:* each adopted idea is a few hundred lookup tables and each declined one is a block not built.

9.21 What is still open in the instruction set

9.21.1 Open, with the reasoning in appendix B or in the instruction-set text, section 12: the thin 6-bit displacement of the shortest scaled-index form; the landing-pad decision (optional first, mandatory later); atomics against the bus (the lock against the monitor; the monitor is chosen); worst-case instruction latency for the block operations (an interrupt-latency claim must state the block granularity); the floating-point condition flags (the signed branches do not treat unordered cleanly); the block operations and the cache (a streaming memory type or a non-temporal variant); directed rounding for arithmetic (a rounding field in the instruction); the leftovers of half precision (minimum, maximum, rounding, bfloat16); conditional select; lazy against eager binding in the ABI; the verification of the memory model against published litmus tests; and the coherence protocol beyond four cores (the design supports at least four and prefers sixteen or more, errata 92; the leanings are in `docs/smp-decisions.md`). *(P and open)*

9.21.2 Settled: the reserved-opcode discipline (a top-level opcode is for an operation whose common case is register to register), the fourth parcel (absent; the code `11` is a second page of 64 one-parcel opcodes), UTF-8 scope (`MVAL` is committed), and the measured length distribution (2, 4, 6 and 8 bytes at 46.5, 42.8, 10.6 and 0.1 percent, before the second page). *(D)*

Commentary. **Trade-offs.** None of the open items blocks the first implementation; the three that must be settled before silicon, the memory model verification, the coherence choice and the worst-case latency figure, are verification tasks, not design changes. **Impact.** *Kernel:* the open items touching the kernel are the user-memory guard, the floating-point state and the block-operation granularity. *Compiler:* conditional select and directed rounding would change instruction selection. *Hardware:* the coherence protocol must be chosen before the cache is built, not after.

Chapter 10 — Conclusion: from FPGA to silicon

The pdp64 is built to be run first on a modest FPGA, to be small enough that its whole core is about sixteen thousand lookup tables or ninety thousand transistors, and to reach silicon only after an OpenBSD built by the project's own toolchain can be logged into over `ssh`. The chapter gives the FPGA plan and its numbers, the transistor budget for a chip, why the first implementations are in order and how the architecture is kept ready for a faster one, and the path from the present toolchain to the finish line. Tags: *(D)* decided; *(E)* an estimate, not a measurement.

10.1 The FPGA plan

10.1.1 The first implementation is a soft core on a system-on-chip FPGA whose hard ARM cores serve as the bring-up harness: they load programs, run the tests and provide the console while the pdp64 core is still being debugged. *(D)*

10.1.2 The first board is the Terasic DE10-Nano (Cyclone V SE: about 110,000 logic elements, dual Cortex-A9, 1 GB of memory on the hard processor, HDMI, and the MiSTer ecosystem); the Digilent Zybo Z7-20 or the AMD Kria KV260 are the move when timing becomes the limit; the open-toolchain ULX3S is kept in view but loses the hard-core harness. The device chat staged devices first on a Zynq UltraScale+ (`decisions-pending.md` E6). *(P)*

10.1.3 A complete core is about 16,000 six-input lookup tables, 10 to 16 multiplier blocks and 11 block RAMs, with an error of about ±40%: the register file is 250 lookup tables of distributed RAM, the 64-bit ALU 600, the barrel shifter 450, the caches 300 and 500 plus 8 block RAMs, the bus bridge 600. *(E)*

10.1.4 Block RAM, not logic, is the scarce resource: the caches, the translation buffer, the framebuffer and the blitter scratch compete for about 5.5 megabits. *(E)*

10.1.5 The clock the FPGA design aims at is 200 to 300 MHz. *(D)* The estimated clock is 50 to 80 MHz on the Cyclone V, 90 to 130 MHz on a Zynq 7020 and 180 to 250 MHz on an UltraScale+; every one of those estimates is below the target except the upper part of the UltraScale+ range, and reaching the target means pipeline stages in the datapath. *(E)* A correct machine at a lower clock still comes before a fast broken one.

10.1.6 The work is staged in phases, each ending in a demonstration and a regression suite: specification and tools; an integer core; exceptions; the memory-management unit; the memory system; floating point; a booting kernel; peripherals; and, last, the other cores: the design supports at least four, and the initial device has four. *(D)*

10.1.7 Phase 2, the fault model, is not optional and does not move; the block operations are built with it, because their whole specification is about what state is visible when an exception arrives. *(D)*

10.1.8 On the paper estimates of the SMP decisions, four cores with the floating-point unit fit only a board of the Kria KV260 class, and sixteen cores fit none of the boards of the plan; no core has been synthesised. *(E)*

Commentary. **Precedent.** The PDP-11's own microprocessor, the T-11, was about 12,000 transistors, a complete integer core without a memory-management unit or floating point; the J-11 that implemented the full PDP-11/70 was about 120,000. The MicroVAX 78032 was 125,000 transistors for a subset of the VAX, with 175 of its 304 instructions and no floating point. **Trade-offs.** The 64-bit carry chain is slow on an FPGA and gates the clock more than it would in silicon, the barrel shifter is a large multiplexer tree, and flags serialise branch resolution behind the ALU; none of that argues for changing the instruction set, only for where the pipeline registers go, for example a virtually indexed, physically tagged first-level cache so that the translation buffer is off the critical path. A multiplier built from two to four multiplier blocks over four to eight cycles closes timing where a full combinational one will not. **Improvements.** The register-transfer description is kept vendor-neutral, with every vendor primitive in one thin directory (under 500 lines), so that moving between boards costs that directory and nothing else. **Benefits.** A single engineer working part-time can reach a booting system with floating point in roughly 10 to 19 months, by the plan's wide-error-bar estimate, with phases 2 and 3 the ones that historically double. **Modern comparables.** The RISC-V soft cores Rocket and VexRiscv are the usual FPGA reference points; the plan treats the compiler back end as the schedule risk, as it is for any new architecture.

10.2 The chip: a transistor budget

10.2.1 For a non-pipelined or short-pipeline implementation in a modern process, the whole core excluding caches is estimated at about 92,000 transistors. *(E)*

Block	Transistors (E)
register file, 16 × 64, two read ports and one write	~6,000
ALU, shifter, address adder	~15,000
decode and sequencer	~6,500
bit unit (popcount tree, count zeros, bitfields)	~3,500
block-move sequencer	~1,500
UTF-8 decode and validity	~1,200
translation buffer (32 entries), walker, window compare	~19,000
floating-point unit with fused multiply-add, classify, min and max	~34,000
control, status word, exceptions, undo buffer	~5,500
total, excluding caches	**~92,000**

10.2.2 The bit unit and the UTF-8 decoder together cost less than 5% of the die; the fused multiply-add datapath is the largest single addition and the least negotiable. *(E)*

Commentary. **Precedent.** The figure sits between the T-11 and the MicroVAX: it is a 64-bit machine with floating point and memory management below the 78032's 125,000 transistors, which had neither. **Trade-offs.** The estimate is not a measurement and excludes caches, which dominate the area of a real chip; an implementation of the translation buffer as a content-addressable memory is larger than the figure assumes, which is why the FPGA build uses a four-way set-associative one. **Benefits.** A chip this size fits in the budget of a small process node with room for caches and the bus controller. **Modern comparables.** The ARM1 of 1985 was about 25,000 transistors without a memory-management unit or floating point; the Intel 80386 about 275,000.

10.3 In order first, ready for out of order

10.3.1 The first implementations, on the FPGA and in silicon, are small in-order cores without speculation; the architecture is kept ready for a large out-of-order design. *(D)*

10.3.2 The condition flags were the structural obstacle: 68.2% of compiled instructions wrote a flag and only 1.5% read one, so the flags form two groups that nothing writes in part, and moves, loads, stores and the compare-and-branch instructions write none. *(D)*

10.3.3 An in-order, non-speculative core has none of the speculative side channels of the last decade, and an out-of-order core must keep that property; the rules that keep a future faster core from repeating them are collected in the hardware-vulnerabilities report, and include no speculation across protection domains, no shared predictor state across privilege boundaries without a flush, and operand-independent timing for a declared set of instructions. *(D as a design aim; the rules are proposals)*

10.3.4 Sequential consistency is nearly free on a small in-order core and is the most expensive memory model for a large speculative one; if a large core is built it may be necessary to move to total store order, which is the least painful retreat. *(P)*

Commentary. **Precedent.** One instruction set has often spanned in-order and out-of-order cores: the ARM small cores beside the large ones, the RISC-V Rocket beside the BOOM, x86 from the Pentium to the P6, Alpha from the 21064 to the 21264. The VAX's long, orthogonal instructions with many side effects are generally blamed for making its later fast implementations so hard. **Trade-offs.** Readiness has a cost, visible here as the compiled-code size paid for the flag rules (0.39%) and as design effort spent on a core that may never be built. **Improvements.** The pain-points report finds that the pdp64 avoids the largest classes of kernel pain by design (barriers, floating-point state, a faulting entry, remote translation flushes, the speculative family) and is exposed in a few places: a single page-table base if a core ever speculates, software accessed and dirty bits, uncached DMA buffers, and no specified constant-time or Rowhammer protection. **Benefits.** Any of these can be addressed in the specification while it is still being written, which is cheaper than after silicon. **Modern comparables.** The common vulnerability-driven changes of recent years, page-table isolation and speculation barriers, are what a speculative core of this architecture would have to add.

10.4 The finish line, and the path to it

10.4.1 Before silicon the project delivers five things: an assembler, a linker, an LLVM compiler back end, an emulator, and a port of OpenBSD that can be logged into with `ssh`. *(D)*

10.4.2 The architecture manual, which includes the instruction set but covers the whole machine, is the primary deliverable; the toolchain is advanced as needed to write and check it. *(D)*

10.4.3 The assembler and linker exist; the assembler lacks the atomics and the linker lacks shared libraries; the pcc compiler back end passes 516 C tests on the simulator and on QEMU; the LLVM back end has its machine-code layer, checked against the project's own assembler on 62 programs and 6,304 instruction forms with no disagreement; the simulator and QEMU run a kernel-mode machine with exceptions, a memory-management unit, a timer and a console; no part of OpenBSD is ported. *(measured)*

10.4.4 The long pole is the LLVM back end and clang. OpenBSD's kernel and base cannot be built by pcc, and the fallback of porting Unix/32V with pcc would not reach the goal, because it has no network stack and no `ssh`. *(D)*

10.4.5 The path, in order, with a test for each step: finish the toolchain gaps (atomics, the user-memory guard, the ABI version); an LLVM back end that passes the C tests; clang, the compiler runtime and `lld`; a machine model for the operating system with an enumerating bus, a disk, a network card and an entropy source (virtio devices first, for speed); the kernel booting to single-user; the userland to a login prompt; the network and `sshd`, and then `ssh user@localhost -p 2222` gives a shell; and finally the same system on the FPGA. *(P)*

Commentary. **Precedent.** OpenBSD's riscv64 port is its newest and smallest, and it measures at 35,600 lines of machine-dependent code, of which 14,873 are the core, 11,771 platform drivers, 4,513 the loader and 4,443 headers; arm64 is 61,613 and amd64 79,167. **Trade-offs.** The pdp64 port should be smaller than the riscv64 one in several places: no firmware tables to parse, uniform eight-register devices instead of a platform's driver zoo, a trivial loader, no cache-maintenance code and no floating-point state code; the core machine-dependent code of about 15,000 lines is the part that has to be written by hand *(estimate)*. The LLVM back end is estimated at three to six months for one experienced engineer, a figure that has not been tested. **Benefits.** An OpenBSD that can be logged into is a demonstration that the architecture, the toolchain and the operating-system interface are complete in the way that matters to users, and it forces every piece (the compiler, the linker, the bus, the devices, the boot) to work together. **Modern comparables.** OpenBSD ports to new architectures have followed this shape; the RISC-V port is the nearest in kind.

10.5 What remains open

The open items are collected, with their options and benefits, in appendix B; ideas that are not decisions are in appendix A. The principal open choices are the transport and interrupt controller (chapters 2 and 5), the details of the device layout, the interrupt controller and virtualization position, the vector-math position, and the several smaller instruction-set proposals.

Part II — The instruction reference tables

The instruction reference tables

This is the reference part of the manual: **one entry for each instruction**, in the layout of a processor reference manual. Each entry gives its mnemonics, its syntax, its encoding (the parcel diagram and the numbers), its operation in pseudocode, the flags it writes, the exceptions it can take, and where the manual explains it. The entries are generated from `spec/pdp64-mnemonics.hbnf`, the one place where the numbers of the architecture are written, so an encoding here cannot disagree with the assembler or the simulators; the words (operation, flags, exceptions) are in `tools/manual/isa_entries.py` and are checked against the reference executor, `common/pdp64exec.c`, which `sim64` and QEMU share. The Status line of each entry says whether the executor implements the instruction.

Notation. `Rd` is the destination register (the `reg` field of parcel 0), `Rs` the first source (`low`), `Rt` the second (`t` of parcel 1), `Ru` the third (`x`); the assembler writes the destination last. `rm` is a general operand: a register at `len = 00`, or a memory operand, an immediate or an absolute address by one of the eight addressing modes (section 9.5). `EA(rm)` is its effective address. `M[a]` is memory, `M8[a]` one byte. `sext` and `zext` sign- and zero-extend. A `len = 01` instruction is two parcels (four bytes) and a `len = 10` instruction three (six bytes); `len = 11` is page 2. `N Z V C` are the condition flags. Bit diagrams number the bits of a parcel from 15 at the left.

Cf. lines point to the chapter that explains the matter an instruction touches: the kernel (chapter 6), the multiprocessor model (chapter 7), the bus (chapter 2).

Where the entries are

- Section 9.6, Moves, conversions and address formation: `MOVQ`, `MOVZL`, `LEA`, `MOVUB`
- Section 9.7, Integer arithmetic: `ADD`, `SUB`, `CMP`, `MUL`, `MULH`, `ADDM`, `INC`, `DEC`, `NEG`, `ADC`, `SBC`, `DIV`, `MULHU`, `ABS`, `MIN`, `SET`
- Section 9.8, Logic, shifts and rotates: `BIC`, `BIS`, `BIT`, `XOR`, `AND`, `SHL`, `CLR`, `COM`, `TST`, `ASL`, `ASR`
- Section 9.9, Bits, bytes and text: `CLZ`, `POPCNT`, `BSWAP`, `BITREV`, `ROR`, `RORI`, `RORI32`, `DSHR`, `EXTB`, `INSB`, `XNORPOP`, `CLMULL`, `UTF8LEN`, `UTF8DEC`, `UTF8ENC`
- Section 9.10, Branches and control flow: `BR`, `SOB`, `CBNE`, `BTAG`, `JMP`, `CALL`, `JMPI`, `JTAG`
- Section 9.11, The stack and block operations: `PUSHM`, `POPM`, `PUSH`, `POP`, `MCPY`, `MSET`, `MCMP`, `MSCN`, `MVAL`
- Section 9.12, Floating point: `FADD`, `FCMP`, `FDIV`, `FADD32`, `FMADD`, `FMADD32`, `FNEG`, `FMIN`, `FCLASS`, `FRINT`, `FCVTDQ`, `FSFLAGS`, `FADD16`, `FCVTSH`
- Section 9.13, Atomics and ordering: `CAS`, `SWAP`, `XADD`, `XAND`, `XMAX`, `LL`
- Section 9.14, System instructions, exceptions and interrupts: `NOP`, `RTI`, `RET`, `HALT`, `BPT`, `FENCE`, `WFE`, `SEV`, `TLBIA`, `PUSHX`, `ISYNC`, `HINT`, `MFSR`, `MTPS`, `SPL`, `TRAP`, `TLBI`, `MTASID`, `MTDP`, `MTDMW`, `MFCPUID`, `ICINV`, `PROBER`

The tables

- The second page: one-parcel abbreviations
- Assembler aliases and alternative spellings
- The architectural constants
- The mnemonics, alphabetically
- The opcodes, in order
- The bit layouts of every opcode: a chart for each layout, generated from the grammar

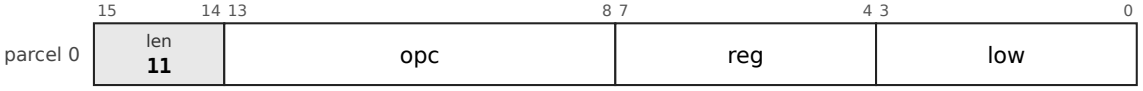
The specification of pdp64/r20 has 270 mnemonic words (234 instructions and 36 aliases) and 45 page-2 opcodes; the reference executor implements 200 of the 234 instructions; the Status line of the entry of each of the rest says that it is not implemented. There are 106 instruction entries, 67 bit-layout charts of the layouts, and an entry for each of the 234 instructions with a chart of each of its 897 forms.

Generated by `tools/mkmanual.py`; do not edit.

The second page: one-parcel abbreviations

Page 2 (`len = 11`) is a second opcode map of 64 one-parcel instructions, 45 of them defined. **Every page-2 instruction is exactly the page-1 instruction it abbreviates**: the same result, the same flags and the same faults. The assembler chooses the shortest encoding, so a mnemonic still assembles to one instruction and a program can never change meaning by using a page-2 form. The opcode numbers of page 2 are independent of those of page 1. `reg` is the register (or the high part of an immediate) named first, `low` the field named second.

Generated by `tools/mkmanual.py` from `spec/pdp64-mnemonics.hbnf` and `tools/manual/isa_entries.py`; do not edit.



Opcode	Name	Operation	reg	low	Long form it abbreviates
000000 (0x00)	ld.q	MOVQ load through (rn)	rd	rn	MOVQ (rn), rd
000001 (0x01)	st.q	MOVQ store through (rn)	rs	rn	MOVQ rs, (rn)
000010 (0x02)	ld.l	MOVL load through (rn)	rd	rn	MOVL (rn), rd
000011 (0x03)	st.l	MOVL store through (rn)	rs	rn	MOVL rs, (rn)
000100 (0x04)	ld.w	MOVW load through (rn)	rd	rn	MOVW (rn), rd
000101 (0x05)	st.w	MOVW store through (rn)	rs	rn	MOVW rs, (rn)
000110 (0x06)	ld.b	MOVB load through (rn)	rd	rn	MOVB (rn), rd
000111 (0x07)	st.b	MOVB store through (rn)	rs	rn	MOVB rs, (rn)
001000 (0x08)	ldz.l	MOVZL load through (rn)	rd	rn	MOVZL (rn), rd
001001 (0x09)	ldz.w	MOVZW load through (rn)	rd	rn	MOVZW (rn), rd
001010 (0x0A)	ldz.b	MOVZB load through (rn)	rd	rn	MOVZB (rn), rd
001011 (0x0B)	lea.sp	LEA 8*d(SP), rd	rd	d (0 to 15)	LEA 8*d(SP), rd
001100 (0x0C)	ldq.sp0	MOVQ load from SP, offset 8*low, 0 to 120	rd	offset / 8	MOVQ d(SP), rd
001101 (0x0D)	ldq.sp1	MOVQ load from SP, offset 8*(16+low), 128 to 248	rd	offset / 8 - 16	MOVQ d(SP), rd
001110 (0x0E)	stq.sp	MOVQ store to SP, offset 8*low, 0 to 120	rs	offset / 8	MOVQ rs, d(SP)
010000 (0x10)	ldl.sp0	MOVL load from SP, offset 4*low, 0 to 60	rd	offset / 4	MOVL d(SP), rd
010001 (0x11)	ldl.sp1	MOVL load from SP, offset 4*(16+low), 64 to 124	rd	offset / 4 - 16	MOVL d(SP), rd
010010 (0x12)	stl.sp0	MOVL store to SP, offset 4*low, 0 to 60	rs	offset / 4	MOVL rs, d(SP)
010011 (0x13)	stl.sp1	MOVL store to SP, offset 4*(16+low), 64 to 124	rs	offset / 4 - 16	MOVL rs, d(SP)
010100 (0x14)	ldq.d1	MOVQ load from rn at offset 8	rd	rn	MOVQ 8(rn), rd
010101 (0x15)	ldq.d2	MOVQ load from rn at offset 16	rd	rn	MOVQ 16(rn), rd
010110 (0x16)	ldq.d3	MOVQ load from rn at offset 24	rd	rn	MOVQ 24(rn), rd
010111 (0x17)	ldq.d4	MOVQ load from rn at offset 32	rd	rn	MOVQ 32(rn), rd
011000 (0x18)	stq.d1	MOVQ store to rn at offset 8	rs	rn	MOVQ rs, 8(rn)
011001 (0x19)	stq.d2	MOVQ store to rn at offset 16	rs	rn	MOVQ rs, 16(rn)
011010 (0x1A)	ldl.d1	MOVL load from rn at offset 4	rd	rn	MOVL 4(rn), rd
011011 (0x1B)	ldl.d2	MOVL load from rn at offset 8	rd	rn	MOVL 8(rn), rd
011100 (0x1C)	ldl.d3	MOVL load from rn at offset 12	rd	rn	MOVL 12(rn), rd
011101 (0x1D)	ldl.d4	MOVL load from rn at offset 16	rd	rn	MOVL 16(rn), rd
011110 (0x1E)	pushm8	push the registers R8 to R15 named by the 8-bit mask reg:low (bit i is R(8+i))	mask high	mask low	PUSHM #(mask << 8)
011111 (0x1F)	popm8	pop the registers named by the mask, as pushm8	mask high	mask low	POPM #(mask << 8)
100000 (0x20)	sub.sp	SUB #(8*n), SP, n = reg:low (0 to 255)	n high	n low	SUB #(8*n), SP
100001 (0x21)	add.sp	ADD #(8*n), SP, n = reg:low	n high	n low	ADD #(8*n), SP
100010 (0x22)	movq.i	MOVQ #imm, rd, imm sign-extended 4 bits (-8 to 7)	rd	imm	MOVQ #imm, rd
100011 (0x23)	shl.i	SHL by low (0 to 15)	rd	count	SHL #n, rd

Opcode	Name	Operation	reg	low	Long form it abbreviates
100100 (0x24)	shr.i	SHR by low (0 to 15)	rd	count	SHR #n, rd
100101 (0x25)	sar.i	SAR by low (0 to 15)	rd	count	SAR #n, rd
100110 (0x26)	and.i	AND with a 4-bit sign-extended immediate	rd	imm	AND #imm, rd
100111 (0x27)	bis.i	BIS with a 4-bit sign-extended immediate	rd	imm	BIS #imm, rd
101000 (0x28)	xor.i	XOR with a 4-bit sign-extended immediate	rd	imm	XOR #imm, rd

Opcode	Name	Operation	reg	low	Long form it abbreviates
101001 (0x29)	add.i	ADD with a 4-bit sign-extended immediate	rd	imm	ADD #imm, rd
101010 (0x2A)	pushx	PUSHX (reg and low zero)	0	0	SYSX sub-opcode 0
101011 (0x2B)	popx	POPX (reg and low zero)	0	0	SYSX sub-opcode 1
101100 (0x2C)	isync	ISYNC (reg and low zero)	0	0	SYSX sub-opcode 2
101101 (0x2D)	hint	HINT #n, n = reg:low (0 to 255)	n high	n low	SYSX sub-opcode 3

Every page-2 opcode not listed (001111 (0x0F) and 101110 (0x2E) to 111111 (0x3F), 19 of 64) is reserved; a reserved page-2 opcode is an illegal-instruction abort.

Why there is a second page

Instructions are at most six bytes: an eight-byte form would serve 48-bit immediates and 44-bit displacements, and only 49 of 373,635 compiled instructions (0.013%) would use it. The length code 11 is therefore a second page of one-parcel instructions, each an exact abbreviation of a long form, chosen from the forms the compiler emits most. The estimate from static counts on the 827-file corpus is about 118 KB, 7% of the 1.67 MB image. The rule that governs the page: a page-2 instruction has the same result, flags and faults as the long form it abbreviates, so the assembler choosing the shortest encoding can never change what a program does.

Assembler aliases and old spellings

A mnemonic that assembles to another instruction. One mnemonic is one instruction (section 9.18); the disassembler never prints an alias.

Generated by `tools/mkmanual.py` *from* `spec/pdp64-mnemonics.hbnf`*; do not edit.*

Written	Assembles to	Group
BHIS	BCC	branch
BLO	BCS	branch
CBHIS	CBCC	cbranch
CBLO	CBCS	cbranch
FADDD	FADD	alu
FADDS	FADD32	fesc
FCLASSD	FCLASS	fesc
FCLASSS	FCLASS32	fesc
FCMPD	FCMP	alu
FCMPS	FCMP32	fesc
FDIVD	FDIV	fesc
FDIVS	FDIV32	fesc
FMADDD	FMADD	fesc
FMADDS	FMADD32	fesc
FMAXD	FMAX	fesc
FMIND	FMIN	fesc
FMSUBD	FMSUB	fesc
FMSUBS	FMSUB32	fesc
FMULD	FMUL	alu
FMULS	FMUL32	fesc
FNMADDD	FNMADD	fesc
FNMADDS	FNMADD32	fesc
FNMSUBD	FNMSUB	fesc
FNMSUBS	FNMSUB32	fesc
FRINTD	FRINT	fesc
FRINTDA	FRINTA	fesc
FRINTDM	FRINTM	fesc
FRINTDP	FRINTP	fesc
FRINTDZ	FRINTZ	fesc
FSQRTD	FSQRT	fesc
FSQRTS	FSQRT32	fesc
FSUBD	FSUB	alu
FSUBS	FSUB32	fesc
MULU	MUL	alu
ROL	RORI	besc
WAIT	WFE	sys-none
ROL rs, #n, rd	RORI rs, #(64-n), rd	assembler, immediate count only

The architectural constants

The numbers of the processor status word, the modes, the exception vectors, the page-table entry, the memory types, the translation geometry, the system registers, the addressing modes and the condition codes, each read from the machine-readable specification (`spec/pdp64-system.hbnf` and `spec/pdp64-mnemonics.hbnf`).
Generated by `tools/mkmanual.py`; do not edit.

The processor status word (16 bits)

Field	Bit	Width	Meaning
C	0	1	carry (borrow after a subtract)
V	1	1	signed overflow
Z	2	1	zero
N	3	1	negative
T	4	1	trace: take a trace trap after the next instruction
PRI	5	3	interrupt priority level, 0 to 7
MODE	8	2	current mode
PREV	10	2	previous mode (the mode before the last exception)

Bits 12 to 15 are reserved and read as zero. Bit 12 is the user-memory guard UA.

The modes

Mode	Value
kernel	0
supervisor	1
user	3

Value 2 is reserved: loading it into either mode field of the status word is an illegal instruction.

The exception vectors

The table has 512 entries of eight bytes at the physical address `VBR + 8 * vector`; ECAUSE equals the vector number.

Vector	Number	Cause
RESET	0	processor reset: the first instruction fetch
ILLEGAL	2	a reserved or undefined opcode, a reserved encoding or an unassigned system register
PRIVILEGED	3	a system instruction executed outside kernel mode
MISALIGNED	4	a data access whose address is not a multiple of its size (the MOVU forms excepted)
NONCANONICAL	5	a virtual address whose bits 63 to 39 differ from bit 38
IFAULT	6	instruction fetch from a page that is not mapped
LFAULT	7	load from a page that is not mapped
SFAULT	8	store to a page that is not mapped
IPROT	9	instruction fetch denied by the page permissions
LPROT	10	load denied by the page permissions
SPROT	11	store denied by the page permissions
DIVZERO	12	division by zero (DIV, DIVU, MOD, MODU)
BUSFAULT	13	a read of non-existent memory (a precise abort)
LPAD	14	an indirect transfer that did not land on LPAD (enforcement off in ABI version 1)
BPT	15	the BPT instruction
TIMER	32	the timer interrupt (level 6)
COUNT	512	the number of vectors; the table has this many eight-byte entries

The page-table entry (64 bits)

Field	Bit	Width	Meaning
`V`	0	1	valid
`R`	1	1	readable
`W`	2	1	writable
`X`	3	1	executable
`U`	4	1	accessible from user mode
`G`	5	1	global: ignores the ASID
`A`	6	1	accessed (managed by software)
`D`	7	1	dirty (managed by software)
`MT`	8	2	memory type
`PPN`	12	28	physical page number: physical address bits 12 to 39, in place

A non-leaf entry has `V` = 1 and `R` = `W` = `X` = 0 and points to the next table. Every bit not named is zero.

The memory types (`MT`)

Type	Value
cached	0
uncached	1
device	2

The translation geometry

Quantity	Value
levels	3

The system registers (`MFSR`, `MTSR`)

Name	Number	Access
`PTBR`	0	read and write
`EPC`	1	read and write
`EVA`	2	read only
`ECAUSE`	3	read only
`DMW0`	4	read and write
`DMW1`	5	read and write
`USP`	6	read and write
`SSP`	7	read and write
`EPSW`	8	read and write
`VBR`	9	read and write
`MMUCTL`	10	read and write
`ASID`	11	read and write
`CPUID`	12	read only

The addressing modes

Mode	Value
base	`0000`
index	`0001`
pcrel	`0010`
dp	`0011`
imm	`0100`
abs	`0101`
postinc	`0110`
predec	`0111`

Modes `1000` to `1111` are reserved. The syntax and the parcel layout of each mode are in section 9.5.

The condition codes

Name	Number	Aliases
al	0	
ne	1	
eq	2	
pl	3	
mi	4	
vc	5	
vs	6	
cc	7	hs
cs	8	lo
ge	9	
lt	10	
gt	11	
le	12	
hi	13	
los	14	

The mnemonics, alphabetically

Generated by `tools/mkmanual.py`; do not edit.

Mnemonic	Entry	Opcode	Sub	Layouts	Group
ABS	ABS (section 9.7)	001110 (0x0E)	00001000 (0x08)	2, entry	Arithmetic (AESC)
ADC	ADC (section 9.7)	010011 (0x13)	0110 (0x6)	9, entry	Single operand (SOP)
ADD	ADD (section 9.7)	010110 (0x16)		9, entry	Moves, address formation and the ALU
ADDM	ADDM (section 9.7)	001111 (0x0F)	00001000 (0x08)	2, entry	Memory escape (MESC)
AND	AND (section 9.8)	011101 (0x1D)		9, entry	Moves, address formation and the ALU
ASL	ASL (section 9.8)	010011 (0x13)	1000 (0x8)	9, entry	Single operand (SOP)
ASR	ASR (section 9.8)	010011 (0x13)	1001 (0x9)	9, entry	Single operand (SOP)
BCC	BCC (section 9.10)	110111 (0x37)		3, entry	Branches
BCS	BCC (section 9.10)	111000 (0x38)		3, entry	Branches
BEQ	BCC (section 9.10)	110010 (0x32)		3, entry	Branches
BGE	BCC (section 9.10)	111001 (0x39)		3, entry	Branches
BGT	BCC (section 9.10)	111011 (0x3B)		3, entry	Branches
BHI	BCC (section 9.10)	111101 (0x3D)		3, entry	Branches
BHIS (alias of BCC)	BCC (section 9.10)	110111 (0x37)		3, entry	Branches
BIC	BIC (section 9.8)	011000 (0x18)		9, entry	Moves, address formation and the ALU
BICM	ADDM (section 9.7)	001111 (0x0F)	00001010 (0x0A)	2, entry	Memory escape (MESC)
BIS	BIS (section 9.8)	011001 (0x19)		9, entry	Moves, address formation and the ALU
BISM	ADDM (section 9.7)	001111 (0x0F)	00001011 (0x0B)	2, entry	Memory escape (MESC)
BIT	BIT (section 9.8)	011011 (0x1B)		9, entry	Moves, address formation and the ALU
BITREV	BITREV (section 9.9)	001100 (0x0C)	00001000 (0x08)	2, entry	Bits and fields (BESC)
BLE	BCC (section 9.10)	111100 (0x3C)		3, entry	Branches
BLO (alias of BCS)	BCC (section 9.10)	111000 (0x38)		3, entry	Branches
BLOS	BCC (section 9.10)	111110 (0x3E)		3, entry	Branches
BLT	BCC (section 9.10)	111010 (0x3A)		3, entry	Branches
BMI	BCC (section 9.10)	110100 (0x34)		3, entry	Branches
BNE	BCC (section 9.10)	110001 (0x31)		3, entry	Branches
BNTAG	BTAG (section 9.10)	010101 (0x15)	00000001 (0x01)	2, entry	Branches
BPL	BCC (section 9.10)	110011 (0x33)		3, entry	Branches
BPT	BPT (section 9.14)	010100 (0x14)	0101 (0x5)	3, entry	System (SYS)
BR	BCC (section 9.10)	110000 (0x30)		3, entry	Branches
BSWAP	BSWAP (section 9.9)	001100 (0x0C)	00000101 (0x05)	2, entry	Bits and fields (BESC)
BSWAPL	BSWAP (section 9.9)	001100 (0x0C)	00000111 (0x07)	2, entry	Bits and fields (BESC)
BSWAPW	BSWAP (section 9.9)	001100 (0x0C)	00000110 (0x06)	2, entry	Bits and fields (BESC)
BTAG	BTAG (section 9.10)	010101 (0x15)	00000000 (0x00)	2, entry	Branches
BVC	BCC (section 9.10)	110101 (0x35)		3, entry	Branches
BVS	BCC (section 9.10)	110110 (0x36)		3, entry	Branches
CALL	CALL (section 9.10)	010011 (0x13)	1101 (0xD)	9, entry	Single operand (SOP)
CALLI	JMPI (section 9.10)	010010 (0x12)	0001 (0x1)	9, entry	Indirect transfers (XFER)
CAS	CAS (section 9.13)	010001 (0x11)	00000000 (0x00)	2, entry	Atomics (ATOM)
CASDB	CAS (section 9.13)	010001 (0x11)	00000001 (0x01)	2, entry	Atomics (ATOM)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
CBCC	CBCC (section 9.10)	101000 (0x28)	00000111 (0x07)	5, entry	Branches
CBCS	CBCC (section 9.10)	101000 (0x28)	00001000 (0x08)	5, entry	Branches
CBEQ	CBCC (section 9.10)	101000 (0x28)	00000010 (0x02)	5, entry	Branches
CBGE	CBCC (section 9.10)	101000 (0x28)	00001001 (0x09)	5, entry	Branches

Mnemonic	Entry	Opcode	Sub	Layouts	Group
CBGT	CBCC (section 9.10)	101000 (0x28)	00001011 (0x0B)	5, entry	Branches
CBHI	CBCC (section 9.10)	101000 (0x28)	00001101 (0x0D)	5, entry	Branches
CBHIS (alias of CBCC)	CBCC (section 9.10)	101000 (0x28)	00000111 (0x07)	5, entry	Branches
CBLE	CBCC (section 9.10)	101000 (0x28)	00001100 (0x0C)	5, entry	Branches
CBLO (alias of CBCS)	CBCC (section 9.10)	101000 (0x28)	00001000 (0x08)	5, entry	Branches
CBLOS	CBCC (section 9.10)	101000 (0x28)	00001110 (0x0E)	5, entry	Branches
CBLT	CBCC (section 9.10)	101000 (0x28)	00001010 (0x0A)	5, entry	Branches
CBMI	CBCC (section 9.10)	101000 (0x28)	00000100 (0x04)	5, entry	Branches
CBNE	CBCC (section 9.10)	101000 (0x28)	00000001 (0x01)	5, entry	Branches
CBPL	CBCC (section 9.10)	101000 (0x28)	00000011 (0x03)	5, entry	Branches
CBVC	CBCC (section 9.10)	101000 (0x28)	00000101 (0x05)	5, entry	Branches
CBVS	CBCC (section 9.10)	101000 (0x28)	00000110 (0x06)	5, entry	Branches
CLMULH	CLMUL (section 9.9)	001100 (0x0C)	00110010 (0x32)	2, entry	Bits and fields (BESC)
CLMULL	CLMUL (section 9.9)	001100 (0x0C)	00110001 (0x31)	2, entry	Bits and fields (BESC)
CLO	COUNT (section 9.9)	001100 (0x0C)	00000010 (0x02)	2, entry	Bits and fields (BESC)
CLR	CLR (section 9.8)	010011 (0x13)	0000 (0x0)	9, entry	Single operand (SOP)
CLZ	COUNT (section 9.9)	001100 (0x0C)	00000000 (0x00)	2, entry	Bits and fields (BESC)
CMP	CMP (section 9.7)	011010 (0x1A)		9, entry	Moves, address formation and the ALU
COM	COM (section 9.8)	010011 (0x13)	0001 (0x1)	9, entry	Single operand (SOP)
CTO	COUNT (section 9.9)	001100 (0x0C)	00000011 (0x03)	2, entry	Bits and fields (BESC)
CTZ	COUNT (section 9.9)	001100 (0x0C)	00000001 (0x01)	2, entry	Bits and fields (BESC)
DEC	DEC (section 9.7)	010011 (0x13)	0011 (0x3)	9, entry	Single operand (SOP)
DIV	DIV (section 9.7)	001110 (0x0E)	00000000 (0x00)	2, entry	Arithmetic (AESC)
DIVU	DIV (section 9.7)	001110 (0x0E)	00000001 (0x01)	2, entry	Arithmetic (AESC)
DSHR	DSHR (section 9.9)	001100 (0x0C)	00100000 (0x20)	2, entry	Bits and fields (BESC)
EXTB	EXTB (section 9.9)	001100 (0x0C)	00100001 (0x21)	2, entry	Bits and fields (BESC)
EXTZB	EXTB (section 9.9)	001100 (0x0C)	00100010 (0x22)	2, entry	Bits and fields (BESC)
FABS	FSIGN (section 9.12)	010000 (0x10)	00100001 (0x21)	2, entry	Floating point (FESC)
FADD	FADD (section 9.12)	100100 (0x24)		9, entry	Moves, address formation and the ALU
FADD16	FHALF (section 9.12)	010000 (0x10)	01010000 (0x50)	2, entry	Floating point (FESC)
FADD32	FSINGLE (section 9.12)	010000 (0x10)	00001000 (0x08)	2, entry	Floating point (FESC)
FADDD (alias of FADD)	FADD (section 9.12)	100100 (0x24)		9, entry	Moves, address formation and the ALU
FADDS (alias of FADD32)	FSINGLE (section 9.12)	010000 (0x10)	00001000 (0x08)	2, entry	Floating point (FESC)
FCLASS	FCLASS (section 9.12)	010000 (0x10)	00101000 (0x28)	2, entry	Floating point (FESC)
FCLASS16	FCLASS (section 9.12)	010000 (0x10)	01011100 (0x5C)	2, entry	Floating point (FESC)
FCLASS32	FCLASS (section 9.12)	010000 (0x10)	00101001 (0x29)	2, entry	Floating point (FESC)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
FCLASSD (alias of FCLASS)	FCLASS (section 9.12)	010000 (0x10)	00101000 (0x28)	2, entry	Floating point (FESC)
FCLASSS (alias of FCLASS32)	FCLASS (section 9.12)	010000 (0x10)	00101001 (0x29)	2, entry	Floating point (FESC)
FCMP	FCMP (section 9.12)	100111 (0x27)		9, entry	Moves, address formation and the ALU
FCMP16	FHALF (section 9.12)	010000 (0x10)	01010101 (0x55)	2, entry	Floating point (FESC)
FCMP32	FSINGLE (section 9.12)	010000 (0x10)	00001101 (0x0D)	2, entry	Floating point (FESC)
FCMPD (alias of FCMP)	FCMP (section 9.12)	100111 (0x27)		9, entry	Moves, address formation and the ALU
FCMPS (alias of FCMP32)	FSINGLE (section 9.12)	010000 (0x10)	00001101 (0x0D)	2, entry	Floating point (FESC)
FCOPYSIGN	FSIGN (section 9.12)	010000 (0x10)	00100010 (0x22)	2, entry	Floating point (FESC)
FCVTDH	FCVTH (section 9.12)	010000 (0x10)	01100010 (0x62)	2, entry	Floating point (FESC)
FCVTDQ	FCVT (section 9.12)	010000 (0x10)	00111000 (0x38)	2, entry	Floating point (FESC)
FCVTDQU	FCVT (section 9.12)	010000 (0x10)	00111110 (0x3E)	2, entry	Floating point (FESC)
FCVTDS	FCVT (section 9.12)	010000 (0x10)	00111010 (0x3A)	2, entry	Floating point (FESC)
FCVTHD	FCVTH (section 9.12)	010000 (0x10)	01100011 (0x63)	2, entry	Floating point (FESC)
FCVTHL	FCVTH (section 9.12)	010000 (0x10)	01100100 (0x64)	2, entry	Floating point (FESC)
FCVTHS	FCVTH (section 9.12)	010000 (0x10)	01100001 (0x61)	2, entry	Floating point (FESC)
FCVTLH	FCVTH (section 9.12)	010000 (0x10)	01100101 (0x65)	2, entry	Floating point (FESC)
FCVTLS	FCVT (section 9.12)	010000 (0x10)	00111101 (0x3D)	2, entry	Floating point (FESC)
FCVTQD	FCVT (section 9.12)	010000 (0x10)	00111001 (0x39)	2, entry	Floating point (FESC)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
FCVTQUD	FCVT (section 9.12)	010000 (0x10)	00111111 (0x3F)	2, entry	Floating point (FESC)
FCVTSD	FCVT (section 9.12)	010000 (0x10)	00111011 (0x3B)	2, entry	Floating point (FESC)
FCVTSH	FCVTH (section 9.12)	010000 (0x10)	01100000 (0x60)	2, entry	Floating point (FESC)
FCVTSL	FCVT (section 9.12)	010000 (0x10)	00111100 (0x3C)	2, entry	Floating point (FESC)
FDIV	FDIV (section 9.12)	010000 (0x10)	00000000 (0x00)	2, entry	Floating point (FESC)
FDIV16	FHALF (section 9.12)	010000 (0x10)	01010011 (0x53)	2, entry	Floating point (FESC)
FDIV32	FSINGLE (section 9.12)	010000 (0x10)	00001011 (0x0B)	2, entry	Floating point (FESC)
FDIVD (alias of FDIV)	FDIV (section 9.12)	010000 (0x10)	00000000 (0x00)	2, entry	Floating point (FESC)
FDIVS (alias of FDIV32)	FSINGLE (section 9.12)	010000 (0x10)	00001011 (0x0B)	2, entry	Floating point (FESC)
FENCE	FENCE (section 9.14)	010100 (0x14)	0110 (0x6)	3, entry	System (SYS)
FMADD	FMADD (section 9.12)	010000 (0x10)	00010000 (0x10)	2, entry	Floating point (FESC)
FMADD16	FHALF (section 9.12)	010000 (0x10)	01011000 (0x58)	2, entry	Floating point (FESC)
FMADD32	FMADD32 (section 9.12)	010000 (0x10)	00011000 (0x18)	2, entry	Floating point (FESC)
FMADDD (alias of FMADD)	FMADD (section 9.12)	010000 (0x10)	00010000 (0x10)	2, entry	Floating point (FESC)
FMADDS (alias of FMADD32)	FMADD32 (section 9.12)	010000 (0x10)	00011000 (0x18)	2, entry	Floating point (FESC)
FMAX	FMIN (section 9.12)	010000 (0x10)	00100100 (0x24)	2, entry	Floating point (FESC)
FMAXD (alias of FMAX)	FMIN (section 9.12)	010000 (0x10)	00100100 (0x24)	2, entry	Floating point (FESC)
FMIN	FMIN (section 9.12)	010000 (0x10)	00100011 (0x23)	2, entry	Floating point (FESC)
FMIND (alias of FMIN)	FMIN (section 9.12)	010000 (0x10)	00100011 (0x23)	2, entry	Floating point (FESC)
FMSUB	FMADD (section 9.12)	010000 (0x10)	00010001 (0x11)	2, entry	Floating point (FESC)
FMSUB16	FHALF (section 9.12)	010000 (0x10)	01011001 (0x59)	2, entry	Floating point (FESC)
FMSUB32	FMADD32 (section 9.12)	010000 (0x10)	00011001 (0x19)	2, entry	Floating point (FESC)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
FMSUBD (alias of FMSUB)	FMADD (section 9.12)	010000 (0x10)	00010001 (0x11)	2, entry	Floating point (FESC)
FMSUBS (alias of FMSUB32)	FMADD32 (section 9.12)	010000 (0x10)	00011001 (0x19)	2, entry	Floating point (FESC)
FMUL	FADD (section 9.12)	100110 (0x26)		9, entry	Moves, address formation and the ALU
FMUL16	FHALF (section 9.12)	010000 (0x10)	01010010 (0x52)	2, entry	Floating point (FESC)
FMUL32	FSINGLE (section 9.12)	010000 (0x10)	00001010 (0x0A)	2, entry	Floating point (FESC)
FMULD (alias of FMUL)	FADD (section 9.12)	100110 (0x26)		9, entry	Moves, address formation and the ALU
FMULS (alias of FMUL32)	FSINGLE (section 9.12)	010000 (0x10)	00001010 (0x0A)	2, entry	Floating point (FESC)
FNEG	FSIGN (section 9.12)	010000 (0x10)	00100000 (0x20)	2, entry	Floating point (FESC)
FNMADD	FMADD (section 9.12)	010000 (0x10)	00010010 (0x12)	2, entry	Floating point (FESC)
FNMADD16	FHALF (section 9.12)	010000 (0x10)	01011010 (0x5A)	2, entry	Floating point (FESC)
FNMADD32	FMADD32 (section 9.12)	010000 (0x10)	00011010 (0x1A)	2, entry	Floating point (FESC)
FNMADDD (alias of FNMADD)	FMADD (section 9.12)	010000 (0x10)	00010010 (0x12)	2, entry	Floating point (FESC)
FNMADDS (alias of FNMADD32)	FMADD32 (section 9.12)	010000 (0x10)	00011010 (0x1A)	2, entry	Floating point (FESC)
FNMSUB	FMADD (section 9.12)	010000 (0x10)	00010011 (0x13)	2, entry	Floating point (FESC)
FNMSUB16	FHALF (section 9.12)	010000 (0x10)	01011011 (0x5B)	2, entry	Floating point (FESC)
FNMSUB32	FMADD32 (section 9.12)	010000 (0x10)	00011011 (0x1B)	2, entry	Floating point (FESC)
FNMSUBD (alias of FNMSUB)	FMADD (section 9.12)	010000 (0x10)	00010011 (0x13)	2, entry	Floating point (FESC)
FNMSUBS (alias of FNMSUB32)	FMADD32 (section 9.12)	010000 (0x10)	00011011 (0x1B)	2, entry	Floating point (FESC)
FRINT	FRINT (section 9.12)	010000 (0x10)	00110000 (0x30)	2, entry	Floating point (FESC)
FRINTA	FRINT (section 9.12)	010000 (0x10)	00110100 (0x34)	2, entry	Floating point (FESC)
FRINTD (alias of FRINT)	FRINT (section 9.12)	010000 (0x10)	00110000 (0x30)	2, entry	Floating point (FESC)
FRINTDA (alias of FRINTA)	FRINT (section 9.12)	010000 (0x10)	00110100 (0x34)	2, entry	Floating point (FESC)
FRINTDM (alias of FRINTM)	FRINT (section 9.12)	010000 (0x10)	00110010 (0x32)	2, entry	Floating point (FESC)
FRINTDP (alias of FRINTP)	FRINT (section 9.12)	010000 (0x10)	00110011 (0x33)	2, entry	Floating point (FESC)
FRINTDZ (alias of FRINTZ)	FRINT (section 9.12)	010000 (0x10)	00110001 (0x31)	2, entry	Floating point (FESC)
FRINTM	FRINT (section 9.12)	010000 (0x10)	00110010 (0x32)	2, entry	Floating point (FESC)
FRINTP	FRINT (section 9.12)	010000 (0x10)	00110011 (0x33)	2, entry	Floating point (FESC)
FRINTZ	FRINT (section 9.12)	010000 (0x10)	00110001 (0x31)	2, entry	Floating point (FESC)
FSFLAGS	FSFLAGS (section 9.12)	010000 (0x10)	01000000 (0x40)	2, entry	Floating point (FESC)
FSQRT	FDIV (section 9.12)	010000 (0x10)	00000001 (0x01)	2, entry	Floating point (FESC)
FSQRT16	FHALF (section 9.12)	010000 (0x10)	01010100 (0x54)	2, entry	Floating point (FESC)
FSQRT32	FSINGLE (section 9.12)	010000 (0x10)	00001100 (0x0C)	2, entry	Floating point (FESC)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
FSQRTD (alias of FSQRT)	FDIV (section 9.12)	010000 (0x10)	00000001 (0x01)	2, entry	Floating point (FESC)
FSQRTS (alias of FSQRT32)	FSINGLE (section 9.12)	010000 (0x10)	00001100 (0x0C)	2, entry	Floating point (FESC)
FSUB	FADD (section 9.12)	100101 (0x25)		9, entry	Moves, address formation and the ALU
FSUB16	FHALF (section 9.12)	010000 (0x10)	01010001 (0x51)	2, entry	Floating point (FESC)
FSUB32	FSINGLE (section 9.12)	010000 (0x10)	00001001 (0x09)	2, entry	Floating point (FESC)
FSUBD (alias of FSUB)	FADD (section 9.12)	100101 (0x25)		9, entry	Moves, address formation and the ALU
FSUBS (alias of FSUB32)	FSINGLE (section 9.12)	010000 (0x10)	00001001 (0x09)	2, entry	Floating point (FESC)
HALT	HALT (section 9.14)	010100 (0x14)	0100 (0x4)	3, entry	System (SYS)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
HINT	HINT (section 9.14)	010100 (0x14)	0011 (0x3)	2, entry	System (SYS)
ICINV	ICINV (section 9.14)	010100 (0x14)	1100 (0xC)	3, entry	System (SYS)
ICINVA	TLBA (section 9.14)	010100 (0x14)	1100 (0xC)	3, entry	System (SYS)
INC	INC (section 9.7)	010011 (0x13)	0010 (0x2)	9, entry	Single operand (SOP)
INSB	INSB (section 9.9)	001100 (0x0C)	00100011 (0x23)	2, entry	Bits and fields (BESC)
ISYNC	ISYNC (section 9.14)	010100 (0x14)	0010 (0x2)	2, entry	System (SYS)
JMP	JMP (section 9.10)	010011 (0x13)	1100 (0xC)	9, entry	Single operand (SOP)
JMPI	JMPI (section 9.10)	010010 (0x12)	0000 (0x0)	9, entry	Indirect transfers (XFER)
JTAG	JTAG (section 9.10)	010010 (0x12)	0010 (0x2)	9, entry	Indirect transfers (XFER)
LEA	LEA (section 9.6)	001011 (0x0B)		9, entry	Moves, address formation and the ALU
LL	LLSC (section 9.13)	010001 (0x11)	00100000 (0x20)	2, entry	Atomics (ATOM)
LPAD	NOP (section 9.14)	010100 (0x14)	1001 (0x9)	3, entry	System (SYS)
MAX	MINMAX (section 9.7)	001110 (0x0E)	00010010 (0x12)	2, entry	Arithmetic (AESC)
MAXU	MINMAX (section 9.7)	001110 (0x0E)	00010011 (0x13)	2, entry	Arithmetic (AESC)
MCMP	MCMP (section 9.11)	001101 (0x0D)	00000011 (0x03)	2, entry	Block memory (MBLK)
MCPY	MCPY (section 9.11)	001101 (0x0D)	00000000 (0x00)	2, entry	Block memory (MBLK)
MFASID	MTASID (section 9.14)	010100 (0x14)	0111 (0x7)	3, entry	System (SYS)
MFCPUID	MFCPUID (section 9.14)	010100 (0x14)	1011 (0xB)	3, entry	System (SYS)
MFDP	MTDP (section 9.14)	010100 (0x14)	1001 (0x9)	3, entry	System (SYS)
MFPS	MTPS (section 9.14)	010100 (0x14)	0010 (0x2)	3, entry	System (SYS)
MFSR	MFSR (section 9.14)	010100 (0x14)	1101 (0xD)	2, entry	System (SYS)
MIN	MINMAX (section 9.7)	001110 (0x0E)	00010000 (0x10)	2, entry	Arithmetic (AESC)
MINU	MINMAX (section 9.7)	001110 (0x0E)	00010001 (0x11)	2, entry	Arithmetic (AESC)
MMOV	MCPY (section 9.11)	001101 (0x0D)	00000001 (0x01)	2, entry	Block memory (MBLK)
MOD	DIV (section 9.7)	001110 (0x0E)	00000010 (0x02)	2, entry	Arithmetic (AESC)
MODU	DIV (section 9.7)	001110 (0x0E)	00000011 (0x03)	2, entry	Arithmetic (AESC)
MOVB	MOV (section 9.6)	000110 (0x06)		9, entry	Moves, address formation and the ALU
MOVL	MOV (section 9.6)	000010 (0x02)		9, entry	Moves, address formation and the ALU
MOVQ	MOV (section 9.6)	000000 (0x00)		9, entry	Moves, address formation and the ALU
MOVUB	MOVU (section 9.6)	001111 (0x0F)	00000000 (0x00)	2, entry	Memory escape (MESC)
MOVUL	MOVU (section 9.6)	001111 (0x0F)	00000100 (0x04)	2, entry	Memory escape (MESC)
MOVUQ	MOVU (section 9.6)	001111 (0x0F)	00000110 (0x06)	2, entry	Memory escape (MESC)
MOVUW	MOVU (section 9.6)	001111 (0x0F)	00000010 (0x02)	2, entry	Memory escape (MESC)
MOVW	MOV (section 9.6)	000100 (0x04)		9, entry	Moves, address formation and the ALU
MOVZB	MOVZ (section 9.6)	001010 (0x0A)		9, entry	Moves, address formation and the ALU
MOVZL	MOVZ (section 9.6)	001000 (0x08)		9, entry	Moves, address formation and the ALU
MOVZW	MOVZ (section 9.6)	001001 (0x09)		9, entry	Moves, address formation and the ALU
MSCN	MSCN (section 9.11)	001101 (0x0D)	00000100 (0x04)	2, entry	Block memory (MBLK)
MSCNN	MSCN (section 9.11)	001101 (0x0D)	00000101 (0x05)	2, entry	Block memory (MBLK)
MSET	MSET (section 9.11)	001101 (0x0D)	00000010 (0x02)	2, entry	Block memory (MBLK)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
MTASID	MTASID (section 9.14)	010100 (0x14)	0110 (0x6)	3, entry	System (SYS)
MTDMW	MTDMW (section 9.14)	010100 (0x14)	1010 (0xA)	3, entry	System (SYS)
MTDP	MTDP (section 9.14)	010100 (0x14)	1000 (0x8)	3, entry	System (SYS)
MTPS	MTPS (section 9.14)	010100 (0x14)	0001 (0x1)	3, entry	System (SYS)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
MTSR	MFSR (section 9.14)	010100 (0x14)	1110 (0xE)	2, entry	System (SYS)
MUL	MUL (section 9.7)	011110 (0x1E)		9, entry	Moves, address formation and the ALU
MULH	MULH (section 9.7)	011111 (0x1F)		9, entry	Moves, address formation and the ALU
MULHU	MULHU (section 9.7)	001110 (0x0E)	00000100 (0x04)	2, entry	Arithmetic (AESC)
MULU (alias of MUL)	MUL (section 9.7)	011110 (0x1E)		9, entry	Moves, address formation and the ALU
MVAL	MVAL (section 9.11)	001101 (0x0D)	00000110 (0x06)	2, entry	Block memory (MBLK)
NEG	NEG (section 9.7)	010011 (0x13)	0100 (0x4)	9, entry	Single operand (SOP)
NOP	NOP (section 9.14)	010100 (0x14)	0000 (0x0)	3, entry	System (SYS)
POP	POP (section 9.11)	010011 (0x13)	1111 (0xF)	9, entry	Single operand (SOP)
POPCNT	POPCNT (section 9.9)	001100 (0x0C)	00000100 (0x04)	2, entry	Bits and fields (BESC)
POPM	POPM (section 9.11)	010011 (0x13)	1011 (0xB)	9, entry	Single operand (SOP)
POPX	PUSHX (section 9.14)	010100 (0x14)	0001 (0x1)	2, entry	System (SYS)
PROBER	PROBE (section 9.14)	010100 (0x14)	1101 (0xD)	3, entry	System (SYS)
PROBEW	PROBE (section 9.14)	010100 (0x14)	1110 (0xE)	3, entry	System (SYS)
PUSH	PUSH (section 9.11)	010011 (0x13)	1110 (0xE)	9, entry	Single operand (SOP)
PUSHM	PUSHM (section 9.11)	010011 (0x13)	1010 (0xA)	9, entry	Single operand (SOP)
PUSHX	PUSHX (section 9.14)	010100 (0x14)	0000 (0x0)	2, entry	System (SYS)
RET	RET (section 9.14)	010100 (0x14)	0011 (0x3)	3, entry	System (SYS)
ROL (alias of RORI)	RORI (section 9.9)	001100 (0x0C)	00010001 (0x11)	2, entry	Bits and fields (BESC)
ROR	ROR (section 9.9)	001100 (0x0C)	00010000 (0x10)	2, entry	Bits and fields (BESC)
RORI	RORI (section 9.9)	001100 (0x0C)	00010001 (0x11)	2, entry	Bits and fields (BESC)
RORI32	RORI32 (section 9.9)	001100 (0x0C)	00010010 (0x12)	2, entry	Bits and fields (BESC)
RTI	RTI (section 9.14)	010100 (0x14)	0001 (0x1)	3, entry	System (SYS)
RTT	RTI (section 9.14)	010100 (0x14)	0010 (0x2)	3, entry	System (SYS)
SAR	SHIFT (section 9.8)	100010 (0x22)		9, entry	Moves, address formation and the ALU
SBC	SBC (section 9.7)	010011 (0x13)	0111 (0x7)	9, entry	Single operand (SOP)
SC	LLSC (section 9.13)	010001 (0x11)	00100001 (0x21)	2, entry	Atomics (ATOM)
SET	SET (section 9.7)	001110 (0x0E)	00100000 (0x20)	2, entry	Arithmetic (AESC)
SEV	SEV (section 9.14)	010100 (0x14)	1000 (0x8)	3, entry	System (SYS)
SHL	SHIFT (section 9.8)	100000 (0x20)		9, entry	Moves, address formation and the ALU
SHR	SHIFT (section 9.8)	100001 (0x21)		9, entry	Moves, address formation and the ALU
SOB	SOB (section 9.10)	111111 (0x3F)	00111111 (0x3F)	2, entry	Branches
SPL	SPL (section 9.14)	010100 (0x14)	0011 (0x3)	3, entry	System (SYS)
SUB	SUB (section 9.7)	010111 (0x17)		9, entry	Moves, address formation and the ALU
SUBM	ADDM (section 9.7)	001111 (0x0F)	00001001 (0x09)	2, entry	Memory escape (MESC)
SWAP	SWAP (section 9.13)	010001 (0x11)	00000010 (0x02)	2, entry	Atomics (ATOM)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
SWAPDB	SWAP (section 9.13)	010001 (0x11)	00000011 (0x03)	2, entry	Atomics (ATOM)
TLBI	TLBI (section 9.14)	010100 (0x14)	0101 (0x5)	3, entry	System (SYS)
TLBIA	TLBA (section 9.14)	010100 (0x14)	1010 (0xA)	3, entry	System (SYS)
TLBSYNC	TLBA (section 9.14)	010100 (0x14)	1011 (0xB)	3, entry	System (SYS)
TRAP	TRAP (section 9.14)	010100 (0x14)	0100 (0x4)	3, entry	System (SYS)
TRAPL	TRAP (section 9.14)	010100 (0x14)	1111 (0xF)	3, entry	System (SYS)
TST	TST (section 9.8)	010011 (0x13)	0101 (0x5)	9, entry	Single operand (SOP)
UTF8DEC	UTF8DEC (section 9.9)	001100 (0x0C)	01000001 (0x41)	2, entry	Bits and fields (BESC)
UTF8ENC	UTF8ENC (section 9.9)	001100 (0x0C)	01000010 (0x42)	2, entry	Bits and fields (BESC)
UTF8LEN	UTF8LEN (section 9.9)	001100 (0x0C)	01000000 (0x40)	2, entry	Bits and fields (BESC)
WAIT (alias of WFE)	WFE (section 9.14)	010100 (0x14)	0111 (0x7)	3, entry	System (SYS)
WFE	WFE (section 9.14)	010100 (0x14)	0111 (0x7)	3, entry	System (SYS)
XADD	XADD (section 9.13)	010001 (0x11)	00000100 (0x04)	2, entry	Atomics (ATOM)
XADDDB	XADD (section 9.13)	010001 (0x11)	00000101 (0x05)	2, entry	Atomics (ATOM)
XAND	XAND (section 9.13)	010001 (0x11)	00000110 (0x06)	2, entry	Atomics (ATOM)
XANDDB	XAND (section 9.13)	010001 (0x11)	00000111 (0x07)	2, entry	Atomics (ATOM)
XMAX	XMAX (section 9.13)	010001 (0x11)	00001100 (0x0C)	2, entry	Atomics (ATOM)
XMAXDB	XMAX (section 9.13)	010001 (0x11)	00001101 (0x0D)	2, entry	Atomics (ATOM)

Mnemonic	Entry	Opcode	Sub	Layouts	Group
XMAXU	XMAX (section 9.13)	010001 (0x11)	00010000 (0x10)	2, entry	Atomics (ATOM)
XMAXUDB	XMAX (section 9.13)	010001 (0x11)	00010001 (0x11)	2, entry	Atomics (ATOM)
XMIN	XMAX (section 9.13)	010001 (0x11)	00001110 (0x0E)	2, entry	Atomics (ATOM)
XMINDB	XMAX (section 9.13)	010001 (0x11)	00001111 (0x0F)	2, entry	Atomics (ATOM)
XMINU	XMAX (section 9.13)	010001 (0x11)	00010010 (0x12)	2, entry	Atomics (ATOM)
XMINUDB	XMAX (section 9.13)	010001 (0x11)	00010011 (0x13)	2, entry	Atomics (ATOM)
XNORPOP	XNORPOP (section 9.9)	001100 (0x0C)	00110000 (0x30)	2, entry	Bits and fields (BESC)
XOR	XOR (section 9.8)	011100 (0x1C)		9, entry	Moves, address formation and the ALU
XOR_	XAND (section 9.13)	010001 (0x11)	00001000 (0x08)	2, entry	Atomics (ATOM)
XOR_DB	XAND (section 9.13)	010001 (0x11)	00001001 (0x09)	2, entry	Atomics (ATOM)
XXOR	XAND (section 9.13)	010001 (0x11)	00001010 (0x0A)	2, entry	Atomics (ATOM)
XXORDB	XAND (section 9.13)	010001 (0x11)	00001011 (0x0B)	2, entry	Atomics (ATOM)

The opcodes, in order

Generated by `tools/mkmanual.py`; do not edit.
The opcodes of the original encoding. The bit layout of each is in the bit layouts of every opcode.

Page 1

Opcode	Use
000000 (0x00)	MOVQ
000001 (0x01)	reserved
000010 (0x02)	MOVL
000011 (0x03)	reserved
000100 (0x04)	MOVW
000101 (0x05)	reserved
000110 (0x06)	MOVB
000111 (0x07)	reserved
001000 (0x08)	MOVZL
001001 (0x09)	MOVZW
001010 (0x0A)	MOVZB
001011 (0x0B)	LEA
001100 (0x0C)	CLZ, CTZ, CLO, CTO, POPCNT and 17 more (sub-opcodes in the entries)
001101 (0x0D)	MCPY, MMOV, MSET, MCMP, MSCN and 2 more (sub-opcodes in the entries)
001110 (0x0E)	DIV, DIVU, MOD, MODU, MULHU and 6 more (sub-opcodes in the entries)
001111 (0x0F)	MOVUB, MOVUW, MOVUL, MOVUQ, ADDM and 3 more (sub-opcodes in the entries)
010000 (0x10)	FDIV, FSQRT, FADD32, FSUB32, FMUL32 and 49 more (sub-opcodes in the entries)
010001 (0x11)	CAS, CASDB, SWAP, SWAPDB, XADD and 17 more (sub-opcodes in the entries)
010010 (0x12)	JMPI, CALLI, JTAG
010011 (0x13)	CLR, COM, INC, DEC, NEG and 11 more (sub-opcodes in the entries)
010100 (0x14)	NOP, RTI, RTT, RET, HALT and 29 more (sub-opcodes in the entries)
010101 (0x15)	BTAG, BNTAG
010110 (0x16)	ADD
010111 (0x17)	SUB
011000 (0x18)	BIC
011001 (0x19)	BIS
011010 (0x1A)	CMP
011011 (0x1B)	BIT
011100 (0x1C)	XOR
011101 (0x1D)	AND
011110 (0x1E)	MUL
011111 (0x1F)	MULH
100000 (0x20)	SHL
100001 (0x21)	SHR
100010 (0x22)	SAR
100011 (0x23)	spare ALU slot (decodes, traps)
100100 (0x24)	FADD
100101 (0x25)	FSUB
100110 (0x26)	FMUL
100111 (0x27)	FCMP

Opcode	Use
101000 (0x28)	CBNE, CBEQ, CBPL, CBMI, CBVC and 9 more (sub-opcodes in the entries)
101001 (0x29)	reserved
101010 (0x2A)	reserved
101011 (0x2B)	reserved
101100 (0x2C)	reserved

Opcode	Use
101101 (0x2D)	reserved
101110 (0x2E)	reserved
101111 (0x2F)	reserved
110000 (0x30)	BR
110001 (0x31)	BNE
110010 (0x32)	BEQ
110011 (0x33)	BPL
110100 (0x34)	BMI
110101 (0x35)	BVC
110110 (0x36)	BVS
110111 (0x37)	BCC
111000 (0x38)	BCS
111001 (0x39)	BGE
111010 (0x3A)	BLT
111011 (0x3B)	BGT
111100 (0x3C)	BLE
111101 (0x3D)	BHI
111110 (0x3E)	BLOS
111111 (0x3F)	SOB

Page 2

Opcode	Name
000000 (0x00)	ld.q
000001 (0x01)	st.q
000010 (0x02)	ld.l
000011 (0x03)	st.l
000100 (0x04)	ld.w
000101 (0x05)	st.w
000110 (0x06)	ld.b
000111 (0x07)	st.b
001000 (0x08)	ldz.l
001001 (0x09)	ldz.w
001010 (0x0A)	ldz.b
001011 (0x0B)	lea.sp
001100 (0x0C)	ldq.sp0
001101 (0x0D)	ldq.sp1
001110 (0x0E)	stq.sp
010000 (0x10)	ldl.sp0
010001 (0x11)	ldl.sp1
010010 (0x12)	stl.sp0
010011 (0x13)	stl.sp1
010100 (0x14)	ldq.d1
010101 (0x15)	ldq.d2
010110 (0x16)	ldq.d3
010111 (0x17)	ldq.d4
011000 (0x18)	stq.d1
011001 (0x19)	stq.d2
011010 (0x1A)	ldl.d1
011011 (0x1B)	ldl.d2
011100 (0x1C)	ldl.d3
011101 (0x1D)	ldl.d4
011110 (0x1E)	pushm8
011111 (0x1F)	popm8
100000 (0x20)	sub.sp
100001 (0x21)	add.sp
100010 (0x22)	movq.i
100011 (0x23)	shl.i
100100 (0x24)	shr.i
100101 (0x25)	sar.i
100110 (0x26)	and.i
100111 (0x27)	bis.i
101000 (0x28)	xor.i

Opcode	Name
101001 (0x29)	add.i

Opcode	Name
101010 (0x2A)	pushx
101011 (0x2B)	popx
101100 (0x2C)	isync
101101 (0x2D)	hint

The bit layouts of every opcode

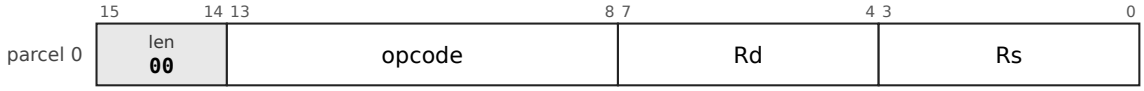
Every opcode of the original encoding has a chart here: one for each distinct layout (class of instruction, length, and body) that the grammar admits. The fields and their widths are the bit rules of `spec/pdp64.hbnf.in` (parcel 0 is `len opcode reg low`, and the body that follows is chosen by the guards of its alternatives), the numbers are the enumerations of `spec/pdp64-mnemonics.hbnf`, and what a field is called is `spec/pdp64-layout.hbnf`. The charts are generated from those files by `tools/mkmanual.py`; changing a field in a grammar changes the chart.

How to read a chart. Each row is one 16-bit parcel, bit 15 on the left, the bit numbers above the boxes. A shaded box has a fixed value, written in it; a white box is a field of the instruction. Fields are named `opN.part` for the part of the Nth operand of the assembly syntax (`op1.d` is the displacement of the first operand); a field the layout splits is `name[1]`, `name[2]` from the top. A register field of 3 or 4 bits holds an index into the hot-register table of the ABI, not a register number.

The charts

Moves, address formation and the ALU

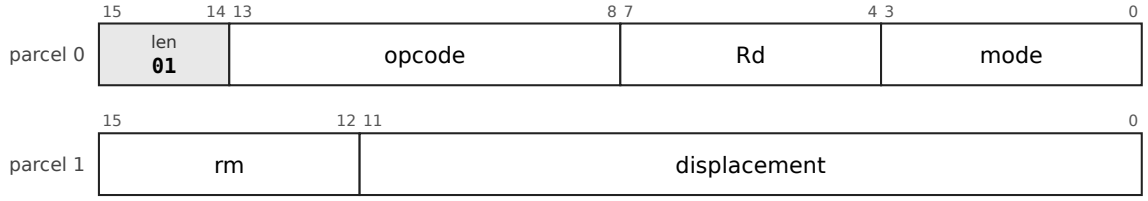
r20-mem-0: Moves, address formation and the ALU, one parcel (`len = 00`)



`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

r20-mem-1: Moves, address formation and the ALU, two parcels (`len = 01`)

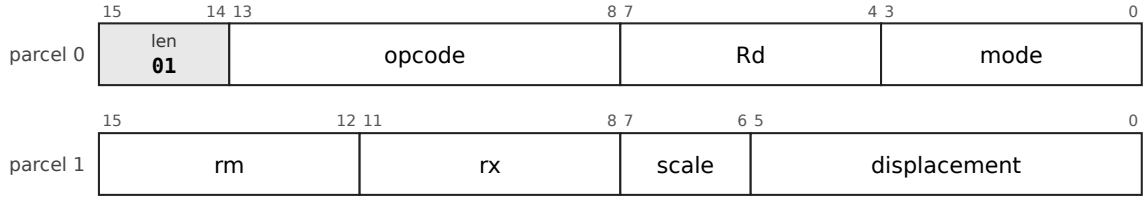


Addressing modes (`low`) that use this layout: `base` 0000, `postinc` 0110, `predec` 0111.

`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

r20-mem-1-2: Moves, address formation and the ALU, two parcels (`len = 01`)



Addressing modes (`low`) that use this layout: `index` 0001.

`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)

Mnemonic	Opcode
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

r20-mem-1-3: Moves, address formation and the ALU, two parcels (`len = 01`)

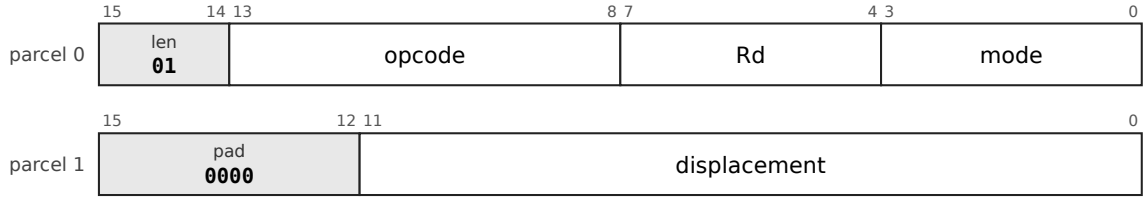


Addressing modes (`low`) that use this layout: `pcrel` 0010, `imm` 0100, `abs` 0101.

`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

r20-mem-1-4: Moves, address formation and the ALU, two parcels (`len = 01`)

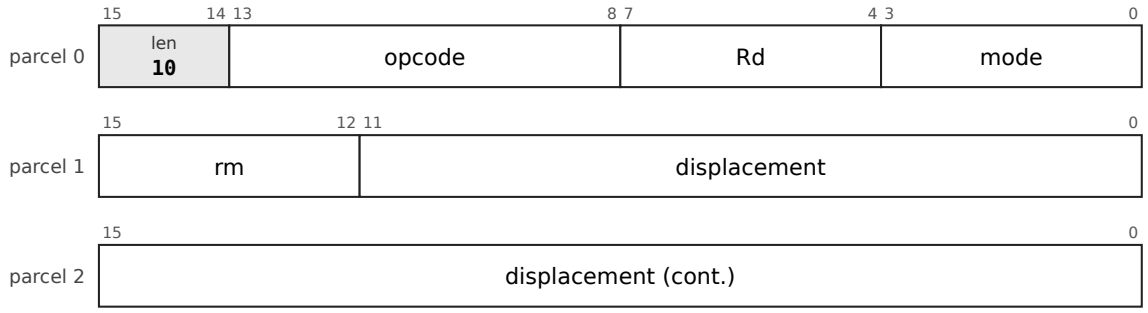


Addressing modes (`low`) that use this layout: `dp` 0011.

`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

r20-mem-2: Moves, address formation and the ALU, three parcels (`len = 10`)



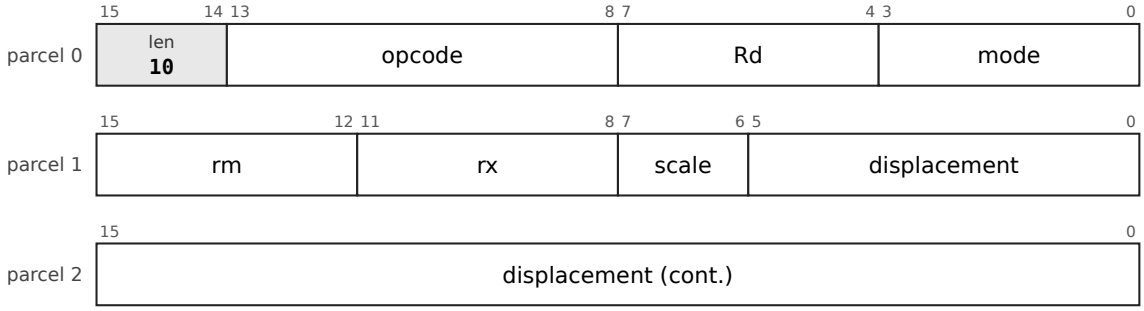
Addressing modes (`low`) that use this layout: `base` 0000, `postinc` 0110, `predec` 0111.

`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)

Mnemonic	Opcode
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

r20-mem-2-2: Moves, address formation and the ALU, three parcels (`len = 10`)

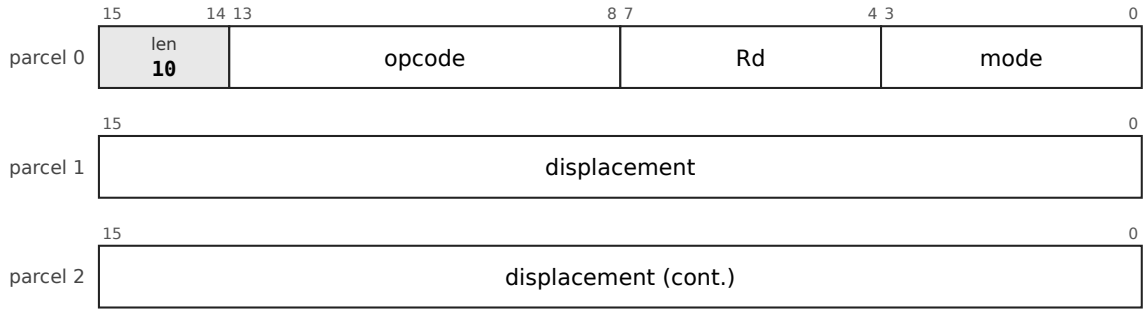


Addressing modes (`low`) that use this layout: `index` 0001.
`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)

Mnemonic	Opcode
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

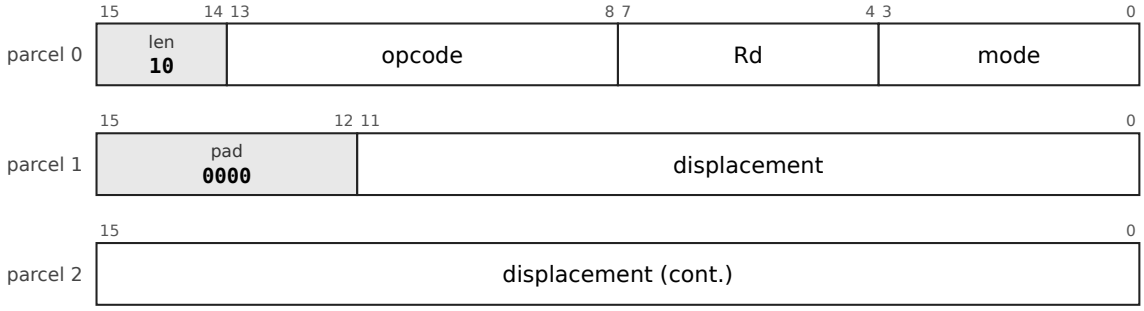
r20-mem-2-3: Moves, address formation and the ALU, three parcels (`len = 10`)



Addressing modes (`low`) that use this layout: `pcrel` 0010, `imm` 0100, `abs` 0101.
`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

r20-mem-2-4: Moves, address formation and the ALU, three parcels (`len = 10`)



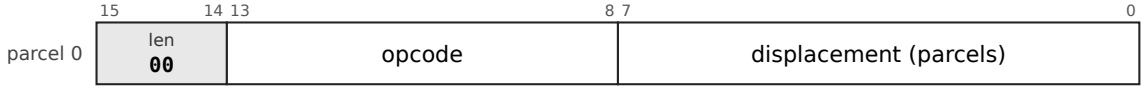
Addressing modes (`low`) that use this layout: `dp` 0011.

`reg` is the register `Rd` (the source of a store); `low` is a second register (`Rs`) at `len = 00` and the addressing mode from `len = 01`; the extension parcels hold the operand (section 9.5).

Mnemonic	Opcode
MOVQ	000000 (0x00), 000001 (0x01)
MOVL	000010 (0x02), 000011 (0x03)
MOVW	000100 (0x04), 000101 (0x05)
MOVB	000110 (0x06), 000111 (0x07)
MOVZL	001000 (0x08)
MOVZW	001001 (0x09)
MOVZB	001010 (0x0A)
LEA	001011 (0x0B)
ADD	010110 (0x16)
SUB	010111 (0x17)
BIC	011000 (0x18)
BIS	011001 (0x19)
CMP	011010 (0x1A)
BIT	011011 (0x1B)
XOR	011100 (0x1C)
AND	011101 (0x1D)
MUL	011110 (0x1E)
MULH	011111 (0x1F)
SHL	100000 (0x20)
SHR	100001 (0x21)
SAR	100010 (0x22)
FADD	100100 (0x24)
FSUB	100101 (0x25)
FMUL	100110 (0x26)
FCMP	100111 (0x27)

Conditional branches

r20-br-0: Conditional branches, one parcel (`len = 00`)

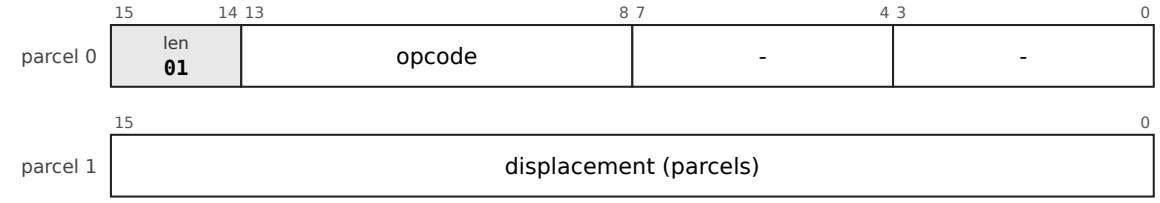


The displacement is signed and counts 2-byte parcels from the first parcel of the instruction.

Mnemonic	Opcode
BR	110000 (0x30)
BNE	110001 (0x31)
BEQ	110010 (0x32)
BPL	110011 (0x33)
BMI	110100 (0x34)
BVC	110101 (0x35)
BVS	110110 (0x36)
BCC	110111 (0x37)
BCS	111000 (0x38)

Mnemonic	Opcode
BGE	111001 (0x39)
BLT	111010 (0x3A)
BGT	111011 (0x3B)
BLE	111100 (0x3C)
BHI	111101 (0x3D)
BLOS	111110 (0x3E)

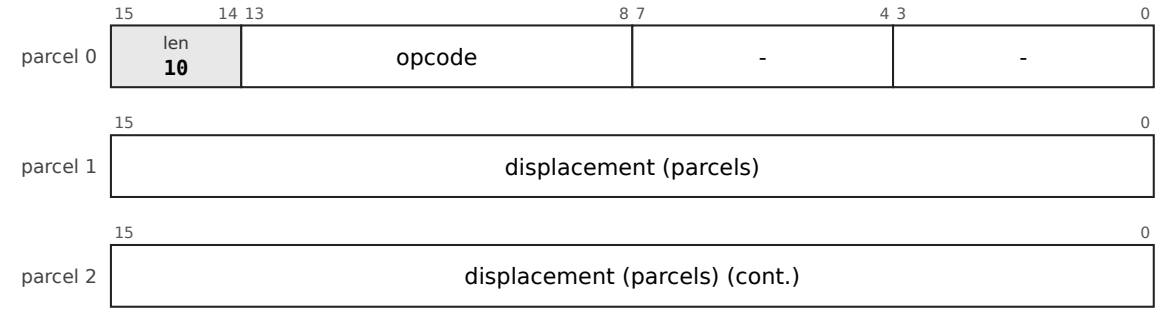
r20-br-1: Conditional branches, two parcels (`len = 01`)



The displacement is signed and counts 2-byte parcels from the first parcel of the instruction.

Mnemonic	Opcode
BR	110000 (0x30)
BNE	110001 (0x31)
BEQ	110010 (0x32)
BPL	110011 (0x33)
BMI	110100 (0x34)
BVC	110101 (0x35)
BVS	110110 (0x36)
BCC	110111 (0x37)
BCS	111000 (0x38)
BGE	111001 (0x39)
BLT	111010 (0x3A)
BGT	111011 (0x3B)
BLE	111100 (0x3C)
BHI	111101 (0x3D)
BLOS	111110 (0x3E)

r20-br-2: Conditional branches, three parcels (`len = 10`)



The displacement is signed and counts 2-byte parcels from the first parcel of the instruction.

Mnemonic	Opcode
BR	110000 (0x30)
BNE	110001 (0x31)
BEQ	110010 (0x32)
BPL	110011 (0x33)
BMI	110100 (0x34)
BVC	110101 (0x35)
BVS	110110 (0x36)
BCC	110111 (0x37)
BCS	111000 (0x38)
BGE	111001 (0x39)
BLT	111010 (0x3A)
BGT	111011 (0x3B)

Mnemonic	Opcode
BLE	111100 (0x3C)
BHI	111101 (0x3D)
BLOS	111110 (0x3E)

Compare and branch

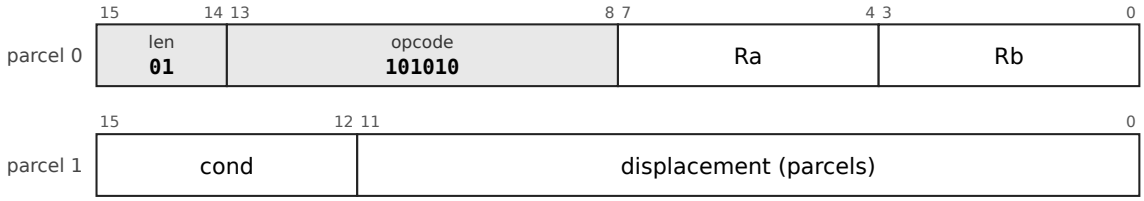
r20-cb-1: Compare and branch, two parcels (`len = 01`)



`low` is the condition; the immediate and the displacement are signed.

Mnemonic	Opcode	low
CBNE	101000 (0x28)	0001 (0x1)
CBEQ	101000 (0x28)	0010 (0x2)
CBPL	101000 (0x28)	0011 (0x3)
CBMI	101000 (0x28)	0100 (0x4)
CBVC	101000 (0x28)	0101 (0x5)
CBVS	101000 (0x28)	0110 (0x6)
CBCC	101000 (0x28)	0111 (0x7)
CBCS	101000 (0x28)	1000 (0x8)
CBGE	101000 (0x28)	1001 (0x9)
CBLT	101000 (0x28)	1010 (0xA)
CBGT	101000 (0x28)	1011 (0xB)
CBLE	101000 (0x28)	1100 (0xC)
CBHI	101000 (0x28)	1101 (0xD)
CBLOS	101000 (0x28)	1110 (0xE)

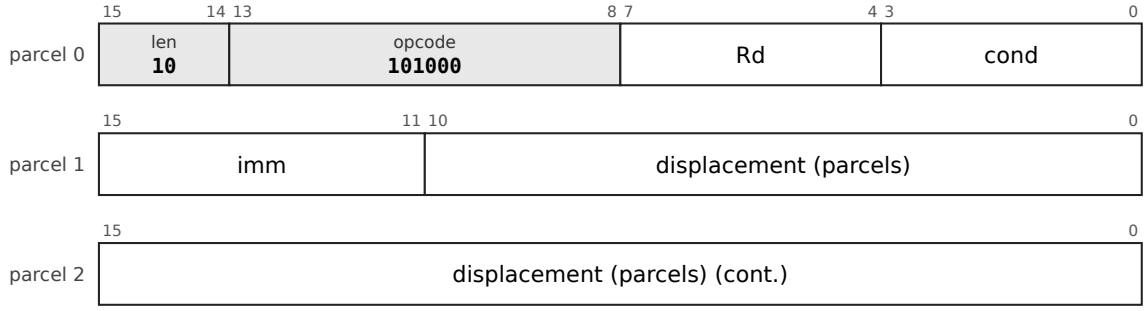
r20-cb-1-2: Compare two registers and branch, two parcels (`len = 01`)



Parcel 1 starts with the condition; `Ra` and `Rb` are compared.

Mnemonic	Opcode	low
CBNE	101010 (0x2A)	0001 (0x1)
CBEQ	101010 (0x2A)	0010 (0x2)
CBPL	101010 (0x2A)	0011 (0x3)
CBMI	101010 (0x2A)	0100 (0x4)
CBVC	101010 (0x2A)	0101 (0x5)
CBVS	101010 (0x2A)	0110 (0x6)
CBCC	101010 (0x2A)	0111 (0x7)
CBCS	101010 (0x2A)	1000 (0x8)
CBGE	101010 (0x2A)	1001 (0x9)
CBLT	101010 (0x2A)	1010 (0xA)
CBGT	101010 (0x2A)	1011 (0xB)
CBLE	101010 (0x2A)	1100 (0xC)
CBHI	101010 (0x2A)	1101 (0xD)
CBLOS	101010 (0x2A)	1110 (0xE)

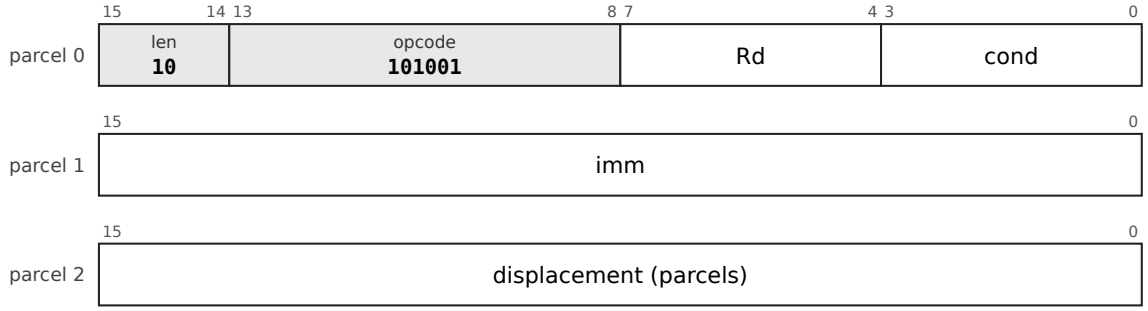
r20-cb-2: Compare and branch, three parcels (`len = 10`)



`low` is the condition; the immediate and the displacement are signed.

Mnemonic	Opcode	`low`
CBNE	101000 (0x28)	0001 (0x1)
CBEQ	101000 (0x28)	0010 (0x2)
CBPL	101000 (0x28)	0011 (0x3)
CBMI	101000 (0x28)	0100 (0x4)
CBVC	101000 (0x28)	0101 (0x5)
CBVS	101000 (0x28)	0110 (0x6)
CBCC	101000 (0x28)	0111 (0x7)
CBCS	101000 (0x28)	1000 (0x8)
CBGE	101000 (0x28)	1001 (0x9)
CBLT	101000 (0x28)	1010 (0xA)
CBGT	101000 (0x28)	1011 (0xB)
CBLE	101000 (0x28)	1100 (0xC)
CBHI	101000 (0x28)	1101 (0xD)
CBLOS	101000 (0x28)	1110 (0xE)

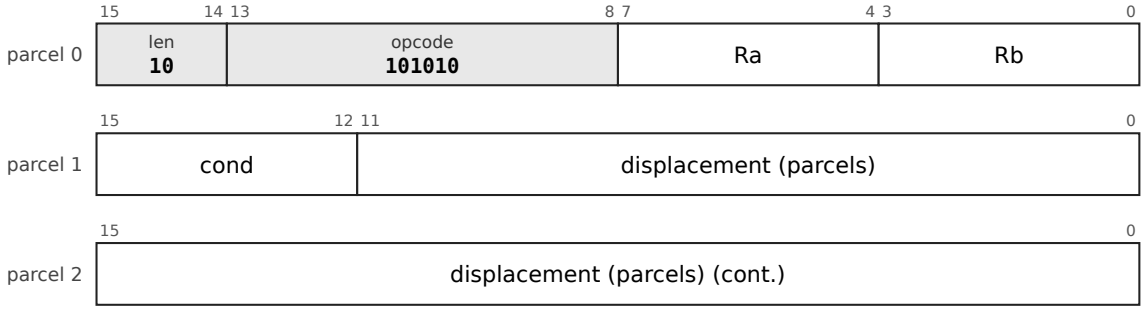
r20-cb-2-2: Compare and branch, three parcels (`len = 10`)



`low` is the condition; the immediate and the displacement are signed.

Mnemonic	Opcode	`low`
CBNE	101001 (0x29)	0001 (0x1)
CBEQ	101001 (0x29)	0010 (0x2)
CBPL	101001 (0x29)	0011 (0x3)
CBMI	101001 (0x29)	0100 (0x4)
CBVC	101001 (0x29)	0101 (0x5)
CBVS	101001 (0x29)	0110 (0x6)
CBCC	101001 (0x29)	0111 (0x7)
CBCS	101001 (0x29)	1000 (0x8)
CBGE	101001 (0x29)	1001 (0x9)
CBLT	101001 (0x29)	1010 (0xA)
CBGT	101001 (0x29)	1011 (0xB)
CBLE	101001 (0x29)	1100 (0xC)
CBHI	101001 (0x29)	1101 (0xD)
CBLOS	101001 (0x29)	1110 (0xE)

r20-cb-2-3: Compare two registers and branch, three parcels (`len = 10`)

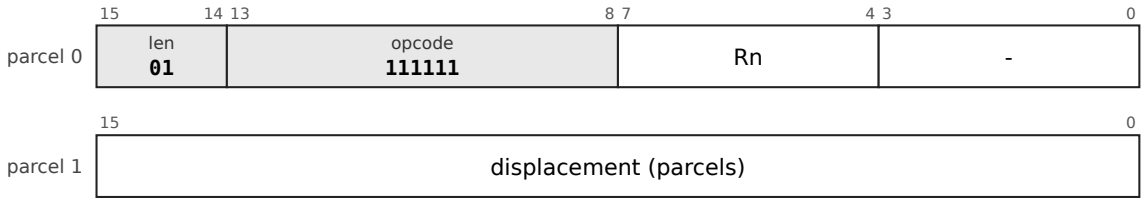


Parcel 1 starts with the condition; `Ra` and `Rb` are compared.

Mnemonic	Opcode	low
CBNE	101010 (0x2A)	0001 (0x1)
CBEQ	101010 (0x2A)	0010 (0x2)
CBPL	101010 (0x2A)	0011 (0x3)
CBMI	101010 (0x2A)	0100 (0x4)
CBVC	101010 (0x2A)	0101 (0x5)
CBVS	101010 (0x2A)	0110 (0x6)
CBCC	101010 (0x2A)	0111 (0x7)
CBCS	101010 (0x2A)	1000 (0x8)
CBGE	101010 (0x2A)	1001 (0x9)
CBLT	101010 (0x2A)	1010 (0xA)
CBGT	101010 (0x2A)	1011 (0xB)
CBLE	101010 (0x2A)	1100 (0xC)
CBHI	101010 (0x2A)	1101 (0xD)
CBLOS	101010 (0x2A)	1110 (0xE)

SOB

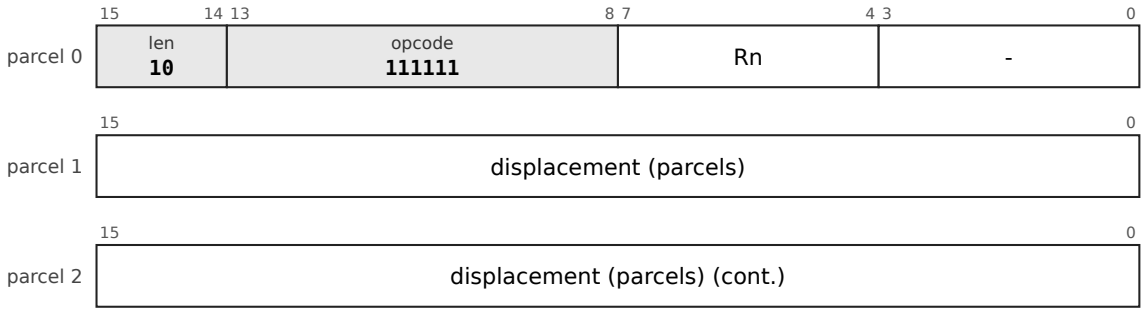
r20-sob-1: SOB, two parcels (`len = 01`)



The displacement is signed and counts parcels.

Mnemonic	Opcode
SOB	111111 (0x3F)

r20-sob-2: SOB, three parcels (`len = 10`)

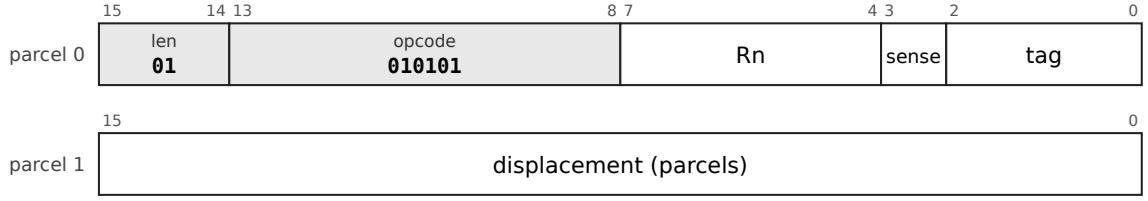


The displacement is signed and counts parcels.

Mnemonic	Opcode
SOB	111111 (0x3F)

Tag branches

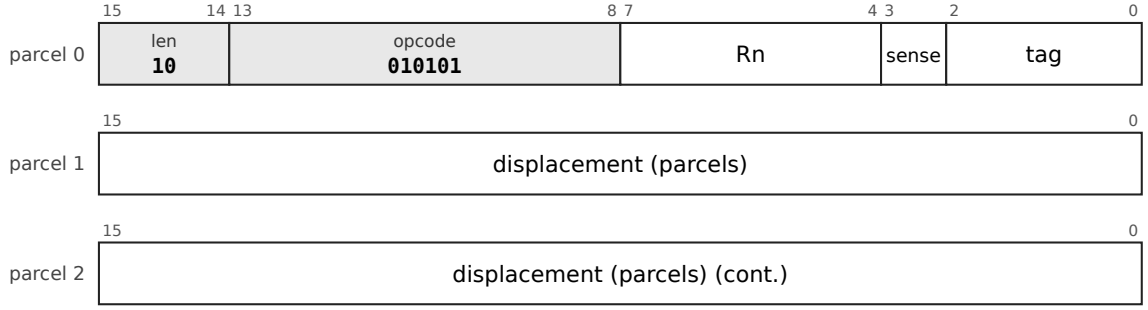
r20-tag-1: Tag branches, two parcels (`len = 01`)



`sense` is 0 for `BTAG` and 1 for `BNTAG`; `tag` is the 3-bit tag.

Mnemonic	Opcode
BTAG	010101 (0x15)
BNTAG	010101 (0x15)

r20-tag-2: Tag branches, three parcels (`len = 10`)

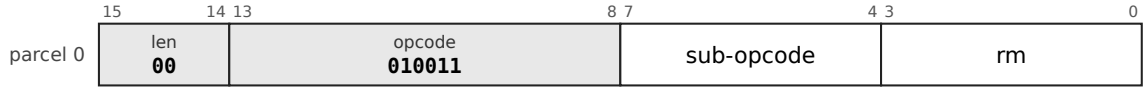


`sense` is 0 for `BTAG` and 1 for `BNTAG`; `tag` is the 3-bit tag.

Mnemonic	Opcode
BTAG	010101 (0x15)
BNTAG	010101 (0x15)

Single operand (SOP)

r20-sop-0: Single operand (SOP), one parcel (`len = 00`)

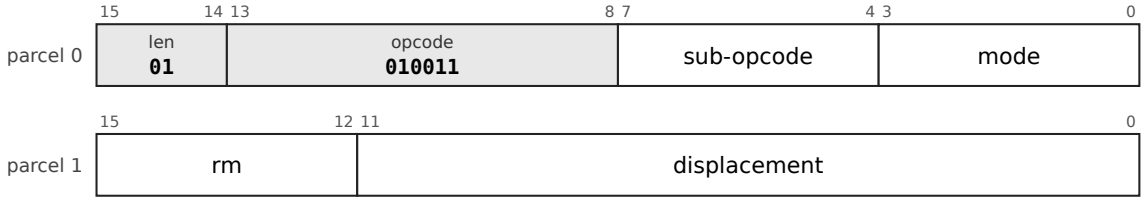


The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)
JMP	010011 (0x13)	1100 (0xC)

Mnemonic	Opcode	reg
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)
POP	010011 (0x13)	1111 (0xF)

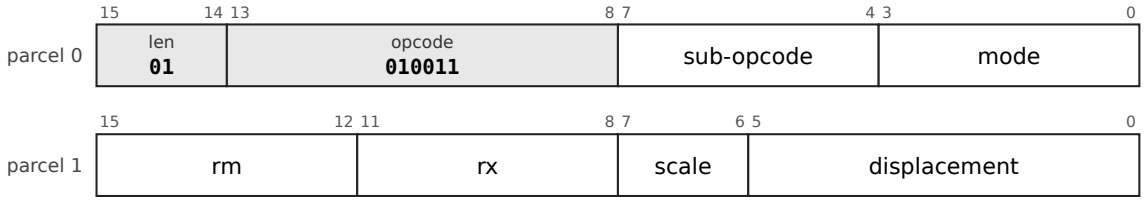
r20-sop-1: Single operand (SOP), two parcels (`len = 01`)



Addressing modes (`low`) that use this layout: `base` 0000, `postinc` 0110, `predec` 0111.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)
JMP	010011 (0x13)	1100 (0xC)
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)
POP	010011 (0x13)	1111 (0xF)

r20-sop-1-2: Single operand (SOP), two parcels (`len = 01`)

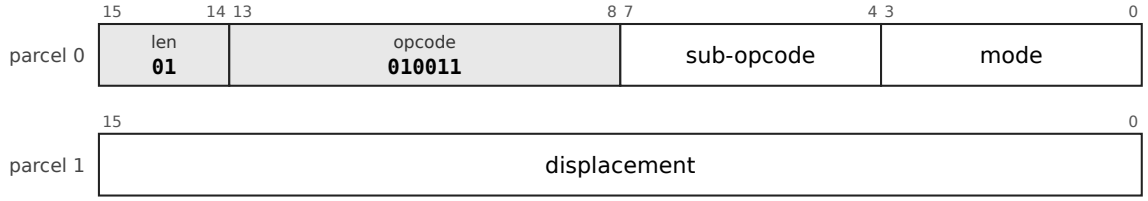


Addressing modes (`low`) that use this layout: `index` 0001.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)
JMP	010011 (0x13)	1100 (0xC)
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)

Mnemonic	Opcode	reg
POP	010011 (0x13)	1111 (0xF)

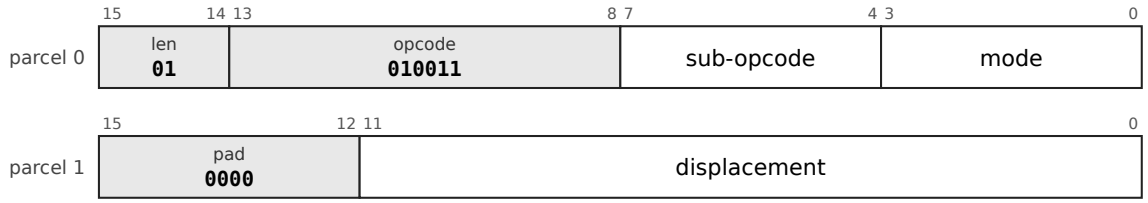
r20-sop-1-3: Single operand (SOP), two parcels (`len = 01`)



Addressing modes (`low`) that use this layout: `pcrel` 0010, `imm` 0100, `abs` 0101.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)
JMP	010011 (0x13)	1100 (0xC)
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)
POP	010011 (0x13)	1111 (0xF)

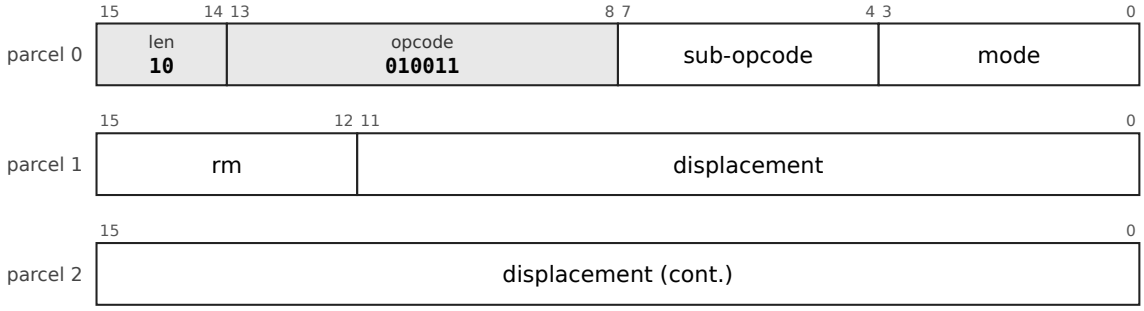
r20-sop-1-4: Single operand (SOP), two parcels (`len = 01`)



Addressing modes (`low`) that use this layout: `dp` 0011.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)
JMP	010011 (0x13)	1100 (0xC)
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)
POP	010011 (0x13)	1111 (0xF)

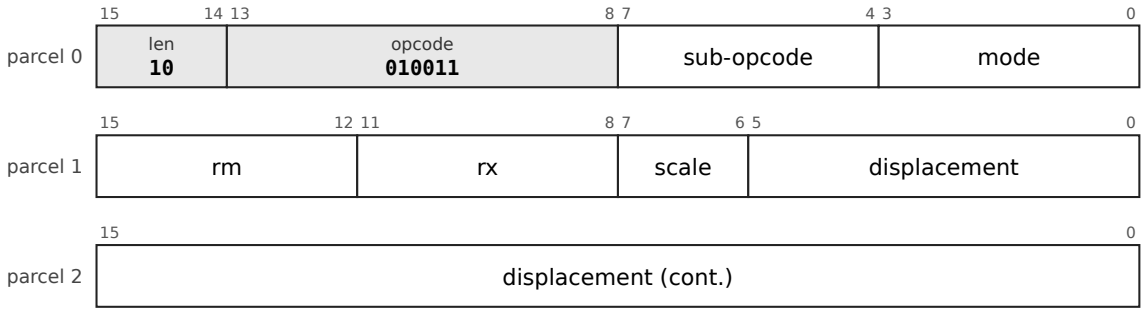
r20-sop-2: Single operand (SOP), three parcels (`len = 10`)



Addressing modes (`low`) that use this layout: `base` 0000, `postinc` 0110, `predec` 0111.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)
JMP	010011 (0x13)	1100 (0xC)
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)
POP	010011 (0x13)	1111 (0xF)

r20-sop-2-2: Single operand (SOP), three parcels (`len = 10`)

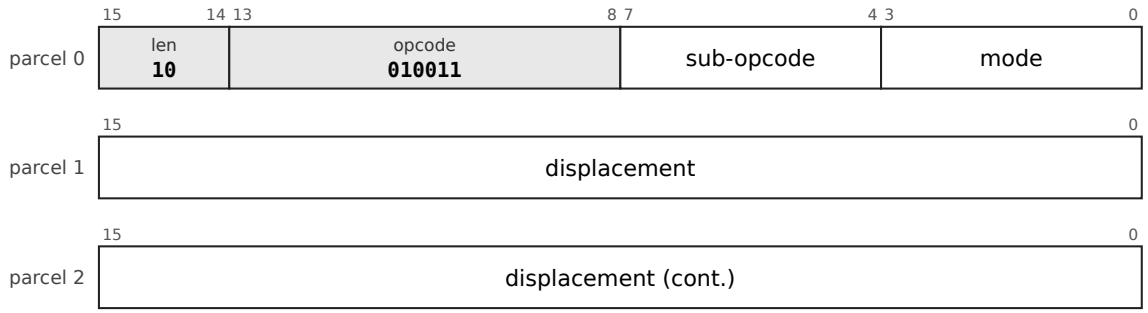


Addressing modes (`low`) that use this layout: `index` 0001.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)

Mnemonic	Opcode	reg
JMP	010011 (0x13)	1100 (0xC)
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)
POP	010011 (0x13)	1111 (0xF)

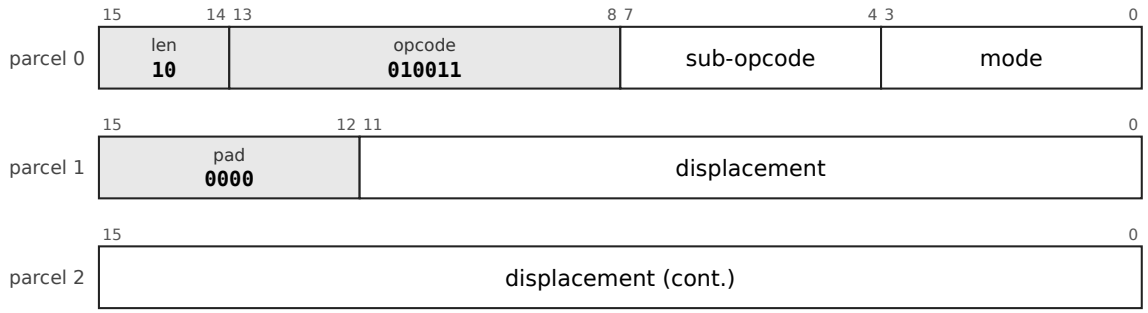
r20-sop-2-3: Single operand (SOP), three parcels (`len = 10`)



Addressing modes (`low`) that use this layout: `pcrel` 0010, `imm` 0100, `abs` 0101.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)
JMP	010011 (0x13)	1100 (0xC)
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)
POP	010011 (0x13)	1111 (0xF)

r20-sop-2-4: Single operand (SOP), three parcels (`len = 10`)



Addressing modes (`low`) that use this layout: `dp` 0011.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
CLR	010011 (0x13)	0000 (0x0)
COM	010011 (0x13)	0001 (0x1)
INC	010011 (0x13)	0010 (0x2)
DEC	010011 (0x13)	0011 (0x3)
NEG	010011 (0x13)	0100 (0x4)
TST	010011 (0x13)	0101 (0x5)

Mnemonic	Opcode	reg
ADC	010011 (0x13)	0110 (0x6)
SBC	010011 (0x13)	0111 (0x7)
ASL	010011 (0x13)	1000 (0x8)
ASR	010011 (0x13)	1001 (0x9)
PUSHM	010011 (0x13)	1010 (0xA)
POPM	010011 (0x13)	1011 (0xB)
JMP	010011 (0x13)	1100 (0xC)
CALL	010011 (0x13)	1101 (0xD)
PUSH	010011 (0x13)	1110 (0xE)
POP	010011 (0x13)	1111 (0xF)

Indirect transfers (XFER)

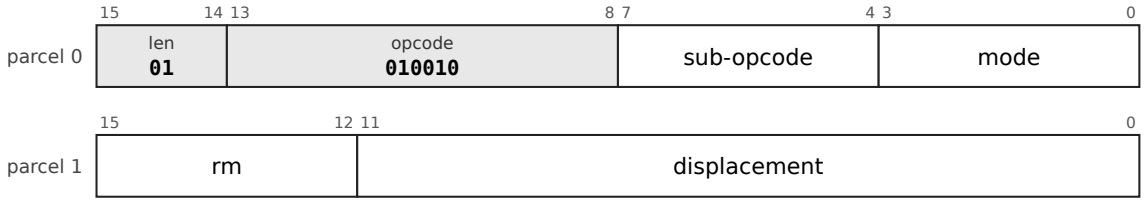
r20-xfer-0: Indirect transfers (XFER), one parcel (`len = 00`)



The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
JMPI	010010 (0x12)	0000 (0x0)
CALLI	010010 (0x12)	0001 (0x1)
JTAG	010010 (0x12)	0010 (0x2)

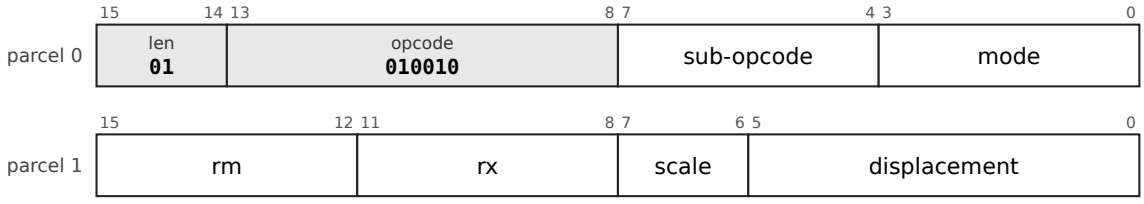
r20-xfer-1: Indirect transfers (XFER), two parcels (`len = 01`)



Addressing modes (`low`) that use this layout: `base` 0000, `postinc` 0110, `predec` 0111.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
JMPI	010010 (0x12)	0000 (0x0)
CALLI	010010 (0x12)	0001 (0x1)
JTAG	010010 (0x12)	0010 (0x2)

r20-xfer-1-2: Indirect transfers (XFER), two parcels (`len = 01`)



Addressing modes (`low`) that use this layout: `index` 0001.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	reg
JMPI	010010 (0x12)	0000 (0x0)
CALLI	010010 (0x12)	0001 (0x1)
JTAG	010010 (0x12)	0010 (0x2)

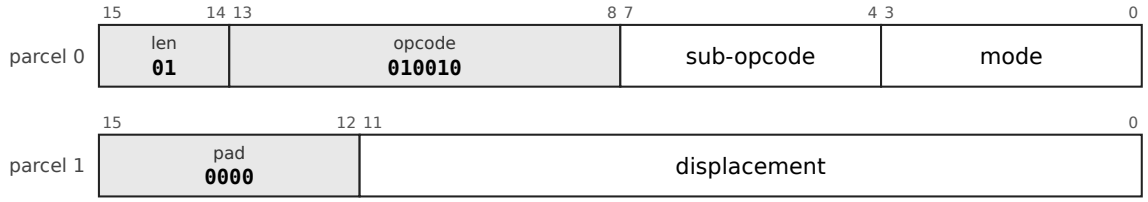
r20-xfer-1-3: Indirect transfers (XFER), two parcels (`len = 01`)



Addressing modes (`low`) that use this layout: `pcrel` 0010, `imm` 0100, `abs` 0101.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
JMPI	010010 (0x12)	0000 (0x0)
CALLI	010010 (0x12)	0001 (0x1)
JTAG	010010 (0x12)	0010 (0x2)

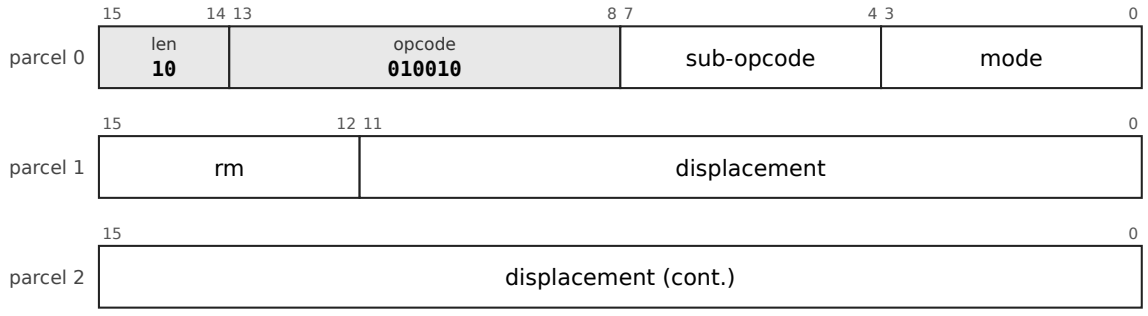
r20-xfer-1-4: Indirect transfers (XFER), two parcels (`len = 01`)



Addressing modes (`low`) that use this layout: `dp` 0011.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
JMPI	010010 (0x12)	0000 (0x0)
CALLI	010010 (0x12)	0001 (0x1)
JTAG	010010 (0x12)	0010 (0x2)

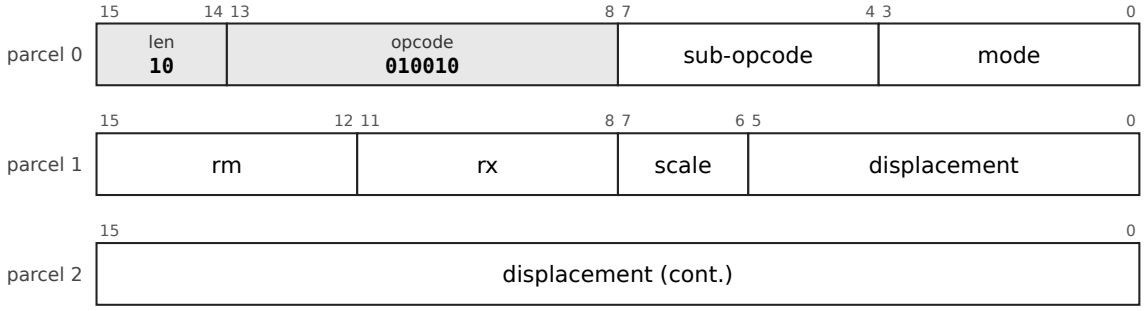
r20-xfer-2: Indirect transfers (XFER), three parcels (`len = 10`)



Addressing modes (`low`) that use this layout: `base` 0000, `postinc` 0110, `predec` 0111.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
JMPI	010010 (0x12)	0000 (0x0)
CALLI	010010 (0x12)	0001 (0x1)
JTAG	010010 (0x12)	0010 (0x2)

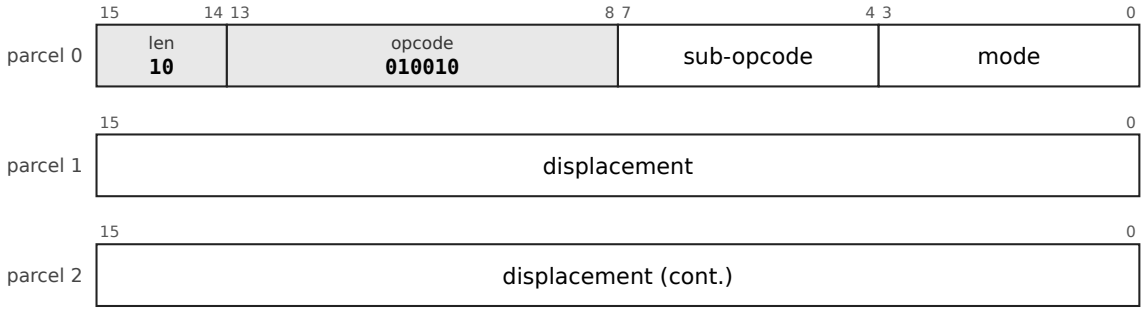
r20-xfer-2-2: Indirect transfers (XFER), three parcels (`len = 10`)



Addressing modes (`low`) that use this layout: `index` 0001.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`JMPI`	`010010 (0x12)`	`0000 (0x0)`
`CALLI`	`010010 (0x12)`	`0001 (0x1)`
`JTAG`	`010010 (0x12)`	`0010 (0x2)`

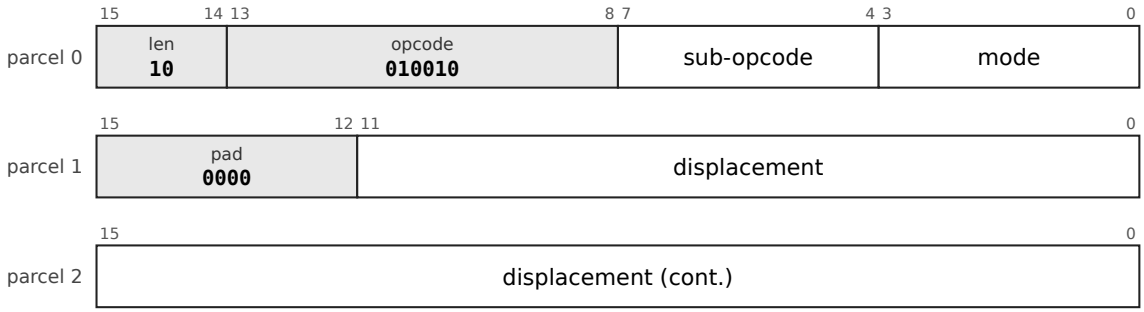
r20-xfer-2-3: Indirect transfers (XFER), three parcels (`len = 10`)



Addressing modes (`low`) that use this layout: `pcrel` 0010, `imm` 0100, `abs` 0101.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`JMPI`	`010010 (0x12)`	`0000 (0x0)`
`CALLI`	`010010 (0x12)`	`0001 (0x1)`
`JTAG`	`010010 (0x12)`	`0010 (0x2)`

r20-xfer-2-4: Indirect transfers (XFER), three parcels (`len = 10`)



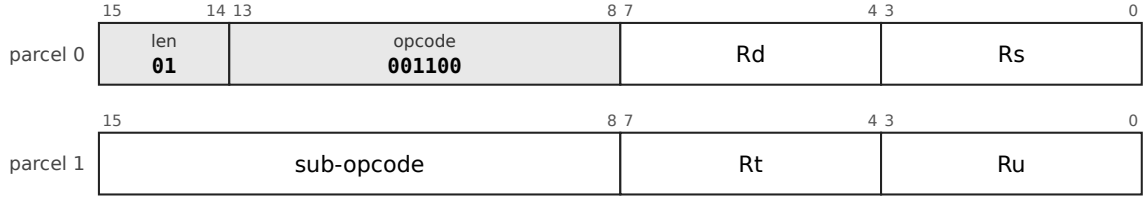
Addressing modes (`low`) that use this layout: `dp` 0011.
The `reg` field is the sub-opcode; the operand is `rm` with its mode, at any length.

Mnemonic	Opcode	`reg`
`JMPI`	`010010 (0x12)`	`0000 (0x0)`

Mnemonic	Opcode	reg
CALLI	010010 (0x12)	0001 (0x1)
JTAG	010010 (0x12)	0010 (0x2)

Bits and fields (BESC)

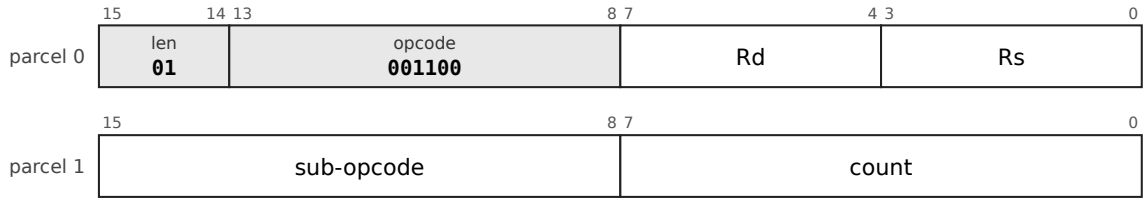
r20-besc-1: Bits and fields (BESC), two parcels (`len = 01`)



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
CLZ	001100 (0x0C)	00000000 (0x00)
CTZ	001100 (0x0C)	00000001 (0x01)
CLO	001100 (0x0C)	00000010 (0x02)
CTO	001100 (0x0C)	00000011 (0x03)
POPCNT	001100 (0x0C)	00000100 (0x04)
BSWAP	001100 (0x0C)	00000101 (0x05)
BSWAPW	001100 (0x0C)	00000110 (0x06)
BSWAPL	001100 (0x0C)	00000111 (0x07)
BITREV	001100 (0x0C)	00001000 (0x08)
ROR	001100 (0x0C)	00010000 (0x10)
DSHR	001100 (0x0C)	00100000 (0x20)
EXTB	001100 (0x0C)	00100001 (0x21)
EXTZB	001100 (0x0C)	00100010 (0x22)
INSB	001100 (0x0C)	00100011 (0x23)
XNORPOP	001100 (0x0C)	00110000 (0x30)
CLMULL	001100 (0x0C)	00110001 (0x31)
CLMULH	001100 (0x0C)	00110010 (0x32)
UTF8LEN	001100 (0x0C)	01000000 (0x40)
UTF8DEC	001100 (0x0C)	01000001 (0x41)
UTF8ENC	001100 (0x0C)	01000010 (0x42)

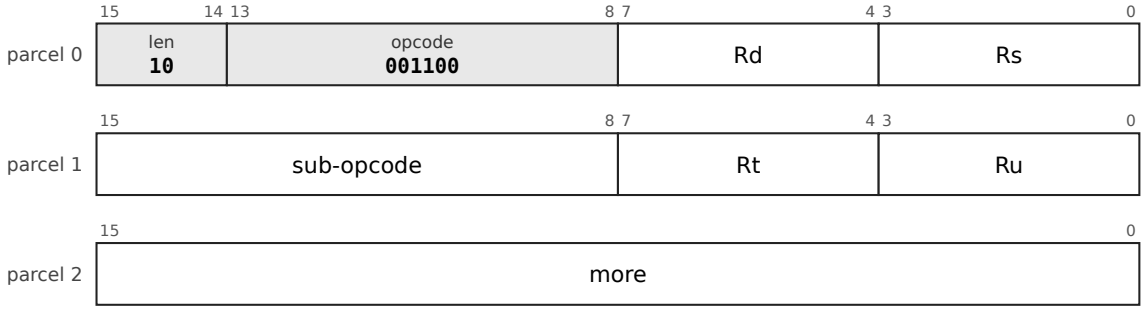
r20-besc-1-2: Bits and fields (BESC), two parcels (`len = 01`)



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
RORI	001100 (0x0C)	00010001 (0x11)
RORI32	001100 (0x0C)	00010010 (0x12)

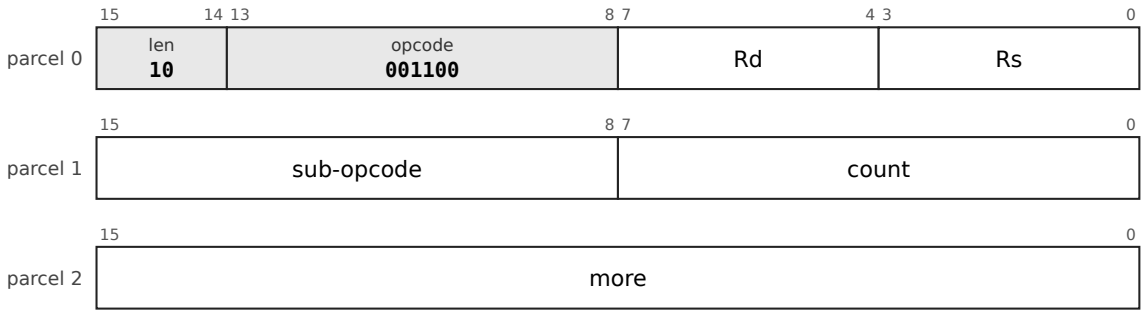
r20-besc-2: Bits and fields (BESC), three parcels (`len = 10`)



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
CLZ	001100 (0x0C)	00000000 (0x00)
CTZ	001100 (0x0C)	00000001 (0x01)
CLO	001100 (0x0C)	00000010 (0x02)
CTO	001100 (0x0C)	00000011 (0x03)
POPCNT	001100 (0x0C)	00000100 (0x04)
BSWAP	001100 (0x0C)	00000101 (0x05)
BSWAPW	001100 (0x0C)	00000110 (0x06)
BSWAPL	001100 (0x0C)	00000111 (0x07)
BITREV	001100 (0x0C)	00001000 (0x08)
ROR	001100 (0x0C)	00010000 (0x10)
XNORPOP	001100 (0x0C)	00110000 (0x30)
CLMULL	001100 (0x0C)	00110001 (0x31)
CLMULH	001100 (0x0C)	00110010 (0x32)
UTF8LEN	001100 (0x0C)	01000000 (0x40)
UTF8DEC	001100 (0x0C)	01000001 (0x41)
UTF8ENC	001100 (0x0C)	01000010 (0x42)

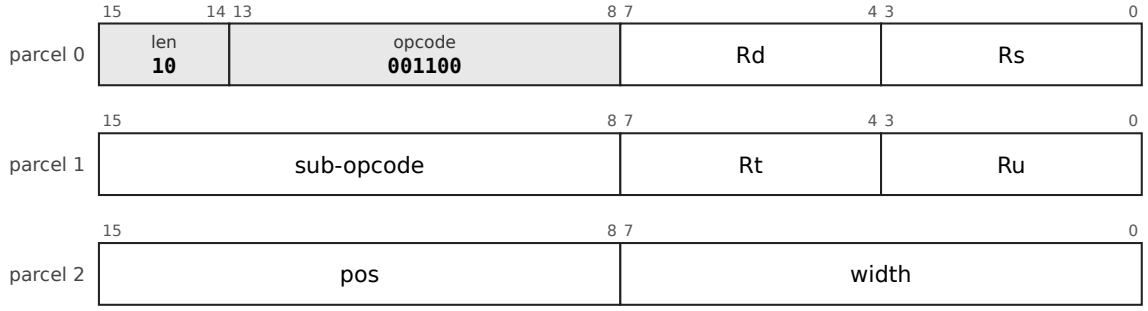
r20-besc-2-2: Bits and fields (BESC), three parcels (`len = 10`)



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
RORI	001100 (0x0C)	00010001 (0x11)
RORI32	001100 (0x0C)	00010010 (0x12)

r20-besc-2-3: Bits and fields (BESC), three parcels (`len = 10`)

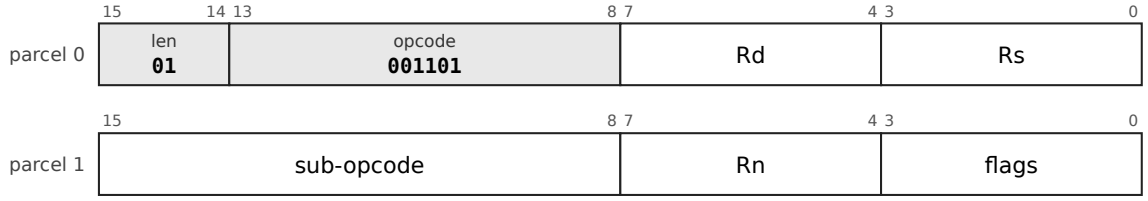


From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
DSHR	001100 (0x0C)	00100000 (0x20)
EXTB	001100 (0x0C)	00100001 (0x21)
EXTZB	001100 (0x0C)	00100010 (0x22)
INSB	001100 (0x0C)	00100011 (0x23)

Block memory (MBLK)

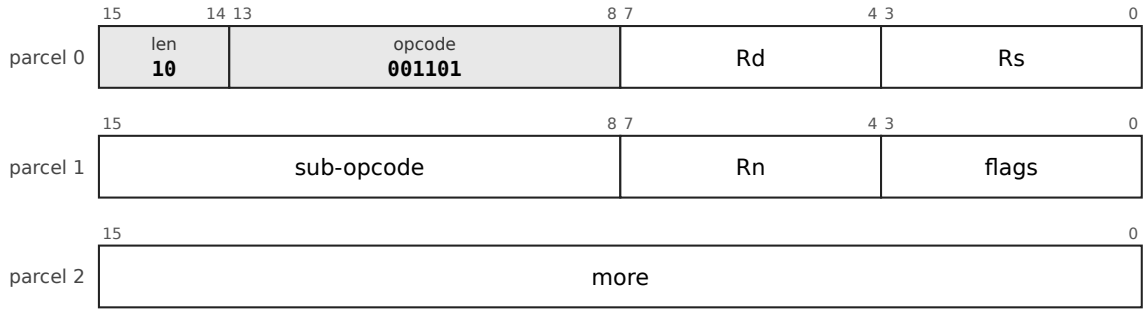
r20-mblk-1: Block memory (MBLK), two parcels (`len = 01`)



`reg` is the destination pointer, `low` the source pointer (or the value register), `Rn` the count register.

Mnemonic	Opcode	Sub-opcode
MCPY	001101 (0x0D)	00000000 (0x00)
MMOV	001101 (0x0D)	00000001 (0x01)
MSET	001101 (0x0D)	00000010 (0x02)
MCMP	001101 (0x0D)	00000011 (0x03)
MSCN	001101 (0x0D)	00000100 (0x04)
MSCNN	001101 (0x0D)	00000101 (0x05)
MVAL	001101 (0x0D)	00000110 (0x06)

r20-mblk-2: Block memory (MBLK), three parcels (`len = 10`)



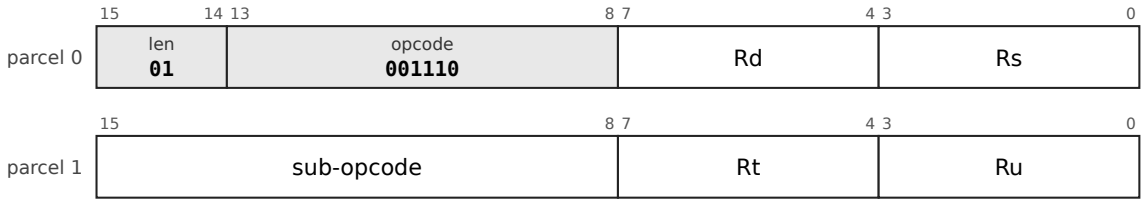
`reg` is the destination pointer, `low` the source pointer (or the value register), `Rn` the count register.

Mnemonic	Opcode	Sub-opcode
MCPY	001101 (0x0D)	00000000 (0x00)
MMOV	001101 (0x0D)	00000001 (0x01)
MSET	001101 (0x0D)	00000010 (0x02)

Mnemonic	Opcode	Sub-opcode
MCMP	001101 (0x0D)	00000011 (0x03)
MSCN	001101 (0x0D)	00000100 (0x04)
MSCNN	001101 (0x0D)	00000101 (0x05)
MVAL	001101 (0x0D)	00000110 (0x06)

Arithmetic (AESC)

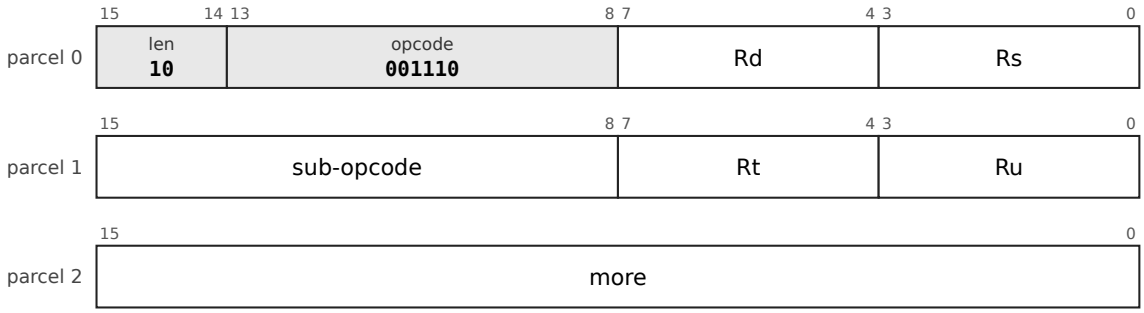
r20-aesc-1: Arithmetic (AESC), two parcels (`len = 01`)



From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
DIV	001110 (0x0E)	00000000 (0x00)
DIVU	001110 (0x0E)	00000001 (0x01)
MOD	001110 (0x0E)	00000010 (0x02)
MODU	001110 (0x0E)	00000011 (0x03)
MULHU	001110 (0x0E)	00000100 (0x04)
ABS	001110 (0x0E)	00001000 (0x08)
MIN	001110 (0x0E)	00010000 (0x10)
MINU	001110 (0x0E)	00010001 (0x11)
MAX	001110 (0x0E)	00010010 (0x12)
MAXU	001110 (0x0E)	00010011 (0x13)
SET	001110 (0x0E)	00100000 (0x20)

r20-aesc-2: Arithmetic (AESC), three parcels (`len = 10`)

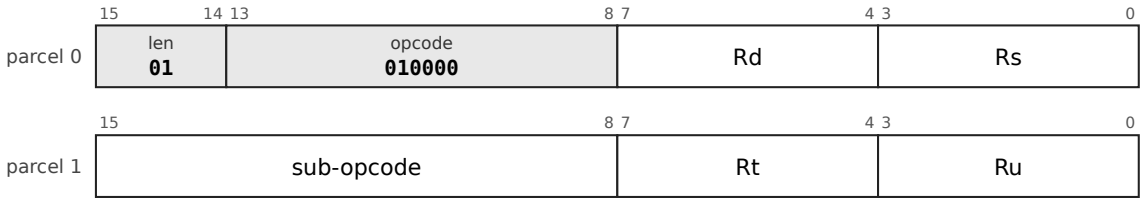


From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
DIV	001110 (0x0E)	00000000 (0x00)
DIVU	001110 (0x0E)	00000001 (0x01)
MOD	001110 (0x0E)	00000010 (0x02)
MODU	001110 (0x0E)	00000011 (0x03)
MULHU	001110 (0x0E)	00000100 (0x04)
ABS	001110 (0x0E)	00001000 (0x08)
MIN	001110 (0x0E)	00010000 (0x10)
MINU	001110 (0x0E)	00010001 (0x11)
MAX	001110 (0x0E)	00010010 (0x12)
MAXU	001110 (0x0E)	00010011 (0x13)
SET	001110 (0x0E)	00100000 (0x20)

Floating point (FESC)

r20-fesc-1: Floating point (FESC), two parcels (`len = 01`)



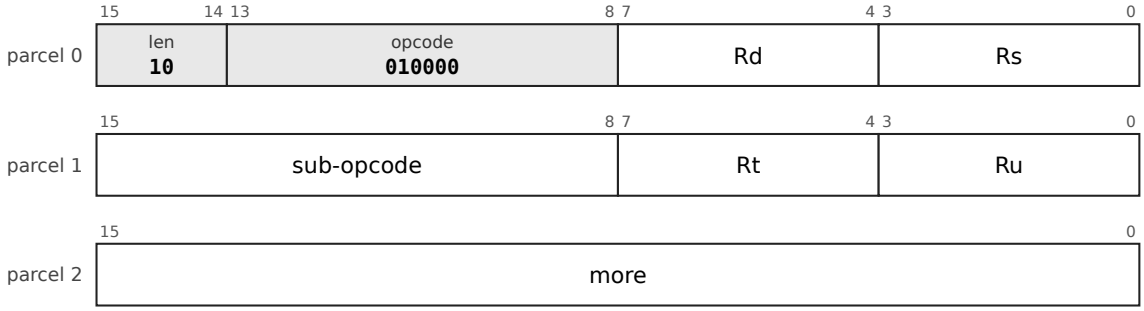
From `len = 01`. In this group `low` is a register, never a mode.

Mnemonic	Opcode	Sub-opcode
FDIV	010000 (0x10)	00000000 (0x00)
FSQRT	010000 (0x10)	00000001 (0x01)
FADD32	010000 (0x10)	00001000 (0x08)
FSUB32	010000 (0x10)	00001001 (0x09)
FMUL32	010000 (0x10)	00001010 (0x0A)
FDIV32	010000 (0x10)	00001011 (0x0B)
FSQRT32	010000 (0x10)	00001100 (0x0C)
FCMP32	010000 (0x10)	00001101 (0x0D)
FMADD	010000 (0x10)	00010000 (0x10)
FMSUB	010000 (0x10)	00010001 (0x11)
FNMADD	010000 (0x10)	00010010 (0x12)
FNMSUB	010000 (0x10)	00010011 (0x13)
FMADD32	010000 (0x10)	00011000 (0x18)
FMSUB32	010000 (0x10)	00011001 (0x19)
FNMADD32	010000 (0x10)	00011010 (0x1A)
FNMSUB32	010000 (0x10)	00011011 (0x1B)
FNEG	010000 (0x10)	00100000 (0x20)
FABS	010000 (0x10)	00100001 (0x21)
FCOPYSIGN	010000 (0x10)	00100010 (0x22)
FMIN	010000 (0x10)	00100011 (0x23)
FMAX	010000 (0x10)	00100100 (0x24)
FCLASS	010000 (0x10)	00101000 (0x28)
FCLASS32	010000 (0x10)	00101001 (0x29)
FRINT	010000 (0x10)	00110000 (0x30)
FRINTZ	010000 (0x10)	00110001 (0x31)
FRINTM	010000 (0x10)	00110010 (0x32)
FRINTP	010000 (0x10)	00110011 (0x33)
FRINTA	010000 (0x10)	00110100 (0x34)
FCVTDQ	010000 (0x10)	00111000 (0x38)
FCVTQD	010000 (0x10)	00111001 (0x39)
FCVTDS	010000 (0x10)	00111010 (0x3A)
FCVTSD	010000 (0x10)	00111011 (0x3B)
FCVTSL	010000 (0x10)	00111100 (0x3C)
FCVTLS	010000 (0x10)	00111101 (0x3D)
FCVTDQU	010000 (0x10)	00111110 (0x3E)
FCVTQUD	010000 (0x10)	00111111 (0x3F)
FSFLAGS	010000 (0x10)	01000000 (0x40)
FADD16	010000 (0x10)	01010000 (0x50)
FSUB16	010000 (0x10)	01010001 (0x51)
FMUL16	010000 (0x10)	01010010 (0x52)

Mnemonic	Opcode	Sub-opcode
FDIV16	010000 (0x10)	01010011 (0x53)
FSQRT16	010000 (0x10)	01010100 (0x54)
FCMP16	010000 (0x10)	01010101 (0x55)
FMADD16	010000 (0x10)	01011000 (0x58)
FMSUB16	010000 (0x10)	01011001 (0x59)
FNMADD16	010000 (0x10)	01011010 (0x5A)
FNMSUB16	010000 (0x10)	01011011 (0x5B)
FCLASS16	010000 (0x10)	01011100 (0x5C)
FCVTSH	010000 (0x10)	01100000 (0x60)
FCVTHS	010000 (0x10)	01100001 (0x61)
FCVTDH	010000 (0x10)	01100010 (0x62)

Mnemonic	Opcode	Sub-opcode
FCVTHD	010000 (0x10)	01100011 (0x63)
FCVTHL	010000 (0x10)	01100100 (0x64)
FCVTLH	010000 (0x10)	01100101 (0x65)

r20-fesc-2: Floating point (FESC), three parcels (`len = 10`)



From `len = 01`. In this group `low` is a register, never a mode.

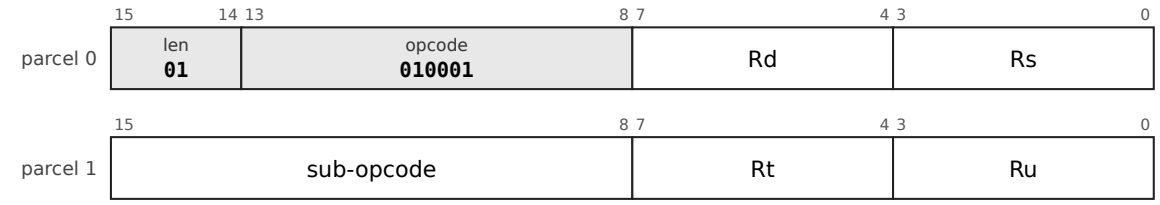
Mnemonic	Opcode	Sub-opcode
FDIV	010000 (0x10)	00000000 (0x00)
FSQRT	010000 (0x10)	00000001 (0x01)
FADD32	010000 (0x10)	00001000 (0x08)
FSUB32	010000 (0x10)	00001001 (0x09)
FMUL32	010000 (0x10)	00001010 (0x0A)
FDIV32	010000 (0x10)	00001011 (0x0B)
FSQRT32	010000 (0x10)	00001100 (0x0C)
FCMP32	010000 (0x10)	00001101 (0x0D)
FMADD	010000 (0x10)	00010000 (0x10)
FMSUB	010000 (0x10)	00010001 (0x11)
FNMADD	010000 (0x10)	00010010 (0x12)
FNMSUB	010000 (0x10)	00010011 (0x13)
FMADD32	010000 (0x10)	00011000 (0x18)
FMSUB32	010000 (0x10)	00011001 (0x19)
FNMADD32	010000 (0x10)	00011010 (0x1A)
FNMSUB32	010000 (0x10)	00011011 (0x1B)
FNEG	010000 (0x10)	00100000 (0x20)
FABS	010000 (0x10)	00100001 (0x21)
FCOPYSIGN	010000 (0x10)	00100010 (0x22)
FMIN	010000 (0x10)	00100011 (0x23)
FMAX	010000 (0x10)	00100100 (0x24)
FCLASS	010000 (0x10)	00101000 (0x28)
FCLASS32	010000 (0x10)	00101001 (0x29)
FRINT	010000 (0x10)	00110000 (0x30)
FRINTZ	010000 (0x10)	00110001 (0x31)
FRINTM	010000 (0x10)	00110010 (0x32)
FRINTP	010000 (0x10)	00110011 (0x33)
FRINTA	010000 (0x10)	00110100 (0x34)
FCVTDQ	010000 (0x10)	00111000 (0x38)
FCVTQD	010000 (0x10)	00111001 (0x39)
FCVTDS	010000 (0x10)	00111010 (0x3A)
FCVTSD	010000 (0x10)	00111011 (0x3B)
FCVTSL	010000 (0x10)	00111100 (0x3C)
FCVTLS	010000 (0x10)	00111101 (0x3D)
FCVTDQU	010000 (0x10)	00111110 (0x3E)
FCVTQUD	010000 (0x10)	00111111 (0x3F)
FSFLAGS	010000 (0x10)	01000000 (0x40)
FADD16	010000 (0x10)	01010000 (0x50)
FSUB16	010000 (0x10)	01010001 (0x51)
FMUL16	010000 (0x10)	01010010 (0x52)

Mnemonic	Opcode	Sub-opcode
FDIV16	010000 (0x10)	01010011 (0x53)
FSQRT16	010000 (0x10)	01010100 (0x54)
FCMP16	010000 (0x10)	01010101 (0x55)

Mnemonic	Opcode	Sub-opcode
FMADD16	010000 (0x10)	01011000 (0x58)
FMSUB16	010000 (0x10)	01011001 (0x59)
FNMADD16	010000 (0x10)	01011010 (0x5A)
FNMSUB16	010000 (0x10)	01011011 (0x5B)
FCLASS16	010000 (0x10)	01011100 (0x5C)
FCVTSH	010000 (0x10)	01100000 (0x60)
FCVTHS	010000 (0x10)	01100001 (0x61)
FCVTDH	010000 (0x10)	01100010 (0x62)
FCVTHD	010000 (0x10)	01100011 (0x63)
FCVTHL	010000 (0x10)	01100100 (0x64)
FCVTLH	010000 (0x10)	01100101 (0x65)

Atomics (ATOM)

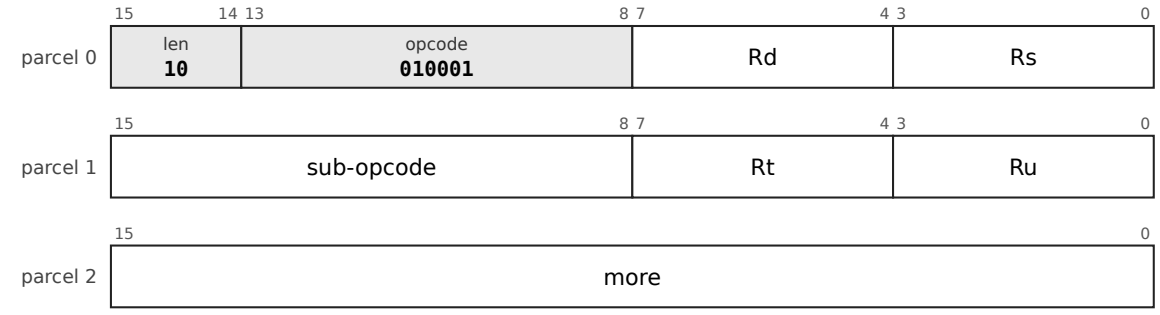
r20-atom-1: Atomics (ATOM), two parcels (`len = 01`)



From `len = 01`. The placement of the operands is not specified.

Mnemonic	Opcode	Sub-opcode
CAS	010001 (0x11)	00000000 (0x00)
CASDB	010001 (0x11)	00000001 (0x01)
SWAP	010001 (0x11)	00000010 (0x02)
SWAPDB	010001 (0x11)	00000011 (0x03)
XADD	010001 (0x11)	00000100 (0x04)
XADDDB	010001 (0x11)	00000101 (0x05)
XAND	010001 (0x11)	00000110 (0x06)
XANDDB	010001 (0x11)	00000111 (0x07)
XOR_	010001 (0x11)	00001000 (0x08)
XOR_DB	010001 (0x11)	00001001 (0x09)
XXOR	010001 (0x11)	00001010 (0x0A)
XXORDB	010001 (0x11)	00001011 (0x0B)
XMAX	010001 (0x11)	00001100 (0x0C)
XMAXDB	010001 (0x11)	00001101 (0x0D)
XMIN	010001 (0x11)	00001110 (0x0E)
XMINDB	010001 (0x11)	00001111 (0x0F)
XMAXU	010001 (0x11)	00010000 (0x10)
XMAXUDB	010001 (0x11)	00010001 (0x11)
XMINU	010001 (0x11)	00010010 (0x12)
XMINUDB	010001 (0x11)	00010011 (0x13)
LL	010001 (0x11)	00100000 (0x20)
SC	010001 (0x11)	00100001 (0x21)

r20-atom-2: Atomics (ATOM), three parcels (`len = 10`)

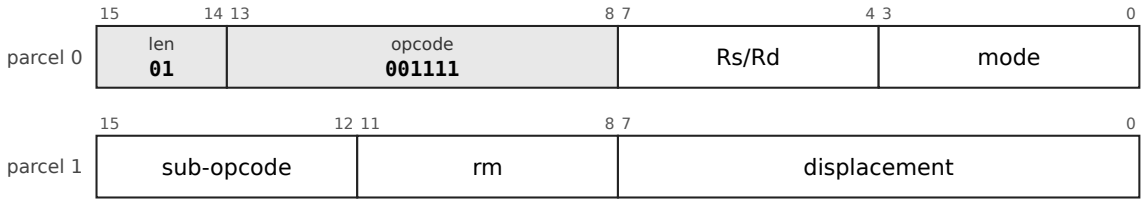


From `len = 01`. The placement of the operands is not specified.

Mnemonic	Opcode	Sub-opcode
CAS	010001 (0x11)	00000000 (0x00)
CASDB	010001 (0x11)	00000001 (0x01)
SWAP	010001 (0x11)	00000010 (0x02)
SWAPDB	010001 (0x11)	00000011 (0x03)
XADD	010001 (0x11)	00000100 (0x04)
XADDDB	010001 (0x11)	00000101 (0x05)
XAND	010001 (0x11)	00000110 (0x06)
XANDDB	010001 (0x11)	00000111 (0x07)
XOR_	010001 (0x11)	00001000 (0x08)
XOR_DB	010001 (0x11)	00001001 (0x09)
XXOR	010001 (0x11)	00001010 (0x0A)
XXORDB	010001 (0x11)	00001011 (0x0B)
XMAX	010001 (0x11)	00001100 (0x0C)
XMAXDB	010001 (0x11)	00001101 (0x0D)
XMIN	010001 (0x11)	00001110 (0x0E)
XMINDB	010001 (0x11)	00001111 (0x0F)
XMAXU	010001 (0x11)	00010000 (0x10)
XMAXUDB	010001 (0x11)	00010001 (0x11)
XMINU	010001 (0x11)	00010010 (0x12)
XMINUDB	010001 (0x11)	00010011 (0x13)
LL	010001 (0x11)	00100000 (0x20)
SC	010001 (0x11)	00100001 (0x21)

Memory escape (MESC)

r20-mesc-1: Memory escape (MESC), two parcels (`len = 01`)

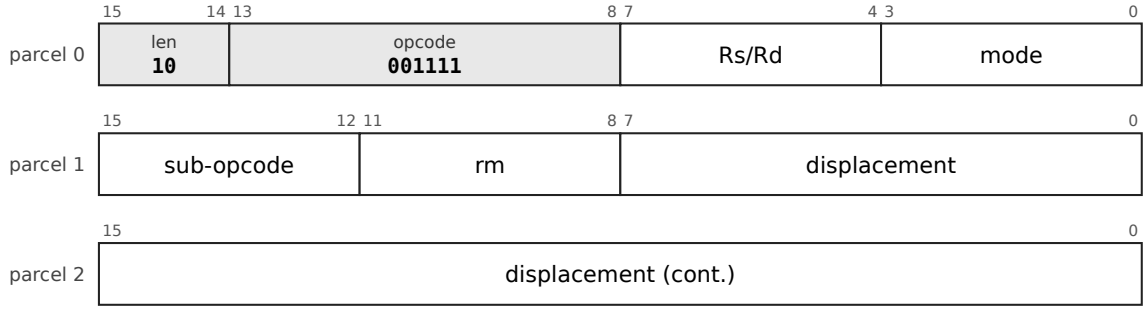


Addressing modes (`low`) that use this layout: `base` 0000, `index` 0001, `pcrel` 0010, `dp` 0011, `imm` 0100, `abs` 0101, `postinc` 0110, `predec` 0111.

`low` of parcel 0 is an addressing mode; parcel 1 holds the sub-opcode, `rm` and the first 8 bits of the displacement.

Mnemonic	Opcode	Sub-opcode
MOVUB	001111 (0x0F)	0000 (0x0)
MOVUW	001111 (0x0F)	0010 (0x2)
MOVUL	001111 (0x0F)	0100 (0x4)
MOVUQ	001111 (0x0F)	0110 (0x6)
ADDM	001111 (0x0F)	1000 (0x8)
SUBM	001111 (0x0F)	1001 (0x9)
BICM	001111 (0x0F)	1010 (0xA)
BISM	001111 (0x0F)	1011 (0xB)

r20-mesc-2: Memory escape (MESC), three parcels (`len = 10`)



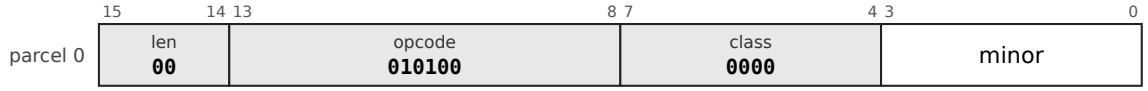
Addressing modes (`low`) that use this layout: `base` 0000, `index` 0001, `pcrel` 0010, `dp` 0011, `imm` 0100, `abs` 0101, `postinc` 0110, `predec` 0111.

`low` of parcel 0 is an addressing mode; parcel 1 holds the sub-opcode, `rm` and the first 8 bits of the displacement.

Mnemonic	Opcode	Sub-opcode
MOVUB	001111 (0x0F)	0000 (0x0)
MOVUW	001111 (0x0F)	0010 (0x2)
MOVUL	001111 (0x0F)	0100 (0x4)
MOVUQ	001111 (0x0F)	0110 (0x6)
ADDM	001111 (0x0F)	1000 (0x8)
SUBM	001111 (0x0F)	1001 (0x9)
BICM	001111 (0x0F)	1010 (0xA)
BISM	001111 (0x0F)	1011 (0xB)

System, no operand

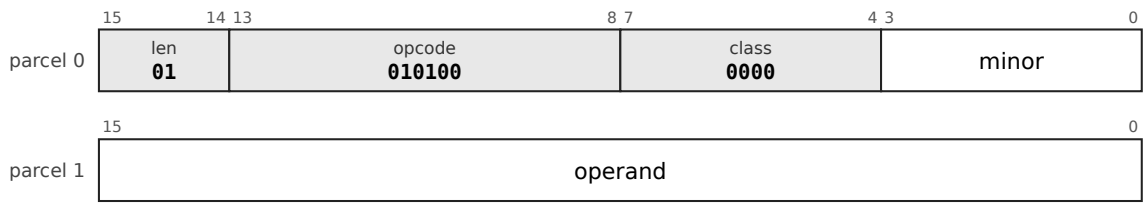
r20-sysn-0: System, no operand, one parcel (`len = 00`)



The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
NOP	010100 (0x14)	0000 (0x0)	0000 (0x0)
RTI	010100 (0x14)	0000 (0x0)	0001 (0x1)
RTT	010100 (0x14)	0000 (0x0)	0010 (0x2)
RET	010100 (0x14)	0000 (0x0)	0011 (0x3)
HALT	010100 (0x14)	0000 (0x0)	0100 (0x4)
BPT	010100 (0x14)	0000 (0x0)	0101 (0x5)
FENCE	010100 (0x14)	0000 (0x0)	0110 (0x6)
WFE	010100 (0x14)	0000 (0x0)	0111 (0x7)
SEV	010100 (0x14)	0000 (0x0)	1000 (0x8)
LPAD	010100 (0x14)	0000 (0x0)	1001 (0x9)
TLBIA	010100 (0x14)	0000 (0x0)	1010 (0xA)
TLBSYNC	010100 (0x14)	0000 (0x0)	1011 (0xB)
ICINVA	010100 (0x14)	0000 (0x0)	1100 (0xC)

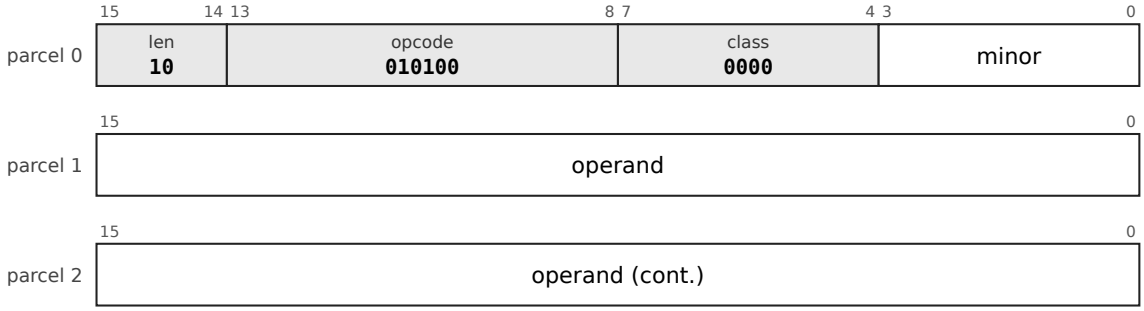
r20-sysn-1: System, no operand, two parcels (`len = 01`)



The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
NOP	010100 (0x14)	0000 (0x0)	0000 (0x0)
RTI	010100 (0x14)	0000 (0x0)	0001 (0x1)
RTT	010100 (0x14)	0000 (0x0)	0010 (0x2)
RET	010100 (0x14)	0000 (0x0)	0011 (0x3)
HALT	010100 (0x14)	0000 (0x0)	0100 (0x4)
BPT	010100 (0x14)	0000 (0x0)	0101 (0x5)
FENCE	010100 (0x14)	0000 (0x0)	0110 (0x6)
WFE	010100 (0x14)	0000 (0x0)	0111 (0x7)
SEV	010100 (0x14)	0000 (0x0)	1000 (0x8)
LPAD	010100 (0x14)	0000 (0x0)	1001 (0x9)
TLBIA	010100 (0x14)	0000 (0x0)	1010 (0xA)
TLBSYNC	010100 (0x14)	0000 (0x0)	1011 (0xB)
ICINVA	010100 (0x14)	0000 (0x0)	1100 (0xC)

r20-sysn-2: System, no operand, three parcels (`len = 10`)



The `reg` field is the class (0); class 0 takes a minor opcode from `low`.

Mnemonic	Opcode	reg	low
NOP	010100 (0x14)	0000 (0x0)	0000 (0x0)
RTI	010100 (0x14)	0000 (0x0)	0001 (0x1)
RTT	010100 (0x14)	0000 (0x0)	0010 (0x2)
RET	010100 (0x14)	0000 (0x0)	0011 (0x3)
HALT	010100 (0x14)	0000 (0x0)	0100 (0x4)
BPT	010100 (0x14)	0000 (0x0)	0101 (0x5)
FENCE	010100 (0x14)	0000 (0x0)	0110 (0x6)
WFE	010100 (0x14)	0000 (0x0)	0111 (0x7)
SEV	010100 (0x14)	0000 (0x0)	1000 (0x8)
LPAD	010100 (0x14)	0000 (0x0)	1001 (0x9)
TLBIA	010100 (0x14)	0000 (0x0)	1010 (0xA)
TLBSYNC	010100 (0x14)	0000 (0x0)	1011 (0xB)
ICINVA	010100 (0x14)	0000 (0x0)	1100 (0xC)

System, SYSX

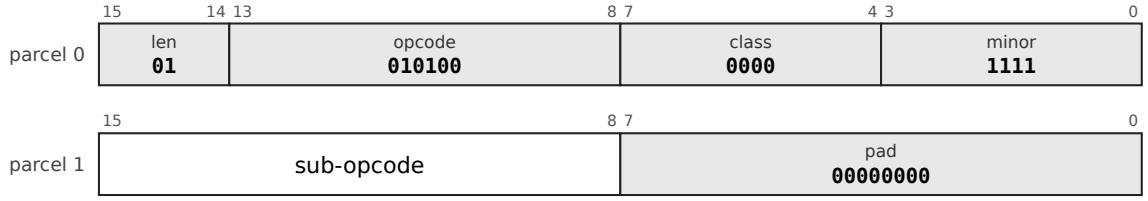
r20-sysx-0: System, SYSX, one parcel (`len = 00`)



Class 0, minor 15: the sub-opcode is in parcel 1.

Mnemonic	Opcode	reg	low	Sub-opcode
PUSHX	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000000 (0x00)
POPX	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000001 (0x01)
ISYNC	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000010 (0x02)
HINT	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000011 (0x03)

r20-sysx-1: System, SYSX, two parcels (`len = 01`)

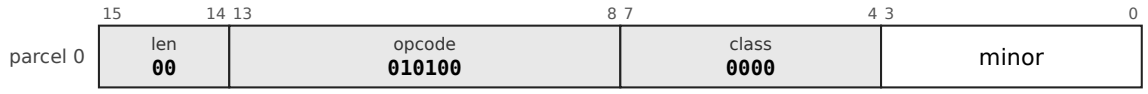


Class 0, minor 15: the sub-opcode is in parcel 1.

Mnemonic	Opcode	reg	low	Sub-opcode
PUSHX	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000000 (0x00)
POPX	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000001 (0x01)
ISYNC	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000010 (0x02)
HINT	010100 (0x14)	0000 (0x0)	1111 (0xF)	00000011 (0x03)

System registers (MFSR, MTSR)

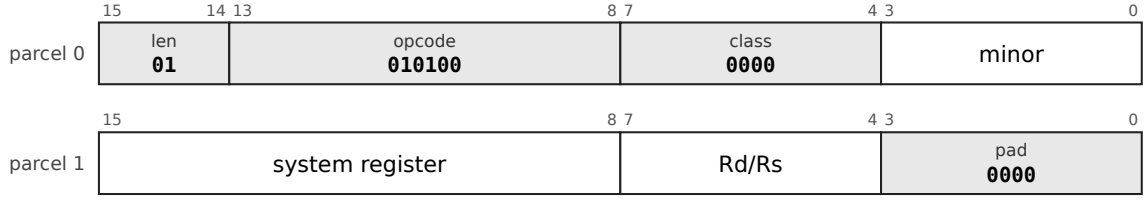
r20-sysreg-0: System registers (MFSR, MTSR), one parcel (`len = 00`)



Class 0: the system register number is in parcel 1.

Mnemonic	Opcode	reg	low
MFSR	010100 (0x14)	0000 (0x0)	1101 (0xD)
MTSR	010100 (0x14)	0000 (0x0)	1110 (0xE)

r20-sysreg-1: System registers (MFSR, MTSR), two parcels (`len = 01`)

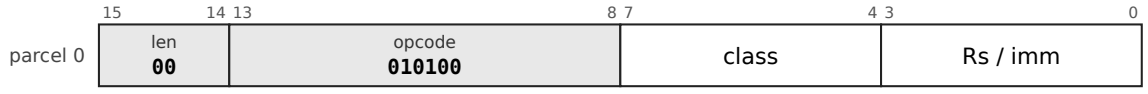


Class 0: the system register number is in parcel 1.

Mnemonic	Opcode	reg	low
MFSR	010100 (0x14)	0000 (0x0)	1101 (0xD)
MTSR	010100 (0x14)	0000 (0x0)	1110 (0xE)

System

r20-sys-0: System, one parcel (`len = 00`)

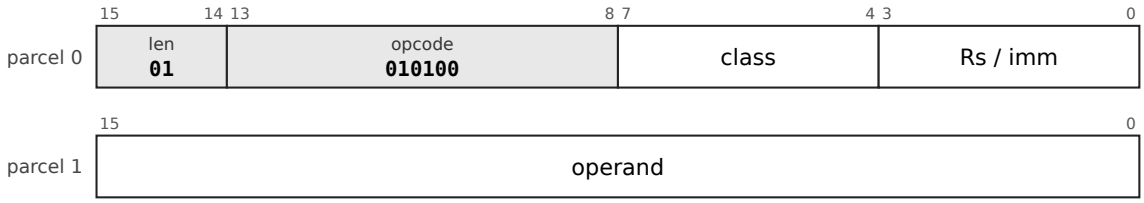


The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	reg
MTPS	010100 (0x14)	0001 (0x1)
MFPS	010100 (0x14)	0010 (0x2)
SPL	010100 (0x14)	0011 (0x3)
TRAP	010100 (0x14)	0100 (0x4)
TLBI	010100 (0x14)	0101 (0x5)
MTASID	010100 (0x14)	0110 (0x6)

Mnemonic	Opcode	reg
MFASID	010100 (0x14)	0111 (0x7)
MTDP	010100 (0x14)	1000 (0x8)
MFDP	010100 (0x14)	1001 (0x9)
MTDMW	010100 (0x14)	1010 (0xA)
MFCPUID	010100 (0x14)	1011 (0xB)
ICINV	010100 (0x14)	1100 (0xC)
PROBER	010100 (0x14)	1101 (0xD)
PROBEW	010100 (0x14)	1110 (0xE)
TRAPL	010100 (0x14)	1111 (0xF)

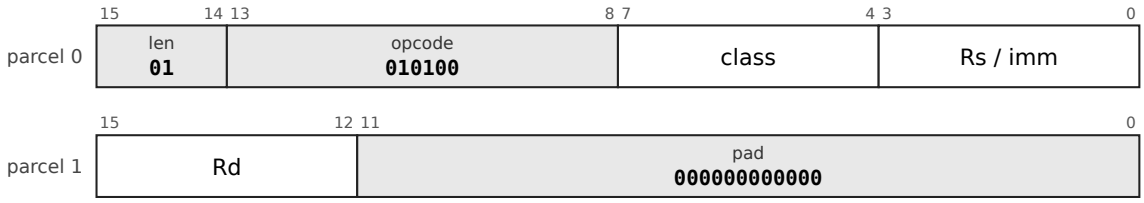
r20-sys-1: System, two parcels (`len = 01`)



The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	reg
MTPS	010100 (0x14)	0001 (0x1)
MFPS	010100 (0x14)	0010 (0x2)
SPL	010100 (0x14)	0011 (0x3)
TRAP	010100 (0x14)	0100 (0x4)
TLBI	010100 (0x14)	0101 (0x5)
MTASID	010100 (0x14)	0110 (0x6)
MFASID	010100 (0x14)	0111 (0x7)
MTDP	010100 (0x14)	1000 (0x8)
MFDP	010100 (0x14)	1001 (0x9)
MTDMW	010100 (0x14)	1010 (0xA)
MFCPUID	010100 (0x14)	1011 (0xB)
ICINV	010100 (0x14)	1100 (0xC)

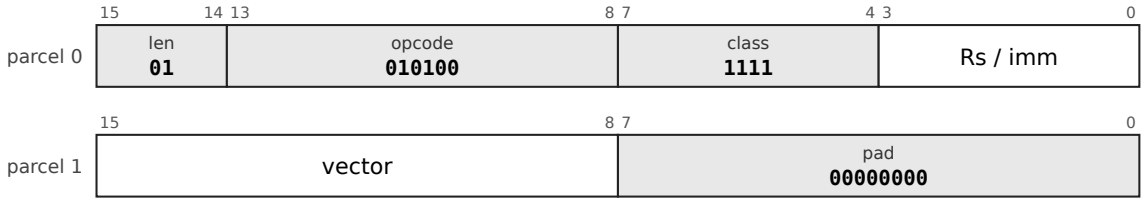
r20-sys-1-2: System, two parcels (`len = 01`)



The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	reg
PROBER	010100 (0x14)	1101 (0xD)
PROBEW	010100 (0x14)	1110 (0xE)

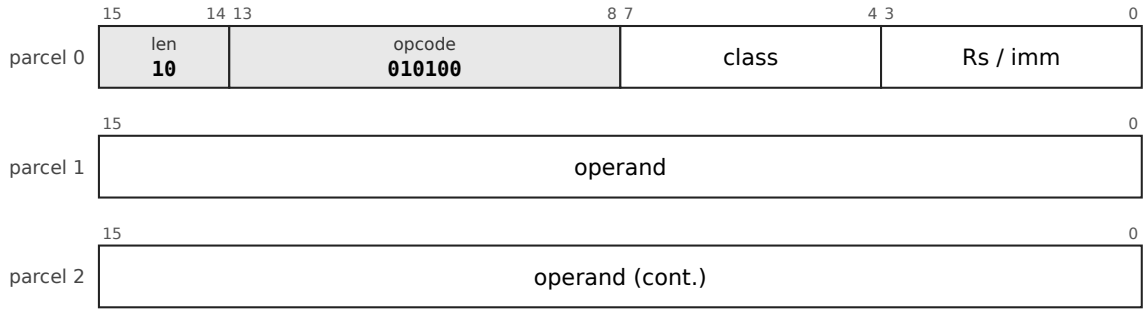
r20-sys-1-3: System, two parcels (`len = 01`)



The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	reg
TRAPL	010100 (0x14)	1111 (0xF)

r20-sys-2: System, three parcels (`len = 10`)

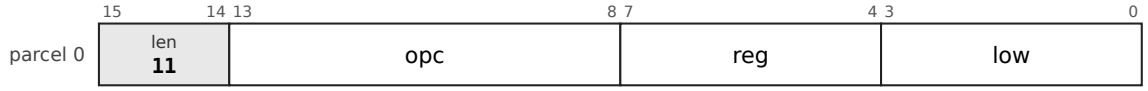


The `reg` field is the class; the other classes take a register or a 4-bit immediate from `low`.

Mnemonic	Opcode	reg
MTPS	010100 (0x14)	0001 (0x1)
MFPS	010100 (0x14)	0010 (0x2)
SPL	010100 (0x14)	0011 (0x3)
TRAP	010100 (0x14)	0100 (0x4)
TLBI	010100 (0x14)	0101 (0x5)
MTASID	010100 (0x14)	0110 (0x6)
MFASID	010100 (0x14)	0111 (0x7)
MTDP	010100 (0x14)	1000 (0x8)
MFDP	010100 (0x14)	1001 (0x9)
MTDMW	010100 (0x14)	1010 (0xA)
MFCPUID	010100 (0x14)	1011 (0xB)
ICINV	010100 (0x14)	1100 (0xC)
PROBER	010100 (0x14)	1101 (0xD)
PROBEW	010100 (0x14)	1110 (0xE)
TRAPL	010100 (0x14)	1111 (0xF)

Page 2

r20-page2: Page 2: a one-parcel instruction with its own opcode map



The index of the mnemonics

For every mnemonic, the charts in which it is a sub-opcode or the whole leaf.

- ABS: r20-aesc-1, r20-aesc-2
- ADC: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- ADD: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- ADDM: r20-mesc-1, r20-mesc-2
- AND: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- ASL: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- ASR: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- BCC: r20-br-0, r20-br-1, r20-br-2
- BCS: r20-br-0, r20-br-1, r20-br-2
- BEQ: r20-br-0, r20-br-1, r20-br-2
- BGE: r20-br-0, r20-br-1, r20-br-2
- BGT: r20-br-0, r20-br-1, r20-br-2

- BHI: r20-br-0, r20-br-1, r20-br-2
- BIC: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- BICM: r20-mesc-1, r20-mesc-2
- BIS: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- BISM: r20-mesc-1, r20-mesc-2
- BIT: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- BITREV: r20-besc-1, r20-besc-2
- BLE: r20-br-0, r20-br-1, r20-br-2
- BLOS: r20-br-0, r20-br-1, r20-br-2
- BLT: r20-br-0, r20-br-1, r20-br-2
- BMI: r20-br-0, r20-br-1, r20-br-2
- BNE: r20-br-0, r20-br-1, r20-br-2
- BNTAG: r20-tag-1, r20-tag-2
- BPL: r20-br-0, r20-br-1, r20-br-2
- BPT: r20-sysn-0, r20-sysn-1, r20-sysn-2
- BR: r20-br-0, r20-br-1, r20-br-2
- BSWAP: r20-besc-1, r20-besc-2
- BSWAPL: r20-besc-1, r20-besc-2
- BSWAPW: r20-besc-1, r20-besc-2
- BTAG: r20-tag-1, r20-tag-2
- BVC: r20-br-0, r20-br-1, r20-br-2
- BVS: r20-br-0, r20-br-1, r20-br-2
- CALL: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- CALLI: r20-xfer-0, r20-xfer-1, r20-xfer-1-2, r20-xfer-1-3, r20-xfer-1-4, r20-xfer-2, r20-xfer-2-2, r20-xfer-2-3, r20-xfer-2-4
- CAS: r20-atom-1, r20-atom-2
- CASDB: r20-atom-1, r20-atom-2
- CBCC: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBCS: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBEQ: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBGE: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBGT: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBHI: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBLE: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBLOS: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBLT: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBMI: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBNE: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBPL: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBVC: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CBVS: r20-cb-1, r20-cb-1-2, r20-cb-2, r20-cb-2-2, r20-cb-2-3
- CLMULH: r20-besc-1, r20-besc-2
- CLMULL: r20-besc-1, r20-besc-2
- CLO: r20-besc-1, r20-besc-2
- CLR: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- CLZ: r20-besc-1, r20-besc-2
- CMP: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4

- `COM`: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- `CTO`: r20-besc-1, r20-besc-2
- `CTZ`: r20-besc-1, r20-besc-2
- `DEC`: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- `DIV`: r20-aesc-1, r20-aesc-2
- `DIVU`: r20-aesc-1, r20-aesc-2
- `DSHR`: r20-besc-1, r20-besc-2-3
- `EXTB`: r20-besc-1, r20-besc-2-3
- `EXTZB`: r20-besc-1, r20-besc-2-3
- `FABS`: r20-fesc-1, r20-fesc-2
- `FADD`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `FADD16`: r20-fesc-1, r20-fesc-2
- `FADD32`: r20-fesc-1, r20-fesc-2
- `FCLASS`: r20-fesc-1, r20-fesc-2
- `FCLASS16`: r20-fesc-1, r20-fesc-2
- `FCLASS32`: r20-fesc-1, r20-fesc-2
- `FCMP`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `FCMP16`: r20-fesc-1, r20-fesc-2
- `FCMP32`: r20-fesc-1, r20-fesc-2
- `FCOPYSIGN`: r20-fesc-1, r20-fesc-2
- `FCVTDH`: r20-fesc-1, r20-fesc-2
- `FCVTDQ`: r20-fesc-1, r20-fesc-2
- `FCVTDQU`: r20-fesc-1, r20-fesc-2
- `FCVTDS`: r20-fesc-1, r20-fesc-2
- `FCVTHD`: r20-fesc-1, r20-fesc-2
- `FCVTHL`: r20-fesc-1, r20-fesc-2
- `FCVTHS`: r20-fesc-1, r20-fesc-2
- `FCVTLH`: r20-fesc-1, r20-fesc-2
- `FCVTLS`: r20-fesc-1, r20-fesc-2
- `FCVTQD`: r20-fesc-1, r20-fesc-2
- `FCVTQUD`: r20-fesc-1, r20-fesc-2
- `FCVTSD`: r20-fesc-1, r20-fesc-2
- `FCVTSH`: r20-fesc-1, r20-fesc-2
- `FCVTSL`: r20-fesc-1, r20-fesc-2
- `FDIV`: r20-fesc-1, r20-fesc-2
- `FDIV16`: r20-fesc-1, r20-fesc-2
- `FDIV32`: r20-fesc-1, r20-fesc-2
- `FENCE`: r20-sysn-0, r20-sysn-1, r20-sysn-2
- `FMADD`: r20-fesc-1, r20-fesc-2
- `FMADD16`: r20-fesc-1, r20-fesc-2
- `FMADD32`: r20-fesc-1, r20-fesc-2
- `FMAX`: r20-fesc-1, r20-fesc-2
- `FMIN`: r20-fesc-1, r20-fesc-2
- `FMSUB`: r20-fesc-1, r20-fesc-2
- `FMSUB16`: r20-fesc-1, r20-fesc-2
- `FMSUB32`: r20-fesc-1, r20-fesc-2
- `FMUL`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4

- FMUL16: r20-fesc-1, r20-fesc-2
- FMUL32: r20-fesc-1, r20-fesc-2
- FNEG: r20-fesc-1, r20-fesc-2
- FNMADD: r20-fesc-1, r20-fesc-2
- FNMADD16: r20-fesc-1, r20-fesc-2
- FNMADD32: r20-fesc-1, r20-fesc-2
- FNMSUB: r20-fesc-1, r20-fesc-2
- FNMSUB16: r20-fesc-1, r20-fesc-2
- FNMSUB32: r20-fesc-1, r20-fesc-2
- FRINT: r20-fesc-1, r20-fesc-2
- FRINTA: r20-fesc-1, r20-fesc-2
- FRINTM: r20-fesc-1, r20-fesc-2
- FRINTP: r20-fesc-1, r20-fesc-2
- FRINTZ: r20-fesc-1, r20-fesc-2
- FSFLAGS: r20-fesc-1, r20-fesc-2
- FSQRT: r20-fesc-1, r20-fesc-2
- FSQRT16: r20-fesc-1, r20-fesc-2
- FSQRT32: r20-fesc-1, r20-fesc-2
- FSUB: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- FSUB16: r20-fesc-1, r20-fesc-2
- FSUB32: r20-fesc-1, r20-fesc-2
- HALT: r20-sysn-0, r20-sysn-1, r20-sysn-2
- HINT: r20-sysx-0, r20-sysx-1
- ICINV: r20-sys-0, r20-sys-1, r20-sys-2
- ICINVA: r20-sysn-0, r20-sysn-1, r20-sysn-2
- INC: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- INSB: r20-besc-1, r20-besc-2-3
- ISYNC: r20-sysx-0, r20-sysx-1
- JMP: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- JMPI: r20-xfer-0, r20-xfer-1, r20-xfer-1-2, r20-xfer-1-3, r20-xfer-1-4, r20-xfer-2, r20-xfer-2-2, r20-xfer-2-3, r20-xfer-2-4
- JTAG: r20-xfer-0, r20-xfer-1, r20-xfer-1-2, r20-xfer-1-3, r20-xfer-1-4, r20-xfer-2, r20-xfer-2-2, r20-xfer-2-3, r20-xfer-2-4
- LEA: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- LL: r20-atom-1, r20-atom-2
- LPAD: r20-sysn-0, r20-sysn-1, r20-sysn-2
- MAX: r20-aesc-1, r20-aesc-2
- MAXU: r20-aesc-1, r20-aesc-2
- MCMP: r20-mblk-1, r20-mblk-2
- MCPY: r20-mblk-1, r20-mblk-2
- MFASID: r20-sys-0, r20-sys-1, r20-sys-2
- MFCPUID: r20-sys-0, r20-sys-1, r20-sys-2
- MFDP: r20-sys-0, r20-sys-1, r20-sys-2
- MFPS: r20-sys-0, r20-sys-1, r20-sys-2
- MFSR: r20-sysreg-0, r20-sysreg-1
- MIN: r20-aesc-1, r20-aesc-2
- MINU: r20-aesc-1, r20-aesc-2
- MMOV: r20-mblk-1, r20-mblk-2
- MOD: r20-aesc-1, r20-aesc-2
- MODU: r20-aesc-1, r20-aesc-2

- `MOVB`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MOVL`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MOVQ`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MOVUB`: r20-mesc-1, r20-mesc-2
- `MOVUL`: r20-mesc-1, r20-mesc-2
- `MOVUQ`: r20-mesc-1, r20-mesc-2
- `MOVUW`: r20-mesc-1, r20-mesc-2
- `MOVW`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MOVZB`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MOVZL`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MOVZW`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MSCN`: r20-mblk-1, r20-mblk-2
- `MSCNN`: r20-mblk-1, r20-mblk-2
- `MSET`: r20-mblk-1, r20-mblk-2
- `MTASID`: r20-sys-0, r20-sys-1, r20-sys-2
- `MTDMW`: r20-sys-0, r20-sys-1, r20-sys-2
- `MTDP`: r20-sys-0, r20-sys-1, r20-sys-2
- `MTPS`: r20-sys-0, r20-sys-1, r20-sys-2
- `MTSR`: r20-sysreg-0, r20-sysreg-1
- `MUL`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MULH`: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- `MULHU`: r20-aesc-1, r20-aesc-2
- `MVAL`: r20-mblk-1, r20-mblk-2
- `NEG`: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- `NOP`: r20-sysn-0, r20-sysn-1, r20-sysn-2
- `POP`: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- `POPCNT`: r20-besc-1, r20-besc-2
- `POPM`: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- `POPX`: r20-sysx-0, r20-sysx-1
- `PROBER`: r20-sys-0, r20-sys-1-2, r20-sys-2
- `PROBEW`: r20-sys-0, r20-sys-1-2, r20-sys-2
- `PUSH`: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- `PUSHM`: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- `PUSHX`: r20-sysx-0, r20-sysx-1
- `RET`: r20-sysn-0, r20-sysn-1, r20-sysn-2
- `ROR`: r20-besc-1, r20-besc-2
- `RORI`: r20-besc-1-2, r20-besc-2-2
- `RORI32`: r20-besc-1-2, r20-besc-2-2
- `RTI`: r20-sysn-0, r20-sysn-1, r20-sysn-2
- `RTT`: r20-sysn-0, r20-sysn-1, r20-sysn-2

- SAR: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- SBC: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- SC: r20-atom-1, r20-atom-2
- SET: r20-aesc-1, r20-aesc-2
- SEV: r20-sysn-0, r20-sysn-1, r20-sysn-2
- SHL: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- SHR: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- SOB: r20-sob-1, r20-sob-2
- SPL: r20-sys-0, r20-sys-1, r20-sys-2
- SUB: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- SUBM: r20-mesc-1, r20-mesc-2
- SWAP: r20-atom-1, r20-atom-2
- SWAPDB: r20-atom-1, r20-atom-2
- TLBI: r20-sys-0, r20-sys-1, r20-sys-2
- TLBIA: r20-sysn-0, r20-sysn-1, r20-sysn-2
- TLBSYNC: r20-sysn-0, r20-sysn-1, r20-sysn-2
- TRAP: r20-sys-0, r20-sys-1, r20-sys-2
- TRAPL: r20-sys-0, r20-sys-1-3, r20-sys-2
- TST: r20-sop-0, r20-sop-1, r20-sop-1-2, r20-sop-1-3, r20-sop-1-4, r20-sop-2, r20-sop-2-2, r20-sop-2-3, r20-sop-2-4
- UTF8DEC: r20-besc-1, r20-besc-2
- UTF8ENC: r20-besc-1, r20-besc-2
- UTF8LEN: r20-besc-1, r20-besc-2
- WFE: r20-sysn-0, r20-sysn-1, r20-sysn-2
- XADD: r20-atom-1, r20-atom-2
- XADDDB: r20-atom-1, r20-atom-2
- XAND: r20-atom-1, r20-atom-2
- XANDDB: r20-atom-1, r20-atom-2
- XMAX: r20-atom-1, r20-atom-2
- XMAXDB: r20-atom-1, r20-atom-2
- XMAXU: r20-atom-1, r20-atom-2
- XMAXUDB: r20-atom-1, r20-atom-2
- XMIN: r20-atom-1, r20-atom-2
- XMINDB: r20-atom-1, r20-atom-2
- XMINU: r20-atom-1, r20-atom-2
- XMINUDB: r20-atom-1, r20-atom-2
- XNORPOP: r20-besc-1, r20-besc-2
- XOR: r20-mem-0, r20-mem-1, r20-mem-1-2, r20-mem-1-3, r20-mem-1-4, r20-mem-2, r20-mem-2-2, r20-mem-2-3, r20-mem-2-4
- XOR_: r20-atom-1, r20-atom-2
- XOR_DB: r20-atom-1, r20-atom-2
- XXOR: r20-atom-1, r20-atom-2
- XXORDB: r20-atom-1, r20-atom-2

Generated by `tools/mkmanual.py`; do not edit.

Part III — Appendices

Appendix A — Possible improvements not decided on

These are ideas that would make the machine better in one respect or another and that are not decisions. They come from the reports on kernel pain points, hardware vulnerabilities and OpenBSD's mitigations, and from the open items of the instruction-set text. Each is stated as a possibility, with its benefit, its cost and what other machines do, so that it can be taken up, rejected or promoted to a decision in appendix B. Nothing here is built or specified unless it says so. The same ordering principle as the rest of the manual applies: code size, then easy recoverability, then speed and efficiency through fewer gates and tighter hot loops.

A.1 Reserve a page bit that enforces landing pads

A.1.1 The `LPAD` instruction marks a legal target of an indirect jump or call; a page bit could make the hardware fault when an indirect transfer lands anywhere else, with enforcement off in ABI version 1 and on from version 2.

A.1.2 A page bit that gates landing pads is approved, with the fault attributed to the branch, to be specified (errata 84, item 5). *(D)*

Commentary. **Precedent.** `LPAD` already exists, and the ABI leaves landing pads off in version 1, with no switch to enforce them. **Trade-offs.** The bit costs one comparator and two bytes per address-taken function, and deciding later gets more expensive because every binary built in the meantime lacks the pads. **Benefits.** It blocks jump-oriented attacks at the transfer, with no recovery code beyond the fault handler that already exists. **Modern comparables.** Intel's indirect-branch tracking and ARM's branch target identification, both of which OpenBSD enforces by default where the hardware has them. **Impact.** *Kernel:* A landing-pad violation is a fault the handler already handles; the kernel sets the page bit at exec time. *Compiler:* The compiler emits `LPAD` at every address-taken function, two bytes each. *Hardware:* One page bit and one comparator on an indirect-transfer target.

A.2 A page bit that says a system call may be made from here

A.2.1 A page-table bit could let the hardware refuse a system call from any page not so marked, so that the kernel's check of where a call came from moves into the instruction.

A.2.2 A page bit for system-call entry (the `SYS` page bit) is approved, to be specified (errata 84, item 8). *(D)*

Commentary. **Precedent.** OpenBSD's `msyscall` and `pinsyscalls` restrict where system calls may be issued from. **Trade-offs.** One bit and one gate; the kernel would no longer need to check the origin on every call. **Benefits.** About four instructions removed from every system call, and region pinning with no hot-path check. **Modern comparables.** No architecture surveyed has this bit; it is done by the kernel in software. **Impact.** *Kernel:* The kernel no longer checks where a system call came from on every call, which removes about four instructions from each. *Compiler:* None. *Hardware:* One page bit and one gate at the `TRAP` decode.

A.3 A check of the stack pointer at system-call entry

A.3.1 The hardware could check that the stack pointer lies in a region registered as a stack when a system call or a fault is taken.

A.3.2 A stack-limit register is approved, to be specified (errata 84, item 4). *(D)*

Commentary. **Precedent.** OpenBSD checks the stack pointer against a `MAP_STACK` region on system calls and faults. **Trade-offs.** It adds a comparison to entry, which is the path the design most wants short, and gives no benefit over the kernel making the same check. **Verdict.** Not recommended; recorded because the question is natural. **Impact.** *Kernel:* None over the check the kernel already makes. *Compiler:* None. *Hardware:* A comparator and a region register on the entry path, which is the path the design most wants short.

A.4 Protecting the return address

A.4.1 The return address lives on the data stack, because `CALL` pushes it and there is no link register; there are four ways to protect it: software canaries and OpenBSD's return-address protection in the compiler, a hardware return stack, a shadow stack, and nothing.

A.4.2 The mirror shadow stack is approved with its recommendations (errata 85, study item 6); return-address authentication is deferred (errata 84, item 7). *(D)*

Commentary. **Precedent.** x86 has the same exposure and uses stack canaries, `retguard` in OpenBSD's clang, and the shadow stacks of Intel's control-flow enforcement; ARM has pointer authentication. **Trade-offs.** A shadow stack is the strongest and brings the most recovery code (a second stack to overflow, switch and unwind), so the analysis does not recommend it by default; software detection costs a few instructions per function and no hardware. **Benefits.** Detection with little or no recovery code. **Verdict.** Start with the compiler; consider a hardware mechanism only if code size allows. **Impact.** *Kernel:* A shadow stack is a second stack for the kernel to manage and unwind; software detection needs only a fault handler. *Compiler:* The compiler adds a canary or a return-address check per function. *Hardware:* Software costs nothing; a hardware return stack or a shadow stack costs a second stack structure and its control.

A.5 Tag or key bits in pointers

A.5.1 The 25 spare bits of a 64-bit pointer (a 39-bit virtual address, the three-level form, uses the rest; 48 and 57 bits leave fewer) could carry a tag or a key, as a memory-tagging or pointer-authentication scheme does.

A.5.2 Full 128-bit CHERI is rejected, and a valid-pointer tag per register and per 8-byte word (stage 1) is kept, with bounds deferred (errata 86, study item 3); granule colouring and reserved colours for uninitialised and freed memory are approved, to be specified, and spend pointer bits that the canonical-address rule refuses to software, so that rule must be restated (errata 84, open). *(D)*

Commentary. **Precedent.** ARM's top-byte-ignore, pointer authentication and memory tagging. **Trade-offs.** The cost is canonical-address rules: pointers must be masked before use, and the hardware must define what a tagged pointer does. **Verdict.** No threat that the operating system asked for requires it; it costs gates and state, so it is not recommended. **Impact.** *Kernel:* The kernel would have to define what a tagged pointer means across the user and kernel boundary. *Compiler:* Every pointer use needs masking, and pointer arithmetic must preserve tags. *Hardware:* Gates and state in the address path.

A.6 A physical address wider than 40 bits: decided, with the entry format open

A.6.1 The physical address is 40 to 64 bits (errata 102, *(D)*), which lets petabytes of memory and device windows be mapped; the present page-table entry holds the physical page number in bits 12 to 39 and keeps bits 40 to 63 zero, so the entry format has to grow its page number into them or change. The format is open. *(S)*

Commentary. **Trade-offs.** The author's reason is that mapping petabytes needs those bits (errata 97). The cost is a wider page number in the entry, in every translation-buffer entry and in every bus address; the benefit is room for devices and memory in one address space without a decoding bit. **Modern comparables.** x86-64 and AArch64 define physical addresses of up to 52 bits and RISC-V up to 56 (stated from memory, not confirmed in the sources read). **Impact.** *Kernel:* a wider entry and a physical-address width to discover. *Compiler:* none. *Hardware:* more bits in the walker, the translation buffer and the bus.

A.7 A second page-table base register

A.7.1 A second page-table base for the kernel half of the address space would let a kernel and a user program run with different tables without flushing, if a speculative core ever needs the isolation.

Commentary. **Precedent.** The cost of separating kernel and user page tables on x86 after Meltdown was about 5% typical and about 30% in the worst case in the developers' own estimates, and the tagged translation buffers (PCID) made it tolerable. **Trade-offs.** The single base is correct and free on a core that does not speculate; the second base register costs a register and a rule, and the question is whether to pay it before the need exists. **Benefits.** The isolation is in the hardware from the start. **Modern comparables.** AArch64 has two translation-table base registers, one for each half of the address space. **Impact.** *Kernel:* The kernel switches one base register at the privilege change instead of managing a shared table. *Compiler:* None. *Hardware:* A second base register and a rule choosing which half uses which.

A.8 Two-byte forms of the user-memory guard instructions

A.8.1 `STUA` and `CLUA` could have one-parcel page-2 forms, saving four bytes per user-copy routine.

Commentary. **Trade-offs.** It uses two of the 19 reserved page-2 slots, which is not worth it until a kernel exists to count the copy routines. **Benefits.** Smaller and faster user copies. **Verdict.** Hold the slots and decide when the kernel exists. **Impact.** *Kernel:* Each user copy routine is four bytes smaller. *Compiler:* None. *Hardware:* Two reserved page-2 slots used.

A.9 Thirty-two-bit forms of add, subtract and shift

A.9.1 If the extension instructions the compiler emits after narrow arithmetic are more than a few percent of the code, 32-bit forms of the common operations on page 2 would remove them.

Commentary. **Precedent.** AArch64's `W` forms and RISC-V's `ADDW` family. **Trade-offs.** A second set of opcodes in the page-2 map against extension instructions in the compiled code; the overhead has never been measured over the whole corpus. **Verdict.** Measure first; it is one script over the compiled corpus. **Impact.** *Kernel:* None. *Compiler:* The compiler stops emitting an extension after common 32-bit operations. *Hardware:* A second set of opcodes in the page-2 map and a little ALU decode.

A.10 Packed-lane instructions in the integer registers

A.10.1 A small set of lane operations (add, subtract, saturating add, compare, minimum, maximum, average, sum of absolute differences, shuffle on 8, 16 and 32-bit lanes) in the existing registers could speed up string, codec and image loops.

Commentary. **Precedent.** Alpha's motion-video instructions and PA-RISC's multimedia instructions did exactly this, in integer registers. **Trade-offs.** It adds no state and little hardware, but it is 64 bits wide so the gain per instruction is modest, and compilers use it poorly. **Verdict.** Collect the evidence first: the compiler corpus was built at a size setting that hides how much vectorisable code there is (appendix B, S1). **Impact.** *Kernel:* String and checksum routines get faster. *Compiler:* Vectorisation into integer lanes is weak, so the library routines would be hand-written. *Hardware:* A lane-aware ALU: little extra hardware and no new state.

A.11 Streaming and non-temporal block operations

A.11.1 A large `MCPY` evicts the whole data cache for data never read again; a non-temporal variant or a streaming memory type would avoid it.

Commentary. **Precedent.** x86 has non-temporal stores; ARM has non-temporal load and store pairs. **Trade-offs.** It shows immediately on an 8 KB cache; the fix is an opcode or a memory type, the non-fix is to leave the cache alone. **Verdict.** Open; decide when the memory system is built. **Impact.** *Kernel:* Large copies would not evict the kernel's working set. *Compiler:* `memcpy` would choose the streaming form above a size. *Hardware:* A non-temporal write path in the cache.

A.12 Smaller possibilities

- **`MULHSU`** (the high half of a signed-by-unsigned product, for a signed top limb meeting an unsigned one) is reserved in the arithmetic escape; it is one correction step in software and has one caller, a bignum library's innermost loop.
- **Hardware page zeroing**, and **guard-page states in the page-table entry that do not consume translation-buffer entries**, would make the allocator hardening of OpenBSD's `malloc` cheaper.
- **A mixed leaf size for the kernel's text** is needed if the kernel is to be write-xor-execute with one-gigabyte leaf pages.
- **Hardware breakpoints and debug registers** are not designed; a debugger has only the breakpoint instruction and the trace bit.
- **A sealed (immutable) page bit and a hardware mask for `pledge`** have no benefit over the kernel doing the checks, and are rejected.

Commentary. **Trade-offs.** None of these is ranked above the decisions in appendix B; the first four sections of this appendix (landing pads, the system-call page bit, the stack check and return-address protection) are the ones OpenBSD's developers have shown the most interest in.

Appendix B — Decisions pending

These are the open decisions, each with its question, the proposed answer, and the precedent, trade-offs and benefits that surround it. Each has five options with their costs in the working document `../decisions-pending.md` (the identifiers P, R, S, N and B below are that document's), and the boot choices are worked through in `../boot.md`. Decided items are not repeated here. Each entry is judged by the three principles in order: code size, easy recoverability (less recovery code), and speed and efficiency through fewer gates and tighter hot loops. Every entry is a proposal *(P)* until it is decided.

B.1 The instruction set and the system

B.1.1 The wake rule of `WFE` (P1)

B.1.1.1 The question is whether `WFE` ends its sleep when an interrupt is pending even if the priority masks it; the proposal is yes. *(P)*

Commentary. **Precedent.** An idle loop must check for work and sleep without a gap, or an interrupt that arrives between the two is lost. V7's idle routine lowered the priority and executed `wait` inside a loop that rechecked the run queue after every interrupt, so a lost wake-up cost at most one clock tick. **Trade-offs.** With priority 7 on entry, "lower the priority, then sleep" misses an interrupt that lands between the two, and "sleep at priority 7" never wakes. **Benefits.** The idle loop becomes `spl #7`, check for work, `wfe`, `spl #0`, with no gap and no timer tick to bound it; no opcode, a few gates. **Modern comparables.** AArch64 and RISC-V `wfi` resume on a pending enabled interrupt (RISC-V's manual says the global enable bit is ignored and that an implementation may wake for any reason, so "the same as RISC-V" holds only if the pdp64's priority mask is treated like that bit); x86 closes the gap with the one-instruction interrupt shadow of `STI`, which is hidden state. **Impact.** *Kernel:* The idle loop needs no tick to bound a lost wake-up. *Compiler:* Spin-wait and idle code use `WFE` and `HINT #1`. *Hardware:* A pending-interrupt line into the wait logic, a few gates.

B.1.2 An unprivileged time counter (P3)

B.1.2.1 The proposal is `RDTIME rd`, an unprivileged read of a free-running, constant-rate, monotonic counter. *(P)*

Commentary. **Precedent.** The PDP-11 and VAX kept the time in memory updated by a line clock and read it with a system call. **Trade-offs.** A user-readable counter is a timing side channel, so the kernel needs an enable bit; the counter's width and rate become architecture. **Benefits.** A clock read without a trap, which is what a fast `clock_gettime` in a shared page needs. **Modern comparables.** x86 `RDTSC`, RISC-V `rdtime`, PowerPC `mftb`, s390 `STCKF`, MIPS `RDHWR`, LoongArch `rdtime`; the x86 counter's history of unreliability is the argument for fixing monotonicity and constant rate at the start. **Use.** Timestamps, timeouts, and the fast path of every library's clock call. **Impact.** *Kernel:* A fast `clock_gettime` with no trap. *Compiler:* An intrinsic for the counter read. *Hardware:* One counter, a user-read enable bit and a `SYSX` decode.

B.1.3 Feature and identification registers (P4)

B.1.3.1 The proposal is a block of read-only system registers (numbers 13 and up) giving the version, the optional features, the cache geometry, the page sizes and the address width. *(P)*

Commentary. **Precedent.** The VAX had a system-identification register the kernel switched on; x86 has `CPUID`, which has grown hundreds of leaves and traps under virtualisation. **Trade-offs.** Each register's layout must be written once in a grammar so it cannot drift. **Benefits.** One binary adapts to the machine, and a kernel need not patch itself per model (OpenBSD carries 75 references to its boot-time patching). **Modern comparables.** AArch64's identification registers, RISC-V's `misa`. **Impact.** *Kernel:* The kernel need not patch itself per model. *Compiler:* Runtime selection of routines by feature bits. *Hardware:* A small block of read-only registers.

B.1.4 TLB invalidation by address-space tag (P5)

B.1.4.1 The proposal is one new instruction, `TLBIASID`, that invalidates every translation of one address space, broadcast like `TLBI`. *(P)*

Commentary. **Precedent.** The VAX flushed process translations on every context switch and kept system ones; x86 added tags (PCID) and `INVPCID` much later. **Trade-offs.** A flush-all works until the first rollover of the tag numbers, after which every process loses its translations. **Benefits.** Process exit and tag reuse become cheap. **Modern comparables.** AArch64 `TLBI ASIDE1IS`, RISC-V `sfence.vma` with an address-space operand, x86 `INVPCID`. **Impact.** *Kernel:* Process exit and tag reuse become one broadcast. *Compiler:* None. *Hardware:* A tag compare in the translation buffer's invalidate path.

B.1.5 What a multi-register instruction leaves behind when it faults (P12)

B.1.5.1 The proposal is a rule and a test: every multi-access instruction (`PUSHM`, `POPM`, `PUSHX`, `POPX`, the block operations) is either all-or-nothing or safe to repeat from the start, and an emulator test interrupts each at every access and checks that a restart reaches the same state. *(P)*

Commentary. **Precedent.** The PDP-11 needed a status register (`MMR1`) and, on the 11/40, a kernel routine that decoded the faulting instruction to undo it; the VAX needed a first-part-done bit; ARM's load-multiple restarts with the base register restored. **Trade-offs.** The area below the stack pointer is dead memory, so a partial push is invisible; the risk is another observer (a device or core) seeing a half-written area. **Benefits.** The fault handler restarts blindly, with no instruction-specific cases, which is what recoverability means. **Modern comparables.** RISC-V has no multi-register instruction; ARM and the VAX restart as above. **Impact.** *Kernel:* The fault handler restarts every multi-access instruction blindly. *Compiler:* None. *Hardware:* Each multi-access instruction checks every address before acting, or is repeatable.

B.1.6 Priority 7 on interrupt entry as well as exception entry (P14)

B.1.6.1 Exceptions enter at priority 7, which is decided; the open question is whether interrupts do too, which the implementation does. *(exceptions D; interrupts P)*

Commentary. **Precedent.** The PDP-11 loaded each vector's own priority and pushed the old state on a stack, so an interrupt at any time could not lose it; trap vectors conventionally held priority 7. **Trade-offs.** Entering in registers needs the priority raised, or a higher-level interrupt could overwrite the saved state before the handler stores it; the cost is one `SPL` in a handler that wants to be interruptible. The alternatives are per-vector priority (the PDP-11's choice), a hardware push, and an interrupt shadow. **Modern comparables.** x86, AArch64 and RISC-V all mask interrupts on entry. **Impact.** *Kernel:* One `SPL` in a handler that wants to be interruptible. *Compiler:* None. *Hardware:* Priority 7 loaded into the new status word at entry.

B.1.7 A prefetch instruction (P15)

B.1.7.1 The proposal is none, with the shape of a non-faulting `PREFETCH` with a memory operand reserved on a free sub-opcode. *(P)*

Commentary. **Precedent.** Neither the PDP-11 nor the VAX had one; caches there were transparent. **Trade-offs.** The kernel survey counts over a hundred prefetch instructions across the four kernels, mostly in tuned list and copy code; an in-order core with no data cache gains nothing. **Modern comparables.** AArch64 `PRFM`, ARM `PLD`, PowerPC `DCBT`, x86 `PREFETCHT0`. **Impact.** *Kernel:* List walks and copy loops can warm the cache. *Compiler:* `PREFETCH` would need compiler heuristics. *Hardware:* A non-faulting load path with no destination; nothing on a core with no data cache.

B.1.8 Performance counters (P16)

B.1.8.1 The proposal is two free-running counters, cycles and instructions retired, readable in user mode, added after the clock (P3) with the same enable bit. *(P)*

Commentary. **Precedent.** Unix profiled by sampling the program counter at clock ticks. **Trade-offs.** A user-readable cycle counter is a timing channel; a full counter block with event selection costs a large block and interrupt machinery. **Modern comparables.** x86 `RDPMC`, the AArch64 PMU, RISC-V's `hpm` counters. **Impact.** *Kernel:* A profiler reads counters instead of sampling at ticks. *Compiler:* Intrinsics. *Hardware:* Two counters and an enable bit.

B.1.9 Constant-time execution (P17)

B.1.9.1 The question is how to guarantee that instruction time does not depend on operand values; the proposal is a core set of instructions that is always constant-time plus a mode bit that extends the promise to divide, `MSCN`, `MCMP` and floating point. *(P)*

B.1.9.2 Closed in part: the branch-free select is adopted, at least one secure core has every timing channel removed (all instructions and cache fetches take fixed time, no periodic tick), and on that core each instruction takes a time independent

of its data, with no ticks or timing interrupts introduced to achieve it (errata 85, 86; `docs/crypto-secure-core.md`); the mode bit of the proposal above is superseded for the secure core. *(D)*

Commentary. **Trade-offs.** Making all integer instructions constant-time would cost about 1% on average by a rough estimate, because divide is only 0.04% of compiled instructions (149 of 370,115; multiply costs nothing, since a fixed-latency multiplier needs less logic than one with an early-out); the mode bit keeps fast paths for ordinary code. A constant-time instruction does not make code constant-time: a secret-dependent branch or table index still leaks through the predictor and the cache. **Precedent.** Neither ancestor addressed it; the PDP-11's multiply and divide were microcoded with operand-dependent times. **Modern comparables.** Arm's `DIT` bit, Intel's data-operand-independent timing mode, RISC-V's scalar-crypto timing subset. **Use.** Cryptography on secret keys. The performance cost of those modes is not established. **Impact.** *Kernel:* Crypto code sets the mode around secret-dependent work. *Compiler:* The back end must not turn a constant-time sequence into a variable-time one. *Hardware:* The core set costs nothing; the mode bit needs the fast paths of divide and the scanners to be gateable.

B.1.10 The literal pool and execute-only code (P18)

B.1.10.1 The proposal is an ABI rule: literal pools move out of `.text` into a read-only section beside it, so code pages hold only instructions and can be execute-only. *(P)*

Commentary. **Precedent.** The PDP-11's immediates sit in the instruction stream, and with separate instruction and data spaces a program could not read its own instructions as data. **Trade-offs.** The PC-relative reach limits how far the pool may be, and the linker must keep the two sections near each other. **Benefits.** Zero hardware; a memory-disclosure bug no longer reveals code. **Modern comparables.** OpenBSD makes code execute-only on amd64 and arm64; ARM compilers offer a flag to keep pools out of code. **Impact.** *Kernel:* None. *Compiler:* Pools are emitted into a read-only section beside the text. *Hardware:* None; the linker keeps the sections within PC-relative reach.

B.1.11 Vector and SIMD math (S1 to S6)

B.1.11.1 The device route is decided: the processor has no vector register file and no SIMD instruction set, and wide data-parallel work (graphics, AI and large-language-model inference, the array languages APL and K) is done by memory-mapped devices (chapter 8). *(D)*

B.1.11.2 Open are the number formats and operations of the devices and the reconciliation of the device descriptors with the standard descriptor; the six questions are: where data-parallel work lives (coprocessor, packed lanes in the integer registers, a fixed-width vector file, a scalable one, a matrix unit); which registers hold vectors; fixed or scalable length; how the machine state is saved on a context switch and whether the kernel may use it; which number formats (half precision, bfloat16, 8-bit integer dot products, 8-bit floating point); and the ABI and compiler support. *(P)*

Commentary. **Precedent.** The PDP-11 sent heavy numerical work to attached array processors; the VAX had a vector extension on its 6000 series, with 16 vector registers of 64 elements, supported by VMS; neither Unix used it much. **Trade-offs.** A fixed-width vector file brings a second register file and its context-switch cost, opcodes, and a new width for every generation (the history of SSE, AVX and AVX-512); a scalable design runs one binary on a small and a large implementation at the price of the most complex specification. Lazy saving of vector state was a security hole on x86, and Linux went back to eager restore. **Benefits.** The coprocessor needs no opcodes and no processor state. **Modern comparables.** ARM SVE and RISC-V V (scalable), ARM NEON, x86 AVX, PowerPC VSX, Alpha MVI. **Use.** Neural-network inference (the LB64 accelerator), graphics, signal processing and the library's string and memory routines. The evidence needed first: compile the corpus at higher optimisation and count vectorised loops, measure the share of time in memory and string routines, and decide the target workloads. **Impact.** *Kernel:* No vector state to save. *Compiler:* No vectoriser for the processor; the runtime lowers array primitives to descriptors. *Hardware:* No register file; the devices carry the width.

B.2 The bus, boot and devices

B.2.1 Data-cache maintenance by address (P6)

B.2.1.1 The proposal is to reserve nothing: DMA buffers are uncached, so no cache instruction is needed until a non-coherent device appears. *(P)*

Commentary. **Precedent.** The 11/70's cache watched DMA writes; the VAX's adapter had data paths to purge. **Trade-offs.** Uncached buffers are slow to read; cache maintenance would speed them and needs three reserved sub-opcodes.

Modern comparables. ARM, MIPS and embedded PowerPC with non-coherent DMA wrap cache maintenance in a DMA-mapping layer; x86 DMA is coherent. **Impact.** *Kernel:* Drivers need no cache code. *Compiler:* None. *Hardware:* No cache-maintenance operations.

B.2.2 Firmware calls (P7)

B.2.2.1 The proposal is none: everything goes through devices, with a required reset register and a watchdog in the bus controller. The kernel survey disagrees and would specify a firmware-call convention on reserved trap vectors. *(P)*

Commentary. **Precedent.** The PDP-11 kernel never called firmware; the VAX had console registers the kernel read and wrote. **Trade-offs.** No firmware means no firmware to trust and no extra privilege level, and every platform must say where its console and reset registers are, which the others get from firmware. **Modern comparables.** RISC-V's supervisor binary interface, ARM's PSCI and secure monitor calls, Alpha's PALcode, SPARC's Open Firmware. **Impact.** *Kernel:* Reboot and power-off through a device register and a watchdog. *Compiler:* None. *Hardware:* A reset register and a watchdog timer in the controller.

B.2.3 Double-width compare-and-swap and a random-number instruction (P8, P9)

B.2.3.1 The proposals are to add neither: the evidence for a 128-bit compare-and-swap is one Linux use (software can tag a pointer with the 25 spare bits instead), and random numbers come from an entropy device on the bus rather than an instruction. *(P)*

B.2.3.2 The random-number pool is one device shared by all cores, with no per-process pool, no version register and no updatable firmware; it assists the operating system's own generator (errata 85). *(D)*

Commentary. **Precedent.** The VAX solved the lock-free queue with whole-queue interlocked instructions; no machine of the period had a random-number instruction. **Modern comparables.** x86 `CMPXCHG16B`, AArch64 `CASP`, RISC-V `AMOCAS.Q`; x86 `RDRAND` and `RDSEED`, PowerPC `DARN`, s390 `PRNO`, AArch64 `RNDR`. **Impact.** *Kernel:* Entropy from a device at boot. *Compiler:* Tagged pointers in software if a double-width compare-and-swap is not wanted. *Hardware:* An entropy source device; no new opcode.

B.2.4 The interrupt controller and routing to cores (R1)

B.2.4.1 The proposal is wired interrupt lines to core 0 with a per-line target register, keeping the eight priority levels, with message-signalled interrupts as an extension. *(P)*

Commentary. **Precedent.** The PDP-11 had no controller; a device asserted one of four request lines and supplied its vector. **Trade-offs.** The smallest design adds no new failure cases and cannot give a fast multi-queue device its own interrupt. **Modern comparables.** x86's APIC, ARM's GIC, RISC-V's PLIC and AIA. **Impact.** *Kernel:* A handler sees one interrupt model. *Compiler:* None. *Hardware:* Wired lines to one core with a target register.

B.2.5 Inter-processor interrupts and starting other cores (R2, B2)

B.2.5.1 The proposal is a per-core doorbell register and a per-core start register in the bus controller (4.7), with `SEV` and `WFE` for waiting. *(P)*

Commentary. **Precedent.** The VAX's console processor started the other processors. **Modern comparables.** x86's INIT and START-UP sequence, ARM's PSCI and spin table, RISC-V's hart-start call. **Impact.** *Kernel:* One register write per core and one per doorbell. *Compiler:* None. *Hardware:* A start register and a doorbell per core.

B.2.6 How coherence scales (R3)

B.2.6.1 The proposal is to keep snooping for the four cores of the first device; the ceiling is now sixteen or more (errata 92), and the protocol for that many is open (7.7.4). *(P)*

Commentary. **Trade-offs.** Directory protocols scale and are large; software coherence is cheap and fragile. **Modern comparables.** x86's snoop filters and directories across sockets, ARM's coherent interconnects. **Impact.** *Kernel:* Cache coherence is invisible to the kernel. *Compiler:* None. *Hardware:* Snooping on two to four cores.

B.2.7 The bus transport (R4)

B.2.7.1 Closed: the transport is PCI Express, with the pdp64's convenience and security circuitry on top (errata 75, 80). *(D)*

Commentary. **Trade-offs.** An ordinary PCI Express device works unchanged, and the Unibus's lock is not a native transaction (the atomics are a monitor, section 2.10). **Modern comparables.** Every current system board. **Impact.** *Kernel:* None. *Compiler:* None. *Hardware:* A standard root complex.

B.2.8 The device layout and the descriptor (R5)

B.2.8.1 Closed: the device interface is virtio, the 16-byte descriptor and the available and used rings of chapter 5 (errata 95). *(D)* The feature subset of each device is open. *(S)*

Commentary. **Precedent.** The RK11 and the DEUNA, as in chapter 5. **Trade-offs.** A standard interface gives up the freedom to shape the registers and gains every existing driver. **Modern comparables.** `virtio`, NVMe, SATA's command lists, Intel Ethernet's done bit. **Impact.** *Kernel:* One transport driver and a thin driver per class. *Compiler:* The descriptor is a generated C structure. *Hardware:* Standard devices.

B.2.9 DMA addressing and protection (R6)

B.2.9.1 Superseded: an IOMMU is part of the design and its five recommendations are accepted (errata 75, 85); bits 40 to 63 of its leaf are free. *(D)*

Commentary. **Precedent.** The Unibus carried only 18 address bits, so the VAX and the larger PDP-11s had map registers that translated device addresses, in effect a simple translation unit for necessity. **Trade-offs.** No protection from a faulty or malicious device; a translation unit matters when virtualisation or untrusted devices do. **Modern comparables.** x86 VT-d, ARM SMMU, the RISC-V IOMMU; Linux's bounce buffers for limited devices. **Impact.** *Kernel:* A driver passes physical addresses. *Compiler:* None. *Hardware:* No translation unit for devices.

B.2.10 Errors raised by devices and the bus (R7)

B.2.10.1 The proposal is the precise and imprecise paths the ISA already has plus a small register block in the bus controller for the cause. *(P)*

Commentary. **Precedent.** A PDP-11 bus timeout trapped through vector 4 and usually ended in a panic; the VAX had a machine-check frame. **Modern comparables.** x86's machine-check banks and PCI Express error reporting, ARM's RAS extension. **Impact.** *Kernel:* One recovery path for device errors. *Compiler:* None. *Hardware:* A small error register block in the controller.

B.2.11 Virtualisation (R8)

B.2.11.1 The proposal is to declare it out of scope for System v1 and fix five constraints: privileged state stays privileged or controllable, the page-table walker could be wrapped by a second level, interrupts are injectable, system-register access can trap, and the translation-enable bit and software accessed and dirty bits are reviewed for what a hypervisor would need. *(P)*

Commentary. **Precedent.** The PDP-11 was not designed for virtualisation and has sensitive instructions that do not trap in user mode; the VAX was not cleanly virtualisable either, and a 1991 paper described the changes needed. **Trade-offs.** Adding virtualisation late cost the VAX instruction changes; settling the constraints early costs a paragraph. **Modern comparables.** x86 VMX with nested paging, ARM exception level 2, the RISC-V hypervisor extension, IBM's trap-and-emulate machines. **Impact.** *Kernel:* A hypervisor could be added without changing a guest. *Compiler:* None. *Hardware:* Constraints on privileged state, the walker and interrupt injection.

B.2.12 Enumeration and hot-plug (N1 to N3)

B.2.12.1 The proposals are enumeration by PCI Express configuration space with the windows placed in the physical address space (N1, closed in outline by errata 80; the placement is open), an interrupt and a rescan on a hot-plug event (N2), and an absent mark with a precise fault and the queued work left in memory (N3). *(P)*

Commentary. See section 2.4 and its commentary: the VAX's nexus scan, PCI's configuration space and all-ones reads, USB's status-change endpoint. **Impact.** *Kernel:* Enumeration and hot-plug are one loop and one rescan. *Compiler:* None. *Hardware:* A standard root complex and the precise fault for an absent device.

B.2.13 The boot decisions (B1, B3, B4, B5, B8)

B.2.13.1 The proposals are a boot ROM that reads block 0 (B1), a boot-information block of memory, cores, the root complex's address and the boot device and console (B3), an `INIT` bit with an optional firmware descriptor (B4) and a block list in the protective MBR's boot code (B8). *(P)* There is no UEFI, with a hand-off and a second stage simple enough for a shim (B5, decided by the author, errata 109, 4.9). *(D)*

Commentary. See chapter 4 and `../boot.md`: the PDP-11's block 0 and ODT, the VAX's boot block and restart parameter block, the BIOS and UEFI (4.9 for the UEFI decision and the shim). **Impact.** *Kernel:* A boot-information struct and a core start register. *Compiler:* The loader and first block are built by the project's compiler. *Hardware:* A boot ROM, a block-0 reader and per-core start registers.

B.2.14 How USB attaches, and virtio first

B.2.14.1 Closed in part: USB is attached over PCI Express, and virtio-net, virtio-gpu and virtio-snd are the standard network, graphics and audio interfaces (errata 75, 79). *(D)* What those decisions do not say is whether USB is a conventional ring-based host controller driven by an existing operating-system USB stack, the proposal of chapter 5, or a controller that does the protocol itself.

Commentary. **Trade-offs.** A controller that does the protocol moves it into the chip and takes the stack out of the operating system; the conventional controller uses code that already exists. `virtio` reuses OpenBSD's existing drivers in the emulator and in silicon. **Modern comparables.** xHCI for the conventional controller; `virtio-mmio` on every emulated RISC-V and ARM board. **Impact.** *Kernel:* A disk driver serves a USB disk; virtio reuses the existing drivers. *Compiler:* None. *Hardware:* USB protocol in the controller costs gates; virtio costs nothing for the emulator.

Appendix C — Glossary

Terms the manual uses, collected in one place. Each entry gives the plain meaning and, in brackets, the chapter or section where the manual deals with it. Terms for the instruction groups (the "escape" opcodes) are listed by name and opcode; the full encodings are in the instruction-set text `../pdp64-isa.md`.

C.1 Architecture and encoding

Term	Meaning
pdp64	the 64-bit extension of the PDP-11 this manual describes, as the PDP-11 extended the PDP-10's ideas down and the VAX extended the PDP-11 up (chapter 1)
tested alternative	pdp64/r20 or pdp64/16: an encoding built and measured against pdp64/32, which is the pdp64, and documented in its own manual (1.8)
parcel	a 16-bit unit of instruction encoding; an instruction is one to three parcels, two to six bytes (9.4)
`len`	bits 15 and 14 of the first parcel: the number of extension parcels for `00`, `01` and `10`; the value `11` selects the second opcode page (9.4)
page 1, page 2	the two opcode maps. Page 1 has 64 top-level opcodes, 58 assigned; page 2 (`len = 11`) has 64 one-parcel entries, 45 defined, each an exact abbreviation of a page-1 instruction (9.1)
top-level opcode	the six bits that select an instruction group; reserved for an operation whose common case is register-to-register (9.1)
escape opcode	a top-level opcode that selects a group by a sub-opcode: `BESC` (bits, `0x0C`), `MBLK` (block memory, `0x0D`), `AESC` (arithmetic, `0x0E`), `MESC` (unaligned and memory ALU, `0x0F`), `FESC` (floating point, `0x10`), `ATOM` (atomics, `0x11`), `XFER` (indirect transfers, `0x12`), `SOP` (single operand, `0x13`), `SYS` (system, `0x14`), `TAGOP` (tag branch, `0x15`)
`CBCC`	compare-immediate-and-branch on condition, opcode `0x28`, the one fused compare and branch (9.10)
`RORI32`	rotate right by an immediate, 32-bit, sign-extended result, a `BESC` sub-opcode
`SOB`	subtract one and branch, either direction (9.10)
`MCPY`, `MMOV`, `MSET`, `MCMP`, `MSCN`, `MSCNN`	the block operations: copy, move (overlap-safe), fill, compare, scan for a byte, scan for the first byte not equal; progress is held in registers so a fault restarts for free (9.11)
`MVAL`	block scan of valid UTF-8, one of the string instructions of the same family (9.9)
`PUSHM`, `POPM`	push and pop a register set named by a mask; the useful half of the VAX's `CALLS` without the frame (9.11)
`JMPI`, `CALLI`	jump and call through a memory word; the only deferred forms, used for PLT stubs and imports (9.10, 9.16)
`JTAG`, `TAGOP`	multiway dispatch on a pointer's low tag bits, and the tag-branch group (9.10)
`LPAD`	marks a legal target of an indirect jump or call; optional in ABI version 1 (appendix A.1)
`CLMUL`	carry-less multiply, the primitive under every CRC and Galois-field operation (9.9)
`XNORPOP`, `POPCNT`	bit-population instructions used by binary operations and bitsets (8.3)
`SET cc, rd`	set a register to 0 or 1 from a condition (9.7)
`DP`	the direct pointer: in user mode it is the thread pointer, in kernel mode the per-CPU pointer; a descendant of the 6502's zero page (9.3, 9.16)
`PSW`	the processor status word: flags, mode, priority and the guard bit (6.2)
`N Z V C`	the four condition flags: negative, zero, overflow, carry (9.3)
canonical address	a 64-bit address whose bits above the top bit of the virtual address (bit 38, 47 or 56 with three, four or five levels) equal that bit; any other value faults, so software cannot hide tags in the high bits (6.6)
`lowtag`	a type tag stored in the low bits of a pointer, free because objects are quad-aligned (instruction-set text, section 6.2; not otherwise used in this manual)
SWAR	SIMD within a register: using one wide scalar register as several narrow lanes through masking and carry-aware arithmetic (instruction-set text, glossary; the manual's bit and byte instructions of 9.9 make it effective)
`FPSR`	the sticky floating-point exception flag register, read and written by `FSFLAGS` (9.12)
ulp	unit in the last place: the spacing between adjacent floating-point numbers, the unit of rounding error
Payne–Hanek	the argument-reduction algorithm that computes x mod π/2 to full precision for large x; what hardware transcendentals cannot do (instruction-set text, section 9.7.1; the manual's account of floating point is 9.12)
Table Maker's Dilemma	that correct rounding of a transcendental may need unboundedly many bits to resolve a near-tie, so no fixed-time circuit can guarantee it (instruction-set text, section 9.7.1)
block floating point	integer mantissas sharing one exponent, used by the LB64 accelerator so fixed-point accumulation is exact (8.3)

C.2 Memory, protection and the kernel

Term	Meaning
`MT`	memory type, a two-bit page attribute: normal cached, normal uncached, device strongly ordered (2.3, 6.7)
`ASID`	address space identifier: tags translation-buffer entries so a context switch flushes nothing (6.8)
`G` **bit**	global: a translation-buffer entry that ignores the `ASID` (6.8)
TLB	translation lookaside buffer: the cache of page-table entries; `TLBI` invalidates one, `TLBIA` all, `TLBSYNC` waits for other cores (7.5)
`DMW`	direct mapping window: a region translated by a fixed offset with no page table; stored without effect, the format is open (6.9)
`MMUCTL`	the register that enables translation and sets the table base (6.5, 6.8)
virtual address, physical address	the number a program uses, and the number the memory answers to; the page tables translate one into the other (6.6)
page, frame, page table	a 4 KB piece of virtual space, a 4 KB piece of physical memory, and the tree of tables of 512 entries that says which frame holds which page (6.6)
PTE	page-table entry: one 64-bit word with the valid, permission, accessed, dirty, global and memory-type bits and the physical page number (6.7)
leaf, pointer	the two kinds of page-table entry: one that maps a page (`R`, `W` or `X` set) and one that points to the next table (6.7)
`KT11`, `PAR`, `PDR`	the PDP-11's memory-management unit, with eight pairs of page address and page descriptor registers for each mode (6.1, 6.6)
`EPC`, `EPSW`, `EVA`, `ECAUSE`	the registers that exception entry writes: where to resume, the old status word, the faulting address, the vector number (6.3)
`VBR`	the vector base register: where the table of 512 eight-byte entries begins (6.3)
`ISYNC`	makes earlier writes to system registers take effect: wait for earlier instructions, discard later ones, fetch again (6.5)
`MFSR`, `MTSR`	move from and to a system register, by number (6.5)
software-managed `A`, `D`	accessed and dirty bits set by the kernel's page-fault handler, never by the walker (6.8)
shootdown	removing a stale translation from the buffers of other cores; on the pdp64 `TLBI` broadcasts and `TLBSYNC` waits (6.8)
walker	the hardware that reads page-table entries on a translation-buffer miss (6.6)
`STUA` / `CLUA`	set and clear the user-access guard bit, which lets kernel code touch user memory only inside the copy routines (6.10)
`PROBER` / `PROBEW`	test whether an address is readable or writable in a given mode without faulting (6.10, 6.13)
W^X	write xor execute: no page is both writable and executable; OpenBSD enforces it (appendix A)
PIE	position-independent executable: loadable at any address; costs nothing here (9.16)
GOT, PLT	global offset table and procedure linkage table: the indirection that lets position-independent code reach imported symbols (9.16)
ABI	application binary interface: calling convention, register use, data layout; version 1 is given in appendix D
trap, exception, interrupt	a deliberate entry (`TRAP #n`), a fault raised by an instruction, and a signal from a device; all enter through the vector table (6.3)
precise fault	a fault after which every earlier instruction has completed and no later one has changed state (6.4)
`SPL`	set the interrupt priority level (6.3)
`WFE`, `SEV`	wait for an event and send an event: the idle and wake-up pair (7.6)
`FENCE`	orders memory accesses; owed to devices and to other cores (2.8, 7.2)
`ATOM`, **monitor**	the atomic operations, built on a reservation monitor as in AXI and ARM, not a locked bus cycle (7.3)
SMP	symmetric multiprocessing: several identical cores sharing memory (chapter 7)
core class	the read-only register of each core that gives its class and capacity, so that software can tell an in-order core from an out-of-order one (7.8.4)
secure core	a core with every timing channel removed (every instruction and cache fetch takes a time independent of its data), private cache lines and no periodic tick (7.8.6)
dedicated core	a core given to one job, graphics, network or audio; proposed (7.8.7)
coherence, snooping MSI	the protocol that keeps each core's cache consistent with the others; snooping means each cache watches the bus; MSI is its three states (modified, shared, invalid) (7.7)
sequential consistency	the memory model in which all cores see all accesses in one order; the pdp64 declares it (7.2)

C.3 The bus, devices and boot

Term	Meaning
Q64	the system bus: the PDP-11 Q-bus lineage widened to 64-bit data and a physical address of 40 to 64 bits (2.3)
Unibus, Q-bus	the PDP-11's buses; Q64 keeps their programming model, with devices in the address space (2.3)
`DATIP` / `DATO`	Unibus and Q-bus transaction types forming a locked read-modify-write (2.10)
window	a region of the physical address space that a device's base-address register describes, a whole number of pages, found by enumeration (2.4, 5.1)
root complex	the PCI Express root of Q64: it enumerates the functions, assigns their windows, and reports arrivals and departures (2.4)
bridge	a function that is itself a bus: it presents the functions behind it, so arbitrary buses (PCI Express, USB) can be connected to Q64 and enumerated to any depth (2.4.3)
MMIO	memory-mapped input/output: device registers reached by ordinary loads and stores (5.1)
DMA	direct memory access: a device moving data to or from memory without the processor (5.4)
virtqueue, descriptor, available ring, used ring, doorbell	the common device protocol: a table of 16-byte descriptors in memory, a ring in which the driver offers chains, a ring in which the device returns them, and a write that tells the device to look (5.2)
`device_status`	the common-configuration field through which the driver and the device handshake; writing zero resets the device (5.2, 5.3)
virtio	a standard family of devices of the same queue-and-descriptor shape, the pdp64's device interface (5.2)
virtio-blk	the standard disk: a virtqueue of read, write and flush requests (5.2.7, 5.5.1)
NVMe, SCSI, ATA, SATA	storage interfaces, each a command queue in memory and a doorbell, which is the same device pattern (5.2); an NVMe device may be attached as any PCI Express device may, but the standard disk is virtio-blk
HDMI transmitter	the chip that takes a pixel stream and does the link protocol; the pdp64 side is a pixel generator and registers (3.14)
`VIC-64X`, `SID-64X`, `BLT64`	the video, sound and blitter devices of the graphics terminal, which is based on the Commodore 64 (video and sound chips) and the Blit terminal (framebuffer and bit-block transfer) (chapter 3)
Blit	Rob Pike's bit-mapped windowing terminal: an 8 MHz Motorola 68000 machine that received downloaded code, characters and editing messages from its host over a serial line and drew every pixel itself (its description as a precursor of the X11 protocol and of NeWS is stated by no source read *(S)*; the lineage the sources confirm runs from its `mux` to 8½ and `rio`); a framebuffer in the address space and a bit-block-transfer operation, done in software, that moves rectangles of pixels; with the C64, one of the two machines the pdp64's terminal is based on (3.1)
blitter, raster operation	a device that copies and combines rectangles of pixels; the raster operation says how source and destination combine (3.10)
scanout, tearing, double buffering	the display reading the picture out of memory row by row at a fixed rate; the seam where a picture changed during the reading shows old above and new below; drawing the next frame in a hidden second picture and swapping at the vertical blank (3.8)
glyph map, glyph atlas	a table of slots each holding the picture of one glyph, filled by a TrueType engine (3.11); the GPU form is one texture holding every glyph the screen uses (3.7)
rio, NeWS	Plan 9's window system, in which each window is a private screen served as files, a descendant of the Blit's `mux` through 8½; and Sun's window server that programs downloaded from the host could extend (3.19)
mixer, ring buffer, underrun	the program that sums the sample streams of several programs into one after converting their rates and formats; the circular buffer the sound device reads at the converter's clock; the reader reaching samples not yet written, audio's tear (3.20)
GPU, shader	a graphics processing unit, thousands of simple lanes; a shader is a small program it runs for every vertex or every pixel (3.6)
bootstrap, boot ROM, block 0	the program that loads programs; the ROM in the controller chip that reads the first block of the boot device into memory and jumps to it; that first 512-byte block (4.1, 4.4)

Term	Meaning
BIOS	the PC's firmware: bring-up and self-test, the first sector, a library of services called by software interrupt, and setup; the pdp64's "BIOS" is its boot ROM and monitor, with no resident services; open source (4.2, 4.3)
first stage, second stage, `installboot`	the program in block 0, which carries a list of where the next program lies; the C program that reads the label and the file system and loads `/bsd`; the tool that writes the first stage and its list (4.3, 4.5, 4.6)
partition, MBR, protective MBR	a named region of a disk; the PC's first sector, with 446 bytes of boot code, a four-entry table and `0xAA55`; the MBR of a GPT disk, with one entry of type `0xEE` covering the disk (4.2, 4.5)
GPT, disk label	the GUID partition table wrapped around the disk's own label so other systems see the disk is in use; the BSD label that names the root, swap and other partitions inside one partition (4.2, 4.5)
boot-information block	the structure whose address the second stage passes in `r1`: memory ranges, cores, the root complex, the boot device, the console, the screen, the command line (4.6)
UEFI, PE/COFF, shim	the PC's later firmware standard, which runs a loader file of the PE/COFF format and offers it boot services; not used by the pdp64; a shim is the small program that would offer those services over virtio to an existing loader (4.2, 4.9)
`ODT`**, monitor, watchdog**	the PDP-11's console debugger; the ROM's console program for examine, deposit, load and start; a timer that restarts the machine if the kernel stops writing to it (4.1, 4.10)
INIT, START-UP, spin table, hart	x86's signals that start another core; ARM's table the waiting cores poll; RISC-V's word for a core; the pdp64 writes a start register instead (4.7)
LB64, MX64, VEC64	the inference accelerator and its companions: sign-flip-accumulate layers, floating-point multiply-accumulate for attention, and element-wise vector work (8.3)
APL, K	array languages whose primitives work on whole arrays; one primitive maps to one descriptor (8.4)

C.4 Process and sources

Term	Meaning
errata	the numbered decision log of the project (`../errata.md`, appendix F)
fair corpus	the 827 or 841 C files that every back end compiles, used for all size comparisons (1.5)
`pcc`	the portable C compiler the project extends to emit pdp64 code (1.5)
`sim64`**, QEMU**	the instruction-set simulator and the full-system emulator the tests run on (1.5, 10.4; not otherwise described in this manual)
(D), (P), (S)	the tags on a numbered statement: decided, proposed, stub (index)
Impact	the line ending each decision's commentary: effect on the kernel, on the compiler and on the hardware layout (index)

Appendix D — The ABI, version 1

The application binary interface fixes how compiled code, the linker and the operating system agree: the sizes of types, which registers carry arguments and results, who saves what, how the stack looks, and how position-independent code reaches its data. These are decided choices (section 9.16 states what the instruction set makes cheap). The source is `../abi.md`; the compiler, the assembler and the linker follow it, and the version number allows a later change without breaking old binaries; in version 1 `long double` is `double`.

D.1 The conventions

Data model	LP64: `char` 8, `short` 16, `int` 32, `long` 64, `long long` 64, pointers 64. Little-endian. `char` is signed. `long double` is `double`.
Alignment	natural; the stack pointer is a multiple of **8** at all times (data alignment is architectural, ISA 7.4: a quad access to an address not a multiple of 8 aborts; the 2-byte parcel alignment is for instructions only). No 16-byte alignment: no type or register is wider than a quad.
Registers	`R0-R5` arguments, `R0` (and `R1`) results, `R0-R7` and `R12` caller-saved; `R8-R11`, `R15` callee-saved; `R12` linker scratch; `R13` frame pointer (callee-saved, optional); `R14` stack pointer.
Floating point	`binary64` in a general register, `binary32` in the low 32 bits sign-extended (ISA 2.3); passed and returned exactly like integers.
Arguments	in order in `R0-R5` (a struct of up to 16 bytes takes one or two consecutive, as many as needed); the rest pushed right to left, each in an 8-byte slot (a larger struct in as many slots as it needs, by value).
Results	`R0`; `R0:R1` for a 128-bit pair or a struct up to 16 bytes; a larger struct through a hidden pointer passed in `R0` (the arguments then start at `R1`), and returned in `R0`.
Calls	`CALL`/`CALLI` push the return address; `RET` pops it. The caller removes stack arguments.
Varargs	the callee stores `R0-R5` in a save area directly below the stack arguments; `va_list` is a pointer into the area, then on into the stack arguments. One area suffices because floats use the same registers.
Red zone	none (version 1).
Symbols	no leading underscore.
Landing pads	off in version 1; the assembler provides `.lpad`.
PIC	globals through `d(PC)`; imported symbols through `MOVQ got(PC),Rd`; calls to imports through `CALLI got(PC)`; TLS local-exec `MOVQ off(DP),Rd`.

D.2 Variadic calls

In a call to a function declared with …, every argument goes in an 8-byte slot, in order: slots 0-5 are `r0-r5`, later slots are on the stack. A structure of any size is passed by value in consecutive slots and may split between `r5` and the stack. A variadic callee leaves its named parameters in their slots (`fp+16+8k`) and `va_start` is the address after the last named slot; `va_list` is a `char *` and `va_copy` is assignment. The prologue pops the return address, pushes `r5`..`r0` and pushes the return address again, so the register save area is contiguous with the stack arguments.

D.3 What the ABI leaves open

Lazy against eager binding, mandatory landing pads and whether imports are always called without a PLT stub beyond version 1 are open (section 9.16, decisions in appendix B).

Appendix E — The assembly language and the object format

The assembler's source layout is the V7 assembler's, `as`; its lexical rules are C's. This appendix is the syntax of a source file: lines, labels, numbers, strings, operators, directives, registers, the operand modes, the mnemonic conventions and the object format. The grammar itself is `spec/pdp64-asm.hbnf`; the source of this appendix is `../asm-syntax.md`. The mnemonics are in the instruction dictionary.

The grammar is `spec/pdp64-asm.hbnf` (hbnf text rules); `tools/check-asmsyntax.sh` generates the parser and runs `tests/asmsyntax.c` over it, and `tools/check-as64.sh` assembles `tests/asm/*.s` and compares the bytes with hand-worked expectations.

- **Lines.** One statement to a line. `;` starts a comment that runs to the end of the line.
- **Labels.** `name:` defines a label; `1:` defines a numeric label, referenced as `1b` (the nearest one before) or `1f` (the nearest after). `sym = expr` assigns. `.` is the location counter. Names are letters, digits, `_`, `.` and `$` and any non-ASCII code point (`café`, `Ж–ブ`); a name may start with a digit as long as the whole bareword is not a number (`3dfx:` is a label; `3`, `0x3d`, `0b101`, `0o17` are numbers; `1f` and `1b` are local references), and a bareword of digits only is a numeric local label. Names are case-sensitive and never truncated. ELF symbol names are any bytes, so a name with other characters is written in double quotes (`"odd name!"`). A mnemonic or register name cannot be a label, and neither can `r` followed only by digits: `r16` is an error, not a name (`r2d2` is a name).
- **UTF-8.** A source file is UTF-8, and the parser refuses anything that is not (a stray byte, an overlong form, a surrogate, a truncated sequence), in names, strings, characters and comments alike; a NUL byte is refused too. In a string, UTF-8 text is copied as bytes and `\u00e9`, `\U0001F600` are written as UTF-8; `'é'` is the code point 0xE9, as is `'\u00e9'`.
- **Numbers and characters** are C's: decimal by default (`010` is ten), `0x`, `0b` or `0o` when asked for; `'a'`, `'\n'`, `'\x41'`, `'\101'`.
- **Strings** are `"text\n"` with C's escapes (`\n \t \r \a \b \f \v \0 \\ \' \" \xHH \ooo`), in `.byte` and `.ascii`.
- **Operators** are C's: `* / % & << >>` and `+ - | ^`, unary `- ~ !`, and parentheses. `/` is division. `*`, `/`, `%`, `&` and the shifts need constants; `+` and `-` accept one relocatable symbol, and the difference of two symbols of one section is a constant.
- **Directives.** `.text .data .bss .globl .comm .byte .word .long .quad .ascii .even .align .space .lpad .pool. .word` is 16 bits, `.long` 32, `.quad` 64 (the ISA's PDP-11 terminology). `.align n` aligns to n bytes (a power of two).
- **Registers.** `r0`..`r15`; `sp` is `r14`, `fp` is `r13`. `pc` and `dp` appear only as a base: `d(pc)`, `d(dp)`.
- **Operand modes** (ISA section 5): `rN`; `#expr`; `@expr`; `expr(rN)` and `(rN)`; `expr(rN)(rX*s)` with `s` 1, 2, 4 or 8; `expr(pc)`; `expr(dp)`; `(rN)+` and `(rN)+d`; `-(rN)` and `(rN)-d`; a bare expression is a PC-relative reference.
- **Mnemonics** are lowercase (`movq`, `add`, `bne`). A multi-register instruction writes its destination last: `add src, rd`, `xnorpop ra, rb, rd`. `cmp` is the exception, `cmp rd, src`, because the encoding computes `rd - src`.
- **Compare and branch.** `cbCC rd, #imm, label` compares `rd` with a 5-bit signed immediate (-16 to 15) and branches on the condition: `cbeq r3, #5, label`. The mnemonics are `cbne cbeq cbpl cbmi cbvc cbvs cbcc cbcs cbge cblt cbgt cble cbhi cblos` and the aliases of the branch group (ISA §4.4.1). With a register as the second operand, `cbCC ra, rb, label` compares two registers (`cbeq r1, r2, label`, ISA §4.4.1), in four bytes or, beyond 4 KB, six. `rori32 rs, #n, rd` rotates the low 32 bits right and sign-extends the result (ISA §4.6). The immediate of `cbCC` may be up to 16 bits (`cbeq r3, #255, label`); the assembler chooses the four-byte, the wide or the long encoding, and refuses a wide immediate whose target is more than 64 KB away (ISA §4.4.1).
- **Literal pool: `=expr`.** An operand written with a leading `=` is syntactic sugar for a pooled constant and has no other meaning: `movq =0x123456789ABC, r1` puts the 64-bit value in the literal pool and loads it with one PC-relative `movq`. It is accepted where a quad memory source is (`movq`, `push`, the ALU group with the register first for `cmp`: `cmp rd, =K`, and the floating-point ALU ops), and refused for sized moves, shifts, stores, `lea` and a relocatable symbol. `=` always pools, even a small value; use `#imm` for a constant that fits the signed 32-bit field. A bare `#K` that does not fit that field is an error ("use =K to load it from the literal pool"): the assembler never pools silently. Entries are 8 bytes, 8-byte aligned, one per distinct value per pool, in order of first use, emitted into `.text` at the end of the section or at the directive `.pool` (flush now; code must jump over a mid-section pool). The `=` marks the sugar on the page, which is why it is allowed.

- **Length.** The assembler chooses the shortest encoding, including the one-parcel forms of page 2 (ISA §4.18); those forms have no mnemonics of their own, so `movq (r3), r1` assembles to the one-parcel form when it fits and to the long form when it does not.
- **Invocation.** `as64 -m pdp64/r20|pdp64/16|pdp64/32 [-o obj] file.s`: `-m` selects the encoding (the default is `pdp64/r20`; `r20`, `16`, `32` and `old` are accepted) and the object records it in its `e_flags`; `ld64 [-M] [-o out] objects...` refuses to link objects of different encodings.
- **`llvm-mc`.** The LLVM back end's assembler (`llvm/`, ISA-checked against `as64` by `llvm/check.sh`) accepts this syntax with these exceptions: no literal pool (`=K` and `.pool`), no names that start with a digit, no non-ASCII names, and `.comm` needs its alignment written (`as64` implies 8). `(pc)` and `(dp)` without a displacement are accepted by `as64` (displacement 0) and by `llvm-mc`.
- **Object format.** ELF64, little-endian, `e_machine` 0xF064 (`EM_PDP64`); relocations `R_PDP64_D64`, `R_PDP64_D32`, `R_PDP64_INSN_ABS`, `R_PDP64_INSN_PC`, `R_PDP64_INSN_BR` (`as64/relocs.h`; LLVM: `ELFRelocs/PDP64.def`).

What it leaves out of V7 `as`: octal numbers by default, `<string>` and the `'\e` escape, `\/ \< \>` and `[ ]` in expressions, `.if`, `.stabs`, symbols cut to eight characters.

Appendix F — The evidence behind the manual

The manual states decisions; these are the documents that hold the evidence, the surveys and the long arguments they were drawn from. Each is a document of the repository, not a chapter of this manual, and each says in its own opening what it is based on and what it did not check. They are listed with what they hold and their status; the *Why it is here* paragraphs of the instruction entries and the *Impact* lines cite them.

F.1 The architecture, in full

Document	What it holds
pdp64-isa.md	the full instruction-set text with every encoding, the rejected features, the idioms lost and restored, the comparison with the PDP-11 and the VAX
pdp64-system-architecture.md	privilege, exceptions, interrupts, memory management and boot as an operating system sees them, with what is undefined stated plainly
pdp64-system-design.md	System v1: the options, how others do it, and how the choices serve the project's goals
pdp64-q64-bus.md	the system bus and its devices
boot.md	boot and bring-up: how the main systems start themselves, whether UEFI is worth having, and the proposal
pdp64-in-order-and-out-of-order.md and pdp64-ooo-readiness.md	what the in-order design point costs and buys, and what the architecture must decide to stay ready for an out-of-order core
pdp64-fpga-plan.md and pdp64-lb64-accelerator.md	the FPGA build plan and the inference accelerator
abi.md and asm-syntax.md	the ABI and the assembly language (appendices D and E are drawn from them)
roadmap.md	the order of work from the present toolchain to an OpenBSD that can be logged into

F.2 The record of decisions

Document	What it holds
errata.md	every decision, numbered, with the reasons
decisions-pending.md	every open decision, with the need, five options, the survey of what other systems do and what the PDP-11 and VAX did (appendix B is drawn from it)
fact-check-2026-10-22.md	the comparison of statements about the outside world with a source where one could be read; the claims it could not check are marked "not confirmed" in the manual

F.3 Measurements and surveys

Document	What it holds
codegen-analysis.md	what compiled C says about the instruction set: instruction use, operand forms, the instructions never emitted
kernel-asm-survey.md	what Linux, FreeBSD, NetBSD and OpenBSD write in assembly, and what pdp64 can learn from it
kernel-pain-points.md	what hurts kernel developers: tight loops, pain points and workarounds in the current kernels, from the source trees and the mailing lists, compared with pdp64
hardware-vulnerabilities.md	the processor, memory and system flaws of the last decade that needed mitigations, and the rules that keep pdp64 from repeating them, including the evidence on constant-time modes
openbsd-mitigations.md	OpenBSD's memory-safety and exploit mitigations, what pdp64 gives for free and what would make the rest soft and easy
sim64.md	the reference emulator
smp-decisions.md	the open symmetric-multiprocessing decisions, proposals awaiting the author's annotations (tag P)

Document	What it holds
crypto-secure-core.md, security-proposals.md, study-proposals.md, retire-stage-candidates.md	the proposals on the secure core, the security mechanisms, the studied instructions and the retire-stage mechanisms, with the author's replies in errata 84 to 86
priority-lists.md	the lists of priorities that errata 83 settled

The data behind the tables is in `docs/codegen-data/`, the generators are in `tools/`, and the sources of the design discussions are `chat01.txt` and the archive of the working session.

Appendix G — The evolution of the design

The design began as a small question and grew by steps. In the author's words of 2026-11-10: "we started off as a pdp16 modernized a tiny bit, then got more and more ambitious", and "the initial idea was, could a pdp16 run effectively today with a blit terminal and off-loaded 8-register nic", with "the blit being a mashup with the c64 for audio". This appendix lists, in point form, how it grew and which influences, inspirations and precedents entered, each with the date and time (UTC) of the message in which it first appeared.

G.1 How it grew

- 2026-08-23 05:50 — the founding question, in his first message about it: an Ethernet card "that held the entire tcp ip udp icmp stack inside itself ... so that a pdp11 could plug it in and use it", and whether the PDP-11 could use the Blit terminal (813:0)
- 2026-08-23 05:58 — wider addresses: "was there a 22 bit pdp11?" (813:6); at 06:19: "What if all addresses were expanded to 22 bit? ... an update to the pdp11 arch" (813:10)
- 2026-08-23 06:21 — "Is there pdp11 fpga?" (813:12); at 06:46: "Could I have dozens of pdp11 on a postage stamp? How about the blit?" with 24-bit colour and 1024x768 (813:18)
- 2026-08-23 19:54 — "combine 8 pdp11 systems plus one blit system and a smart nic ethernet card all together on one silicon die" (519:2)
- 2026-08-28 19:31 — "if one implemented pdp11 without the peripherals, what is involved?" (792:0)
- 2026-08-29 00:29 — the first design brief: "assume mmu and fp units"; in the same message the C64 graphics and audio, a modern USB keyboard and mouse and an NVMe disk are asked for (531:0)
- 2026-08-29 00:29 — the Blit as the terminal, mashed up with the C64: "could we revamp the blit to be c64 adjacent", since "c64 is superior to blit terminal in many ways" (531:0)
- 2026-08-29 00:49 — the 8502 chosen over the plain C64 as the partner: "they seem much closer to each other"; the same message fixes little-endian and a 200 to 300 MHz FPGA clock (531:4)
- 2026-08-30 09:13 — the 32-bit step skipped: "skip 32 bit version and go straight to 64bit" (533:8)
- 2026-08-30 09:15 — and the widening stated: "basically upgrade pdp-11 straight to 64 bit" (533:10)
- 2026-08-30 09:30 — a constraint on the widening: "only 2 byte memory alignment; don't put 8 byte memory alignment constraints" (533:14)
- 2026-10-20 19:59 — devices become standard hardware on the bus: "we can put standard gpu on the bus too?" (533:36)
- 2026-10-20 22:44 — multiprocessing enters: what the design needs "to allow smp, symmetric mutiprocessing" (533:61)
- 2026-10-22 16:16 — the ambition of a large machine: "ready for the out of order large scale design" (1889:2236)
- 2026-10-22 20:35 — the early rule of no mode bits becomes "a very soft rule" (1889:2500)
- 2026-10-22 20:48 — the priorities written down: "code size, easy recoverability, and speed/efficiency through fewer gates and tighter hot loops" (1889:2506)
- 2026-10-22 23:01 — security as a design goal: "there should be constant time flag ... so instructions always take an equal amount of time" (1889:2605)
- 2026-10-23 00:49 — the terminal takes the C64's audio: a Blitter terminal "based on a modernized and updated c64 with sid chip and also modern 48 bit audio and 48 bit pixels" (1889:2704)
- 2026-10-23 00:53 — the VAX idea of a device space in the address map is added to the book (1889:j79495)
- 2026-11-04 00:07 — the bus becomes PCI Express with NVMe and USB, with the security fixes of the pain-points document
- 2026-11-04 00:57 — cores dedicated to devices: "also one core for audio processing, doing midi, dsp, etc." (1882:j541)
- 2026-11-04 08:07 — a new instruction encoding: "an encoding scheme that minimizes code size and fits them all in with 32 reg and with 16 reg" (1882:172)

- 2026-11-04 09:57 — "the pdp64 now has children, the pdp64/32 and pdp64/16" (1882:j2434)
- 2026-11-04 19:21 — the first device: "4 cores is enough hopefully that fits on the fpga" (1882:258)
- 2026-11-08 02:37 — the link between Commodore and DEC found: "I found the evidence, there IS a link between C64 and DEC!" (1882:354). The source, a Commodore history of the industrial designer Ira Velinsky, shows one designer who worked for both companies (section 1.1). The author's own description, 2026-11-10 06:47: "the smoking gun that there was indeed intercourse between dec and commodore" (1882:j7644)
- 2026-11-09 07:02 — virtio across the board for the devices, approved for the bus chapter (592:15)
- 2026-11-10 05:30 — "pdp64/32 is the decided upon pdp64"; r20 and pdp64/16 are tested alternatives, no longer part of it (1882:j4575)

G.2 The influences, inspirations and precedents, in the order they were introduced

- Each line: date and time of the first message that named it · the referent · who named it · its stance when introduced (**chosen**: the choice follows it; **rejected**: turned down; **considered**: compared or cited, not decided) · the part of the design it bears on · the source in parentheses (archive thread and message number, or `j` for a message found only in the session files).
- "Later" gives the first date and time at which its stance changed, where it did.
- Only what the messages say: a referent proposed by the assistant is marked so, and counts as an influence only where its line says it was accepted.
- 2025-07-23 22:43 · MS-DOS console font · named by the author · considered — C64 console (c64x), the earlier SDL console: font "reminiscent of the old Commodore 64 font, or the old MSDOS console font" (2224:0)
- 2026-06-18 00:31 · kitty · named by the author · considered — C64 console font and 9p interface: c64x is much the same "but with peek/poke instead of byte stream" (397:9)
- 2026-06-18 00:31 · xterm-256color subset shim · named by the author · chosen — C64 console font and 9p interface: "we'll have a shim that implements xterm-256color subset" (397:9)
- 2026-06-18 00:48 · 9P · named by the author · chosen — C64 console font and 9p interface: "9p all the way"; also a 9p server for window info [397:9] (397:12)
- 2026-06-18 00:48 · vsock · proposed by the assistant, not taken up · considered — C64 console font and 9p interface: he "didn't understand" the vsock question (397:12)
- 2026-06-18 02:54 · Commodore 64 · named by the author · chosen — C64 console (c64x), the earlier SDL console: "as sweet and usable as the C64" (397:44); later rejected 2026-06-25 05:41
- 2026-06-18 02:54 · newLISP · named by the author · chosen — C64 console (c64x), the earlier SDL console: as driver of the console (397:44)
- 2026-06-25 05:41 · RPi 500 keyboard · named by the author · chosen — C64 in spirit, not the hardware: "adequate" (1058:12)
- 2026-06-25 05:57 · original Lisp · named by the author · chosen — newlisp environment: bare function name at beginning of a line (1058:18)
- 2026-06-25 05:57 · Tcl · named by the author · chosen — newlisp environment: some idioms incorporated (1058:18)
- 2026-06-25 05:57 · Smalltalk · named by the author · chosen — newlisp environment: idioms and code browser, but showing all contexts (1058:18)
- 2026-06-25 05:57 · vim · named by the author · chosen — newlisp environment: "incorporate some of vim" into the environment (1058:18)
- 2026-08-23 05:51 · Blit · named by the author · considered — asked how much memory it needs and whether the PDP-11 could use it (813:0); later chosen 2026-10-23 18:57
- 2026-08-23 06:10 · Plan 9 · named by the author · considered — asked how Plan 9 did demand paging (813:8)
- 2026-08-23 06:28 · VAX ISA and DEC Alpha · named by the author · considered — asked how the VAX instruction set compares with the Alpha and the PDP-11 (813:14)
- 2026-08-23 06:36 · DEUNA and DELUA network cards · named by the author · considered — asked about their history and how much of the sockets API and TCP stack they provided (813:16)
- 2026-08-23 06:54 · SIMH · named by the author · considered — asked what it supports (813:22); later chosen 2026-10-13 17:05

- 2026-08-23 06:55 · Qbus and Unibus against PCI and SCSI · named by the author · considered — asked how they differ from modern buses (813:24)
- 2026-08-23 19:45 · Linux and Plan 9 virtual memory · named by the author · considered — asked which is better for the 32V virtual-memory retrofit (519:0)
- 2026-08-29 00:28 · PDP-11 · named by the author · chosen — Mashup of PDP-11 with C64 / M8502: "use pdp-11 as the base" (788:0); later rejected 2026-08-29 00:49, considered 2026-08-30 08:56
- 2026-08-29 00:29 · SID · named by the author · chosen — FPGA target and clock: fold in the good parts of c64 (531:0); later considered 2026-10-24 23:11
- 2026-08-29 00:29 · USB · named by the author · chosen — Modern I/O hardware (USB, NVMe): modern usb keyboard, mouse and hard drive (531:0)
- 2026-08-29 00:29 · NVMe · named by the author · chosen — Modern I/O hardware (USB, NVMe): "nvme hard drive" (531:0)
- 2026-08-29 00:29 · Raspberry Pi SD-card storage · named by the author · considered — Modern I/O hardware (USB, NVMe): asked how those little SanDisk drives interface with rpi (531:0)
- 2026-08-29 00:40 · QNX/QUNIX · named by the author · considered — Mashup of PDP-11 with C64 / M8502: fold design decisions into V7 for realtime guarantees (531:2)
- 2026-08-29 00:40 · Commodore 128 · named by the author · considered — Blit and C64 as the terminal model: asked what advances it had over the c64 (531:2); later chosen 2026-11-04 00:58
- 2026-08-29 00:40 · V7 Unix · named by the author · chosen — Realtime V7 with QNX ideas: "assume v7 source code as the base" (531:2); later considered 2026-10-22 07:46
- 2026-08-29 00:49 · middle-endian longword order · named by the author · rejected — Endianness: "none of this middle endian stuff, that gets fixed right now" (531:4)
- 2026-08-29 00:49 · 6502 · named by the author · considered — Endianness: 8-bit, so word order did not matter there (531:4)
- 2026-08-29 00:49 · IEEE 754 floating point · named by the author · chosen — Floating-point format: new FP unit compatible with the modern representation (531:4)
- 2026-08-29 00:49 · m8502 · named by the author · chosen — FPGA target and clock: mash it up with the PDP-11 instead of the c64, they seem closer (531:4)
- 2026-08-29 01:38 · 8502/6502 compatibility in FPGA · named by the author · considered — Spec revisions and write-up: all registers readable by holding the rdy pin (531:24)
- 2026-08-29 01:38 · 8502 RDY pin hold · named by the author · chosen — C64 island: 8502 registers readable from outside: freeze the CPU to read and write registers (531:24)
- 2026-08-29 07:52 · HDMI · named by the author · chosen — Video output (HDMI): drive it however we want (531:29)
- 2026-08-29 07:52 · PAL/NTSC · named by the author · considered — Video output (HDMI): 50 fps only for legacy code, backwards compat (531:29)
- 2026-08-30 08:23 · x86 · named by the author · chosen — Backward compatibility: adopt what is needed from it to keep transistor count down (533:2)
- 2026-08-30 08:23 · Strecker VAX-11/780 paper · proposed by the assistant, not taken up · considered — Backward compatibility: DEC considered extending PDP-11; opcode-space exhaustion was the reason (533:2)
- 2026-08-30 08:23 · M8502 and x86 · named by the author · chosen — MMU and FP units included: adopt what is needed from them, low transistor count (533:2)
- 2026-08-30 08:23 · VAX-11 · proposed by the assistant, not taken up · considered — Widening the PDP-11 to 64 bits: widened PDP-11 pleasant to write and target (533:2); later chosen 2026-10-20 22:44
- 2026-08-30 09:08 · x86 ah/al · named by the author · considered — Register width: asked for 64-bit registers addressable as two 32-bit halves, sort of like ah/al (533:6)
- 2026-08-30 09:13 · UTF-8 · named by the author · chosen — Word size: showed how the length-first encoding can be done (533:8)
- 2026-08-30 09:13 · ARM32 LDRD / MIPS32 FP pairs · proposed by the assistant, not taken up · considered — Word size: register-pair cost of 32-bit FP, in the case against a 32-bit version (533:8)

- 2026-08-30 09:30 · AArch64 · proposed by the assistant, not taken up · considered — Spec revisions and write-up: holds both rules at once; cited for partial writes (533:14)
- 2026-08-30 09:50 · Lisp machine · named by the author · chosen — Tagged pointers: tagged pointers; hardware tag checking on dereference not added (533:18)
- 2026-10-12 13:17 · dmr cc · named by the author · chosen — dmr optimizations: goal and scope: make the modern pcc pdp11 backend emit identical code, including optimizer (1889:0); later considered 2026-10-13 18:37
- 2026-10-12 13:42 · modern pcc -xssa · named by the author · chosen — dmr optimizations: goal and scope: run in parallel with dmr optimizations (1889:j968)
- 2026-10-12 13:42 · modern pcc SSA optimizations · named by the author · considered — SSA and pcc optimizations alongside dmr: allowed alongside dmr; dropped meanwhile for getnum bug (1889:j968)
- 2026-10-12 13:46 · dmr optimization pass · named by the author · chosen — SSA and pcc optimizations alongside dmr: other optimizations should run after it (1889:44)
- 2026-10-13 15:50 · VAX and x86 platforms · named by the author · chosen — dmr optimizations on other targets: final step after the pdp-11 backend is complete (1889:j5862)
- 2026-10-13 17:22 · pcc · named by the author · chosen — Upstream PortableCC: keep tree up to date with it; keep upstream's fix (1889:j7146); later considered 2026-10-14 00:43
- 2026-10-13 17:54 · VAX backend · named by the author · chosen — mips backend: prioritised over mips (1889:406)
- 2026-10-13 17:54 · nova second pass in pcc · named by the author · considered — mips backend: if pcc ships it, bring the dmr c2 pass in too (1889:406)
- 2026-10-13 18:10 · V7 disk image · named by the author · chosen — simh testing: used for the simh tests (1889:j7762)
- 2026-10-13 18:36 · 32V assembler · named by the author · chosen — VAX output for the 32V assembler: flag so the vax backend emits what 32v as accepts (1889:j7879)
- 2026-10-13 19:46 · 32V cc · named by the author · chosen — dmr optimizations on other targets: original cc, checked for optimizations beyond dmr package (1889:j8486)
- 2026-10-13 19:57 · dmr and VAX tricks · named by the author · chosen — x86 backend changes: document how they worked and bring them to x86 and other backends (1889:434)
- 2026-10-13 19:57 · VAX pcc as good as dmr on PDP-11 · named by the author · considered — x86 backend changes: remarked it sounds like VAX pcc was about as good (1889:434)
- 2026-10-13 19:57 · VAX pcc / 32V · named by the author · considered — Optimization tricks document and optimization docs: "about as good as dmr was on pdp11" (1889:434)
- 2026-10-13 19:57 · x86, then VAX · named by the author · chosen — Optimization tricks document and optimization docs: "do the x86 ones first, then vax" (1889:434)
- 2026-10-17 09:28 · optim2.c · named by the author · chosen — Optimizer placement (optim2.c): put whatever possible there (1889:601)
- 2026-10-17 10:25 · original Unix code (dmr) · named by the author · considered — Documenting bugs and defects: "note any dmr errors in the docs" (1889:j16765)
- 2026-10-17 10:59 · pdp11/vax/x86 backends only · named by the author · chosen — Optimization tricks document and optimization docs: other backends left alone for now (1889:j17625)
- 2026-10-17 21:39 · V7 and 32V old K&R forms · named by the author · considered — Old forms and long long libc documentation: forms pcc doesn't support; cost of supporting them (1889:j24650)
- 2026-10-17 21:39 · C99 · named by the author · considered — Old forms and long long libc documentation: regressions from C99 if old forms were supported (1889:j24650)
- 2026-10-18 06:30 · V7/32V libc side · named by the author · rejected — Long long support and libc helpers: asked to do longlong helpers on pcc side instead (1889:j29864)
- 2026-10-18 07:33 · PDP-11 and VAX-11 · named by the author · considered — Documenting bugs and defects: asks whether pre-existing bugs are only pdp11 or vax too (1889:1075); later chosen 2026-10-22 06:52
- 2026-10-18 16:03 · clang -Os · named by the author · considered — aarch64 against clang -Os: asked if -xdmr beats clang -Os on aarch64 (1889:1114); later chosen 2026-10-22 07:18
- 2026-10-18 16:36 · Lua and SQLite · named by the author · chosen — Benchmark report and corpus: fetch them as test corpora, use for each arch (1889:1124)

- 2026-10-18 16:57 · clang · named by the author · considered — Benchmark report and corpus: asked which backends pcc and clang share; test all (1889:1135)
- 2026-10-18 18:03 · yacc · named by the author · chosen — as32v in the v7unix-toolchain repo and tool naming: use it for the old-yacc grammar (1889:j41256)
- 2026-10-18 18:06 · v7as / v7cc prefix naming · named by the author · rejected — as32v in the v7unix-toolchain repo and tool naming: change policy to asv7, ccv7, cppv7 suffixes (1889:j41544)
- 2026-10-18 22:43 · 32V pcc · named by the author · considered — Benchmark and size reports: name to use for the 32V cc (1889:j44109)
- 2026-10-18 23:52 · modern C inline semantics · named by the author · considered — Benchmark freeze and fixes against stock: inline is a hint, not a command (1889:j44538)
- 2026-10-19 01:56 · GCC 4.7 or older (plain C) · proposed by the assistant, not taken up · considered — gcc cross-build and gcc comparison: would avoid C++ host build; judged not worth the trouble (1889:1291)
- 2026-10-19 04:13 · upstream git history · named by the author · chosen — Upstream PortableCC: source for the pattern of what ragge accepts (1889:j46712)
- 2026-10-19 04:41 · PR #47 aarch64 backend · named by the author · chosen — aarch64 PR 47 reconciliation: compare ours with it and fold in anything good (1889:1346)
- 2026-10-19 07:16 · Debian multiarch include path · proposed by the assistant, accepted · chosen — aarch64 PR 47 reconciliation: /usr/include/aarch64-linux-gnu (1889:1410)
- 2026-10-19 07:16 · Apple Mach-O variadic convention · proposed by the assistant, accepted · chosen — aarch64 PR 47 reconciliation: fix variadic calls for Mach-O (1889:1410)
- 2026-10-19 07:57 · libgcc __addtf3 family · proposed by the assistant, accepted · chosen — aarch64 binary128 support: stage 3: arithmetic and conversions through library calls (1889:1421)
- 2026-10-19 09:22 · GNU libgcc runtime · proposed by the assistant, not taken up · considered — aarch64 long double default (A64QUAD): option 1; depends on GCC being installed (1889:1431)
- 2026-10-19 09:22 · own runtime tools/a64quad/tf.c · proposed by the assistant, not taken up · considered — aarch64 long double default (A64QUAD): option 2; assistant's recommendation, not chosen (1889:1431)
- 2026-10-19 09:27 · gcc cross-compilers for pdp11 and vax · named by the author · chosen — gcc cross-build and gcc comparison: built and measured as comparison reference (1889:1436)
- 2026-10-19 10:33 · gcc -Os · proposed by the assistant, accepted · considered — Benchmark and size reports: size gap explained as GCC deleting undefined-behaviour code (1889:j53715); later chosen 2026-10-21 08:03
- 2026-10-19 20:58 · gcc · named by the author · chosen — Fair corpus and compiler comparison: files gcc cannot compile excluded from the fair corpus (1889:j55479)
- 2026-10-19 22:05 · sjpcc · named by the author · chosen — Fair corpus and compiler comparison: include in the evaluation on vax and pdp11 (1889:j57024)
- 2026-10-20 19:59 · LoongArch64 / AArch64 · named by the author · considered — R7 as a view of the PC: asked how they handle the PC (533:36)
- 2026-10-20 19:59 · LoongArch64 · named by the author · considered — ISA comparison table: asked how they handle the PC (533:36)
- 2026-10-20 20:28 · GPU · named by the author · considered — GPU and accelerator meaning, helper chips: meant as "a helper/accelerator that uses the same ram, does dma" (533:51)
- 2026-10-20 20:28 · SIMD/AVX · named by the author · considered — GPU and accelerator meaning, helper chips: helps like a GPU's, "the simd/avax stuff" (533:51)
- 2026-10-20 20:28 · Samsung LittleBit-2 · named by the author · considered — GPU and accelerator meaning, helper chips: a GPU optimized purely for its encoding (533:51)
- 2026-10-20 23:14 · ror over rol · named by the author · chosen — Mnemonic and width spelling: assistant made the case; assembler could map rol to ror (533:93)
- 2026-10-21 13:17 · Ockham ("ockam") · named by the author · chosen — Efficiency order for PIE: entity-multiplication rule (533:117)
- 2026-10-21 15:20 · asv7 and ldv7 · named by the author · chosen — pdp64 toolchain elements and build plan: port them to assemble and link pdp64 (1889:1699)

- 2026-10-21 15:21 · ELF · named by the author · chosen — pdp64 toolchain elements and build plan: “actually.... elf linker is ok” (1889:j68834)
- 2026-10-21 15:21 · QEMU · named by the author · chosen — QEMU and LLVM targets: wants a plan for a qemu backend for the pdp64 (1889:j68877)
- 2026-10-21 15:29 · V7 a.out with 64-bit variant · proposed by the assistant, not taken up · considered — pdp64 toolchain elements and build plan: assistant’s alternative to ELF64 output (1889:1706)
- 2026-10-21 15:29 · GNU ld / lld · proposed by the assistant, not taken up · considered — pdp64 toolchain elements and build plan: no back end for this ISA; judged more work (1889:1706)
- 2026-10-21 15:29 · ELF64 output (widen ld.c) · proposed by the assistant, accepted · chosen — Spec revisions and write-up: said “yes to all” to the assistant’s proposal (1889:1706)
- 2026-10-21 15:29 · V7 a.out 64-bit variant · proposed by the assistant, not taken up · considered — Spec revisions and write-up: offered as the smaller change (1889:1706)
- 2026-10-21 15:45 · PDP-11 and x86 signed char · proposed by the assistant, accepted · chosen — pdp64 ABI and long double: signed char matches their tradition (1889:1712)
- 2026-10-21 15:45 · modern ELF symbol names (no underscore) · proposed by the assistant, accepted · chosen — pdp64 ABI and long double: instead of V7 leading underscore (1889:1712)
- 2026-10-21 15:45 · software 128-bit long double · proposed by the assistant, not taken up · considered — pdp64 ABI and long double: alternative to same-as-double (1889:1712)
- 2026-10-21 20:37 · asv7 (V7 as) · named by the author · rejected — Assembler syntax and names: not to copy its warts or anachronisms (1889:1745)
- 2026-10-21 20:38 · V7 octal-default numbers · named by the author · rejected — Assembler syntax and names: numbers default to decimal (1889:j69889)
- 2026-10-21 20:46 · C syntax for numbers and strings · named by the author · chosen — Assembler syntax and names: instead of V7 <...> strings and comment slash (1889:1753)
- 2026-10-21 20:48 · modern ELF symbol names · named by the author · chosen — Assembler syntax and names: names and labels follow it (1889:1757)
- 2026-10-21 22:09 · PowerPC 601 · named by the author · considered — Opcode usage, instruction count and pruning: cited as the measure pdp64 already beats; against cutting to about 70 (1889:1845); later chosen 2026-10-23 00:15
- 2026-10-21 22:13 · C round() / libm · proposed by the assistant, accepted · chosen — FP rounding variants: ties-away-from-zero rounding added as a qualifier (1889:1847)
- 2026-10-21 22:16 · carry-less multiply · named by the author · chosen — CRC instructions: replaces both CRC variants (1889:1851)
- 2026-10-21 22:27 · cmov/csel/fsel · named by the author · considered — Integer abs/min/max and conditional select: wants to know more; abs/min/max as assembler aliases on top (1889:j71358)
- 2026-10-21 22:36 · ARM csel family · proposed by the assistant, not taken up · considered — Integer abs/min/max and conditional select: select variants nearly free once the first exists (1889:1865)
- 2026-10-22 06:21 · manpage-style SEE ALSO section · named by the author · chosen — Links and navigation between docs: wants “a manpage style section at the bottom” listing repos and docs (1889:1969)
- 2026-10-22 07:46 · LLVM · named by the author · considered — Undefined items: asks how hard one would be (1889:2073); later chosen 2026-10-23 00:04
- 2026-10-22 07:46 · Linux port (GCC or LLVM back end) · proposed by the assistant, not taken up · considered — Undefined items: a BSD or V7-style Unix is closer to the toolchain (1889:2073)
- 2026-10-22 08:09 · gcc -Os x86-64 / clang -Os · proposed by the assistant, not taken up · considered — One-parcel forms: pdp64 averages 3.36 bytes per instruction against 3.66 and 3.78 (1889:2102)
- 2026-10-22 15:37 · OpenBSD · named by the author · considered — QEMU and LLVM targets: “how does openbsd bring up a new architecture?” vs going straight to 3bsd (1889:2199); later chosen 2026-10-22 18:52
- 2026-10-22 15:37 · 3BSD · named by the author · considered — QEMU and LLVM targets: premise of his question: going straight to 3bsd (1889:2199); later rejected 2026-10-22 18:52

- 2026-10-22 15:38 · LLVM target sources under llvm/ in pdp64 repo · proposed by the assistant, not taken up · considered — QEMU and LLVM targets: assistant proposal, not a GitHub fork; he answered only about machine (1889:2202)
- 2026-10-22 15:47 · major operating systems and architectures · named by the author · considered — Comparison with other architectures: asked how the options compared against them (1889:2207)
- 2026-10-22 15:47 · gcc cross-compilers on other arches · named by the author · considered — System doc: asks what cross-compile infrastructure exists (1889:2207)
- 2026-10-22 16:00 · out-of-order design · named by the author · considered — System doc: asks what supporting it needs and what refusing it costs (1889:2215)
- 2026-10-22 16:16 · ARM32 SRS/RFE · proposed by the assistant, not taken up · considered — PUSHX and POPX: same idea; assistant said from memory, unchecked (1889:2236)
- 2026-10-22 16:28 · Linux and BSD kernel assembly · named by the author · considered — Kernel assembly survey: survey for instructions like popx/pushx that compilers never emit (1889:2258); later chosen 2026-10-22 16:28
- 2026-10-22 16:28 · clang backends · named by the author · considered — Kernel assembly survey: compile the corpus with each to find instructions to adopt (1889:2258)
- 2026-10-22 16:28 · each clang backend that we have · named by the author · chosen — Comparison with other architectures: compile the fair corpus with each to find instructions to adopt (1889:2258)
- 2026-10-22 16:29 · other backends with -Os · named by the author · considered — pdp64 measurement against clang and gcc: asked how pdp64 compares with them (1889:j75313)
- 2026-10-22 16:37 · VAX svpctx/ldpctx · proposed by the assistant, not taken up · considered — Kernel assembly survey document: hardware context switch sidebar (1889:2277)
- 2026-10-22 18:52 · Unix/32V · named by the author · chosen — First operating system port: fallback "if openbsd proves too hard"; has demand paging (1889:2401)
- 2026-10-22 18:56 · AArch64 ldtr/sttr and m68k moves · proposed by the assistant, not taken up · considered — Decisions document: option B: separate access-as-user load and store forms (1889:2406)
- 2026-10-22 18:56 · x86, ARM and RISC-V kernels (incl. OpenBSD) · proposed by the assistant, not taken up · considered — Decisions document: assistant's reason for recommending option A (1889:2406)
- 2026-10-22 19:03 · unix on PDP-11 and VAX-11 · named by the author · considered — Decisions document: asked to compare decisions with what they used to do, and tradeoffs (1889:2413)
- 2026-10-22 19:22 · VAX/PDP · named by the author · considered — SMP and its decisions document: survey "what all major modern os and cpu do, plus vax/pdp" (1889:2426)
- 2026-10-22 20:07 · TUHS source · named by the author · chosen — Fact-checking: already on the box, no download needed (1889:2456)
- 2026-10-22 20:23 · kernel source code · named by the author · considered — Fact-checking: suggested instead of hard-to-access manuals (1889:2484)
- 2026-10-22 20:35 · PDP-11 SR0 / VAX MAPEN · proposed by the assistant, not taken up · considered — Mode bits and the "no mode bits" rule: translation on/off switch, called the heritage choice by the assistant (1889:2500)
- 2026-10-22 20:56 · Linux and BSD current kernel source · named by the author · chosen — Pain-points research: current version only, not full history (1889:2519)
- 2026-10-22 21:01 · mailing lists · named by the author · chosen — Pain-points research: expected pain points from mailing lists, not only source (1889:2526)
- 2026-10-22 21:11 · lwn.net · named by the author · rejected — Browser use, politeness and site bans: avoided after it triggered spammer code; later ban removed [1889:j82932] (1889:2542); later considered 2026-10-22 21:12
- 2026-10-22 23:01 · Spectre · named by the author · rejected — Constant-time execution: design must not repeat it (1889:2605)
- 2026-10-22 23:01 · Rowhammer · named by the author · rejected — Constant-time execution: written as "hammer-row"; design must not repeat it (1889:2605)
- 2026-10-22 23:01 · timing channel attacks · named by the author · rejected — CPU vulnerability avoidance: wants a constant-time flag so instructions take equal time (1889:2605)

- 2026-10-22 23:01 · OpenBSD mitigations · named by the author · chosen — Memory safety from OpenBSD mitigations: make memory-safety mitigations "soft and easy"; improvements "theo would love" (1889:j78280)
- 2026-10-22 23:05 · Linux security · named by the author · chosen — Pain-points research: same list from its people (1889:2612)
- 2026-10-22 23:08 · UEFI · named by the author · considered — Boot process, BIOS and open firmware: asked to look into it (1889:2614)
- 2026-10-22 23:15 · OpenBSD disk partitioning · named by the author · considered — Boot process, BIOS and open firmware: asked where it originated and how it differs (1889:2620)
- 2026-10-22 23:15 · GPT · named by the author · considered — Boot process, BIOS and open firmware: asked how it plays into boot (1889:2620); later chosen 2026-10-22 23:23
- 2026-10-22 23:23 · majority boot block offset · named by the author · chosen — Disk label and boot block: "follow the majority as far as boot block offset" (1889:j78553)
- 2026-10-23 00:12 · PDP-10, PDP-11 and VAX-11 · named by the author · chosen — Manual: deliverable and chapter order: manual to cover what was so great about programming them (1889:2668)
- 2026-10-23 00:14 · old Sanskrit books (aphorism plus commentary) · named by the author · chosen — Manual purpose, style and notation: style: aphorism followed by commentary (1889:j78927)
- 2026-10-23 00:14 · Mishnah and Gemara · named by the author · chosen — Manual purpose, style and notation: same aphorism-then-commentary structure (1889:j78927)
- 2026-10-23 00:17 · VAX and PDP-11 (minimality) · named by the author · considered — Design philosophy and optimization layers: pdp64 is "even more minimal" in hindsight (1889:j78988)
- 2026-10-23 00:41 · ARM-style memory-mapped device · named by the author · chosen — SIMD and vector work: "mmap device arm style route" for SIMD/AVX2 vector work (1889:j79318)
- 2026-10-23 00:41 · AVX2 · named by the author · considered — SIMD and vector work: the SIMD work the chapter covers (1889:j79318)
- 2026-10-23 00:41 · APL/K · named by the author · chosen — SIMD and vector work: mention as making vector languages easier to support (1889:j79318)
- 2026-10-23 00:49 · HDMI, SATA, USB · named by the author · considered — Standard GPU, NIC and audio interfaces: "standard kit", our interface goes through mmio (1889:2704)
- 2026-10-23 00:53 · VAX I/O space · proposed by the assistant, accepted · considered — Address space (VAX I/O space): memory low, I/O above; to be noted in the book (1889:j79495)
- 2026-10-23 01:06 · SCSI/ATA/SATA/NVMe · named by the author · considered — Physical address bit that selects device space: all follow the same device pattern, not limited to 8-register mmio (1889:2713)
- 2026-10-23 01:47 · mkbook.py · named by the author · rejected — Book toolchain, build image and aports: reimplementing what buildabook does (1889:2744)
- 2026-10-23 08:12 · alpine:edge docker with ada-on-alpine · named by the author · chosen — Book toolchain, build image and aports: build and install all the aports on it (1889:2796)
- 2026-10-23 08:57 · buildabook · named by the author · chosen — Book toolchain, build image and aports: replace mkbook.py and push buildabook (1889:2828)
- 2026-10-23 19:01 · USB enumerated over PCIe · named by the author · chosen — Bus enumeration and attached buses: model for q64 enumerating arbitrary attached buses (1889:2941)
- 2026-10-23 20:06 · Plan 9 draw(3)/devdraw · proposed by the assistant, not taken up · considered — Architecture recommendations and open proposals: compared with ring-based command encoding for BLT64 (1889:2962)
- 2026-11-04 00:07 · PCI Express · named by the author · chosen — Q64 bus: DMA design, document and PCI Express base: pcie for the q64 bus, with pain points solved or mitigated (1882:23)
- 2026-11-04 00:07 · IOMMU mapping of C64/C128 · named by the author · considered — Q64 bus: DMA design, document and PCI Express base: "i think we should, but that is extraneous" (1882:23)
- 2026-11-04 15:28 · Steve Johnson pcc VAX optimizations · named by the author · chosen — Dual-encoding toolchain, mnemonics and integration: fold into the -xdmr flag (1882:j2662)
- 2026-11-04 15:30 · hbnf · named by the author · chosen — Grammar as the single source of truth: source of truth for the tools (1882:j2701)

- 2026-11-04 15:30 · Python pivot · named by the author · rejected — Grammar as the single source of truth: "undo your pivot to python" (1882:j2701)
- 2026-11-05 06:58 · hbnf C, Ada, Rust and Zig backends · proposed by the assistant, not taken up · considered — hbnf layout grammar and templates: chart backend would sit alongside them (1882:304)
- 2026-11-05 06:58 · Python chart generator · proposed by the assistant, not taken up · considered — hbnf layout grammar and templates: interim, behind one layout model (1882:304)
- 2026-11-05 07:01 · Mustache · named by the author · rejected — hbnf layout grammar and templates: "the mustache templates are going away" once rewrite rules exist (1882:307)
- 2026-11-05 07:39 · hbnf %emit{} %action{} with prologue/epilogue · named by the author · considered — hbnf layout grammar and templates: asks if C code in the grammar can do the job until rewrite rules land (1882:329)
- 2026-11-08 05:33 · Velinsky (DEC Rainbow and Commodore designer) · proposed by the assistant, accepted · considered — Velinsky link between C64 and DEC: common designer as link between DEC and Commodore (1882:363)
- 2026-11-08 05:34 · C64 and PDP-11 similarities · named by the author · considered — Velinsky link between C64 and DEC: commonalities found, cited as evidence (1882:365)
- 2026-11-09 06:52 · PCIe/NVMe/USB · named by the author · chosen — Device register model: native 8-register devices to virtio: standardizing on the latest buses (592:13)
- 2026-11-09 06:52 · 12 register + dma · named by the author · considered — Device register model: native 8-register devices to virtio: compared with virtio devices (592:13)
- 2026-11-09 07:02 · virtio · named by the author · chosen — Q64 bus: DMA design, document and PCI Express base: "virtio across the board" (592:15); later considered 2026-11-09 07:14
- 2026-11-09 07:02 · native 8-register devices · named by the author · rejected — Q64 bus: DMA design, document and PCI Express base: "forget the native 8 register devices. gone." (592:15)
- 2026-11-09 07:02 · Loongson 5-level TLB · named by the author · considered — Physical address bit that selects device space: asked what else is affected if pdp64 had it (592:15)
- 2026-11-09 09:10 · RISC-V · named by the author · considered — Prefix-coded encodings, 16-register and 32-register grammars: faulted for no comparison of opcode counts or bit layouts with pdp64 (592:38)